The pdp64 Architecture Manual

A 64-bit extension of the PDP-11

Digital Equity Collective's Pragmatic Digital Processor, 64 bit

David Walther

Copyright © 2026 David Walther. Released under the BSD 2-Clause licence (the file `COPYING` of the repository).
The source of this book is https://github.com/moebiusV/pdp64 (`docs/manual/`, the manual of each member in `docs/manual/pdp64-r20/`, `pdp64-16/`, `pdp64-32/`); the instruction dictionary is generated from the machine-readable specification. Typeset with Typst in EB Garamond and DejaVu Sans Mono.

CONTENTS

About this manual ..... **ii**
Part I — The architecture **1**
Chapter 1 — Introduction 2
Chapter 2 — The bus 7
Chapter 3 — Graphics: the terminal and its devices 11
Chapter 4 — Boot, the BIOS, the disk label and the GPT 15
Chapter 5 — The device driver model 19
Chapter 6 — The kernel: modes, exceptions and memory management 24
Chapter 7 — Multiple processors 30
Chapter 8 — Vector work, AI and array languages: the device route 36
Chapter 9 — The instruction set 40
Chapter 10 — Conclusion: from FPGA to silicon 418
Part II — The instruction reference tables **421**
The instruction reference tables 422
Assembler aliases and old spellings 423
The architectural constants 424
The mnemonics, alphabetically 427
The bit layouts of every opcode 434
Part III — Appendices **516**
Appendix A — Possible improvements not decided on 517
Appendix B — Decisions pending 520
Appendix C — Glossary 526
Appendix D — The ABI, version 1 529
Appendix E — The assembly language and the object format 531
Appendix F — The evidence behind the manual 533

About this manual

Each section is a few numbered aphorisms followed by a commentary. The aphorisms say what is true of the machine, in plain short statements, and read on their own as the main text. The commentary under them gives, as they apply: the *Precedent* (what earlier machines did), the *Trade-offs* (what the choice costs), the *Improvements* (how the pdp64 departs from the precedent), the *Benefits*, the *Modern comparables* (what current architectures do), and, for an instruction that is not obvious, its *Use*. Every decision also carries an *Impact* line saying what it means for an efficient **kernel**, for efficient **compilation** and for an efficient **hardware layout**: the three things a processor designer asks of a choice. An instruction as plain as `MOV` or `ADD` needs no statement of use. The reference entries follow the layout of the classic CPU manuals, in which every instruction has its encoding diagram, its operation and its effects.

Each aphorism carries a status. *(D)* means decided. *(P)* means a proposal that is not decided. *(S)* means a stub: the statement names what is left unspecified. An aphorism with no tag describes the architecture as it is built and tested.

Numbers. Where a figure is measured, the text says what was measured; where it is an estimate, it says so. Claims about other machines are checked against the fact-check record listed in appendix F.

The PDF. The same text as a book, `book/pdp64-16.pdf`, built by buildabook and Typst from the sources of this directory (`tools/mkbook.sh`; see `book/README.md`).

Evidence documents. All of them are listed in appendix F: the full instruction set text (`../pdp64-isa.md`), the record of decisions (`../errata.md`) and the open decisions with five options each (`../decisions-pending.md`).

Part I — The architecture

Chapter 1 — Introduction

The pdp64 is a 64-bit successor to the PDP-11, designed to be as pleasant to program as the PDP-11 and the VAX-11 were, and as small to build as the PDP-11 was. It is a little-endian machine with sixteen 64-bit registers, instructions of two, four or six bytes, and the PDP-11's addressing modes and branches, brought up to date with what fifty years of compilers and operating systems have shown to matter. It is a 64-bit machine that supports several cores from the start (symmetric multiprocessing, chapter 7), and it is designed to be efficient with in-order and out-of-order execution implementations (1.4.4). Sections 1.1 to 1.4 give the reasons for the machine, what made its ancestors good, and the principles the rest of the manual follows; section 1.8 says how this CPU and the other CPUs of the family relate.

1.1 Why the machine exists

1.1.1 The PDP-11 and the VAX-11 were pleasant, even fun, to program in assembly language.

1.1.2 Afterward computer design went in many directions, and that pleasure was largely lost.

1.1.3 The pdp64 carries it forward, using the lessons of the last fifty years: which instructions are needed to get things done and which are not, what a processor is used for, and which instructions it needs the most.

1.1.4 The PDP-11 is a source, not an obligation. What earned its place is kept, what did not is dropped, and what fifty years showed to be missing is added.

Commentary. **Precedent.** The claim in 1.1.1 is old and widely made of the PDP-11: a small, regular instruction set that a programmer could learn in a week and keep in mind, with a set of addressing modes that fit the C language so closely that the language and the machine seem to have grown together. **Improvements.** Measurements contradict several received ideas. A second unsigned multiply is the same instruction as the first, while the instruction that returns the *high half* of a product is the one multi-precision arithmetic needs. A rotate by a register count has almost no users, because cryptographic code rotates by constants. A divide is rare, because compilers turn division by a constant into a multiply and a shift. Dedicated absolute-value, minimum and maximum instructions are worth under a tenth of a percent of compiled code. And CRC instructions become unnecessary once a carry-less multiply exists. **Benefits.** A machine built on these findings is small enough for a single engineer to hold in mind, and, as later chapters show, small enough to fit in a modest FPGA. **Modern comparables.** The same re-examination produced the RISC designs of the 1980s and, more recently, the compact RISC-V and LoongArch instruction sets; the pdp64 differs by keeping the PDP-11's memory operands, addressing modes and variable-length encoding where those were found to save code and to cost little. **A thread between the two lineages.** The pdp64 joins a DEC machine's instruction set to a Commodore machine's devices, and the two companies are linked by at least one person: Ira Velinsky, the industrial designer credited with modifying the Commodore 64's case, had earlier worked at Digital Equipment Corporation (he is credited there with the Rainbow 100 of 1981), and a Commodore history credits his DEC work with drawing the attention of Commodore's management. The author predicted such a link before finding it. The hypothesis he draws from it is that personnel passed between the two companies and that the similarities between the C64 and the PDP-11 are too many to be chance *(P)*; the source itself claims only the person, and no influence of DEC designs on Commodore's, and the similarities are not yet listed with sources (the record is in `docs/c64-devices-from-chat.md`, section 8).

1.2 What made the PDP-10, the PDP-11 and the VAX-11 good to program

1.2.1 The PDP-10 gave a programmer one word per instruction that said everything: the operation, the register, the index register, the indirect bit and the address.

1.2.2 The PDP-10's registers were also memory: the sixteen accumulators answered to the addresses 0 to 15.

1.2.3 The PDP-10 had half-word and byte operations, stack and skip instructions, and a procedure call that was one instruction.

1.2.4 The PDP-11 made every instruction and every addressing mode available with every register, so that the language had few exceptions to remember.

1.2.5 The PDP-11's program counter and stack pointer were ordinary registers, and its autoincrement and autodecrement modes made `*p++` and `*--p` single instructions.

1.2.6 The PDP-11 had no input or output instructions: a device was a few memory locations, and a driver was an ordinary program.

1.2.7 The VAX-11 widened the PDP-11 to 32 bits and added three-operand arithmetic, a complete virtual-memory system, and instructions for the jobs programs actually do: procedure calls, queues, character strings, array indexing and polynomial evaluation.

Commentary. **Precedent.** These are the qualities the design starts from. The PDP-10's simplicity of format and its habit of making the machine's own state addressable (1.2.1, 1.2.2) are the origin of the idea that the machine should hold together in the head. The PDP-11's orthogonality (1.2.4) is the property that mattered most to assembly programmers and, later, to compiler writers: an instruction that takes a register takes an address, an immediate or a stack slot with the same syntax. Its memory-mapped devices (1.2.6) are the reason the pdp64's devices are still eight addresses each (chapter 5). **Trade-offs.** Each of these qualities had a price that the later machines paid. The VAX-11's richness (1.2.7) bought programmability and cost an unbounded amount of hidden state in its longest instructions, so that a page fault in the middle of a character move could not be resumed without a special bit in the status word. The Motorola 68000, which took the PDP-11's orthogonality and widened it, could not restart an instruction after a bus error and so could not support demand-paged virtual memory until its successor patched the problem. **Improvements.** The pdp64 keeps what made programming pleasant and removes what made implementation hard: no hidden instruction state (every instruction can be restarted), at most one memory operand, a fixed small maximum instruction length, and no mode bits that change what unrelated instructions mean unless their benefit outweighs the cost. **Benefits.** The measurements in the instruction-set chapter show where this paid off: the processor fits in about 16,000 FPGA lookup tables and the compiler's code is the size of `gcc -Os`. **Modern comparables.** Of today's architectures, x86-64 kept the memory operands and variable-length encoding of this lineage; AArch64 and RISC-V chose fixed-length register-to-register encodings; the pdp64 sits between them, closer to x86 in the form of its instructions and closer to RISC-V in the discipline of its encoding.

1.3 More minimal than its ancestors, where hindsight allows

1.3.1 In several ways the pdp64 is more minimal than the PDP-11 and the VAX-11: it gives up theoretical completeness where fifty years of use showed the completeness was never exercised, and it spends what that saves on power that was.

1.3.2 Not every instruction works with every addressing mode on every operand: at most one operand of an instruction is a memory reference. *(D)*

1.3.3 The deferred addressing modes are gone; an indirection is a `JMPI`, a `CALLI` or an ordinary load. *(D)*

1.3.4 The ALU has one width, there is no arithmetic in the narrow sizes, and there is no conversion opcode group: conversions are moves. *(D)*

1.3.5 There is one rotate direction, and no slot exists only to complete a table.

1.3.6 The VAX's packed decimal, string editing and `CALLS`/`CALLG` procedure-call instructions are absent. *(D)*

1.3.7 There are no transcendental instructions, no separate floating-point register file and no rounding-mode bits. *(D)*

1.3.8 What the saving buys is power that programs use: a bit unit (population count, count of leading and trailing zeros, byte reversal, bitfields, a carry-less multiply), block operations that restart from their registers, a fused multiply-add with the IEEE exception flags, the full set of atomics, UTF-8 instructions, and a user-memory guard. *(D)*

Commentary. **Precedent.** The PDP-11 was orthogonal, any operation with any mode on any register, and the VAX carried that to completeness: every operation in every data type, with up to six operands, each operand of any mode. Both were designed when memory was dear and compilers were weak, and the completeness did two things at once: it made assembly programming regular and pleasant, and it made a compiler's job simple. **Trade-offs.** The completeness was never free. It bought a decoder that had to parse specifier after specifier before the next instruction's address was known, and an instruction that could fault in the middle with unbounded state to save. What hindsight lets the pdp64 do is separate the two benefits: keep the regularity that programmers and compilers use (every register is general, every mode means the same thing everywhere it is allowed) and drop the completeness that nobody exercised (a memory-to-memory arithmetic

instruction, a deferred mode on every operand, a sized version of every operation). The price is a few more instructions in rare cases, such as a load and a store where the VAX had one memory-to-memory move. **Improvements.** Each piece of dropped completeness was weighed against what it freed: the one memory operand lets the fault matrix collapse to three rows with the third empty and pays for sixteen registers and a 12-bit displacement in four bytes; dropping the sized ALU removes byte-lane multiplexing and partial-register writes altogether; dropping the transcendentals leaves the `FMA` datapath, which is what accurate library maths is built from. **Benefits.** The decoder is small, the exception machinery is small, and the die area that completeness would have used goes to a bit unit and a block-move sequencer that together cost less than 5% of the core. **Modern comparables.** The RISC machines of the 1980s went further and made every instruction register to register except loads and stores; x86 kept one memory operand and survived on it; the pdp64 takes the middle position, which keeps the compact code of a memory operand and the restartability of a load/store design.

1.4 The principles

1.4.1 The design follows these priorities, in this order: **security and safety**, meaning errors that can never arise, then errors that are easy to detect, then errors that are cheap to recover from (restart and recovery go together, and an instruction does what its name says and nothing else); **opcode choice**, so that fewer instructions executed means fewer memory fetches; **opcode encoding**, dense, with rare instructions in longer encodings; **speed and power**, served by a low gate count; a **good compiler target**; and **pleasant in assembler**. *(D)*

1.4.2 All the rules are soft rules: a rule gives way when breaking it is worth more than it costs, with the cost written down. *(D)*

1.4.3 Between two designs that both work, prefer by this order: the one with no strange side effects (an instruction does what its name says and nothing else); the one that does not push an existing instruction into a longer encoding. Beside the ranking, at every level, do not multiply entities without benefit. Mode bits are decided case by case: by default a mode is visible state, and a bit in a register that is saved anyway costs nothing extra.

1.4.4 The architecture is designed to be efficient with in-order and out-of-order implementations. The first implementations are in order; nothing is specified in a way that is safe only because the core does not speculate or reorder, and nothing is specified that an out-of-order core could implement only at a large cost. *(D)*

1.4.5 An instruction is judged by its weight in the loops where it is used, not by how often it appears: a rarely used instruction that sits inside a critical, tight hot loop can earn its place. The measure is profile-weighted (executions multiplied by the instructions and bytes it saves), not a static count. *(D)*

Commentary. **Precedent.** Security and safety rank first; opcode choice and encoding, which cut memory fetches, rank next, then speed and power with the gate count that serves them. The priorities agree on almost everything, because a small, regular machine is also compact in code and cheap in gates; the stated order decides which wins when they conflict. Code size here means the instructions a program executes and the bytes it fetches, not the number of opcodes the machine has. **Trade-offs.** Error prevention, detection and recovery is the principle that most often changes a decision. It means counting the cost of an option not only in gates and bytes but in the code a kernel must write to handle its failures: a feature that adds a new fault, a new error return or a new half-finished state costs recovery code in every system that uses it, and a feature that makes an operation repeatable from the start costs none. **Improvements.** The soft-rules principle covers the two strongest defaults, "no mode bits" and "natural alignment of quad accesses". Breaking either has a large cost, but each is weighed like any other choice. Three mode bits are accepted in the design on those terms: the translation-enable bit that is set once at boot, the user-memory guard that lives in the status word, and the sticky floating-point flags, which never change what an instruction does. **Benefits.** A reader can tell why a choice was made by asking which principle it serves. **Modern comparables.** Most architectures state no priorities; the RISC-V project's published principle of keeping the base small and the extensions optional is the nearest in spirit.

1.5 What has been built and measured

1.5.1 The machine is specified by machine-readable grammars; the assembler, linker, simulator, a compiler back end and the QEMU target are generated from or checked against them, so that the numbers cannot drift apart.

1.5.2 On 827 real source files that every compiler accepts, the first compiler's output, in code and data, is within 0.2% of `gcc -Os` and 6.5% smaller than `clang -Os` on amd64, using 1.19 and 1.10 times as many (shorter) instructions. That is the figure of the original CPU; the figures of this CPU are in 9.2.13 and appendix F.

1.5.3 The C test programs pass on the simulator for this CPU, 963 of 963 runs (programs under the flag sets of the harness); the six large programs (lz4, zlib, Lua and three cryptographic kernels) print exactly what the host build prints; QEMU does not yet execute this encoding.

1.5.4 The finish line before silicon is an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D)*

Commentary. **Precedent.** The habit of generating every tool from one description, rather than writing each and hoping they agree, is the discipline that other instruction-set projects adopted late; the pdp64 follows it from the start, which is why a change to an opcode number is made once. **Trade-offs.** The measurements are of compiled C at the compiler's size-optimising setting; they are static counts of what is written, not run-time measurements, and the compiler is one of two (the LLVM back end, which OpenBSD needs, has an assembler and object writer that agree with the project's own on every form tested). **Benefits.** The comparison gives the architecture a number against which later changes are measured: the flag rules cost 0.39% in code size, accepted for the benefit they give to faster implementations. **Modern comparables.** `gcc` and `clang` at `-Os` are the standard measuring sticks for code size on x86-64 and AArch64; the pdp64 figure is compared with them on the same source files.

1.6 Does the instruction set matter?

1.6.1 The instruction set is not interchangeable with any other: it fixes what every implementation must preserve, and so what a processor costs to build, to verify, to program and to secure.

1.6.2 The common view, that a good implementation hides the instruction set, is true of one thing: the peak speed of a very large core, where a great deal of hardware can absorb almost any encoding.

1.6.3 It is not true of the cost of a small core, the size of the code, the amount of software a system needs, or the number of ways a system can fail; those follow the instruction set, and they can be measured.

1.6.4 Each of the rows below is a measurement or an estimate from this project, and each is explained in the chapter named beside it.

What differs	The figure	Where
The logic of a complete implementation, same company and era	the PDP-11's T-11, a full integer core, was about 12,000 transistors; the MicroVAX 78032, a subset of the VAX with no floating point, about 125,000	10.1, 10.2
What it takes to find where the next instruction starts	pdp64: a prefix of 3 to 16 bits of the first parcel; PDP-11: both operand specifiers; x86: prefixes, opcode, ModR/M and SIB in sequence; VAX: every specifier in order	9.4
Whether flags block a faster implementation	68.2% of compiled instructions wrote a flag and 1.5% read one, so the flag rules are set at a measured cost of 0.39% in code size	9.3, 10.3
Size of compiled code	on the 841-file corpus the output of the new compiler for this CPU is 13.78% smaller than the original CPU's	1.5
System-call path, instructions in and out (paper estimates)	x86-64 Linux about 45 and 30; AArch64 about 60 and 45; RISC-V about 66 and 45; VAX about 11 and 8; m68k 6 and 8; pdp64 4 to 5 and 4	6.5
Lines of kernel code to start a second core	x86: 1,528 plus 1,258 of real-mode code; arm64: 124 or 127; RISC-V: 108 or 58; pdp64: one register write	4.5, 7.8
Machine-dependent code in an OpenBSD port	riscv64 35,600 lines; arm64 61,613; amd64 79,167	10.4
Errata, workaround and quirk lines per 1,000 code lines in Linux's architecture directories	x86 0.8; arm64 0.7; arm 0.5; powerpc 0.4; mips 0.3; m68k 0.1; riscv 0.1; s390 0.0	pain-points report
Hardware-erratum options in Linux's configuration	arm64 64; arm 32; riscv 14	pain-points report

Commentary. **Precedent.** The view that instruction sets are equivalent has real support. Published comparisons of x86 and ARM implementations (the 2013 study usually cited is "Power Struggles: Revisiting the RISC vs. CISC Debate on Contemporary ARM and x86 Architectures", a citation not verified) found that at the same performance level the instruction set explains little of the difference in power or speed, and the very large cores of both families have converged. The pdp64 does not dispute that. **Trade-offs.** The claim is about steady-state performance of comparable large cores, and the table is about other costs. The table's figures are measurements of real machines and real software, not controlled experiments: some of the differences between architectures in the Linux figures are partly a matter of age and deployment (x86 is the oldest and most widely deployed), and the system-call and start-up figures are estimates and line counts of what is written,

not run times. They show that the instruction set is correlated with these costs, and the chapters show the mechanism in each case. **Improvements.** Each row is a place where this design made a different choice from the architecture it is compared with, and the choices were made against these costs, not against peak speed. **Benefits.** A reader who thinks the instruction set does not matter can check any row against its chapter: the answer is in what the instruction set obliges an implementation to do (precise exceptions, ordering, flag behaviour, instruction length, restart) and in the code a system must then write. **Modern comparables.** RISC-V's own case for a clean base instruction set and x86's survival by microarchitecture are the two sides of the argument.

1.7 How the manual is organised

1.7.1 Chapter 2 is the bus. Chapter 3 is graphics: the terminal, based on the C64 and the Blit and modernised, and how it drives HDMI and the other ports. Chapter 4 is boot. Chapter 5 is the device driver model. Chapter 6 is the kernel: modes, exceptions and memory management. Chapter 7 is multiple processors. Chapter 8 is vector work, AI and array languages, done by devices. Chapter 9 is the instruction set. Chapter 10 is the path to silicon.

1.7.2 Each section states its aphorisms first and its commentary after; the status tag *(D)* or *(P)* marks what is decided and what is proposed.

Commentary. **Precedent.** The form follows the old commentary traditions, in which a short text can be read straight through and the discussion of each line sits beneath it; and the layout of the classic CPU manuals for the reference material, in which every instruction has a fixed entry. **Benefits.** A designer who wants only the architecture reads the aphorisms; one who wants the reasons reads on.

1.8 The three CPUs

1.8.1 The pdp64 is an architecture with three CPUs, pdp64/r20, pdp64/16 and pdp64/32. They share the roles of the registers, the memory model, the exception mechanism, the page tables, the bus and the devices, the calling convention and the assembly language; they differ in how an instruction is encoded and in the size of the register file. *(D)*

1.8.2 pdp64/r20 is the original CPU: sixteen registers and an instruction whose first parcel is `len`, opcode, `reg` and `low`. pdp64/16 has sixteen registers and pdp64/32 has thirty-two, and both encode an instruction by a prefix code over the first parcel; pdp64/32 has 5-bit register fields in its one-parcel forms. This manual is the manual of pdp64/16. *(D)*

1.8.3 Each CPU has its own manual, because the CPUs are different, and all three are generated from one text, so that what is common is written once. The assembler selects the encoding (`-m pdp64/r20`, `-m pdp64/16`, `-m pdp64/32`), the object records it, and the linker refuses to link objects of different encodings. *(D)*

1.8.4 This CPU has 125 instruction entries (340 mnemonic words with the aliases): the 106 entries of the original CPU and 19 more for the 70 new words, the fused pairs, the selects, the checked arithmetic, TRAPCC, the deferred and user-qualified moves and the AES and SHA words, which the original CPU does not accept. *(D for the fused pairs, the selects, MOVQL, the checked arithmetic and TRAPCC; P for the rest, tagged in their entries)*

The manuals of the family: pdp64/r20 and pdp64/32; the landing page is `docs/manual/`.

Commentary. **Precedent.** One architecture with more than one encoding is not new: AArch32 has the ARM and Thumb encodings and RISC-V a compressed extension; the PDP-11 family had many implementations of one instruction set. **Trade-offs.** Three manuals cost the reader a choice and the maintainers a generator; one manual with a line of availability on every instruction would have mixed three layouts on every page, and the author decided against it. **Benefits.** A reader of one manual meets only the encoding of that CPU; the bus, the kernel and the devices are the same text in all three. **Impact.** *Kernel:* the system architecture is common, so one kernel serves all three. *Compiler:* one back end with three targets (`ccom -m`), a different set of instructions on the original CPU. *Hardware:* the CPUs have different decoders and register files and the same programming model above the instruction set.

Chapter 2 — The bus

The processor reaches the world through one bus, Q64, on which every device is a handful of memory locations, the bus itself is a device that says what is plugged into it, and every transfer of data is a descriptor in memory that a device carries out and marks done. The chapter states what the processor owes the bus and how the bus enumerates and maps devices and tells the processor when one comes or goes. How a driver talks to a device, the eight-register layout and the default devices are in chapter 5. Decided statements are tagged *(D)*; proposed ones *(P)*. Interrupts and modes are chapter 6, and the other cores are chapter 7. The full discussion is in `../pdp64-q64-bus.md`; the options still open are in `../decisions-pending.md` (R1 to R8, Q10).

2.1 Q64 and the address space

> ***PCI Express.*** *Since errata 75 and 80 (2026-11-03) Q64 is PCI Express, backward compatible, with convenience and security circuitry added on top, and storage is NVMe and USB; the standard network, graphics and audio devices are virtio-net, virtio-gpu and virtio-snd over the virtio PCI Express transport (errata 79). Devices are enumerated by PCI Express configuration space and base-address registers and interrupt by MSI-X, which replace the slot numbers, the slot-derived vectors and the eight-register device rule of the statements below (errata 80, in outline). This chapter describes the programming model that the pdp64 keeps on top of PCI Express; its full rewrite waits for the author's answers to the remaining items of* `decisions-pending.md` *Part F. Statements keep their tags.*

2.1.1 Q64 is the system bus: PCI Express, so that an ordinary PCI Express device works on it unchanged, with the programming model of the PDP-11 Q-bus (devices as memory locations in the one address space) widened to 64-bit data and a 40-bit physical address, and with convenience and security circuitry added. *(D)*

2.1.2 Devices live in the ordinary address space; the processor has no input or output instructions. *(D)*

2.1.3 Each device occupies its own 4 KB page of physical address space, so that one device can be mapped to one protection domain. *(P)*

2.1.4 Every page has a memory type: normal cached, normal uncached, device strongly ordered, or device weakly ordered. *(D)*

2.1.5 The transactions of Q64 are those of PCI Express; the circuitry the pdp64 adds is for convenience (overlays, helper registers) and security (the IOMMU, error and reset rules), and is measured against the list of pain points of `kernel-pain-points.md`. An extension is announced to a card, through a PCI Express extended capability, only if the card must take part in it; a mitigation that lives on the processor side is invisible to cards. *(D)* The detail of the extensions is not specified. *(S)*

2.1.6 The top bit of the 40-bit physical address (bit 39) selects the space (errata 60, the author's decision): set, the access goes to a device; clear, it goes to memory. Memory is then 512 GB and device space 512 GB, with one bit and no range comparison deciding where an access goes. This is the VAX's arrangement (below). *(D)*

Commentary. **Precedent.** The PDP-11's Unibus put every device in the address space at a fixed location, which is why a PDP-11 driver is an ordinary program; the Q-bus did the same with multiplexed lines. **Trade-offs.** The memory type is per page, not per access, so the code that writes a device register need not know that it is a device; the price is one page per device, which wastes address space but not memory. Using AXI as the transport saves a bridge between two protocols (estimated at about 600 lookup tables on the FPGA) and loses the Unibus's locked read-modify-write as a native bus transaction. **Improvements.** A 40-bit physical address puts devices and memory in one terabyte, with room for every device to sit in its own page. **Benefits.** An uncached, strongly ordered page needs no special instruction to reach a register. **Modern comparables.** RISC-V and ARM systems use memory-mapped devices on AXI or TileLink; x86 keeps both port I/O

and memory-mapped devices. **Impact.** *Kernel:* Devices are pages in the kernel's address map, found by the bus controller. *Compiler:* Nothing special. *Hardware:* Address decode is by page; the transport is AXI if the proposal is accepted.

Commentary on 2.1.6. **Precedent.** The VAX-11/780 divided its physical address space in two: memory in the low half and the I/O space above `0x20000000` (the 4.3BSD kernel's `NEX780`), which holds one 8 KB window (`NEXSIZE`) per adaptor slot, with the Unibus adapters' device registers reached in a space of their own beyond it; the bit that is set for the whole of I/O space is the one the hardware decodes first (read in the 4.3BSD `nexus.h`, which also records the 8600's, 8200's and MicroVAX's different bases, so the VAX itself kept changing the map from model to model, which the pdp64's single fixed bit would avoid); the PDP-11 put its devices in a fixed page at the top of the address space (a V7 driver's pointer to its registers is a constant in that page). **Trade-offs.** One bit makes the first routing decision in the bus fabric trivial and makes two classes of bug detectable: a page-table entry that maps a normal cached page onto device space, or a device page onto memory, can be a fault because the entry's memory type and the address bit disagree; and the bus controller can refuse a device's DMA into device space, so that no device can write another's registers, which is a free partial protection until devices have a translation unit. The cost is a change to the address map (the emulator's timer and console are in the upper half) and, if the bit were a forty-first, one more bit in every translation-buffer entry and every bus address. The reading of "bit 40" that costs nothing is the top bit of the existing 40-bit address (bit 39 counting from zero), because the page-table entry already holds physical page numbers up to that bit and DRAM then fills exactly 39 bits, the width of a virtual address. **Benefits.** Fewer gates in decode, and fewer ways to get a device mapping wrong. **Modern comparables.** Most systems split the map by address ranges chosen by the platform; the VAX and the PDP-11 used a fixed region; x86 has a separate port space. **Impact.** *Kernel:* a device mapping that is cached by mistake is a fault, not a silent bug, and a driver cannot be given a DMA address that reaches registers. *Compiler:* none. *Hardware:* a one-bit decode at the head of the fabric and a consistency check in the page-table walk.

2.2 What the processor owes the bus

2.2.1 The processor owes the bus exactly three things: a memory type per page, a `FENCE` instruction whose meaning against Q64 is defined, and atomics that can reach device memory. *(D)*

2.2.2 A driver can be sure a descriptor is visible before it writes the doorbell: it builds the descriptor, executes `FENCE`, and writes the doorbell register. *(D)*

Commentary. **Precedent.** The PDP-11 needed none of this because there was one processor and no cache. **Trade-offs.** Three obligations is the whole contract, which keeps the device side free to change without respecifying the processor. **Benefits.** A device and a driver can be written against these three promises without knowing the processor's pipeline. **Modern comparables.** ARM's device-memory types and `DMB`/`DSB`, and RISC-V's `FENCE` with I/O bits, play the same roles. **Impact.** *Kernel:* One `FENCE` before the doorbell is the whole ordering rule. *Compiler:* `FENCE` is an intrinsic. *Hardware:* The memory type is two bits per page-table entry and the bus honours it.

2.3 When an access fails

2.3.1 A read or instruction fetch of non-existent memory is a precise abort with cause `BUSFAULT`, at the faulting instruction. *(D)*

2.3.2 A posted write that times out, and an uncorrectable memory error, are machine checks: imprecise and not restartable. A `FENCE` before a probe makes even a posted write precise at a known point. *(D)*

2.3.3 `PROBER` and `PROBEW` test whether an address can be read or written without faulting. *(D)*

2.3.4 Resetting the bus is a write to a register of the bus controller, not an instruction. *(D)*

Commentary. **Precedent.** A PDP-11 read timeout was an unrestartable bus error, fatal to the process; the `nofault` idiom of V6 and V7 existed to probe for devices without dying. **Improvements.** The read timeout is a precise, restartable abort, and `PROBE` does the work of the idiom. **Benefits.** A driver that probes for a device handles a missing one in a few lines of ordinary fault-handling code, which is the recoverability principle at work. **Modern comparables.** x86 machine checks and ARM's `SError` correspond to the imprecise class; PCI returns all ones for a read of a removed device, which every driver must then learn to distrust, and the pdp64 avoids that convention. **Impact.** *Kernel:* A driver probes a missing device in a few lines of ordinary fault handling. *Compiler:* None. *Hardware:* A timeout counter in the bridge and a way to make a read abort precise.

2.4 Coherence and the hand-off

2.4.1 Between processors the data caches are coherent without any software action, by snooping with three states (modified, shared, invalid), for two to four cores. *(D)*

2.4.2 Between a processor and a DMA device they are not: buffers are mapped uncached and the hand-off is explicit. *(D)*

2.4.3 The hand-off is five steps: the processor builds the descriptors and data in uncached memory, executes `FENCE`, writes the device's doorbell register; the device walks the descriptors, moves the data by DMA, writes the result and a status word, and raises an interrupt; the processor reads the result. *(D)*

2.4.4 The write that raises a device's interrupt cannot overtake the data writes that precede it on the same path. *(D)*

Commentary. **Precedent.** The 11/70's cache watched Unibus writes so that DMA appeared coherent; the VAX's Unibus adapter had buffered data paths that a driver had to purge. **Trade-offs.** Snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off, so two mechanisms are used, each where it is cheap; the cost is that the processor reads received data from uncached memory, which is slow, and a cache-maintenance instruction would remove it for a non-coherent device that may come later. **Benefits.** No driver can forget a cache clean or invalidate, because there is none to forget. **Modern comparables.** x86 DMA is coherent; ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer in the kernel to place cache maintenance correctly. **Impact.** *Kernel:* Uncached DMA buffers mean no driver cache code. *Compiler:* None. *Hardware:* A snoop port on each data cache for core coherence only; no snoop port for DMA.

2.5 Atomics on the bus

2.5.1 The atomics are defined on a monitor: a load-linked takes a reservation, a write by any initiator clears it, and a store-conditional succeeds only if it is intact; the Unibus's `DATIP`/`DATO` locked pair is not an architectural primitive. *(D)*

2.5.2 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** The PDP-11's locked read-modify-write and the VAX's interlocked instructions are bus locks. **Trade-offs.** A lock is simple and a bottleneck as cores multiply; a monitor maps onto AXI's exclusive access and the memory systems of FPGAs and modern chips, at the cost of forward-progress rules. **Modern comparables.** RISC-V `LR`/`SC`, AArch64's exclusive loads and stores, and the large-system atomics that perform the operation at the memory. **Impact.** *Kernel:* Lock words and lock-free structures use the atomics of chapter 7. *Compiler:* C11 atomics lower to them. *Hardware:* A reservation monitor and an exclusive access, not a bus lock.

2.6 The bus as a device

2.6.1 The bus controller is itself a device with eight memory-mapped registers, through which the processor detects what is on the bus and asks it to map a device to an address range. *(D)*

2.6.2 The bus alerts the processor when a device enters or leaves. *(D)*

Offset	Register	Meaning
`0x00`	`ID`	bus controller, version, number of slots
`0x08`	`CONTROL`	enable events, rescan, reset the bus
`0x10`	`STATUS`	event pending, queue overflowed, busy
`0x18`	`SLOT`	select a slot
`0x20`	`INFO`	for the selected slot: present, type, version, window size, interrupt level
`0x28`	`MAPADDR`	the address at which the device's window appears
`0x30`	`MAPCTL`	enable the mapping; request an orderly eject
`0x38`	`EVENT`	next event (arrival, departure, fault) and its slot; zero if none

2.6.3 Each slot starts at a fixed 4 KB page; `MAPADDR` is for devices that need a particular address or size. *(P)*

2.6.4 An arrival or departure puts an event in a small queue and raises an interrupt; if the queue overflows a flag says events were lost and the driver rescans the bus, which is also what it does at boot. *(P)*

2.6.5 When a device leaves, its slot is marked absent at once: a read of its window is a precise bus fault, a posted write is dropped and counted, and its in-flight bus transactions are aborted. *(P)*

2.6.6 The queued work of a departed device stays in memory untouched, so the driver can read, save, resend or fail it. *(P)*

2.6.7 A device on Q64 may itself be a bus: a bridge presents the devices behind it as slots with the same registers as the bus controller, so any bus (PCI Express, USB, SATA, a legacy bus) can be connected to Q64 and its devices enumerated, to any depth, as USB devices are enumerated over a PCI Express bus. *(D)*

2.6.8 The kernel's enumeration is therefore one recursive walk: it reads a bus controller's slots, and a slot of the type "bus" is another bus controller to read. How a slot says it is a bus, how an arrival or departure behind a bridge is reported, and how the windows behind a bridge are mapped into device space are unspecified. *(S)*

2.6.9 A bus reset produces no slot events, so a reset is not mistaken for the removal of every device, and the `ID` register of every slot is defined after the bus has lost power. *(D)*

Commentary. **Precedent.** The VAX scanned its interconnect: `probenexi()` in 4.3BSD reads each nexus slot's status register, whose type field says what is plugged in; the Unibus had no enumeration at all, with devices at switch-set addresses and `badaddr()` probes, and nothing told the kernel when a device left. PCI sizes each device's windows by writing all ones to its base registers and assigns addresses; Amiga AutoConfig lets each board announce itself at a fixed address and be told where to live; NuBus gave each slot a fixed share of the address space. **Trade-offs.** A fixed page per slot needs no assignment step and limits the slot count; assignment on demand supports any layout at a few gates of logic per slot. **Improvements.** On a surprise removal PCI returns all ones, so every driver must treat all ones as a possible error; the pdp64 uses the precise bus fault the instruction set already defines, so the driver has a single recovery path. The queue's overflow rule makes boot and recovery the same code. **Benefits.** The kernel needs one fixed address, the bus controller's, to find every device, and the hand-off at boot shrinks to memory, cores and that address. **Modern comparables.** PCI Express enumerates through configuration space with hot-plug signalled by a slot-status interrupt; USB hubs report port changes through a status-change endpoint; NVMe and `virtio` keep their queues in host memory and re-queue them after a reset. **Impact.** *Kernel:* Enumeration is one loop over the slots; hot-plug is an interrupt and a queue the kernel drains. *Compiler:* None. *Hardware:* A slot register, an information register, an event queue of about eight entries and one presence bit per slot.

Commentary on 2.6.7 and 2.6.8. **Precedent.** The VAX did this: the Unibus hung off the VAX-11/780's interconnect through a Unibus adapter, a device on the nexus whose own registers include a table of map registers for DMA and whose Unibus memory (the devices' registers) appears in a window of its own (`UMEM780(i)` in `vaxuba/ubareg.h`, 4.3BSD); `unifind()` in `vax/autoconf.c` probes the devices behind each adapter. PCI-to-PCI bridges nest buses, a USB host controller (itself a PCI Express function) enumerates a tree of hubs and devices, and Thunderbolt tunnels PCI Express over a cable. **Trade-offs.** Each level adds a mapping of windows and a hop of latency, and a departure deep in the tree must be reported to a driver that does not know the path; the cure for the first is that a bridge's windows are ordinary device space (half the 40-bit address space is device space, section 2.1.6), and for the second that every bridge uses the same event register. **Improvements.** A new bus costs one bridge device and no change to the processor, the kernel's walk or the driver model, which is how the design stays small as buses change (the chapter 5 convention for storage, USB, network and display is the same pattern one level down). **Benefits.** Any bus a board offers is connected by a bridge; the PCI Express graphics card of section 3.5 is one such bridge. **Modern comparables.** PCI Express switches and bridges, USB hubs, Thunderbolt, CXL, and the device-tree and ACPI descriptions of nested buses. **Impact.** *Kernel:* one recursive enumeration routine and one hot-plug path for every bus. *Compiler:* none. *Hardware:* a bridge is a device with the standard registers plus a window translation, built once per bus type.

2.7 Still open

The bus chapter's open items, each with five options in the decisions document: the interrupt controller and routing (R1), starting the other cores and inter-processor interrupts (R2, B2), coherence beyond four cores (R3), the register layout and descriptor details (R5; chapter 5), the detail of the IOMMU (its five recommendations are accepted, errata 85), errors raised by devices (R7), virtualization (R8), and whether to use `virtio` devices in the emulator to reach an OpenBSD login sooner (Q10 and the roadmap).

Chapter 3 — Graphics: the terminal and its devices

Iteration two. *The devices of this chapter (VIC-64X with its 48-bit RGBA pixel, SID-64X with 48-bit samples, BLT64, and the eight-register face) are planned for the future, together with the author's 12-register network, sound and graphics cards (errata 75, 2026-11-03). The first iteration uses the most standard, secure and easy-to-support network, graphics and audio interfaces over PCI Express, NVMe and USB; this chapter describes what follows them. Statements keep their tags; the decisions of errata 73 and 74 stand.*

The pdp64's display is built on two machines, the Commodore 64 and the Blit terminal, modernised and extended, as memory-mapped devices: a video device, a sound device and a blitter, each with the eight-register face of chapter 5, and a driver for each that is a page of register writes. The pdp64 is an extension of the PDP-11 and its terminal is an extension of two older machines: from the C64 it takes video and sound as chips with registers, and from the Blit a framebuffer in the address space with a bit-block-transfer engine beside it. The idea in both is the same: a machine whose display is chips with registers, not a graphics card with a programming model, at a modern width. The processor never sees the display protocol: HDMI, like SATA and USB, is a transmitter chip on the board with its own standard, and the pdp64's side of it is a small set of registers. The terminal is the first thing a user sees and the first device the boot software drives. Tags: *(D)* decided; *(P)* proposed; *(S)* stub.

3.1 The terminal: VIC-64X, SID-64X and BLT64

3.1.1 The graphics terminal is based on two machines, not one: the **Commodore 64**, for its video and sound as chips with registers that a program writes, and the **Blit terminal** (Rob Pike's bit-mapped windowing terminal), for its framebuffer in the address space and the bit-block-transfer operation that moves rectangles of pixels. The pdp64 modernises and extends both. *(D)*

3.1.2 The video device VIC-64X is the Commodore 64's video chip (the VIC-II) extended: a framebuffer in the address space with 48-bit pixels, scanned out through a line buffer. A pixel is RGBA with 12 bits per channel (errata 73, the author's decision of 2026-10-24). The register formats and the other pixel formats are unspecified (3.2). *(D)*

3.1.3 The sound device SID-64X is the C64's sound chip (the SID) extended: voices with filters, and also a general sound path that is not limited to the SID's voices; audio samples are 48 bits (errata 74, the author's decision of 2026-10-24). The register formats and the layout of a 48-bit sample are unspecified (3.2). *(D)*

3.1.4 The blitter BLT64, the Blit's bit-block-transfer operation as a device, copies and combines rectangles of pixels with raster operations: registers for the source and destination base addresses, the strides, the width and height, a raster operation (the sixteen binary operations of source against destination and the common ternary ones), a mask or transparent colour, and a control register with direction, go, busy and interrupt on done. The device chat chose a three-operand `draw` instead (`decisions-pending.md` E3). *(P)*

3.1.5 Each of the three is a device in a slot: it is enumerated by the bus, mapped into its own 4 KB page, and found by its identification register like any other (chapters 2 and 5). The device chat put the C64 devices in one 8 KB window per process (`decisions-pending.md` E5). *(P)*

3.1.6 The framebuffer stays plain memory, so that software can draw without BLT64; the blitter is an assist that can be bypassed, not the only way to draw (the Blit authors' advice in 1985 was "transparent, so it can be bypassed"). *(P)*

3.1.7 Four raster operations are required of BLT64: store, OR, clear and exclusive-OR (these four sufficed on the Blit, and exclusive-OR dominated its use); the other twelve binary operations and the ternary ones are optional. *(P)*

3.1.8 Whether the video device has a hardware cursor or sprite is unspecified (the vertical-blank interrupt is 3.1.9); the Blit read the mouse position at the retrace interrupt and drew its cursor in software, in exclusive-OR. *(S)*

3.1.9 Every register of the video device that takes effect at a frame boundary is shadowed: one write to its `ARM` register latches them all together at the start of the next vertical blank, and a status bit says whether the arm was taken in this frame or missed it. A sticky underrun bit and an underrun counter report a scan-out that ran dry. The vertical-blank interrupt occurs at a fixed line of the frame, whose number a register holds. The sound path's audio clock relation (3.3.4) is part of the same mode setting. *(D)*

Commentary. **Precedent.** The Commodore 64's VIC-II and SID were chips whose registers a program poked directly, with a screen in memory that the video chip read by direct memory access; **the Blit**, Rob Pike's terminal, had a framebuffer in the address space and a bit-block-transfer operation that the terminal did in software (Pike, Locanthi and Reiser, 1985: the terminal had "no pixel-moving hardware"), where the Amiga and other 1980s machines added hardware blitters. The Blit was a Motorola 68000 machine (8 MHz) that received downloaded code, characters and editing messages from its host over a serial line and drew every pixel itself; its description as a precursor of the X11 protocol and of the NeWS window system is not stated by any source read *(S)*, while the lineage the sources confirm runs from the Blit's `mux` to 8½ and `rio` (`docs/kernel-pain-points.md` section 14I). The display was a small computer driven by messages, where here it is a set of devices on the bus driven by registers and descriptors, so the pdp64 needs no display protocol between host and terminal (the nearest thing is the descriptor ring of chapter 5); the pdp64's terminal takes the framebuffer and the bit-block-transfer operation from the Blit (the pdp64 adds the hardware engine, `BLT64`) and the video and sound chips from the C64, and both are modernised. The Blit authors give the cost of the split (a slow link, and a program in two halves) as the reason their design was what it was; the pdp64 avoids the split by putting the display on the bus. Plan 9 later moved its window system into files (`/dev/bitblt`, then `/dev/draw`) for the sake of size and remote use: a stream of small commands written to a device, the same shape as an entry in a descriptor ring (reading of the commands as a ring is an inference, not a source's statement). **Trade-offs.** The blitter covers scrolling a terminal, moving a window, drawing a glyph, filling a rectangle and exclusive-oring a cursor, which is the working set of a text-and-windows machine; it does not do three-dimensional graphics. `MCPY` is no substitute: it is one-dimensional and occupies the processor, where a blit has a stride and a raster operation and runs concurrently. 48-bit pixels (presumably 16 bits per channel) exceed what any display connector carries today, so the display device converts to the connector's depth on the way out; 48-bit audio samples likewise exceed what a converter takes, so the sound device keeps the precision for mixing and filtering and reduces it at the output. **Improvements.** The C64's video and sound registers were scattered and irregular; the pdp64 versions use the uniform register face, so one driver skeleton serves them. **Benefits.** The blitter is about 1,500 lookup tables and a line buffer, nearly all address arithmetic; the whole terminal is a handful of small devices. **Modern comparables.** Every embedded display controller has a 2-D blitter and a scan-out engine; sound chips are audio codecs behind an I2S interface. **Impact.** *Kernel:* a framebuffer, a sound and a blit driver are each a mapped page and a few register writes; no cache maintenance is needed because the framebuffer is uncached. *Compiler:* none. *Hardware:* a scan-out engine, a line buffer, a small address-arithmetic unit for the blitter and a mixer for the sound device, each independent of the processor and of the others.

3.2 The stub: the C64 and Blit documentation

3.2.1 The register-level definitions of `VIC-64X`, `SID-64X` and `BLT64` (register names, offsets, bit fields, the 48-bit pixel and sample formats, the sprite, raster-interrupt and voice models inherited from the C64, and the bit-block-transfer operations and windowing model inherited from the Blit) are a stub: only the `BLT64` registers listed in 3.1.4 are specified. The author's earlier device chat decided much of the rest (register conventions, the 8502 island, the display rules); it is extracted in `docs/c64-devices-from-chat.md` and awaits adoption (`decisions-pending.md` E1 to E11). *(S)*

3.2.2 What is fixed: the devices exist, they follow the eight-register convention in the terminal's slots, and the interrupt, `FENCE` and uncached-framebuffer rules of section 3.4 apply. *(S)*

Commentary. This section is the placeholder for the C64- and Blit-derived specification. The Blit side draws on the papers read in section 14I of `docs/kernel-pain-points.md` (the four primitives `bitblt`, `texture`, `segment` and `point`, the `Bitmap`, `Rectangle`, `Point` and `Texture` types, half-open rectangles, and overlapping windows with obscured parts kept off screen); the video chip is built after the platform it runs on. **Impact.** *Kernel:* none until defined. *Compiler:* none. *Hardware:* the block RAM budget (section 3.4) is settled before the video device is built.

3.3 How the terminal drives HDMI and the other ports

3.3.1 The display connector is standard kit: HDMI (and DVI and DisplayPort) is a transmitter chip that takes pixels and sync signals in parallel and does the link protocol, the encoding and the electrical signalling itself; on the first board it is the DE10-Nano's ADV7513, set up once over an I2C-style control bus. *(D)*

3.3.2 The video device's output side is therefore a pixel stream with horizontal and vertical sync, a clock and a data-enable, which is what every such transmitter accepts; the device's registers set the mode (width, height, blanking, pixel clock, the pixel format), the base address of the framebuffer and the display on and off. *(P)*

3.3.3 Other ports are the same shape: a different transmitter (VGA digital-to-analogue converter, DisplayPort, an LVDS panel) takes the same pixel stream, so the pdp64's side does not change, and each port is one more slot or one more output of the video device. *(P)*

3.3.4 The sound device's output is likewise a standard digital audio stream (I2S or the HDMI audio channel) into a converter or the transmitter. The processor never sees the HDMI packets, but whoever sets the mode must also program the audio clock relation: the transmitter needs N (and CTS, or a hardware CTS calculation) for each pixel clock and sample rate, and a driver such as the ADV7511's computes both. *(P)*

3.3.5 The ports that carry data, SATA for storage and USB for peripherals, follow the same pattern: the protocol is in the chip or controller block, and the processor's side is a slot with the common registers and a ring (chapter 5). *(D)*

Commentary. **Precedent.** The MiSTer framework on the DE10-Nano already does exactly this (the retro cores produce a pixel stream and the framework's scaler and the HDMI transmitter do the rest), which is why the plan buys HDMI there. **Trade-offs.** The control bus of the transmitter needs a small amount of one-time setup that a boot routine does by writing a few registers; the pixel clock must be generated to the mode, which on an FPGA is a phase-locked loop and in silicon is the same block. **Improvements.** Because the interface is a stream and a register set, replacing the transmitter or the connector changes nothing above the video device, as a SATA controller behind its ring can be replaced without changing the disk driver. **Benefits.** No part of the operating system carries an HDMI, SATA or USB protocol stack: each is one driver of the common shape. **Modern comparables.** Single-board ARM computers with a display controller and an HDMI transmitter; the video output of every SoC. **Impact.** *Kernel:* a display driver sets a mode and a base address and nothing else; hot-plug of a monitor is a bus event if the transmitter reports one. *Compiler:* none. *Hardware:* a pixel generator, a clock and an off-chip transmitter; no protocol logic in the pdp64's own gates.

3.4 The framebuffer, the memory type and the memory budget

3.4.1 The framebuffer is mapped with the uncached memory type, or the processor's writes sit in the data cache while the video device scans out stale memory; a `fence` separates writing a device's registers from setting its go bit. *(D)*

3.4.2 On the first board the block RAM (about 5.5 megabits on the DE10-Nano) cannot hold a modern frame (a 1920 by 1080 frame at 32 bits is 8.3 MB, and at the 48-bit pixel of 3.1.2 it is 12.4 MB), so the framebuffer lives in main memory and the block RAM holds the scan-out line buffer of a few kilobytes. *(D)*

3.4.3 The block RAM budget (caches, translation buffer, the blitter's scratch and the sound voices' state) is settled before the video device is built. *(D)*

3.4.4 If the address-space split of section 2.1.6 is adopted, the framebuffer is ordinary memory in the lower half and the devices' registers are in the upper half, so a page-table entry that maps the framebuffer as a device (or a register page as cached memory) is detectably wrong. *(P)*

Commentary. **Trade-offs.** An uncached framebuffer makes a single pixel write slow and a long run of writes no slower per byte, which suits the blitter (it writes the bus directly) and punishes pixel-by-pixel drawing from the processor, which is what a blitter is for. **Modern comparables.** Write-combining memory types on x86 and ARM serve the same purpose. **Impact.** *Kernel:* the mapping rule is one memory-type value in the page-table entry. *Compiler:* none. *Hardware:* the scan-out reads main memory by bus mastering, which competes with the processor for bandwidth.

3.5 A standard graphics card

3.5.1 A PCI Express graphics card is not excluded by the architecture: a bridge could present PCI Express functions as slots on the bus, enumerated like any other device. *(P)*

3.5.2 Against a standard card: PCI Express configuration space, base-address mapping and message-signalled interrupts, a driver stack for a programming model that for some vendors must be reverse-engineered, the card's firmware and the initialisation a PC's firmware performs, and no hard PCI Express block on the first board (the Cyclone V and the Zynq 7020 lack one; a Zynq UltraScale+ has it). *(D)*

3.5.3 On the other side PCI Express is a standard enumerated bus, which suits the bus-as-a-device design (each function a slot, as each USB device is), and OpenBSD carries graphics drivers for several vendors' cards ported from Linux *(not confirmed in the sources read). (P)*

Commentary. **Trade-offs.** The terminal described here works on the first board and costs a few thousand lookup tables; a standard card needs a root complex, which is a large block when it is not already on the chip, and a graphics driver ported to the pdp64, which is a larger task than the architecture. **Modern comparables.** PowerPC and RISC-V workstations plug in PCI Express graphics cards with open drivers. **Impact.** *Kernel:* a PCI Express card would need a large graphics driver ported. *Compiler:* none. *Hardware:* a PCI Express root complex and bridge to the bus.

Chapter 4 — Boot, the BIOS, the disk label and the GPT

The machine starts the way the PDP-11 did: a small ROM in the controller chip loads one block from the boot device and jumps to it, and every later stage is an ordinary program, open source, compiled by the project's own compiler. The chapter covers what starts the machine, what takes the place of a BIOS, how the disk is laid out, how other cores and devices are started, and why UEFI is not used. The full review of how other systems boot is `../boot.md`. Tags: *(D)* decided; *(P)* proposed.

4.1 What takes the place of a BIOS

> ***PCI Express.*** *Since errata 75 and 80 (2026-11-03) Q64 is PCI Express, backward compatible, with convenience and security circuitry added on top, and storage is NVMe and USB; the standard network, graphics and audio devices are virtio-net, virtio-gpu and virtio-snd over the virtio PCI Express transport (errata 79). Devices are enumerated by PCI Express configuration space and base-address registers and interrupt by MSI-X, which replace the slot numbers, the slot-derived vectors and the eight-register device rule of the statements below (errata 80, in outline). Where this chapter finds devices by slot, the boot ROM finds them by PCI Express enumeration; the full rewrite of the chapter waits for the author's answers to the remaining items of* `decisions-pending.md` *Part F. Statements keep their tags.*

4.1.1 The pdp64's "BIOS" is the controller chip's boot ROM and its monitor. It brings up the machine, loads the first block, and offers a console monitor when nothing boots. It has no resident services and nothing is called after boot. *(P)*

4.1.2 The boot software is open source and part of the specification: the ROM and monitor, the first-block program, the second-stage loader and the boot tools. *(D)*

4.1.3 Everything in the project, the boot software included, is under the BSD 2-clause licence. *(D)*

Commentary. **Precedent.** A BIOS did four jobs: hardware bring-up and self-test, the first code (read the boot sector and jump), a runtime library of device services called by software interrupt (the "I/O" in its name), and configuration. UEFI replaced the second and third with a loader file and a set of boot and runtime services, and the other architectures have the same roles under other names: Open Firmware on SPARC and PowerPC, U-Boot with trusted firmware on ARM, OpenSBI on RISC-V, the SRM console on Alpha, the console processor on the VAX, and the boot ROMs and `ODT` monitor on the PDP-11. **Trade-offs.** The pdp64 needs only the first, second and fourth jobs in a small form. The third disappears because the devices have one uniform register layout (chapter 5) that the loader and the kernel share, so the loader needs no firmware to read a disk. The cost is that the second-stage loader carries its own drivers, which is small because a driver drives a small register set (the common eight and a few of its own). **Improvements.** There is no firmware API to specify, no compatibility layer, no resident management code and no closed blob at the bottom of the trust chain. **Benefits.** The failure cases reduce to one: the first block did not load, and the monitor is the recovery. **Modern comparables.** x86 firmware keeps resident system-management code that the operating system cannot see; arm64 and RISC-V servers need trusted firmware and, usually, UEFI beside it. **Impact.** *Kernel:* There are no firmware services to call or to bug-fix. *Compiler:* The ROM, loader and tools are built by the project's own compiler. *Hardware:* A boot ROM of a few hundred bytes to a few kilobytes, with a driver per boot device.

4.2 The reset state and the first block

4.2.1 At reset the controller brings up memory, holds every core in reset, and looks for a boot device in a fixed order: disk or flash, then SD card, then the serial line. *(P)*

4.2.2 It reads block 0 of the first device that answers into low memory, checks the `0x55AA` signature in the last two bytes, and starts core 0 at that address. It does not parse a partition table. *(P)*

4.2.3 Core 0 starts in kernel mode at priority 7 with translation off (every address physical), `VBR` zero and the user-memory guard clear; `r0` holds the boot device number, `r1` the address of the boot-information block, `r14` the top of the first 64 KB of memory, and the other registers are zero. *(P)*

4.2.4 If no boot device answers, or a key is held at power-up, the monitor runs on the serial console. *(P)*

Commentary. **Precedent.** The PDP-11 read block 0 of the boot device into low memory and jumped to it, from a boot ROM or a few words keyed in on the front panel; the VAX console processor did the same with the boot block and passed the boot device in registers; the mainframe's IPL record and the Alpha console's boot block are the same idea. In V7 the second stage was about 250 lines of PDP-11 assembler that read a pathname and loaded the kernel. **Trade-offs.** A boot ROM with a driver per boot device is more code in the controller than a "jump to a fixed flash address" scheme, and much less than firmware; the first block must fit in one sector. **Improvements.** The reset state is specified in full, where the earlier machines left it to the console. **Benefits.** The first stage is the same on every board. **Modern comparables.** A PC's BIOS reads one 512-byte sector; ARM boards load a first-stage loader from the boot medium; UEFI runs a file from a FAT partition. **Impact.** *Kernel:* The reset state is fully specified, so the first kernel code makes no guesses. *Compiler:* The first block is 446 bytes of assembler. *Hardware:* The ROM reads one block and starts one core.

4.3 The disk: a GPT around the BSD label

4.3.1 The disk carries a GUID partition table with a protective MBR; the operating system's own partition holds the BSD disk label, which names the root, swap and other partitions. *(D)*

4.3.2 Block 0 is the protective MBR, so the first stage lives in its 446 boot-code bytes, carrying a short block list (start block, count, load address) to the second stage; neither the ROM nor the first stage parses the GPT. *(P)*

4.3.3 Inside the BSD partition, the label is at sector 1, offset 0; the boot area is 8192 bytes at the start of the partition; the file system begins after it, its superblock at offset 8192. *(D)*

Sector (512 bytes) of the BSD partition	Contents
0	the partition's own boot block
1	the disk label, at offset 0
2 to 15	the rest of the 8 KB boot area
16 on	the file system, superblock at offset 8192

4.3.4 Sector counts are 64-bit throughout; the label has 16 partitions and its sizes carry high fields beyond 32 bits. *(D)*

Commentary. **Precedent.** V7 had no partition table on the disk: each driver compiled in an array of eight sizes and starting cylinders (`hp_sizes[8]` in `hp.c`), and changing the layout meant recompiling the kernel. The disk label arrived with 4.3BSD-Tahoe in 1988, in sector 0 or 1 "possibly offset from the beginning to leave room for a bootstrap", with the magic number `0x82564557`; OpenBSD's label is its descendant, with the same magic number, 16 partitions and 64-bit sizes. In the OpenBSD tree seven architecture headers put the label at sector 1 offset 0 (amd64, i386, arm, arm64, riscv64, hppa, powerpc64); Alpha and SPARC64 put it at sector 0. The PC used the MBR, a four-partition table in sector 0 with 446 bytes of boot code, and UEFI requires the GPT, whose header is in the second block, with a backup at the end of the disk and a protective MBR in block 0. **Trade-offs.** The label alone would have been the smallest design and would have been unreadable to other systems' tools; the GPT wrapper costs a GPT reader in the second stage and the kernel (OpenBSD's kernel already has one) and puts the protective MBR in block 0, and gives a disk that other operating systems and partitioning tools recognise. Following the majority on the offsets means the port adds one header of a dozen lines. **Improvements.** The first stage does not depend on a GPT that may be damaged, so a corrupted table cannot stop the machine reaching the second stage, which can repair it. **Benefits.** The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. **Modern comparables.** BSDs on UEFI machines use a GPT with the label inside an OpenBSD-type partition and a FAT partition for the loader; Linux has no scheme of its own. **Impact.** *Kernel:* The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. *Compiler:* None. *Hardware:* None.

4.4 The second stage and the hand-off

4.4.1 The second stage is a program in C that understands the root file system, reads the GPT and the label, loads `/bsd` (an ELF file) and jumps to it with `r0` and `r1` as before and a command line in the boot-information block. *(P)*

4.4.2 The boot-information block names the memory ranges, the core count, the address of the bus controller and the slots of the boot device and the console; the kernel enumerates every other device by asking the bus. *(P)*

4.4.3 The block's layout is defined once, in a grammar from which the headers are generated, and carries a checksum and a version. *(P)*

4.4.4 The boot-information block carries the video device's base address, size, pitch and pixel format when the boot loader has set it up, and nothing about the device's clock or power state is changed behind the kernel: its pixel clock stays up until the kernel's driver owns the device. *(D)*

Commentary. **Precedent.** The VAX passed a restart parameter block, the Alpha a hardware restart parameter block, both fixed-layout records of the machine; the PDP-11 configured the kernel for the machine at build time and probed device addresses; x86 uses ACPI tables, and arm64 and RISC-V use a device tree. **Trade-offs.** A fixed-layout block with a checksum has two failure cases (bad checksum, unknown version), against the parser a device tree or ACPI requires; a device tree has the advantage that OpenBSD's arm64 and riscv64 ports already parse one and attach drivers from it, which could save kernel code if the port reuses many of their drivers. **Improvements.** Because the bus describes its own devices, the block shrinks to memory, cores and the bus controller's address. **Benefits.** The hand-off is a struct; the generated header cannot disagree with the loader. **Modern comparables.** The device tree and ACPI are the two standard answers. **Impact.** *Kernel:* The hand-off is a structure the kernel reads. *Compiler:* The header is generated from one grammar, so it cannot drift from the loader. *Hardware:* None.

4.5 Starting the other cores and the devices

4.5.1 The controller holds every core except core 0 in reset; a write to a core's start register in the controller gives it an entry address and an argument and releases it into kernel mode at priority 7, with its core number in `r0` and the argument in `r1`. *(P)*

4.5.2 A stuck core is reset by a write to a core-reset register. *(P)*

4.5.3 A device is initialised by writing its `INIT` bit and waiting for `READY`; a device that wants firmware is given it through a descriptor operation once the root file system is available, and a device the boot depends on needs no firmware. *(P)*

Commentary. **Precedent.** On x86 the boot core sends INIT and START-UP interrupts through the APIC and each other core wakes in 16-bit real mode, then has to be walked through protected mode into 64-bit mode by a trampoline: the Linux `smpboot.c` is 1,528 lines and the real-mode code another 1,258. arm64 uses a firmware call (PSCI) in 124 lines or a spin table in 127; RISC-V asks the firmware to start a hart in 108 lines or spins in 58. The VAX's console processor started its other processors. **Trade-offs.** The pdp64 has no mode ladder, so a core can start in its normal mode at any address and the whole protocol is one register write, at the cost of a small register block in the controller. **Improvements.** There is no trampoline and no firmware call. **Benefits.** Fewer lines than any system measured and one recovery path, a core reset. **Modern comparables.** PSCI, the SBI hart-state calls, and x86's INIT and START-UP sequence. **Impact.** *Kernel:* Starting a core is a register write and a device start is `INIT`. *Compiler:* None. *Hardware:* A start register and a reset hold per core; an `INIT` bit per device.

4.6 Why UEFI is not used

4.6.1 The pdp64 does not use UEFI: its boot is the block-0 route of sections 4.2 and 4.3, its hand-off is the boot-information block, and its devices describe themselves. *(P)*

4.6.2 The hand-off and the second stage are kept simple enough that a UEFI shim could be written by someone who wanted one. *(P)*

Commentary. **Precedent.** UEFI is a firmware specification of boot services, runtime services, a memory map, the GPT, the PE/COFF executable format and a driver model, implemented usually by the open-source EDK II or by a vendor. Its aim is that one installer boots on any machine that offers it. OpenBSD's arm64 and riscv64 ports use UEFI and a device tree, and its own loaders for them are 7,195 and 4,513 lines; for amd64 the loaders total 15,440 lines, including a BIOS boot block of 862 lines and a second stage of 189 beside the UEFI loader of 4,593. **Trade-offs.** UEFI would let existing generic loaders run unchanged, offers a standard memory map and a secure-boot chain, and costs a great deal of firmware (an EDK II port to a new architecture, in a new executable format), adds failure modes (variable storage, drivers, boot-order state), enlarges the attack surface, and still needs a device tree or ACPI to describe the devices. **Improvements.** The pdp64 has one architecture and one platform description, which is the case UEFI was not built for. **Benefits.** Smaller (principle 1), fewer failures (principle 2), fewer gates and less flash (principle 3). **Modern comparables.** x86 servers, arm64 servers and RISC-V servers all use UEFI; OpenBSD itself still ships a BIOS-style loader beside the UEFI one on amd64. **Impact.** *Kernel:*

No firmware interface, no UEFI services, no memory-map translation. *Compiler:* No PE/COFF format in the toolchain. *Hardware:* A smaller flash and no firmware stack.

4.7 Recovery

4.7.1 The controller's ROM includes a monitor reached on the serial console when nothing boots: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. *(P)*

4.7.2 A hardware watchdog restarts the machine if the kernel hangs. *(P)*

Commentary. **Precedent.** The PDP-11's console `ODT`, a monitor in ROM for examining and depositing memory and starting at an address, and the VAX console processor were the way in when nothing booted. **Trade-offs.** A monitor is a few hundred bytes of ROM and needs a working serial port and nothing else. **Benefits.** The same monitor brings up a new board before any boot device works, and it is the simplest recovery path of all: it needs the least code to write. **Modern comparables.** U-Boot's console and OpenSBI's debug console fill this role; PC firmware has a setup screen. **Impact.** *Kernel:* A monitor is the way in when nothing boots. *Compiler:* None. *Hardware:* A serial port and a few hundred bytes of ROM.

4.8 Still open

The boot chapter's open items, in `../boot.md`: the address the first block is loaded to, the partition type of the boot partition (and whether a dedicated boot partition is needed at all, since the first stage's block list can point at the second-stage file in the root file system), the device-tree option, the details of the GPT, which are not checked against the UEFI specification, and the PDP-11 power-up behaviour and the VAX and Alpha boot protocols, which are not checked against their manuals.

Chapter 5 — The device driver model

A driver on the pdp64 is a short ordinary program: it maps a device's page, writes its registers (at least eight), hands the device descriptors in memory, and finds them marked done, and because the device is always the same shape the same driver skeleton serves a disk, a network card and a serial line. The chapter gives the driver writer's view of the memory-mapped registers and of DMA: what a driver sees, the uniform eight-register convention and its descriptor, the life of a driver from probe to detach, how data moves and what the driver need not do, the default devices, how errors and departures are handled, interrupts, and how this maps onto OpenBSD's driver interfaces. The bus the devices sit on is chapter 2. Tags: *(D)* decided; *(P)* proposed. A driver is small because a device is eight registers.

5.1 What a driver sees

> ***PCI Express.*** *Since errata 75 and 80 (2026-11-03) Q64 is PCI Express, backward compatible, with convenience and security circuitry added on top, and storage is NVMe and USB; the standard network, graphics and audio devices are virtio-net, virtio-gpu and virtio-snd over the virtio PCI Express transport (errata 79). Devices are enumerated by PCI Express configuration space and base-address registers and interrupt by MSI-X, which replace the slot numbers, the slot-derived vectors and the eight-register device rule of the statements below (errata 80, in outline). The eight-register convention and its descriptor describe the pdp64's own devices (the iteration-two cards of chapter 3) and the shape of a driver; the standard devices of iteration one are PCI Express devices with their own drivers; the full rewrite of the chapter waits for the author's answers to the remaining items of* `decisions-pending.md` *Part F. Statements keep their tags.*

5.1.1 A device is at least eight 64-bit registers at the start of its own 4 KB page (a larger device takes more pages, section 5.2.1), and the page is mapped with the device memory type, strongly ordered. *(P for the page per device; D for the memory types)*

5.1.2 A driver reaches a register with an ordinary load or store; there are no input or output instructions and no port space. *(D)*

5.1.3 The registers are 64 bits wide, naturally aligned and accessed whole; the machine is little-endian throughout, so a driver has no byte-order or byte-lane cases. *(D)*

5.1.4 Accesses to a device page happen in program order; one `FENCE` separates the last write to memory the device will read from the write of the doorbell. *(D)*

Commentary. **Precedent.** A V7 driver is a structure laid over a fixed address: `rk.c` declares `struct device { int rkds; int rker; int rkcs; int rkwc; caddr_t rkba; int rkda; }` and a pointer to it at `0177400`, and reads and writes the fields. **Trade-offs.** One page per device wastes address space and no memory, and lets the kernel give one device to one protection domain. **Improvements.** The register's address is a virtual address the kernel maps, so the driver is position-independent and the same code serves any slot. **Benefits.** There is no accessor layer to learn and nothing to get wrong with endianness or alignment. **Modern comparables.** Linux and BSD wrap register access in an accessor (`readl`, `bus_space_read_8`) to hide port and memory differences and byte order; on the pdp64 those wrappers reduce to a load or a store. **Impact.** *Kernel:* No accessor layer: a register access is a load or a store. *Compiler:* Register access is a 64-bit `volatile` load or store through a pointer to a structure. *Hardware:* A device is decoded by page address; its eight registers are an eight-way decode within the page.

5.2 The device convention: eight registers (at least) and DMA

5.2.1 Each device is at least eight memory-mapped registers and moves its data by DMA. *(D)* Eight is the minimum every device has, not a limit: a device that needs more (a controller with many queues, a framebuffer, a display's mode and palette registers) maps a larger window in its slot, the first eight registers keep the common layout, and the rest are the device's own. *(D)*

5.2.1a The storage interfaces are all this one pattern: SCSI, ATA, SATA and NVMe each present a command queue in host memory, a doorbell and a completion in memory, which is the ring and the descriptor of 5.2.3 with a different descriptor body; an NVMe controller's submission and completion queues are the closest existing form of it. A disk driver for any of them is the same skeleton with a different command encoding. *(D)*

5.2.2 The first three registers are the same in every device: `ID`, `CONTROL` and `STATUS`. *(P)*

Offset	Register	Meaning
`0x00`	`ID`	type, variant, version, feature bits (read-only)
`0x08`	`CONTROL`	enable, `INIT` (initialise yourself), interrupt enable
`0x10`	`STATUS`	`READY`, `ERROR`, `BUSY`, error code
`0x18`	`RING_BASE`	physical address of the descriptor ring
`0x20`	`RING_SIZE`	the ring's size in entries (a power of two)
`0x28`	`DOORBELL`	write the producer index to start work
`0x30`	`IRQ`	cause bits (complete, error, attention); write 1 to clear
`0x38`	`DEVICE`	device-specific (for the polled devices, the data register)

5.2.3 A transfer is a 32-byte descriptor in a ring in host memory: four 64-bit words giving an operation and flags, a status field the device writes, a length, a buffer address, an argument (a block number, for a disk) and a result the device writes. *(P)*

```
word 0:  OP(8) FLAGS(8) STATUS(16) LENGTH(32)     FLAGS: OWN, STARTED, INTERRUPT, MORE
word 1:  ADDRESS (physical)
word 2:  ARGUMENT
word 3:  RESULT (written by the device)
```

5.2.4 The host sets `OWN` when it hands a descriptor to the device; the device clears it when it is finished, after writing the data, the status and the result. Progress is therefore in memory, not in the device. *(P)*

5.2.5 `INIT` makes a device initialise itself without touching the ring; a device may take a firmware image through an ordinary descriptor operation; boot-critical devices work without any image. *(P)*

5.2.6 Operations common to all devices: no-op, identify (the device writes an identification block into a buffer) and load-firmware; each device adds its own from `0x10`. *(P)*

5.2.7 A device that carries traffic in two directions (a network card, a serial line) is two slots, one ring each; a console, a timer and an entropy source are polled devices with their data in the last register. *(P)*

Commentary. **Precedent.** The V7 RK11 disk is six registers (drive status, error, control and status, word count, bus address, disk address) with DMA by a word count and a bus address; the 4.3BSD DEUNA Ethernet is four registers plus transmit and receive rings in memory, each ring entry holding a segment length, an address, flags and errors, with an `OWN` bit that is zero when the descriptor belongs to the driver. The proposal is the second design, kept uniform. **Trade-offs.** A single ring per device makes duplex devices two slots, which is simple and slightly unusual, in exchange for in-order completion and no out-of-order bookkeeping; the ownership bit costs one status write per descriptor. **Improvements.** Because progress lives in the descriptor and not in a device register, a driver can recover from a departed or reset device with one routine: scan the ring for descriptors that are not done, then read, save, resend or fail them. **Benefits.** A driver is a skeleton shared by all devices, and the hardware is one reusable ring engine that fetches a descriptor, hands the operation to the device's own logic and writes the status back. **Modern comparables.** `virtio` rings, NVMe's submission and completion queues, the descriptor-done bit of Intel Ethernet controllers and the command lists of SATA controllers are all rings with a done mark; the 16550 serial chip is the classic eight-register device. **Impact.** *Kernel:* One driver skeleton serves every device, and the layout is the same for a disk, a network card and a serial line. *Compiler:* The descriptor is a 32-byte aligned C structure, and register offsets are generated constants. *Hardware:* One reusable ring engine fetches a descriptor, hands the operation to the device's logic and writes the status back.

5.3 The life of a driver

5.3.1 *Probe.* The kernel reads the bus controller's slots (section 2.6), reads each present device's `ID` register, and matches the type and version against the drivers it has. *(P)*

5.3.2 *Attach.* The driver maps the slot's page, writes `INIT`, waits for `READY`, and reads the device's identification block by submitting an identify descriptor. *(P)*

5.3.3 *Run.* The driver puts the ring in uncached memory, writes `RING_BASE` and `RING_SIZE`, fills descriptors with `OWN` set, executes `FENCE`, and writes `DOORBELL`; a descriptor is complete when the device has cleared `OWN`, which the driver sees by polling or by the interrupt. *(P)*

5.3.4 *Fail.* A failed descriptor completes with an error code in its status; a device that stops answering is reset with `INIT` and its ring reprogrammed from the first descriptor that is not done. *(P)*

5.3.5 *Detach.* A departure event, or a bus fault from an absent slot, tells the driver to stop; it reads, saves, resends or fails the descriptors still queued. *(P)*

Commentary. **Precedent.** A V7 disk driver is a strategy routine that queues a buffer, a start routine that writes the registers, and an interrupt routine that completes the buffer; 4.3BSD's Unibus drivers add a probe routine that tests whether a device answers at an address and an attach routine, which BSD's autoconfiguration calls in order. **Trade-offs.** The five steps are the same for every device, so the work that differs is the personality: what the operations mean. **Improvements.** Probe reads the bus and not a configuration file, and detach is a defined path because the bus reports departures. **Benefits.** A new device is a new set of operations in a driver that already has the skeleton. **Modern comparables.** OpenBSD's autoconfiguration calls a driver's `match`, `attach` and `detach` functions; Linux's driver model has `probe` and `remove`. **Impact.** *Kernel:* The five steps are the same for every driver, so only the operations differ. *Compiler:* Nothing special. *Hardware:* The `INIT` and `READY` handshake is a small state machine in each device.

5.4 DMA and the hand-off

5.4.1 Buffers and descriptors are in memory mapped uncached, and the driver does no cache clean or invalidate. *(D)*

5.4.2 A device addresses physical memory directly, with 40-bit addresses, and the driver gives it physical addresses; until an address-translation unit for devices exists, every device must be able to address all of memory. *(P)*

5.4.3 A buffer that crosses a page is described by several descriptors chained with the `MORE` flag; the driver does not make buffers contiguous. *(P)*

5.4.4 The device writes the data, then the status and result, then clears `OWN`; the driver reads `OWN` and then the data. *(P)*

Commentary. **Precedent.** The Unibus carried 18 address bits, so the VAX and the larger PDP-11s had map registers (`ubalonce` in 4.3BSD) translating device addresses to memory, and the VAX's adapter had data paths a driver had to purge before reading; the 11/70's cache watched DMA writes so that the data appeared coherent. **Trade-offs.** Uncached buffers make the processor's reads of received data slow, in return for a driver that cannot forget a cache operation; a non-coherent device that came later would call for cache-maintenance instructions. No address-translation unit means no protection from a faulty or malicious device. **Benefits.** The sequence build, `FENCE`, doorbell, complete is the whole of the hand-off. **Modern comparables.** ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer that places cache maintenance; x86 DMA is coherent; `swiotlb` in Linux copies for devices that cannot address all memory. **Impact.** *Kernel:* No cache clean or invalidate in any driver, so none can be forgotten. *Compiler:* None. *Hardware:* DMA goes around the processor's data cache, so the cache needs no snoop port for devices.

5.5 The default devices

5.5.1 The **disk** has the operations read, write and flush at 512-byte sectors, with the block number in the argument word; its identification block gives the capacity in sectors and the sector size; the boot ROM reads block 0 with one descriptor. *(P)*

5.5.2 The **network card** is a transmit slot and a receive slot: the transmit ring carries frames to send, the receive ring carries buffers the host posts and the device fills, completing each with the frame length; link changes raise an attention interrupt. *(P)*

5.5.3 USB is presented by a controller chip that does the protocol and the enumeration and shows each attached device as a slot on the bus with the personality of its class: a mass-storage device looks like a disk, a keyboard or mouse like a receive-only device, a serial adapter like a console. *(P)*

5.5.4 The **console** is a polled serial device with a transmit-ready and a receive-ready bit and one data register; the **timer** is a 64-bit free-running counter, a compare register and a control register; the **entropy source** returns 64 random bits from its data register and fills buffers by descriptor. *(console and timer D; entropy P)*

5.5.5 A device raises one interrupt line at the level its slot asks for, with the vector numbered from the slot. *(P)*

Commentary. **Precedent.** The PDP-11 had a disk with eight registers, a serial line with four, and network controllers with rings; this is the same set with a common layout. **Trade-offs.** USB in eight registers is the hard case: a conventional host controller exposes command, event and transfer rings and many registers. Presenting each device as a bus slot moves the protocol into the controller chip, which costs gates or a small embedded processor in the controller and takes the USB stack out of the operating system; the alternative is a conventional ring-based host controller, which an existing operating-system USB stack can drive at the price of that large stack. The choice between them is undecided. **Improvements.** Treating every USB port as a hot-pluggable slot reuses the bus's own enumeration and hot-plug machinery instead of a second one. **Benefits.** A disk driver written for the bus disk works for a USB disk. **Modern comparables.** `virtio` block and network devices have the same shape; xHCI is the conventional USB controller; a USB-to-Ethernet adapter appears to the host as a network class device. **Impact.** *Kernel:* A disk driver written for the bus disk also drives a USB disk; the network card is two slots. *Compiler:* None. *Hardware:* Each personality is a small block of logic behind the common ring engine; the USB protocol, if done in the controller, is where the gates go.

5.6 Errors, departure and recovery

5.6.1 A descriptor's status carries a code: success; bad operation; bad address (a DMA access faulted); bad length; media error; not ready; aborted (the device was reset); bad image. *(P)*

5.6.2 A device that is reset with `INIT` leaves its ring in memory untouched and clears its ring registers; the driver reprograms them and starts at the first descriptor that is not done. *(P)*

5.6.3 A device that leaves the bus leaves its queued work in memory; the driver finds it by scanning the ring from its last completed descriptor. *(P)*

5.6.4 Operations are repeatable where possible (reads, writes of the same data, sends, buffer posts, identify), so that resending an unfinished descriptor is safe; an operation that must not run twice carries a `STARTED` flag so the driver can tell "never started" from "started, outcome unknown". *(P)*

Commentary. **Precedent.** 4.3BSD's `ubareset()` resets the Unibus adapter and then calls each driver's reset routine, so each driver re-issues what was pending: resending as a design, driver by driver. **Trade-offs.** The cost is a status write per descriptor and the ordering rule between data and the done mark. **Improvements.** One recovery routine serves a device that fails and is reset, a device replaced by another, and, with a saved ring, a hand-over between cores. **Benefits.** This is the recoverability principle in the driver: fewer failure outcomes (the precise bus fault, not all ones on a removed device) and one path to recover from any of them. **Modern comparables.** NVMe and virtio drivers requeue outstanding requests after a reset; PCI returns all ones for a read of a removed device. **Impact.** *Kernel:* One recovery routine handles a failed, reset, replaced or departed device. *Compiler:* None. *Hardware:* The device writes data before status before clearing the ownership bit, and the bus keeps one presence bit per slot.

5.7 Interrupts from devices

5.7.1 A device raises one interrupt line, at the priority level its slot asks for, and its cause is in the `IRQ` register, where each bit is cleared by writing a 1. *(P)*

5.7.2 The vector of a device interrupt is its slot number added to 64. *(P)*

5.7.3 A polled device (the console, the timer, the entropy source) may be used with its interrupts masked. *(D)*

5.7.4 A changeable interrupt target is changed with its vector masked and then read back, or is a single atomic register write. *(D)*

Commentary. **Precedent.** A PDP-11 device asserted one of four bus-request lines and supplied its own vector, set by switches on the card. **Trade-offs.** A vector derived from the slot needs no per-device switch and no assignment, and limits the device interrupts to 192 slots. **Benefits.** A driver's interrupt routine reads one register to learn why. **Modern comparables.** The causes and the write-one-to-clear convention are those of most memory-mapped controllers; message-signalled interrupts, which need no line, are an open option. **Impact.** *Kernel:* An interrupt routine reads one register to learn why and clears it by writing the same bits. *Compiler:* None. *Hardware:* One line per device into an eight-level priority encoder, with the vector derived from the slot number.

5.8 Drivers in user space, and in OpenBSD

5.8.1 Because a device is one page, a driver can run in user space with only that page and its buffers mapped. *(P)*

5.8.2 Until devices have an address-translation unit, a user-space driver can instruct its device to write anywhere in memory, so it is as trusted as the device. *(P)*

5.8.3 In OpenBSD the register accessors become loads and stores, the DMA mapping interface returns physical addresses with no cache-synchronisation work to do, and the autoconfiguration framework attaches a driver to each slot the bus reports. *(P)*

Commentary. **Precedent.** Microkernels and exokernels run drivers in user space where the hardware lets one device be mapped to one process; the PDP-11's Unibus gave every device a fixed address and no protection. **Trade-offs.** Without a translation unit the mapping protects against stray register writes but not against DMA. **Benefits.** A driver can be tested, restarted and replaced without taking the kernel with it. **Modern comparables.** Linux's `vfio` and `uio` and DPDK run drivers in user space, relying on an I/O translation unit for protection. **Impact.** *Kernel:* The accessors reduce to loads and stores, the DMA interface to physical addresses, and attach to the bus's slots. *Compiler:* None. *Hardware:* One device per page lets the translation unit protect a device, which a device-sharing page layout would not.

5.9 Still open

The register layout and descriptor details (decision R5), whether USB is a per-slot controller or a conventional ring controller, whether the emulator uses `virtio` devices first to reach an OpenBSD login sooner, DMA addressing and protection (R6), errors raised by devices (R7), and the details of the interrupt vector and cause (R1). The proposal is in the working documents (`../decisions-pending.md`, `../pdp64-q64-bus.md`) and appendix B.

Chapter 6 — The kernel: modes, exceptions and memory management

An operating system on the pdp64 sees three things: a status word with a mode and a priority, an exception mechanism that writes only registers, and a three-level page table with software-managed accessed and dirty bits. The chapter covers what a kernel writer needs: the modes and the privileged instructions, the exception and interrupt entry, the vector table, the system registers, translation and the page-table entry, the user-memory guard, system calls, and how the machine starts. It is the design called System v1; the options and the comparisons with other architectures are in `../pdp64-system-architecture.md` (section 15) and `../pdp64-system-design.md`. Tags: *(D)* decided; *(P)* proposed.

6.1 Modes and the status word

6.1.1 There are three modes, with the PDP-11's encoding: kernel 0, supervisor 1, user 3. The supervisor mode exists in the status word and in the banked registers, and no code uses it. *(D)*

6.1.2 The status word holds the flags in bits 0 to 3, the trace bit in bit 4, the interrupt priority in bits 5 to 7, the current mode in bits 8 and 9, the previous mode in bits 10 and 11, and the user-access guard in bit 12; the other bits are zero. *(D)*

6.1.3 `MTPS` in user mode writes only the flags; in kernel mode it writes every writable bit. `SPL #n` is kernel-only. *(D)*

6.1.4 The stack pointer `R14` and the direct-page register `DP` each have one copy per mode; the mode selects which one an instruction names. *(D)*

6.1.5 These instructions abort with the privileged-instruction vector outside kernel mode: `RTI`, `RTT`, `HALT`, `SPL`, `MTASID`, `MFASID`, `TLBI`, `TLBIA`, `TLBSYNC`, `MTDMW`, `MFSR`, `MTSR`, `PROBER`, `PROBEW`, `WFE`, `ISYNC`, `STUA`, `CLUA`, `PUSHX`, `POPX`. *(D)*

6.1.6 These are not privileged: `MTDP` and `MFDP` (in user mode, the thread pointer), `ICINV` and `ICINVA` (a JIT needs them), `FENCE`, `SEV`, `HINT`, `TRAP`, `BPT`, `NOP`, and `MTPS` and `MFPS` with the restriction above. *(D)*

Commentary. **Precedent.** The PDP-11 had kernel, supervisor and user modes, with a status word whose priority and mode fields the pdp64 keeps, and a stack pointer per mode so that a trap arriving in user mode finds a kernel stack without searching for one. **Trade-offs.** Three modes cost a third copy of two registers and no logic the kernel uses; supervisor mode has no user, and is included because adding a mode would change the status word and every saved copy of it. **Benefits.** Entry from user mode needs no stack switch code; a threaded program's pointer and a kernel's per-CPU pointer are the same register in different modes. **Modern comparables.** x86 has four rings of which two are used; AArch64 has exception levels 0 to 3; RISC-V has user, supervisor and machine modes. **Use.** `MFSR` and `MTSR` are the whole of system control; `PROBER` and `PROBEW` do the work of the `nofault` idiom of V6 and V7. **Impact.** *Kernel:* Banked stack pointers mean a trap from user mode needs no stack switch, and the direct-page copies give a per-CPU pointer in the kernel and a thread pointer in user mode. *Compiler:* User-mode thread-local storage is one `MOVQ d(DP), Rd`. *Hardware:* Three copies of two registers, selected by the mode, and a mode comparison on privileged decode.

6.2 Exceptions and interrupts

6.2.1 Entry to an exception or an interrupt writes only registers: `EPC` receives the address of the faulting instruction (for an abort or a progress exception) or of the next instruction (for a trap or an interrupt), `EPSW` the old status word, `EVA` the faulting address (zero except for aborts), `ECAUSE` the vector number. *(D)*

6.2.2 The new status word has the current mode kernel, the previous mode the old current mode, priority 7, and the trace bit and the flags cleared. *(D)*

6.2.3 The new program counter is the 64-bit word at the physical address `VBR + 8 × vector`; a bus error on that fetch is a machine check that halts. *(D)*

6.2.4 The hardware pushes nothing. A handler that can be interrupted saves `EPC` and `EPSW` with `PUSHX`, lowers the priority with `SPL`, and before returning restores them with `POPX` and executes `RTI`. *(D)*

6.2.5 `RTI` loads the program counter from `EPC` and the status word from `EPSW`; `RTT` does the same and sets the trace bit after the return. *(D)*

6.2.6 The vector table has 512 entries and `ECAUSE` equals the vector number. *(D)*

Vectors	Meaning
0	reset (not used by the QEMU machine)
1	machine check
2, 3, 4, 5	illegal instruction, privileged instruction, misaligned access, non-canonical address
6, 7, 8	instruction, load and store page fault
9, 10, 11	protection violation on instruction, load and store
12, 13	divide by zero, bus fault
14, 15	landing-pad violation, breakpoint
16	check, taken by `TRAPCC` *(P)*
17 to 31	reserved
32 to 63	interrupts (in the virt machine, 32 timer, 33 console input, 34 console output)
64 to 255	reserved (the proposal assigns device interrupts here, vector 64 plus the slot)
256 + n	`TRAP #n` and `TRAPL #n`, n from 0 to 255

6.2.7 There are eight interrupt priority levels; the processor takes an interrupt between instructions when its level is greater than the priority in the status word; the timer is at level 6 and the console at level 4. *(D)*

Commentary. **Precedent.** The PDP-11 pushed the program counter and the status word on a stack and loaded new ones from a two-word vector, and its trap vectors conventionally held priority 7, so a PDP-11 handler began with interrupts masked and chose when to lower them; its interrupts took the priority of their device from the vector. The VAX pushed the program counter and the processor status longword and raised the priority to the interrupt's level for an interrupt, leaving it unchanged for an exception. **Trade-offs.** Entering in registers is short and cannot itself fault (no memory is written), and costs the handler an explicit save; it needs the priority raised on entry, because a higher-level interrupt could otherwise arrive before the handler had stored the pair and overwrite it, which is why the entry sets priority 7 for every exception and interrupt, a fixed choice where the PDP-11 let each vector choose. **Improvements.** `PUSHX` and `POPX` make the save and restore one instruction each; the aborts are precise, so a page-fault handler repairs the mapping and executes `RTI` to run the instruction again. **Benefits.** The kernel survey estimates a system-call entry at four or five instructions on paper, against 45 to 70 on RISC kernels; no entry code is needed to find a stack. **Modern comparables.** AArch64 and RISC-V latch the return address and status in registers; x86 pushes five words on the stack; every one of them masks interrupts on entry. **Impact.** *Kernel:* Entry is four or five instructions on paper; priority 7 on entry removes the code that would otherwise guard the saved state. *Compiler:* Handlers written in C need a short assembler stub to save and restore; no other code generation is affected. *Hardware:* Entry writes registers only, so it cannot fault, and the vector fetch is one 64-bit read.

6.3 System registers

6.3.1 The system registers are reached by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

Number	Register	Meaning
0	`PTBR`	physical address of the level-2 page table
1	`EPC`	exception program counter
2	`EVA`	exception virtual address
3	`ECAUSE`	exception cause (the vector number)
4, 5	`DMW0`, `DMW1`	direct-mapping windows (stored; the format is unspecified)
6, 7	`USP`, `SSP`	the user and supervisor stack pointers
8	`EPSW`	exception status word
9	`VBR`	vector base register
10	`MMUCTL`	memory-management control (bit 0 enables translation)
11	`ASID`	address-space identifier
12	`CPUID`	the core number (read-only)
13 and up		reserved (feature and identification registers are proposed here)

6.3.2 A write to `PTBR`, `ASID` or `MMUCTL`, or an `ICINV`, is guaranteed to take effect only after the next `ISYNC`. *(D)*

Commentary. **Precedent.** The VAX had `MTPR` and `MFPR` with an internal-register number, the same design. The PDP-11 kept its control registers at fixed high addresses and reached them with ordinary moves. **Trade-offs.** A general pair of instructions with 243 free numbers costs two opcodes and no decoding as the registers are added; separate instructions per register would have spent opcode space with every register. **Benefits.** A new register is a number, not an instruction. **Modern comparables.** AArch64's `MRS` and `MSR`, RISC-V's `CSRR` and `CSRW`, PowerPC's `MFSPR` and `MTSPR`; the kernel survey found privileged-register access to be 48% of all the compiler-invisible instructions in four kernels. **Impact.** *Kernel:* A single pair of instructions reaches every system register, so adding a register adds a number and no instruction. *Compiler:* The compiler reaches them through an intrinsic with a constant register number. *Hardware:* A control-register file addressed by an 8-bit number, with a read-only decode for the identification registers.

6.4 Memory management

6.4.1 Virtual addresses are 39 bits, translated by three levels of nine bits over 4 KB pages: address bits 38 to 30 index level 2, 29 to 21 level 1 and 20 to 12 level 0; `PTBR` holds the physical address of the level-2 table. *(D)*

6.4.2 A page-table entry is 64 bits: `V` (valid) in bit 0, `R`, `W`, `X`, `U` and `G` (global) in bits 1 to 5, `A` (accessed) and `D` (dirty) in bits 6 and 7, the memory type `MT` in bits 8 and 9, and the physical page number in bits 12 to 39, in place; all other bits are zero, and a set reserved bit is a page fault. *(D)*

6.4.3 A non-leaf entry has `V` set and `R`, `W` and `X` clear and points to the next table; a leaf at level 2 maps 1 GB, at level 1 maps 2 MB and at level 0 maps 4 KB. *(D)*

6.4.4 A user-mode access needs `U`; kernel mode may read and write user pages but may not execute them; `W` without `R` is reserved. *(D)*

6.4.5 The accessed and dirty bits are managed by software: an access to a page with `A` clear, or a write to a page with `D` clear, raises the page fault of its kind and the kernel sets the bit; the hardware walker never writes a page table. *(D)*

6.4.6 Translation is off at reset (`MMUCTL` is zero and every address is physical) and is enabled by bit 0 of `MMUCTL`; the `ASID` selects non-global entries. *(D)*

6.4.7 `TLBI rN` invalidates the translation of the page containing the address in `rN` for the current address space and the global entries; `TLBIA` invalidates all; both broadcast to every core and `TLBSYNC` waits for the broadcast to finish. *(D)*

6.4.8 The memory type of a page is one of normal cached, normal uncached, device strongly ordered and device weakly ordered. *(D)*

6.4.9 The translation is split by the top bit of the virtual address: bit 38 clear selects the lower half, translated from the table at `PTBR` (one per address space), and bit 38 set selects the upper half, translated from the table at the system register `KTBR` (one for the whole system, the kernel's). In user mode the upper half is never walked: an access to it takes a translation fault without consulting any page table, so no kernel translation is present for a speculating core to read around a permission check. In kernel mode both halves are walked, so the copy routines of 6.5 need no change of root. Entry to an exception changes the mode in hardware and so changes the halves walked; no entry trampoline is mapped in user space. Translation-buffer entries carry the half they came from, and a mode change flushes nothing. The register number of `KTBR` and the layout of the level-2 tables of the two roots are unspecified. *(D)*

Commentary. **Precedent.** The PDP-11 had eight segments per mode, each with a base and a length, and no page tables; the VAX had a linear page table per region with a hardware modify bit and no reference bit, so 4.3BSD emulated reference bits in software. **Trade-offs.** Software-managed bits cost one extra fault on the first touch of each page and one on the first write, and they make the walker small and make it safe against the chip bugs of hardware management that other architectures have had; a hardware-managed option, advertised in a feature register, is a possible extension. A fourth page-table level would give 48 bits at the price of a longer walk on every miss. **Improvements.** The entry has the physical address in place, so no shifting; a leaf at any level makes large pages free to use for the kernel's own mapping. **Benefits.** A walker that only reads is small; a kernel's page-fault handler is the same code for first touch, first write and demand paging, differing only in the bits it sets. **Modern comparables.** The geometry is RISC-V's Sv39; x86-64 and AArch64 use four levels and hardware-managed accessed and dirty bits; MIPS refills its translation buffer in software. **Impact.** *Kernel:* The handler sets accessed and dirty bits; leaves at any level let the kernel map itself with 1 GB pages; `TLBI` broadcasts so there is no shootdown code. *Compiler:* Position-independent code uses PC-relative addressing; the canonical 39-bit form means pointers need no masking. *Hardware:* The walker only reads; the physical page number is in place, so no shifting; three levels of nine bits keep the walk to three reads.

6.5 The user-memory guard and system calls

6.5.1 With the guard bit `UA` clear, a kernel-mode data access to a user page faults; `STUA` sets it and `CLUA` clears it; exception entry clears it and `EPSW` keeps the old value. *(D)*

6.5.2 `TRAP #n` and `TRAPL #n` (n up to 255, two bytes each: one word with an 8-bit vector number) enter the kernel through vector 256 + n; the system-call number and arguments travel in registers by the ABI. *(D)*

6.5.3 A copy routine opens the guard, copies, and closes it; every error path out of the copy must close it, and entry clearing it bounds the damage if one does not. *(D)*

6.5.4 A move with the qualifier `.U` (`MOVQ.U`, `MOVL.U` and the others, seven words) accesses its memory operand with user privilege, so that a copy-in or copy-out routine needs no window in the status word: a fault on that operand is attributed to the instruction and recovered through a fixup table. The five leanings of the study (the fixup table, the `.U` spelling, keeping `UA`, `STUA` and `CLUA` until the OpenBSD port confirms them, bit 15 of `ECAUSE`, the atomics taking the user form once their operand placement is fixed) are accepted; the specification is not complete, and the reference executor treats the qualified move as a plain move. *(P)*

Commentary. **Precedent.** The PDP-11 reached user memory with the explicit instructions `MFPI` and `MTPI`, since its kernel and user address spaces were separate, so accidental access was impossible; the VAX read user addresses directly after a `PROBER` check. **Trade-offs.** The guard is a mode bit, accepted because it lives in the status word, which is saved anyway; it makes a stray kernel pointer a clean fault at the cost of an open and a close around every user copy. **Benefits.** A kernel bug that dereferences a user pointer by mistake does not succeed silently. **Modern comparables.** x86 `SMAP` with `STAC` and `CLAC`, AArch64 `PAN` and RISC-V `SUM`. **Use.** The user copy routines (`copyin`, `copyout`, `copyinstr`). **Impact.** *Kernel:* Every user copy opens and closes the guard; every error path must close it, and entry clearing it bounds the damage of one that does not. *Compiler:* The copy routines are written in assembler or inline assembler; system-call stubs are one `TRAP`. *Hardware:* One status-word bit and one check on a kernel-mode data access to a user page.

6.6 Reset and the first kernel

6.6.1 At reset core 0 starts in kernel mode at priority 7 with translation off, the guard clear and `VBR` zero; the kernel sets `VBR`, builds page tables, writes `PTBR`, sets `MMUCTL` bit 0 and executes `ISYNC`. *(D for the QEMU machine; P for the architectural reset, chapter 4)*

6.6.2 The QEMU machine starts at the entry point of the ELF given with `-kernel`, with the kernel stack at the top of memory; the option `kernel-mode=on` turns off the system-call stand-in of user-mode programs so that `TRAP` enters the kernel. *(D)*

6.6.3 The first operating system is OpenBSD, with Unix/32V as the fallback if OpenBSD proves too hard. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX turned the memory manager on with a bit in a status register and a jump to a mapped address, each needing a transition in the boot code. **Trade-offs.** Translation-off at reset is the same arrangement, and the single bit is a machine-wide mode bit accepted for that reason. **Benefits.** The boot code runs at physical addresses until the kernel is ready, with no identity-map trampoline of the kind x86 needs. **Modern comparables.** RISC-V's `satp` with a bare mode, AArch64's `SCTLR.M`. **Impact.** *Kernel:* Boot code runs at physical addresses until the kernel is ready, with no identity-map trampoline. *Compiler:* The first kernel code is position-independent and uses no virtual address until translation is on. *Hardware:* Translation-enable is one bit.

6.7 System instructions, exceptions and interrupts

6.7.1 There are three modes (kernel, supervisor, user); supervisor is defined and unused. System instructions executed outside kernel mode abort with the privileged-instruction vector. *(D)*

6.7.2 `TRAP #n` and `TRAPL #n` (vectors 0 to 255, two bytes each) enter the kernel; `RTI` returns, and `RTT` returns and sets the trace bit after. *(D)*

6.7.3 An exception or interrupt writes only registers: `EPC` receives the address of the faulting instruction (or the next one, for a trap or interrupt), `EPSW` the old status word, `EVA` the faulting address and `ECAUSE` the vector number; the new program counter is the 64-bit word at `VBR + 8 × vector`. *(D)*

6.7.4 Entry sets the priority to 7 for every exception and every interrupt; the handler lowers it. *(D)*

6.7.5 There are eight interrupt priority levels; `SPL #n` sets the priority without touching the flags. *(D)*

6.7.6 MFSR and MTSR read and write the system registers by number; TLBI, TLBIA and TLBSYNC invalidate translations (broadcast to other cores); MTASID and MFASID set the address-space tag; ICINV and ICINVA invalidate the instruction cache; PROBER and PROBEW test an address; MFCPUID identifies the processor. *(D)*

6.7.7 UA in the status word guards user memory: with it clear, a kernel-mode data access to a user page faults; STUA sets it and CLUA clears it; entry clears it. *(D)*

6.7.8 TRAPCC.cc (fourteen words, one for each condition; one parcel, no operand) enters the kernel when its condition holds; it is a trap and not an abort, so the instruction completes and EPC is the next instruction. *(D)* The reference executor takes vector 16 (the check vector of the study) with EVA the address of the TRAPCC; both are assumptions and the vector number is a proposal. ADDV, SUBV and MULV take the same vector on signed overflow, in the two-register form of the grammar, as an abort that writes nothing. *(P)*

Commentary. The modes and exceptions are sections 6.1 to 6.3 and the multiprocessor instructions are chapter 7. **Precedent.** The PDP-11 pushed the program counter and status word on a stack and loaded new ones from a two-word vector; its trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers is short and cannot fault, and costs the handler an explicit save; raising the priority on entry closes the window in which an interrupt could overwrite the saved state. **Improvements.** PUSHX and POPX save the pair in one instruction, and the user-memory guard turns a stray kernel pointer into a clean fault. **Benefits.** The kernel survey estimates system-call entry at four or five instructions against 45 to 70 on RISC kernels (a paper estimate, not a measurement). **Modern comparables.** AArch64 and RISC-V save the return address and status in registers on exception entry; x86 pushes five words; x86's SMAP with STAC/CLAC, AArch64's PAN and RISC-V's SUM are the user-memory guards. **Use.** PROBER and PROBEW replace the `nofault` idiom of V6 and V7; ISYNC follows a write to the page-table base or the vector base. **Impact.** *Kernel:* This is the kernel's whole instruction vocabulary: MFSR, MTSR, TLBI, RTI, SPL, ISYNC, the probes. *Compiler:* The compiler never emits them except through intrinsics. *Hardware:* The system group takes its sub-opcode from the register field, and the privilege check is one comparison on the current mode.

The instruction entries for this section are in chapter 9, section 9.14: each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

Entry	Mnemonics
No operation and landing pad	NOP, LPAD
Return from interrupt or exception	RTI, RTT
Return from subroutine	RET
Halt the processor	HALT
Breakpoint	BPT
Memory fence	FENCE
Wait for an event	WFE
Send an event	SEV
Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache	TLBIA, TLBSYNC, ICINVA
Push and pop the exception state	PUSHX, POPX
Instruction synchronisation	ISYNC
Hint	HINT
Move from and to a system register	MFSR, MTSR
Move to and from the processor status word	MTPS, MFPS
Set the interrupt priority level	SPL
System call and trap	TRAP, TRAPL
Invalidate a translation	TLBI
Move to and from the address-space identifier	MTASID, MFASID
Move to and from the direct pointer	MTDP, MFDP
Move to a direct-mapping window	MTDMW
Read the processor identifier	MFCPUID
Invalidate the instruction cache for an address	ICINV
Probe an address for read or write access	PROBER, PROBEW
Trap on condition	TRAPCC.NE, TRAPCC.EQ, TRAPCC.PL, TRAPCC.MI, TRAPCC.VC, TRAPCC.VS, TRAPCC.CC, TRAPCC.CS, TRAPCC.GE, TRAPCC.LT, TRAPCC.GT, TRAPCC.LE, TRAPCC.HI, TRAPCC.LOS
Reserved system words	SYSPG0, SYSPG1, SYSPG2, SYSPG3, SYSPG4, SYSPG5

6.8 Faults, restart and memory management

6.8.1 Faults fall in four categories: traps (the instruction completes), aborts (it is annulled and restarted), progress (a block operation stopped part-way, restarted from its registers) and machine checks (not restartable). *(D)*

6.8.2 The register update of an auto-increment or auto-decrement mode is held until the instruction retires and discarded by an abort, so a handler never undoes anything. *(D)*

6.8.3 A read of non-existent memory is a precise abort with cause `BUSFAULT`; a posted write that times out and an uncorrectable memory error are machine checks. *(D)*

6.8.4 Virtual addresses are 39 bits, translated by three levels of nine bits over 4 KB pages to a 40-bit physical address. *(D)*

6.8.5 A page-table entry carries the physical page number in place and read, write, execute and user permission bits; accessed and dirty bits are managed by software; one memory type per page selects cached, uncached, strongly ordered or weakly ordered. *(D)*

6.8.6 Two direct-mapping windows, `DMW0` and `DMW1`, are stored and their format is unspecified; `MMUCTL` bit 0 enables translation and is zero at reset, so every address is physical until the kernel turns it on. *(D)*

6.8.7 A deferred load (`MOVQ.D` and the six other `.D` words) does not fault at the load: a faulting load leaves a poison mark on its destination register and the fault is taken at the first instruction that writes the flags from a poisoned source; the kernel decides the fault of a deferred load, and the kernel dispatcher, not the hardware, tests the argument registers for poison. The combination `.UD` of the two qualifiers has no text of its own, and the poison bit is not decided. *(P)*

Commentary. **Precedent.** The PDP-11 recorded its register side effects in `MMR1` and left the undo to software; the VAX needed resumable microstate and a status-word bit; the 68000 had no restart path. **Trade-offs.** Thirty-nine bits is 512 GB, what a three-level table indexes with nine bits per level; a fourth level would give 48 bits at the price of a longer walk on every TLB miss. Software-managed accessed and dirty bits cost one extra fault on first touch and one on first write per page, in exchange for a simpler walker (appendix B). **Improvements.** The one-memory-operand rule makes the undo buffer a single entry and row 3 of the fault matrix provably empty, so there is no journal and no undo protocol. **Benefits.** An FPGA walker is small, and a core with no speculation has no page-table side channels. **Modern comparables.** RISC-V's Sv39 has the same geometry; x86-64 and AArch64 have 48-bit spaces with four levels and hardware-managed accessed and dirty bits. **Use.** The memory type on a page lets a driver map a device register uncached and strongly ordered without a special instruction. **Impact.** *Kernel:* Precise faults and software-managed accessed and dirty bits make one small page-fault handler for first touch, first write and demand paging. *Compiler:* No scheduling restriction arises from faults, because the register update of an addressing mode is held until retirement. *Hardware:* A single-entry undo buffer, a read-only walker, and (on the FPGA) a four-way set-associative translation buffer rather than a content-addressable one.

6.9 Still open

The format of the direct-mapping windows `DMW0` and `DMW1` (stored but without effect), the use of supervisor mode, hardware breakpoints and watchdogs, performance counters, machine-check reporting beyond halting, feature and identification registers from register 13 up, and a TLB invalidate by address-space tag (appendix B).

Chapter 7 — Multiple processors

The pdp64 is built so that adding a second core needs seventeen operations, one per-core register, and nothing a kernel writer has to work around: memory is sequentially consistent, the atomics are one instruction each, the per-CPU pointer is a register that already exists, and the other cores' translation buffers are invalidated by a single broadcast. The chapter collects everything the architecture adds for symmetric multiprocessing, what a kernel does with it, what is left out, and what the device side supplies. The instruction-set text is `../pdp64-isa.md`, section 17. Tags: *(D)* decided; *(P)* proposed.

7.1 The seventeen operations

7.1.1 The instructions that exist for multiple processors fall under one umbrella of seventeen operations: ten atomics and seven others. *(D)*

Group	Operations
Atomics (10)	compare-and-swap (`CAS`), swap (`SWAP`), fetch-and-add (`XADD`), fetch-and-and (`XAND`), fetch-and-or (`XOR_`), fetch-and-exclusive-or (`XXOR`), fetch-and-maximum (`XMAX`, with `XMAXU`), fetch-and-minimum (`XMIN`, with `XMINU`), load-linked (`LL`), store-conditional (`SC`)
Others (7)	`FENCE`, wait-for-event (`WFE`), send-event (`SEV`), `TLBI` (with `TLBIA` as its "all" form), `TLBSYNC`, instruction-cache invalidate (`ICINV`, with `ICINVA`), read the core number (`MFCPUID`)

7.1.2 Every atomic has a variant, `_DB`, that folds a barrier into the instruction, so that a lock acquire or release is one instruction. *(D)*

7.1.3 The atomics live in the atomic leaves and the other operations in the system leaves; the umbrella is conceptual, not an encoding. *(D)*

7.1.4 Supporting these are the banked direct-page register, the address-space tag, the global-page bit and the banked stack pointer (section 7.4). *(D)*

Commentary. **Precedent.** The PDP-11/70's read-modify-write instructions were indivisible on the Unibus (`DATIP` and `DATO`); the VAX added interlocked instructions (`BBSSI`, `ADAWI` and the queue instructions) and multiprocessor models, and the 4.3BSD kernel on them still used `spl` for exclusion. **Trade-offs.** The set is the union of the three designs found on other machines (compare-and-swap, load-linked and store-conditional, fetch-and-operate), which is more atomic operations than any architecture surveyed, in exchange for nothing a kernel writer has to emulate. **Improvements.** The `_DB` forms were taken from LoongArch: a lock acquire should be one instruction, with no separate fence. **Benefits.** The seven non-atomic operations are the minimum that a kernel cannot synthesise: which core am I, wait and signal, invalidate everywhere, and make the instruction fetch see what the data side wrote. **Modern comparables.** RISC-V's `LR`/`SC` and `AMO` set, AArch64's exclusive loads and stores and its large-system atomics, x86's `LOCK`-prefixed read-modify-write. **Impact.** *Kernel:* Each of the seventeen operations is one instruction in the kernel's lock, idle, shootdown and per-CPU code. *Compiler:* C11 atomics and the kernel's intrinsics map to them directly. *Hardware:* The atomics live in one escape group and the other seven in the system group, so the decoder has two places to look.

7.2 The memory model

7.2.1 Memory is sequentially consistent: the result is as if all cores' accesses happened in one order that respects each program's order. *(D)*

7.2.2 A store buffer is allowed, provided loads snoop it and stores drain in program order. *(D)*

7.2.3 Quad accesses are naturally aligned, so every access is atomic within its own size; no misaligned access can be torn across cores. *(D)*

7.2.4 The machine is in order and does not speculate, so no speculative side channel exists to be mitigated. *(D)*

7.2.5 The ABI says nothing about memory ordering, because with sequential consistency there is nothing to say. *(D)*

7.2.6 Memory is sequentially consistent per byte: a load returns, for each byte, the value written by the last store to that byte in the single order, and an aligned access of N bytes is single-copy atomic. A load that overlaps a buffered store of a different size either waits for the buffer to drain or merges the buffer and memory in one step; it never returns a mixture that no order of the stores could produce. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX ran one processor at a time in the model that mattered, so the question did not arise; ARM's memory model took about a decade to formalise, and the C11 and C++11 models exist largely to paper over the differences between weak architectures. **Trade-offs.** Sequential consistency is the model with no cost to software and the highest cost to a large speculative core; software written against a weak model runs on a strong machine and not the reverse, so the choice is close to irreversible, which is why the choice is made once. If a large out-of-order core is ever built, total store order is the least painful retreat. **Benefits.** None of the barrier code that fills other kernels: the kernel survey found barrier vocabulary in 5,596 lines of Linux code and 3,016 lines of `memory-barriers.txt` documentation, and none of it arises here. **Modern comparables.** x86 and SPARC are total store order; ARM, POWER and RISC-V are weak. **Impact.** *Kernel:* None of the 5,596 lines of ordering code found in Linux, nor the barrier documentation, is needed. *Compiler:* The optimiser may move non-volatile accesses freely, and no fence is inserted for ordering. *Hardware:* In-order with a snooped store buffer; sequential consistency is nearly free on this core and the expensive part only appears on a speculative one.

7.3 The atomics in detail

7.3.1 `LL` takes a reservation on a granule of memory; a write to the granule by any initiator, core or device, clears it; `SC` succeeds exactly when the reservation is intact and otherwise writes nothing. *(D)*

7.3.2 A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule. *(D)*

7.3.3 The read-modify-write atomics are single-copy atomic with respect to every initiator; the memory system may build them from an exclusive access with hardware retry. *(D)*

7.3.4 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** A bus lock, the Unibus's `DATIP`/`DATO` pair, is simple and a bottleneck as cores multiply; a monitor maps onto the exclusive access of AXI and the memory systems of FPGAs and modern chips. **Trade-offs.** A monitor needs forward-progress rules and, under contention, can livelock if not specified; RISC-V's guarantee of eventual success for constrained sequences is the model. The pain-points report records that the kernels' simple lock does not scale (OpenBSD's ticket kernel lock measured 155% worse than an Anderson lock in one developer's test) and that Linux moved from test-and-set through ticket locks to queued locks; the instruction set provides the primitives and the lock design is the kernel's. **Benefits.** A kernel's lock needs one instruction; lock-free queues need compare-and-swap. **Modern comparables.** RISC-V `LR`/`SC` with its constrained-loop guarantee, AArch64 load-exclusive and `CAS`, x86 `LOCK CMPXCHG`. **Use.** `CAS` for lock-free structures and lock words; `XADD` for counters and ticket numbers; `LL`/`SC` for everything else. **Impact.** *Kernel:* A lock is one `_DB` instruction. *Compiler:* A `LL`/`SC` loop must be emitted in the constrained form, with no other memory access between the two. *Hardware:* A reservation granule tag in the cache controller and an exclusive-access path to memory.

7.4 Per-CPU state and cheap context

7.4.1 `MFCPUID` returns the number of the core that executes it. It cannot be synthesised and per-CPU data needs it. *(D)*

7.4.2 `DP`, the direct-page register, has a copy per mode; in kernel mode it is the per-CPU pointer and a field of the per-CPU structure is one six-byte instruction, `MOVQ d(DP), Rd`. *(D)*

7.4.3 The address-space tag makes a context switch constant-time, and global page-table entries and the direct-mapping windows keep the kernel's own mappings out of the invalidation path. *(D)*

7.4.4 The stack pointer is banked by mode, so an interrupt arriving in user mode on any core finds that core's kernel stack without a search. *(D)*

Commentary. **Precedent.** Every SMP kernel builds a per-CPU base by hand: x86 uses the `GS` base and a `swapgs` instruction, ARM uses `TPIDR_EL1`, RISC-V the supervisor scratch register, MIPS reserves two registers; the kernel survey found seven architectures paying for the problem of having no free register at the moment of entry. **Improvements.** `DP` serves code density and is also what both an SMP kernel and a threaded C runtime need most, in the mode where each needs it, and the two copies do not conflict. **Benefits.** No `swapgs`, no reserved scratch register, no code spent finding the CPU. **Modern comparables.** `GS` and `TPIDR` as above. **Impact.** *Kernel:* Per-CPU data is one six-byte `MOVQ d(DP), Rd`, with no `swapgs` and no reserved register. *Compiler:* The compiler addresses per-CPU variables relative to `DP`. *Hardware:* Three copies of `DP`, one per mode.

7.5 Translation and instruction-cache coherence

7.5.1 `TLBI` and `TLBIA` broadcast to all cores; `TLBSYNC` waits until the broadcast has completed everywhere. *(D)*

7.5.2 Instruction caches are not coherent with data caches; `ICINV` invalidates a line and `ICINVA` all of it, and neither is privileged, because a JIT needs them. *(D)*

7.5.3 A write to the page-table base, the address-space tag, the translation-enable bit or the vector base takes effect only after the next `ISYNC`. *(D)*

Commentary. **Precedent.** x86 flushes remote translation buffers with an inter-processor interrupt and a software handshake, which is slow and intricate; AMD added a broadcast invalidate only late. **Trade-offs.** A broadcast is simpler in hardware and far faster for two to four cores; it needs the completion barrier, because a broadcast one cannot wait for is no use. Making instruction caches coherent would cost a snoop port for a case that almost never happens. **Improvements.** The kernel's own mappings are global or window-mapped and never part of a shootdown. **Benefits.** The pain-points report finds that remote translation-buffer shootdown, a source of deadlocks in the BSDs and of much code in Linux, does not arise. **Modern comparables.** ARM's broadcast `TLBI` and `DSB`; RISC-V's `SFENCE.VMA` still needs software shootdown. **Impact.** *Kernel:* A shootdown is `TLBI` then `TLBSYNC`; a module or JIT write is `ICINV` then `ISYNC`. *Compiler:* A JIT or a trampoline generator must call `ICINV` after writing code. *Hardware:* A broadcast on the interconnect and a counter for completion; the instruction cache has an invalidate port but no snoop port.

7.6 Waiting and signalling

7.6.1 `WFE` sleeps until another core executes `SEV` or an interrupt arrives, and `HINT #1` is the spin-wait pause. *(D)*

7.6.2 `WFE` ends its sleep when an interrupt is pending even if the priority masks it, so that the idle loop is `SPL #7`, check for work, `WFE`, `SPL #0`, with no gap. *(P)*

7.6.3 On one core `WFE` is the idle instruction, and `WAIT` is its assembler alias. *(D)*

Commentary. **Precedent.** x86's `PAUSE` tells a spinning core to back off; ARM's `WFE` lets it stop until an event. V7's idle routine lowered the priority and executed `wait` in a loop that rechecked the run queue after every interrupt, so a missed wake-up cost one clock tick. **Trade-offs.** On an FPGA a spinning core consumes memory bandwidth the other cores want, which is the reason to prefer a real wait. **Benefits.** An idle core costs nothing and wakes on a signal. **Modern comparables.** ARM `WFE` and `SEV`, RISC-V `WFI`. **Impact.** *Kernel:* An idle core waits on `WFE` and costs no bandwidth. *Compiler:* Spin-wait loops use `HINT #1`. *Hardware:* `WFE` stalls or gates the core's clock until an event or an interrupt.

7.7 Cache coherence

7.7.1 Between processors, coherence is transparent to software, by snooping with three states (modified, shared, invalid), for the four cores of the first device. *(D)*

7.7.2 A DMA device is the opposite case: the buffers are uncached and the hand-off is explicit, because snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off. *(D)*

7.7.3 The design supports at least four cores; sixteen or more is the preferred ceiling that no decision may preclude; the initial device has four cores, which it is hoped will fit the FPGA. *(D)*

7.7.4 The protocol for more than four coherent cores is open. The leaning of the SMP decisions is MESI with a snoop filter inside clusters of four and a directory between clusters, with dedicated cores as non-coherent partners that use uncached rings; the architecture promises only that memory is coherent and sequentially consistent. *(P)*

Commentary. **Trade-offs.** MESI would add an exclusive state to avoid a bus transaction when writing a clean line nobody else holds; at four cores the benefit is small and the state machine larger; a directory scales further and is not needed below about eight cores. Software-managed coherence is cheaper in hardware and is where concurrency bugs live, since every missing flush is a race that reproduces once a week. **Benefits.** The instruction set owes nothing to the coherence protocol, which keeps the two separate. **Modern comparables.** x86 snoops with MESIF or MOESI and directories across sockets; ARM's coherent interconnects use directories or snoop filters. **Impact.** *Kernel:* No coherence code, in either direction between cores. *Compiler:* None. *Hardware:* A snoop port on each data cache, about 500 lookup tables and a small state machine for three states, plus about 600 per core.

7.8 Starting and signalling the other cores

7.8.1 The controller holds every core but core 0 in reset; a write to a per-core start register gives a core an entry address and an argument and releases it into kernel mode at priority 7, with its number in `r0`. *(P)*

7.8.2 An inter-processor interrupt is a write to a per-core register in the interrupt controller; no APIC-like machinery is needed. *(P)*

7.8.3 The routing of device interrupts to cores, and a shared second-level cache or a coherent interconnect, are device-side and unspecified. *(P)*

7.8.4 Each core has a read-only identification register giving its class and capacity, defined from the first system although every core is alike, and a stop-and-drain register that a kernel writes to take a core offline: the core finishes its outstanding bus transactions and stops. *(D)*

7.8.5 In-order and out-of-order cores may share a die if software can tell which is which, which the class register of 7.8.4 reports; the memory model and the semantics of every instruction are identical on every core, and a core class may differ in speed and never in what a program can observe. *(D)*

7.8.6 At least one core is a secure core: every timing channel is removed (every instruction and every cache fetch takes a time independent of its data), it has private cache lines and takes no periodic timer tick, and it is stopped and preempted by an inter-processor interrupt. The AES and SHA words of this CPU are the constant-time instructions proposed for it. *(D)*

7.8.7 The author's recommendation is dedicated cores for graphics, for the network and for audio, then general-purpose cores. *(P)*

Commentary. **Precedent.** An x86 core wakes in 16-bit real mode and has to be walked through three modes by a trampoline (1,528 lines of start-up code and 1,258 of real-mode code in Linux); arm64 uses a firmware call in 124 lines or a spin table in 127; RISC-V 108 or 58; the VAX's console processor started the others. **Benefits.** A core can start in its normal mode at any address, so the whole protocol is one register write. **Modern comparables.** x86's INIT and START-UP interrupts, ARM's PSCI, RISC-V's hart-start call. **Impact.** *Kernel:* Starting a core is one register write, and a stuck core is reset by another. *Compiler:* None. *Hardware:* A start register and a reset-hold per core in the controller.

7.9 A kernel's view

7.9.1 The kernel starts a core by writing its start register, keeps its per-CPU structure at the address in `dp`, and takes a lock with one `_db` atomic. *(P)*

7.9.2 It unmaps a page by changing the entry, issuing `tlbi`, and waiting with `tlbsync`. *(D)*

7.9.3 It makes new instructions visible by writing them, issuing `icinv` and executing `isync`. *(D)*

7.9.4 It idles a core with `wfe` and wakes it with `sev` or an inter-processor interrupt. *(P)*

Commentary. Each item is one instruction or one register write, and each is a place where other architectures need a trampoline, a handshake or a fence.

7.10 What was avoided

7.10.1 The architecture has no weak ordering, no speculative execution, no hardware transactional memory, no software-managed coherence between processors, no coherent instruction caches, no inter-processor-interrupt shootdown as the main mechanism, and no memory-model-dependent ABI. *(D)*

Commentary. **Trade-offs.** Intel's transactional memory produced a decade of errata and side channels for a feature few programs used; compare-and-swap and `ll`/`sc` suffice. Each avoided item is a tax on every concurrent program, or on every implementation, in exchange for performance the small in-order core cannot use. **Impact.** *Kernel:* Each avoided feature is code a kernel does not have to write: no barriers, no software coherence, no shootdown interrupts. *Compiler:* Each is analysis the compiler does not have to do: no memory-model-dependent code generation. *Hardware:* Each is a block not built: no speculation, no transactional memory, no coherent instruction cache.

7.11 What the model does not give for free, and the road to out of order

7.11.1 A simple memory model does not make an atomic implementation simple by itself: cache coherence, exclusive ownership, invalidation, ordering and the interaction with interrupts still have to be designed and tested.

7.11.2 Out-of-order execution is not ruled out: the first implementations are in order, and the architecture is kept ready for a large out-of-order design. *(D)*

7.11.3 Sequential consistency on a speculative core needs loads that are replayed when another core intervenes; the readiness review calls this awkward but defined, and names the flag rules, the reservation rules and the ISYNC and HINT instructions as what a speculative core needs. *(D)*

Commentary. **Precedent.** One instruction set often spans small in-order and large out-of-order cores: ARM's small and large cores, RISC-V's Rocket and BOOM, the x86 Pentium and P6, Alpha's 21064 and 21264. **Trade-offs.** What the pdp64 gains by refusing speculation is a smaller design with fewer error cases and none of the speculative side channels; what it gives up is the performance of an aggressive core, which an FPGA or a small chip could not build anyway. The readiness document (`../pdp64-ooo-readiness.md`) and the hardware-vulnerabilities report give the rules a future core would follow. **Benefits.** The decision is reversible in the direction that matters: total store order can be the retreat for a large core. **Impact.** *Kernel:* None. *Compiler:* None. *Hardware:* A speculative core would need loads that replay when another core intervenes, and the flag and reservation rules keep that path open.

7.12 The atomic and ordering instructions

7.12.1 The atomic group provides CAS, SWAP, XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU and XMINU, each with a _DB variant that adds a barrier, and LL and SC.

7.12.2 LL takes a reservation on a granule of memory; any write to it by any initiator clears it; SC succeeds exactly when the reservation is intact. A short LL/SC sequence must eventually succeed if no other initiator keeps writing the granule. *(D)*

7.12.3 The read-modify-write atomics are atomic with respect to every initiator, cores and devices alike. *(D)*

7.12.4 FENCE orders device accesses; WFE and SEV wait for and signal an event; ISYNC makes earlier system-register writes take effect; HINT #n has no architectural effect (1 pause, 2 speculation barrier, others reserved). *(D)*

7.12.5 The operands of the atomic words sit in three register fields (four for CAS) in the grammar, with the address in a register and the size in the sub-opcode; the roles of the registers and the size encoding of CAS are not fixed in the text, and the reference executor does not implement the group. The study leans to fixing three register fields now, with the fourth field reserved zero. *(P)*

Commentary. **Precedent.** The PDP-11's read-modify-write instructions were indivisible on the Unibus (the DATIP/DATO pair); the VAX had interlocked queue and bit instructions. **Trade-offs.** A bus lock is simple and does not scale as cores multiply; a monitor (reservation) maps onto the memory systems that FPGAs and modern chips use, at the cost of forward-progress rules. **Improvements.** The pdp64 offers all three designs found on other machines (load-linked/store-conditional, compare-and-swap, fetch-and-operate), more atomic operations than any architecture surveyed. **Benefits.** A kernel lock needs one instruction. **Modern comparables.** RISC-V LR/SC and the AMO set; AArch64 load-exclusive and its large-system atomics; x86 LOCK-prefixed read-modify-write. **Use.** CAS for lock-free structures; XADD for counters; LL/SC for anything else; WFE for the idle loop. **Impact.** *Kernel:* A lock is one instruction, and the _DB forms need no separate fence. *Compiler:* C11 atomics map one-to-one, no ordering fences are emitted under sequential consistency, and a LL/SC loop must be generated in the constrained form (short, no other memory access inside). *Hardware:* The reservation is a granule tag in the cache controller, and the read-modify-write atomics use an exclusive access with hardware retry.

The instruction entries for this section are in chapter 9, section 9.13: each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

Entry	Mnemonics
Compare and swap	CAS, CASDB
Atomic exchange	SWAP, SWAPDB
Atomic fetch and add	XADD, XADDDB
Atomic fetch and and, or, xor	XAND, XANDDB, XOR, XOR_DB, XXOR, XXORDB
Atomic fetch and minimum or maximum	XMAX, XMAXDB, XMIN, XMINDB, XMAXU, XMAXUDB, XMINU, XMINUDB
Load-linked and store-conditional	LL, SC

7.13 Still open

The mapping of the bus's atomics onto PCI Express (the transport of decision R4 is settled by errata 80), the interrupt controller and routing (R1), starting the other cores (R2, B2), how coherence scales beyond four cores (R3), the wake rule of `wfe` (P1), and lock scaling above a few cores, which is a kernel design question the instruction set does not settle.

Chapter 8 — Vector work, AI and array languages: the device route

Wide data-parallel work, whether neural-network inference or the primitives of an array language, is done by memory-mapped devices with their own memory and driven by descriptors, not by SIMD registers in the processor. There is no vector register file and no AVX2-style instruction set in the pdp64. The chapter states that decision, the reason for it, and how it serves AI and large-language-model inference and the array languages APL and K. Graphics has its own chapter (chapter 3), since its devices are the first a user sees. It is the route ARM-based systems take for their neural-network and graphics engines, and the one the PDP-11 took for its attached array processors. The discussion of the alternatives is appendix B (S1 to S6); the accelerator designs are `../pdp64-lb64-accelerator.md` and section 12 of `../pdp64-fpga-plan.md`. Tags: *(D)* decided; *(P)* proposed.

8.1 The decision

8.1.1 The processor has no vector register file and no SIMD instruction set of the SSE, AVX2 or NEON kind. *(D)*

8.1.2 Wide work is described to a device by descriptors in memory: an operation, the addresses of the arrays and their length. *(D)*

8.1.3 The processor owes such a device what it owes every device: a memory type per page, a `FENCE`, and atomics that reach device memory; the instruction set is unchanged. *(D)*

8.1.4 Packed-lane operations in the ordinary integer registers (a handful of instructions for bytes, halves and words in a 64-bit register) are not part of this decision; they remain a possibility to be settled by measurement. *(P)*

Commentary. **Precedent.** The PDP-11's heavy numerical work went to attached array processors driven by library calls (the Floating Point Systems AP-120B attached to PDP-11 hosts, by the one brochure checked), and the VAX gave its 6000 series a vector extension of 16 registers of 64 elements, supported by VMS but little used by Unix. x86 went from MMX through SSE, AVX and AVX2 (256-bit) to AVX-512, a new width and a new set of state each time; ARM has NEON in the processor and scalable vectors beyond, and attaches its neural-network and graphics engines as memory-mapped devices; RISC-V has a scalable vector extension. **Trade-offs.** A device needs a setup (a descriptor) and a completion (a status), so small arrays are faster on the processor and large ones on the device; a SIMD register file has no setup and a cost in state, opcodes and width that grows with every generation. **Improvements.** The decision removes the questions that a vector register file raises: whether a context switch saves the vector state (x86's lazy saving of floating-point state was a security hole and Linux went back to eager restore), whether the kernel may use the registers, how wide they are, and what the calling convention does with them. **Benefits.** A processor that stays small and in order and that gains graphics and inference by adding devices. **Modern comparables.** ARM's Ethos neural engines and Mali graphics engines, Apple's matrix coprocessor reached through a library, Google's TPU on PCI Express. **Impact.** *Kernel:* no vector state to save at a context switch and no rule about the kernel using vector registers; a device is one more slot, mapped like any other. *Compiler:* no auto-vectoriser targets the device; the library and the language runtime call it at the level of an operation on an array. *Hardware:* no second register file, no wide load and store path, no opcodes spent; the processor's cost does not grow with the workload.

8.2 Why not SIMD in the processor

8.2.1 A SIMD register file is a second wide register file, which the instruction set deliberately avoided (floating point shares the general registers), and it brings wide loads and stores entangled with the alignment rules and a large demand on a first-page opcode map that is 58 of 64 full. *(D)*

8.2.2 A fixed-width design (128-bit NEON, 256-bit AVX2) bakes the width into every instruction, so a wider generation needs new instructions, and old binaries cannot use the new width. *(D)*

8.2.3 A scalable design (ARM SVE, RISC-V V) avoids that and is the most complex to specify and to implement; it is the only in-processor form that would suit this project's range from a small FPGA core to a possible large one. *(P)*

8.2.4 The compiled corpus shows little vectorisable code at the size-optimising setting; whether it shows more at higher optimisation is a measurement to make before any in-processor form is reconsidered. *(P)*

Commentary. **Precedent.** x86's history of SSE, AVX, AVX2 and AVX-512 is the argument against a fixed width: each added instructions and state, and AVX-512 was withdrawn from some client chips and later proposed to be unified. **Trade-offs.** A scalable vector extension would let one binary run on a small and a large implementation, at the price of the largest specification in the architecture and a compiler model that must handle an unknown vector length. **Improvements.** Moving the work to a device makes the length a descriptor field, which is the scalable idea without a register file. **Benefits.** The block RAM of a small FPGA, the scarce resource of the plan, is not spent on vector registers. **Modern comparables.** AVX2 on x86, NEON on ARM, SVE and the RISC-V vector extension. **Impact.** *Kernel:* nothing to save and no lazy-save hazard. *Compiler:* no vector types in the ABI and no vectorisation pass for the processor. *Hardware:* no area for a register file or wide data paths; the FPGA's memory goes to the caches and the device buffers.

8.3 AI and large-language-model inference

8.3.1 The inference accelerator, LB64, evaluates the layers of a binary-weight language model as sign-flip accumulates with no multipliers, taking a chain of descriptors and writing a status back on completion. *(D)*

8.3.2 A layer is two descriptors, the second accumulating into the first's result for the residual path, and a transformer block is a chain; the processor writes the chain once and is then idle. *(D)*

8.3.3 The work is partitioned between small engines, each doing one thing with its own local memory: LB64 for the projections and the feed-forward layers, a floating-point multiply-accumulate engine for the attention products, a vector engine for normalisation, softmax, activation and rotary position encoding, and the processor for the descriptor chains, sampling and the tokeniser. *(P)*

8.3.4 The accelerator has no instruction set: its computation has exactly one shape, so a programmable sequencer would spend its area on flexibility the work does not use. *(D)*

8.3.5 The processor keeps `POPCNT` and `XNORPOP` for the binary operations that small models and bitsets use; they are not what the accelerator uses. *(D)*

Commentary. **Precedent.** A descriptor chain is how every competent DMA engine since the PDP-11's RK11 has worked, and the partition into small single-purpose engines fed by chains is the idea of the 144-processor GA144 chip applied to a workload that suits it. **Trade-offs.** The accelerator is fast at what it does and does nothing else: attention is not helped, the element-wise work needs another engine, and a training accelerator is a different design again. The accelerator's descriptor is larger than the 32-byte descriptor of chapter 5, so the two must be reconciled: the proposal is that the standard descriptor's address points at a device-defined work block, which can chain, and the ring carries pointers to work blocks. **Benefits.** The lane count is set by the memory bandwidth and not by the amount of logic, so a small board can run a model the processor alone could not. **Modern comparables.** ARM's Ethos engines, Google's TPU, and the many inference accelerators on small boards. **Impact.** *Kernel:* a driver builds a chain and starts it; the kernel owns no model state and the only scheduling is of the device. *Compiler:* none; inference frameworks emit descriptors. *Hardware:* adders and sign-flip logic and no multipliers in LB64; the multiply-accumulate engine needs multiplier blocks; the memory interface sets the size.

8.4 Array languages: APL and K

8.4.1 Array languages express computation as operations on whole arrays: each primitive (add, multiply, compare, reduce, scan, index, where, sort) is one operation over contiguous data.

8.4.2 A device that takes a descriptor naming an operation, the addresses of the arrays and a length is the natural implementation of those primitives: one descriptor per primitive, the length in the descriptor. *(P)*

8.4.3 The interpreter stays on the processor and sends a primitive to the device when the array is long enough to repay the setup; short arrays run on the processor. *(P)*

8.4.4 There is no vector length in the language and none in a register: the length is a field of the descriptor, so one implementation serves any array size. *(P)*

8.4.5 A vector engine for such primitives would provide element-wise arithmetic and comparison, selection, reductions, scans, gather and scatter, compress (`where`) and index generation, on integer, floating-point and bit-array types. *(P)*

Commentary. **Precedent.** Iverson's APL made whole-array operations the language, and its descendants J and Arthur Whitney's K (the language of the kdb+ database) are interpreters whose speed comes from running long loops over contiguous arrays in a few tight kernels; the Cray vector machines were built around the same shape of loop, and DEC offered APL for the PDP-11 (not confirmed in the sources read). **Trade-offs.** Offloading a primitive has a fixed cost, so the benefit is for arrays above a size threshold; reductions and scans carry a dependence along the array that the device must support directly; sorting and gather are bandwidth-limited. **Improvements.** The language supplies the parallelism: the programmer says "add these two arrays", so there is no vectorising compiler to find it, which is the hardest part of making an ordinary language fast on SIMD hardware. The interpretive overhead is paid once per primitive, not once per element, and the device streams the data at memory bandwidth. **Benefits.** Vector languages are easier to support (a small interpreter with a library of primitives that map onto descriptors) and more efficient (the per-element work is hardware, and the per-primitive work is one descriptor), and the same device that serves the array languages serves the elementwise work of inference (section 8.3) and the lane-wise work of graphics. **Modern comparables.** K and kdb+ interpreters use AVX2 intrinsics heavily on x86 *(not confirmed in the sources read)*; array-language compilers for GPUs (Futhark, Co-dfns) target accelerators through offload, the same shape as this. **Impact.** *Kernel:* a language runtime that drives a device from user space needs the device's page mapped into its address space, with the protection caveat of chapter 5 (without a translation unit a user-space driver can direct its device to write anywhere). *Compiler:* none; the interpreter or compiler for the array language lowers primitives to descriptors, and a conventional compiler never sees a vector. *Hardware:* a streaming engine with a small set of element-wise, reduce, scan and gather units behind the ring engine, sized to the memory bandwidth.

8.5 Number formats

8.5.1 The processor handles `binary64`, `binary32` and `binary16` as scalar floating point; other formats (bfloat16, 8-bit integers with 32-bit accumulation, binary weights, 8-bit floating point) are the device's. *(D for the processor; P for the devices)*

8.5.2 Each vector device names its operations and formats in its identification block and feature bits, so a library can choose by asking. *(P)*

Commentary. **Trade-offs.** A format fixed in the processor's instruction set is a format that cannot change when the machine-learning field changes it, as it has several times; a format in a device is replaced by replacing the device. **Modern comparables.** x86 has half precision, bfloat16 and `VNNI` dot products in the instruction set; ARM has `BF16`, `SDOT` and matrix extensions; accelerators handle the rest. **Impact.** *Kernel:* none. *Compiler:* only the scalar formats; the machine-learning formats are descriptor fields. *Hardware:* the devices' multiply-accumulate units, not the processor's, are built for the formats in use.

8.6 Driving a vector device

8.6.1 A program fills a descriptor naming the operation and the arrays, executes `FENCE`, and writes the doorbell; the device processes the descriptor, writes the results and a status, and clears the ownership bit; the program polls or takes the interrupt. *(P)*

8.6.2 A chain of descriptors runs without the processor, and a work block that is a chain is how a transformer layer or a sequence of array primitives is expressed. *(P)*

8.6.3 The buffers are uncached, as for every device, so the processor's reads of results are slow; a large result is left in the device's memory or read back by the next descriptor. *(P)*

Commentary. This is the same driver skeleton as for a disk (chapter 5), with an operation field that means "add these arrays". **Trade-offs.** The slow uncached readback is the cost of the explicit hand-off; an interpreter that needs many small results back on the processor should leave them in device memory. **Impact.** *Kernel:* one driver skeleton. *Compiler:* none. *Hardware:* one ring engine shared with every device.

8.7 Still open

The reconciliation of the accelerators' descriptors with the standard descriptor, the formats and operation set of the vector engine, the target workloads, and the evidence that would reopen the question of packed integer lanes: compile the corpus

at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and estimate the block RAM cost (appendix B, S1 to S6).

Chapter 9 — The instruction set

The instruction set is built around one idea: the first parcel of an instruction says both what the instruction is and how long it is, so the commonest instructions are two bytes and nothing is longer than six, one literal excepted. The chapter covers how the opcode space is divided into prefix-coded leaves, the properties and principles that hold throughout, the registers, the format and the addressing modes, each group of instructions, faults, restart and memory management, position-independent code, what is rejected, the idioms of the PDP-11 and VAX that are restored, lost or replaced and the idioms of the compiler, the comparison with the PDP-11 and VAX, what is taken from LoongArch, and what is open. Every instruction has its entry (syntax, bit-layout chart, operation, flags, exceptions) in the section of its group below; the atomic instructions are in section 7.12 and the system instructions in section 6.7; the reference tables hold the aliases, the constants, the lists of mnemonics and the chart of every leaf. The grammars the tools are generated from are described in `../../../spec/README.md`; every number here comes from them or from the measurements cited.

9.1 How the opcode space is divided

9.1.1 Every instruction begins with a 16-bit parcel (the unit of instruction encoding), and the first parcel alone fixes what the instruction is and how long it is: its top 3 to 16 bits are a prefix, and the prefix selects a leaf. *(D)*

9.1.2 A leaf is a group of mnemonics that share one form. A form lists the operand shapes (how each operand is written in assembly, and how many bits each part of it has); the leaf fixes where every field lies. After the prefix comes the sub-opcode, which selects one mnemonic of the leaf, and then the pieces of the operands: register fields from bit 0 of the first parcel upward, the last register operand lowest, displacements and immediates after them, the widest field last. No bit is padding. pdp64/16 has 208 forms in 270 leaves.

9.1.3 The prefixes form a prefix code (a set of bit strings none of which begins another, so that the first parcel matches at most one leaf and the match needs no look-ahead). The lengths were chosen by an optimiser that was given the compiled corpus, so the shortest prefixes belong to the forms the compiler emits most. *(D)*

9.1.4 A leaf is one, two or three parcels long, which is two, four or six bytes; 55, 149 and 65 of the leaves are of those lengths. One leaf, `MOVQL`, is five parcels (ten bytes) because it carries a 64-bit literal. *(D)*

9.1.5 3,277 of the 65,536 patterns of a first parcel (5.0%) match no prefix and are illegal instructions; so is a sub-opcode that no mnemonic uses and a hot-register index beyond the table (9.4.6), so that a wild jump fails early and nothing is undefined.

9.1.6 The assembly language and every mnemonic of the original encoding are kept. The leaves add 70 words that pdp64/r20 does not have: the four fused pairs (`MOVCALL`, `POPRET`, `POPMRET`, `MOV2`), `MASKEQZ` and `MASKNEZ`, `BSEL`, `MOVQL`, the checked arithmetic `ADDV`, `SUBV` and `MULV`, the fourteen `TRAPCC` words, the deferred and user-qualified moves, the AES and SHA words and the placeholders `SYSPG0` to `SYSPG5`. Each entry says how firm its words are.

9.1.7 This CPU and pdp64/r20 are different CPUs of one architecture: the assembler selects the encoding with `-m`, the object records it, and the linker refuses to link objects of different encodings. The manual of the original CPU is here. *(D)*

Commentary. **Precedent.** The PDP-11 partitioned its opcodes by the addressing modes of the first word; the VAX by operand specifiers parsed in sequence; x86 by prefixes, opcode, ModR/M and SIB read one after another. The prefix code is the device of a Huffman code (the commonest symbols get the shortest strings) applied to the first parcel, with the length of the instruction as a by-product of the match. **Trade-offs.** The decoder reads up to 16 bits to find the length where pdp64/r20 reads two. In the timing model the prefix-coded CPUs pay one cycle of length-decode latency on every taken branch and refill (pdp64/r20 pays none), and fetch-wait is 5.5% of the cycles at four bytes per cycle against 1.7% for pdp64/r20. What the prefix buys is density: on the 841-file corpus the new compiler's output for this CPU is 13.78% smaller than the old compiler's output for pdp64/r20; the encoding alone, measured on the old compiler's code, was 6.48% smaller for the model of pdp64/16 and 9.66% smaller with the four fused pairs, and most of the saving in instructions executed is the compiler (appendix F, F.4). **Improvements.** Every bit is a prefix, sub-opcode or operand bit, so an unused code is an illegal

instruction and not a silent no-op; the fields of every leaf come from the grammar, so no table is written twice. **Benefits.** The common operations, a register move, a load from the stack, a branch on a register against zero, cost two bytes with no look at a second parcel, and the rare ones are as long as they need to be. **Modern comparables.** Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms with the length in a few bits; x86 finds the length by parsing; the prefix code here is the general case of the first and avoids the second. **Impact.** *Kernel:* nothing in the kernel depends on the opcode map; a debugger and the fault handler's step-over read the same generated table as the assembler. *Compiler:* the cost model charges 2, 4 or 6 bytes (10 for `movql`) and knows which operands fit a one-parcel form, the hot registers above all (9.4.6). *Hardware:* the length is a function of up to 16 bits of the first parcel, a lookup in the fetch path where pdp64/r20 has a two-bit counter; a front end that decodes several instructions a cycle needs such a lookup at each parcel position, a cost that the timing model does not measure beyond the one cycle above.

9.2 Properties and principles

9.2.1 The length of an instruction is a function of its first parcel alone: the prefix selects the leaf, and the leaf has 1, 2 or 3 parcels, for 2, 4 or 6 bytes. *(D)*

9.2.2 At most one operand of an instruction may be a memory reference; the other, where present, is a register. The block operations are the one admitted exception. *(D)*

9.2.3 No instruction is longer than six bytes, with one exception: `movql` carries a 64-bit literal and is ten bytes, the literal not being counted as part of the instruction. *(D)*

9.2.4 A register is always written in full: narrow loads sign- or zero-extend, and there is no partial-register write. *(D)*

9.2.5 Anything that changes what an instruction means is, by default, visible state set by an instruction that names it, not hidden state; whether a mode bit is worth its cost is decided case by case (errata 83). Compatibility is by conversion instructions where that is cheap. *(D)*

9.2.6 Every instruction can be restarted after a fault, and exceptions are precise; the machine has no hidden instruction state. *(D)*

9.2.7 Floating point is done in the general registers; there is no separate floating-point register file. *(D)*

9.2.8 Memory is sequentially consistent between processors. *(D)*

9.2.9 The ALU has one width, 64 bits; sizes appear only on loads, stores and conversions. *(D)*

9.2.10 Addresses are canonical: 39 bits of virtual address, 40 bits of physical address. *(D)*

9.2.11 Instructions are aligned to two bytes; quad data accesses are naturally aligned, and the explicit unaligned forms exist for the cases that need them. *(D)*

9.2.12 An instruction does what its name says and nothing else; no flag is set that its entry does not list. *(D)*

9.2.13 Measured on the 841-file corpus of the compiler (instruction bytes, `as64 --stats`), the output of the new compiler for this CPU is 934,470 bytes in 300,821 instructions (3.11 bytes an instruction), 13.78% fewer bytes than the old compiler's 1,083,870 for pdp64/r20; over six large programs it executes 19.32% fewer instructions and 30.96% fewer bytes than the same programs on pdp64/r20, an average of 2.678 bytes an executed instruction against 3.129. The fused pairs account for 19,661 instructions of the corpus. Linker relaxation (9.5.5) takes a further 3.86% off the text of the corpus. The tables and their provenance are in appendix F.

Commentary. **Precedent.** 9.2.2 is x86's ModR/M idea made architectural; it is also what lets the exception machinery be small (the PDP-11 recorded at most two register side effects per instruction because it had two specifiers; with one memory operand there is at most one). 9.2.6 exists because the Motorola 68000, which was very nearly this machine, could not restart an instruction after a bus error and so could not do demand paging until its successor retrofitted a fault model. 9.2.4 is the rule x86-64 adopted for 32-bit writes and failed to apply to the 8- and 16-bit forms. 9.2.7 is the arrangement RISC-V standardised as Zfinx. **Trade-offs.** 9.2.9 means narrow arithmetic is done wide and converted, which costs an extension instruction after some narrow operations; whether that cost is large enough to justify 32-bit forms of add and subtract is an open question that the compiled corpus can answer (appendix B). 9.2.8 is the cheapest model on a small in-order core and the most expensive on a large speculative one. 9.2.10 gives up the 48-bit space of x86-64 and AArch64 for a three-level page table. **Improvements.** The rules are soft defaults: each says what it protects and what breaking it costs. **Benefits.** Together they make the machine small, restartable and easy for a compiler; the measured code size is in chapter 1. **Modern comparables.** AArch64 and RISC-V have fixed-length register-to-register encodings and weak memory models; x86-64 has

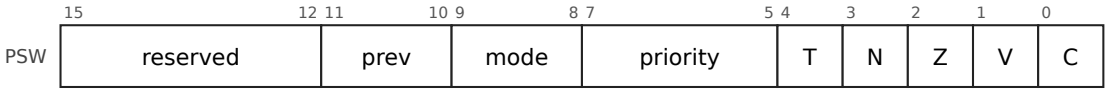
memory operands and total store order; the pdp64 takes memory operands, strong ordering and a bounded fault surface. **Impact.** *Kernel:* Restartable instructions and precise exceptions mean a page-fault handler repairs the mapping and returns, with no instruction-specific cases, and sequential consistency means no barrier code. *Compiler:* One memory operand keeps instruction selection and register allocation simple; a register written in full removes partial-register merges; the single ALU width costs an extension instruction after some narrow arithmetic. *Hardware:* One address generator, at most one register side effect, a single-entry undo buffer, no byte-lane multiplexing and a length that is a lookup on one parcel are what keep the die small.

9.3 State: registers and flags

9.3.1 There are sixteen 64-bit general registers, `R0` to `R15`. `R14` is the stack pointer `SP`, with three copies (kernel, supervisor, user) chosen by the current mode; `R13` is the frame pointer by convention only; the others carry no meaning to the hardware. *(D)*

9.3.2 The program counter is not a general register; it is read with `LEA 0(PC), Rn`. *(D)*

9.3.3 The processor status word is 16 bits. *(D)*



(Bit positions as in System v1: `C` 0, `V` 1, `Z` 2, `N` 3, `T` 4, priority 5-7, current mode 8-9, previous mode 10-11, `UA` 12.)

9.3.4 The flags form two groups, `C` and `{N, Z, V}`, and no instruction writes part of `{N, Z, V}` or writes it conditionally. Moves, loads, stores and the compare-and-branch instructions write no flags; shifts write the same flags for every count; rotates and the funnel shift write none; the by-one left shift sets `V` to `N` xor `C`. *(D)*

9.3.5 `DP`, the direct-page base, has three copies by mode. It makes `d(DP)` an address form that needs no base register (a 28-bit displacement in six bytes; the grammar has loads through `d(DP)` and no store); in kernel mode it is the per-CPU pointer, and in user mode the thread pointer. *(D)*

9.3.6 Other special registers: `FPSR` (five sticky floating-point flags), `ASID`, `PTBR`, `DMW0` and `DMW1`, `EPC`, `EPSW`, `EVA`, `ECAUSE`, `VBR`, `MMUCTL`; the system registers are read and written by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

9.3.7 A bit `DIT` in the status word (its position is unspecified) promises data-independent timing: while it is set no data-dependent timing effect of any kind remains, neither in instruction latency (divide, `MSCN`, `MCMP` and floating point included), nor in frequency or power management (a core that scales frequency holds it steady while the bit is set, or says in a feature register that it cannot), nor from any data-dependent prefetcher or value predictor (a core may have one, but it is off while the bit is set). User code may set it for its own use. *(D)*

9.3.8 A register-save mask (`PUSHM`, `POPM`, `POPMRET`) names the registers by bit: bit n is `Rn`, and the mask is 16 bits (bits 0 to 15 exist). The forms carry the mask in 8 bits (two bytes), 16 bits (four bytes) and 32 bits (six bytes); the fused `POPMRET` has an 8-bit field scaled by 256, which names `R8` to `R15` and no other register. A mask containing `R14` is reserved.

Commentary. **Precedent.** The PDP-11's PC and SP were ordinary registers and its flags were set by nearly every instruction. The VAX put the PC in R15. **Trade-offs.** Keeping the PC out of the register file means a jump to a computed address uses `JMP`, not a write to a register, so that writing `SP` has no control-flow consequence and the hazard logic treats it like any other destination. Condition codes are kept (rather than a flagless design) because compare-and-branch and flag-setting arithmetic make for dense code; the flag rules make them cheap for an out-of-order implementation, because the compiled corpus shows that 68% of instructions write a flag and only 1.5% read one. **Improvements.** The status word carries a user-memory guard bit (9.14) and the per-mode `DP` serves two jobs, the per-CPU pointer in the kernel and the thread pointer in user mode. **Benefits.** An exception handler finds a kernel stack and a per-CPU pointer without first searching for them; a threaded program's thread-local variable is one instruction. **Modern comparables.** x86-64 keeps `FS` and `GS` base registers for the same two jobs; AArch64 has `TPIDR_EL0` and `TPIDR_EL1`; RISC-V uses `tp` and the supervisor scratch register. **Impact.** *Kernel:* Banked stack and direct-page registers let entry find a kernel stack and a per-CPU pointer with no code, and the status word is saved by one `PUSHX`. *Compiler:* The flag rules decide when a compare may fuse with its branch (1,893 pairs stay separate because a second branch reads the flags) and what the compiler may move between a compare and a branch. *Hardware:* Two flag groups need one or two rename targets, not one per bit; the register file is 16 by 64 bits; the banked copies are a three-way mux.

9.4 The instruction format

9.4.1 An instruction is one, two or three parcels (two, four or six bytes; `MOVQL` is five); the first parcel holds the prefix, the sub-opcode and the operand pieces that fit, and the later parcels hold the rest. The bit layout of every leaf is a chart in the reference tables. *(D)*

9.4.2 Instructions are parcel-aligned and little-endian; an instruction may cross any boundary of 8 bytes, a cache line or a page. *(D)*

9.4.3 Most instructions are two or four bytes. Register-to-register work and the whole conversion set are one parcel; so is a memory operand through a hot base register with a displacement of up to three bits, a load or store from the stack pointer at a small offset, and a constant of up to seven bits. A base register with a 12-bit displacement, a post-increment or pre-decrement with a 12-bit stride, a scaled index with a 6- or 8-bit displacement, a pc-relative address within ±32 KiB and an immediate of up to 18 bits are two parcels. Displacements to ±128 MiB, absolute addresses, 32-bit immediates and branches to ±4 GiB are three. *(D)*

9.4.4 Where an addressing mode names no base register (immediate, absolute, PC-relative), no field is spent on one: the later parcels are all value, so those modes reach 16 bits at four bytes and 32 bits at six. *(D)*

9.4.5 A constant wider than 32 bits is either written in the instruction, `MOVQL #imm64, Rd`, which is ten bytes with the literal in the four later parcels, or loaded from a literal pool with `MOVQ =K, Rd`, written with an explicit `=`; nothing is pooled silently, and a `MOVQ #imm` whose constant fits no `MOVQ` form is assembled as `MOVQL`. *(D)*

9.4.6 In a one-parcel form a 3-bit register field is not a register number but an index into a table of the eight hottest registers, `R0 R1 R8 R2 SP R3 R9 R10`. The tables are part of the ABI, and the compiler's register allocator prefers the hot registers. *(D)*

Commentary. **Precedent.** The PDP-11 had the length in the first word, but as a function of both of its six-bit specifiers; Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms with the length in a few bits. Here the length and the identity of the instruction are one match on the prefix. **Trade-offs.** Byte-granular encoding, as in x86, buys perhaps a tenth of the code size and costs a full alignment rotator and cross-line splice logic in the fetch path on a 64-bit datapath; this CPU refuses it. A hot-register index buys a one-parcel form and costs a rule for the compiler: a value that lives in one of the eight hottest registers is cheaper to use, which the allocator learns, and the ABI cannot move the tables without changing the encoding. Forcing an instruction to stay inside an eight-byte window would simplify fetch and would cost 5% to 10% of code size in padding. **Improvements.** `MOVQL` takes a 64-bit constant in one instruction instead of a literal-pool load, at ten bytes against six, the literal not counting against the three-parcel rule (errata 85). **Benefits.** The measured average is 2.7 to 3.3 bytes an instruction (9.2.13). **Modern comparables.** AArch64 and RISC-V have fixed or two-length encodings; x86 has byte-granular ones; this CPU has parcel-granular ones with three common lengths. **Impact.** *Kernel:* a one-parcel form is cheaper in kernel text with no source change. *Compiler:* the assembler picks the shortest form, branch and call relaxation is a length choice, and the cost model knows 2, 4 and 6 bytes and which registers are hot. *Hardware:* the fetch unit has a parcel buffer and a length lookup on the first parcel; no instruction crosses more than two parcel-aligned windows of eight bytes except the ten-byte `MOVQL`.

9.5 Addressing modes

9.5.1 The addressing mode is not a field: it is the leaf, whose form lists the shape of each operand. The assembly language has the eight modes below, and the assembler gives each operand the shortest leaf that holds it. *(D)*

Syntax	Effect	One parcel	Two parcels	Three parcels
`d(Rn)`	base plus displacement	`(Rn)` for any register, or a hot base register with a displacement of 1 to 3 bits; `d(SP)` with a displacement of 4 to 7 bits	8-, 10- or 12-bit displacement (scaled by the size of the access in most forms)	28-bit signed displacement, ±128 MiB
`d(Rn)(Rx*s)`	scaled index, `s` in {1, 2, 4, 8}	none	6- or 8-bit displacement	22-bit displacement
`d(PC)`	PC-relative; the base is the first parcel of the instruction	none	12-, 14- or 16-bit byte displacement: ±2 KiB, ±8 KiB, ±32 KiB	32-bit displacement, ±2 GiB
`d(DP)`	direct page, loads only	none	none	28-bit displacement
`#imm`	immediate, sign-extended	3 to 7 bits	up to 18 bits	32 bits (`MOVQL`: 64 bits, five parcels)
`@abs`	absolute, sign-extended	none	none	32-bit address

Syntax	Effect	One parcel	Two parcels	Three parcels
`(Rn)+d`	post-increment by a stride `d` (0: the operand size)	none	12-bit stride	none
`-(Rn)` or `(Rn)-d`	pre-decrement by a stride (0: the operand size)	none	12-bit stride	none

9.5.2 The deferred modes of the PDP-11 are removed; indirection is done with `JMPI`, `CALLI` and a load. *(D)*

9.5.3 At most one register side effect occurs per instruction, and it is committed at retirement; a fault discards it. *(D)*

9.5.4 A pc-relative operand is measured in bytes from the first parcel of the instruction; the field counts bytes for a memory operand and parcels for a branch or call. A branch has a field of 6 to 9 bits in one parcel (±64 to ±512 bytes), 16 bits in two (±64 KiB) and 32 bits in three (±4 GiB); a `CALL` has 18 bits in two parcels (±256 KiB) and 32 in three; a `LEA` or a load through `d(PC)` reaches ±2 KiB to ±32 KiB in two parcels and ±2 GiB in three. The dictionary lists the width and reach of every operand shape.

9.5.5 The assembler cannot know the distance to a symbol in another object, so it writes such a `CALL` or `LEA` of a data symbol in its long form (six bytes) with a relocation; the linker lays out the sections, gives each such instruction the shortest form that reaches (a 4-byte `CALL` within ±256 KiB, a 4-byte `LEA` or pc-relative load within ±32 KiB, a 4-byte `JMP` or `MOVCALL` within ±64 KiB) and moves everything behind it. It grows each instruction from its shortest form, so the lengths only grow and the iteration ends. Relaxation is on by default; `as64 --no-relax` and `ld64 --no-relax` give the unrelaxed bytes. *(D)*

Commentary on 9.5.1, 9.5.4 and 9.5.5. **Precedent.** The PDP-11 chose the mode in each six-bit specifier; x86 chooses it in ModR/M and SIB; here the leaf chooses it, and the cost of a mode is the length of the leaf that has it. **Trade-offs.** The scaled index is the thin case: its shortest form has a 6-bit displacement, and `d(DP)` has only the six-byte load form, so a thread-local access through `DP` costs six bytes. Relaxation turns the assembler's guess into the linker's measurement: it saved 3.86% of the text of the corpus (an upper bound for the reach of the 4-byte call, since the corpus stub places symbols right behind each file) and 1.73% of the linked text of the six large programs; external calls, 6 bytes each, are the bulk. **Improvements.** The stride of the post-increment and pre-decrement is the same field in every leaf, so an array of 16-byte records is one instruction per element; the scaled index takes a conversion of a narrower index in the compiler (`bs_ec`'s inner loop, 31% fewer instructions). **Benefits.** An object file from `as64` is correct without relaxation and smaller with it, and a program linked either way produces the same output on the six large programs. **Modern comparables.** RISC-V's linker relaxes `call` and `lui`/`addi` pairs for the same reason; AArch64 needs linker veneers for far branches. **Impact.** *Kernel:* the kernel is relaxed like any other program, and text near the reach limits is smaller by the 4-byte forms. *Compiler:* it emits the long form for an external symbol and leaves the choice to the linker; it prefers scaled index and strided autoincrement where its loops allow (appendix F, F.4). *Hardware:* nothing: the address adder takes a field and a base in every leaf.

Commentary. **Precedent.** The PDP-11 had eight modes for each of eight registers, with a deferred flavour of each, and it made `*p++` and `*--p` single instructions. **Trade-offs.** Dropping the deferred modes removes the case where a fault occurs after a register has been updated at an address that itself came from a load; keeping the strided autoincrement costs nothing, because the undo information was already a delta (the PDP-11's `MMR1` recorded an amount for the same reason). **Improvements.** The increment is a stride, so walking an array of 16-byte records is one instruction; the PC-relative modes count from the instruction itself, so a branch needs no adjustment and `LEA` reads the PC. **Benefits.** Address generation is one adder pass, never load-then-add, which is the difference between one cycle and a long path on an FPGA. **Modern comparables.** AArch64 has pre- and post-indexed forms with an immediate; x86 has scaled index with a base; RISC-V has base plus immediate only. **Impact.** *Kernel:* A strided autoincrement walks a table of records, and `d(DP)` reaches the per-CPU structure in one instruction. *Compiler:* The modes match C idioms (`*p++`, `a[i]`) one-to-one, and dropping the deferred modes removes the cases the selector would otherwise have to consider. *Hardware:* Address generation is one adder pass, never load-then-add, and the PC-relative base is the first parcel, so no adjustment is needed.

9.6 Moves, conversions and address formation

The instructions. Every instruction has an entry in the section of its group, laid out as in a processor reference manual: its mnemonics, its syntax, its encoding (the chart of its layout and the list of all its layouts, each with a chart of its own), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains the matter it touches (*Cf.*). Every instruction, the atomic and the system instructions included, has its official entry below (sections 9.13 and 9.14 for those two groups); the chapters on the kernel and on several processors give the reasons, and an entry may point back

to them for a fuller explanation. In the pseudocode, ← is assignment, = and ≠ compare, × and ÷ multiply and divide, and `M[a]` is memory at address `a`. Numbers that are encodings (prefixes, sub-opcodes, field values) are written in binary. Masks are binary too, with a subscript 2 to mark the base (`Rn & 111₂` keeps the low three bits). The layouts are generated from the grammars of the encodings, `spec/pdp64-enc.hbnf` and the grammar of the CPU, so a chart here cannot disagree with the assembler, the linker or the simulator; the words (operation, flags, exceptions) are checked against the reference executor, `common/pdp64exec.c` and `common/pdp64exec_enc.inc`, which `sim64` runs, and the Status line of each entry says whether the executor implements the instruction. The aliases, the constants, the lists of the mnemonics and the bit layouts of every opcode are in the reference tables.

The stories. An instruction that is not common (the PDP-11 core and the operations every architecture has carry none) is followed by *Why it is here*: the machine the idea came from and the reason it was chosen, with the measurements that bear on it. Where no reason is recorded, the paragraph says so. The documents behind the paragraphs are listed in appendix F.

Notation. `Rd` is the destination register, `Rs` the first source, `Rt` the second, `Ru` the third; the assembler writes the destination last, and the operands of a chart are numbered `op1`, `op2`, ... in the order of the assembly syntax. `rm` is a general operand: a register, or a memory operand, an immediate, a pc-relative or an absolute address, by the addressing forms that the layouts of the instruction give (section 9.5). `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. `N Z V C` are the condition flags. In the charts the bits of a parcel are numbered from 15 at the left, a shaded box is a fixed value, and a white box is a field that varies.

Registers. pdp64/16 has 16 registers, `R0` to `R15`. A register field of an instruction is the full number of bits (4), or fewer where the field names one of the hot registers; the hot-register tables are part of the ABI: the 3-bit fields hold, in order, `R0 R1 R8 R2 SP R3 R9 R10`.

Cf. section 6.4 for the memory types that decide whether an access is cached, and section 6.8 for what a faulting load leaves behind.

9.6.1 `MOVQ`, `MOVL`, `MOVW` and `MOVB` load and store 64, 32, 16 and 8 bits; the loads sign-extend. `MOVZL`, `MOVZW` and `MOVZB` load with zero extension.

9.6.2 With a register operand, the same opcodes are the conversion set: `MOVB R1, R2` sign-extends the low byte, `MOVZW R1, R2` zero-extends the low word.

9.6.3 `LEA` forms an effective address; `LEA 0(PC), Rn` reads the program counter.

9.6.4 Moves, loads and stores write no flags. *(D)*

9.6.5 The explicit unaligned forms `MOVUB`, `MOVUW`, `MOVUL` and `MOVUQ` exist in both directions. *(D)*

9.6.6 The one-parcel forms abbreviate the common cases: loads and stores through a hot base register or the stack pointer at small offsets, `LEA` from `SP`, and `MOVQ #imm, rd` with an immediate of up to seven bits. *(D)*

Commentary. **Precedent.** The PDP-11 had `MOV` and `MOVB` with a byte sign-extended into a register; the VAX had a sized `MOV` for every width. **Trade-offs.** Keeping the sized loads costs four extra opcodes against a quad-only design, but without `MOVL` a struct field load becomes two `MOVW`s, a shift and an or, and ELF, timestamps and most on-disk and wire structures are full of 32- and 16-bit fields. **Improvements.** There is no separate `CVT` group: the conversions are the moves with a register source, so they cost nothing. **Benefits.** No partial-register state anywhere. **Modern comparables.** AArch64 and RISC-V have the same sized loads with sign and zero extension; x86-64 keeps 8- and 16-bit register writes that preserve the upper bits, which makes narrow writes depend on the old value. **Use.** `LEA 0(PC)` is how a position-independent program finds itself; the unaligned forms are for packed on-disk and network structures. **Impact.** *Kernel:* `LEA 0(PC)` lets position-independent kernel code find itself; the unaligned forms read packed on-disk and network structures. *Compiler:* Sized loads avoid the shift, mask and or of a struct-field read, and the conversions are moves, so no separate conversion selection exists. *Hardware:* No partial-register write anywhere; the conversions reuse the load-extend path.

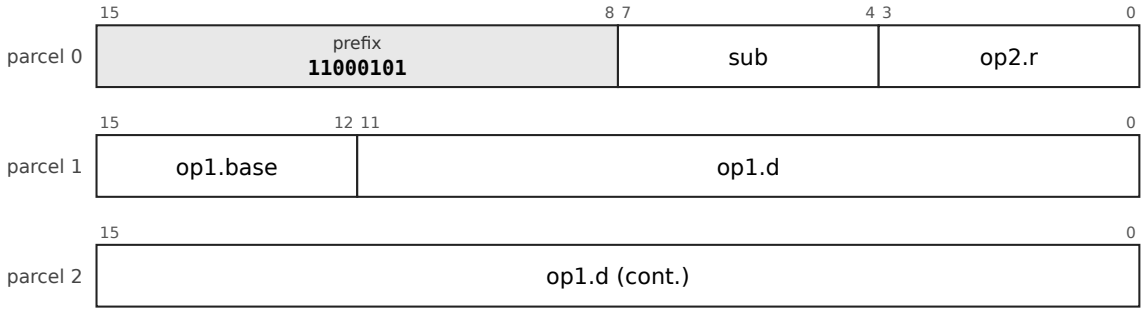
MOV: Move, load and store (64, 32, 16 and 8 bits)

Mnemonics: `MOVQ`, `MOVL`, `MOVW`, `MOVB`

Syntax

```
MOVQ rm, Rd        ; load
MOVQ Rs, rm        ; store
MOVL, MOVW, MOVB likewise
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-STF-B28-1`, `16-STF-X22-1`, `16-STF-PI12-1`, `16-STF-PD12-1`, `16-STF-PC32-1`, `16-STF-A32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDI12-a-1`, `16-STBs8-a-1`, `16-LDPC12-a-1`, `16-LDBs8-a-1`, `16-LDPC14-a-1`, `16-LDI14-a-1`, `16-STX2-a-1`, `16-LDI16-a-1`, `16-IMM3-a-1`, `16-LDPC16-a-1`, `16-LDBu10-a-1`, `16-LDHHH1-a-1`, `16-STHHH1-a-1`, `16-STBu10-a-1`, `16-LDSP4-a-1`, `16-STSP4-a-1`, `16-LDBu12-a-1`, `16-IMM5-a-1`, `16-LDHHH3-a-1`, `16-LDX6-a-1`, `16-IMM7-a-1`, `16-STBu12-a-1`, `16-STX6-a-1`, `16-LDBs10-a-1`, `16-LDSP5-a-1`, `16-STHHH3-a-1`, `16-LD0-a-1`, `16-STSP5-a-1`, `16-LDI18-a-1`, `16-ST0-a-1`, `16-LDBs14-a-1`, `16-STBs10-a-1`, `16-LDHRH2-a-1`, `16-LDSP7-a-1`, `16-LDX8-a-1`, `16-LDHHR2-a-1`, `16-LDFI-I32-2`, `16-STBu8-b-1`, `16-LDBu8-b-1`, `16-STX2-b-1`, `16-LDX2-b-1`, `16-STBs8-b-1`, `16-STBu10-b-1`, `16-STHHH1-b-1`, `16-STBu12-b-1`, `16-LDBu10-b-1`, `16-LDHHH1-b-1`, `16-LDBu12-b-1`, `16-STX6-b-1`, `16-LDPC12-b-1`, `16-LDBs8-b-1`, `16-LDX4-b-1`, `16-LDPC14-b-1`, `16-STSP4-b-1`, `16-STHHH2-b-1`, `16-STBs10-b-1`, `16-LDI12-d-2`.

Operation

```
size = 8 (MOVQ), 4 (MOVL), 2 (MOVW) or 1 (MOVB) bytes
load :  Rd ← sext(M[EA(rm)], size)          ; len = 00: Rd ← sext(Rs[8×size-1:0])
store:  M[EA(rm)] ← Rs[8×size-1:0]          ; len = 00: Rd ← sext(Rs[8×size-1:0])
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

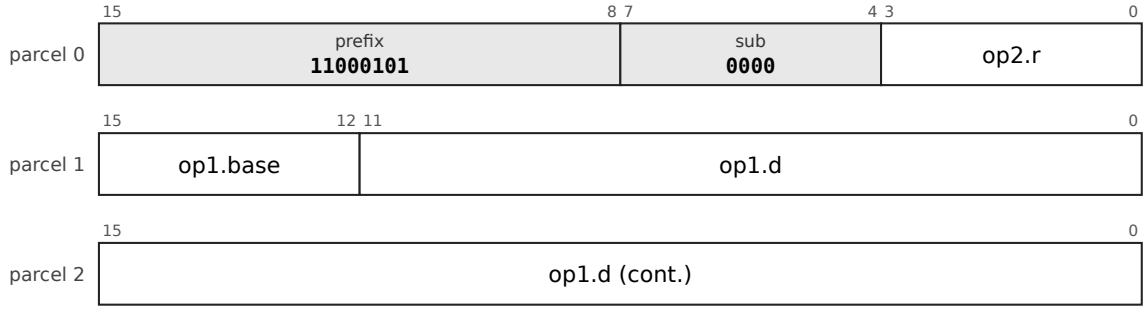
A register is always written in full: a narrow load sign-extends to 64 bits and there is no partial-register write anywhere. The register-to-register forms of the sized moves are the conversion set: `MOVB R1, R2` sign-extends the low byte of `R1` into `R2`.

Cf. 6.4 for the memory type that decides whether the access is cached; 6.8 for what a faulting access leaves behind.

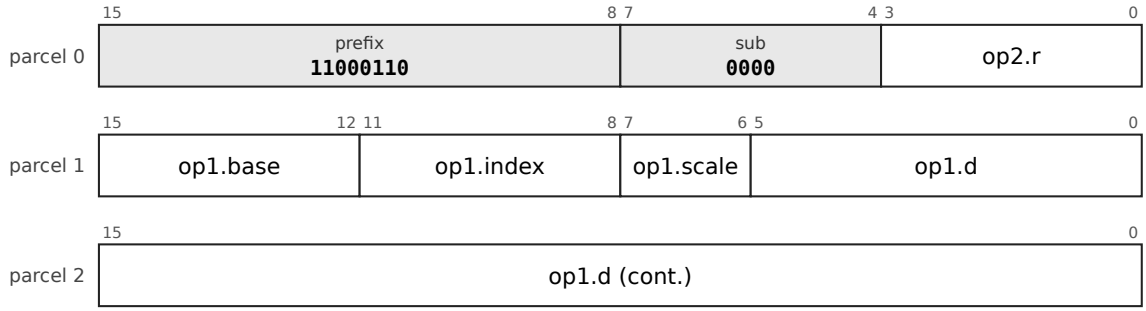
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MOVQ

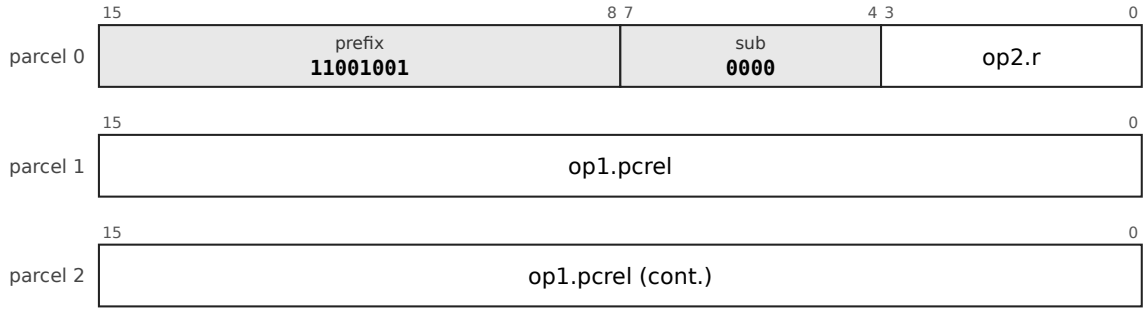
MOVQ: LDF.B28, prefix 11000101, sub-opcode 0000; operands b4d28s-x1, reg4; 6 bytes



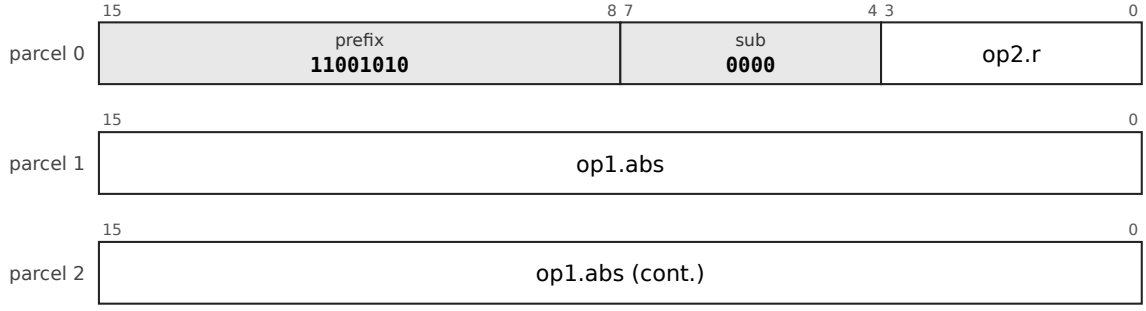
MOVQ: LDF.X22, prefix 11000110, sub-opcode 0000; operands x4-4d22s, reg4; 6 bytes



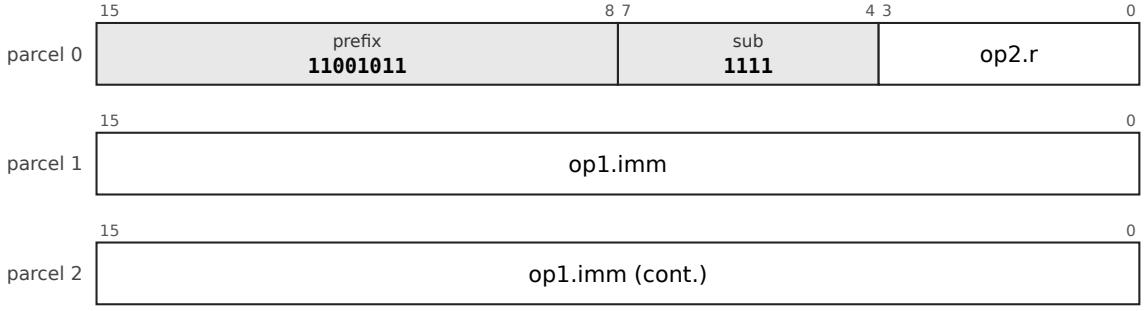
MOVQ: LDF.PC32, prefix 11001001, sub-opcode 0000; operands pc32-u1, reg4; 6 bytes



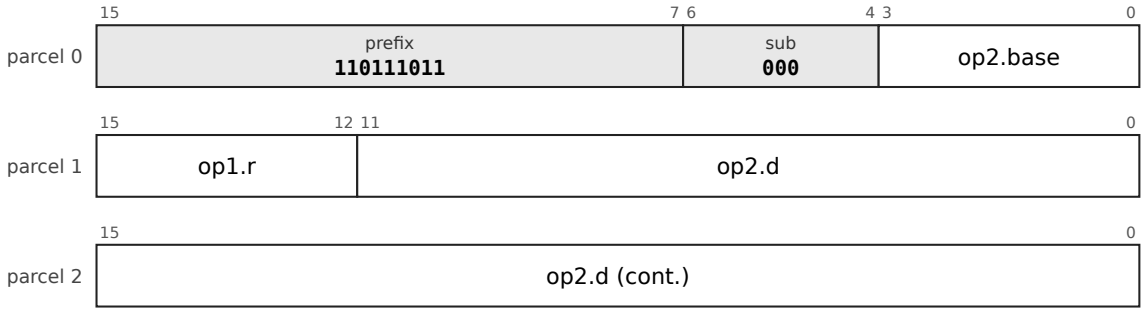
MOVQ: LDF.A32, prefix 11001010, sub-opcode 0000; operands abs32, reg4; 6 bytes



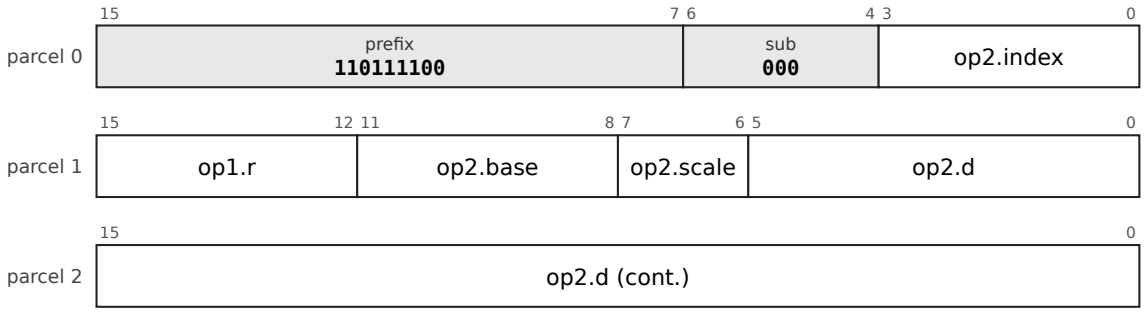
MOVQ: LDFI.I32, prefix 11001011, sub-opcode 1111; operands imm32s, reg4; 6 bytes



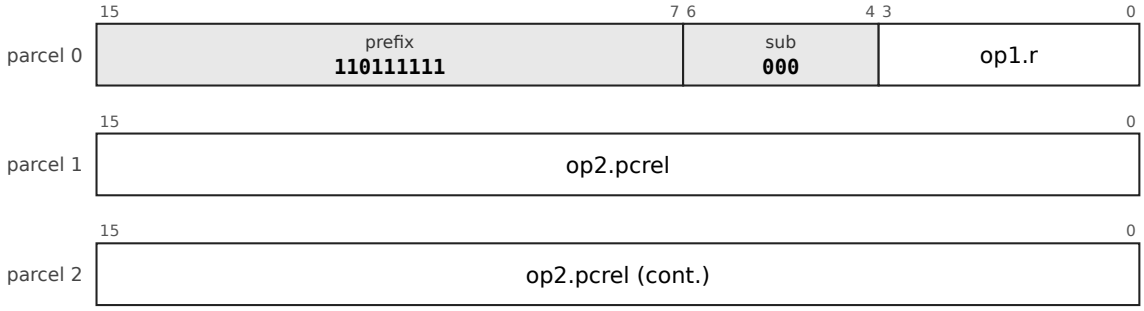
MOVQ: STF.B28, prefix 110111011, sub-opcode 000; operands reg4, b4d28s-x1; 6 bytes



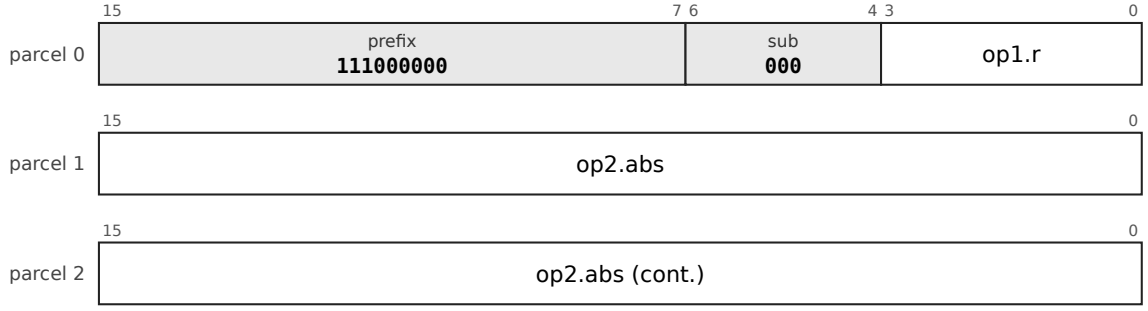
MOVQ: STF.X22, prefix 110111100, sub-opcode 000; operands reg4, x4-4d22s; 6 bytes



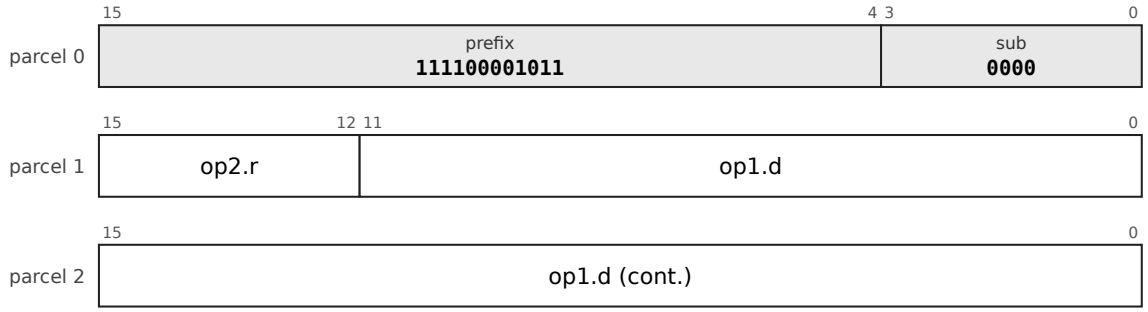
MOVQ: STF.PC32, prefix 110111111, sub-opcode 000; operands reg4, pc32-u1; 6 bytes



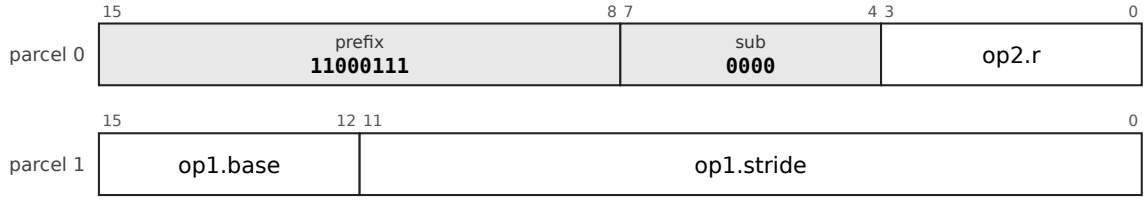
MOVQ: STF.A32, prefix 111000000, sub-opcode 000; operands reg4, abs32; 6 bytes



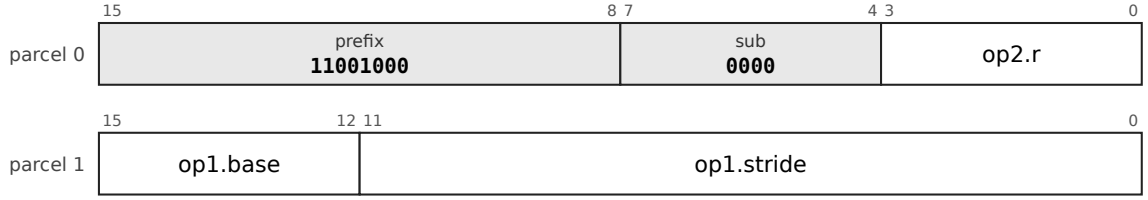
MOVQ: DPF.D28, prefix 111100001011, sub-opcode 0000; operands dpd28s-x1, reg4; 6 bytes



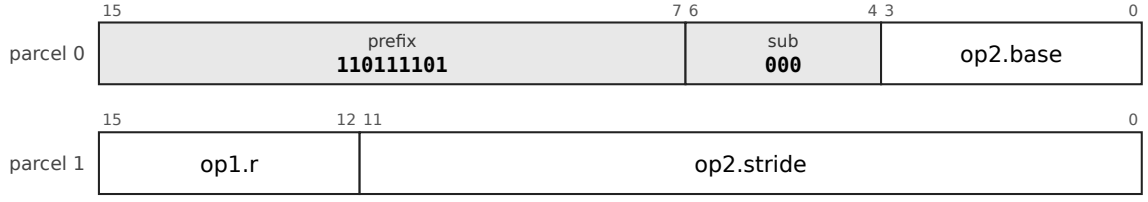
MOVQ: LDF.PI12, prefix 11000111, sub-opcode 0000; operands pi4-s12, reg4; 4 bytes



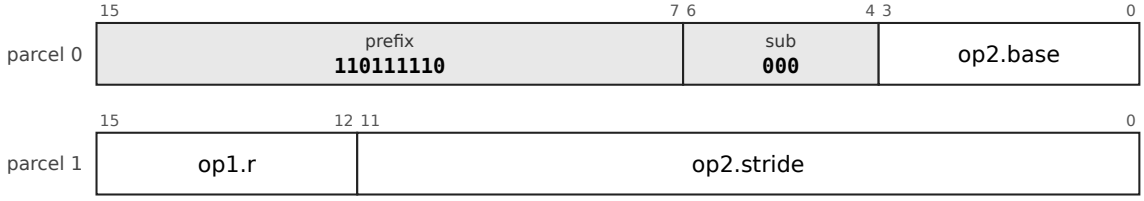
MOVQ: LDF.PD12, prefix 11001000, sub-opcode 0000; operands pd4-s12, reg4; 4 bytes



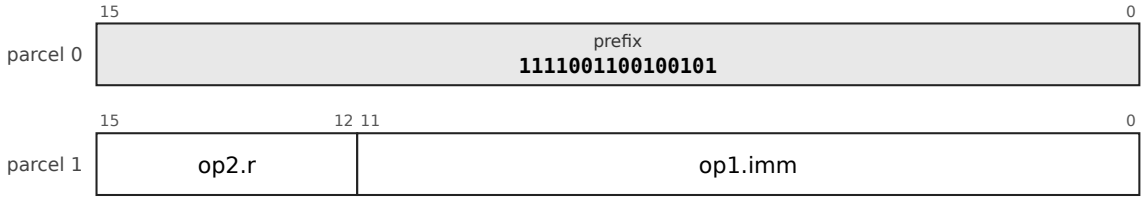
MOVQ: STF.PI12, prefix 110111101, sub-opcode 000; operands reg4, pi4-s12; 4 bytes



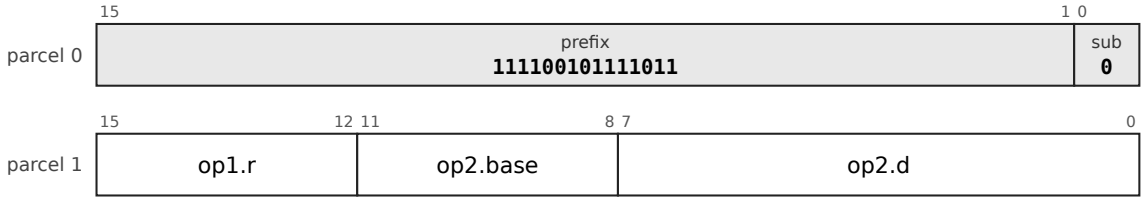
MOVQ: STF.PD12, prefix 110111110, sub-opcode 000; operands reg4, pd4-s12; 4 bytes



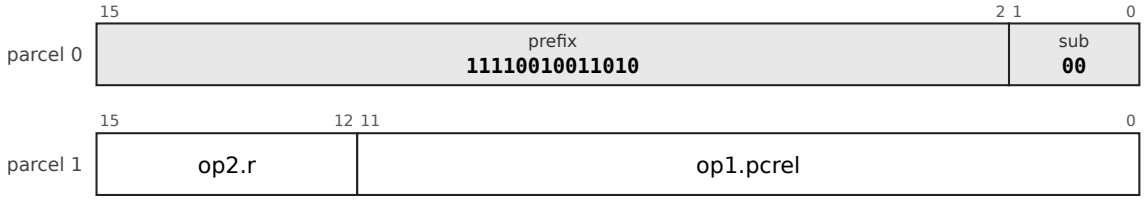
MOVQ: LDI12/a, prefix 1111001100100101; operands imm12s, reg4; 4 bytes



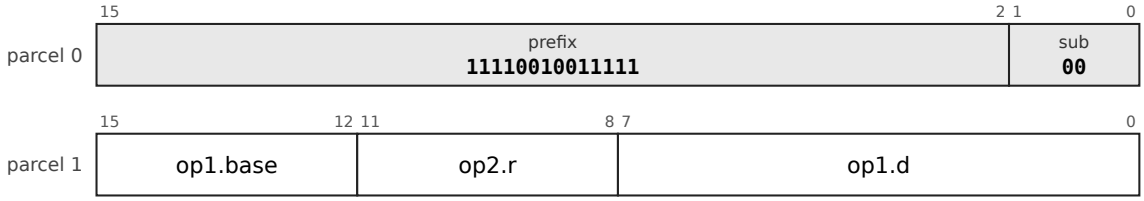
MOVQ: STBs8/a, prefix 111100101111011, sub-opcode 0; operands reg4, b4d8s-xsz; 4 bytes



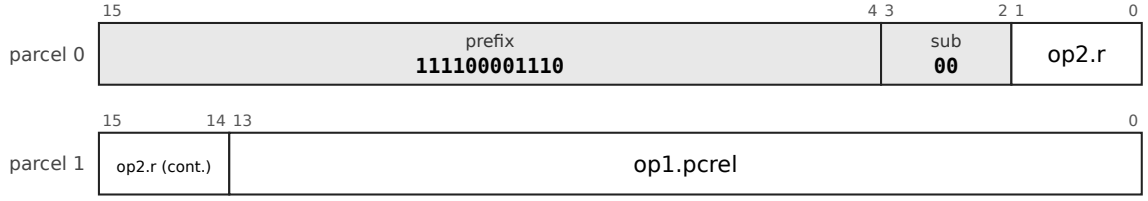
MOVQ: LDPC12/a, prefix 11110010011010, sub-opcode 00; operands pc12-u1, reg4; 4 bytes



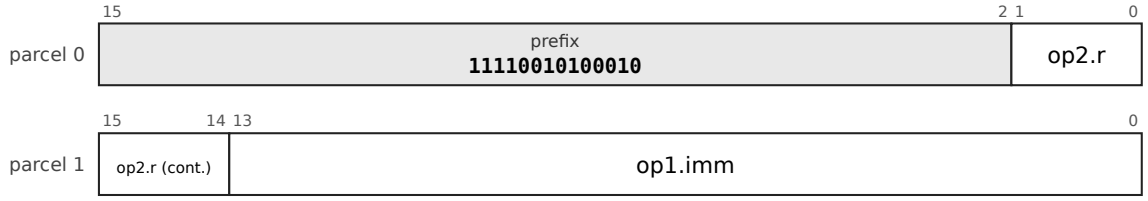
MOVQ: LDBs8/a, prefix 11110010011111, sub-opcode 00; operands b4d8s-xsz, reg4; 4 bytes



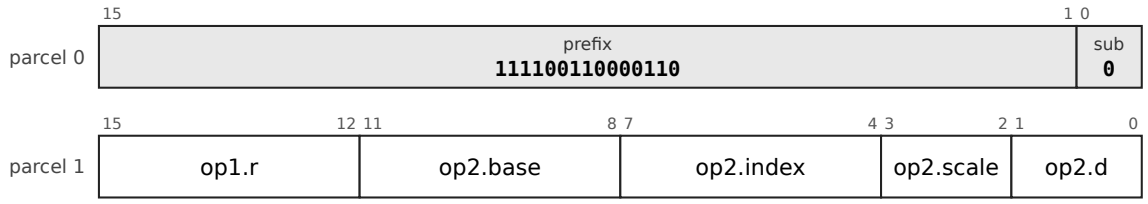
MOVQ: LDPC14/a, prefix 111100001110, sub-opcode 00; operands pc14-u1, reg4; 4 bytes



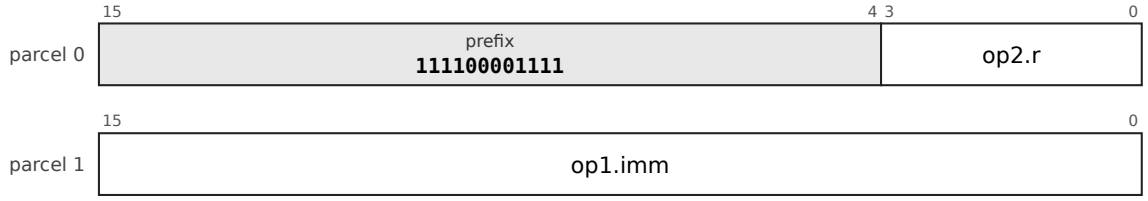
MOVQ: LDI14/a, prefix 11110010100010; operands imm14s, reg4; 4 bytes



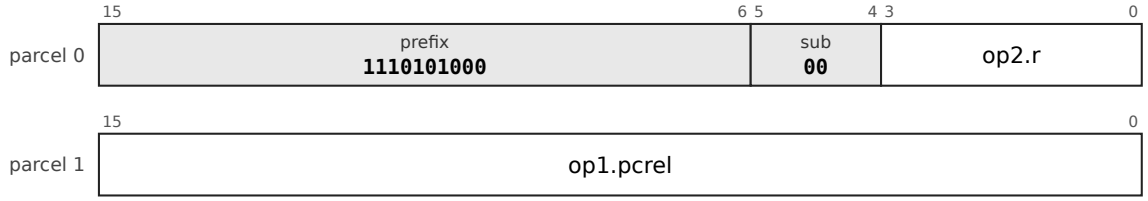
MOVQ: STX2/a, prefix 111100110000110, sub-opcode 0; operands reg4, x4-4d2s; 4 bytes



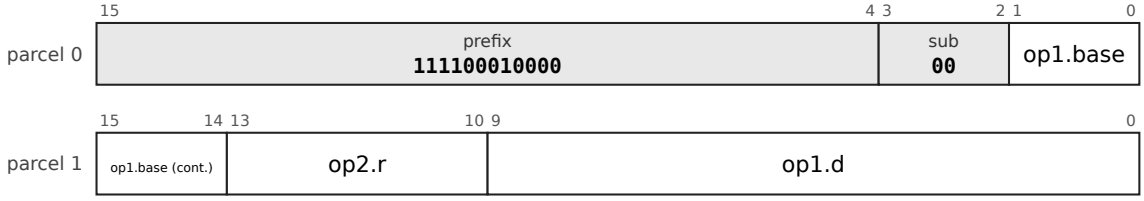
MOVQ: LDI16/a, prefix 111100001111; operands imm16s, reg4; 4 bytes



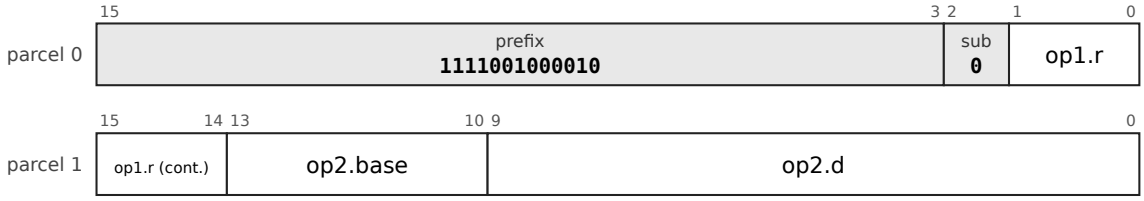
MOVQ: LDPC16/a, prefix 1110101000, sub-opcode 00; operands pc16-u1, reg4; 4 bytes



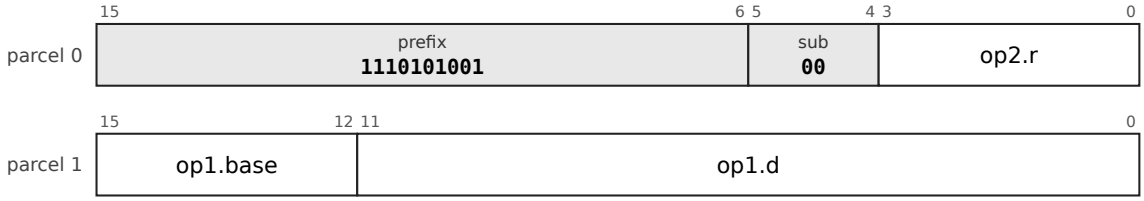
MOVQ: LDBu10/a, prefix 111100010000, sub-opcode 00; operands b4d10u-xsz, reg4; 4 bytes



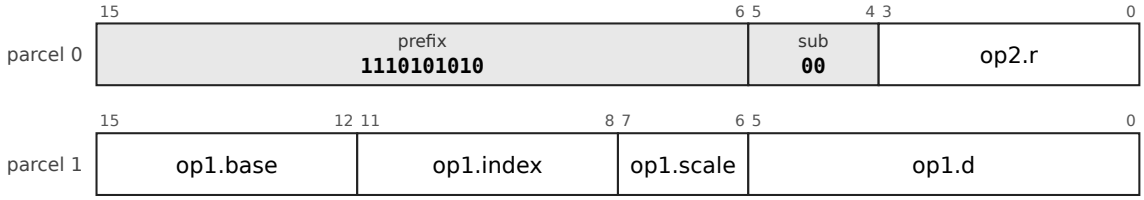
MOVQ: STBu10/a, prefix 1111001000010, sub-opcode 0; operands reg4, b4d10u-xsz; 4 bytes



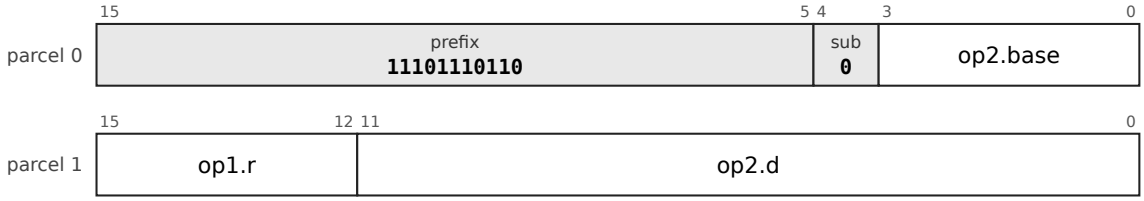
MOVQ: LDBu12/a, prefix 1110101001, sub-opcode 00; operands b4d12u-xsz, reg4; 4 bytes



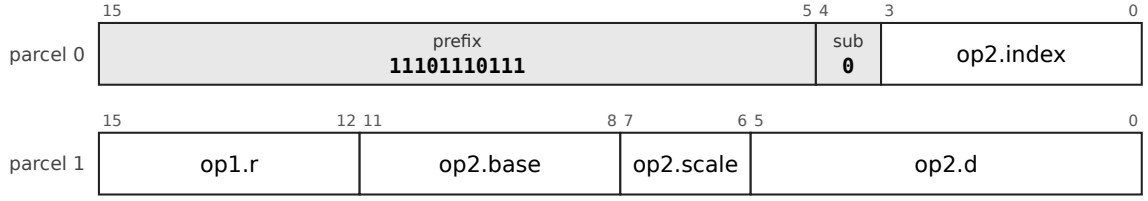
MOVQ: LDX6/a, prefix 1110101010, sub-opcode 00; operands x4-4d6s, reg4; 4 bytes



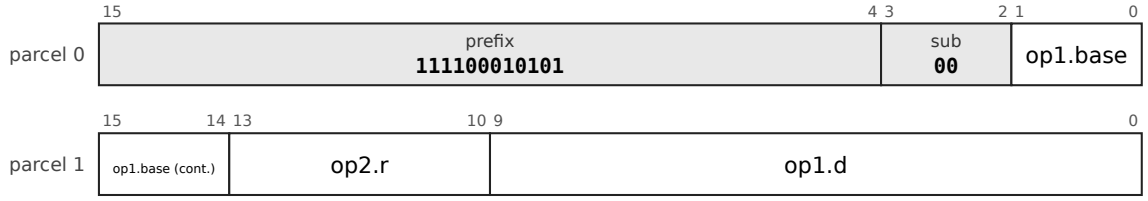
MOVQ: STBu12/a, prefix 11101110110, sub-opcode 0; operands reg4, b4d12u-xsz; 4 bytes



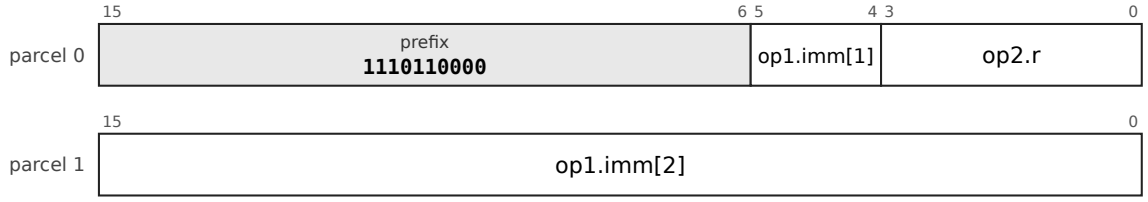
MOVQ: STX6/a, prefix 11101110111, sub-opcode 0; operands reg4, x4-4d6s; 4 bytes



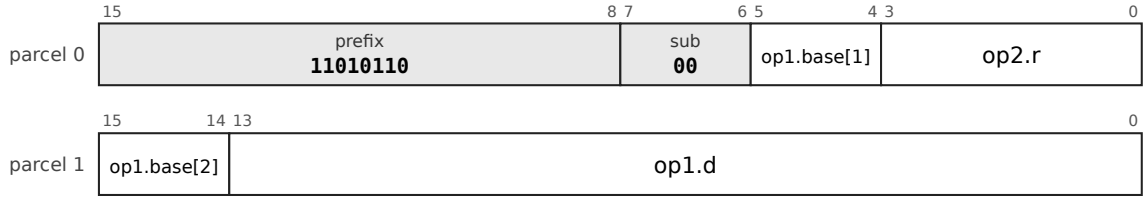
MOVQ: LDBs10/a, prefix 111100010101, sub-opcode 00; operands b4d10s-xsz, reg4; 4 bytes



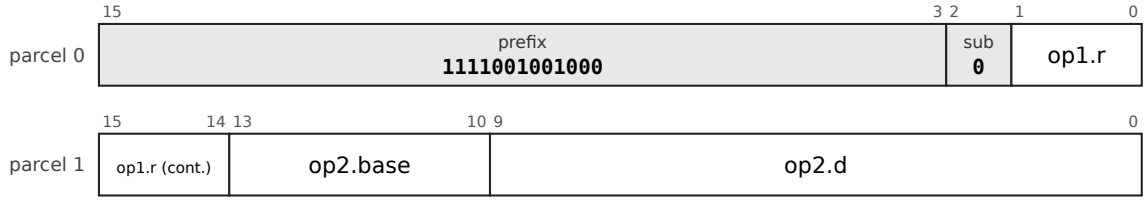
MOVQ: LDI18/a, prefix 1110110000; operands imm18s, reg4; 4 bytes



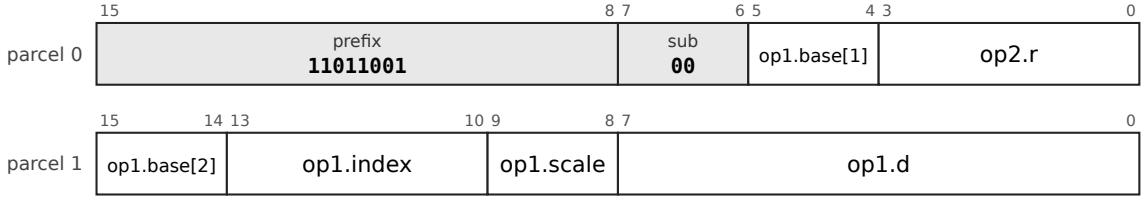
MOVQ: LDBs14/a, prefix 11010110, sub-opcode 00; operands b4d14s-xsz, reg4; 4 bytes



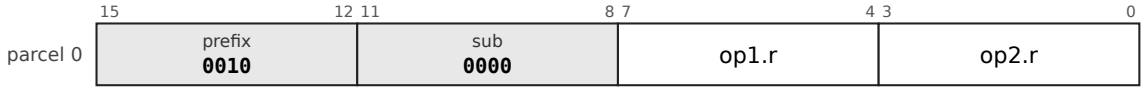
MOVQ: STBs10/a, prefix 1111001001000, sub-opcode 0; operands reg4, b4d10s-xsz; 4 bytes



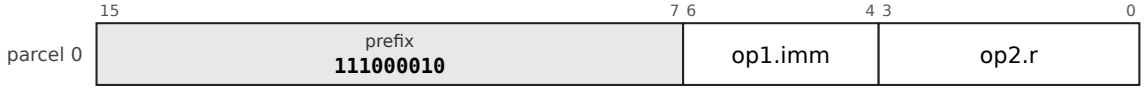
MOVQ: LDX8/a, prefix 11011001, sub-opcode 00; operands x4-4d8s, reg4; 4 bytes



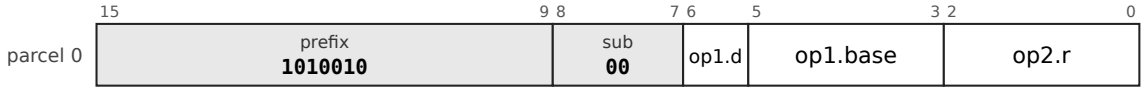
MOVQ: RRF, prefix 0010, sub-opcode 0000; operands reg4, reg4; 2 bytes



MOVQ: IMM3/a, prefix 111000010; operands imm3s, reg4; 2 bytes



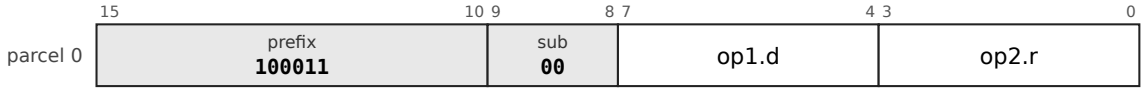
MOVQ: LDHHH1/a, prefix 1010010, sub-opcode 00; operands b3d1u-xsz, reg3; 2 bytes



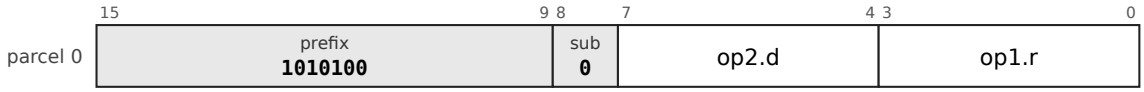
MOVQ: STHHH1/a, prefix 11001111, sub-opcode 0; operands reg3, b3d1u-xsz; 2 bytes



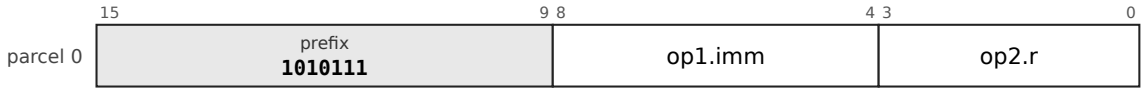
MOVQ: LDSP4/a, prefix 100011, sub-opcode 00; operands spd4u-xsz, reg4; 2 bytes



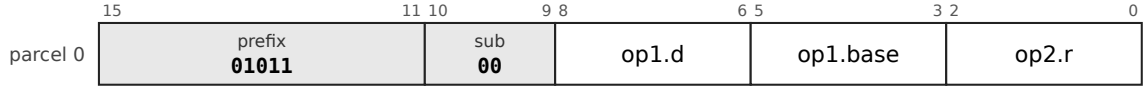
MOVQ: STSP4/a, prefix 1010100, sub-opcode 0; operands reg4, spd4u-xsz; 2 bytes



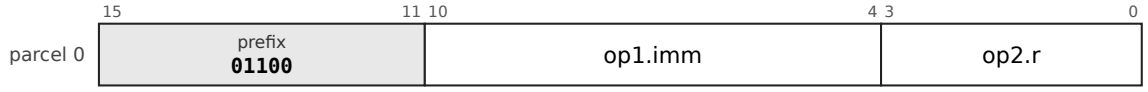
MOVQ: IMM5/a, prefix 1010111; operands imm5s, reg4; 2 bytes



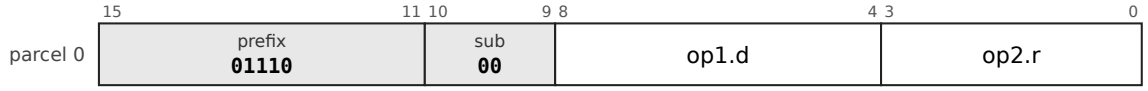
MOVQ: LDHHH3/a, prefix 01011, sub-opcode 00; operands b3d3u-xsz, reg3; 2 bytes



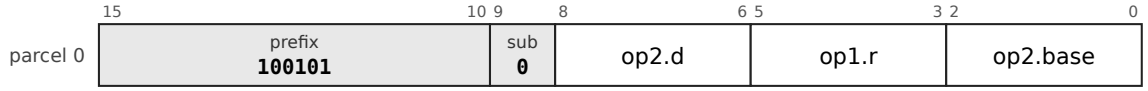
MOVQ: IMM7/a, prefix 01100; operands imm7s, reg4; 2 bytes



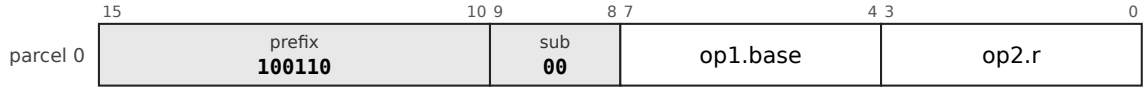
MOVQ: LDSP5/a, prefix 01110, sub-opcode 00; operands spd5u-xsz, reg4; 2 bytes



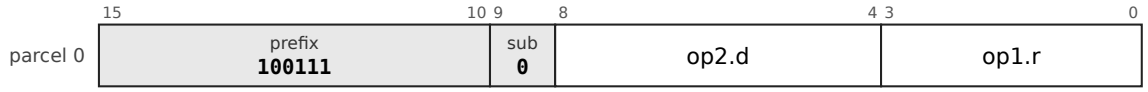
MOVQ: STHHH3/a, prefix 100101, sub-opcode 0; operands reg3, b3d3u-xsz; 2 bytes



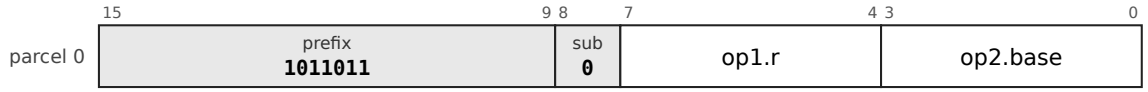
MOVQ: LD0/a, prefix 100110, sub-opcode 00; operands b4d0s-xsz, reg4; 2 bytes



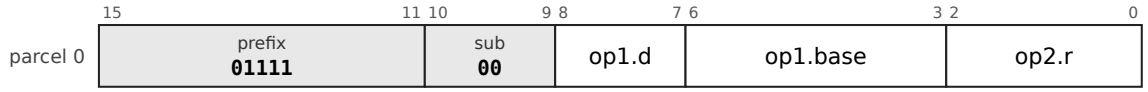
MOVQ: STSP5/a, prefix 100111, sub-opcode 0; operands reg4, spd5u-xsz; 2 bytes



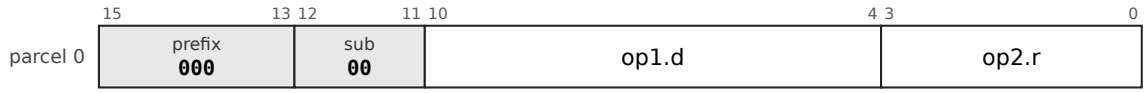
MOVQ: ST0/a, prefix 1011011, sub-opcode 0; operands reg4, b4d0s-xsz; 2 bytes



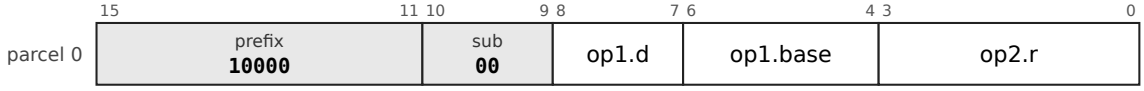
MOVQ: LDHRH2/a, prefix 01111, sub-opcode 00; operands b4d2u-xsz, reg3; 2 bytes



MOVQ: LDSP7/a, prefix 000, sub-opcode 00; operands spd7u-xsz, reg4; 2 bytes



MOVQ: LDHHR2/a, prefix 10000, sub-opcode 00; operands b3d2u-xsz, reg4; 2 bytes

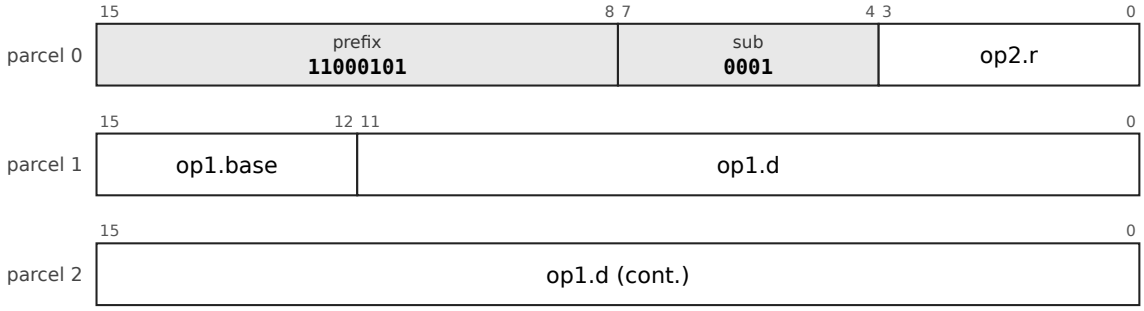



```
MOVQ rm, Rd
MOVQ Rs, rm
```

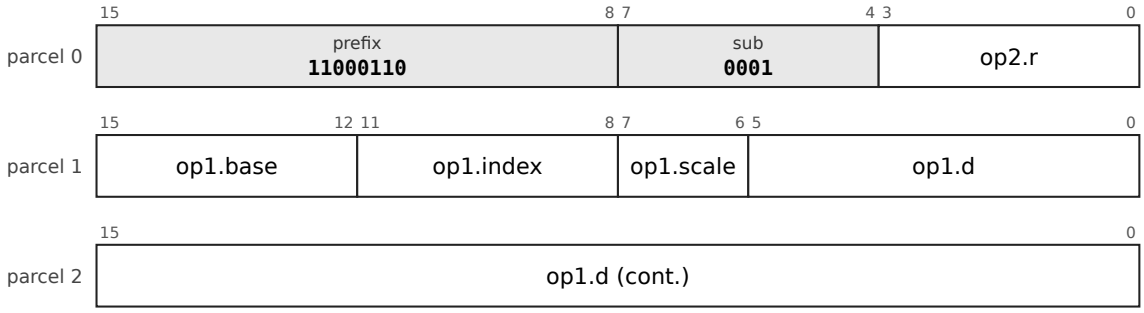
Operation, flags and exceptions as for the entry MOV above; 51 layout forms, each a chart of its own (above).

MOVL

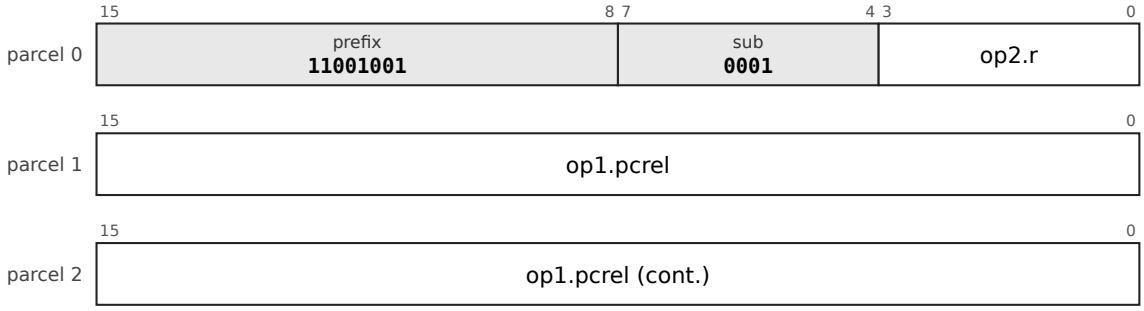
MOVL: LDF.B28, prefix 11000101, sub-opcode 0001; operands b4d28s-x1, reg4; 6 bytes



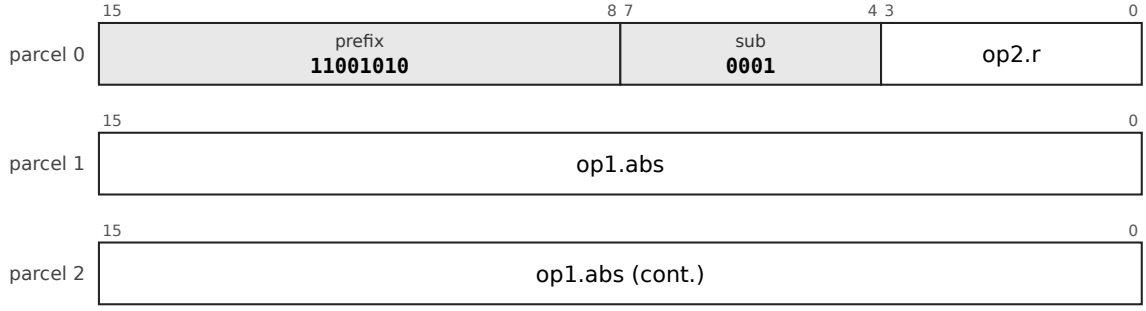
MOVL: LDF.X22, prefix 11000110, sub-opcode 0001; operands x4-4d22s, reg4; 6 bytes



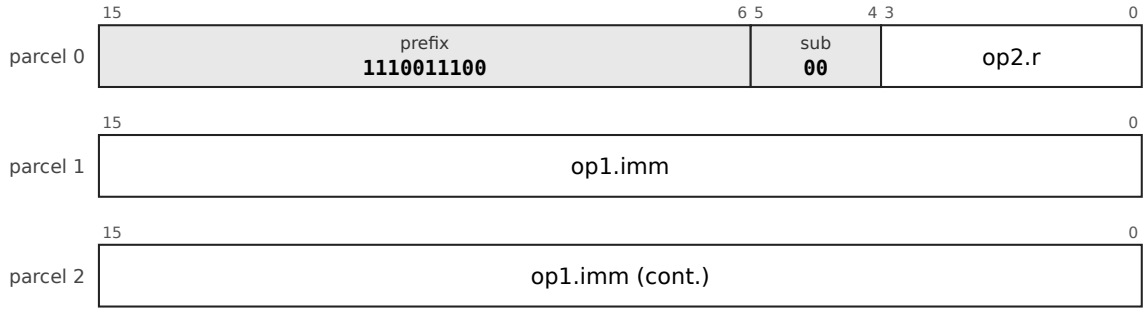
MOVL: LDF.PC32, prefix 11001001, sub-opcode 0001; operands pc32-u1, reg4; 6 bytes



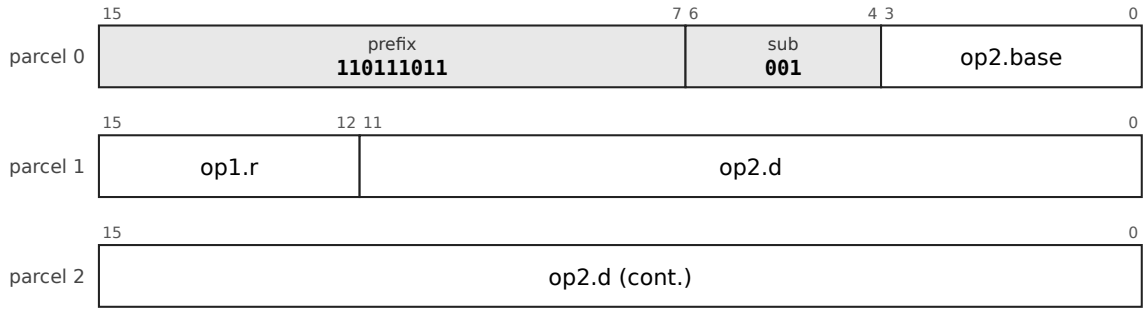
MOVL: LDF.A32, prefix 11001010, sub-opcode 0001; operands abs32, reg4; 6 bytes



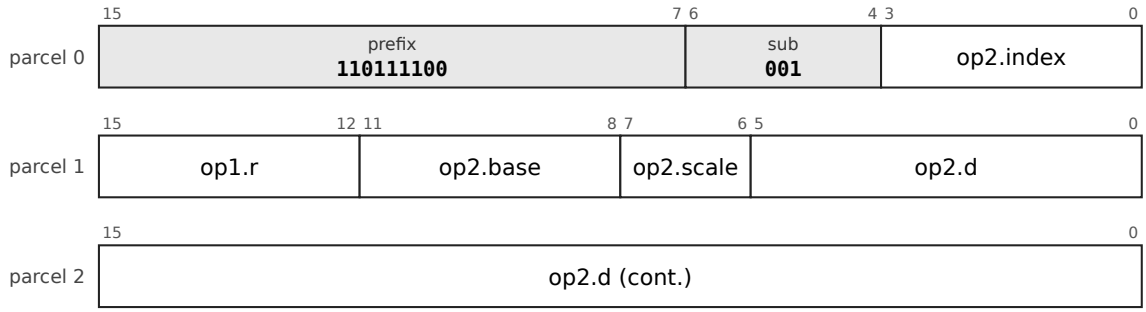
MOVL: LDFI.I32, prefix 1110011100, sub-opcode 00; operands imm32s, reg4; 6 bytes



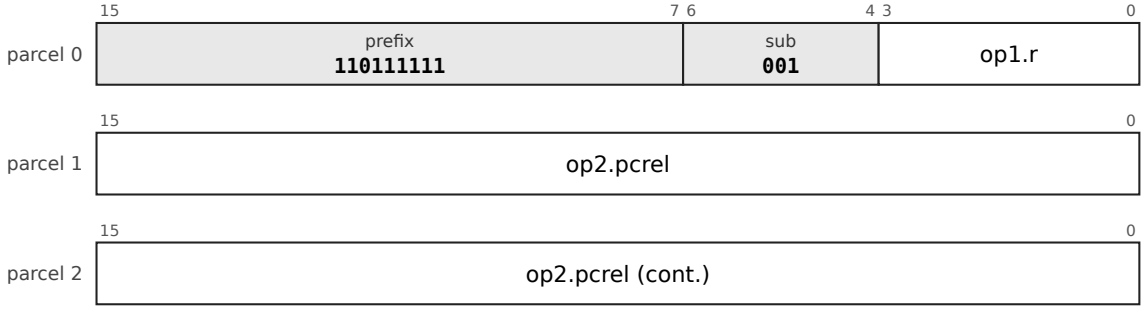
MOVL: STF.B28, prefix 110111011, sub-opcode 001; operands reg4, b4d28s-x1; 6 bytes



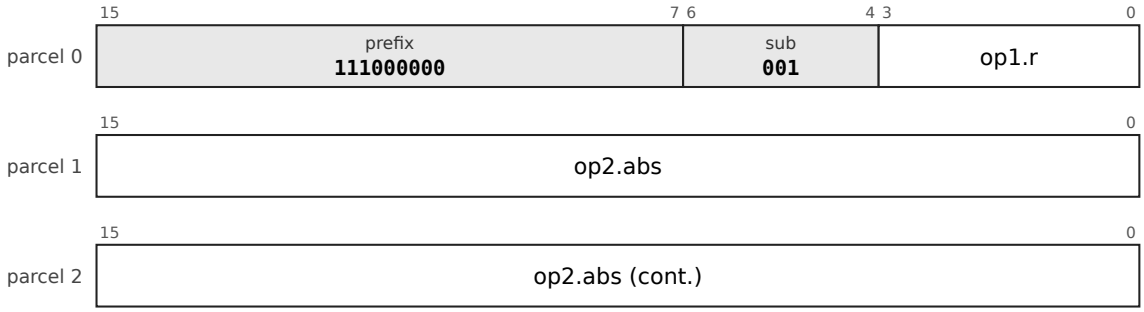
MOVL: STF.X22, prefix 110111100, sub-opcode 001; operands reg4, x4-4d22s; 6 bytes



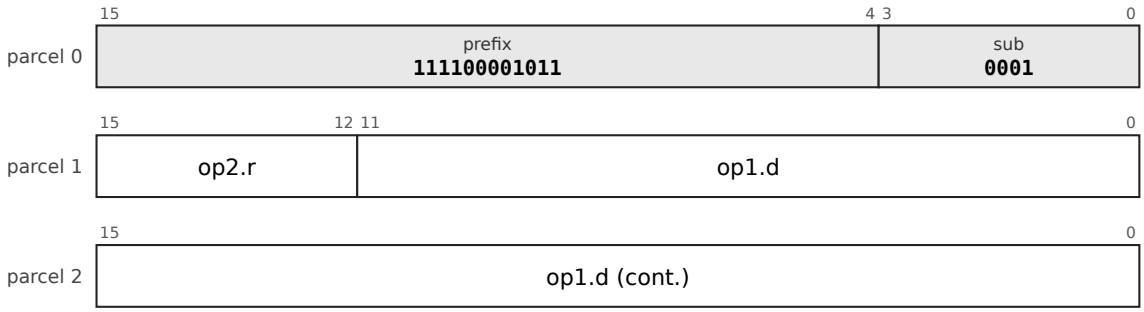
MOVL: STF.PC32, prefix 110111111, sub-opcode 001; operands reg4, pc32-u1; 6 bytes



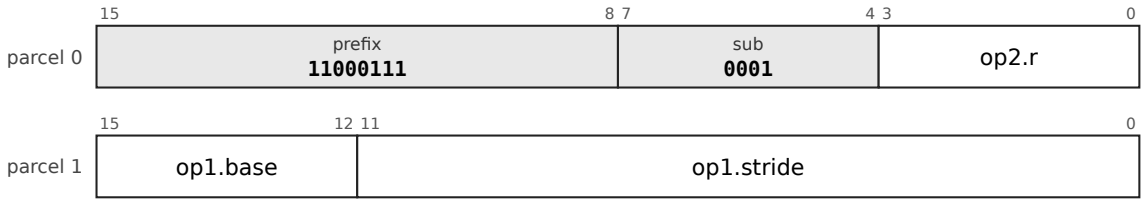
MOVL: STF.A32, prefix 111000000, sub-opcode 001; operands reg4, abs32; 6 bytes



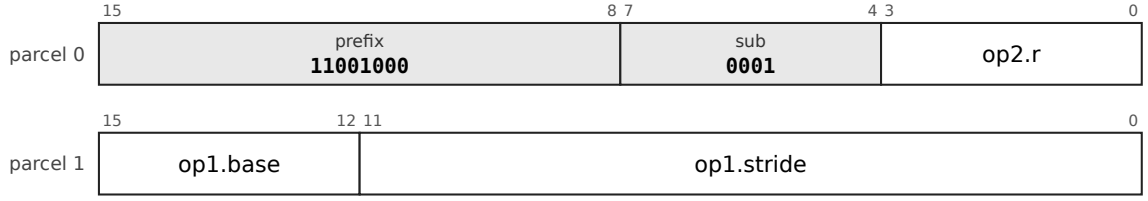
MOVL: DPF.D28, prefix 111100001011, sub-opcode 0001; operands dpd28s-x1, reg4; 6 bytes



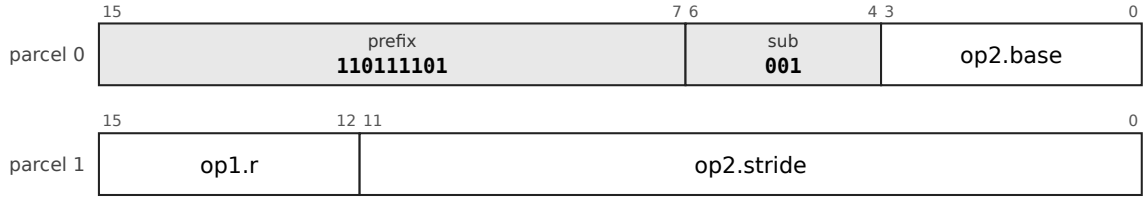
MOVL: LDF.PI12, prefix 11000111, sub-opcode 0001; operands pi4-s12, reg4; 4 bytes



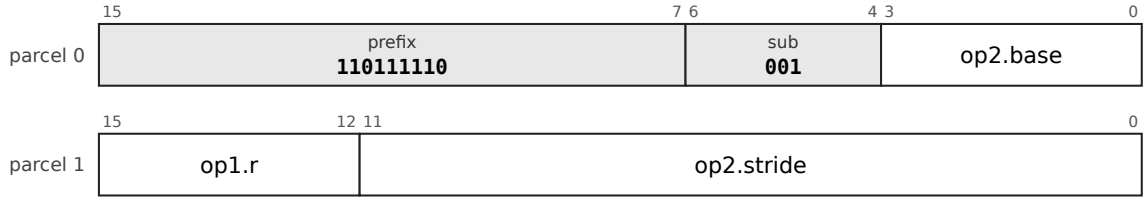
MOVL: LDF.PD12, prefix 11001000, sub-opcode 0001; operands pd4-s12, reg4; 4 bytes



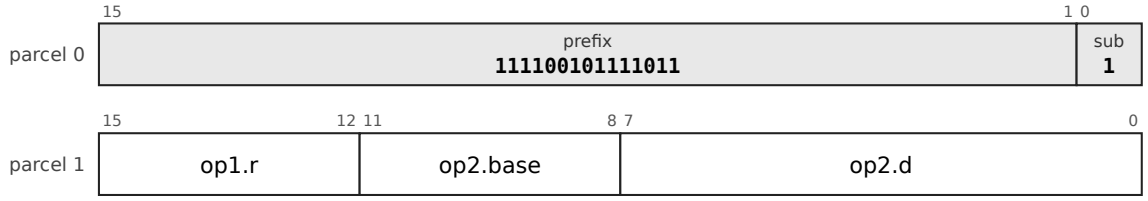
MOVL: STF.PI12, prefix 110111101, sub-opcode 001; operands reg4, pi4-s12; 4 bytes



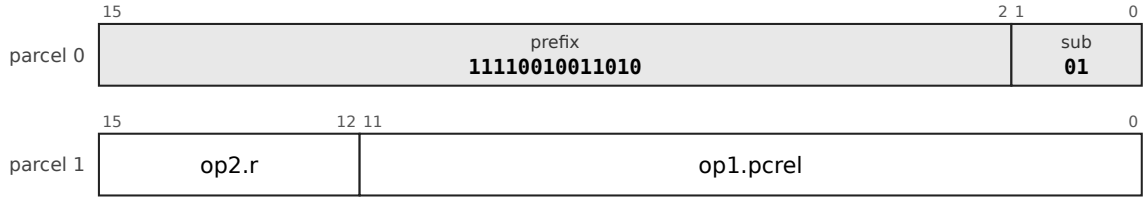
MOVL: STF.PD12, prefix 110111110, sub-opcode 001; operands reg4, pd4-s12; 4 bytes



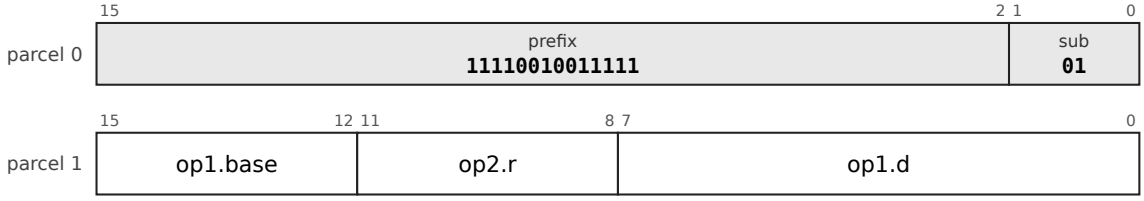
MOVL: STBs8/a, prefix 111100101111011, sub-opcode 1; operands reg4, b4d8s-xsz; 4 bytes



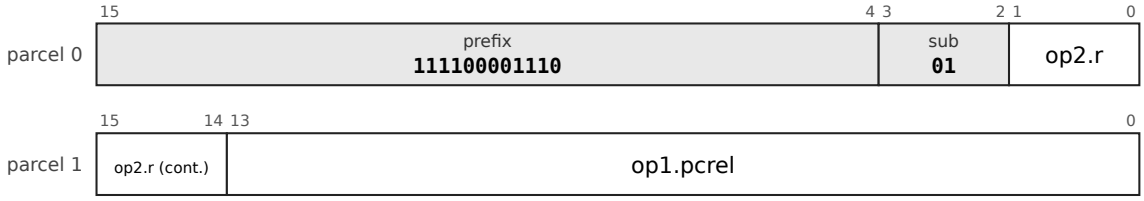
MOVL: LDPC12/a, prefix 11110010011010, sub-opcode 01; operands pc12-u1, reg4; 4 bytes



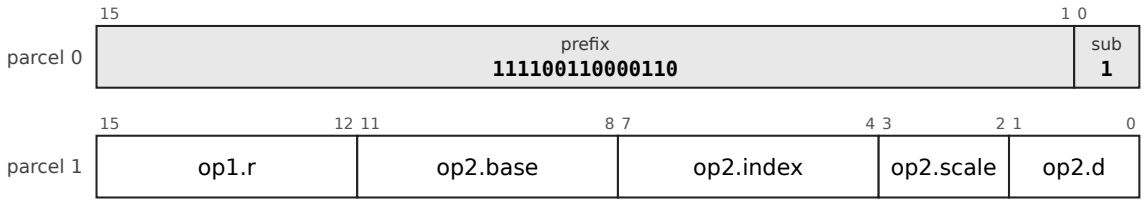
MOVL: LDBs8/a, prefix 11110010011111, sub-opcode 01; operands b4d8s-xsz, reg4; 4 bytes



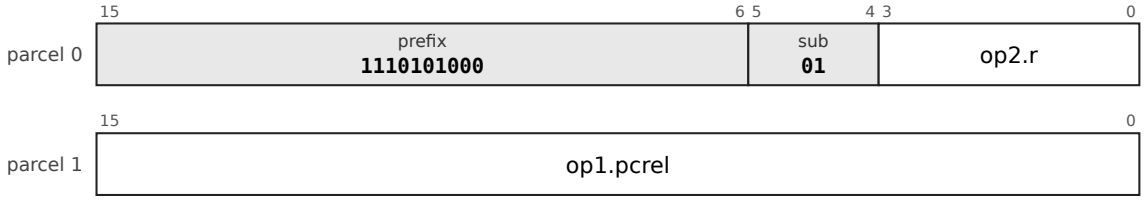
MOVL: LDPC14/a, prefix 111100001110, sub-opcode 01; operands pc14-u1, reg4; 4 bytes



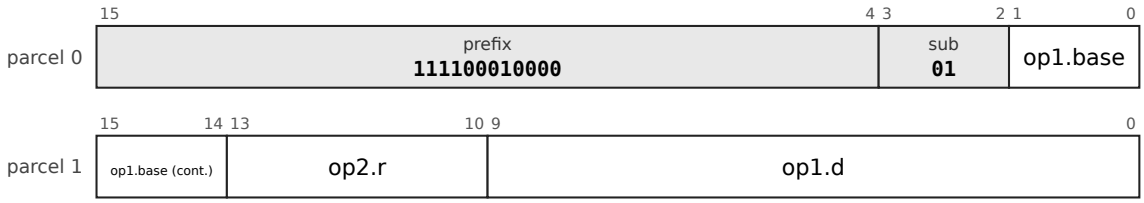
MOVL: STX2/a, prefix 111100110000110, sub-opcode 1; operands reg4, x4-4d2s; 4 bytes



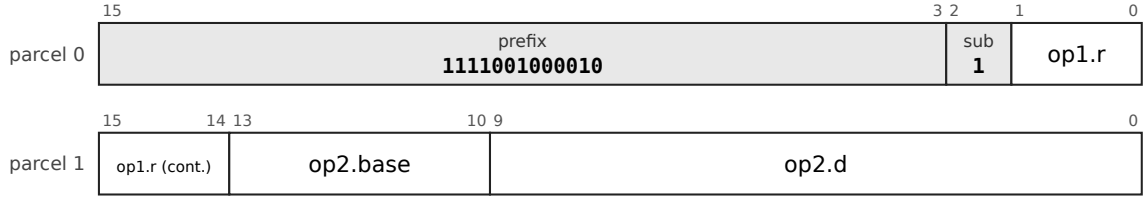
MOVL: LDPC16/a, prefix 1110101000, sub-opcode 01; operands pc16-u1, reg4; 4 bytes



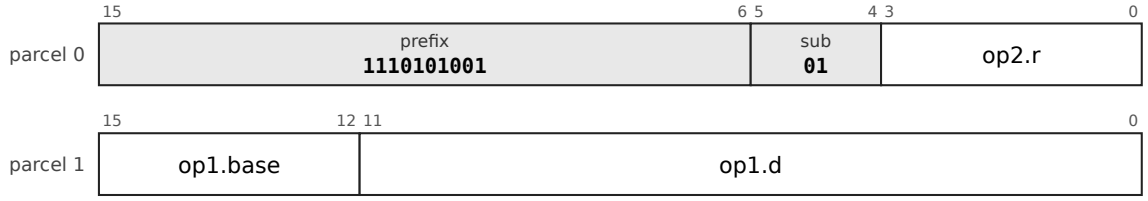
MOVL: LDBu10/a, prefix 111100010000, sub-opcode 01; operands b4d10u-xsz, reg4; 4 bytes



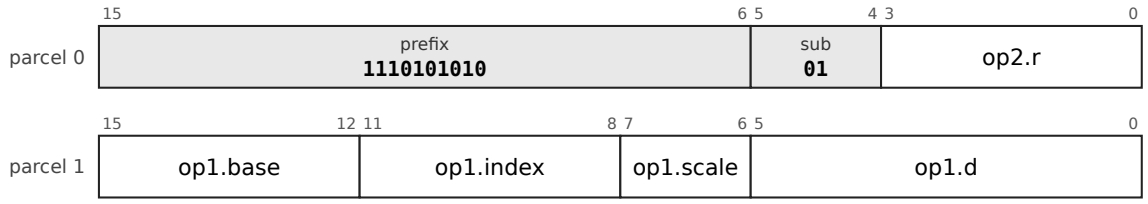
MOVL: STBu10/a, prefix 1111001000010, sub-opcode 1; operands reg4, b4d10u-xsz; 4 bytes



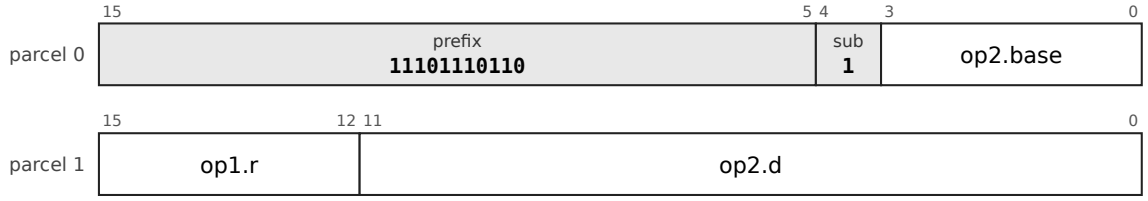
MOVL: LDBu12/a, prefix 1110101001, sub-opcode 01; operands b4d12u-xsz, reg4; 4 bytes



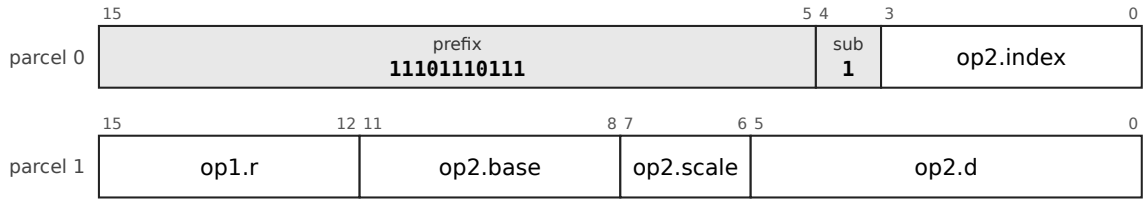
MOVL: LDX6/a, prefix 1110101010, sub-opcode 01; operands x4-4d6s, reg4; 4 bytes



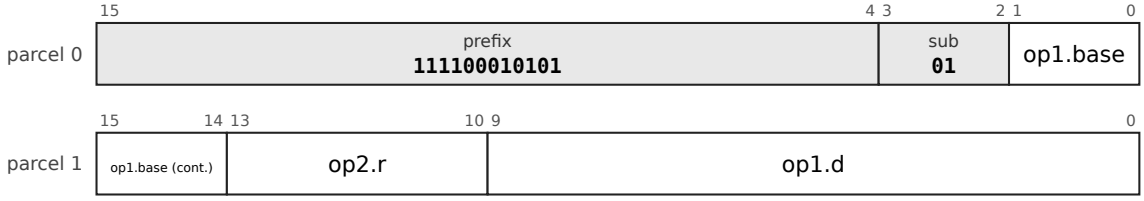
MOVL: STBu12/a, prefix 11101110110, sub-opcode 1; operands reg4, b4d12u-xsz; 4 bytes



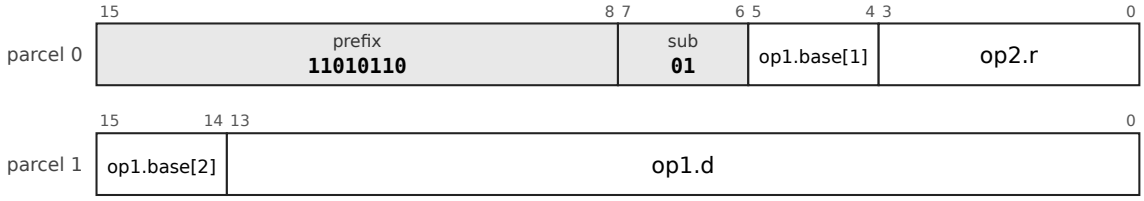
MOVL: STX6/a, prefix 11101110111, sub-opcode 1; operands reg4, x4-4d6s; 4 bytes



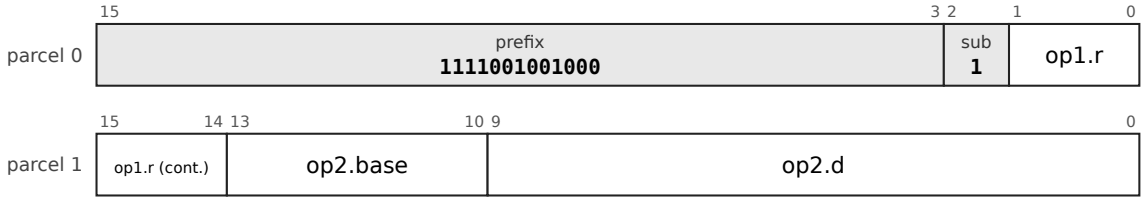
MOVL: LDBs10/a, prefix 111100010101, sub-opcode 01; operands b4d10s-xsz, reg4; 4 bytes



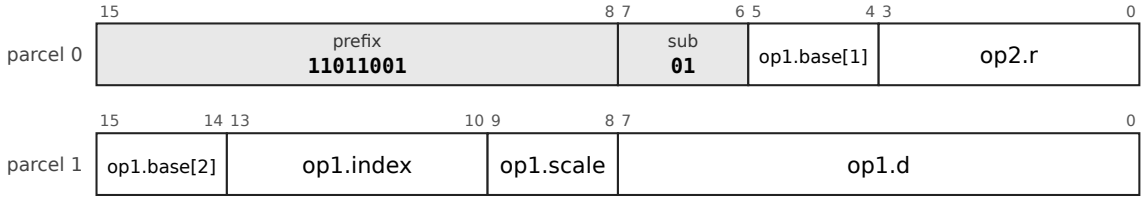
MOVL: LDBs14/a, prefix 11010110, sub-opcode 01; operands b4d14s-xsz, reg4; 4 bytes



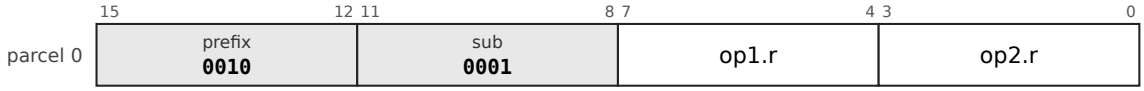
MOVL: STBs10/a, prefix 1111001001000, sub-opcode 1; operands reg4, b4d10s-xsz; 4 bytes



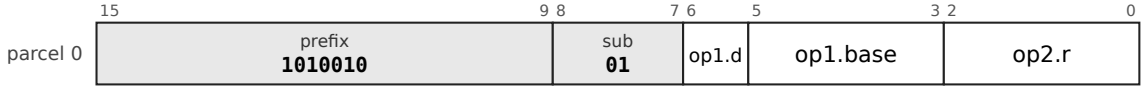
MOVL: LDX8/a, prefix 11011001, sub-opcode 01; operands x4-4d8s, reg4; 4 bytes



MOVL: RRF, prefix 0010, sub-opcode 0001; operands reg4, reg4; 2 bytes



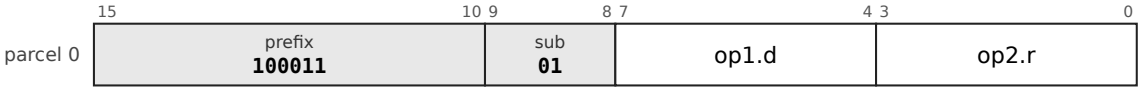
MOVL: LDHHH1/a, prefix 1010010, sub-opcode 01; operands b3d1u-xsz, reg3; 2 bytes



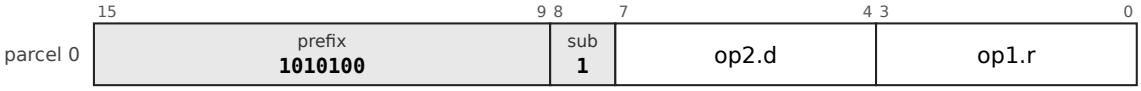
MOVL: STHHH1/a, prefix 11001111, sub-opcode 1; operands reg3, b3d1u-xsz; 2 bytes



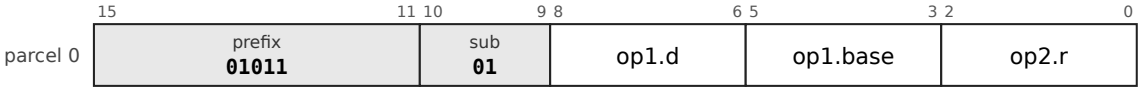
MOVL: LDSP4/a, prefix 100011, sub-opcode 01; operands spd4u-xsz, reg4; 2 bytes



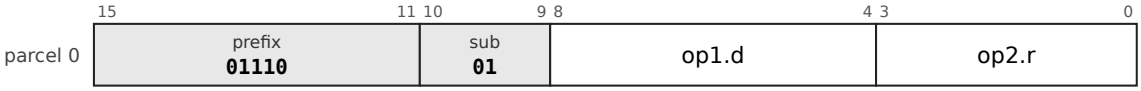
MOVL: STSP4/a, prefix 1010100, sub-opcode 1; operands reg4, spd4u-xsz; 2 bytes



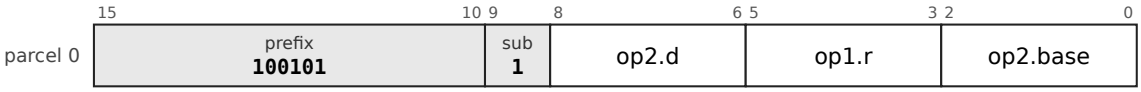
MOVL: LDHHH3/a, prefix 01011, sub-opcode 01; operands b3d3u-xsz, reg3; 2 bytes



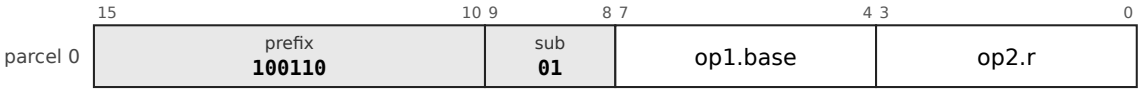
MOVL: LDSP5/a, prefix 01110, sub-opcode 01; operands spd5u-xsz, reg4; 2 bytes



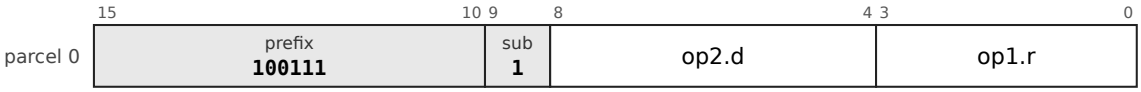
MOVL: STHHH3/a, prefix 100101, sub-opcode 1; operands reg3, b3d3u-xsz; 2 bytes



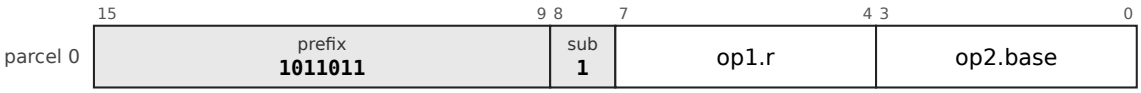
MOVL: LD0/a, prefix 100110, sub-opcode 01; operands b4d0s-xsz, reg4; 2 bytes



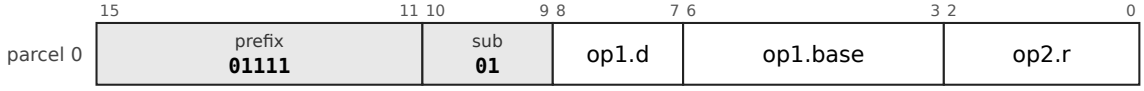
MOVL: STSP5/a, prefix 100111, sub-opcode 1; operands reg4, spd5u-xsz; 2 bytes



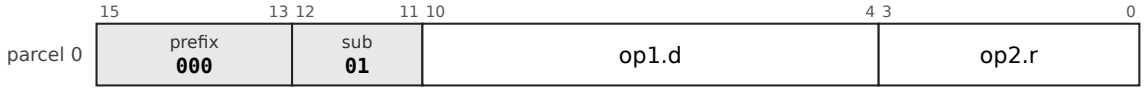
MOVL: ST0/a, prefix 1011011, sub-opcode 1; operands reg4, b4d0s-xsz; 2 bytes



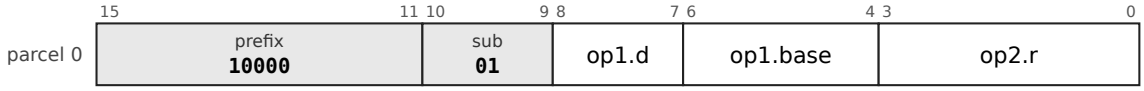
MOVL: LDHRH2/a, prefix 01111, sub-opcode 01; operands b4d2u-xsz, reg3; 2 bytes



MOVL: LDSP7/a, prefix 000, sub-opcode 01; operands spd7u-xsz, reg4; 2 bytes



MOVL: LDHHR2/a, prefix 10000, sub-opcode 01; operands b3d2u-xsz, reg4; 2 bytes

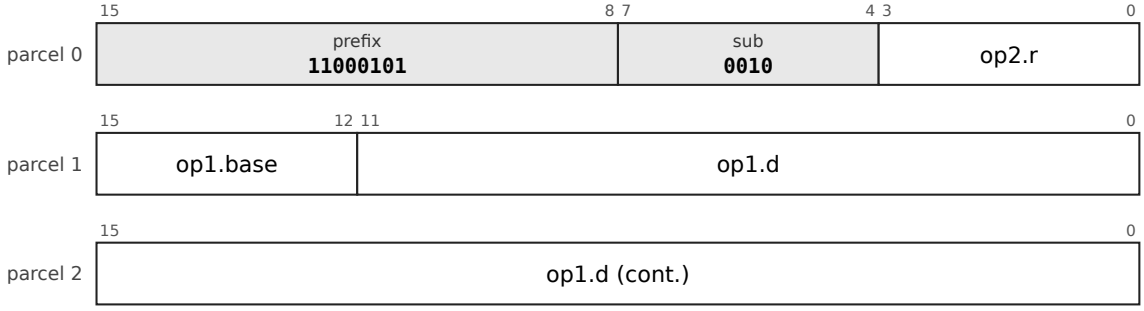



```
MOVL, MOVW, MOVB likewise
```

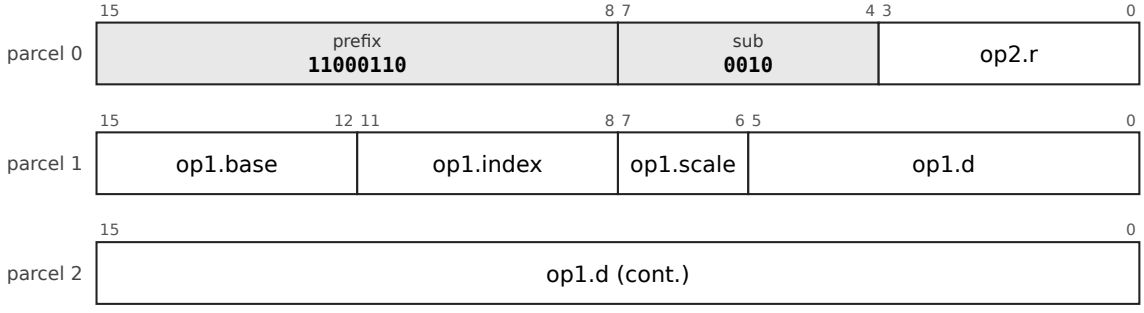
Operation, flags and exceptions as for the entry MOV above; 44 layout forms, each a chart of its own (above).

MOVW

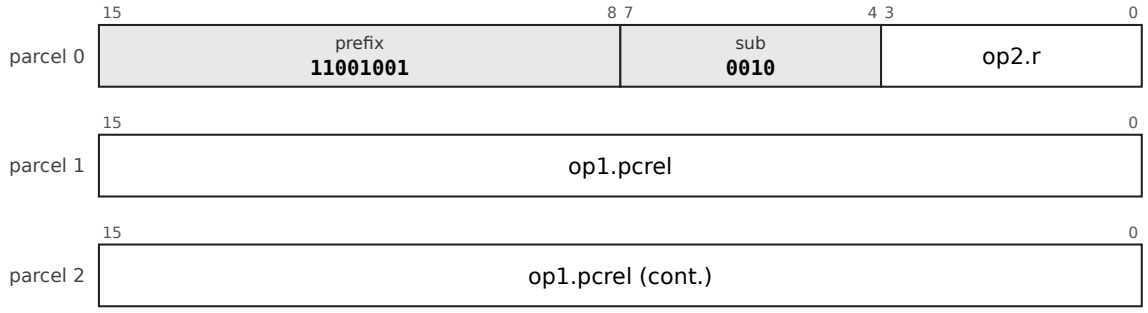
MOVW: LDF.B28, prefix 11000101, sub-opcode 0010; operands b4d28s-x1, reg4; 6 bytes



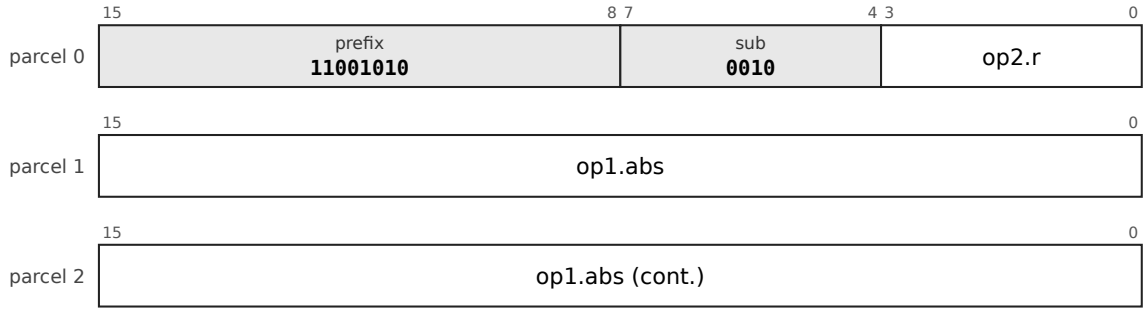
MOVW: LDF.X22, prefix 11000110, sub-opcode 0010; operands x4-4d22s, reg4; 6 bytes



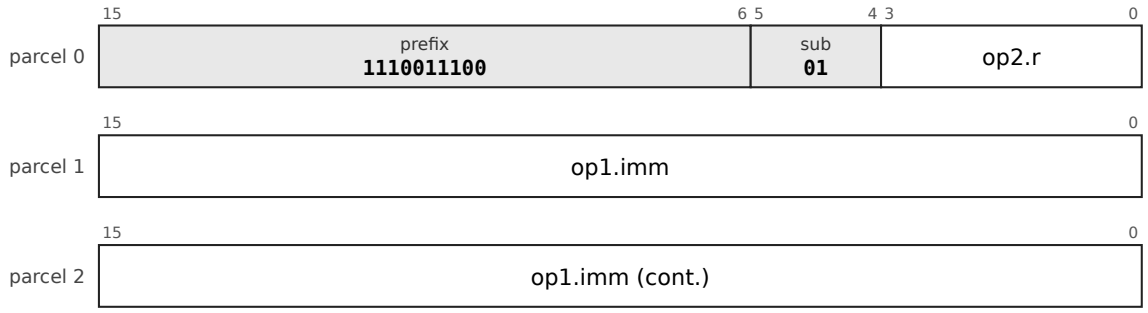
MOVW: LDF.PC32, prefix 11001001, sub-opcode 0010; operands pc32-u1, reg4; 6 bytes



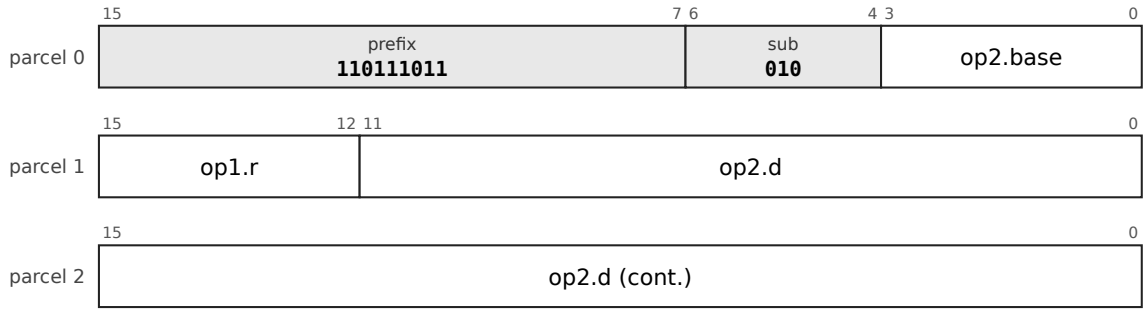
MOVW: LDF.A32, prefix 11001010, sub-opcode 0010; operands abs32, reg4; 6 bytes



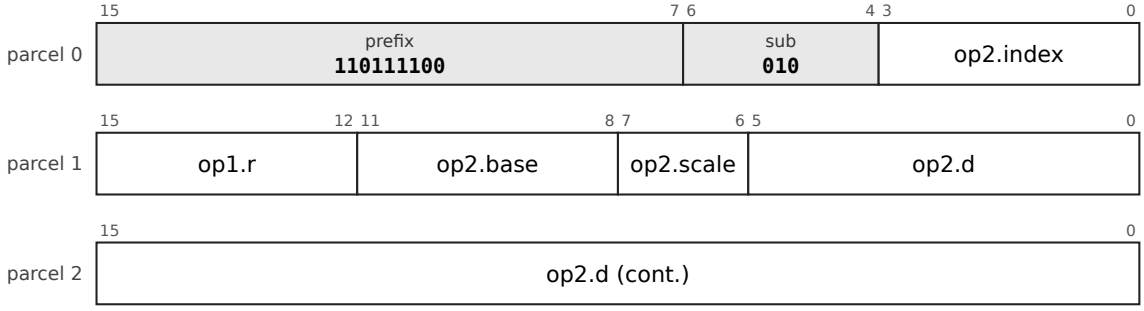
MOVW: LDFI.I32, prefix 1110011100, sub-opcode 01; operands imm32s, reg4; 6 bytes



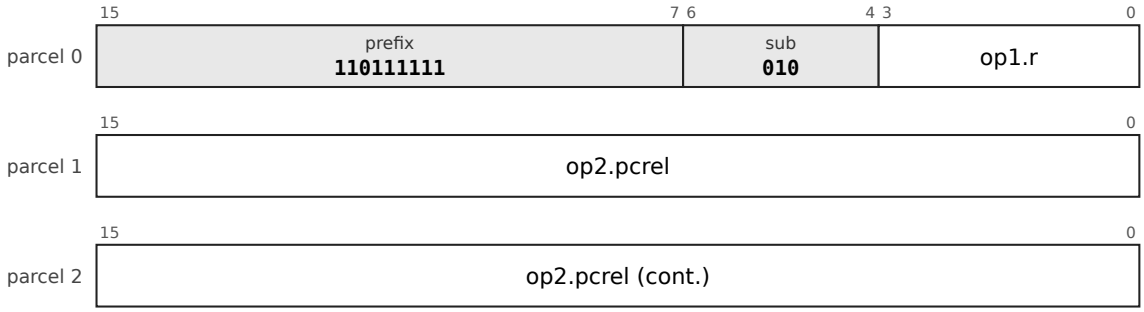
MOVW: STF.B28, prefix 110111011, sub-opcode 010; operands reg4, b4d28s-x1; 6 bytes



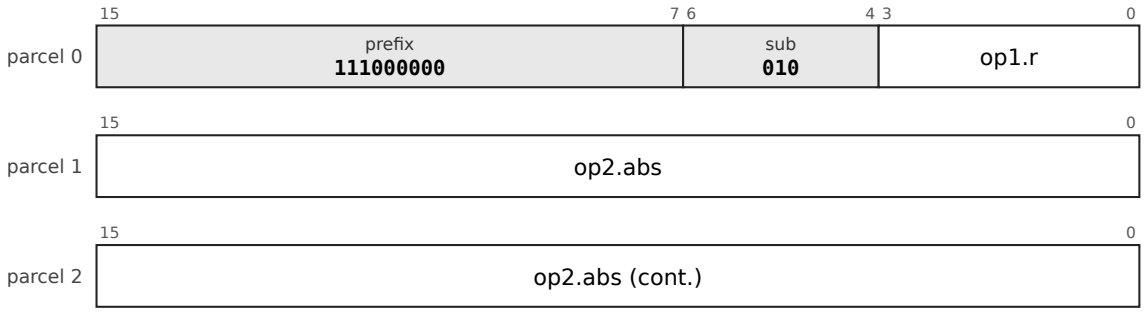
MOVW: STF.X22, prefix 110111100, sub-opcode 010; operands reg4, x4-4d22s; 6 bytes



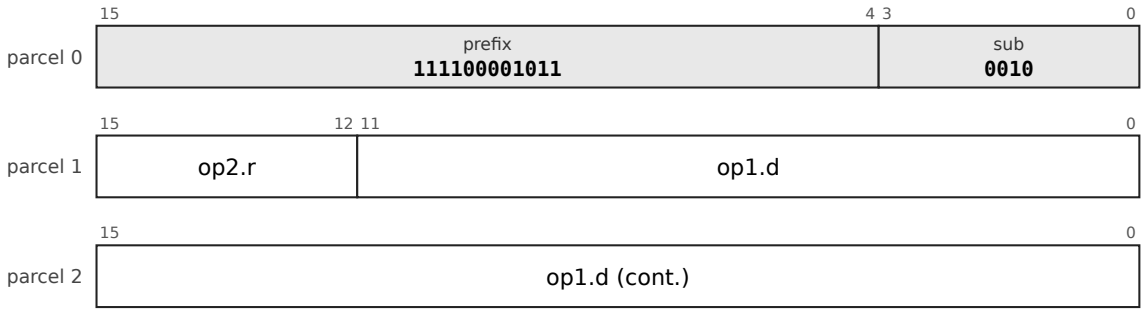
MOVW: STF.PC32, prefix 110111111, sub-opcode 010; operands reg4, pc32-u1; 6 bytes



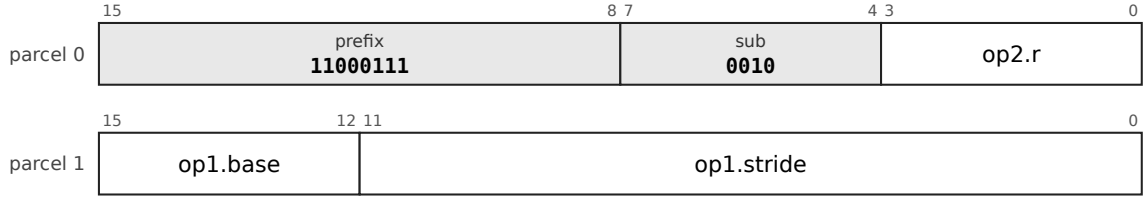
MOVW: STF.A32, prefix 111000000, sub-opcode 010; operands reg4, abs32; 6 bytes



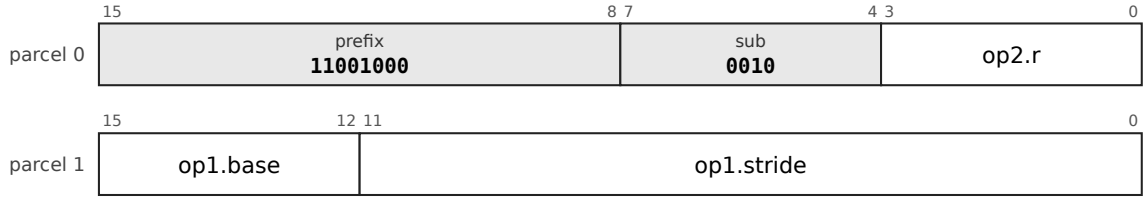
MOVW: DPF.D28, prefix 111100001011, sub-opcode 0010; operands dpd28s-x1, reg4; 6 bytes



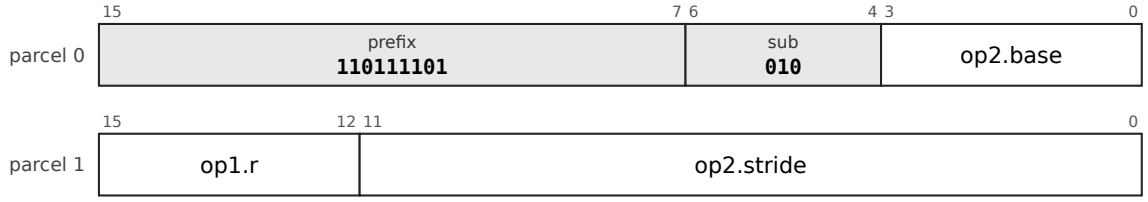
MOVW: LDF.PI12, prefix 11000111, sub-opcode 0010; operands pi4-s12, reg4; 4 bytes



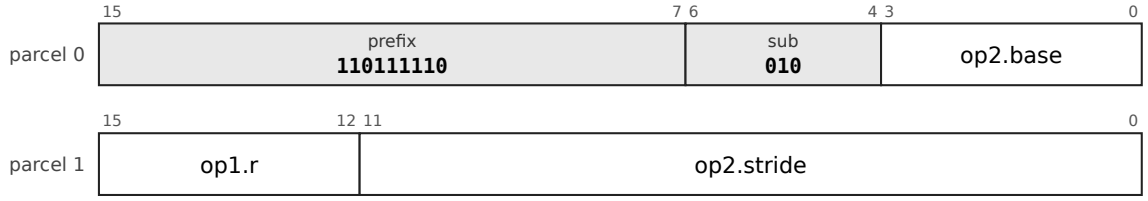
MOVW: LDF.PD12, prefix 11001000, sub-opcode 0010; operands pd4-s12, reg4; 4 bytes



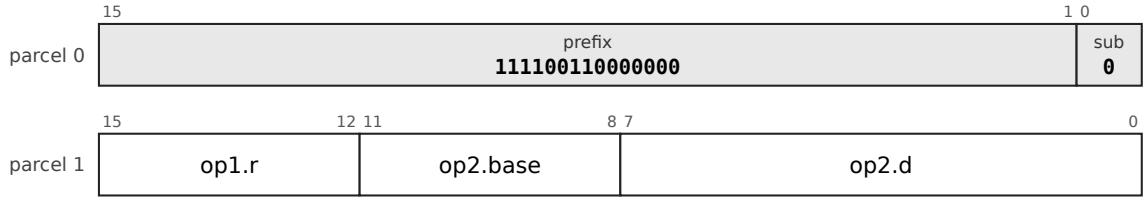
MOVW: STF.PI12, prefix 110111101, sub-opcode 010; operands reg4, pi4-s12; 4 bytes



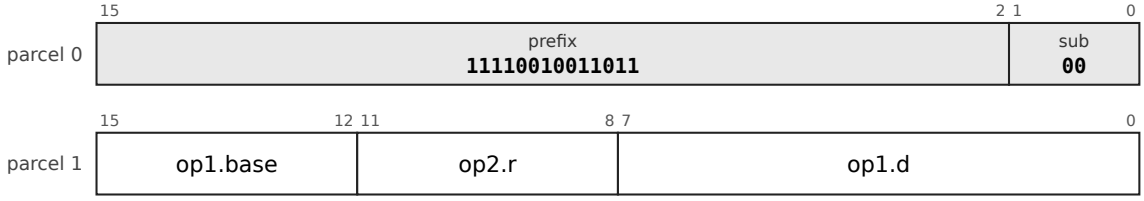
MOVW: STF.PD12, prefix 110111110, sub-opcode 010; operands reg4, pd4-s12; 4 bytes



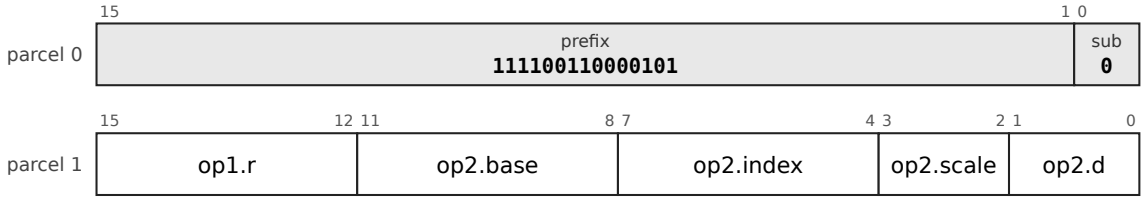
MOVW: STBu8/b, prefix 111100110000000, sub-opcode 0; operands reg4, b4d8u-xsz; 4 bytes



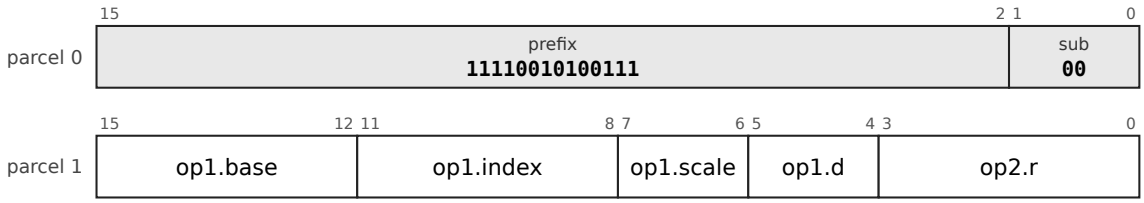
MOVW: LDBu8/b, prefix 11110010011011, sub-opcode 00; operands b4d8u-xsz, reg4; 4 bytes



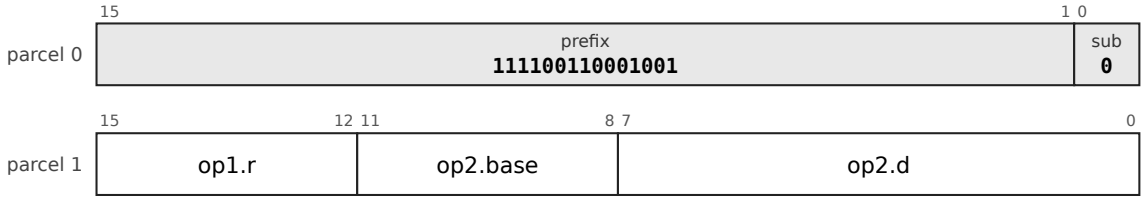
MOVW: STX2/b, prefix 111100110000101, sub-opcode 0; operands reg4, x4-4d2s; 4 bytes



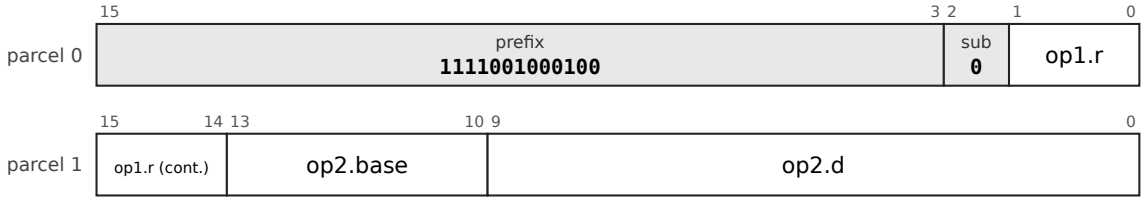
MOVW: LDX2/b, prefix 11110010100111, sub-opcode 00; operands x4-4d2s, reg4; 4 bytes



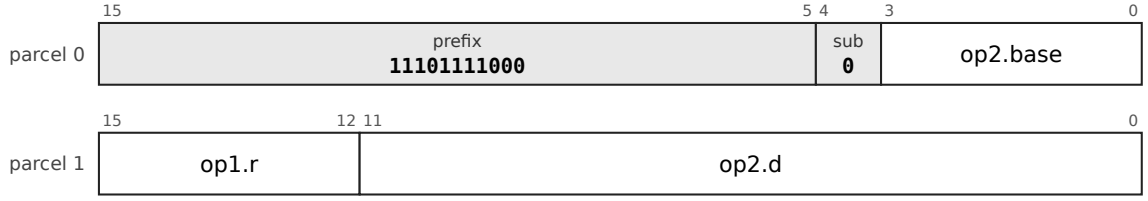
MOVW: STBs8/b, prefix 111100110001001, sub-opcode 0; operands reg4, b4d8s-xsz; 4 bytes



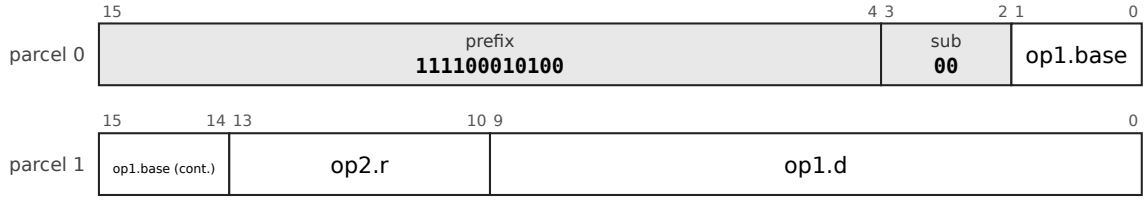
MOVW: STBu10/b, prefix 1111001000100, sub-opcode 0; operands reg4, b4d10u-xsz; 4 bytes



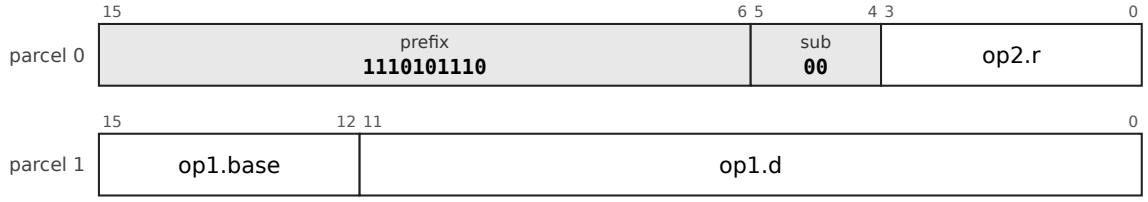
MOVW: STBu12/b, prefix 11101111000, sub-opcode 0; operands reg4, b4d12u-xsz; 4 bytes



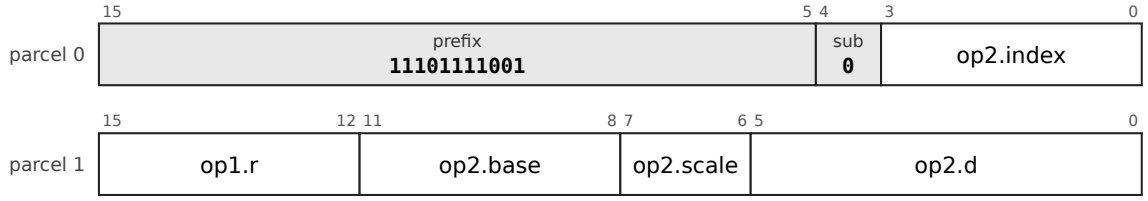
MOVW: LDBu10/b, prefix 111100010100, sub-opcode 00; operands b4d10u-xsz, reg4; 4 bytes



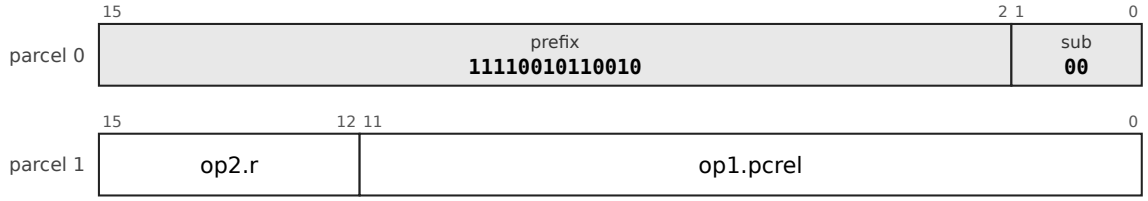
MOVW: LDBu12/b, prefix 1110101110, sub-opcode 00; operands b4d12u-xsz, reg4; 4 bytes



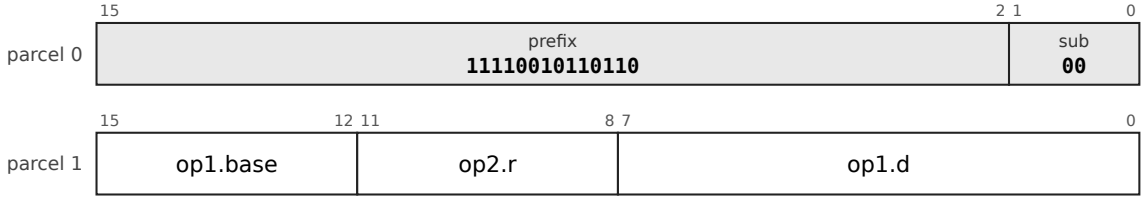
MOVW: STX6/b, prefix 11101111001, sub-opcode 0; operands reg4, x4-4d6s; 4 bytes



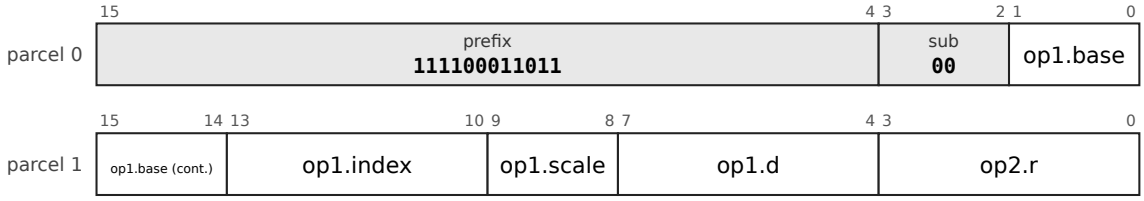
MOVW: LDPC12/b, prefix 11110010110010, sub-opcode 00; operands pc12-u1, reg4; 4 bytes



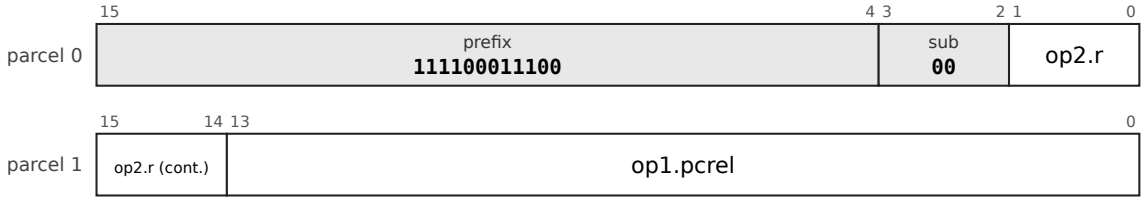
MOVW: LDBs8/b, prefix 11110010110110, sub-opcode 00; operands b4d8s-xsz, reg4; 4 bytes



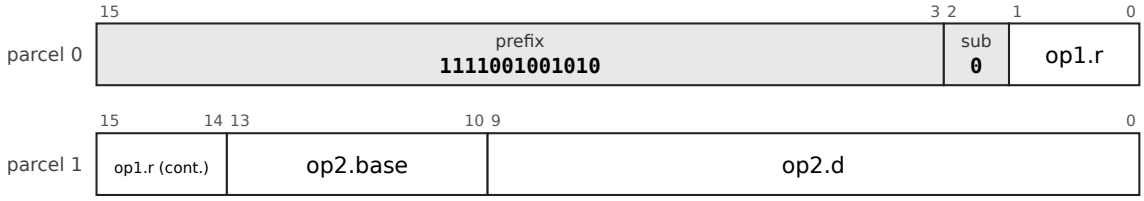
MOVW: LDX4/b, prefix 111100011011, sub-opcode 00; operands x4-4d4s, reg4; 4 bytes



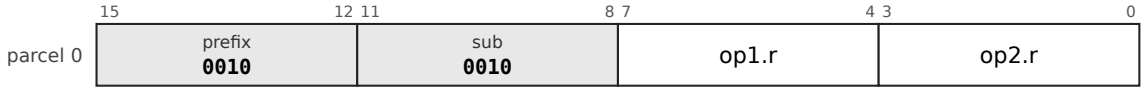
MOVW: LDPC14/b, prefix 111100011100, sub-opcode 00; operands pc14-u1, reg4; 4 bytes



MOVW: STBs10/b, prefix 1111001001010, sub-opcode 0; operands reg4, b4d10s-xsz; 4 bytes



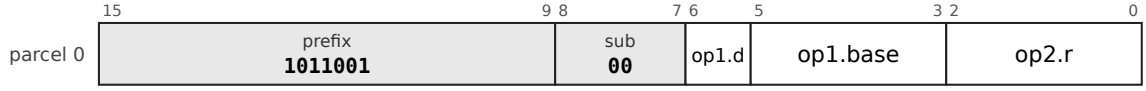
MOVW: RRF, prefix 0010, sub-opcode 0010; operands reg4, reg4; 2 bytes



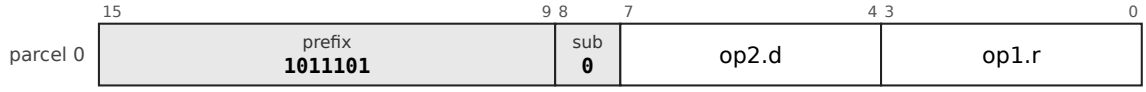
MOVW: STHHH1/b, prefix 11010011, sub-opcode 0; operands reg3, b3d1u-xsz; 2 bytes



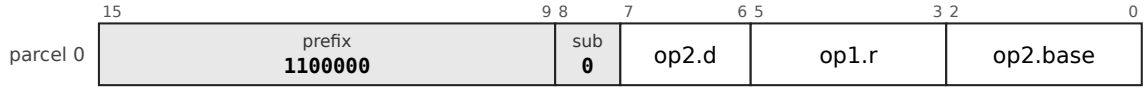
MOVW: LDHHH1/b, prefix 1011001, sub-opcode 00; operands b3d1u-xsz, reg3; 2 bytes



MOVW: STSP4/b, prefix 1011101, sub-opcode 0; operands reg4, spd4u-xsz; 2 bytes



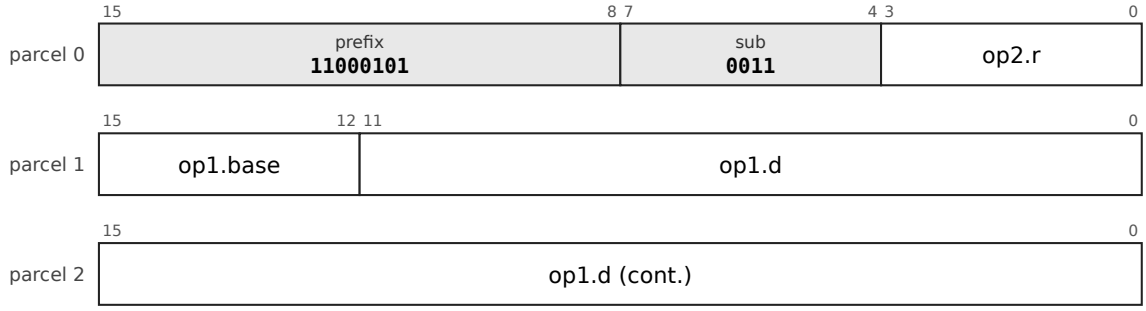
MOVW: STHHH2/b, prefix 1100000, sub-opcode 0; operands reg3, b3d2u-xsz; 2 bytes



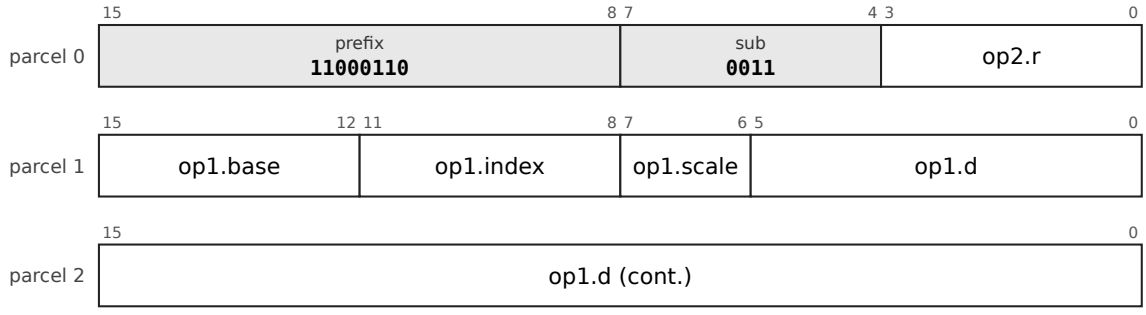
Operation, flags and exceptions as for the entry MOV above; 34 layout forms, each a chart of its own (above).

MOVB

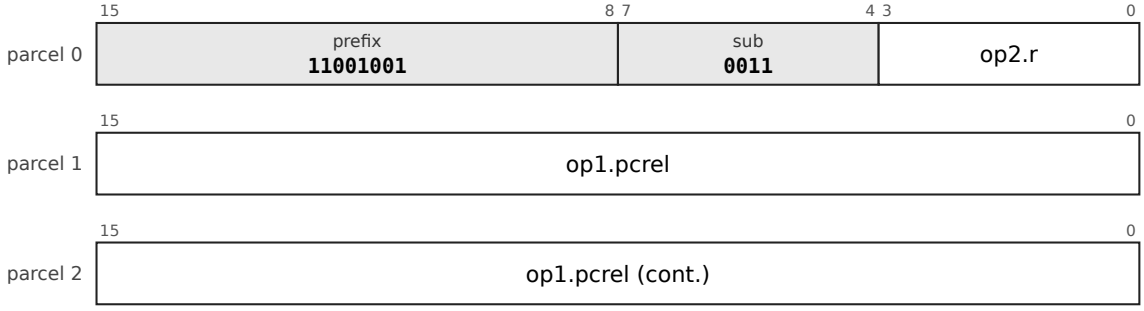
MOVB: LDF.B28, prefix 11000101, sub-opcode 0011; operands b4d28s-x1, reg4; 6 bytes



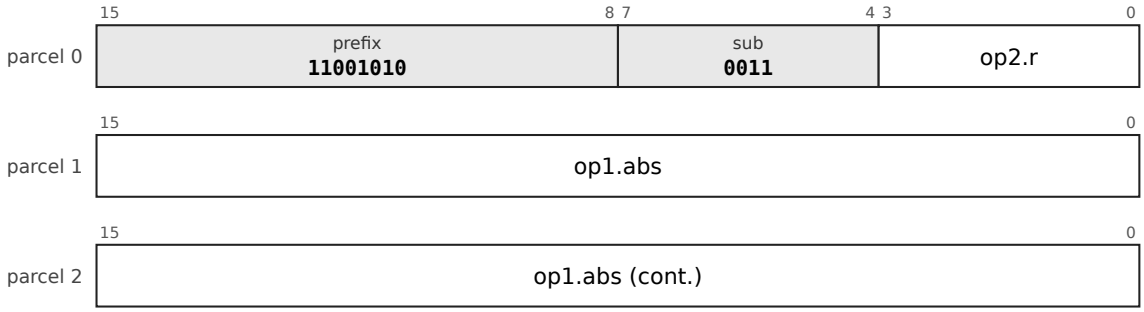
MOVB: LDF.X22, prefix 11000110, sub-opcode 0011; operands x4-4d22s, reg4; 6 bytes



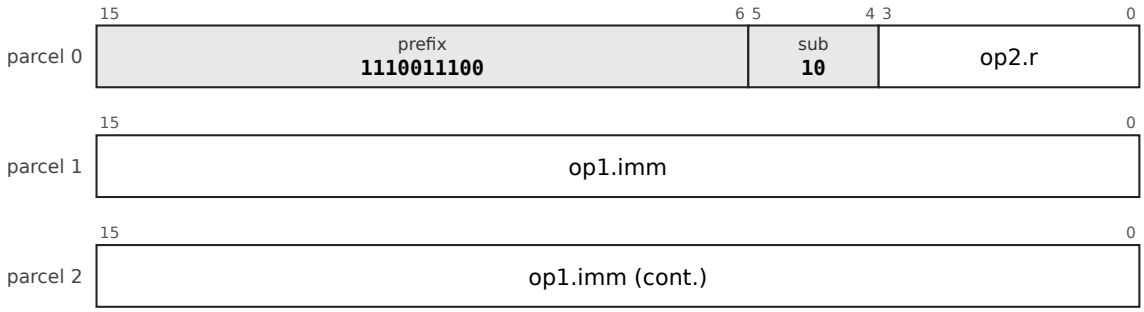
MOVB: LDF.PC32, prefix 11001001, sub-opcode 0011; operands pc32-u1, reg4; 6 bytes



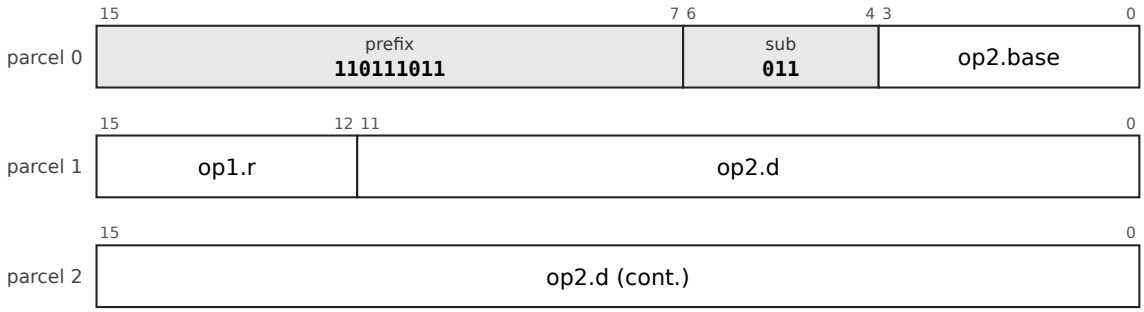
MOVB: LDF.A32, prefix 11001010, sub-opcode 0011; operands abs32, reg4; 6 bytes



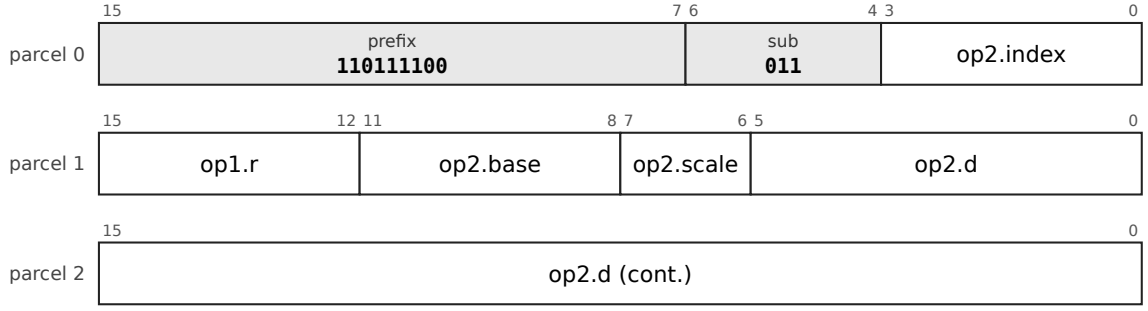
MOVB: LDFI.I32, prefix 1110011100, sub-opcode 10; operands imm32s, reg4; 6 bytes



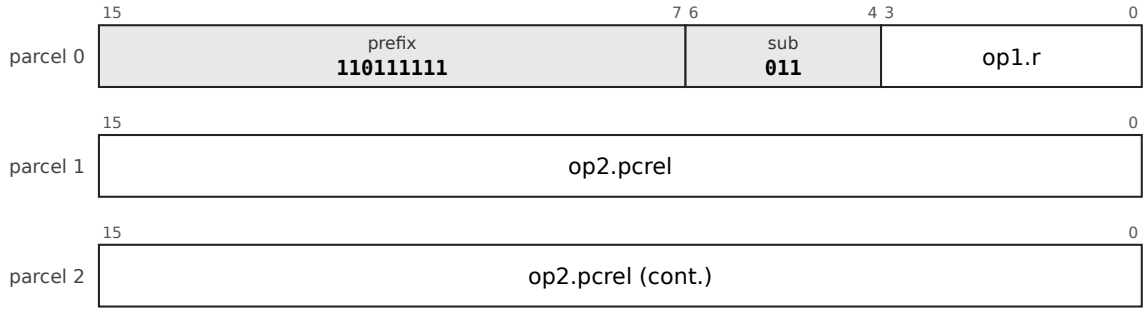
MOVB: STF.B28, prefix 110111011, sub-opcode 011; operands reg4, b4d28s-x1; 6 bytes



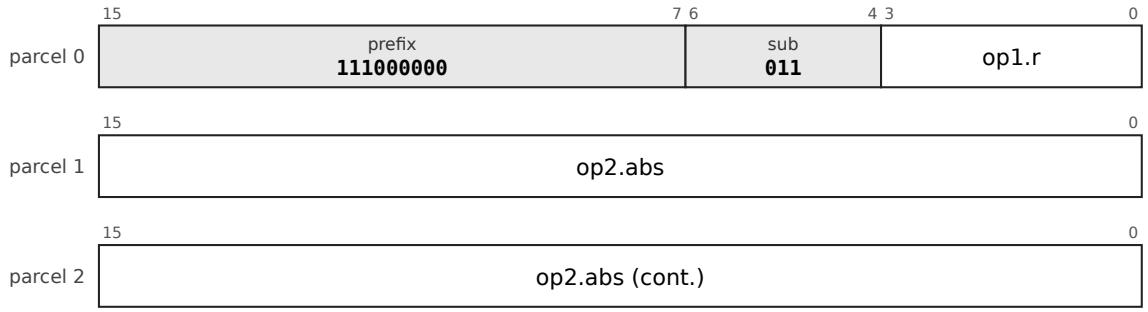
MOVB: STF.X22, prefix 110111100, sub-opcode 011; operands reg4, x4-4d22s; 6 bytes



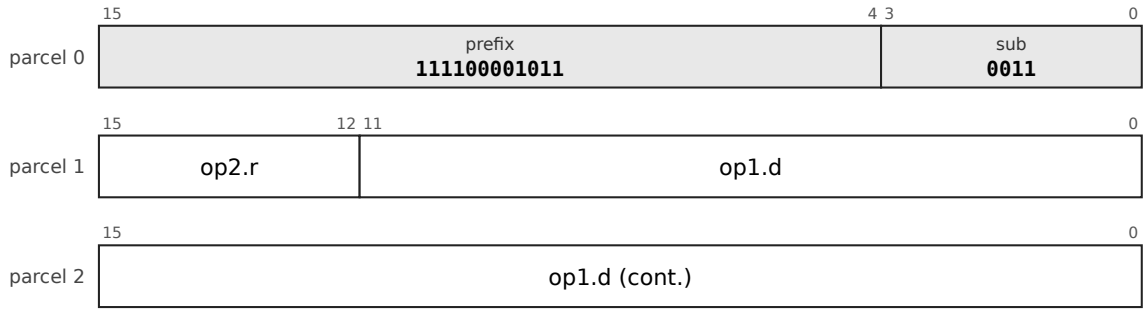
MOVB: STF.PC32, prefix 110111111, sub-opcode 011; operands reg4, pc32-u1; 6 bytes



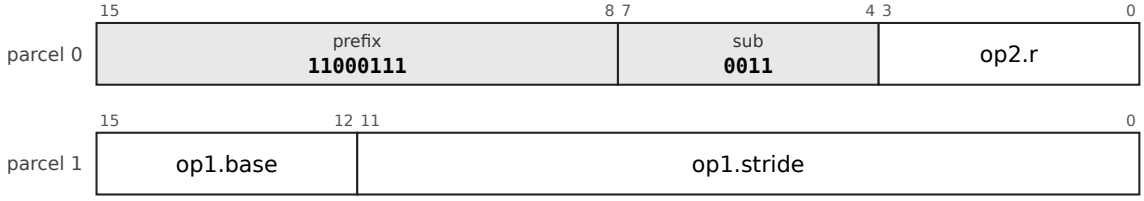
MOVB: STF.A32, prefix 111000000, sub-opcode 011; operands reg4, abs32; 6 bytes



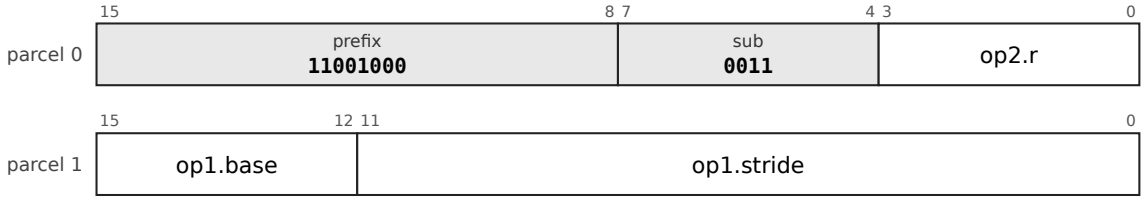
MOVB: DPF.D28, prefix 111100001011, sub-opcode 0011; operands dpd28s-x1, reg4; 6 bytes



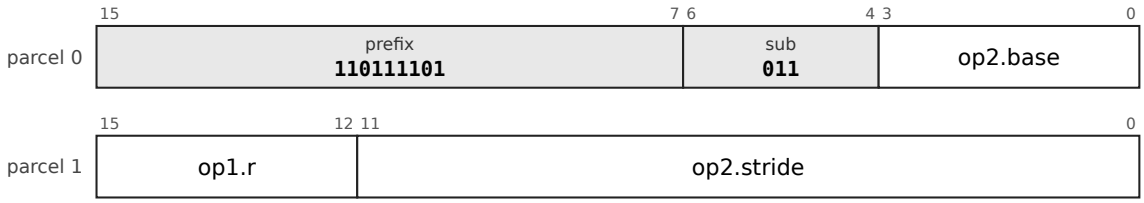
MOVB: LDF.PI12, prefix 11000111, sub-opcode 0011; operands pi4-s12, reg4; 4 bytes



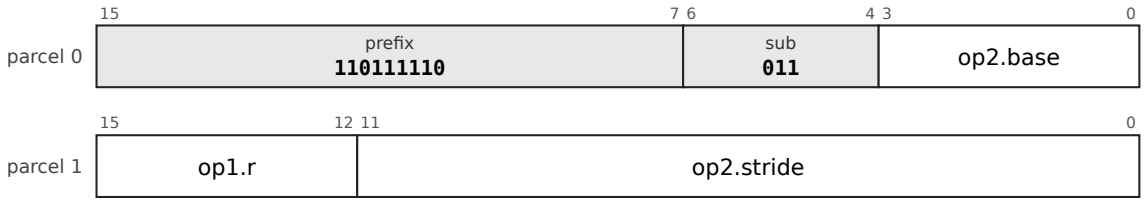
MOVB: LDF.PD12, prefix 11001000, sub-opcode 0011; operands pd4-s12, reg4; 4 bytes



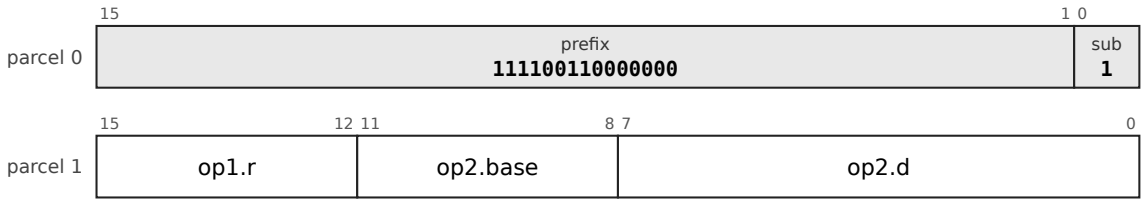
MOVB: STF.PI12, prefix 110111101, sub-opcode 011; operands reg4, pi4-s12; 4 bytes



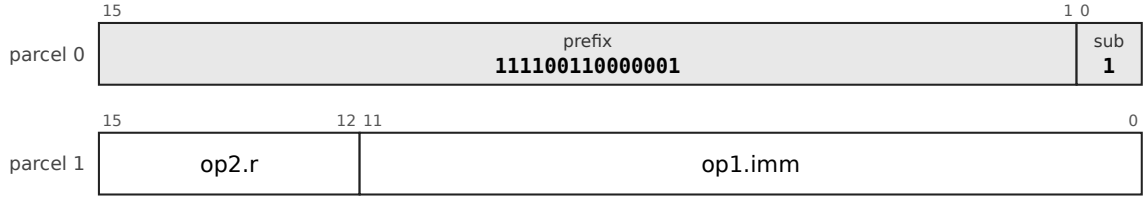
MOVB: STF.PD12, prefix 110111110, sub-opcode 011; operands reg4, pd4-s12; 4 bytes



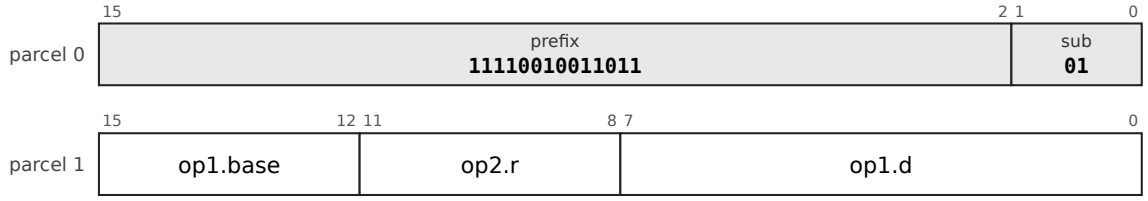
MOVB: STBu8/b, prefix 111100110000000, sub-opcode 1; operands reg4, b4d8u-xsz; 4 bytes



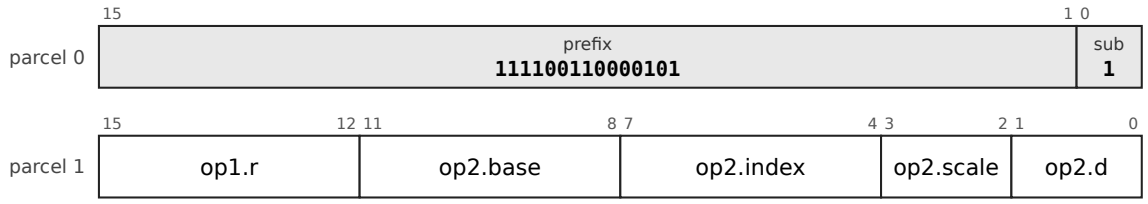
MOVB: LDI12/d, prefix 111100110000001, sub-opcode 1; operands imm12s, reg4; 4 bytes



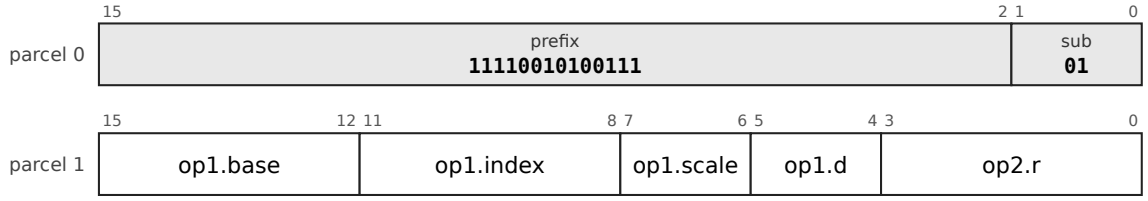
MOVB: LDBu8/b, prefix 11110010011011, sub-opcode 01; operands b4d8u-xsz, reg4; 4 bytes



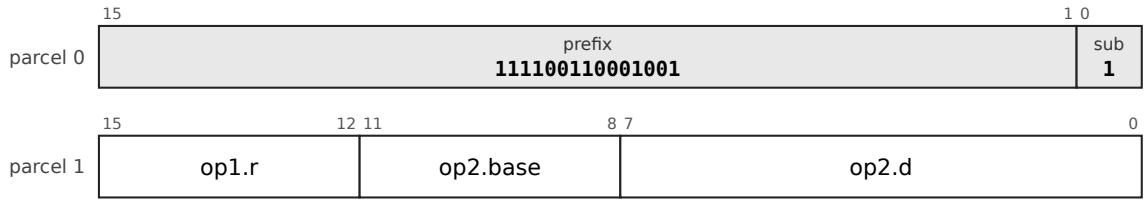
MOVB: STX2/b, prefix 111100110000101, sub-opcode 1; operands reg4, x4-4d2s; 4 bytes



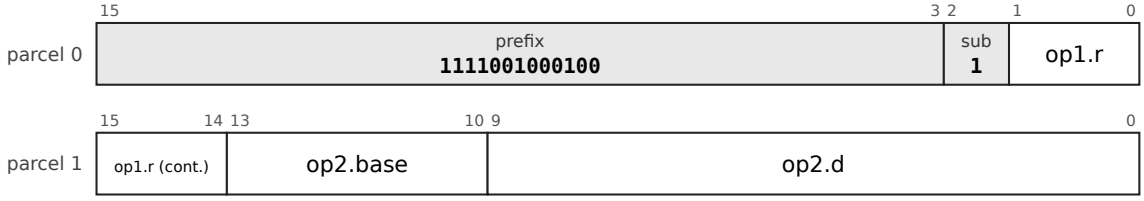
MOVB: LDX2/b, prefix 11110010100111, sub-opcode 01; operands x4-4d2s, reg4; 4 bytes



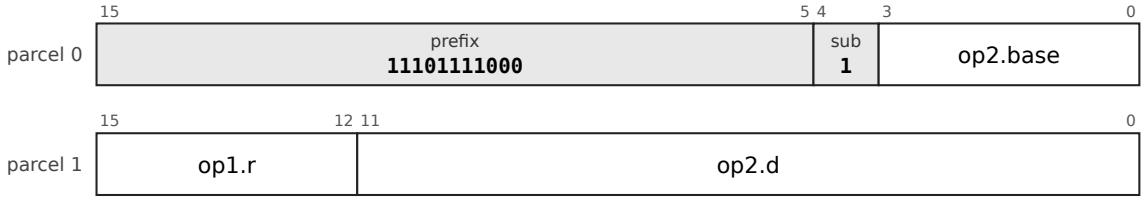
MOVB: STBs8/b, prefix 111100110001001, sub-opcode 1; operands reg4, b4d8s-xsz; 4 bytes



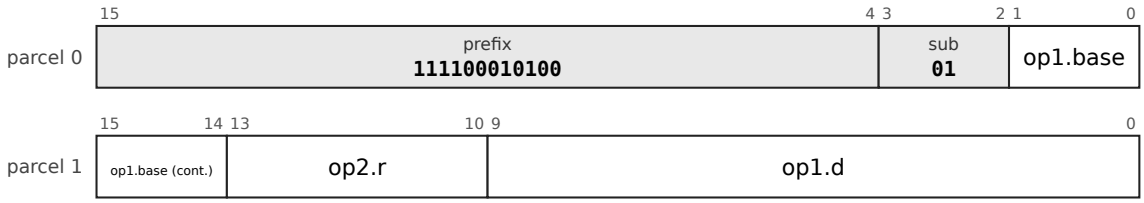
MOVB: STBu10/b, prefix 1111001000100, sub-opcode 1; operands reg4, b4d10u-xsz; 4 bytes



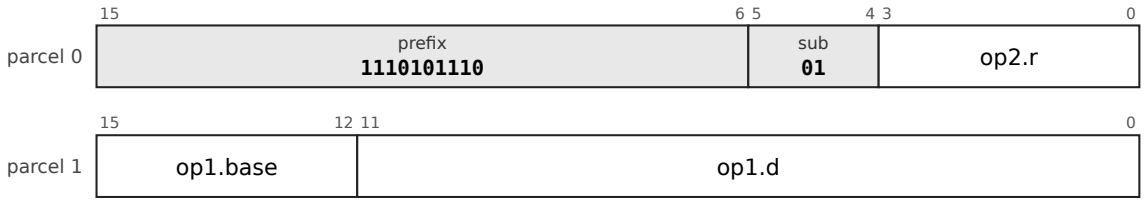
MOVB: STBu12/b, prefix 11101111000, sub-opcode 1; operands reg4, b4d12u-xsz; 4 bytes



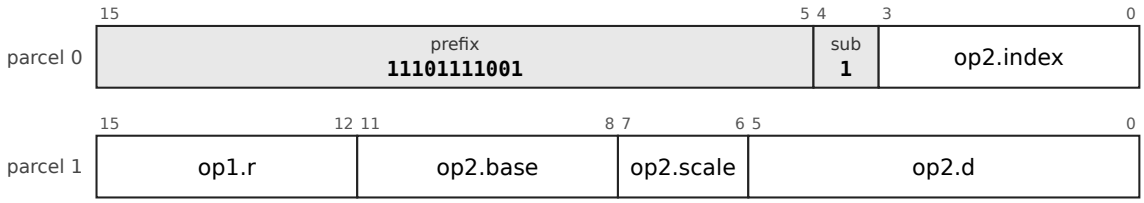
MOVB: LDBu10/b, prefix 111100010100, sub-opcode 01; operands b4d10u-xsz, reg4; 4 bytes



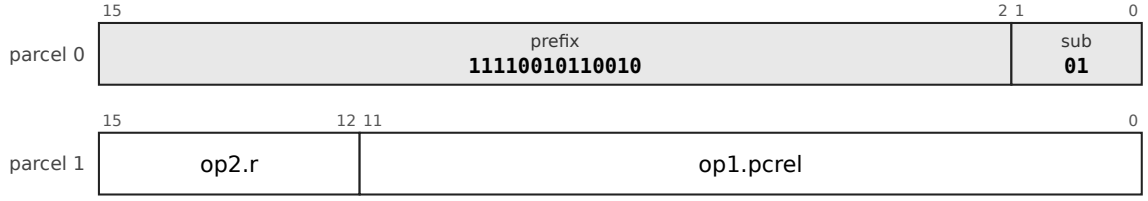
MOVB: LDBu12/b, prefix 1110101110, sub-opcode 01; operands b4d12u-xsz, reg4; 4 bytes



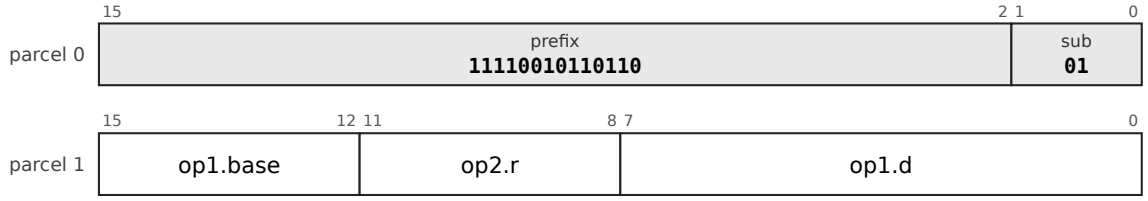
MOVB: STX6/b, prefix 11101111001, sub-opcode 1; operands reg4, x4-4d6s; 4 bytes



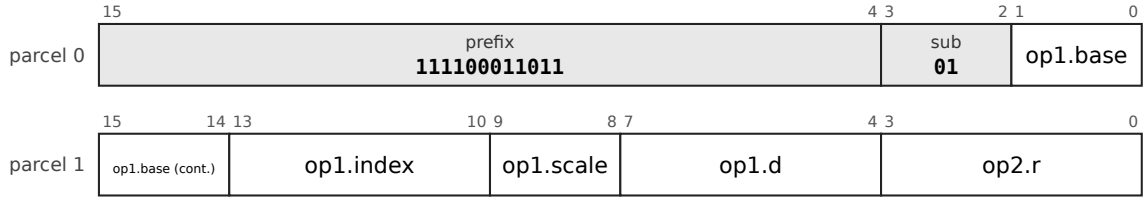
MOVB: LDPC12/b, prefix 11110010110010, sub-opcode 01; operands pc12-u1, reg4; 4 bytes



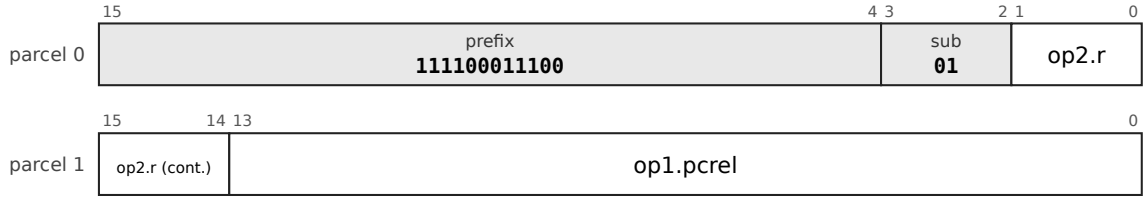
MOVB: LDBs8/b, prefix 11110010110110, sub-opcode 01; operands b4d8s-xsz, reg4; 4 bytes



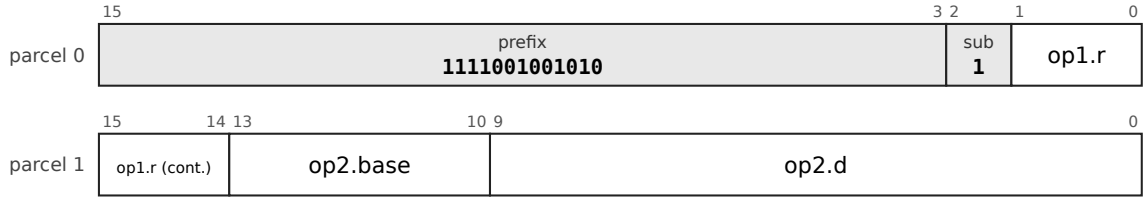
MOVB: LDX4/b, prefix 111100011011, sub-opcode 01; operands x4-4d4s, reg4; 4 bytes



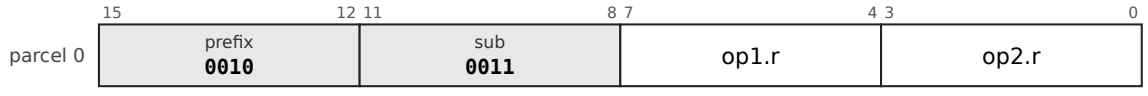
MOVB: LDPC14/b, prefix 111100011100, sub-opcode 01; operands pc14-u1, reg4; 4 bytes



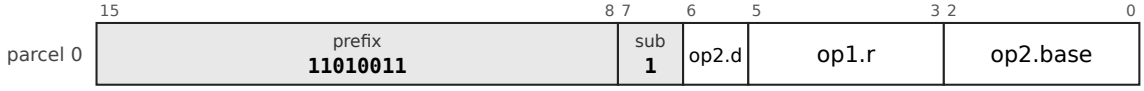
MOVB: STBs10/b, prefix 1111001001010, sub-opcode 1; operands reg4, b4d10s-xsz; 4 bytes



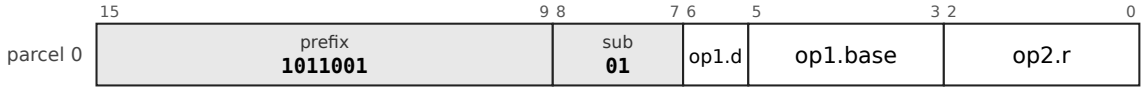
MOVB: RRF, prefix 0010, sub-opcode 0011; operands reg4, reg4; 2 bytes



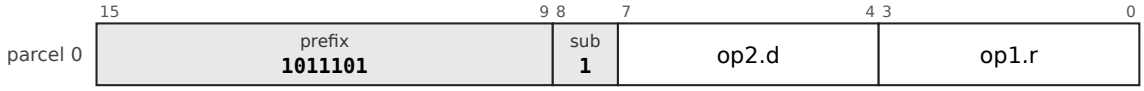
MOVB: STHHH1/b, prefix 11010011, sub-opcode 1; operands reg3, b3d1u-xsz; 2 bytes



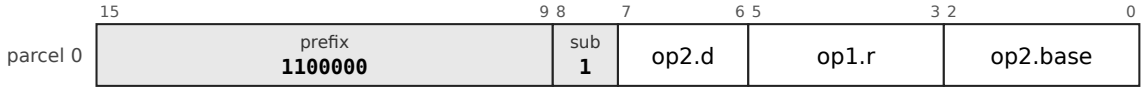
MOVB: LDHHH1/b, prefix 1011001, sub-opcode 01; operands b3d1u-xsz, reg3; 2 bytes



MOVB: STSP4/b, prefix 1011101, sub-opcode 1; operands reg4, spd4u-xsz; 2 bytes



MOVB: STHHH2/b, prefix 1100000, sub-opcode 1; operands reg3, b3d2u-xsz; 2 bytes



Operation, flags and exceptions as for the entry MOV above; 35 layout forms, each a chart of its own (above).

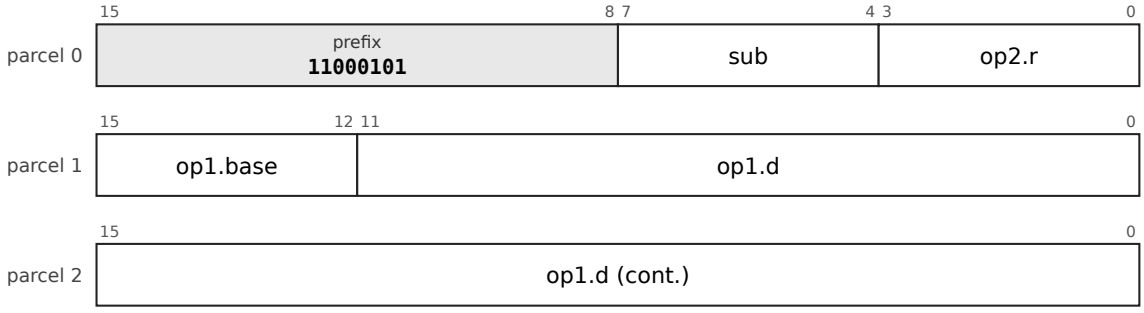
MOVZ: Move zero-extended (32, 16 and 8 bits)

Mnemonics: `MOVZL`, `MOVZW`, `MOVZB`

Syntax

```
MOVZL rm, Rd
MOVZW rm, Rd
MOVZB rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-2`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDBu8-b-1`, `16-LDX2-b-1`, `16-LDBu10-b-1`, `16-LDHHH1-b-1`, `16-LDBu12-b-1`, `16-LDPC12-b-1`, `16-LDBs8-b-1`, `16-LDX4-b-1`, `16-LDPC14-b-1`, `16-LDFI-I32-3`,

16-LDPC12-a-1, 16-LDBs8-a-1, 16-LDPC14-a-1, 16-LDPC16-a-1, 16-LDBu10-a-1, 16-LDHHH1-a-1, 16-LDSP4-a-1, 16-LDBu12-a-1, 16-LDHHH3-a-1, 16-LDX6-a-1, 16-LDBs10-a-1, 16-LDSP5-a-1, 16-LD0-a-1, 16-LDBs14-a-1, 16-LDHRH2-a-1, 16-LDSP7-a-1, 16-LDX8-a-1, 16-LDHHR2-a-1.

Operation

```
Rd ← zext(M[EA(rm)], size)      ; len = 00: Rd ← zext(Rs[8×size-1:0])
```

Flags None written.

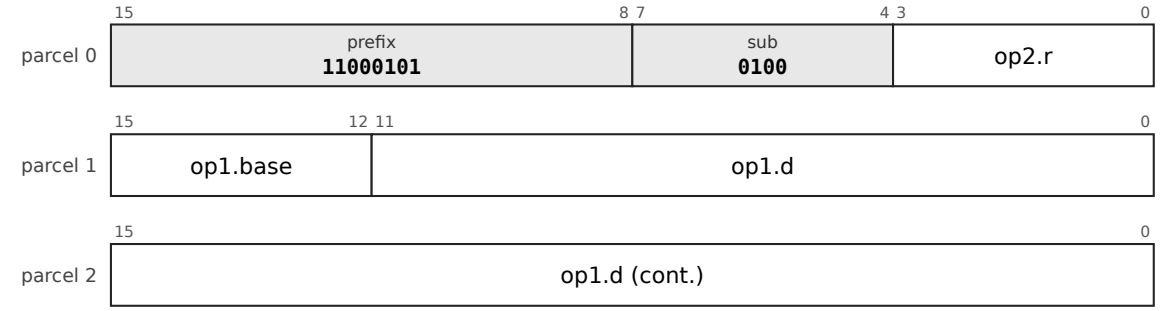
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

There is no zero-extending 64-bit move (it would be MOVQ). The register-to-register forms are the zero-extending half of the conversion set.

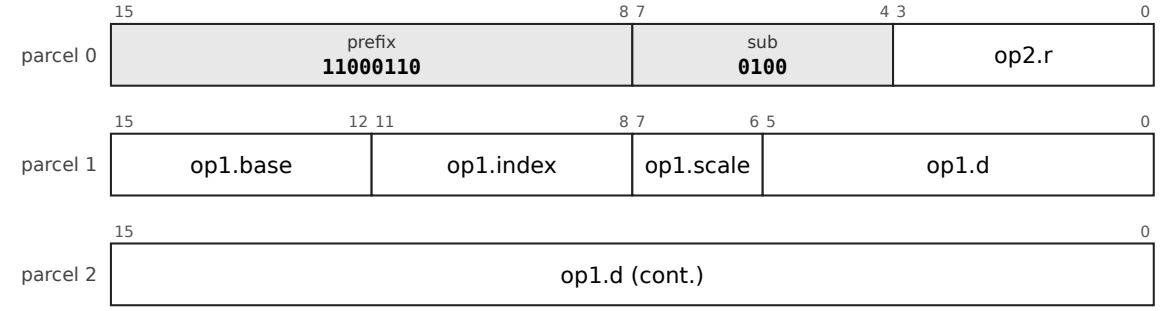
Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

MOVZL

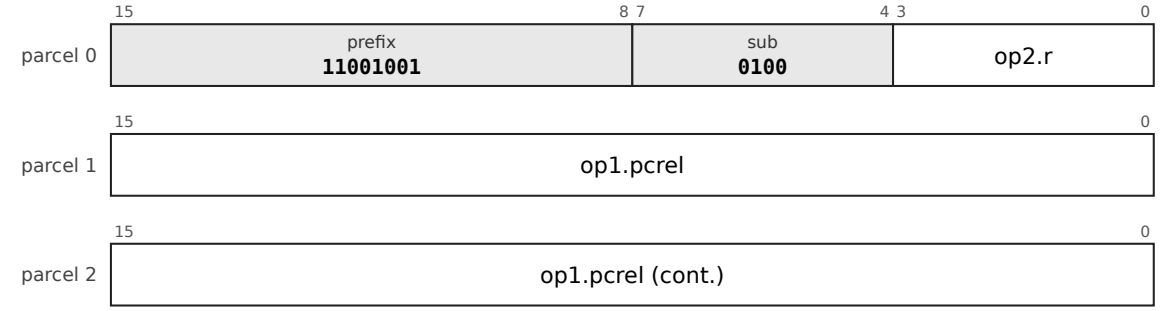
MOVZL: LDF.B28, prefix 11000101, sub-opcode 0100; operands b4d28s-x1, reg4; 6 bytes



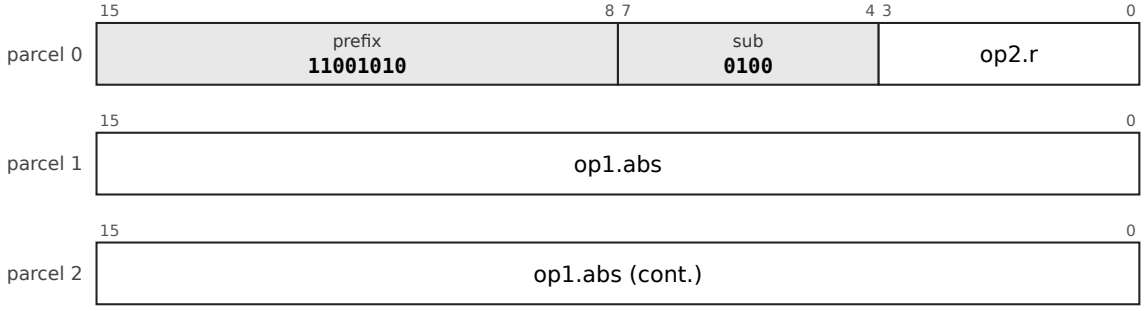
MOVZL: LDF.X22, prefix 11000110, sub-opcode 0100; operands x4-4d22s, reg4; 6 bytes



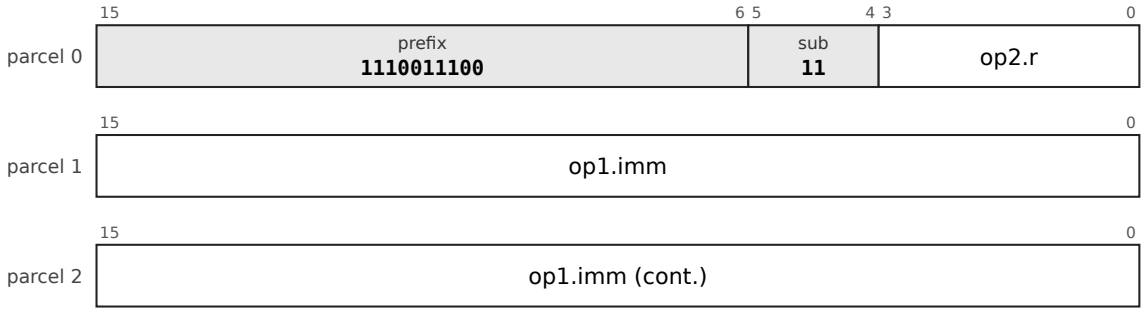
MOVZL: LDF.PC32, prefix 11001001, sub-opcode 0100; operands pc32-u1, reg4; 6 bytes



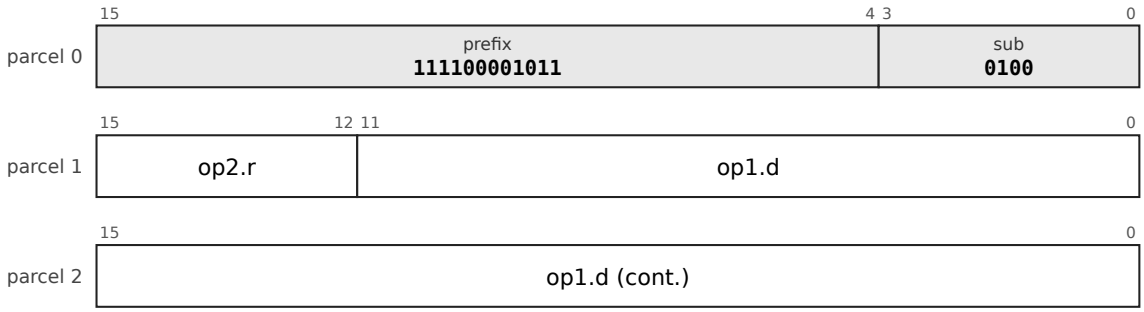
MOVZL: LDF.A32, prefix 11001010, sub-opcode 0100; operands abs32, reg4; 6 bytes



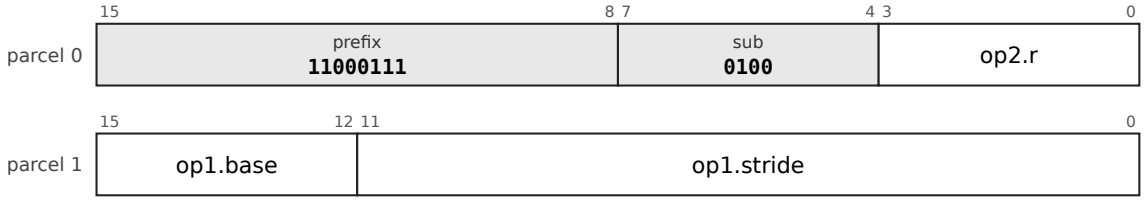
MOVZL: LDFI.I32, prefix 1110011100, sub-opcode 11; operands imm32s, reg4; 6 bytes



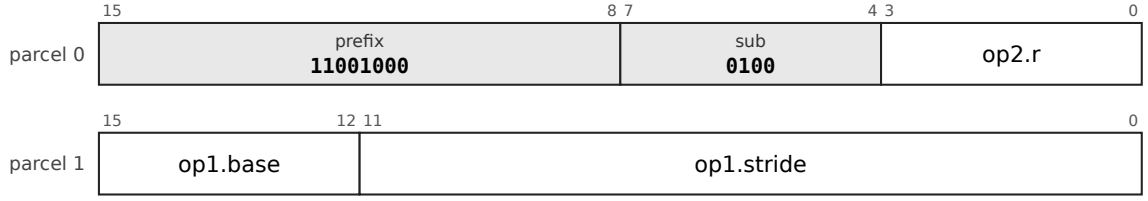
MOVZL: DPF.D28, prefix 111100001011, sub-opcode 0100; operands dpd28s-x1, reg4; 6 bytes



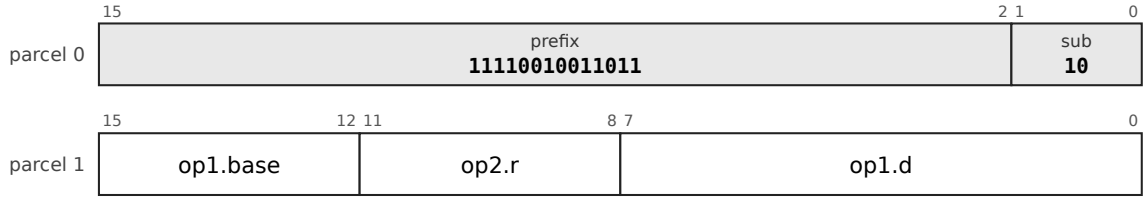
MOVZL: LDF.PI12, prefix 11000111, sub-opcode 0100; operands pi4-s12, reg4; 4 bytes



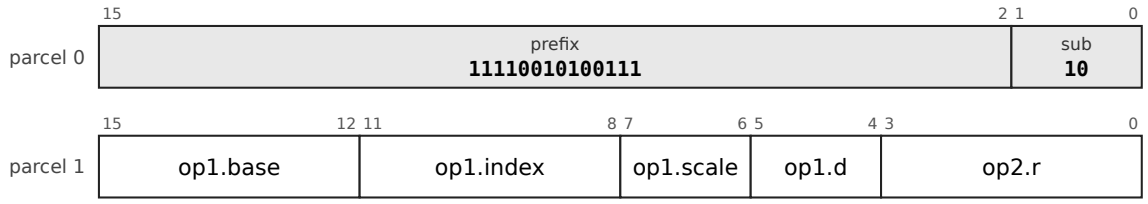
MOVZL: LDF.PD12, prefix 11001000, sub-opcode 0100; operands pd4-s12, reg4; 4 bytes



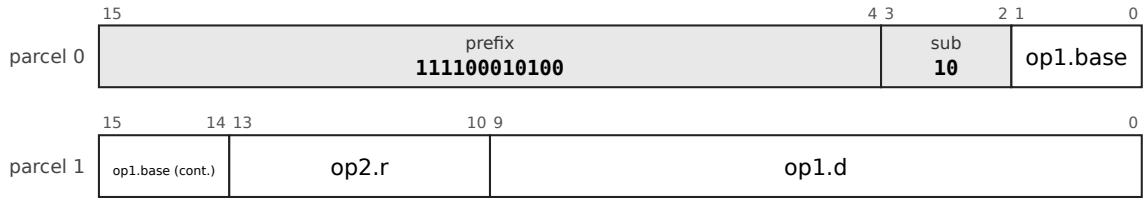
MOVZL: LDBu8/b, prefix 11110010011011, sub-opcode 10; operands b4d8u-xsz, reg4; 4 bytes



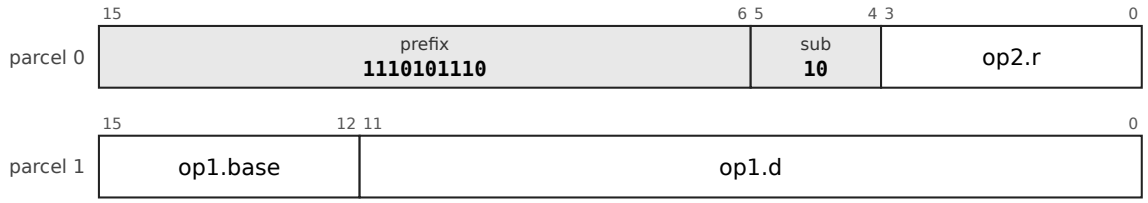
MOVZL: LDX2/b, prefix 11110010100111, sub-opcode 10; operands x4-4d2s, reg4; 4 bytes



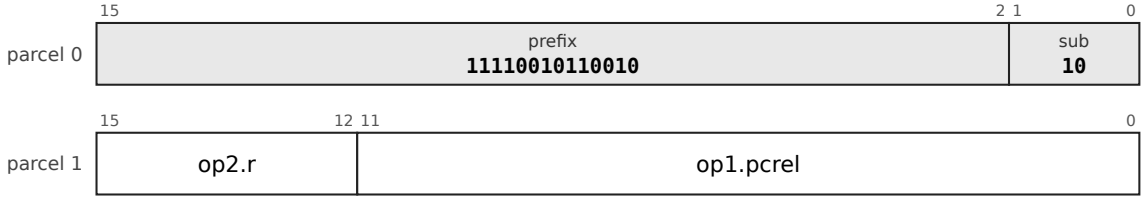
MOVZL: LDBu10/b, prefix 111100010100, sub-opcode 10; operands b4d10u-xsz, reg4; 4 bytes



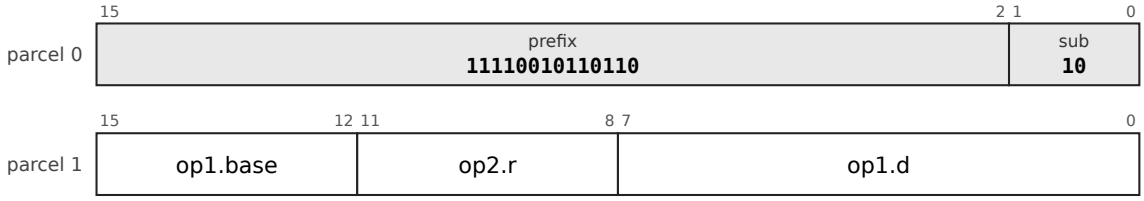
MOVZL: LDBu12/b, prefix 1110101110, sub-opcode 10; operands b4d12u-xsz, reg4; 4 bytes



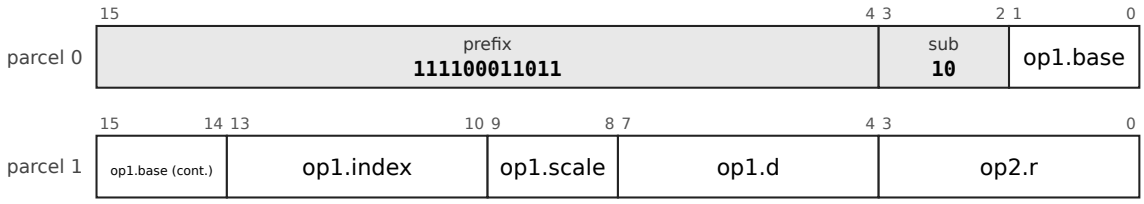
MOVZL: LDPC12/b, prefix 11110010110010, sub-opcode 10; operands pc12-u1, reg4; 4 bytes



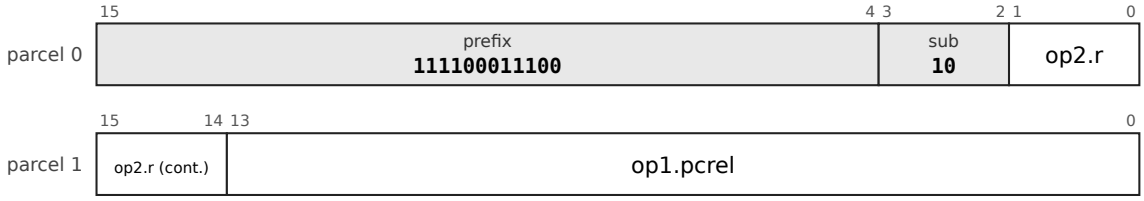
MOVZL: LDBs8/b, prefix 11110010110110, sub-opcode 10; operands b4d8s-xsz, reg4; 4 bytes



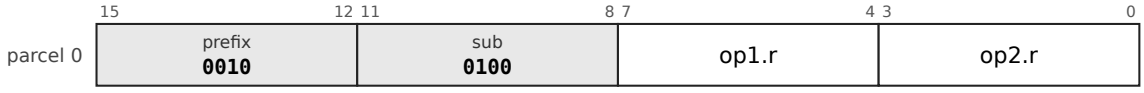
MOVZL: LDX4/b, prefix 111100011011, sub-opcode 10; operands x4-4d4s, reg4; 4 bytes



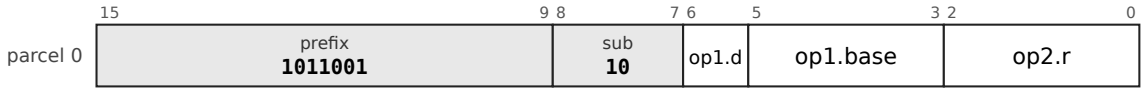
MOVZL: LDPC14/b, prefix 111100011100, sub-opcode 10; operands pc14-u1, reg4; 4 bytes



MOVZL: RRF, prefix 0010, sub-opcode 0100; operands reg4, reg4; 2 bytes



MOVZL: LDHHH1/b, prefix 1011001, sub-opcode 10; operands b3d1u-xsz, reg3; 2 bytes

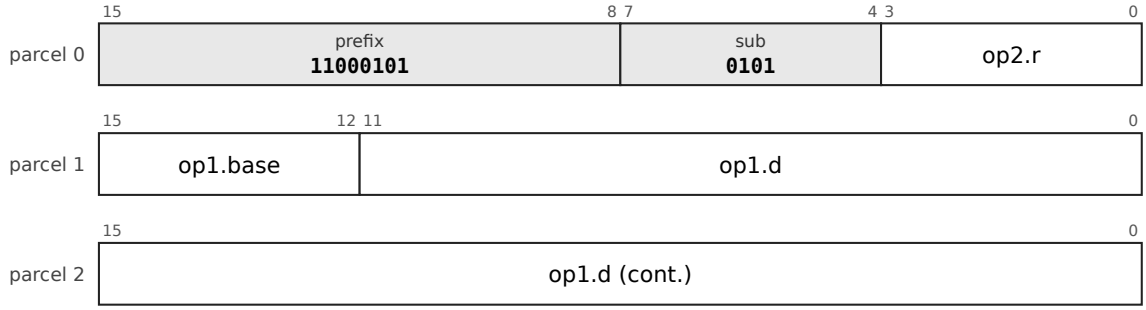



```
MOVZL rm, Rd
```

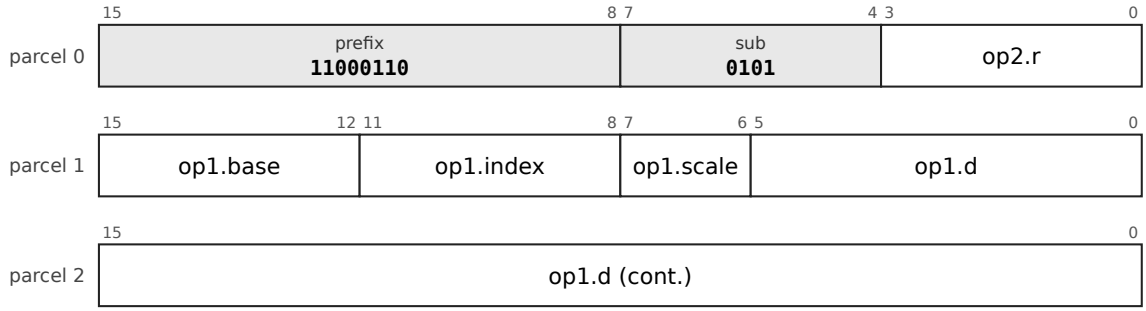
Operation, flags and exceptions as for the entry MOVZ above; 18 layout forms, each a chart of its own (above).

MOVZW

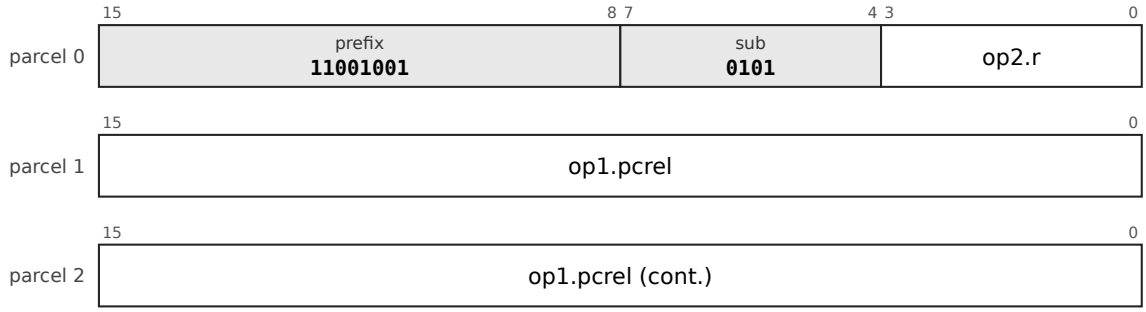
MOVZW: LDF.B28, prefix 11000101, sub-opcode 0101; operands b4d28s-x1, reg4; 6 bytes



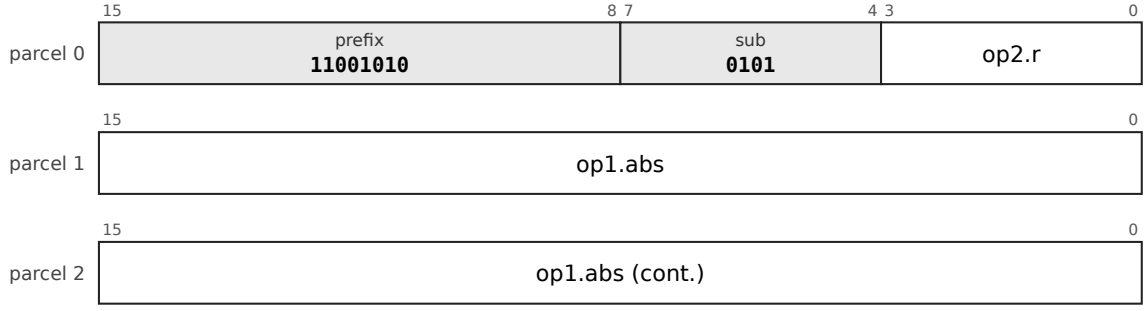
MOVZW: LDF.X22, prefix 11000110, sub-opcode 0101; operands x4-4d22s, reg4; 6 bytes



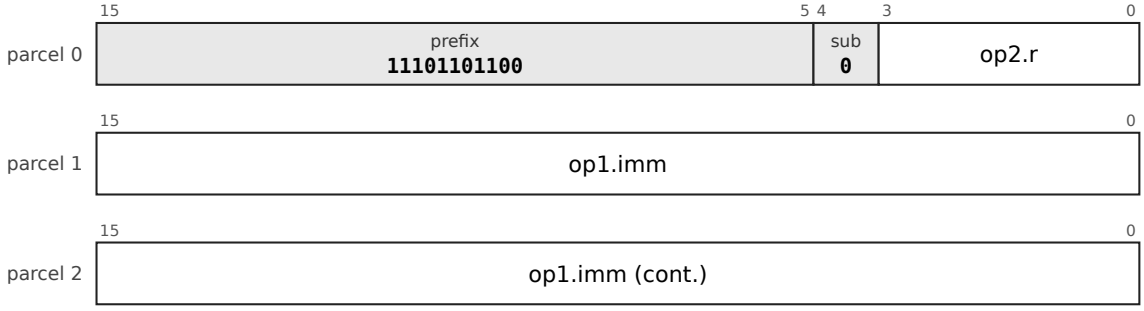
MOVZW: LDF.PC32, prefix 11001001, sub-opcode 0101; operands pc32-u1, reg4; 6 bytes



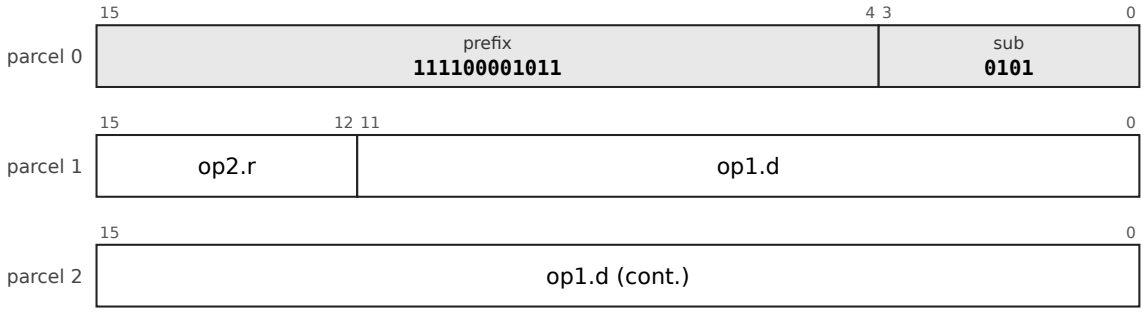
MOVZW: LDF.A32, prefix 11001010, sub-opcode 0101; operands abs32, reg4; 6 bytes



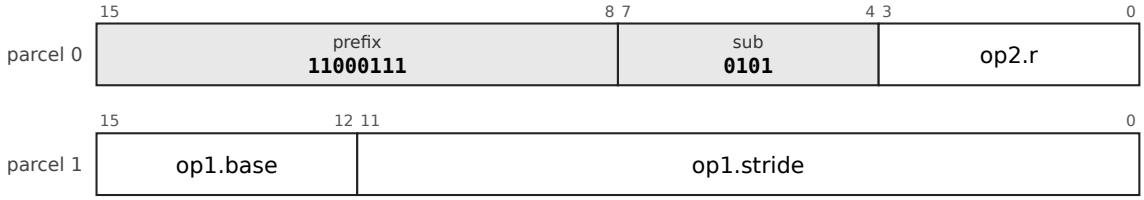
MOVZW: LDFI.I32, prefix 11101101100, sub-opcode 0; operands imm32s, reg4; 6 bytes



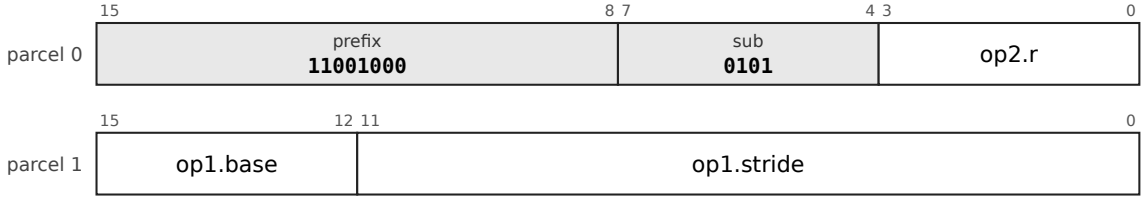
MOVZW: DPF.D28, prefix 111100001011, sub-opcode 0101; operands dpd28s-x1, reg4; 6 byte



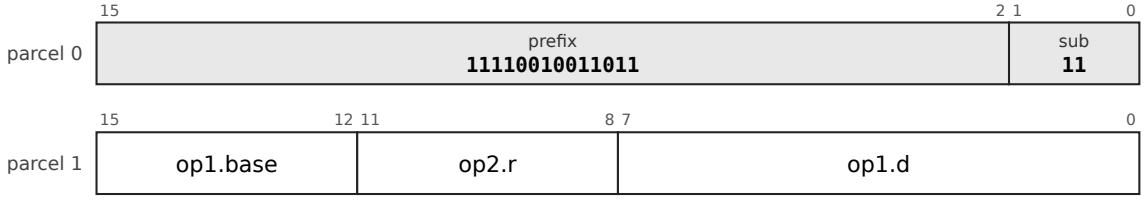
MOVZW: LDF.PI12, prefix 11000111, sub-opcode 0101; operands pi4-s12, reg4; 4 bytes



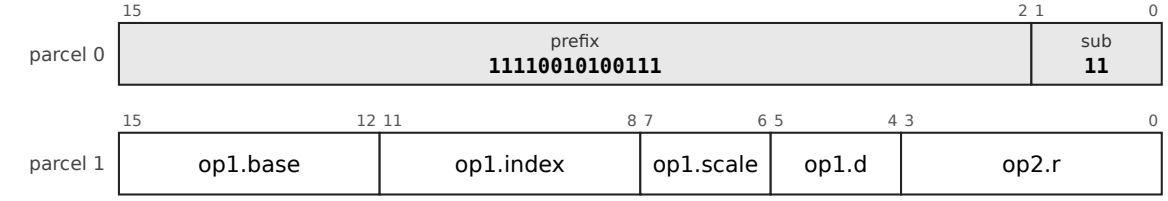
MOVZW: LDF.PD12, prefix 11001000, sub-opcode 0101; operands pd4-s12, reg4; 4 bytes



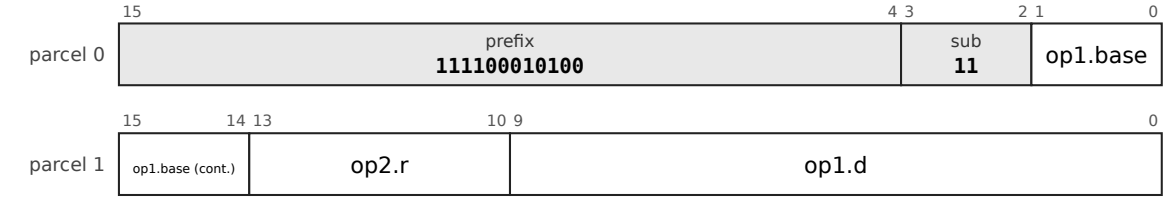
MOVZW: LDBu8/b, prefix 11110010011011, sub-opcode 11; operands b4d8u-xsz, reg4; 4 byte



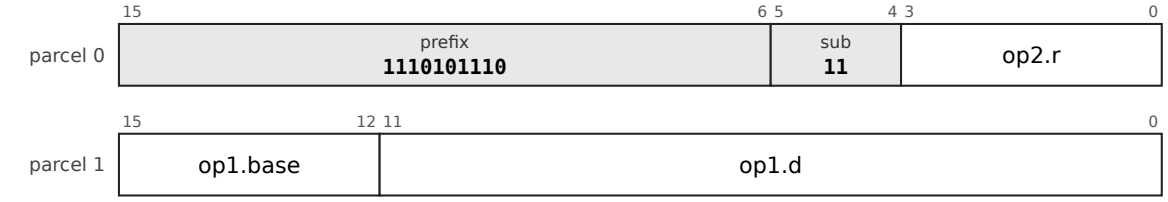
MOVZW: LDX2/b, prefix 11110010100111, sub-opcode 11; operands x4-4d2s, reg4; 4 bytes



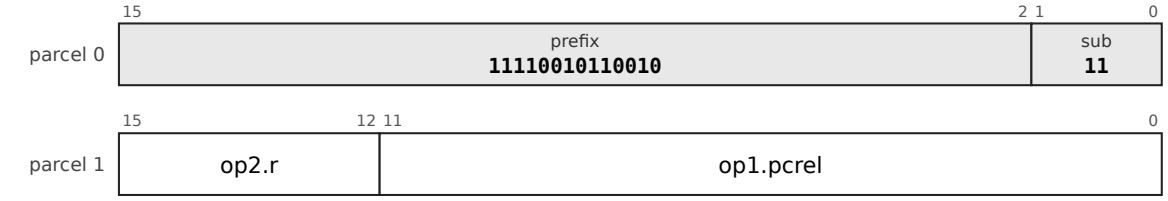
MOVZW: LDBu10/b, prefix 111100010100, sub-opcode 11; operands b4d10u-xsz, reg4; 4 byte



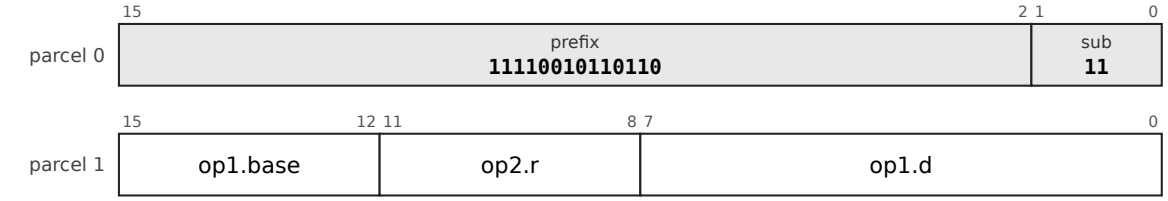
MOVZW: LDBu12/b, prefix 1110101110, sub-opcode 11; operands b4d12u-xsz, reg4; 4 bytes



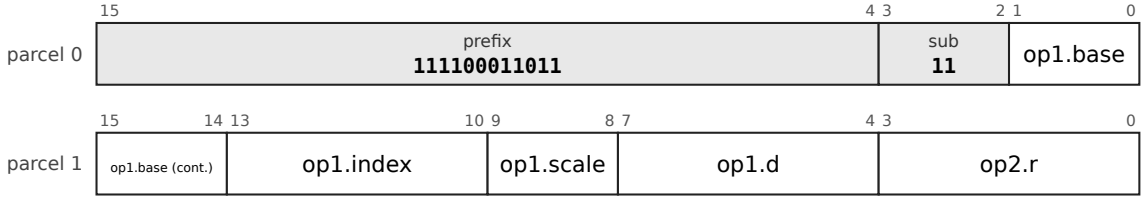
MOVZW: LDPC12/b, prefix 11110010110010, sub-opcode 11; operands pc12-u1, reg4; 4 bytes



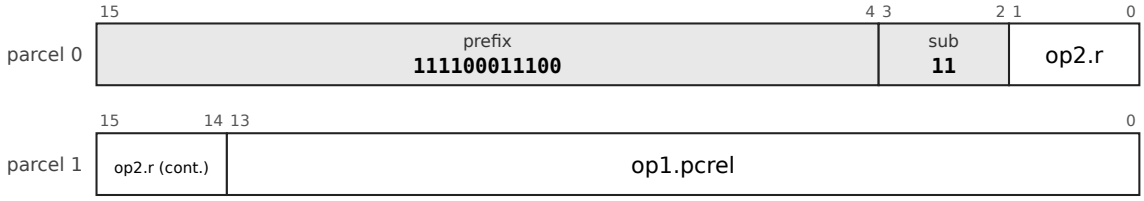
MOVZW: LDBs8/b, prefix 11110010110110, sub-opcode 11; operands b4d8s-xsz, reg4; 4 byte



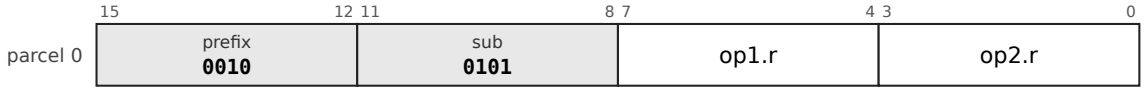
MOVZW: LDX4/b, prefix 111100011011, sub-opcode 11; operands x4-4d4s, reg4; 4 bytes



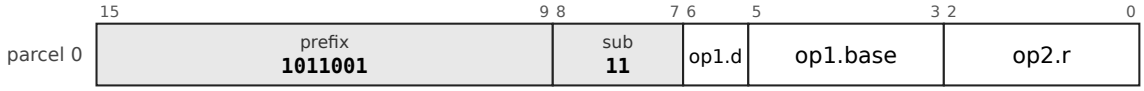
MOVZW: LDPC14/b, prefix 111100011100, sub-opcode 11; operands pc14-u1, reg4; 4 bytes



MOVZW: RRF, prefix 0010, sub-opcode 0101; operands reg4, reg4; 2 bytes



MOVZW: LDHHH1/b, prefix 1011001, sub-opcode 11; operands b3d1u-xsz, reg3; 2 bytes

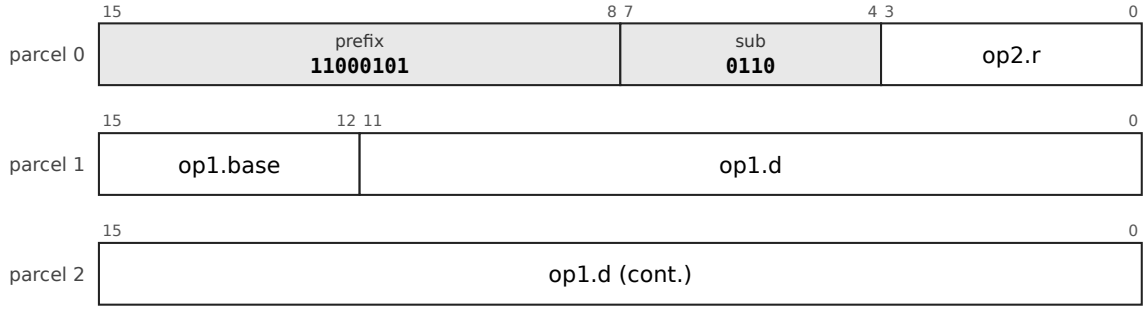



```
MOVZW rm, Rd
```

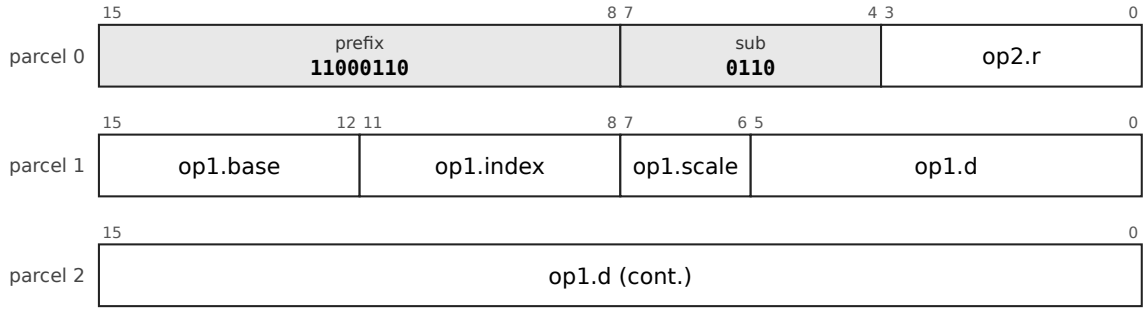
Operation, flags and exceptions as for the entry MOVZ above; 18 layout forms, each a chart of its own (above).

MOVZB

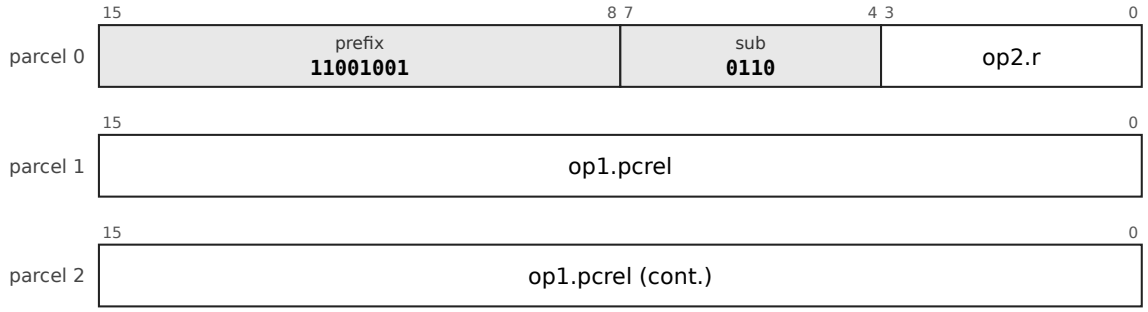
MOVZB: LDF.B28, prefix 11000101, sub-opcode 0110; operands b4d28s-x1, reg4; 6 bytes



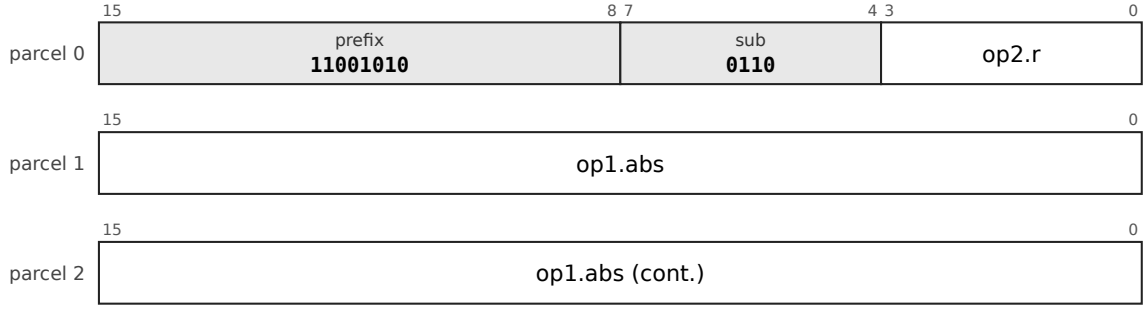
MOVZB: LDF.X22, prefix 11000110, sub-opcode 0110; operands x4-4d22s, reg4; 6 bytes



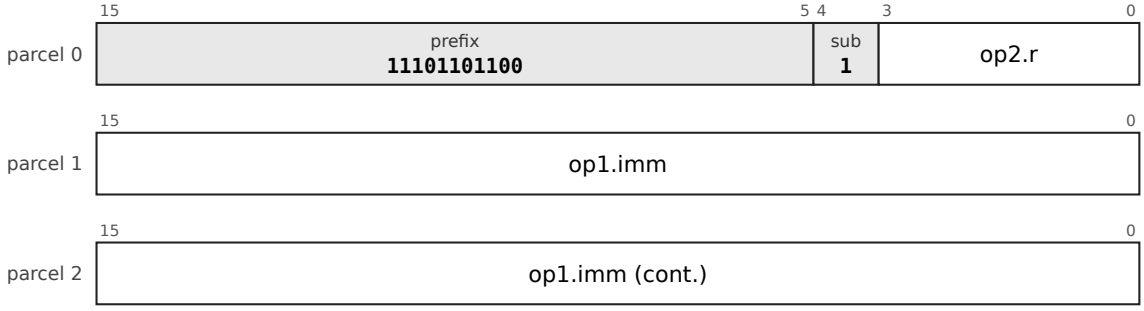
MOVZB: LDF.PC32, prefix 11001001, sub-opcode 0110; operands pc32-u1, reg4; 6 bytes



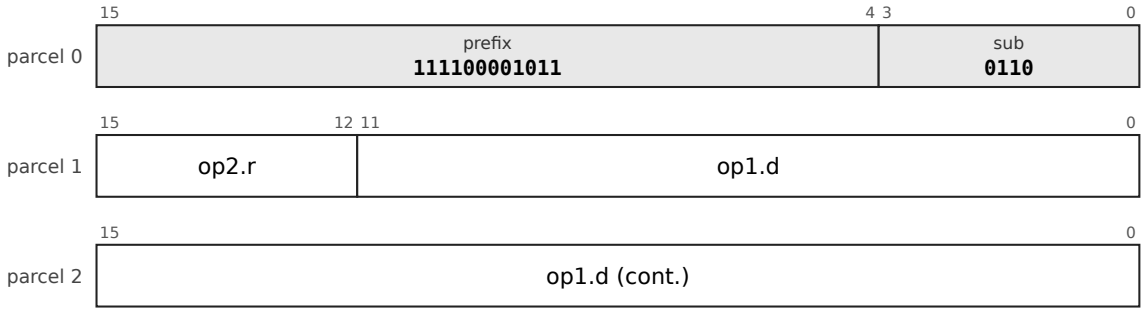
MOVZB: LDF.A32, prefix 11001010, sub-opcode 0110; operands abs32, reg4; 6 bytes



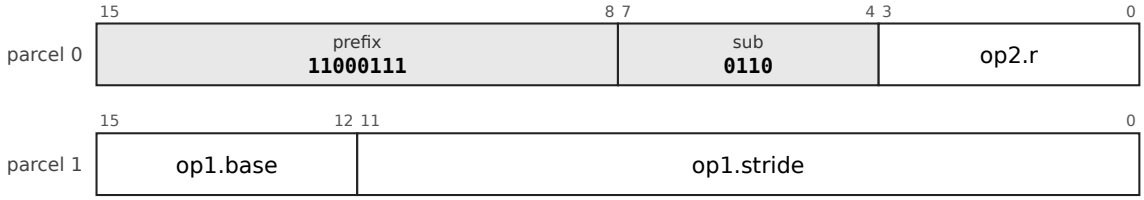
MOVZB: LDFI.I32, prefix 11101101100, sub-opcode 1; operands imm32s, reg4; 6 bytes



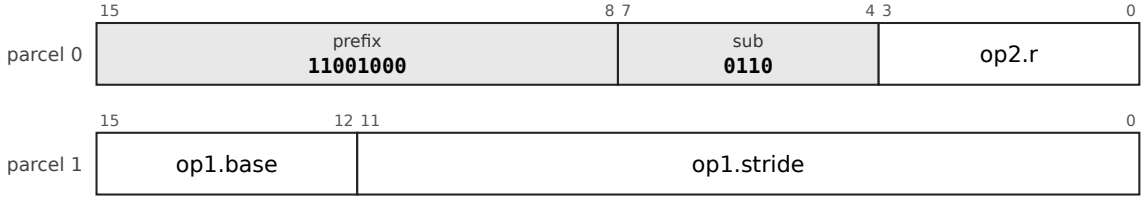
MOVZB: DPF.D28, prefix 111100001011, sub-opcode 0110; operands dpd28s-x1, reg4; 6 byte



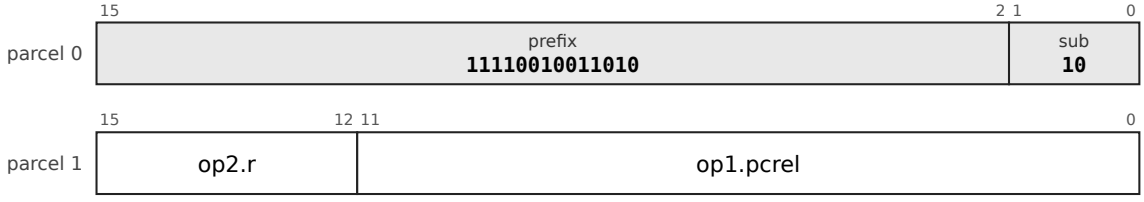
MOVZB: LDF.PI12, prefix 11000111, sub-opcode 0110; operands pi4-s12, reg4; 4 bytes



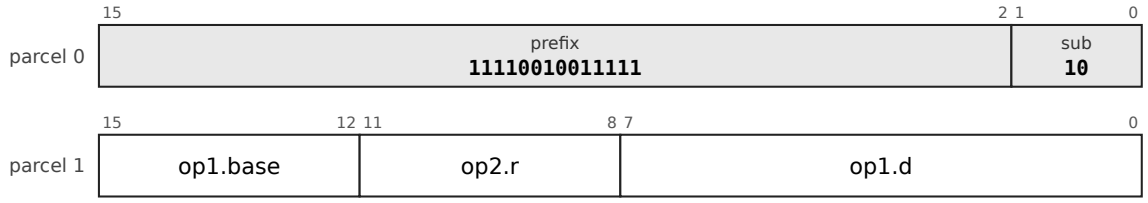
MOVZB: LDF.PD12, prefix 11001000, sub-opcode 0110; operands pd4-s12, reg4; 4 bytes



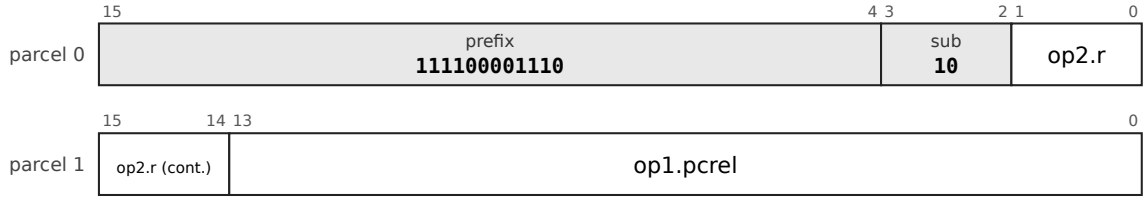
MOVZB: LDPC12/a, prefix 11110010011010, sub-opcode 10; operands pc12-u1, reg4; 4 bytes



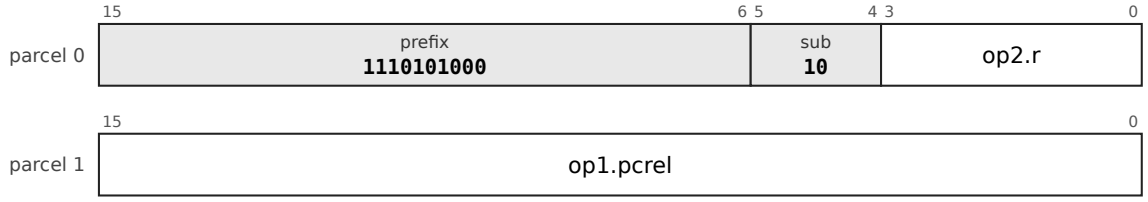
MOVZB: LDBs8/a, prefix 11110010011111, sub-opcode 10; operands b4d8s-xsz, reg4; 4 bytes



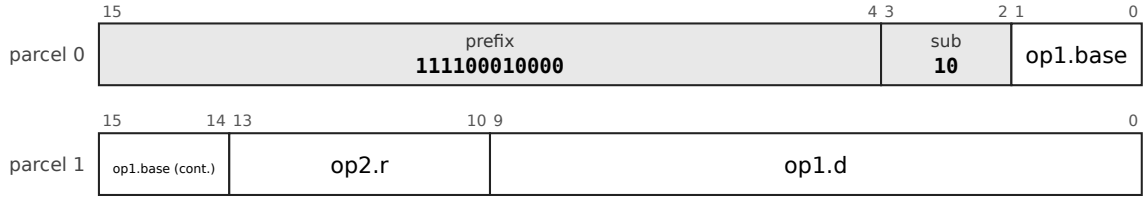
MOVZB: LDPC14/a, prefix 111100001110, sub-opcode 10; operands pc14-u1, reg4; 4 bytes



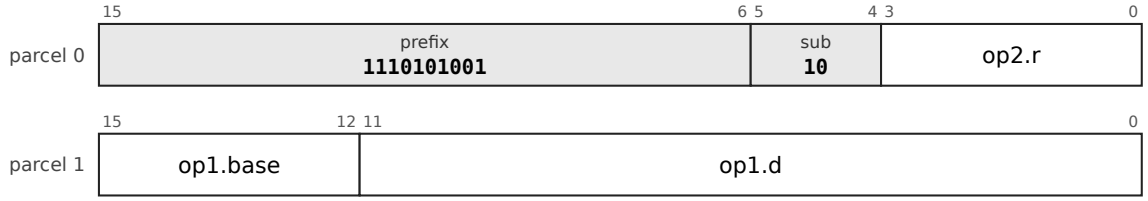
MOVZB: LDPC16/a, prefix 1110101000, sub-opcode 10; operands pc16-u1, reg4; 4 bytes



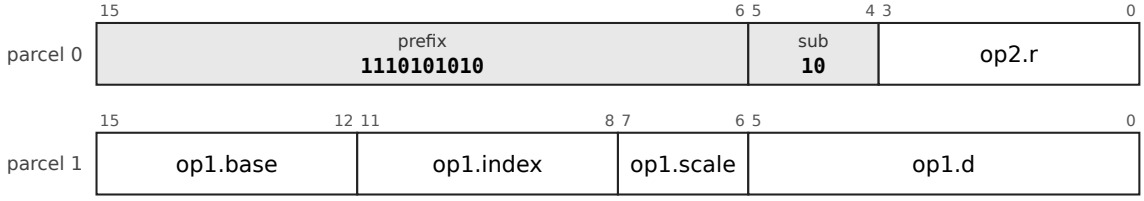
MOVZB: LDBu10/a, prefix 111100010000, sub-opcode 10; operands b4d10u-xsz, reg4; 4 bytes



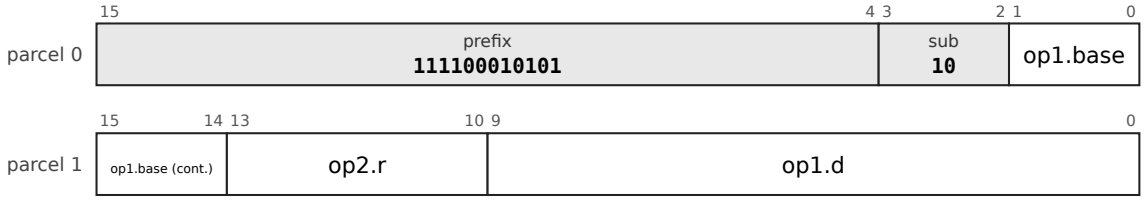
MOVZB: LDBu12/a, prefix 1110101001, sub-opcode 10; operands b4d12u-xsz, reg4; 4 bytes



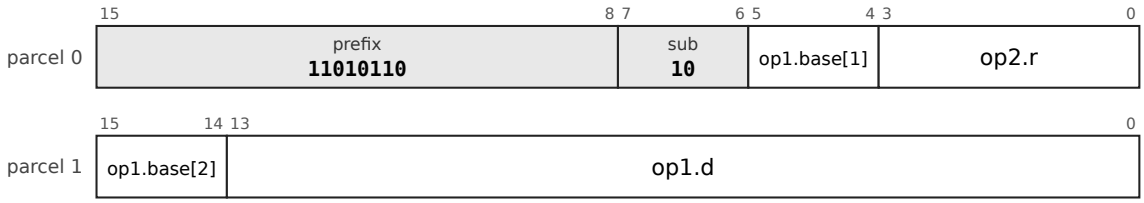
MOVZB: LDX6/a, prefix 1110101010, sub-opcode 10; operands x4-4d6s, reg4; 4 bytes



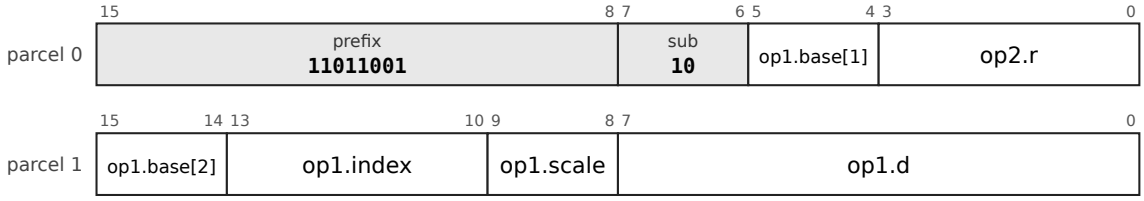
MOVZB: LDBs10/a, prefix 111100010101, sub-opcode 10; operands b4d10s-xsz, reg4; 4 bytes



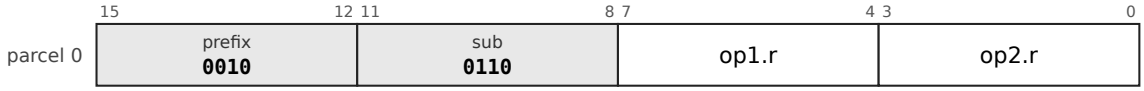
MOVZB: LDBs14/a, prefix 11010110, sub-opcode 10; operands b4d14s-xsz, reg4; 4 bytes



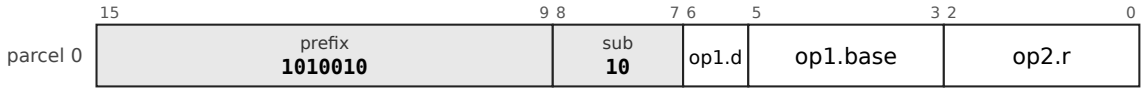
MOVZB: LDX8/a, prefix 11011001, sub-opcode 10; operands x4-4d8s, reg4; 4 bytes



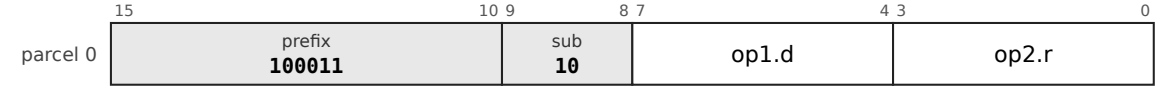
MOVZB: RRF, prefix 0010, sub-opcode 0110; operands reg4, reg4; 2 bytes



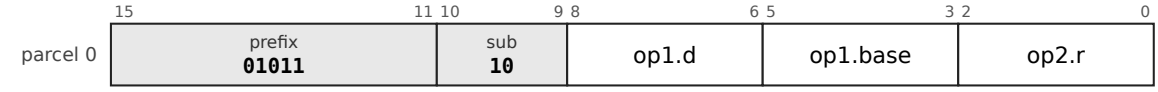
MOVZB: LDHHH1/a, prefix 1010010, sub-opcode 10; operands b3d1u-xsz, reg3; 2 bytes



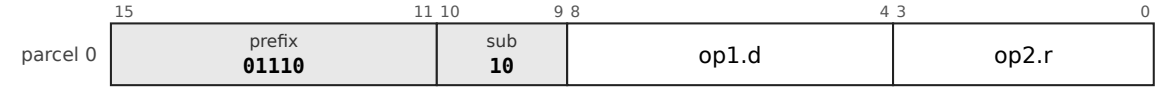
MOVZB: LDSP4/a, prefix 100011, sub-opcode 10; operands spd4u-xsz, reg4; 2 bytes



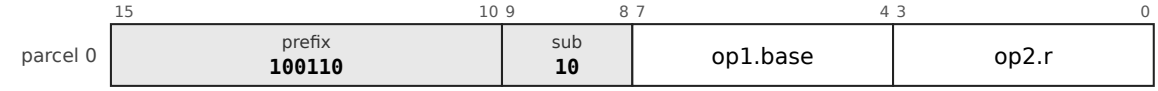
MOVZB: LDHHH3/a, prefix 01011, sub-opcode 10; operands b3d3u-xsz, reg3; 2 bytes



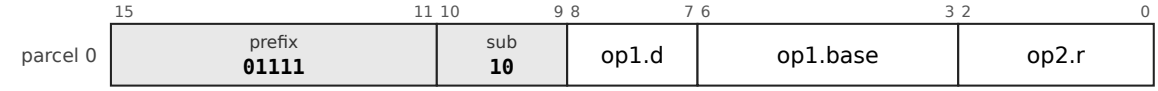
MOVZB: LDSP5/a, prefix 01110, sub-opcode 10; operands spd5u-xsz, reg4; 2 bytes



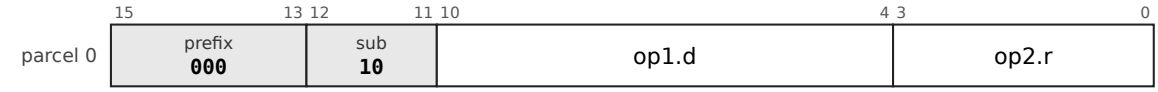
MOVZB: LD0/a, prefix 100110, sub-opcode 10; operands b4d0s-xsz, reg4; 2 bytes



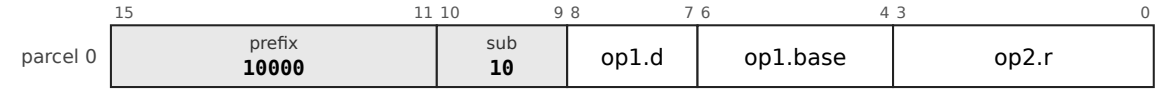
MOVZB: LDHRH2/a, prefix 01111, sub-opcode 10; operands b4d2u-xsz, reg3; 2 bytes



MOVZB: LDSP7/a, prefix 000, sub-opcode 10; operands spd7u-xsz, reg4; 2 bytes



MOVZB: LDHHR2/a, prefix 10000, sub-opcode 10; operands b3d2u-xsz, reg4; 2 bytes

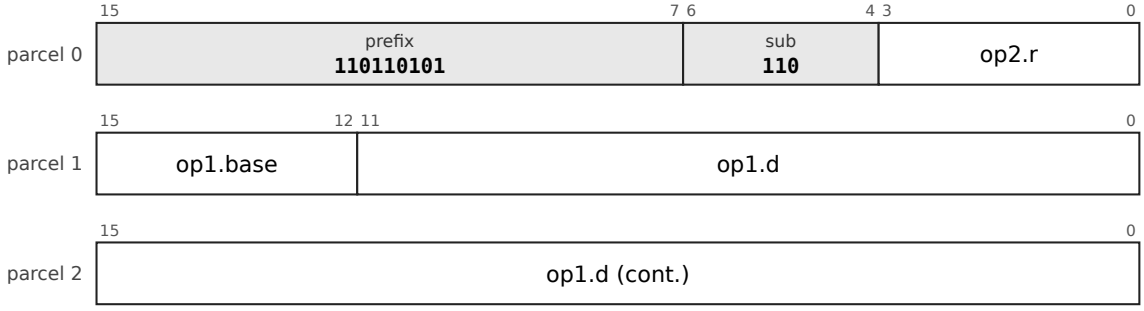



```
MOVZB rm, Rd
```

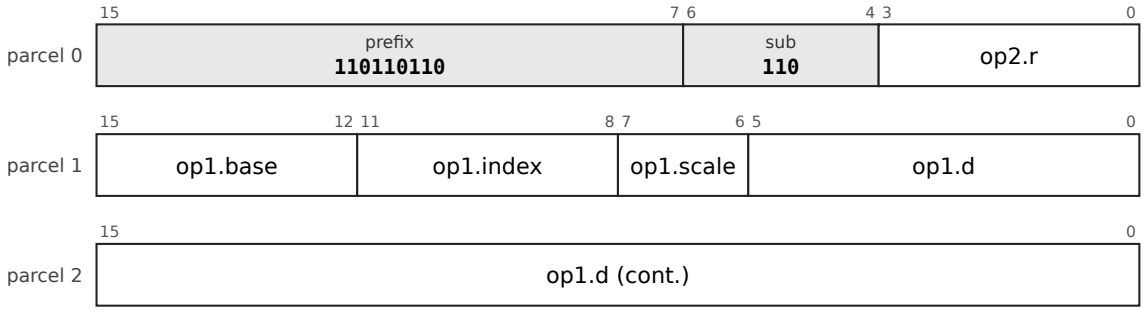
Operation, flags and exceptions as for the entry MOVZ above; 27 layout forms, each a chart of its own (above).

LEA: Load effective address

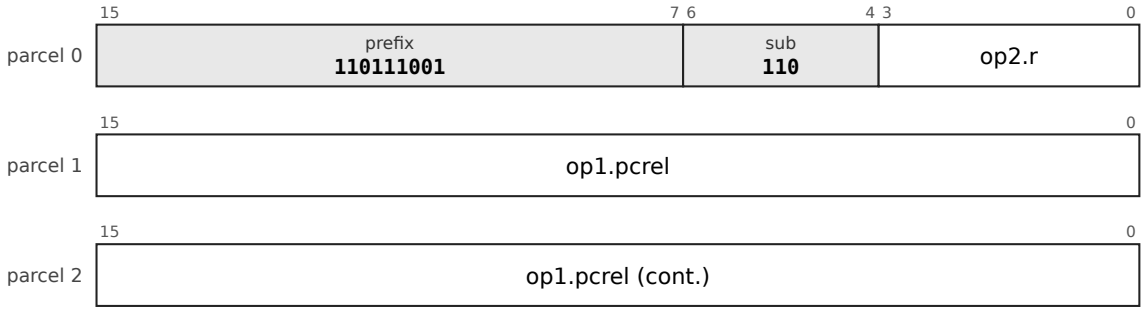
LEA: LDF.B28, prefix 110110101, sub-opcode 110; operands b4d28s-x1, reg4; 6 bytes



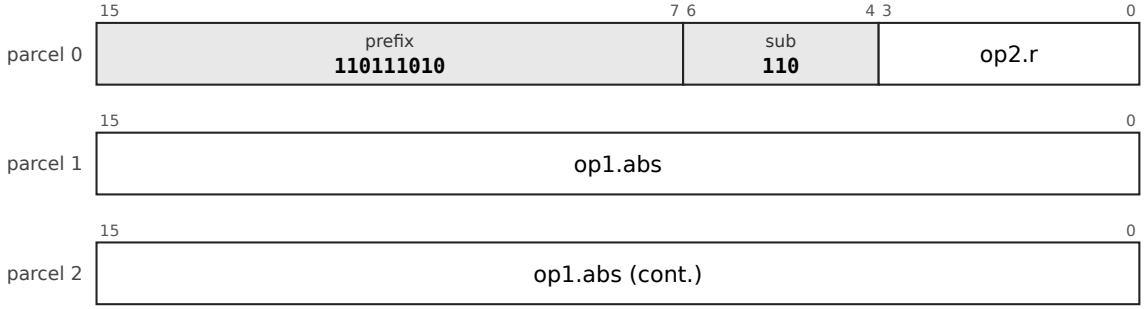
LEA: LDF.X22, prefix 110110110, sub-opcode 110; operands x4-4d22s, reg4; 6 bytes



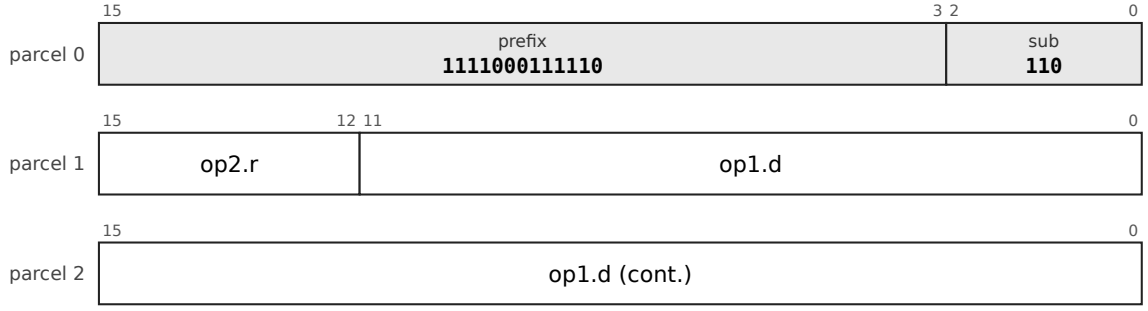
LEA: LDF.PC32, prefix 110111001, sub-opcode 110; operands pc32-u1, reg4; 6 bytes



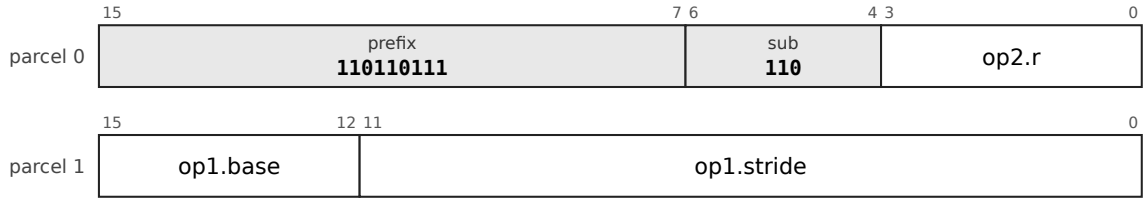
LEA: LDF.A32, prefix 110111010, sub-opcode 110; operands abs32, reg4; 6 bytes



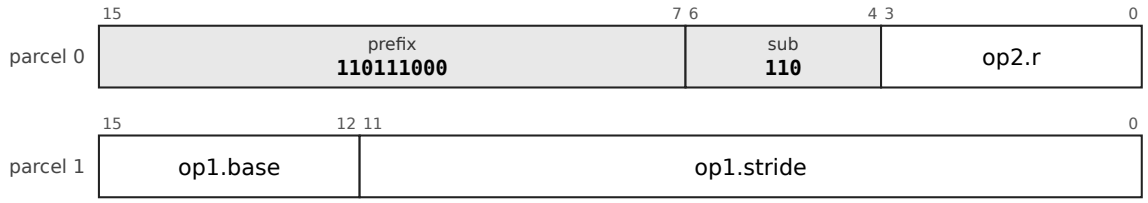
LEA: DPF.D28, prefix 1111000111110, sub-opcode 110; operands dpd28s-x1, reg4; 6 bytes



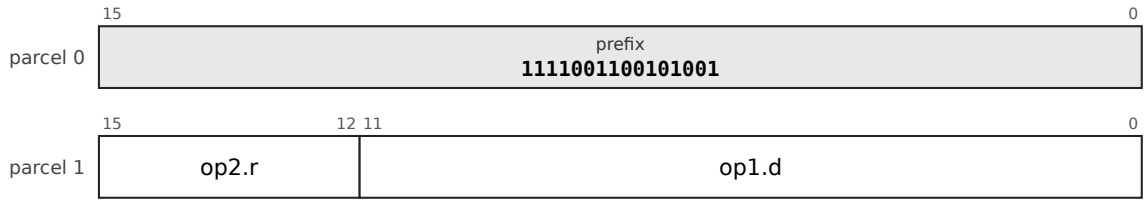
LEA: LDF.PI12, prefix 110110111, sub-opcode 110; operands pi4-s12, reg4; 4 bytes



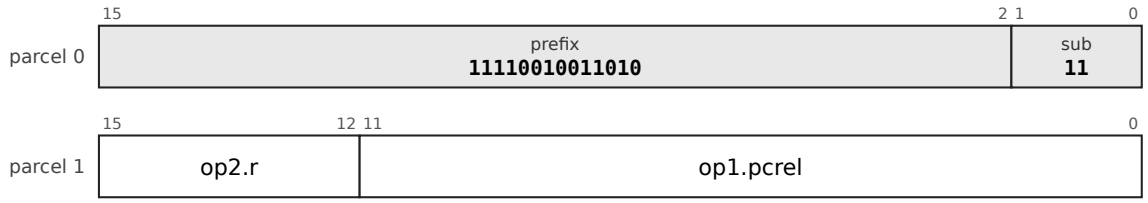
LEA: LDF.PD12, prefix 110111000, sub-opcode 110; operands pd4-s12, reg4; 4 bytes



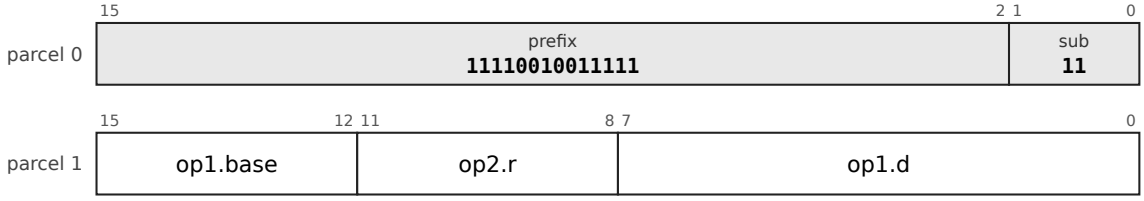
LEA: LEASx12/a, prefix 1111001100101001; operands spd12s-x1, reg4; 4 bytes



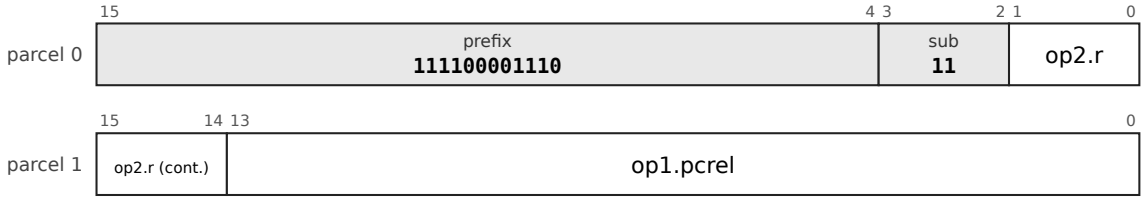
LEA: LDPC12/a, prefix 11110010011010, sub-opcode 11; operands pc12-u1, reg4; 4 bytes



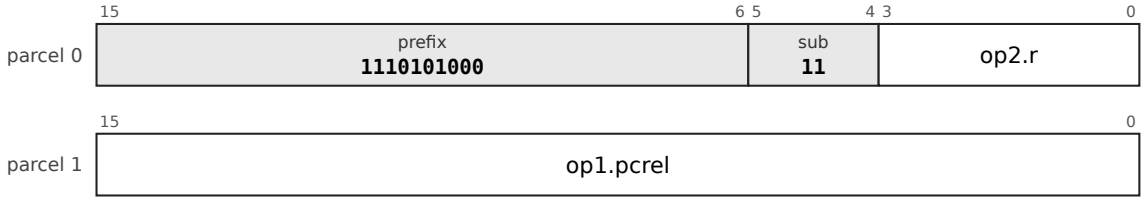
LEA: LDBs8/a, prefix 11110010011111, sub-opcode 11; operands b4d8s-xsz, reg4; 4 bytes



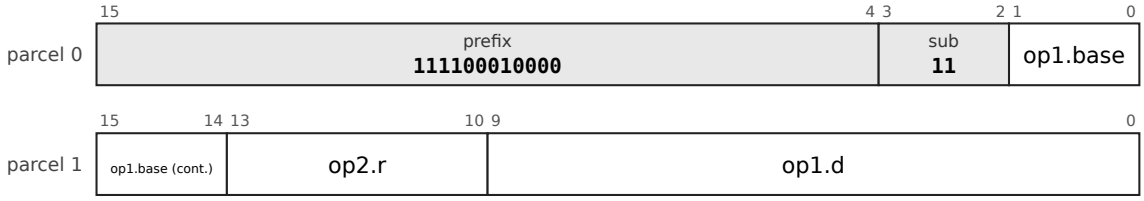
LEA: LDPC14/a, prefix 111100001110, sub-opcode 11; operands pc14-u1, reg4; 4 bytes



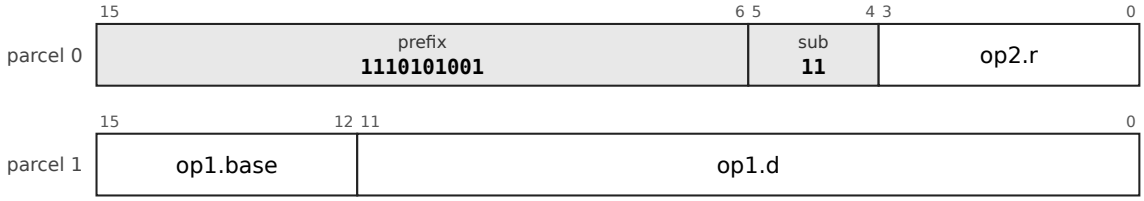
LEA: LDPC16/a, prefix 1110101000, sub-opcode 11; operands pc16-u1, reg4; 4 bytes



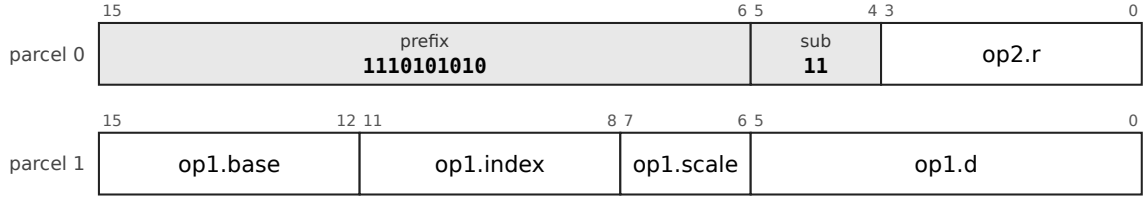
LEA: LDBu10/a, prefix 111100010000, sub-opcode 11; operands b4d10u-xsz, reg4; 4 bytes



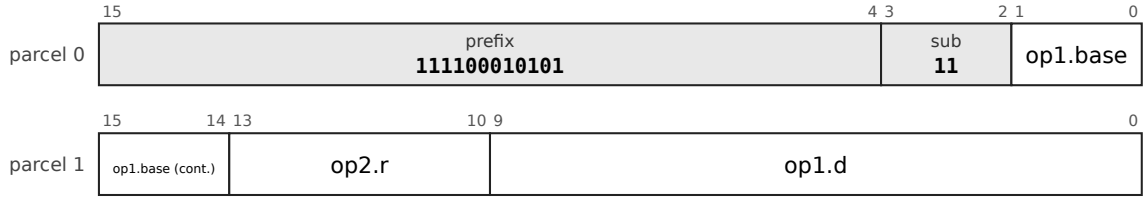
LEA: LDBu12/a, prefix 1110101001, sub-opcode 11; operands b4d12u-xsz, reg4; 4 bytes



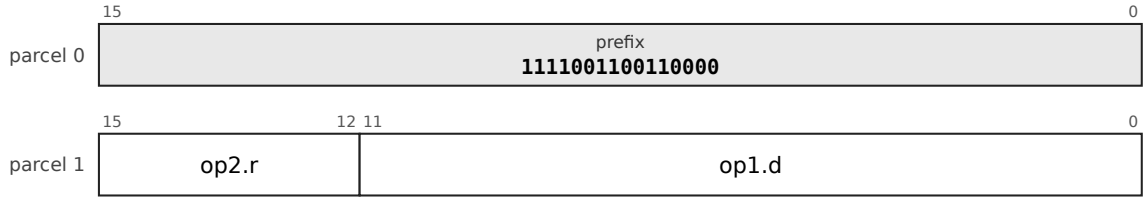
LEA: LDX6/a, prefix 1110101010, sub-opcode 11; operands x4-4d6s, reg4; 4 bytes



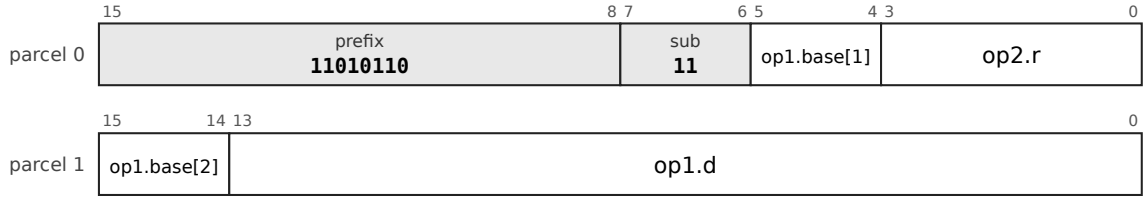
LEA: LDBs10/a, prefix 111100010101, sub-opcode 11; operands b4d10s-xsz, reg4; 4 bytes



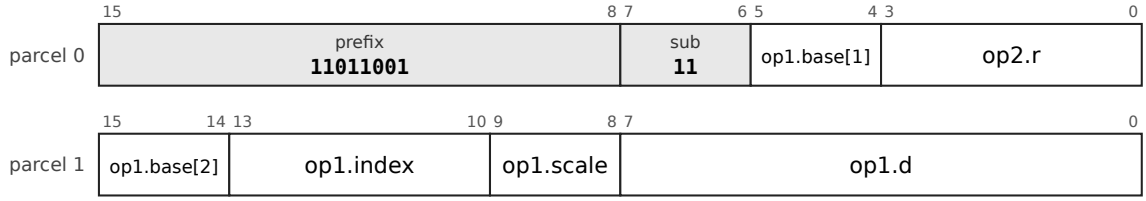
LEA: LEAS12/a, prefix 1111001100110000; operands spd12u-x8, reg4; 4 bytes



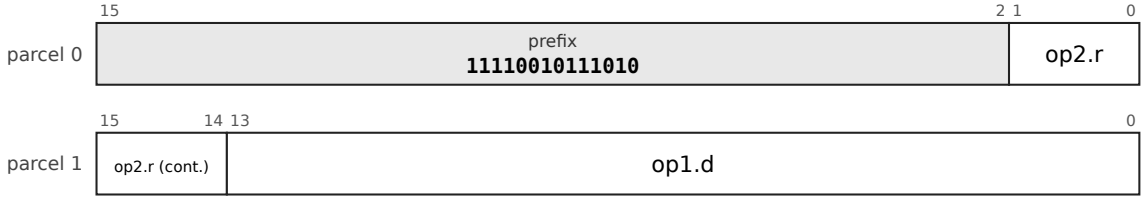
LEA: LDBs14/a, prefix 11010110, sub-opcode 11; operands b4d14s-xsz, reg4; 4 bytes



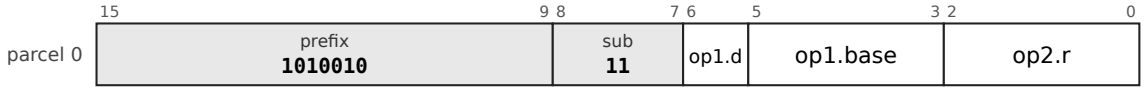
LEA: LDX8/a, prefix 11011001, sub-opcode 11; operands x4-4d8s, reg4; 4 bytes



LEA: LEAS14/a, prefix 11110010111010; operands spd14u-x8, reg4; 4 bytes



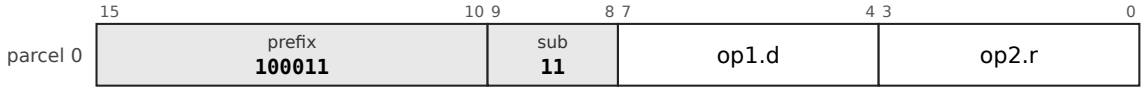
LEA: LDHHH1/a, prefix 1010010, sub-opcode 11; operands b3d1u-xsz, reg3; 2 bytes



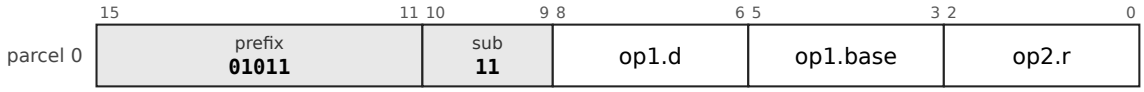
LEA: LEASP4/a, prefix 11010000; operands spd4u-x8, reg4; 2 bytes



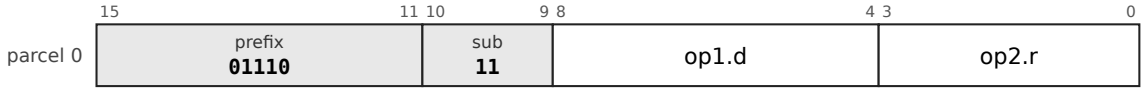
LEA: LDSP4/a, prefix 100011, sub-opcode 11; operands spd4u-xsz, reg4; 2 bytes



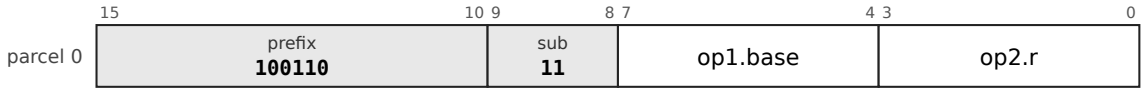
LEA: LDHHH3/a, prefix 01011, sub-opcode 11; operands b3d3u-xsz, reg3; 2 bytes



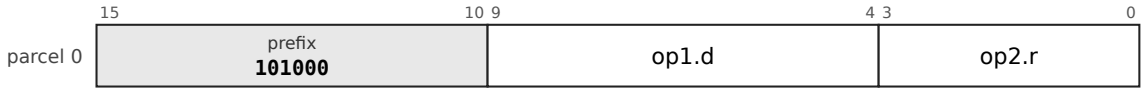
LEA: LDSP5/a, prefix 01110, sub-opcode 11; operands spd5u-xsz, reg4; 2 bytes



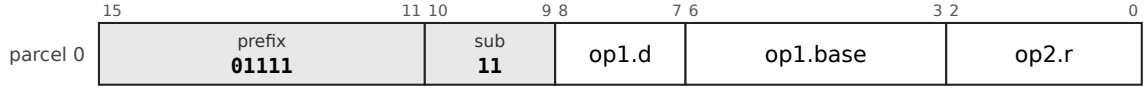
LEA: LD0/a, prefix 100110, sub-opcode 11; operands b4d0s-xsz, reg4; 2 bytes



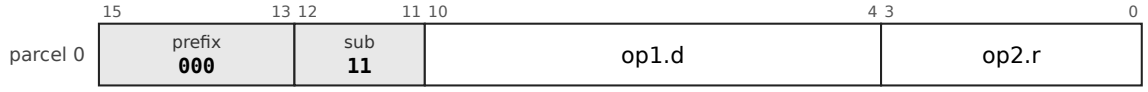
LEA: LEASP6/a, prefix 101000; operands spd6u-x8, reg4; 2 bytes



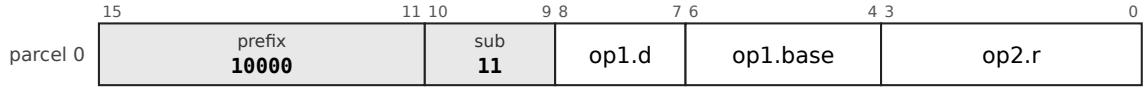
LEA: LDHRH2/a, prefix 01111, sub-opcode 11; operands b4d2u-xsz, reg3; 2 bytes



LEA: LDSP7/a, prefix 000, sub-opcode 11; operands spd7u-xsz, reg4; 2 bytes



LEA: LDHHR2/a, prefix 10000, sub-opcode 11; operands b3d2u-xsz, reg4; 2 bytes

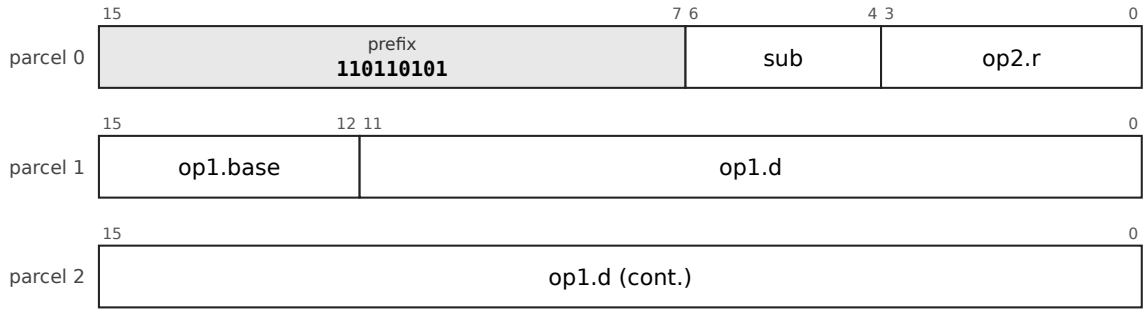


Mnemonics: `LEA`

Syntax

```
LEA d(Rn), Rd
LEA d(PC), Rd
LEA d(Rn)(Rx×s), Rd
```

Encoding



The chart is the leaf `16-LDF-B28-2`, LDF.B28, prefix `110110101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-2`, `16-LDF-X22-2`, `16-LDF-PI12-2`, `16-LDF-PD12-2`, `16-LDF-PC32-2`, `16-LDF-A32-2`, `16-DPF-D28-2`, `16-LEASx12-a-1`, `16-LDPC12-a-1`, `16-LDBs8-a-1`, `16-LDPC14-a-1`, `16-LDPC16-a-1`, `16-LDBu10-a-1`, `16-LDHHH1-a-1`, `16-LEASP4-a-1`, `16-LDSP4-a-1`, `16-LDBu12-a-1`, `16-LDHHH3-a-1`, `16-LDX6-a-1`, `16-LDBs10-a-1`, `16-LDSP5-a-1`, `16-LEAS12-a-1`, `16-LD0-a-1`, `16-LEASP6-a-1`, `16-LDBs14-a-1`, `16-LDHRH2-a-1`, `16-LDSP7-a-1`, `16-LDX8-a-1`, `16-LDHHR2-a-1`, `16-LEAS14-a-1`.

Operation

```
Rd ← EA(rm)          ; no memory access
```

Flags None written.

Exceptions The operand must name memory (a register or an immediate is not a form). No memory access is made, so the instruction takes no memory fault. A reserved encoding aborts with the illegal-instruction vector.

`LEA 0(PC), Rd` is how the program counter is read: it puts the address of the `LEA` itself in `Rd`. The base of every PC-relative displacement is the address of the first parcel of the instruction.

Why it is here. The PDP-11 and the VAX made the program counter a register (R7, R15). Here the program counter is kept out of the register file, because every register write would then be a potential branch, and a read-only view of it (R7 as a view of the PC) is refused, because it would fix for ever *which* PC is meant (ARM32 is stuck with `PC+8`). `LEA 0(PC), Rn` gives the capability without the commitment: the address of the instruction itself, in one instruction. The same instruction forms any address PC-relatively, which is what makes position-independent code free (section 9.16).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

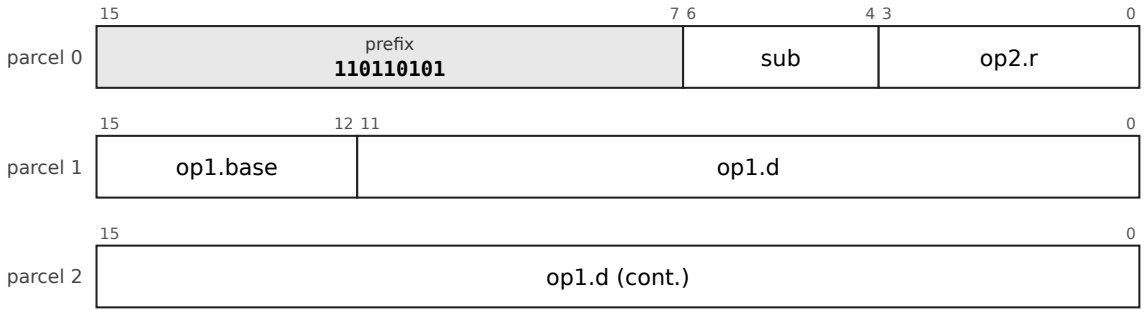
MOVU: Unaligned load and store

Mnemonics: `MOVUB`, `MOVUW`, `MOVUL`, `MOVUQ`

Syntax

```
MOVUQ rm, Rd        ; load, any alignment
MOVUQ Rs, rm        ; store, any alignment
MOVUW, MOVUL, MOVUB likewise
```

Encoding



The chart is the leaf `16-LDF-B28-2`, LDF.B28, prefix `110110101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-2`, `16-LDF-X22-2`, `16-LDF-PI12-2`, `16-LDF-PD12-2`, `16-LDF-PC32-2`, `16-LDF-A32-2`, `16-STF-B28-1`, `16-STF-X22-1`, `16-STF-PI12-1`, `16-STF-PD12-1`, `16-STF-PC32-1`, `16-STF-A32-1`, `16-DPF-D28-2`, `16-LDF-B28-3`, `16-LDF-X22-3`, `16-LDF-PI12-3`, `16-LDF-PD12-3`, `16-LDF-PC32-3`, `16-LDF-A32-3`, `16-DPF-D28-3`, `16-LDF-B28-4`, `16-LDF-X22-4`, `16-LDF-PI12-4`, `16-LDF-PD12-4`, `16-LDF-PC32-4`, `16-LDF-A32-4`, `16-DPF-D28-4`.

Operation

```
size = 1, 2, 4 or 8 bytes
load :  Rd ← sext(M[EA(rm)], size)           ; the access may cross an alignment boundary
store:  M[EA(rm)] ← Rs[8×size-1:0]
```

Flags None written.

Exceptions Page and protection faults and bus errors as for any memory operand; **no alignment fault**.

The ordinary moves fault on a misaligned address (the alignment fault is the tag check of section 9.5); these are the explicit forms for data that is not aligned.

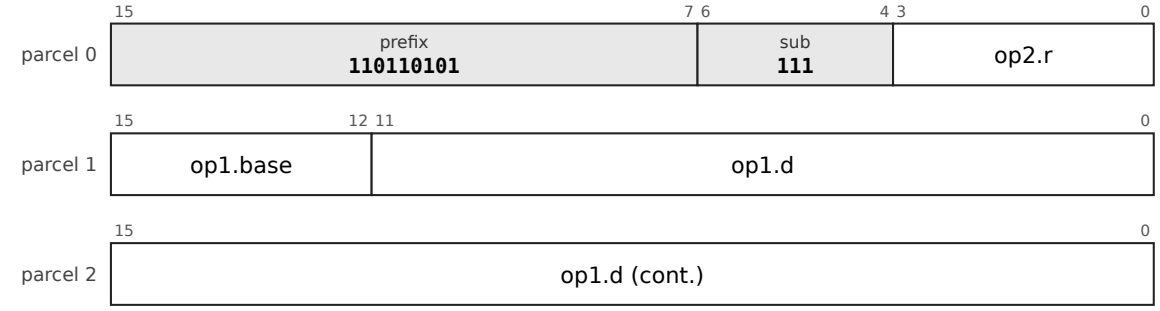
Why it is here. Data alignment is architectural on this machine (9.2.11): an ordinary quad access to an address that is not a multiple of 8 aborts, and that abort doubles as the tag check for tagged pointers; a load form that ignores low pointer bits is rejected for that reason. Packed on-disk and network structures still need unaligned access, so the architecture has explicit unaligned forms rather than weakening the rule for every load. No further reason is recorded for these four.

Cf. 6.4, 6.8.

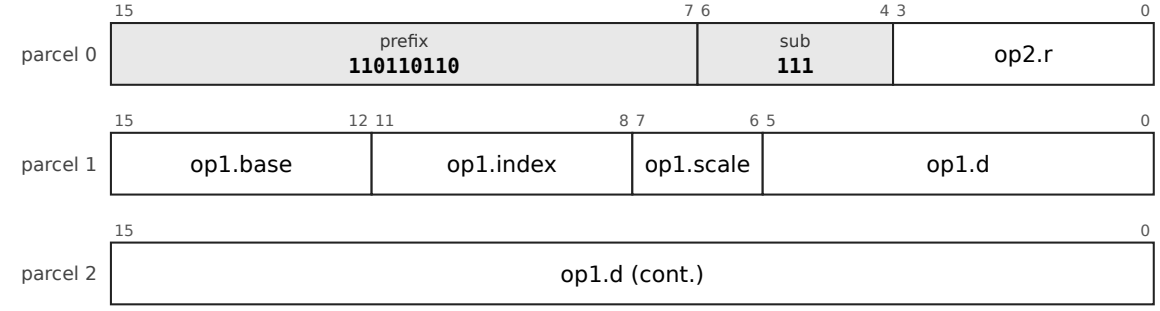
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MOVUB

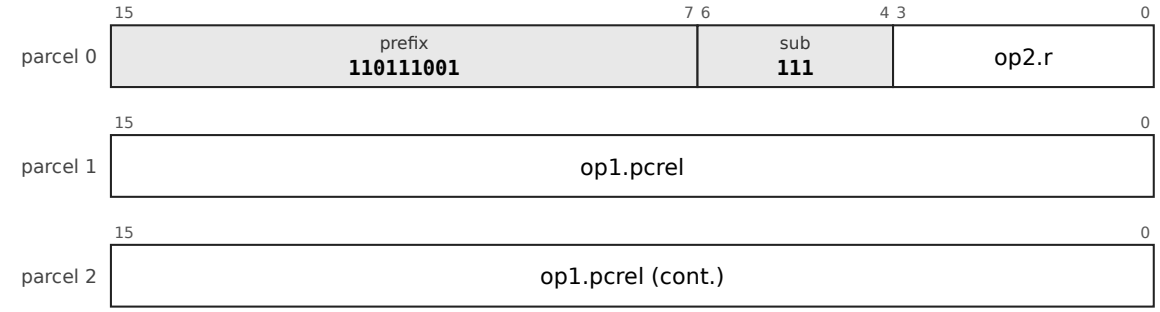
MOVUB: LDF.B28, prefix 110110101, sub-opcode 111; operands b4d28s-x1, reg4; 6 bytes



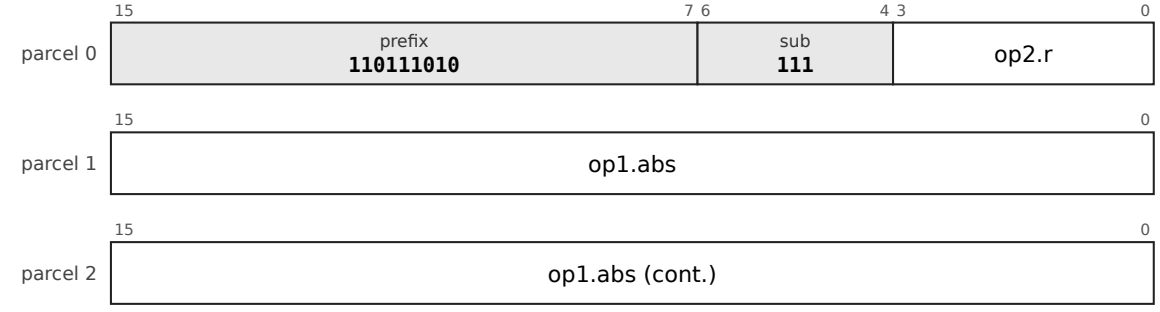
MOVUB: LDF.X22, prefix 110110110, sub-opcode 111; operands x4-4d22s, reg4; 6 bytes



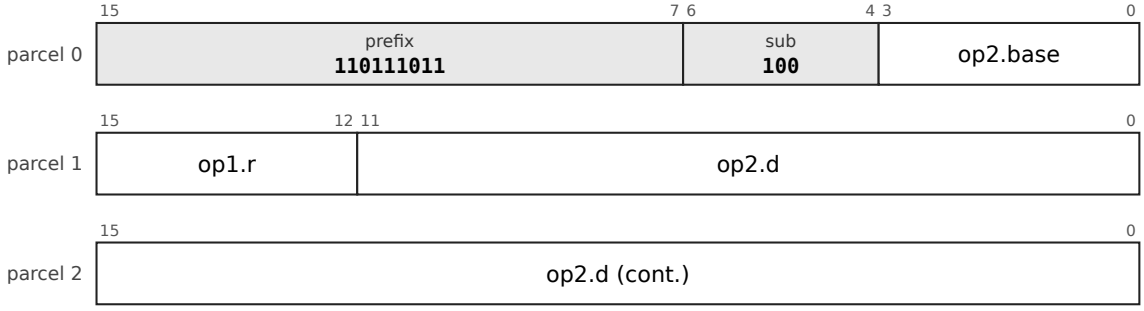
MOVUB: LDF.PC32, prefix 110111001, sub-opcode 111; operands pc32-u1, reg4; 6 bytes



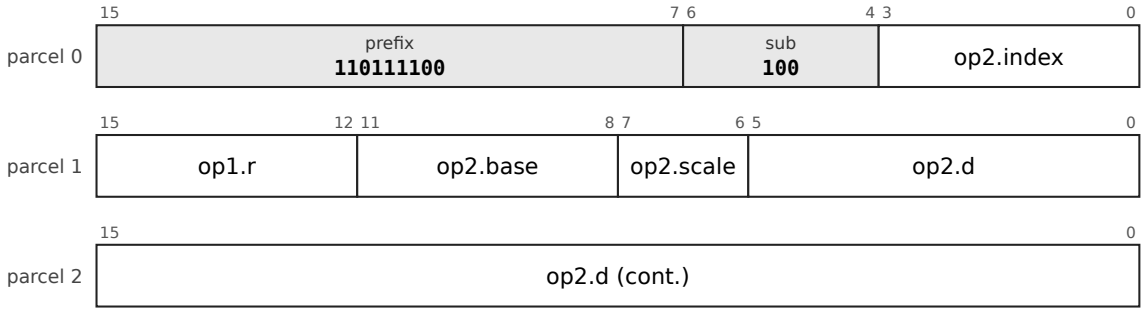
MOVUB: LDF.A32, prefix 110111010, sub-opcode 111; operands abs32, reg4; 6 bytes



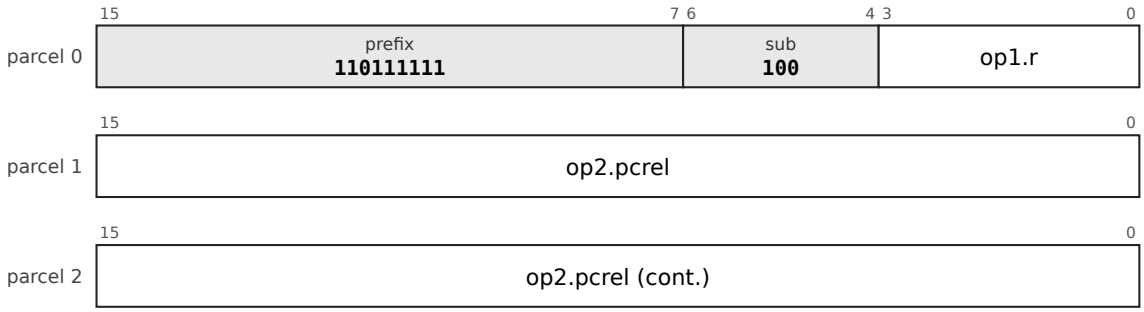
MOVUB: STF.B28, prefix 110111011, sub-opcode 100; operands reg4, b4d28s-x1; 6 bytes



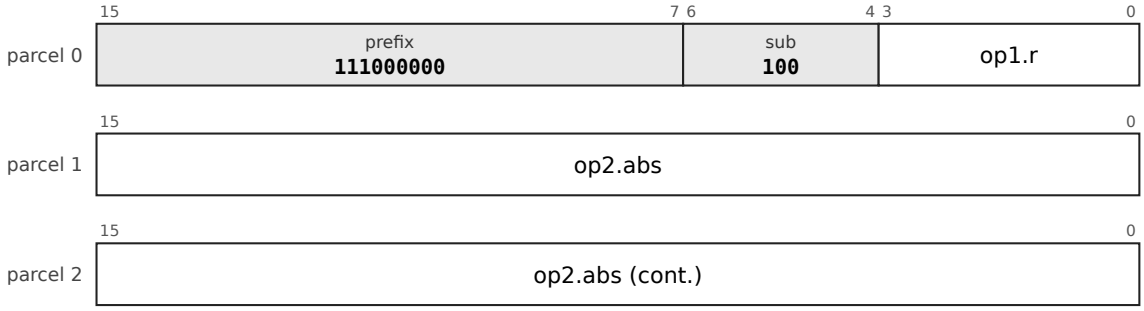
MOVUB: STF.X22, prefix 110111100, sub-opcode 100; operands reg4, x4-4d22s; 6 bytes



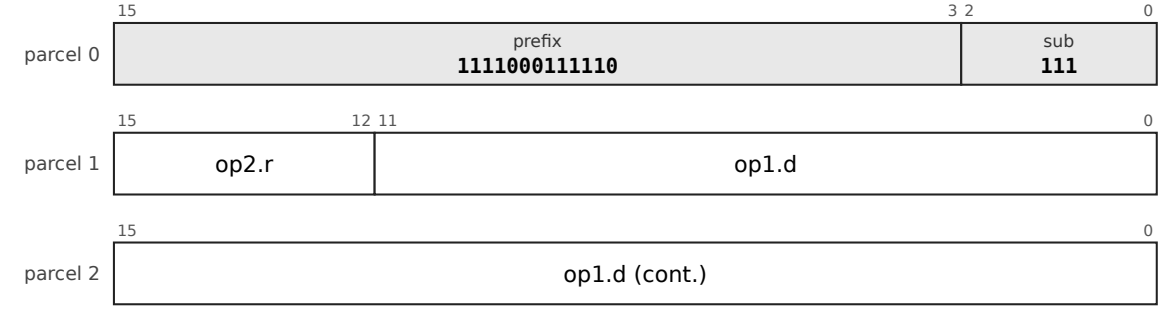
MOVUB: STF.PC32, prefix 110111111, sub-opcode 100; operands reg4, pc32-u1; 6 bytes



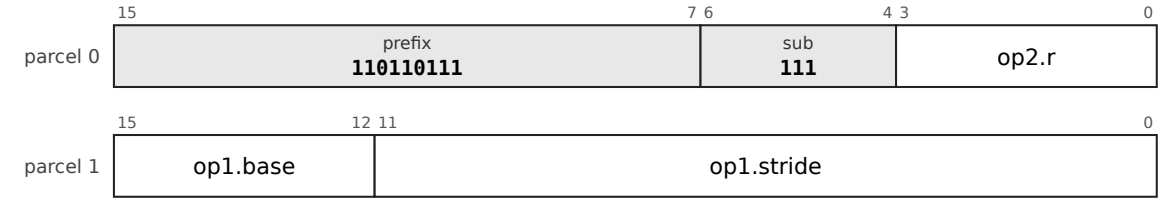
MOVUB: STF.A32, prefix 111000000, sub-opcode 100; operands reg4, abs32; 6 bytes



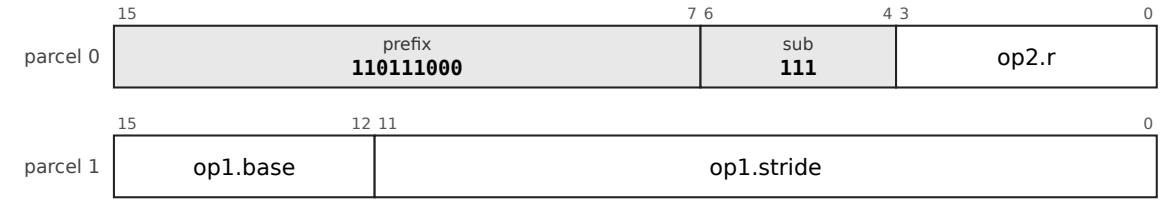
MOVUB: DPF.D28, prefix 1111000111110, sub-opcode 111; operands dpd28s-x1, reg4; 6 byte



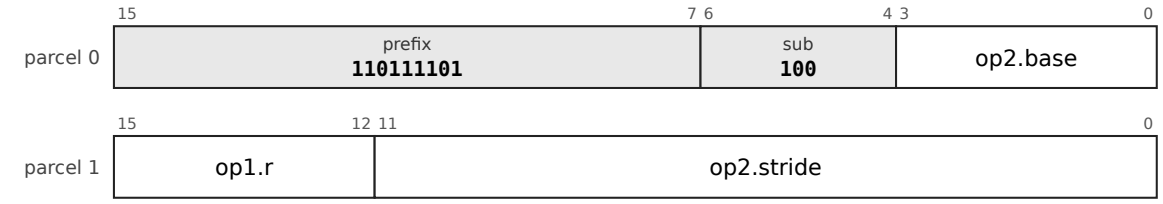
MOVUB: LDF.PI12, prefix 110110111, sub-opcode 111; operands pi4-s12, reg4; 4 bytes



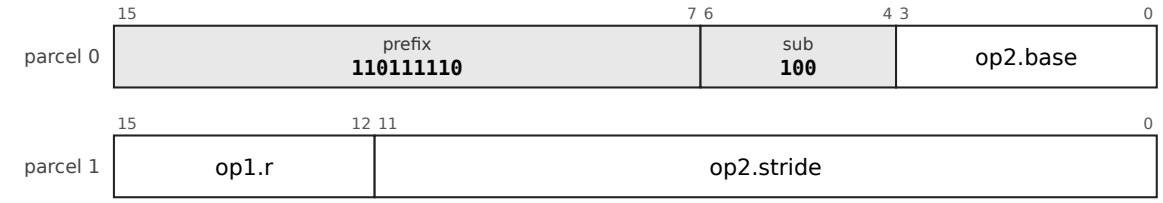
MOVUB: LDF.PD12, prefix 110111000, sub-opcode 111; operands pd4-s12, reg4; 4 bytes



MOVUB: STF.PI12, prefix 110111101, sub-opcode 100; operands reg4, pi4-s12; 4 bytes



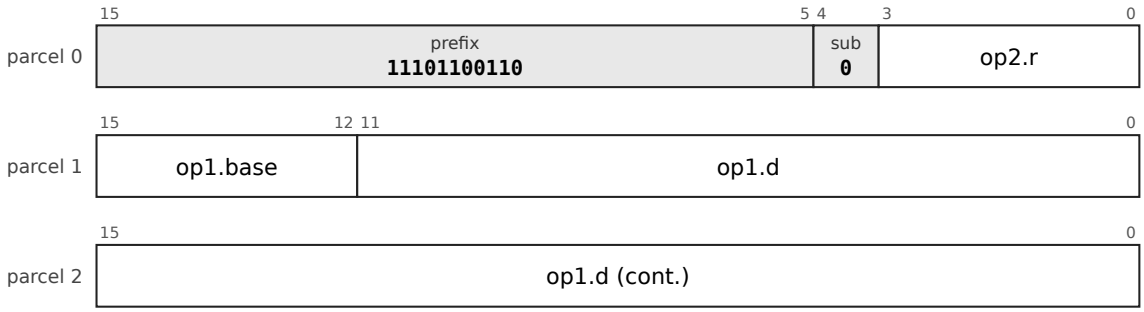
MOVUB: STF.PD12, prefix 110111110, sub-opcode 100; operands reg4, pd4-s12; 4 bytes



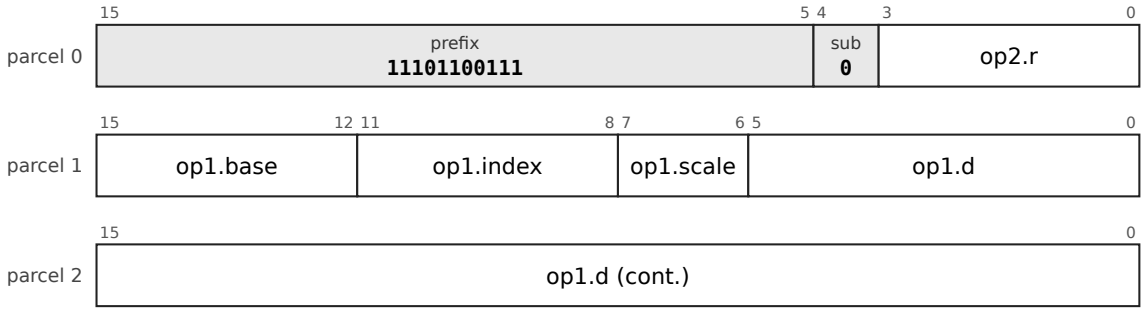
Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

MOVUW

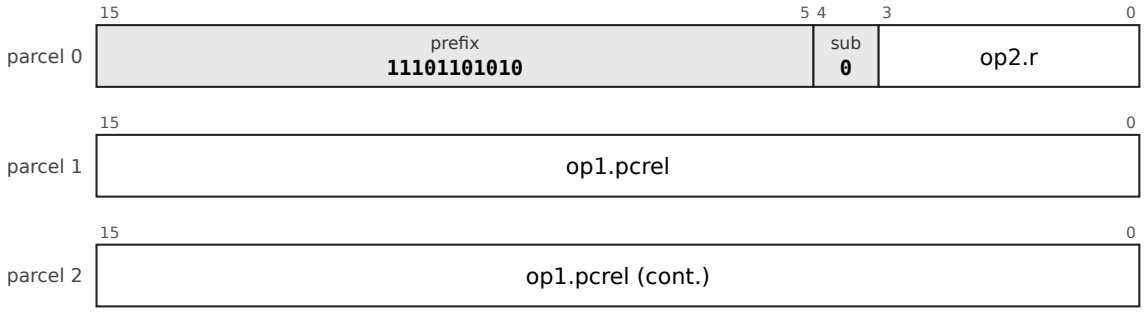
MOVUW: LDF.B28, prefix 11101100110, sub-opcode 0; operands b4d28s-x1, reg4; 6 bytes



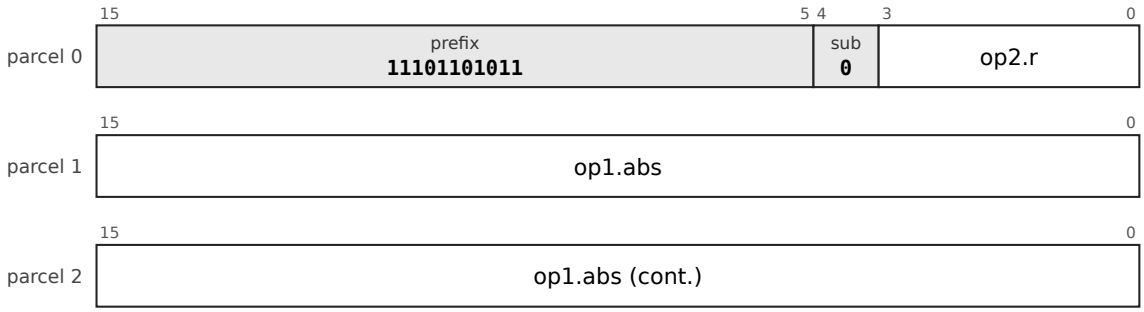
MOVUW: LDF.X22, prefix 11101100111, sub-opcode 0; operands x4-4d22s, reg4; 6 bytes



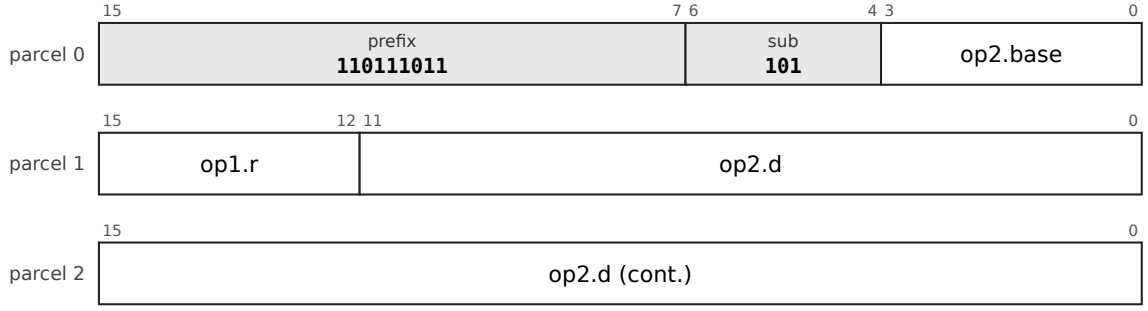
MOVUW: LDF.PC32, prefix 11101101010, sub-opcode 0; operands pc32-u1, reg4; 6 bytes



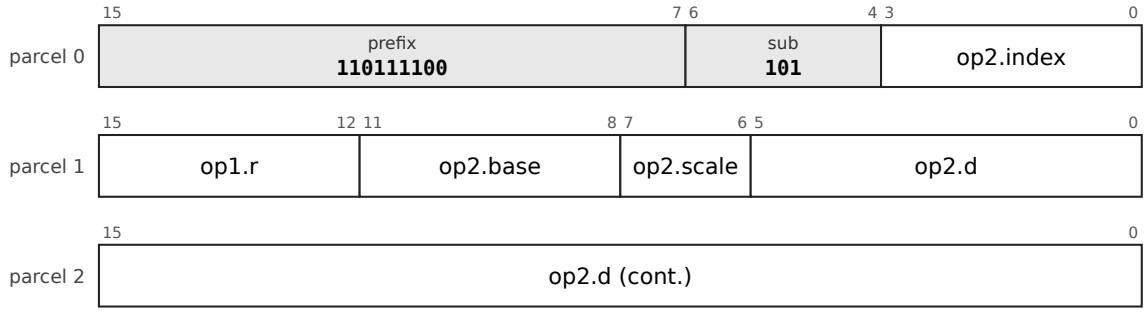
MOVUW: LDF.A32, prefix 11101101011, sub-opcode 0; operands abs32, reg4; 6 bytes



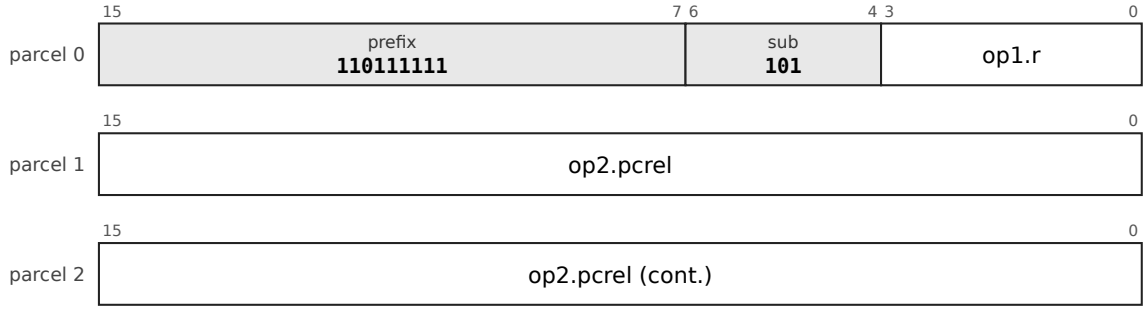
MOVUW: STF.B28, prefix 110111011, sub-opcode 101; operands reg4, b4d28s-x1; 6 bytes



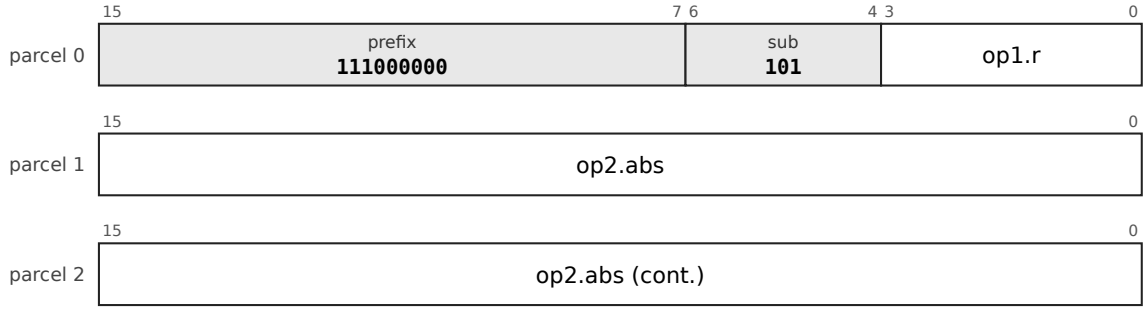
MOVUW: STF.X22, prefix 110111100, sub-opcode 101; operands reg4, x4-4d22s; 6 bytes



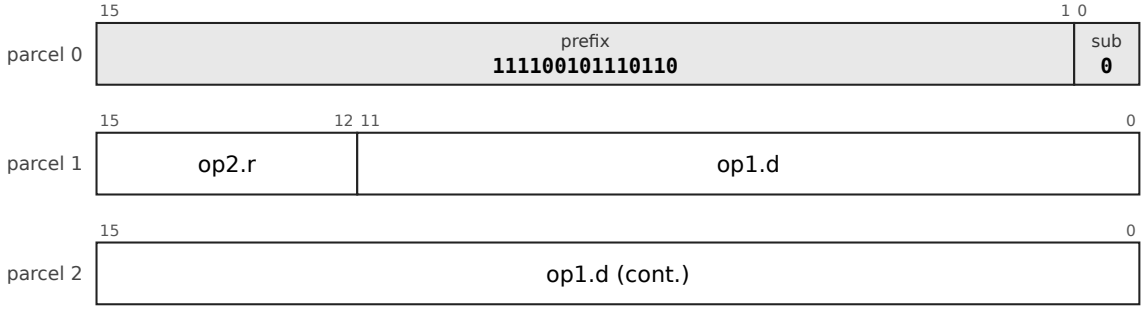
MOVUW: STF.PC32, prefix 110111111, sub-opcode 101; operands reg4, pc32-u1; 6 bytes



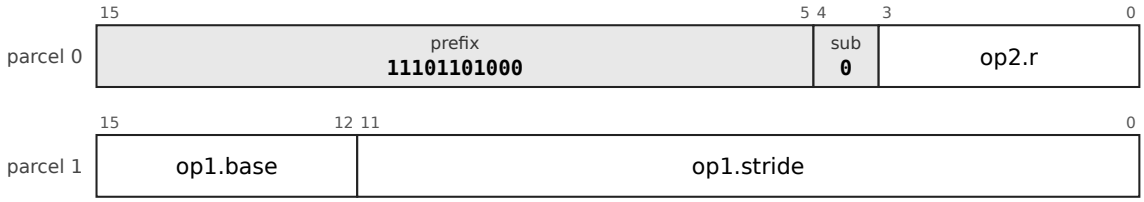
MOVUW: STF.A32, prefix 111000000, sub-opcode 101; operands reg4, abs32; 6 bytes



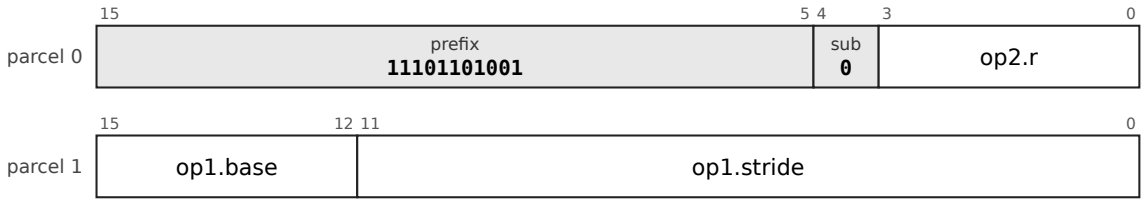
MOVUW: DPF.D28, prefix 111100101110110, sub-opcode 0; operands dpd28s-x1, reg4; 6 byte



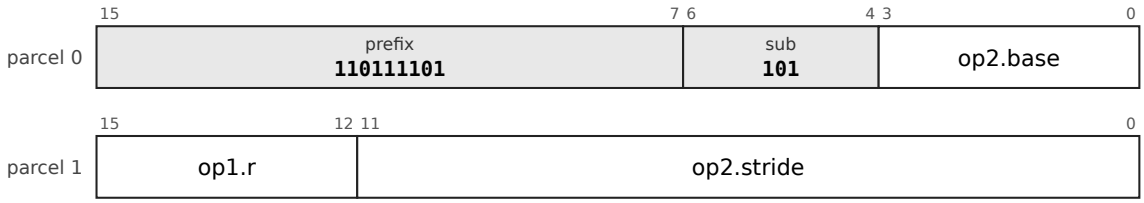
MOVUW: LDF.PI12, prefix 11101101000, sub-opcode 0; operands pi4-s12, reg4; 4 bytes



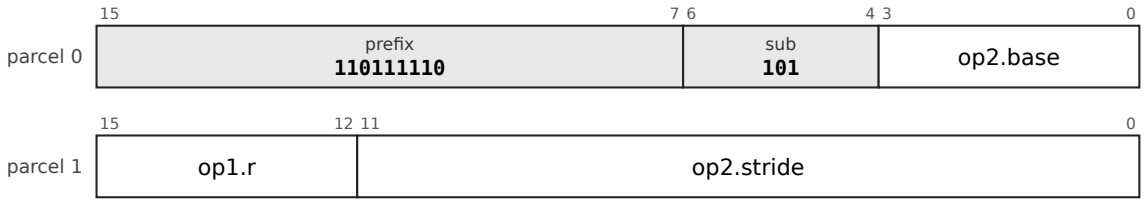
MOVUW: LDF.PD12, prefix 11101101001, sub-opcode 0; operands pd4-s12, reg4; 4 bytes



MOVUW: STF.PI12, prefix 110111101, sub-opcode 101; operands reg4, pi4-s12; 4 bytes



MOVUW: STF.PD12, prefix 110111110, sub-opcode 101; operands reg4, pd4-s12; 4 bytes

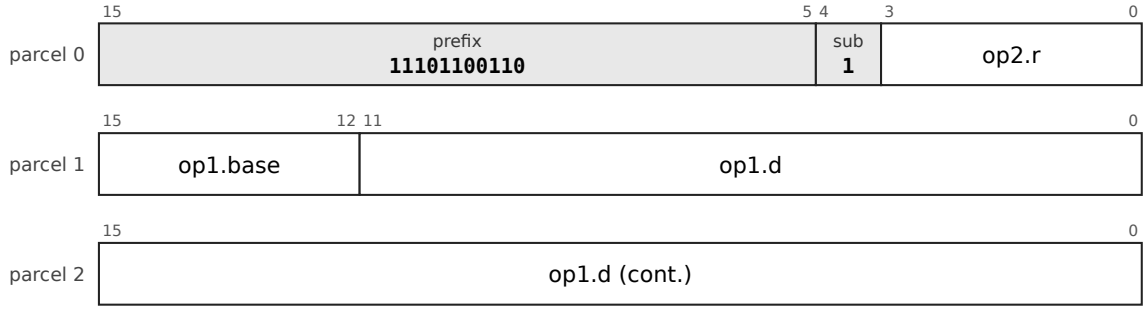



```
MOVUW, MOVUL, MOVUB likewise
```

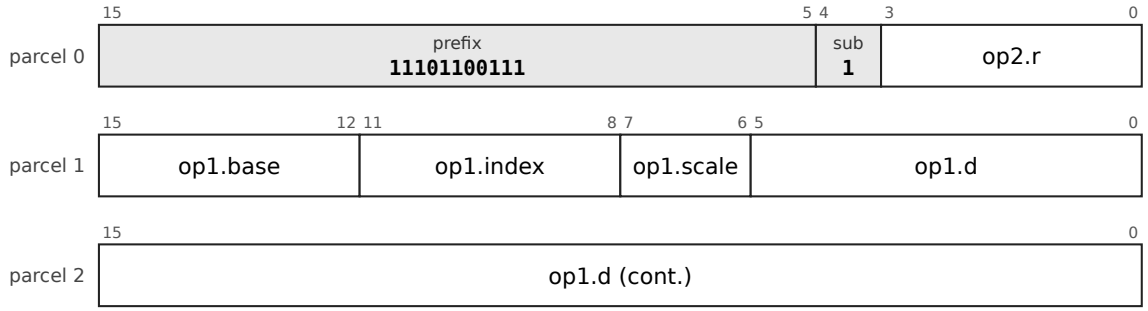
Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

MOVUL

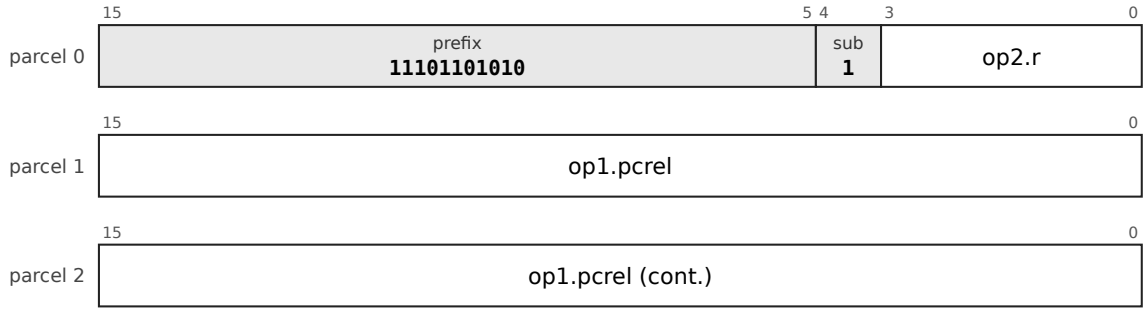
MOVUL: LDF.B28, prefix 11101100110, sub-opcode 1; operands b4d28s-x1, reg4; 6 bytes



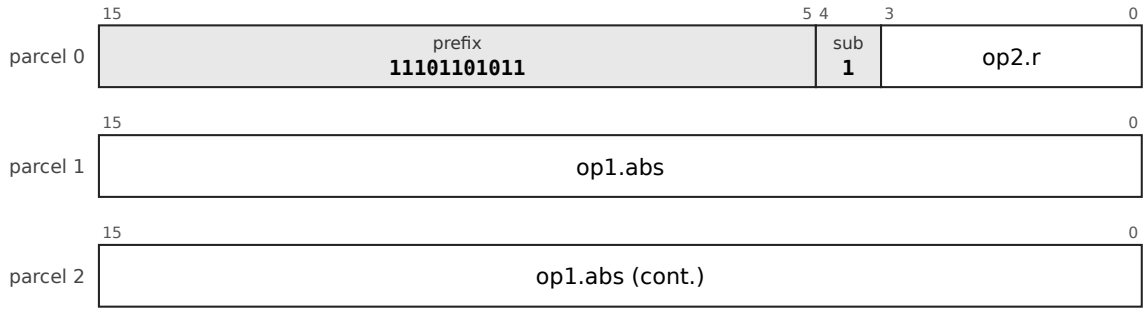
MOVUL: LDF.X22, prefix 11101100111, sub-opcode 1; operands x4-4d22s, reg4; 6 bytes



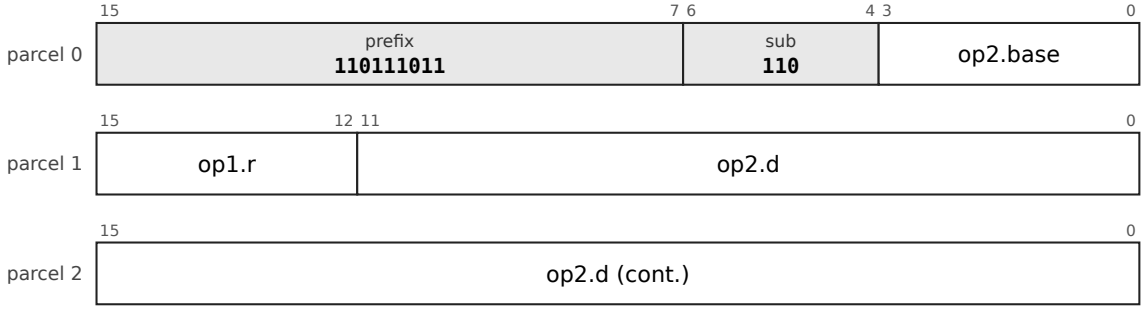
MOVUL: LDF.PC32, prefix 11101101010, sub-opcode 1; operands pc32-u1, reg4; 6 bytes



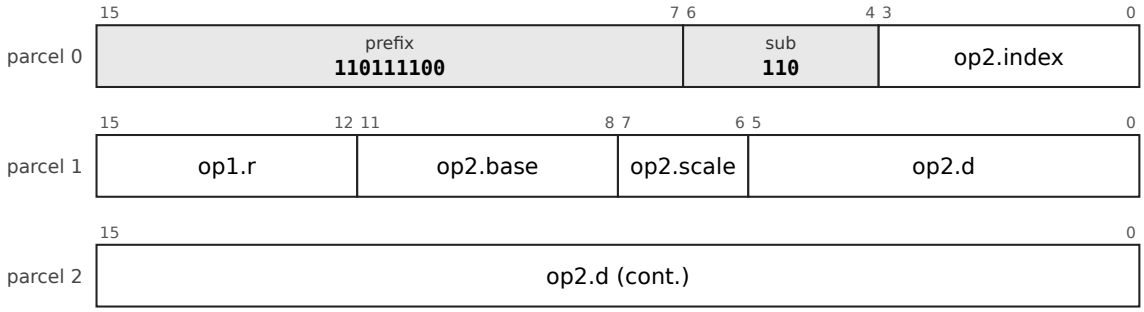
MOVUL: LDF.A32, prefix 11101101011, sub-opcode 1; operands abs32, reg4; 6 bytes



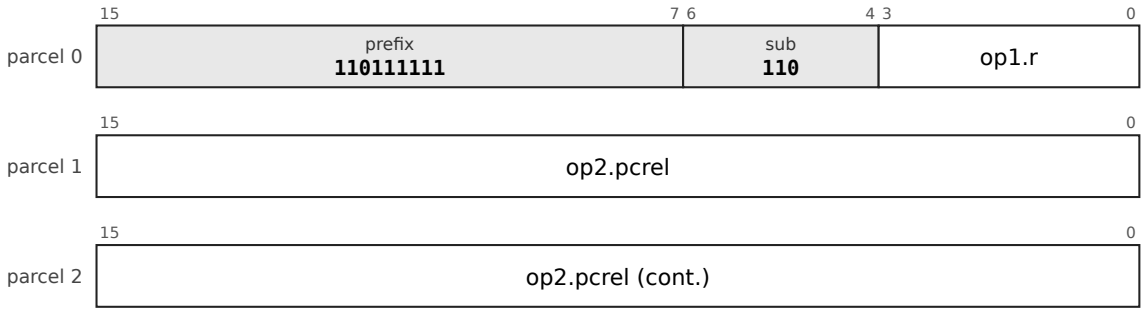
MOVUL: STF.B28, prefix 110111011, sub-opcode 110; operands reg4, b4d28s-x1; 6 bytes



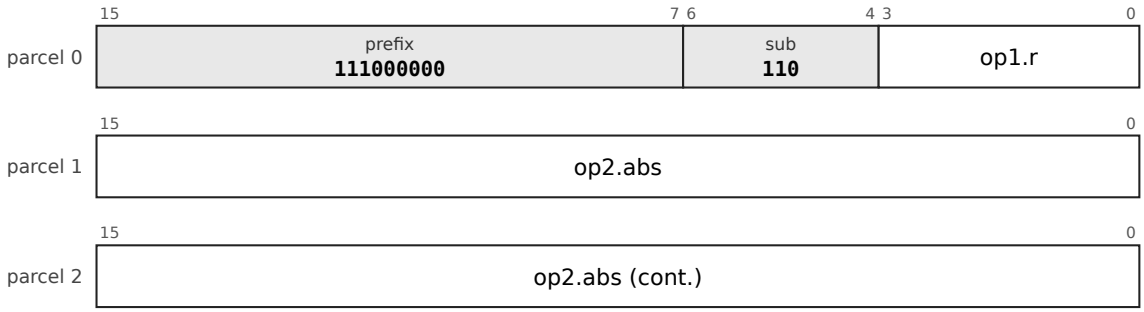
MOVUL: STF.X22, prefix 110111100, sub-opcode 110; operands reg4, x4-4d22s; 6 bytes



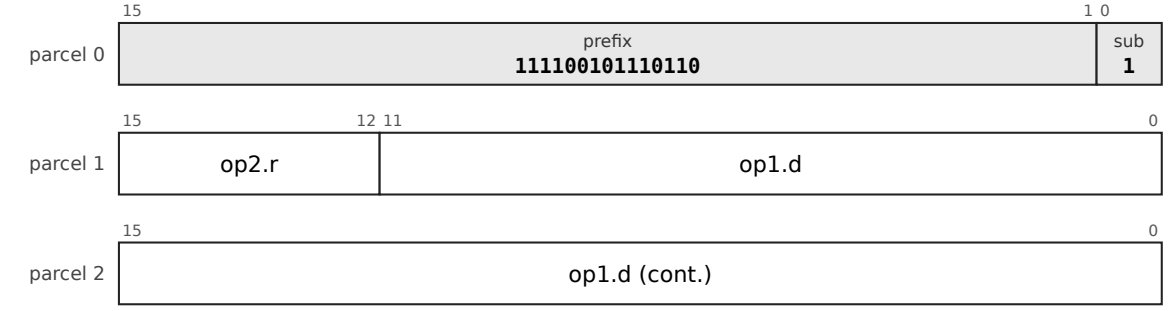
MOVUL: STF.PC32, prefix 110111111, sub-opcode 110; operands reg4, pc32-u1; 6 bytes



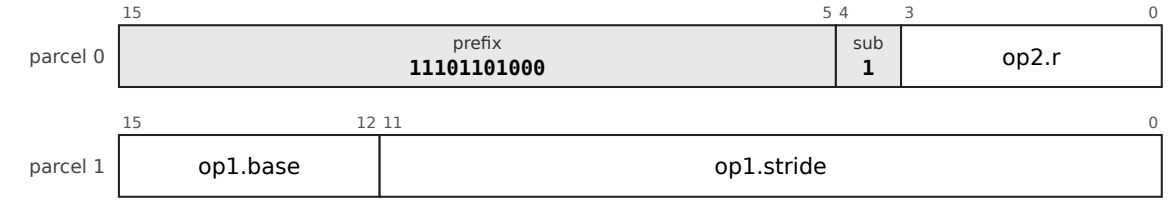
MOVUL: STF.A32, prefix 111000000, sub-opcode 110; operands reg4, abs32; 6 bytes



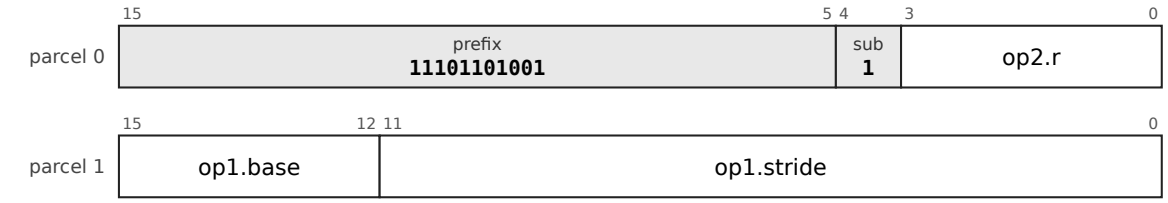
MOVUL: DPF.D28, prefix 111100101110110, sub-opcode 1; operands dpd28s-x1, reg4; 6 byte



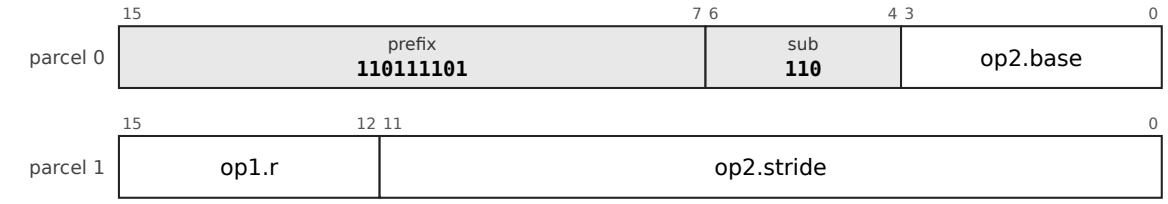
MOVUL: LDF.PI12, prefix 11101101000, sub-opcode 1; operands pi4-s12, reg4; 4 bytes



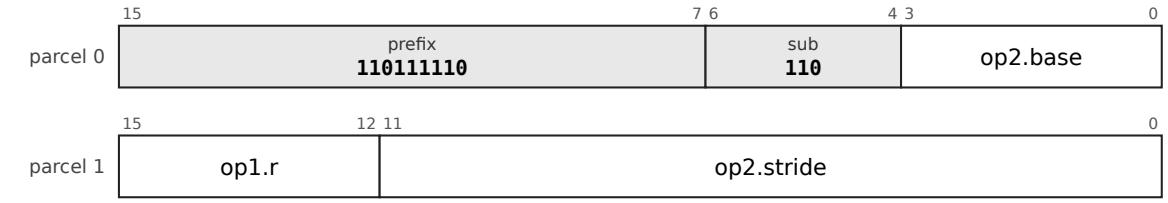
MOVUL: LDF.PD12, prefix 11101101001, sub-opcode 1; operands pd4-s12, reg4; 4 bytes



MOVUL: STF.PI12, prefix 110111101, sub-opcode 110; operands reg4, pi4-s12; 4 bytes



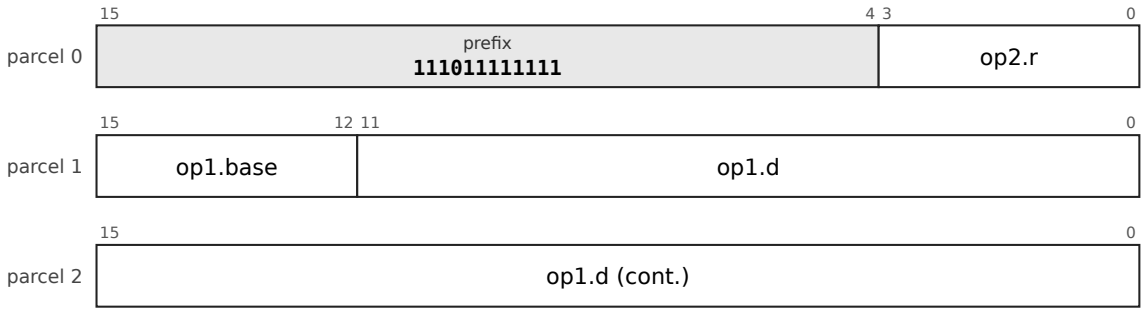
MOVUL: STF.PD12, prefix 110111110, sub-opcode 110; operands reg4, pd4-s12; 4 bytes



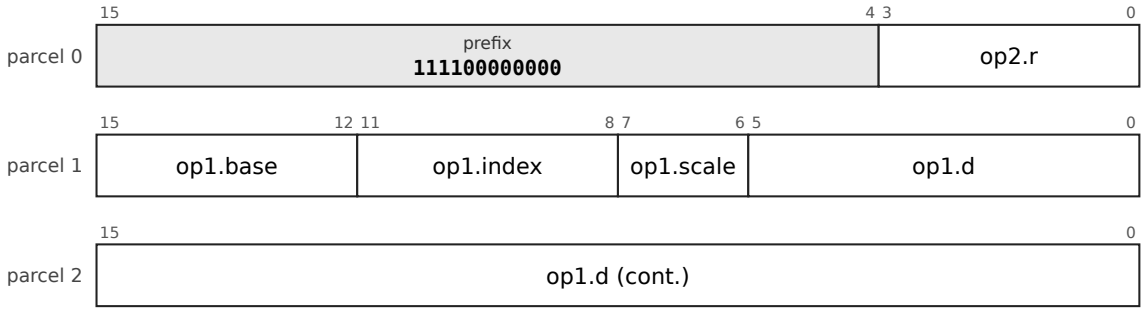
Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

MOVUQ

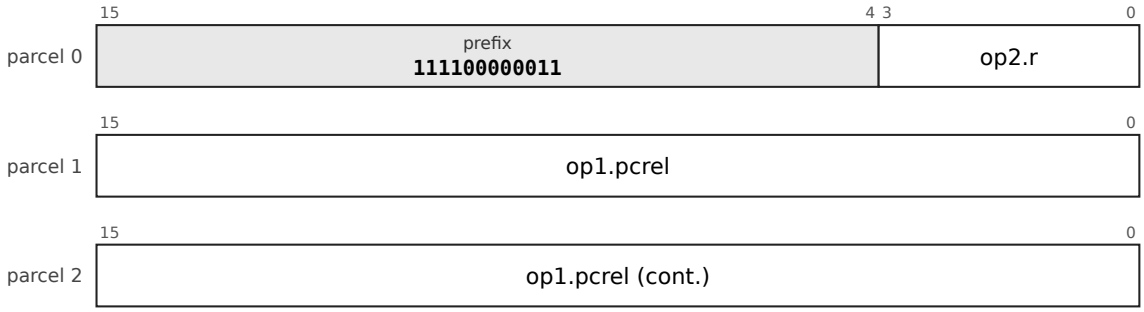
MOVUQ: LDF.B28, prefix 111011111111; operands b4d28s-x1, reg4; 6 bytes



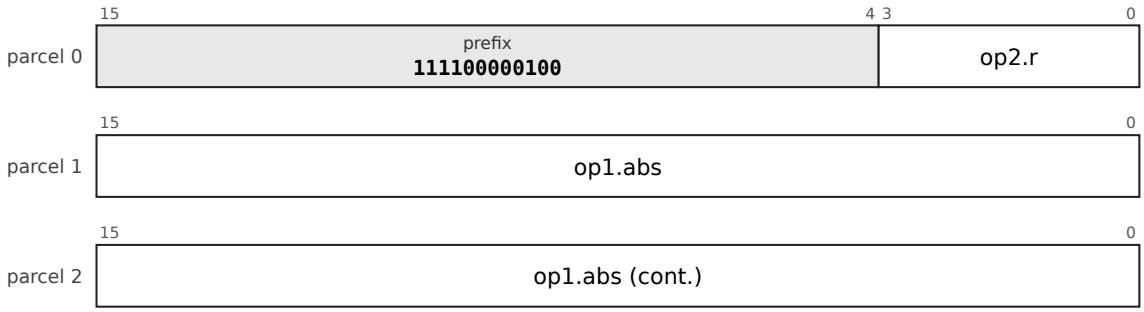
MOVUQ: LDF.X22, prefix 111100000000; operands x4-4d22s, reg4; 6 bytes



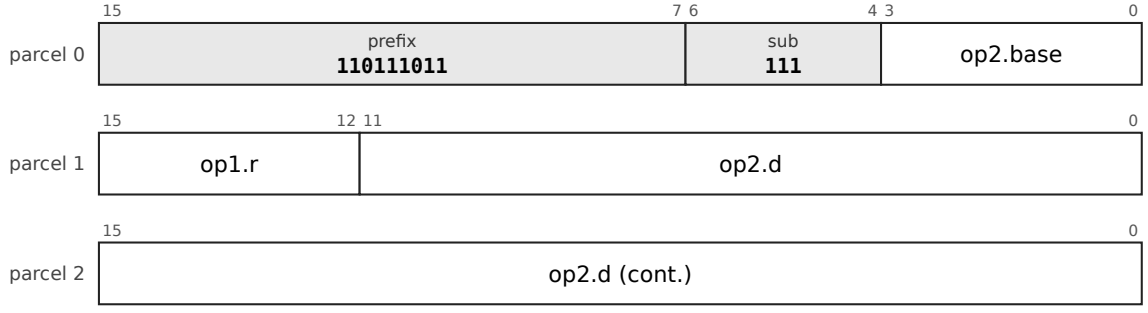
MOVUQ: LDF.PC32, prefix 111100000011; operands pc32-u1, reg4; 6 bytes



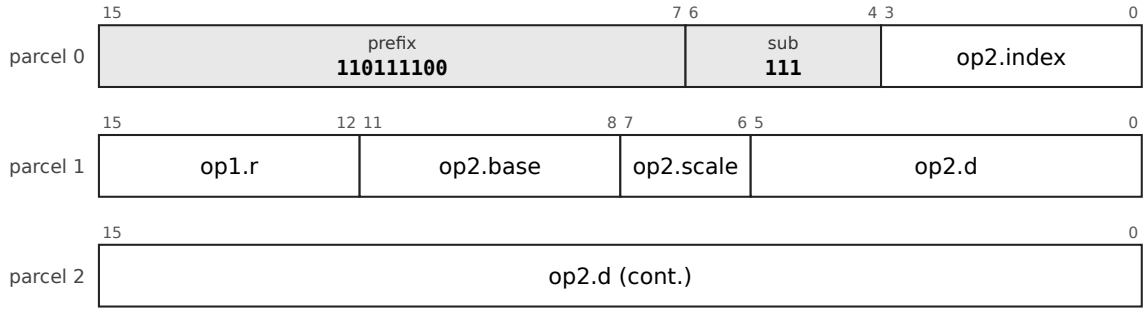
MOVUQ: LDF.A32, prefix 111100000100; operands abs32, reg4; 6 bytes



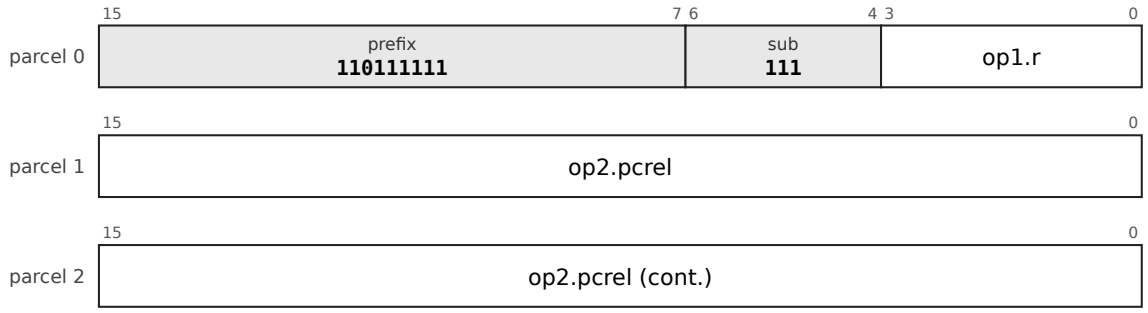
MOVUQ: STF.B28, prefix 110111011, sub-opcode 111; operands reg4, b4d28s-x1; 6 bytes



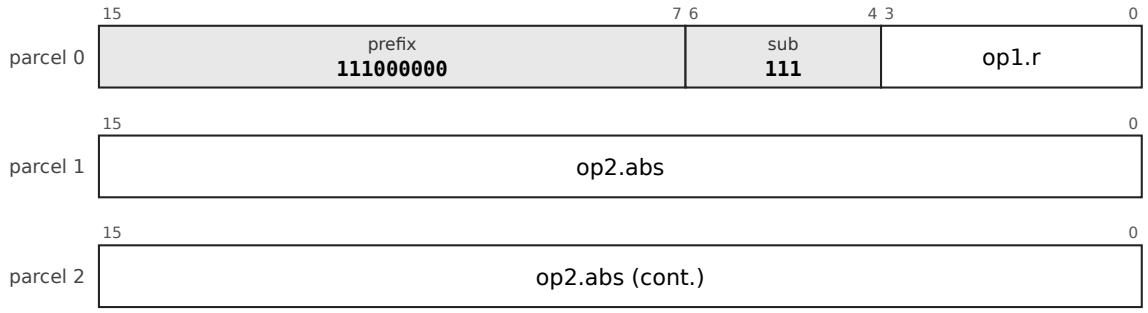
MOVUQ: STF.X22, prefix 110111100, sub-opcode 111; operands reg4, x4-4d22s; 6 bytes



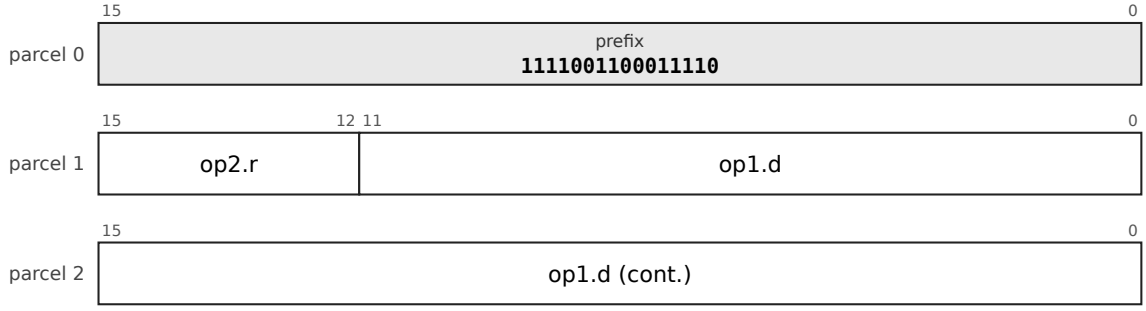
MOVUQ: STF.PC32, prefix 110111111, sub-opcode 111; operands reg4, pc32-u1; 6 bytes



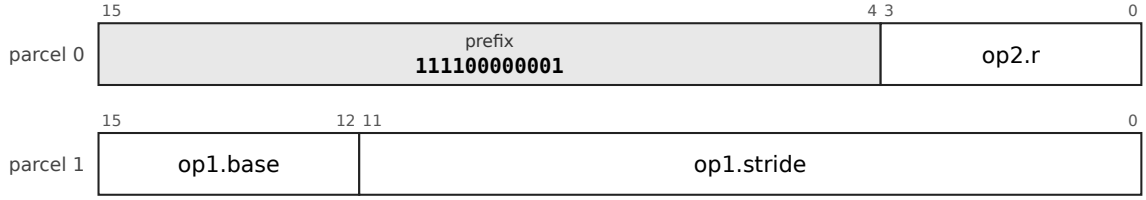
MOVUQ: STF.A32, prefix 111000000, sub-opcode 111; operands reg4, abs32; 6 bytes



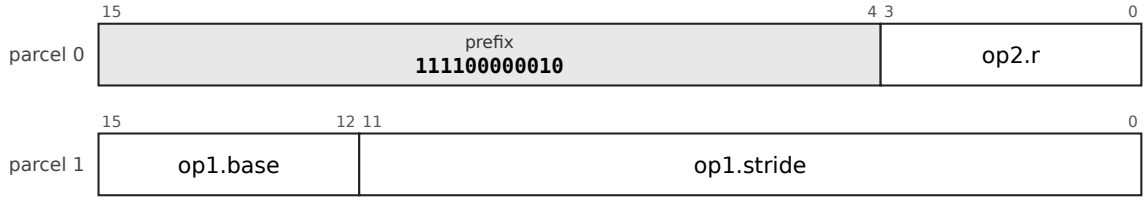
MOVUQ: DPF.D28, prefix 1111001100011110; operands dpd28s-x1, reg4; 6 bytes



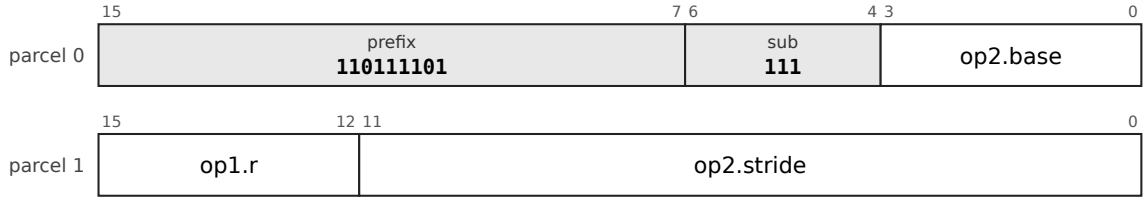
MOVUQ: LDF.PI12, prefix 111100000001; operands pi4-s12, reg4; 4 bytes



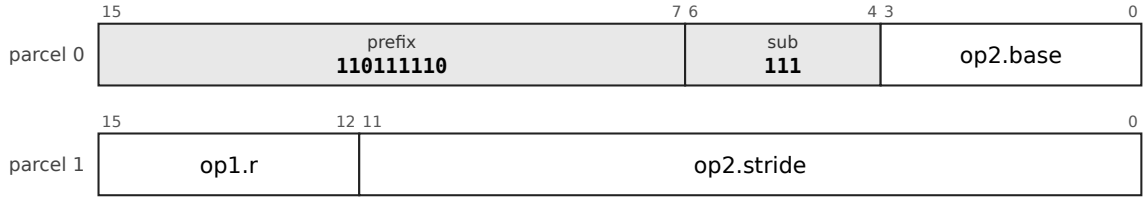
MOVUQ: LDF.PD12, prefix 111100000010; operands pd4-s12, reg4; 4 bytes



MOVUQ: STF.PI12, prefix 110111101, sub-opcode 111; operands reg4, pi4-s12; 4 bytes



MOVUQ: STF.PD12, prefix 110111110, sub-opcode 111; operands reg4, pd4-s12; 4 bytes

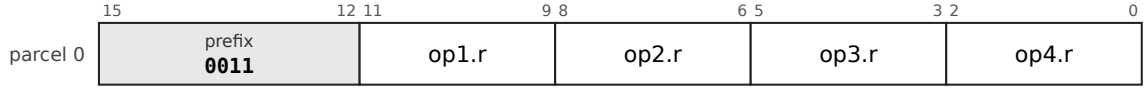



```
MOVUQ rm, Rd
MOVUQ Rs, rm
```

Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

MOV2: Two register moves

MOV2: FX.MOV2, prefix 0011; operands reg3, reg3, reg3, reg3; 2 bytes

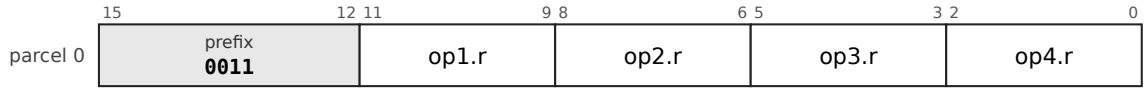


Mnemonics: `MOV2`

Syntax

```
MOV2 Ra, Rb, Rc, Rd
```

Encoding



The chart is the leaf `16-FX-MOV2-1`, FX.MOV2, prefix `0011`. `op1` is `reg3`: register, 3 bits (an index into the hot-register table). `op2` is `reg3`: register, 3 bits (an index into the hot-register table). `op3` is `reg3`: register, 3 bits (an index into the hot-register table). `op4` is `reg3`: register, 3 bits (an index into the hot-register table).

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-FX-MOV2-1`.

Operation

```
Rb ← Ra
Rd ← Rc          ; in this order
```

Flags None written.

Exceptions None.

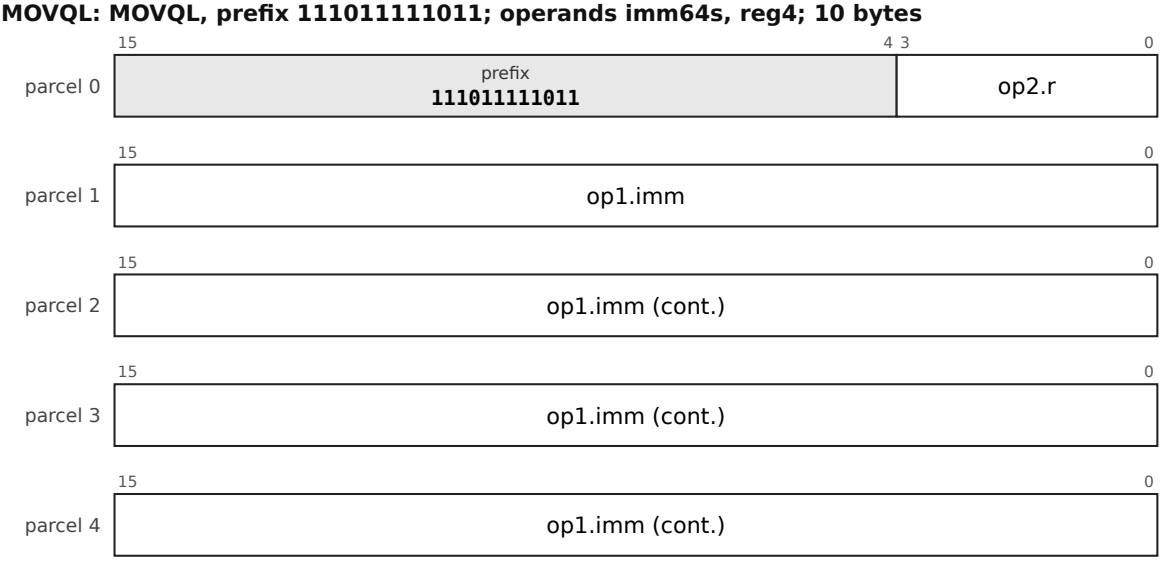
(D) `MOVQ Ra, Rb` followed by `MOVQ Rc, Rd`, fused (errata 87). The four operands are 3-bit fields: only the eight hot registers (`R0 R1 R8 R2 SP R3 R9 R10`, the ABI table of the encoding) can be named, and the instruction takes one sixteenth of the first-parcel space, which is why the pdp64/16 and pdp64/32 grammars give it a prefix of four bits. If `Rb` is `Rc` the second move reads the new value, as two instructions would.

Why it is here. The author decided the four fused pairs in errata 87 (`movcall`, `popmret`, `popret`, `mov2`) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of `docs/encoding-proposal.md` section 8 (static size of the training corpus, in pdp64/16 and pdp64/32): `MOVQ Ra, Rb` followed by `MOVQ Rc, Rd` occurs 6,742 times and saves 0.61% (pdp64/16) and 0.50% (pdp64/32), the smallest gain of the four and the dearest in code space: four 3-bit register fields use one sixteenth of the first-parcel space and can name only the eight hot registers. The proposal lists both as costs, and one instruction that writes two registers (the rename width of an out-of-order core).

Cf. Appendix D (the hot-register tables).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MOVQL: Move a 64-bit literal

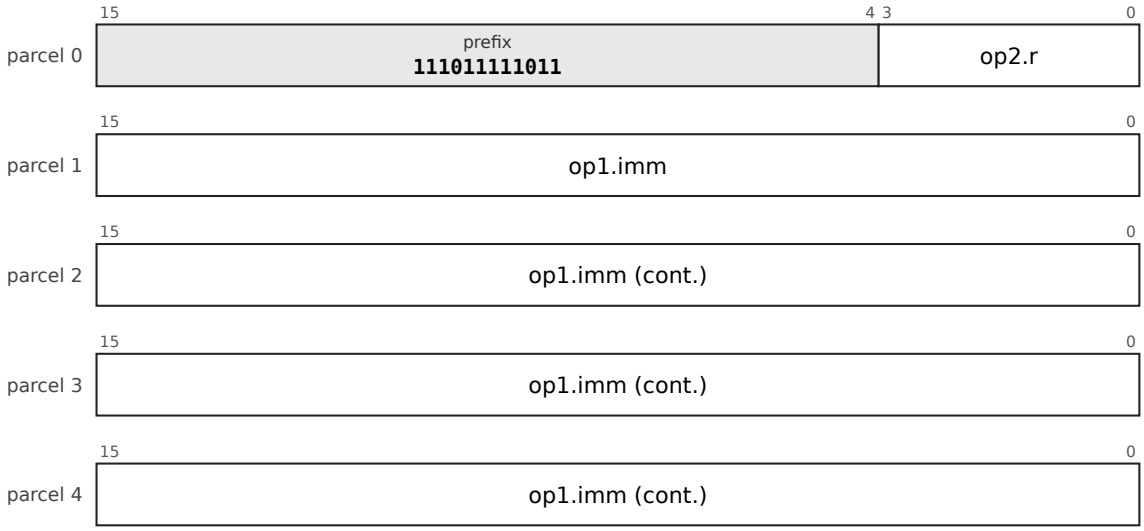


Mnemonics: `MOVQL`

Syntax

```
MOVQL #imm64, Rd
```

Encoding



The chart is the leaf `16-MOVQL-1`, MOVQL, prefix `111011111011`. `op1` is `imm64s`: signed 64-bit immediate, -9223372036854775808 to 9223372036854775807. `op2` is `reg4`: register, 4 bits.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-MOVQL-1`.

Operation

```
Rd ← imm64
```

Flags None written.

Exceptions None.

(D) Errata 85: `MOVQL #imm64, Rd` is accepted, and breaking the three-parcel rule for it is acceptable because a 64-bit literal is not counted as part of the instruction. The instruction is five parcels, ten bytes: the literal is inside the instruction, so the length is still known from parcel 0 and the fetch has no exception for a trailing literal (`docs/encoding-proposal.md` section 3). The assembler uses it for a `MOVQ #imm, Rd` that fits no `MOVQ` form (errata 87, assumption 6).

Why it is here. The author accepted it in the second round of security answers (errata 85, proposal P18): "we can do the same trick for 32 bit immediates too", the author added, which the grammars have not taken up. A 64-bit constant otherwise comes from the literal pool; the literal inside the instruction is one instruction, and it is not counted against the three-parcel rule because it is data, not instruction. The encoding work found no use of a trailing literal worth its exception to the fetch rule, so the literal is part of a five-parcel instruction (`docs/encoding-proposal.md` section 3).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

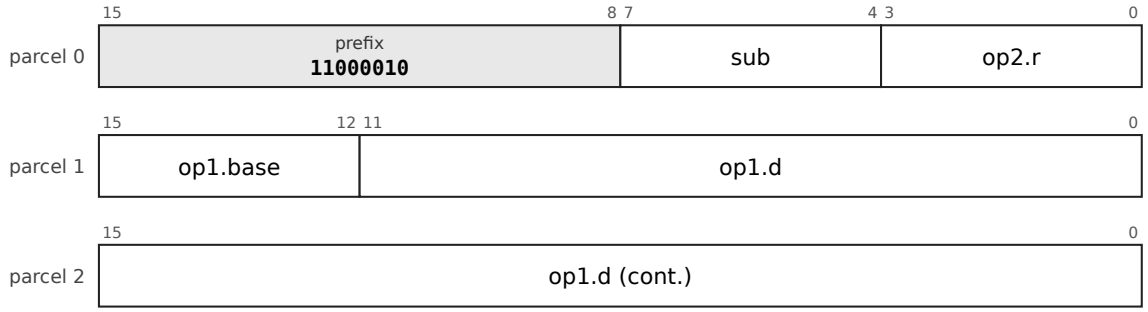
MOVD: Deferred-fault load

Mnemonics: `MOVQ.D`, `MOVL.D`, `MOVW.D`, `MOVB.D`, `MOVZL.D`, `MOVZW.D`, `MOVZB.D`

Syntax

```
MOVQ.D d(Rn), Rd
MOVL.D ..., MOVW.D ..., MOVB.D ...
MOVZL.D ..., MOVZW.D ..., MOVZB.D ...
```

Encoding



The chart is the leaf `16-LDU-B28-1`, LDU.B28, prefix `11000010`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDU-B28-1`, `16-LDU-B12-1`, `16-LDU-X6-1`.

Operation

```
Rd ← M[EA]                    ; as the load without .D, when the access succeeds
when the access faults: the fault is precise as always; the kernel's handler repairs the cause and restarts,
    or, if the access is invalid, marks Rd as poisoned in the saved state and goes on at the next
instruction
```

Flags None written.

Exceptions The ordinary faults of a load (page fault, protection fault, alignment fault, bus error), precise; what the handler does with them is the difference.

(P) The deferred-fault load is item 10 of `docs/study-proposals.md` (the poison bit); the author accepted its three leanings (errata 86: the fault is taken at the instruction that writes the flags from a poisoned source; the kernel decides a deferred load's fault; the kernel dispatcher, not the hardware, tests the argument registers for poison), but the poison bit itself is a proposal, so

the semantics above are *(P)*. The proposal spells the load `MOVQ.s`; the grammars spell it `.D`. Base and indexed operands only (a mode that needs a base register needs `LEA` first). **Not implemented** in the reference executor: it traps as an unimplemented instruction.

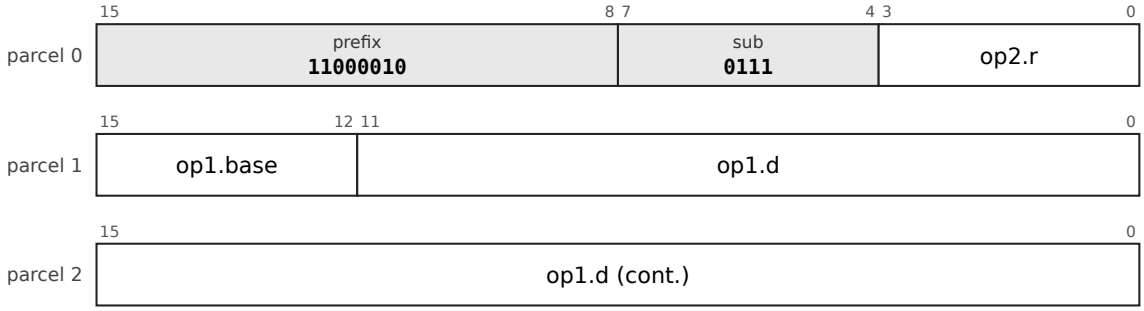
Why it is here. The poison bit of item 10 of the study (`docs/study-proposals.md`) wants a load that can be hoisted above its guard: it either gives a value or a poisoned value, and a use of a poisoned value is the fault the guard would have prevented. That removes the recovery code of if-conversion and software pipelining, gives a deterministic Spectre version 1 form (a deferred, bounds-checked address computation yields poison, not a branch the predictor can train), and makes a chain of checks one fault. The author accepted the three leanings of the proposal (errata 86); the record does not show the author deciding the poison bit itself, so the instruction is a proposal.

Cf. 6.2, docs/study-proposals.md item 10.

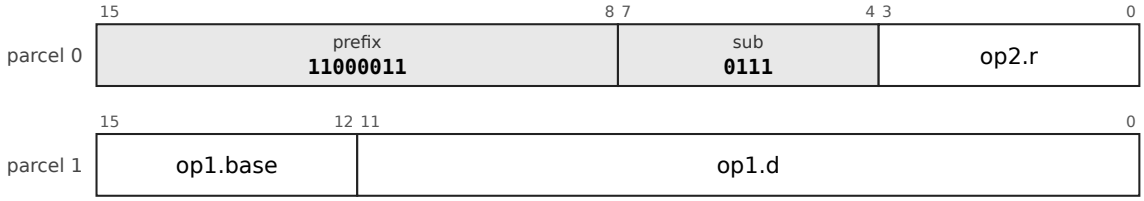
Status specified; **not yet implemented** in the reference executor.

MOVQ.D

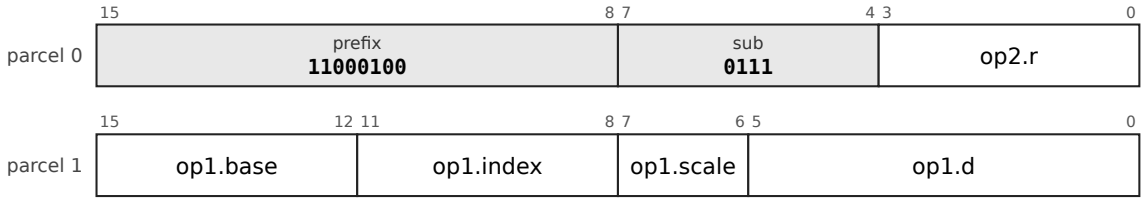
MOVQ.D: LDU.B28, prefix 11000010, sub-opcode 0111; operands b4d28s-x1, reg4; 6 bytes



MOVQ.D: LDU.B12, prefix 11000011, sub-opcode 0111; operands b4d12s-x1, reg4; 4 bytes



MOVQ.D: LDU.X6, prefix 11000100, sub-opcode 0111; operands x4-4d6s, reg4; 4 bytes

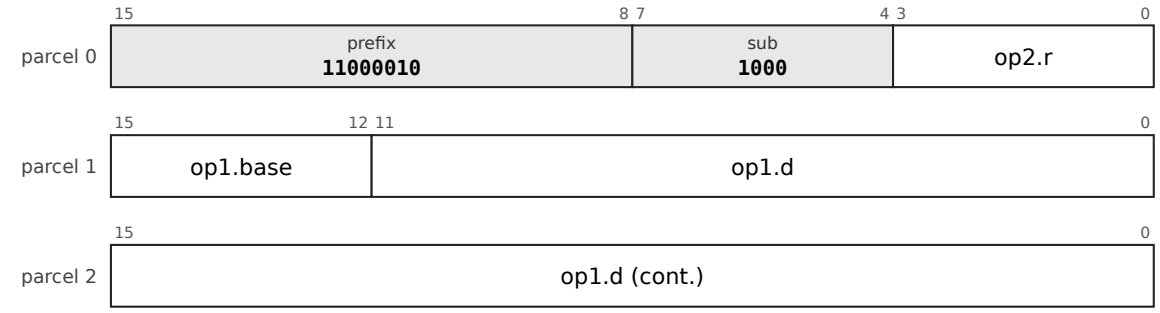



```
MOVQ.D d(Rn), Rd
```

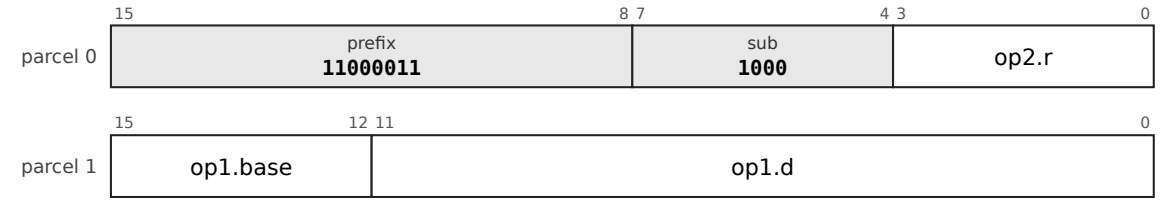
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVL.D

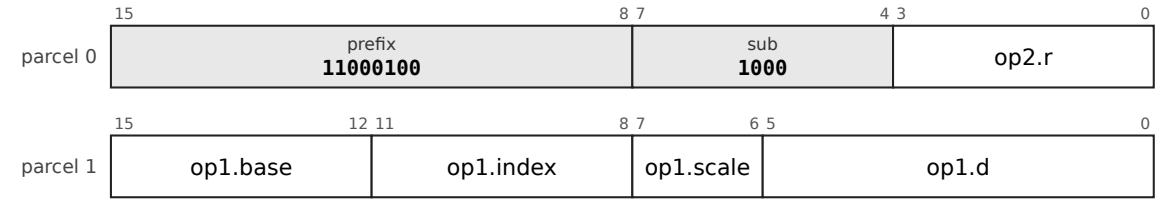
MOVL.D: LDU.B28, prefix 11000010, sub-opcode 1000; operands b4d28s-x1, reg4; 6 bytes



MOVL.D: LDU.B12, prefix 11000011, sub-opcode 1000; operands b4d12s-x1, reg4; 4 bytes



MOVL.D: LDU.X6, prefix 11000100, sub-opcode 1000; operands x4-4d6s, reg4; 4 bytes

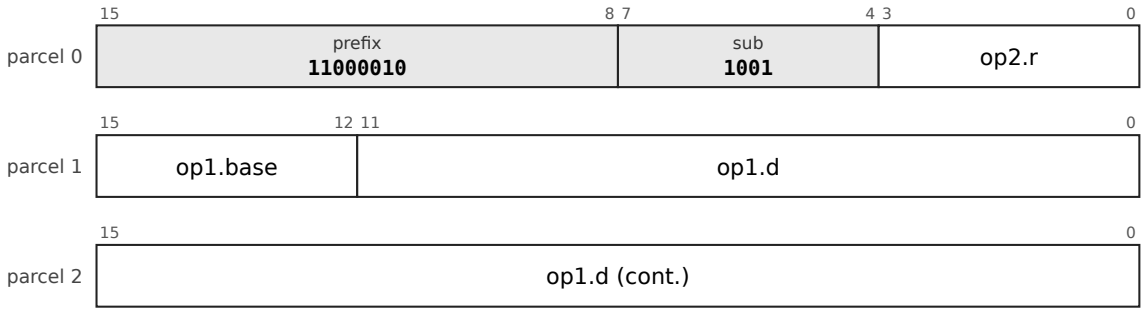



```
MOVL.D ..., MOVW.D ..., MOVB.D ...
```

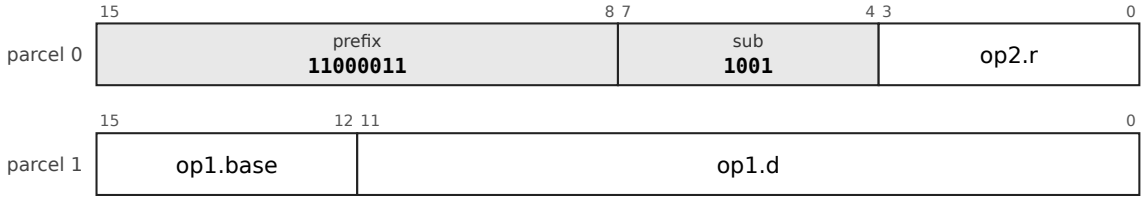
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVW.D

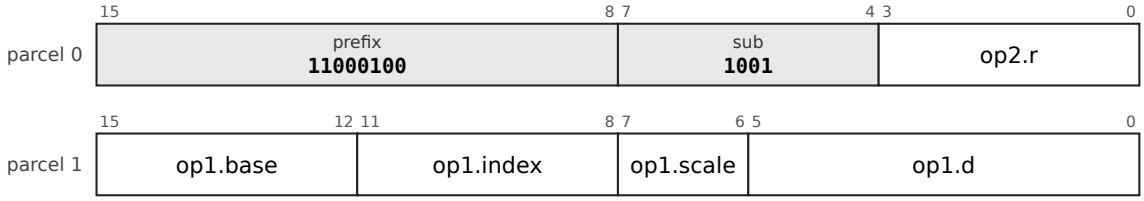
MOVW.D: LDU.B28, prefix 11000010, sub-opcode 1001; operands b4d28s-x1, reg4; 6 bytes



MOVW.D: LDU.B12, prefix 11000011, sub-opcode 1001; operands b4d12s-x1, reg4; 4 bytes



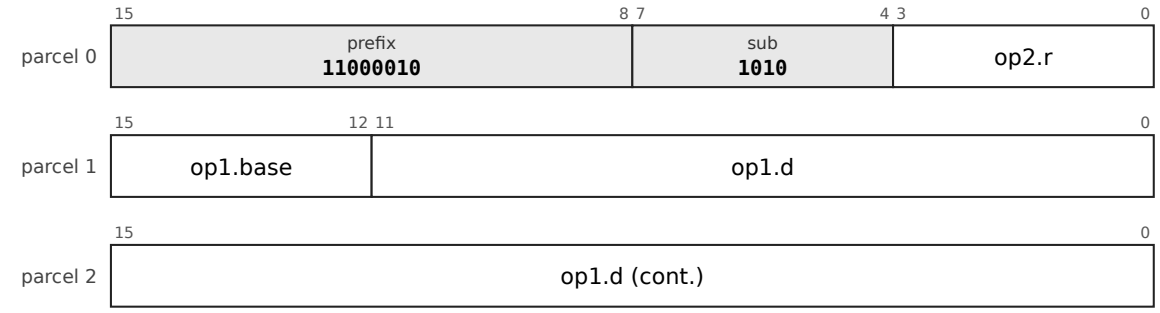
MOVW.D: LDU.X6, prefix 11000100, sub-opcode 1001; operands x4-4d6s, reg4; 4 bytes



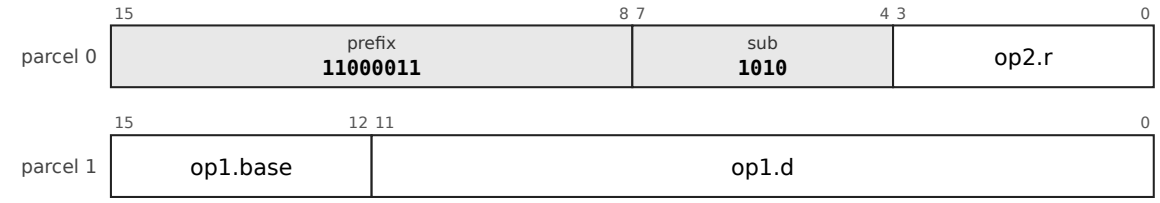
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVB.D

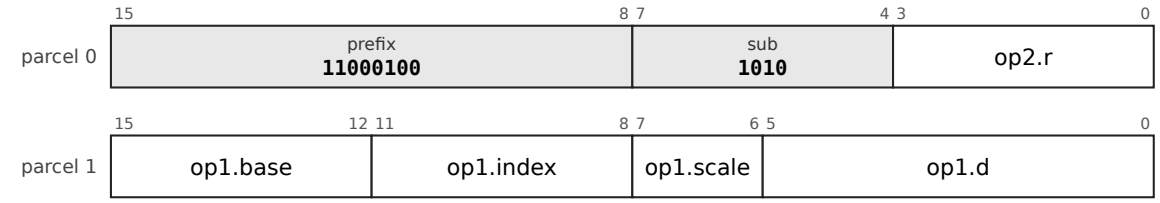
MOVB.D: LDU.B28, prefix 11000010, sub-opcode 1010; operands b4d28s-x1, reg4; 6 bytes



MOVB.D: LDU.B12, prefix 11000011, sub-opcode 1010; operands b4d12s-x1, reg4; 4 bytes



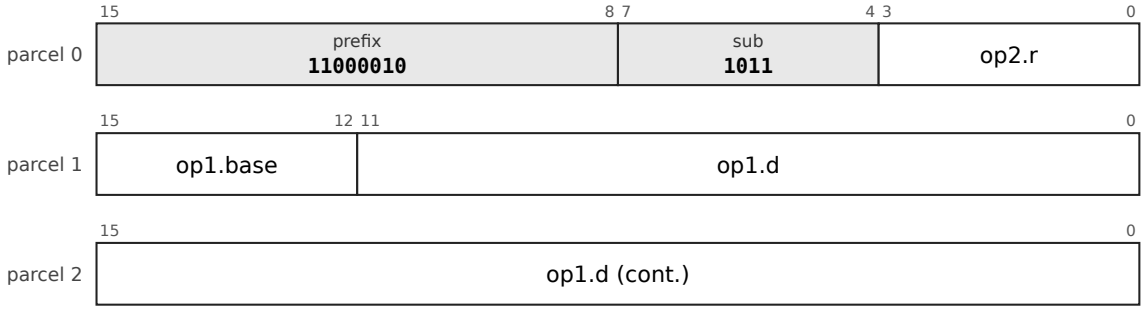
MOVB.D: LDU.X6, prefix 11000100, sub-opcode 1010; operands x4-4d6s, reg4; 4 bytes



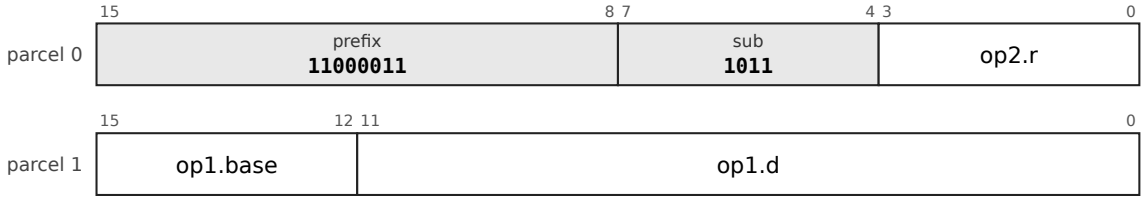
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVZL.D

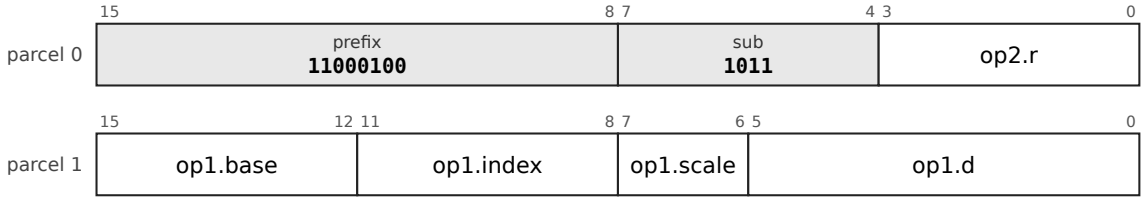
MOVZL.D: LDU.B28, prefix 11000010, sub-opcode 1011; operands b4d28s-x1, reg4; 6 bytes



MOVZL.D: LDU.B12, prefix 11000011, sub-opcode 1011; operands b4d12s-x1, reg4; 4 bytes



MOVZL.D: LDU.X6, prefix 11000100, sub-opcode 1011; operands x4-4d6s, reg4; 4 bytes

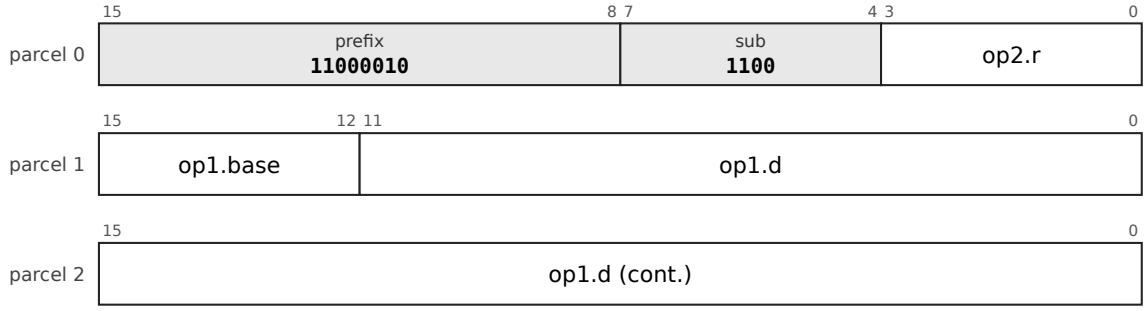



```
MOVZL.D ..., MOVZW.D ..., MOVZB.D ...
```

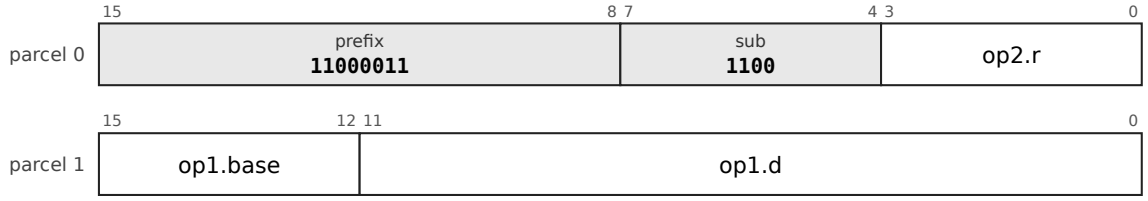
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVZW.D

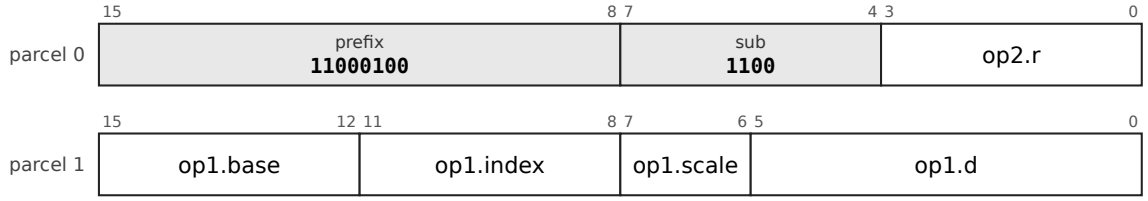
MOVZW.D: LDU.B28, prefix 11000010, sub-opcode 1100; operands b4d28s-x1, reg4; 6 bytes



MOVZW.D: LDU.B12, prefix 11000011, sub-opcode 1100; operands b4d12s-x1, reg4; 4 bytes



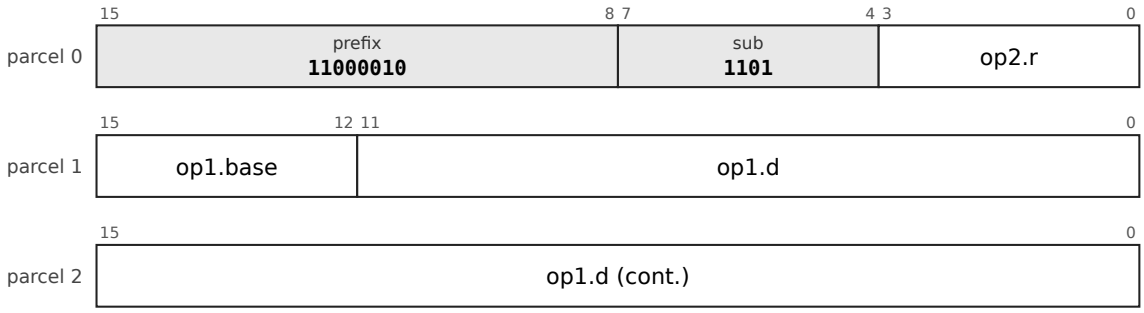
MOVZW.D: LDU.X6, prefix 11000100, sub-opcode 1100; operands x4-4d6s, reg4; 4 bytes



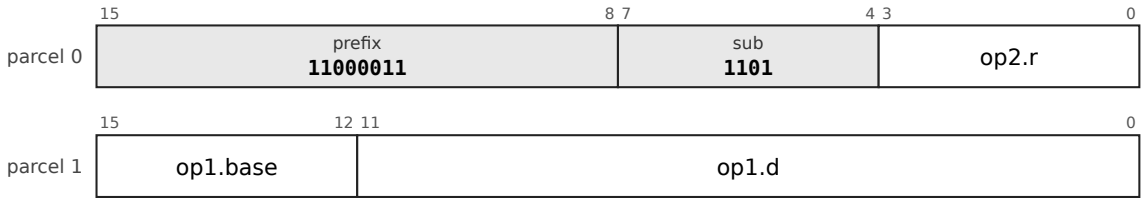
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

MOVZB.D

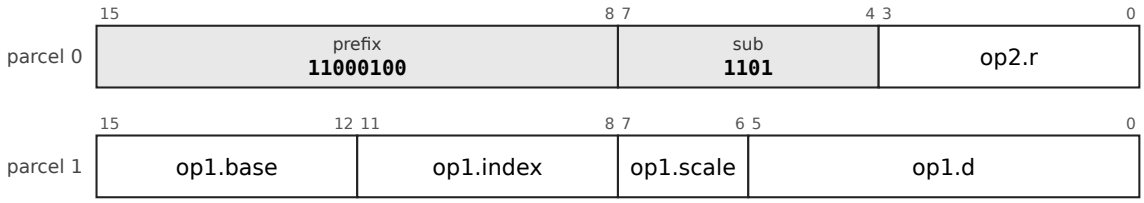
MOVZB.D: LDU.B28, prefix 11000010, sub-opcode 1101; operands b4d28s-x1, reg4; 6 bytes



MOVZB.D: LDU.B12, prefix 11000011, sub-opcode 1101; operands b4d12s-x1, reg4; 4 bytes



MOVZB.D: LDU.X6, prefix 11000100, sub-opcode 1101; operands x4-4d6s, reg4; 4 bytes



Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

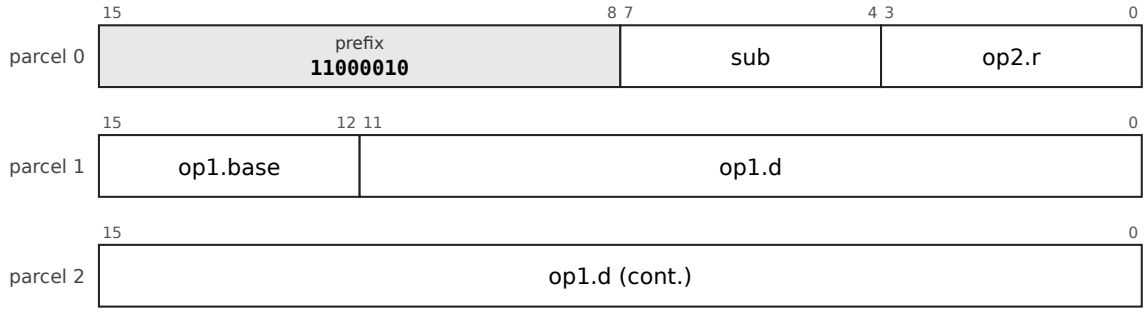
MOVUSER: User-qualified load and store

Mnemonics: `MOVQ.U`, `MOVL.U`, `MOVW.U`, `MOVB.U`, `MOVZL.U`, `MOVZW.U`, `MOVZB.U`

Syntax

```
MOVQ.U d(Rn), Rd
MOVQ.U Rs, d(Rn)
MOVL.U ..., MOVW.U ..., MOVB.U ...
MOVZL.U ..., MOVZW.U ..., MOVZB.U ...    ; loads only
```

Encoding



The chart is the leaf `16-LDU-B28-1`, LDU.B28, prefix `11000010`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDU-B28-1`, `16-STU-B28-1`, `16-LDU-B12-1`, `16-STU-B12-1`, `16-LDU-X6-1`, `16-STU-X6-1`.

Operation

```
the access of MOVQ, MOVL, MOVW, MOVB and MOVZ* with the operand checked as a user-mode access:
the page must be present, have U = 1, and permit R (a load) or W (a store); a kernel page takes the
protection abort
whatever PSW.UA says
```

Flags None written.

Exceptions The faults of the access, as a user-mode access. In user mode the encoding is a privileged-instruction abort.

(P) Item 18 of `docs/study-proposals.md`: user access is an attribute of one memory operand, so it cannot outlive the instruction, cannot leak across an interrupt, and `RTI` re-executes the instruction with the attribute. The author accepted the five leanings (errata 86): a fixup table in place of `pcb_onfault`, the `.U` spelling, keeping `PSW.UA`, `STUA` and `CLUA` until the OpenBSD port confirms, `ECAUSE` bit 15 to say that the access was user-qualified, and the atomics taking the user form once the operand placement of `ATOM` is fixed. The proposal is not specified in full, so the text is *(P)*. (Not to be confused with `MOVU`, the unaligned move.) The reference executor runs a `.U` move as a plain move, because it has no MMU in that mode.

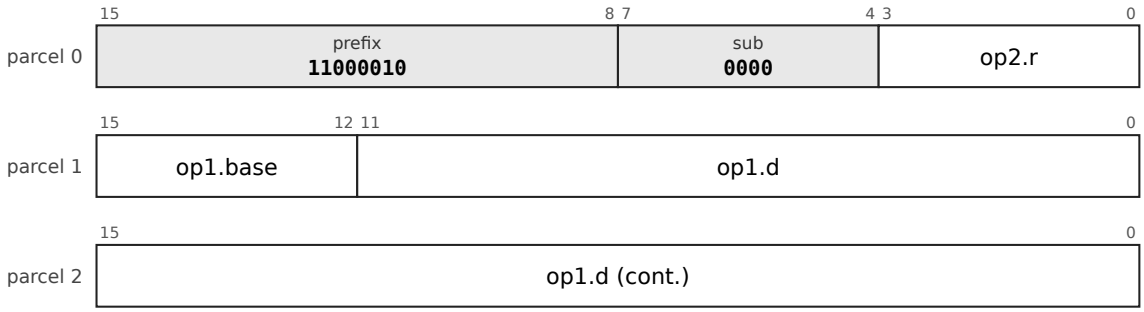
Why it is here. Copying between the kernel and the user is where kernels pay for the user-memory guard. x86 opens a window with `STAC` and closes it with `CLAC` (35 and 53 sites in the kernel-assembly corpus), and a forgotten `CLAC` on an error path is the bug class; Arm's `LDTR` and `STTR` make the access itself unprivileged (52 sites), but only for plain loads and stores. Item 18 of the study makes user access an attribute of one operand, so that no window exists to leave open and the range check of the copy routines (`addr + len` against the top of the user half) disappears into the hardware: a kernel address faults at the first byte. The author accepted its five leanings (errata 86) and kept `PSW.UA`, `STUA` and `CLUA` until the OpenBSD port confirms; the instruction is a proposal until then.

Cf. 6.4 (the user-memory guard), docs/study-proposals.md item 18.

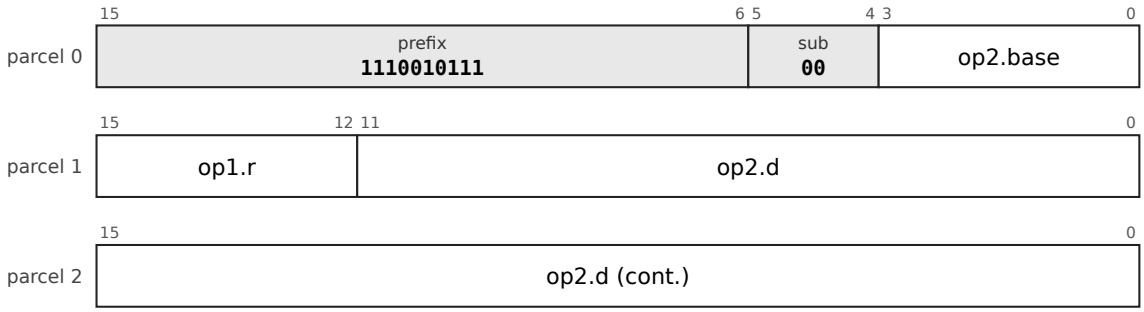
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MOVQ.U

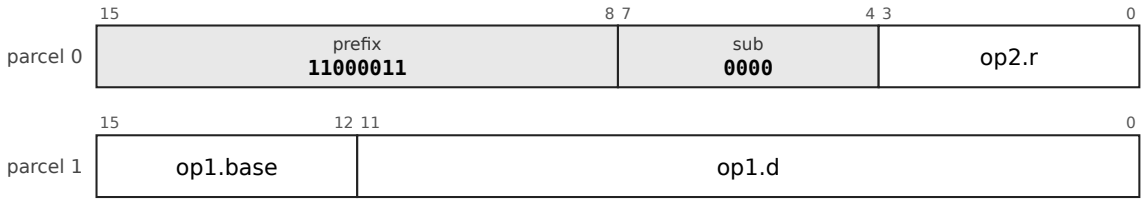
MOVQ.U: LDU.B28, prefix 11000010, sub-opcode 0000; operands b4d28s-x1, reg4; 6 bytes



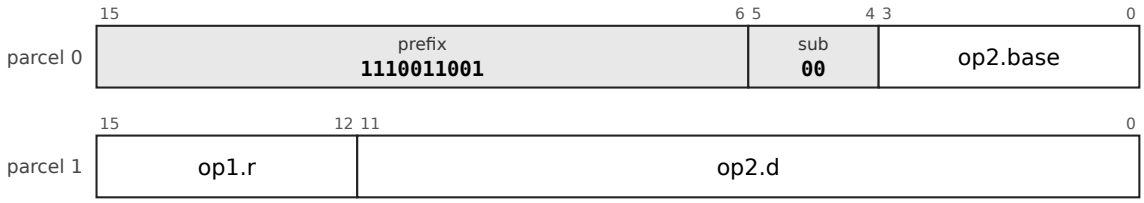
MOVQ.U: STU.B28, prefix 1110010111, sub-opcode 00; operands reg4, b4d28s-x1; 6 bytes



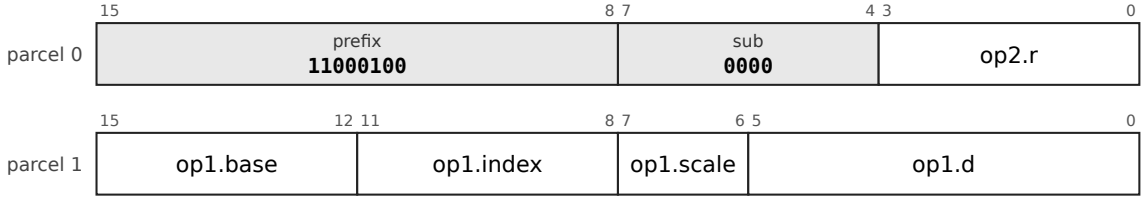
MOVQ.U: LDU.B12, prefix 11000011, sub-opcode 0000; operands b4d12s-x1, reg4; 4 bytes



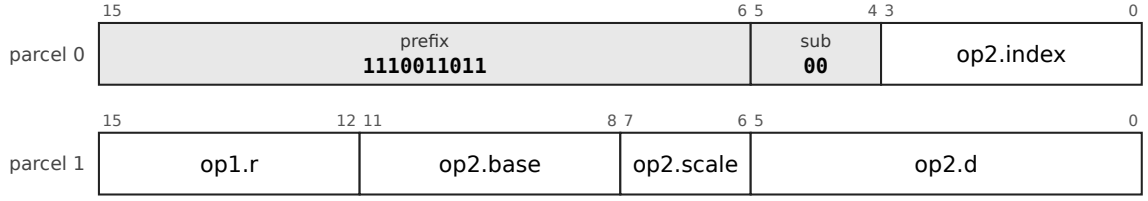
MOVQ.U: STU.B12, prefix 1110011001, sub-opcode 00; operands reg4, b4d12s-x1; 4 bytes



MOVQ.U: LDU.X6, prefix 11000100, sub-opcode 0000; operands x4-4d6s, reg4; 4 bytes



MOVQ.U: STU.X6, prefix 1110011011, sub-opcode 00; operands reg4, x4-4d6s; 4 bytes

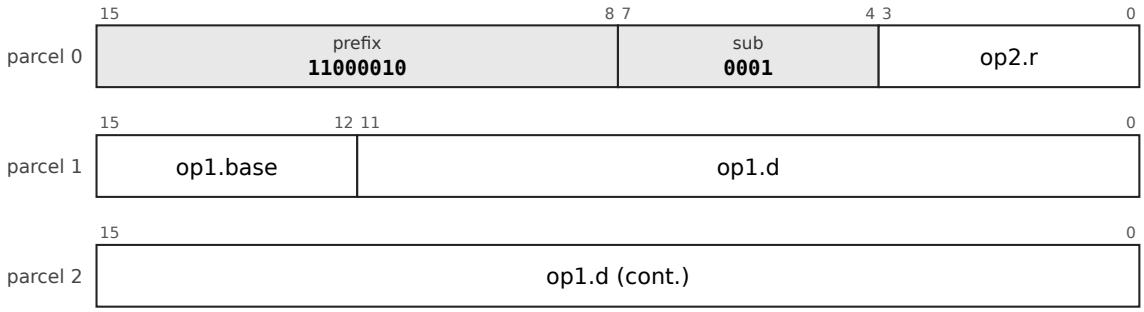



```
MOVQ.U d(Rn), Rd
MOVQ.U Rs, d(Rn)
```

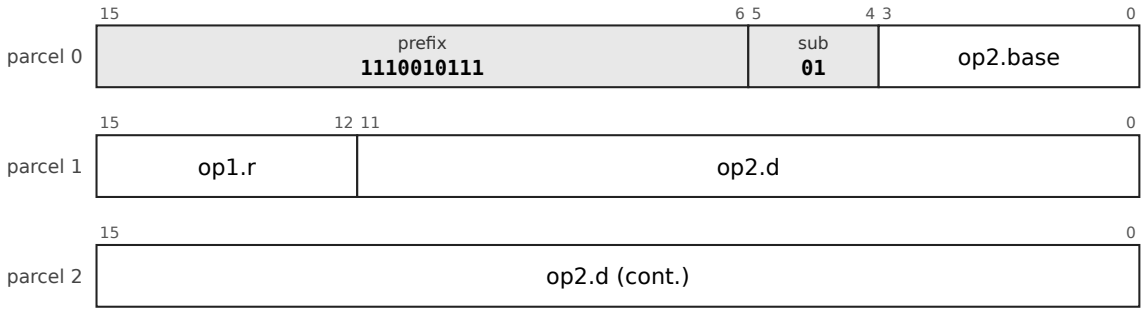
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

MOVL.U

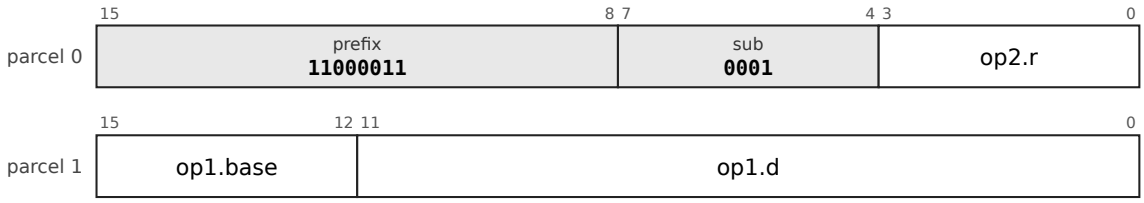
MOVL.U: LDU.B28, prefix 11000010, sub-opcode 0001; operands b4d28s-x1, reg4; 6 bytes



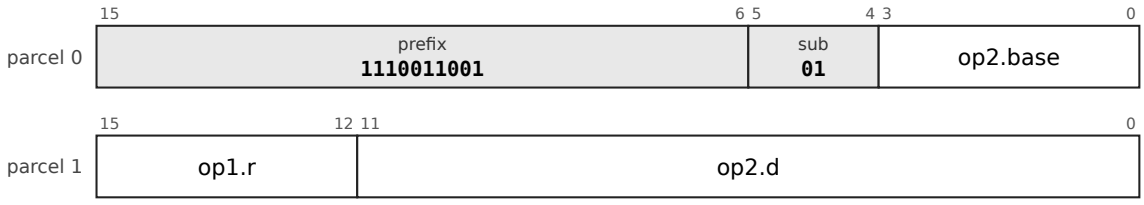
MOVL.U: STU.B28, prefix 1110010111, sub-opcode 01; operands reg4, b4d28s-x1; 6 bytes



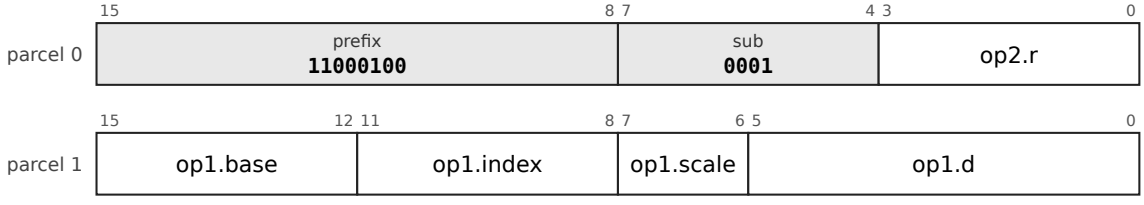
MOVL.U: LDU.B12, prefix 11000011, sub-opcode 0001; operands b4d12s-x1, reg4; 4 bytes



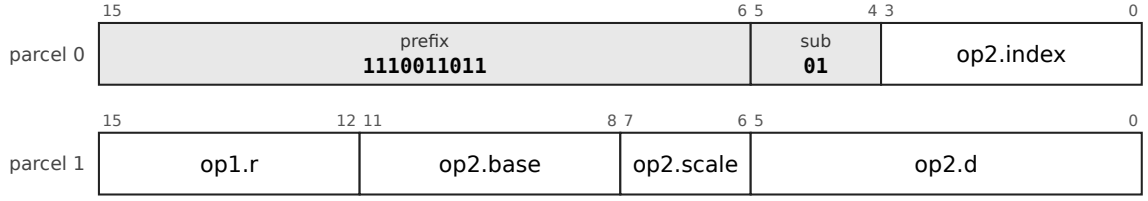
MOVL.U: STU.B12, prefix 1110011001, sub-opcode 01; operands reg4, b4d12s-x1; 4 bytes



MOVL.U: LDU.X6, prefix 11000100, sub-opcode 0001; operands x4-4d6s, reg4; 4 bytes



MOVL.U: STU.X6, prefix 1110011011, sub-opcode 01; operands reg4, x4-4d6s; 4 bytes

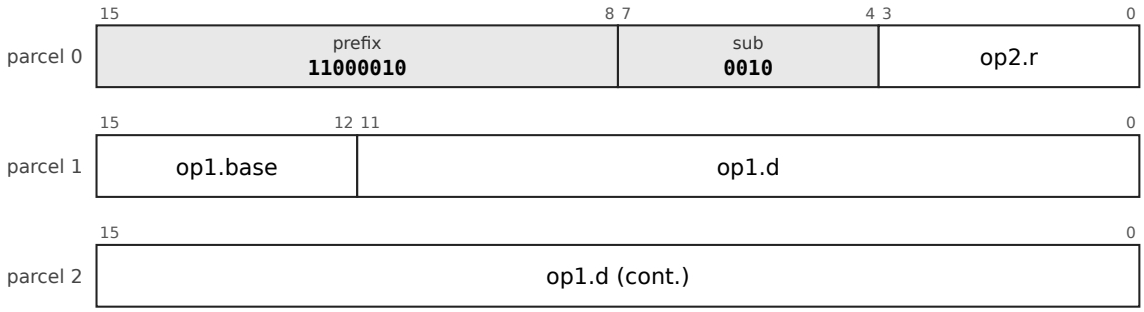



```
MOVL.U ..., MOVW.U ..., MOVB.U ...
```

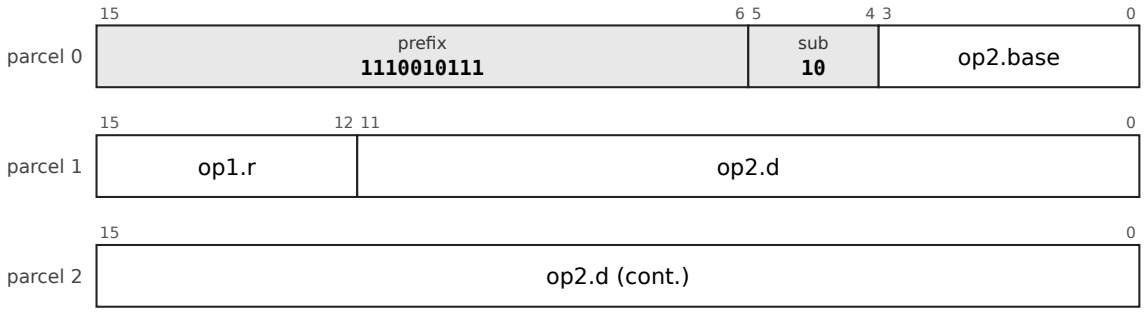
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

MOVW.U

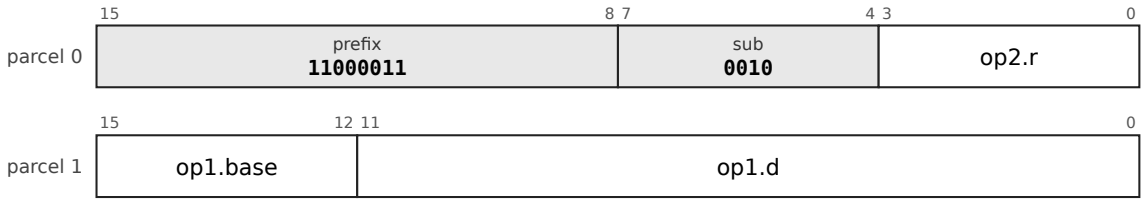
MOVW.U: LDU.B28, prefix 11000010, sub-opcode 0010; operands b4d28s-x1, reg4; 6 bytes



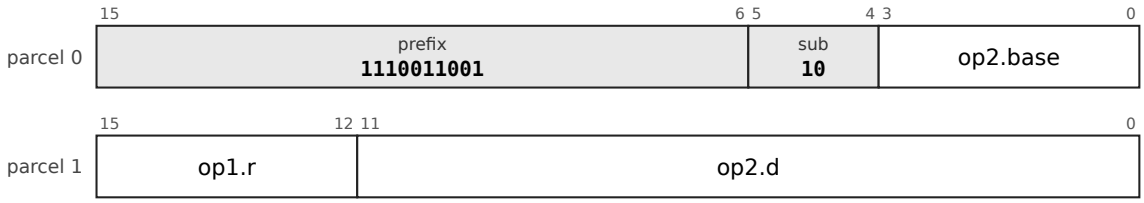
MOVW.U: STU.B28, prefix 1110010111, sub-opcode 10; operands reg4, b4d28s-x1; 6 bytes



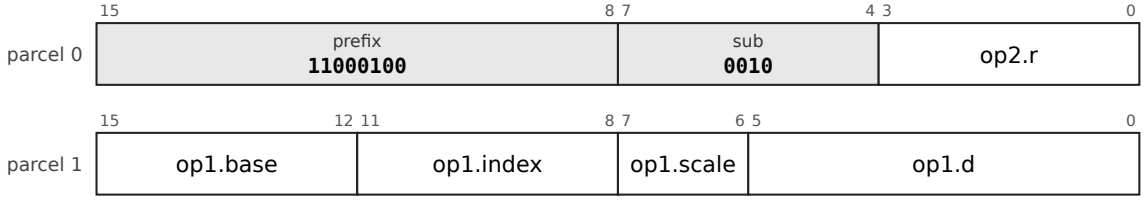
MOVW.U: LDU.B12, prefix 11000011, sub-opcode 0010; operands b4d12s-x1, reg4; 4 bytes



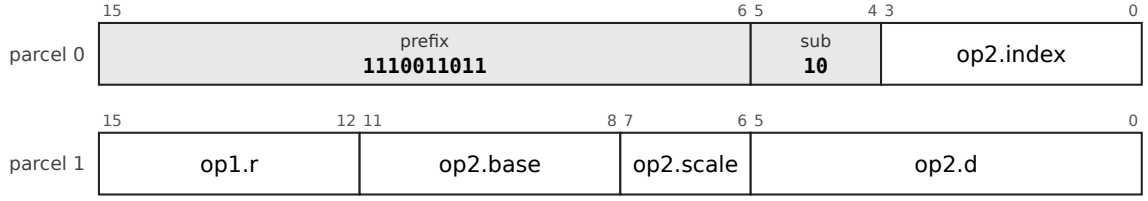
MOVW.U: STU.B12, prefix 1110011001, sub-opcode 10; operands reg4, b4d12s-x1; 4 bytes



MOVW.U: LDU.X6, prefix 11000100, sub-opcode 0010; operands x4-4d6s, reg4; 4 bytes



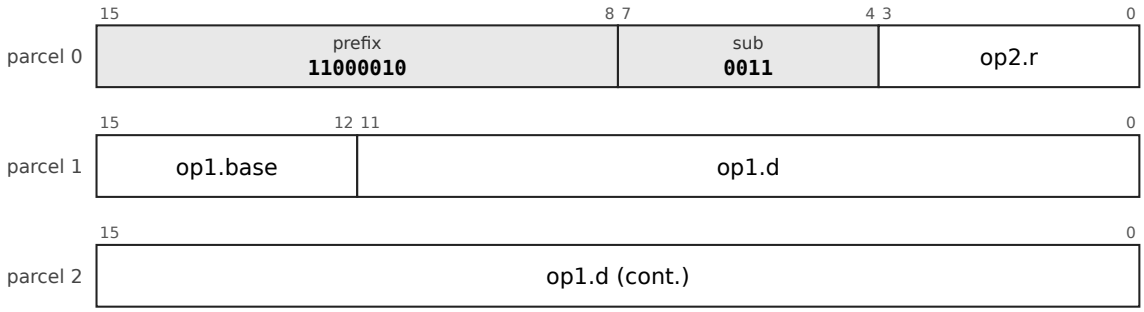
MOVW.U: STU.X6, prefix 1110011011, sub-opcode 10; operands reg4, x4-4d6s; 4 bytes



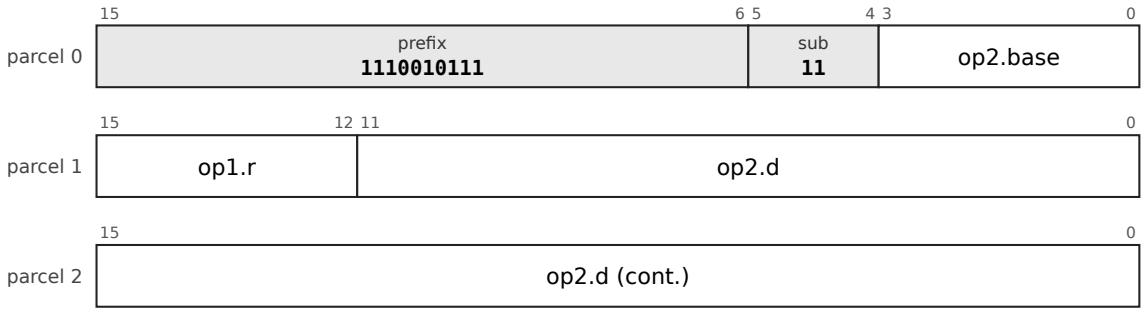
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

MOVB.U

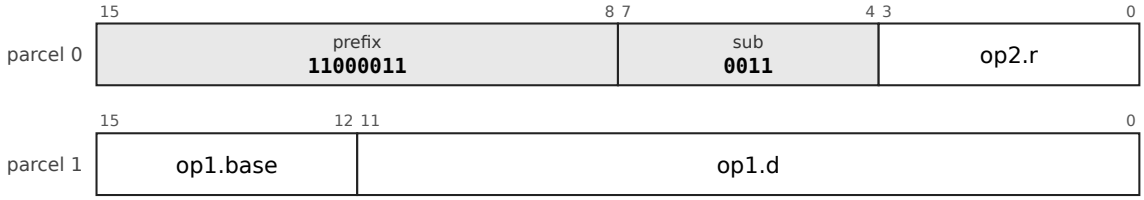
MOVB.U: LDU.B28, prefix 11000010, sub-opcode 0011; operands b4d28s-x1, reg4; 6 bytes



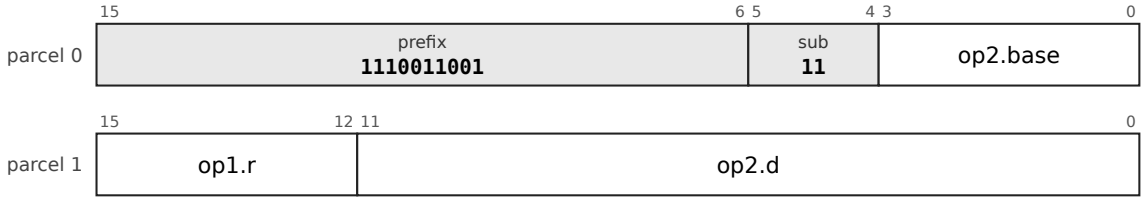
MOVB.U: STU.B28, prefix 1110010111, sub-opcode 11; operands reg4, b4d28s-x1; 6 bytes



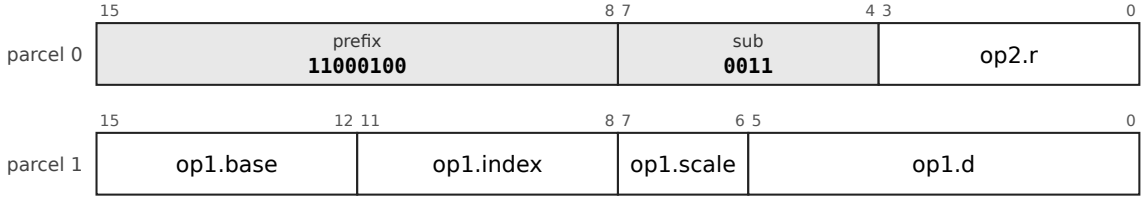
MOVB.U: LDU.B12, prefix 11000011, sub-opcode 0011; operands b4d12s-x1, reg4; 4 bytes



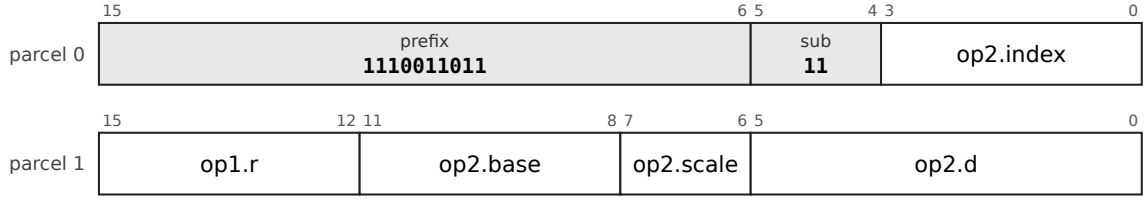
MOVB.U: STU.B12, prefix 1110011001, sub-opcode 11; operands reg4, b4d12s-x1; 4 bytes



MOVB.U: LDU.X6, prefix 11000100, sub-opcode 0011; operands x4-4d6s, reg4; 4 bytes



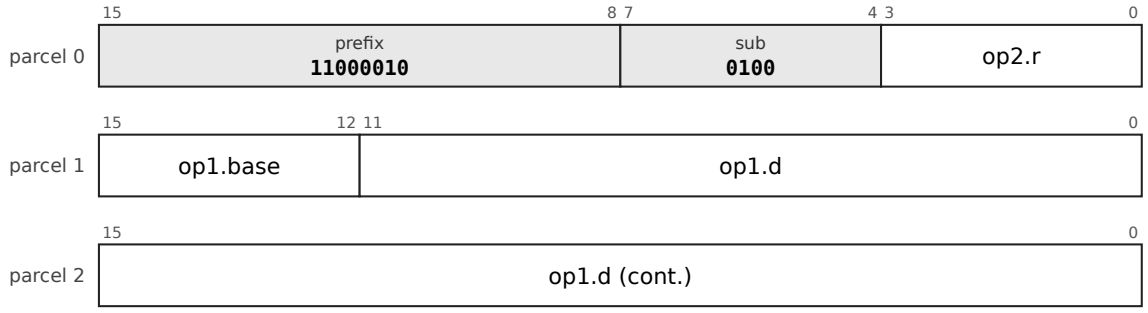
MOVB.U: STU.X6, prefix 1110011011, sub-opcode 11; operands reg4, x4-4d6s; 4 bytes



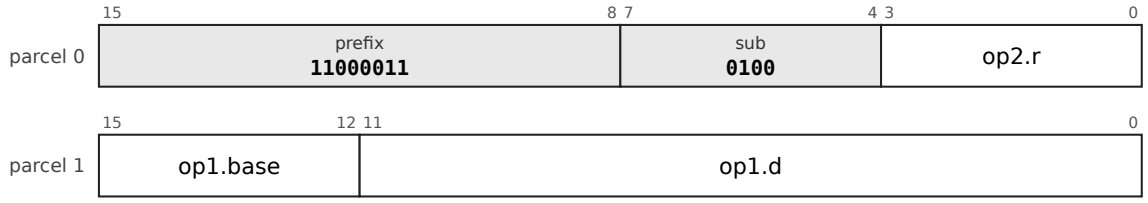
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

MOVZL.U

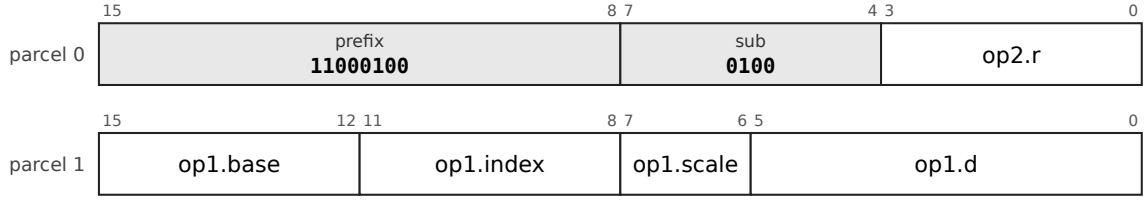
MOVZL.U: LDU.B28, prefix 11000010, sub-opcode 0100; operands b4d28s-x1, reg4; 6 bytes



MOVZL.U: LDU.B12, prefix 11000011, sub-opcode 0100; operands b4d12s-x1, reg4; 4 bytes



MOVZL.U: LDU.X6, prefix 11000100, sub-opcode 0100; operands x4-4d6s, reg4; 4 bytes

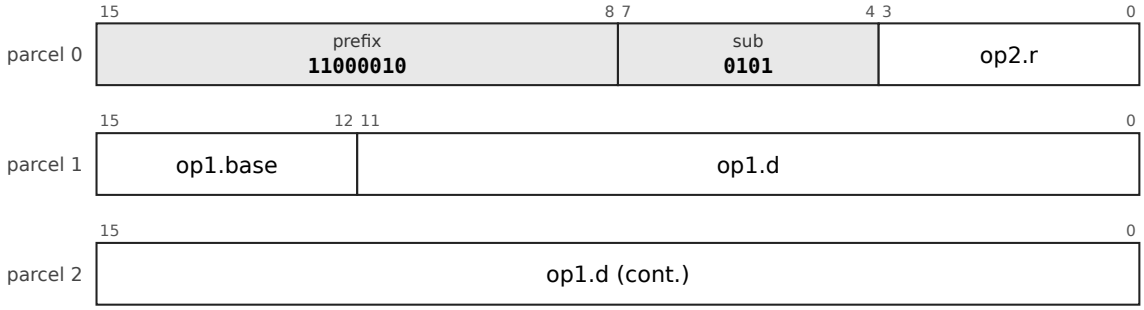



```
MOVZL.U ..., MOVZW.U ..., MOVZB.U ...
```

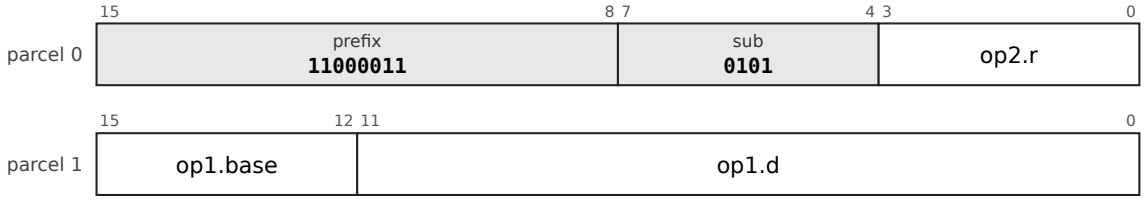
Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

MOVZW.U

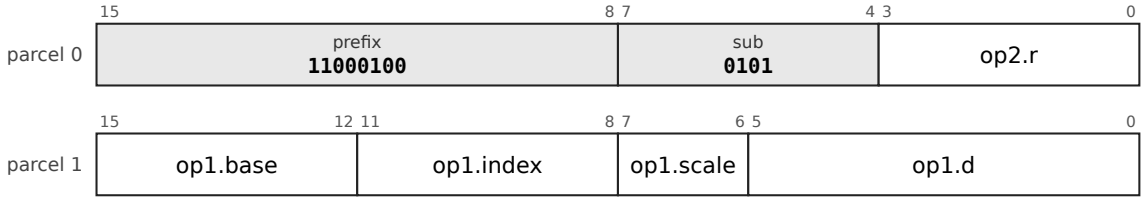
MOVZW.U: LDU.B28, prefix 11000010, sub-opcode 0101; operands b4d28s-x1, reg4; 6 bytes



MOVZW.U: LDU.B12, prefix 11000011, sub-opcode 0101; operands b4d12s-x1, reg4; 4 bytes



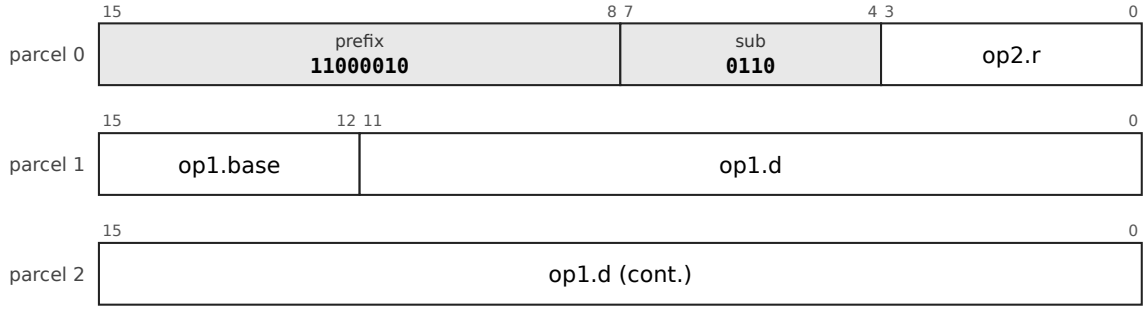
MOVZW.U: LDU.X6, prefix 11000100, sub-opcode 0101; operands x4-4d6s, reg4; 4 bytes



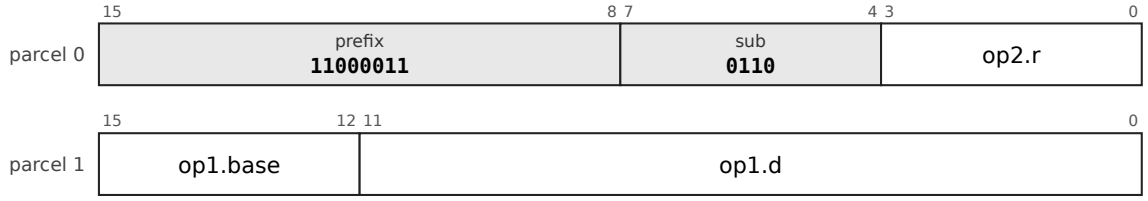
Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

MOVZB.U

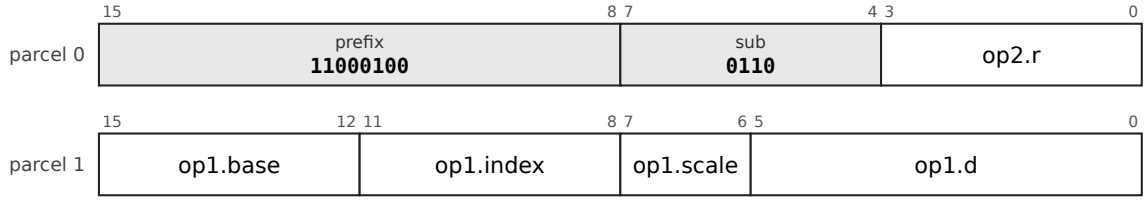
MOVZB.U: LDU.B28, prefix 11000010, sub-opcode 0110; operands b4d28s-x1, reg4; 6 bytes



MOVZB.U: LDU.B12, prefix 11000011, sub-opcode 0110; operands b4d12s-x1, reg4; 4 bytes



MOVZB.U: LDU.X6, prefix 11000100, sub-opcode 0110; operands x4-4d6s, reg4; 4 bytes



Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

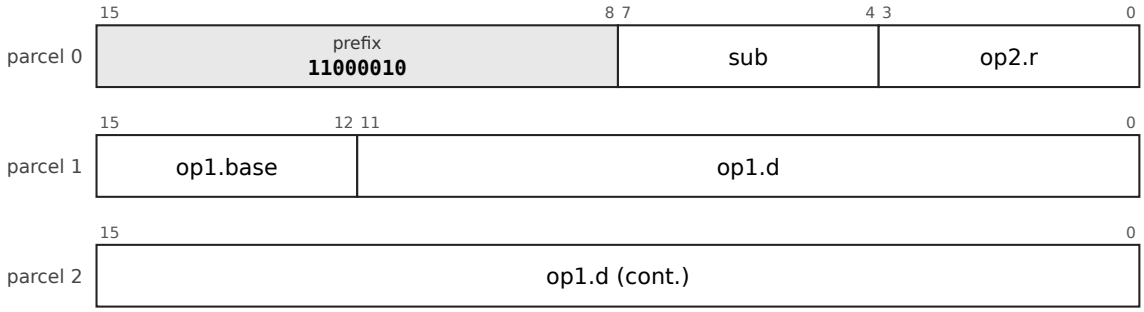
MOVUSERD: User-qualified deferred-fault load

Mnemonics: `MOVQ.UD`, `MOVL.UD`, `MOVW.UD`, `MOVB.UD`, `MOVZL.UD`, `MOVZW.UD`, `MOVZB.UD`

Syntax

```
MOVQ.UD d(Rn), Rd
MOVL.UD ..., MOVW.UD ..., MOVB.UD ...
MOVZL.UD ..., MOVZW.UD ..., MOVZB.UD ...
```

Encoding



The chart is the leaf `16-LDU-B28-1`, LDU.B28, prefix `11000010`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDU-B28-1`, `16-LDU-B12-1`, `16-LDU-X6-1`, `16-LDU-B28-2`, `16-LDU-B12-2`, `16-LDU-X6-2`, `16-LDU-B28-3`, `16-LDU-B12-3`, `16-LDU-X6-3`.

Operation

```
Rd ← M[EA], the access being both user-qualified (MOVU) and deferred (MOVD)
```

Flags None written.

Exceptions The faults of `MOVU` and `MOVD` together.

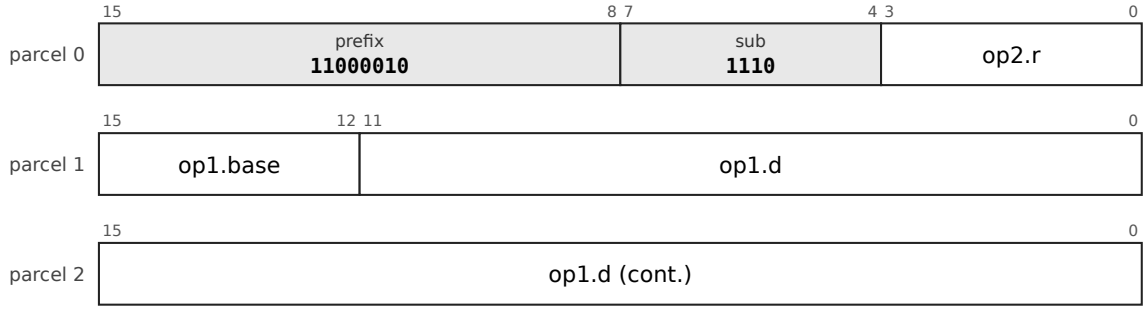
(P) The combination of the two qualifiers. The repository defines each qualifier alone (items 10 and 18 of `docs/study-proposals.md`) and gives the combination no text of its own: a kernel that reads a user word without wanting to fail on it (a probe of a user pointer) is the case the combination serves, and its meaning is the composition. Loads only, base and indexed operands. **Not implemented** in the reference executor.

Why it is here. See `MOVD` and `MOVUSER`. The record gives no separate reason for the combination; it is listed because both qualifiers exist and a probe of a user pointer wants both.

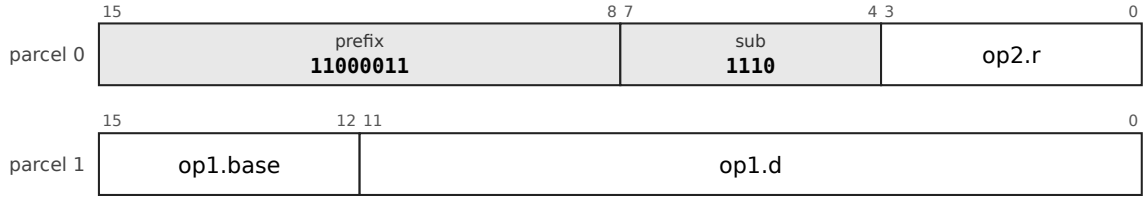
Status specified; **not yet implemented** in the reference executor.

MOVQ.UD

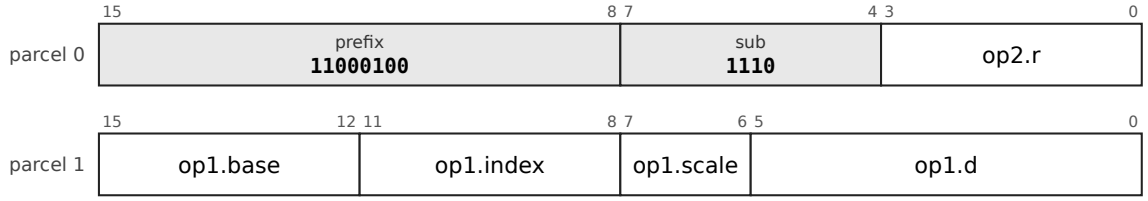
MOVQ.UD: LDU.B28, prefix 11000010, sub-opcode 1110; operands b4d28s-x1, reg4; 6 bytes



MOVQ.UD: LDU.B12, prefix 11000011, sub-opcode 1110; operands b4d12s-x1, reg4; 4 bytes



MOVQ.UD: LDU.X6, prefix 11000100, sub-opcode 1110; operands x4-4d6s, reg4; 4 bytes

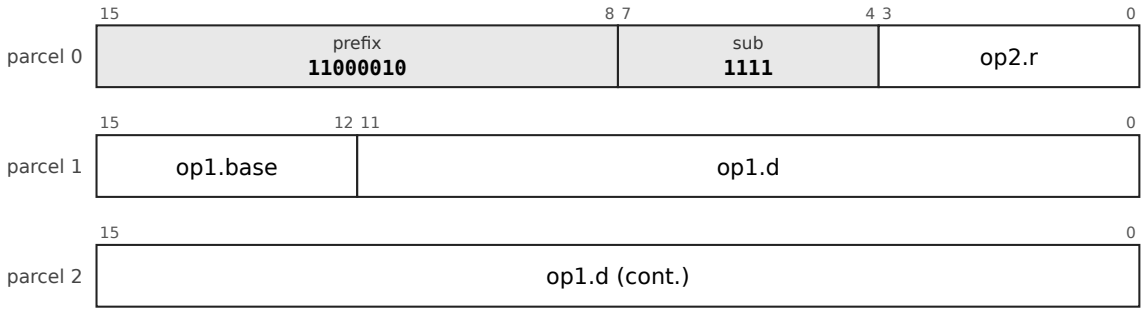



```
MOVQ.UD d(Rn), Rd
```

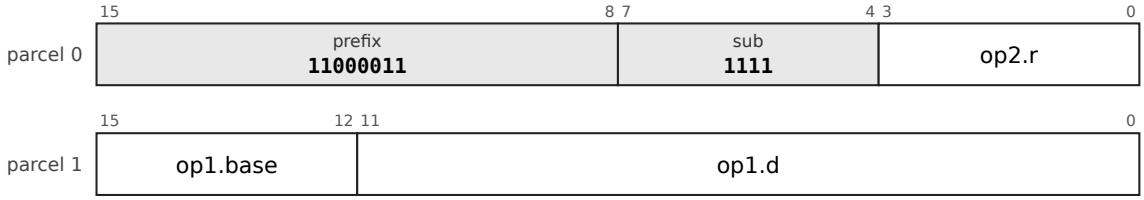
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVL.UD

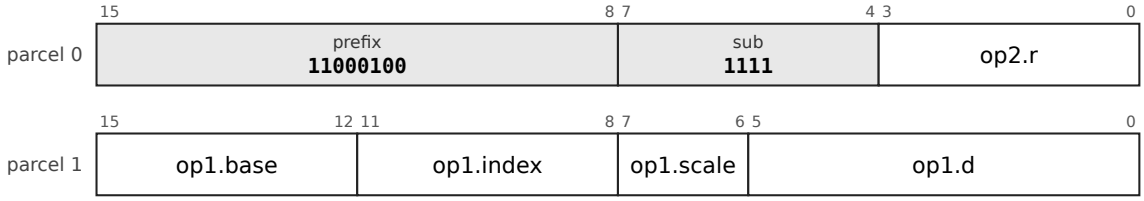
MOVL.UD: LDU.B28, prefix 11000010, sub-opcode 1111; operands b4d28s-x1, reg4; 6 bytes



MOVL.UD: LDU.B12, prefix 11000011, sub-opcode 1111; operands b4d12s-x1, reg4; 4 bytes



MOVL.UD: LDU.X6, prefix 11000100, sub-opcode 1111; operands x4-4d6s, reg4; 4 bytes

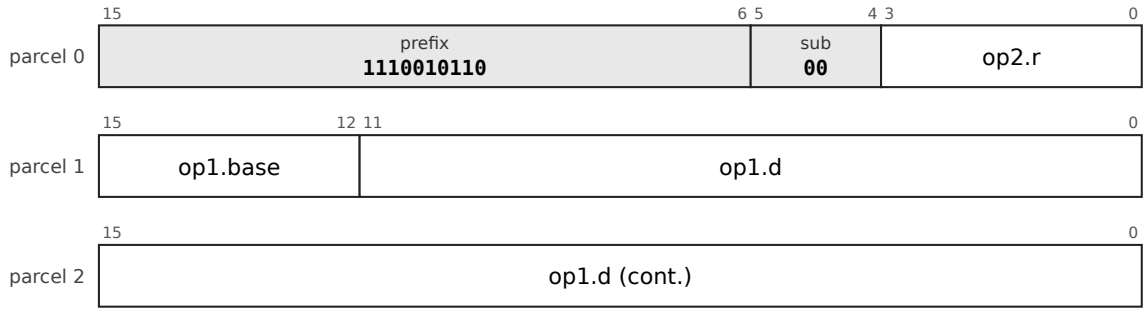



```
MOVL.UD ..., MOVW.UD ..., MOVB.UD ...
```

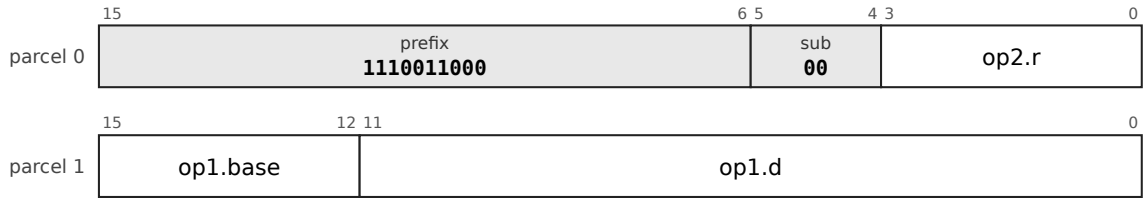
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVW.UD

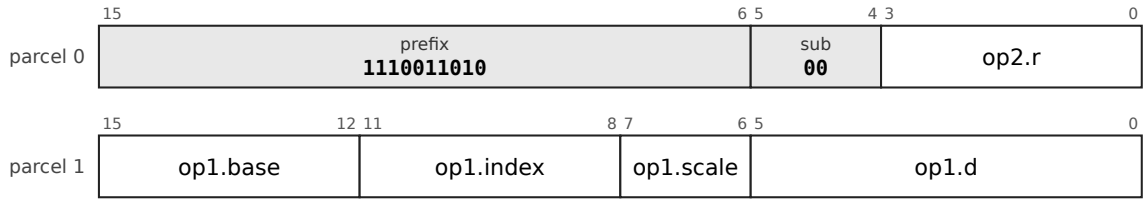
MOVW.UD: LDU.B28, prefix 1110010110, sub-opcode 00; operands b4d28s-x1, reg4; 6 bytes



MOVW.UD: LDU.B12, prefix 1110011000, sub-opcode 00; operands b4d12s-x1, reg4; 4 bytes



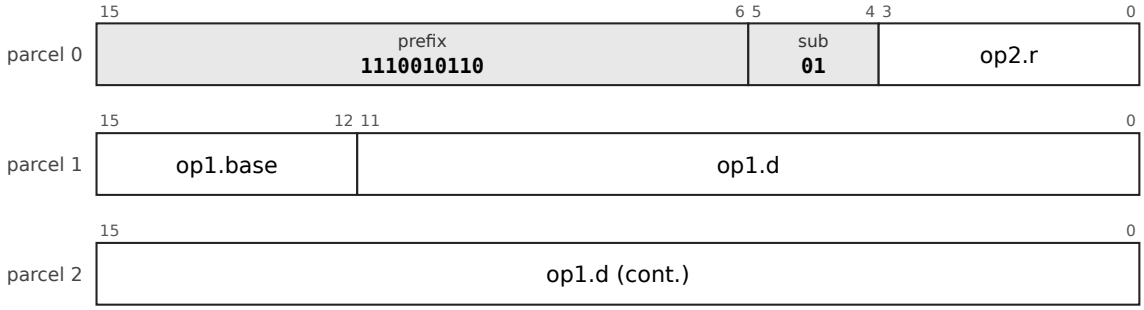
MOVW.UD: LDU.X6, prefix 1110011010, sub-opcode 00; operands x4-4d6s, reg4; 4 bytes



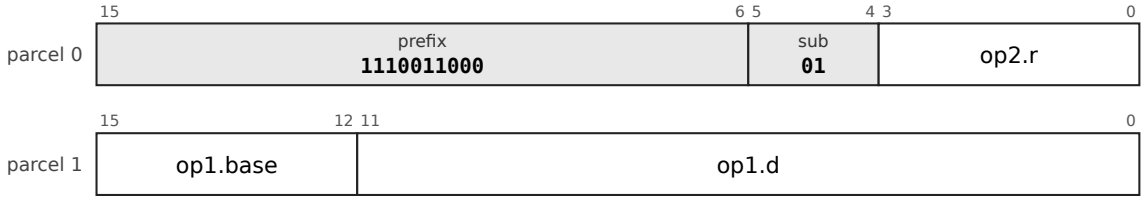
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVB.UD

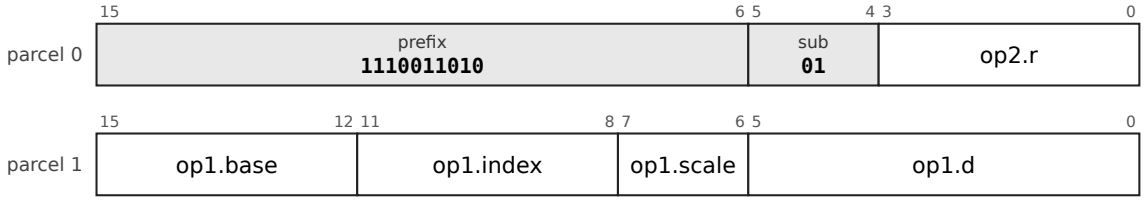
MOVB.UD: LDU.B28, prefix 1110010110, sub-opcode 01; operands b4d28s-x1, reg4; 6 bytes



MOVB.UD: LDU.B12, prefix 1110011000, sub-opcode 01; operands b4d12s-x1, reg4; 4 bytes



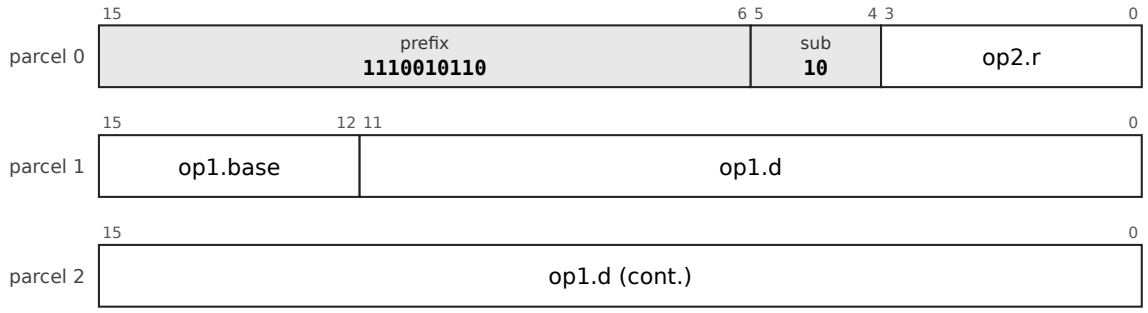
MOVB.UD: LDU.X6, prefix 1110011010, sub-opcode 01; operands x4-4d6s, reg4; 4 bytes



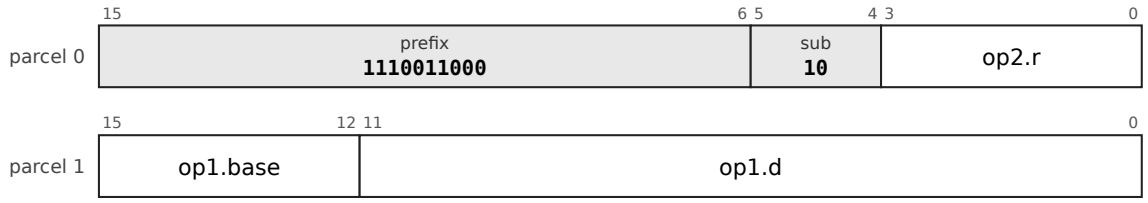
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVZL.UD

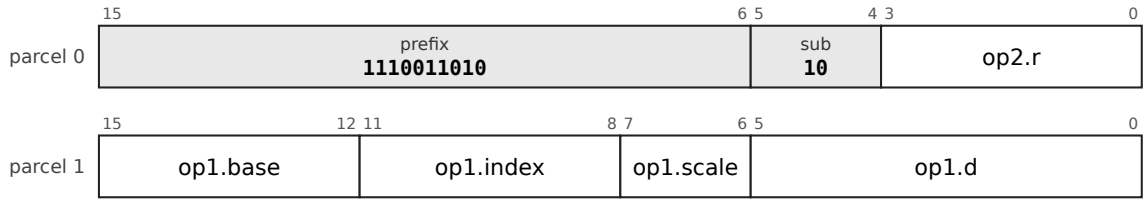
MOVZL.UD: LDU.B28, prefix 1110010110, sub-opcode 10; operands b4d28s-x1, reg4; 6 bytes



MOVZL.UD: LDU.B12, prefix 1110011000, sub-opcode 10; operands b4d12s-x1, reg4; 4 bytes



MOVZL.UD: LDU.X6, prefix 1110011010, sub-opcode 10; operands x4-4d6s, reg4; 4 bytes

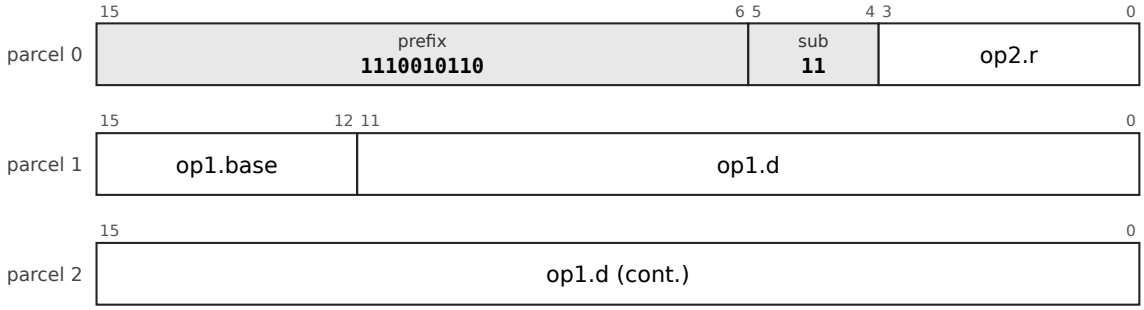



```
MOVZL.UD ..., MOVZW.UD ..., MOVZB.UD ...
```

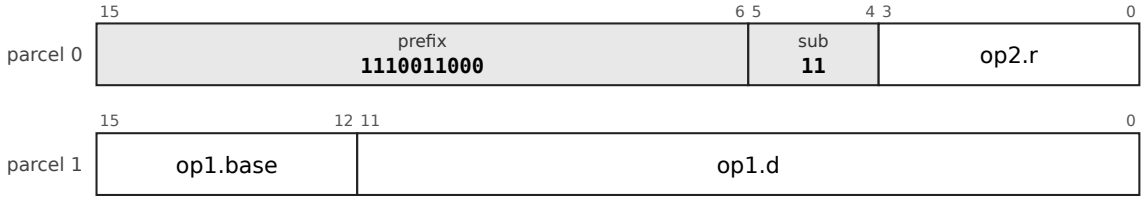
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVZW.UD

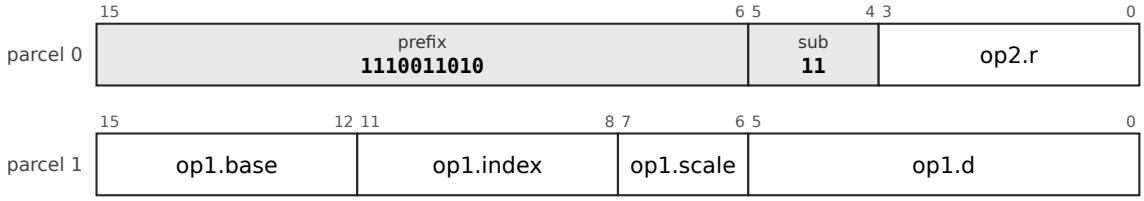
MOVZW.UD: LDU.B28, prefix 1110010110, sub-opcode 11; operands b4d28s-x1, reg4; 6 bytes



MOVZW.UD: LDU.B12, prefix 1110011000, sub-opcode 11; operands b4d12s-x1, reg4; 4 bytes



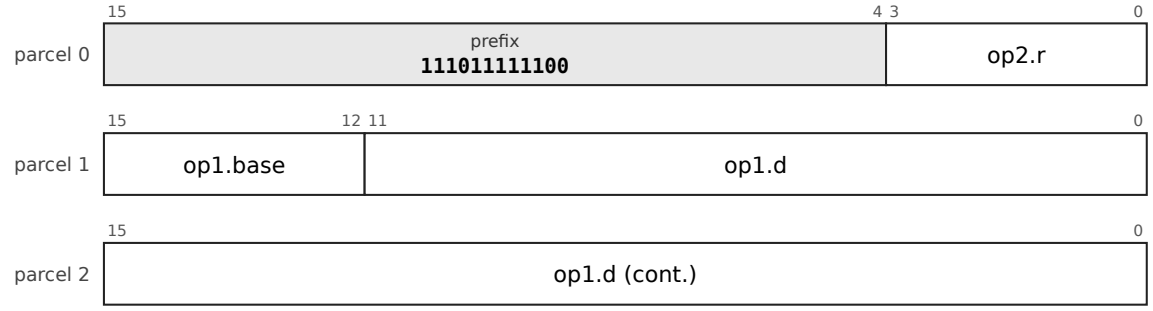
MOVZW.UD: LDU.X6, prefix 1110011010, sub-opcode 11; operands x4-4d6s, reg4; 4 bytes



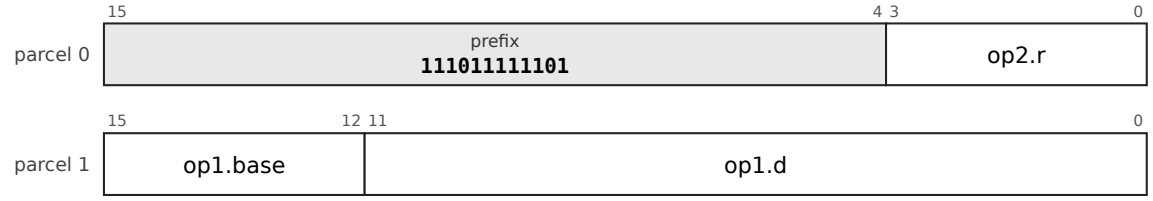
Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

MOVZB.UD

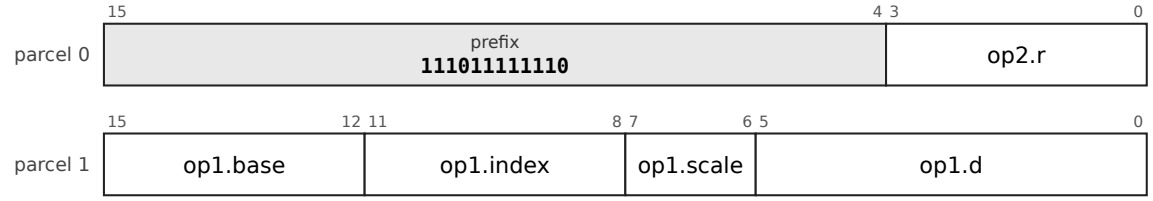
MOVZB.UD: LDU.B28, prefix 111011111100; operands b4d28s-x1, reg4; 6 bytes



MOVZB.UD: LDU.B12, prefix 111011111101; operands b4d12s-x1, reg4; 4 bytes



MOVZB.UD: LDU.X6, prefix 111011111110; operands x4-4d6s, reg4; 4 bytes



Operation, flags and exceptions as for the entry MOVUSERD above; 3 layout forms, each a chart of its own (above).

9.7 Integer arithmetic

Cf. section 6.2 for the divide-by-zero vector (`DIV`, `DIVU`, `MOD`, `MODU`).

9.7.1 `ADD`, `SUB`, `CMP`, `MUL` and `MULH` are two bytes with register operands; `MULHU` (high half of an unsigned product), `DIV`, `DIVU`, `MOD` and `MODU` are in the arithmetic escape. *(D)*

9.7.2 `CMP` is written `CMP Rd, rm`, register first, and computes `Rd - rm`. *(D)*

9.7.3 `ADD` sets `C` on a carry; `SUB` and `CMP` set `C` on a borrow, the PDP-11 sense, so `BHIS` and `BCC` mean "no borrow", unsigned greater or equal. *(D)*

9.7.4 `ADC` and `SBC` add and subtract the carry; `INC`, `DEC`, `NEG`, `CLR` and `TST` are single-operand forms that work on a register or memory.

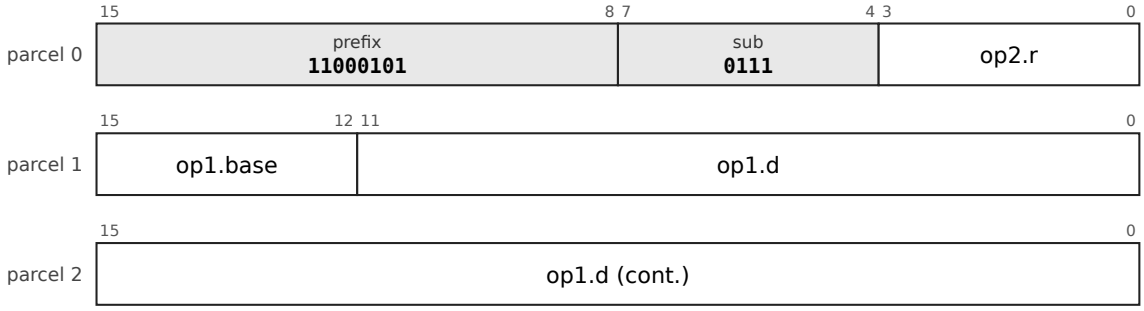
9.7.5 `ABS`, `MIN`, `MAX`, `MINU` and `MAXU`, and `SET cc, Rd` (write 1 if the condition holds, else 0), are in the arithmetic escape. *(D)*

Commentary. **Precedent.** `ADD`, `SUB`, `CMP`, `BIT`, `BIS`, `BIC` and the single-operand set are the PDP-11's, unchanged in name and meaning except for the order of `CMP`'s operands. **Trade-offs.** The PDP-11 wrote `CMP a, b` to compute `a - b`; since the destination of an ALU operation here must be a register, a literal `src, dst` reading would force every comparison against an immediate or memory to be reversed, so the register comes first. Divide is in the escape group, at four bytes, because compilers turn division by a constant into `MULH` plus a shift, so a real divide is rare. **Improvements.** The high-half multiply that bignum code needs is present; a second unsigned multiply is the same instruction as the first and is not provided. **Benefits.** `SET` replaces a branch around a move for the very common "store the truth value of a comparison", and absolute value, minimum and maximum remove a compare-and-branch from small loops. **Modern comparables.** RISC-V has `MULH`, `MULHU` and `MULHSU`, and

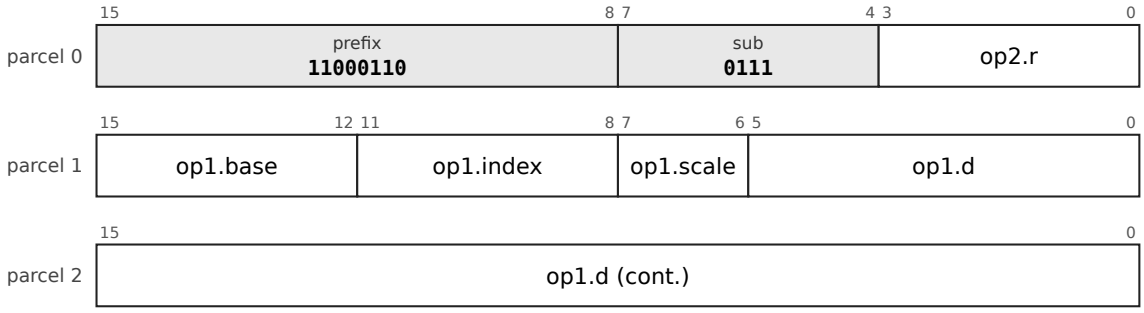
AArch64 `SMULH` and `UMULH`; x86 has a one-operand multiply that produces both halves; AArch64's conditional select and set instructions are the nearest to `SET`. **Use.** `MULH` and `MULHU` are for multi-precision arithmetic and for division by constants; `ADC` and `SBC` carry a multi-word add or subtract; `MIN`, `MAX` and `ABS` appear under a tenth of a percent of compiled code but each replaces a branch. **Impact.** *Kernel:* `ADC` and `SBC` carry multiword arithmetic for the kernel's 128-bit counters and for the bignum code in the base system. *Compiler:* `MULH` turns division by a constant into a multiply and a shift; `SET` materialises a comparison without a branch; `CMP` takes the register first, as the compiler orders its operands. *Hardware:* One adder on the carry chain; on an FPGA the multiplier is built from two to four multiplier blocks over four to eight cycles; the divide is a sequencer.

ADD: Add

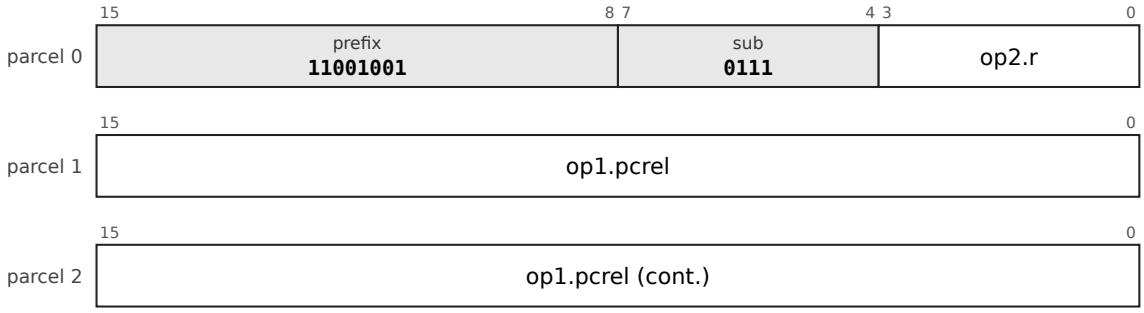
ADD: LDF.B28, prefix 11000101, sub-opcode 0111; operands b4d28s-x1, reg4; 6 bytes



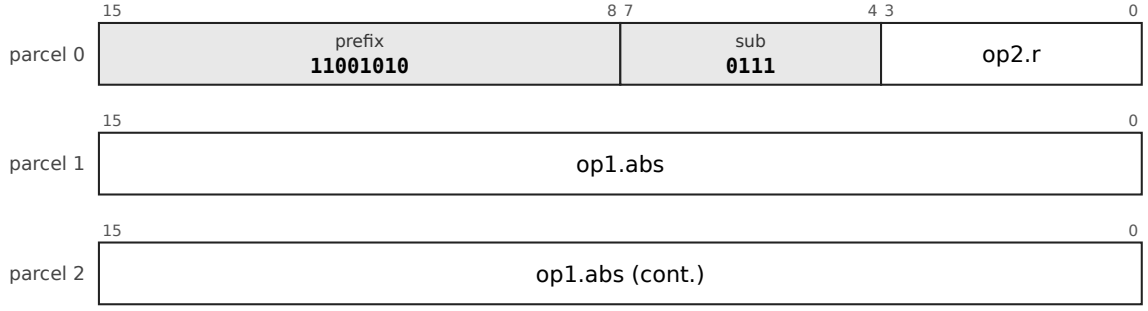
ADD: LDF.X22, prefix 11000110, sub-opcode 0111; operands x4-4d22s, reg4; 6 bytes



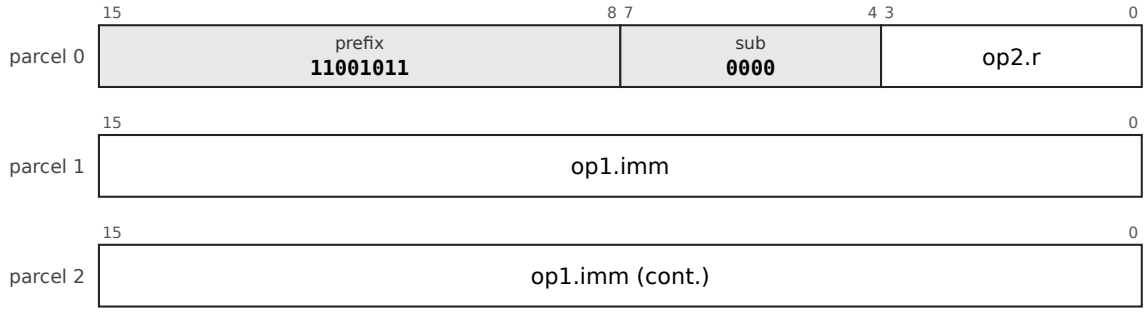
ADD: LDF.PC32, prefix 11001001, sub-opcode 0111; operands pc32-u1, reg4; 6 bytes



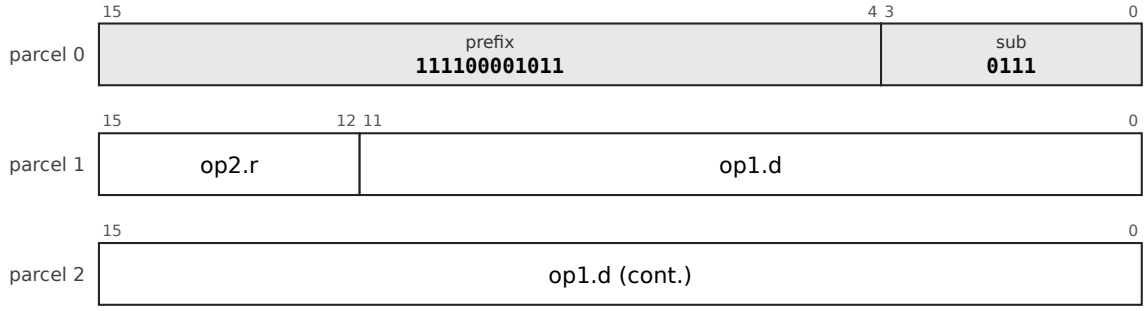
ADD: LDF.A32, prefix 11001010, sub-opcode 0111; operands abs32, reg4; 6 bytes



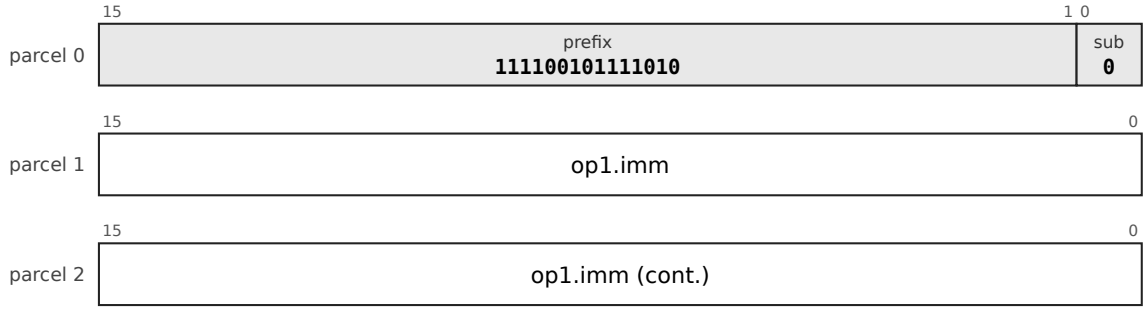
ADD: LDFI.I32, prefix 11001011, sub-opcode 0000; operands imm32s, reg4; 6 bytes



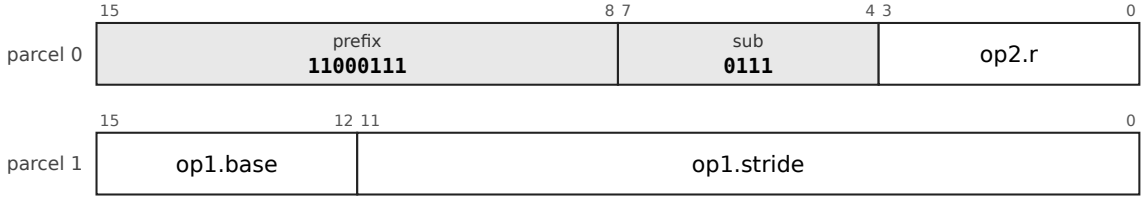
ADD: DPF.D28, prefix 111100001011, sub-opcode 0111; operands dpd28s-x1, reg4; 6 bytes



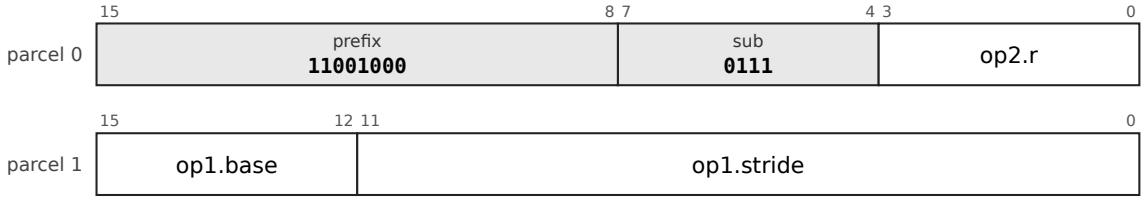
ADD: SPADJF, prefix 111100101111010, sub-opcode 0; operands imm32s, k-sp; 6 bytes



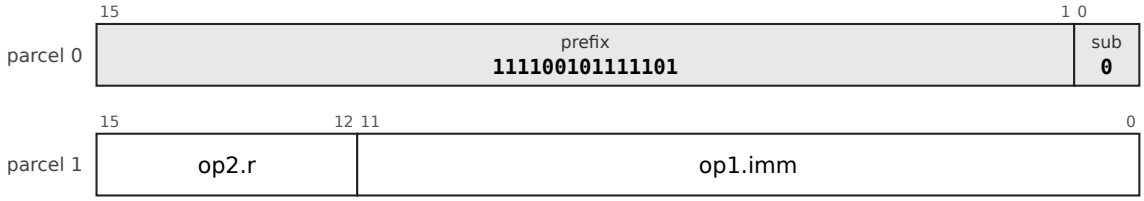
ADD: LDF.PI12, prefix 11000111, sub-opcode 0111; operands pi4-s12, reg4; 4 bytes



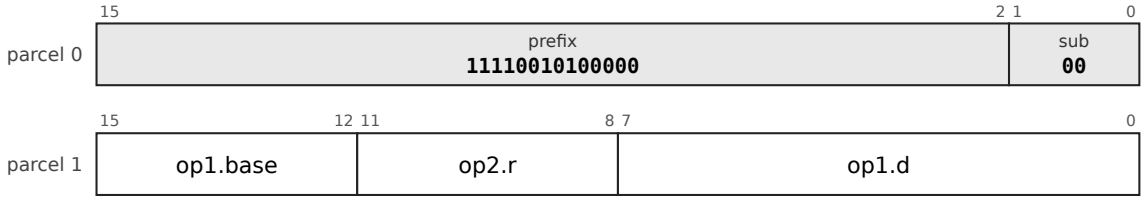
ADD: LDF.PD12, prefix 11001000, sub-opcode 0111; operands pd4-s12, reg4; 4 bytes



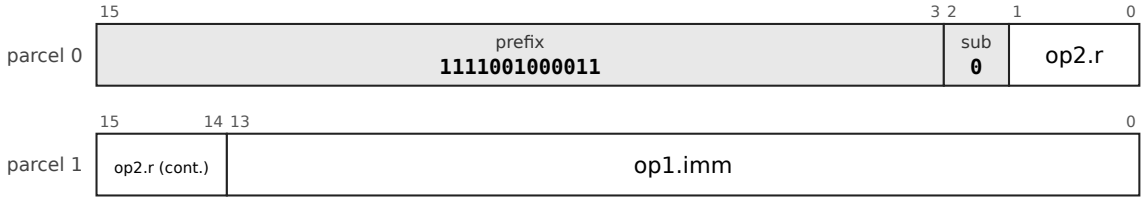
ADD: LDI12/b, prefix 111100101111101, sub-opcode 0; operands imm12s, reg4; 4 bytes



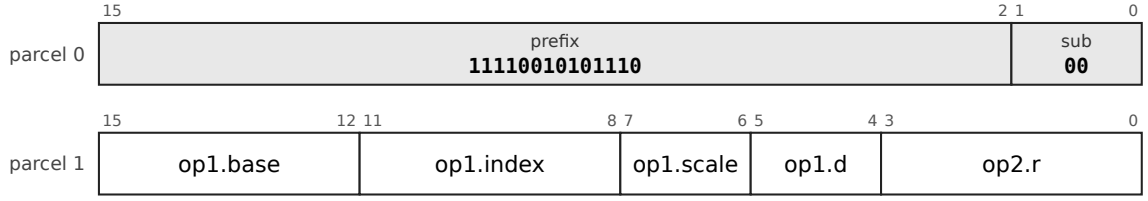
ADD: LDBu8/c, prefix 11110010100000, sub-opcode 00; operands b4d8u-xsz, reg4; 4 bytes



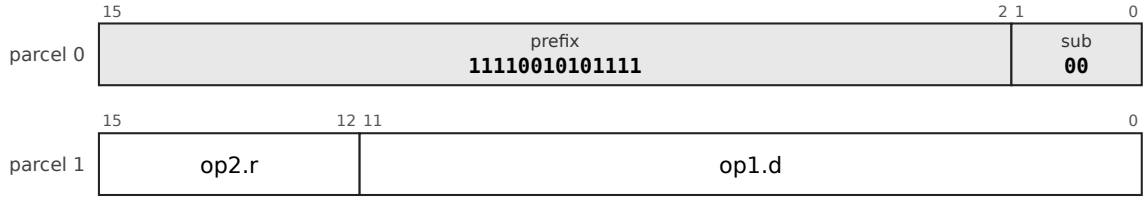
ADD: LDI14/b, prefix 1111001000011, sub-opcode 0; operands imm14s, reg4; 4 bytes



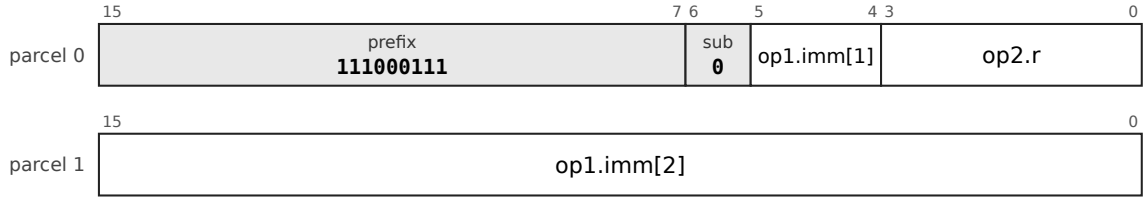
ADD: LDX2/c, prefix 11110010101110, sub-opcode 00; operands x4-4d2s, reg4; 4 bytes



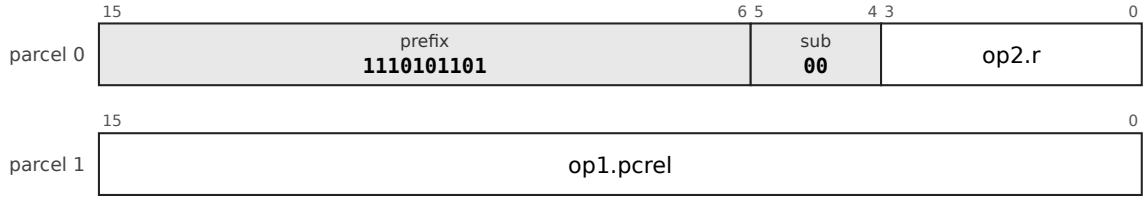
ADD: LDS12/c, prefix 11110010101111, sub-opcode 00; operands spd12u-xsz, reg4; 4 bytes



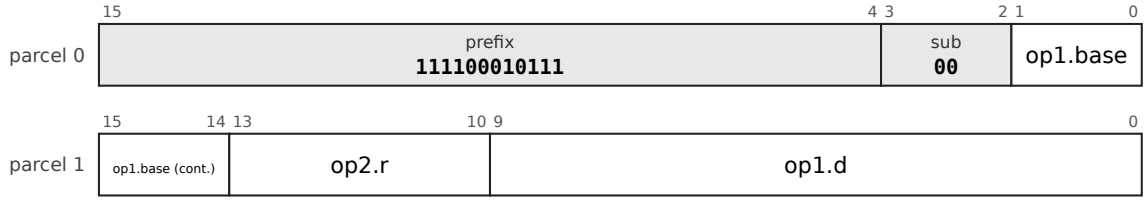
ADD: LDI18/b, prefix 111000111, sub-opcode 0; operands imm18s, reg4; 4 bytes



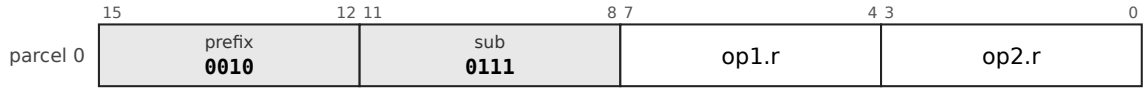
ADD: LDPC16/c, prefix 1110101101, sub-opcode 00; operands pc16-u1, reg4; 4 bytes



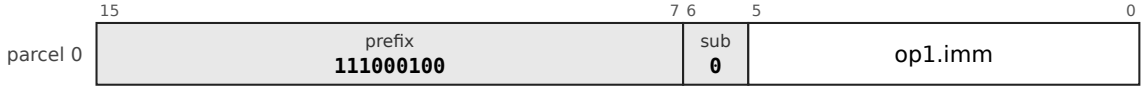
ADD: LDBs10/c, prefix 111100010111, sub-opcode 00; operands b4d10s-xsz, reg4; 4 bytes



ADD: RRF, prefix 0010, sub-opcode 0111; operands reg4, reg4; 2 bytes



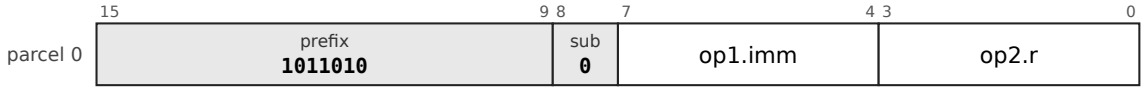
ADD: SPADJ6/a, prefix 111000100, sub-opcode 0; operands imm6u-x8, k-sp; 2 bytes



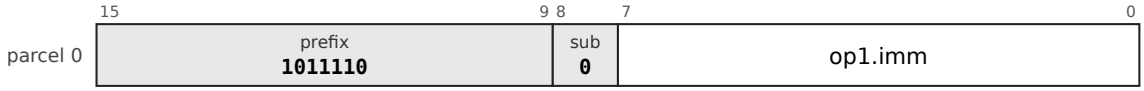
ADD: IMM3/b, prefix 11010010, sub-opcode 0; operands imm3s, reg4; 2 bytes



ADD: IMM4/b, prefix 1011010, sub-opcode 0; operands imm4s, reg4; 2 bytes



ADD: SPADJ8/a, prefix 1011110, sub-opcode 0; operands imm8u-x8, k-sp; 2 bytes

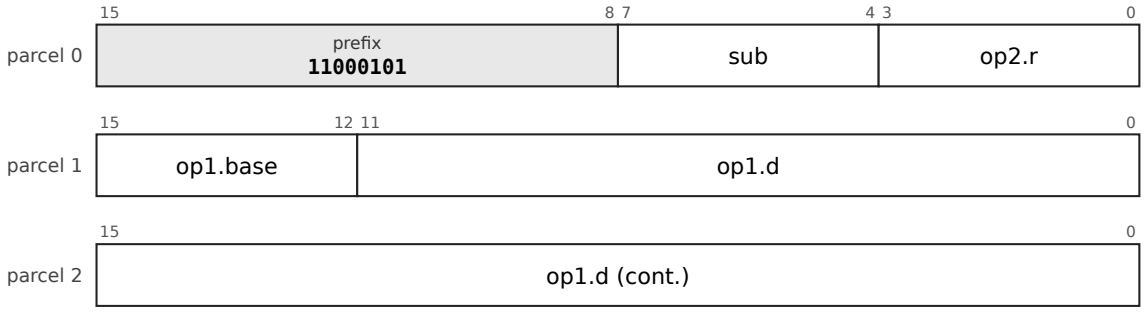


Mnemonics: `ADD`

Syntax

```
ADD rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-SPADJF-1`, `16-LDI12-b-1`, `16-LDBu8-c-1`, `16-SPADJ6-a-1`, `16-LDI14-b-1`, `16-LDX2-c-1`, `16-IMM3-b-1`, `16-LDS12-c-1`, `16-LDI18-b-1`, `16-LDPC16-c-1`, `16-LDBs10-c-1`, `16-IMM4-b-1`, `16-SPADJ8-a-1`.

Operation

```
Rd ← Rd + rm
```

Flags N and Z from the result, V set on signed overflow, C set on a carry out of bit 63.

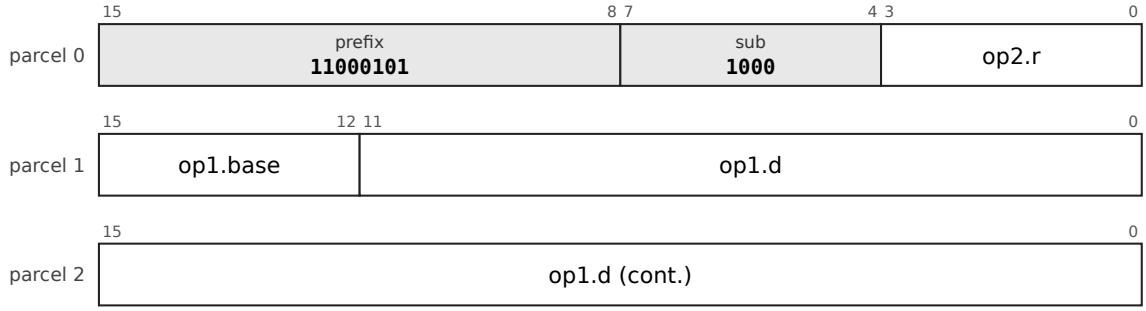
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

All ALU operations are 64 bits, with no size field.

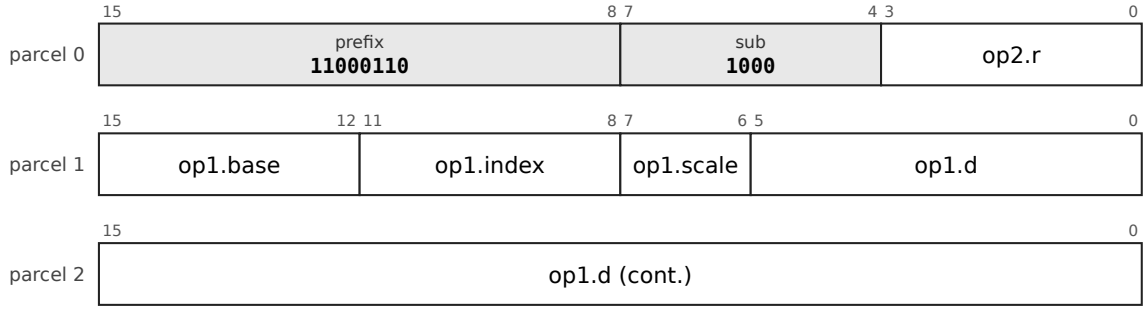
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

SUB: Subtract

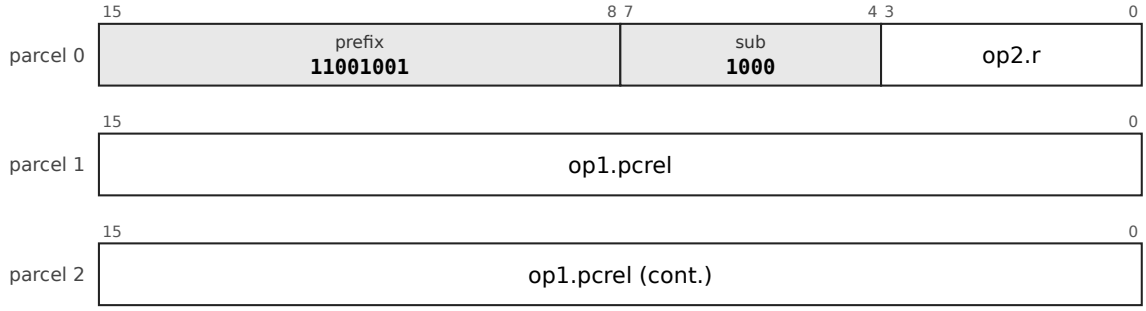
SUB: LDF.B28, prefix 11000101, sub-opcode 1000; operands b4d28s-x1, reg4; 6 bytes



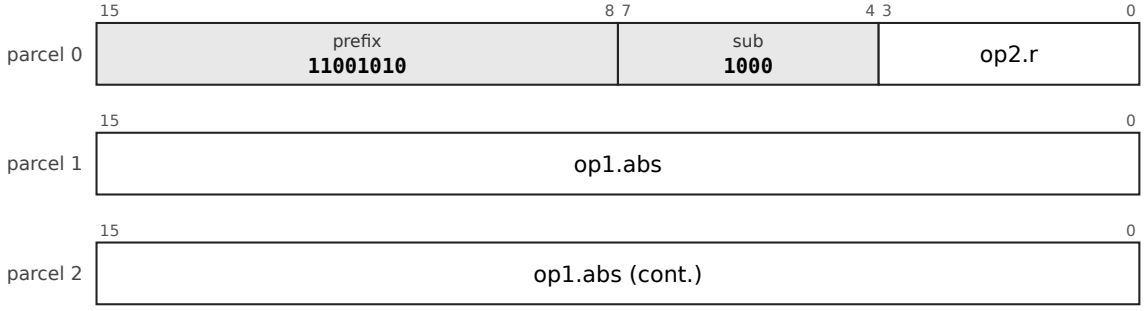
SUB: LDF.X22, prefix 11000110, sub-opcode 1000; operands x4-4d22s, reg4; 6 bytes



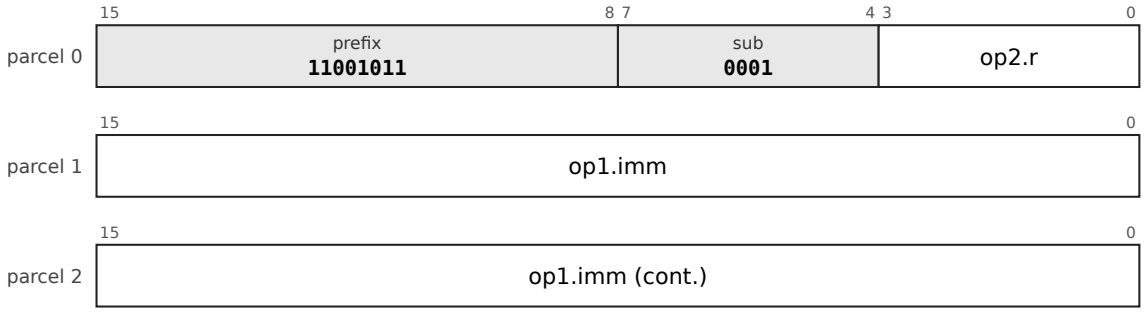
SUB: LDF.PC32, prefix 11001001, sub-opcode 1000; operands pc32-u1, reg4; 6 bytes



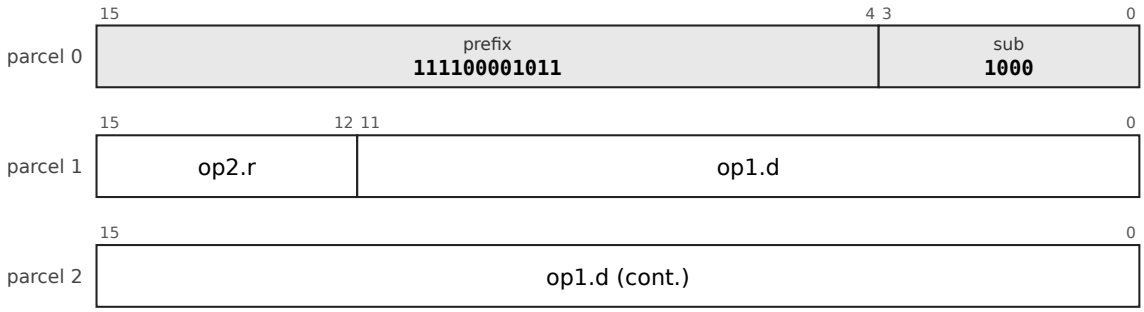
SUB: LDF.A32, prefix 11001010, sub-opcode 1000; operands abs32, reg4; 6 bytes



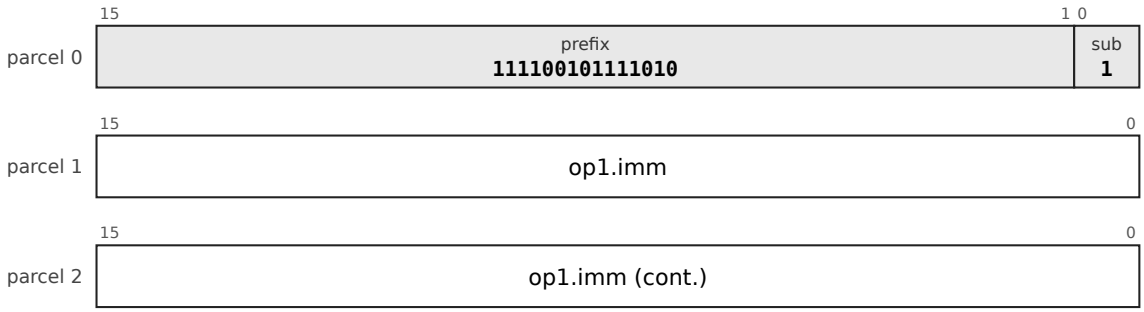
SUB: LDFI.I32, prefix 11001011, sub-opcode 0001; operands imm32s, reg4; 6 bytes



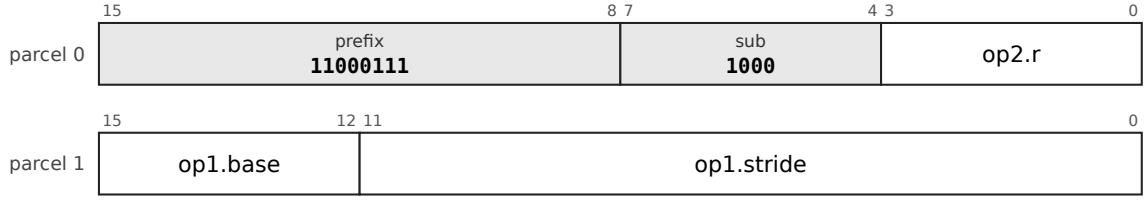
SUB: DPF.D28, prefix 111100001011, sub-opcode 1000; operands dpd28s-x1, reg4; 6 bytes



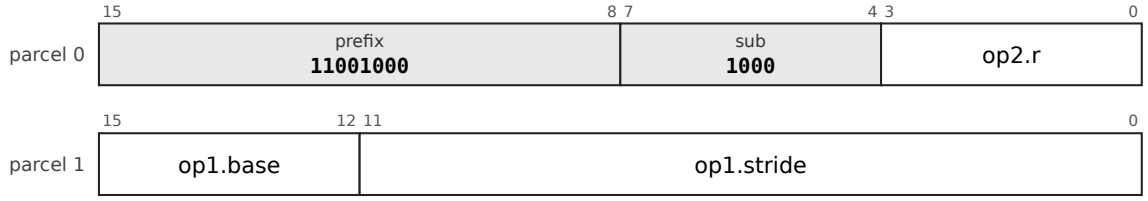
SUB: SPADJF, prefix 111100101111010, sub-opcode 1; operands imm32s, k-sp; 6 bytes



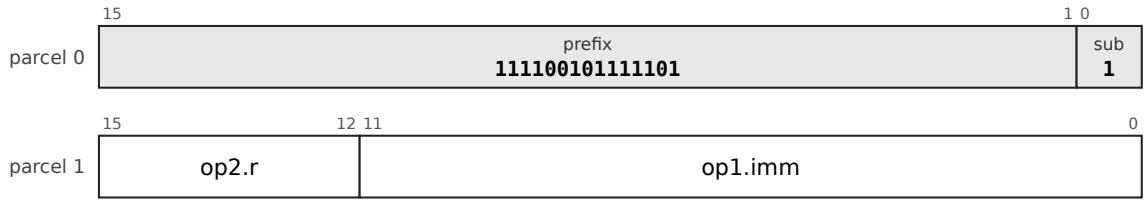
SUB: LDF.PI12, prefix 11000111, sub-opcode 1000; operands pi4-s12, reg4; 4 bytes



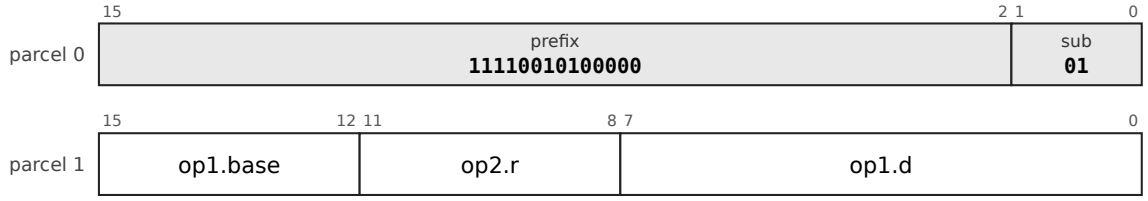
SUB: LDF.PD12, prefix 11001000, sub-opcode 1000; operands pd4-s12, reg4; 4 bytes



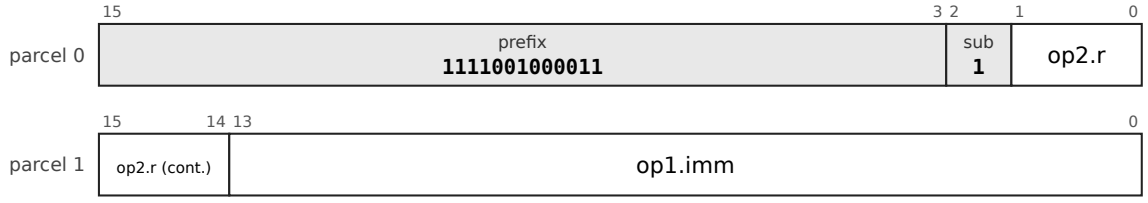
SUB: LDI12/b, prefix 111100101111101, sub-opcode 1; operands imm12s, reg4; 4 bytes



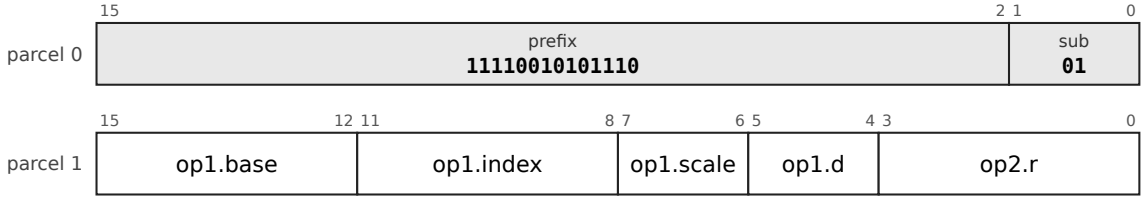
SUB: LDBu8/c, prefix 11110010100000, sub-opcode 01; operands b4d8u-xsz, reg4; 4 bytes



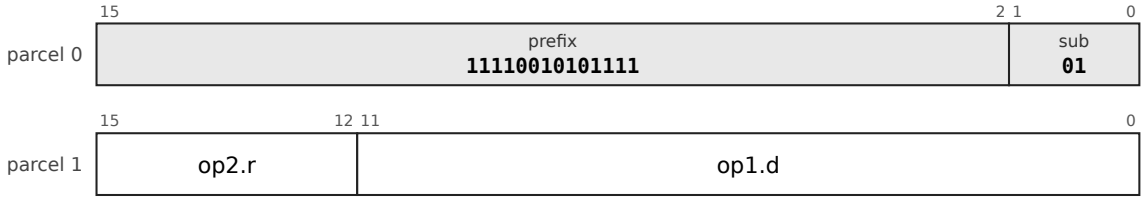
SUB: LDI14/b, prefix 1111001000011, sub-opcode 1; operands imm14s, reg4; 4 bytes



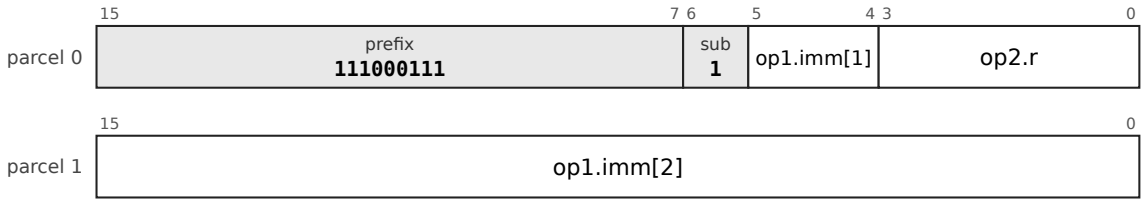
SUB: LDX2/c, prefix 11110010101110, sub-opcode 01; operands x4-4d2s, reg4; 4 bytes



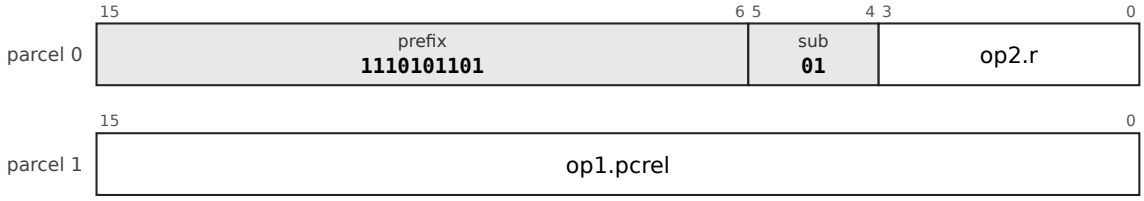
SUB: LDS12/c, prefix 11110010101111, sub-opcode 01; operands spd12u-xsz, reg4; 4 bytes



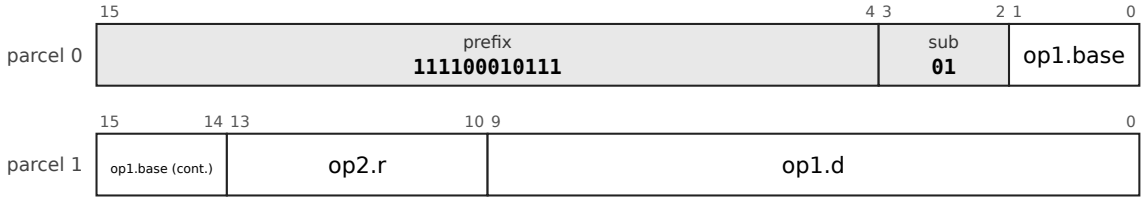
SUB: LDI18/b, prefix 111000111, sub-opcode 1; operands imm18s, reg4; 4 bytes



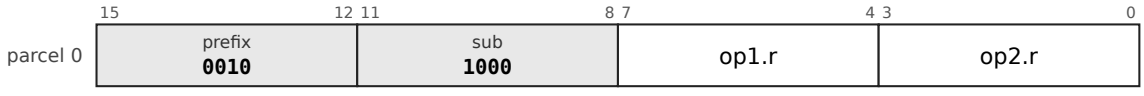
SUB: LDPC16/c, prefix 1110101101, sub-opcode 01; operands pc16-u1, reg4; 4 bytes



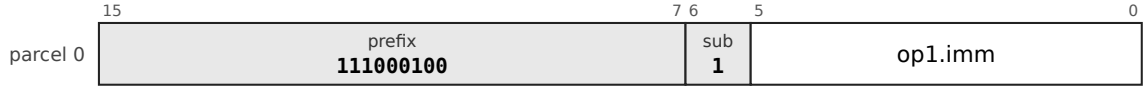
SUB: LDBs10/c, prefix 111100010111, sub-opcode 01; operands b4d10s-xsz, reg4; 4 bytes



SUB: RRF, prefix 0010, sub-opcode 1000; operands reg4, reg4; 2 bytes



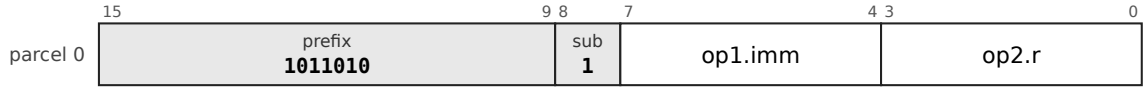
SUB: SPADJ6/a, prefix 111000100, sub-opcode 1; operands imm6u-x8, k-sp; 2 bytes



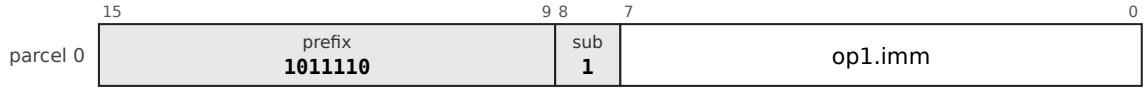
SUB: IMM3/b, prefix 11010010, sub-opcode 1; operands imm3s, reg4; 2 bytes



SUB: IMM4/b, prefix 1011010, sub-opcode 1; operands imm4s, reg4; 2 bytes



SUB: SPADJ8/a, prefix 1011110, sub-opcode 1; operands imm8u-x8, k-sp; 2 bytes

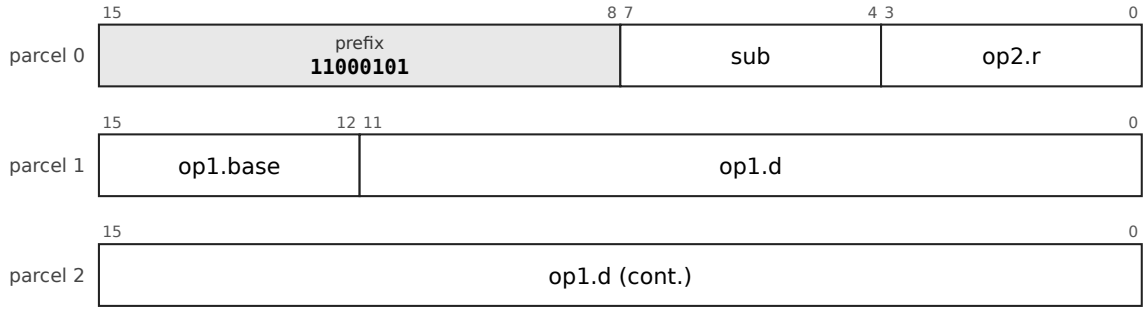


Mnemonics: `SUB`

Syntax

```
SUB rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-SPADJF-1`, `16-LDI12-b-1`, `16-LDBu8-c-1`, `16-SPADJ6-a-1`, `16-LDI14-b-1`, `16-LDX2-c-1`, `16-IMM3-b-1`, `16-LDS12-c-1`, `16-LDI18-b-1`, `16-LDPC16-c-1`, `16-LDBs10-c-1`, `16-IMM4-b-1`, `16-SPADJ8-a-1`.

Operation

```
Rd ← Rd - rm
```

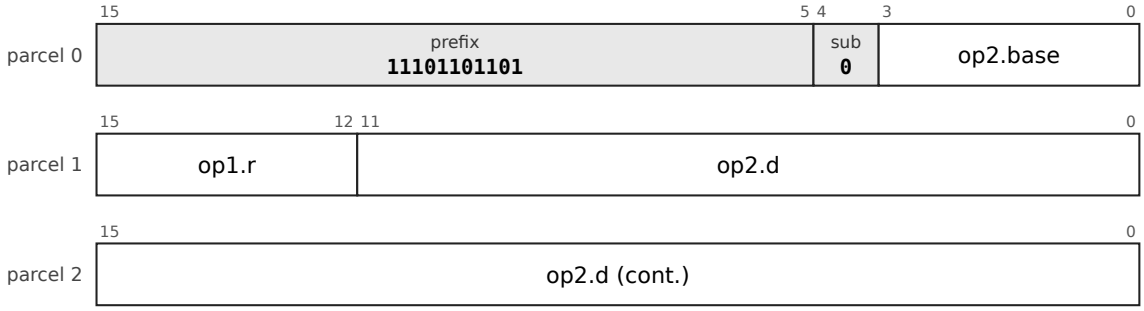
Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

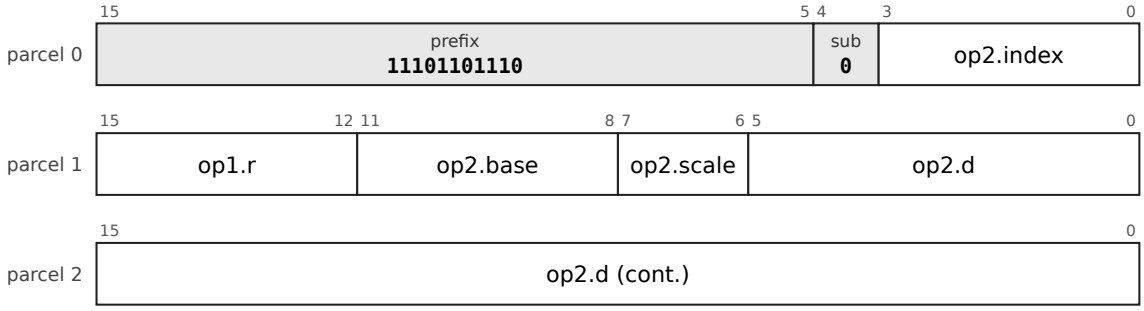
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

CMP: Compare

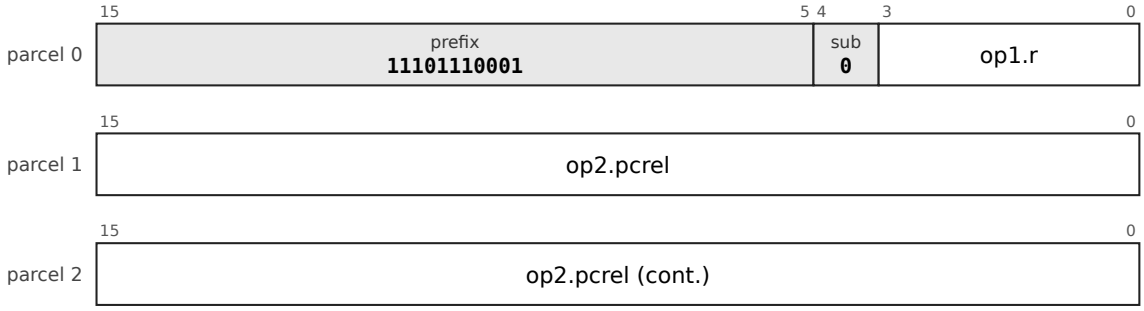
CMP: CMF.B28, prefix 11101101101, sub-opcode 0; operands reg4, b4d28s-x1; 6 bytes



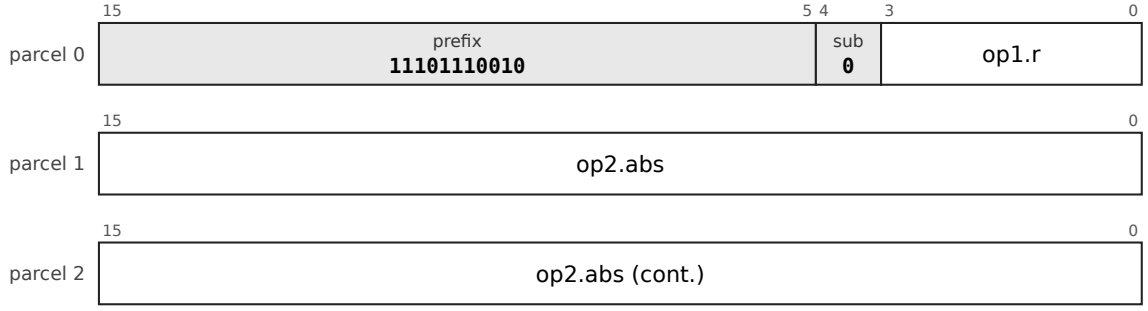
CMP: CMF.X22, prefix 11101101110, sub-opcode 0; operands reg4, x4-4d22s; 6 bytes



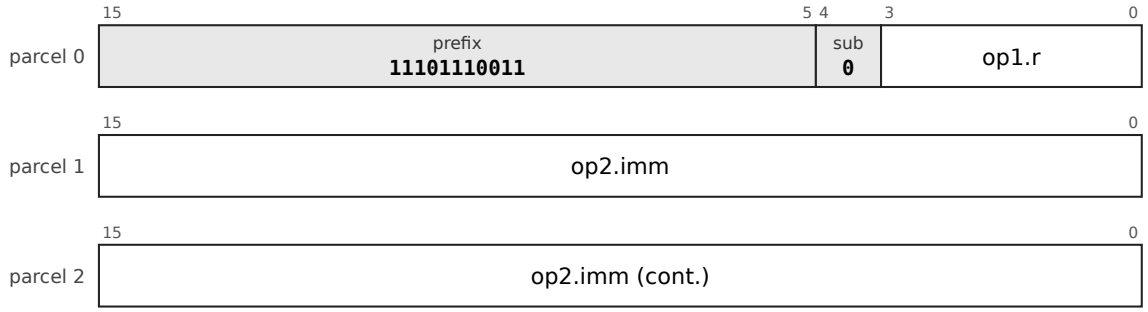
CMP: CMF.PC32, prefix 11101110001, sub-opcode 0; operands reg4, pc32-u1; 6 bytes



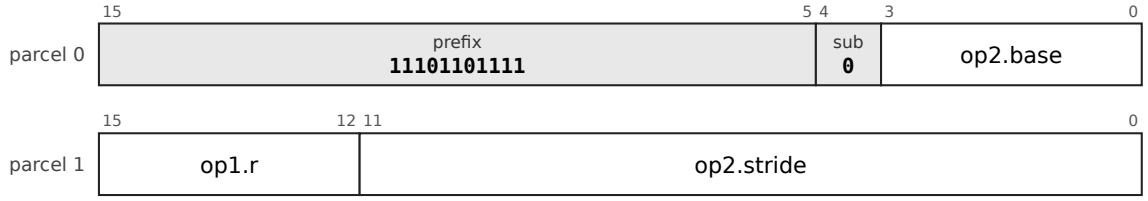
CMP: CMF.A32, prefix 11101110010, sub-opcode 0; operands reg4, abs32; 6 bytes



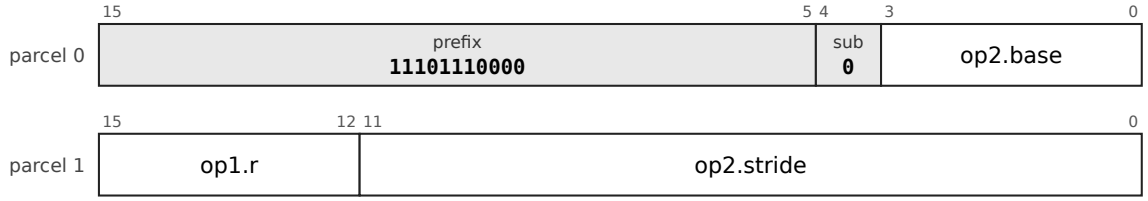
CMP: CMF.I32, prefix 11101110011, sub-opcode 0; operands reg4, imm32s; 6 bytes



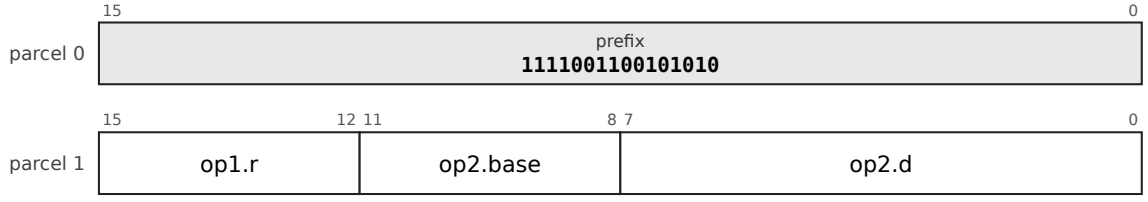
CMP: CMF.PI12, prefix 11101101111, sub-opcode 0; operands reg4, pi4-s12; 4 bytes



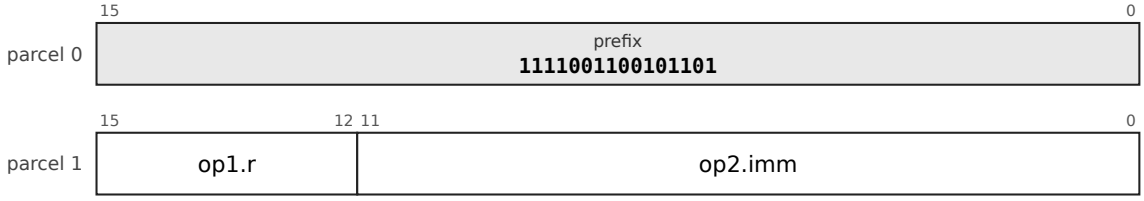
CMP: CMF.PD12, prefix 11101110000, sub-opcode 0; operands reg4, pd4-s12; 4 bytes



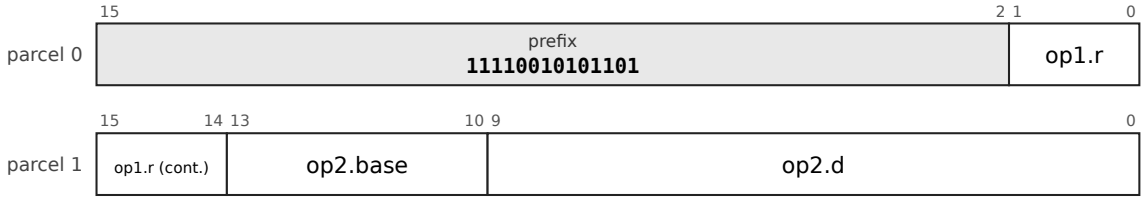
CMP: CMBu8/a, prefix 1111001100101010; operands reg4, b4d8u-xsz; 4 bytes



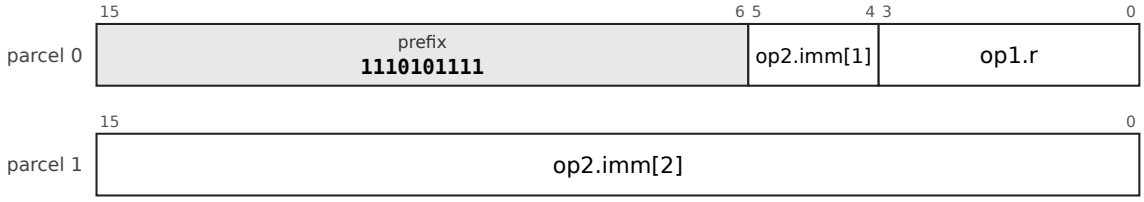
CMP: CMI12/a, prefix 1111001100101101; operands reg4, imm12s; 4 bytes



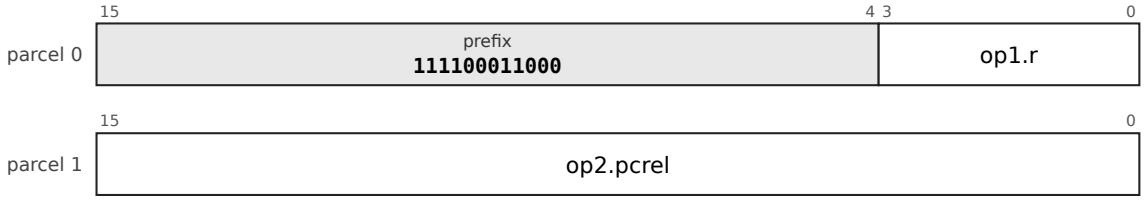
CMP: CMBu10/a, prefix 11110010101101; operands reg4, b4d10u-xsz; 4 bytes



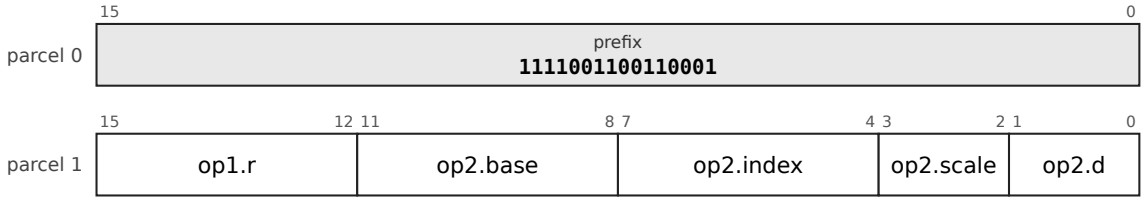
CMP: CMI18/a, prefix 1110101111; operands reg4, imm18s; 4 bytes



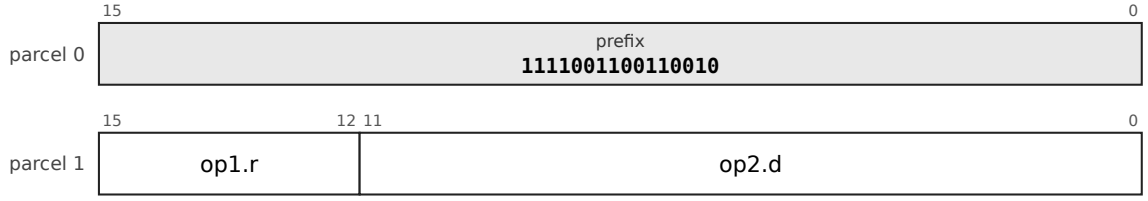
CMP: CMPC16/a, prefix 111100011000; operands reg4, pc16-u1; 4 bytes



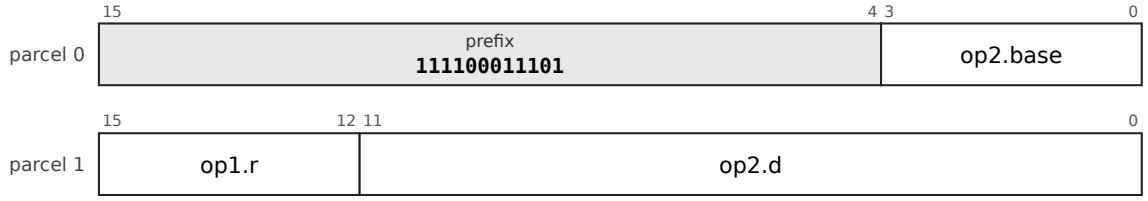
CMP: CMX2/a, prefix 1111001100110001; operands reg4, x4-4d2s; 4 bytes



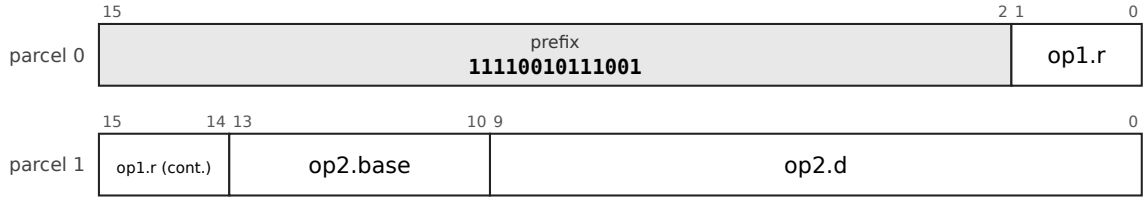
CMP: CMS12/a, prefix 1111001100110010; operands reg4, spd12u-xsz; 4 bytes



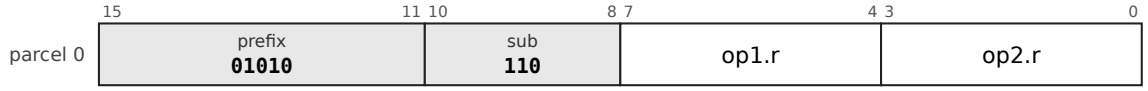
CMP: CMBu12/a, prefix 111100011101; operands reg4, b4d12u-xsz; 4 bytes



CMP: CMBs10/a, prefix 11110010111001; operands reg4, b4d10s-xsz; 4 bytes



CMP: RRF, prefix 01010, sub-opcode 110; operands reg4, reg4; 2 bytes

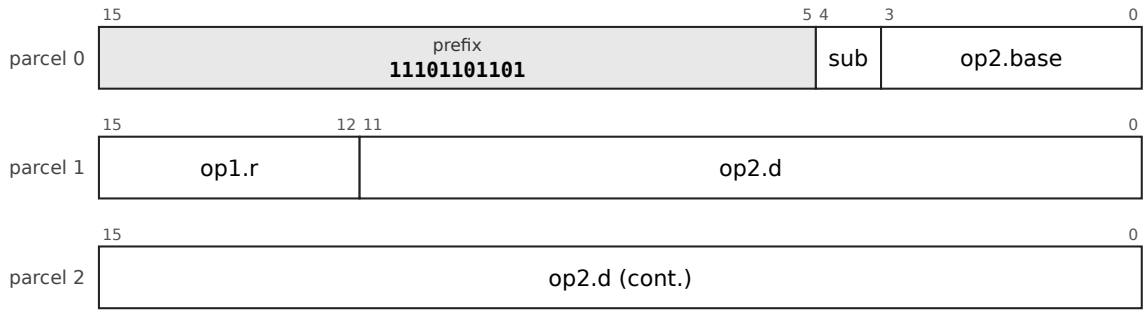


Mnemonics: `CMP`

Syntax

```
CMP Rd, rm        ; register first
```

Encoding



The chart is the leaf `16-CMF-B28-1`, CMF.B28, prefix `11101101101`. `op1` is `reg4`: register, 4 bits. `op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-CMF-B28-1`, `16-CMF-X22-1`, `16-CMF-PI12-1`, `16-CMF-PD12-1`, `16-CMF-PC32-1`, `16-CMF-A32-1`, `16-CMF-I32-1`, `16-RRF-2`, `16-CMBu8-a-1`, `16-CMI12-a-1`, `16-CMBu10-a-1`, `16-CMI18-a-1`, `16-CMPC16-a-1`, `16-CMX2-a-1`, `16-CMS12-a-1`, `16-CMBu12-a-1`, `16-CMBs10-a-1`.

Operation

```
tmp ← Rd - rm         ; the result is discarded
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

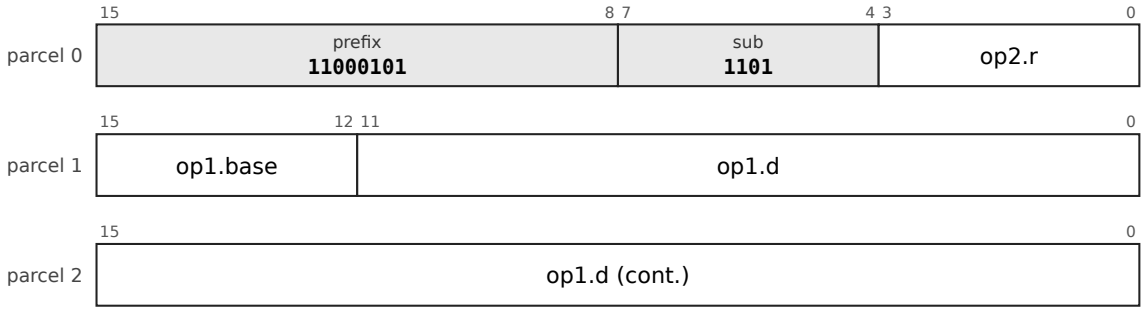
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The operands are in the order register, then the other operand, the reverse of the PDP-11 `CMP src,dst`: `CMP Rd, rm` computes `Rd - rm`, so a compare of a register against an immediate or memory operand does not reverse its branch condition. After `CMP`, `BHIS`/`BCC` mean unsigned `>=` (no borrow) and `BLO`/`BCS` mean a borrow occurred.

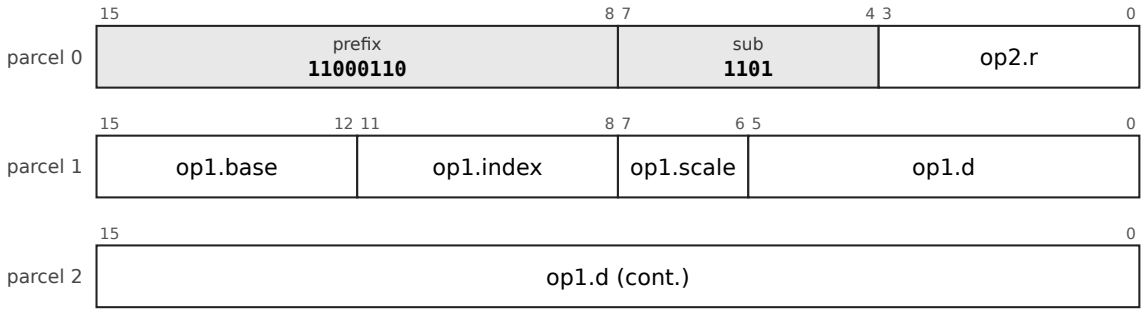
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MUL: Multiply, low 64 bits

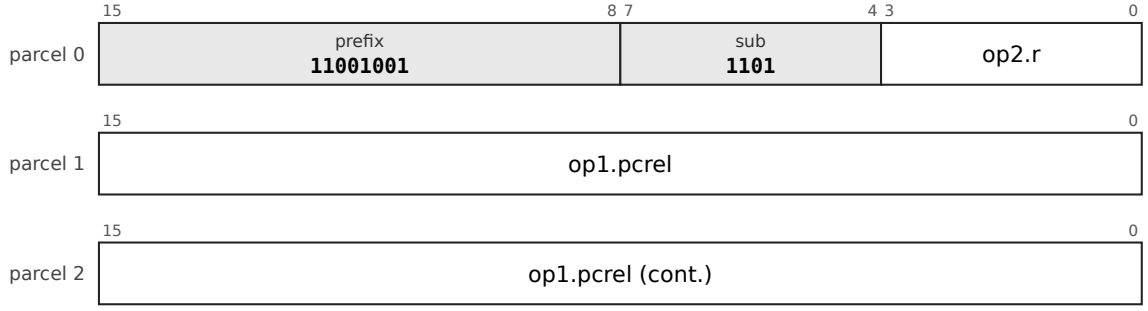
MUL: LDF.B28, prefix 11000101, sub-opcode 1101; operands b4d28s-x1, reg4; 6 bytes



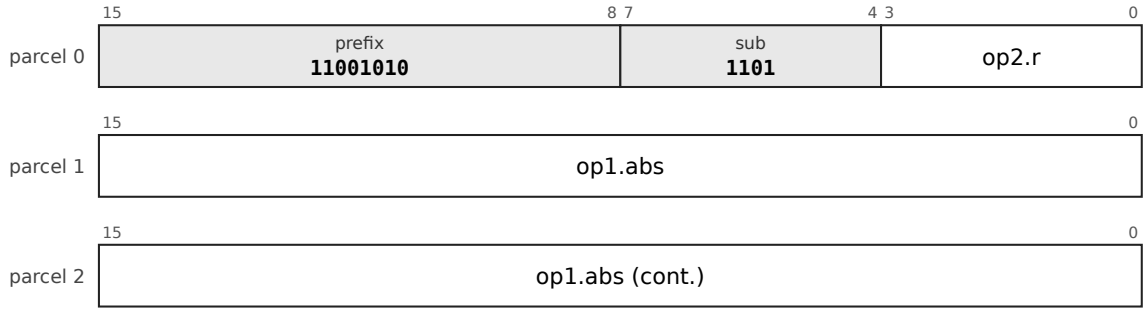
MUL: LDF.X22, prefix 11000110, sub-opcode 1101; operands x4-4d22s, reg4; 6 bytes



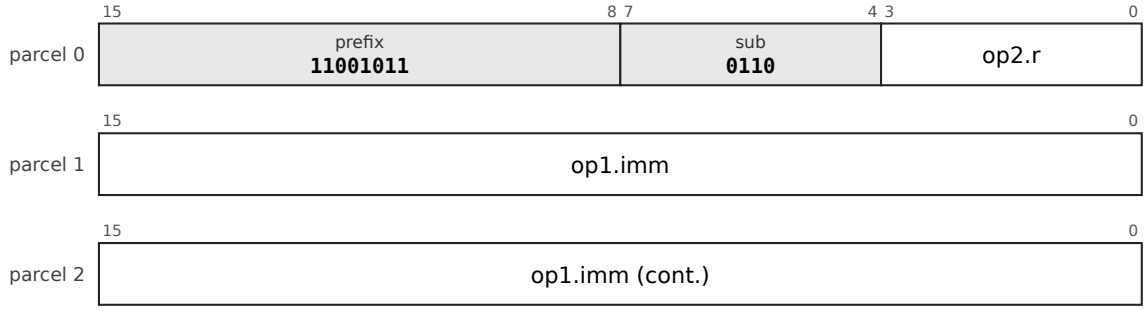
MUL: LDF.PC32, prefix 11001001, sub-opcode 1101; operands pc32-u1, reg4; 6 bytes



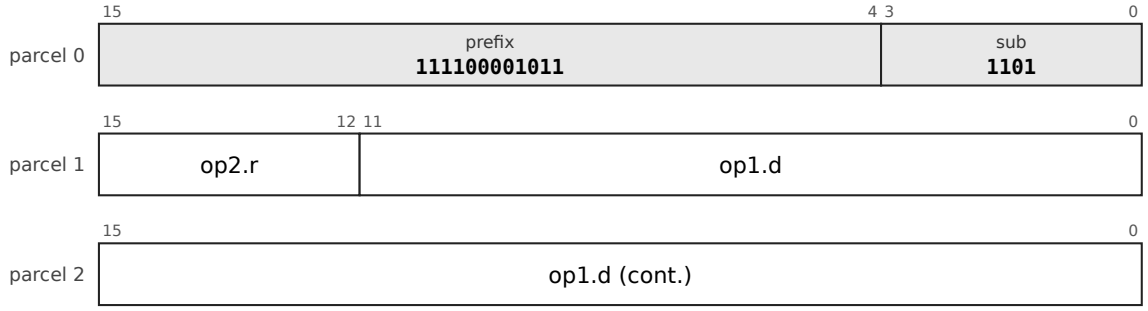
MUL: LDF.A32, prefix 11001010, sub-opcode 1101; operands abs32, reg4; 6 bytes



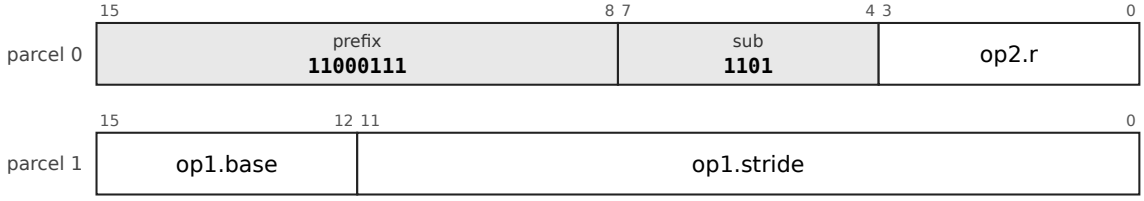
MUL: LDFI.I32, prefix 11001011, sub-opcode 0110; operands imm32s, reg4; 6 bytes



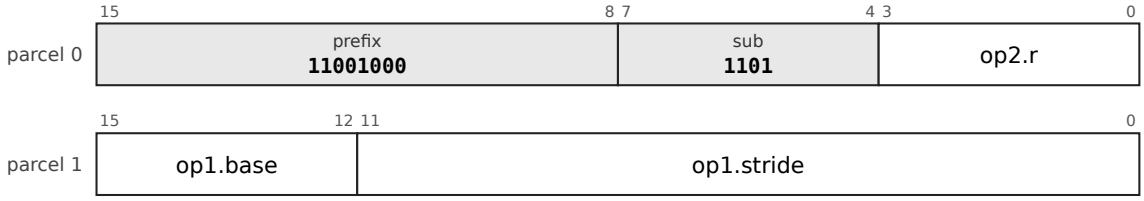
MUL: DPF.D28, prefix 111100001011, sub-opcode 1101; operands dpd28s-x1, reg4; 6 bytes



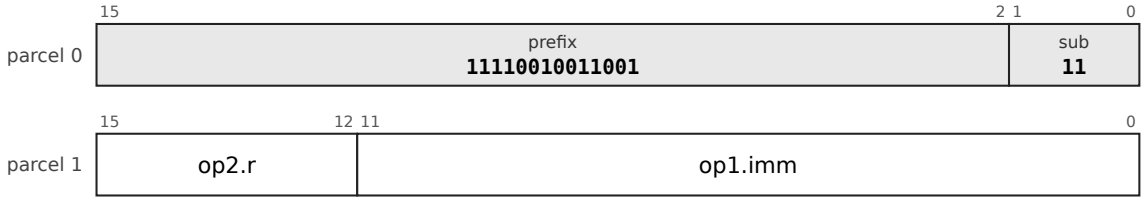
MUL: LDF.PI12, prefix 11000111, sub-opcode 1101; operands pi4-s12, reg4; 4 bytes



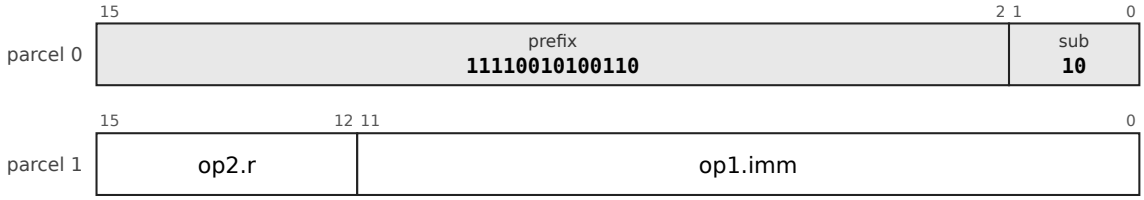
MUL: LDF.PD12, prefix 11001000, sub-opcode 1101; operands pd4-s12, reg4; 4 bytes



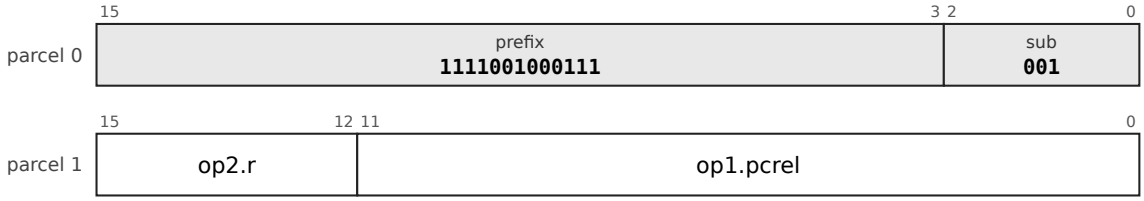
MUL: LDI12/d, prefix 11110010011001, sub-opcode 11; operands imm12s, reg4; 4 bytes



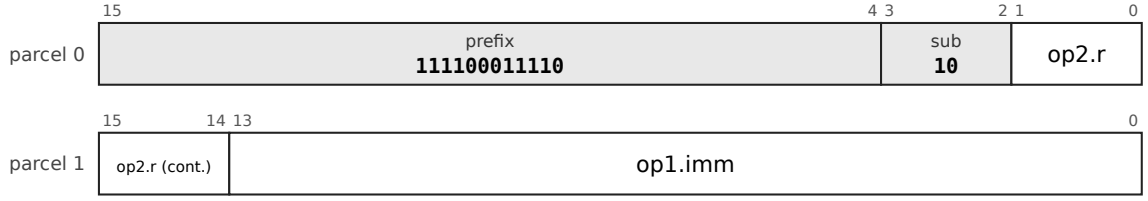
MUL: LDI12/c, prefix 11110010100110, sub-opcode 10; operands imm12s, reg4; 4 bytes



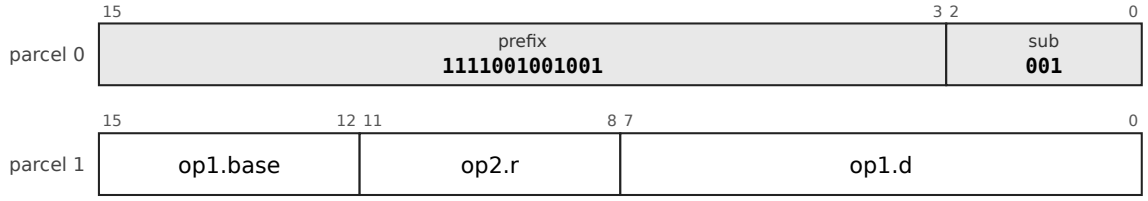
MUL: LDPC12/d, prefix 1111001000111, sub-opcode 001; operands pc12-u1, reg4; 4 bytes



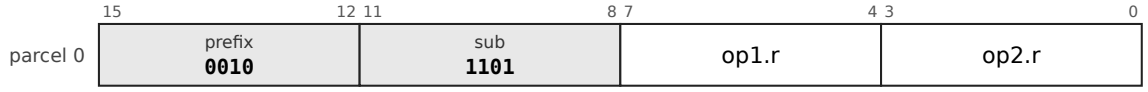
MUL: LDI14/c, prefix 111100011110, sub-opcode 10; operands imm14s, reg4; 4 bytes



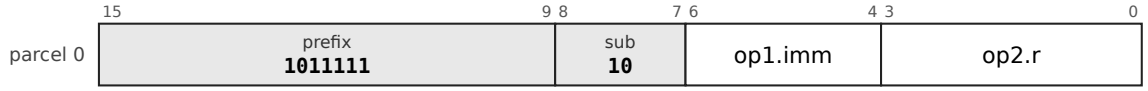
MUL: LDBs8/d, prefix 1111001001001, sub-opcode 001; operands b4d8s-xsz, reg4; 4 bytes



MUL: RRF, prefix 0010, sub-opcode 1101; operands reg4, reg4; 2 bytes



MUL: IMM3/c, prefix 1011111, sub-opcode 10; operands imm3s, reg4; 2 bytes

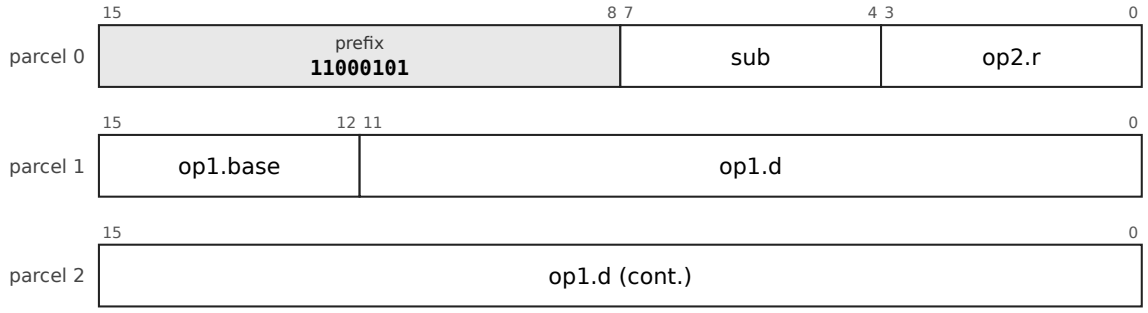


Mnemonics: `MUL` (aliases: `MULU`)

Syntax

```
MUL rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDI12-d-1`, `16-LDI12-c-1`, `16-LDPC12-d-1`, `16-IMM3-c-1`, `16-LDI14-c-1`, `16-LDBs8-d-1`.

Operation

```
Rd ← (Rd × rm)[63:0]        ; sign-agnostic
```

Flags `N` and `Z` from the result, `V` set when the signed 128-bit product does not fit in 64 bits, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

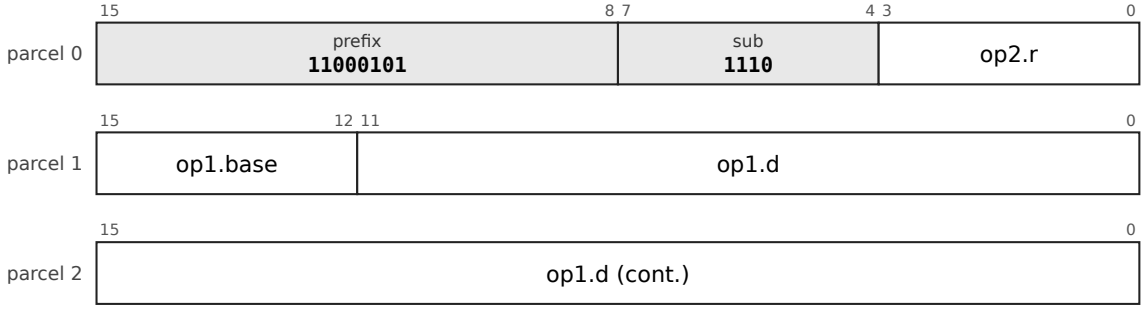
The low 64 bits of a two's-complement product are the same whether the operands are read as signed or unsigned, so there is one low-half multiply. The mnemonic `MULU` is an alias of `MUL`. The high half is `MULH` (signed) and `MULHU` (unsigned).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

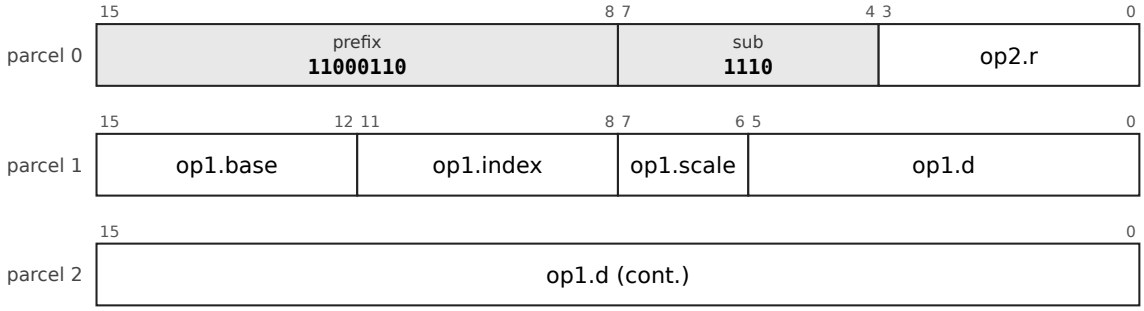
`MULU` is an alias of `MUL`: the assembler writes it as that word.

MULH: Multiply, high 64 bits, signed

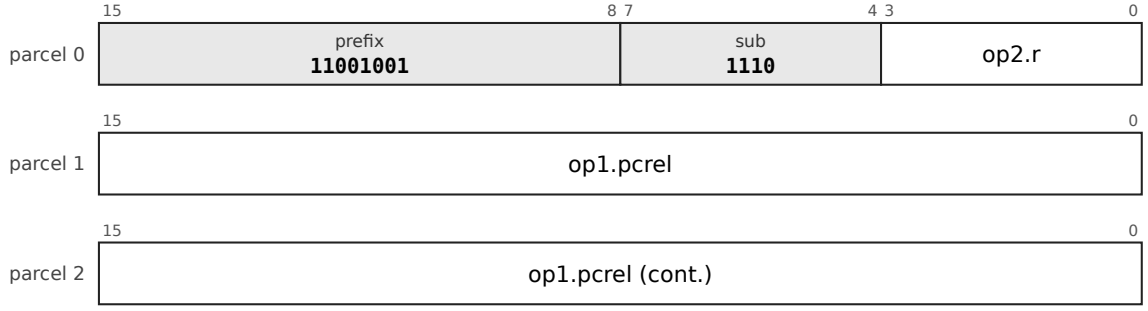
MULH: LDF.B28, prefix 11000101, sub-opcode 1110; operands b4d28s-x1, reg4; 6 bytes



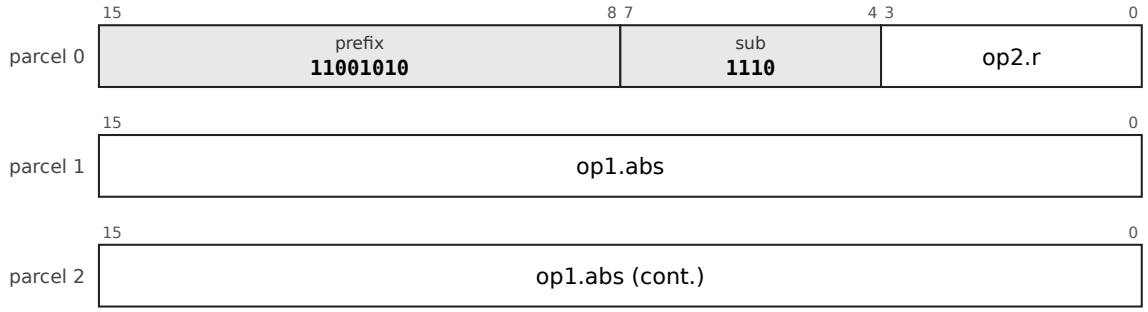
MULH: LDF.X22, prefix 11000110, sub-opcode 1110; operands x4-4d22s, reg4; 6 bytes



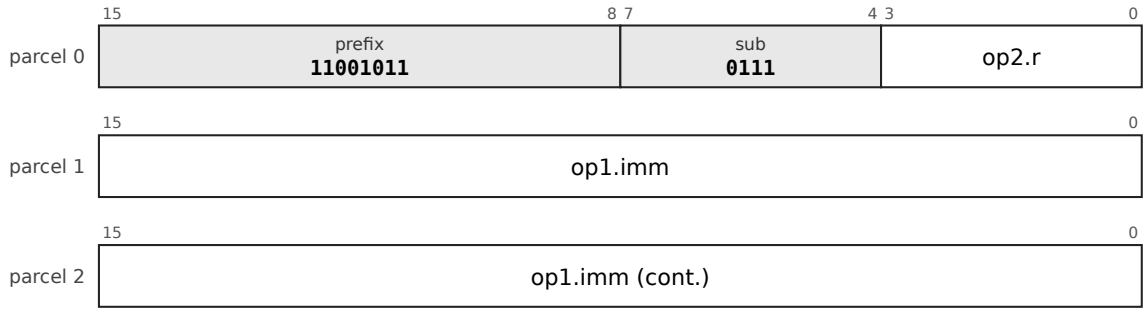
MULH: LDF.PC32, prefix 11001001, sub-opcode 1110; operands pc32-u1, reg4; 6 bytes



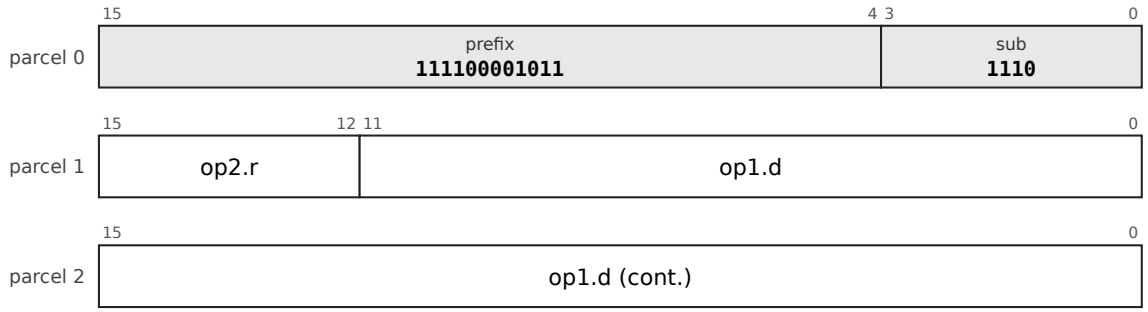
MULH: LDF.A32, prefix 11001010, sub-opcode 1110; operands abs32, reg4; 6 bytes



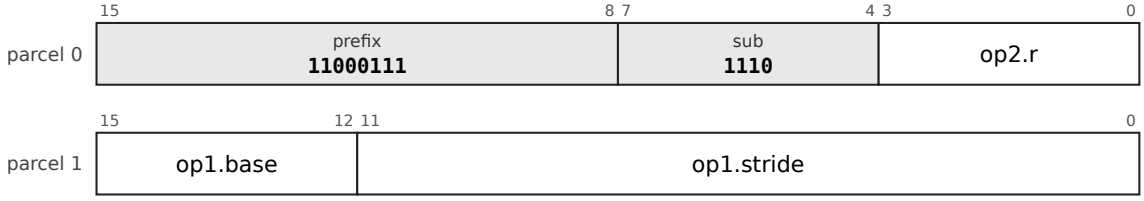
MULH: LDFI.I32, prefix 11001011, sub-opcode 0111; operands imm32s, reg4; 6 bytes



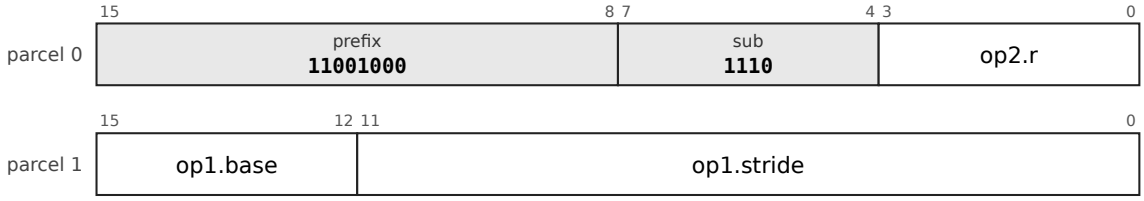
MULH: DPF.D28, prefix 111100001011, sub-opcode 1110; operands dpd28s-x1, reg4; 6 bytes



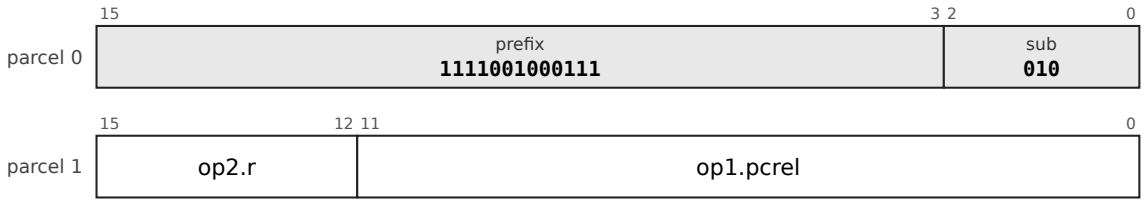
MULH: LDF.PI12, prefix 11000111, sub-opcode 1110; operands pi4-s12, reg4; 4 bytes



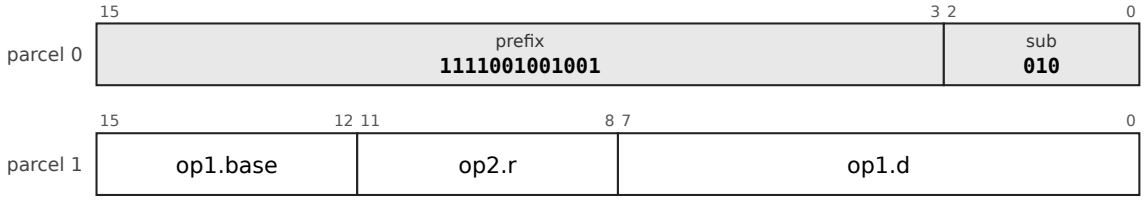
MULH: LDF.PD12, prefix 11001000, sub-opcode 1110; operands pd4-s12, reg4; 4 bytes



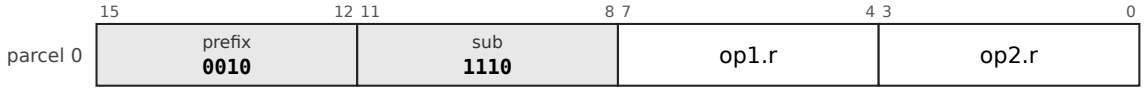
MULH: LDPC12/d, prefix 1111001000111, sub-opcode 010; operands pc12-u1, reg4; 4 bytes



MULH: LDBs8/d, prefix 1111001001001, sub-opcode 010; operands b4d8s-xsz, reg4; 4 bytes



MULH: RRF, prefix 0010, sub-opcode 1110; operands reg4, reg4; 2 bytes

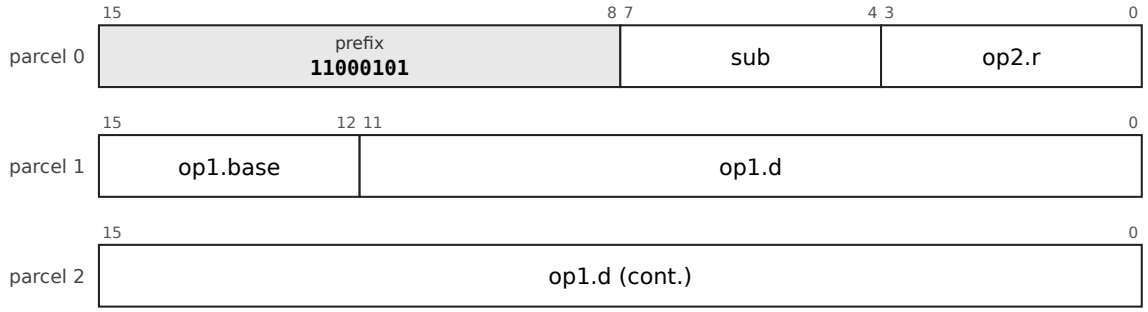


Mnemonics: `MULH`

Syntax

```
MULH rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDPC12-d-1`, `16-LDBs8-d-1`.

Operation

```
Rd ← (sext(Rd) × sext(rm))[127:64]
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `MULH`, division by a constant is a multiply by a reciprocal and a shift, which is what compilers emit.

Why it is here. The low 64 bits of a two's-complement product are identical signed or unsigned, so a second multiply (`MULU`) would duplicate `MUL`; what the architecture needs in that place is the *high* half of the product. Without it a 64 by 64 to 128-bit multiply, every bignum multiply, 128-bit fixed point and divide-by-a-constant by multiplying by the reciprocal are impossible. `MULH` is that instruction. It is also why a divide is rare in compiled code: compilers turn division by a constant into `MULH` and a shift.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

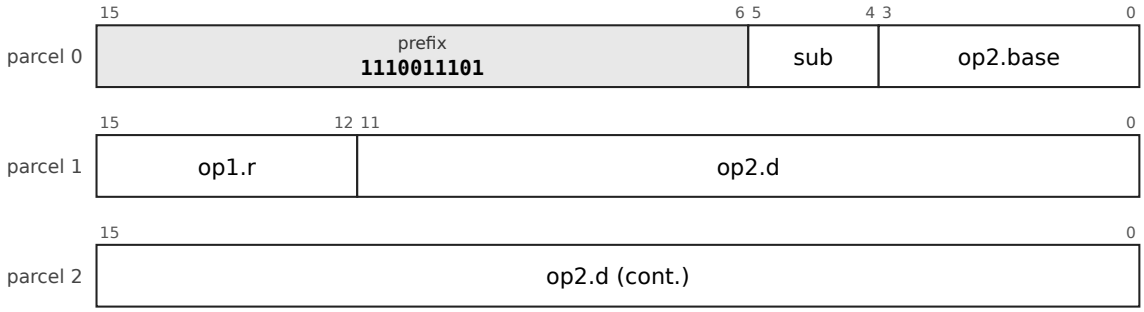
ADDM: ALU operation with a memory destination

Mnemonics: `ADDM`, `SUBM`, `BICM`, `BISM`

Syntax

```
ADDM Rs, rm
SUBM Rs, rm
BICM Rs, rm
BISM Rs, rm
```

Encoding



The chart is the leaf `16-STF-B28-2`, STF.B28, prefix `1110011101`. `op1` is `reg4`: register, 4 bits. `op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-STF-B28-2`, `16-STF-X22-2`, `16-STF-PI12-2`, `16-STF-PD12-2`, `16-STF-PC32-2`, `16-STF-A32-2`.

Operation

```
ADDM: M[EA] ← M[EA] + Rs
SUBM: M[EA] ← M[EA] - Rs
BICM: M[EA] ← M[EA] AND NOT Rs
BISM: M[EA] ← M[EA] OR Rs
```

Flags As for `ADD`, `SUB`, `BIC` and `BIS`.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Write permission is checked during the read access, so the write cannot fault after the read has succeeded.

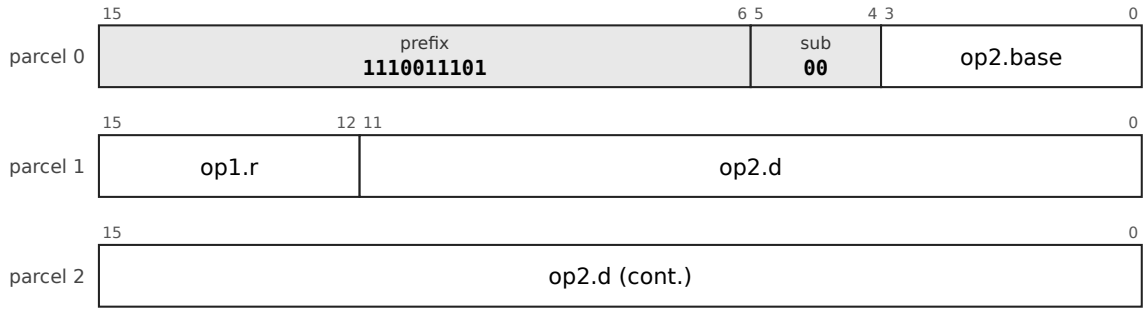
The forms with a memory destination are in the ordinary memory families of the encoding, with every addressing form (errata 87).

Why it is here. These are the read-modify-write forms with a memory destination. They cover the case where the second operand is a register, for example `BISM Rmask, flags(DP)`. The compiled corpus has 265 sites (1.6 KB) for them, measured for the original encoding, which is why they are not worth an opcode of their own there.

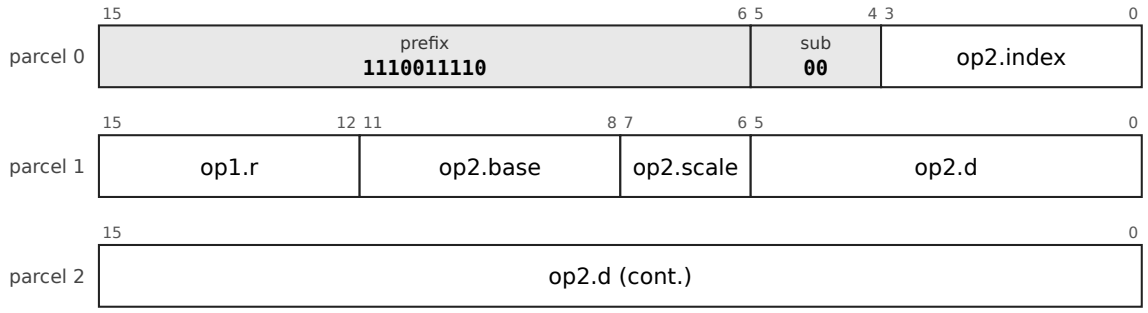
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ADDM

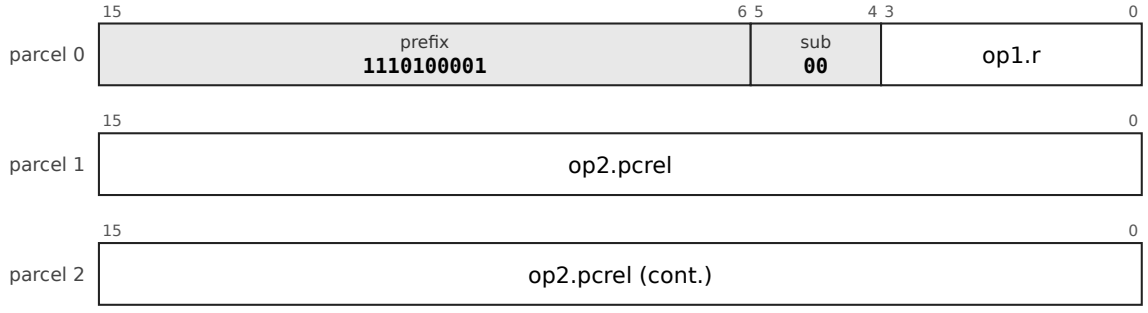
ADDM: STF.B28, prefix 1110011101, sub-opcode 00; operands reg4, b4d28s-x1; 6 bytes



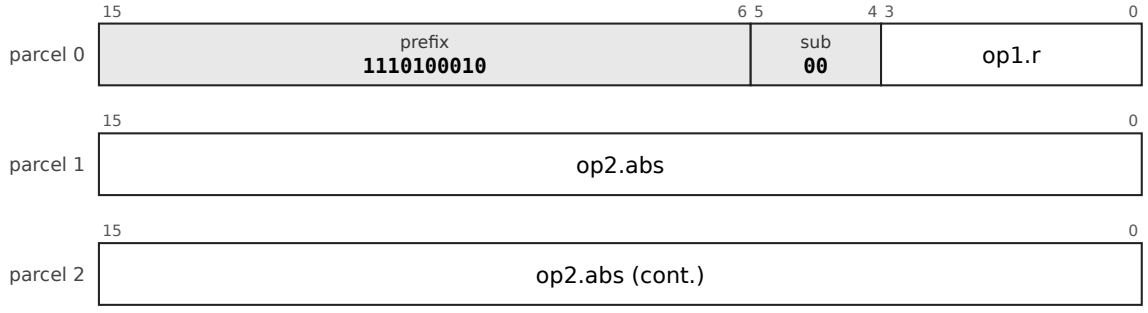
ADDM: STF.X22, prefix 1110011110, sub-opcode 00; operands reg4, x4-4d22s; 6 bytes



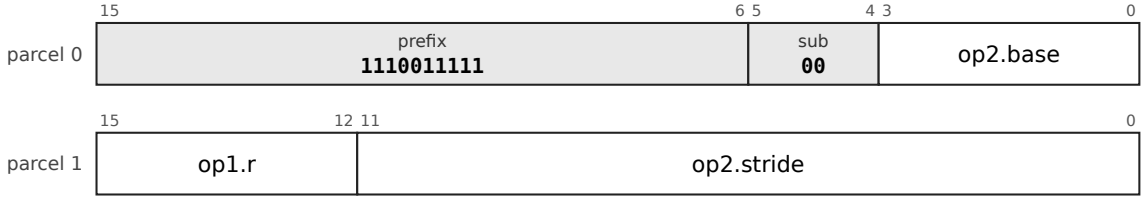
ADDM: STF.PC32, prefix 1110100001, sub-opcode 00; operands reg4, pc32-u1; 6 bytes



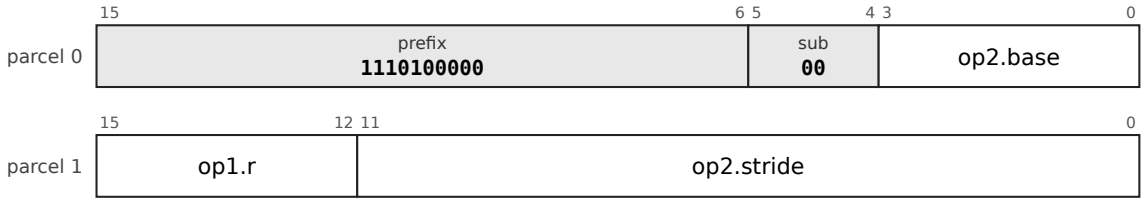
ADDM: STF.A32, prefix 1110100010, sub-opcode 00; operands reg4, abs32; 6 bytes



ADDM: STF.PI12, prefix 1110011111, sub-opcode 00; operands reg4, pi4-s12; 4 bytes



ADDM: STF.PD12, prefix 1110100000, sub-opcode 00; operands reg4, pd4-s12; 4 bytes

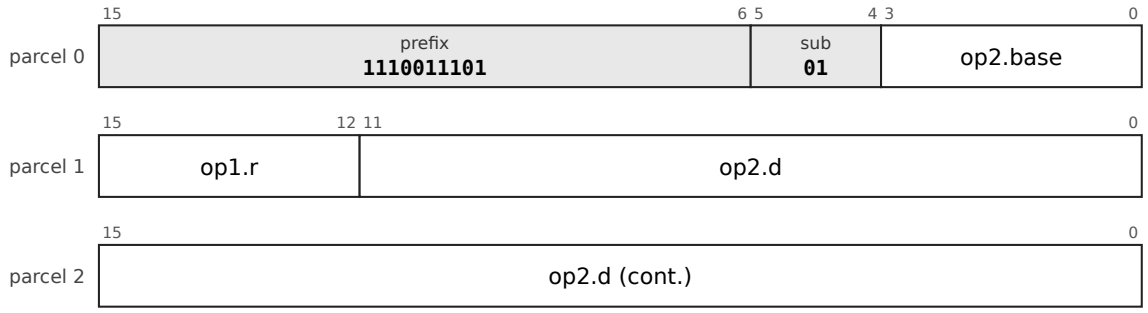



```
ADDM Rs, rm
```

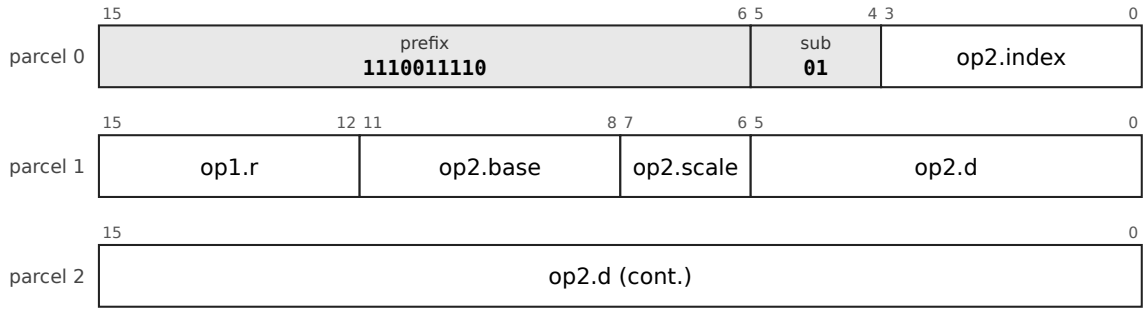
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

SUBM

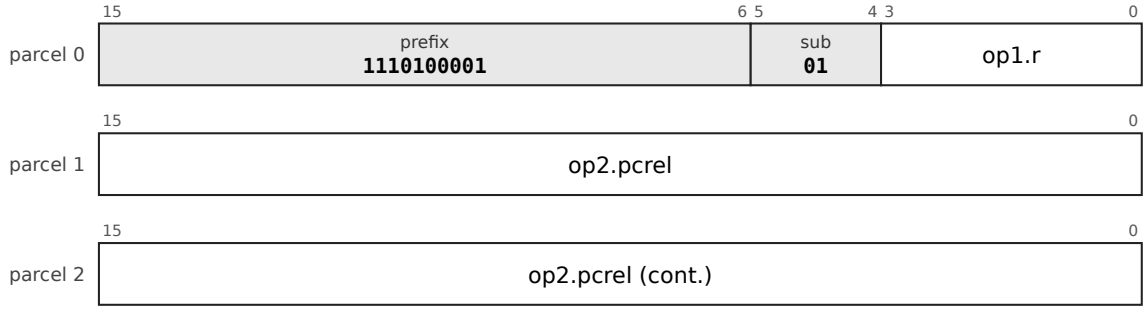
SUBM: STF.B28, prefix 1110011101, sub-opcode 01; operands reg4, b4d28s-x1; 6 bytes



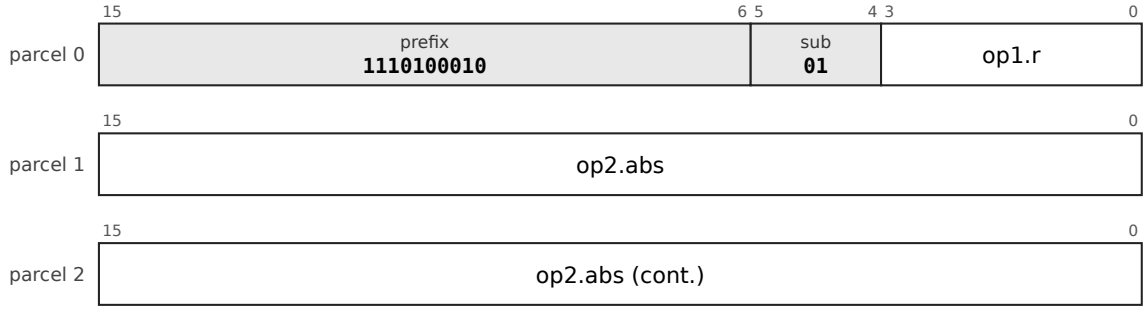
SUBM: STF.X22, prefix 1110011110, sub-opcode 01; operands reg4, x4-4d22s; 6 bytes



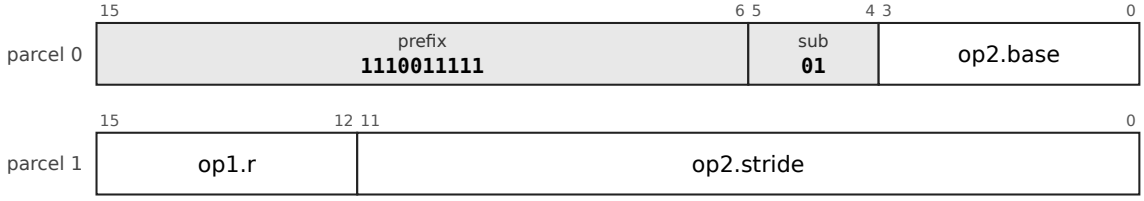
SUBM: STF.PC32, prefix 1110100001, sub-opcode 01; operands reg4, pc32-u1; 6 bytes



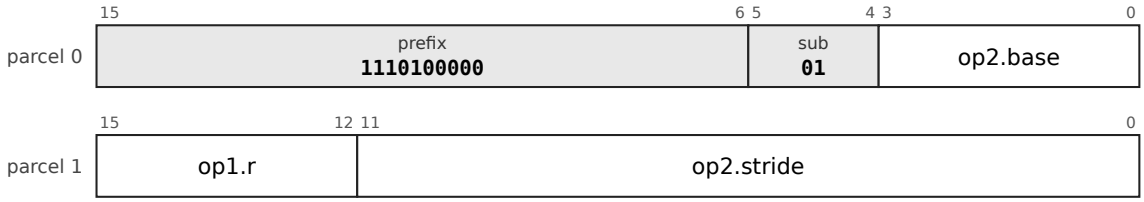
SUBM: STF.A32, prefix 1110100010, sub-opcode 01; operands reg4, abs32; 6 bytes



SUBM: STF.PI12, prefix 1110011111, sub-opcode 01; operands reg4, pi4-s12; 4 bytes



SUBM: STF.PD12, prefix 1110100000, sub-opcode 01; operands reg4, pd4-s12; 4 bytes

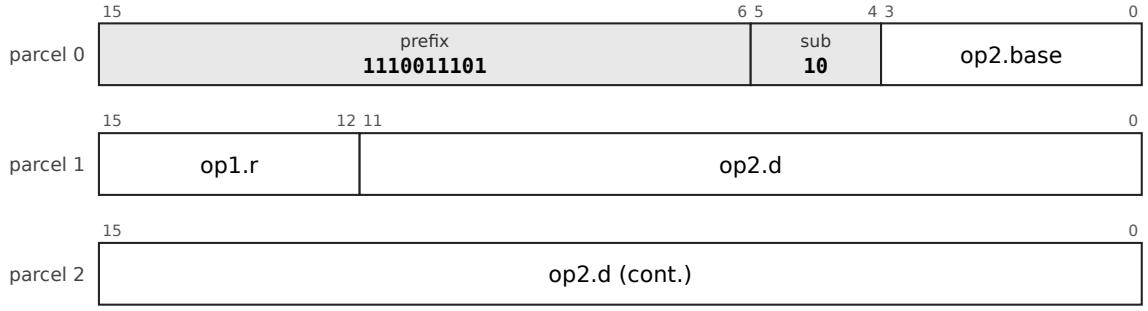



```
SUBM Rs, rm
```

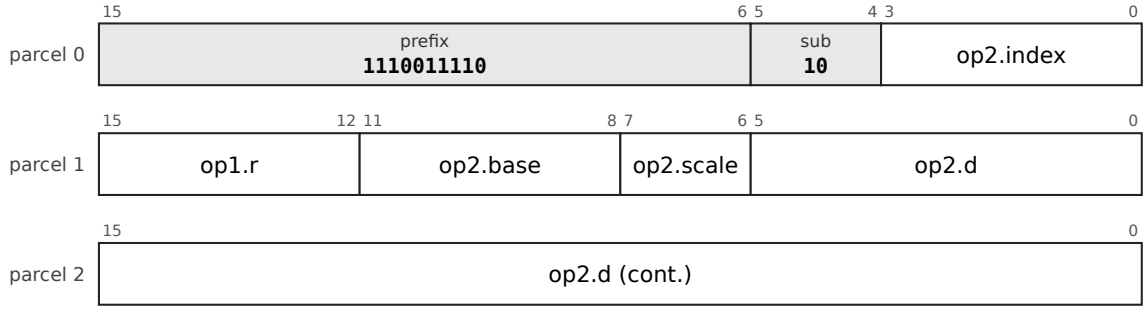
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

BICM

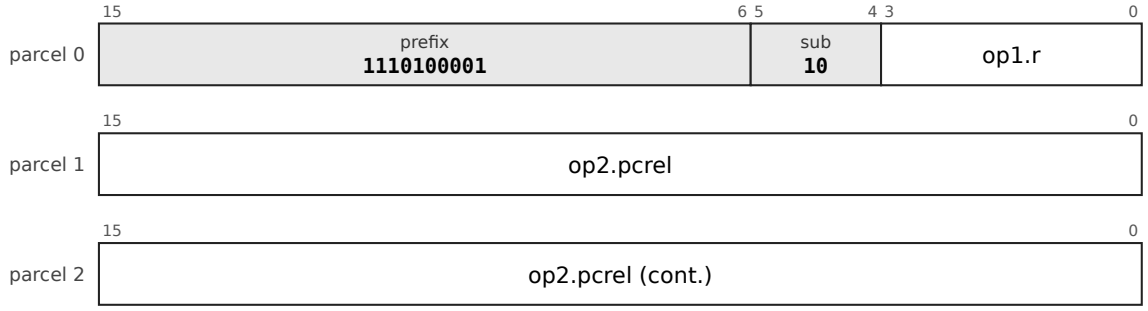
BICM: STF.B28, prefix 1110011101, sub-opcode 10; operands reg4, b4d28s-x1; 6 bytes



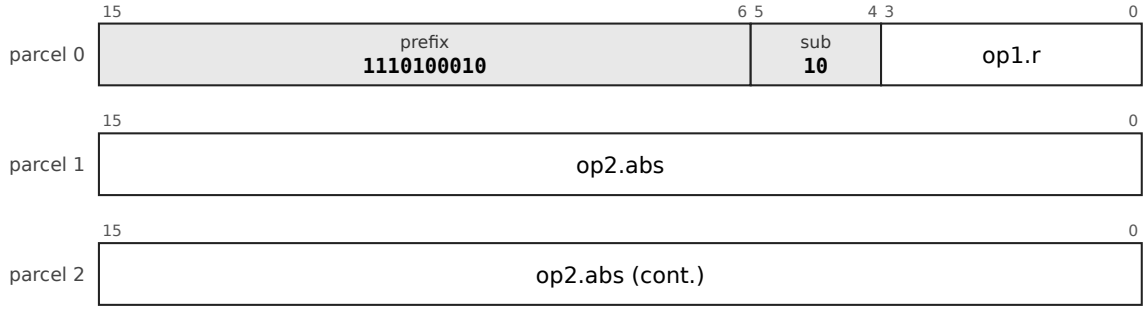
BICM: STF.X22, prefix 1110011110, sub-opcode 10; operands reg4, x4-4d22s; 6 bytes



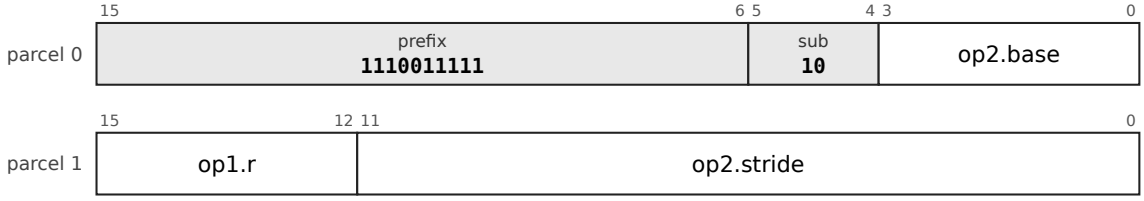
BICM: STF.PC32, prefix 1110100001, sub-opcode 10; operands reg4, pc32-u1; 6 bytes



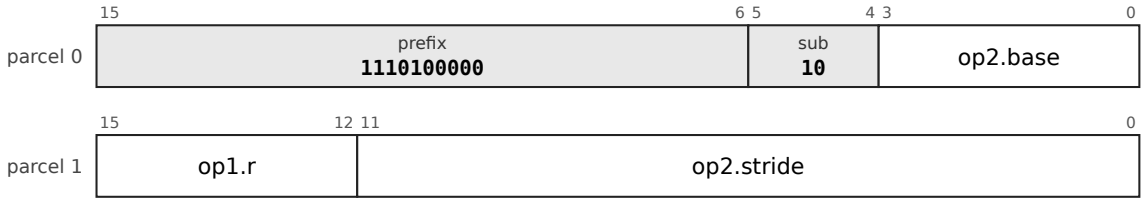
BICM: STF.A32, prefix 1110100010, sub-opcode 10; operands reg4, abs32; 6 bytes



BICM: STF.PI12, prefix 1110011111, sub-opcode 10; operands reg4, pi4-s12; 4 bytes



BICM: STF.PD12, prefix 1110100000, sub-opcode 10; operands reg4, pd4-s12; 4 bytes

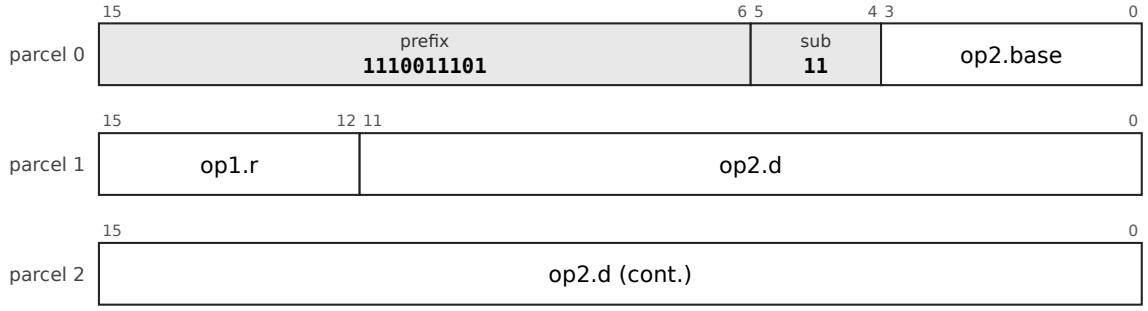



```
BICM Rs, rm
```

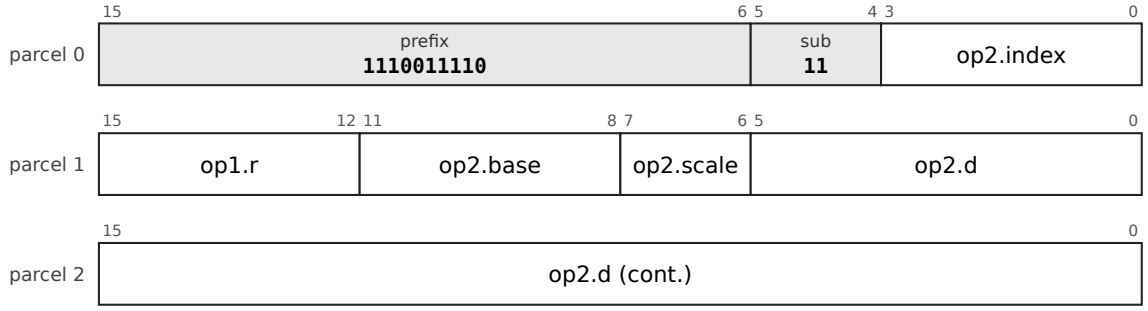
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

BISM

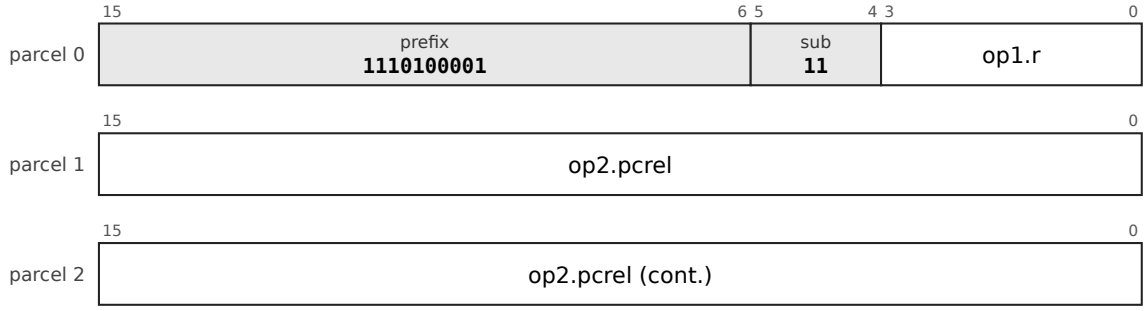
BISM: STF.B28, prefix 1110011101, sub-opcode 11; operands reg4, b4d28s-x1; 6 bytes



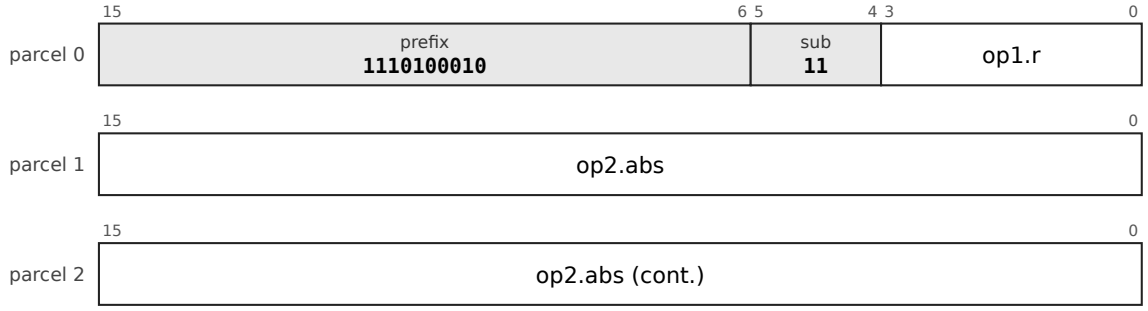
BISM: STF.X22, prefix 1110011110, sub-opcode 11; operands reg4, x4-4d22s; 6 bytes



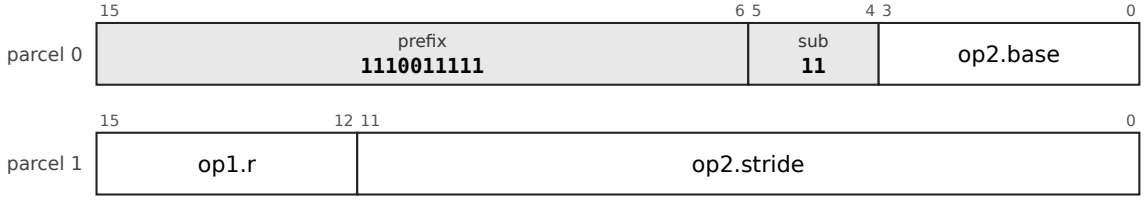
BISM: STF.PC32, prefix 1110100001, sub-opcode 11; operands reg4, pc32-u1; 6 bytes



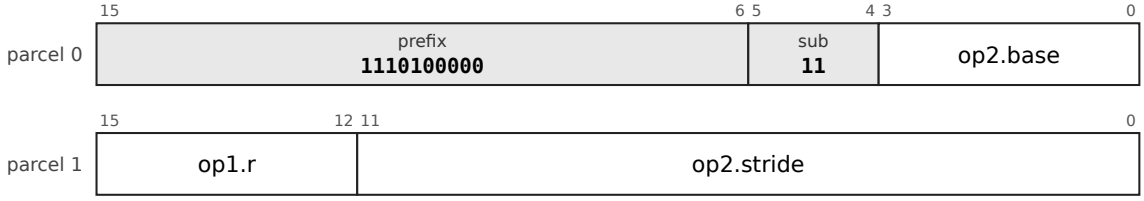
BISM: STF.A32, prefix 1110100010, sub-opcode 11; operands reg4, abs32; 6 bytes



BISM: STF.PI12, prefix 1110011111, sub-opcode 11; operands reg4, pi4-s12; 4 bytes



BISM: STF.PD12, prefix 1110100000, sub-opcode 11; operands reg4, pd4-s12; 4 bytes

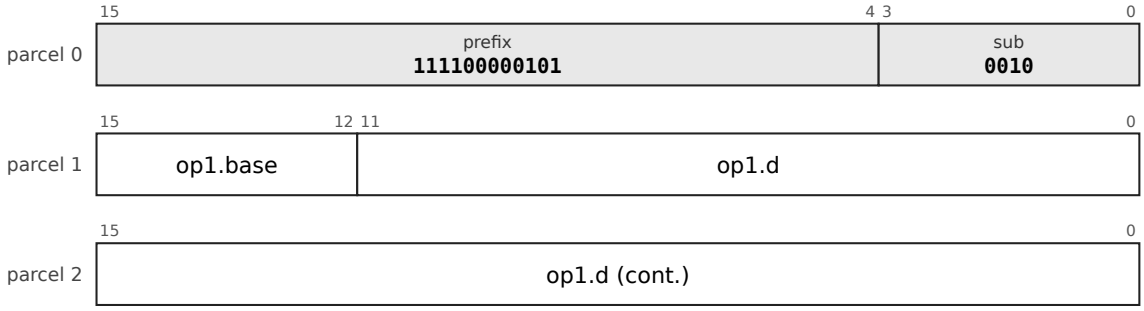



```
BISM Rs, rm
```

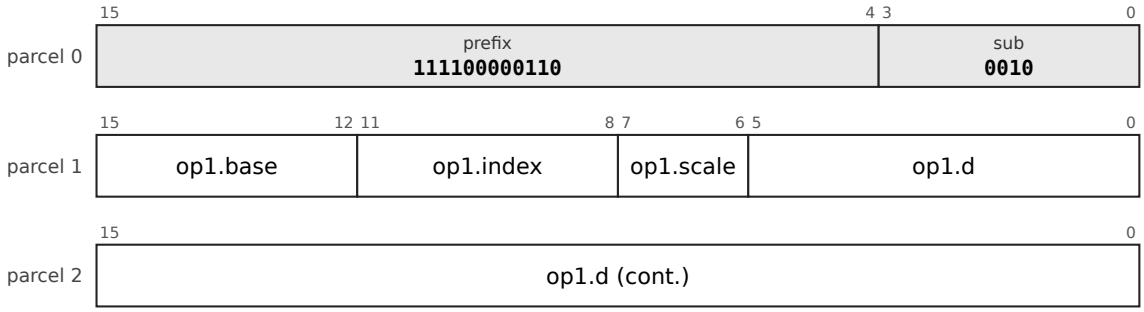
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

INC: Increment

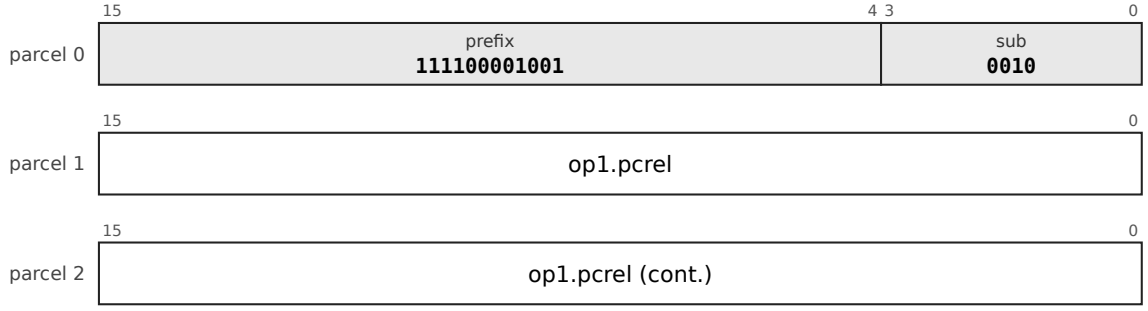
INC: SOF.B28, prefix 111100000101, sub-opcode 0010; operands b4d28s-x1; 6 bytes



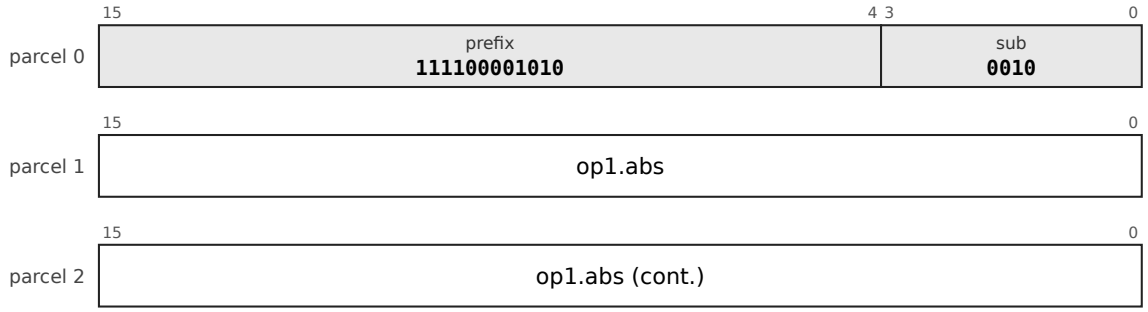
INC: SOF.X22, prefix 111100000110, sub-opcode 0010; operands x4-4d22s; 6 bytes



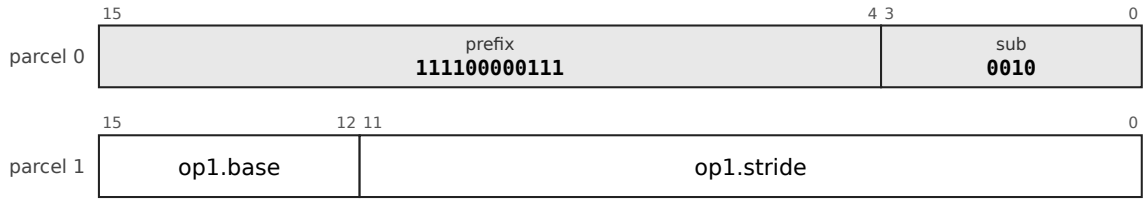
INC: SOF.PC32, prefix 111100001001, sub-opcode 0010; operands pc32-u1; 6 bytes



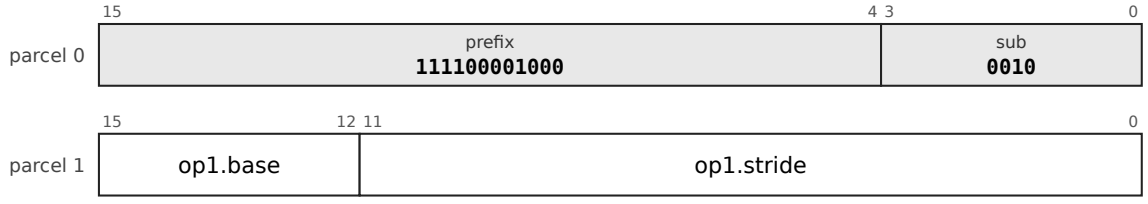
INC: SOF.A32, prefix 111100001010, sub-opcode 0010; operands abs32; 6 bytes



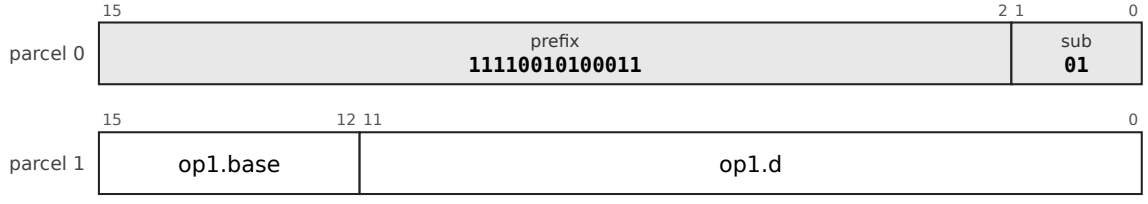
INC: SOF.PI12, prefix 111100000111, sub-opcode 0010; operands pi4-s12; 4 bytes



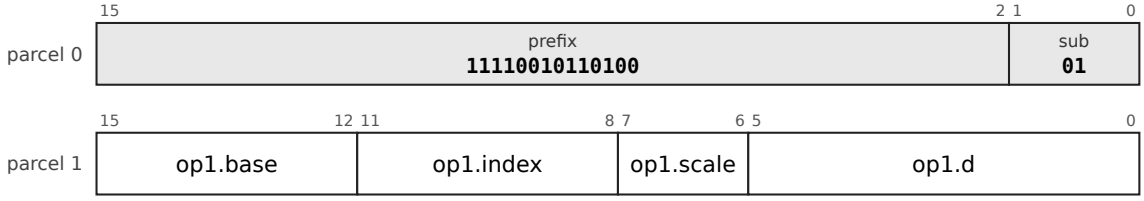
INC: SOF.PD12, prefix 111100001000, sub-opcode 0010; operands pd4-s12; 4 bytes



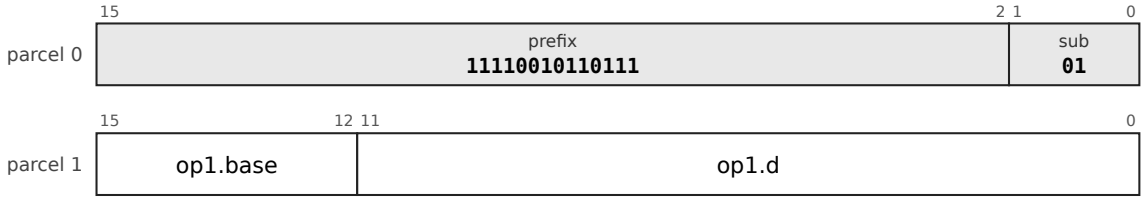
INC: SOBu12/a, prefix 11110010100011, sub-opcode 01; operands b4d12u-xsz; 4 bytes



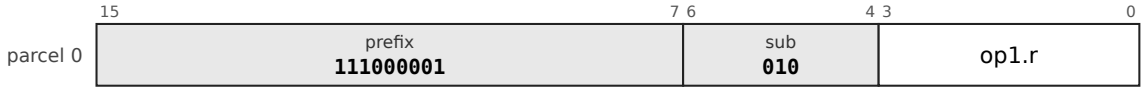
INC: SOX6/a, prefix 11110010110100, sub-opcode 01; operands x4-4d6s; 4 bytes



INC: SOBs12/a, prefix 11110010110111, sub-opcode 01; operands b4d12s-xsz; 4 bytes



INC: SOPR, prefix 111000001, sub-opcode 010; operands reg4; 2 bytes

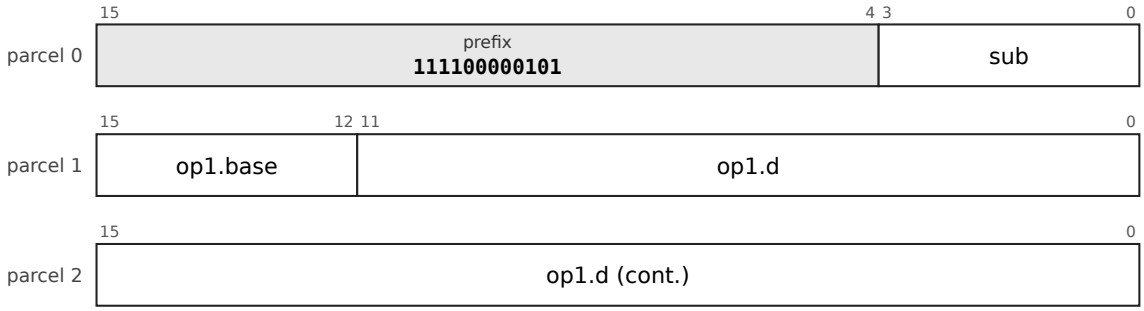


Mnemonics: `INC`

Syntax

```
INC rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`, `16-SOBu12-a-1`, `16-SOX6-a-1`, `16-SOBs12-a-1`.

Operation

```
rm ← rm + 1
```

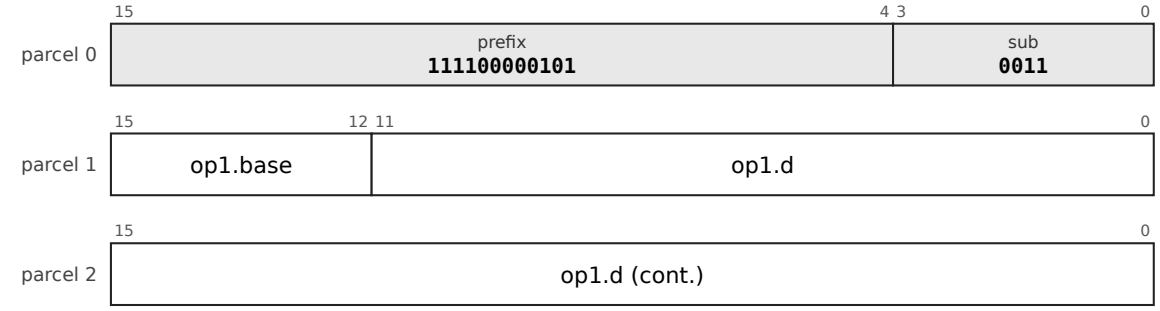
Flags `N`, `Z` and `V` as for `ADD`; `C` **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

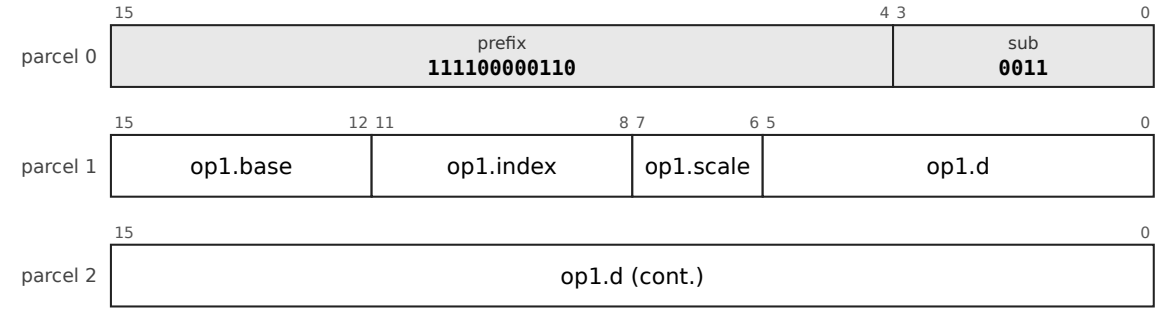
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

DEC: Decrement

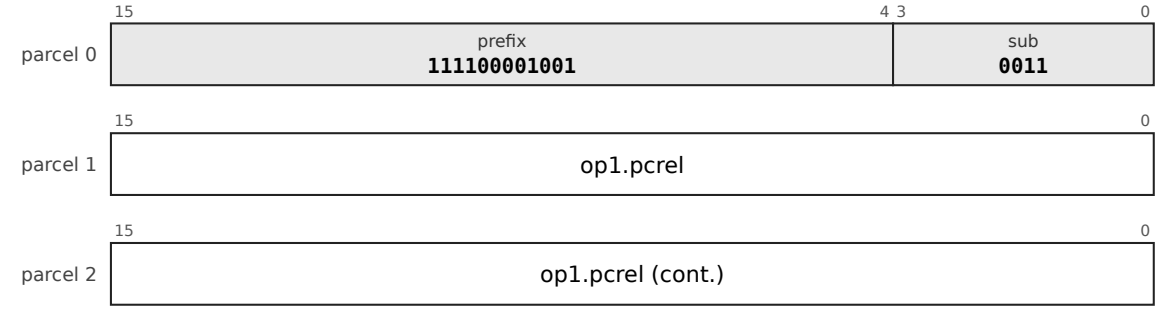
DEC: SOF.B28, prefix 111100000101, sub-opcode 0011; operands b4d28s-x1; 6 bytes



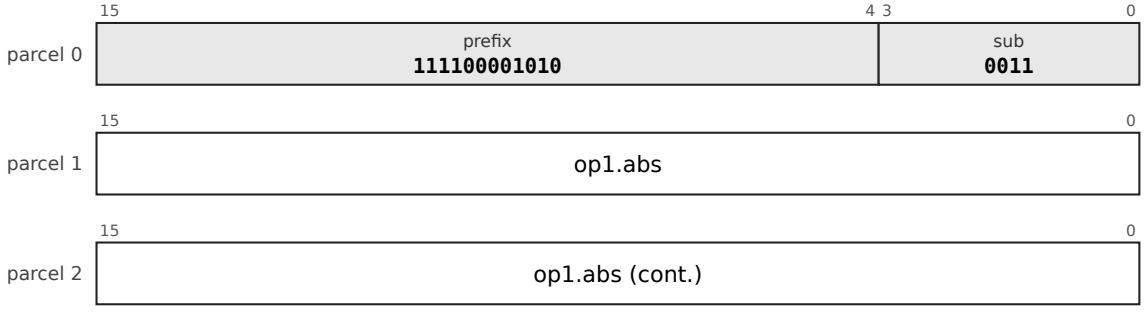
DEC: SOF.X22, prefix 111100000110, sub-opcode 0011; operands x4-4d22s; 6 bytes



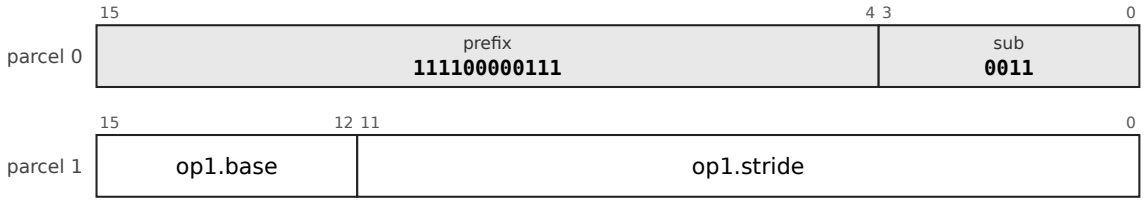
DEC: SOF.PC32, prefix 111100001001, sub-opcode 0011; operands pc32-u1; 6 bytes



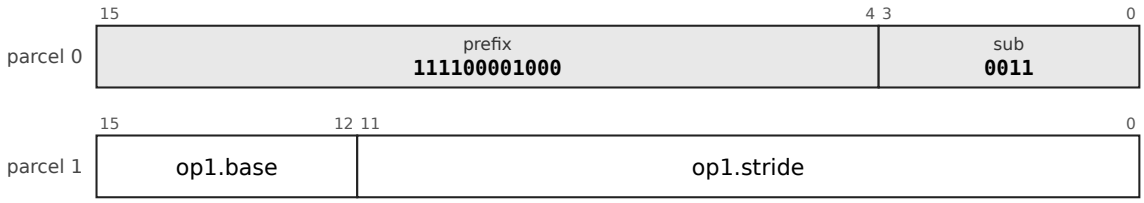
DEC: SOF.A32, prefix 111100001010, sub-opcode 0011; operands abs32; 6 bytes



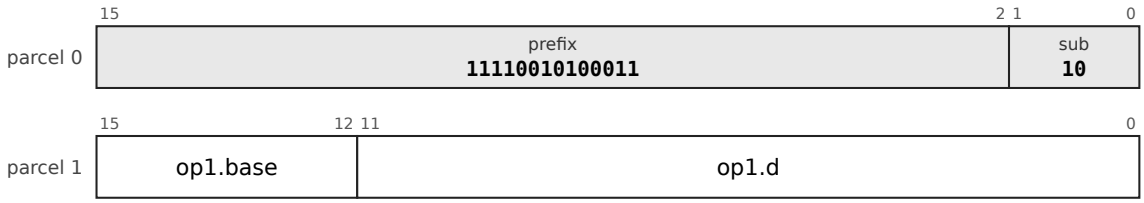
DEC: SOF.PI12, prefix 111100000111, sub-opcode 0011; operands pi4-s12; 4 bytes



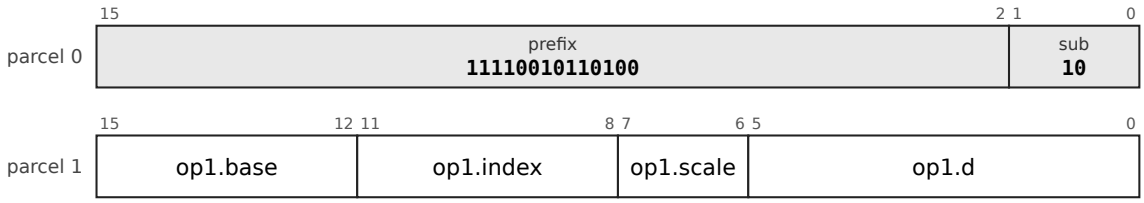
DEC: SOF.PD12, prefix 111100001000, sub-opcode 0011; operands pd4-s12; 4 bytes



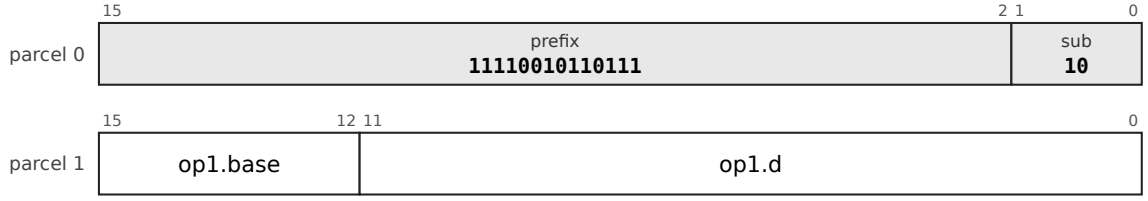
DEC: SOBu12/a, prefix 11110010100011, sub-opcode 10; operands b4d12u-xsz; 4 bytes



DEC: SOX6/a, prefix 11110010110100, sub-opcode 10; operands x4-4d6s; 4 bytes



DEC: SOBs12/a, prefix 11110010110111, sub-opcode 10; operands b4d12s-xsz; 4 bytes



DEC: SOPR, prefix 111000001, sub-opcode 011; operands reg4; 2 bytes

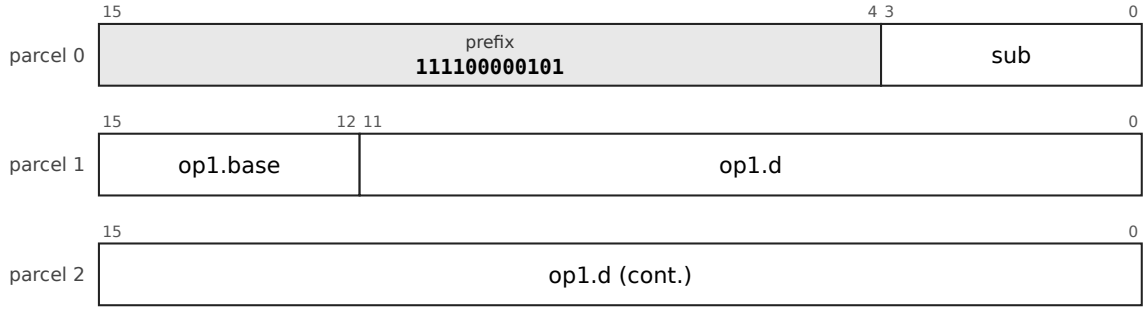


Mnemonics: `DEC`

Syntax

```
DEC rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`, `16-SOBu12-a-1`, `16-SOX6-a-1`, `16-SOBs12-a-1`.

Operation

```
rm ← rm - 1
```

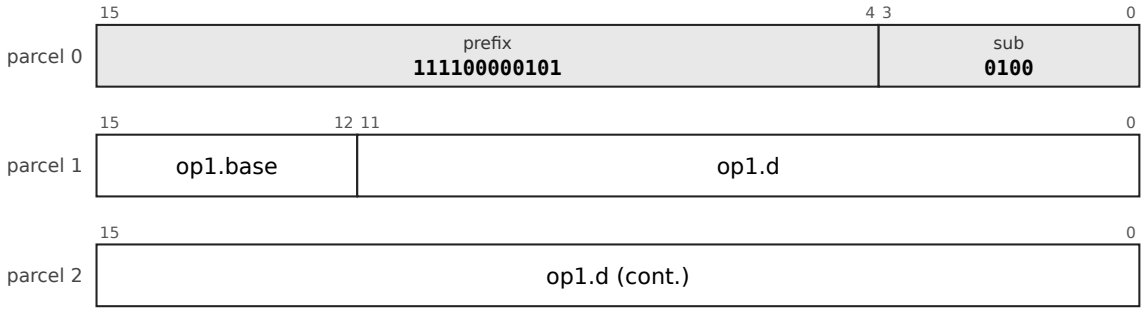
Flags `N`, `Z` and `V` as for `SUB`; `C` **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

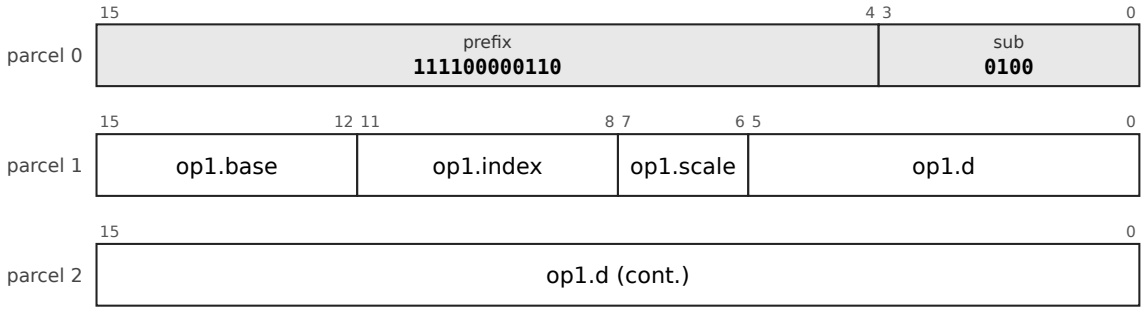
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

NEG: Negate

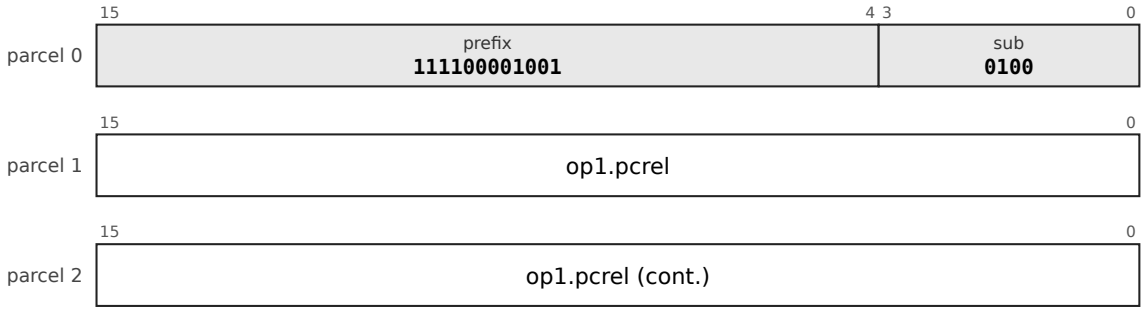
NEG: SOF.B28, prefix 111100000101, sub-opcode 0100; operands b4d28s-x1; 6 bytes



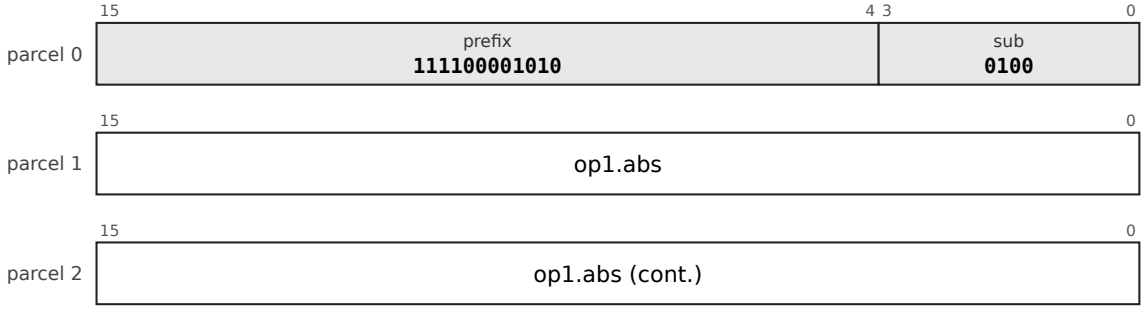
NEG: SOF.X22, prefix 111100000110, sub-opcode 0100; operands x4-4d22s; 6 bytes



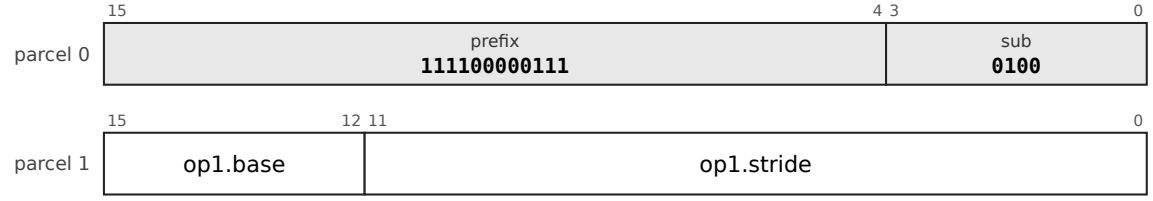
NEG: SOF.PC32, prefix 111100001001, sub-opcode 0100; operands pc32-u1; 6 bytes



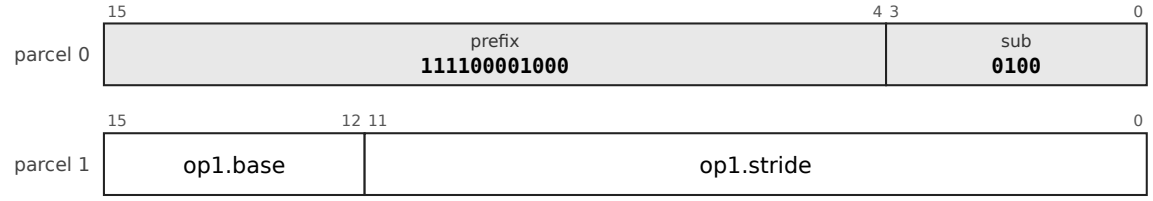
NEG: SOF.A32, prefix 111100001010, sub-opcode 0100; operands abs32; 6 bytes



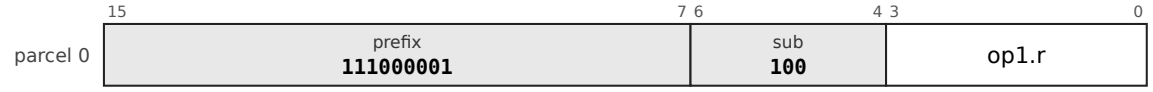
NEG: SOF.PI12, prefix 111100000111, sub-opcode 0100; operands pi4-s12; 4 bytes



NEG: SOF.PD12, prefix 111100001000, sub-opcode 0100; operands pd4-s12; 4 bytes



NEG: SOPR, prefix 111000001, sub-opcode 100; operands reg4; 2 bytes

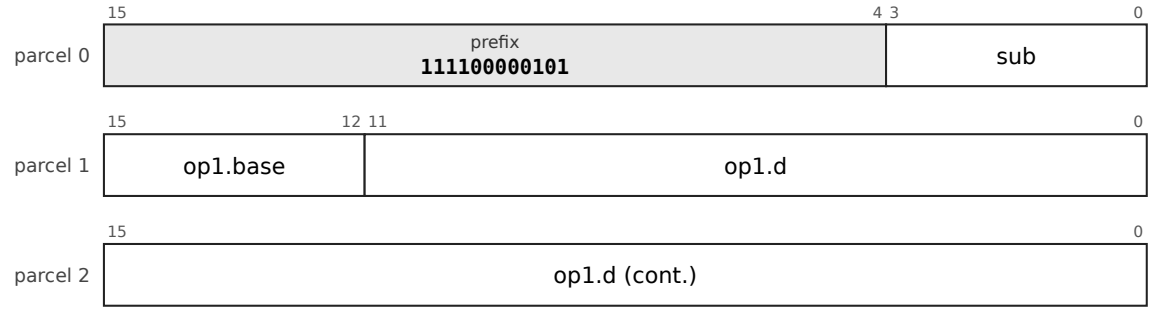


Mnemonics: `NEG`

Syntax

```
NEG rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`.

Operation

```
rm ← 0 - rm
```

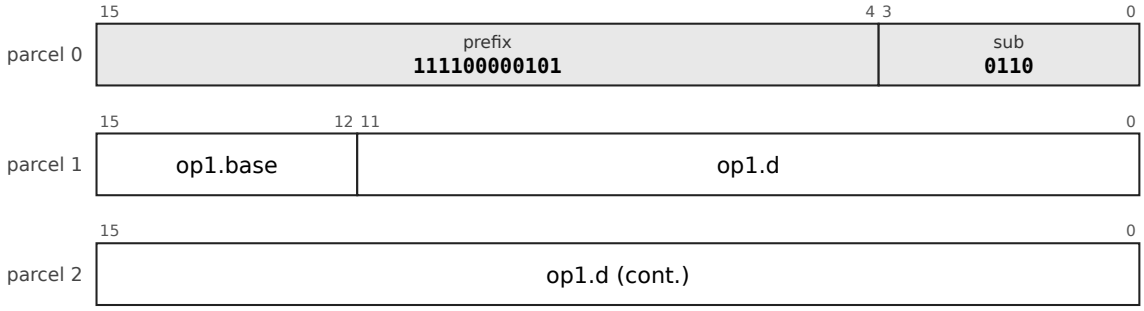
Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger. (`C` = 1 unless the operand was 0.)

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

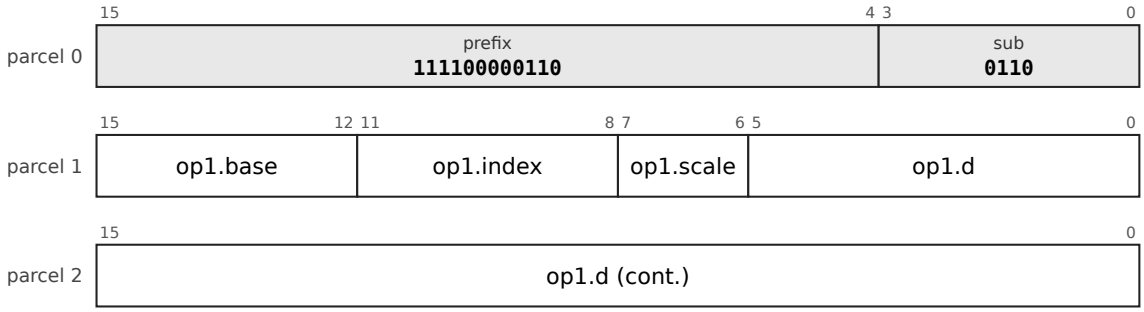
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ADC: Add carry

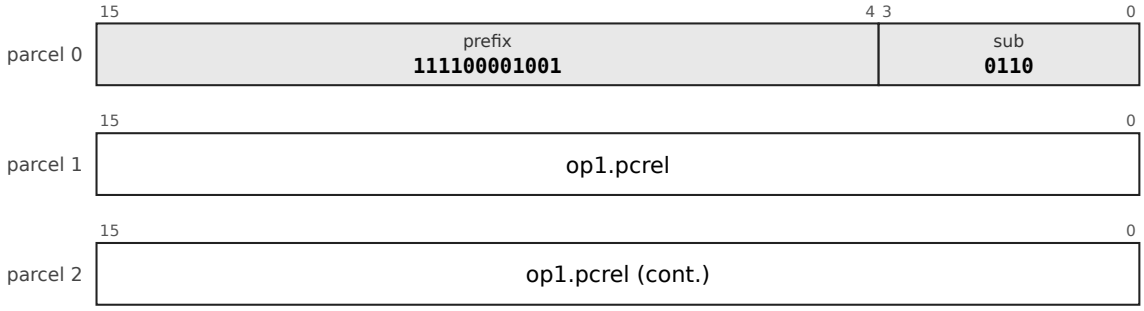
ADC: SOF.B28, prefix 111100000101, sub-opcode 0110; operands b4d28s-x1; 6 bytes



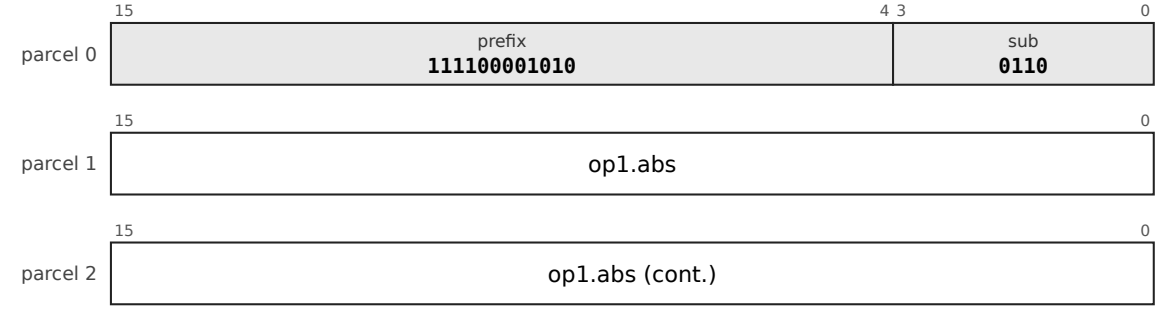
ADC: SOF.X22, prefix 111100000110, sub-opcode 0110; operands x4-4d22s; 6 bytes



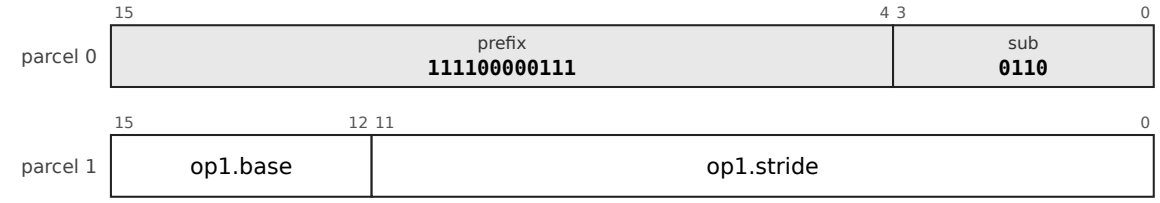
ADC: SOF.PC32, prefix 111100001001, sub-opcode 0110; operands pc32-u1; 6 bytes



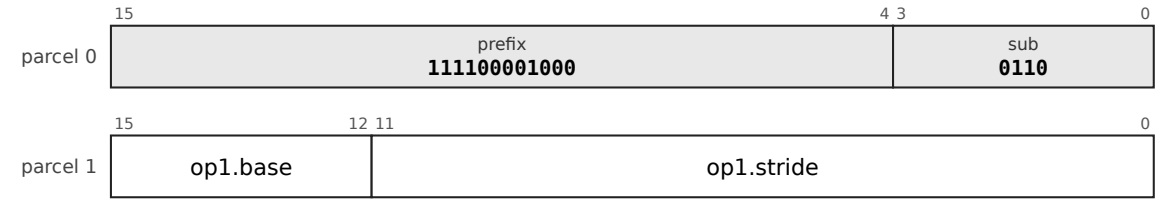
ADC: SOF.A32, prefix 111100001010, sub-opcode 0110; operands abs32; 6 bytes



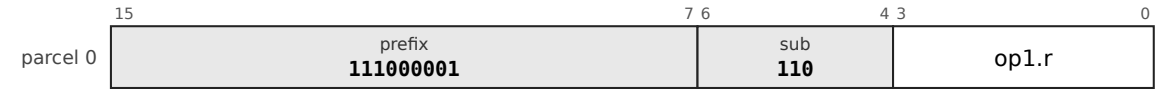
ADC: SOF.PI12, prefix 111100000111, sub-opcode 0110; operands pi4-s12; 4 bytes



ADC: SOF.PD12, prefix 111100001000, sub-opcode 0110; operands pd4-s12; 4 bytes



ADC: SOPR, prefix 111000001, sub-opcode 110; operands reg4; 2 bytes

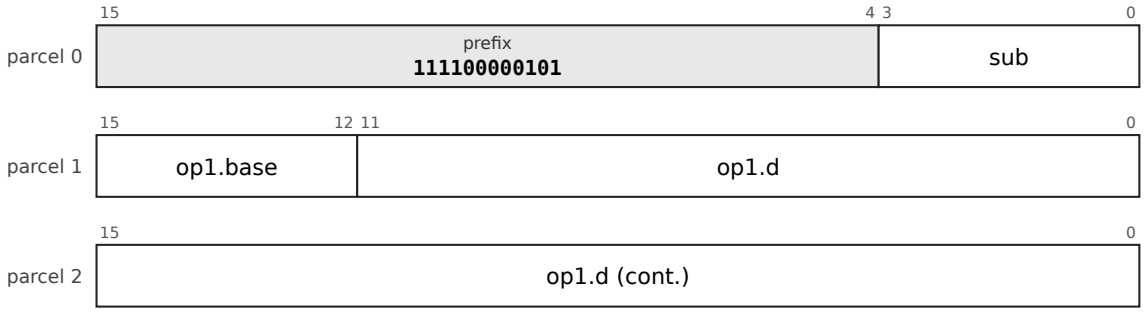


Mnemonics: ADC

Syntax

```
ADC rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`.

Operation

```
rm ← rm + C
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.

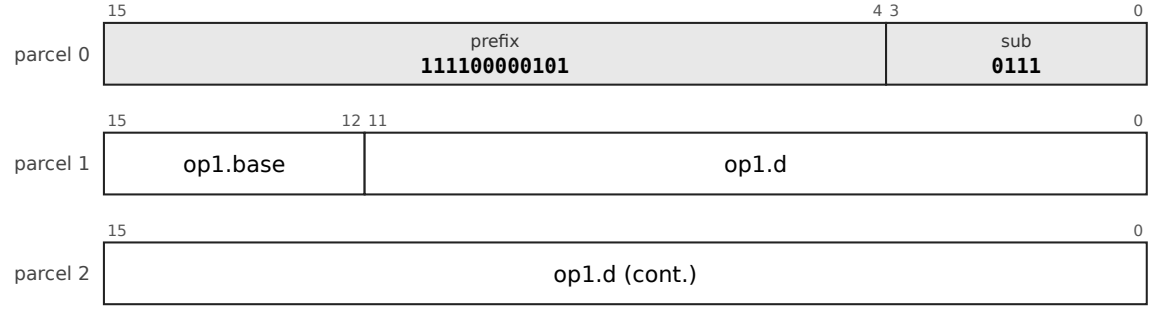
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `ADD`, multi-word addition.

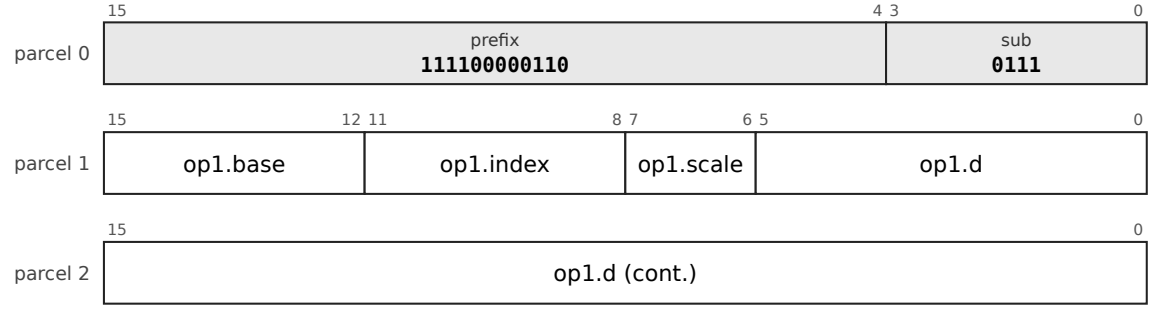
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

SBC: Subtract carry

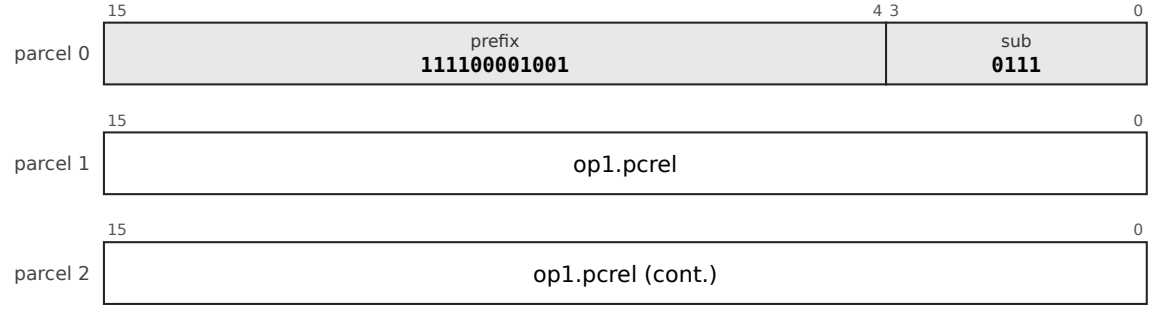
SBC: SOF.B28, prefix 111100000101, sub-opcode 0111; operands b4d28s-x1; 6 bytes



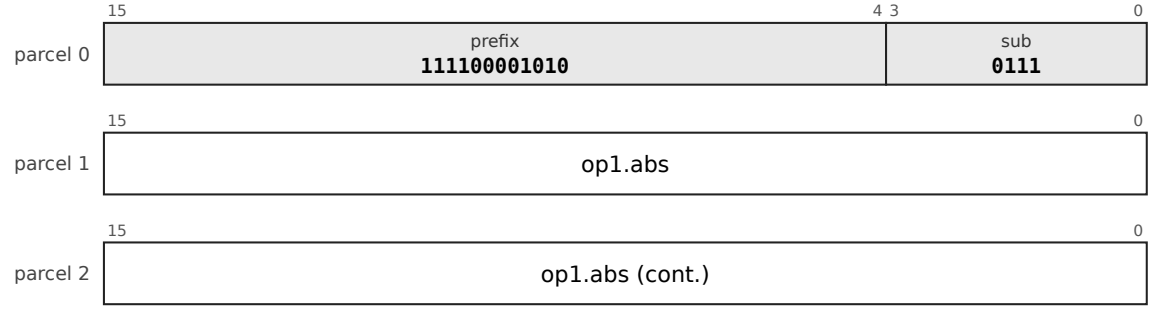
SBC: SOF.X22, prefix 111100000110, sub-opcode 0111; operands x4-4d22s; 6 bytes



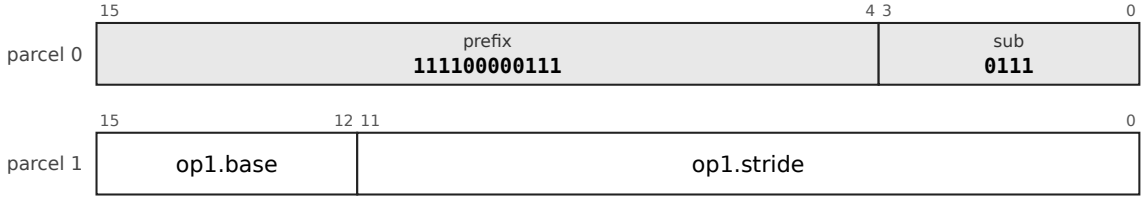
SBC: SOF.PC32, prefix 111100001001, sub-opcode 0111; operands pc32-u1; 6 bytes



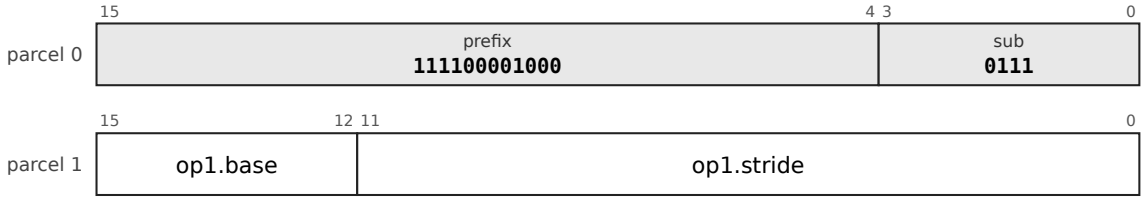
SBC: SOF.A32, prefix 111100001010, sub-opcode 0111; operands abs32; 6 bytes



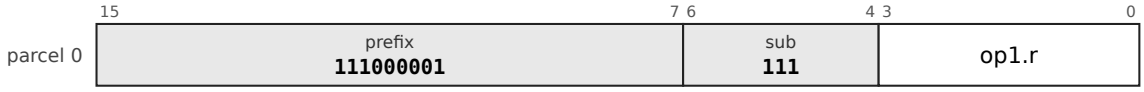
SBC: SOF.PI12, prefix 111100000111, sub-opcode 0111; operands pi4-s12; 4 bytes



SBC: SOF.PD12, prefix 111100001000, sub-opcode 0111; operands pd4-s12; 4 bytes



SBC: SOPR, prefix 111000001, sub-opcode 111; operands reg4; 2 bytes

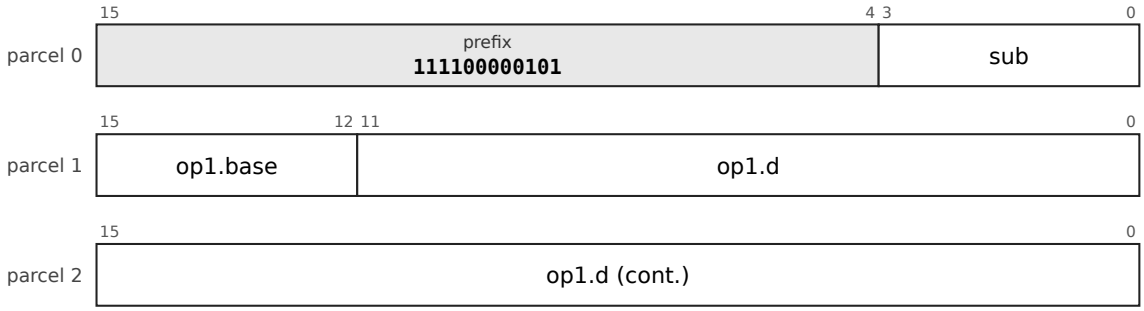


Mnemonics: `SBC`

Syntax

```
SBC rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`.

Operation

```
rm ← rm - C
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

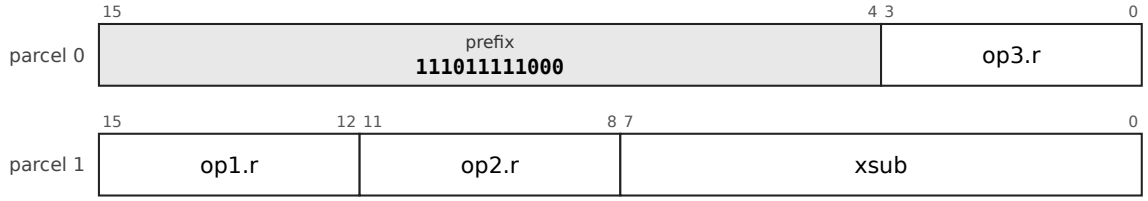
DIV: Divide and remainder, signed and unsigned

Mnemonics: DIV, DIVU, MOD, MODU

Syntax

```
DIV Ra, Rb, Rd
DIVU Ra, Rb, Rd
MOD Ra, Rb, Rd
MODU Ra, Rb, Rd
```

Encoding



The chart is the leaf 16-R3-1, R3, prefix 111011111000. op1 is reg4: register, 4 bits. op2 is reg4: register, 4 bits. op3 is reg4: register, 4 bits. xsub selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): 16-R3-1.

Operation

```
if Rb = 0 then divide-by-zero abort
DIV : Rd ← Ra ÷ Rb  (signed, rounded toward zero; INT_MIN ÷ -1 = INT_MIN)
DIVU: Rd ← Ra ÷ Rb  (unsigned)
MOD : Rd ← Ra mod Rb (signed, the sign of Ra; any value mod -1 = 0)
MODU: Rd ← Ra mod Rb (unsigned)
```

Flags None written.

Exceptions Division by zero aborts with the divide-by-zero vector (cause DIVZERO). There is no overflow trap: INT_MIN / -1 returns INT_MIN.

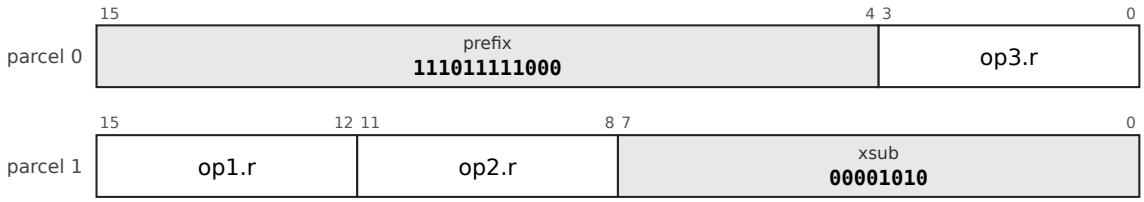
Division by a constant is a MULH and a shift, so a real divide is rare; this is why divide has no two-byte form.

Cf. 6.2 (the divide-by-zero vector).

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

DIV

DIV: R3, prefix 111011111000, sub-opcode 00001010; operands reg4, reg4, reg4; 4 bytes

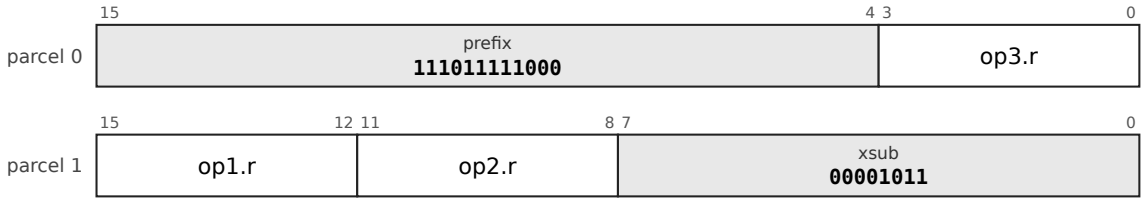



```
DIV Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

DIVU

DIVU: R3, prefix 111011111000, sub-opcode 00001011; operands reg4, reg4, reg4; 4 bytes

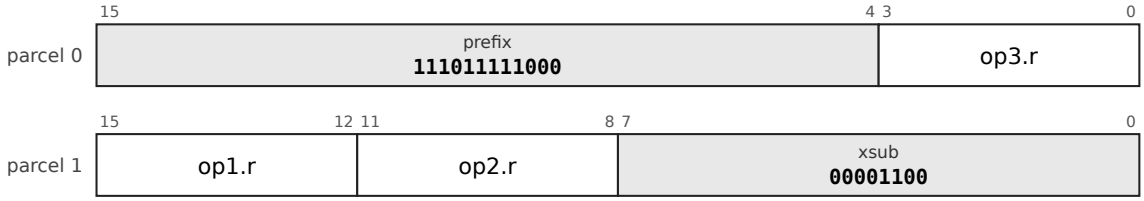



```
DIVU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

MOD

MOD: R3, prefix 111011111000, sub-opcode 00001100; operands reg4, reg4, reg4; 4 bytes

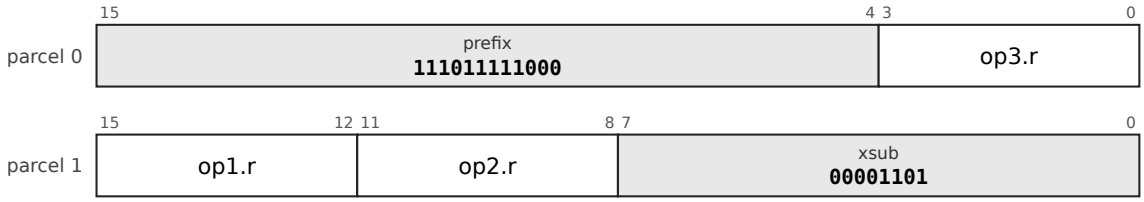



```
MOD Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

MODU

MODU: R3, prefix 111011111000, sub-opcode 00001101; operands reg4, reg4, reg4; 4 bytes

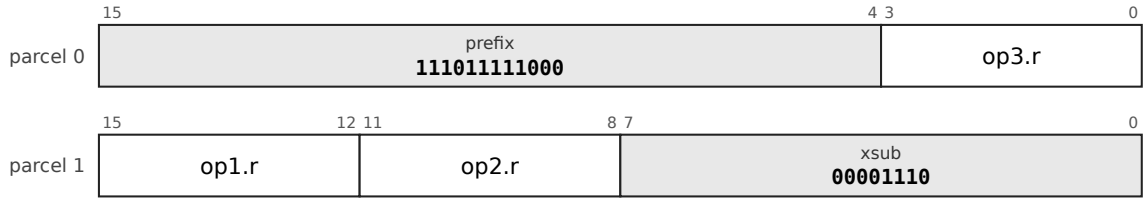



```
MODU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

MULHU: Multiply, high 64 bits, unsigned

MULHU: R3, prefix 111011111000, sub-opcode 00001110; operands reg4, reg4, reg4; 4 bytes

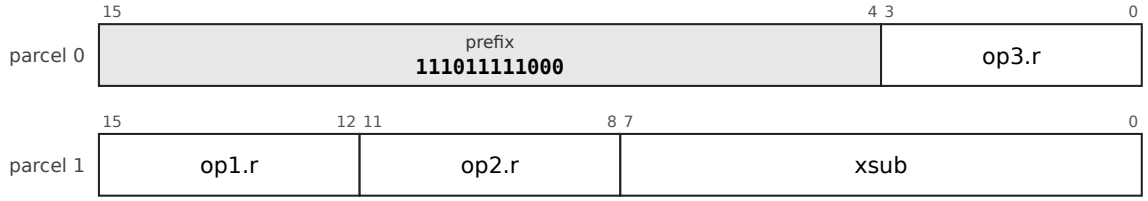


Mnemonics: `MULHU`

Syntax

```
MULHU Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
Rd ← (Ra × Rb)[127:64]    ; unsigned
```

Flags None written.

Exceptions None.

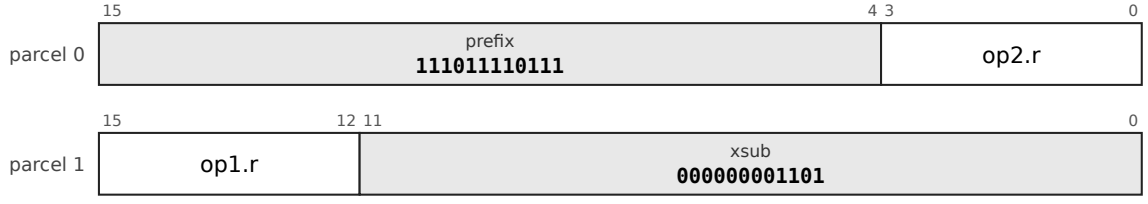
`MULHSU` (signed by unsigned) is reserved and not committed.

Why it is here. The unsigned partner of `MULH`, for the same reason (the high half of the product). `MULHSU`, signed times unsigned for a bignum's top limb, is reserved because it can be built from `MULH` with one correction.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ABS: Absolute value

ABS: R2, prefix 111011110111, sub-opcode 000000001101; operands reg4, reg4; 4 bytes

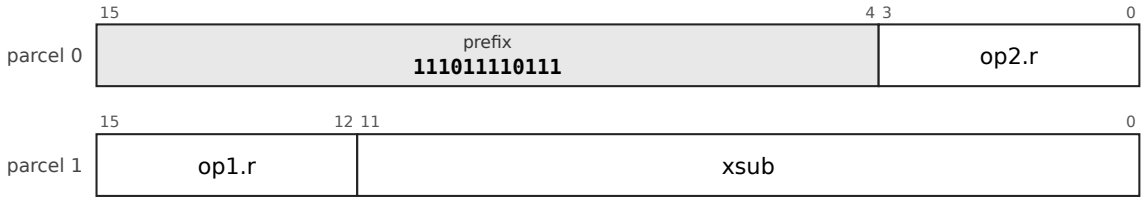


Mnemonics: `ABS`

Syntax

```
ABS Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← |Rs|
```

Flags `N` and `Z` from the result, `V` set if `Rs` is the most negative value (the result is then still negative), `C` unchanged.

Exceptions None.

Why it is here. Integer absolute value is the counterpart of `FABS`. Its cost is about 150 to 250 lookup tables, no new operation by the folding rule (integer against float and signed against unsigned are qualifiers), and the compiler's work of recognising `x < 0 ? -x : x`, which is the real cost. The compiled corpus has 18 uses of `abs` and 5 of its unsigned form, under 0.1% of code, so the case is clarity and the idiom more than size.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

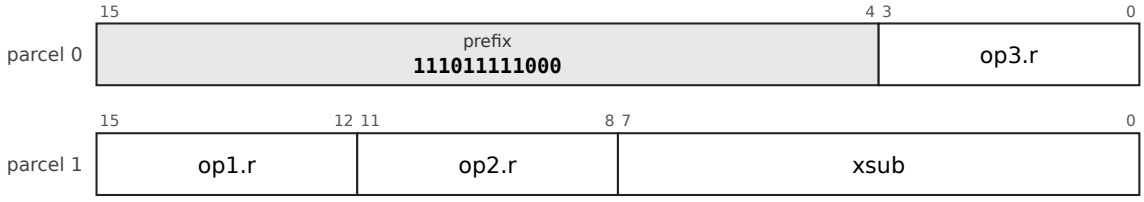
MINMAX: Minimum and maximum, signed and unsigned

Mnemonics: `MIN`, `MINU`, `MAX`, `MAXU`

Syntax

```
MIN Ra, Rb, Rd
MINU Ra, Rb, Rd
MAX Ra, Rb, Rd
MAXU Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
MIN : Rd ← smaller of Ra, Rb (signed)        MINU: unsigned
MAX : Rd ← larger of Ra, Rb (signed)         MAXU: unsigned
```

Flags N and Z from the result, V = 0, C unchanged.

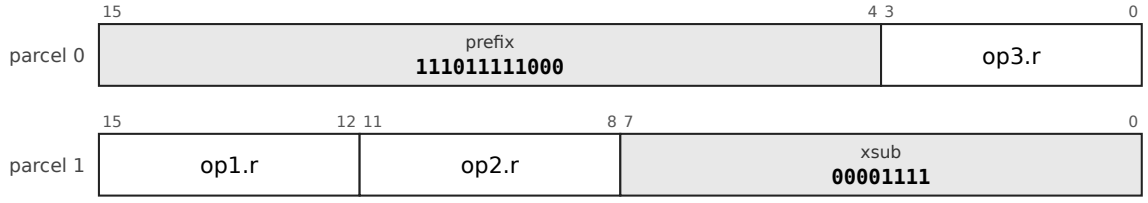
Exceptions None.

Why it is here. The integer counterparts of FMIN and FMAX, provided with ABS. By the folding rule they add no operation, and the hardware reuses the subtract-and-compare that CMP already does plus a 64-bit select. The corpus has 171 uses of MIN and 52 of MAX across 95 files, under 0.1% of code in all. A general conditional select is the alternative; it is open (9.21) for want of a speed model.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MIN

MIN: R3, prefix 111011111000, sub-opcode 00001111; operands reg4, reg4, reg4; 4 bytes

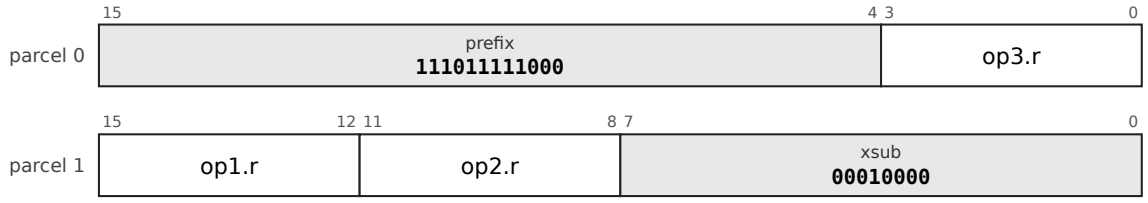



```
MIN Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry MINMAX above; 1 layout form, each a chart of its own (above).

MINU

MINU: R3, prefix 111011111000, sub-opcode 00010000; operands reg4, reg4, reg4; 4 bytes

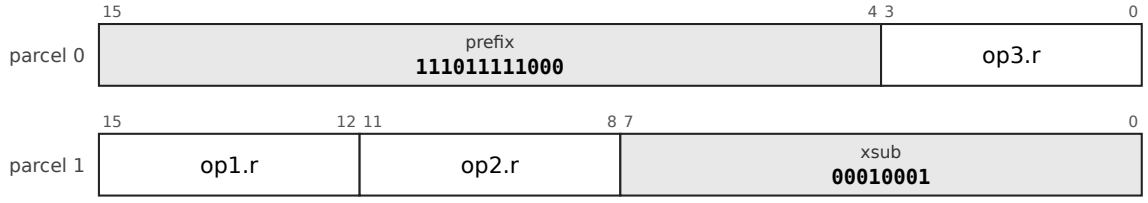



```
MINU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry MINMAX above; 1 layout form, each a chart of its own (above).

MAX

MAX: R3, prefix 111011111000, sub-opcode 00010001; operands reg4, reg4, reg4; 4 bytes

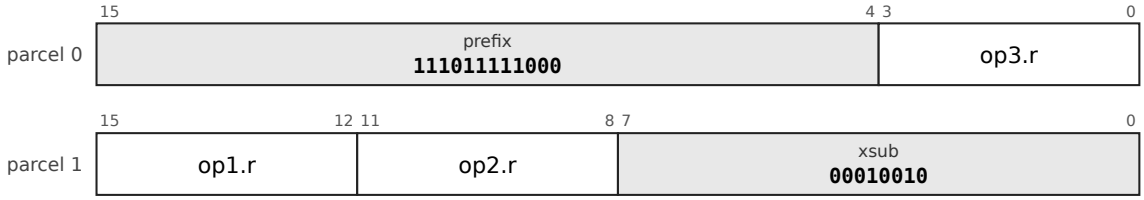



```
MAX Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry MINMAX above; 1 layout form, each a chart of its own (above).

MAXU

MAXU: R3, prefix 111011111000, sub-opcode 00010010; operands reg4, reg4, reg4; 4 bytes

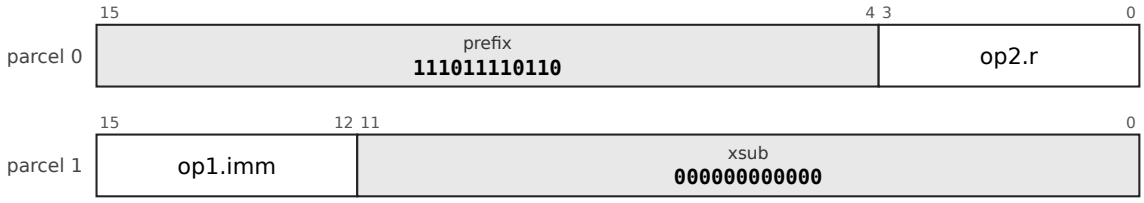



```
MAXU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry MINMAX above; 1 layout form, each a chart of its own (above).

SET: Set a register from a condition

SET: SETF, prefix 111011110110, sub-opcode 000000000000; operands imm4u, reg4; 4 bytes

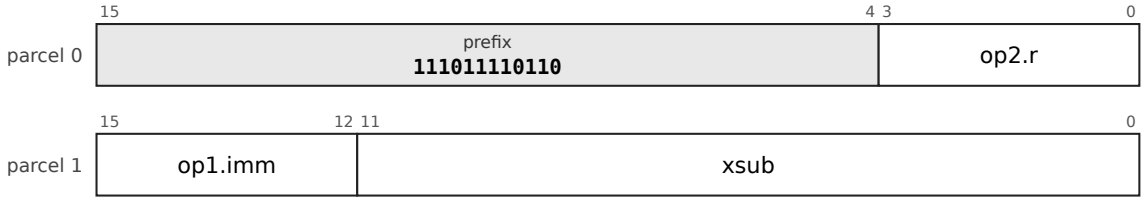


Mnemonics: `SET`

Syntax

```
SET cc, Rd        ; for example SET EQ, R1
```

Encoding



The chart is the leaf `16-SETF-1`, SETF, prefix `111011110110`. `op1` is `imm4u`: unsigned 4-bit immediate, 0 to 15. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SETF-1`.

Operation

```
Rd ← 1 if condition cc holds on the current flags, else 0
```

Flags None written.

Exceptions None.

The condition is numbered as the low nibble of the branch opcodes (`al` 0, `ne` 1, `eq` 2, ... `los` 14).

Why it is here. The first instruction written with a condition word: one instruction, `SET cc, Rd`, rather than one opcode per condition, with the condition numbered like the low nibble of the branch opcodes. It turns a comparison used as a value into 0 or 1 without a branch. gcc `-Os` for amd64 emits 673 `set<cc>` on the corpus, and `SET` would remove about six bytes per

site, around 0.4% of code; the compiler's first pass turns such a comparison into branches before its second pass sees it, so the saving is not reached in compiled code.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

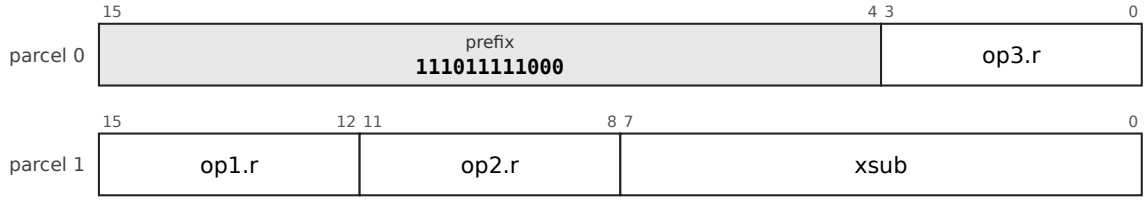
MASK: Mask by a condition register

Mnemonics: `MASKEQZ`, `MASKNEZ`

Syntax

```
MASKEQZ Ra, Rc, Rd
MASKNEZ Ra, Rc, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
MASKEQZ: Rd ← (Rc = 0) ? 0 : Ra
MASKNEZ: Rd ← (Rc ≠ 0) ? 0 : Ra
```

Flags None written.

Exceptions None.

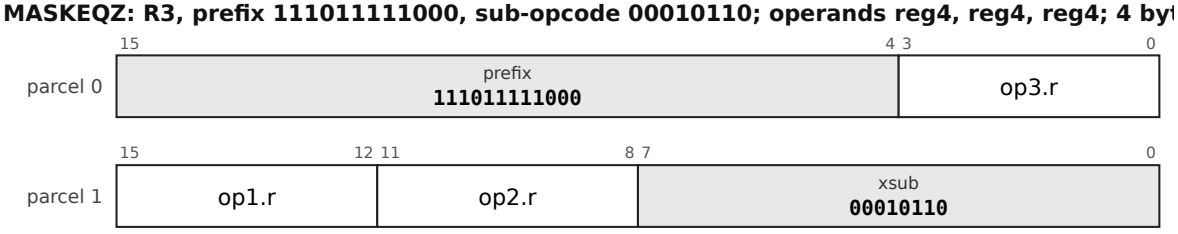
(D) Errata 85: the branch-free select is adopted. `Rc` is tested whole, all 64 bits, so any non-zero value is true; no flag is read or written, and the latency is fixed. `Rd` may be `Ra` or `Rc`. A constant-time select `d = c ? a : b` is `MASKEQZ a, c, t`, `MASKNEZ b, c, d`, `BIS t, d`. The semantics are those of LoongArch `maskeqz` and `masknez` and of RISC-V Zicond (the crypto proposal states this from memory). A flag-reading `CSEL` is not in the set (reserved, recommended only if the compiler work shows selects on comparisons matter).

Why it is here. The author adopted the branch-free select (errata 85) when the crypto proposal asked for it: constant-time code must choose between two values without a branch, and a flag-reading conditional move would put a secret in a flag. `MASKEQZ` and `MASKNEZ` are the select of LoongArch and of RISC-V Zicond (stated from memory in the proposal, not checked against a manual); the pair and an `OR` make `c ? a : b` in three instructions, against two for `CMP` and a conditional move, but the conditional move would be used at 783 sites in 284,641 instructions (0.28%), and the case for the pair is the constant-time idiom, where the condition is already a register. The author confirmed the pair and wrote "looks good" on the section (errata 86). About 100 look-up tables, one cycle (the proposal's estimate).

Cf. 9.7 (SET), docs/crypto-secure-core.md section 2.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

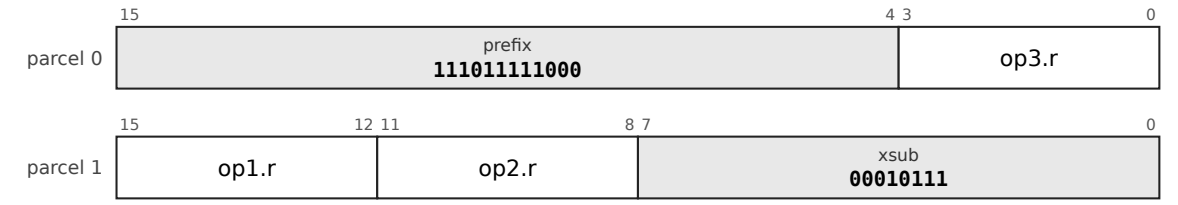
MASKEQZ




```
MASKEQZ Ra, Rc, Rd
```

Operation, flags and exceptions as for the entry MASK above; 1 layout form, each a chart of its own (above).

MASKNEZ




```
MASKNEZ Ra, Rc, Rd
```

Operation, flags and exceptions as for the entry MASK above; 1 layout form, each a chart of its own (above).

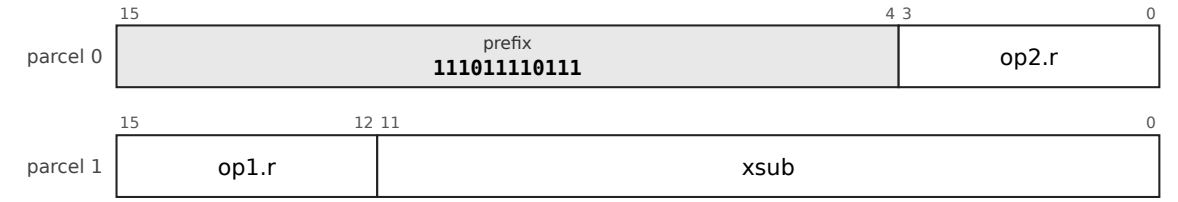
OVF: Add, subtract and multiply, trapping on signed overflow

Mnemonics: `ADDV`, `SUBV`, `MULV`

Syntax

```
ADDV Rs, Rd
SUBV Rs, Rd
MULV Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
t ← Rd op Rs                ; op is +, - or ×, on signed 64-bit values
if t does not fit in 64 bits then abort at this instruction (vector CHECK), Rd unwritten
else Rd ← t
```

Flags None written.

Exceptions Aborts with the CHECK vector on signed overflow, with EPC at the instruction and the destination unwritten; a blind restart faults again unless the handler changed an operand.

(D) Errata 85: TRAPCC with the fused ADDV, SUBV and MULV forms. The tables of the grammars hold the two-register form (`Rd <- Rd op Rs`, like ADD, SUB, MUL) and no immediate or memory form; `docs/study-proposals.md` item 9 writes a three-register form (`Rd <- Ra op Rb`), which no table has *(P)*. The check vector is number 16 (CHECK) in `docs/study-proposals.md` item 9 and in the reference executor; `spec/pdp64-system.hbnf` has no vector of that name yet, so the number is *(P)*.

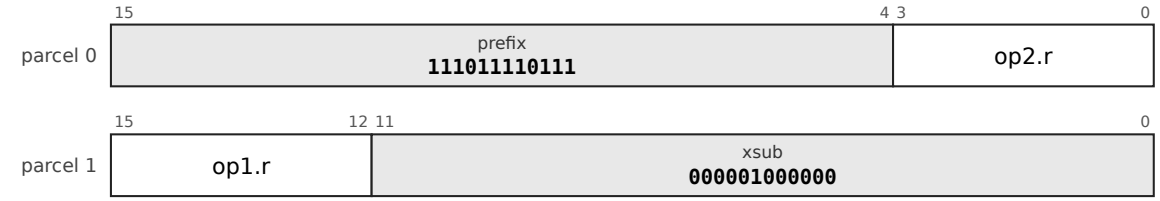
Why it is here. Checked arithmetic is what clang `-ftrapv` and `__builtin_*_overflow`, Rust's debug overflow checks, Swift's default arithmetic and Ada's `Constraint_Error` need. The author decided TRAPCC with the fused ADDV, SUBV and MULV and no compare-and-trap form (errata 85), the fused forms as the case where one non-destructive instruction beats ADD and TRAPCC. The corpus cannot size the use: it contains no BVS or BVC at all, because the overflow builtins were not implemented in the compiler that produced it, and the proposal says that sanitiser builds are the target. The record gives the precedent only for the trap, not for the fused forms.

Cf. 9.14 (TRAPCC), docs/study-proposals.md item 9.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ADDV

ADDV: R2, prefix 111011110111, sub-opcode 000001000000; operands reg4, reg4; 4 bytes

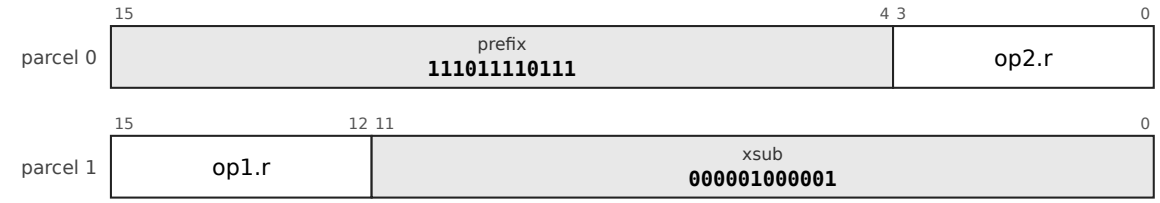



```
ADDV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

SUBV

SUBV: R2, prefix 111011110111, sub-opcode 000001000001; operands reg4, reg4; 4 bytes

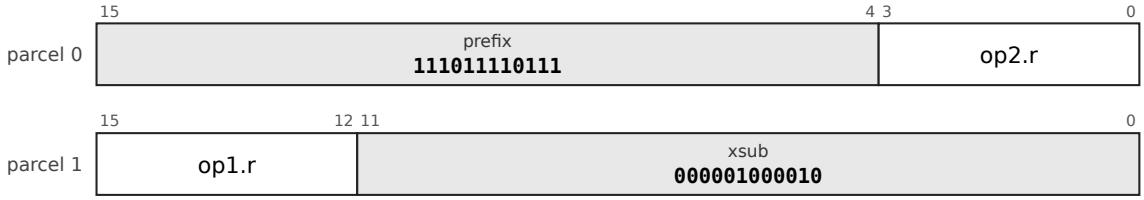



```
SUBV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

MULV

MULV: R2, prefix 111011110111, sub-opcode 000001000010; operands reg4, reg4; 4 bytes




```
MULV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

9.8 Logic, shifts and rotates

9.8.1 `AND`, `BIC` (bit clear), `BIS` (bit set, the or), `XOR` and `BIT` (test) are two-byte ALU operations; `COM` is one's complement.

9.8.2 `SHL`, `SHR` and `SAR` shift by a count taken from `rm` (a register, an immediate, or memory); `ASL` and `ASR` are the by-one forms in two bytes. *(D)*

9.8.3 A shift count of 64 or more shifts the value entirely away; `C` receives the last bit that left. *(D)*

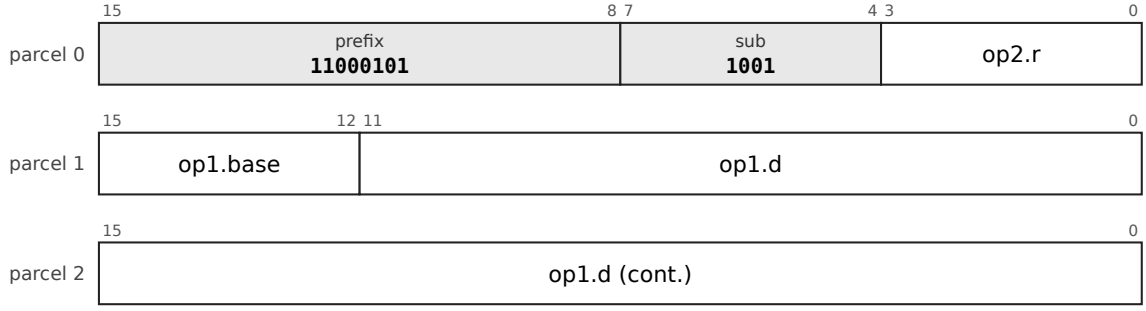
9.8.4 `ROR` rotates right by a register count and `RORI` by an immediate; `ROL Rd, Rs, #n` assembles to `RORI` with `64 - n`; `RORI32` rotates the low 32 bits and sign-extends. Rotates write no flags. *(D)*

9.8.5 `DSHR` is a double-width funnel shift. It writes no flags. *(D)*

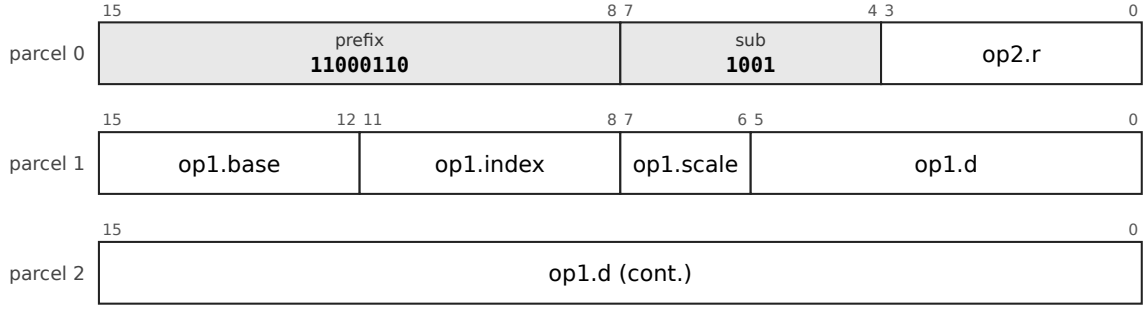
Commentary. **Precedent.** The PDP-11 had `ASL`, `ASR`, `ROL` and `ROR` by one through the carry; x86's `SHRD` and ARM's `EXTR` are the funnel shifts. **Trade-offs.** One rotate direction is enough, so there is no by-one `ROL` in a two-byte slot; rotates are not through the carry because a carry chain is fragile when the carry flag is shared. **Improvements.** The shift count comes from `rm`, not from a fixed register like x86's `CL`, which costs a register shuffle in every hand-written loop. **Benefits.** Multi-precision shifting is `DSHR`, two instructions for any distance. **Modern comparables.** RISC-V and AArch64 have rotates by register and by immediate; x86 has `ROL`/`ROR` through the carry as well. **Use.** `RORI` and `RORI32` are for hashes and ciphers, which rotate by constants; `DSHR` shifts a bignum. **Impact.** *Kernel:* Bit tests and mask operations on the status and page-table words are single instructions. *Compiler:* The shift count may be any register, immediate or memory operand, so the compiler never has to move a count into a fixed register; hashes and ciphers rotate by constants. *Hardware:* The barrel shifter (about 450 lookup tables) serves shifts and rotates, and writing no flag for rotates removes a result path.

BIC: Bit clear

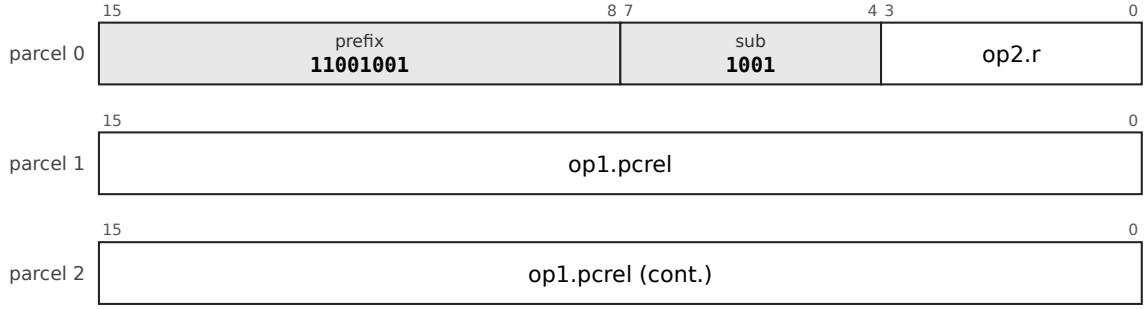
BIC: LDF.B28, prefix 11000101, sub-opcode 1001; operands b4d28s-x1, reg4; 6 bytes



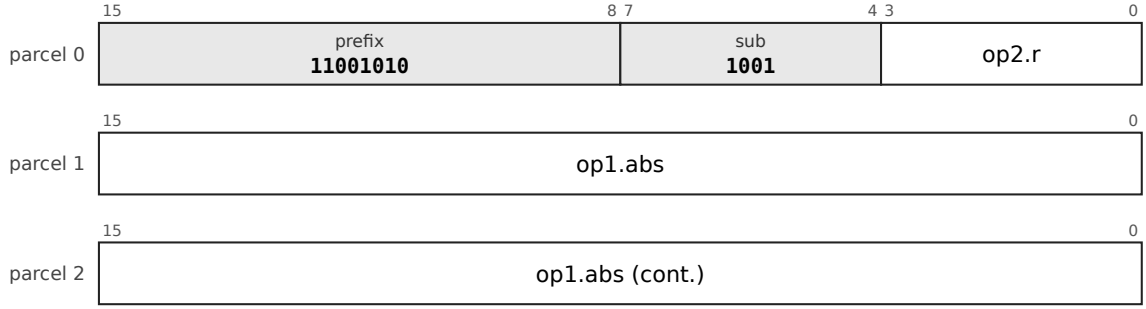
BIC: LDF.X22, prefix 11000110, sub-opcode 1001; operands x4-4d22s, reg4; 6 bytes



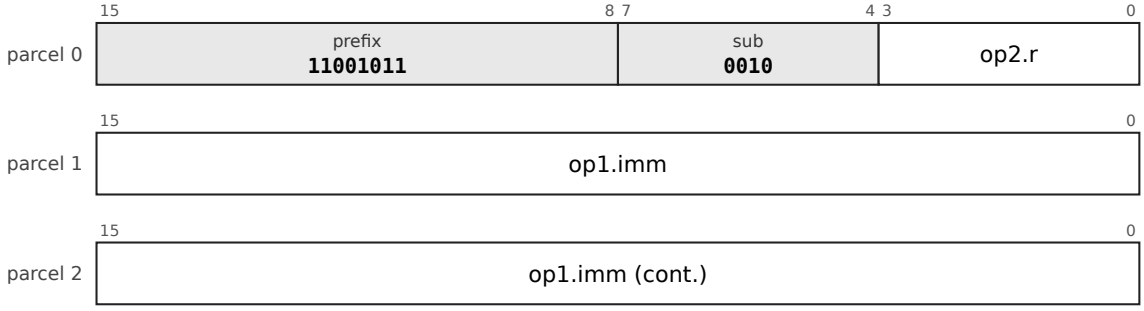
BIC: LDF.PC32, prefix 11001001, sub-opcode 1001; operands pc32-u1, reg4; 6 bytes



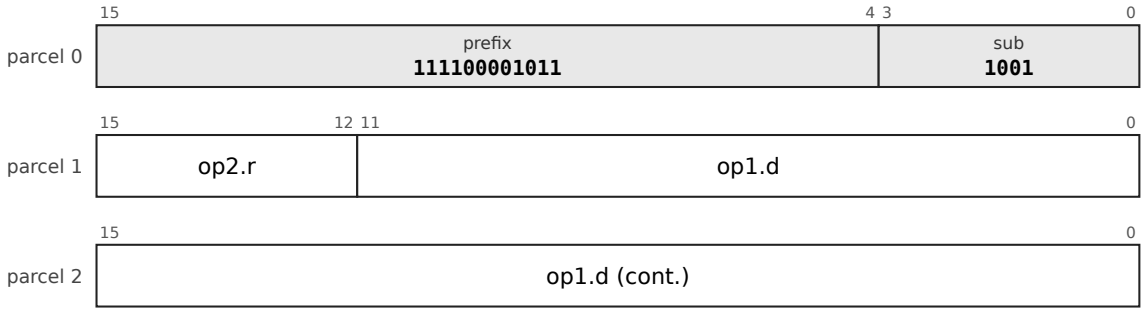
BIC: LDF.A32, prefix 11001010, sub-opcode 1001; operands abs32, reg4; 6 bytes



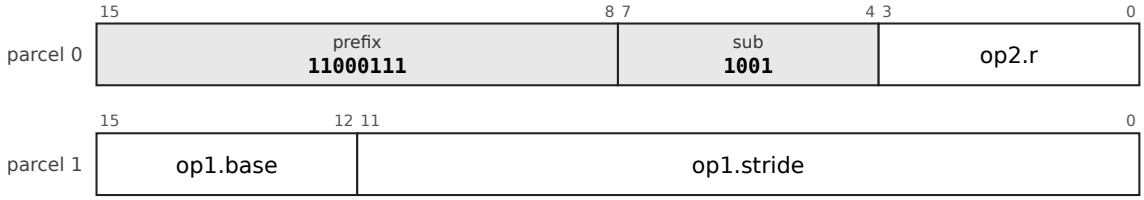
BIC: LDFI.I32, prefix 11001011, sub-opcode 0010; operands imm32s, reg4; 6 bytes



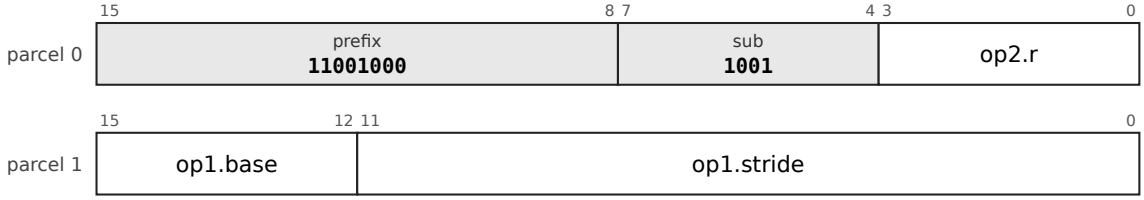
BIC: DPF.D28, prefix 111100001011, sub-opcode 1001; operands dpd28s-x1, reg4; 6 bytes



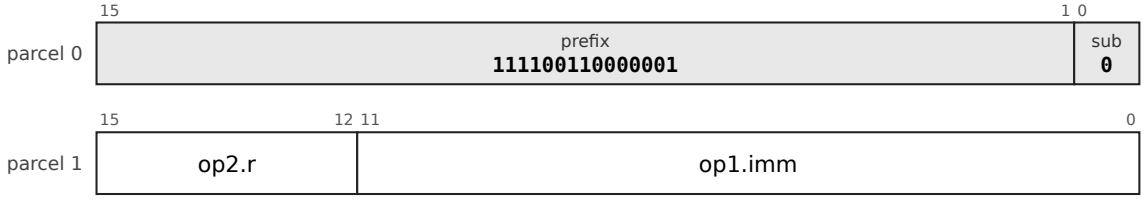
BIC: LDF.PI12, prefix 11000111, sub-opcode 1001; operands pi4-s12, reg4; 4 bytes



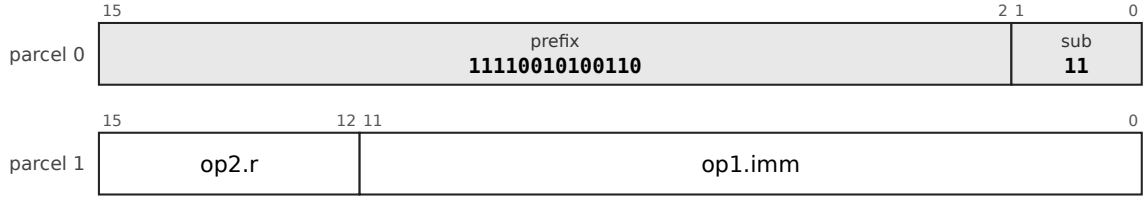
BIC: LDF.PD12, prefix 11001000, sub-opcode 1001; operands pd4-s12, reg4; 4 bytes



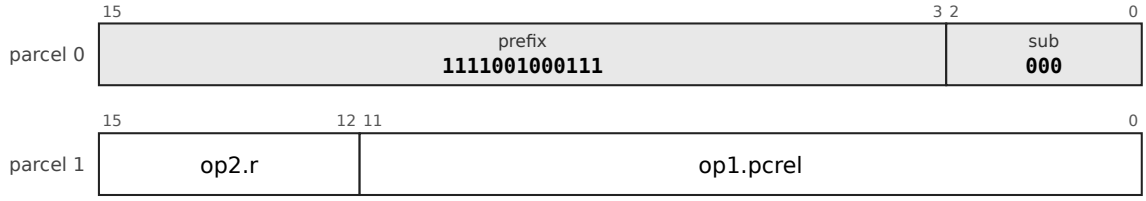
BIC: LDI12/d, prefix 111100110000001, sub-opcode 0; operands imm12s, reg4; 4 bytes



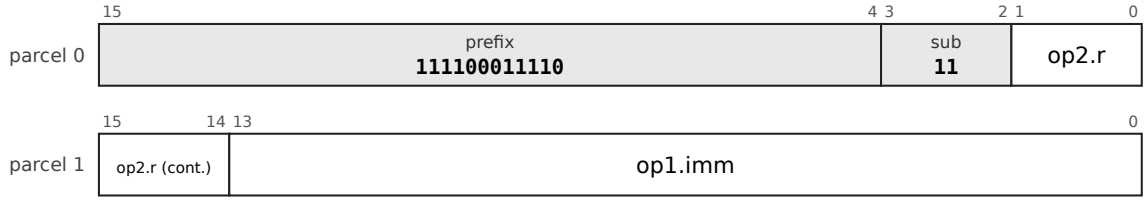
BIC: LDI12/c, prefix 11110010100110, sub-opcode 11; operands imm12s, reg4; 4 bytes



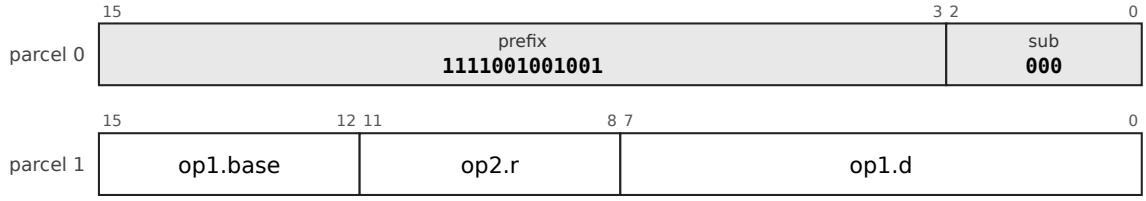
BIC: LDPC12/d, prefix 1111001000111, sub-opcode 000; operands pc12-u1, reg4; 4 bytes



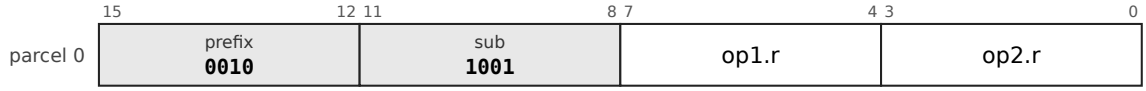
BIC: LDI14/c, prefix 111100011110, sub-opcode 11; operands imm14s, reg4; 4 bytes



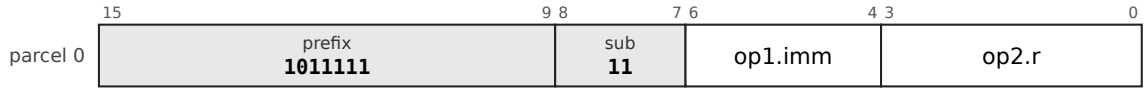
BIC: LDBs8/d, prefix 1111001001001, sub-opcode 000; operands b4d8s-xsz, reg4; 4 bytes



BIC: RRF, prefix 0010, sub-opcode 1001; operands reg4, reg4; 2 bytes



BIC: IMM3/c, prefix 1011111, sub-opcode 11; operands imm3s, reg4; 2 bytes

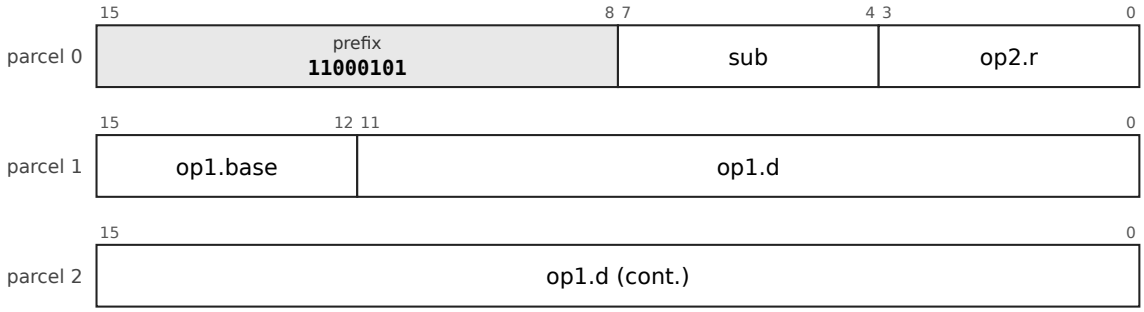


Mnemonics: `BIC`

Syntax

```
BIC rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDI12-d-2`, `16-LDI12-c-1`, `16-LDPC12-d-1`, `16-IMM3-c-1`, `16-LDI14-c-1`, `16-LDBs8-d-1`.

Operation

```
Rd ← Rd AND NOT rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

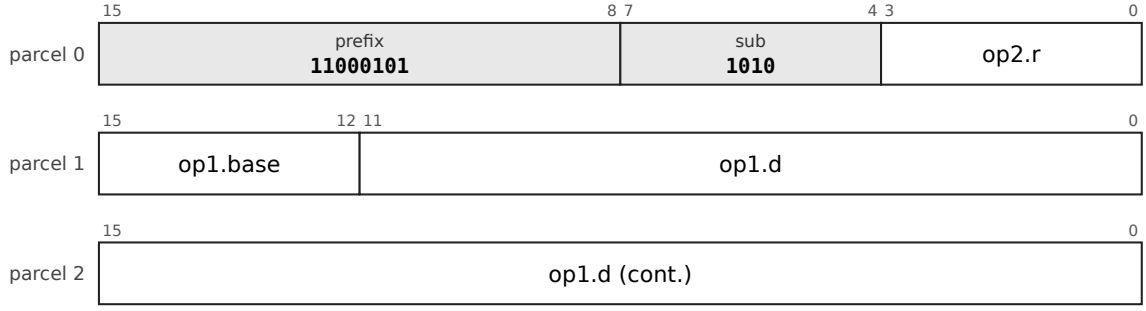
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The assembler also accepts `BIC #imm, rm` as `AND #~imm, rm`.

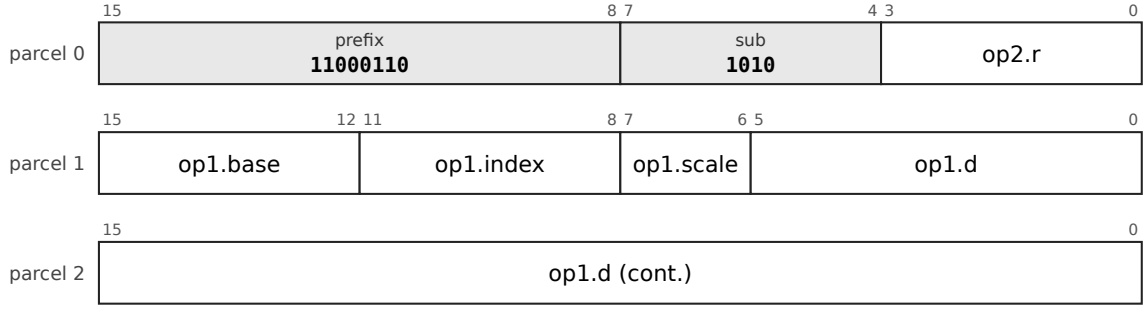
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BIS: Bit set (inclusive or)

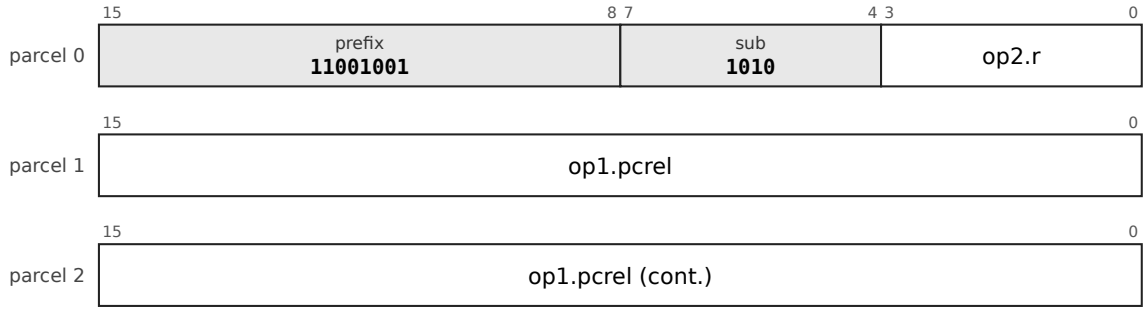
BIS: LDF.B28, prefix 11000101, sub-opcode 1010; operands b4d28s-x1, reg4; 6 bytes



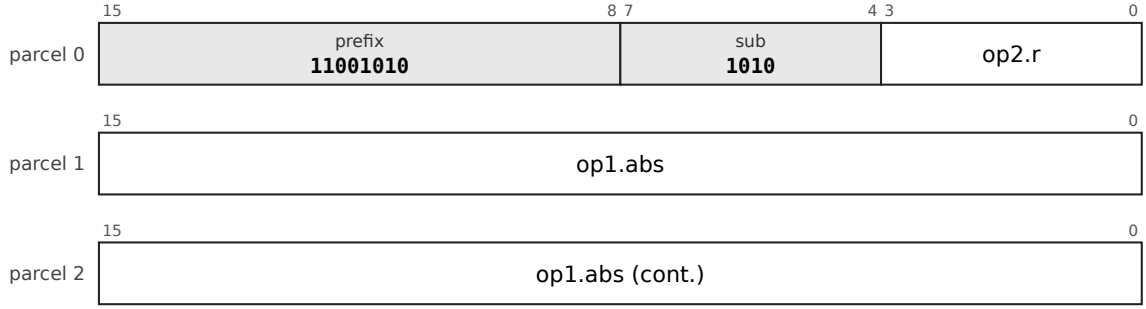
BIS: LDF.X22, prefix 11000110, sub-opcode 1010; operands x4-4d22s, reg4; 6 bytes



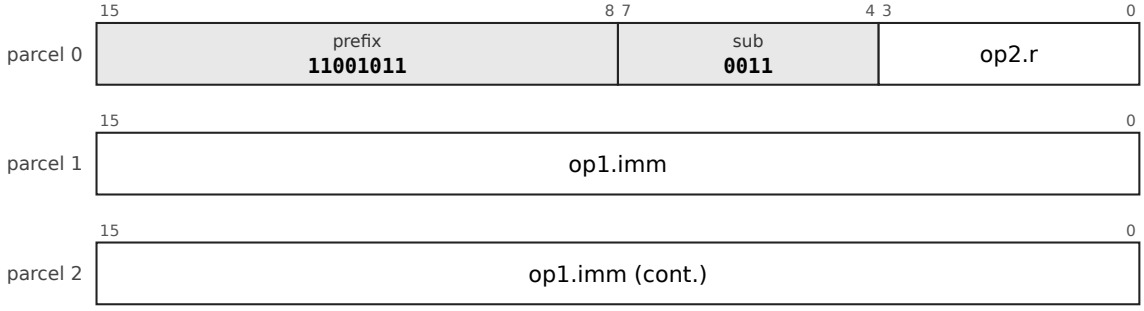
BIS: LDF.PC32, prefix 11001001, sub-opcode 1010; operands pc32-u1, reg4; 6 bytes



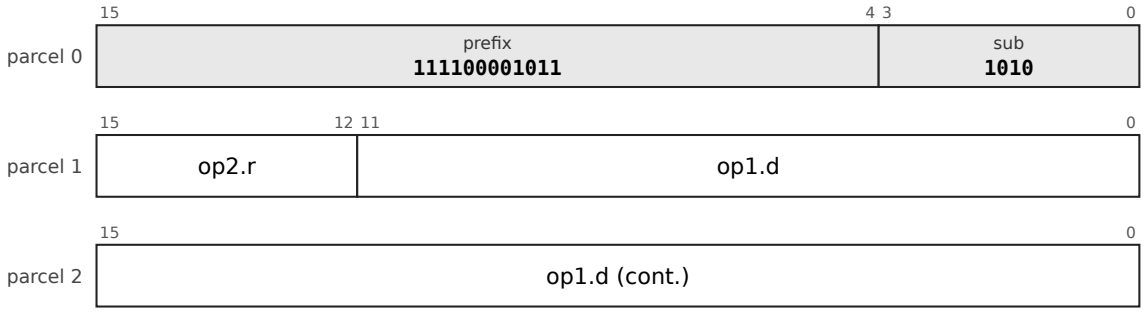
BIS: LDF.A32, prefix 11001010, sub-opcode 1010; operands abs32, reg4; 6 bytes



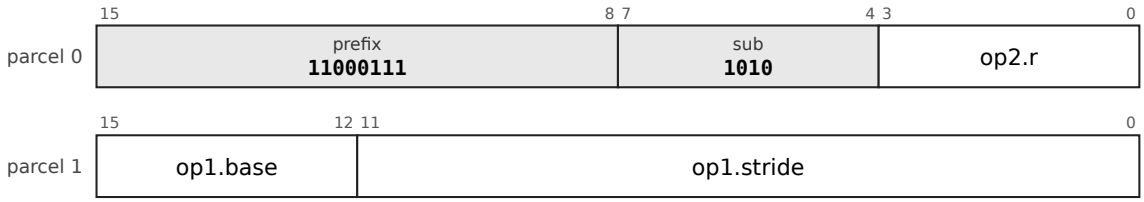
BIS: LDFI.I32, prefix 11001011, sub-opcode 0011; operands imm32s, reg4; 6 bytes



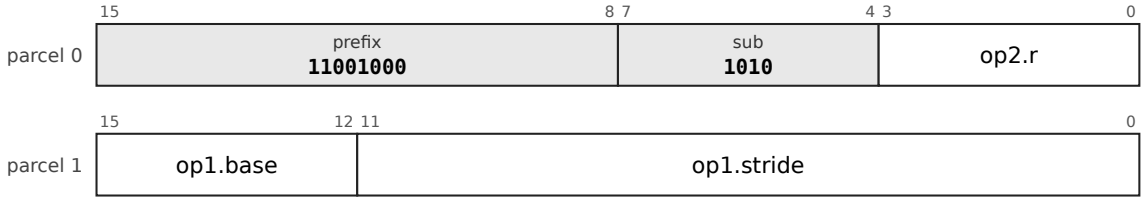
BIS: DPF.D28, prefix 111100001011, sub-opcode 1010; operands dpd28s-x1, reg4; 6 bytes



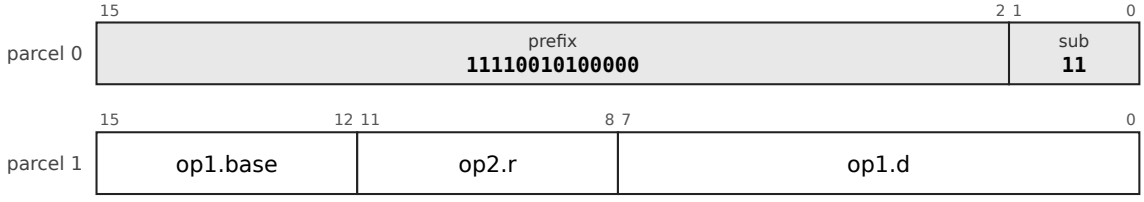
BIS: LDF.PI12, prefix 11000111, sub-opcode 1010; operands pi4-s12, reg4; 4 bytes



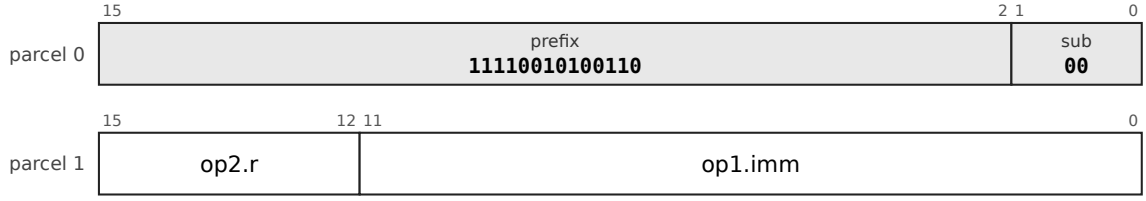
BIS: LDF.PD12, prefix 11001000, sub-opcode 1010; operands pd4-s12, reg4; 4 bytes



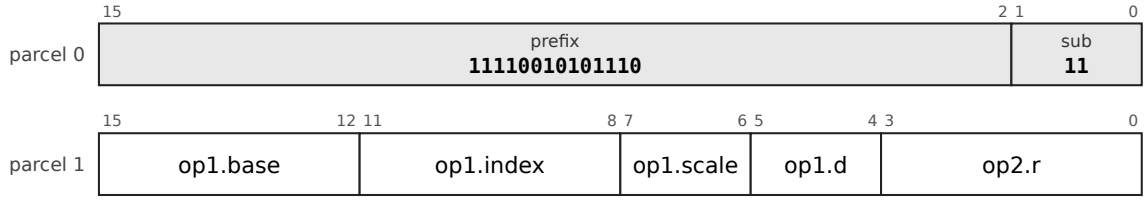
BIS: LDBu8/c, prefix 11110010100000, sub-opcode 11; operands b4d8u-xsz, reg4; 4 bytes



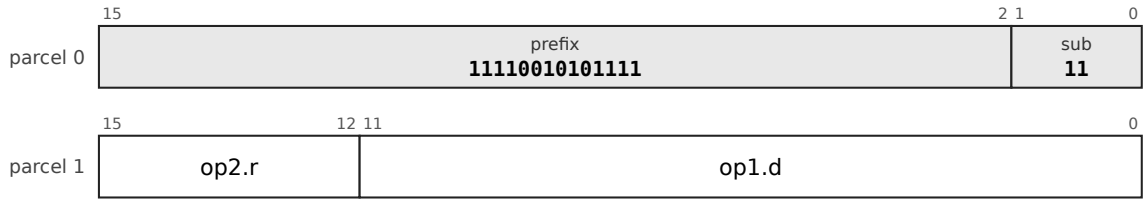
BIS: LDI12/c, prefix 11110010100110, sub-opcode 00; operands imm12s, reg4; 4 bytes



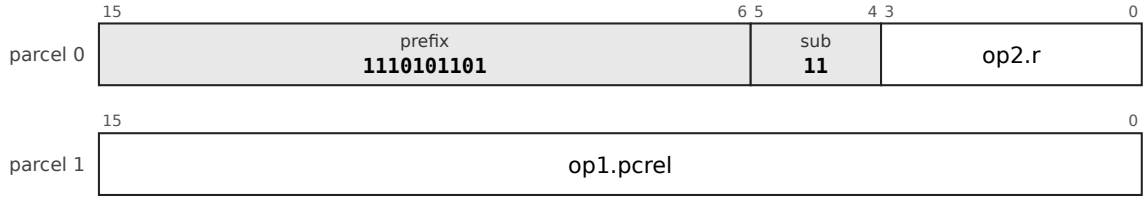
BIS: LDX2/c, prefix 11110010101110, sub-opcode 11; operands x4-4d2s, reg4; 4 bytes



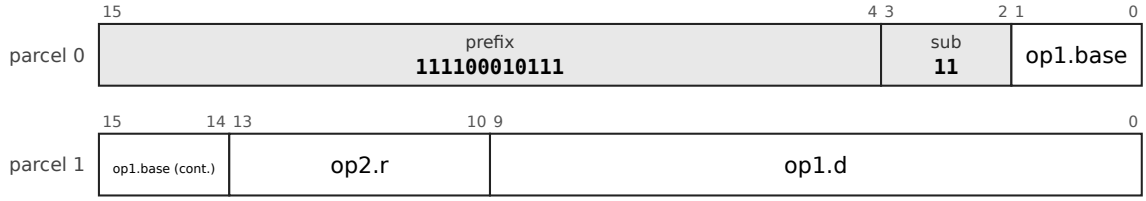
BIS: LDS12/c, prefix 11110010101111, sub-opcode 11; operands spd12u-xsz, reg4; 4 bytes



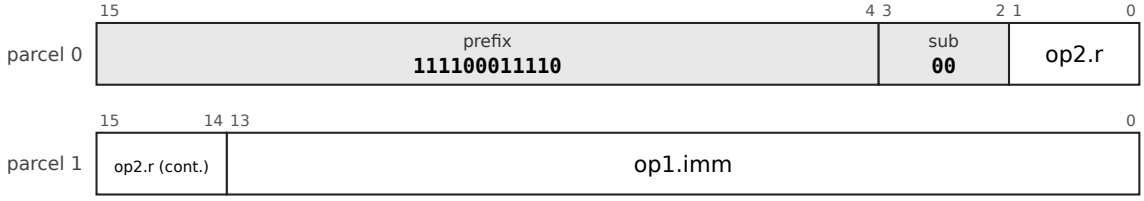
BIS: LDPC16/c, prefix 1110101101, sub-opcode 11; operands pc16-u1, reg4; 4 bytes



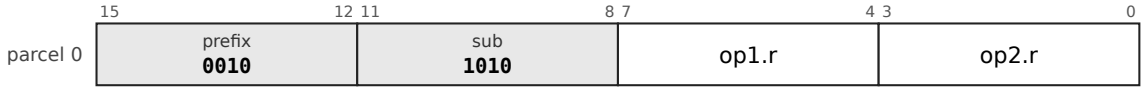
BIS: LDBs10/c, prefix 111100010111, sub-opcode 11; operands b4d10s-xsz, reg4; 4 bytes



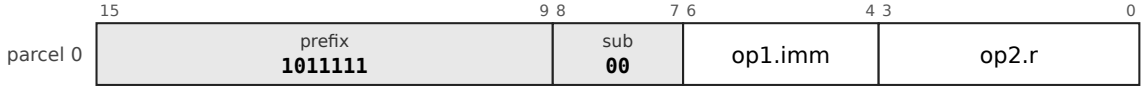
BIS: LDI14/c, prefix 111100011110, sub-opcode 00; operands imm14s, reg4; 4 bytes



BIS: RRF, prefix 0010, sub-opcode 1010; operands reg4, reg4; 2 bytes



BIS: IMM3/c, prefix 1011111, sub-opcode 00; operands imm3s, reg4; 2 bytes

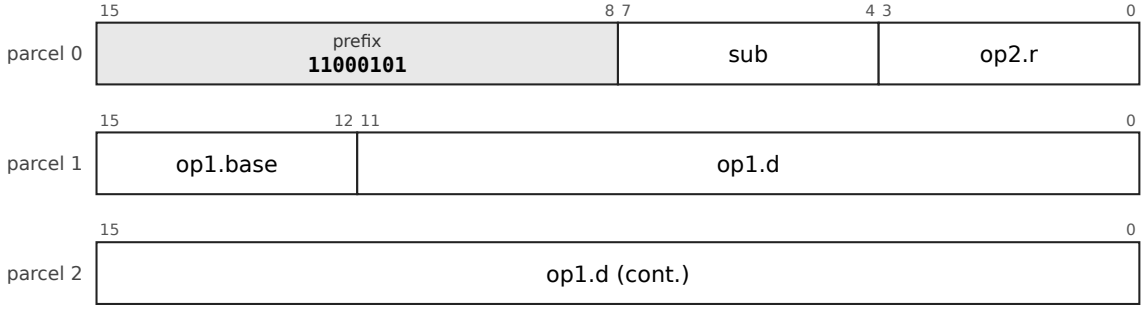


Mnemonics: `BIS`

Syntax

```
BIS rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDBu8-c-1`, `16-LDI12-c-1`, `16-LDX2-c-1`, `16-LDS12-c-1`, `16-LDPC16-c-1`, `16-LDBs10-c-1`, `16-IMM3-c-1`, `16-LDI14-c-1`.

Operation

```
Rd ← Rd OR rm
```

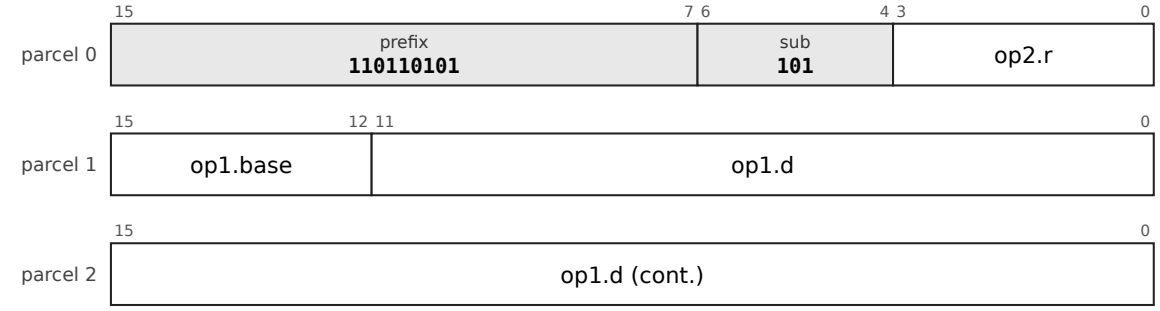
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

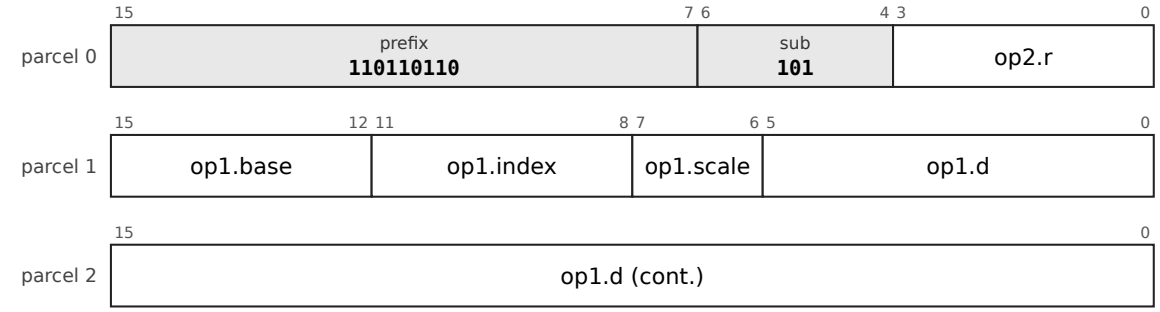
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BIT: Bit test

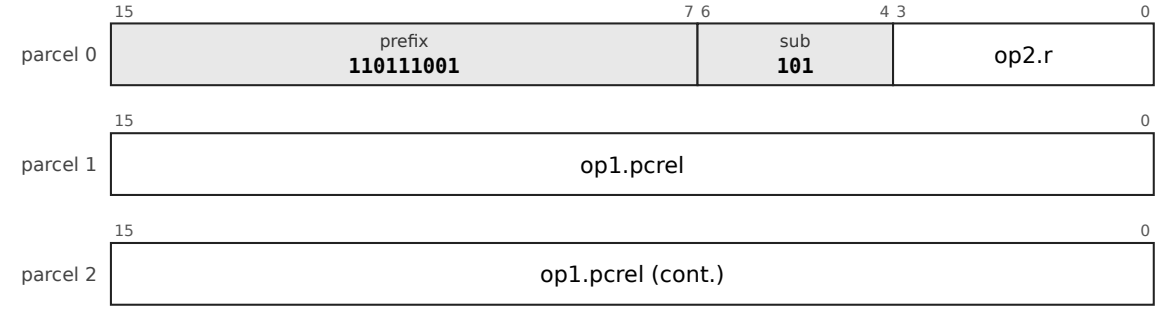
BIT: LDF.B28, prefix 110110101, sub-opcode 101; operands b4d28s-x1, reg4; 6 bytes



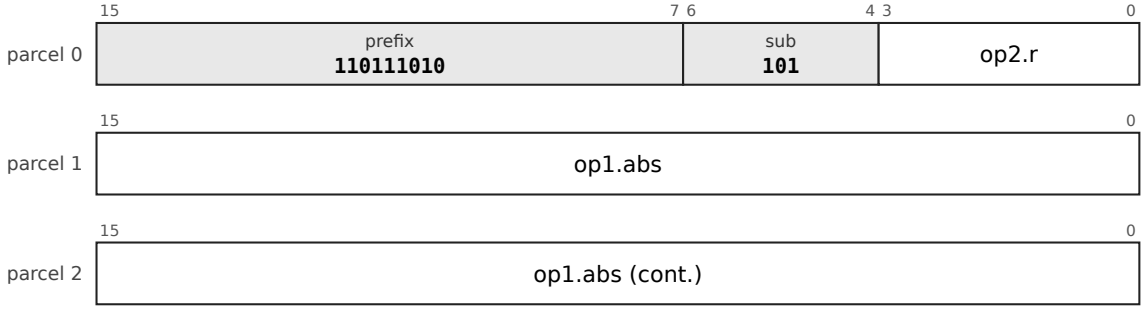
BIT: LDF.X22, prefix 110110110, sub-opcode 101; operands x4-4d22s, reg4; 6 bytes



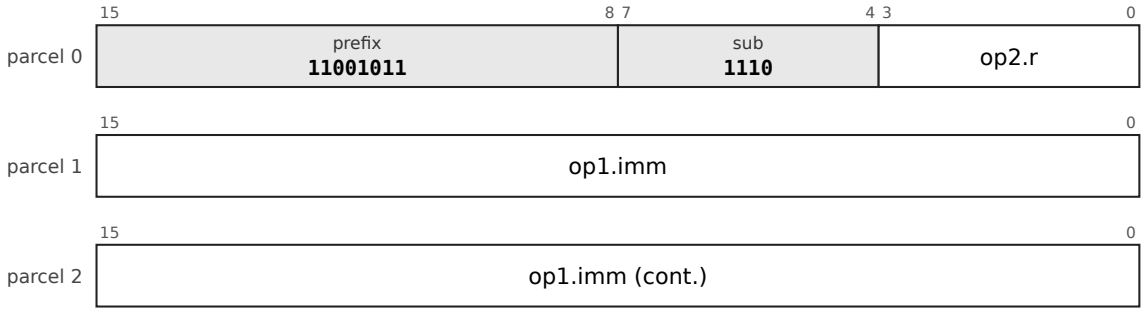
BIT: LDF.PC32, prefix 110111001, sub-opcode 101; operands pc32-u1, reg4; 6 bytes



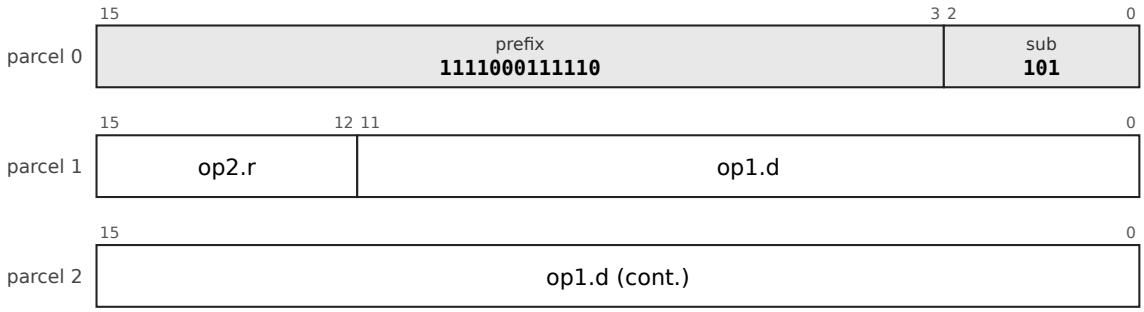
BIT: LDF.A32, prefix 110111010, sub-opcode 101; operands abs32, reg4; 6 bytes



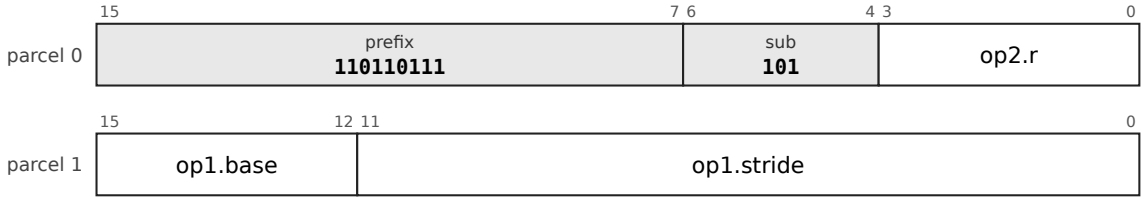
BIT: LDFI.I32, prefix 11001011, sub-opcode 1110; operands imm32s, reg4; 6 bytes



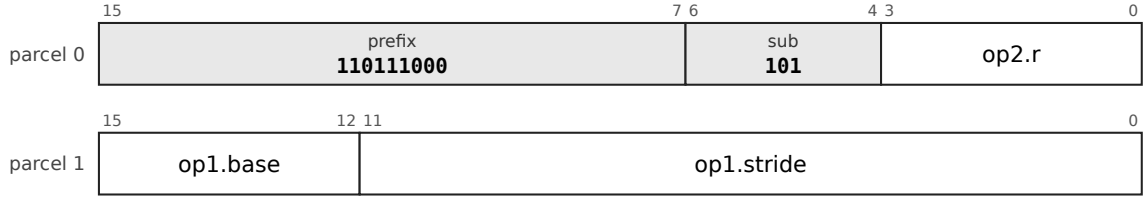
BIT: DPF.D28, prefix 1111000111110, sub-opcode 101; operands dpd28s-x1, reg4; 6 bytes



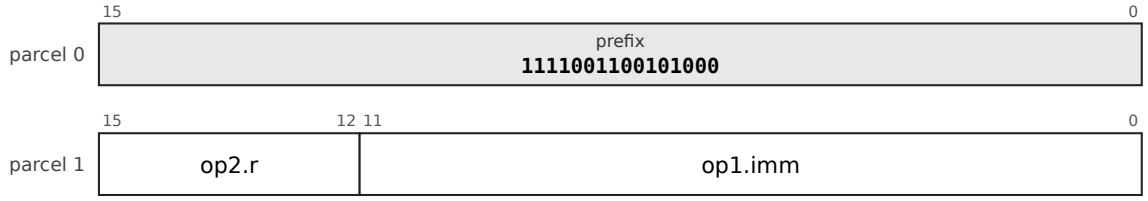
BIT: LDF.PI12, prefix 110110111, sub-opcode 101; operands pi4-s12, reg4; 4 bytes



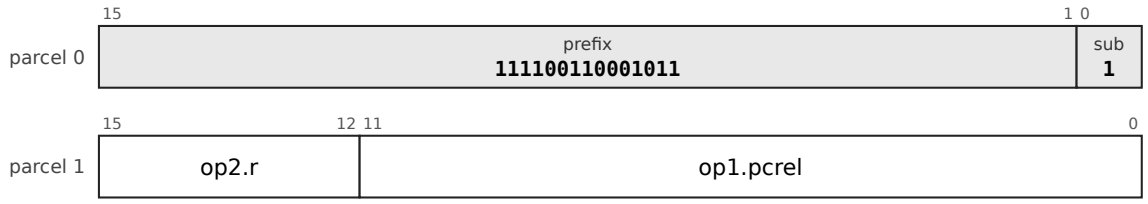
BIT: LDF.PD12, prefix 110111000, sub-opcode 101; operands pd4-s12, reg4; 4 bytes



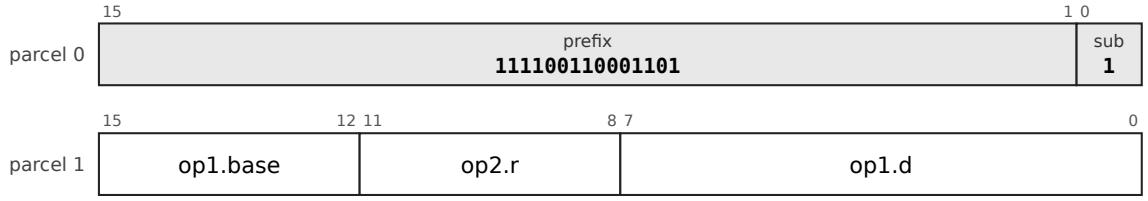
BIT: LDI12/d, prefix 1111001100101000; operands imm12s, reg4; 4 bytes



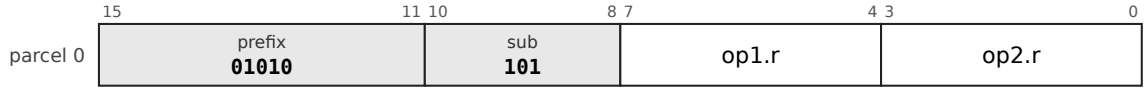
BIT: LDPC12/d, prefix 111100110001011, sub-opcode 1; operands pc12-u1, reg4; 4 bytes



BIT: LDBs8/d, prefix 111100110001101, sub-opcode 1; operands b4d8s-xsz, reg4; 4 bytes



BIT: RRF, prefix 01010, sub-opcode 101; operands reg4, reg4; 2 bytes

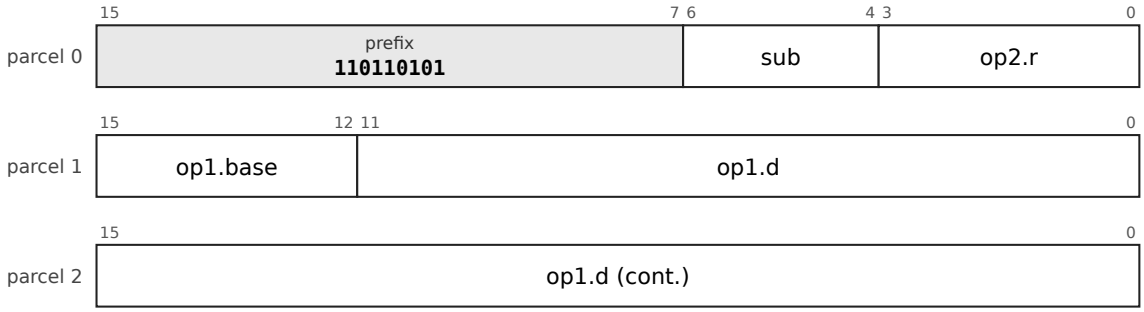


Mnemonics: `BIT`

Syntax

```
BIT rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-2`, LDF.B28, prefix `110110101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-2`, `16-LDF-X22-2`, `16-LDF-PI12-2`, `16-LDF-PD12-2`, `16-LDF-PC32-2`, `16-LDF-A32-2`, `16-LDFI-I32-1`, `16-DPF-D28-2`, `16-RRF-2`, `16-LDI12-d-3`, `16-LDPC12-d-2`, `16-LDBs8-d-2`.

Operation

```
tmp ← Rd AND rm         ; the result is discarded
```

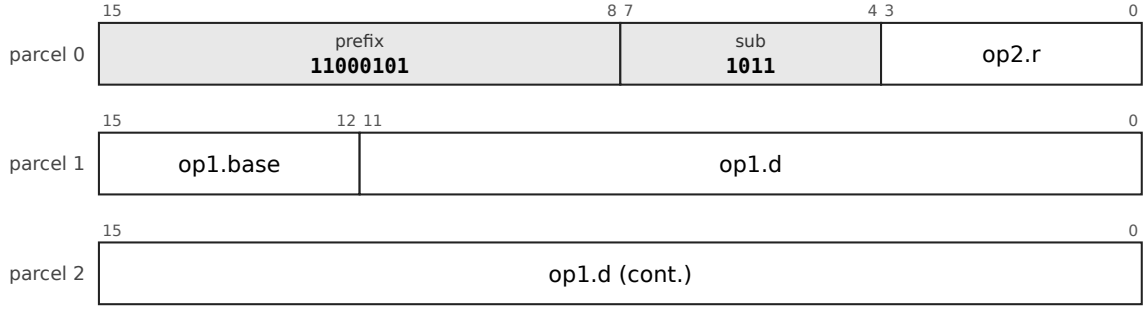
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

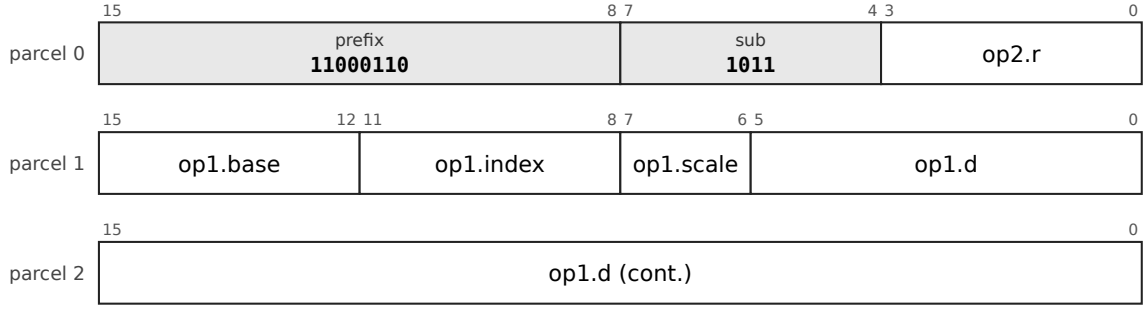
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

XOR: Exclusive or

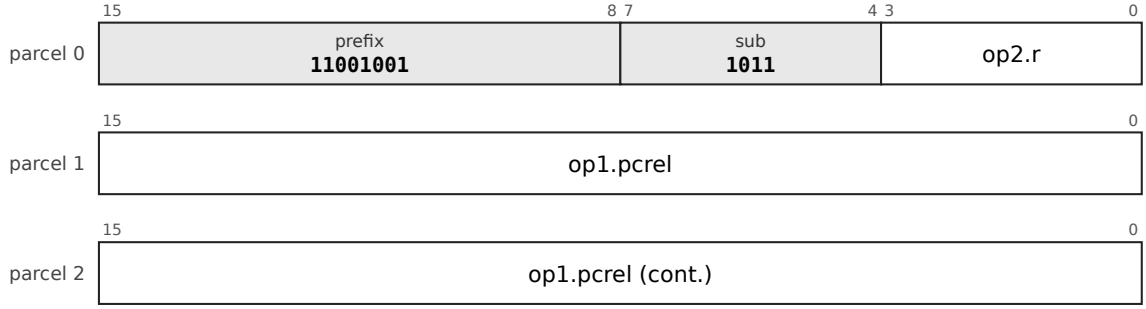
XOR: LDF.B28, prefix 11000101, sub-opcode 1011; operands b4d28s-x1, reg4; 6 bytes



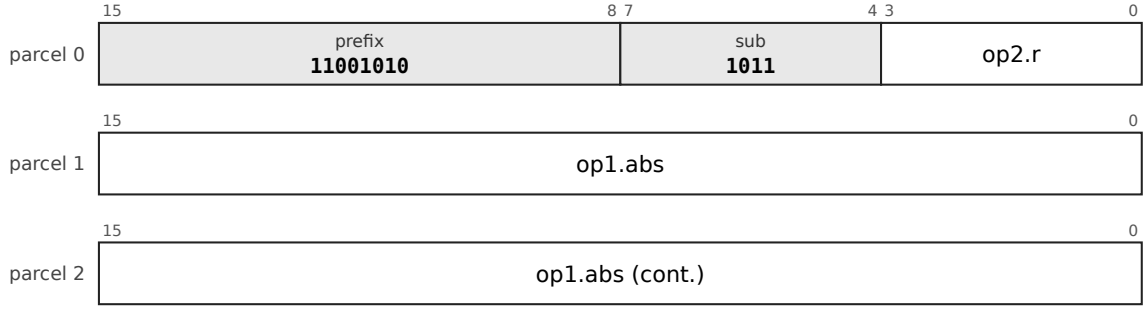
XOR: LDF.X22, prefix 11000110, sub-opcode 1011; operands x4-4d22s, reg4; 6 bytes



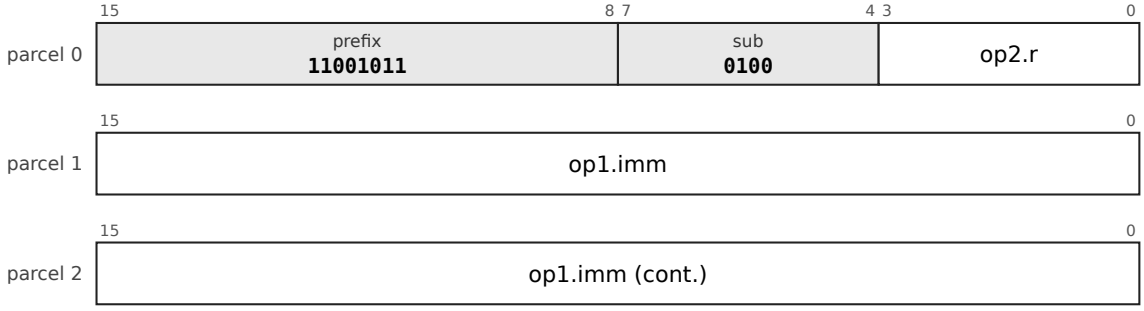
XOR: LDF.PC32, prefix 11001001, sub-opcode 1011; operands pc32-u1, reg4; 6 bytes



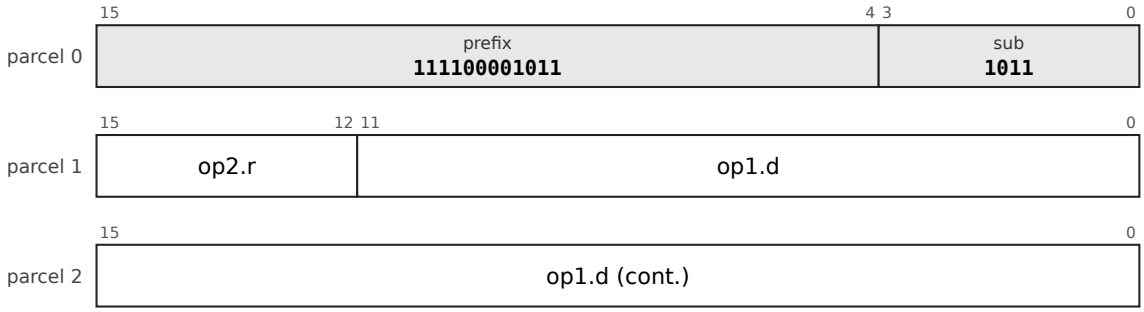
XOR: LDF.A32, prefix 11001010, sub-opcode 1011; operands abs32, reg4; 6 bytes



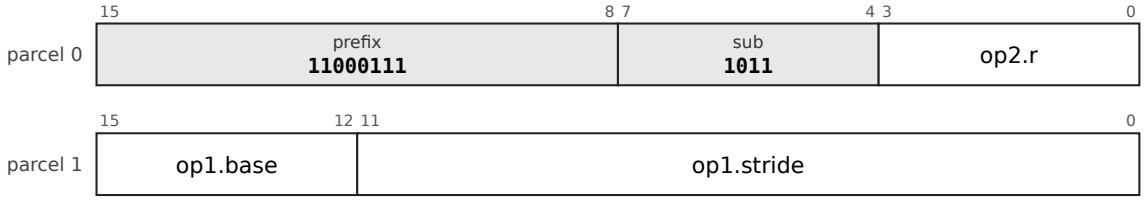
XOR: LDFI.I32, prefix 11001011, sub-opcode 0100; operands imm32s, reg4; 6 bytes



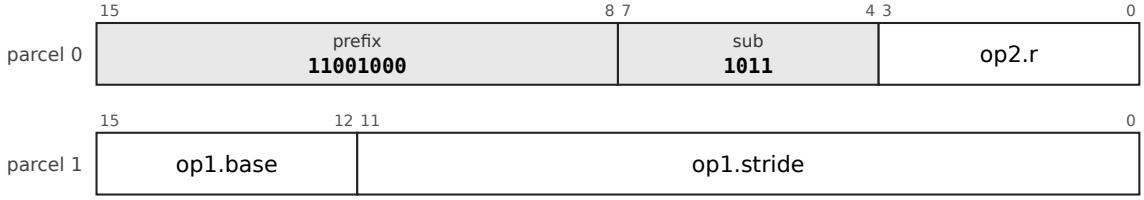
XOR: DPF.D28, prefix 111100001011, sub-opcode 1011; operands dpd28s-x1, reg4; 6 bytes



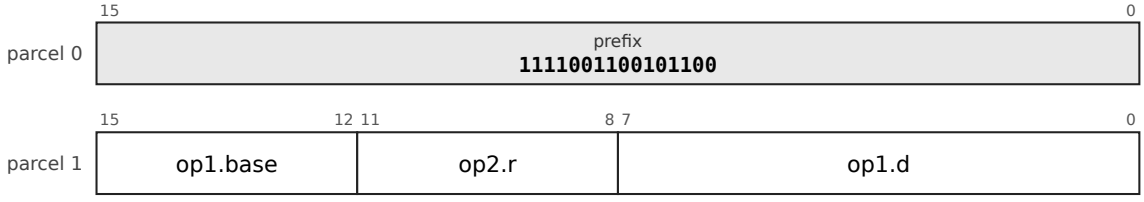
XOR: LDF.PI12, prefix 11000111, sub-opcode 1011; operands pi4-s12, reg4; 4 bytes



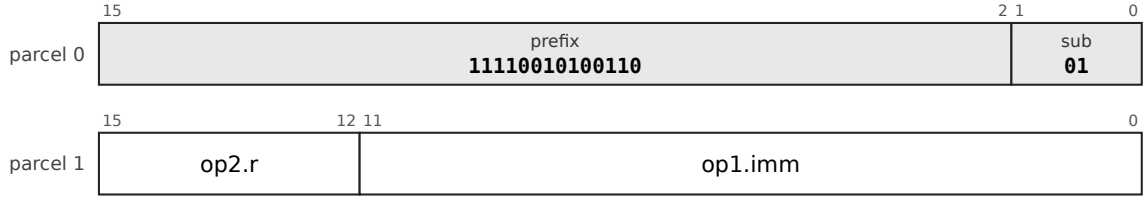
XOR: LDF.PD12, prefix 11001000, sub-opcode 1011; operands pd4-s12, reg4; 4 bytes



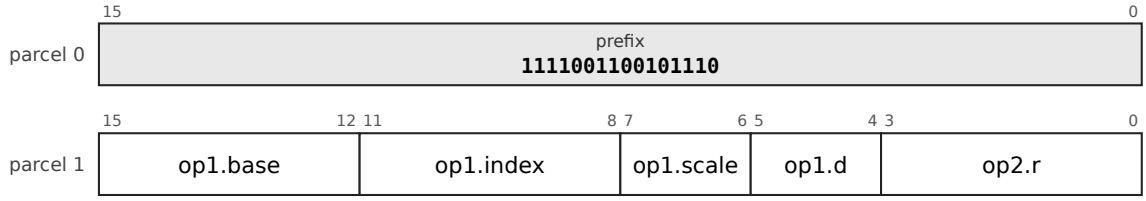
XOR: LDBu8/c, prefix 1111001100101100; operands b4d8u-xsz, reg4; 4 bytes



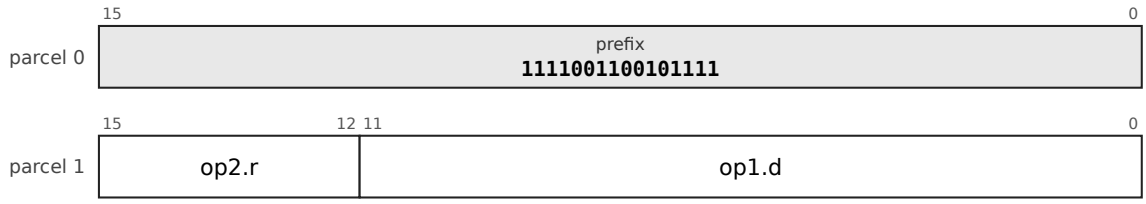
XOR: LDI12/c, prefix 11110010100110, sub-opcode 01; operands imm12s, reg4; 4 bytes



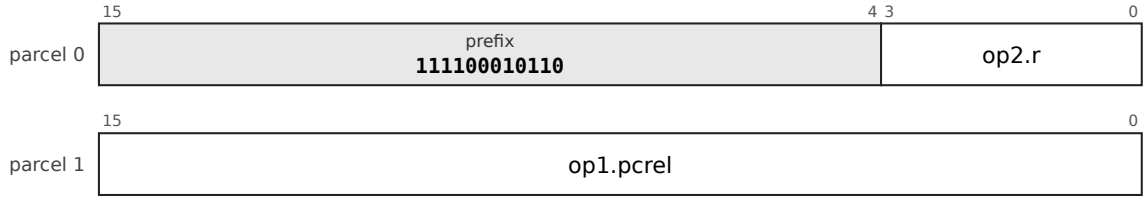
XOR: LDX2/c, prefix 1111001100101110; operands x4-4d2s, reg4; 4 bytes



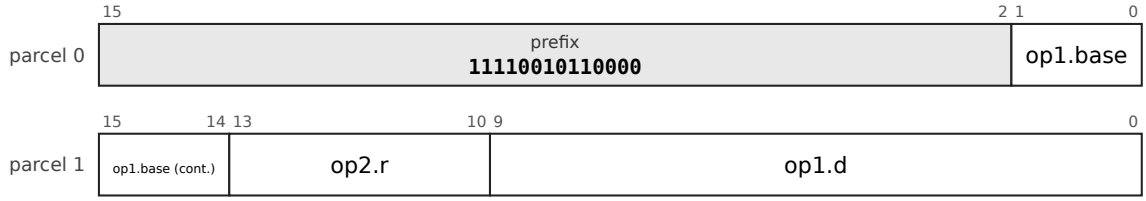
XOR: LDS12/c, prefix 1111001100101111; operands spd12u-xsz, reg4; 4 bytes



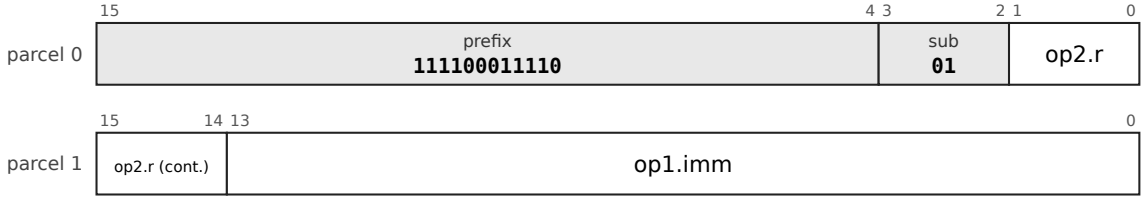
XOR: LDPC16/c, prefix 111100010110; operands pc16-u1, reg4; 4 bytes



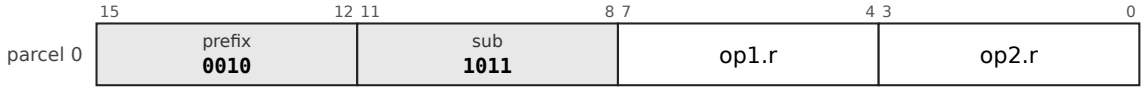
XOR: LDBs10/c, prefix 11110010110000; operands b4d10s-xsz, reg4; 4 bytes



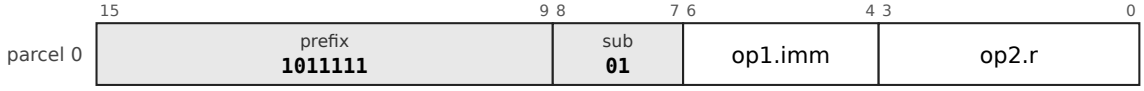
XOR: LDI14/c, prefix 111100011110, sub-opcode 01; operands imm14s, reg4; 4 bytes



XOR: RRF, prefix 0010, sub-opcode 1011; operands reg4, reg4; 2 bytes



XOR: IMM3/c, prefix 1011111, sub-opcode 01; operands imm3s, reg4; 2 bytes

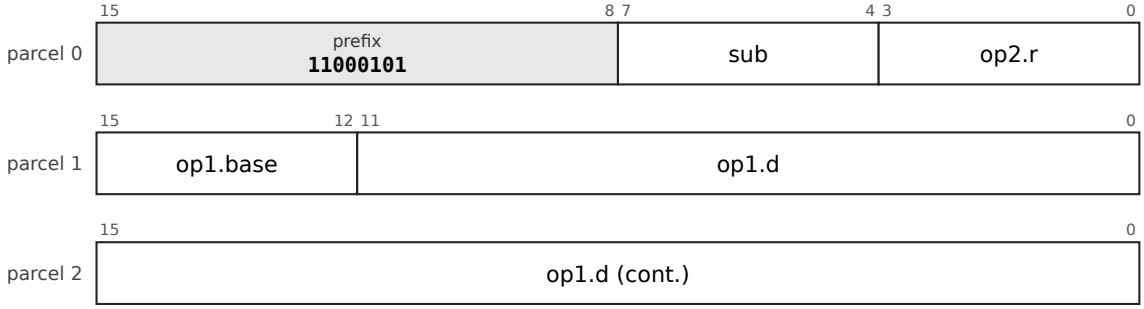


Mnemonics: `XOR`

Syntax

```
XOR rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDBu8-c-2`, `16-LDI12-c-1`, `16-LDX2-c-2`, `16-LDS12-c-2`, `16-LDPC16-c-2`, `16-LDBs10-c-2`, `16-IMM3-c-1`, `16-LDI14-c-1`.

Operation

```
Rd ← Rd XOR rm
```

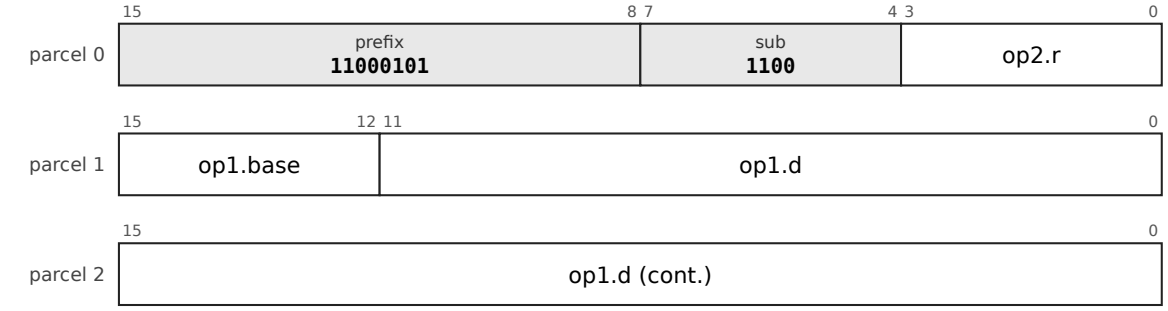
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

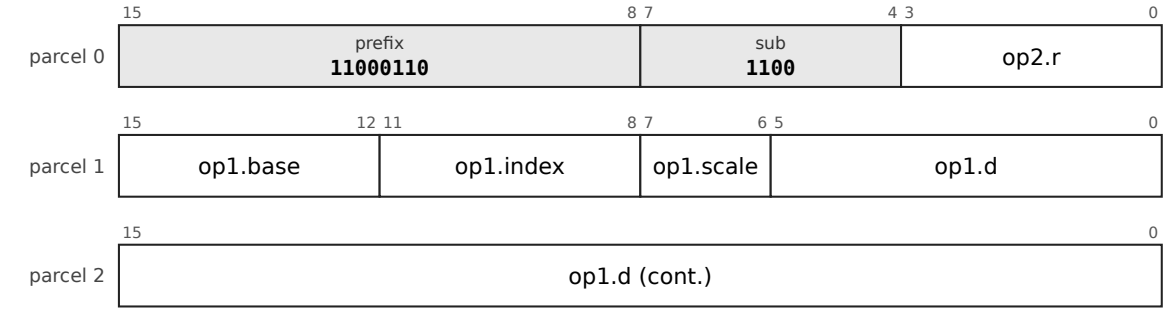
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

AND: And

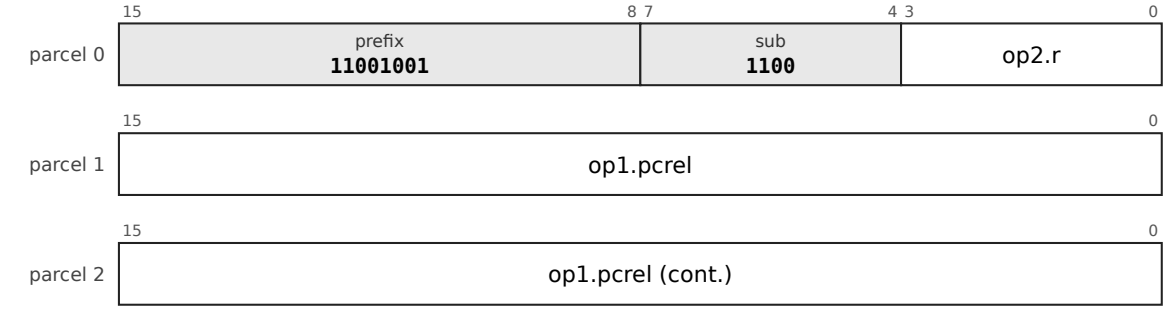
AND: LDF.B28, prefix 11000101, sub-opcode 1100; operands b4d28s-x1, reg4; 6 bytes



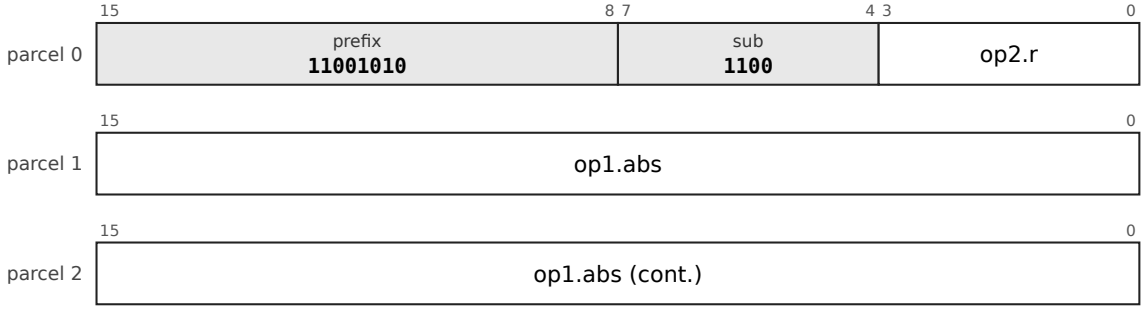
AND: LDF.X22, prefix 11000110, sub-opcode 1100; operands x4-4d22s, reg4; 6 bytes



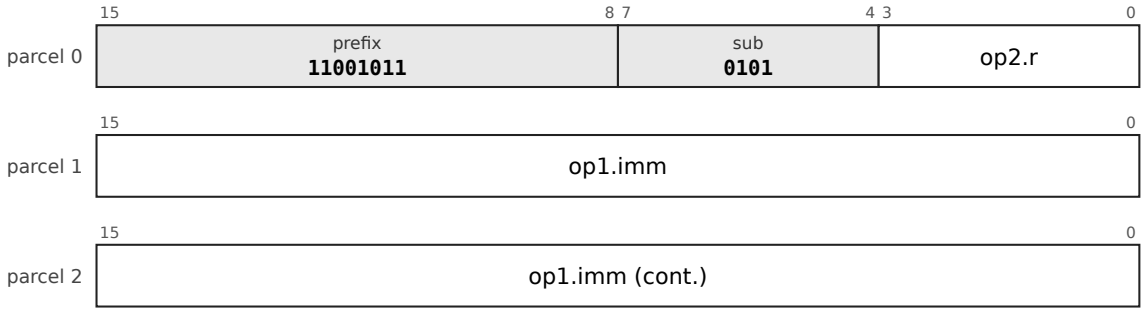
AND: LDF.PC32, prefix 11001001, sub-opcode 1100; operands pc32-u1, reg4; 6 bytes



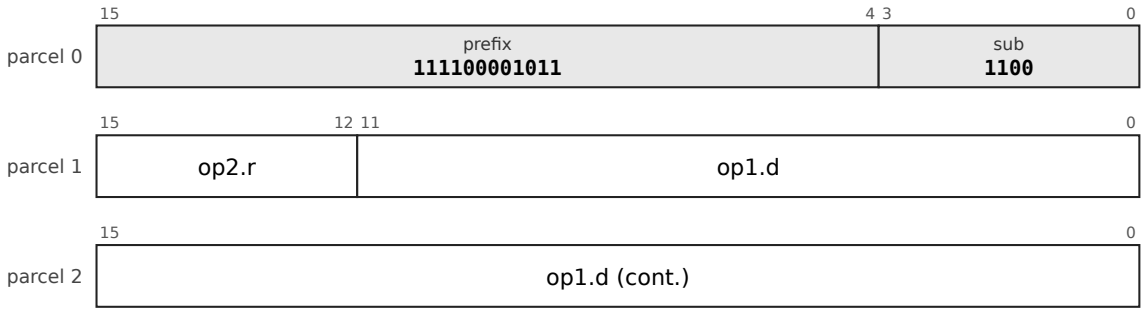
AND: LDF.A32, prefix 11001010, sub-opcode 1100; operands abs32, reg4; 6 bytes



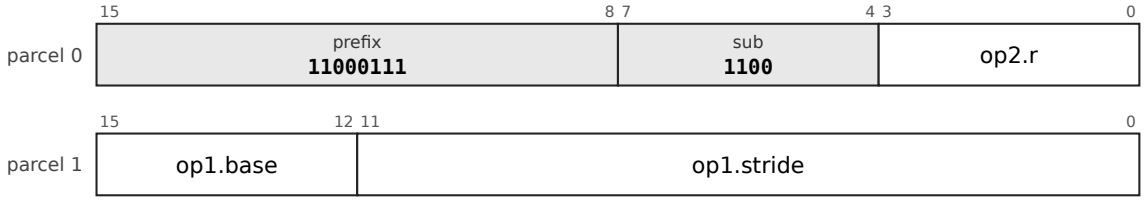
AND: LDFI.I32, prefix 11001011, sub-opcode 0101; operands imm32s, reg4; 6 bytes



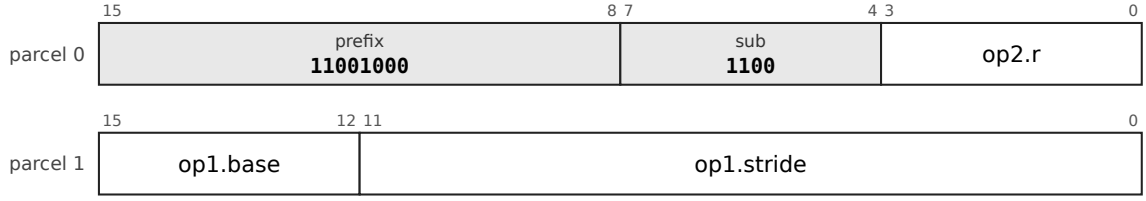
AND: DPF.D28, prefix 111100001011, sub-opcode 1100; operands dpd28s-x1, reg4; 6 bytes



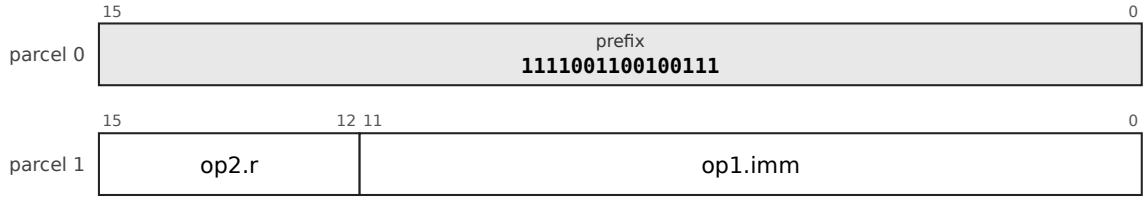
AND: LDF.PI12, prefix 11000111, sub-opcode 1100; operands pi4-s12, reg4; 4 bytes



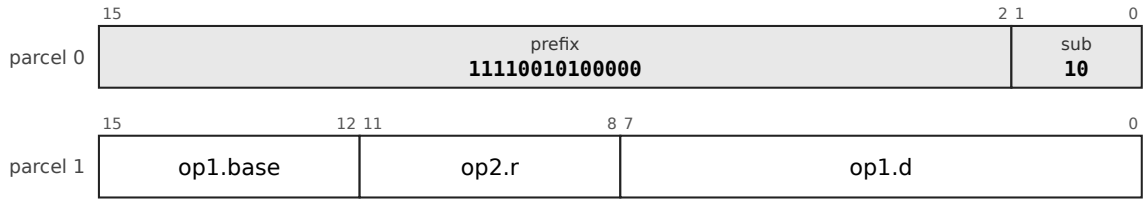
AND: LDF.PD12, prefix 11001000, sub-opcode 1100; operands pd4-s12, reg4; 4 bytes



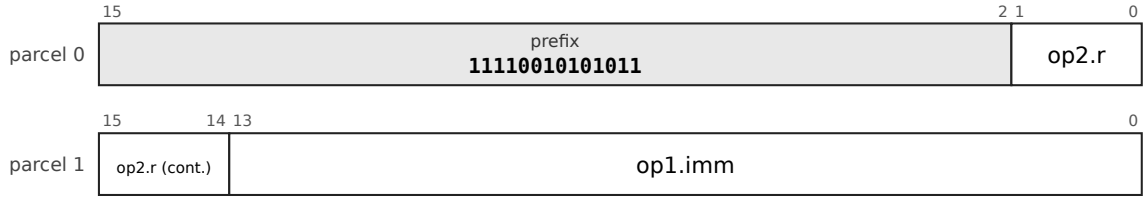
AND: LDI12/b, prefix 1111001100100111; operands imm12s, reg4; 4 bytes



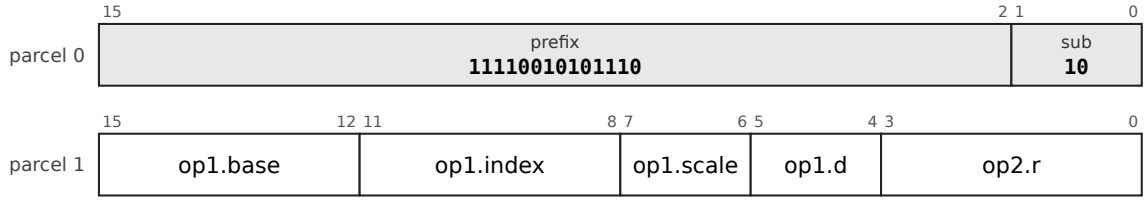
AND: LDBu8/c, prefix 11110010100000, sub-opcode 10; operands b4d8u-xsz, reg4; 4 bytes



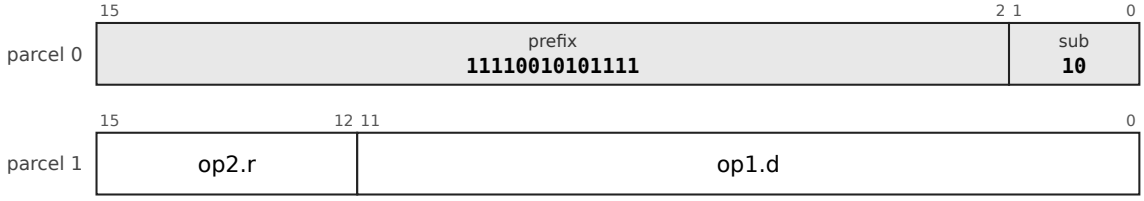
AND: LDI14/b, prefix 11110010101011; operands imm14s, reg4; 4 bytes



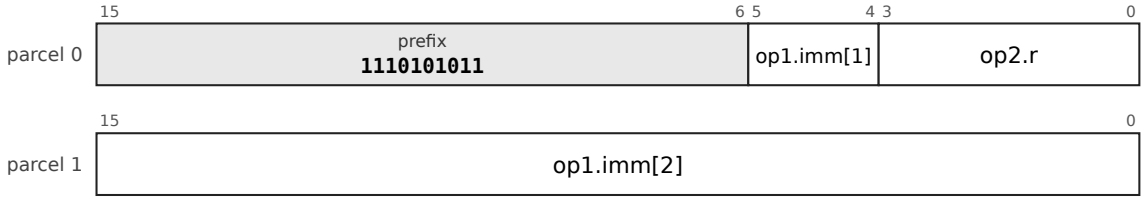
AND: LDX2/c, prefix 11110010101110, sub-opcode 10; operands x4-4d2s, reg4; 4 bytes



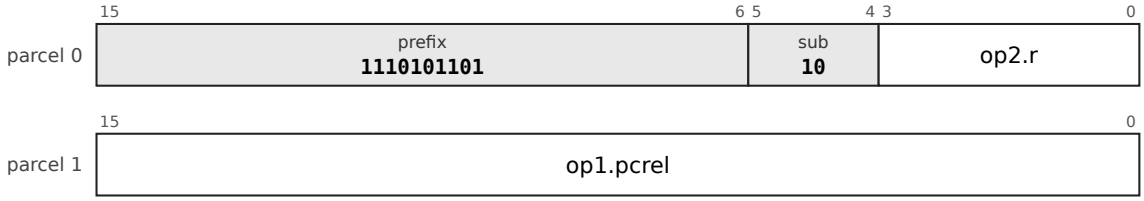
AND: LDS12/c, prefix 11110010101111, sub-opcode 10; operands spd12u-xsz, reg4; 4 bytes



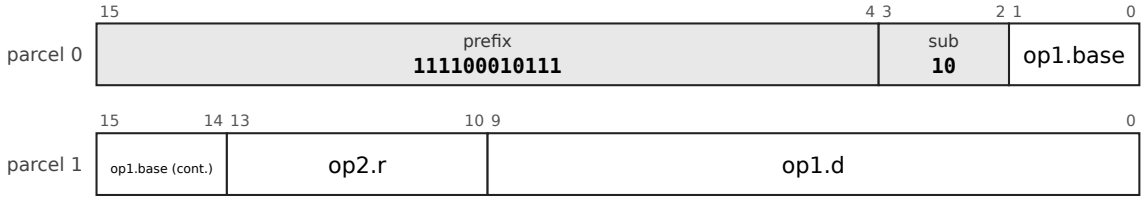
AND: LDI18/b, prefix 1110101011; operands imm18s, reg4; 4 bytes



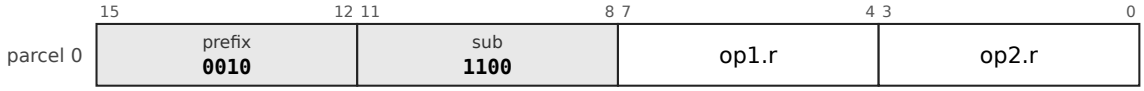
AND: LDPC16/c, prefix 1110101101, sub-opcode 10; operands pc16-u1, reg4; 4 bytes



AND: LDBs10/c, prefix 111100010111, sub-opcode 10; operands b4d10s-xsz, reg4; 4 bytes



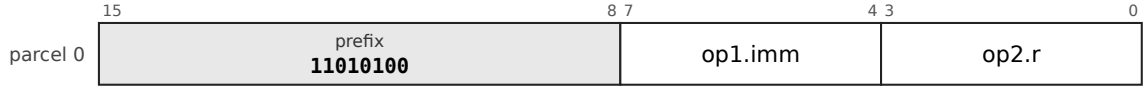
AND: RRF, prefix 0010, sub-opcode 1100; operands reg4, reg4; 2 bytes



AND: IMM3/b, prefix 111000110; operands imm3s, reg4; 2 bytes



AND: IMM4/b, prefix 11010100; operands imm4s, reg4; 2 bytes

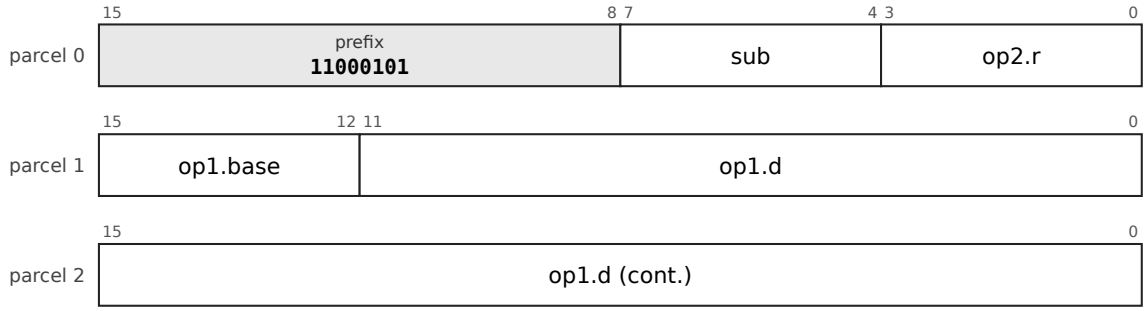


Mnemonics: `AND`

Syntax

```
AND rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDI12-b-2`, `16-LDBu8-c-1`, `16-LDI14-b-2`, `16-LDX2-c-1`, `16-IMM3-b-2`, `16-LDS12-c-1`, `16-LDI18-b-2`, `16-LDPC16-c-1`, `16-LDBs10-c-1`, `16-IMM4-b-2`.

Operation

```
Rd ← Rd AND rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

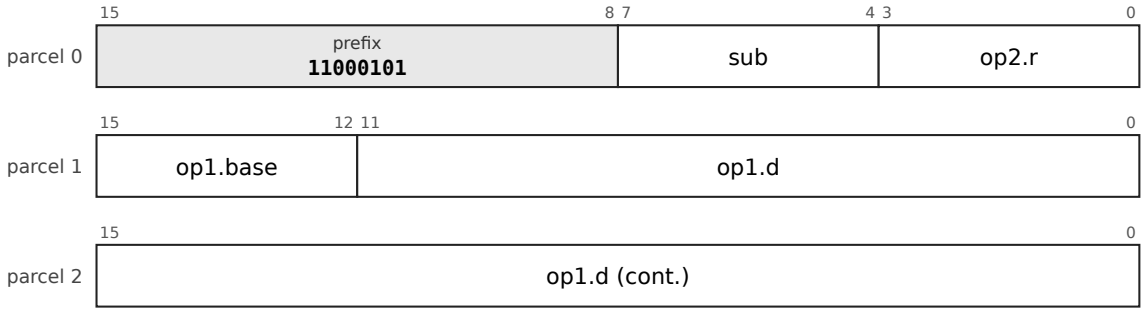
SHIFT: Shift left, shift right logical, shift right arithmetic

Mnemonics: `SHL`, `SHR`, `SAR`

Syntax

```
SHL rm, Rd
SHR rm, Rd
SAR rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-1`, LDF.B28, prefix `11000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-1`, `16-LDF-X22-1`, `16-LDF-PI12-1`, `16-LDF-PD12-1`, `16-LDF-PC32-1`, `16-LDF-A32-1`, `16-LDFI-I32-1`, `16-DPF-D28-1`, `16-RRF-1`, `16-LDI12-d-1`, `16-SHI3-a-1`, `16-SHI5-a-1`, `16-LDPC12-d-1`, `16-LDBs8-d-1`, `16-LDF-B28-2`, `16-LDF-X22-2`, `16-LDF-PI12-2`, `16-LDF-PD12-2`, `16-LDF-PC32-2`, `16-LDF-A32-2`, `16-DPF-D28-2`, `16-RRF-2`.

Operation

```
n ← rm                                ; the count: a register, an immediate, or memory
SHL: Rd ← (n ≥ 64) ? 0 : Rd << n
SHR: Rd ← (n ≥ 64) ? 0 : Rd >>u n
SAR: Rd ← (n ≥ 64) ? (Rd < 0 ? -1 : 0) : Rd >>s n
```

Flags `N` and `Z` from the result, `V` cleared, `C` the last bit shifted out (bit 63 for `SHL`, bit 0 for `SHR` and `SAR`). **The same flags are written for every count**: a count of zero gives `N` and `Z` from the unchanged operand, `V` = 0 and `C` = 0.

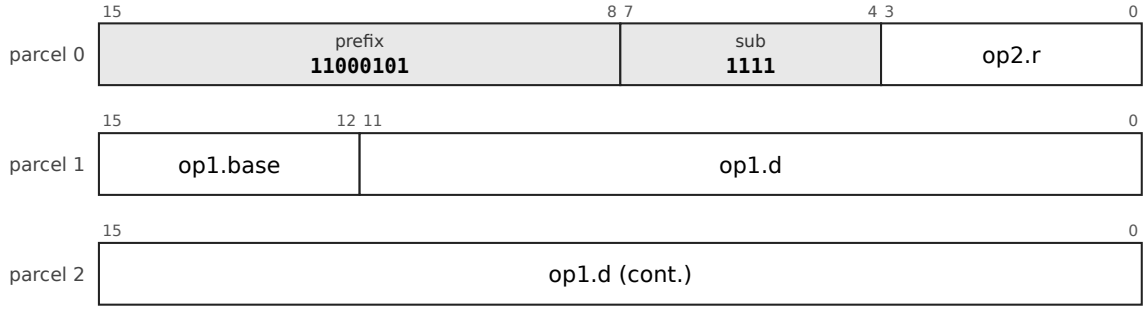
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The count is read as an unsigned 64-bit value, so a count of 64 or more shifts the value entirely away. Small immediate counts have one-parcel forms (the layouts of this entry); `ASL` and `ASR` are the by-one forms; rotates are `ROR` and `RORI`.

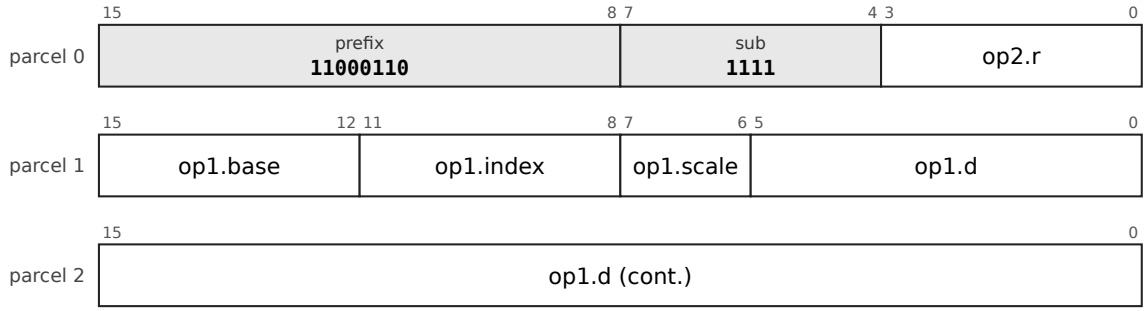
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

SHL

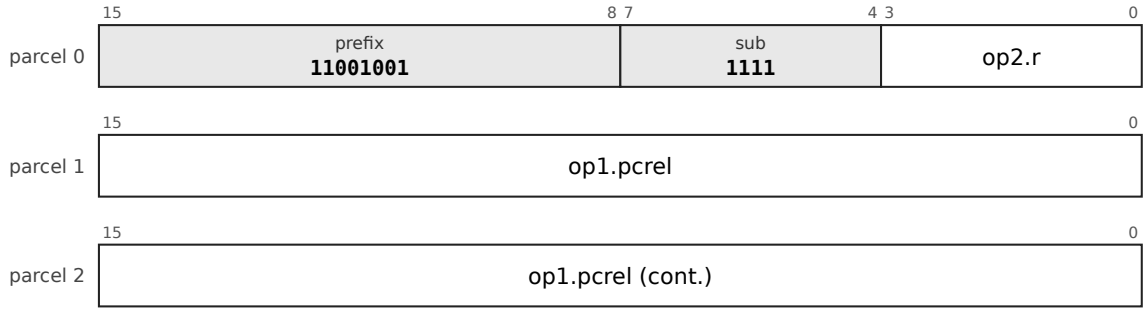
SHL: LDF.B28, prefix 11000101, sub-opcode 1111; operands b4d28s-x1, reg4; 6 bytes



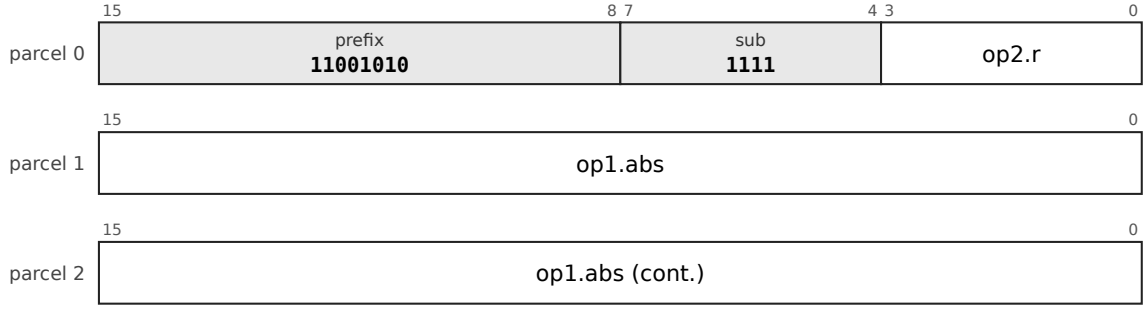
SHL: LDF.X22, prefix 11000110, sub-opcode 1111; operands x4-4d22s, reg4; 6 bytes



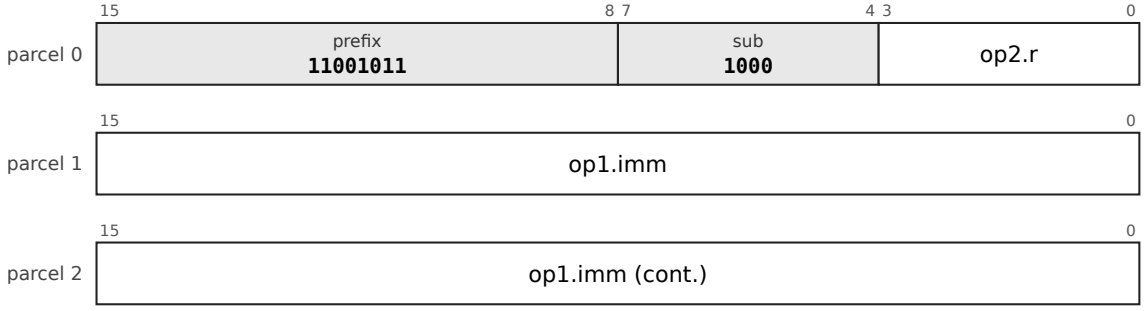
SHL: LDF.PC32, prefix 11001001, sub-opcode 1111; operands pc32-u1, reg4; 6 bytes



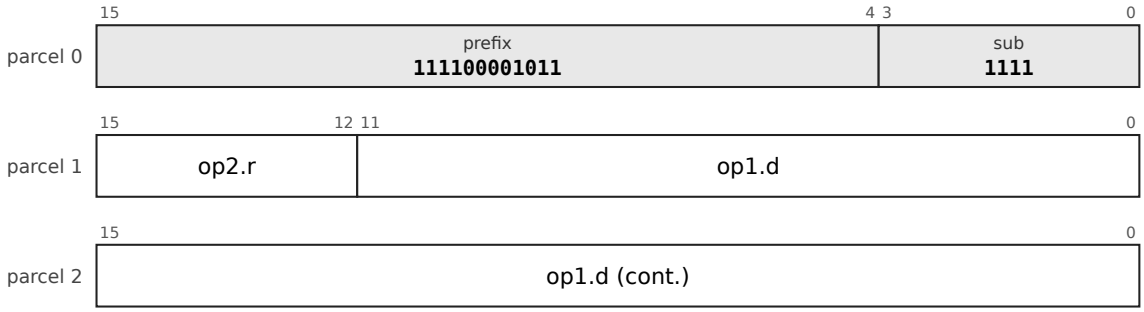
SHL: LDF.A32, prefix 11001010, sub-opcode 1111; operands abs32, reg4; 6 bytes



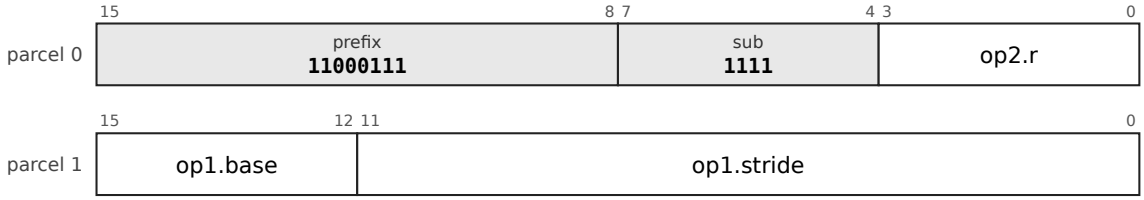
SHL: LDFI.I32, prefix 11001011, sub-opcode 1000; operands imm32s, reg4; 6 bytes



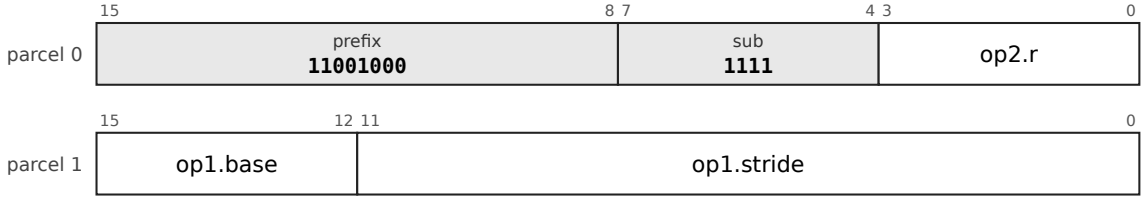
SHL: DPF.D28, prefix 111100001011, sub-opcode 1111; operands dpd28s-x1, reg4; 6 bytes



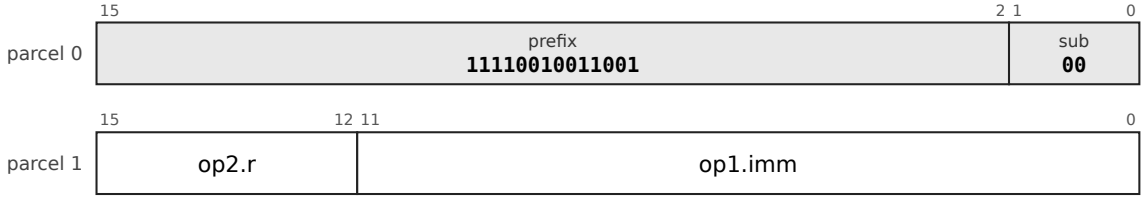
SHL: LDF.PI12, prefix 11000111, sub-opcode 1111; operands pi4-s12, reg4; 4 bytes



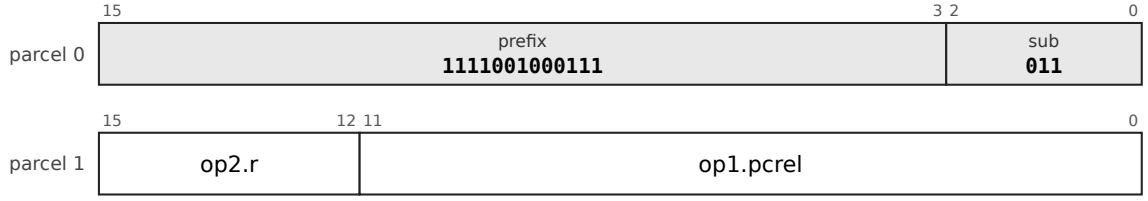
SHL: LDF.PD12, prefix 11001000, sub-opcode 1111; operands pd4-s12, reg4; 4 bytes



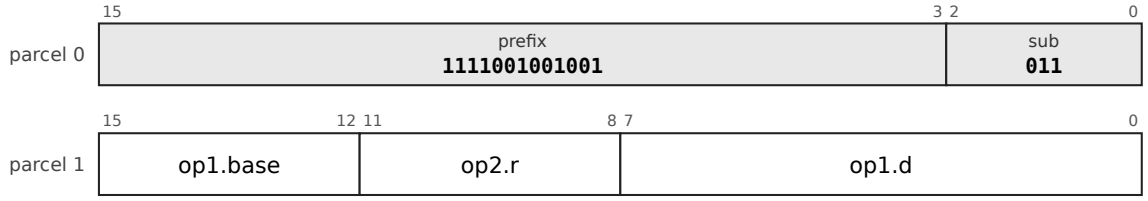
SHL: LDI12/d, prefix 11110010011001, sub-opcode 00; operands imm12s, reg4; 4 bytes



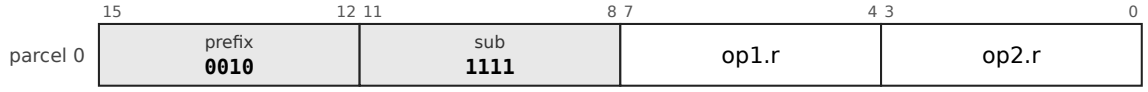
SHL: LDPC12/d, prefix 1111001000111, sub-opcode 011; operands pc12-u1, reg4; 4 bytes



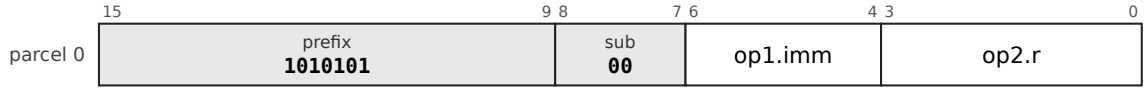
SHL: LDBs8/d, prefix 1111001001001, sub-opcode 011; operands b4d8s-xsz, reg4; 4 bytes



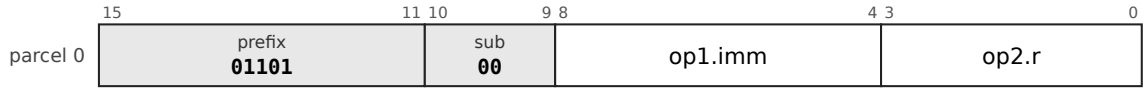
SHL: RRF, prefix 0010, sub-opcode 1111; operands reg4, reg4; 2 bytes



SHL: SHI3/a, prefix 1010101, sub-opcode 00; operands imm3u, reg4; 2 bytes



SHL: SHI5/a, prefix 01101, sub-opcode 00; operands imm5u, reg4; 2 bytes

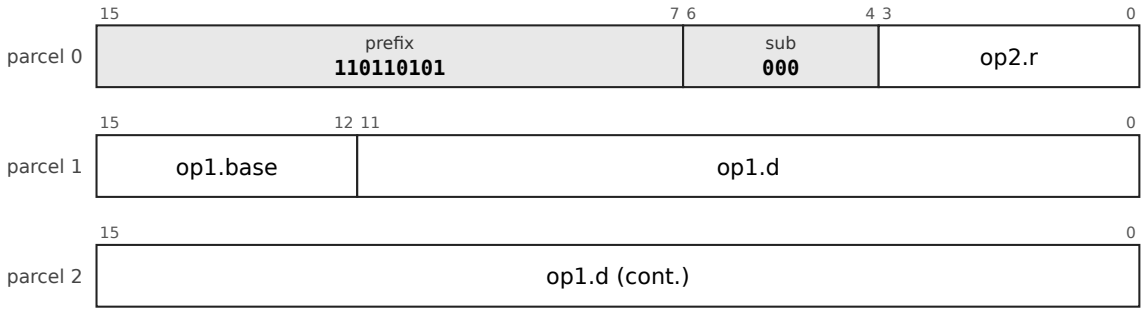



```
SHL rm, Rd
```

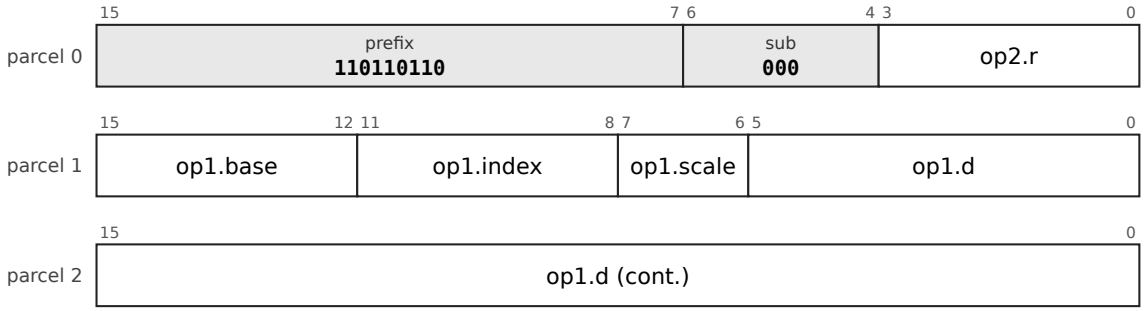
Operation, flags and exceptions as for the entry SHIFT above; 14 layout forms, each a chart of its own (above).

SHR

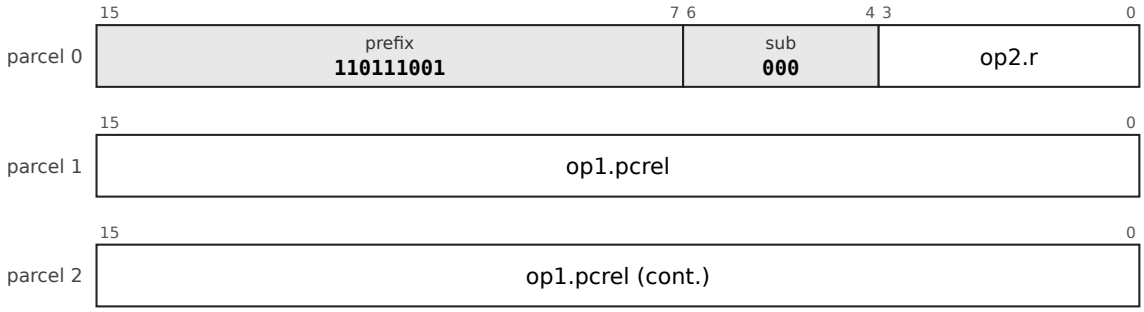
SHR: LDF.B28, prefix 110110101, sub-opcode 000; operands b4d28s-x1, reg4; 6 bytes



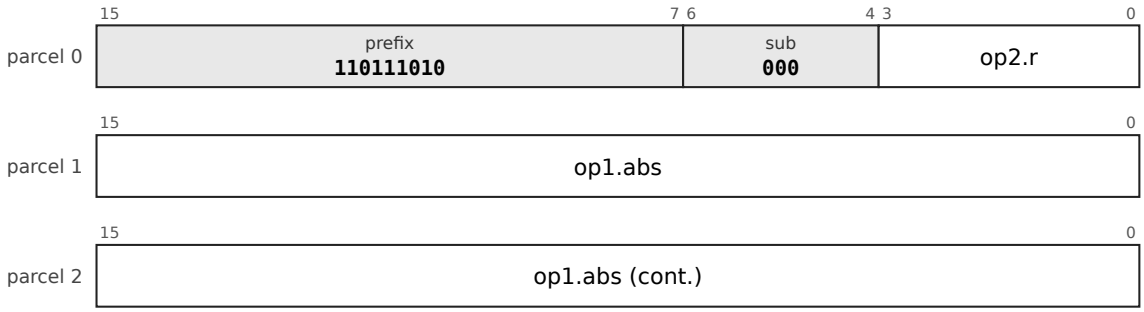
SHR: LDF.X22, prefix 110110110, sub-opcode 000; operands x4-4d22s, reg4; 6 bytes



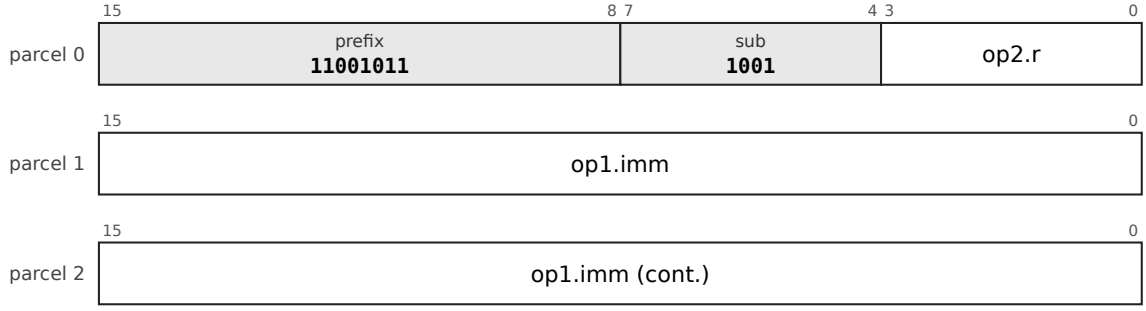
SHR: LDF.PC32, prefix 110111001, sub-opcode 000; operands pc32-u1, reg4; 6 bytes



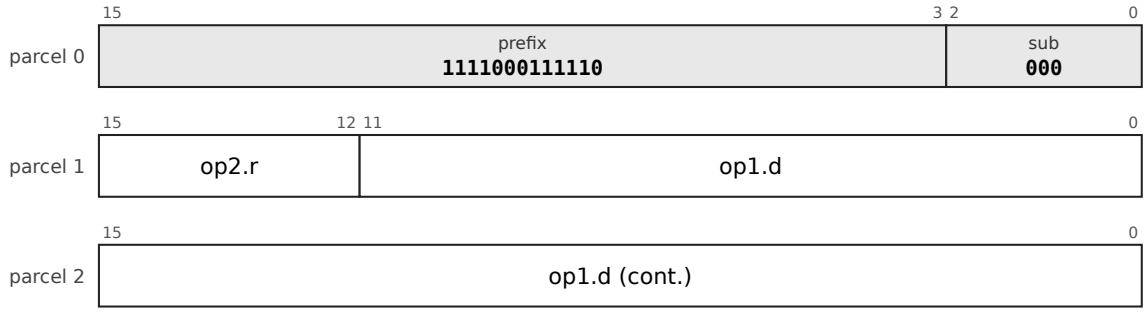
SHR: LDF.A32, prefix 110111010, sub-opcode 000; operands abs32, reg4; 6 bytes



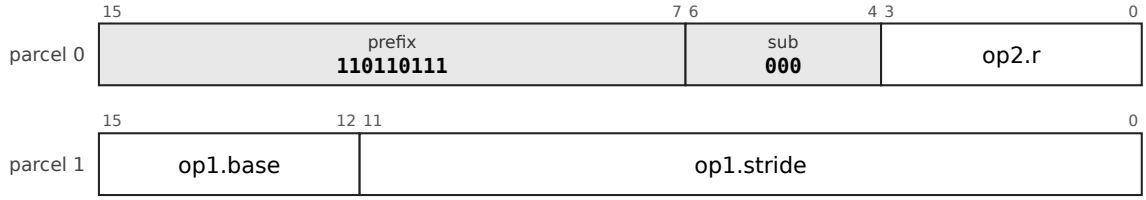
SHR: LDFI.I32, prefix 11001011, sub-opcode 1001; operands imm32s, reg4; 6 bytes



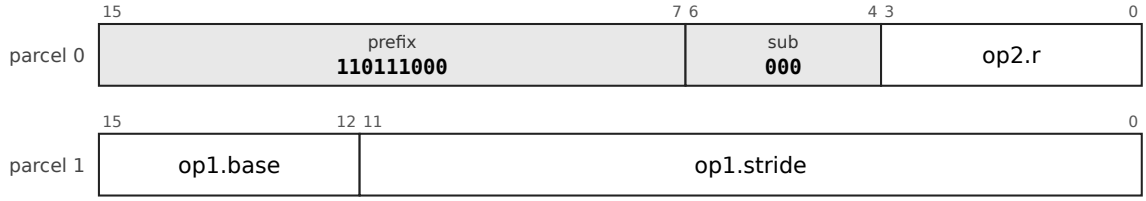
SHR: DPF.D28, prefix 1111000111110, sub-opcode 000; operands dpd28s-x1, reg4; 6 bytes



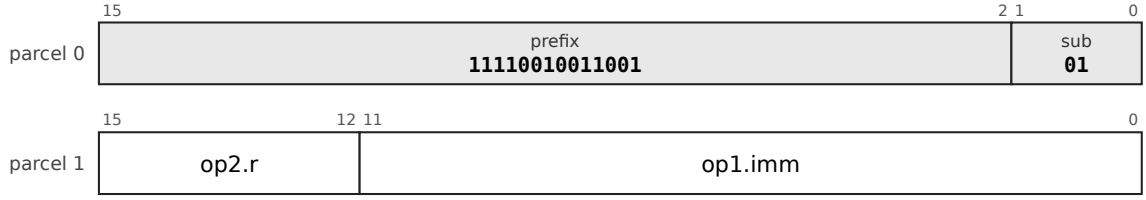
SHR: LDF.PI12, prefix 110110111, sub-opcode 000; operands pi4-s12, reg4; 4 bytes



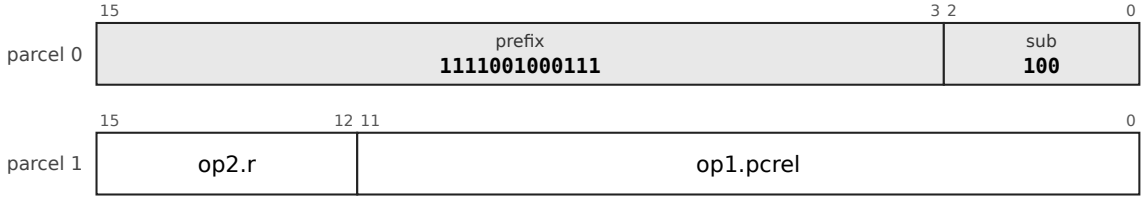
SHR: LDF.PD12, prefix 110111000, sub-opcode 000; operands pd4-s12, reg4; 4 bytes



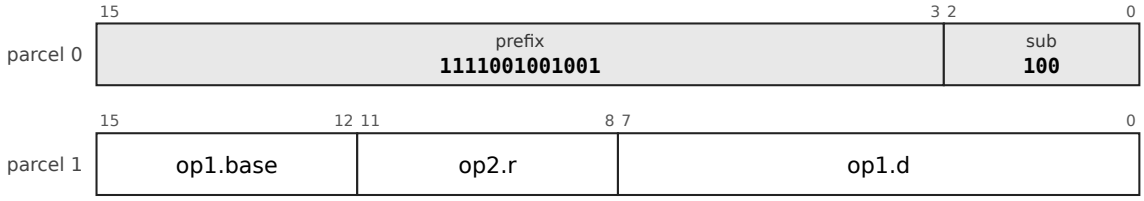
SHR: LDI12/d, prefix 11110010011001, sub-opcode 01; operands imm12s, reg4; 4 bytes



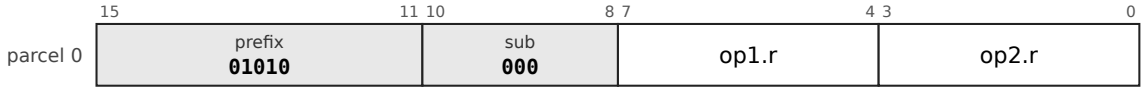
SHR: LDPC12/d, prefix 1111001000111, sub-opcode 100; operands pc12-u1, reg4; 4 bytes



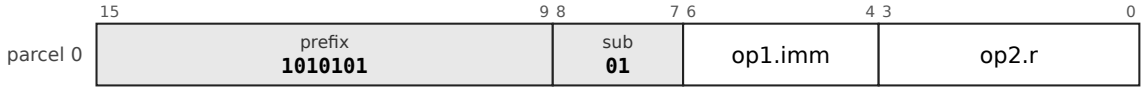
SHR: LDBs8/d, prefix 1111001001001, sub-opcode 100; operands b4d8s-xsz, reg4; 4 bytes



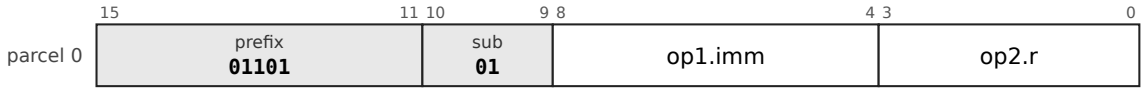
SHR: RRF, prefix 01010, sub-opcode 000; operands reg4, reg4; 2 bytes



SHR: SHI3/a, prefix 1010101, sub-opcode 01; operands imm3u, reg4; 2 bytes



SHR: SHI5/a, prefix 01101, sub-opcode 01; operands imm5u, reg4; 2 bytes

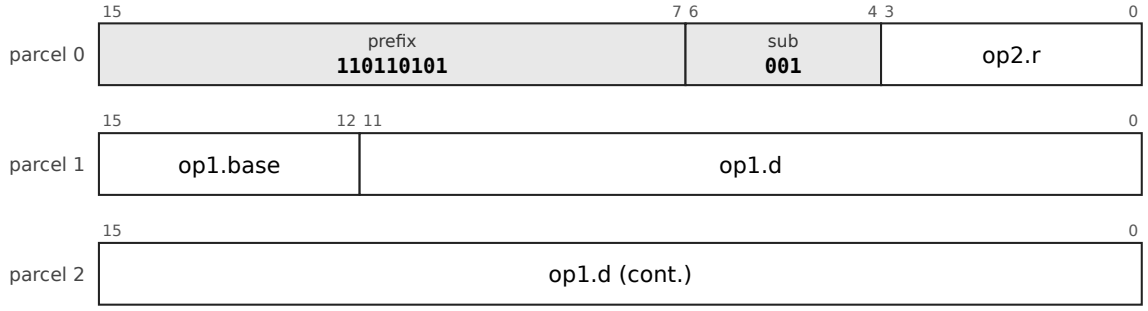



```
SHR rm, Rd
```

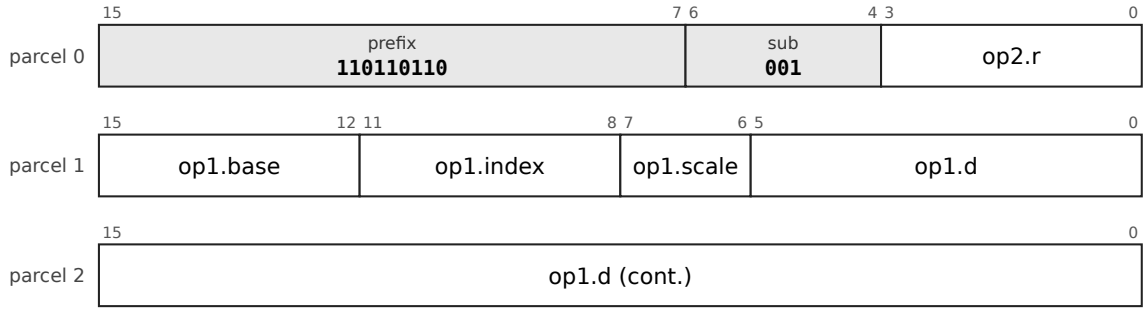
Operation, flags and exceptions as for the entry SHIFT above; 14 layout forms, each a chart of its own (above).

SAR

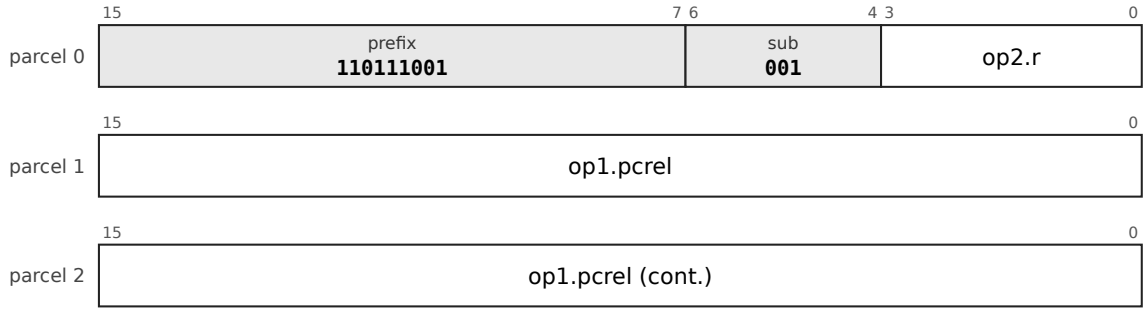
SAR: LDF.B28, prefix 110110101, sub-opcode 001; operands b4d28s-x1, reg4; 6 bytes



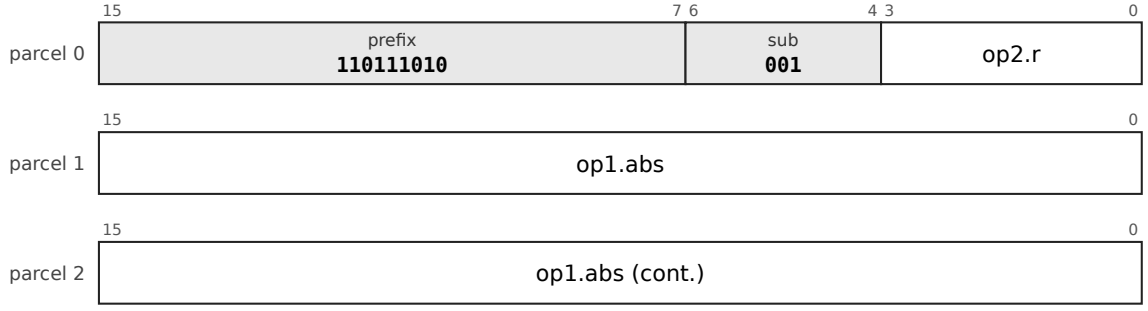
SAR: LDF.X22, prefix 110110110, sub-opcode 001; operands x4-4d22s, reg4; 6 bytes



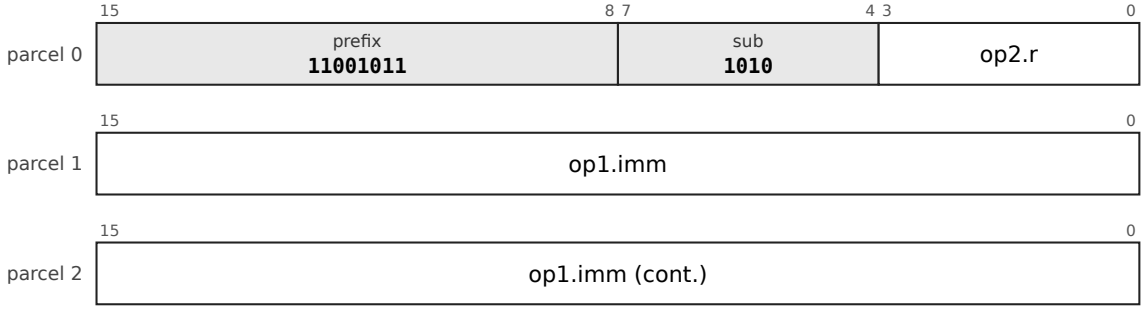
SAR: LDF.PC32, prefix 110111001, sub-opcode 001; operands pc32-u1, reg4; 6 bytes



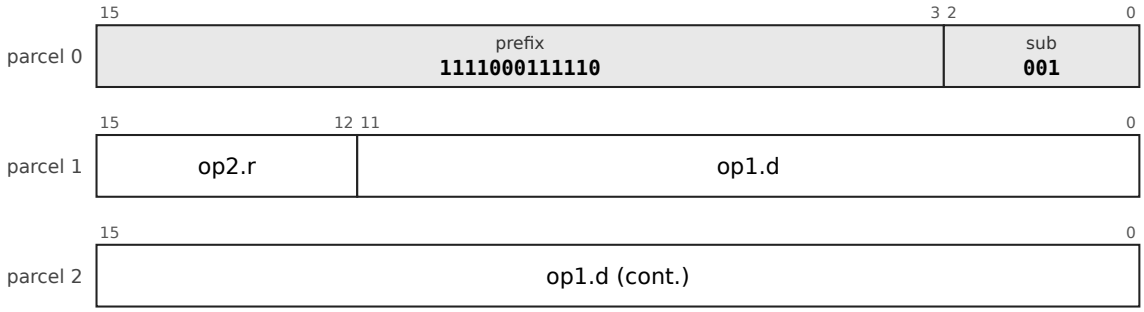
SAR: LDF.A32, prefix 110111010, sub-opcode 001; operands abs32, reg4; 6 bytes



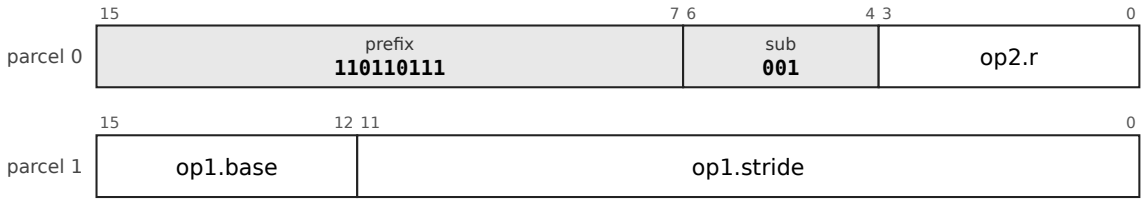
SAR: LDFI.I32, prefix 11001011, sub-opcode 1010; operands imm32s, reg4; 6 bytes



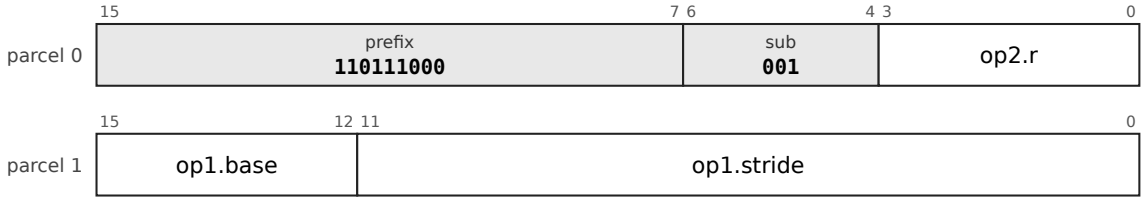
SAR: DPF.D28, prefix 1111000111110, sub-opcode 001; operands dpd28s-x1, reg4; 6 bytes



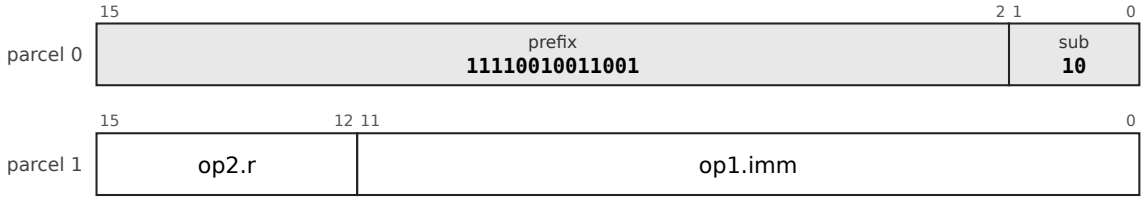
SAR: LDF.PI12, prefix 110110111, sub-opcode 001; operands pi4-s12, reg4; 4 bytes



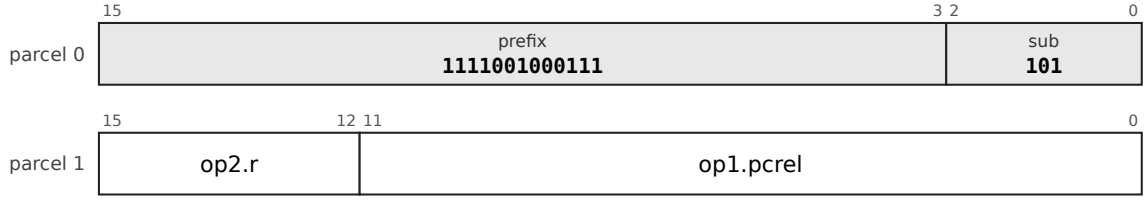
SAR: LDF.PD12, prefix 110111000, sub-opcode 001; operands pd4-s12, reg4; 4 bytes



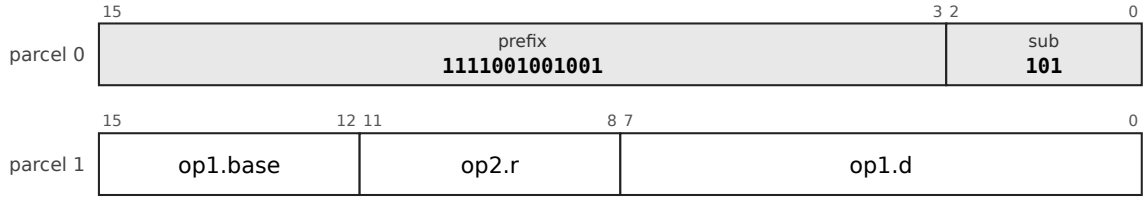
SAR: LDI12/d, prefix 11110010011001, sub-opcode 10; operands imm12s, reg4; 4 bytes



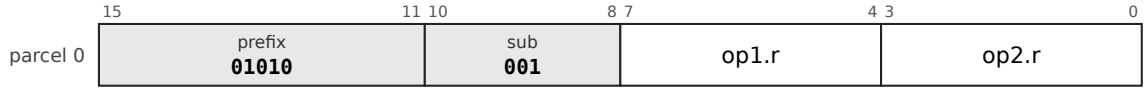
SAR: LDPC12/d, prefix 1111001000111, sub-opcode 101; operands pc12-u1, reg4; 4 bytes



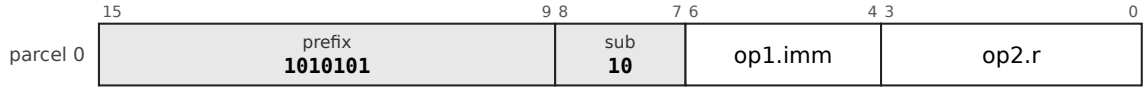
SAR: LDBs8/d, prefix 1111001001001, sub-opcode 101; operands b4d8s-xsz, reg4; 4 bytes



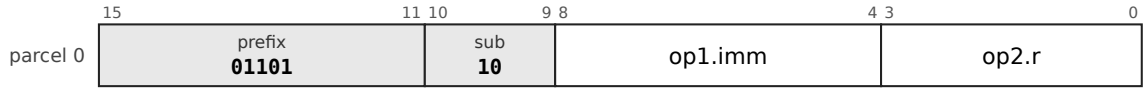
SAR: RRF, prefix 01010, sub-opcode 001; operands reg4, reg4; 2 bytes



SAR: SHI3/a, prefix 1010101, sub-opcode 10; operands imm3u, reg4; 2 bytes



SAR: SHI5/a, prefix 01101, sub-opcode 10; operands imm5u, reg4; 2 bytes

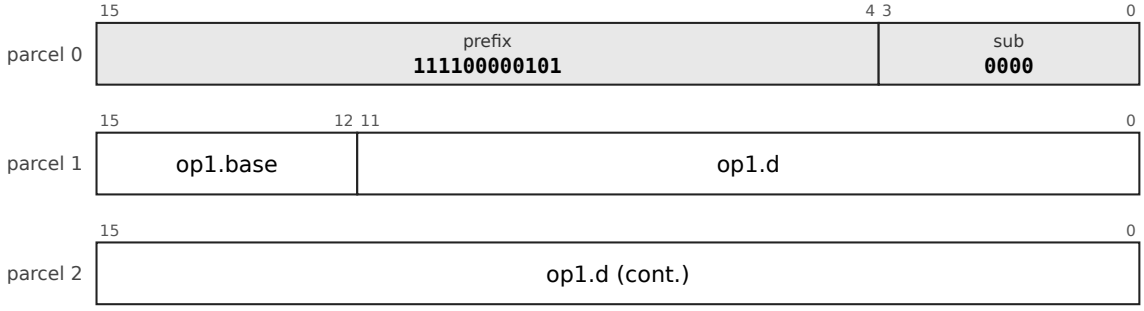



```
SAR rm, Rd
```

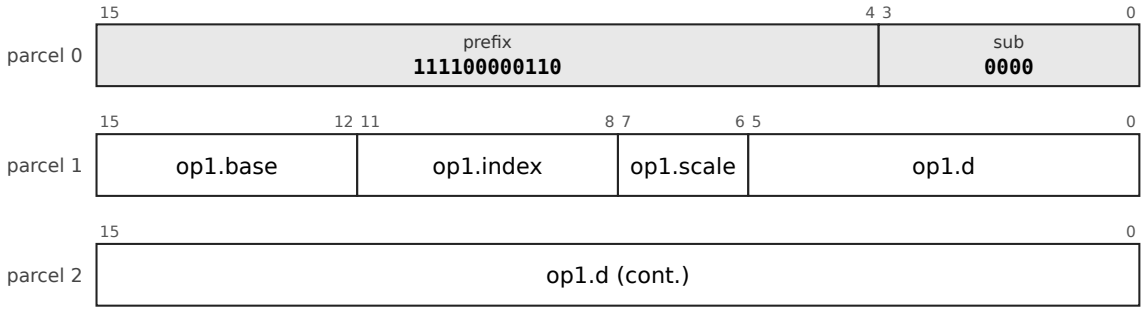
Operation, flags and exceptions as for the entry SHIFT above; 14 layout forms, each a chart of its own (above).

CLR: Clear

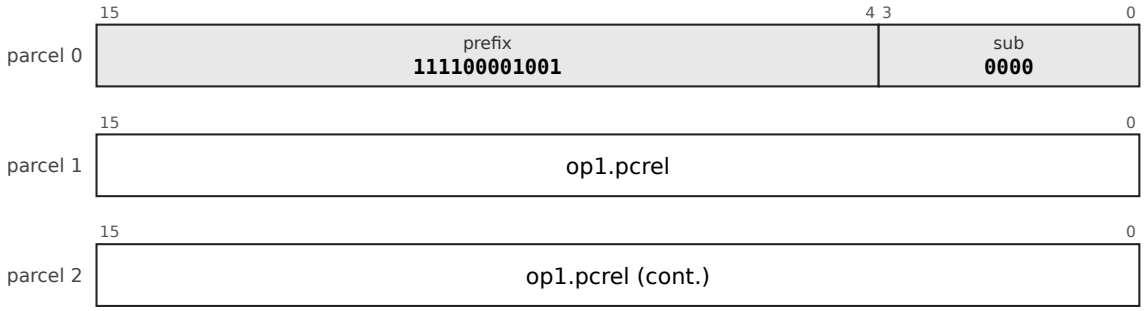
CLR: SOF.B28, prefix 111100000101, sub-opcode 0000; operands b4d28s-x1; 6 bytes



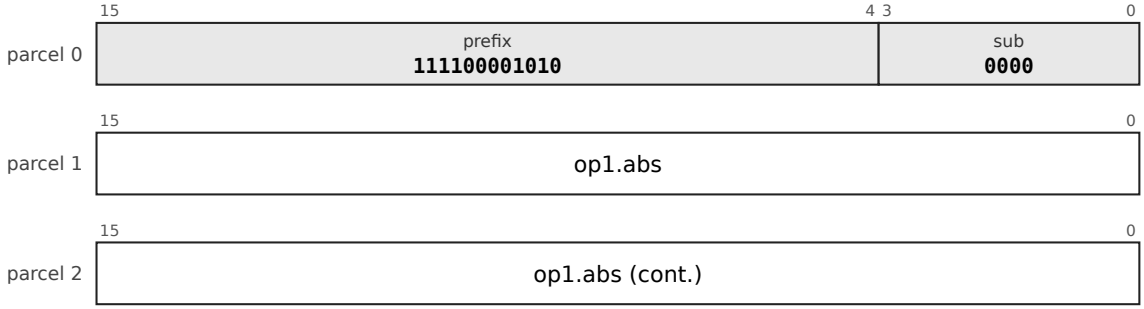
CLR: SOF.X22, prefix 111100000110, sub-opcode 0000; operands x4-4d22s; 6 bytes



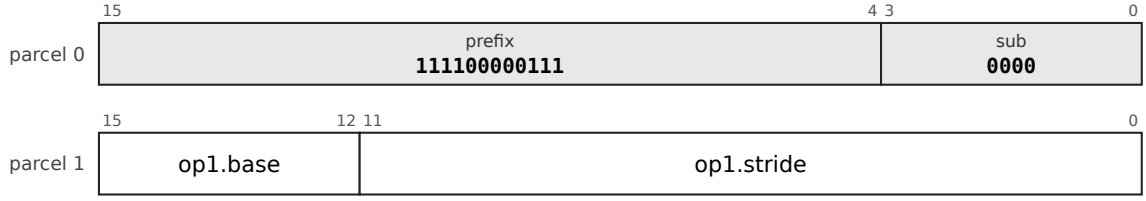
CLR: SOF.PC32, prefix 111100001001, sub-opcode 0000; operands pc32-u1; 6 bytes



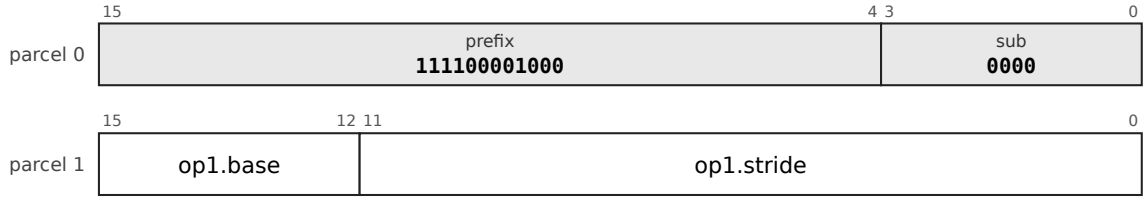
CLR: SOF.A32, prefix 111100001010, sub-opcode 0000; operands abs32; 6 bytes



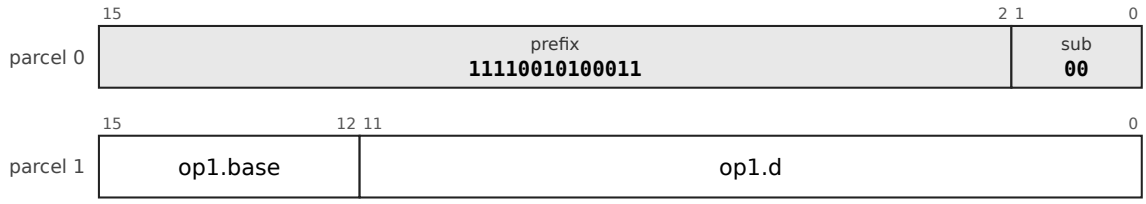
CLR: SOF.PI12, prefix 111100000111, sub-opcode 0000; operands pi4-s12; 4 bytes



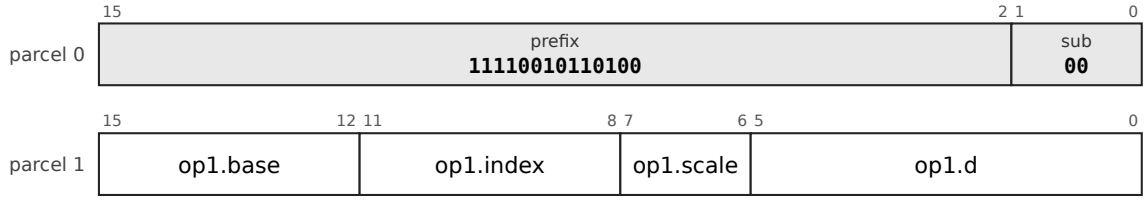
CLR: SOF.PD12, prefix 111100001000, sub-opcode 0000; operands pd4-s12; 4 bytes



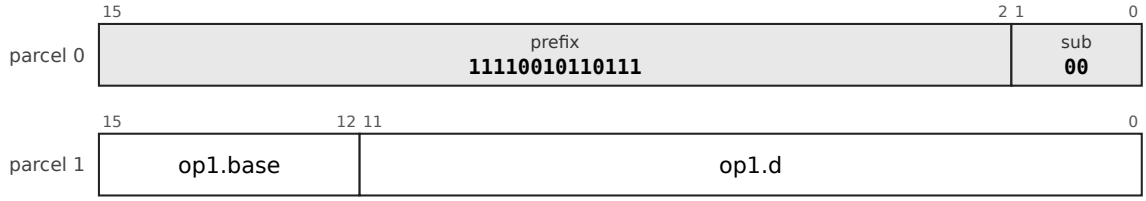
CLR: SOBu12/a, prefix 11110010100011, sub-opcode 00; operands b4d12u-xsz; 4 bytes



CLR: SOX6/a, prefix 11110010110100, sub-opcode 00; operands x4-4d6s; 4 bytes



CLR: SOBs12/a, prefix 11110010110111, sub-opcode 00; operands b4d12s-xsz; 4 bytes



CLR: SOPR, prefix 111000001, sub-opcode 000; operands reg4; 2 bytes

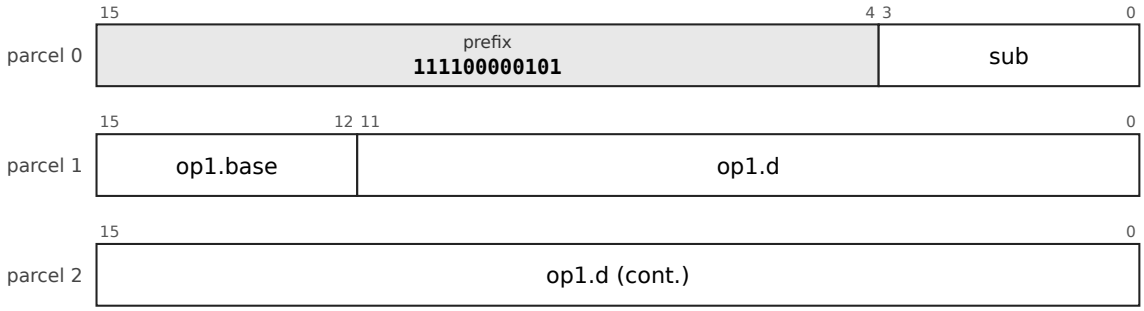


Mnemonics: CLR

Syntax

```
CLR rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`, `16-SOBu12-a-1`, `16-SOX6-a-1`, `16-SOBs12-a-1`.

Operation

```
rm ← 0
```

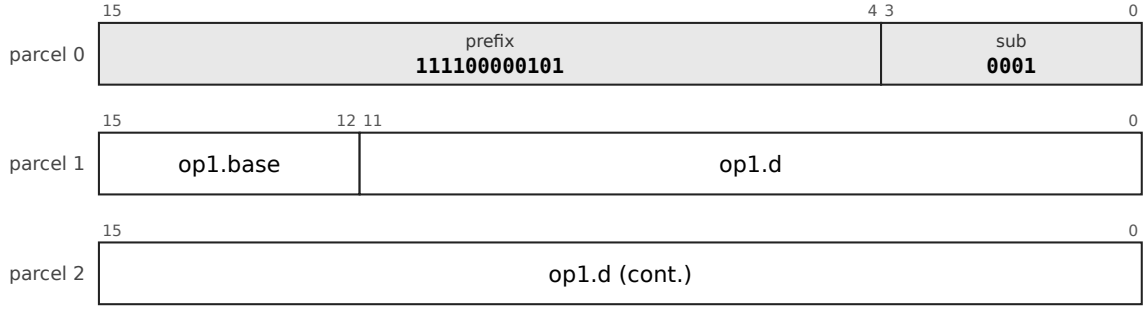
Flags `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

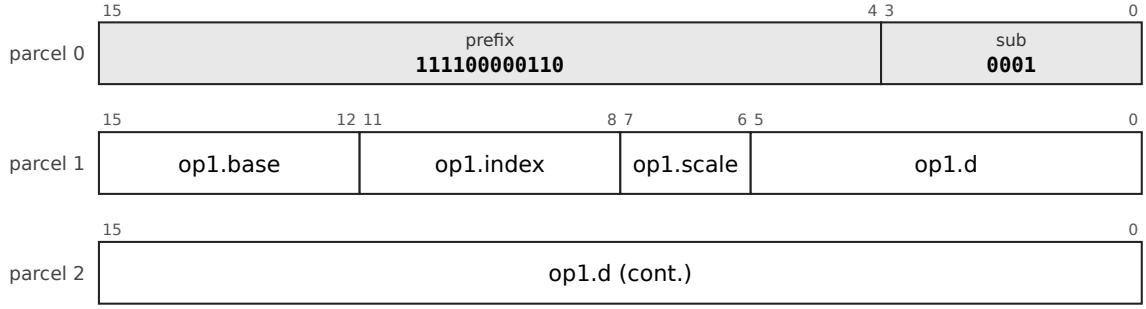
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

COM: Complement

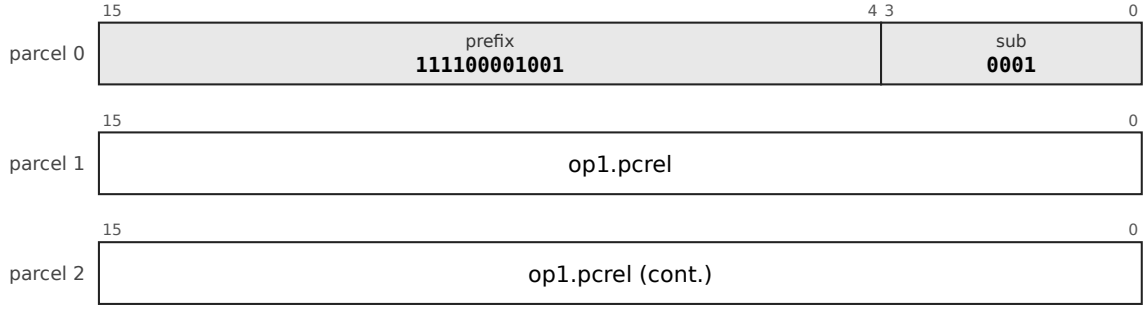
COM: SOF.B28, prefix 111100000101, sub-opcode 0001; operands b4d28s-x1; 6 bytes



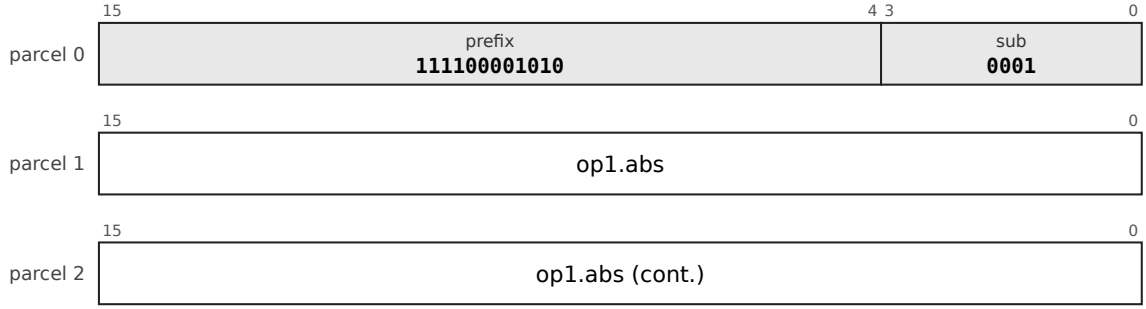
COM: SOF.X22, prefix 111100000110, sub-opcode 0001; operands x4-4d22s; 6 bytes



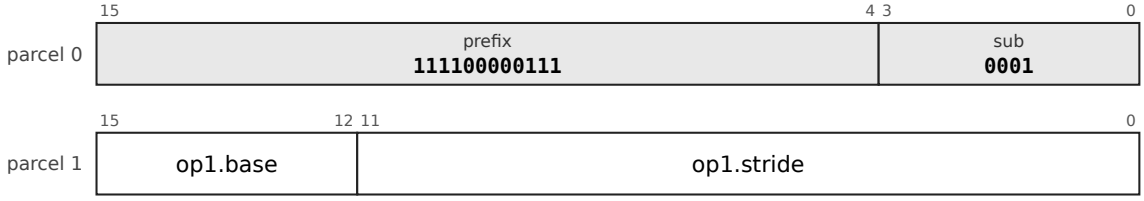
COM: SOF.PC32, prefix 111100001001, sub-opcode 0001; operands pc32-u1; 6 bytes



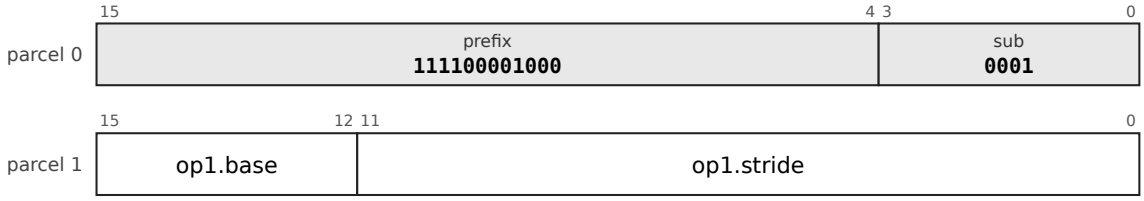
COM: SOF.A32, prefix 111100001010, sub-opcode 0001; operands abs32; 6 bytes



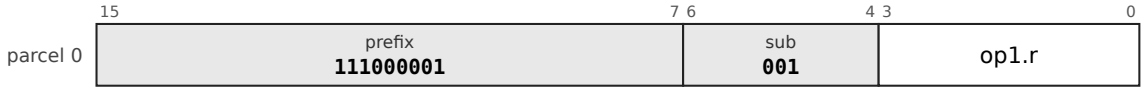
COM: SOF.PI12, prefix 111100000111, sub-opcode 0001; operands pi4-s12; 4 bytes



COM: SOF.PD12, prefix 111100001000, sub-opcode 0001; operands pd4-s12; 4 bytes



COM: SOPR, prefix 111000001, sub-opcode 001; operands reg4; 2 bytes

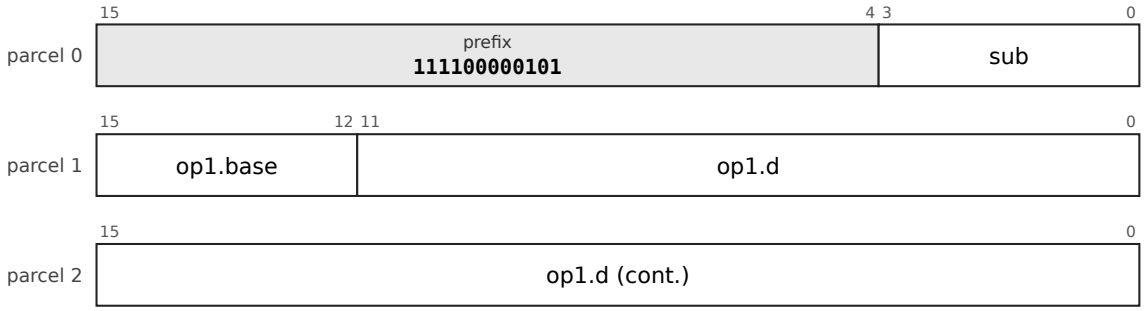


Mnemonics: `COM`

Syntax

```
COM rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`.

Operation

```
rm ← NOT rm
```

Flags `N` and `Z` from the result, `V` = 0, `C` = 1 (the PDP-11 rule).

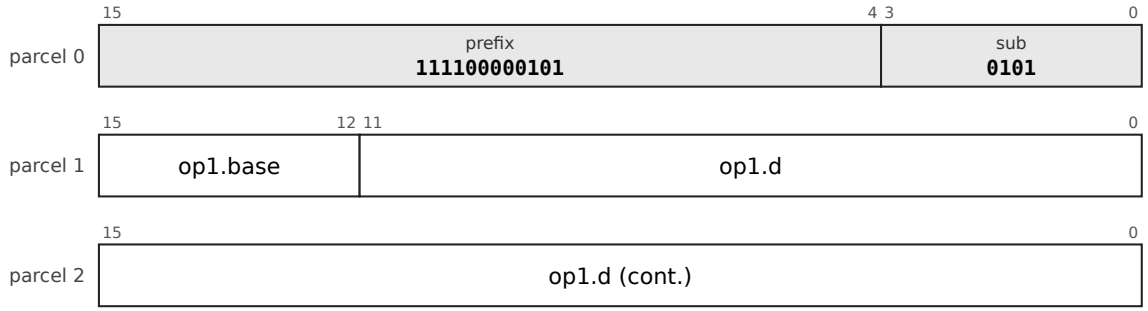
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A two-byte instruction for a register operand.

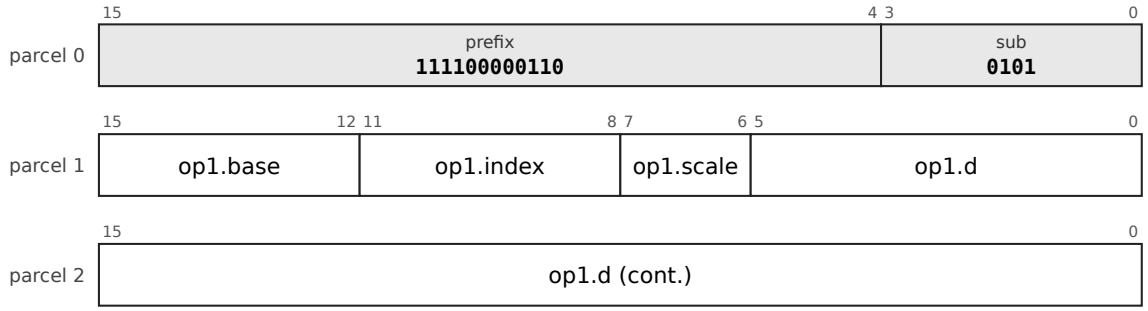
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

TST: Test

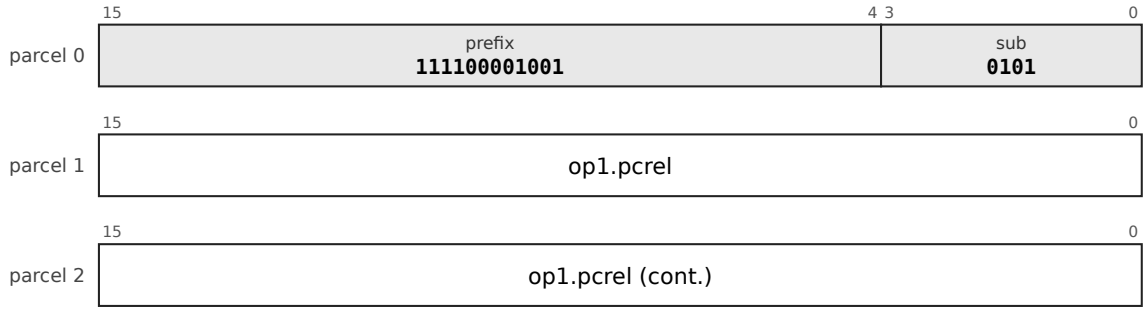
TST: SOF.B28, prefix 111100000101, sub-opcode 0101; operands b4d28s-x1; 6 bytes



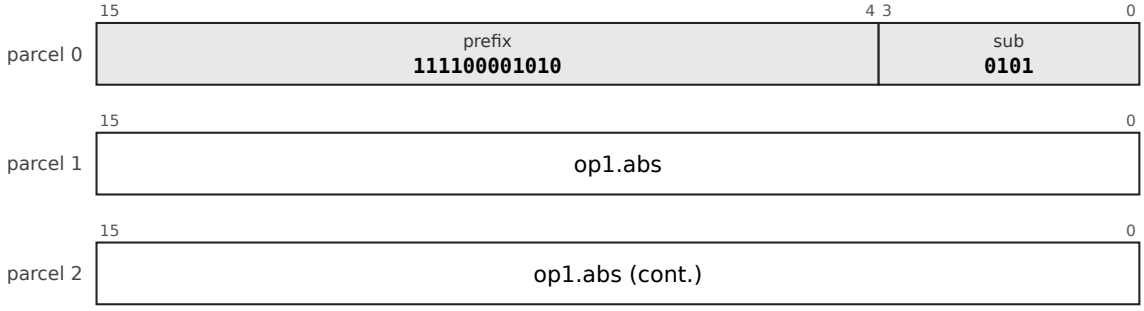
TST: SOF.X22, prefix 111100000110, sub-opcode 0101; operands x4-4d22s; 6 bytes



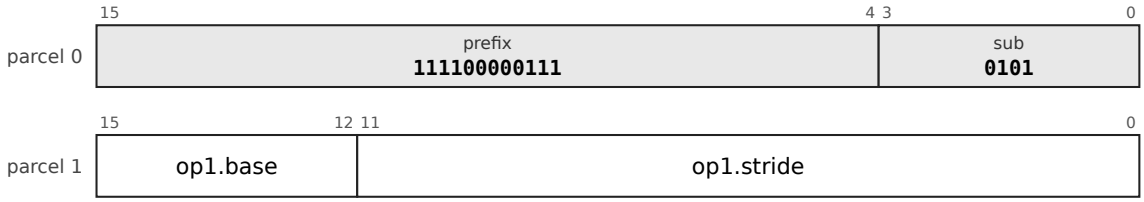
TST: SOF.PC32, prefix 111100001001, sub-opcode 0101; operands pc32-u1; 6 bytes



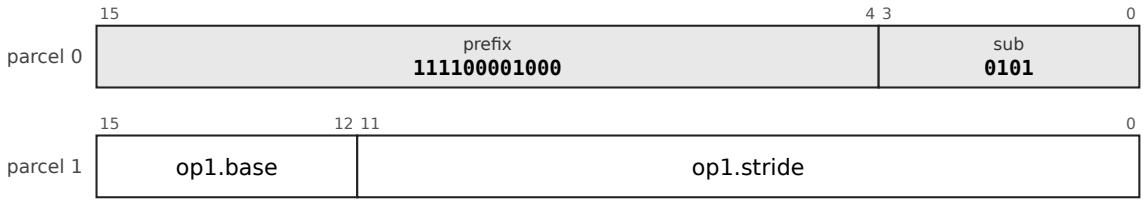
TST: SOF.A32, prefix 111100001010, sub-opcode 0101; operands abs32; 6 bytes



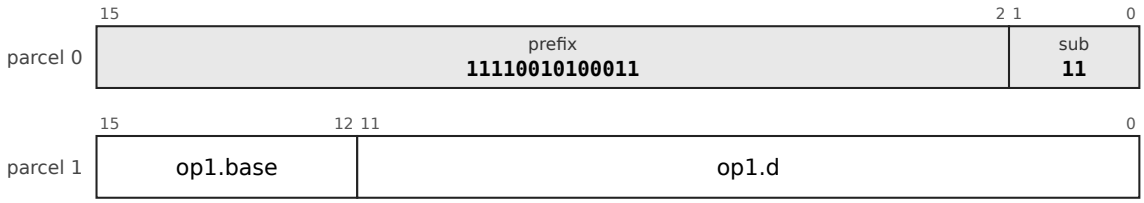
TST: SOF.PI12, prefix 111100000111, sub-opcode 0101; operands pi4-s12; 4 bytes



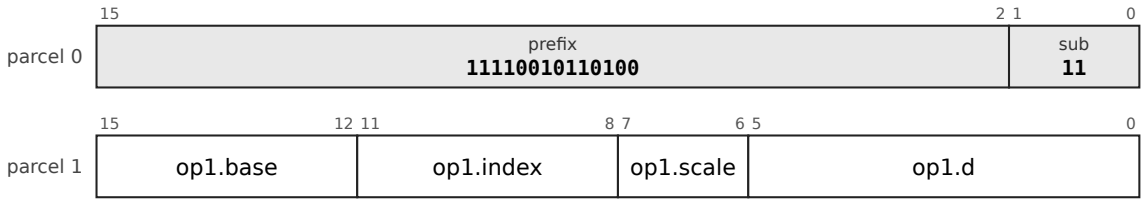
TST: SOF.PD12, prefix 111100001000, sub-opcode 0101; operands pd4-s12; 4 bytes



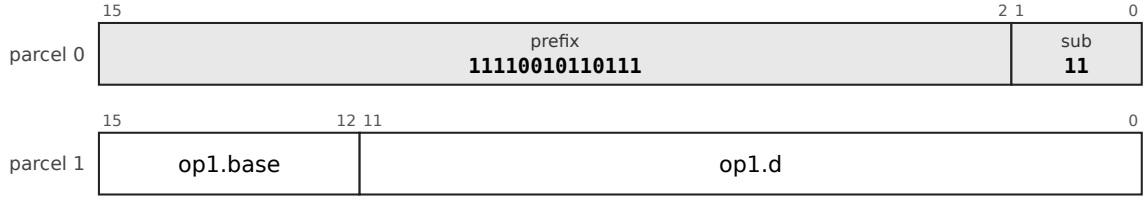
TST: SOBu12/a, prefix 11110010100011, sub-opcode 11; operands b4d12u-xsz; 4 bytes



TST: SOX6/a, prefix 11110010110100, sub-opcode 11; operands x4-4d6s; 4 bytes



TST: SOBs12/a, prefix 11110010110111, sub-opcode 11; operands b4d12s-xsz; 4 bytes



TST: SOPR, prefix 111000001, sub-opcode 101; operands reg4; 2 bytes

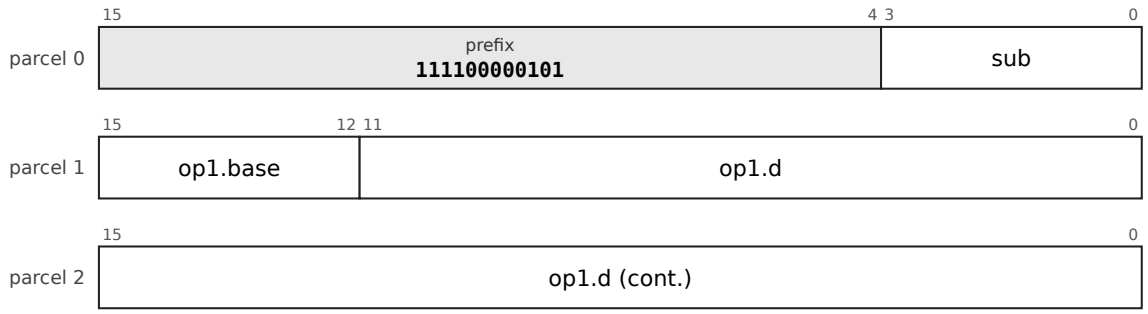


Mnemonics: `TST`

Syntax

```
TST rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-1`, `16-SOBu12-a-1`, `16-SOX6-a-1`, `16-SOBs12-a-1`.

Operation

```
set flags from rm; nothing is written
```

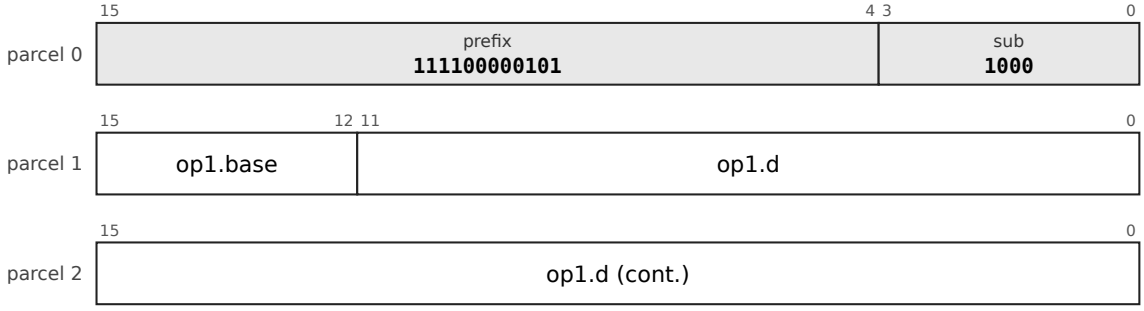
Flags `N` and `Z` from the operand, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

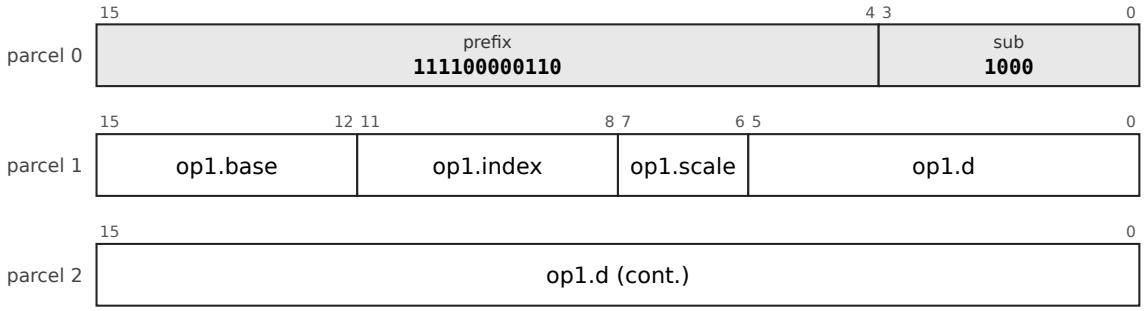
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ASL: Arithmetic shift left by one

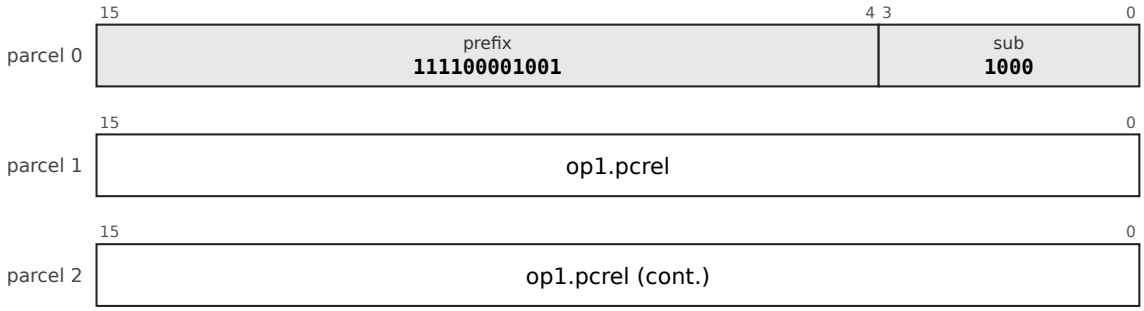
ASL: SOF.B28, prefix 111100000101, sub-opcode 1000; operands b4d28s-x1; 6 bytes



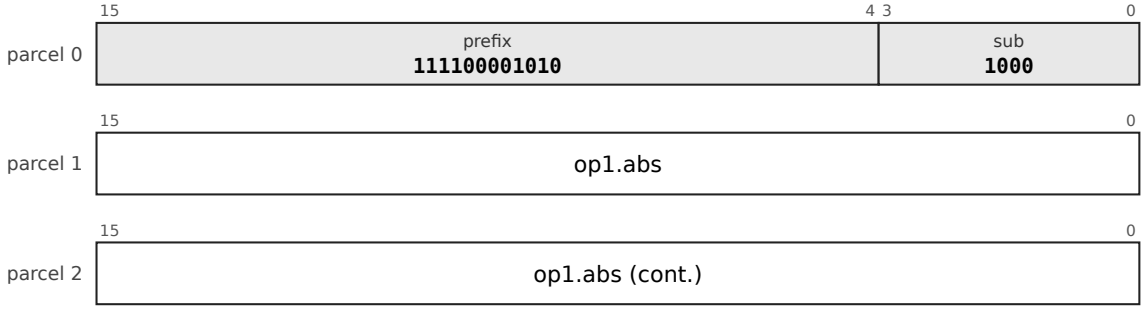
ASL: SOF.X22, prefix 111100000110, sub-opcode 1000; operands x4-4d22s; 6 bytes



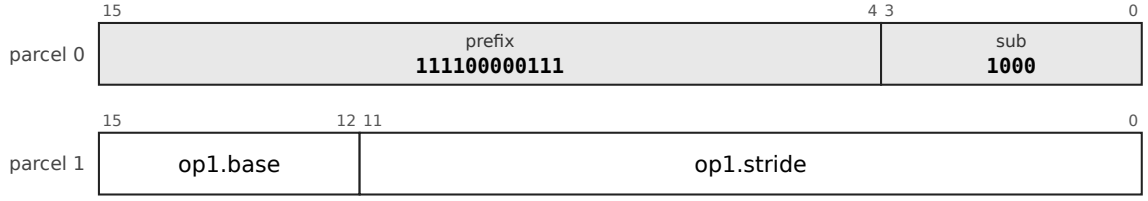
ASL: SOF.PC32, prefix 111100001001, sub-opcode 1000; operands pc32-u1; 6 bytes



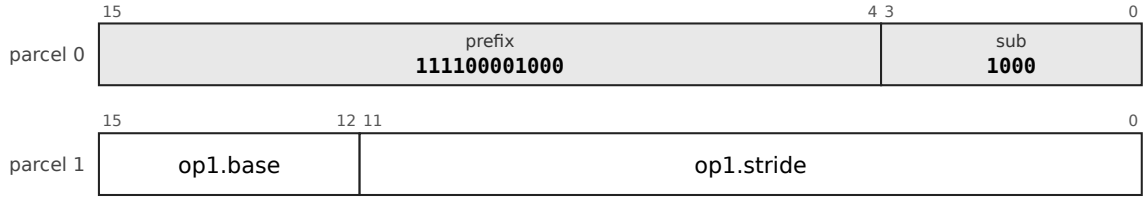
ASL: SOF.A32, prefix 111100001010, sub-opcode 1000; operands abs32; 6 bytes



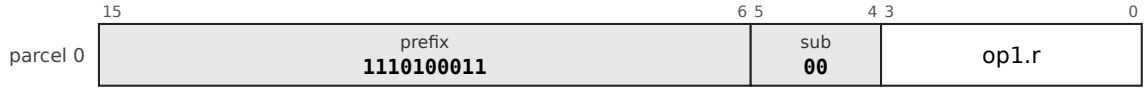
ASL: SOF.PI12, prefix 111100000111, sub-opcode 1000; operands pi4-s12; 4 bytes



ASL: SOF.PD12, prefix 111100001000, sub-opcode 1000; operands pd4-s12; 4 bytes



ASL: SOPR, prefix 1110100011, sub-opcode 00; operands reg4; 2 bytes

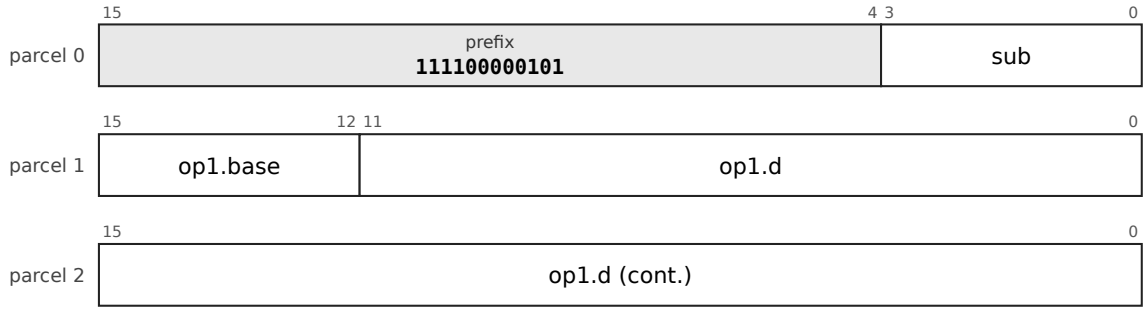


Mnemonics: `ASL`

Syntax

```
ASL rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-2`.

Operation

```
C ← rm[63] ; rm ← rm << 1
```

Flags `N` and `Z` from the result, `C` the bit shifted out of bit 63, `V` = `N` xor `C` (set if the sign changed: the PDP-11 rule).

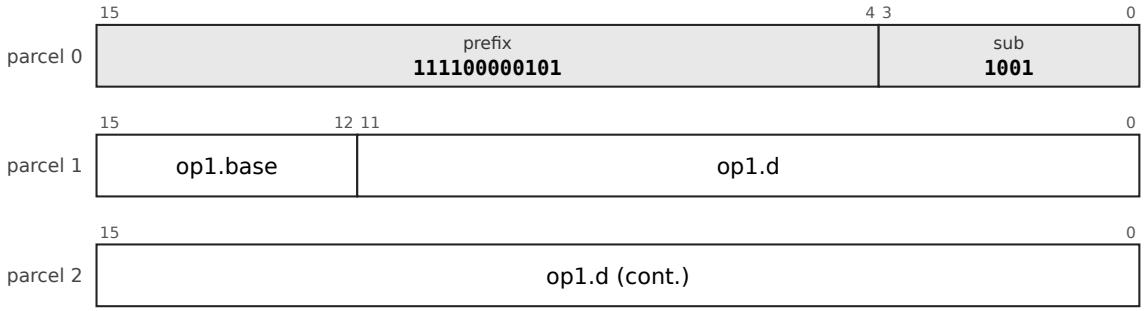
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The by-one form of SHL.

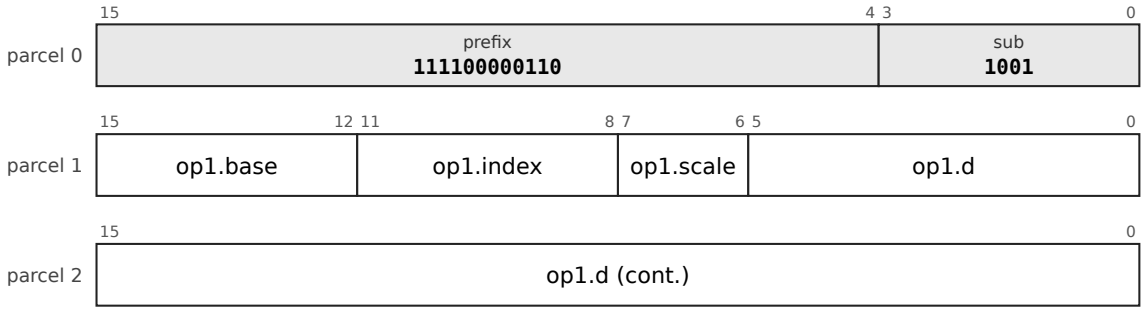
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ASR: Arithmetic shift right by one

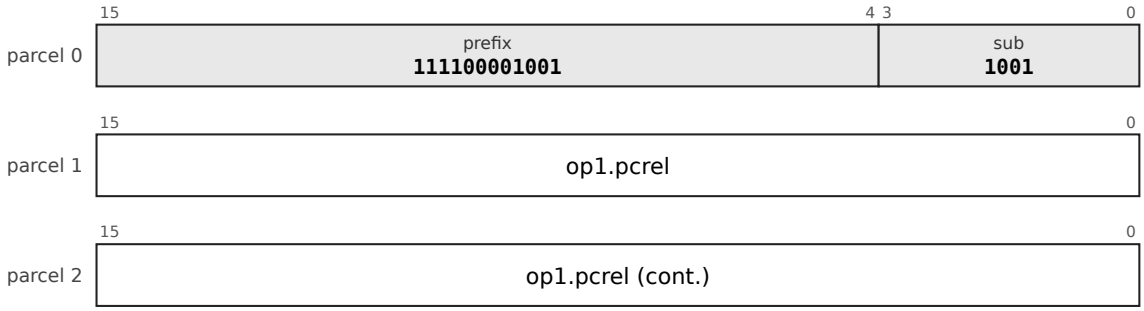
ASR: SOF.B28, prefix 111100000101, sub-opcode 1001; operands b4d28s-x1; 6 bytes



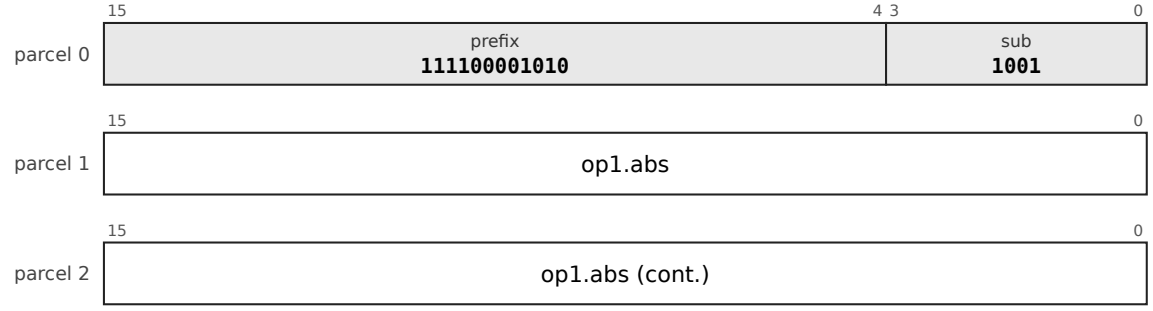
ASR: SOF.X22, prefix 111100000110, sub-opcode 1001; operands x4-4d22s; 6 bytes



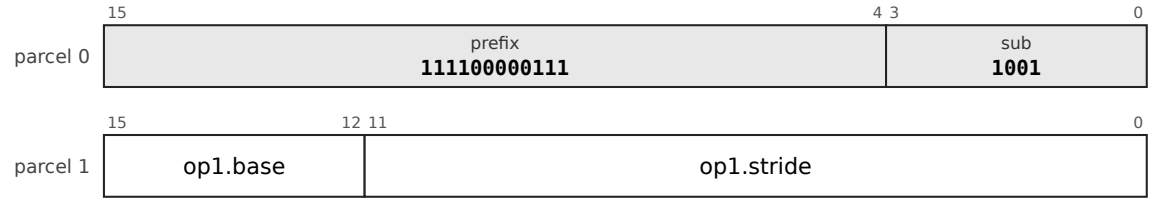
ASR: SOF.PC32, prefix 111100001001, sub-opcode 1001; operands pc32-u1; 6 bytes



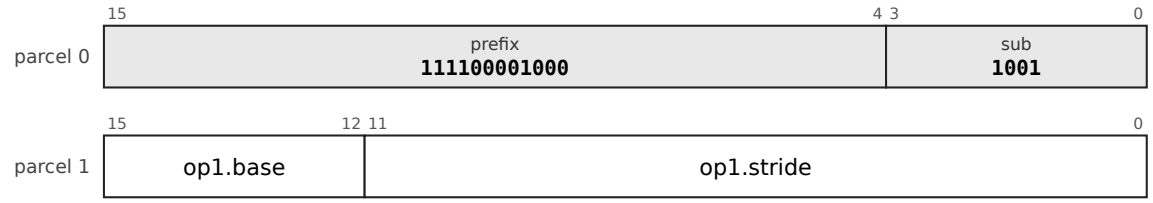
ASR: SOF.A32, prefix 111100001010, sub-opcode 1001; operands abs32; 6 bytes



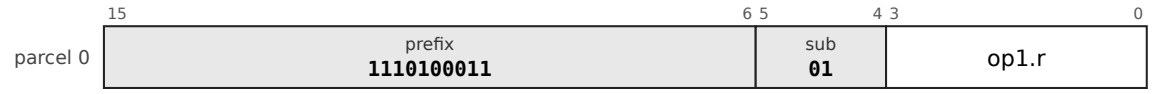
ASR: SOF.PI12, prefix 111100000111, sub-opcode 1001; operands pi4-s12; 4 bytes



ASR: SOF.PD12, prefix 111100001000, sub-opcode 1001; operands pd4-s12; 4 bytes



ASR: SOPR, prefix 1110100011, sub-opcode 01; operands reg4; 2 bytes

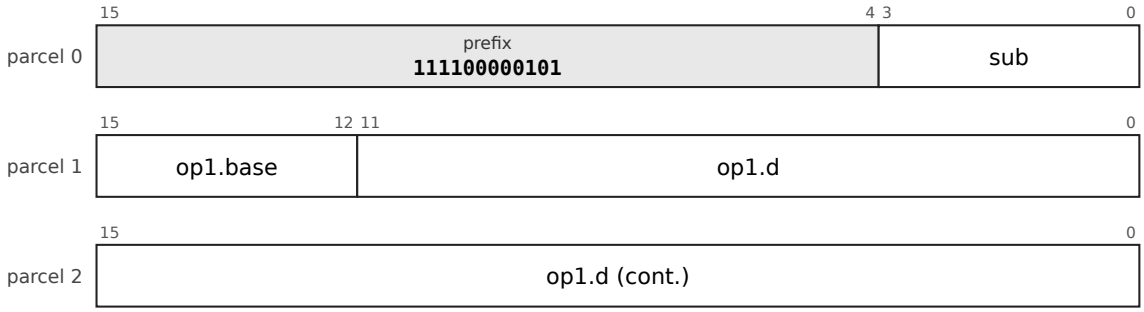


Mnemonics: ASR

Syntax

```
ASR rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-2`.

Operation

```
C ← rm[0] ; rm ← rm >>s 1
```

Flags `N` and `Z` from the result, `C` the bit shifted out of bit 0, `V` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The by-one form of `SAR`.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

9.9 Bits, bytes and text

9.9.1 The bit-and-field escape provides `CLZ`, `CTZ`, `CLO`, `CTO` (count leading and trailing zeros and ones), `POPCNT`, `BSWAP`, `BSWAPW`, `BSWAPL` and `BITREV`.

9.9.2 `EXTB` and `EXTZB` extract a bitfield at a position and width, sign- or zero-extended, and `INSB` inserts one; the position and the width are 6-bit fields (a width of 64 cannot be written), so these are six bytes. *(D)*

9.9.3 `XNORPOP` computes the population count of the exclusive-nor of two registers. *(D)*

9.9.4 `CLMULL` and `CLMULH` are the low and high halves of a carry-less multiply; no polynomial is built into the architecture. *(D)*

9.9.5 `UTF8LEN`, `UTF8DEC` and `UTF8ENC` find the length of, decode and encode a UTF-8 sequence, and reject overlong forms, surrogates, out-of-range values and bad continuation bytes. *(D)*

Commentary. **Precedent.** The PDP-11 had `SWAB`; the VAX had bit-field instructions and a table-driven `CRC`; x86 added `POPCNT`, `LZCNT` and `PCLMULQDQ` late, and committed `CRC32` to one polynomial. **Trade-offs.** A bit unit is one shared structure: `POPCNT`, `XNORPOP` and a garbage collector's mark-bitmap scan all want the same 64-input adder tree, and `CLZ` and its relatives want a priority encoder over the same operand. **Improvements.** A carry-less multiply stands in for a `CRC` instruction: no polynomial is baked in, and the same instruction serves CRC, the Galois-counter-mode authentication used in TLS, and hashing. Every UTF-8 security hole of the past is three comparators, which an instruction cannot omit and software often does. **Benefits.** The bit unit and the UTF-8 decoder together cost less than 5% of the die. **Modern comparables.** RISC-V's bit-manipulation and carry-less-multiply extensions, AArch64's `CLZ`, `RBIT`, `REV` and `PMULL`, and x86's `PCLMULQDQ`. **Use.** `POPCNT` for bitsets and Hamming distance; `XNORPOP` for binary-weight neural networks and bitset intersection; `CLMUL` for CRC, GCM and fingerprinting; `UTF8*` for text validation in parsers and kernels. **Impact.** *Kernel:* `CTZ` and `CLZ` find the first set bit in a run-queue or free-page bitmap; `POPCNT` counts bits in bitmaps; `BSWAP` converts network byte order. *Compiler:* The compiler maps `__builtin_clz`, `__builtin_popcount`

and `__builtin_bswap` directly, and a carry-less multiply is the primitive for CRC and hashing code. *Hardware:* One bit unit holds a shared 64-input adder tree and a priority encoder, and the UTF-8 comparators are about 300 lookup tables.

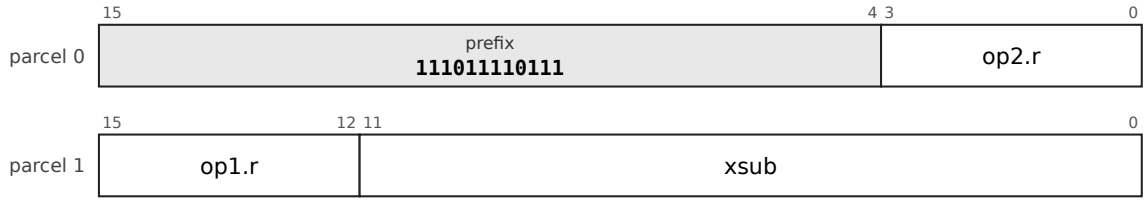
COUNT: Count leading and trailing zeros and ones

Mnemonics: `CLZ`, `CTZ`, `CLO`, `CTO`

Syntax

```
CLZ Rs, Rd
CTZ Rs, Rd
CLO Rs, Rd
CTO Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
CLZ: Rd ← number of leading zero bits of Rs        (64 if Rs = 0)
CTZ: Rd ← number of trailing zero bits of Rs       (64 if Rs = 0)
CLO: Rd ← number of leading one bits of Rs         (64 if Rs = all ones)
CTO: Rd ← number of trailing one bits of Rs        (64 if Rs = all ones)
```

Flags None written.

Exceptions None.

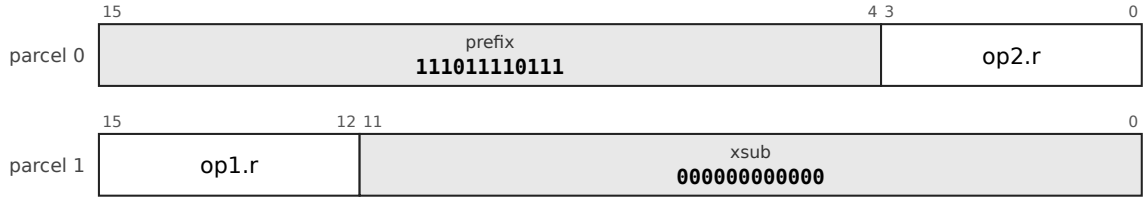
`CTZ` is the VAX `FFS`. Bit manipulation is core on a machine that does filesystem and bignum work, not an extension.

Why it is here. Bit manipulation is not an extension on a machine that does filesystem work, bignum arithmetic and the like. The counts are the VAX `FFS` and `FFC` idiom (find first set, find first clear), and LoongArch, which has `CLO`, `CLZ`, `CTO` and `CTZ` in its base, is the model for putting them there. The compiler emits none of them, since C has no operator for them; the corpus shows zero uses.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

CLZ

CLZ: R2, prefix 111011110111, sub-opcode 000000000000; operands reg4, reg4; 4 bytes

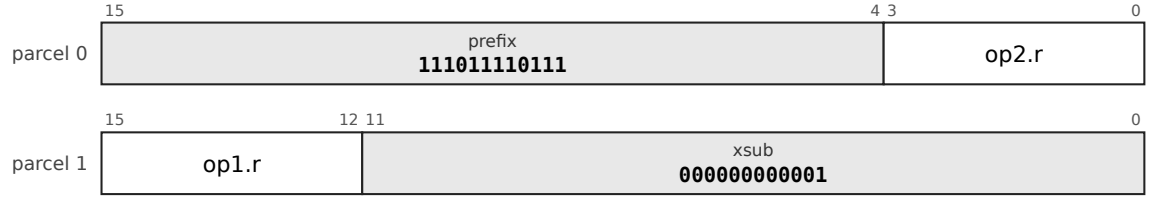



```
CLZ Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

CTZ

CTZ: R2, prefix 111011110111, sub-opcode 000000000001; operands reg4, reg4; 4 bytes

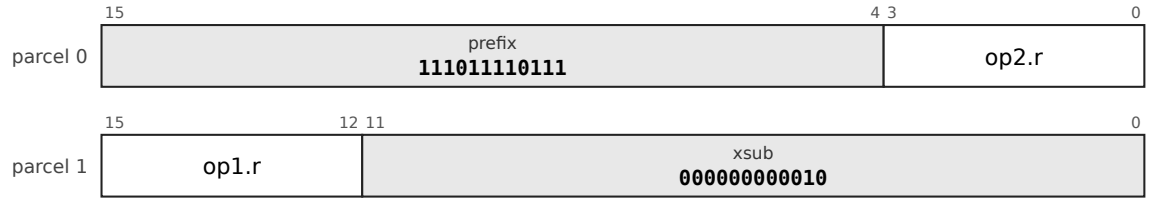



```
CTZ Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

CLO

CLO: R2, prefix 111011110111, sub-opcode 000000000010; operands reg4, reg4; 4 bytes

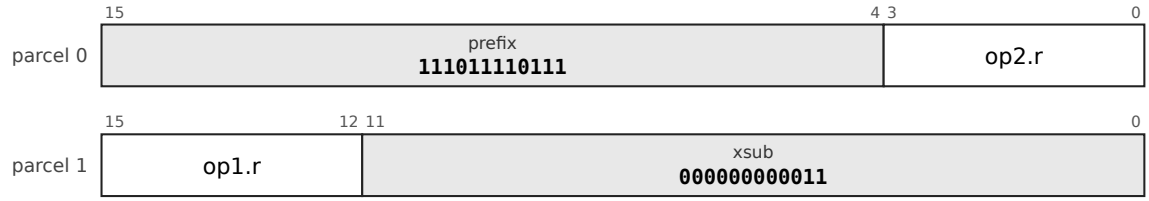



```
CLO Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

CTO

CTO: R2, prefix 111011110111, sub-opcode 000000000011; operands reg4, reg4; 4 bytes

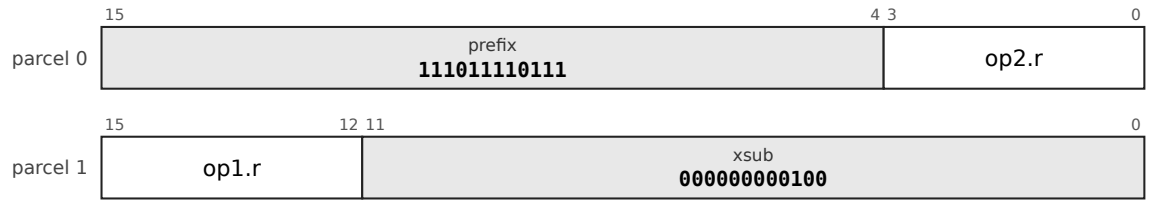



```
CTO Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

POPCNT: Population count

POPCNT: R2, prefix 111011110111, sub-opcode 000000000100; operands reg4, reg4; 4 bytes

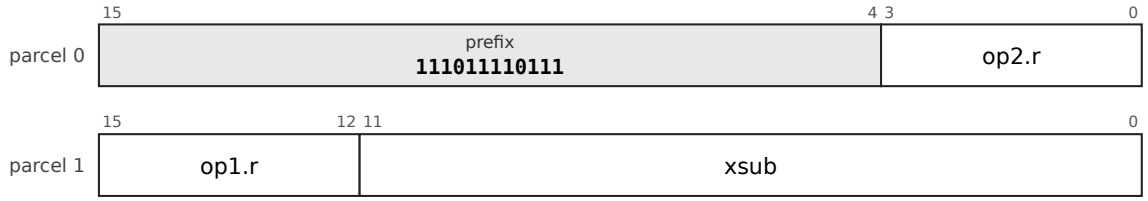


Mnemonics: `POPCNT`

Syntax

```
POPCNT Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← number of one bits in Rs
```

Flags None written.

Exceptions None.

Why it is here. One of the counts; it is also the building block of `XNORPOP`. A compiler needs `__builtin_popcount` to emit it, so the corpus shows no use (see `COUNT`).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

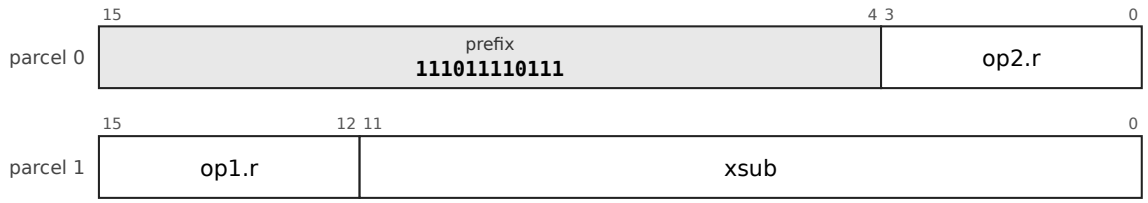
BSWAP: Byte swap

Mnemonics: `BSWAP`, `BSWAPW`, `BSWAPL`

Syntax

```
BSWAP Rs, Rd
BSWAPW Rs, Rd
BSWAPL Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
BSWAP : Rd ← Rs with all 8 bytes reversed
BSWAPW: Rd ← Rs with the bytes of each 16-bit word reversed
BSWAPL: Rd ← Rs with the bytes of each 32-bit long reversed
```

Flags None written.

Exceptions None.

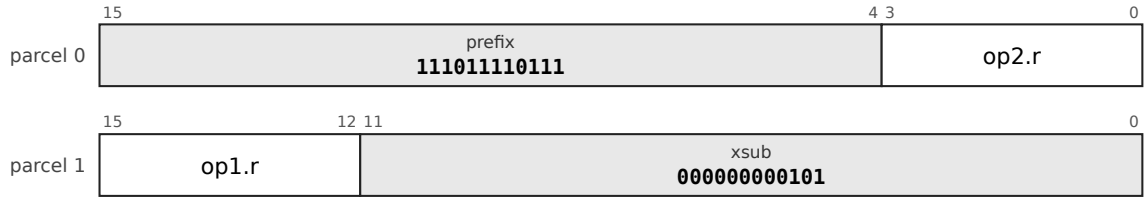
The generalisation of the PDP-11 `SWAB` to 64 bits.

Why it is here. The PDP-11 `SWAB` swapped the bytes of a word; `BSWAP` generalises it to 64 bits, with word and long variants. `SWAB` is therefore subsumed, not lost (9.18).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BSWAP

BSWAP: R2, prefix 111011110111, sub-opcode 000000000101; operands reg4, reg4; 4 bytes

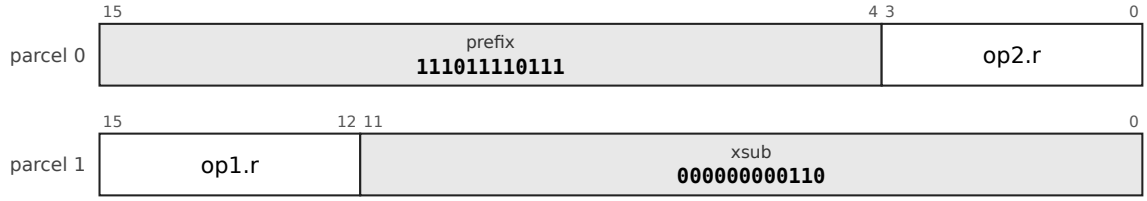



```
BSWAP Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

BSWAPW

BSWAPW: R2, prefix 111011110111, sub-opcode 000000000110; operands reg4, reg4; 4 byte

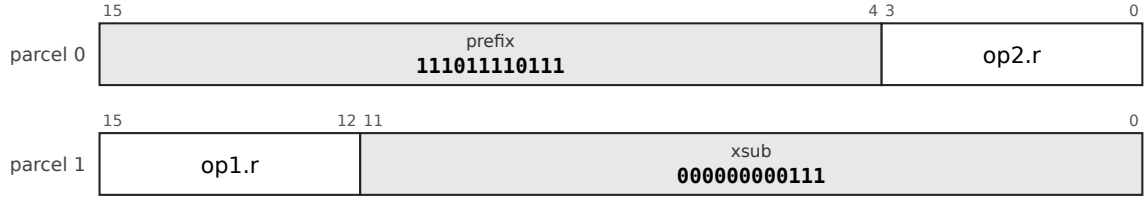



```
BSWAPW Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

BSWAPL

BSWAPL: R2, prefix 111011110111, sub-opcode 000000000111; operands reg4, reg4; 4 bytes

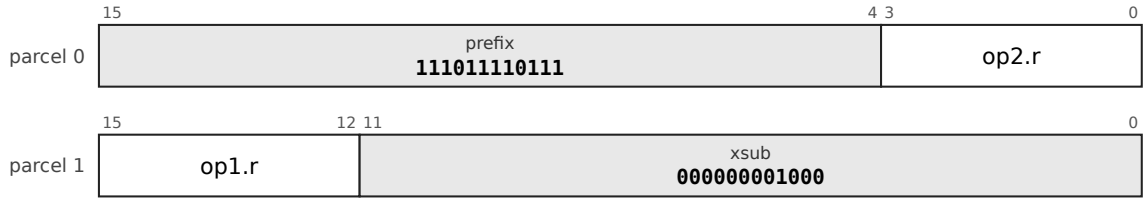



```
BSWAPL Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

BITREV: Reverse the bits

BITREV: R2, prefix 111011110111, sub-opcode 000000001000; operands reg4, reg4; 4 bytes

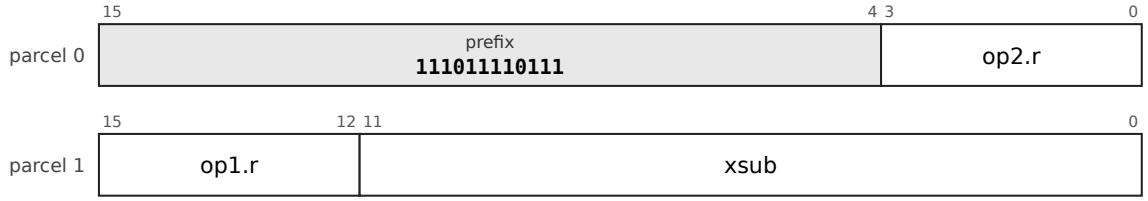


Mnemonics: `BITREV`

Syntax

```
BITREV Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← Rs with bit i moved to bit 63-i
```

Flags None written.

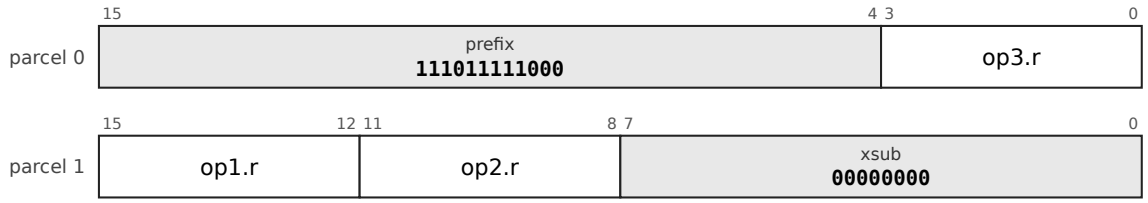
Exceptions None.

Why it is here. Borrowed with the other LoongArch bit operations (`REVB`, `REVH`, `BITREV`). No workload from the corpus is recorded.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ROR: Rotate right by a register count

ROR: R3, prefix 111011111000, sub-opcode 00000000; operands reg4, reg4, reg4; 4 bytes

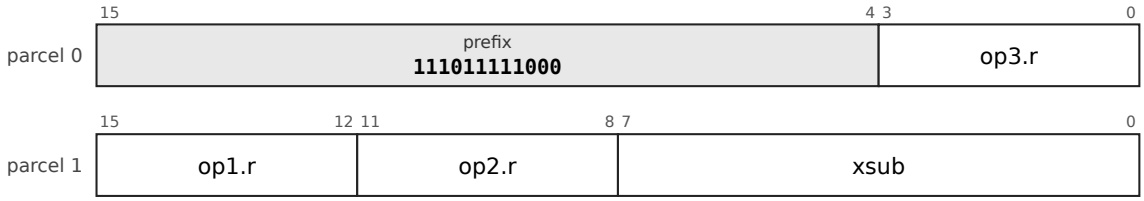


Mnemonics: `ROR`

Syntax

```
ROR Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
Rd ← Ra rotated right by (Rb mod 64)
```

Flags None written (rotates write no flags).

Exceptions None.

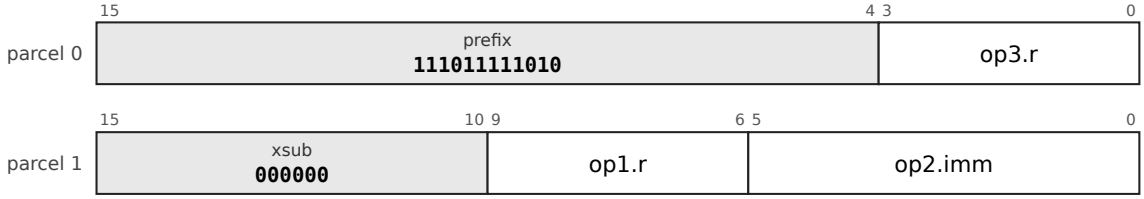
There is no rotate left instruction: `ROL rs, #n, rd` is an assembler spelling of `RORI rs, #(64-n), rd`. No rotate is through carry; multi-word shifting uses `DSHR`.

Why it is here. One rotate direction suffices and `ROR` is the one to keep: `ROL x, #n` is exactly `ROR x, #(64-n)`, and a funnel shifter yields `ROR` directly. No rotate is through carry (see `DSHR`). `ROR` is a bit operation and not an ALU operation: cryptographic code rotates by compile-time constants, so an immediate form is needed either way, and the register-count form has almost no users. Rotates write no flags.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

RORI: Rotate right by an immediate

RORI: RRI, prefix 111011111010, sub-opcode 000000; operands reg4, imm6u, reg4; 4 bytes

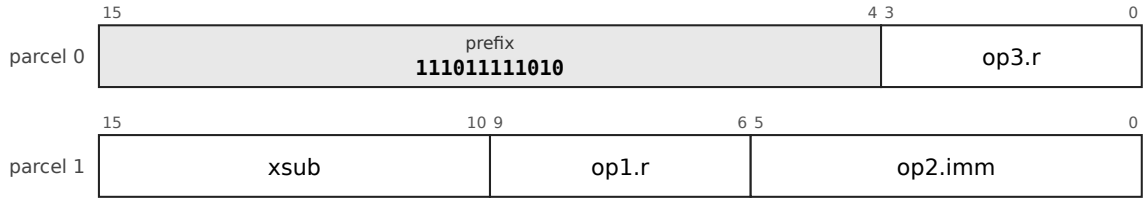


Mnemonics: `RORI` (aliases: `ROL`)

Syntax

```
RORI Rs, #n, Rd
ROL Rs, #n, Rd        ; assembles to RORI Rs, #(64-n), Rd
```

Encoding



The chart is the leaf `16-RRI-1`, RRI, prefix `111011111010`. `op1` is `reg4`: register, 4 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-RRI-1`.

Operation

```
Rd ← Rs rotated right by (n mod 64)
```

Flags None written.

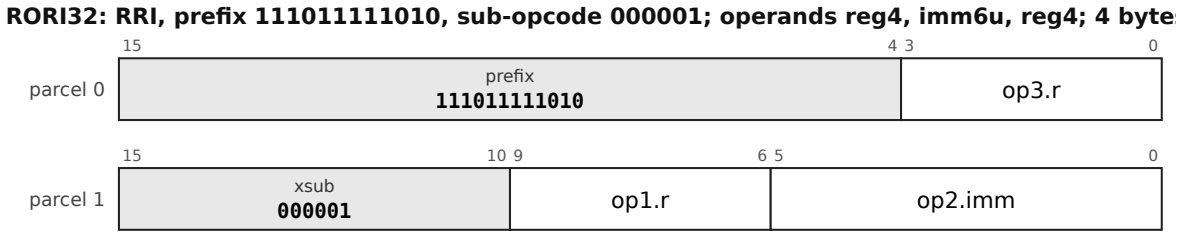
Exceptions None.

Why it is here. It has its own mnemonic because an immediate count and a register count are different encodings. `ROL Rs, #n, Rd` is accepted by the assembler as `RORI Rs, #(64-n), Rd`: the complement costs nothing at assembly time, so there is no reason to make programmers do it, and a register-count `ROL` is rejected so that one mnemonic stays one instruction.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

`ROL` is an alias of `RORI`: the assembler writes it as that word.

RORI32: Rotate right by an immediate, 32 bits

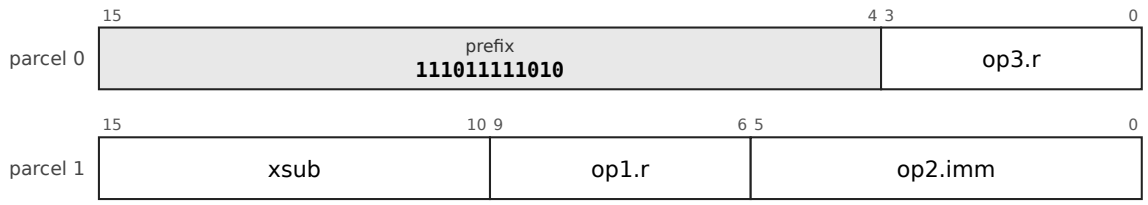


Mnemonics: `RORI32`

Syntax

```
RORI32 Rs, #n, Rd
```

Encoding



The chart is the leaf `16-RRI-1`, RRI, prefix `111011111010`. `op1` is `reg4`: register, 4 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-RRI-1`.

Operation

```
Rd ← sext32(ror32(Rs[31:0], n mod 32))
```

Flags None written.

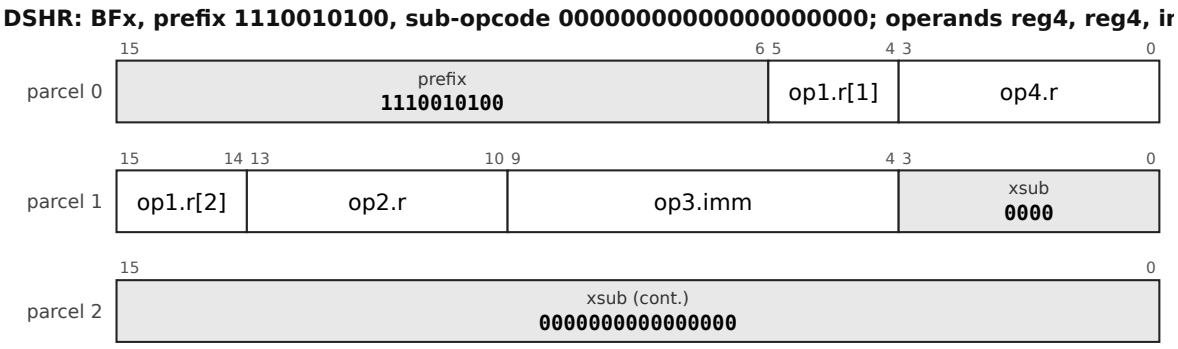
Exceptions None.

The 32-bit rotate with the result sign-extended like every 32-bit result; the primitive of most hash functions and ciphers.

Why it is here. A 32-bit rotate by a constant, the shape of crypto and hash inner loops, estimated together with `CBCC` at 22 KB (1.3%) of the compiled image. The result is sign-extended like the rest of the 32-bit operations, so that no instruction is needed after it. It removes 44 `MOVL` instructions in the corpus (the demand analysis already removes nearly all of them).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

DSHR: Double-width funnel shift right

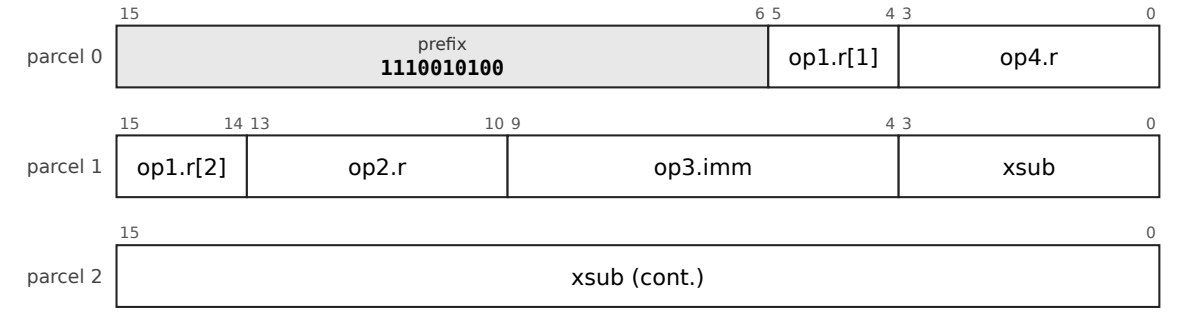


Mnemonics: `DSHR`

Syntax

```
DSHR Ra, Rb, #n, Rd
```

Encoding



The chart is the leaf `16-BFx-1`, BFx, prefix `1110010100`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-BFx-1`.

Operation

```
t ← Ra:Rb                   ; 128 bits, Ra the high half
Rd ← (t >> n)[63:0]         ; n = 0..63
```

Flags None written.

Exceptions None.

x86 SHRD, ARM EXTR. A 128-bit left shift by k is `DSHR hi, hi, lo, #(64-k)` then `SHL #k, lo`.

Why it is here. A double-width funnel shift (x86 SHRD, ARM EXTR), the right primitive for multi-word shifting. Two instructions shift a 128-bit value by any distance, where rotate-through-carry takes one bit per instruction, and that is why no rotate is through carry. It also subsumes ROR when both sources are the same register. It leaves every flag alone.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

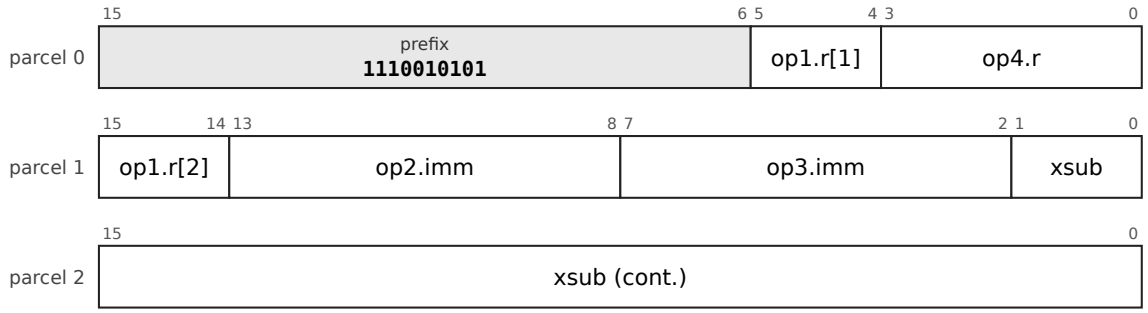
EXTB: Extract a bit field

Mnemonics: EXTB, EXTZB

Syntax

```
EXTB Rs, #pos, #width, Rd       ; sign-extended
EXTZB Rs, #pos, #width, Rd      ; zero-extended
```

Encoding



The chart is the leaf `16-BFy-1`, BFy, prefix `1110010101`. `op1` is `reg4`: register, 4 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-BFy-1`.

Operation

```
Rd ← ext((Rs >> pos)[width-1:0])          ; sext for EXTB, zext for EXTZB; width 0 or > 64 means 64
```

Flags None written.

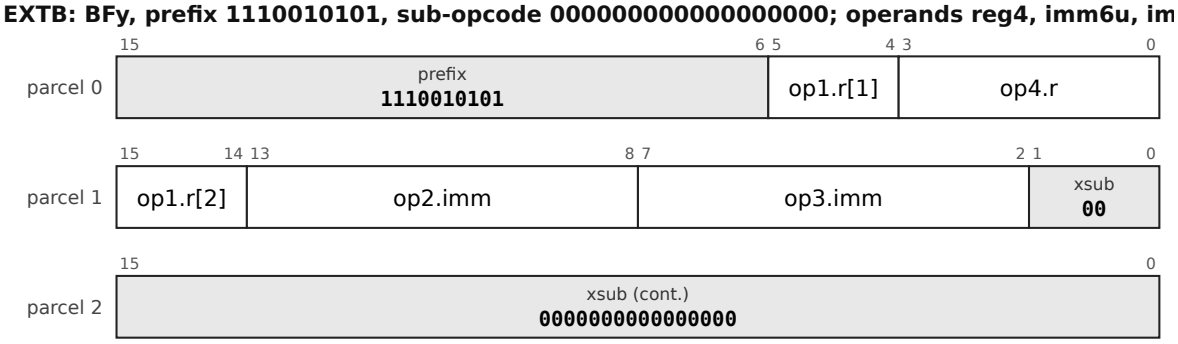
Exceptions None.

Position and width are immediate operands; a width of 64 cannot be encoded (errata 87, assumption 8).

Why it is here. Bit-field extract and insert are the VAX EXTV, EXTZV and INSV idiom. The compiler does not use them: the shift-pair extraction pattern occurs 151 times in 26 files of the corpus, about 600 bytes at best (0.03%), and there are four true bit-field extracts in all. They are for hand-written protocol and bit-stream code.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

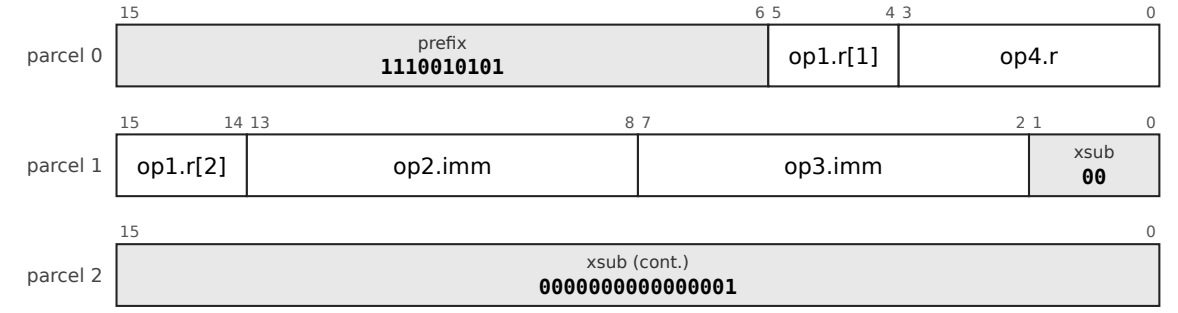
EXTB




```
EXTB Rs, #pos, #width, Rd
```

Operation, flags and exceptions as for the entry EXTB above; 1 layout form, each a chart of its own (above).

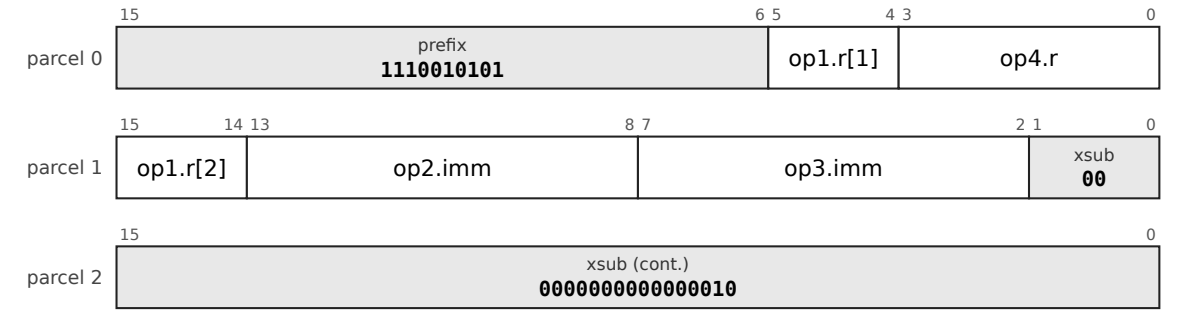
EXTZB




```
EXTZB Rs, #pos, #width, Rd
```

Operation, flags and exceptions as for the entry EXTB above; 1 layout form, each a chart of its own (above).

INSB: Insert a bit field

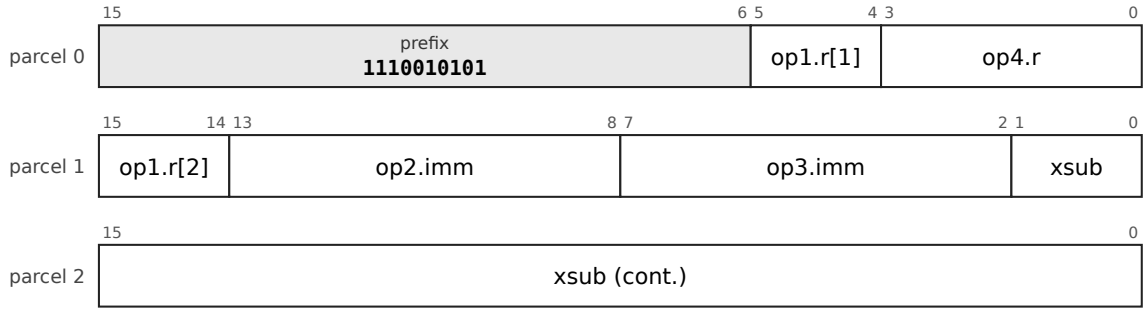


Mnemonics: `INSB`

Syntax

```
INSB Rs, #pos, #width, Rd
```

Encoding



The chart is the leaf `16-BFy-1`, BFy, prefix `1110010101`. `op1` is `reg4`: register, 4 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-BFy-1`.

Operation

```
mask ← ((1 << width) - 1) << pos
Rd ← (Rd AND NOT mask) OR ((Rs << pos) AND mask)
```

Flags None written.

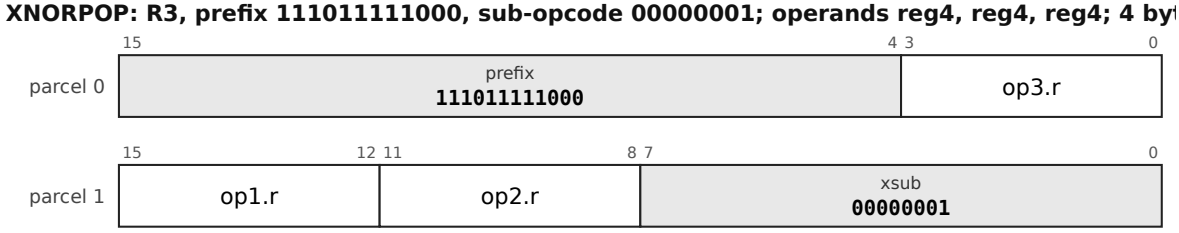
Exceptions None.

Six bytes. `Rd` is both source and destination.

Why it is here. The VAX `INSV` idiom, with `EXTB`. No use in the compiled corpus is recorded.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

XNORPOP: Population count of the equality of two words

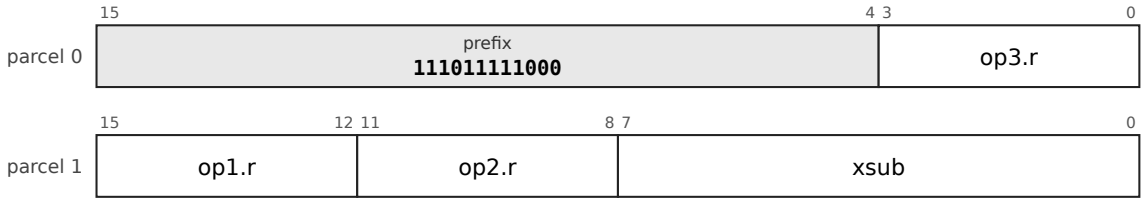


Mnemonics: `XNORPOP`

Syntax

```
XNORPOP Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
Rd ← popcount(NOT (Ra XOR Rb))
```

Flags None written.

Exceptions None.

For binary-activation networks, Hamming distance and bitset intersection cardinality. It does **not** serve sub-1-bit language models (section 8.3).

Why it is here. It serves binary-activation networks, Hamming distance, bitset intersection counts and counting a collector's mark bitmap. It does not serve sub-1-bit language models: their activations (in the LittleBit work) are FP16 and the inner loop is a conditional add, not a popcount. The sub-1-bit kernel is bandwidth-bound and belongs to a bus-mastering accelerator, not an opcode (chapter 8).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

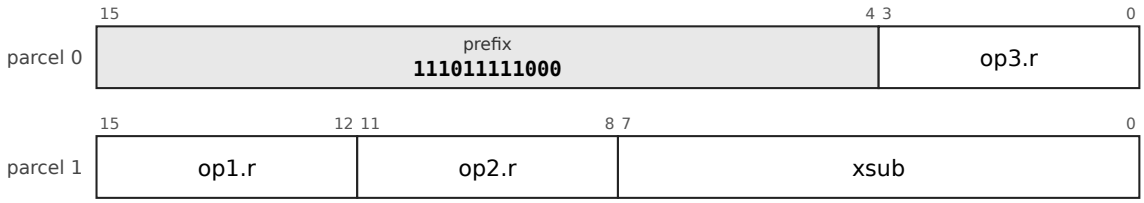
CLMUL: Carry-less multiply

Mnemonics: `CLMULL`, `CLMULH`

Syntax

```
CLMULL Ra, Rb, Rd
CLMULH Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
p ← carry-less (polynomial over GF(2)) product of Ra and Rb      ; 128 bits
CLMULL: Rd ← p[63:0]
CLMULH: Rd ← p[127:64]
```

Flags None written.

Exceptions None.

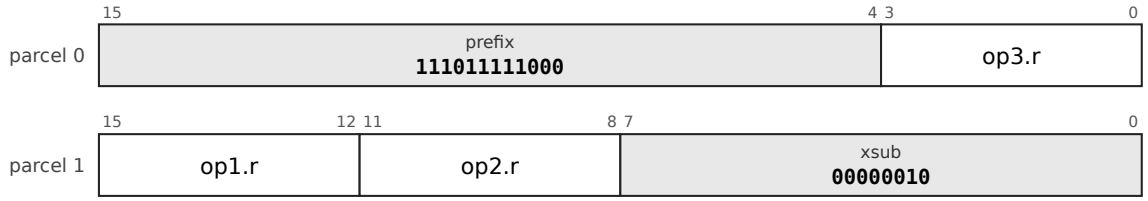
The primitive under every CRC (with a Barrett reduction), GF(2^k) arithmetic and GCM; no polynomial is committed to the architecture.

Why it is here. Carry-less multiply is the primitive beneath every CRC polynomial (a CRC is a carry-less multiply plus a Barrett reduction), GF(2^k) erasure codes and AES-GCM. It is chosen over a `CRC` instruction because x86 and ARM each committed a polynomial to the architecture; there are no `CRC32` or `CRC32C` words.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

CLMULL

CLMULL: R3, prefix 111011111000, sub-opcode 00000010; operands reg4, reg4, reg4; 4 byte

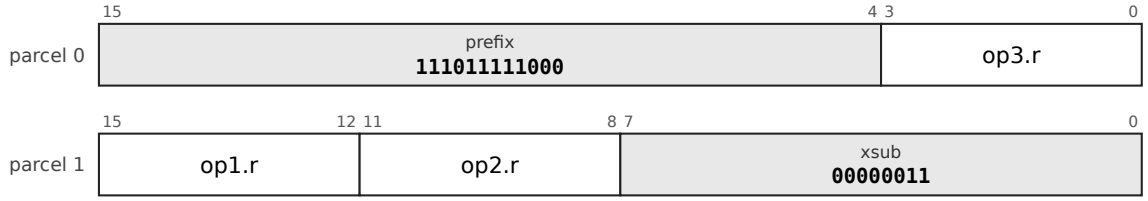



```
CLMULL Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry CLMUL above; 1 layout form, each a chart of its own (above).

CLMULH

CLMULH: R3, prefix 111011111000, sub-opcode 00000011; operands reg4, reg4, reg4; 4 byte

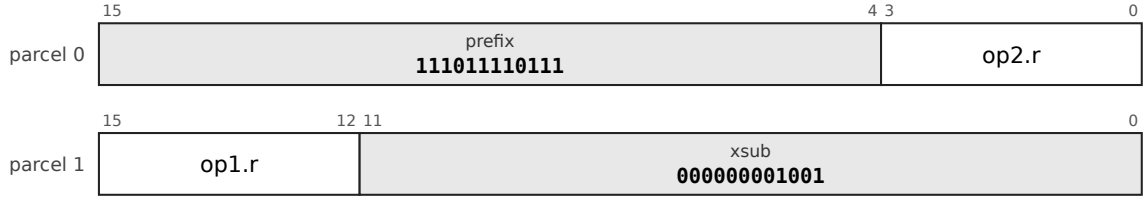



```
CLMULH Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry CLMUL above; 1 layout form, each a chart of its own (above).

UTF8LEN: Length of a UTF-8 sequence from its leading byte

UTF8LEN: R2, prefix 111011110111, sub-opcode 000000001001; operands reg4, reg4; 4 byte

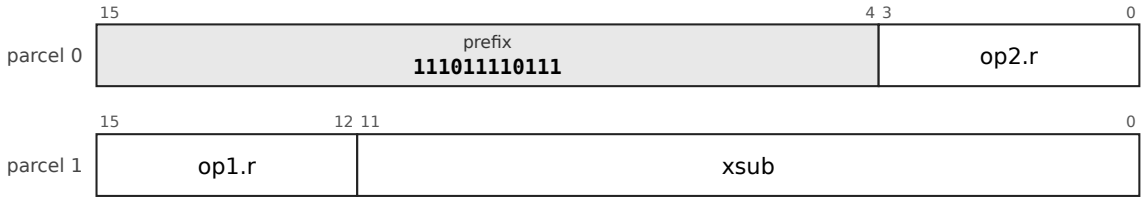


Mnemonics: `UTF8LEN`

Syntax

```
UTF8LEN Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← length (1 to 4) implied by the leading byte Rs[7:0]; 0 if it cannot begin a valid sequence
```

Flags None written.

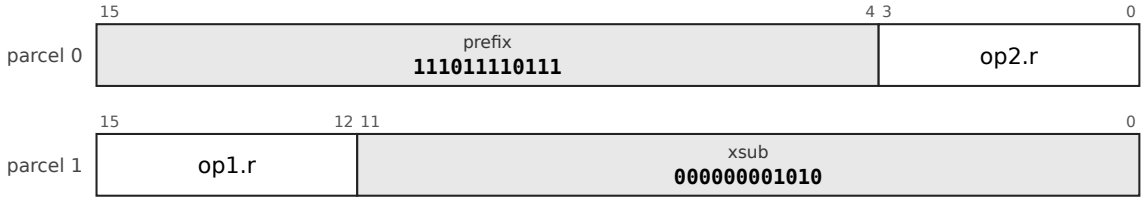
Exceptions None.

Why it is here. Most of what UTF-8 needs is general bit manipulation, but the validity rules are where the bugs are, so the group has three instructions. `UTF8LEN` is the one that is nearly free: `CLO` on the leading byte, mapped to 1, 2, 3 or 4.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

UTF8DEC: Decode a UTF-8 sequence

UTF8DEC: R2, prefix 111011110111, sub-opcode 000000001010; operands reg4, reg4; 4 byte

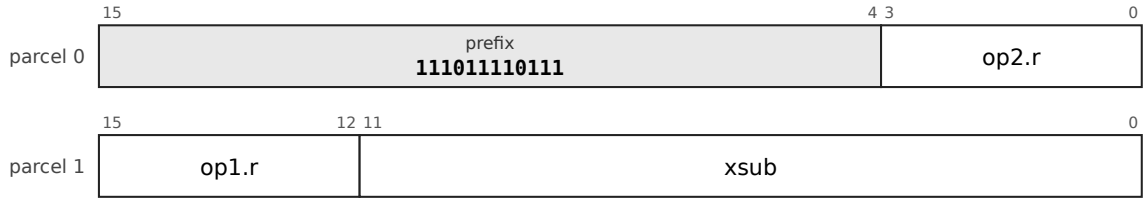


Mnemonics: `UTF8DEC`

Syntax

```
UTF8DEC Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rs holds up to 4 bytes, leading byte in bits 7..0
Rd ← the scalar value
V  ← 1 if the sequence is not a valid encoding
```

Flags `V` set for a bad continuation byte, an overlong form, a surrogate (U+D800 to U+DFFF), a value above U+10FFFF, or a leading byte of `11000000 (0xC0)`, `11000001 (0xC1)` or `11110101 (0xF5)` to `11111111 (0xFF)`; the length consumed is reported in a scratch field of the status word (details in the instruction-set text, section 4.6.1).

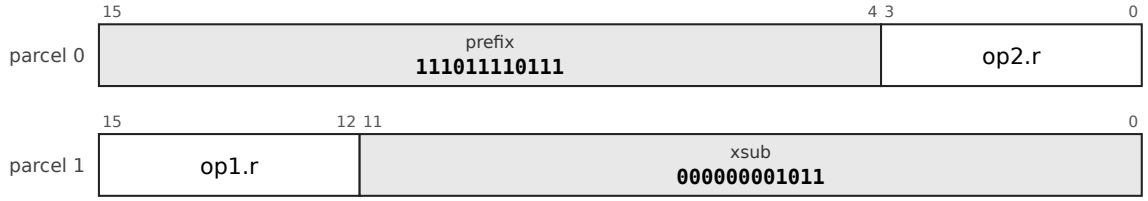
Exceptions None.

Why it is here. Not for speed but for correctness: overlong forms, surrogates, values above U+10FFFF and the bytes that can never begin a sequence are each a historical security hole, each is three comparators in hardware, and software implementations get them wrong or omit them for speed (about 300 lookup tables).

Status specified; **not yet implemented** in the reference executor.

UTF8ENC: Encode a Unicode scalar value

UTF8ENC: R2, prefix 111011110111, sub-opcode 000000001011; operands reg4, reg4; 4 byte

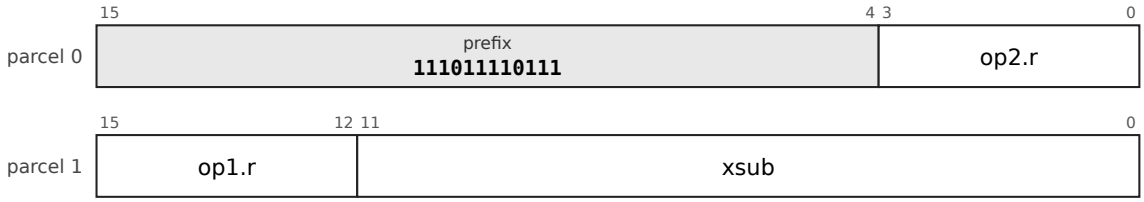


Mnemonics: `UTF8ENC`

Syntax

```
UTF8ENC Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← the UTF-8 bytes of the scalar value Rs, leading byte in bits 7..0
V  ← 1 if Rs is not a Unicode scalar value
```

Flags V set if `Rs` is not a Unicode scalar value (a surrogate or above U+10FFFF).

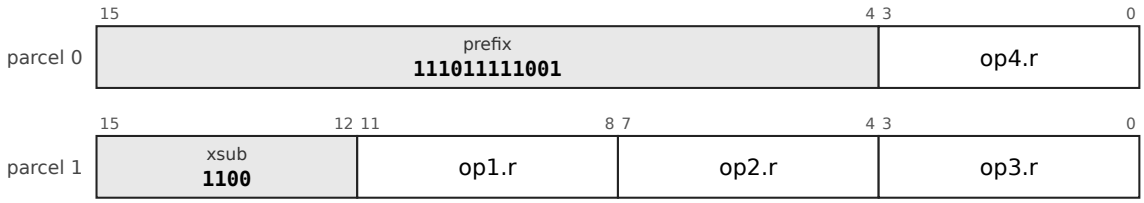
Exceptions None.

Why it is here. The encoding half of the UTF-8 pair; it refuses surrogates and values above U+10FFFF so that an encoder cannot produce what a decoder must reject.

Status specified; **not yet implemented** in the reference executor.

BSEL: Bitwise select

BSEL: R4, prefix 111011111001, sub-opcode 1100; operands reg4, reg4, reg4, reg4; 4 bytes

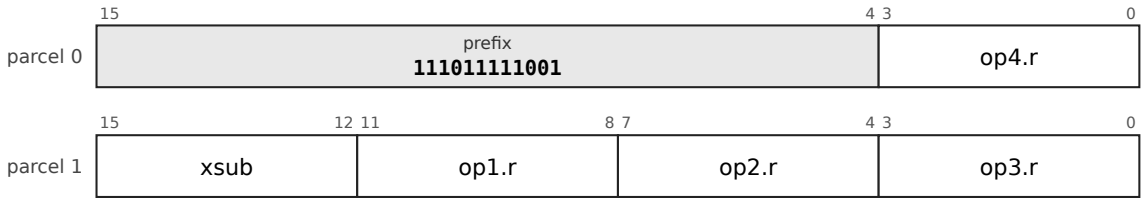


Mnemonics: `BSEL`

Syntax

```
BSEL Rm, Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R4-1`, R4, prefix `111011111001`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R4-1`.

Operation

```
Rd ← (Rm AND Ra) OR (NOT Rm AND Rb)       ; bit by bit
```

Flags None written.

Exceptions None.

(P) In the crypto proposal (`docs/crypto-secure-core.md` section 3.2), marked "adopt" there and not separately confirmed by the author; the reference executor implements it. `Ch(e, f, g)` of SHA-2 is `BSEL e, f, g`; `Maj(a, b, c)` is `BSEL (a XOR b), c, b`; a mask-controlled swap of two registers is two `BSEL`. Three sources and a destination: the four register fields of the three-operand group.

Why it is here. A bit select takes three sources and a destination, the four register fields of the three-operand group. Its case in the crypto proposal is SHA-2 (`Ch` and `Maj` are one `BSEL` each) and the constant-time swap: the X25519 ladder swaps ten limbs per step, 50 instructions with the mask pair against about 25 with `BSEL`. It is the select of ARM (`BSL`) and of x86 ternary logic, and it serves MD5 and SHA-1 round functions, blends and bit insertion. The proposal names it as the one instruction to drop "if the author wants fewer entities than the minimum", and the record does not show the author deciding it separately.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

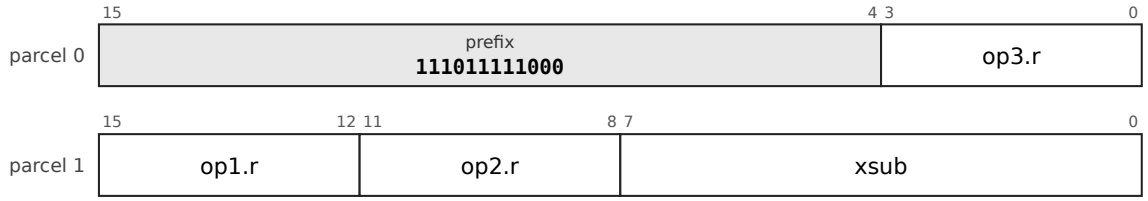
AESR: AES round on a 128-bit block in two registers

Mnemonics: `AESES`, `AESESM`, `AESDS`, `AESDSM`

Syntax

```
AESES Ra, Rb, Rd
AESESM Ra, Rb, Rd
AESDS Ra, Rb, Rd
AESDSM Ra, Rb, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
the 16 bytes of the block are Rb:Ra (Ra holds bytes 0 to 7, columns 0 and 1; Rb bytes 8 to 15)
AESES  : ShiftRows, then SubBytes
AESESM : ShiftRows, SubBytes, then MixColumns
AESDS  : inverse ShiftRows, then inverse SubBytes
AESDSM : inverse ShiftRows, inverse SubBytes, then inverse MixColumns
Rd ← the 8 bytes of the new state that belong to columns 0 and 1; with Ra and Rb exchanged, columns 2 and 3
```

Flags None written.

Exceptions None.

(P) `docs/crypto-secure-core.md` section 3.2; the author approved the secure-core design and the select instructions (errata 86) and has not yet answered sections 3 to 5 of that document. Any two registers hold a block (no even-odd pairing): a round on one block is `AESESM Ra, Rb, T` and `AESESM Rb, Ra, Rb`, with the round key XORed by `XOR` with a memory operand. The latency

is fixed and the instruction reads and writes no memory, so it has no cache-timing channel; the architecture does not say whether the S-box is a table inside the unit or a circuit. The definitions are those of FIPS 197 and must be checked against it before the text is final. **Not implemented** in the reference executor.

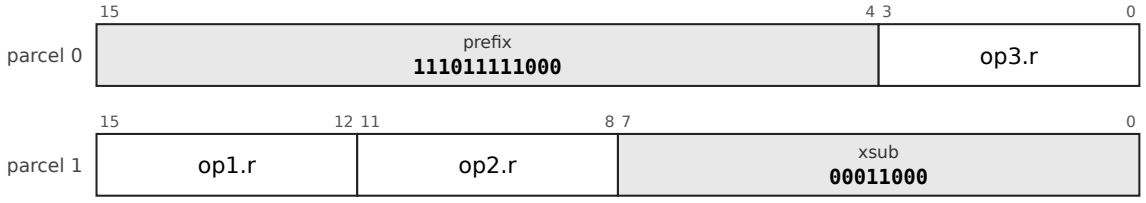
Why it is here. Constant-time software AES takes about 1,000 instructions a block (BearSSL's bitsliced code, scaled; the proposal's estimate), and table AES leaks through the cache. The author asked for constant-time AES and SHA instructions "or whichever opcodes would speed up and enable aes and sha" for the secure core (errata 85). The design is RISC-V's scalar cryptography (`aes64es` and its kin): the state is in the general registers, so no vector register file is needed and, unlike the AES drivers of OpenBSD for x86 and ARM, no `fpu_kernel_enter()` bracket either: a kernel AES loop saves nothing and is preemptible with the ordinary register save. A block costs 42 instructions for AES-128, 20 to 25 times fewer than the software sequence (estimates). Precedent for the pair-in-registers shape: RISC-V; the other machines (x86 AES-NI, ARM `AESE`) use vector registers.

Cf. docs/crypto-secure-core.md section 3.

Status specified; **not yet implemented** in the reference executor.

AESES

AESES: R3, prefix 111011111000, sub-opcode 00011000; operands reg4, reg4, reg4; 4 bytes

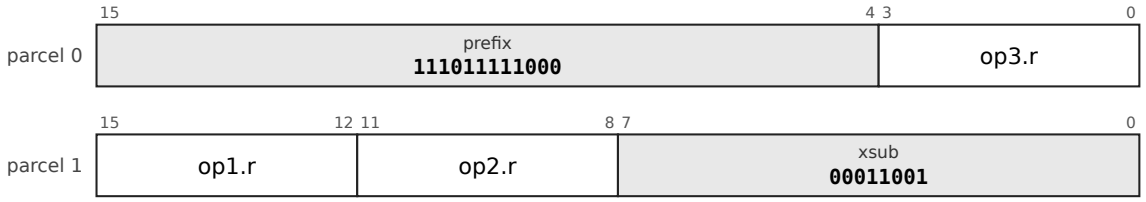



```
AESES Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

AESESM

AESESM: R3, prefix 111011111000, sub-opcode 00011001; operands reg4, reg4, reg4; 4 byte

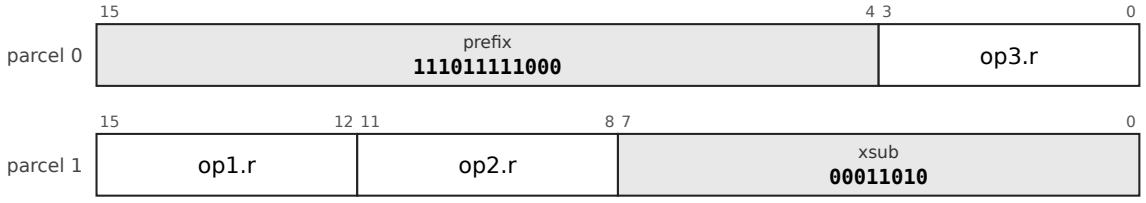



```
AESESM Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

AESDS

AESDS: R3, prefix 111011111000, sub-opcode 00011010; operands reg4, reg4, reg4; 4 bytes

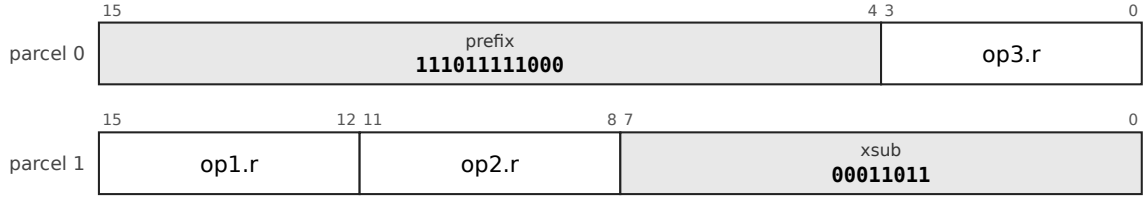



```
AESDS Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

AESDSM

AESDSM: R3, prefix 111011111000, sub-opcode 00011011; operands reg4, reg4, reg4; 4 byte




```
AESDSM Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

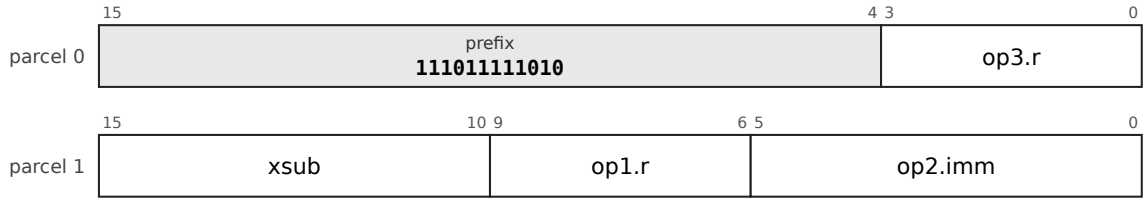
AESK: AES key schedule steps

Mnemonics: `AESKS1`, `AESKS2`, `AESIM`

Syntax

```
AESKS1 Rs, #n, Rd
AESKS2 Ra, Rb, Rd
AESIM Rs, Rd
```

Encoding



The chart is the leaf `16-RRI-1`, RRI, prefix `111011111010`. `op1` is `reg4`: register, 4 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-RRI-1`, `16-R3-1`, `16-R2-1`.

Operation

```
AESKS1: Rd ← (SubWord(RotWord(Rs[63:32])) XOR rcon(n)), replicated in both halves   ; n = 10: no round
constant (AES-256)
AESKS2: Rd[31:0] ← Ra[31:0] XOR Rb[63:32] ; Rd[63:32] ← Ra[63:32] XOR Rd[31:0]
AESIM : Rd ← InvMixColumns of the two columns in Rs         ; for the decryption key schedule
```

Flags None written.

Exceptions None.

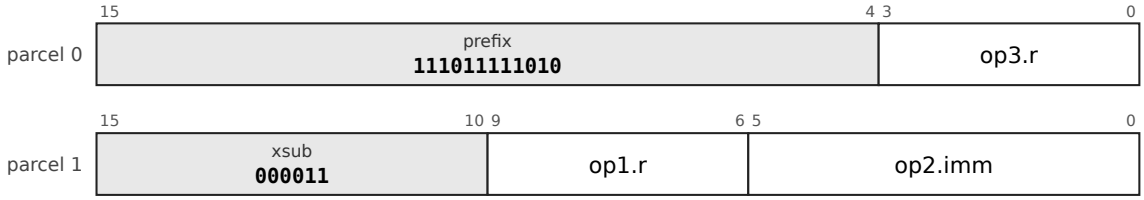
(P) As `AESR`: `docs/crypto-secure-core.md` section 3.2, awaiting the author's answer. The key schedule of AES-128 takes three instructions per round. **Not implemented** in the reference executor.

Why it is here. The key schedule and the inverse mix of the round key complete the round instructions: three instructions per round of the AES-128 schedule, about 30 for the whole schedule instead of about 150, once per key; `AESIM` adds 18 for the decryption schedule. The same precedent as `AESR`.

Status specified; **not yet implemented** in the reference executor.

AESKS1

AESKS1: RRI, prefix 111011111010, sub-opcode 000011; operands reg4, imm6u, reg4; 4 byte

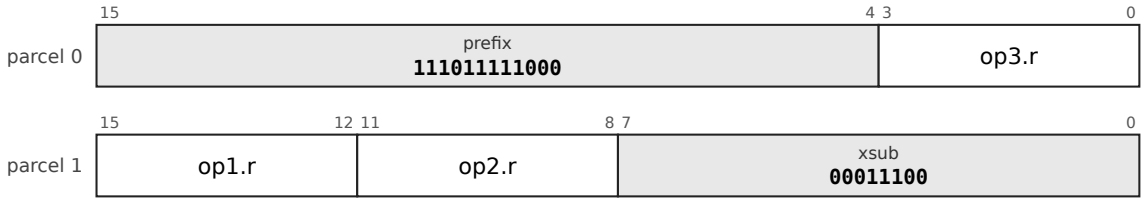



```
AESKS1 Rs, #n, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

AESKS2

AESKS2: R3, prefix 111011111000, sub-opcode 00011100; operands reg4, reg4, reg4; 4 byte

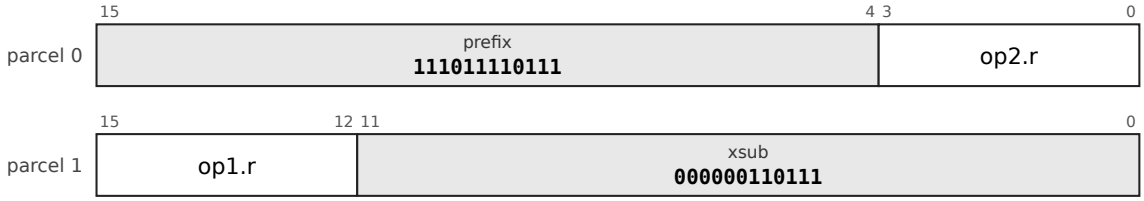



```
AESKS2 Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

AESIM

AESIM: R2, prefix 111011110111, sub-opcode 000000110111; operands reg4, reg4; 4 bytes




```
AESIM Rs, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

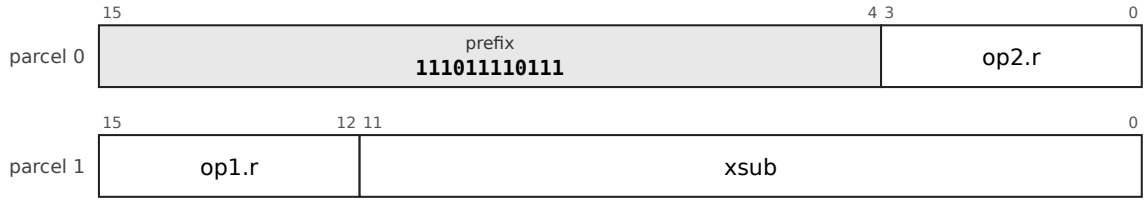
SHA256: The SHA-256 functions

Mnemonics: `SHA256SIG0`, `SHA256SIG1`, `SHA256SUM0`, `SHA256SUM1`

Syntax

```
SHA256SIG0 Rs, Rd
SHA256SIG1 Rs, Rd
SHA256SUM0 Rs, Rd
SHA256SUM1 Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
on the low 32 bits x of Rs; the result is in the low 32 bits of Rd, the upper bits zero
SIG0: ror(x,7)  XOR ror(x,18) XOR shr(x,3)
SIG1: ror(x,17) XOR ror(x,19) XOR shr(x,10)
SUM0: ror(x,2)  XOR ror(x,13) XOR ror(x,22)
SUM1: ror(x,6)  XOR ror(x,11) XOR ror(x,25)
```

Flags None written.

Exceptions None.

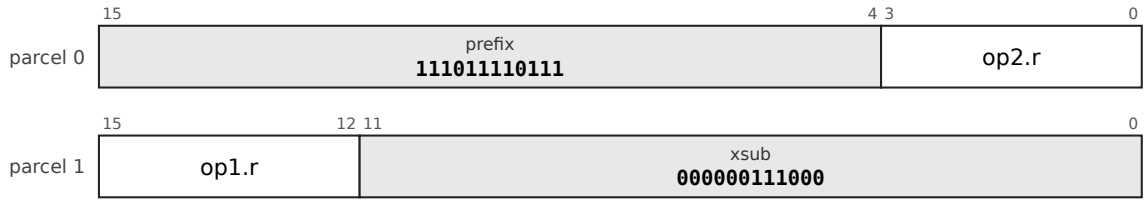
(P) `docs/crypto-secure-core.md` section 3.2, awaiting the author's answer. The result is zero-extended there; every other 32-bit result of the machine is sign-extended (`RORI32`), so the choice is open. Rotations and three-input exclusive ors, one cycle in hardware, no data-dependent timing. **Not implemented** in the reference executor.

Why it is here. The SHA-2 rounds are rotations and three-input exclusive ors, which the integer instructions do in several steps; the RISC-V scalar cryptography extension has the eight functions as instructions. The crypto proposal counts the gain honestly: the primitives remove half the work, not nineteen twentieths, because a round still needs the adds and the register traffic (about 1,200 against 2,500 instructions for a 64-byte block with `BSEL`, estimates).

Status specified; **not yet implemented** in the reference executor.

SHA256SIG0

SHA256SIG0: R2, prefix 111011110111, sub-opcode 000000111000; operands reg4, reg4; 4 b

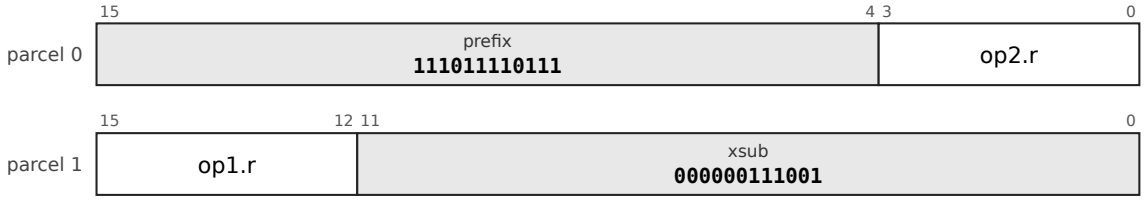



```
SHA256SIG0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

SHA256SIG1

SHA256SIG1: R2, prefix 111011110111, sub-opcode 000000111001; operands reg4, reg4; 4 b

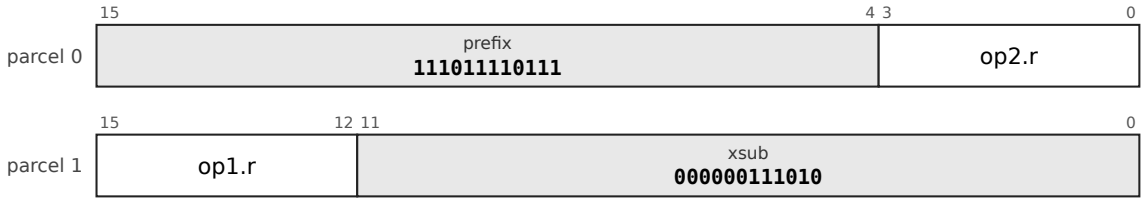



```
SHA256SIG1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

SHA256SUM0

SHA256SUM0: R2, prefix 111011110111, sub-opcode 000000111010; operands reg4, reg4; 4

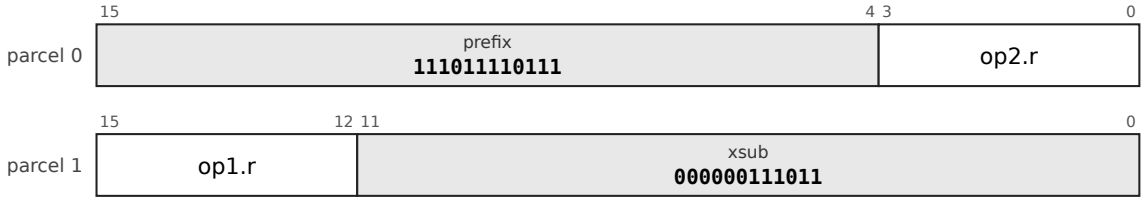



```
SHA256SUM0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

SHA256SUM1

SHA256SUM1: R2, prefix 111011110111, sub-opcode 000000111011; operands reg4, reg4; 4




```
SHA256SUM1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

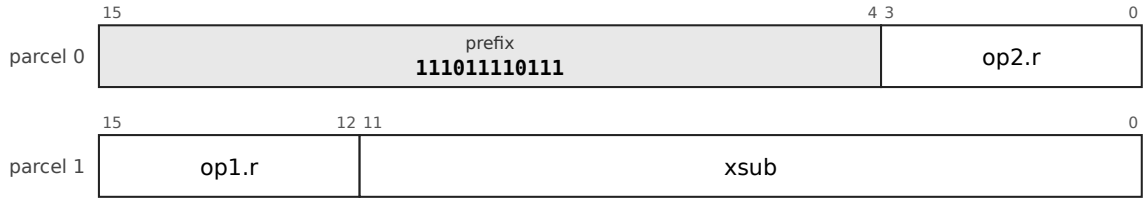
SHA512: The SHA-512 functions

Mnemonics: `SHA512SIG0`, `SHA512SIG1`, `SHA512SUM0`, `SHA512SUM1`

Syntax

```
SHA512SIG0 Rs, Rd
SHA512SIG1 Rs, Rd
SHA512SUM0 Rs, Rd
SHA512SUM1 Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
on the 64 bits x of Rs
SIG0: ror(x,1)  XOR ror(x,8)  XOR shr(x,7)
SIG1: ror(x,19) XOR ror(x,61) XOR shr(x,6)
SUM0: ror(x,28) XOR ror(x,34) XOR ror(x,39)
SUM1: ror(x,14) XOR ror(x,18) XOR ror(x,41)
```

Flags None written.

Exceptions None.

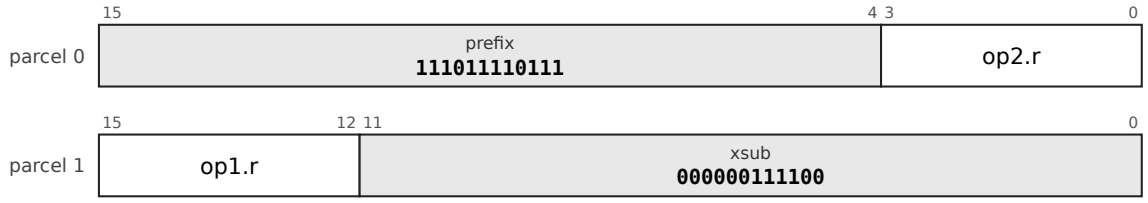
(P) As `SHA256`. **Not implemented** in the reference executor.

Why it is here. See `SHA256`: the four 64-bit functions of SHA-512, with the same gain of about two (about 1,500 against 3,100 instructions for a 128-byte block, estimates).

Status specified; **not yet implemented** in the reference executor.

SHA512SIG0

SHA512SIG0: R2, prefix 111011110111, sub-opcode 000000111100; operands reg4, reg4; 4 b

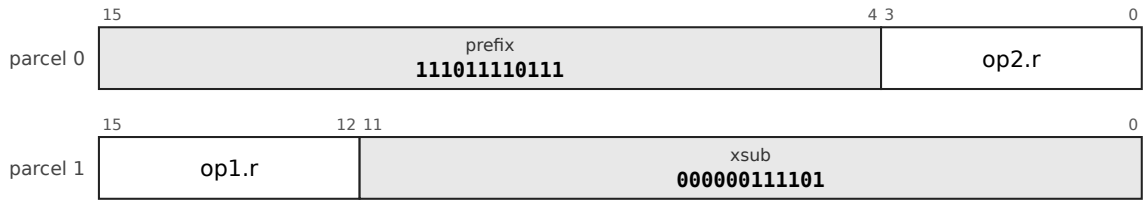



```
SHA512SIG0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

SHA512SIG1

SHA512SIG1: R2, prefix 111011110111, sub-opcode 000000111101; operands reg4, reg4; 4 b

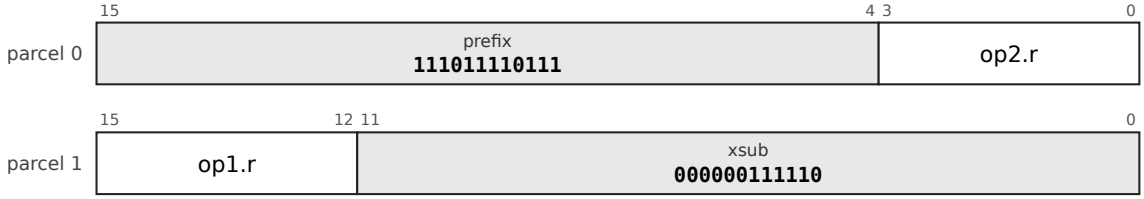



```
SHA512SIG1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

SHA512SUM0

SHA512SUM0: R2, prefix 111011110111, sub-opcode 000000111110; operands reg4, reg4; 4

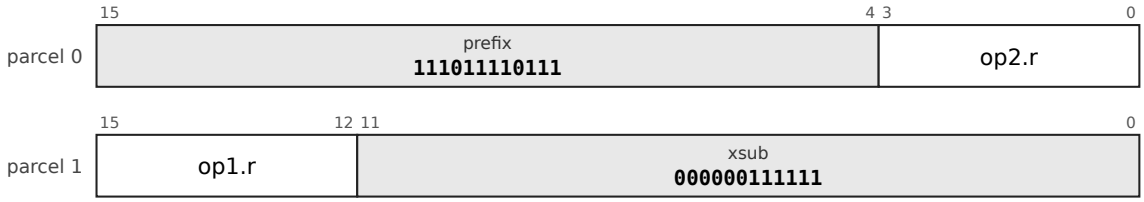



```
SHA512SUM0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

SHA512SUM1

SHA512SUM1: R2, prefix 111011110111, sub-opcode 000000111111; operands reg4, reg4; 4




```
SHA512SUM1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

9.10 Branches and control flow

Cf. section 6.7 for TRAP, RTI *and* RTT, *and section 6.2 for how a trap enters the kernel; section 9.16 for* CALLI *and* JMPI *in position-independent code.*

9.10.1 There is an unconditional branch BR and fourteen conditional branches, two bytes with a field of 6 to 9 bits, four with 16 bits, six with 32 bits, counting two-byte parcels from the first parcel of the branch: ±64 to ±512 bytes, ±64 KiB, ±4 GiB.

9.10.2 The conditions are `ne`, `eq`, `pl`, `mi`, `vc`, `vs`, `cc`/`hs`, `cs`/`lo`, `ge`, `lt`, `gt`, `le`, `hi` and `los`.

9.10.3 `SOB Rn, dst` subtracts one from `Rn` and branches if the result is nonzero, forward or backward. It is four bytes, with a 16-bit field of parcels (±64 KiB).

9.10.4 CBCC compares a register with zero, with an immediate or with a second register and branches on the condition, and writes no flags. It is one parcel for CBEQ and CBNE against zero (a field of 4 to 7 bits: ±16 to ±128 bytes); two parcels for an immediate of 3 to 10 bits or a second register, with a field of 8 to 14 bits (±256 bytes to ±16 KiB); three parcels for a 16-bit immediate with a 12-bit field (±4 KiB), an 8-bit immediate with a 20-bit field (±1 MiB) or two registers with a 24-bit field (±16 MiB). *(D)*

9.10.5 JMP and CALL take any address mode; RET returns; JMPI and CALLI jump and call through a pointer in memory; JTAG dispatches on the tag in the low three bits of a register; BTAG and BNTAG branch on a tag. *(D)*

9.10.6 A call pushes the return address on the stack; there is no link register. *(D)*

9.10.7 Four pairs of instructions are fused into one. `MOVCALL Rs, Rd, label` is `MOVQ Rs, Rd` followed by `CALL label`, in four bytes (a 16-bit field of parcels, ±64 KiB) or six (32 bits); `POPRET Rd` is `POP Rd` followed by RET, and `POPMRET #mask` is `POPM #mask` followed by RET (the 8-bit mask field, scaled by 256, names `R8` to `R15`), each in two bytes; `MOV2 Ra, Rb, Rc, Rd` is `MOVQ Ra, Rb` followed by `MOVQ Rc, Rd`, in two bytes, with all four registers in the 3-bit hot table. The assembler never fuses: choosing a fused instruction is the compiler's job. A fused instruction is restarted whole after a fault. *(D)*

9.10.8 A CBCC for which no form reaches (a label in another object, an operand pair that no form holds) is assembled as CMP followed by BCC. This writes the flags, which CBCC does not, and executes two instructions for one; the compiler forms a CBCC only where the flags are dead after the branch, so the program is unchanged. On the 841-file corpus no site needs the pair; in the six large programs none is executed.

Commentary. **Precedent.** The complete PDP-11 branch set, which the long-branch problem of the PDP-11 (±256 bytes) made awkward, here reaches ±4 GB with no long-branch opcode and no trampoline, because the displacement widens with the length. SOB was the PDP-11's, which could only branch backward. **Trade-offs.** A compare-and-branch that writes no flags cannot feed a second branch that reads them, so a three-way compare is compare-then-branch; the compiler fuses only where the flags are dead afterwards, which costs about 0.4% in code size. **Improvements.** The tag branches serve a Lisp or other tagged runtime in one instruction. **Benefits.** Branch relaxation in the assembler is choosing a length. **Modern comparables.** RISC-V and MIPS are compare-and-branch machines with no flags; AArch64 has CBZ and TBZ; x86 uses flags like the pdp64's branches. **Use.** JMPI and CALLI are the shared-library call and the virtual call; JTAG is the dispatch of a dynamic-language interpreter. **Impact.** *Kernel:* JMPI, CALLI and JTAG are the shared-library call, the virtual call and the dispatch of an interpreter; SOB and CBCC are the loops of the kernel's inner routines. *Compiler:* Branch relaxation is choosing a length; fusing a compare and a branch is legal only where the flags are dead afterwards, so the compiler needs a flag-liveness analysis. *Hardware:* The next-PC mux takes the load-return path for the indirect forms; the compare-and-branch compare runs in the branch unit beside the ALU; displacements count parcels, so no shifter is needed.

Commentary on 9.10.4, 9.10.7 and 9.10.8. **Precedent.** The PDP-11 branched on the flags of the previous instruction only; compare-and-branch machines (MIPS, RISC-V) branch on two registers in one instruction, and the long-branch problem of both was solved by a longer encoding, here by the three lengths of CBCC. Macro-op fusion in current x86 and ARM cores joins a compare and a branch in the decoder; here the compiler does the joining and the instruction set names the result. **Trade-offs.** The reach of the shortest CBCC forms was set from the corpus: the optimiser trained on the compiler's own output, priced a CBCC that no form reaches as the CMP and BCC that the assembler substitutes, and offered the three-parcel forms; the cost is that one-parcel CBCC in pdp64/32 reaches less than in pdp64/16, and lz4 fetches 1.2% more bytes for it (appendix F). The four fused pairs save two bytes a pair: 39,674 bytes (4.16%) in the corpus of pdp64/16 when the compiler was first given them. **Improvements.** The fused call has a three-parcel form, so a call beyond ±64 KiB is still one instruction; and the assembler's CMP plus BCC makes every program assemble, so a form that does not reach is a slower program and never an error. **Benefits.** Fusion is a decision of the compiler, so the instruction set stays free of microarchitectural pairing rules, and a core that does not fuse in hardware loses nothing. **Modern comparables.** The fused pairs are the ones the compiled corpus shows most often adjacent: a move before a call, a pop before a return. **Impact.** *Kernel:* POPRET and POPMRET are the epilogue of every function and MOVCALL the call of every wrapper; they are smaller kernel text with no source change. *Compiler:* it must know the flags are dead before forming CBCC, fuse after register allocation, and keep a mask of R24 to R31 out of POPMRET (the field names R8 to R15 only). *Hardware:* a fused instruction is decoded as two operations that retire together; a hardware implementation may execute them as one or two.

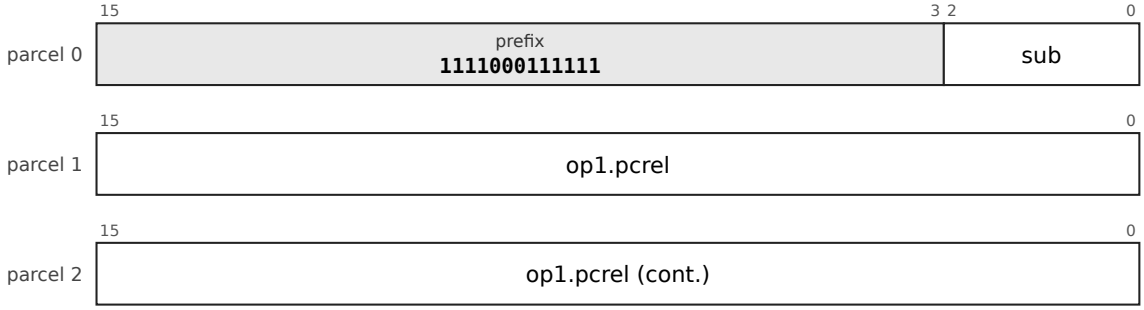
BCC: Branch on condition

Mnemonics: BR, BNE, BEQ, BPL, BMI, BVC, BVS, BCC, BCS, BGE, BLT, BGT, BLE, BHI, BLOS (aliases: BHIS, BLO)

Syntax

```
BR target
BNE target
BEQ target ... and the rest
```

Encoding



The chart is the leaf `16-BRF-1`, BRF, prefix `1111000111111`. `op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes). `sub` (3 bits) selects the mnemonic: `br` 000, `bne` 001, `beq` 010, `bpl` 011, `bmi` 100, `bvc` 101, `bvs` 110, `bcc` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-BRF-1`, `16-BR2-16-a-1`, `16-BR1-6-a-1`, `16-BR1-8-a-1`, `16-BR1-9-a-1`, `16-BR2-16-b-1`, `16-BR1-6-b-1`, `16-BR1-7-b-1`, `16-BR2-16-c-1`, `16-BR1-6-c-1`, `16-BRF-2`, `16-BR2-16-c-2`, `16-BR1-6-c-2`, `16-BRF-3`, `16-BRF-4`.

Operation

```
if cond(c) then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None; a branch to an unmapped or non-executable address faults on the fetch of the target.

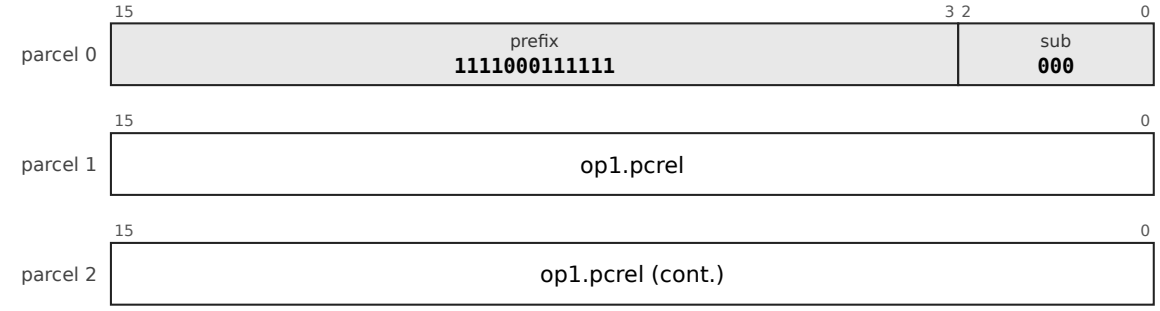
Condition numbers: `al` 0 (`BR`), `ne` 1, `eq` 2, `pl` 3, `mi` 4, `vc` 5, `vs` 6, `cc`/`hs` 7, `cs`/`lo` 8, `ge` 9, `lt` 10, `gt` 11, `le` 12, `hi` 13, `los` 14. Conditions on the flags: `ne` `!Z`; `eq` `Z`; `pl` `!N`; `mi` `N`; `vc` `!V`; `vs` `V`; `cc` `!C`; `cs` `C`; `ge` `N == V`; `lt` `N != V`; `gt` `!Z and N == V`; `le` `Z or N != V`; `hi` `!C and !Z`; `los` `C or Z`. The displacement counts 2-byte parcels; the reach of each form is in its chart.

Cf. 6.7 for `TRAP`, `RTI` and `RTT`.

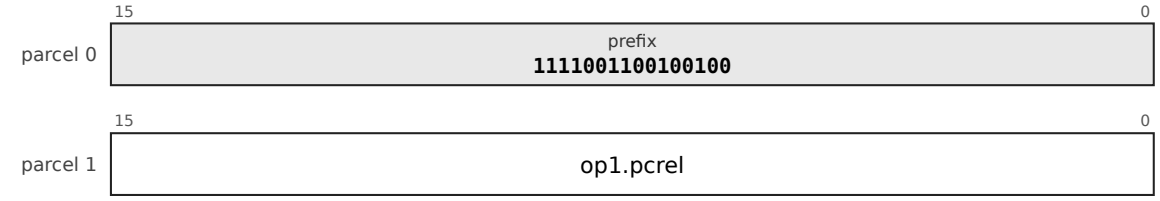
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BR

BR: BRF, prefix 1111000111111, sub-opcode 000; operands pc32-u2; 6 bytes



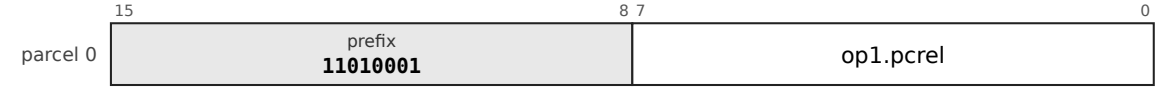
BR: BR2_16/a, prefix 1111001100100100; operands pc16-u2; 4 bytes



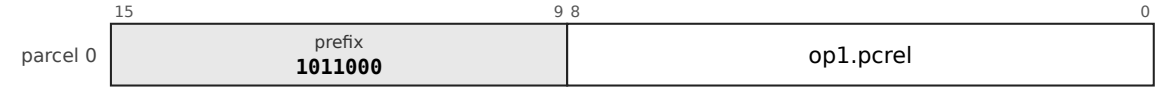
BR: BR1_6/a, prefix 1110100110; operands pc6-u2; 2 bytes



BR: BR1_8/a, prefix 11010001; operands pc8-u2; 2 bytes



BR: BR1_9/a, prefix 1011000; operands pc9-u2; 2 bytes

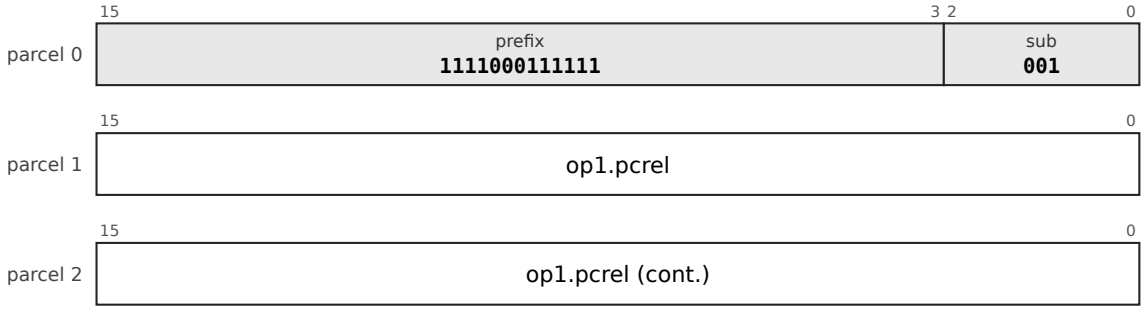



```
BR target
```

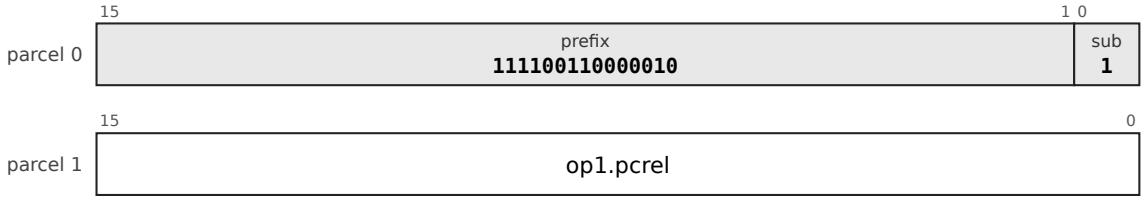
Operation, flags and exceptions as for the entry BCC above; 5 layout forms, each a chart of its own (above).

BNE

BNE: BRF, prefix 1111000111111, sub-opcode 001; operands pc32-u2; 6 bytes



BNE: BR2_16/b, prefix 111100110000010, sub-opcode 1; operands pc16-u2; 4 bytes



BNE: BR1_6/b, prefix 111000101, sub-opcode 1; operands pc6-u2; 2 bytes



BNE: BR1_7/b, prefix 11010111, sub-opcode 1; operands pc7-u2; 2 bytes

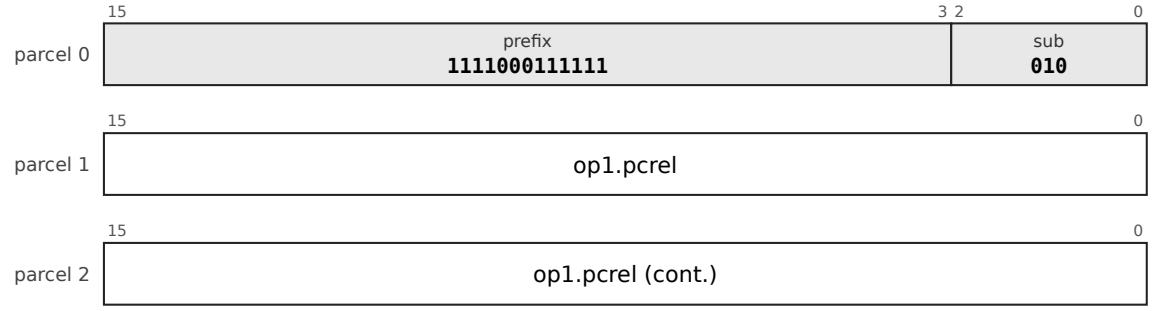



```
BNE target
```

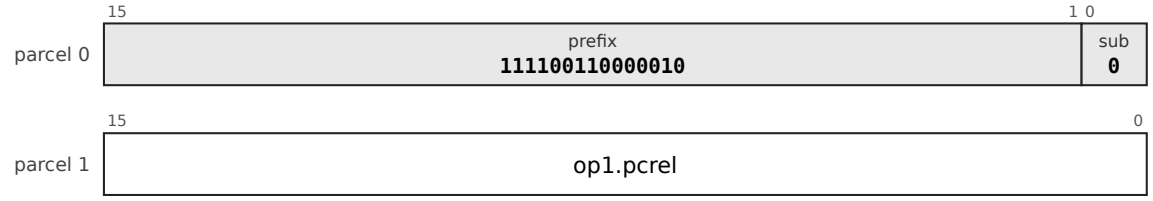
Operation, flags and exceptions as for the entry BCC above; 4 layout forms, each a chart of its own (above).

BEQ

BEQ: BRF, prefix 1111000111111, sub-opcode 010; operands pc32-u2; 6 bytes



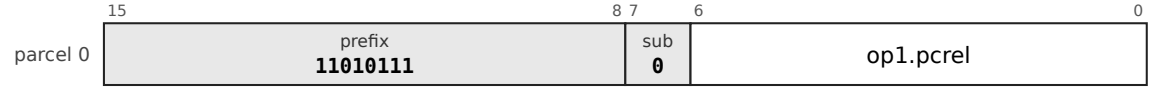
BEQ: BR2_16/b, prefix 111100110000010, sub-opcode 0; operands pc16-u2; 4 bytes



BEQ: BR1_6/b, prefix 111000101, sub-opcode 0; operands pc6-u2; 2 bytes



BEQ: BR1_7/b, prefix 11010111, sub-opcode 0; operands pc7-u2; 2 bytes

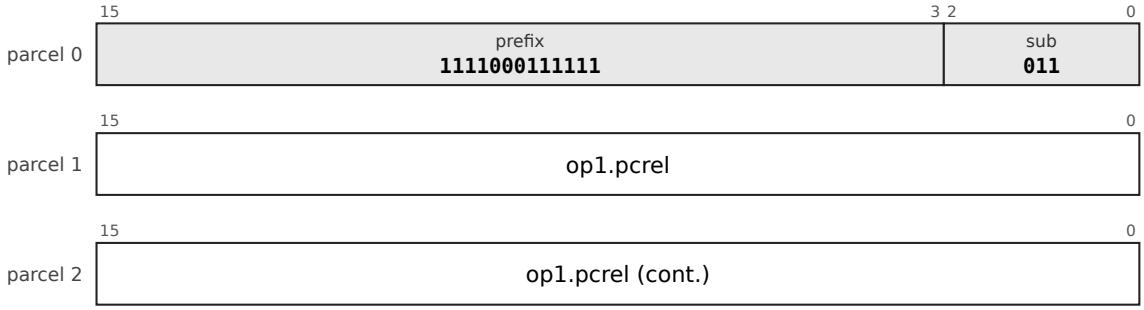



```
BEQ target ... and the rest
```

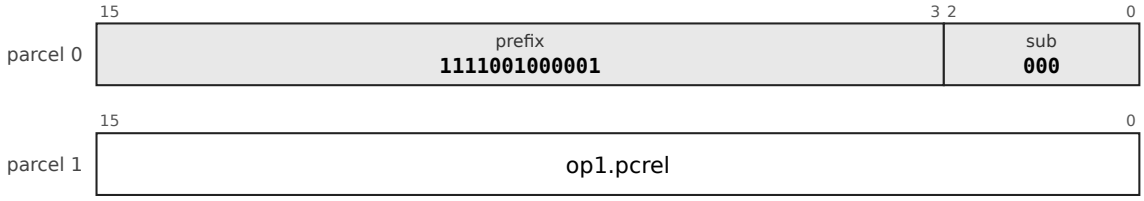
Operation, flags and exceptions as for the entry BCC above; 4 layout forms, each a chart of its own (above).

BPL

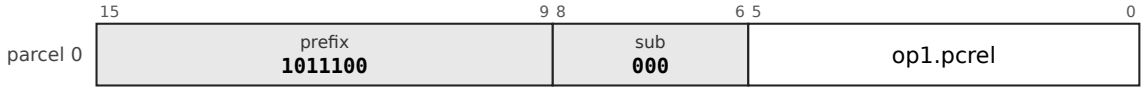
BPL: BRF, prefix 1111000111111, sub-opcode 011; operands pc32-u2; 6 bytes



BPL: BR2_16/c, prefix 1111001000001, sub-opcode 000; operands pc16-u2; 4 bytes



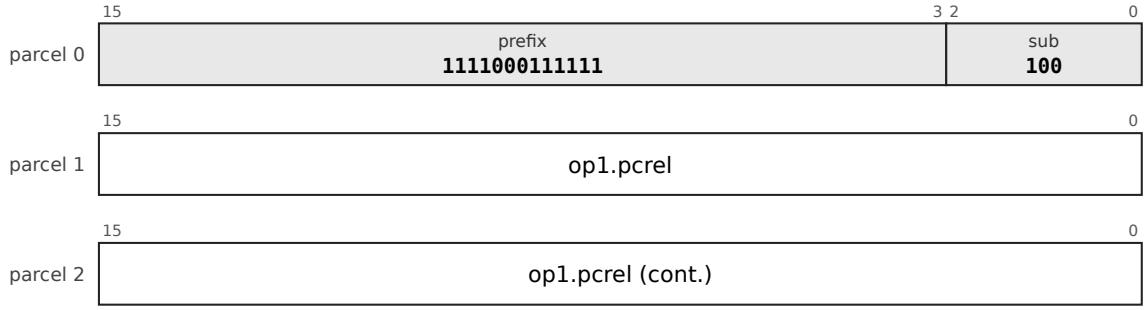
BPL: BR1_6/c, prefix 1011100, sub-opcode 000; operands pc6-u2; 2 bytes



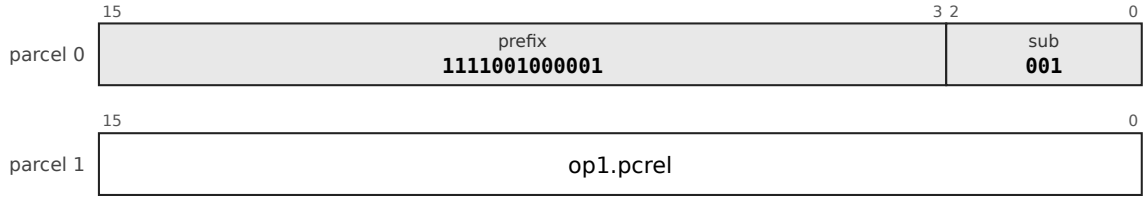
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BMI

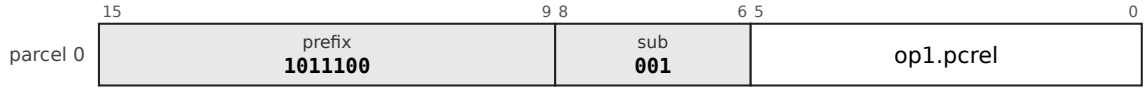
BMI: BRF, prefix 1111000111111, sub-opcode 100; operands pc32-u2; 6 bytes



BMI: BR2_16/c, prefix 1111001000001, sub-opcode 001; operands pc16-u2; 4 bytes



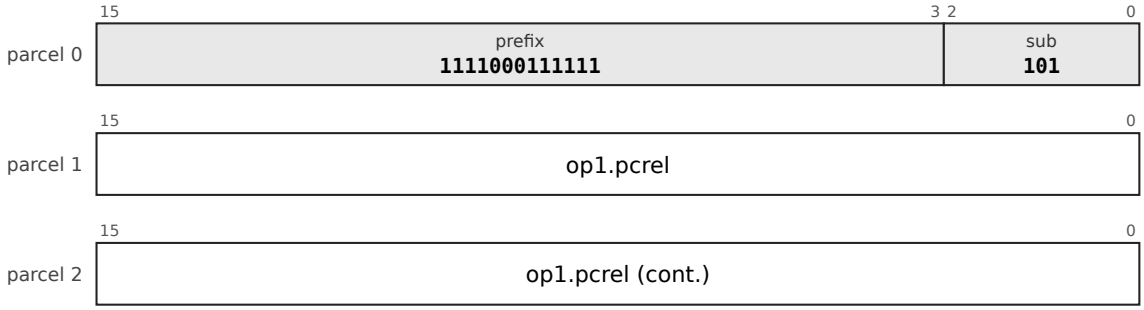
BMI: BR1_6/c, prefix 1011100, sub-opcode 001; operands pc6-u2; 2 bytes



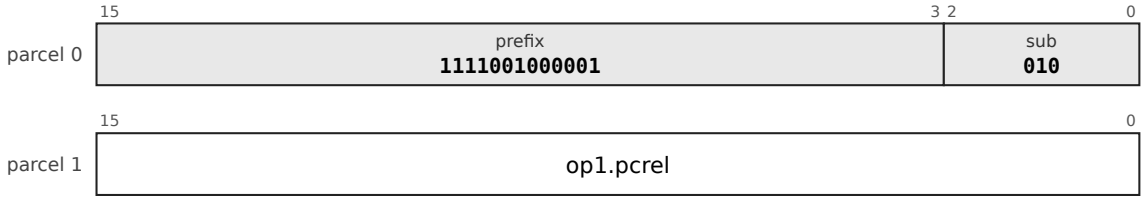
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BVC

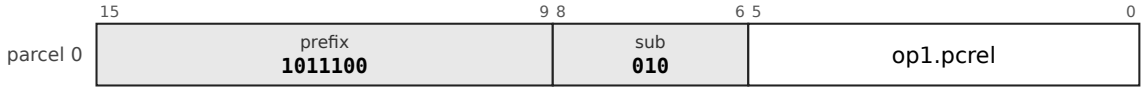
BVC: BRF, prefix 1111000111111, sub-opcode 101; operands pc32-u2; 6 bytes



BVC: BR2_16/c, prefix 1111001000001, sub-opcode 010; operands pc16-u2; 4 bytes



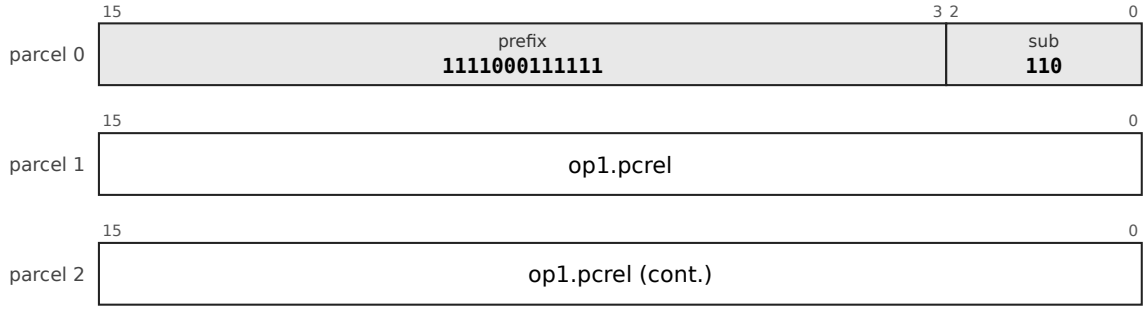
BVC: BR1_6/c, prefix 1011100, sub-opcode 010; operands pc6-u2; 2 bytes



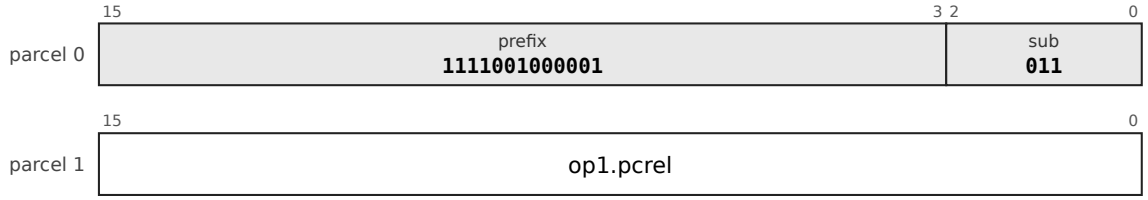
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BVS

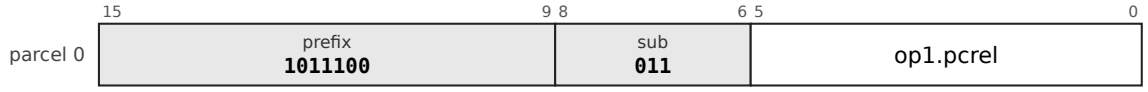
BVS: BRF, prefix 1111000111111, sub-opcode 110; operands pc32-u2; 6 bytes



BVS: BR2_16/c, prefix 1111001000001, sub-opcode 011; operands pc16-u2; 4 bytes



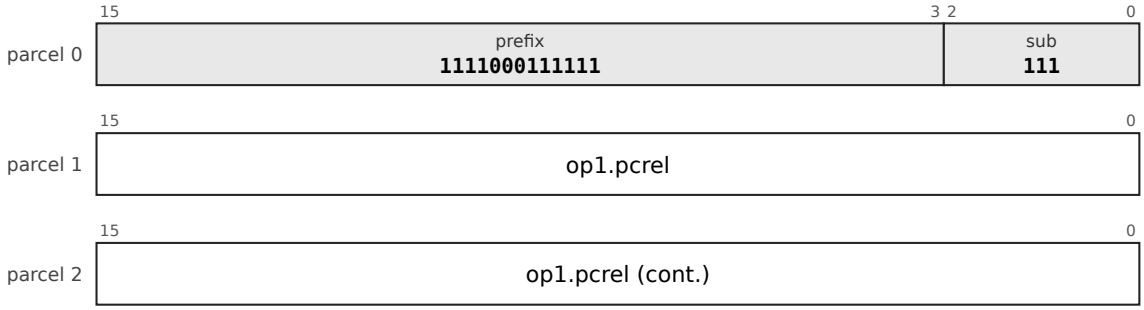
BVS: BR1_6/c, prefix 1011100, sub-opcode 011; operands pc6-u2; 2 bytes



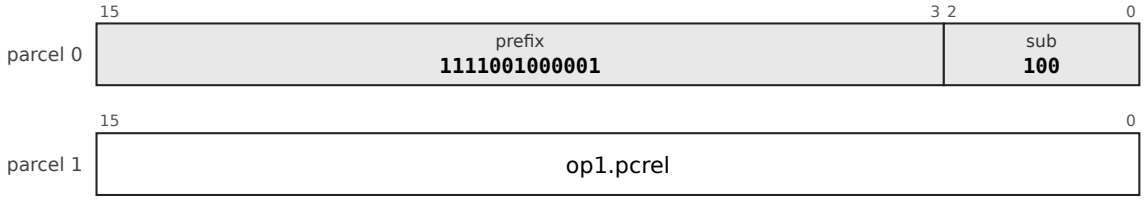
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BCC

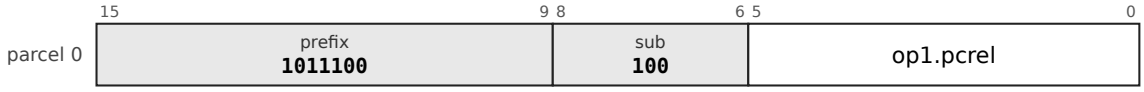
BCC: BRF, prefix 1111000111111, sub-opcode 111; operands pc32-u2; 6 bytes



BCC: BR2_16/c, prefix 1111001000001, sub-opcode 100; operands pc16-u2; 4 bytes



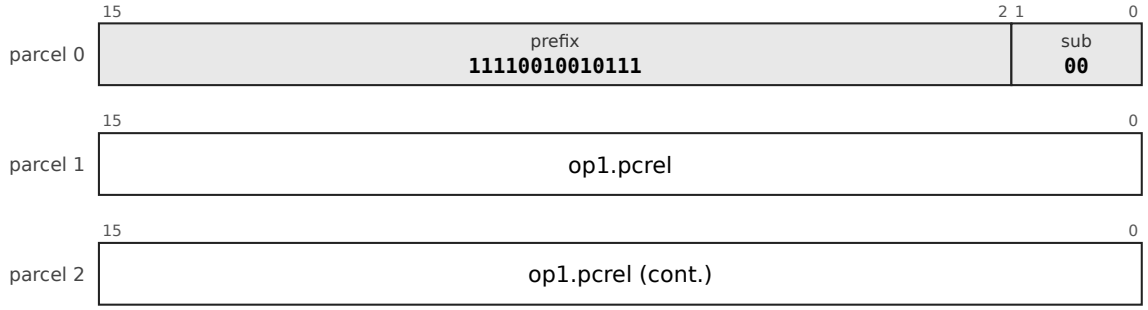
BCC: BR1_6/c, prefix 1011100, sub-opcode 100; operands pc6-u2; 2 bytes



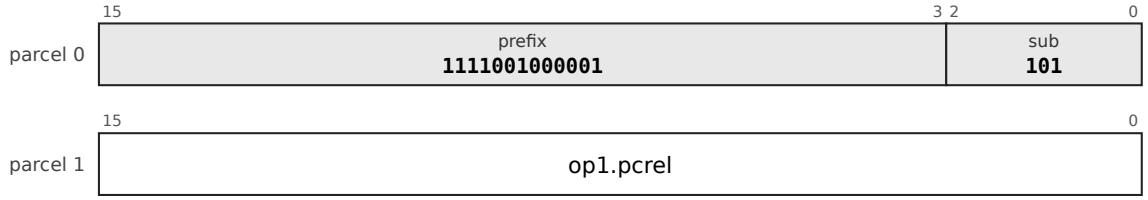
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BCS

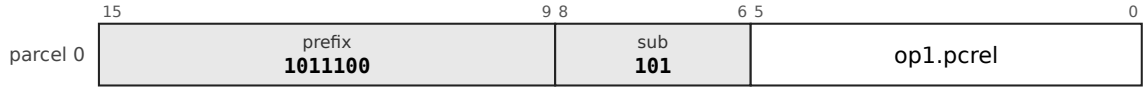
BCS: BRF, prefix 11110010010111, sub-opcode 00; operands pc32-u2; 6 bytes



BCS: BR2_16/c, prefix 1111001000001, sub-opcode 101; operands pc16-u2; 4 bytes



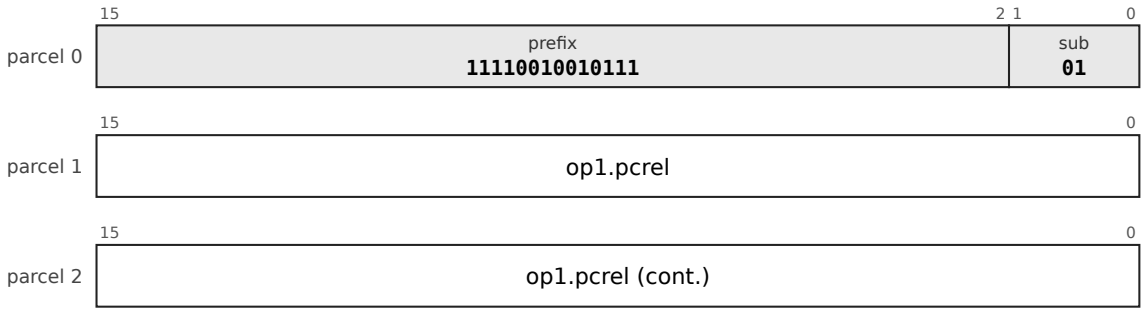
BCS: BR1_6/c, prefix 1011100, sub-opcode 101; operands pc6-u2; 2 bytes



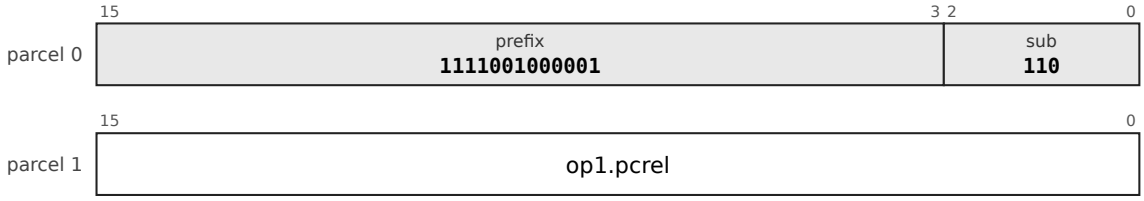
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BGE

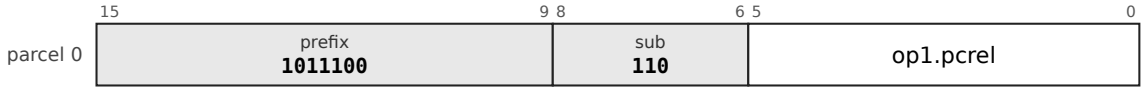
BGE: BRF, prefix 11110010010111, sub-opcode 01; operands pc32-u2; 6 bytes



BGE: BR2_16/c, prefix 1111001000001, sub-opcode 110; operands pc16-u2; 4 bytes



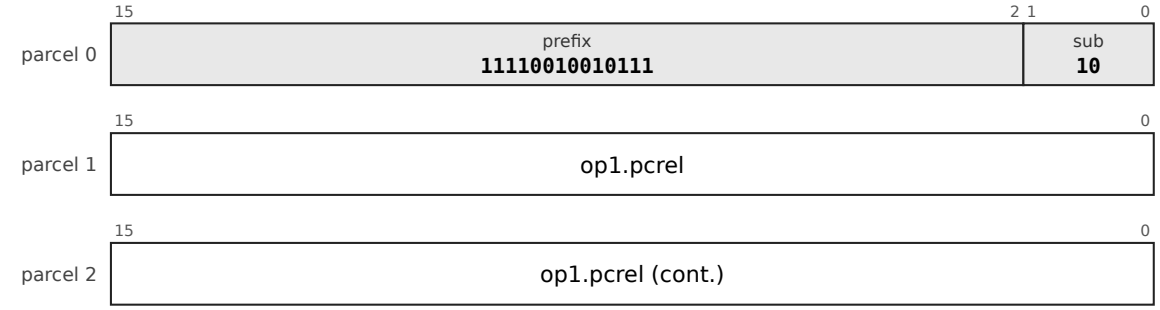
BGE: BR1_6/c, prefix 1011100, sub-opcode 110; operands pc6-u2; 2 bytes



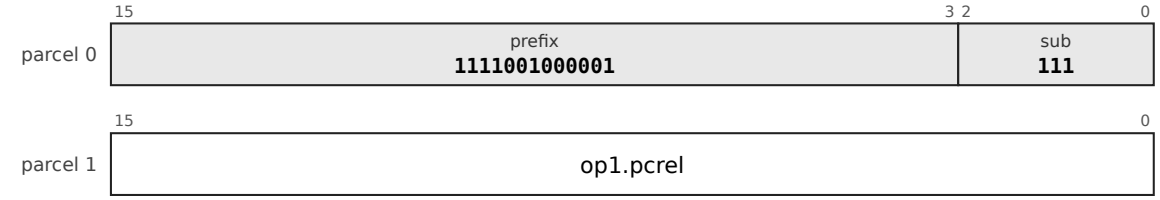
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLT

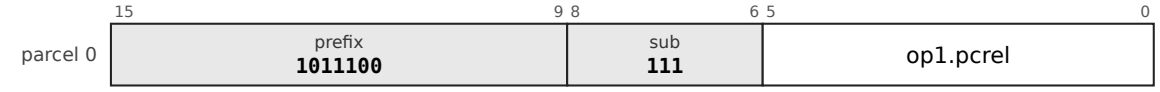
BLT: BRF, prefix 11110010010111, sub-opcode 10; operands pc32-u2; 6 bytes



BLT: BR2_16/c, prefix 1111001000001, sub-opcode 111; operands pc16-u2; 4 bytes



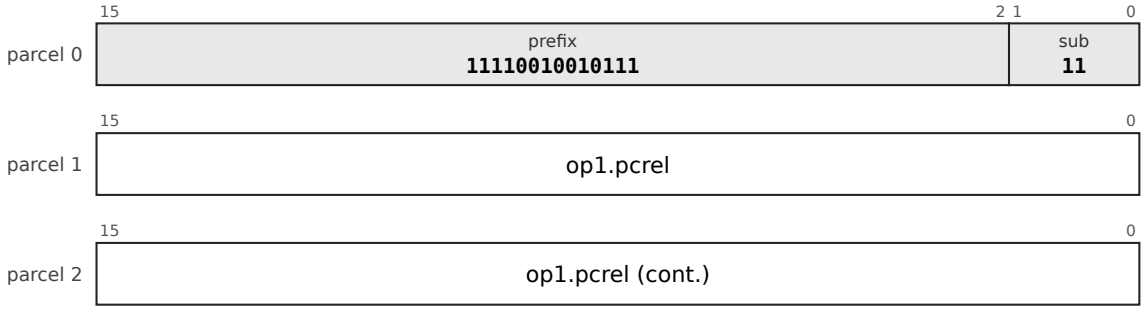
BLT: BR1_6/c, prefix 1011100, sub-opcode 111; operands pc6-u2; 2 bytes



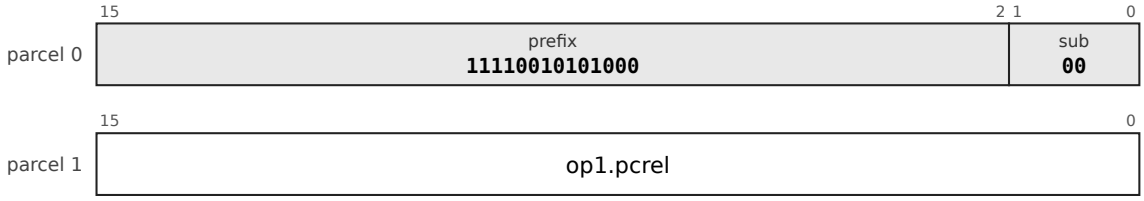
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BGT

BGT: BRF, prefix 11110010010111, sub-opcode 11; operands pc32-u2; 6 bytes



BGT: BR2_16/c, prefix 11110010101000, sub-opcode 00; operands pc16-u2; 4 bytes



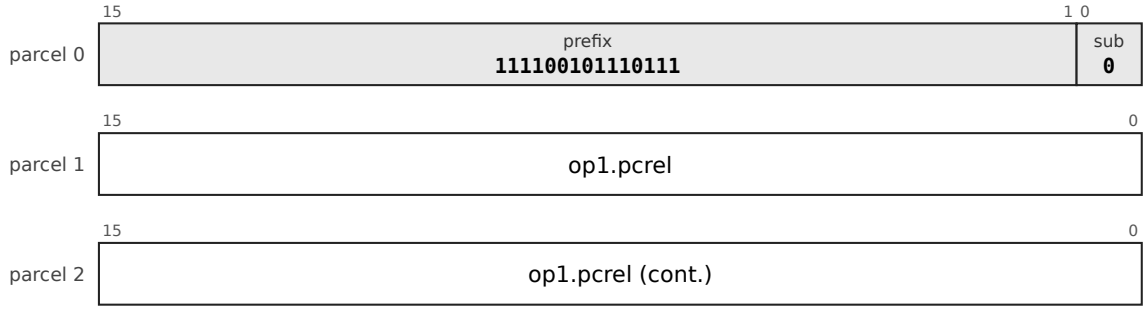
BGT: BR1_6/c, prefix 11010101, sub-opcode 00; operands pc6-u2; 2 bytes



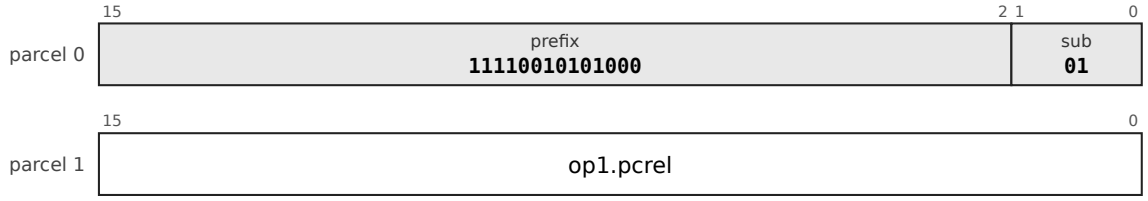
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLE

BLE: BRF, prefix 111100101110111, sub-opcode 0; operands pc32-u2; 6 bytes



BLE: BR2_16/c, prefix 11110010101000, sub-opcode 01; operands pc16-u2; 4 bytes



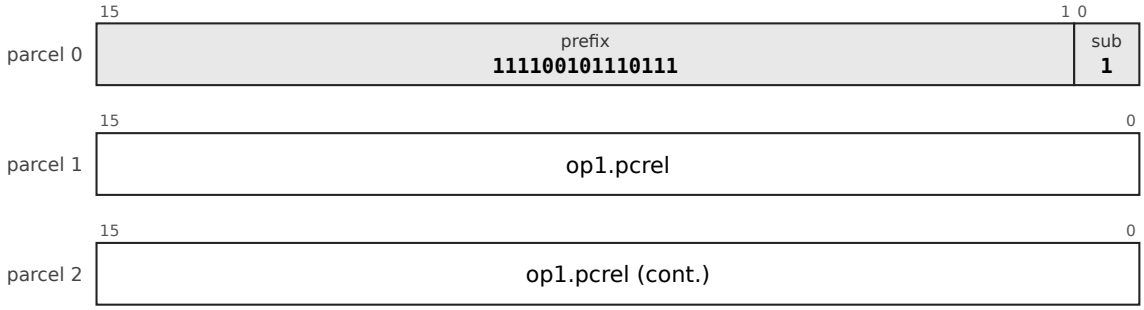
BLE: BR1_6/c, prefix 11010101, sub-opcode 01; operands pc6-u2; 2 bytes



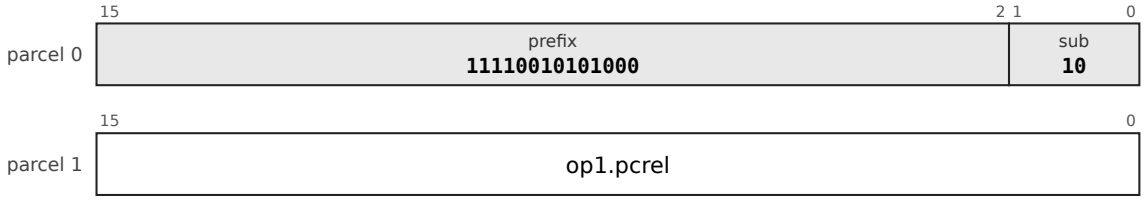
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BHI

BHI: BRF, prefix 111100101110111, sub-opcode 1; operands pc32-u2; 6 bytes



BHI: BR2_16/c, prefix 11110010101000, sub-opcode 10; operands pc16-u2; 4 bytes



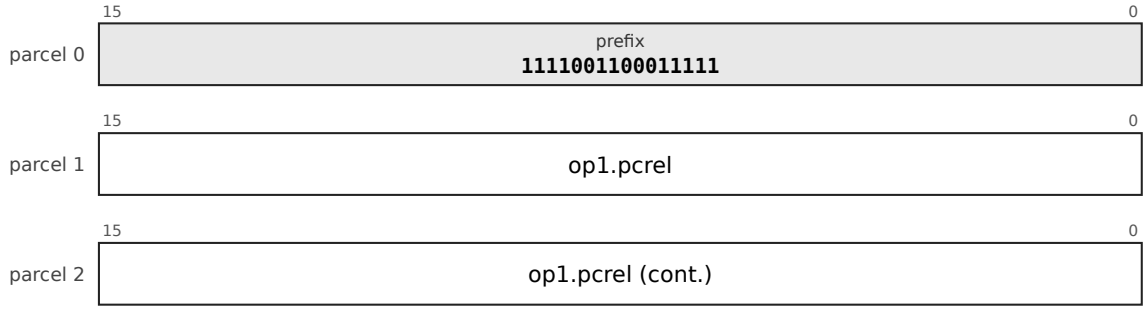
BHI: BR1_6/c, prefix 11010101, sub-opcode 10; operands pc6-u2; 2 bytes



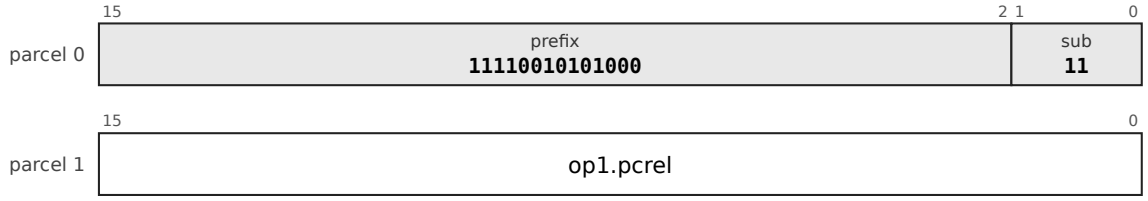
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLOS

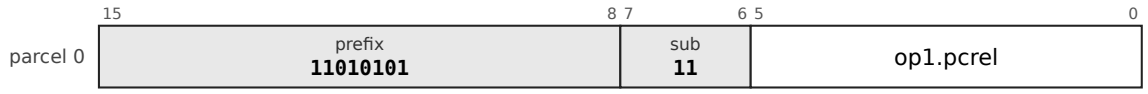
BLOS: BRF, prefix 1111001100011111; operands pc32-u2; 6 bytes



BLOS: BR2_16/c, prefix 11110010101000, sub-opcode 11; operands pc16-u2; 4 bytes



BLOS: BR1_6/c, prefix 11010101, sub-opcode 11; operands pc6-u2; 2 bytes



Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BHIS

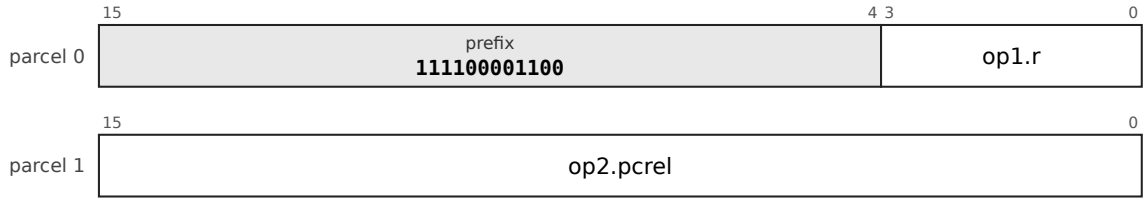
An alias of BCC: the assembler writes it as that word.

BLO

An alias of BCS: the assembler writes it as that word.

SOB: Subtract one and branch

SOB: SOB, prefix 111100001100; operands reg4, pc16-u2; 4 bytes

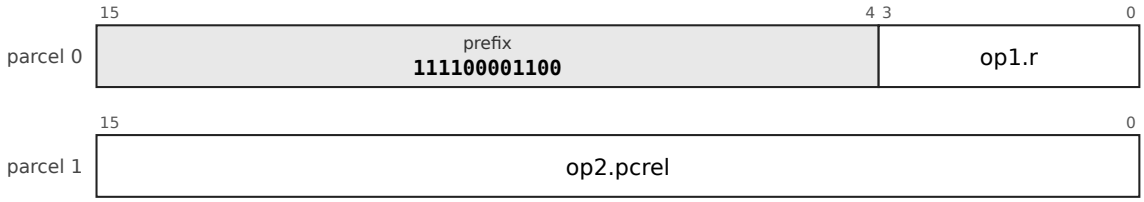


Mnemonics: SOB

Syntax

```
SOB Rn, target
```

Encoding



The chart is the leaf `16-SOB-1`, SOB, prefix `111100001100`. `op1` is `reg4`: register, 4 bits. `op2` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOB-1`.

Operation

```
Rn ← Rn - 1
if Rn ≠ 0 then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None.

From the PDP-11, where it could branch only backward; here the signed displacement goes either way. Four bytes at least.

Why it is here. Taken from the PDP-11. The PDP-11's could only branch backward (a 6-bit unsigned displacement); this one branches either way. Against the `cbnz` idiom of other architectures it does not save bytes (`DEC` plus `BNE` is also four in the original encoding) but it saves an instruction issue, a cycle per iteration on a machine that does not pipeline: same size, one instruction fewer (static `dec; bne` pairs exist).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

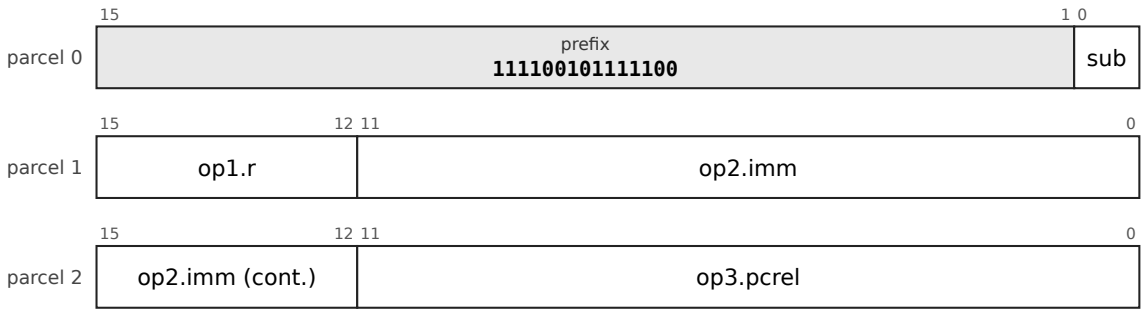
CBCC: Compare and branch on condition

Mnemonics: `CBNE`, `CBEQ`, `CBPL`, `CBMI`, `CBVC`, `CBVS`, `CBCC`, `CBCS`, `CBGE`, `CBLT`, `CBGT`, `CBLE`, `CBHI`, `CBLOS` (aliases: `CBHIS`, `CBLO`)

Syntax

```
CBNE Rd, #imm, target
CBEQ Ra, Rb, target ... and the rest
```

Encoding



The chart is the leaf `16-CB3I16-12-a-1`, CB3I16_12/a, prefix `111100101111100`. `op1` is `reg4`: register, 4 bits. `op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767. `op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes). `sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-CB3I16-12-a-1`, `16-CBR8-a-1`, `16-CBZ2-12-a-1`, `16-CBI5-8-a-1`, `16-CBI6-8-a-1`, `16-CBI10-8-a-1`, `16-CB3I8-20-a-1`, `16-CBZ1-4-a-1`, `16-`

CBI5-10-a-1, 16-CB3R24-a-1, 16-CBZ1-5-a-1, 16-CBI8-10-a-1, 16-CBR10-a-1, 16-CBI4-12-a-1, 16-CBZ1-7-a-1, 16-CBZ2-14-a-1, 16-CBI6-12-a-1, 16-CBDI-1, 16-CBDR-1, 16-CBR8-c-1, 16-CB3I16-12-c-1, 16-CBI4-8-c-1, 16-CBI6-8-c-1, 16-CB3R24-c-1, 16-CBR10-c-1, 16-CBI8-8-c-1, 16-CBI4-10-c-1, 16-CB3I8-20-c-1, 16-CBI10-8-c-1, 16-CBI6-10-c-1, 16-CB3I16-12-b-1, 16-CBR8-b-1, 16-CBI8-8-b-1, 16-CBZ2-12-b-1, 16-CB3I8-20-b-1, 16-CB3R24-b-1, 16-CBI4-10-b-1.

Operation

```
tmp ← Rd - (imm or Rb)         ; as CMP, for the decision only
if cond(c) on that comparison then PC ← address of this instruction + 2 × disp
```

Flags None written: the instruction computes the comparison for its own decision and leaves N Z V C as they were.

Exceptions None.

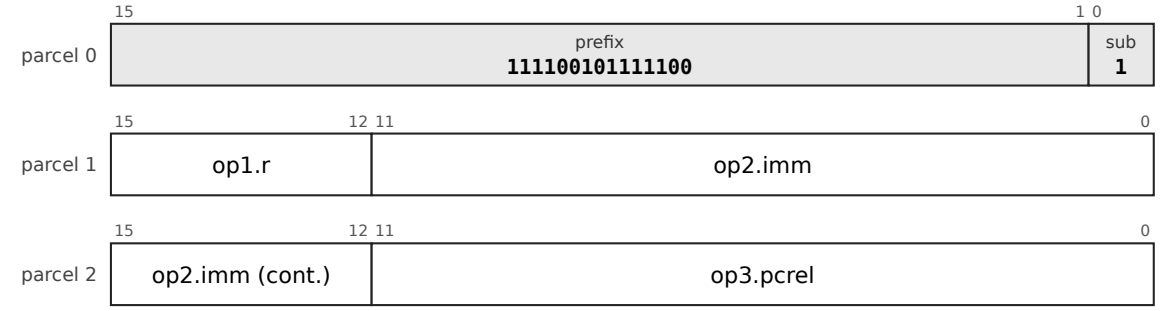
The family has several forms for each condition (the layouts of this entry): the second operand is a register or an immediate of some width, and the reach differs. A cbCC that no form reaches is assembled by as64 as CMP and bCC, which **writes the flags** (errata 87, assumption 6; the integration measured 4 such sites in 34,387 for pdp64/16 and 17 in 34,524 for pdp64/32, errata 90).

Why it is here. A fused compare-and-branch, estimated with RORI32 at 22 KB (1.3%) of the image of the original encoding. There the wide-immediate form removes 11,735 instructions (3.4%) and 1,828 bytes, and the register-register compare-and-branch removes 4,631 instructions (1.4%) and 142 bytes. It is not a replacement for flags: LoongArch and RISC-V chose fused compare-and-branch instead of flags, and this architecture declines that (9.20) because it needs a comparator in the branch unit and a third read port; here the fused form is added beside the flags. CBCC writes no flags.

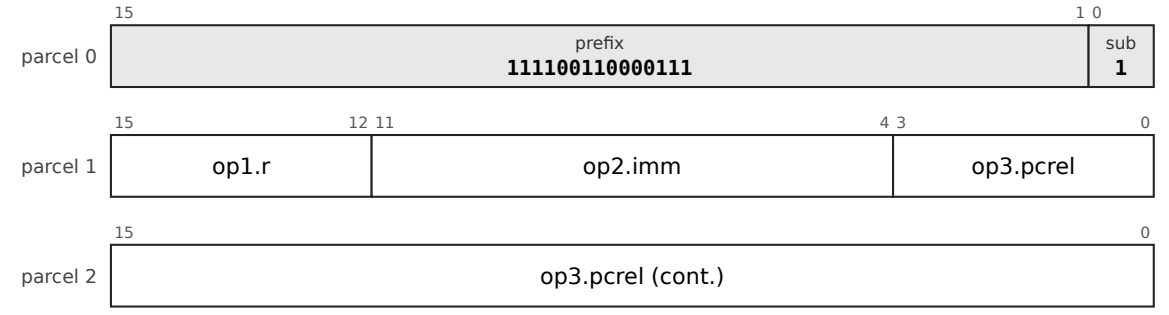
Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

CBNE

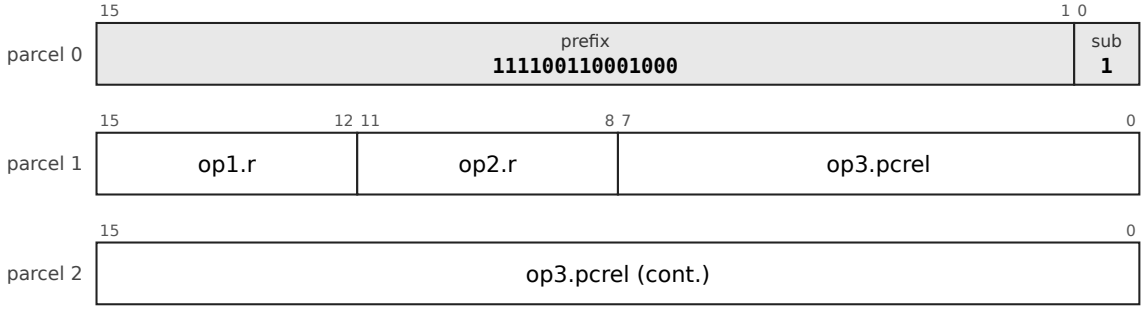
CBNE: CB3I16_12/a, prefix 111100101111100, sub-opcode 1; operands reg4, imm16s, pc12-u



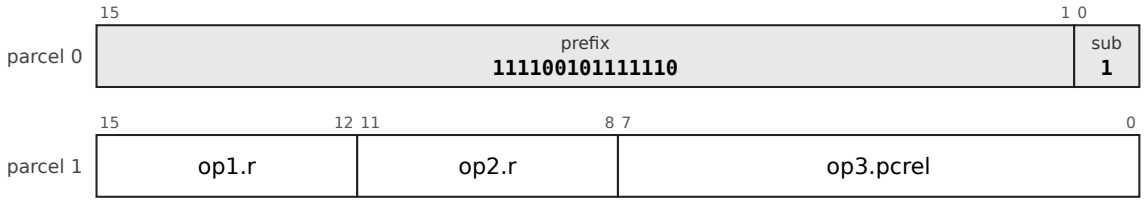
CBNE: CB3I8_20/a, prefix 111100110000111, sub-opcode 1; operands reg4, imm8s, pc20-u2;



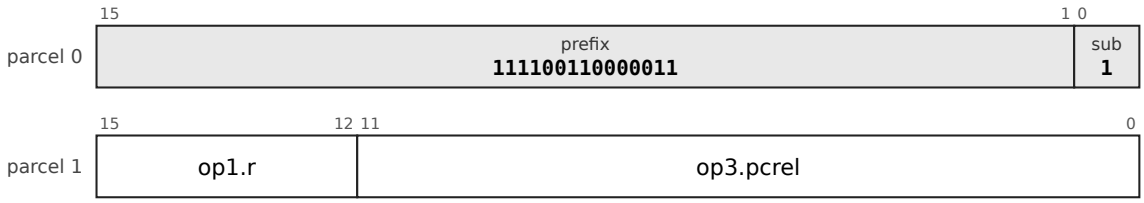
CBNE: CB3R24/a, prefix 111100110001000, sub-opcode 1; operands reg4, reg4, pc24-u2; 6 by



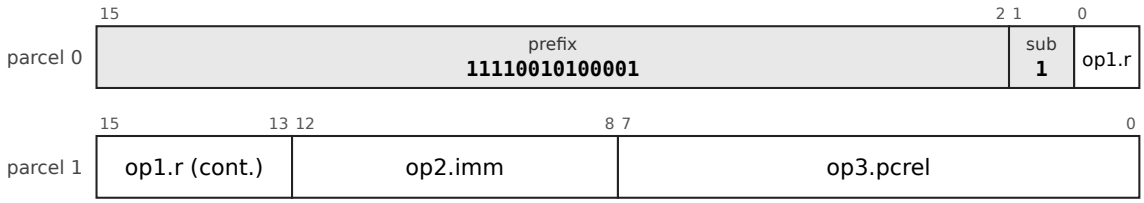
CBNE: CBR8/a, prefix 111100101111110, sub-opcode 1; operands reg4, reg4, pc8-u2; 4 bytes



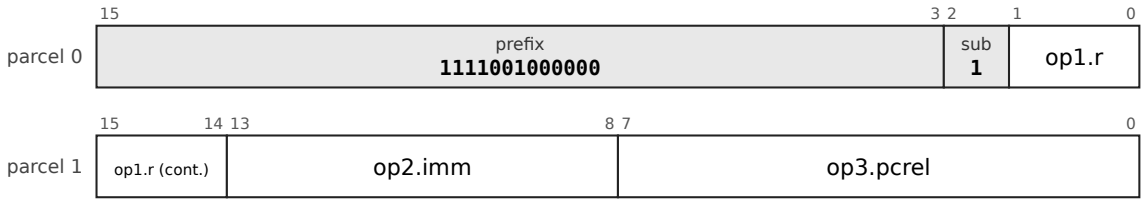
CBNE: CBZ2_12/a, prefix 111100110000011, sub-opcode 1; operands reg4, k-imm0, pc12-u2;



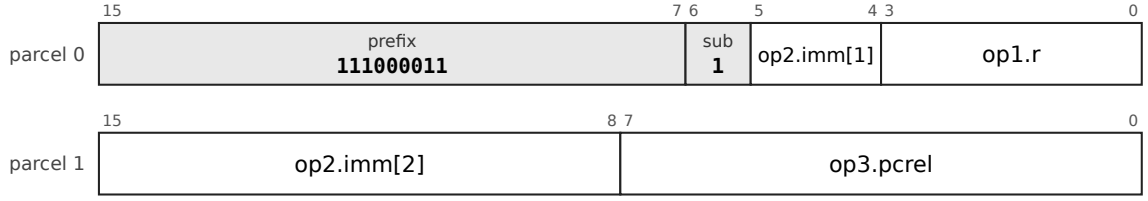
CBNE: CBI5_8/a, prefix 11110010100001, sub-opcode 1; operands reg4, imm5s, pc8-u2; 4 byt



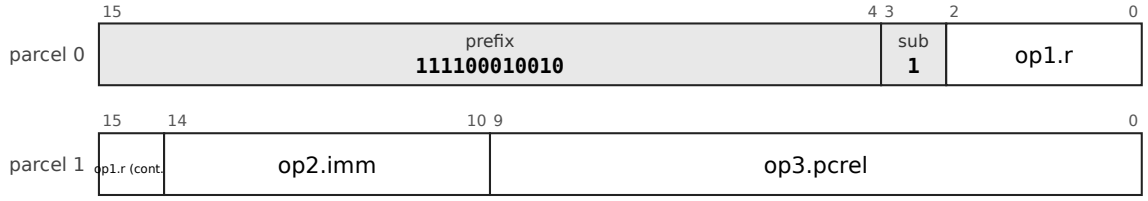
CBNE: CBI6_8/a, prefix 1111001000000, sub-opcode 1; operands reg4, imm6s, pc8-u2; 4 byte



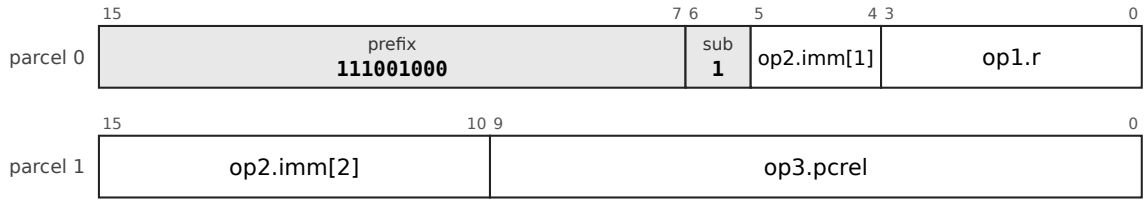
CBNE: CBI10_8/a, prefix 111000011, sub-opcode 1; operands reg4, imm10s, pc8-u2; 4 bytes



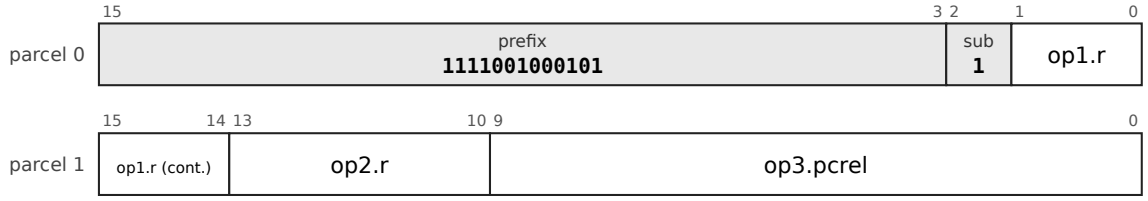
CBNE: CBI5_10/a, prefix 111100010010, sub-opcode 1; operands reg4, imm5s, pc10-u2; 4 byt



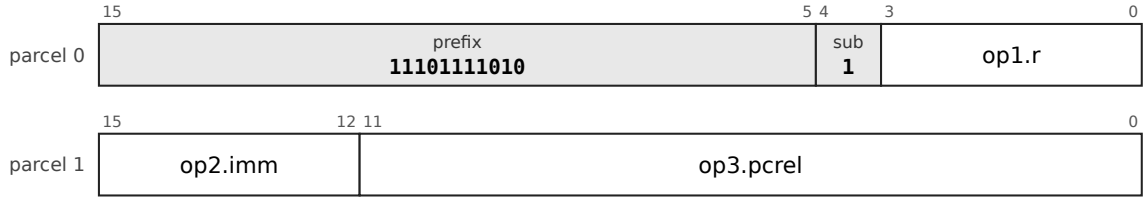
CBNE: CBI8_10/a, prefix 111001000, sub-opcode 1; operands reg4, imm8s, pc10-u2; 4 bytes



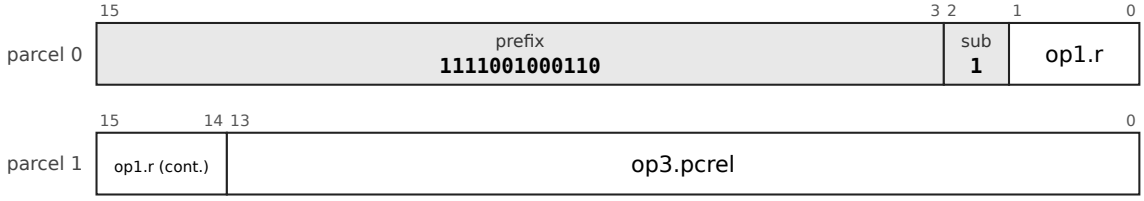
CBNE: CBR10/a, prefix 1111001000101, sub-opcode 1; operands reg4, reg4, pc10-u2; 4 bytes



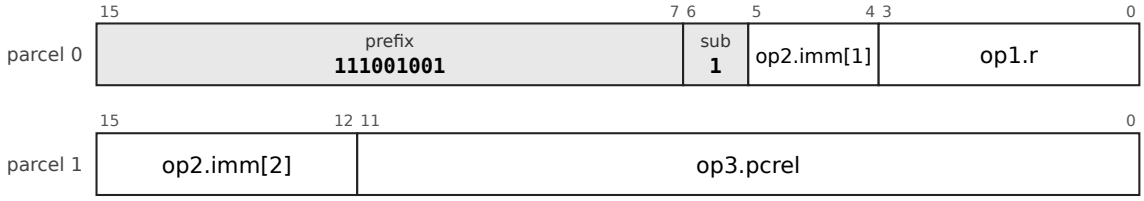
CBNE: CBI4_12/a, prefix 11101111010, sub-opcode 1; operands reg4, imm4s, pc12-u2; 4 byte



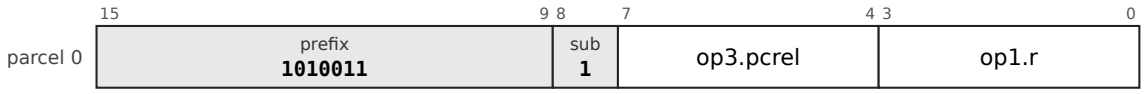
CBNE: CBZ2_14/a, prefix 1111001000110, sub-opcode 1; operands reg4, k-imm0, pc14-u2; 4 l



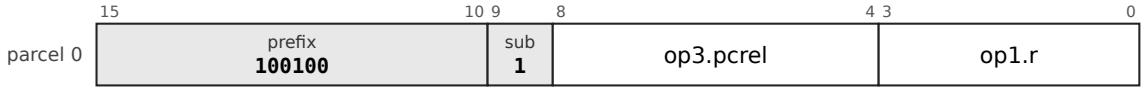
CBNE: CBI6_12/a, prefix 111001001, sub-opcode 1; operands reg4, imm6s, pc12-u2; 4 bytes



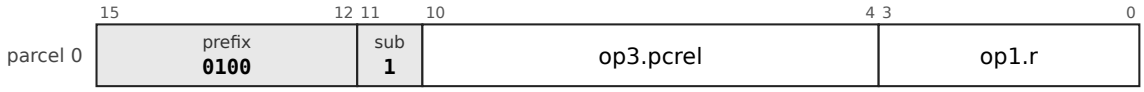
CBNE: CBZ1_4/a, prefix 1010011, sub-opcode 1; operands reg4, k-imm0, pc4-u2; 2 bytes



CBNE: CBZ1_5/a, prefix 100100, sub-opcode 1; operands reg4, k-imm0, pc5-u2; 2 bytes



CBNE: CBZ1_7/a, prefix 0100, sub-opcode 1; operands reg4, k-imm0, pc7-u2; 2 bytes

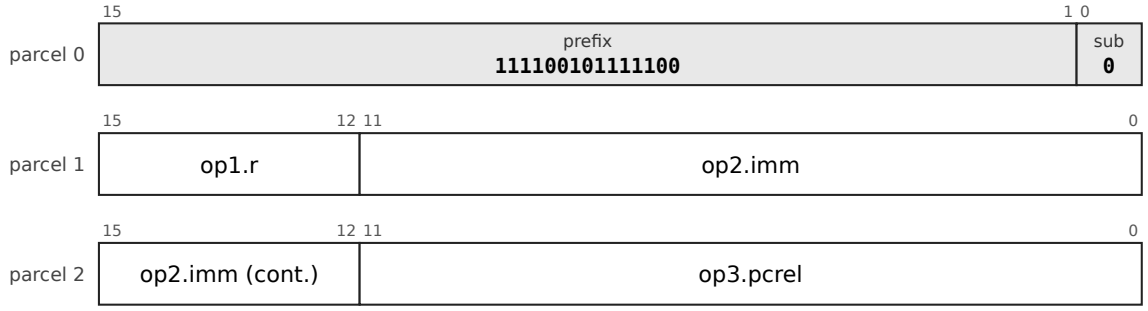



```
CBNE Rd, #imm, target
```

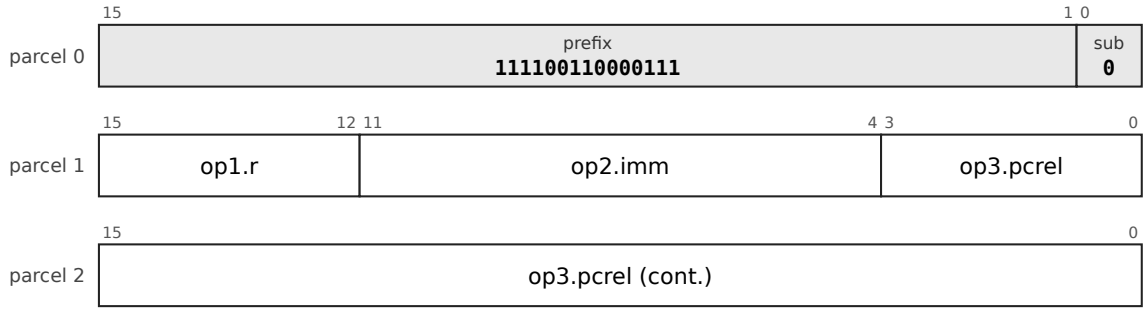
Operation, flags and exceptions as for the entry CBCC above; 17 layout forms, each a chart of its own (above).

CBEQ

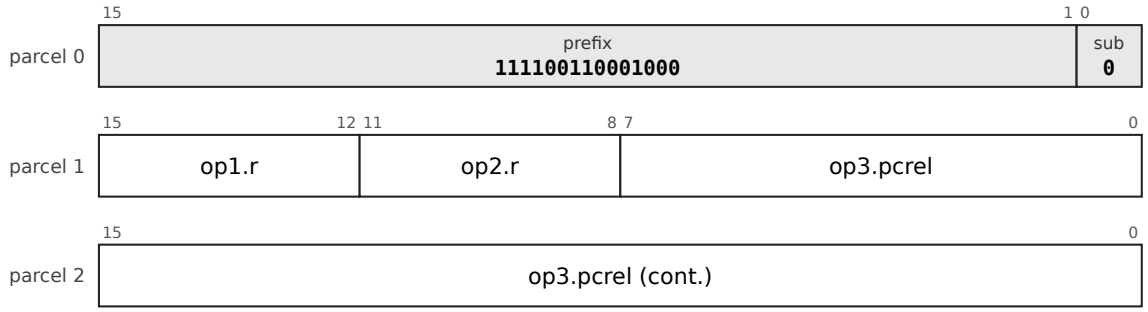
CBEQ: CB3I16_12/a, prefix 111100101111100, sub-opcode 0; operands reg4, imm16s, pc12-u



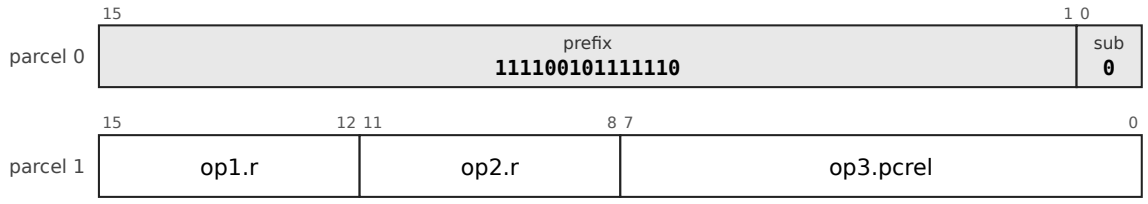
CBEQ: CB3I8_20/a, prefix 111100110000111, sub-opcode 0; operands reg4, imm8s, pc20-u2;



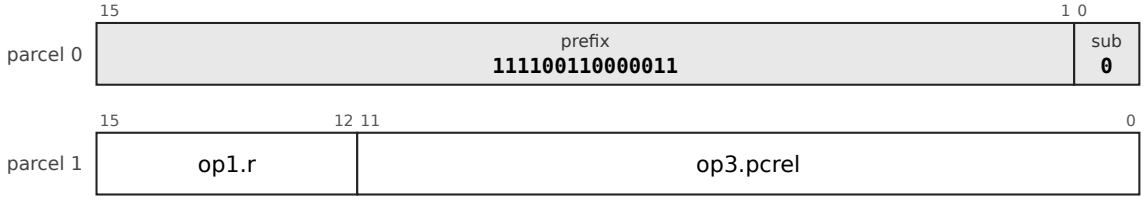
CBEQ: CB3R24/a, prefix 111100110001000, sub-opcode 0; operands reg4, reg4, pc24-u2; 6 by



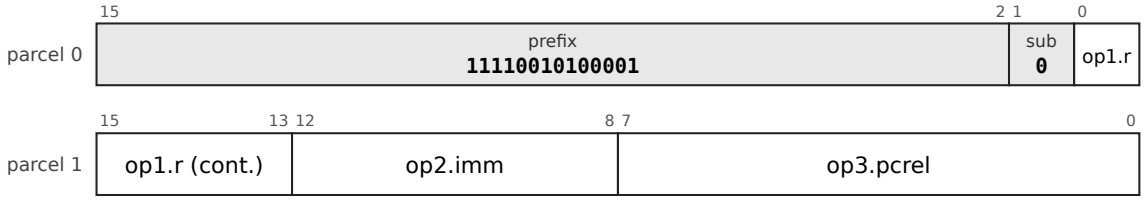
CBEQ: CBR8/a, prefix 111100101111110, sub-opcode 0; operands reg4, reg4, pc8-u2; 4 bytes



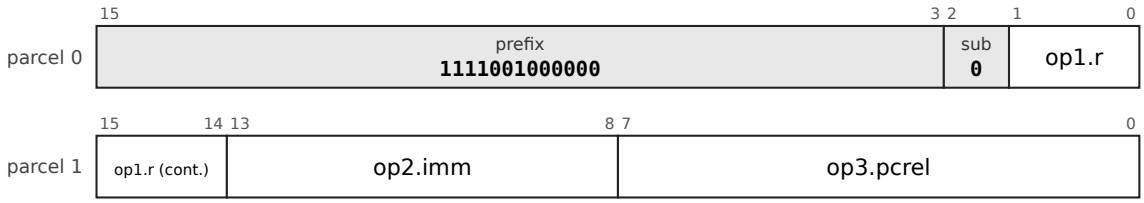
CBEQ: CBZ2_12/a, prefix 111100110000011, sub-opcode 0; operands reg4, k-imm0, pc12-u2;



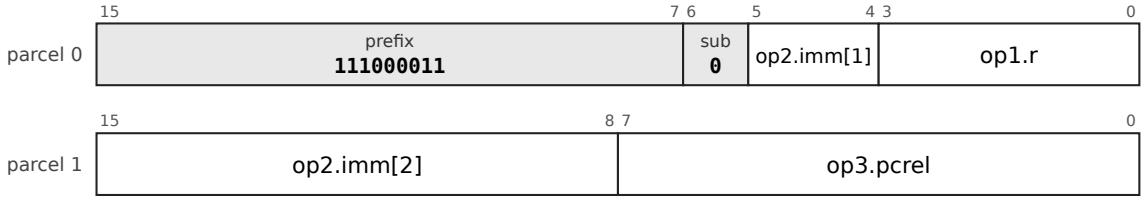
CBEQ: CBI5_8/a, prefix 11110010100001, sub-opcode 0; operands reg4, imm5s, pc8-u2; 4 byt



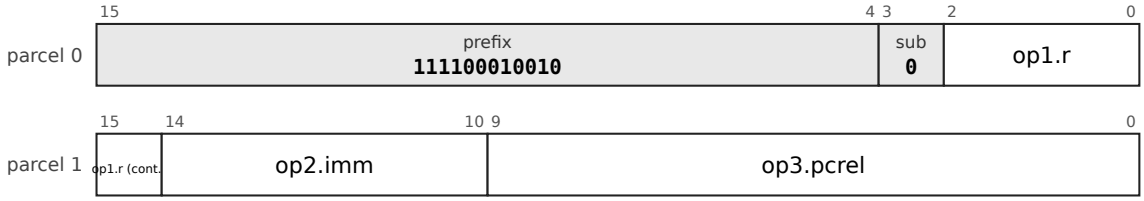
CBEQ: CBI6_8/a, prefix 1111001000000, sub-opcode 0; operands reg4, imm6s, pc8-u2; 4 byte



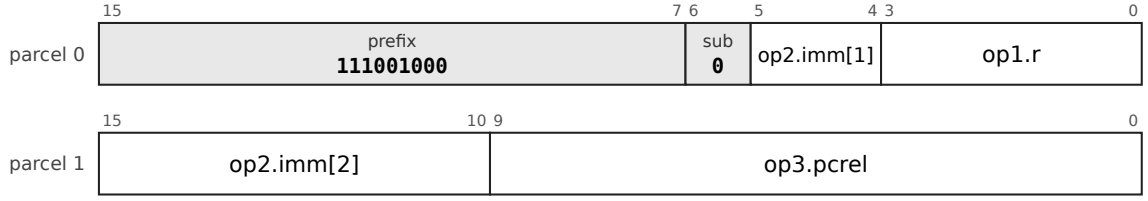
CBEQ: CBI10_8/a, prefix 111000011, sub-opcode 0; operands reg4, imm10s, pc8-u2; 4 bytes



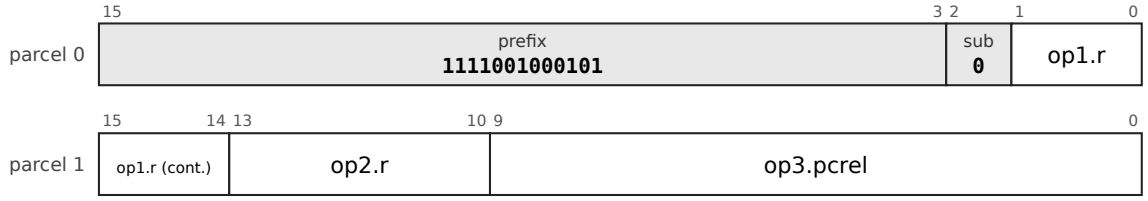
CBEQ: CBI5_10/a, prefix 111100010010, sub-opcode 0; operands reg4, imm5s, pc10-u2; 4 byt



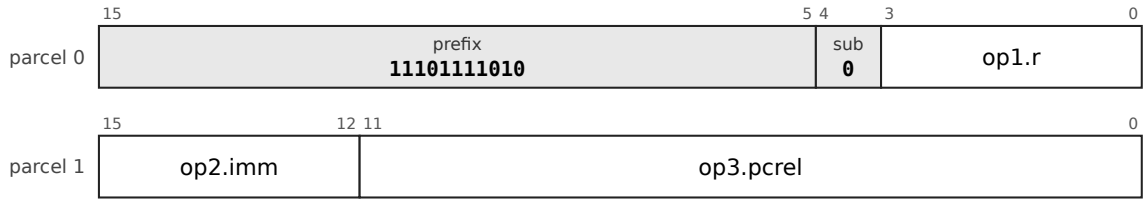
CBEQ: CBI8_10/a, prefix 111001000, sub-opcode 0; operands reg4, imm8s, pc10-u2; 4 bytes



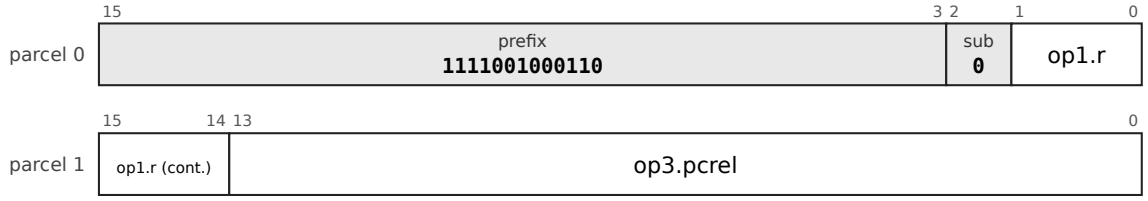
CBEQ: CBR10/a, prefix 1111001000101, sub-opcode 0; operands reg4, reg4, pc10-u2; 4 bytes



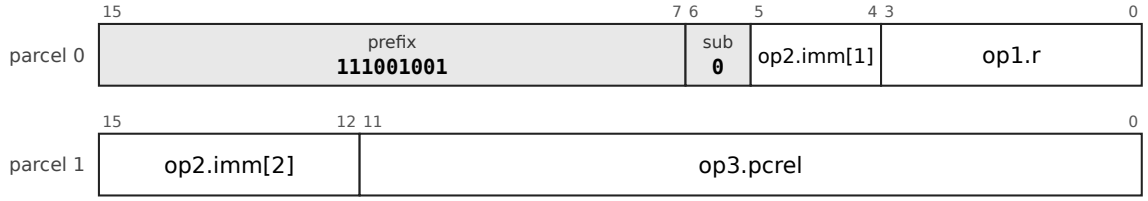
CBEQ: CBI4_12/a, prefix 11101111010, sub-opcode 0; operands reg4, imm4s, pc12-u2; 4 byte



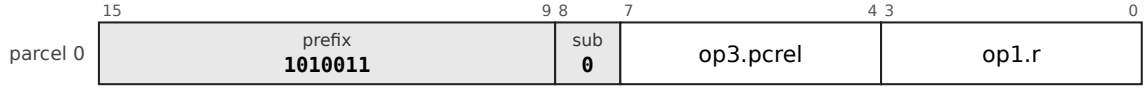
CBEQ: CBZ2_14/a, prefix 1111001000110, sub-opcode 0; operands reg4, k-imm0, pc14-u2; 4 l



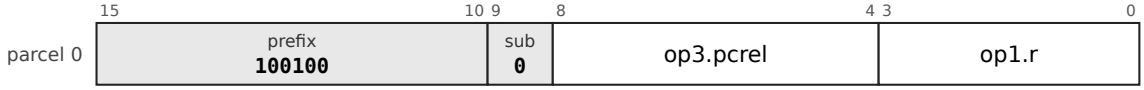
CBEQ: CBI6_12/a, prefix 111001001, sub-opcode 0; operands reg4, imm6s, pc12-u2; 4 bytes



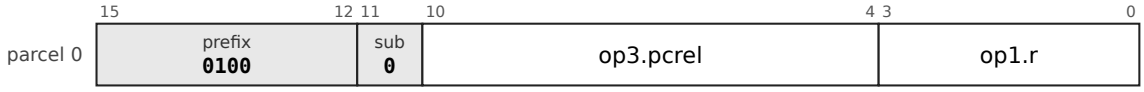
CBEQ: CBZ1_4/a, prefix 1010011, sub-opcode 0; operands reg4, k-imm0, pc4-u2; 2 bytes



CBEQ: CBZ1_5/a, prefix 100100, sub-opcode 0; operands reg4, k-imm0, pc5-u2; 2 bytes



CBEQ: CBZ1_7/a, prefix 0100, sub-opcode 0; operands reg4, k-imm0, pc7-u2; 2 bytes

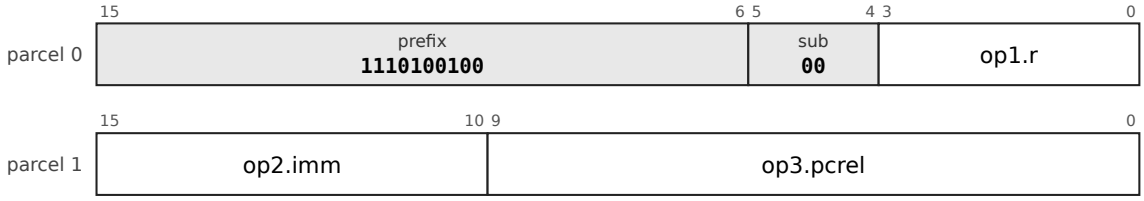



```
CBEQ Ra, Rb, target ... and the rest
```

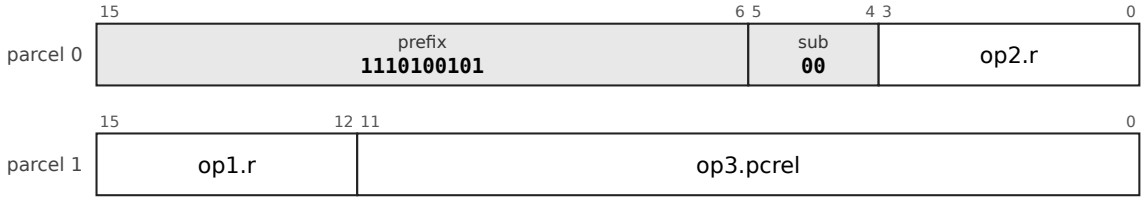
Operation, flags and exceptions as for the entry CBCC above; 17 layout forms, each a chart of its own (above).

CBPL

CBPL: CBDI, prefix 1110100100, sub-opcode 00; operands reg4, imm6s, pc10-u2; 4 bytes



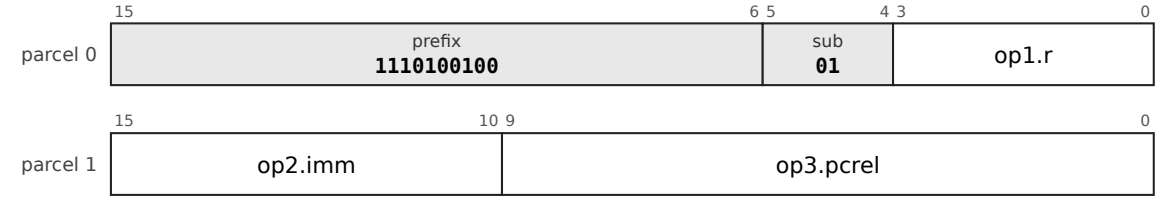
CBPL: CBDR, prefix 1110100101, sub-opcode 00; operands reg4, reg4, pc12-u2; 4 bytes



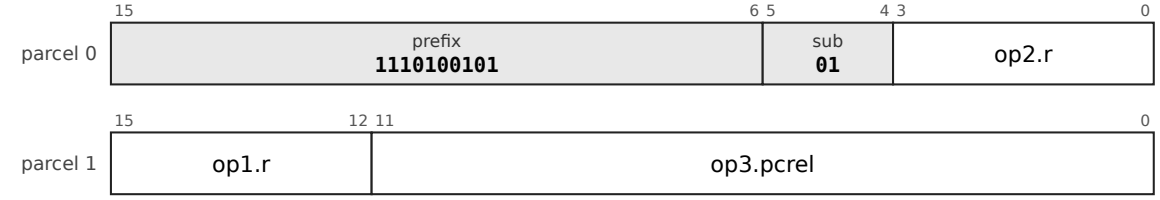
Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

CBMI

CBMI: CBDI, prefix 1110100100, sub-opcode 01; operands reg4, imm6s, pc10-u2; 4 bytes



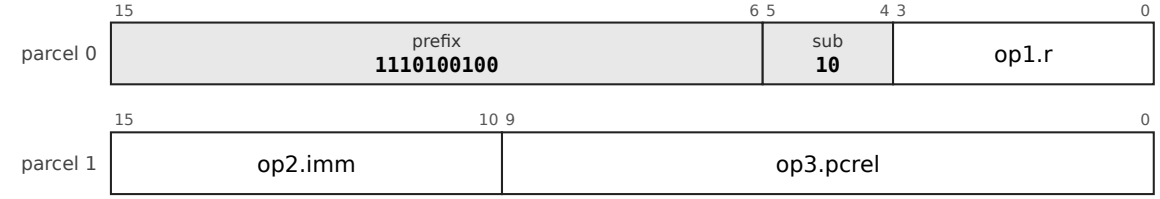
CBMI: CBDR, prefix 1110100101, sub-opcode 01; operands reg4, reg4, pc12-u2; 4 bytes



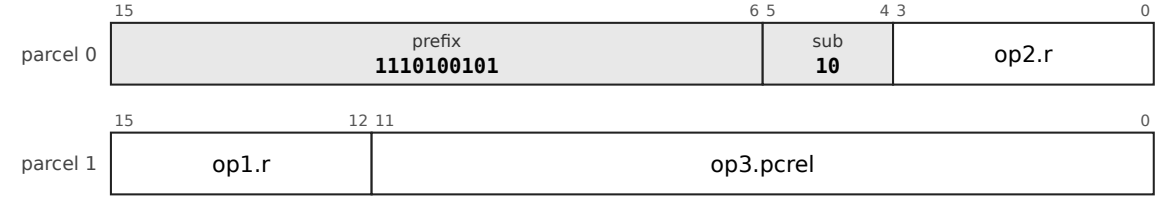
Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

CBVC

CBVC: CBDI, prefix 1110100100, sub-opcode 10; operands reg4, imm6s, pc10-u2; 4 bytes



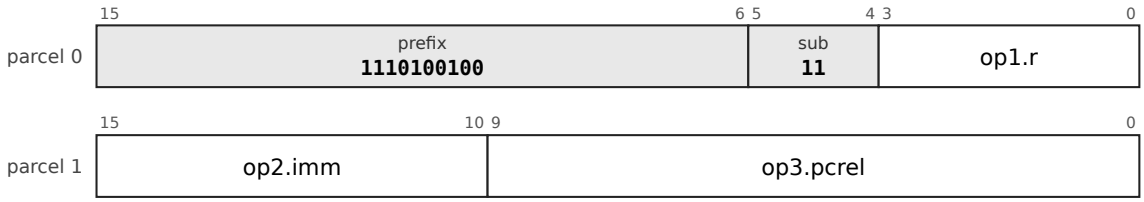
CBVC: CBDR, prefix 1110100101, sub-opcode 10; operands reg4, reg4, pc12-u2; 4 bytes



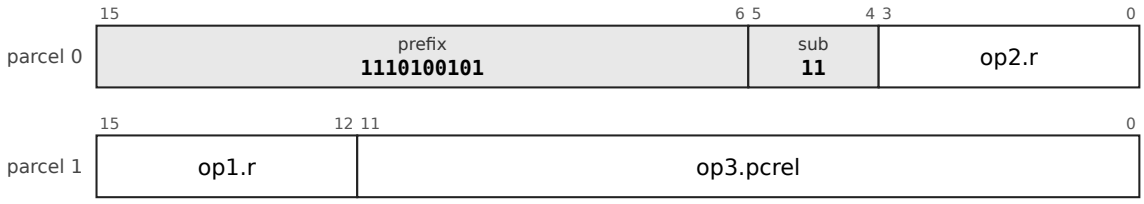
Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

CBVS

CBVS: CBDI, prefix 1110100100, sub-opcode 11; operands reg4, imm6s, pc10-u2; 4 bytes



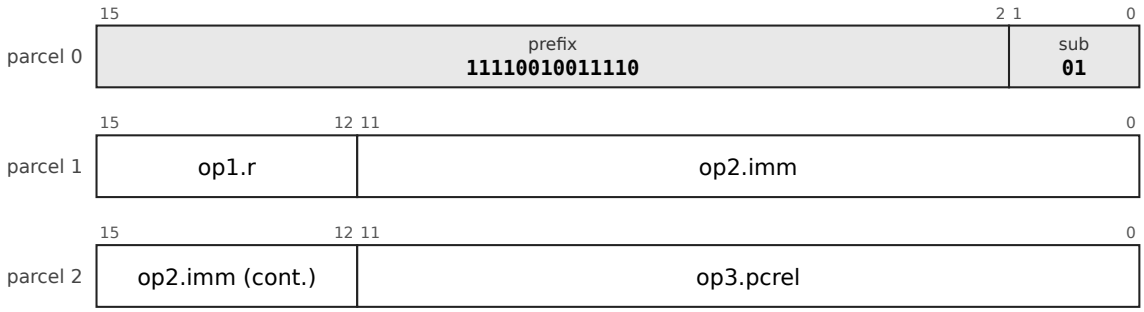
CBVS: CBDR, prefix 1110100101, sub-opcode 11; operands reg4, reg4, pc12-u2; 4 bytes



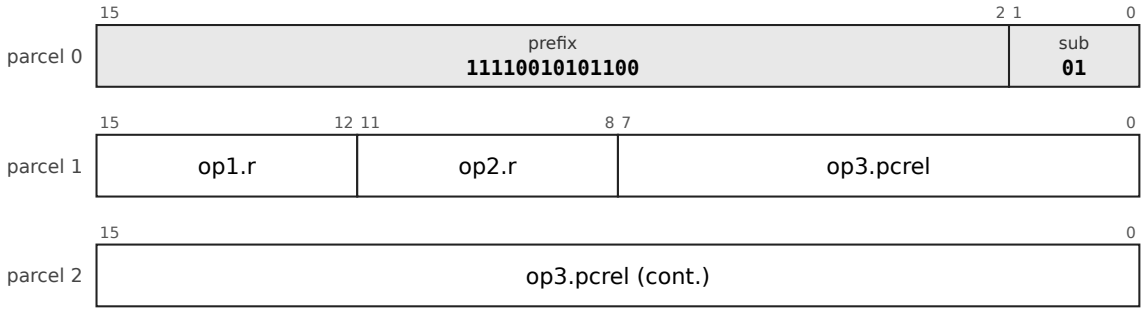
Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

CBCC

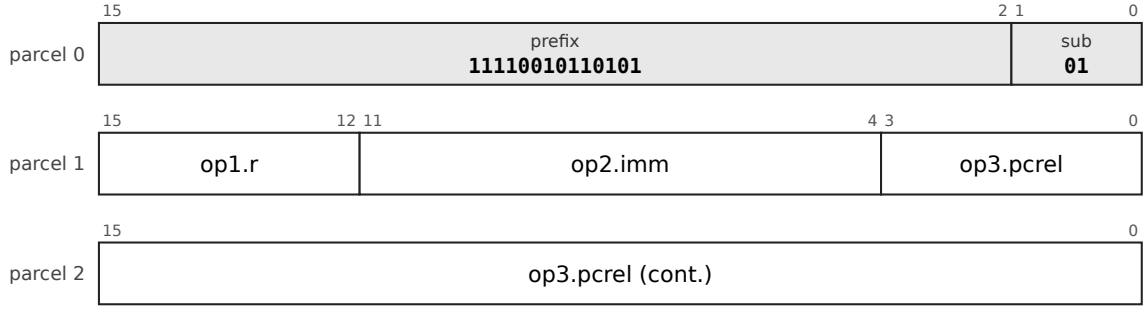
CBCC: CB3I16_12/c, prefix 11110010011110, sub-opcode 01; operands reg4, imm16s, pc12-u



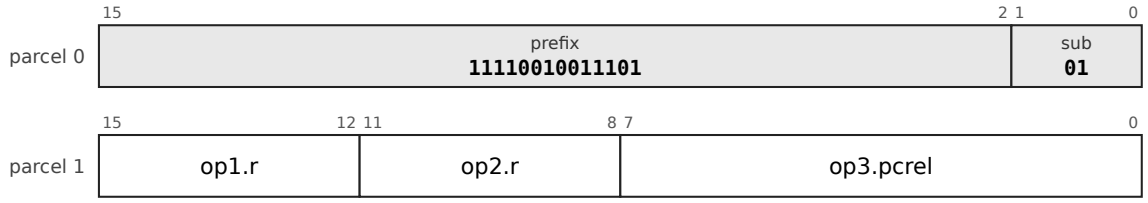
CBCC: CB3R24/c, prefix 11110010101100, sub-opcode 01; operands reg4, reg4, pc24-u2; 6 by



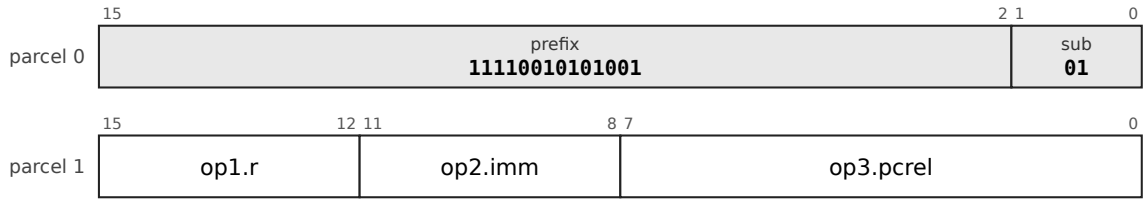
CBCC: CB3I8_20/c, prefix 11110010110101, sub-opcode 01; operands reg4, imm8s, pc20-u2; (



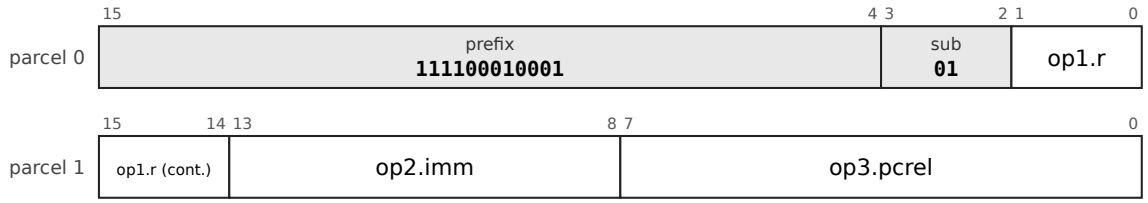
CBCC: CBR8/c, prefix 11110010011101, sub-opcode 01; operands reg4, reg4, pc8-u2; 4 bytes



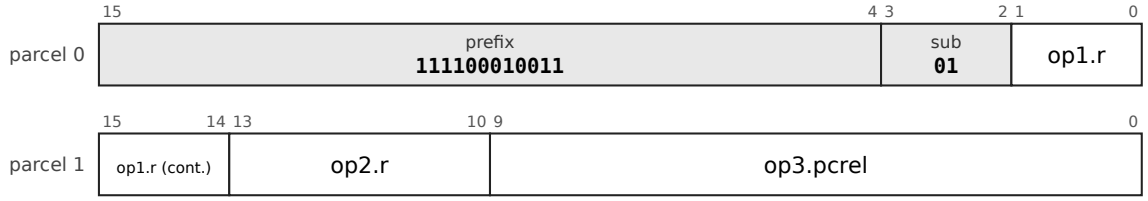
CBCC: CBI4_8/c, prefix 11110010101001, sub-opcode 01; operands reg4, imm4s, pc8-u2; 4 by



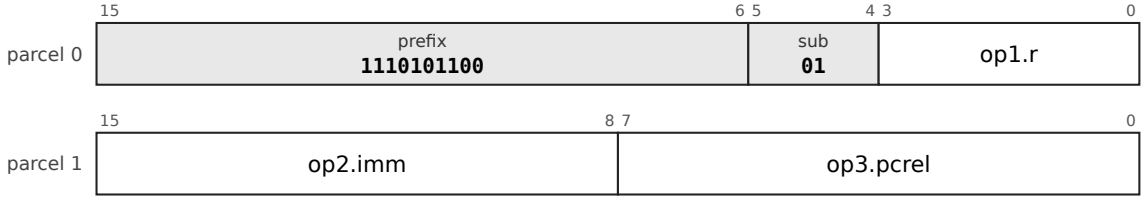
CBCC: CBI6_8/c, prefix 111100010001, sub-opcode 01; operands reg4, imm6s, pc8-u2; 4 bytes



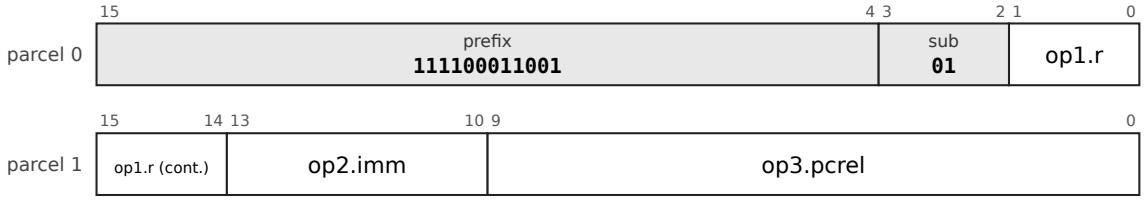
CBCC: CBR10/c, prefix 111100010011, sub-opcode 01; operands reg4, reg4, pc10-u2; 4 bytes



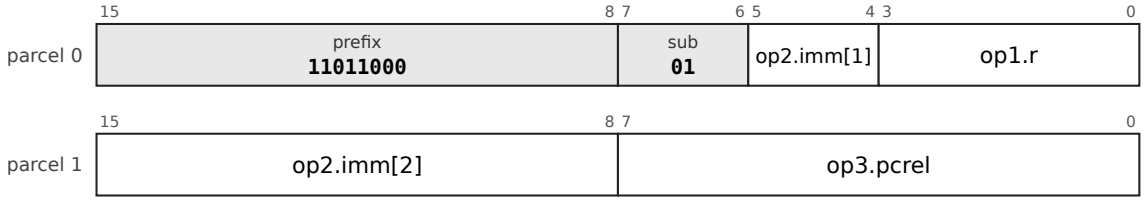
CBCC: CBI8_8/c, prefix 1110101100, sub-opcode 01; operands reg4, imm8s, pc8-u2; 4 bytes



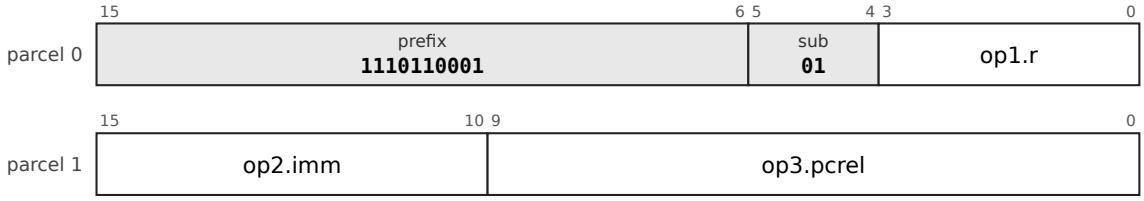
CBCC: CBI4_10/c, prefix 111100011001, sub-opcode 01; operands reg4, imm4s, pc10-u2; 4 by



CBCC: CBI10_8/c, prefix 11011000, sub-opcode 01; operands reg4, imm10s, pc8-u2; 4 bytes



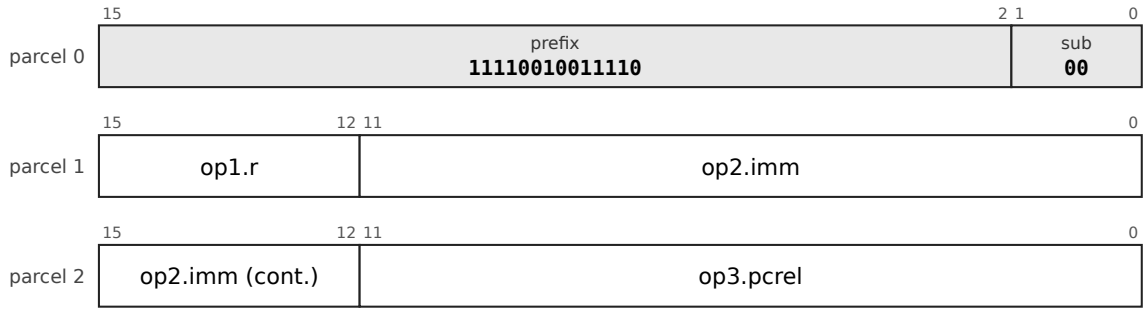
CBCC: CBI6_10/c, prefix 1110110001, sub-opcode 01; operands reg4, imm6s, pc10-u2; 4 byte



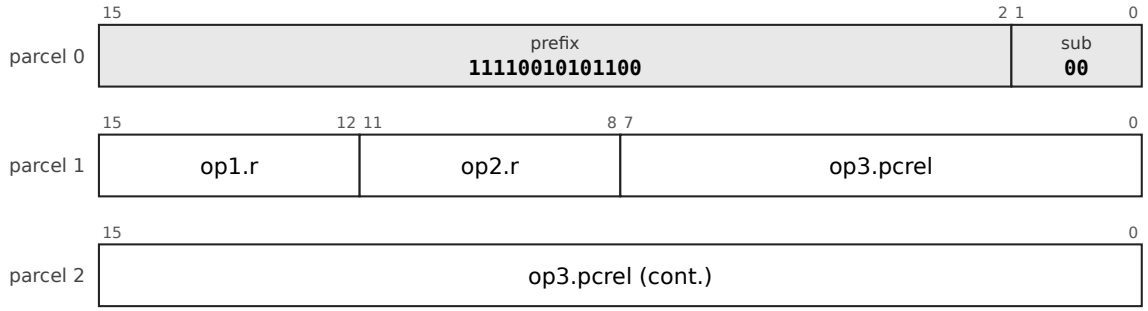
Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

CBCS

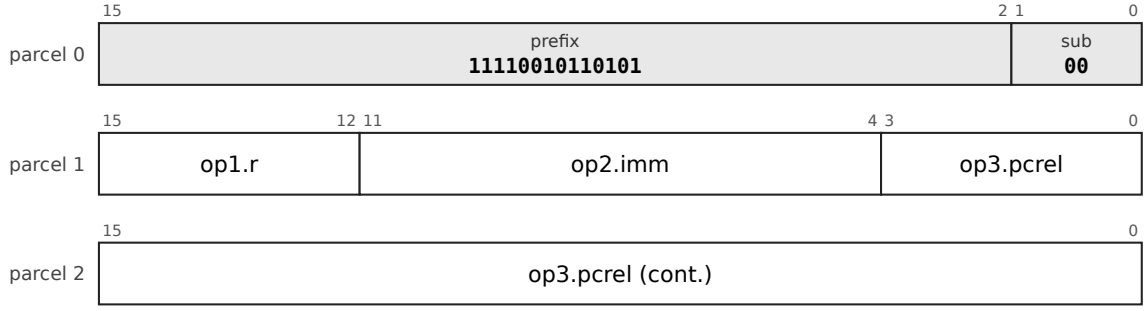
CBCS: CB3I16_12/c, prefix 11110010011110, sub-opcode 00; operands reg4, imm16s, pc12-u



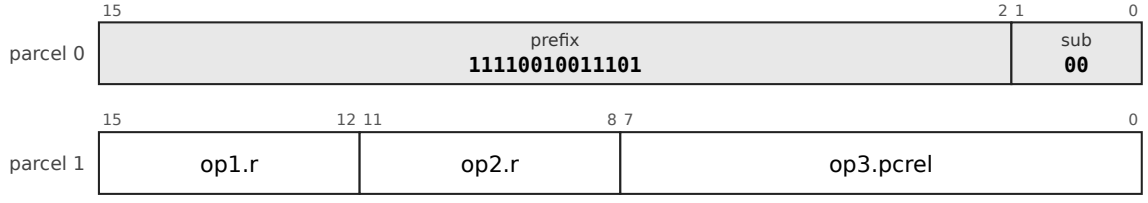
CBCS: CB3R24/c, prefix 11110010101100, sub-opcode 00; operands reg4, reg4, pc24-u2; 6 by



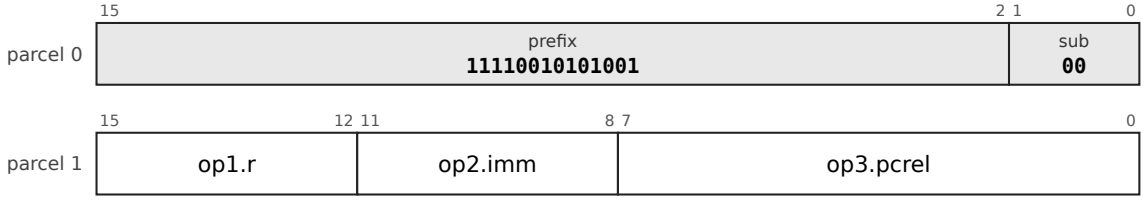
CBCS: CB3I8_20/c, prefix 11110010110101, sub-opcode 00; operands reg4, imm8s, pc20-u2; (



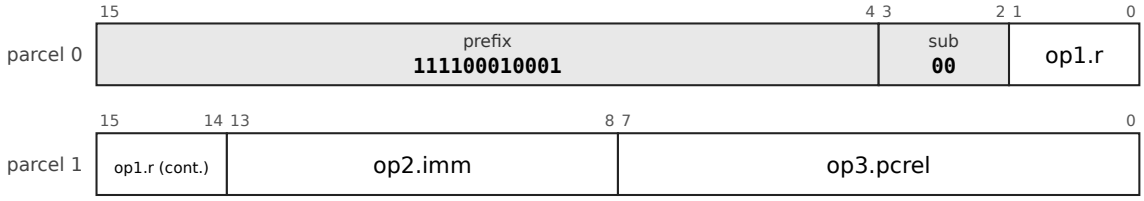
CBCS: CBR8/c, prefix 11110010011101, sub-opcode 00; operands reg4, reg4, pc8-u2; 4 bytes



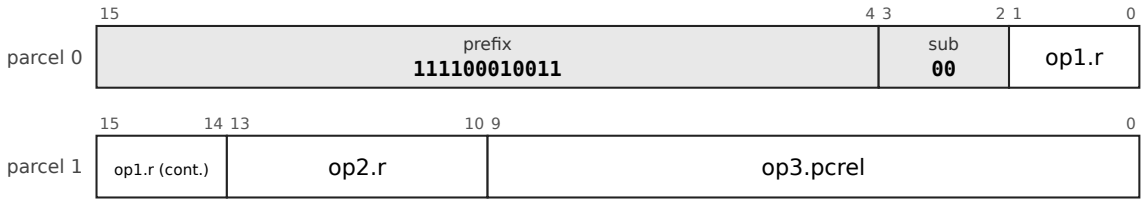
CBCS: CBI4_8/c, prefix 11110010101001, sub-opcode 00; operands reg4, imm4s, pc8-u2; 4 by



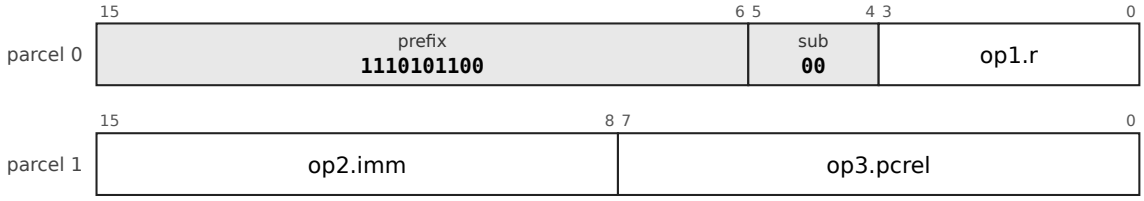
CBCS: CBI6_8/c, prefix 111100010001, sub-opcode 00; operands reg4, imm6s, pc8-u2; 4 byte



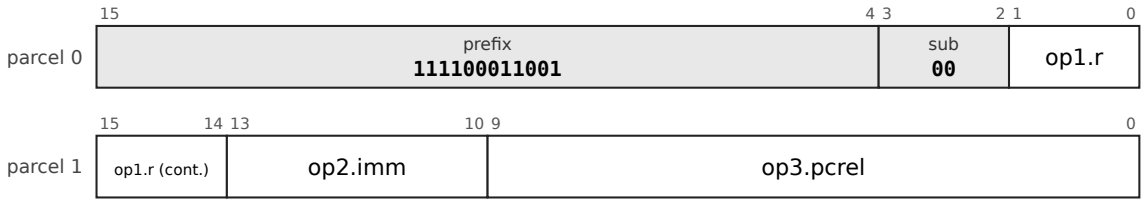
CBCS: CBR10/c, prefix 111100010011, sub-opcode 00; operands reg4, reg4, pc10-u2; 4 bytes



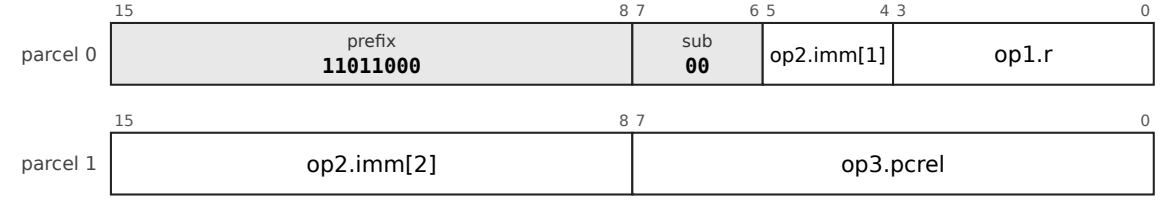
CBCS: CBI8_8/c, prefix 1110101100, sub-opcode 00; operands reg4, imm8s, pc8-u2; 4 bytes



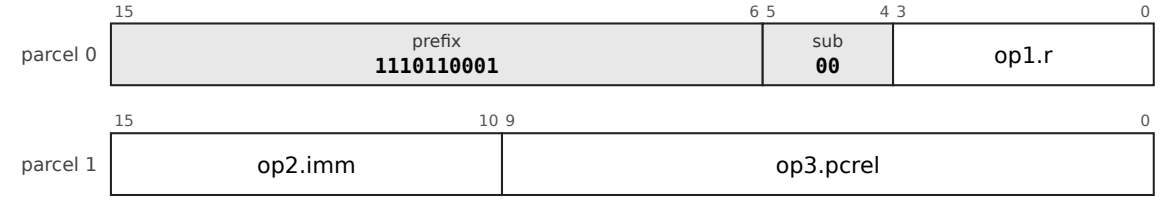
CBCS: CBI4_10/c, prefix 111100011001, sub-opcode 00; operands reg4, imm4s, pc10-u2; 4 by



CBCS: CBI10_8/c, prefix 11011000, sub-opcode 00; operands reg4, imm10s, pc8-u2; 4 bytes



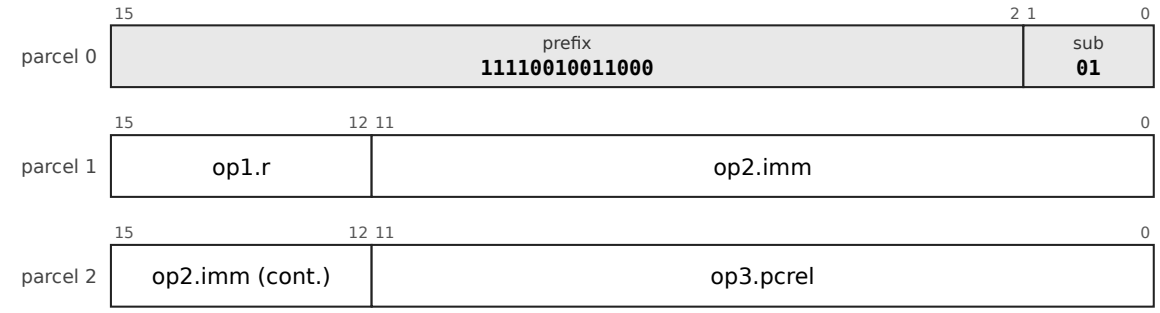
CBCS: CBI6_10/c, prefix 1110110001, sub-opcode 00; operands reg4, imm6s, pc10-u2; 4 byte



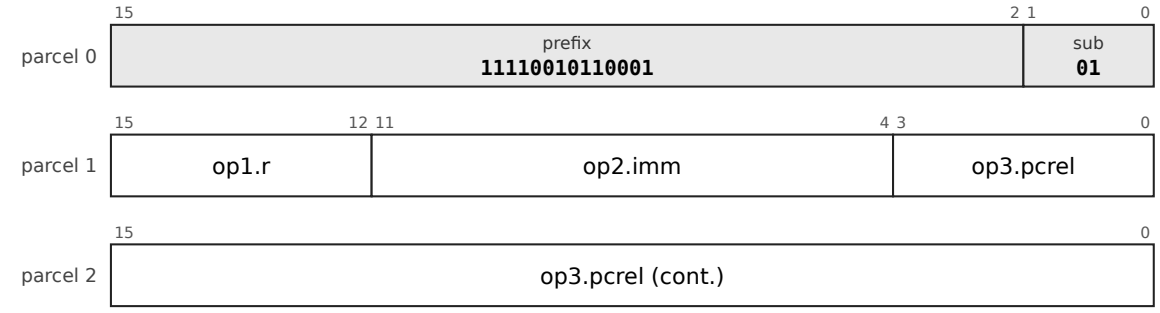
Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

CBGE

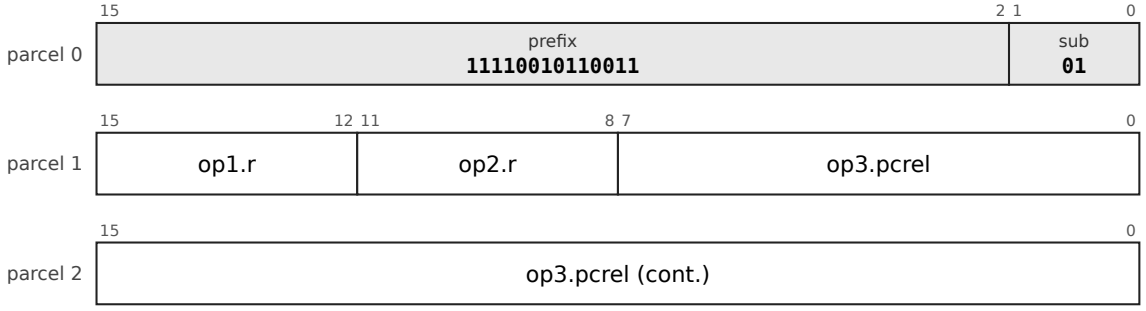
CBGE: CB3I16_12/b, prefix 11110010011000, sub-opcode 01; operands reg4, imm16s, pc12-u



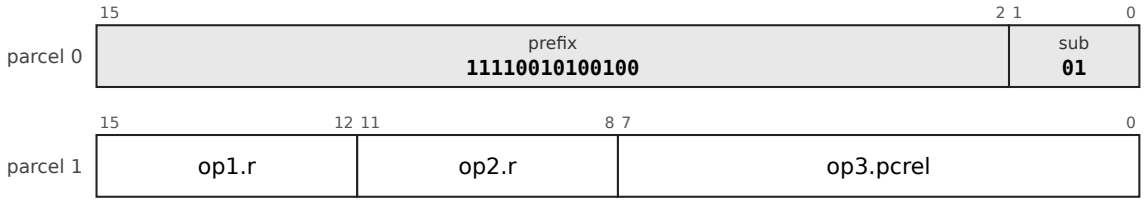
CBGE: CB3I8_20/b, prefix 11110010110001, sub-opcode 01; operands reg4, imm8s, pc20-u2;



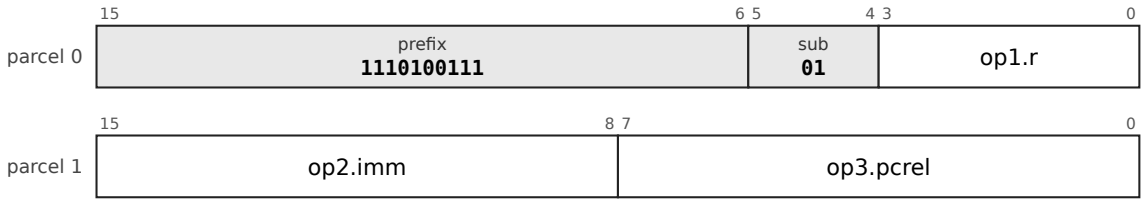
CBGE: CB3R24/b, prefix 11110010110011, sub-opcode 01; operands reg4, reg4, pc24-u2; 6 by



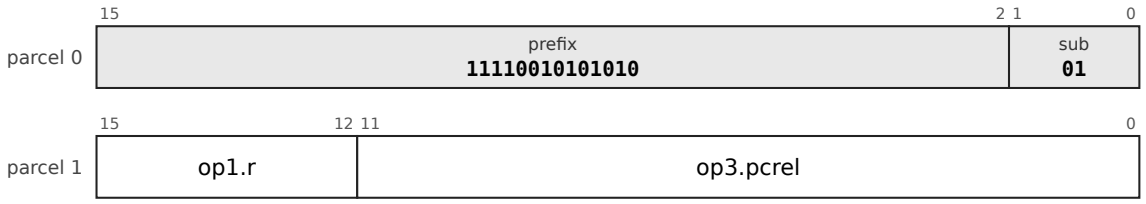
CBGE: CBR8/b, prefix 11110010100100, sub-opcode 01; operands reg4, reg4, pc8-u2; 4 bytes



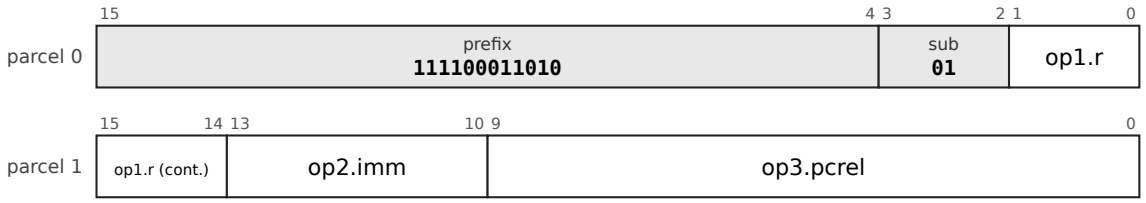
CBGE: CBI8_8/b, prefix 1110100111, sub-opcode 01; operands reg4, imm8s, pc8-u2; 4 bytes



CBGE: CBZ2_12/b, prefix 11110010101010, sub-opcode 01; operands reg4, k-imm0, pc12-u2;



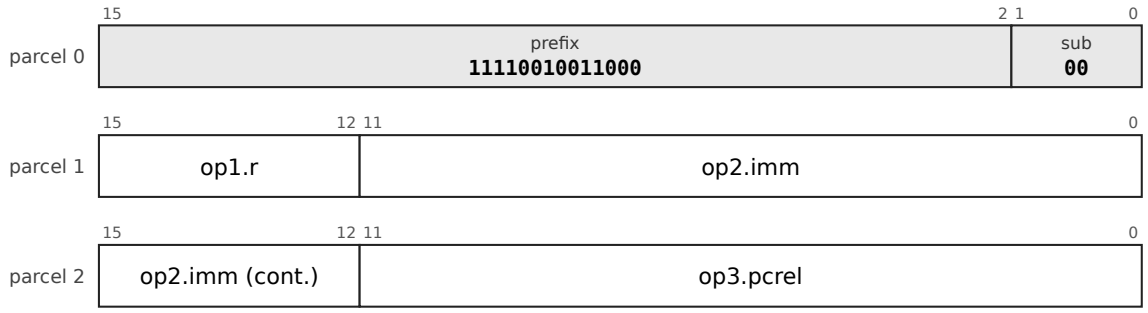
CBGE: CBI4_10/b, prefix 111100011010, sub-opcode 01; operands reg4, imm4s, pc10-u2; 4 by



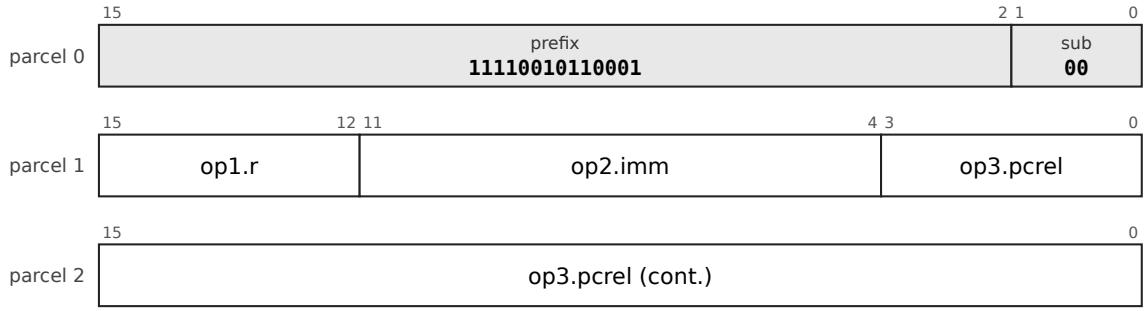
Operation, flags and exceptions as for the entry CBCC above; 7 layout forms, each a chart of its own (above).

CBLT

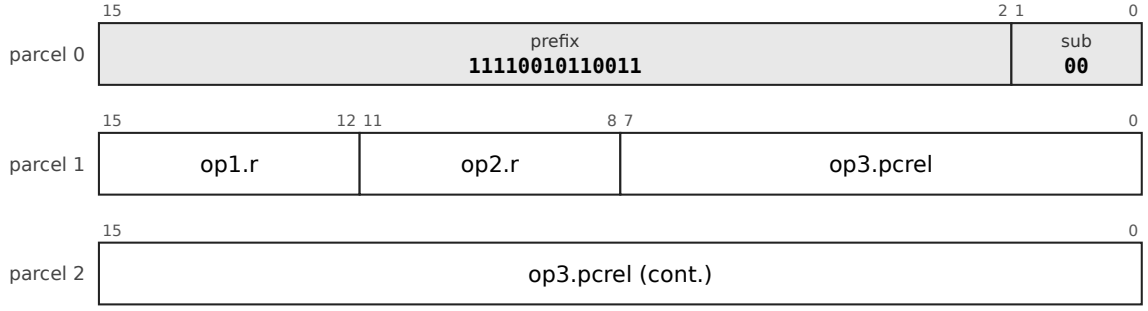
CBLT: CB3I16_12/b, prefix 11110010011000, sub-opcode 00; operands reg4, imm16s, pc12-u2



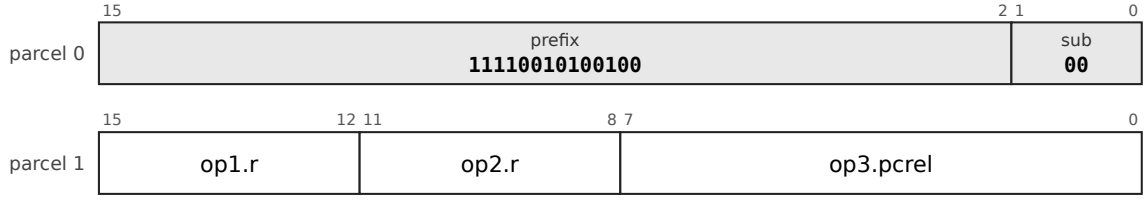
CBLT: CB3I8_20/b, prefix 11110010110001, sub-opcode 00; operands reg4, imm8s, pc20-u2; 6



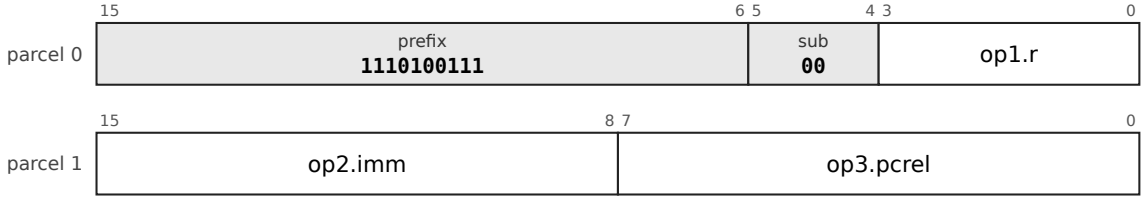
CBLT: CB3R24/b, prefix 11110010110011, sub-opcode 00; operands reg4, reg4, pc24-u2; 6 by



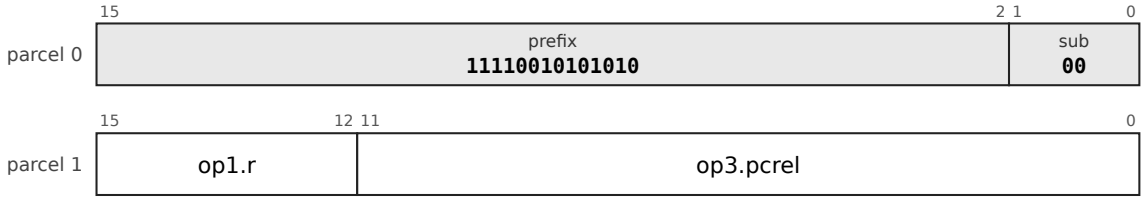
CBLT: CBR8/b, prefix 11110010100100, sub-opcode 00; operands reg4, reg4, pc8-u2; 4 bytes



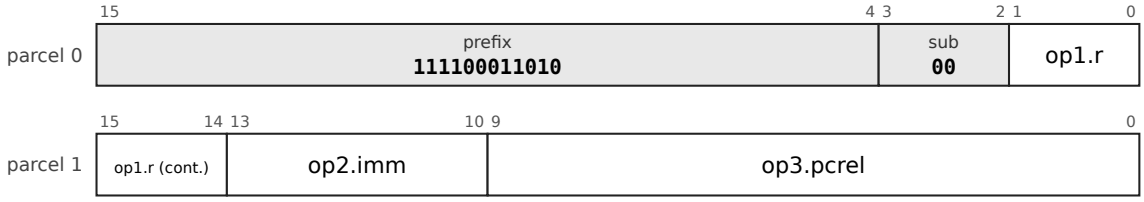
CBLT: CBI8_8/b, prefix 1110100111, sub-opcode 00; operands reg4, imm8s, pc8-u2; 4 bytes



CBLT: CBZ2_12/b, prefix 11110010101010, sub-opcode 00; operands reg4, k-imm0, pc12-u2; 4



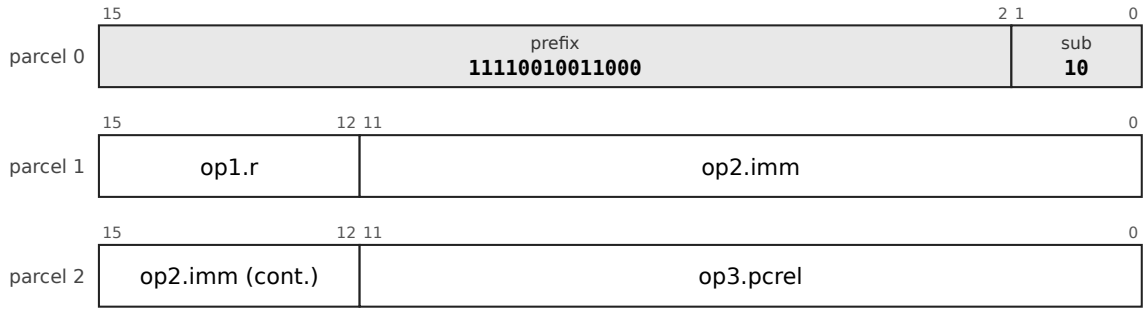
CBLT: CBI4_10/b, prefix 111100011010, sub-opcode 00; operands reg4, imm4s, pc10-u2; 4 byt



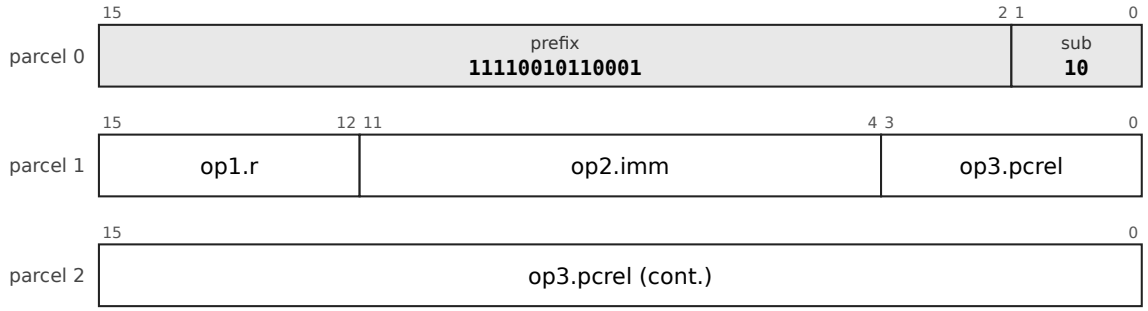
Operation, flags and exceptions as for the entry CBCC above; 7 layout forms, each a chart of its own (above).

CBGT

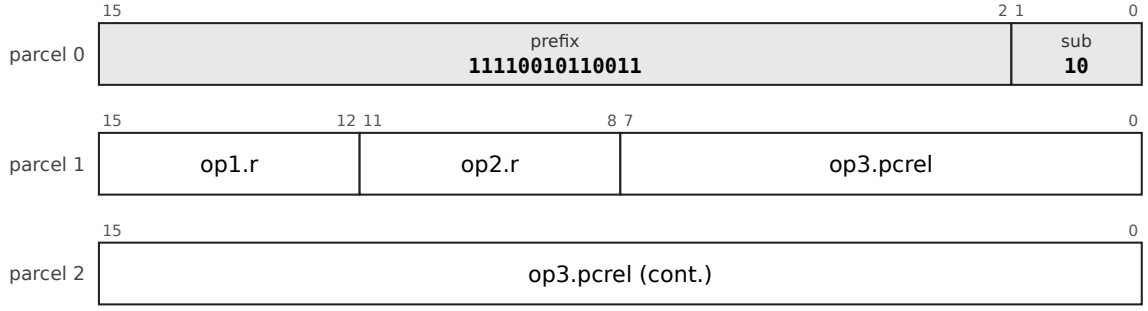
CBGT: CB3I16_12/b, prefix 11110010011000, sub-opcode 10; operands reg4, imm16s, pc12-u



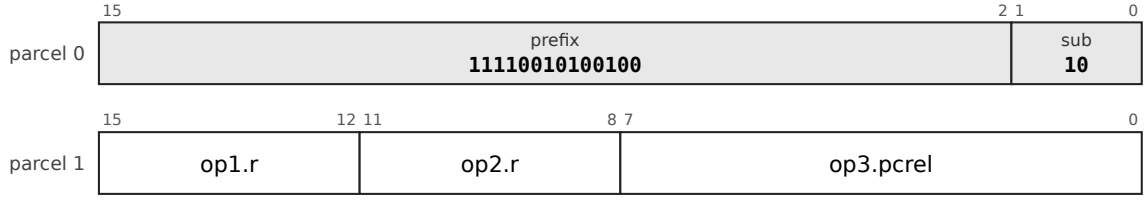
CBGT: CB3I8_20/b, prefix 11110010110001, sub-opcode 10; operands reg4, imm8s, pc20-u2; (



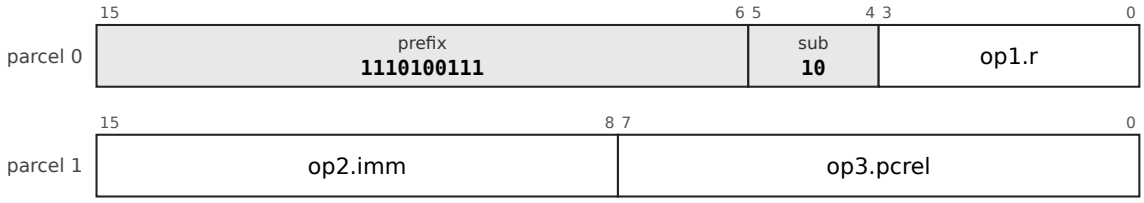
CBGT: CB3R24/b, prefix 11110010110011, sub-opcode 10; operands reg4, reg4, pc24-u2; 6 by



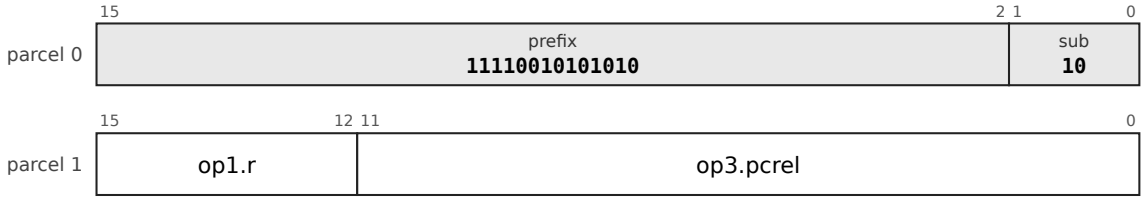
CBGT: CBR8/b, prefix 11110010100100, sub-opcode 10; operands reg4, reg4, pc8-u2; 4 bytes



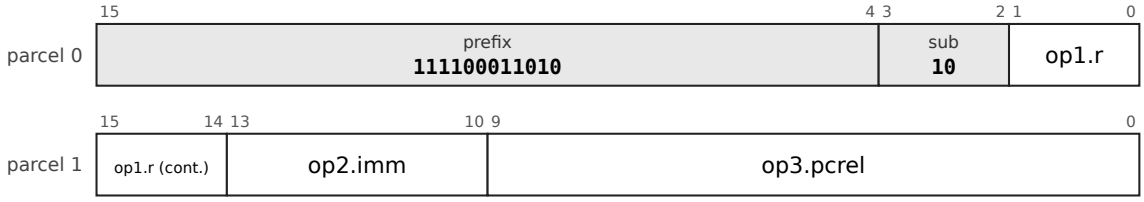
CBGT: CBI8_8/b, prefix 1110100111, sub-opcode 10; operands reg4, imm8s, pc8-u2; 4 bytes



CBGT: CBZ2_12/b, prefix 11110010101010, sub-opcode 10; operands reg4, k-imm0, pc12-u2;



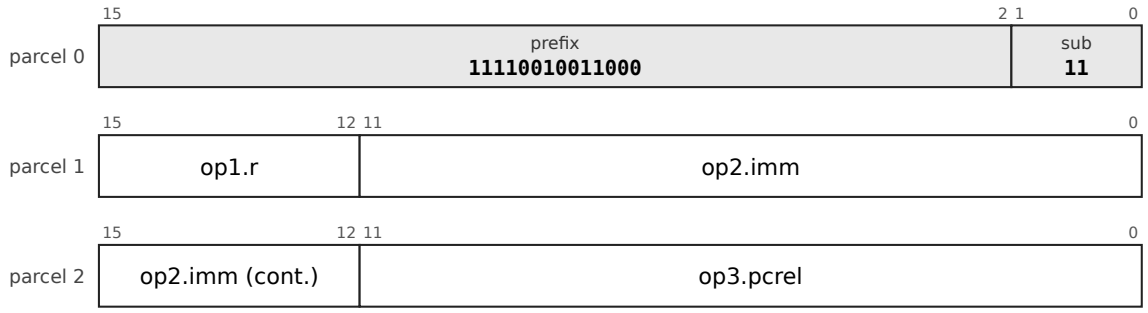
CBGT: CBI4_10/b, prefix 111100011010, sub-opcode 10; operands reg4, imm4s, pc10-u2; 4 by



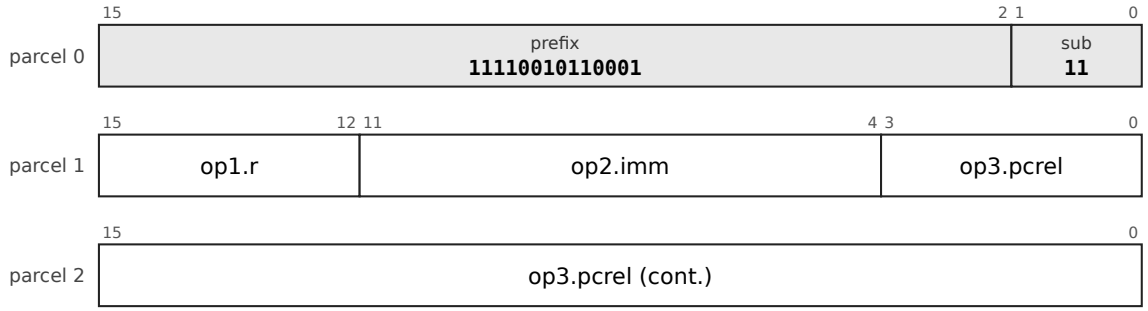
Operation, flags and exceptions as for the entry CBCC above; 7 layout forms, each a chart of its own (above).

CBLE

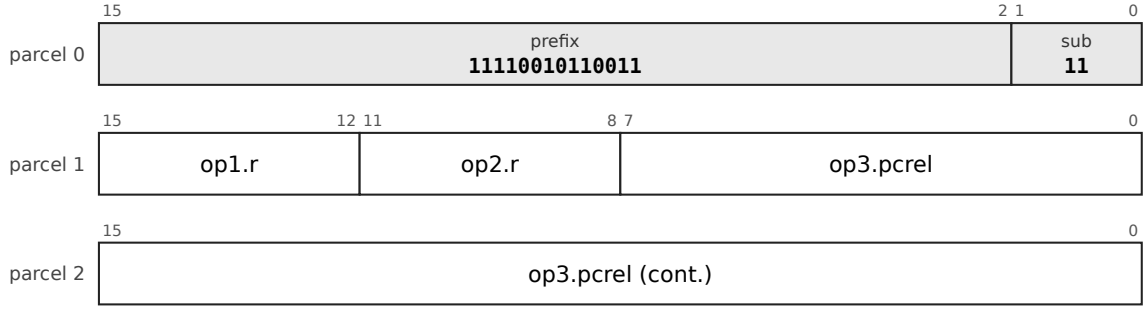
CBLE: CB3I16_12/b, prefix 11110010011000, sub-opcode 11; operands reg4, imm16s, pc12-u2



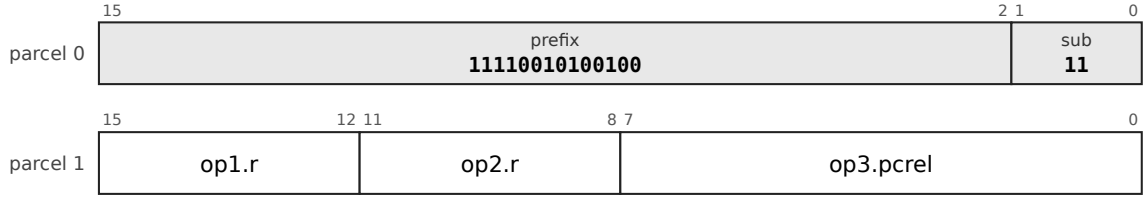
CBLE: CB3I8_20/b, prefix 11110010110001, sub-opcode 11; operands reg4, imm8s, pc20-u2; 6



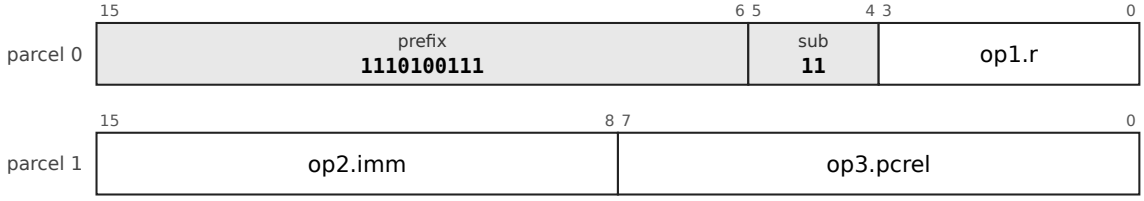
CBLE: CB3R24/b, prefix 11110010110011, sub-opcode 11; operands reg4, reg4, pc24-u2; 6 by



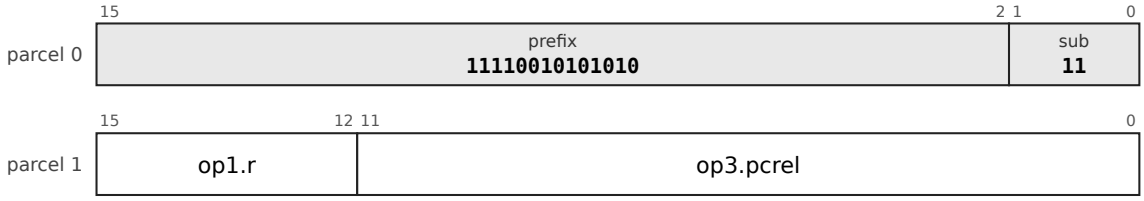
CBLE: CBR8/b, prefix 11110010100100, sub-opcode 11; operands reg4, reg4, pc8-u2; 4 bytes



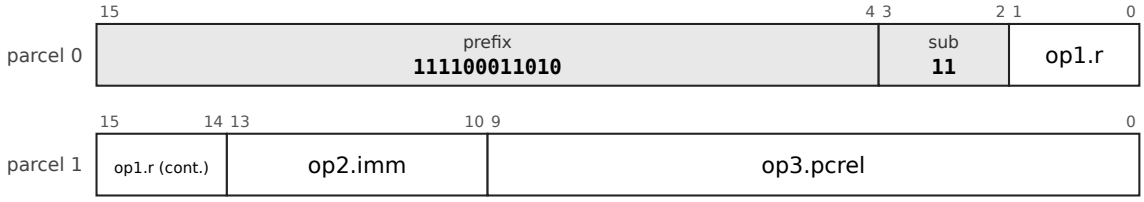
CBLE: CBI8_8/b, prefix 1110100111, sub-opcode 11; operands reg4, imm8s, pc8-u2; 4 bytes



CBLE: CBZ2_12/b, prefix 11110010101010, sub-opcode 11; operands reg4, k-imm0, pc12-u2;



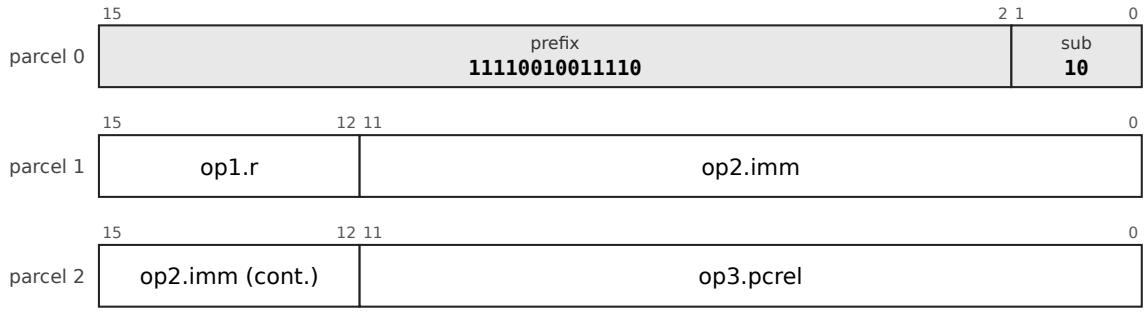
CBLE: CBI4_10/b, prefix 111100011010, sub-opcode 11; operands reg4, imm4s, pc10-u2; 4 by



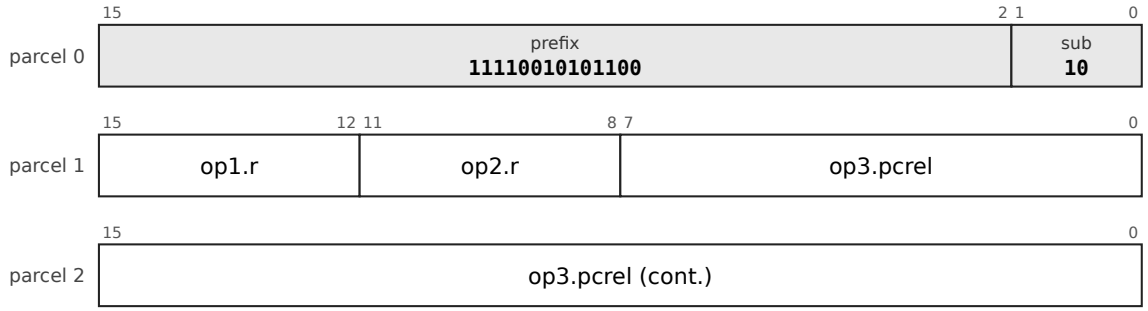
Operation, flags and exceptions as for the entry CBCC above; 7 layout forms, each a chart of its own (above).

CBHI

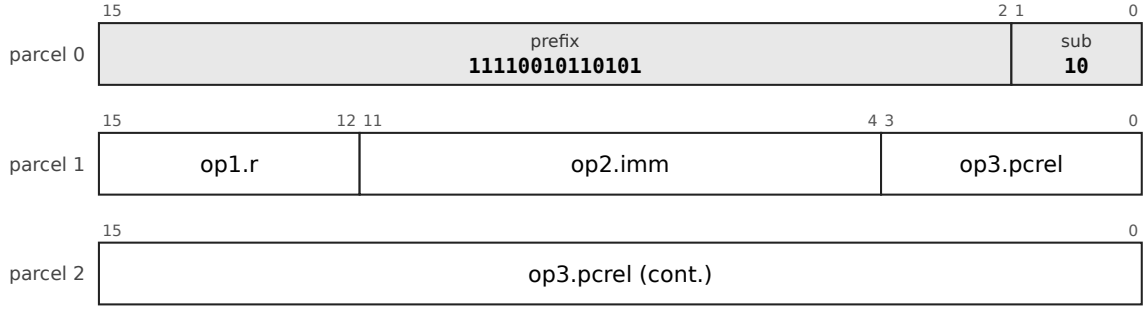
CBHI: CB3I16_12/c, prefix 11110010011110, sub-opcode 10; operands reg4, imm16s, pc12-u2



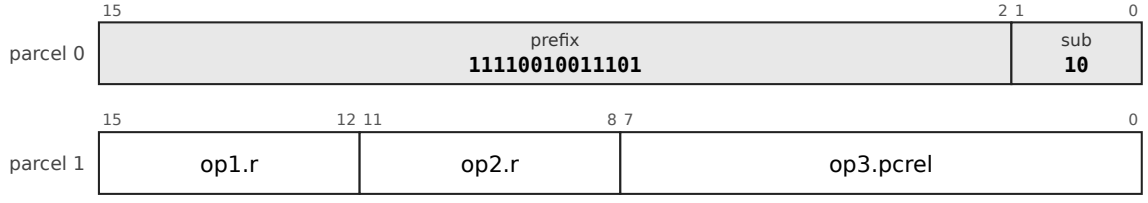
CBHI: CB3R24/c, prefix 11110010101100, sub-opcode 10; operands reg4, reg4, pc24-u2; 6 by



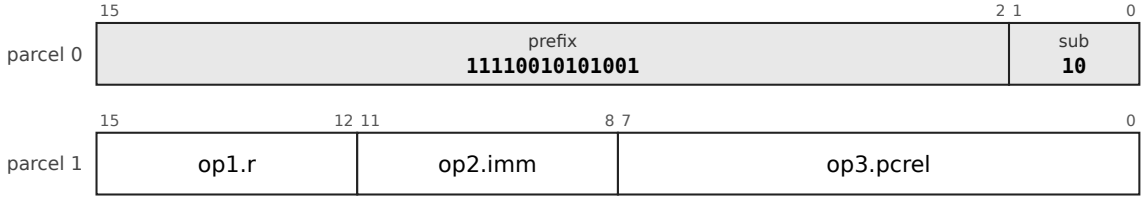
CBHI: CB3I8_20/c, prefix 11110010110101, sub-opcode 10; operands reg4, imm8s, pc20-u2; 6



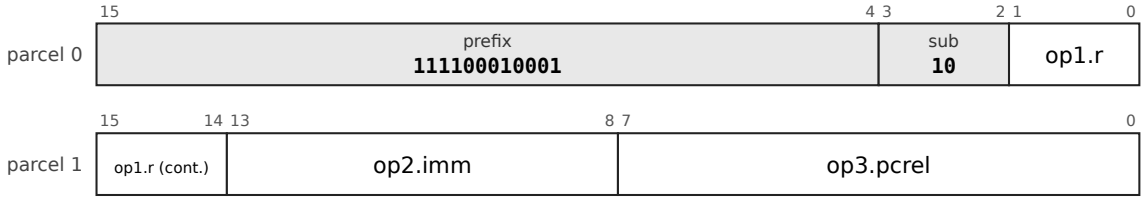
CBHI: CBR8/c, prefix 11110010011101, sub-opcode 10; operands reg4, reg4, pc8-u2; 4 bytes



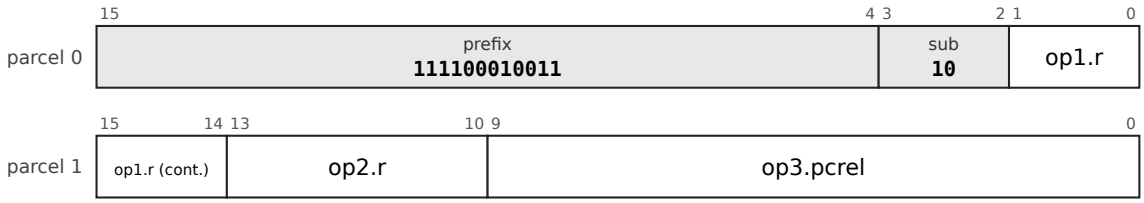
CBHI: CBI4_8/c, prefix 11110010101001, sub-opcode 10; operands reg4, imm4s, pc8-u2; 4 byt



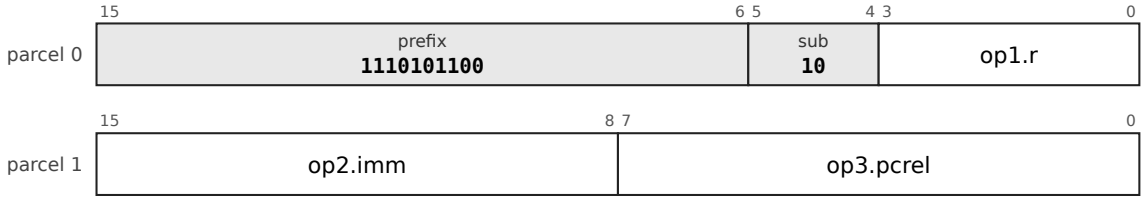
CBHI: CBI6_8/c, prefix 111100010001, sub-opcode 10; operands reg4, imm6s, pc8-u2; 4 bytes



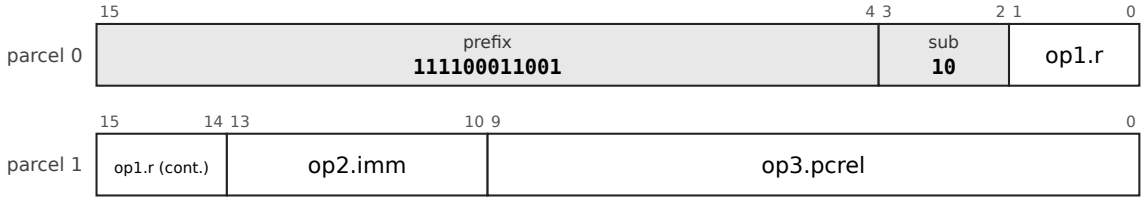
CBHI: CBR10/c, prefix 111100010011, sub-opcode 10; operands reg4, reg4, pc10-u2; 4 bytes



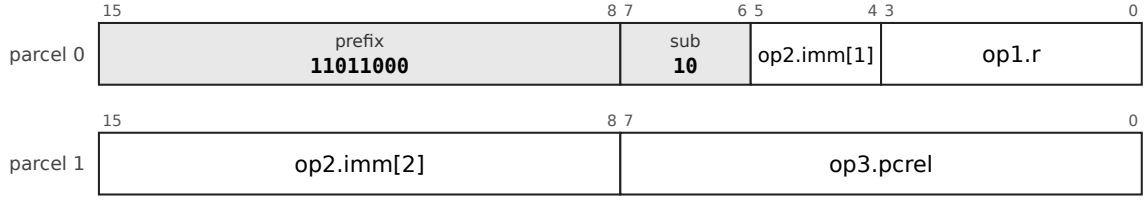
CBHI: CBI8_8/c, prefix 1110101100, sub-opcode 10; operands reg4, imm8s, pc8-u2; 4 bytes



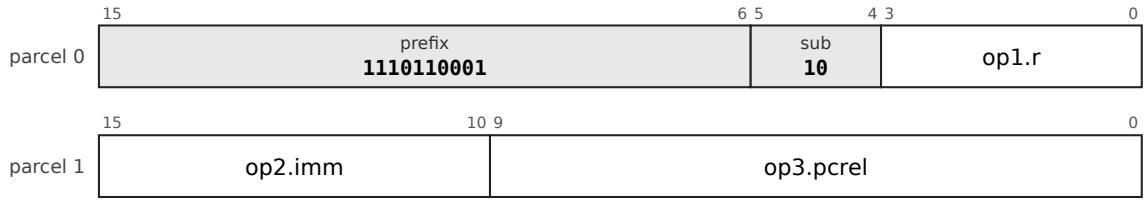
CBHI: CBI4_10/c, prefix 111100011001, sub-opcode 10; operands reg4, imm4s, pc10-u2; 4 byt



CBHI: CBI10_8/c, prefix 11011000, sub-opcode 10; operands reg4, imm10s, pc8-u2; 4 bytes



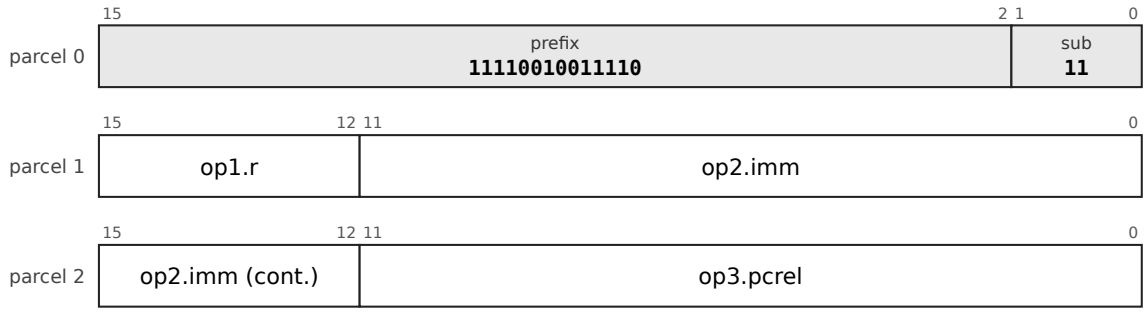
CBHI: CBI6_10/c, prefix 1110110001, sub-opcode 10; operands reg4, imm6s, pc10-u2; 4 bytes



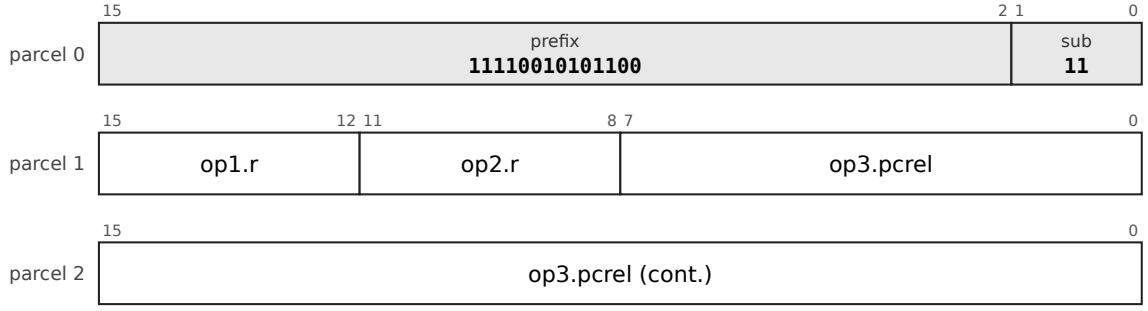
Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

CBLOS

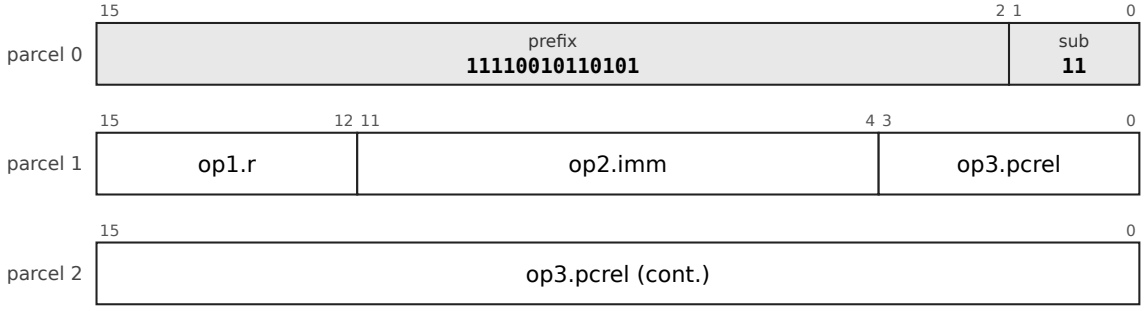
CBLOS: CB3I16_12/c, prefix 11110010011110, sub-opcode 11; operands reg4, imm16s, pc12-



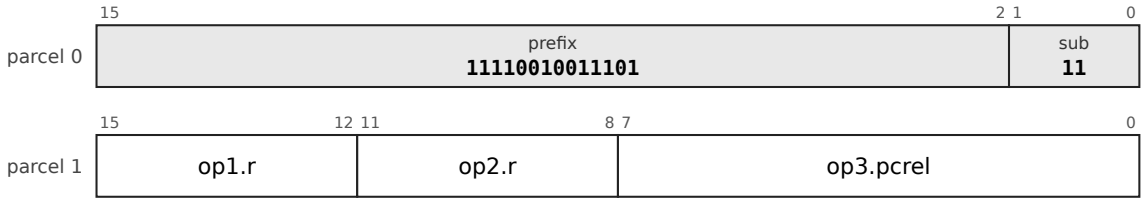
CBLOS: CB3R24/c, prefix 11110010101100, sub-opcode 11; operands reg4, reg4, pc24-u2; 6 b



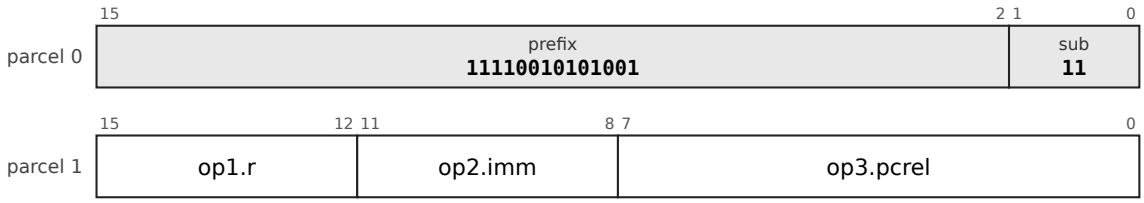
CBLOS: CB3I8_20/c, prefix 11110010110101, sub-opcode 11; operands reg4, imm8s, pc20-u2;



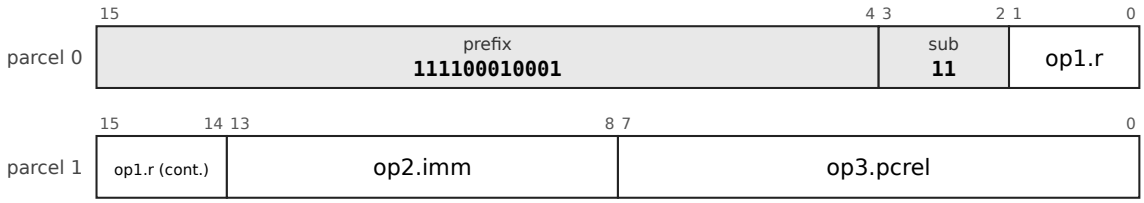
CBLOS: CBR8/c, prefix 11110010011101, sub-opcode 11; operands reg4, reg4, pc8-u2; 4 byte



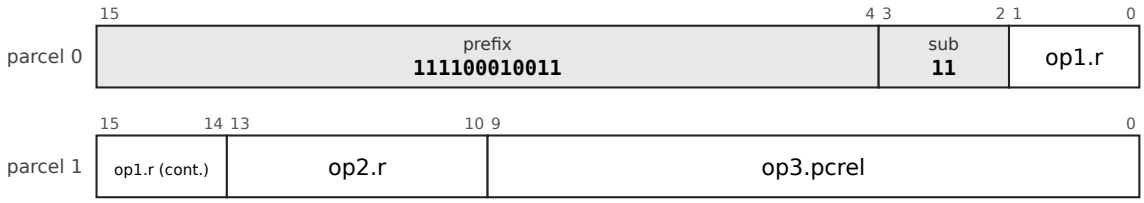
CBLOS: CBI4_8/c, prefix 11110010101001, sub-opcode 11; operands reg4, imm4s, pc8-u2; 4 b



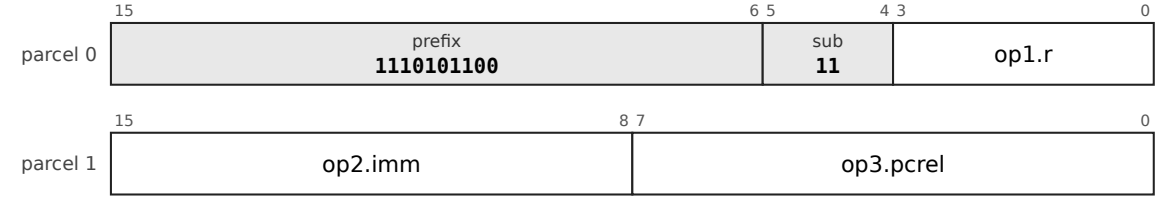
CBLOS: CBI6_8/c, prefix 111100010001, sub-opcode 11; operands reg4, imm6s, pc8-u2; 4 byt



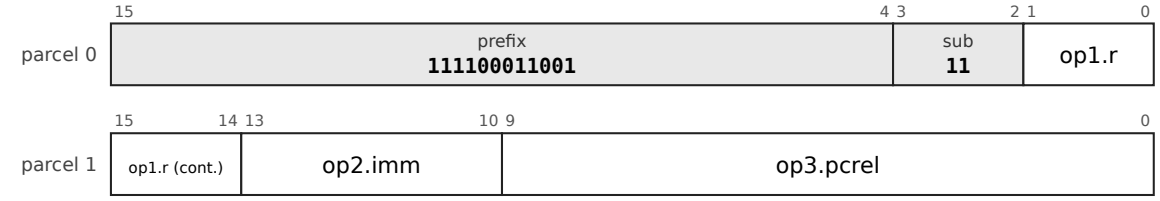
CBLOS: CBR10/c, prefix 111100010011, sub-opcode 11; operands reg4, reg4, pc10-u2; 4 byte



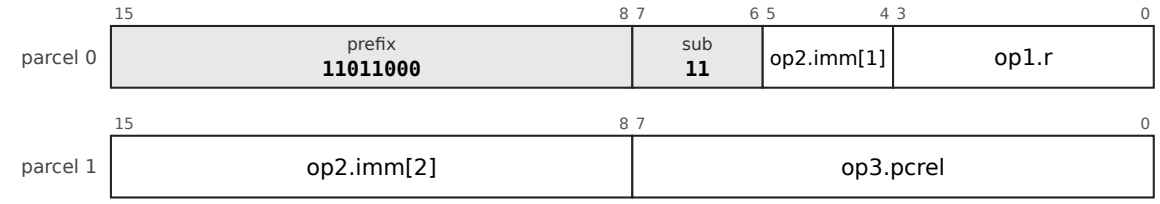
CBLOS: CBI8_8/c, prefix 1110101100, sub-opcode 11; operands reg4, imm8s, pc8-u2; 4 bytes



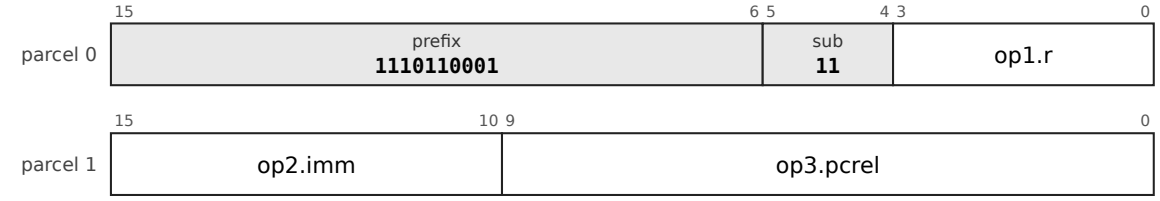
CBLOS: CBI4_10/c, prefix 111100011001, sub-opcode 11; operands reg4, imm4s, pc10-u2; 4 b



CBLOS: CBI10_8/c, prefix 11011000, sub-opcode 11; operands reg4, imm10s, pc8-u2; 4 bytes



CBLOS: CBI6_10/c, prefix 1110110001, sub-opcode 11; operands reg4, imm6s, pc10-u2; 4 byt



Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

CBHIS
An alias of CBCC: the assembler writes it as that word.

CBLO
An alias of CBCS: the assembler writes it as that word.

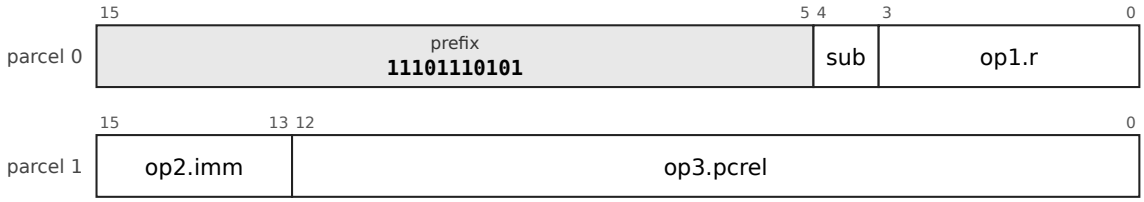
BTAG: Branch on pointer tag

Mnemonics: BTAG, BNTAG

Syntax

```
BTAG Rn, #tag, target
BNTAG Rn, #tag, target
```

Encoding



The chart is the leaf `16-TAG-1`, TAG, prefix `11101110101`. `op1` is `reg4`: register, 4 bits. `op2` is `imm3u`: unsigned 3-bit immediate, 0 to 7. `op3` is `pc13-u2`: pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes). `sub` (1 bit) selects the mnemonic: `btag` 0, `bntag` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-TAG-1`.

Operation

```
BTAG : if (Rn & 111₂) = tag then PC ← address of this instruction + 2 × disp
BNTAG: if (Rn & 111₂) ≠ tag then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None.

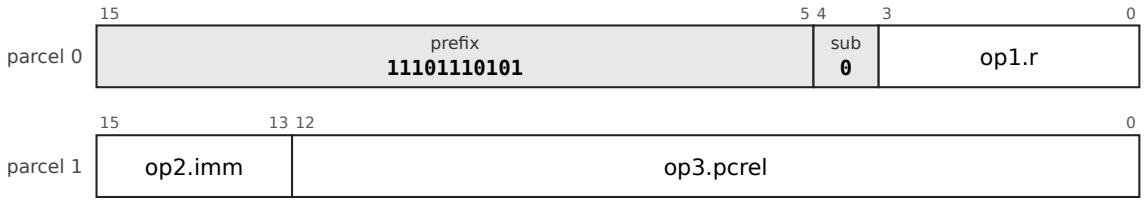
For runtimes that keep a 3-bit type tag in the low bits of a pointer (`lowtag`). The displacement is signed and counts parcels.

Why it is here. The tag branches `BTAG` and `BNTAG` and the table dispatch `JTAG` make tagged pointers cheap to use rather than merely possible. Because quad-aligned objects have zero in the low three bits, the tag is free, and an unaligned access is a fault. The compiler for C never emits them (the corpus shows none); they are for Lisp and other tagged runtimes.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BTAG

BTAG: TAG, prefix 11101110101, sub-opcode 0; operands reg4, imm3u, pc13-u2; 4 bytes

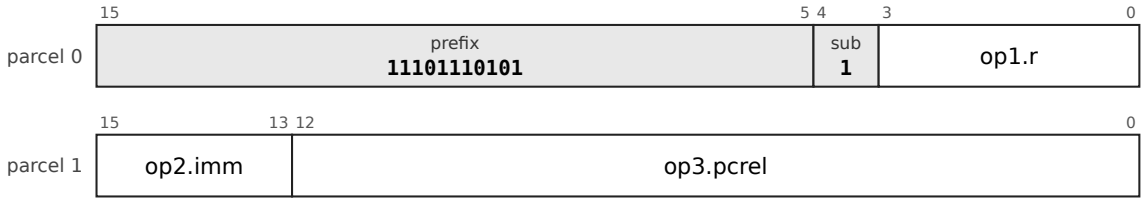



```
BTAG Rn, #tag, target
```

Operation, flags and exceptions as for the entry BTAG above; 1 layout form, each a chart of its own (above).

BNTAG

BNTAG: TAG, prefix 11101110101, sub-opcode 1; operands reg4, imm3u, pc13-u2; 4 bytes

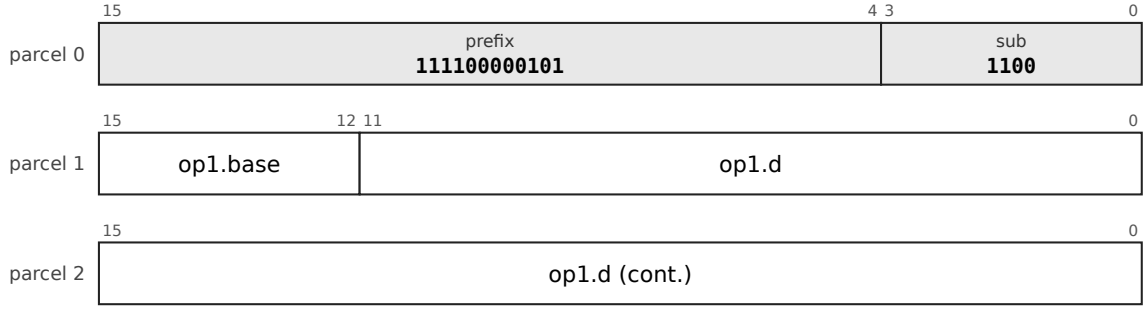



```
BNTAG Rn, #tag, target
```

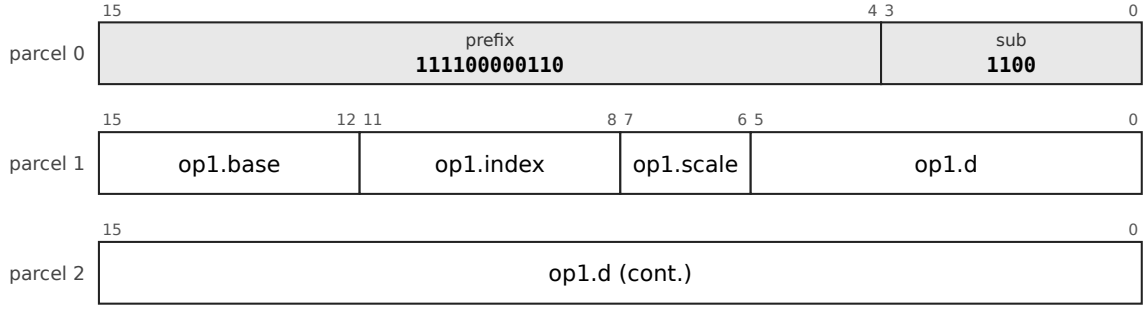
Operation, flags and exceptions as for the entry BTAG above; 1 layout form, each a chart of its own (above).

JMP: Jump

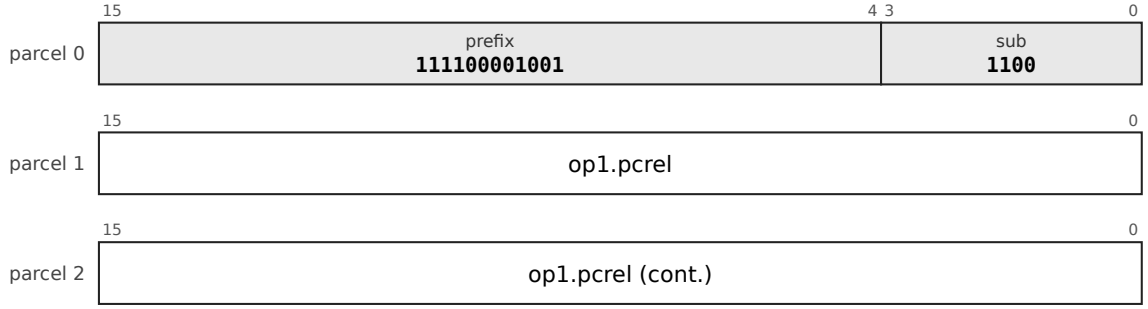
JMP: SOF.B28, prefix 111100000101, sub-opcode 1100; operands b4d28s-x1; 6 bytes



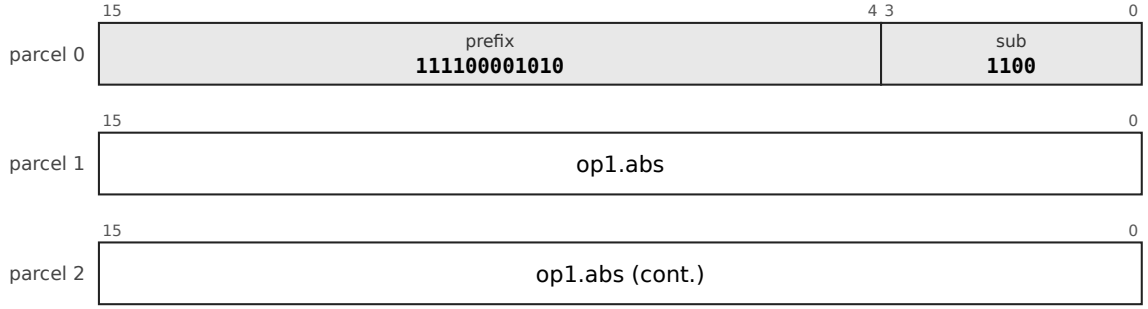
JMP: SOF.X22, prefix 111100000110, sub-opcode 1100; operands x4-4d22s; 6 bytes



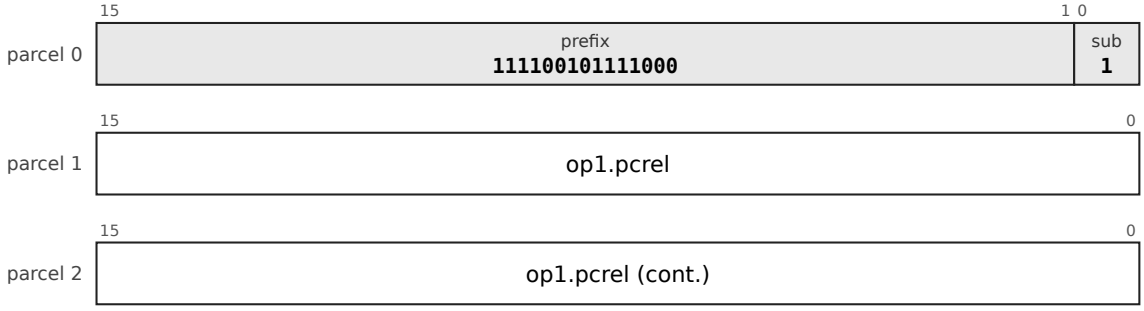
JMP: SOF.PC32, prefix 111100001001, sub-opcode 1100; operands pc32-u1; 6 bytes



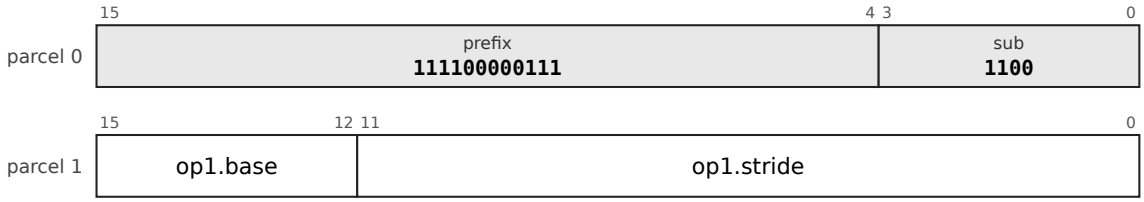
JMP: SOF.A32, prefix 111100001010, sub-opcode 1100; operands abs32; 6 bytes



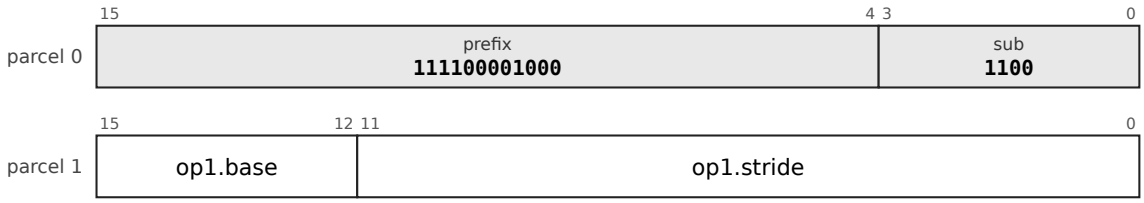
JMP: CALLF, prefix 111100101111000, sub-opcode 1; operands pc32-u2; 6 bytes



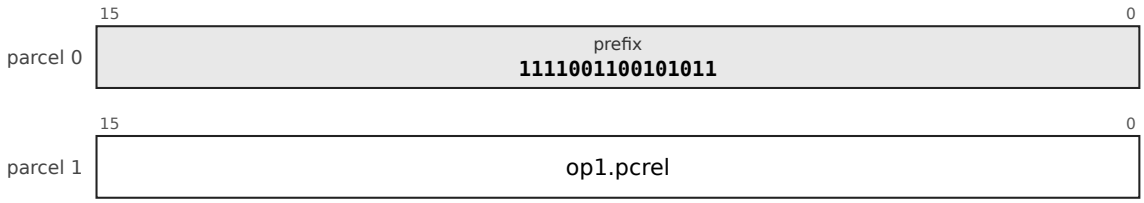
JMP: SOF.PI12, prefix 111100000111, sub-opcode 1100; operands pi4-s12; 4 bytes



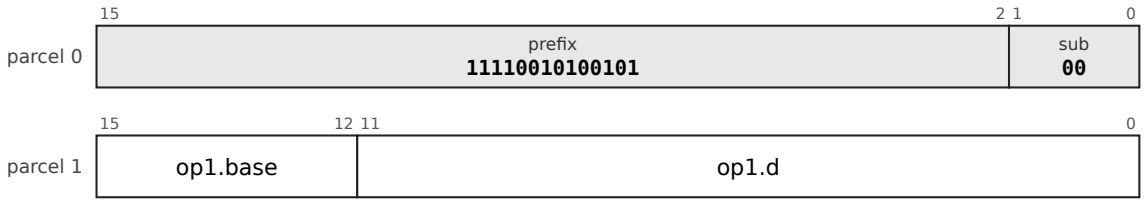
JMP: SOF.PD12, prefix 111100001000, sub-opcode 1100; operands pd4-s12; 4 bytes



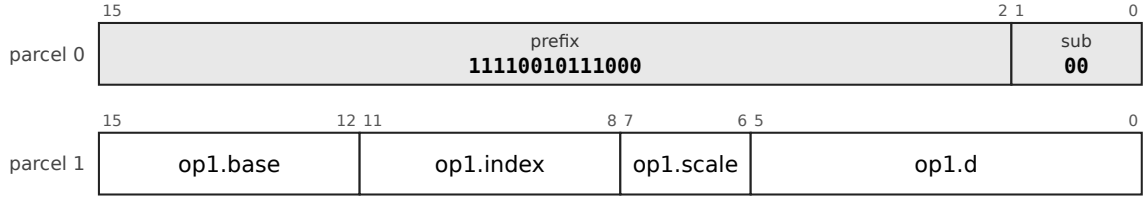
JMP: CALL2_16/j, prefix 1111001100101011; operands pc16-u2; 4 bytes



JMP: SOBu12/x, prefix 11110010100101, sub-opcode 00; operands b4d12u-xsz; 4 bytes



JMP: SOX6/x, prefix 11110010111000, sub-opcode 00; operands x4-4d6s; 4 bytes



JMP: SOPR, prefix 11101110100, sub-opcode 0; operands reg4; 2 bytes

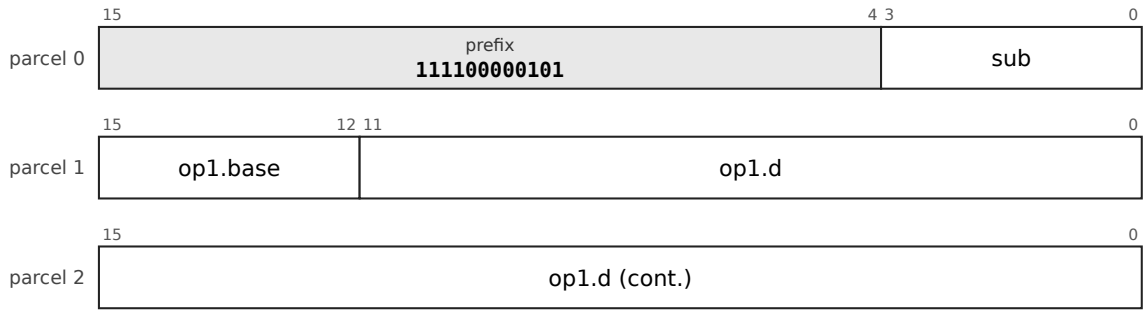


Mnemonics: `JMP`

Syntax

```
JMP rm        ; a register, or an address by any memory mode
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-CALLF-1`, `16-SOPR-3`, `16-CALL2-16-j-1`, `16-SOBu12-x-1`, `16-SOX6-x-1`.

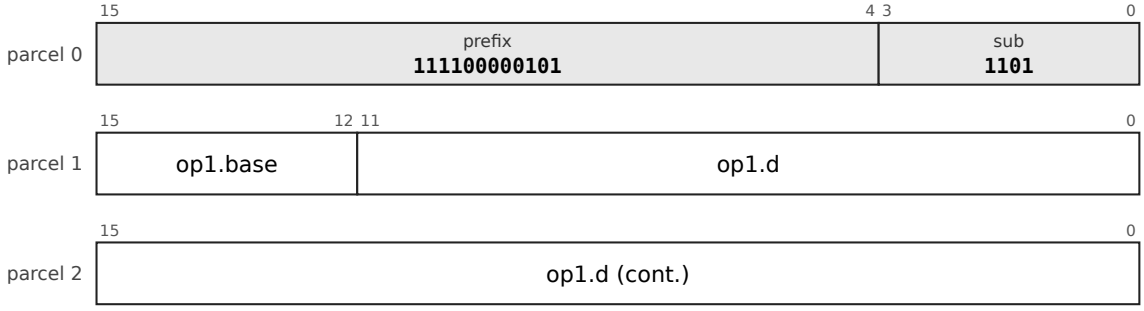
Operation

```
PC ← (register operand) ? Rm : EA(rm)
```

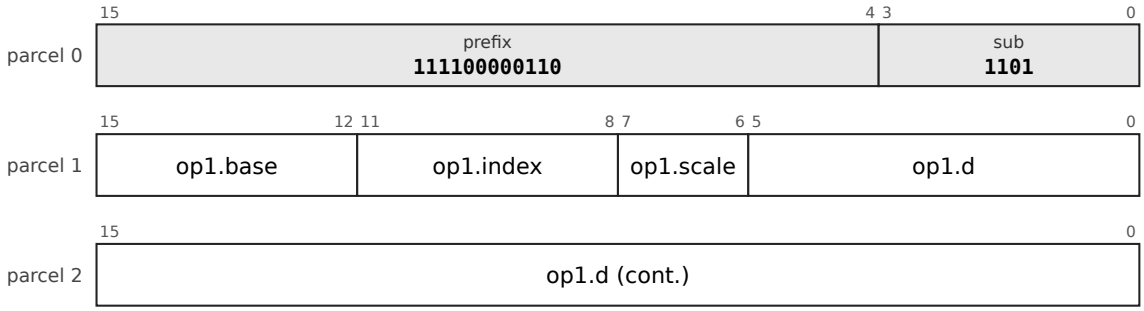
Flags None written.
Exceptions A jump to an unmapped or non-executable address faults on the fetch of the target.
A switch is a `CMP` against the limit, a `BHIS` to the default and a `JMP` through a scaled index.
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

CALL: Call

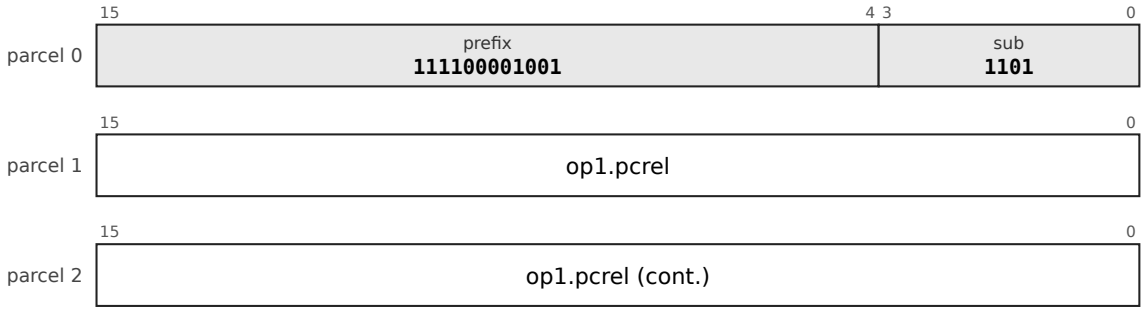
CALL: SOF.B28, prefix 111100000101, sub-opcode 1101; operands b4d28s-x1; 6 bytes



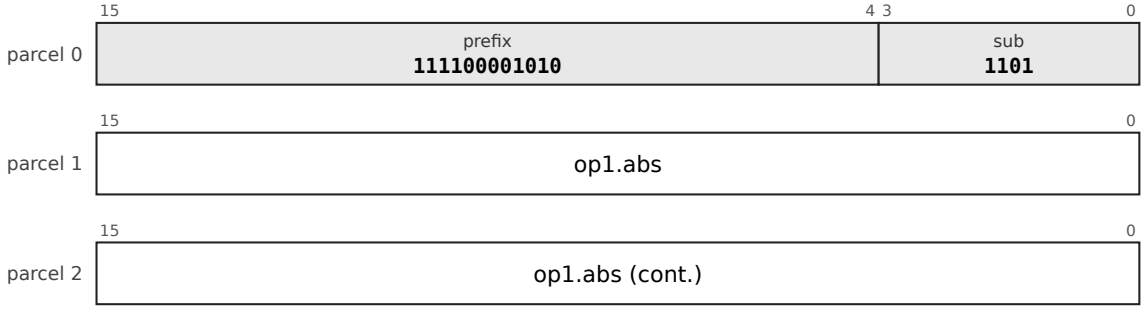
CALL: SOF.X22, prefix 111100000110, sub-opcode 1101; operands x4-4d22s; 6 bytes



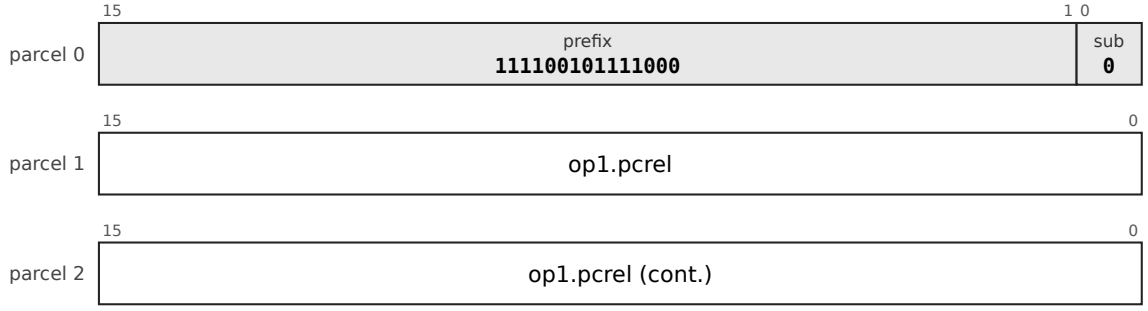
CALL: SOF.PC32, prefix 111100001001, sub-opcode 1101; operands pc32-u1; 6 bytes



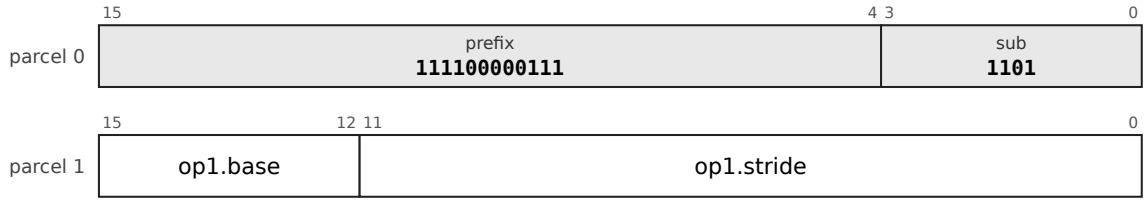
CALL: SOF.A32, prefix 111100001010, sub-opcode 1101; operands abs32; 6 bytes



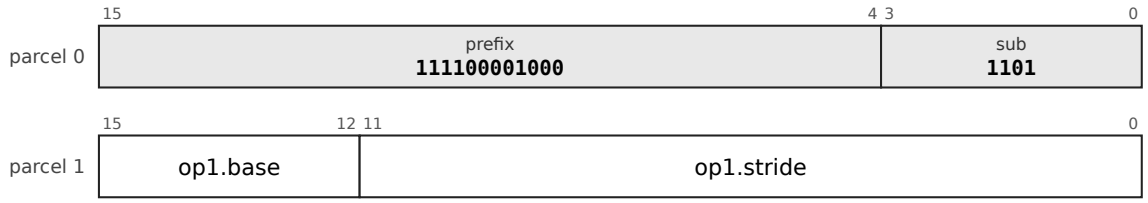
CALL: CALLF, prefix 111100101111000, sub-opcode 0; operands pc32-u2; 6 bytes



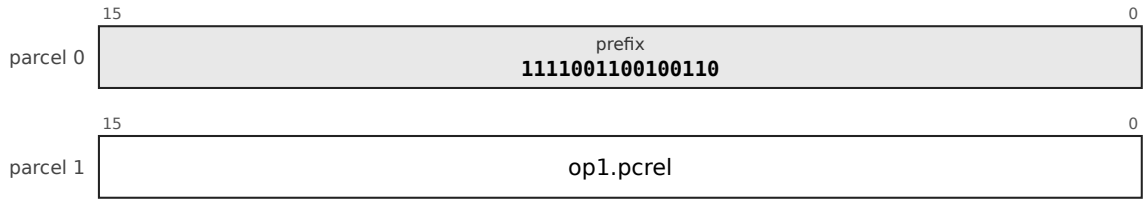
CALL: SOF.PI12, prefix 111100000111, sub-opcode 1101; operands pi4-s12; 4 bytes



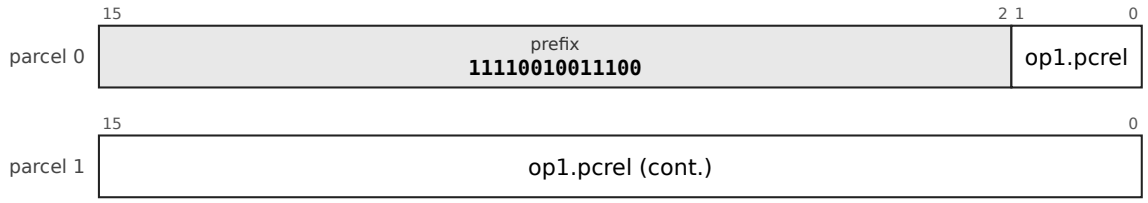
CALL: SOF.PD12, prefix 111100001000, sub-opcode 1101; operands pd4-s12; 4 bytes



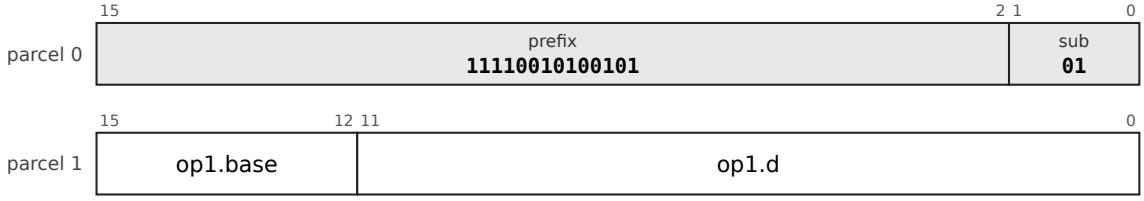
CALL: CALL2_16/c, prefix 1111001100100110; operands pc16-u2; 4 bytes



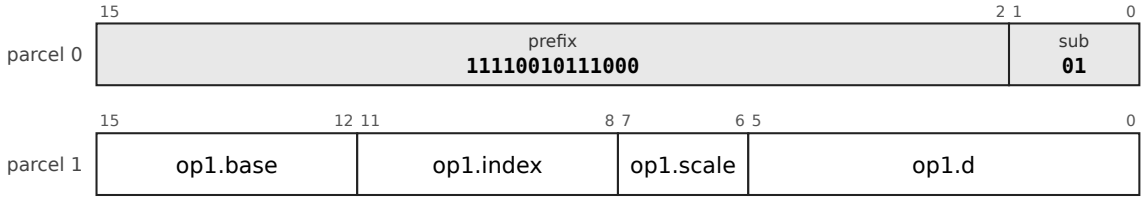
CALL: CALL2_18/c, prefix 11110010011100; operands pc18-u2; 4 bytes



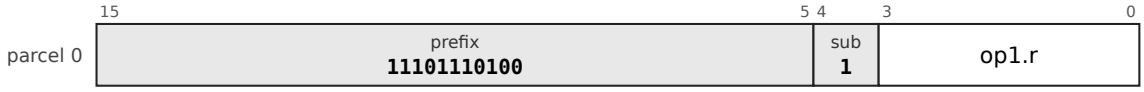
CALL: SOBu12/x, prefix 11110010100101, sub-opcode 01; operands b4d12u-xsz; 4 bytes



CALL: SOX6/x, prefix 11110010111000, sub-opcode 01; operands x4-4d6s; 4 bytes



CALL: SOPR, prefix 11101110100, sub-opcode 1; operands reg4; 2 bytes

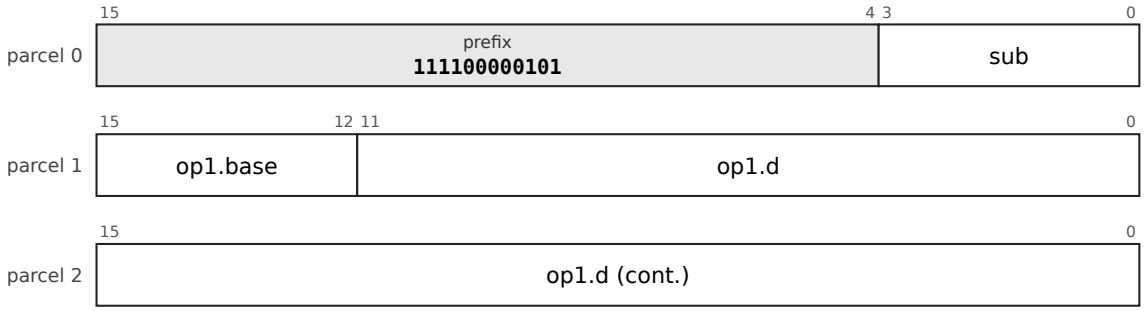


Mnemonics: `CALL`

Syntax

```
CALL rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-CALLF-1`, `16-SOPR-3`, `16-CALL2-16-c-1`, `16-CALL2-18-c-1`, `16-SOBu12-x-1`, `16-SOX6-x-1`.

Operation

```
t ← (register operand) ? Rm : EA(rm)
SP ← SP - 8 ; M[SP] ← address of the next instruction
PC ← t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The return address is on the data stack where every unwinder and debugger expects it. RET pops it.

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

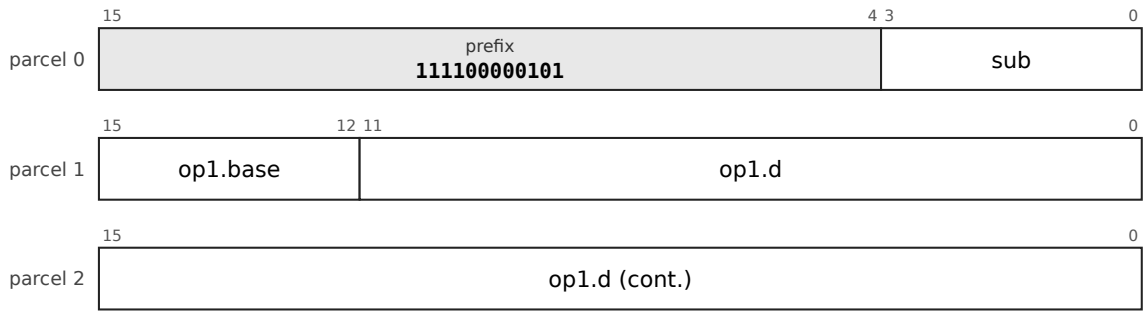
JMPI: Jump and call through memory

Mnemonics: JMPI, CALLI

Syntax

```
JMPI rm        ; PC ← M[EA]
CALLI rm       ; push the return address, then PC ← M[EA]
```

Encoding



The chart is the leaf 16-SOF-B28-1, SOF.B28, prefix 111100000101. op1 is b4d28s-x1: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. sub (4 bits) selects the mnemonic: clr 0000, com 0001, inc 0010, dec 0011, neg 0100, tst 0101, adc 0110, sbc 0111, asl 1000, asr 1001, push 1010, pop 1011, jmp 1100, call 1101, jmpi 1110, calli 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOBu12-x-1, 16-SOX6-x-1.

Operation

```
JMPI : PC ← M[EA(rm)]
CALLI: t ← M[EA(rm)] ; SP ← SP - 8 ; M[SP] ← address of the next instruction ; PC ← t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. For CALLI the order is specified: load the target, then push, then transfer; a fault on the load leaves nothing changed.

The one deferred reference the architecture admits, for control transfer only: the PLT stub (JMPI got_slot(PC)) and virtual-method dispatch (CALLI vtbl_off(Robj)). It needs a memory operand.

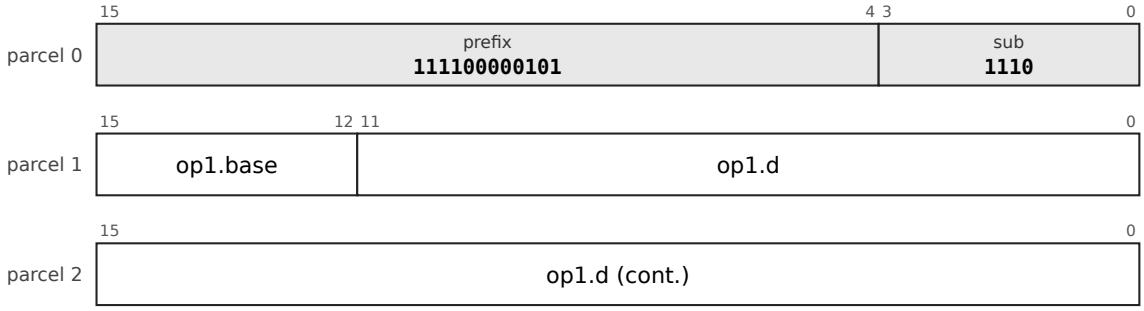
Why it is here. Exactly one deferred reference, for control transfer only: the PLT stub of every dynamically linked program (JMPI got_slot(PC)) and virtual-method dispatch (CALLI vtbl_off(Robj)), each two instructions and a scratch register without it. It is admitted because it breaks none of the reasons deferred modes are removed (no register is side-effected, the reference is

in the branch path and not the operand path, a fault is clean by ordering). Jump tables through `JMPI` are not smaller than a compare chain, because a table entry is 8 bytes and a case in a compare chain is also 8.

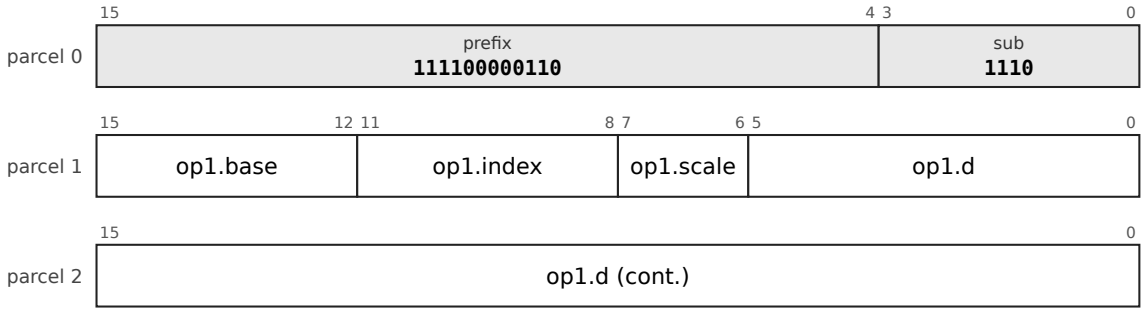
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

JMPI

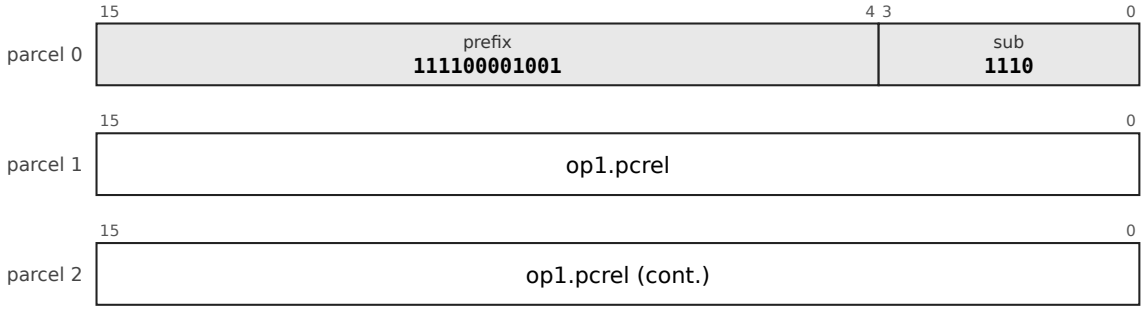
JMPI: SOF.B28, prefix 111100000101, sub-opcode 1110; operands b4d28s-x1; 6 bytes



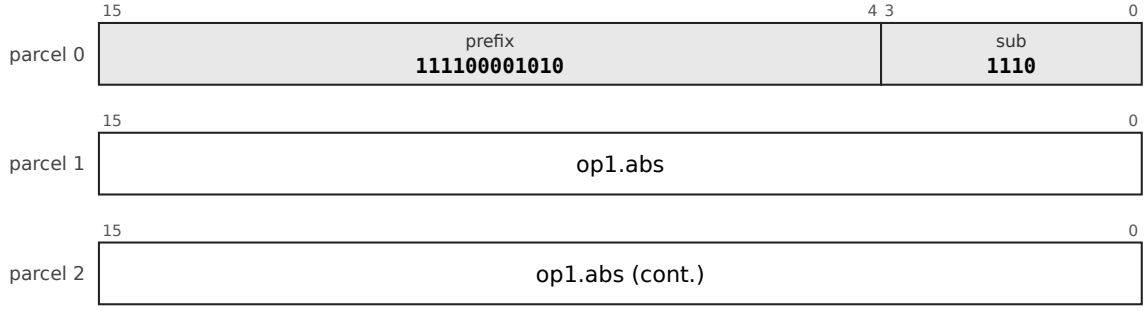
JMPI: SOF.X22, prefix 111100000110, sub-opcode 1110; operands x4-4d22s; 6 bytes



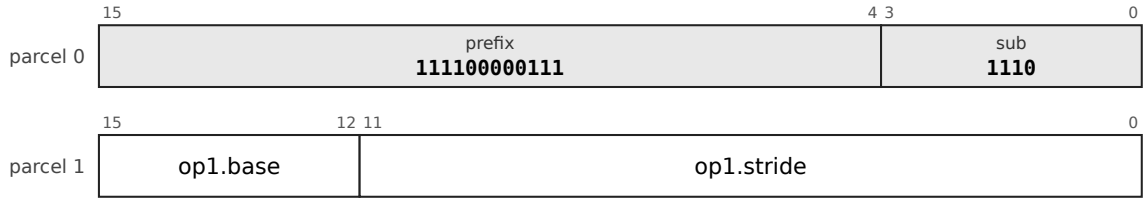
JMPI: SOF.PC32, prefix 111100001001, sub-opcode 1110; operands pc32-u1; 6 bytes



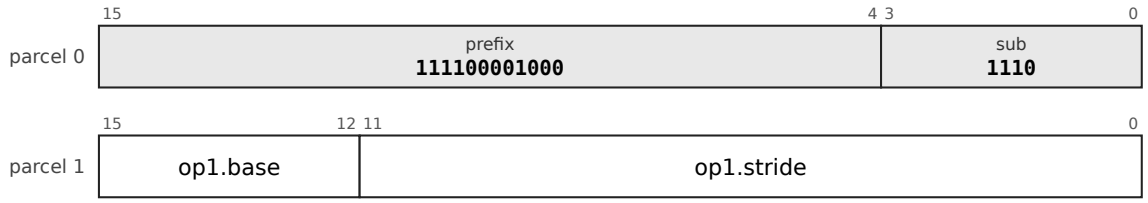
JMPI: SOF.A32, prefix 111100001010, sub-opcode 1110; operands abs32; 6 bytes



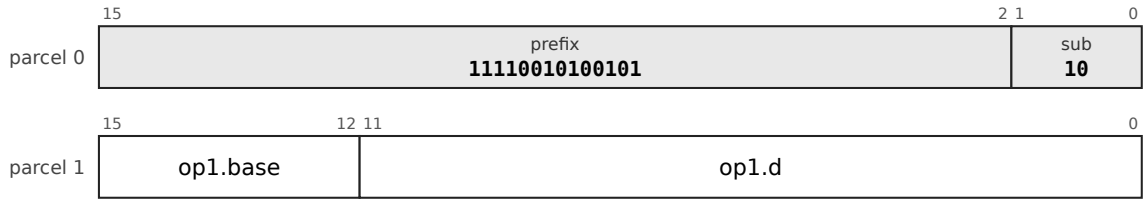
JMPI: SOF.PI12, prefix 111100000111, sub-opcode 1110; operands pi4-s12; 4 bytes



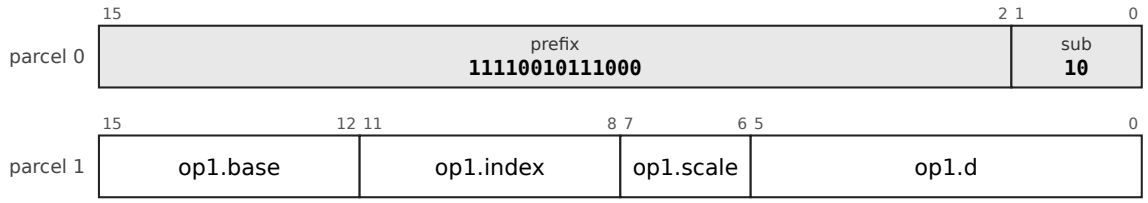
JMPI: SOF.PD12, prefix 111100001000, sub-opcode 1110; operands pd4-s12; 4 bytes



JMPI: SOBu12/x, prefix 11110010100101, sub-opcode 10; operands b4d12u-xsz; 4 bytes



JMPI: SOX6/x, prefix 11110010111000, sub-opcode 10; operands x4-4d6s; 4 bytes

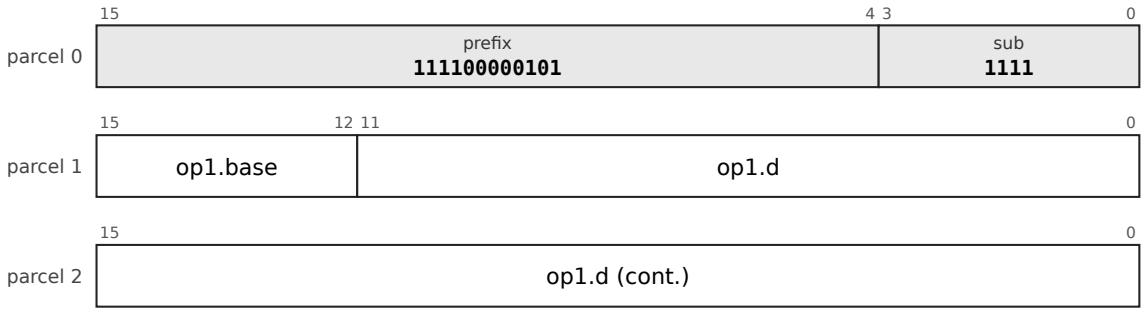



```
JMPI rm
```

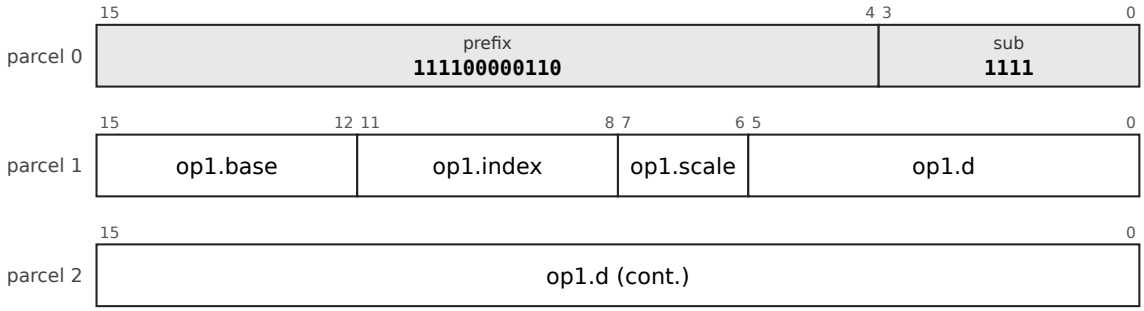
Operation, flags and exceptions as for the entry JMPI above; 8 layout forms, each a chart of its own (above).

CALLI

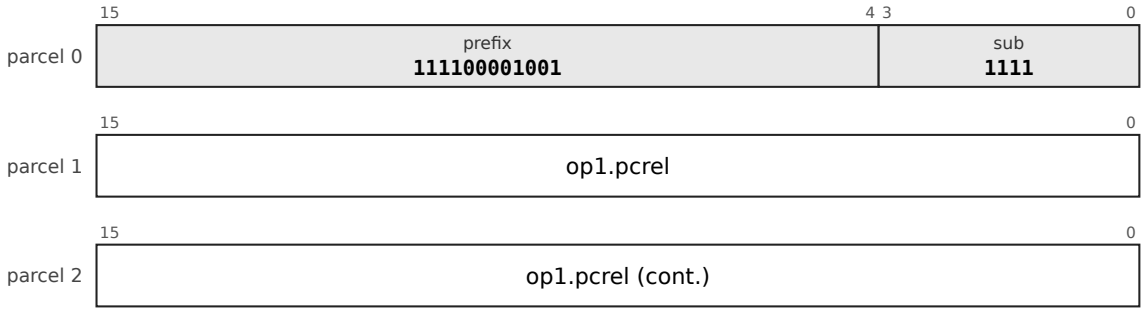
CALLI: SOF.B28, prefix 111100000101, sub-opcode 1111; operands b4d28s-x1; 6 bytes



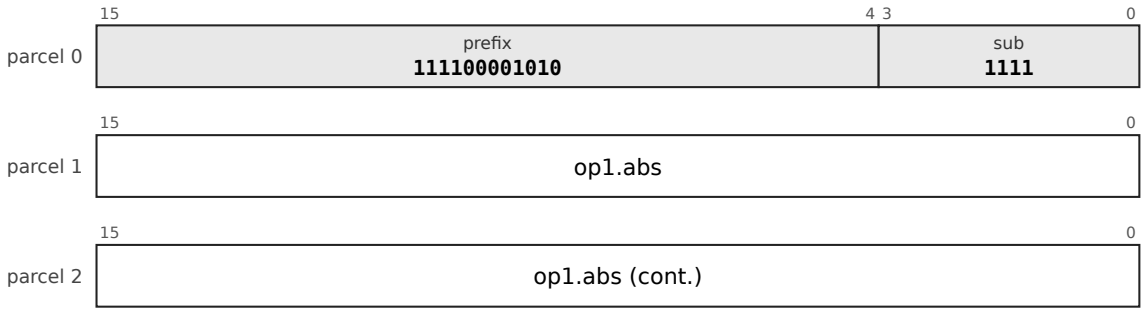
CALLI: SOF.X22, prefix 111100000110, sub-opcode 1111; operands x4-4d22s; 6 bytes



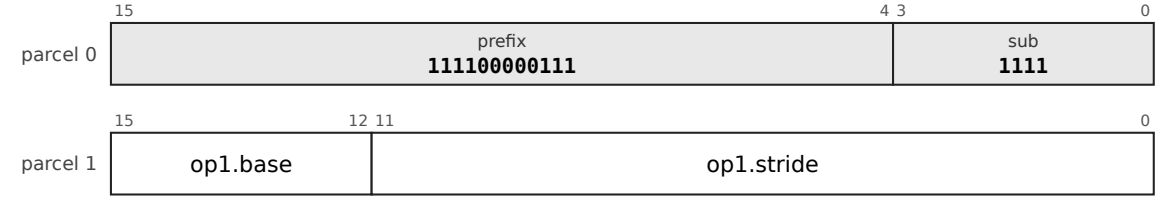
CALLI: SOF.PC32, prefix 111100001001, sub-opcode 1111; operands pc32-u1; 6 bytes



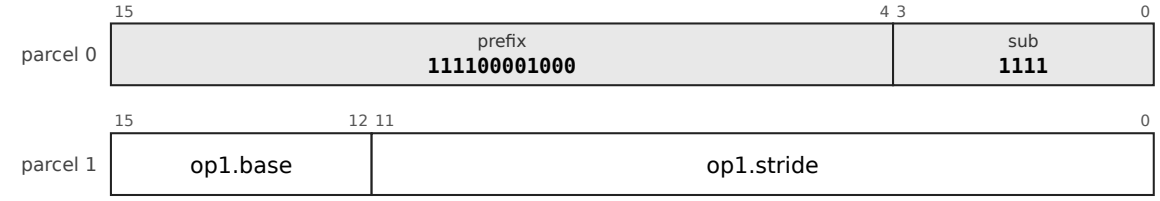
CALLI: SOF.A32, prefix 111100001010, sub-opcode 1111; operands abs32; 6 bytes



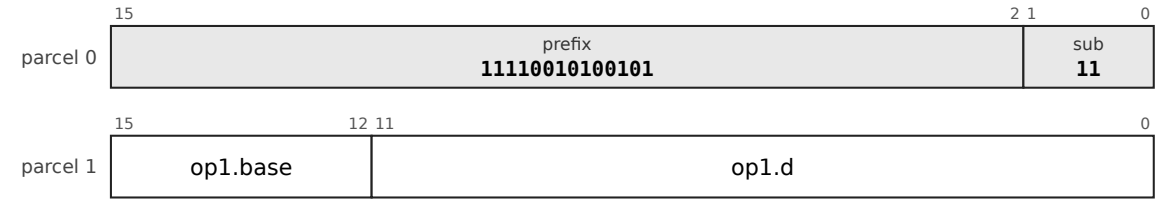
CALLI: SOF.PI12, prefix 111100000111, sub-opcode 1111; operands pi4-s12; 4 bytes



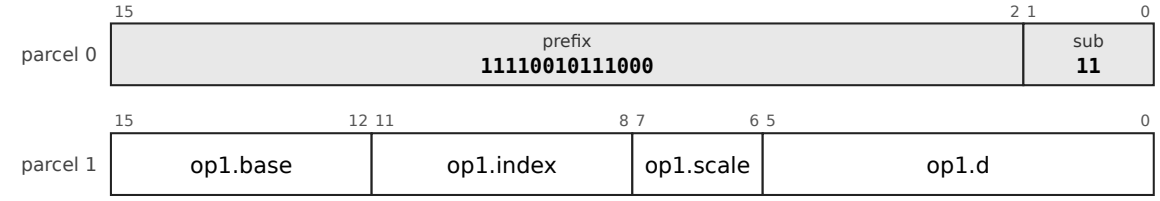
CALLI: SOF.PD12, prefix 111100001000, sub-opcode 1111; operands pd4-s12; 4 bytes



CALLI: SOBu12/x, prefix 11110010100101, sub-opcode 11; operands b4d12u-xsz; 4 bytes



CALLI: SOX6/x, prefix 11110010111000, sub-opcode 11; operands x4-4d6s; 4 bytes

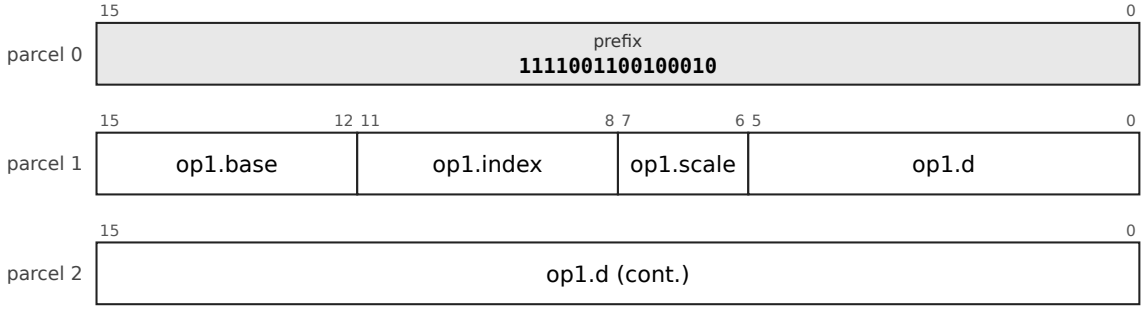



```
CALLI rm
```

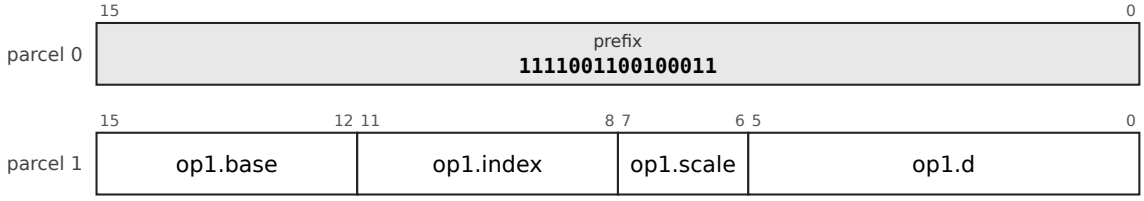
Operation, flags and exceptions as for the entry JMPI above; 8 layout forms, each a chart of its own (above).

JTAG: Jump through a tag table

JTAG: JTAG3, prefix 1111001100100010; operands x4-4d22s; 6 bytes



JTAG: JTAG2, prefix 1111001100100011; operands x4-4d6s; 4 bytes

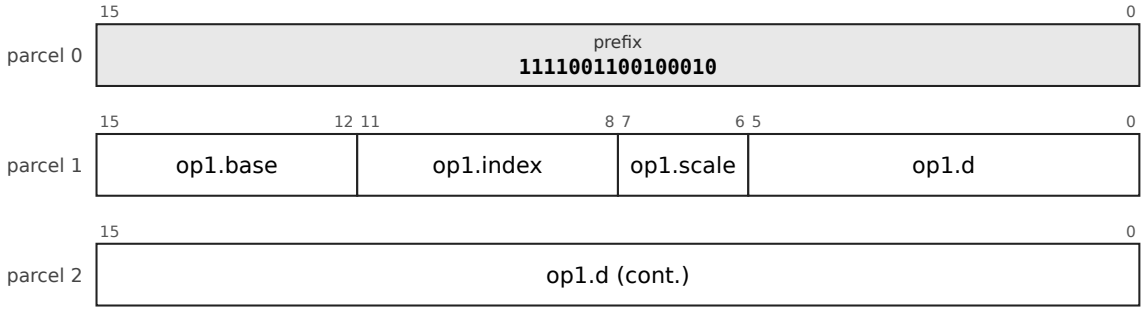


Mnemonics: `JTAG`

Syntax

```
JTAG d(Rb)(Rx×s)
```

Encoding



The chart is the leaf `16-JTAG3-1`, JTAG3, prefix `1111001100100010`. `op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-JTAG3-1`, `16-JTAG2-1`.

Operation

```
PC ← M[Rb + d + 8 × (Rx & 111₂)]
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

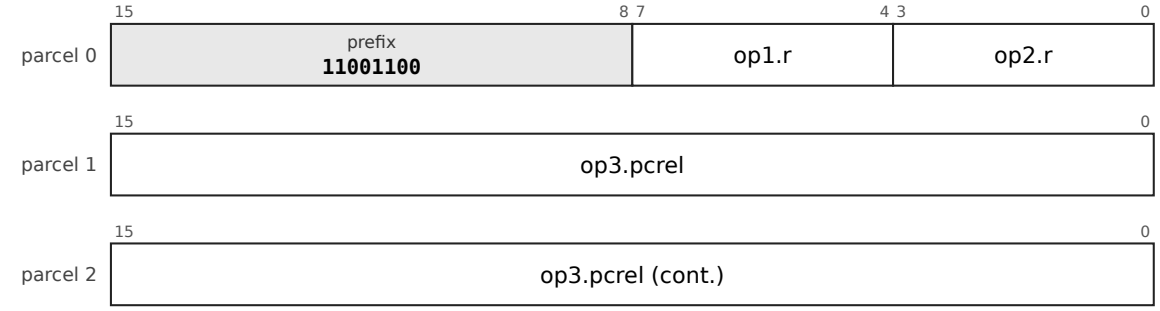
Legal only with the scaled-index mode (`0001`). Dispatches on the low three bits of a tagged pointer through a table of eight entries; the 3-bit tag cannot index out of range.

Why it is here. The table-dispatch partner of `BTAG`. It is legal only in the scaled-index mode, the one mode with a free register slot for the tagged value. It replaces the PDP-11 dispatch-table idiom `@(Rn)+` better, because it needs no pointer register and cannot index out of range.

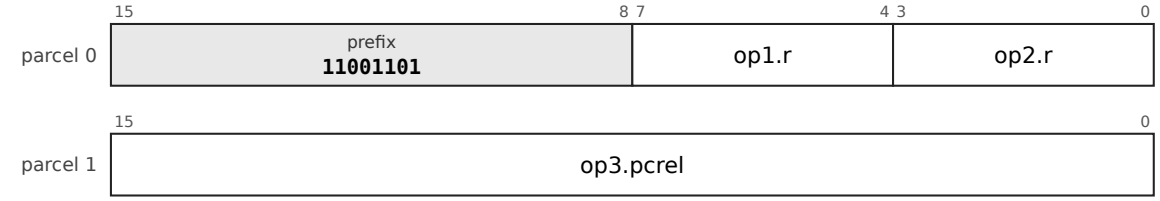
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MOVCALL: Move a register and call

MOVCALL: FX.MOVCALL3, prefix 11001100; operands reg4, reg4, pc32-u2; 6 bytes



MOVCALL: FX.MOVCALL2, prefix 11001101; operands reg4, reg4, pc16-u2; 4 bytes

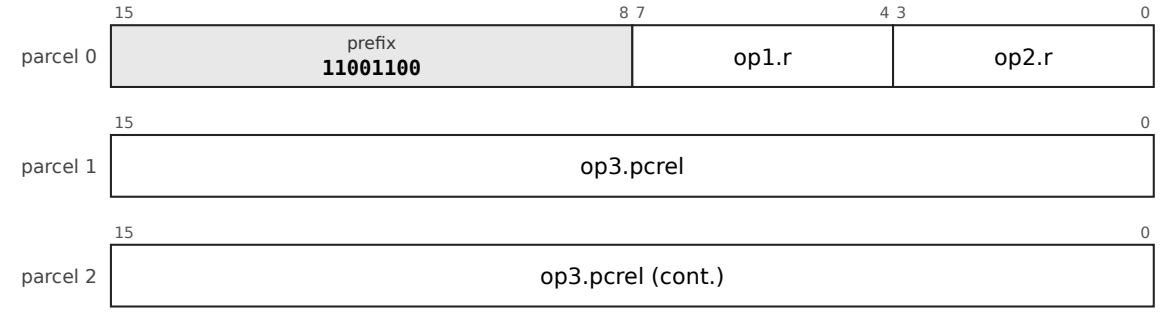


Mnemonics: `MOVCALL`

Syntax

```
MOVCALL Rs, Rd, target
```

Encoding



The chart is the leaf `16-FX-MOVCALL3-1`, FX.MOVCALL3, prefix `11001100`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-FX-MOVCALL3-1`, `16-FX-MOVCALL2-1`.

Operation

```
Rd ← Rs
SP ← SP - 8 ; M[SP] ← address of the next instruction
PC ← target
```

Flags None written.

Exceptions The store of the return address can fault (the memory faults of `CALL`): the instruction restarts whole, because the first half reads a register that the second half does not change.

(D) One of the four fused pairs of errata 87: `MOVQ Rs, Rd` followed by `CALL target`, in one instruction. The target is a pc-relative field in parcels, measured from the first parcel of the fused instruction (errata 87, assumption 9); the instruction is two parcels (a 16-bit field) or three (a 32-bit field), and the linker grows it from its shortest form like a `CALL` (`docs/relaxation.md`). **The assembler never fuses**: the compiler writes `MOVCALL` where it wants it. An instruction that writes two registers (`Rd` and the stack pointer) and stores to memory is a two-micro-operation instruction for an out-of-order core (errata 87).

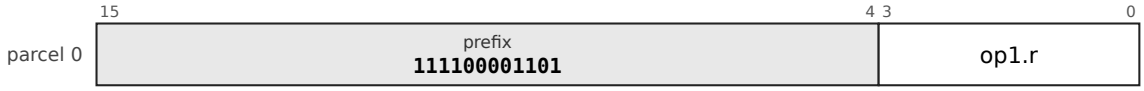
Why it is here. The author decided the four fused pairs in errata 87 (`movcall`, `popmret`, `popret`, `mov2`) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of `docs/encoding-proposal.md` section 8 (static size of the training corpus, in pdp64/16 and pdp64/32): `MOVQ Rs, Rd` followed by `CALL` occurs 11,123 times, and fusing it alone shrinks the code by 2.19% (pdp64/16) and 2.39% (pdp64/32), the largest of the four; all four together fuse 20,428 pairs and save 3.29% and 3.18% of the bytes and about 1.1% of the executed instructions. The call keeps its reach (four or six bytes). The cost, named in the proposal: a new mnemonic, and one instruction that writes two registers (the destination and the stack pointer), which is two micro-operations for an out-of-order core. The record gives no earlier machine for this particular pair; the reasoning is the fourth priority of the definitive list (errata 83), fewer instructions executed, where coalescing is good.

Cf. 9.5 (the pc-relative operand and the relaxation), 6.2 (restart after a fault).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

POPRET: Pop a register and return

POPRET: FX.POPRET, prefix 111100001101; operands reg4; 2 bytes

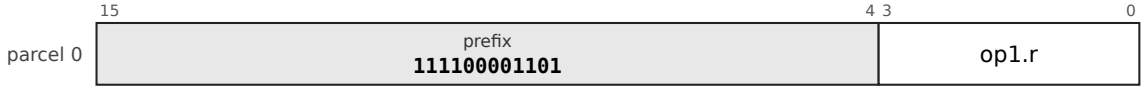


Mnemonics: `POPRET`

Syntax

```
POPRET Rd
```

Encoding



The chart is the leaf `16-FX-POPRET-1`, FX.POPRET, prefix `111100001101`. `op1` is `reg4`: register, 4 bits.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-FX-POPRET-1`.

Operation

```
Rd ← M[SP] ; SP ← SP + 8
PC ← M[SP] ; SP ← SP + 8
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

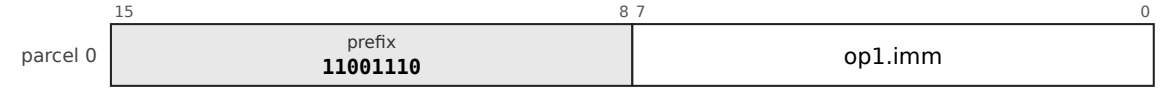
(D) POP Rd followed by RET, fused (errata 87). The register is the one the function restores last, usually a callee-saved register or the frame pointer. One parcel of 16 bits at pdp64/16 with the register in a 4-bit field; at pdp64/32 the 5-bit field makes the prefix one bit shorter.

Why it is here. The author decided the four fused pairs in errata 87 (movcall, popmret, popret, mov2) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of docs/encoding-proposal.md section 8 (static size of the training corpus, in pdp64/16 and pdp64/32): POP Rd followed by RET and POPM followed by RET (POPMRET) occur 4,862 times together and save 0.95% (pdp64/16) and 1.00% (pdp64/32); this is the epilogue of nearly every function. The fault of the second half restarts the whole instruction, which is idempotent because the first half reads a register the second does not change and POPM is already specified restartable (the proposal's decision 2).

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

POPMRET: Pop several registers and return

POPMRET: FX.POPMRET, prefix 11001110; operands imm8u-x256; 2 bytes

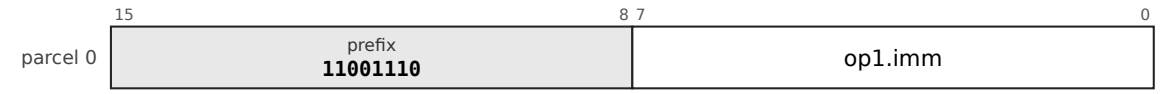


Mnemonics: POPMRET

Syntax

```
POPMRET #mask
```

Encoding



The chart is the leaf 16-FX-POPMRET-1, FX.POPMRET, prefix 11001110. op1 is imm8u-x256: unsigned 8-bit immediate times 256, 0 to 65280.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): 16-FX-POPMRET-1.

Operation

```
for each bit i set in mask, highest register first:
    Ri ← M[SP] ; SP ← SP + 8
PC ← M[SP] ; SP ← SP + 8
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A fault after some registers have been popped leaves them popped: the instruction restarts whole and pops them again, which is idempotent because the stack pointer is restored with the rest of the state at the start of the instruction.

(D) POPM #mask followed by RET, fused (errata 87): the epilogue of every function that saved registers with PUSHM. The mask is an 8-bit field scaled by 256, so the instruction names R8 to R15; any other set of registers is POPM and RET. One parcel.

Why it is here. See POPRET: the two are measured together (4,862 pairs, 0.95% and 1.00% of the bytes). The mask is an 8-bit field scaled by 256 because one parcel has no room for a wider one; the epilogue that restores R8 to R15 is the common case the corpus shows, and any other set stays POPM and RET.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

9.11 The stack and block operations

Cf. section 6.8 for restarting a block operation after a fault (progress is held in registers) and section 6.4 for the page faults it can take; section 7.2 for how block operations order their accesses.

9.11.1 PUSH and POP move one register; PUSHM and POPM move a set named by a mask (a mask of 8 bits is two bytes). A mask containing R14 is reserved. *(D)*

9.11.2 PUSHX and POPX save and restore the exception state EPSW and EPC in one instruction each; both words are checked before either is accessed, so an abort leaves nothing changed. *(D)*

9.11.3 The block operations MCPY, MMOV (overlap-safe), MSET, MCMP, MSCN and MSCNN take a destination pointer, a source pointer or value and a count in registers; MVAL validates a buffer as UTF-8.

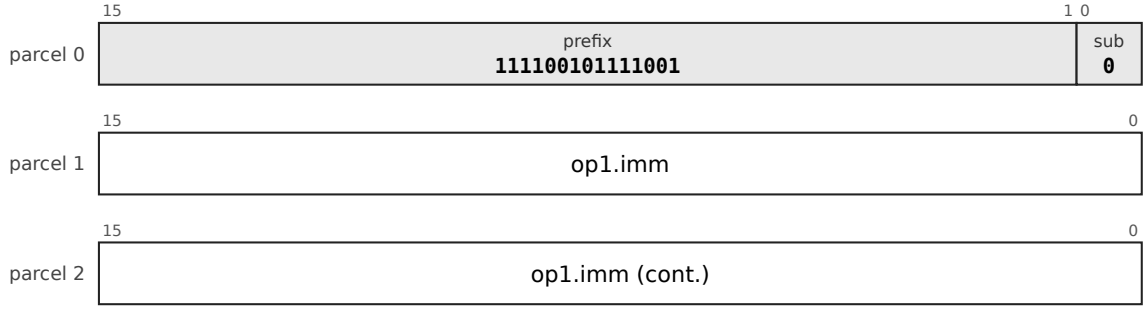
9.11.4 A block operation keeps its progress in its registers: when it completes, the count is zero and the pointers are one past the end; when it stops early or is interrupted, they point where it stopped, and it restarts by running again. *(D)*

9.11.5 An implementation may move more than a byte per step, provided the registers are consistent with byte-at-a-time progress wherever an exception can be taken. *(D)*

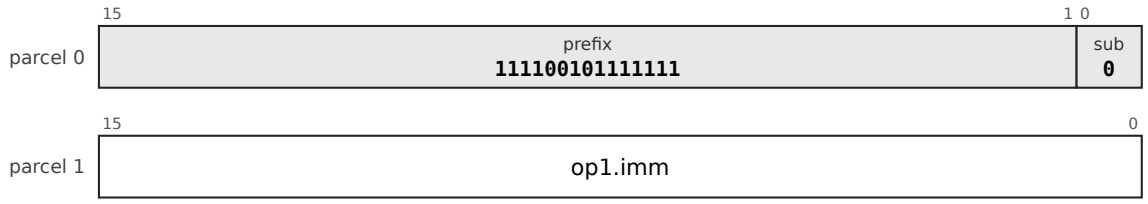
Commentary. **Precedent.** The VAX's MOVC5 is the canonical argument against such instructions: its operands could be memory, its architectural definition needed the original values to survive, and so it required hidden state and a first-part-done bit in the status word to resume. The PDP-10 had a block transfer, and x86's REP MOVS keeps its progress in registers as the pdp64's do. **Trade-offs.** This is the one admitted exception to the one-memory-operand rule, accepted because its mechanism, hidden state, is absent: the state is the registers, which are already architectural. A large copy can evict the whole data cache, and a non-temporal variant is an open question. **Improvements.** ARM reached the same design in 2022 with FEAT_MOPS, whose specification says the registers are updated so that progress is not lost. **Benefits.** The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors. **Modern comparables.** x86 REP MOVSB and REP STOSB with fast-string microcode; AArch64 CPYP/CPYM/CPYE and SETP/SETM/SETE; s390 MVC. **Use.** `memcpy`, `memmove`, `memset`, `memcmp` and `memchr` in the C library; MVAL in parsers and kernels. **Impact.** *Kernel:* PUSHM, POPM, PUSHX and POPX shorten entry, exit and the context switch, and the block operations are `copyin`, `copyout`, page clearing and `memcpy`. *Compiler:* A prologue and an epilogue are one instruction each, and `memcpy` and `memset` lower to the block operations. *Hardware:* The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors, with no hidden state.

PUSHM: Push registers by mask

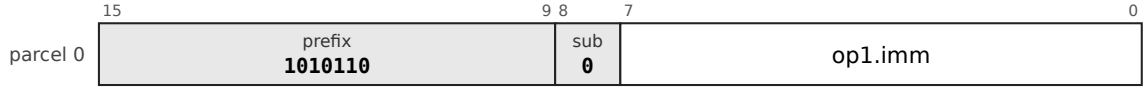
PUSHM: PUSHMF, prefix 111100101111001, sub-opcode 0; operands imm32u; 6 bytes



PUSHM: PUSHM16a/a, prefix 111100101111111, sub-opcode 0; operands imm16u; 4 bytes



PUSHM: PUSHM8/a, prefix 1010110, sub-opcode 0; operands imm8u-x256; 2 bytes

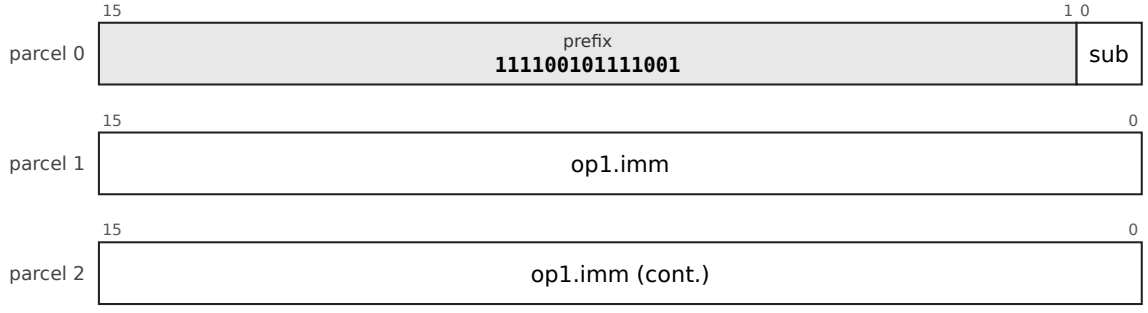


Mnemonics: `PUSHM`

Syntax

```
PUSHM #mask        ; a 16-bit immediate, bit n is Rn
```

Encoding



The chart is the leaf `16-PUSHMF-1`, PUSHMF, prefix `111100101111001`. `op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295. `sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-PUSHMF-1`, `16-PUSHM16a-a-1`, `16-PUSHM8-a-1`.

Operation

```
for n = 0 to 15, if mask bit n:
    SP ← SP - 8 ; M[SP] ← Rn
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A mask that includes R14 (SP) is reserved.

The registers are pushed in ascending order, so POPM with the same mask restores them. The mask is the low 32 bits of the number (errata 87, assumption 7); the one-parcel form takes an 8-bit mask scaled by 256, which names R8 to R15.

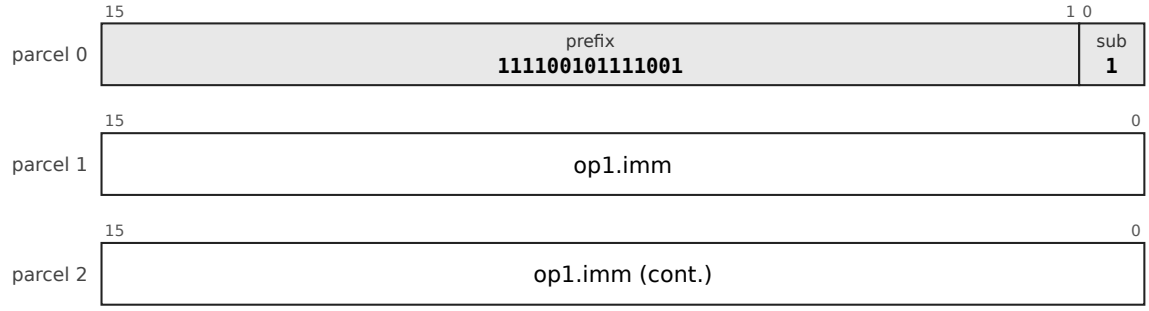
Why it is here. The VAX procedure entry mask was the useful half of CALLS; PUSHM and POPM keep that half without the frame (hardware call frames ran slower than the sequence on later VAXes). The one-parcel forms take an 8-bit mask for R8 to R15, because all 2,392 push sites and 2,392 pop sites in the measured corpus use a mask within R8 to R15.

Cf. 6.8 for the restart rule.

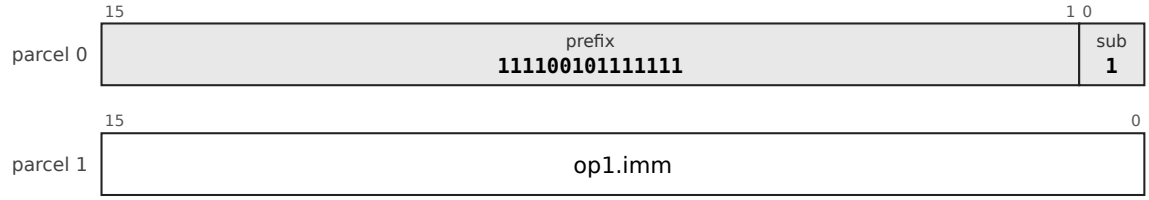
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

POPM: Pop registers by mask

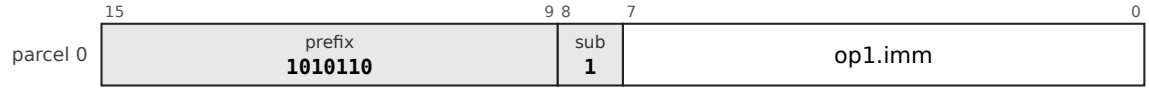
POPM: PUSHMF, prefix 111100101111001, sub-opcode 1; operands imm32u; 6 bytes



POPM: PUSHM16a/a, prefix 111100101111111, sub-opcode 1; operands imm16u; 4 bytes



POPM: PUSHM8/a, prefix 1010110, sub-opcode 1; operands imm8u-x256; 2 bytes

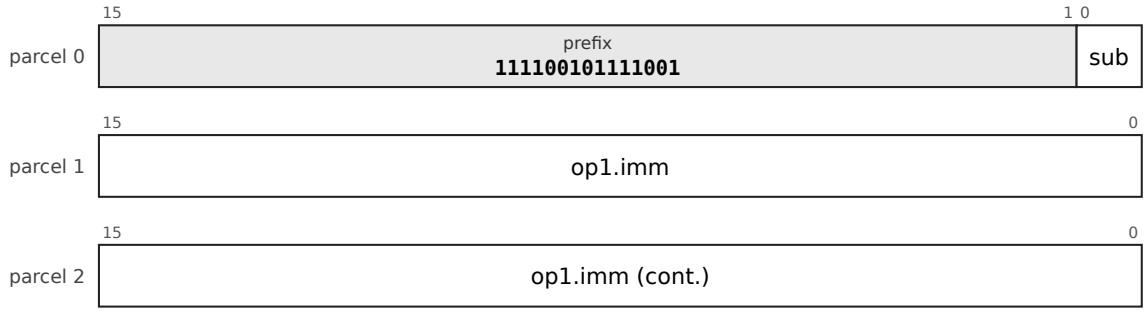


Mnemonics: POPM

Syntax

```
POPM #mask
```

Encoding



The chart is the leaf `16-PUSHMF-1`, PUSHMF, prefix `111100101111001`. `op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295. `sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-PUSHMF-1`, `16-PUSHM16a-a-1`, `16-PUSHM8-a-1`.

Operation

```
for n = 15 down to 0, if mask bit n:
    Rn ← M[SP] ; SP ← SP + 8
```

Flags None written.

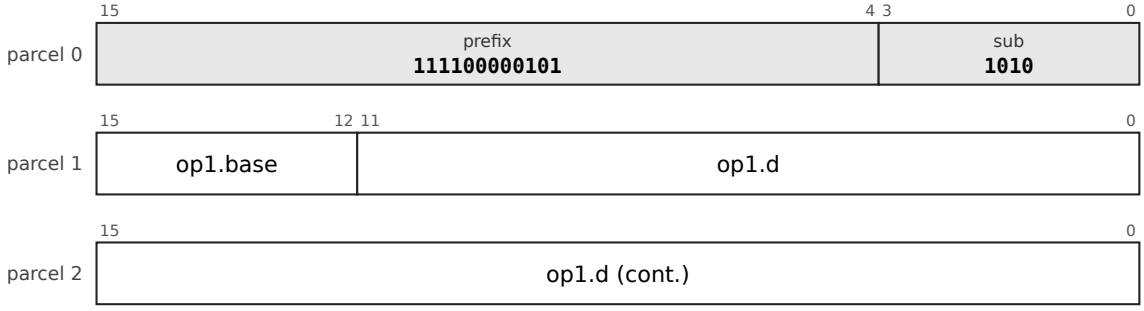
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The mirror of `PUSHM`.

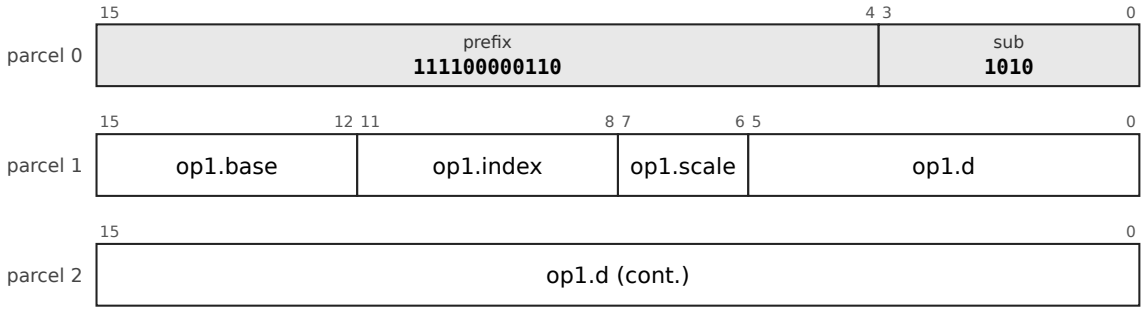
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

PUSH: Push

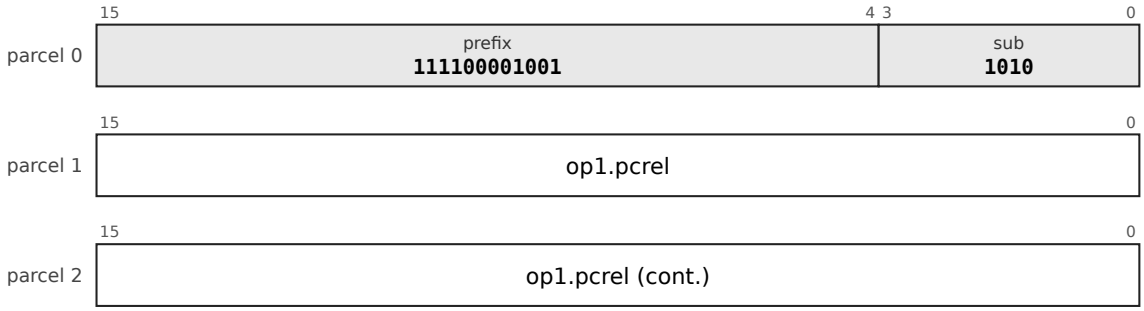
PUSH: SOF.B28, prefix 111100000101, sub-opcode 1010; operands b4d28s-x1; 6 bytes



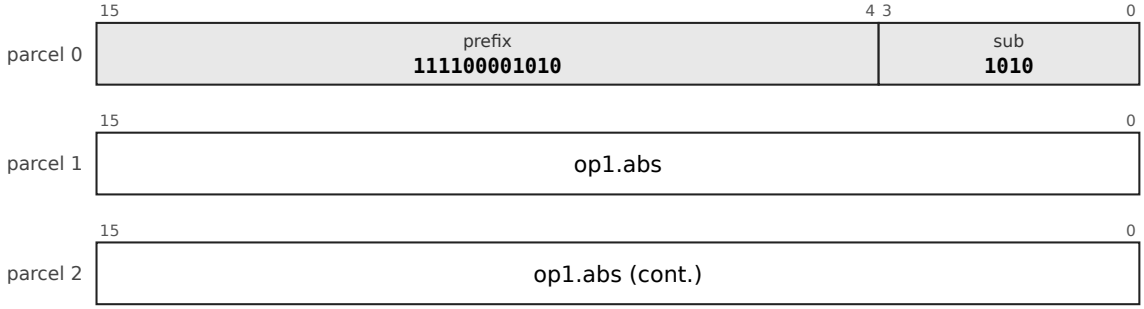
PUSH: SOF.X22, prefix 111100000110, sub-opcode 1010; operands x4-4d22s; 6 bytes



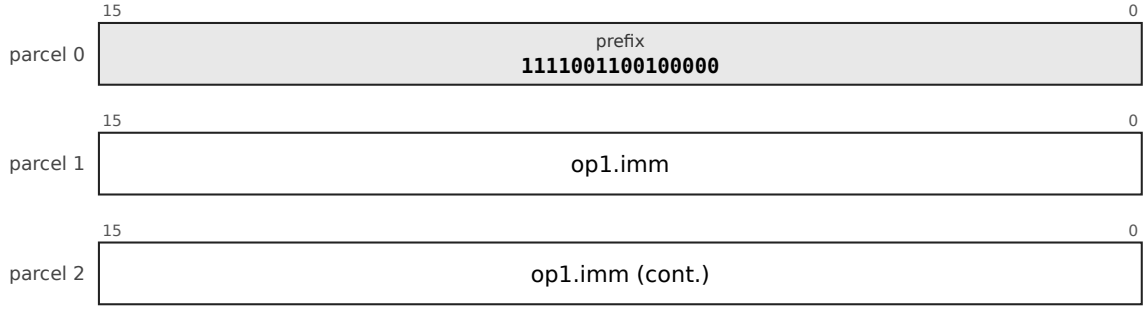
PUSH: SOF.PC32, prefix 111100001001, sub-opcode 1010; operands pc32-u1; 6 bytes



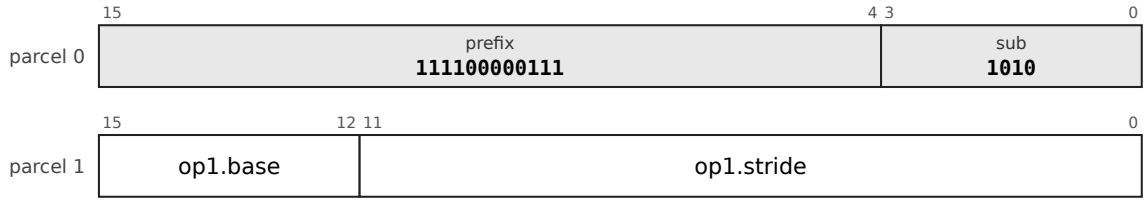
PUSH: SOF.A32, prefix 111100001010, sub-opcode 1010; operands abs32; 6 bytes



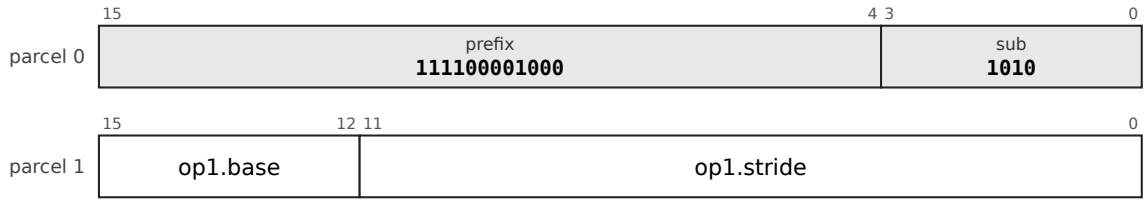
PUSH: PUSHI3, prefix 1111001100100000; operands imm32s; 6 bytes



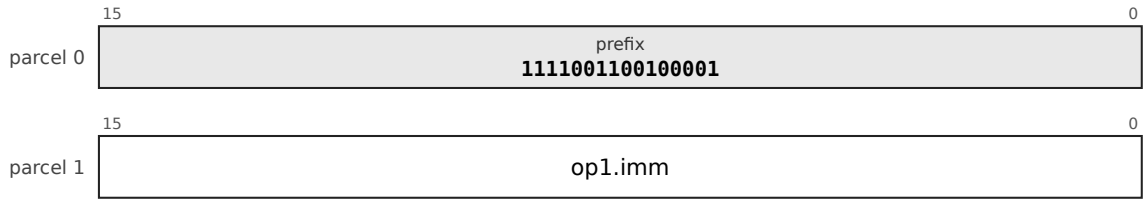
PUSH: SOF.PI12, prefix 111100000111, sub-opcode 1010; operands pi4-s12; 4 bytes



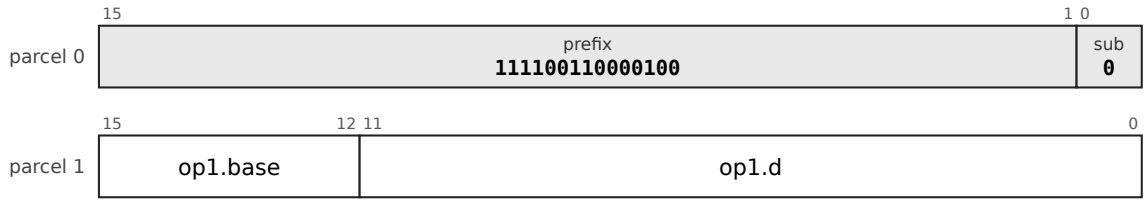
PUSH: SOF.PD12, prefix 111100001000, sub-opcode 1010; operands pd4-s12; 4 bytes



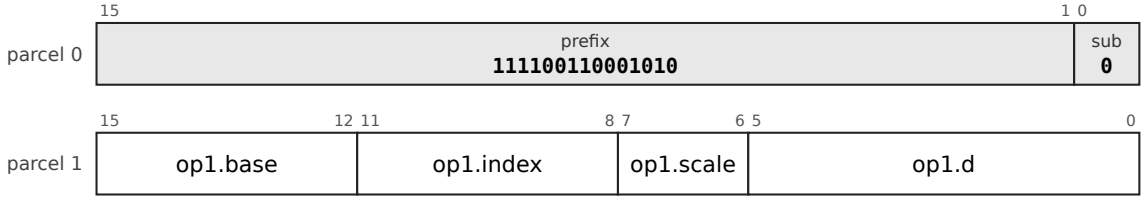
PUSH: PUSHI2, prefix 1111001100100001; operands imm16s; 4 bytes



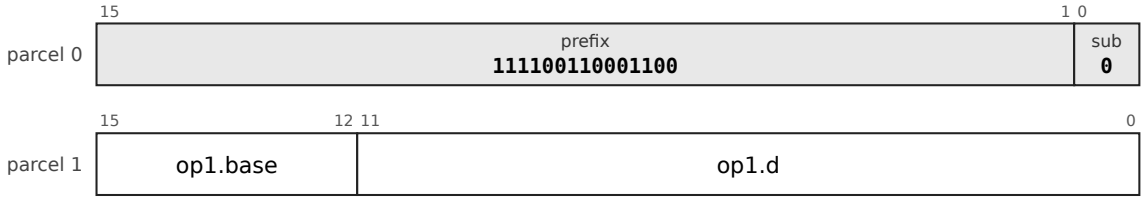
PUSH: SOBu12/a, prefix 111100110000100, sub-opcode 0; operands b4d12u-xsz; 4 bytes



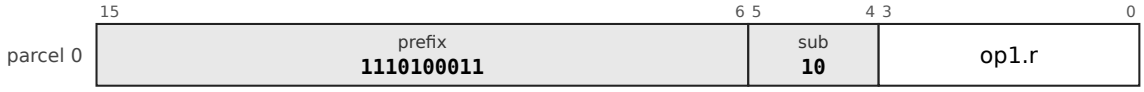
PUSH: SOX6/a, prefix 111100110001010, sub-opcode 0; operands x4-4d6s; 4 bytes



PUSH: SOBs12/a, prefix 111100110001100, sub-opcode 0; operands b4d12s-xsz; 4 bytes



PUSH: SOPR, prefix 1110100011, sub-opcode 10; operands reg4; 2 bytes

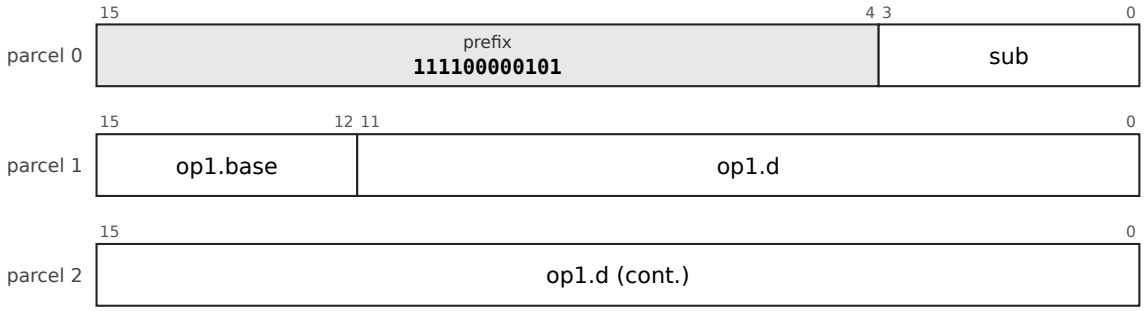


Mnemonics: `PUSH`

Syntax

```
PUSH rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-2`, `16-PUSHI3-1`, `16-PUSHI2-1`, `16-SOBu12-a-2`, `16-SOX6-a-2`, `16-SOBs12-a-2`.

Operation

```
v ← rm ; SP ← SP - 8 ; M[SP] ← v
```

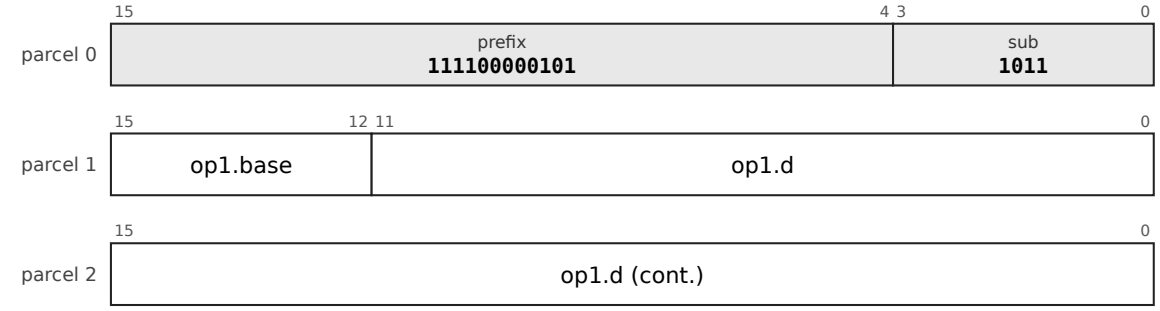
Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

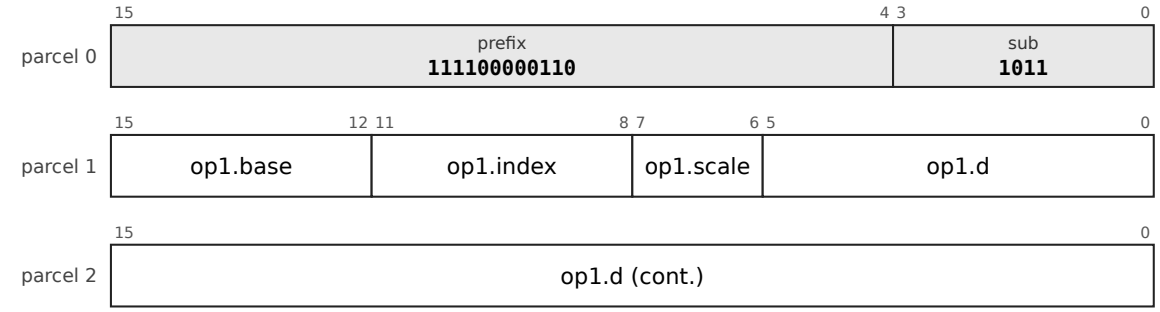
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

POP: Pop

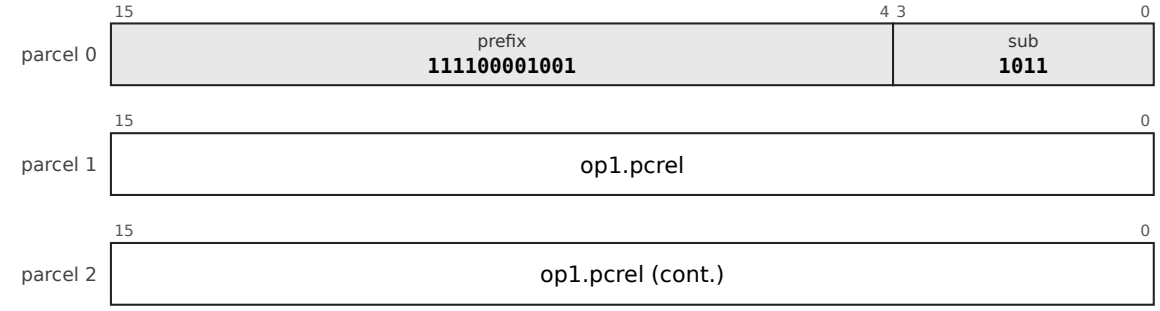
POP: SOF.B28, prefix 111100000101, sub-opcode 1011; operands b4d28s-x1; 6 bytes



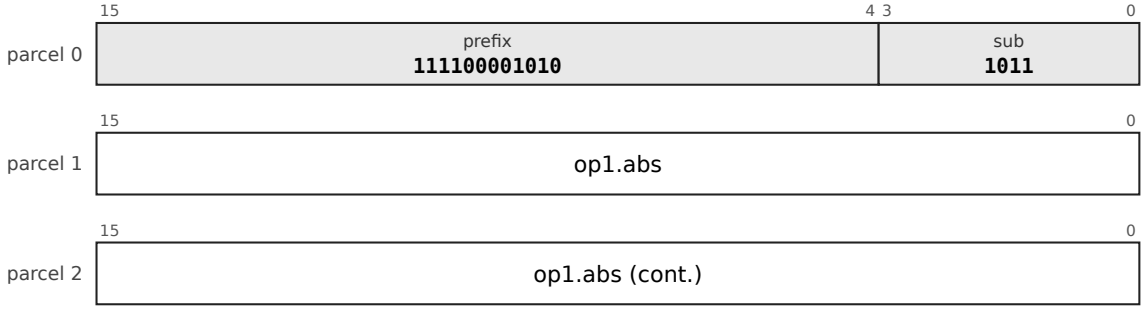
POP: SOF.X22, prefix 111100000110, sub-opcode 1011; operands x4-4d22s; 6 bytes



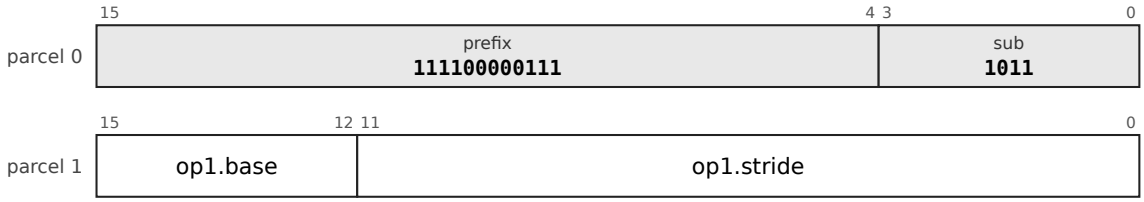
POP: SOF.PC32, prefix 111100001001, sub-opcode 1011; operands pc32-u1; 6 bytes



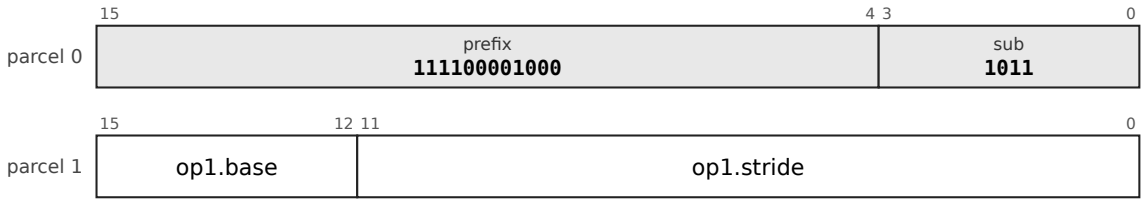
POP: SOF.A32, prefix 111100001010, sub-opcode 1011; operands abs32; 6 bytes



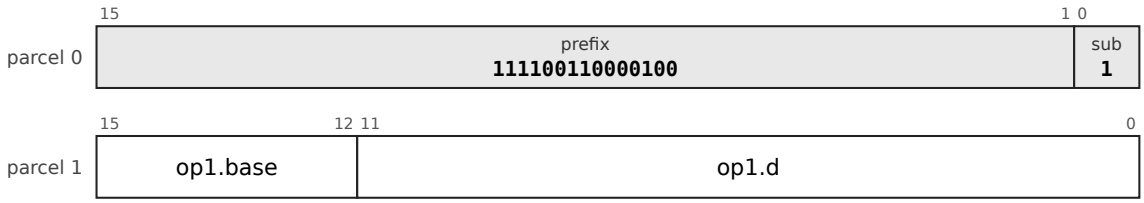
POP: SOF.PI12, prefix 111100000111, sub-opcode 1011; operands pi4-s12; 4 bytes



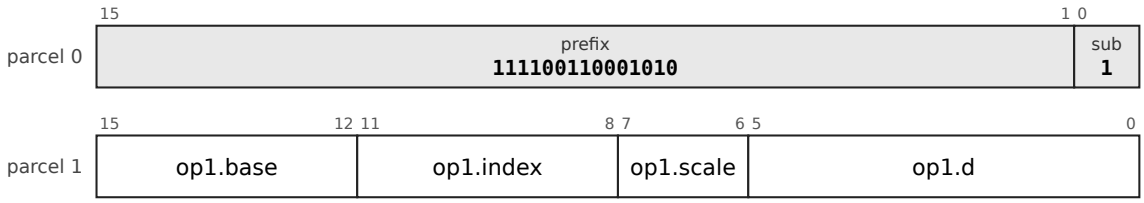
POP: SOF.PD12, prefix 111100001000, sub-opcode 1011; operands pd4-s12; 4 bytes



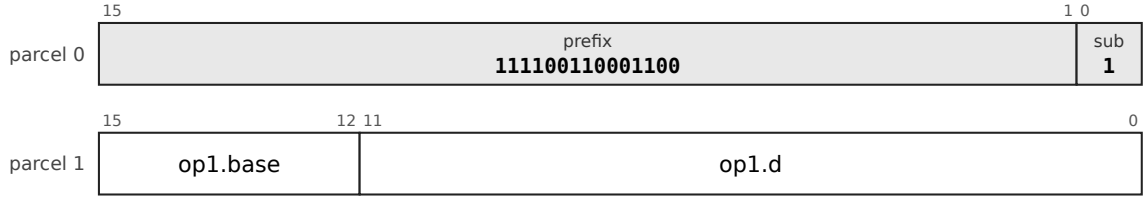
POP: SOBu12/a, prefix 111100110000100, sub-opcode 1; operands b4d12u-xsz; 4 bytes



POP: SOX6/a, prefix 111100110001010, sub-opcode 1; operands x4-4d6s; 4 bytes



POP: SOBs12/a, prefix 111100110001100, sub-opcode 1; operands b4d12s-xsz; 4 bytes



POP: SOPR, prefix 1110100011, sub-opcode 11; operands reg4; 2 bytes

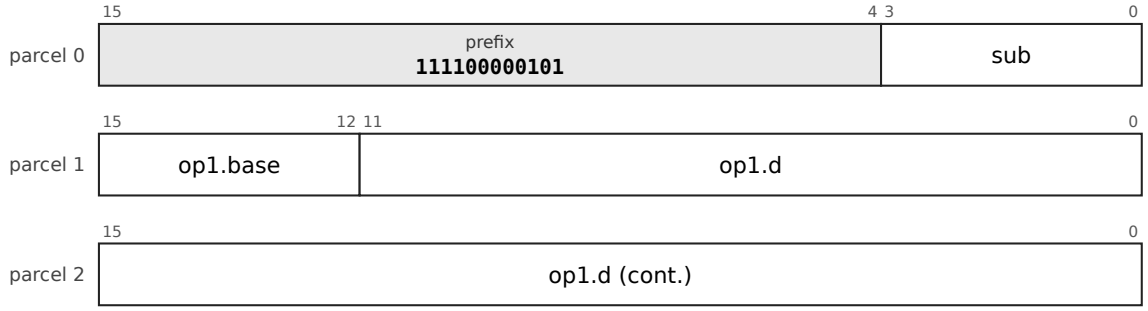


Mnemonics: `POP`

Syntax

```
POP rm
```

Encoding



The chart is the leaf `16-SOF-B28-1`, SOF.B28, prefix `111100000101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SOF-B28-1`, `16-SOF-X22-1`, `16-SOF-PI12-1`, `16-SOF-PD12-1`, `16-SOF-PC32-1`, `16-SOF-A32-1`, `16-SOPR-2`, `16-SOBu12-a-2`, `16-SOX6-a-2`, `16-SOBs12-a-2`.

Operation

```
v ← M[SP] ; SP ← SP + 8 ; rm ← v
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

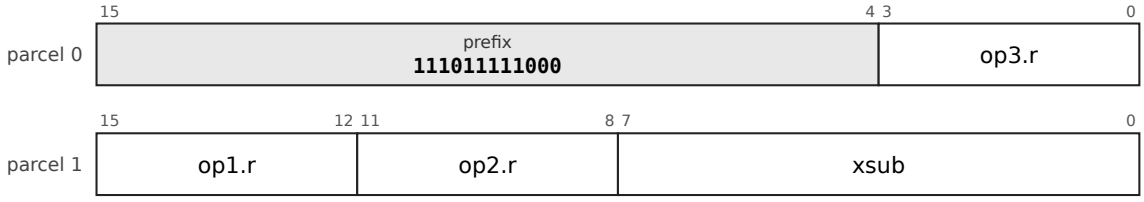
MCPY: Copy a block of bytes

Mnemonics: `MCPY`, `MMOV`

Syntax

```
MCPY Rd, Rs, Rn        ; destination pointer, source pointer, count
MMOV Rd, Rs, Rn        ; correct for overlap
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
MCPY:                           ; ascending; overlap is undefined
  while Rn ≠ 0:  M8[Rd] ← M8[Rs] ; Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
MMOV: as MCPY, but the direction is chosen from the pointers so that an overlapping copy is correct
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. The three registers are updated as work completes, so a fault leaves a consistent state and the instruction restarts from its registers.

The one granted exception to the one-memory-operand rule. All three registers are architectural state: on completion `Rn` is 0 and `Rd` and `Rs` point one past the end. An implementation may move more than a byte per step, so long as the register state at every point where an exception can be taken is consistent with a byte-at-a-time copy. The accesses made inside one block operation are unordered with respect to each other.

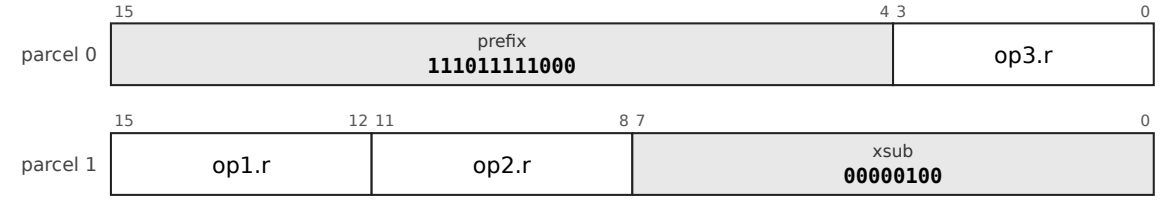
Why it is here. The block operations are the one granted exception to the one-memory-operand rule (an exception to a principle is allowed where there is a clear benefit), made safe by keeping the progress in architectural registers: the VAX `MOVC5` kept it in hidden microstate, which is why it is rejected, and ARM reached the same design in 2022 with `FEAT_MOPS`. The names `MCPY`, `MMOV` and `MSET` follow the memory operations of IBM System z; `MCMP` and `MSCN` complete the set. `MCPY` replaces the PDP-11 `MOV (R1)+,(R2)+` loop, and the `rep movsb` of x86, with one restartable instruction.

Cf. 6.8 (restart, progress faults); 7.2 (ordering of the accesses).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MCPY

MCPY: R3, prefix 111011111000, sub-opcode 00000100; operands reg4, reg4, reg4; 4 bytes

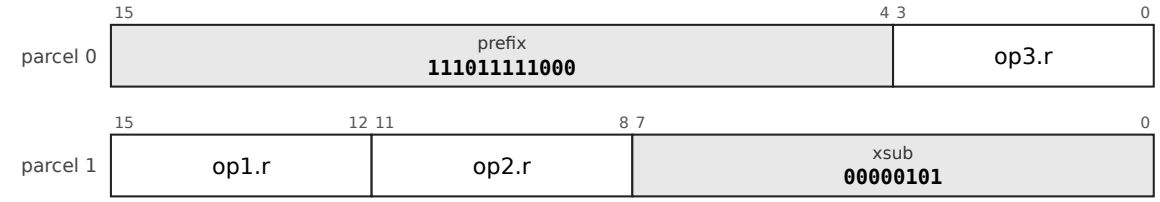



```
MCPY Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MCPY above; 1 layout form, each a chart of its own (above).

MMOV

MMOV: R3, prefix 111011111000, sub-opcode 00000101; operands reg4, reg4, reg4; 4 bytes

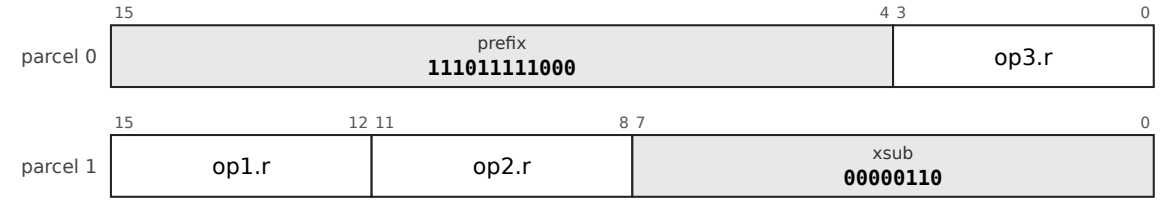



```
MMOV Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MCPY above; 1 layout form, each a chart of its own (above).

MSET: Fill a block of bytes

MSET: R3, prefix 111011111000, sub-opcode 00000110; operands reg4, reg4, reg4; 4 bytes

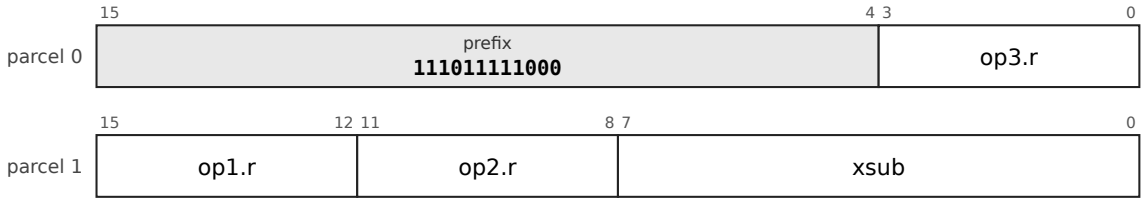


Mnemonics: `MSET`

Syntax

```
MSET Rd, Rv, Rn
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
while Rn ≠ 0:  M8[Rd] ← Rv[7:0] ; Rd ← Rd + 1 ; Rn ← Rn - 1
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

`MSET` with `Rv = 0` is the range-zeroing instruction: `bzero` and the clear of `calloc` are one instruction.

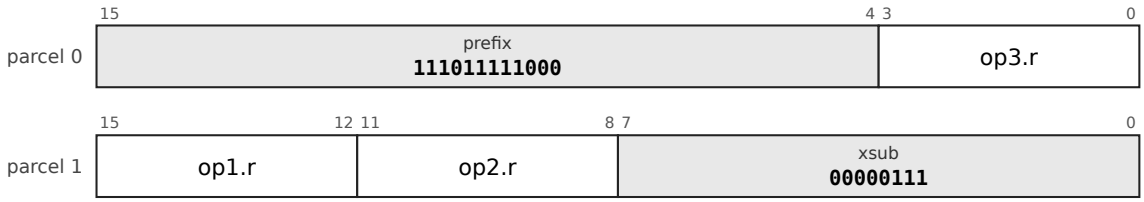
Why it is here. With a zero value it is the range-zeroing instruction, so `bzero` and the clearing of `calloc` need no separate opcode.

Cf. 6.8.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MCMP: Compare two blocks of bytes

MCMP: R3, prefix 111011111000, sub-opcode 00000111; operands reg4, reg4, reg4; 4 bytes

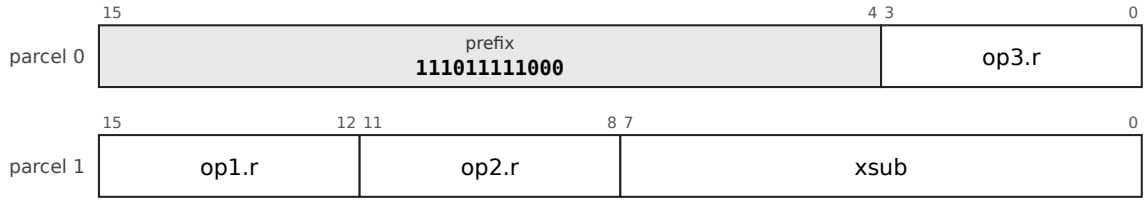


Mnemonics: `MCMP`

Syntax

```
MCMP Rd, Rs, Rn
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
while Rn ≠ 0:
    x ← M8[Rd] ; y ← M8[Rs]
    if x ≠ y then  flags ← flags of (x - y) ; stop      ; pointers left AT the differing byte
    Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
if equal:  N = 0, Z = 1, V = 0, C = 0
```

Flags From the first differing byte as for `CMP` of the two bytes (`C` set if `x < y` unsigned, `N` and `V` from the signed difference); for equal blocks `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

When the instruction stops at a difference, `Rd` and `Rs` point at the differing bytes and `Rn` is the count remaining, so the position of the difference is in the registers.

Why it is here. Seven of the VAX's ten character-string instructions are present (`MOVC3`, `CMPC3`, `LOCC` and `SKPC` among them); the rejected ones are `EDITPC` and packed decimal. `MCMP` is the `CMPC3` equivalent.

Cf. 6.8.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

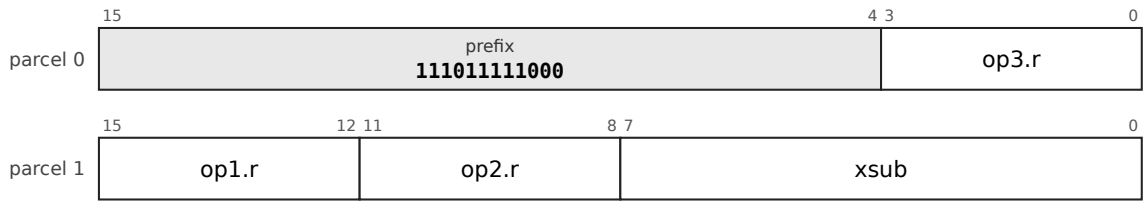
MSCN: Scan a block for a byte, or for a byte not equal to it

Mnemonics: `MSCN`, `MSCNN`

Syntax

```
MSCN Rd, Rv, Rn
MSCNN Rd, Rv, Rn
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
while Rn ≠ 0:
    hit ← (M8[Rd] = Rv[7:0])            ; MSCNN: hit ← (M8[Rd] ≠ Rv[7:0])
    if hit then  Z = 1 ; stop             ; Rd points AT the byte found
    Rd ← Rd + 1 ; Rn ← Rn - 1
not found: Z = 0
```

Flags All four flags are written: `Z` = 1 if the byte was found, else 0; `N`, `V` and `C` are cleared.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

`MSCN` is `memchr`; with an explicit count it gives `strnlen`, and composing it with `MCPY` gives a bounded `strcpy` in two instructions. There are no NUL-terminated string instructions.

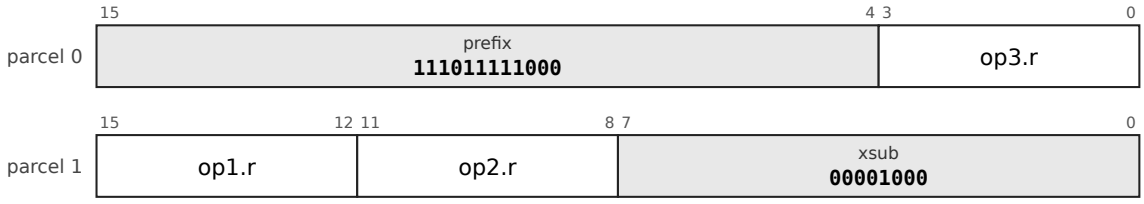
Why it is here. `MSCN` is `LOCC` and `memchr`; `MSCNN` fills the `SKPC` gap (the first byte that is *not* equal). Both cost almost no hardware. With an explicit count `MSCN` gives `strnlen`, and with `MCPY` a bounded `strcpy` in two instructions. NUL-terminated string instructions (the System/z `MVST`, `SRST`, `CLST`) are rejected, because they encode an unbounded read, the largest single source of memory-safety failures in C (9.17).

Cf. 6.8.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MSCN

MSCN: R3, prefix 111011111000, sub-opcode 00001000; operands reg4, reg4, reg4; 4 bytes

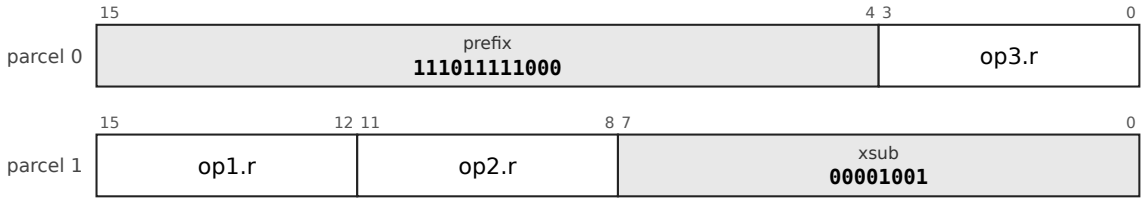



```
MSCN Rd, Rv, Rn
```

Operation, flags and exceptions as for the entry MSCN above; 1 layout form, each a chart of its own (above).

MSCNN

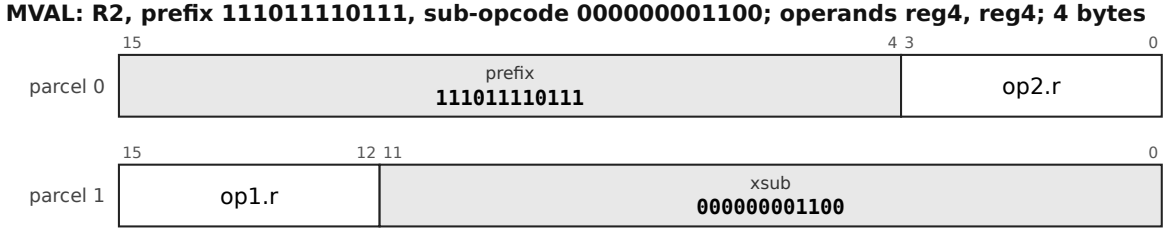
MSCNN: R3, prefix 111011111000, sub-opcode 00001001; operands reg4, reg4, reg4; 4 bytes




```
MSCNN Rd, Rv, Rn
```

Operation, flags and exceptions as for the entry MSCN above; 1 layout form, each a chart of its own (above).

MVAL: Validate a block as UTF-8

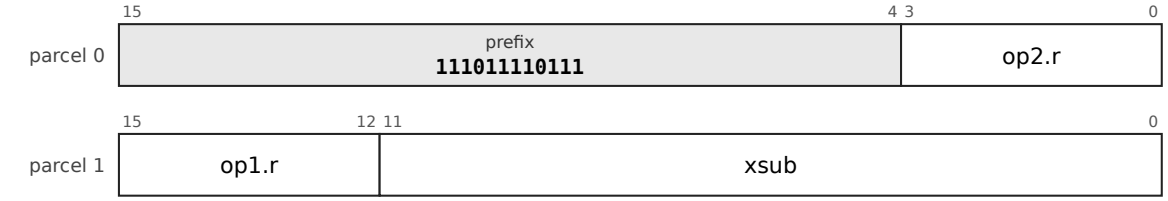


Mnemonics: `MVAL`

Syntax

```
MVAL Rd, Rn
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
scan Rn bytes at Rd; stop at the first byte that cannot be part of a valid UTF-8 sequence (the rules of
UTF8DEC)
```

Flags On completion `Rn` is 0 and the buffer is valid; otherwise `Rd` points at the offending sequence and `Rn` counts the bytes remaining (details in the instruction-set text, section 4.7.1).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The `UTF8DEC` comparators wired to the block sequencer.

Why it is here. The UTF-8 validity comparators already built for `UTF8DEC`, wired to the streaming sequencer already built for `MCPY`, so it costs almost no hardware. The alternative, a SWAR loop (SIMD within a register), runs at about two to three cycles a byte and is respectable, which is why the justification is cost and not throughput.

Cf. 6.8.

Status specified; **not yet implemented** in the reference executor.

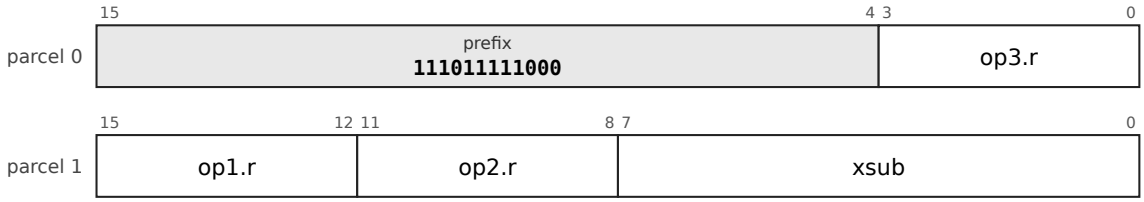
MSHA: Hash whole blocks of memory

Mnemonics: `MSHA256`, `MSHA512`

Syntax

```
MSHA256 Rd, Rs, Rn
MSHA512 Rd, Rs, Rn
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
hash the Rn bytes (whole blocks) at (Rs) into the state at (Rd)
progress is held in Rd, Rs and Rn; an interrupt or a fault restarts the instruction where it stopped
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

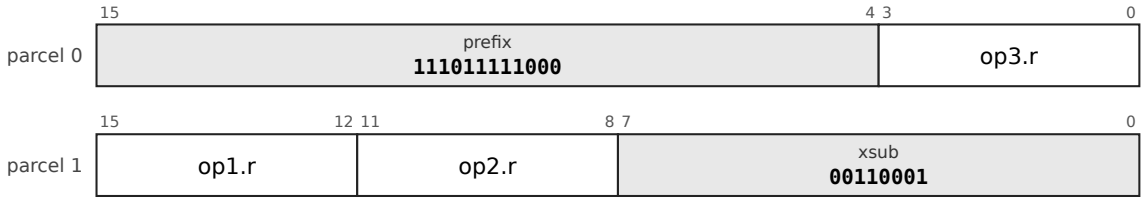
(P) Stage 2 of the crypto proposal (`docs/crypto-secure-core.md` section 3.4): the whole-buffer form, in the manner of the s390 CPACF instructions, to be built "only after measurement"; the grammars reserve its encoding. The unit would need about 2,000 look-up tables and 1,000 flip-flops (the proposal's estimate). **Not implemented** in the reference executor.

Why it is here. Stage 2 of the crypto proposal, in the manner of the s390 CPACF instructions (`KIMD` and kin): one instruction hashes a buffer, the padding and chaining are done in hardware and so cannot be got wrong, and progress is held in registers, so a fault or an interrupt restarts it, as for `MCPY`. The estimate is 64 cycles for a 64-byte SHA-256 block against about 1,200 instructions, roughly 18 times; the cost is gates in proportion to the algorithms frozen into opcodes. The proposal says to build `MSHA256` first and the others only after measurement, and the author has not decided it.

Status specified; **not yet implemented** in the reference executor.

MSHA256

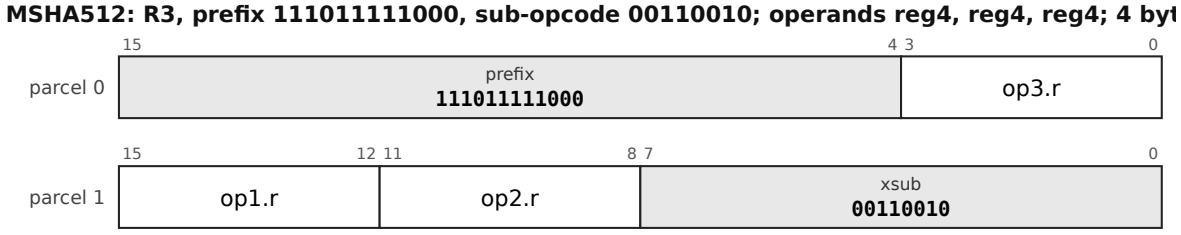
MSHA256: R3, prefix 111011111000, sub-opcode 00110001; operands reg4, reg4, reg4; 4 byt




```
MSHA256 Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MSHA above; 1 layout form, each a chart of its own (above).

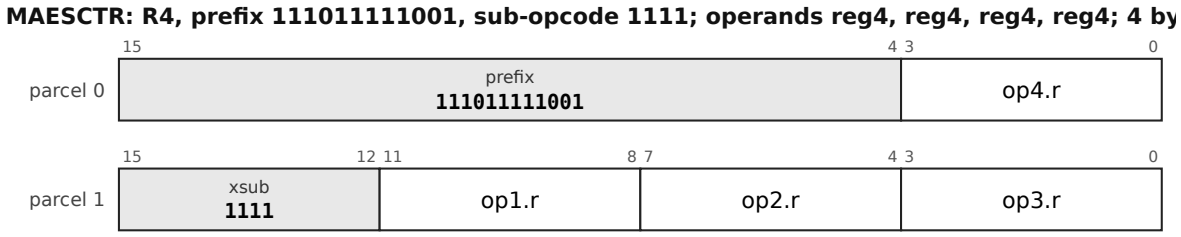
MSHA512




```
MSHA512 Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MSHA above; 1 layout form, each a chart of its own (above).

MAESCTR: AES in counter mode over a buffer

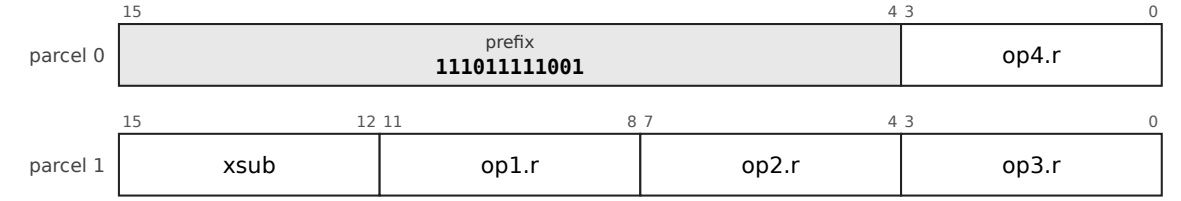


Mnemonics: `MAESCTR`

Syntax

```
MAESCTR Rd, Rs, Rn, Rp
```

Encoding



The chart is the leaf `16-R4-1`, R4, prefix `111011111001`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R4-1`.

Operation

```
AES counter mode over the Rn bytes at (Rs), written at (Rd); the key, the counter and the number of rounds
are in the parameter block at (Rp)
progress is held in the registers and the parameter block; an interrupt or a fault restarts the instruction
where it stopped
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

(P) Stage 2, as `MSHA`; the layout of the parameter block is not specified. **Not implemented** in the reference executor.

Why it is here. Stage 2, as `MSHA`; whole-buffer AES gains less (about 4 times: 0.6 against 2.6 instructions per byte), because the round instructions are already good, and the proposal puts it after `MSHA256` and `MSHA512`.

Status specified; **not yet implemented** in the reference executor.

9.12 Floating point

9.12.1 Floating point uses the general registers: a `binary64` value fills one register, and the instruction, not the register, decides how its bits are read. *(D)*

9.12.2 `FADD`, `FSUB`, `FMUL` and `FCMP` are two-byte instructions; `FDIV`, `FSQRT`, the fused multiply-add family, `FABS`, `FMIN`, `FMAX`, `FCLASS`, the roundings and the `binary32` forms are in the floating-point escape; `binary16` has seventeen operations and conversions. *(D)*

9.12.3 The five IEEE flags (invalid, divide by zero, overflow, underflow, inexact) accumulate in `FPSR`; no floating-point operation traps; `FSFLAGS` reads and clears them. *(D)*

9.12.4 Every NaN that an operation creates is the canonical quiet NaN. *(D)*

9.12.5 There are no rounding-mode or precision mode bits: the precision is in the opcode, and so is the rounding mode wherever it is not nearest-even. *(D)*

9.12.6 There are no transcendental instructions. *(D)*

Commentary. **Precedent.** The PDP-11's FP11 had `AC0` to `AC5` and mode bits for single and double precision; the VAX had F, D, G and H formats. **Trade-offs.** An integer operation applied to a register holding a `binary64` destroys it, with no trap and no tag; that is intended and the ABI states it. A separate register file would cost a second set of read ports and a floating-point context; on a small implementation the register file is among the largest blocks. **Improvements.** The fused multiply-add is the largest single addition to the datapath (about 9,000 transistors) and the least negotiable; the IEEE exception flags are recorded without traps, which is what C99 `<fenv.h>` needs. **Benefits.** No transfer instructions between files and a single calling convention. **Modern comparables.** RISC-V Zfinx and Zdinx use the integer register file for the same reason; AArch64 and x86 have separate vector and floating-point files. **Use.** `FCLASS` begins every library maths function; `FMA` is the building block of accurate polynomial evaluation. **Impact.** *Kernel:* There is no separate floating-point state to save at a context switch beyond the registers already saved, and the sticky flags go with the status word. *Compiler:* One calling convention for integers and floats, fused multiply-add contraction, and no rounding-mode state for the compiler to track. *Hardware:* No second register file (one of the largest blocks of a small floating-point unit); the floating-point unit with fused multiply-add is about 34,000 transistors.

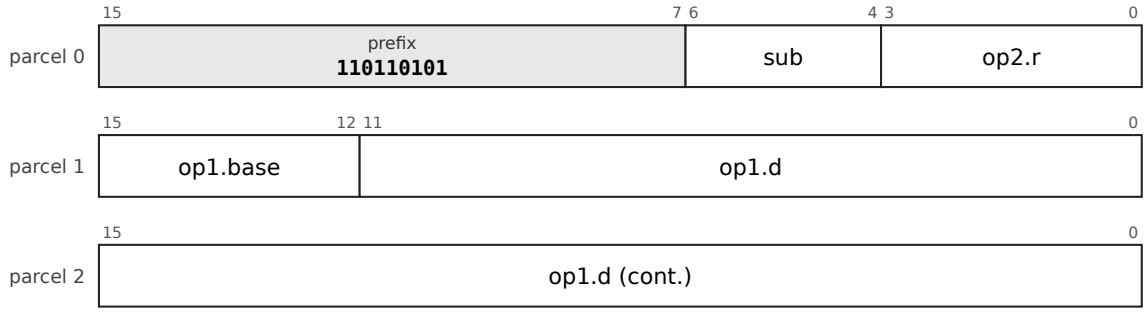
FADD: Floating add, subtract and multiply, binary64

Mnemonics: `FADD`, `FSUB`, `FMUL` (aliases: `FADDD`, `FSUBD`, `FMULD`)

Syntax

```
FADD rm, Rd
FSUB rm, Rd
FMUL rm, Rd
```

Encoding



The chart is the leaf `16-LDF-B28-2`, LDF.B28, prefix `110110101`. `op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-LDF-B28-2`, `16-LDF-X22-2`, `16-LDF-PI12-2`, `16-LDF-PD12-2`, `16-LDF-PC32-2`, `16-LDF-A32-2`, `16-LDFI-I32-1`, `16-DPF-D28-2`, `16-RRF-2`, `16-LDPC12-d-1`, `16-LDBs8-d-1`, `16-LDPC12-d-2`, `16-LDBs8-d-2`.

Operation

```
Rd ← Rd + rm   (FADD)
Rd ← Rd - rm   (FSUB)
Rd ← Rd × rm   (FMUL)       ; IEEE 754 binary64, round to nearest even
```

Flags None written. The five IEEE exception flags are ORed into `FPSR`; no instruction traps.

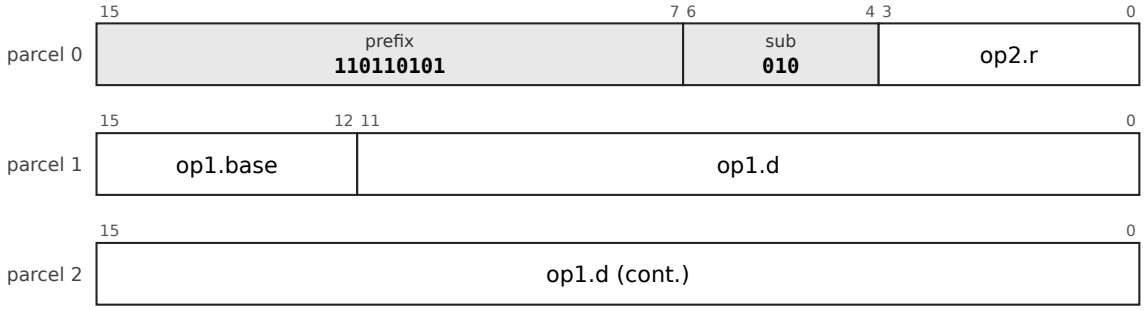
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Floating-point exceptions never trap.

Floating-point values live in the general registers; there is no separate register file and no mode bit. A NaN result is the canonical quiet NaN `0x7FF8000000000000`. The alternative spellings `FADDD`, `FSUBD`, `FMULD` are aliases.

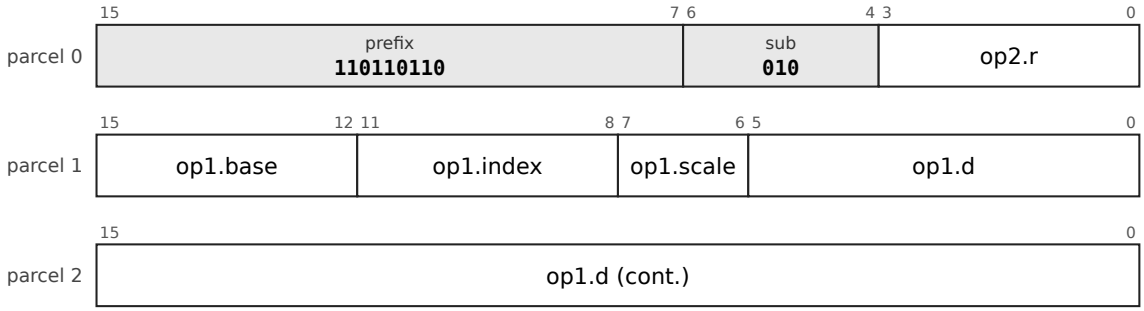
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FADD

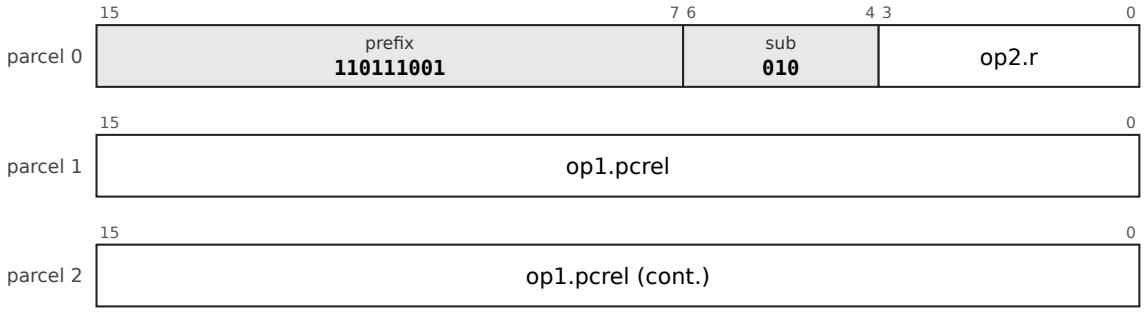
FADD: LDF.B28, prefix 110110101, sub-opcode 010; operands b4d28s-x1, reg4; 6 bytes



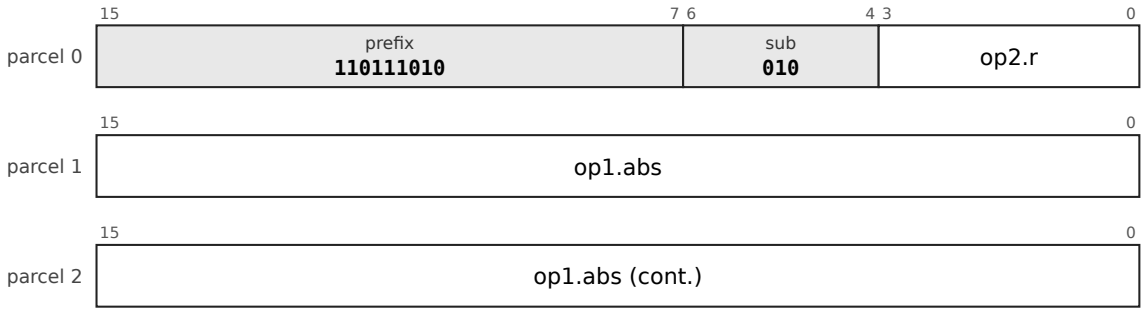
FADD: LDF.X22, prefix 110110110, sub-opcode 010; operands x4-4d22s, reg4; 6 bytes



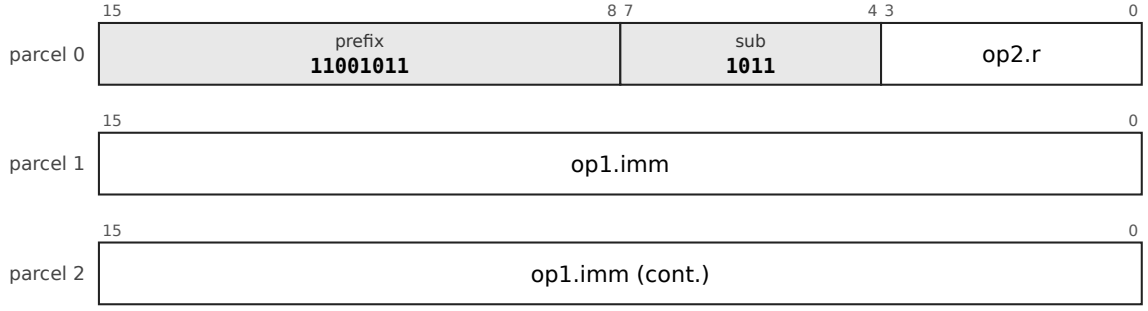
FADD: LDF.PC32, prefix 110111001, sub-opcode 010; operands pc32-u1, reg4; 6 bytes



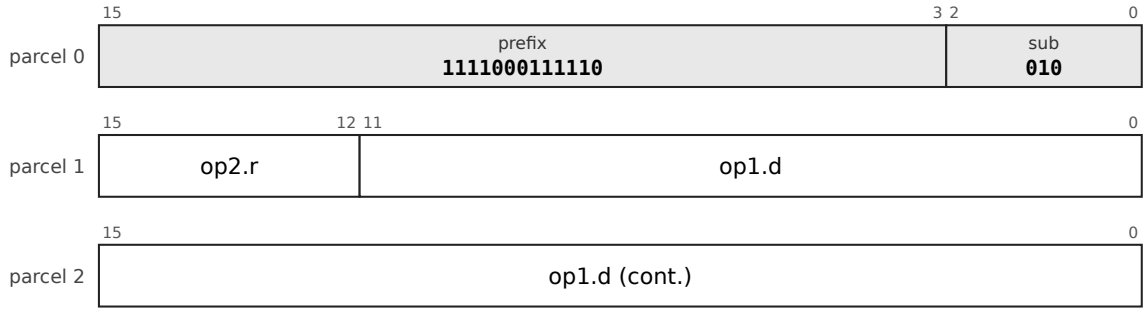
FADD: LDF.A32, prefix 110111010, sub-opcode 010; operands abs32, reg4; 6 bytes



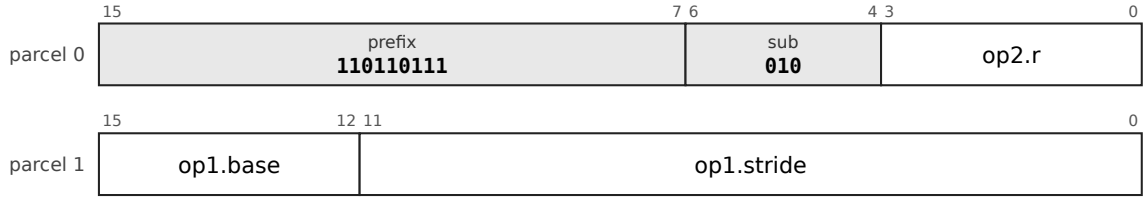
FADD: LDFI.I32, prefix 11001011, sub-opcode 1011; operands imm32s, reg4; 6 bytes



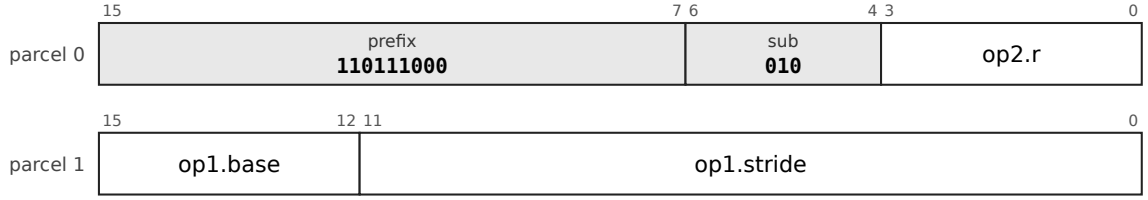
FADD: DPF.D28, prefix 1111000111110, sub-opcode 010; operands dpd28s-x1, reg4; 6 bytes



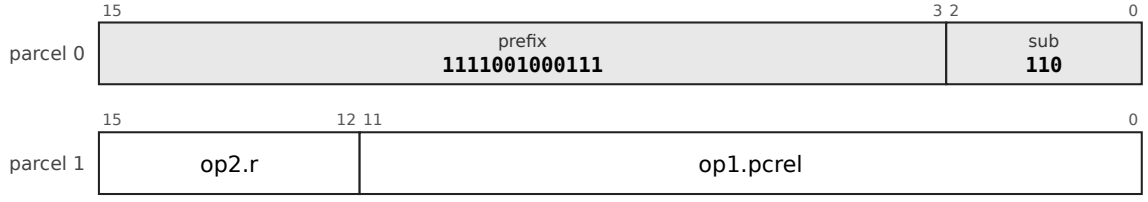
FADD: LDF.PI12, prefix 110110111, sub-opcode 010; operands pi4-s12, reg4; 4 bytes



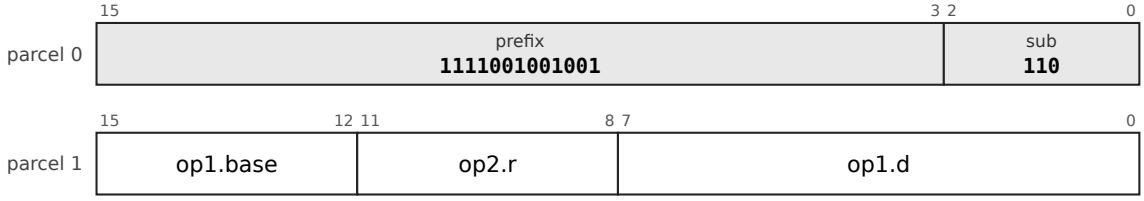
FADD: LDF.PD12, prefix 110111000, sub-opcode 010; operands pd4-s12, reg4; 4 bytes



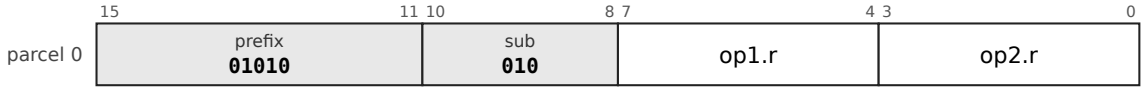
FADD: LDPC12/d, prefix 1111001000111, sub-opcode 110; operands pc12-u1, reg4; 4 bytes



FADD: LDBs8/d, prefix 1111001001001, sub-opcode 110; operands b4d8s-xsz, reg4; 4 bytes



FADD: RRF, prefix 01010, sub-opcode 010; operands reg4, reg4; 2 bytes

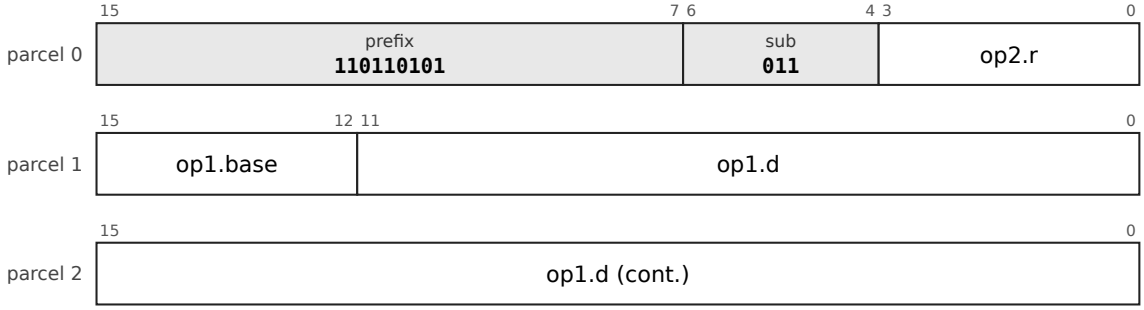



```
FADD rm, Rd
```

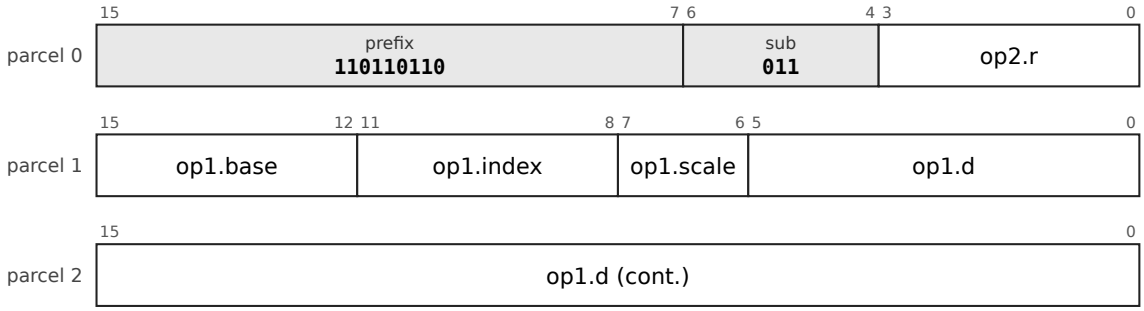
Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

FSUB

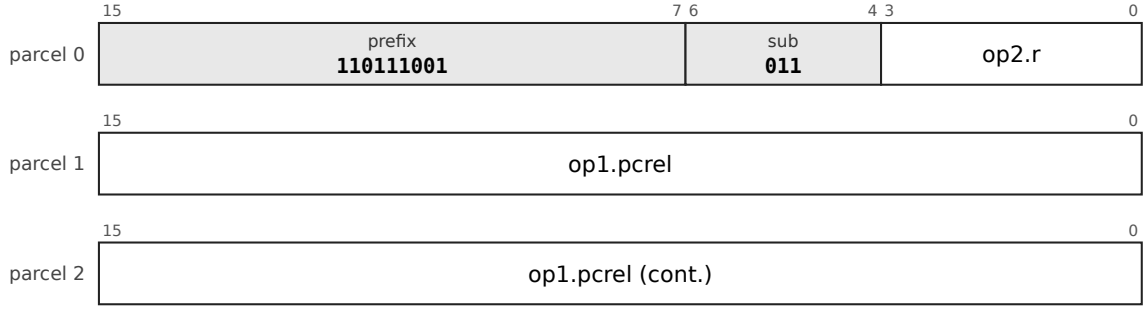
FSUB: LDF.B28, prefix 110110101, sub-opcode 011; operands b4d28s-x1, reg4; 6 bytes



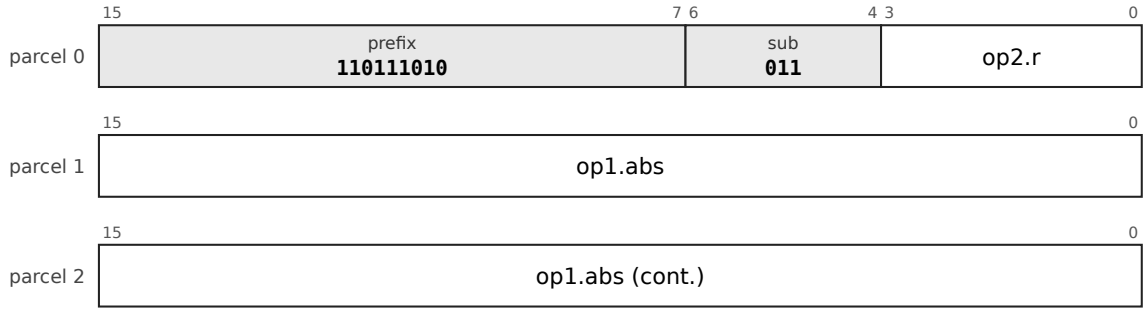
FSUB: LDF.X22, prefix 110110110, sub-opcode 011; operands x4-4d22s, reg4; 6 bytes



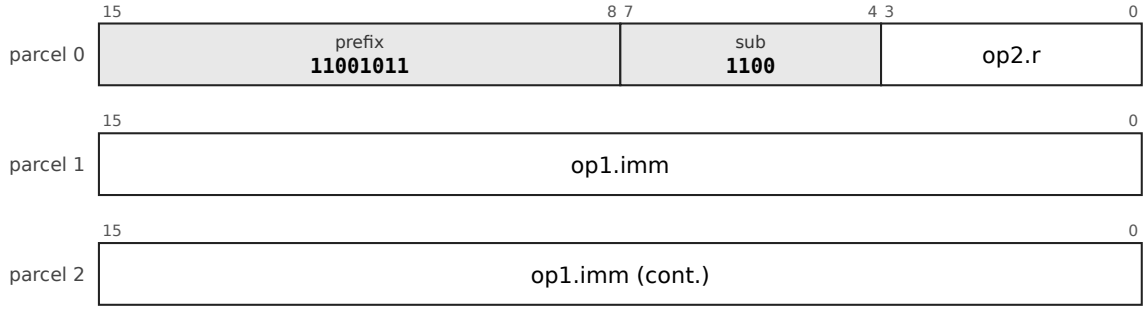
FSUB: LDF.PC32, prefix 110111001, sub-opcode 011; operands pc32-u1, reg4; 6 bytes



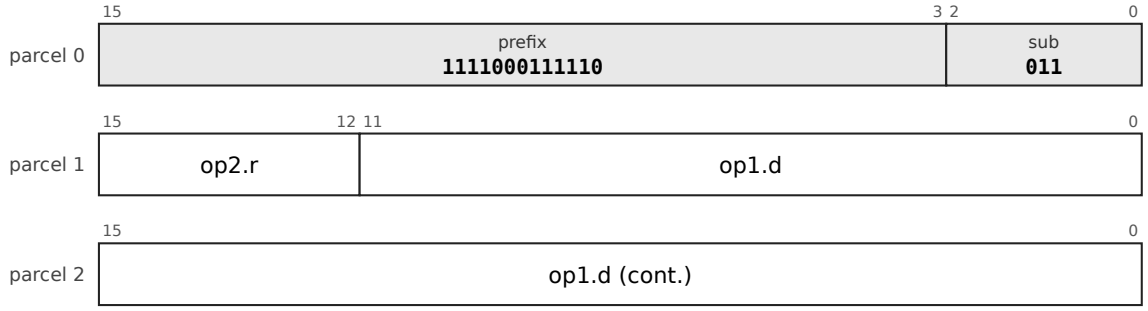
FSUB: LDF.A32, prefix 110111010, sub-opcode 011; operands abs32, reg4; 6 bytes



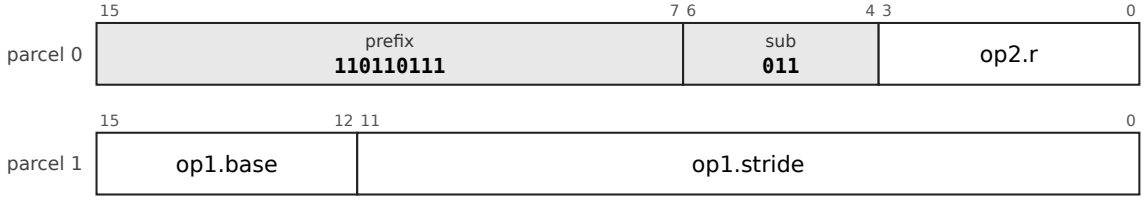
FSUB: LDFI.I32, prefix 11001011, sub-opcode 1100; operands imm32s, reg4; 6 bytes



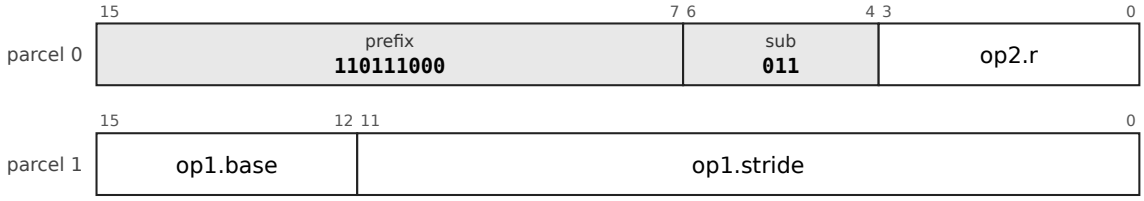
FSUB: DPF.D28, prefix 1111000111110, sub-opcode 011; operands dpd28s-x1, reg4; 6 bytes



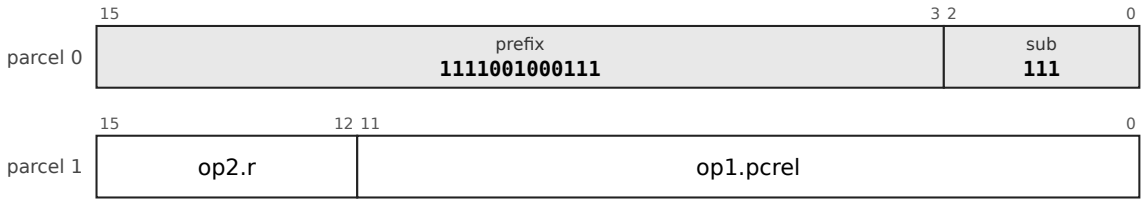
FSUB: LDF.PI12, prefix 110110111, sub-opcode 011; operands pi4-s12, reg4; 4 bytes



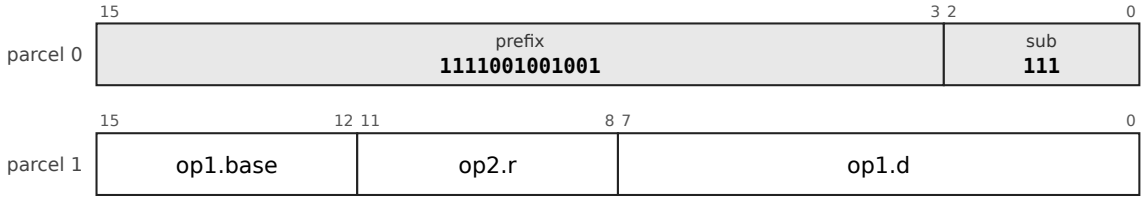
FSUB: LDF.PD12, prefix 110111000, sub-opcode 011; operands pd4-s12, reg4; 4 bytes



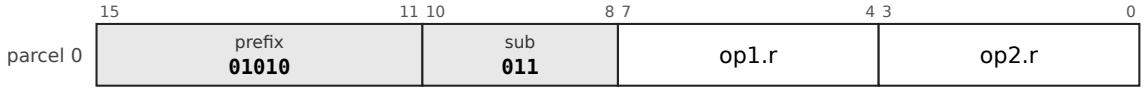
FSUB: LDPC12/d, prefix 1111001000111, sub-opcode 111; operands pc12-u1, reg4; 4 bytes



FSUB: LDBs8/d, prefix 1111001001001, sub-opcode 111; operands b4d8s-xsz, reg4; 4 bytes



FSUB: RRF, prefix 01010, sub-opcode 011; operands reg4, reg4; 2 bytes

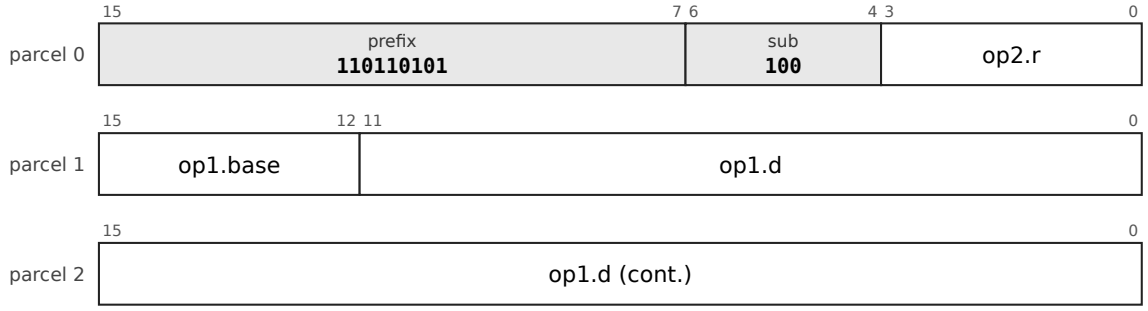



```
FSUB rm, Rd
```

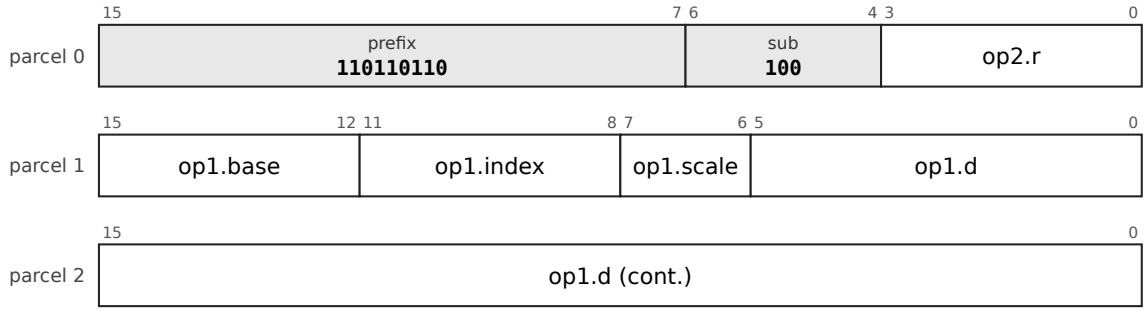
Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

FMUL

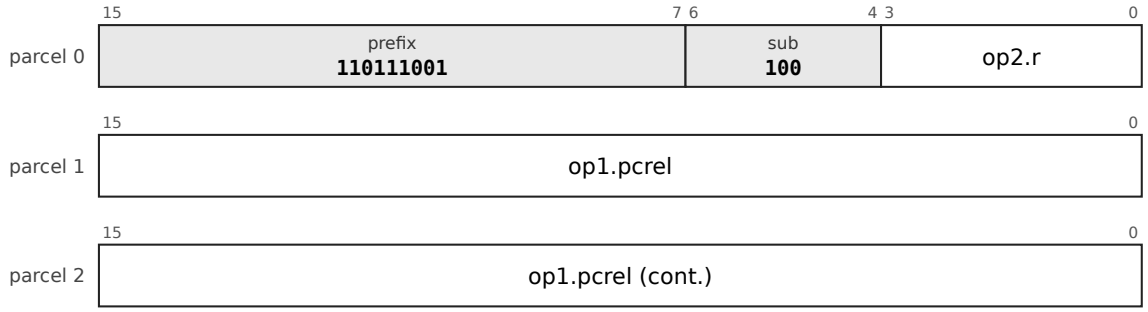
FMUL: LDF.B28, prefix 110110101, sub-opcode 100; operands b4d28s-x1, reg4; 6 bytes



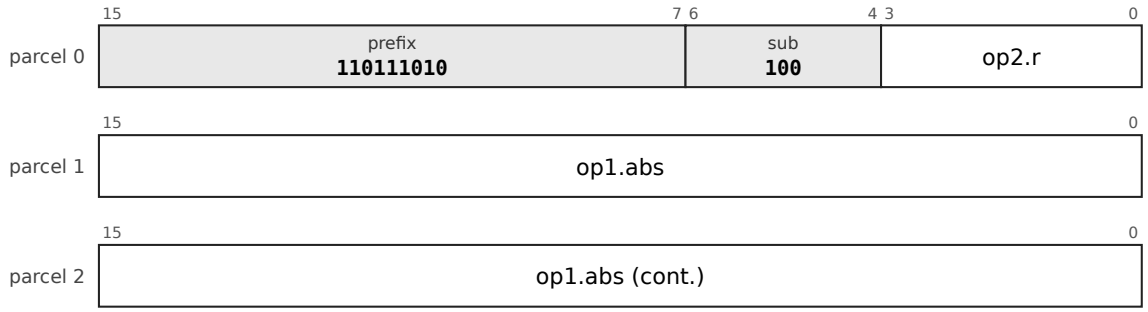
FMUL: LDF.X22, prefix 110110110, sub-opcode 100; operands x4-4d22s, reg4; 6 bytes



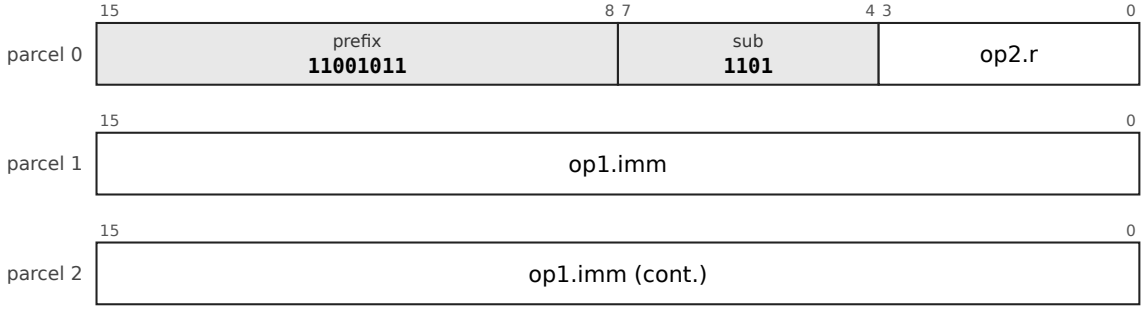
FMUL: LDF.PC32, prefix 110111001, sub-opcode 100; operands pc32-u1, reg4; 6 bytes



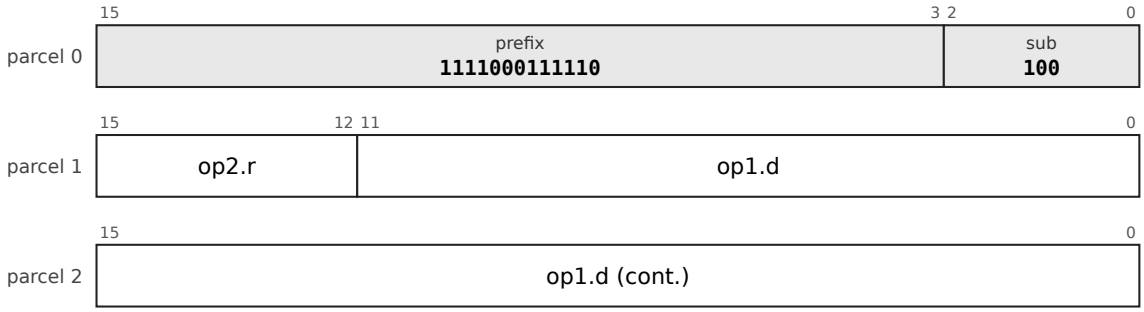
FMUL: LDF.A32, prefix 110111010, sub-opcode 100; operands abs32, reg4; 6 bytes



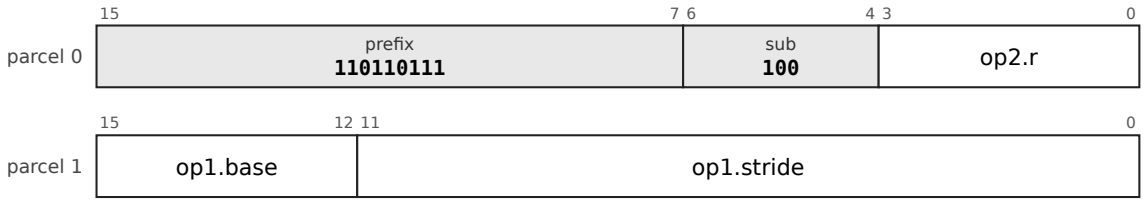
FMUL: LDFI.I32, prefix 11001011, sub-opcode 1101; operands imm32s, reg4; 6 bytes



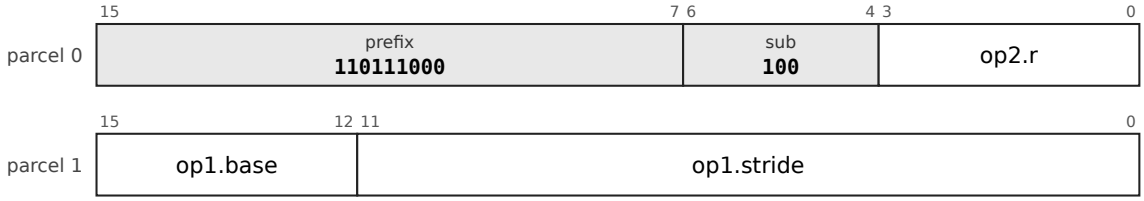
FMUL: DPF.D28, prefix 1111000111110, sub-opcode 100; operands dpd28s-x1, reg4; 6 bytes



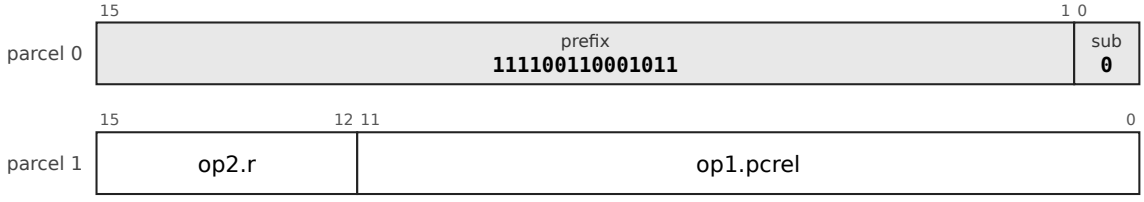
FMUL: LDF.PI12, prefix 110110111, sub-opcode 100; operands pi4-s12, reg4; 4 bytes



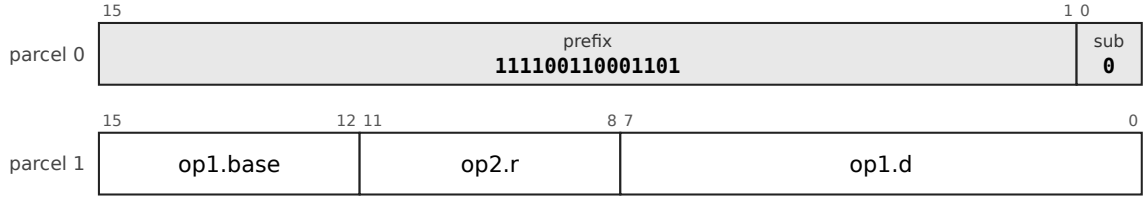
FMUL: LDF.PD12, prefix 110111000, sub-opcode 100; operands pd4-s12, reg4; 4 bytes



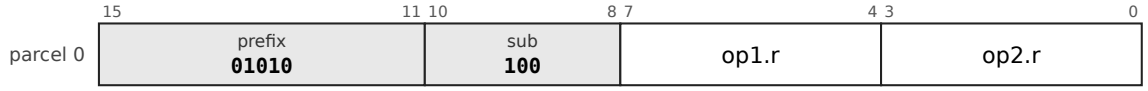
FMUL: LDPC12/d, prefix 111100110001011, sub-opcode 0; operands pc12-u1, reg4; 4 bytes



FMUL: LDBs8/d, prefix 111100110001101, sub-opcode 0; operands b4d8s-xsz, reg4; 4 bytes



FMUL: RRF, prefix 01010, sub-opcode 100; operands reg4, reg4; 2 bytes




```
FMUL rm, Rd
```

Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

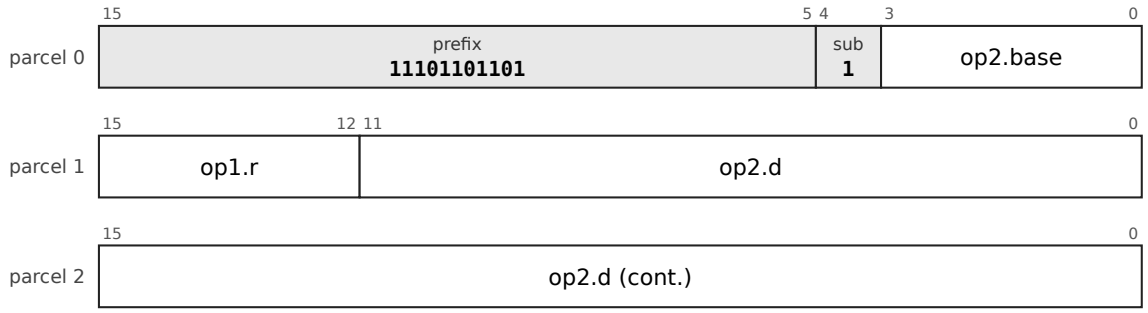
FADDD
An alias of FADD: the assembler writes it as that word.

FSUBD
An alias of FSUB: the assembler writes it as that word.

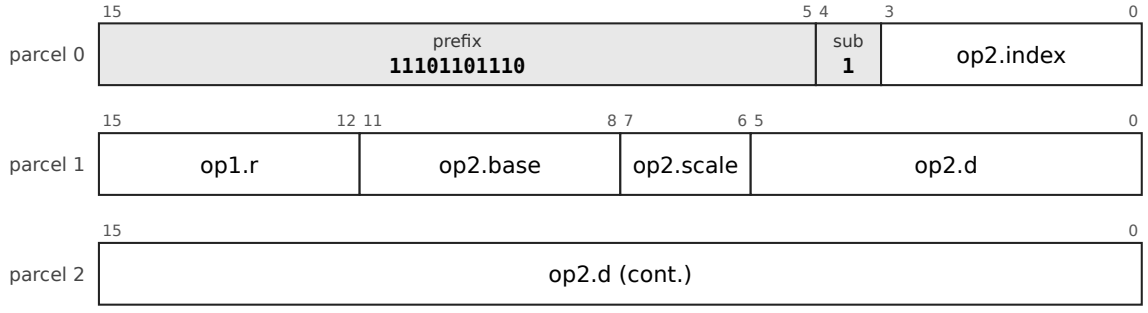
FMULD
An alias of FMUL: the assembler writes it as that word.

FCMP: Floating compare, binary64

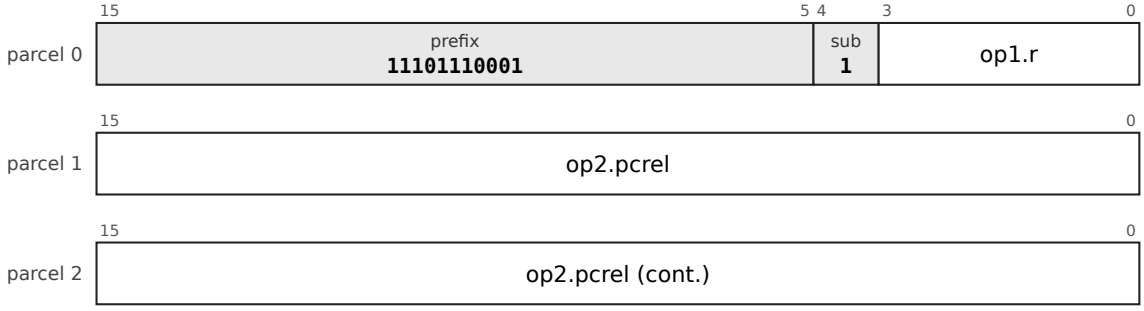
FCMP: CMF.B28, prefix 11101101101, sub-opcode 1; operands reg4, b4d28s-x1; 6 bytes



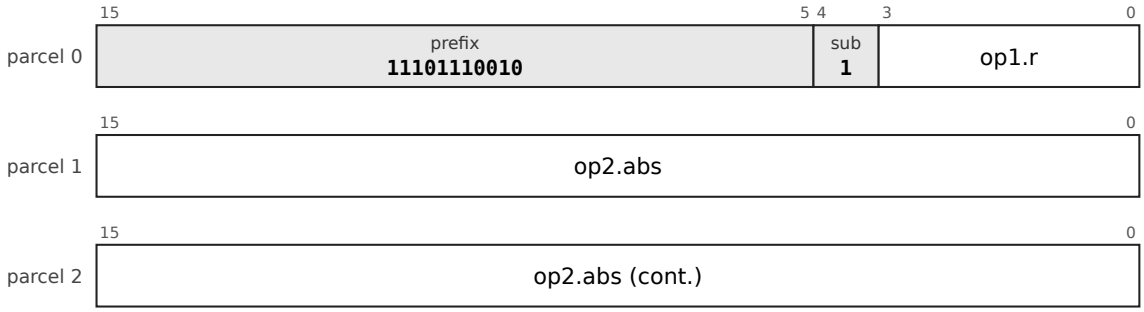
FCMP: CMF.X22, prefix 11101101110, sub-opcode 1; operands reg4, x4-4d22s; 6 bytes



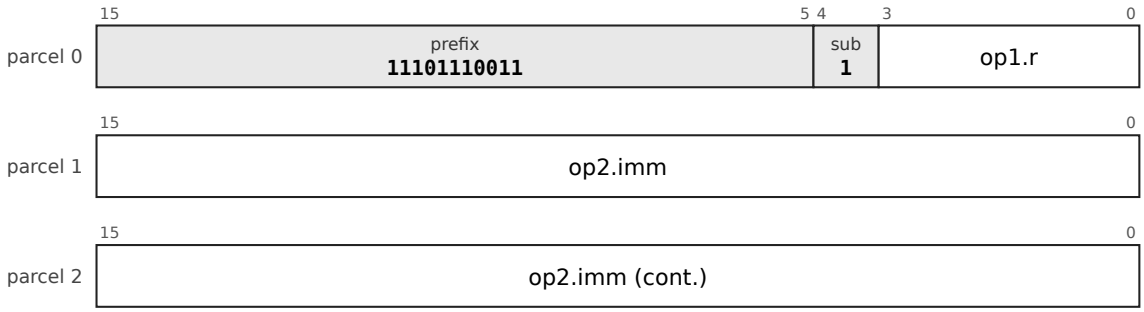
FCMP: CMF.PC32, prefix 11101110001, sub-opcode 1; operands reg4, pc32-u1; 6 bytes



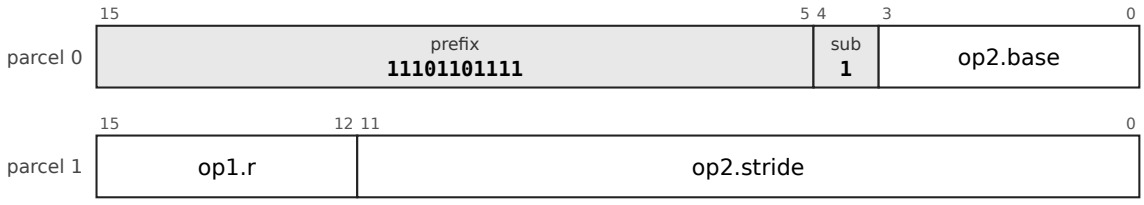
FCMP: CMF.A32, prefix 11101110010, sub-opcode 1; operands reg4, abs32; 6 bytes



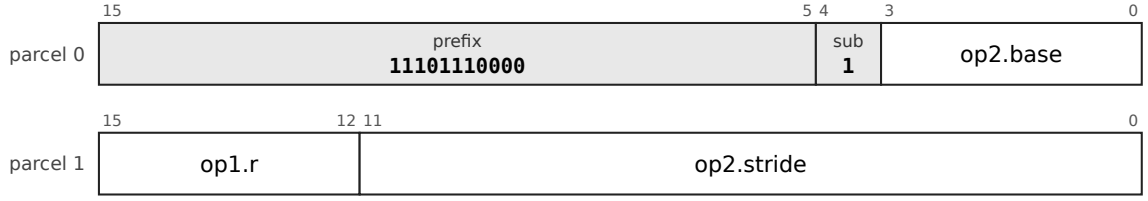
FCMP: CMF.I32, prefix 11101110011, sub-opcode 1; operands reg4, imm32s; 6 bytes



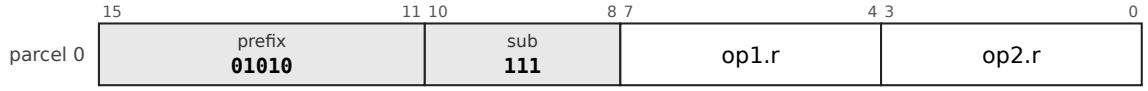
FCMP: CMF.PI12, prefix 11101101111, sub-opcode 1; operands reg4, pi4-s12; 4 bytes



FCMP: CMF.PD12, prefix 11101110000, sub-opcode 1; operands reg4, pd4-s12; 4 bytes



FCMP: RRF, prefix 01010, sub-opcode 111; operands reg4, reg4; 2 bytes

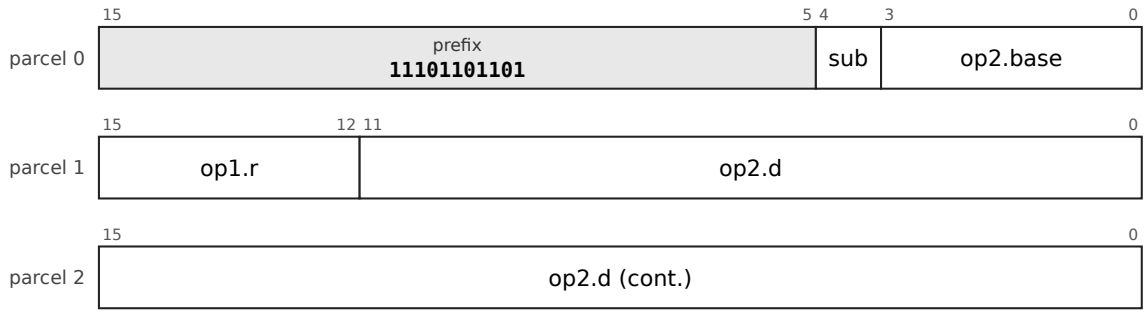


Mnemonics: `FCMP` (aliases: `FCMPD`)

Syntax

```
FCMP Rd, rm        ; register first
```

Encoding



The chart is the leaf `16-CMF-B28-1`, CMF.B28, prefix `11101101101`. `op1` is `reg4`: register, 4 bits. `op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-CMF-B28-1`, `16-CMF-X22-1`, `16-CMF-PI12-1`, `16-CMF-PD12-1`, `16-CMF-PC32-1`, `16-CMF-A32-1`, `16-CMF-I32-1`, `16-RRF-2`.

Operation

```
N ← Rd < rm ; Z ← Rd = rm ; V ← unordered ; C ← 0
```

Flags `N` set if `Rd < rm`, `Z` if equal, `V` if the operands are unordered (either is a NaN), `C` cleared. A signalling NaN raises `NV` in `FPSR`.
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
The signed branches do not treat the unordered case cleanly (`BLT` after an unordered compare takes `N xor V` = 1); this is an open item of the instruction set.
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).
`FCMPD` is an alias of `FCMP`: the assembler writes it as that word.

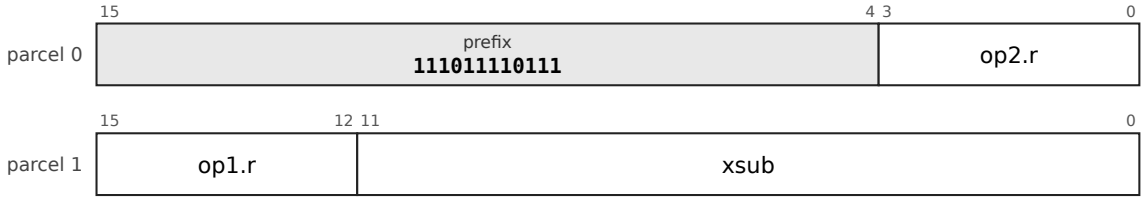
FDIV: Floating divide and square root, binary64

Mnemonics: `FDIV`, `FSQRT` (aliases: `FDIVD`, `FSQRTD`)

Syntax

```
FDIV Rs, Rd
FSQRT Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
FDIV : Rd ← Rd ÷ Rs
FSQRT: Rd ← sqrt(Rs)
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

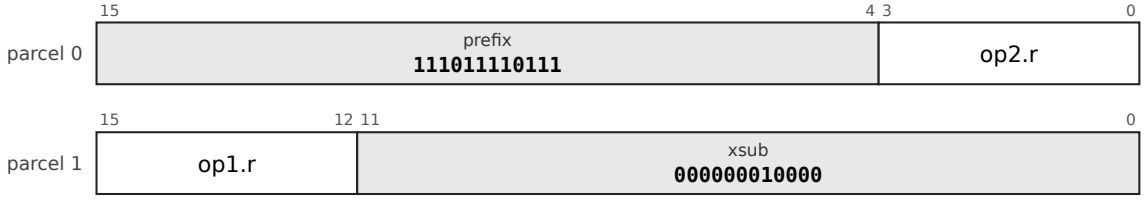
Exceptions None.

`FDIV` and `FSQRT` are multi-cycle sequencer operations and rarely appear in a one-parcel form. The alternative spellings `FDIVD`, `FSQRTD` are aliases.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FDIV

FDIV: R2, prefix 111011110111, sub-opcode 000000010000; operands reg4, reg4; 4 bytes

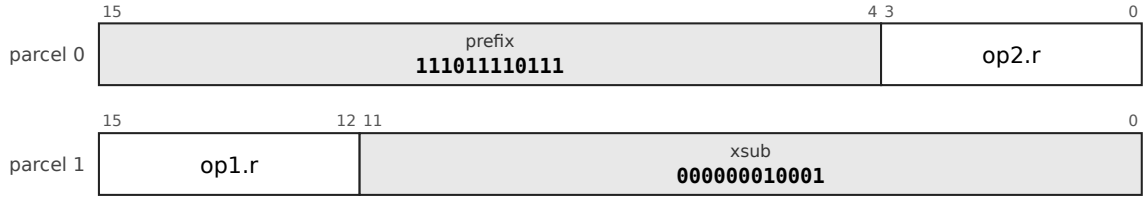



```
FDIV Rs, Rd
```

Operation, flags and exceptions as for the entry FDIV above; 1 layout form, each a chart of its own (above).

FSQRT

FSQRT: R2, prefix 111011110111, sub-opcode 000000010001; operands reg4, reg4; 4 bytes




```
FSQRT Rs, Rd
```

Operation, flags and exceptions as for the entry FDIV above; 1 layout form, each a chart of its own (above).

FDIVD

An alias of `FDIV`: the assembler writes it as that word.

FSQRTD

An alias of `FSQRT`: the assembler writes it as that word.

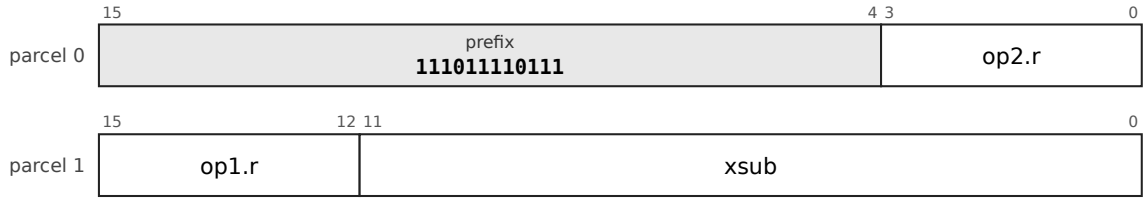
FSINGLE: Floating arithmetic, binary32

Mnemonics: `FADD32`, `FSUB32`, `FMUL32`, `FDIV32`, `FSQRT32`, `FCMP32` (aliases: `FADDS`, `FSUBS`, `FMULS`, `FDIVS`, `FSQRTS`, `FCMPS`)

Syntax

```
FADD32 Rs, Rd
FSUB32 Rs, Rd
FMUL32 Rs, Rd
FDIV32 Rs, Rd
FSQRT32 Rs, Rd
FCMP32 Rd, Rs
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← Rd op Rs  (FADD32 FSUB32 FMUL32 FDIV32); Rd ← sqrt(Rs) (FSQRT32)
FCMP32: flags as FCMP, comparing Rd with Rs
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP32` writes `N Z V C` as `FCMP` does.

Exceptions None.

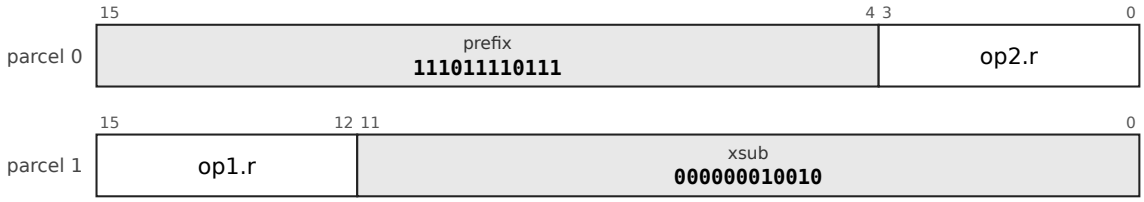
A `binary32` lives in the low 32 bits of a register, sign-extended to 64 like every 32-bit result. Results are correctly rounded to nearest even. The alternative spellings `FADDS`, `FSUBS`, ... are aliases.

Why it is here. `binary64` has first-class opcodes because on a 64-bit machine it is the common case; `binary32` has the same operations, held in the low 32 bits of a register and sign-extended like every other 32-bit value (see also the ABI). The naming rule is an `F` in front of the integer mnemonic (`FADD`) and a size suffix for the other formats (`FADD16`); the older spellings (`FADDS` and so on) are aliases.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FADD32

FADD32: R2, prefix 111011110111, sub-opcode 000000010010; operands reg4, reg4; 4 bytes

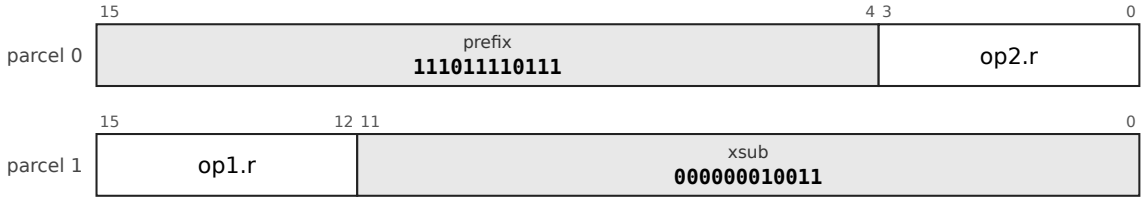



```
FADD32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FSUB32

FSUB32: R2, prefix 111011110111, sub-opcode 000000010011; operands reg4, reg4; 4 bytes

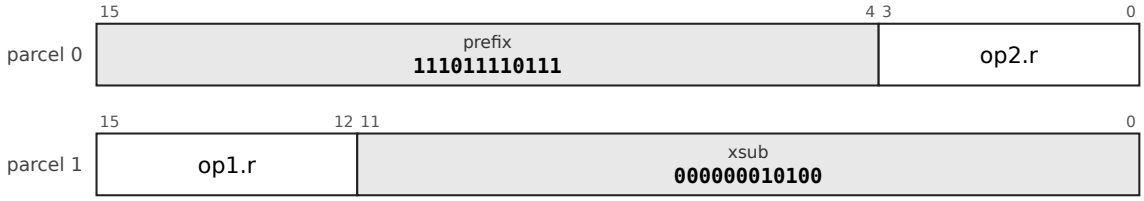



```
FSUB32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FMUL32

FMUL32: R2, prefix 111011110111, sub-opcode 000000010100; operands reg4, reg4; 4 bytes

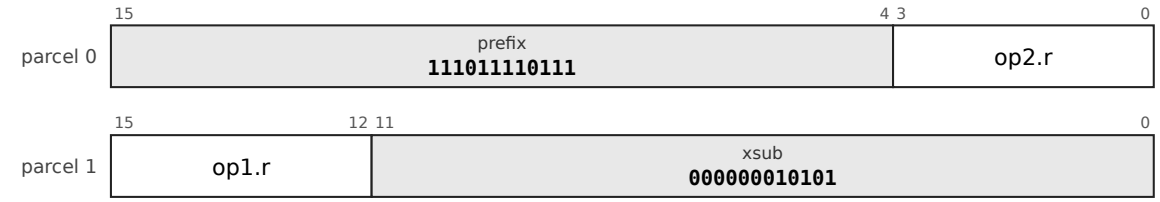



```
FMUL32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FDIV32

FDIV32: R2, prefix 111011110111, sub-opcode 000000010101; operands reg4, reg4; 4 bytes

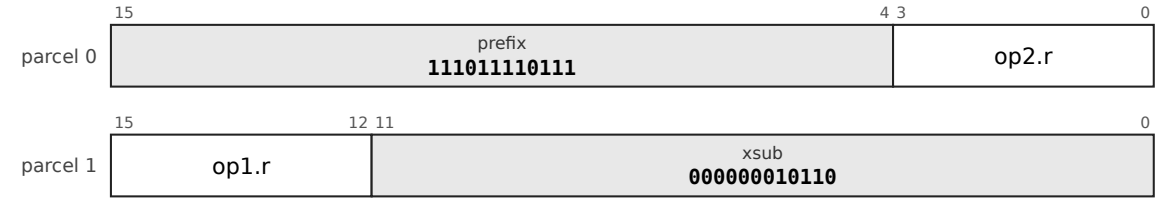



```
FDIV32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FSQRT32

FSQRT32: R2, prefix 111011110111, sub-opcode 000000010110; operands reg4, reg4; 4 byte

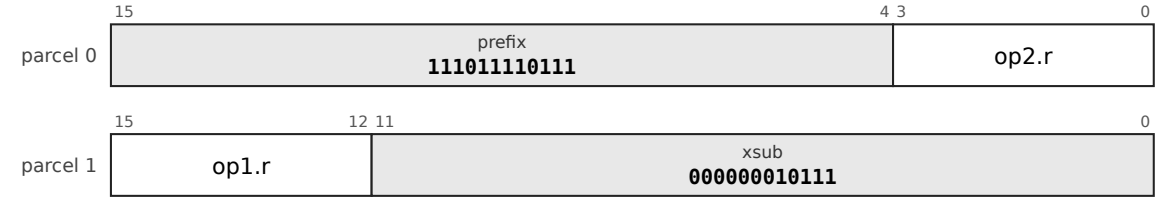



```
FSQRT32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FCMP32

FCMP32: R2, prefix 111011110111, sub-opcode 000000010111; operands reg4, reg4; 4 bytes




```
FCMP32 Rd, Rs
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

FADDS

An alias of `FADD32`: the assembler writes it as that word.

FSUBS

An alias of `FSUB32`: the assembler writes it as that word.

FMULS

An alias of `FMUL32`: the assembler writes it as that word.

FDIVS

An alias of `FDIV32`: the assembler writes it as that word.

FSQRTS

An alias of `FSQRT32`: the assembler writes it as that word.

FCMPS

An alias of `FCMP32`: the assembler writes it as that word.

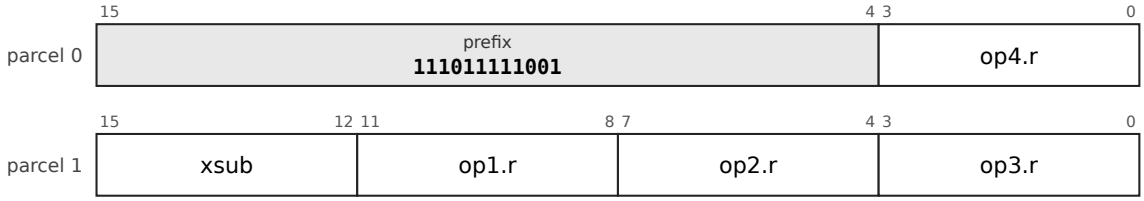
FMADD: Fused multiply-add, binary64

Mnemonics: `FMADD`, `FMSUB`, `FNMADD`, `FNMSUB` (aliases: `FMADDD`, `FMSUBD`, `FNMADDD`, `FNMSUBD`)

Syntax

```
FMADD Rs, Rt, Ru, Rd
FMSUB Rs, Rt, Ru, Rd
FNMADD Rs, Rt, Ru, Rd
FNMSUB Rs, Rt, Ru, Rd
```

Encoding



The chart is the leaf `16-R4-1`, R4, prefix `111011111001`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R4-1`.

Operation

```
FMADD : Rd ← Rs×Rt + Ru
FMSUB : Rd ← Rs×Rt - Ru
FNMADD: Rd ← -(Rs×Rt) - Ru
FNMSUB: Rd ← -(Rs×Rt) + Ru          ; each rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

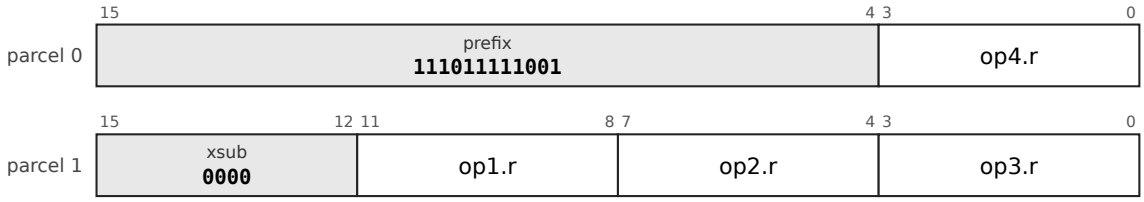
The alternative spellings `FMADDD`, ... are aliases.

Why it is here. The whole fused multiply-add family is in the instruction set because its absence would be the single worst gap in the floating-point set: one rounding instead of two, and the inner step of dot products, polynomial evaluation and every matrix kernel. It has four register operands. It replaces the VAX `POLY`, which was the best answer before fused multiply-add.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FMADD

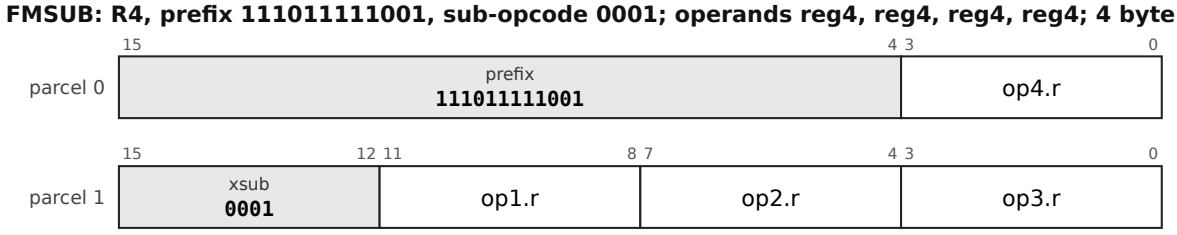
FMADD: R4, prefix 111011111001, sub-opcode 0000; operands reg4, reg4, reg4, reg4; 4 byte




```
FMADD Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

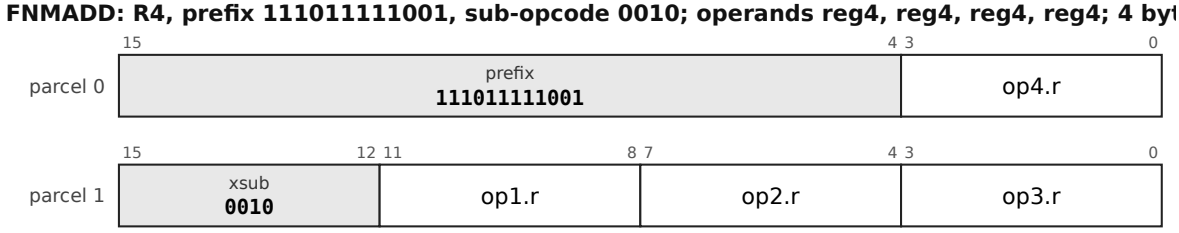
FMSUB




```
FMSUB Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

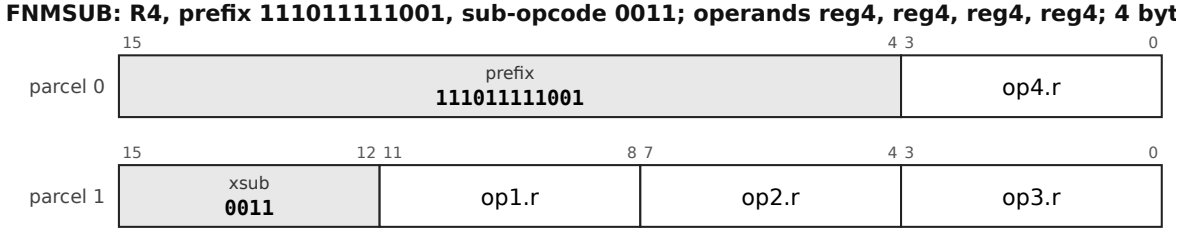
FNMADD




```
FNMADD Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

FNMSUB




```
FNMSUB Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

FMADDD
An alias of `FMADD`: the assembler writes it as that word.

FMSUBD
An alias of `FMSUB`: the assembler writes it as that word.

FNMADDD
An alias of `FNMADD`: the assembler writes it as that word.

FNMSUBD
An alias of `FNMSUB`: the assembler writes it as that word.

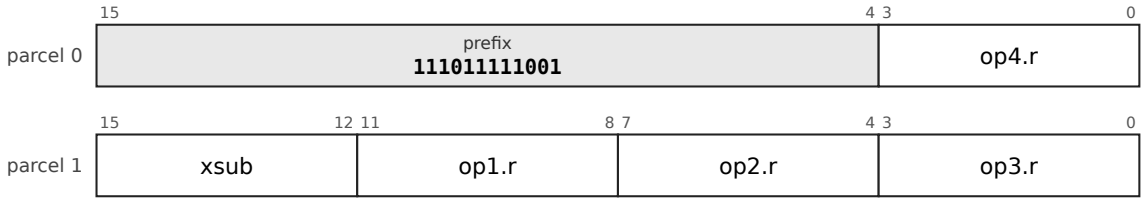
FMADD32: Fused multiply-add, binary32

Mnemonics: `FMADD32`, `FMSUB32`, `FNMADD32`, `FNMSUB32` (aliases: `FMADDS`, `FMSUBS`, `FNMADDS`, `FNMSUBS`)

Syntax

```
FMADD32 Rs, Rt, Ru, Rd
FMSUB32 ...
FNMADD32 ...
FNMSUB32 ...
```

Encoding



The chart is the leaf `16-R4-1`, R4, prefix `111011111001`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R4-1`.

Operation

```
As FMADD, in binary32, rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

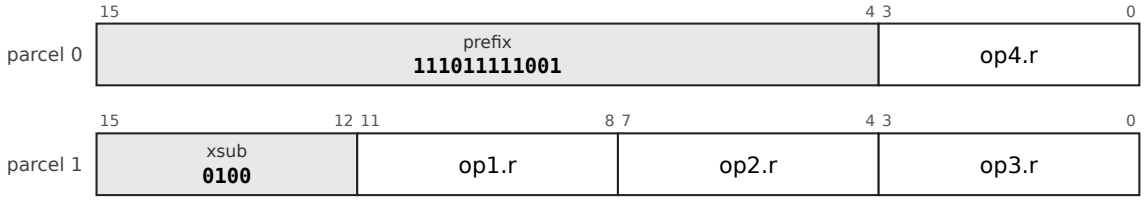
Exceptions None.

Why it is here. The `binary32` forms of the fused multiply-add family.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FMADD32

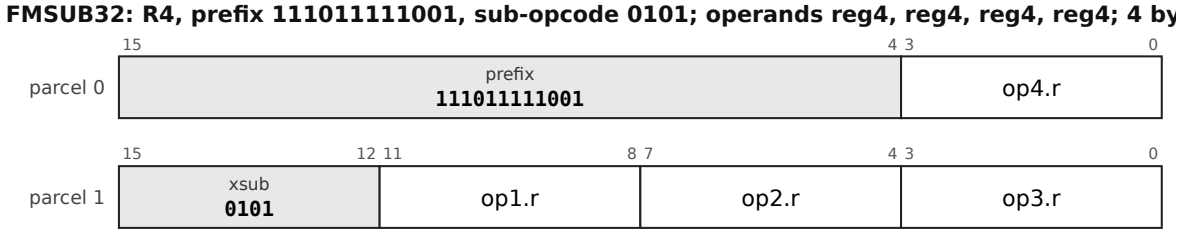
FMADD32: R4, prefix 111011111001, sub-opcode 0100; operands reg4, reg4, reg4, reg4; 4 by




```
FMADD32 Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

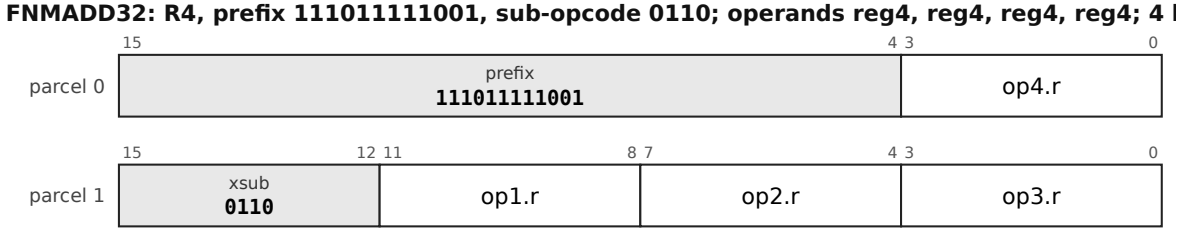
FMSUB32




```
FMSUB32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

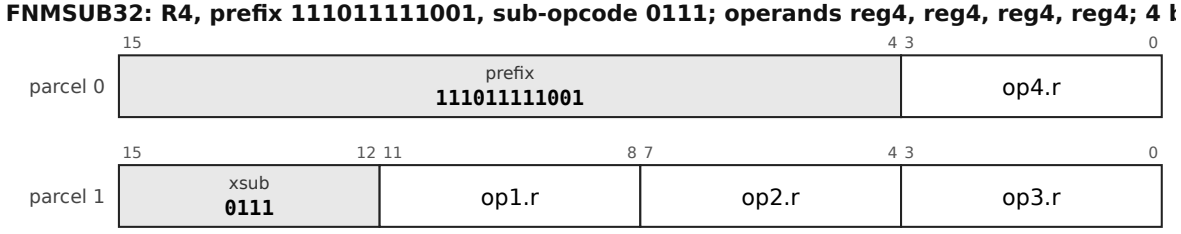
FNMADD32




```
FNMADD32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

FNMSUB32




```
FNMSUB32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

FMADDS
An alias of `FMADD32`: the assembler writes it as that word.

FMSUBS
An alias of `FMSUB32`: the assembler writes it as that word.

FNMADDS
An alias of `FNMADD32`: the assembler writes it as that word.

FNMSUBS
An alias of `FNMSUB32`: the assembler writes it as that word.

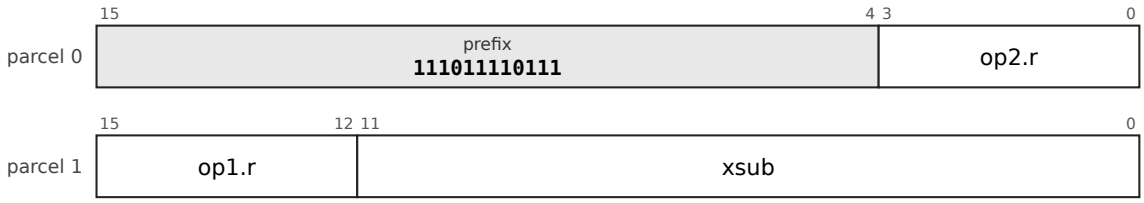
FSIGN: Sign operations on the bits

Mnemonics: `FNEG`, `FABS`, `FCOPYSIGN`

Syntax

```
FNEG Rs, Rd
FABS Rs, Rd
FCOPYSIGN Rs, Rt, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`, `16-R3-1`.

Operation

```
FNEG     : Rd ← Rs with the sign bit flipped
FABS     : Rd ← Rs with the sign bit cleared
FCOPYSIGN: Rd ← magnitude of Rs, sign of Rt
```

Flags None. These operate on the bits, do not canonicalise a NaN and raise no exception flag.

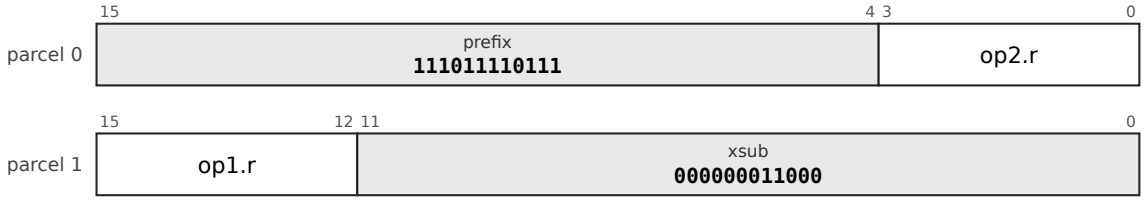
Exceptions None.

Why it is here. Sign and selection operations (negate, absolute value and the like) work on the bits, do not canonicalise a NaN and raise no flag, so they cost almost nothing. The PowerPC 601 floating-point set has the same operations (stated from memory, not checked against a manual). No separate reason is recorded for these three.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FNEG

FNEG: R2, prefix 111011110111, sub-opcode 000000011000; operands reg4, reg4; 4 bytes

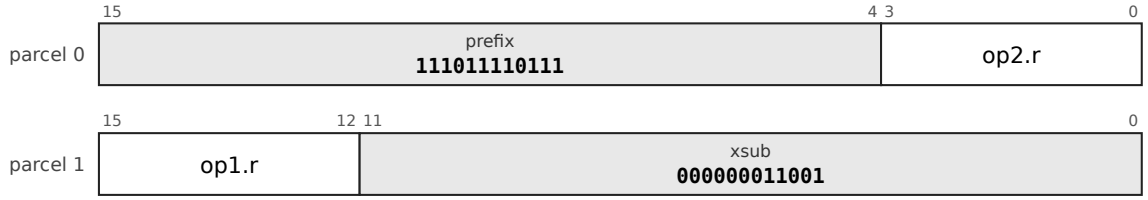



```
FNEG Rs, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

FABS

FABS: R2, prefix 111011110111, sub-opcode 000000011001; operands reg4, reg4; 4 bytes

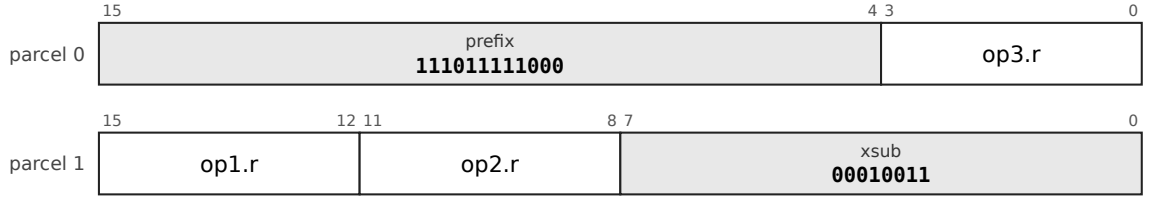



```
FABS Rs, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

FCOPYSIGN

FCOPYSIGN: R3, prefix 111011111000, sub-opcode 00010011; operands reg4, reg4, reg4; 4 b




```
FCOPYSIGN Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

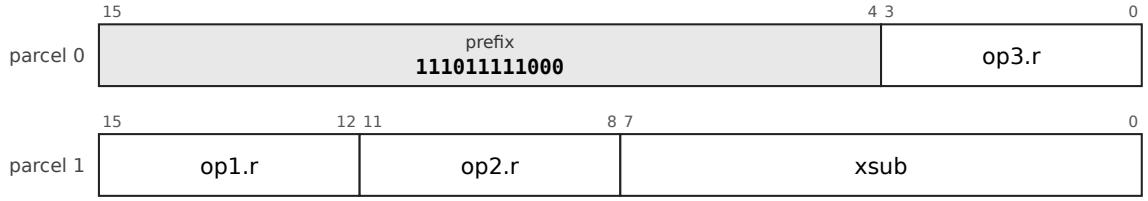
FMIN: Floating minimum and maximum, binary64

Mnemonics: `FMIN`, `FMAX` (aliases: `FMIND`, `FMAXD`)

Syntax

```
FMIN Rs, Rt, Rd
FMAX Rs, Rt, Rd
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
if Rs is a NaN then Rd ← Rt ; else if Rt is a NaN then Rd ← Rs ; else Rd ← smaller (FMIN) or larger (FMAX)
if both are NaNs the result is the canonical quiet NaN
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

Exceptions None.

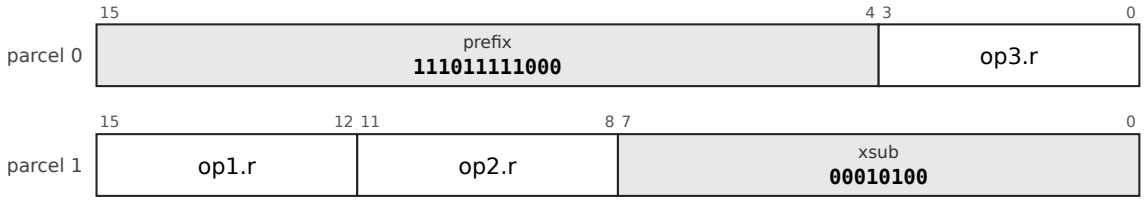
The alternative spellings FMIND, FMAXD are aliases.

Why it is here. The floating counterparts of the integer MIN and MAX. A NaN operand loses to a number and two NaNs give the canonical quiet NaN.

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

FMIN

FMIN: R3, prefix 111011111000, sub-opcode 00010100; operands reg4, reg4, reg4; 4 bytes

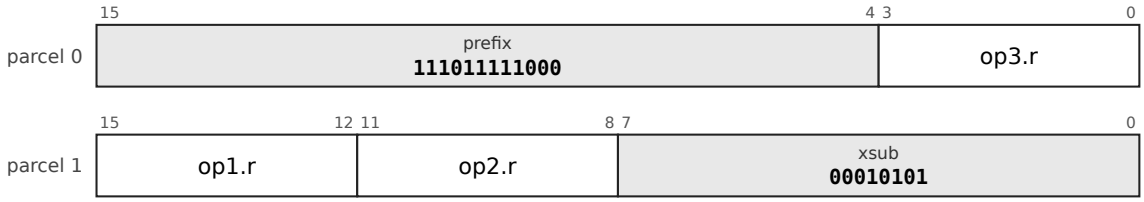



```
FMIN Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FMIN above; 1 layout form, each a chart of its own (above).

FMAX

FMAX: R3, prefix 111011111000, sub-opcode 00010101; operands reg4, reg4, reg4; 4 bytes




```
FMAX Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FMIN above; 1 layout form, each a chart of its own (above).

FMIND

An alias of FMIN: the assembler writes it as that word.

FMAXD

An alias of FMAX: the assembler writes it as that word.

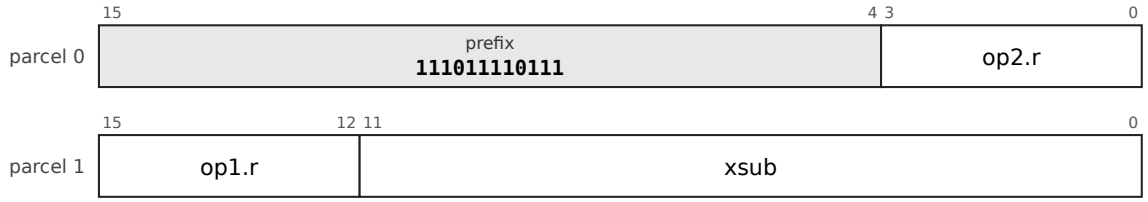
FCLASS: Classify a floating-point value

Mnemonics: FCLASS, FCLASS32, FCLASS16 (aliases: FCLASSD, FCLASSS)

Syntax

```
FCLASS Rs, Rd
FCLASS32 Rs, Rd
FCLASS16 Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← a bit vector with exactly one bit set:
bit 0 -infinity      bit 1 -normal      bit 2 -subnormal   bit 3 -zero
bit 4 +zero          bit 5 +subnormal   bit 6 +normal      bit 7 +infinity
bit 8 signalling NaN bit 9 quiet NaN
```

Flags None.

Exceptions None.

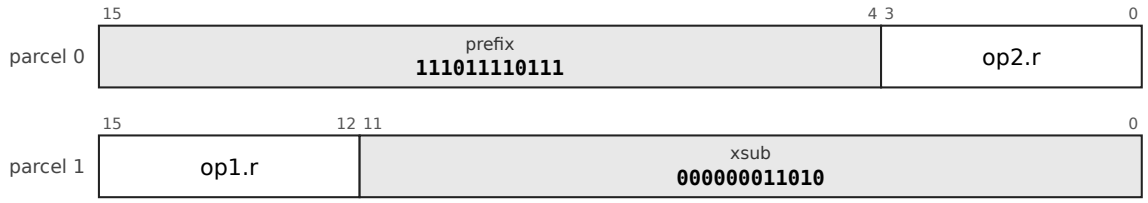
Every libm function begins with this dispatch. The alternative spellings `FCLASSD`, `FCLASSS` are aliases.

Why it is here. Every libm function begins with this dispatch: one instruction returns which of ten kinds a value is (signalling or quiet NaN, infinity, normal, subnormal, zero, each signed), where software needs several compares and masks. It is justified by a workload, library mathematics, and not by compiled C, as the tag branches are. Nothing in the corpus emits it, because the compiler does not use it.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FCLASS

FCLASS: R2, prefix 111011110111, sub-opcode 000000011010; operands reg4, reg4; 4 bytes

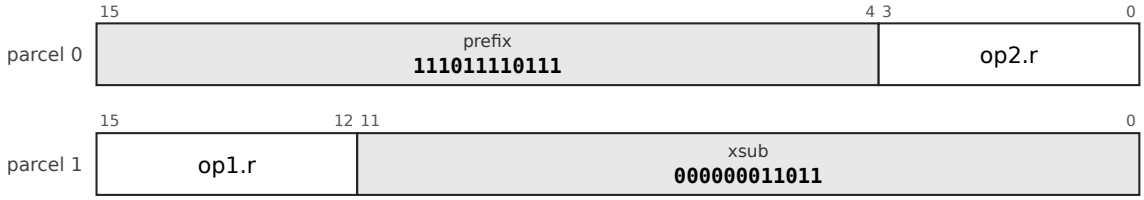



```
FCLASS Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

FCLASS32

FCLASS32: R2, prefix 111011110111, sub-opcode 000000011011; operands reg4, reg4; 4 byt

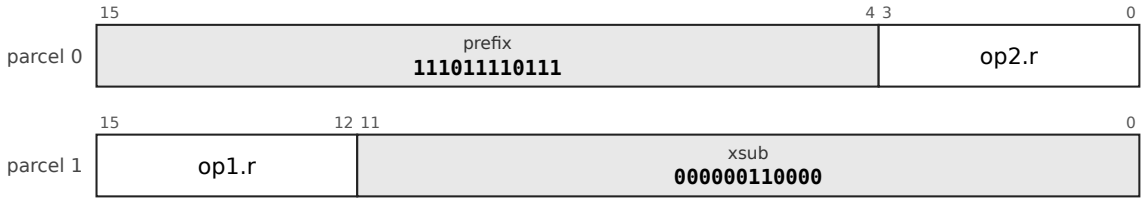



```
FCLASS32 Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

FCLASS16

FCLASS16: R2, prefix 111011110111, sub-opcode 000000110000; operands reg4, reg4; 4 byt




```
FCLASS16 Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

FCLASSD

An alias of `FCLASS`: the assembler writes it as that word.

FCLASSS

An alias of `FCLASS32`: the assembler writes it as that word.

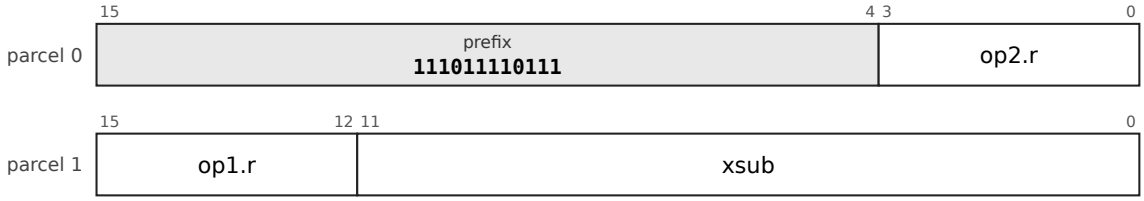
FRINT: Round to an integral value, binary64

Mnemonics: `FRINT`, `FRINTZ`, `FRINTM`, `FRINTP`, `FRINTA` (aliases: `FRINTD`, `FRINTDZ`, `FRINTDM`, `FRINTDP`, `FRINTDA`)

Syntax

```
FRINT Rs, Rd
FRINTZ Rs, Rd
FRINTM Rs, Rd
FRINTP Rs, Rd
FRINTA Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
FRINT : Rd ← Rs rounded to nearest, ties to even
FRINTZ: toward zero         FRINTM: toward -infinity
FRINTP: toward +infinity    FRINTA: to nearest, ties away from zero (C round())
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

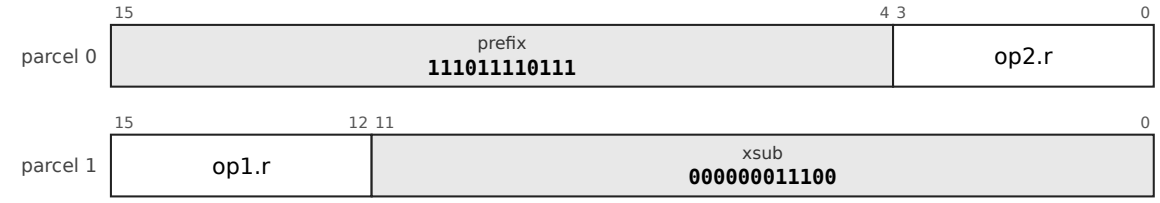
The rounding mode is in the instruction, never in a mode bit. The alternative spellings `FRINTD`, `FRINTDZ`, ... are aliases.

Why it is here. Rounding to an integral value in each direction (nearest even, toward zero, down, up) is in the set, and `FRINTA`, ties away from zero, serves C's `round()`, which is common in libm; it is a qualifier of the round operation and so costs no new operation.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FRINT

FRINT: R2, prefix 111011110111, sub-opcode 000000011100; operands reg4, reg4; 4 bytes

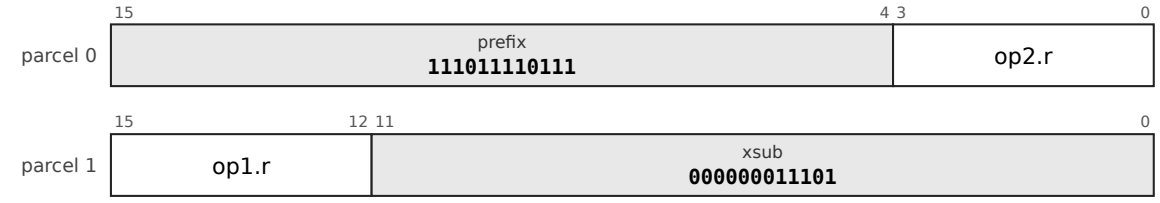



```
FRINT Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

FRINTZ

FRINTZ: R2, prefix 111011110111, sub-opcode 000000011101; operands reg4, reg4; 4 bytes

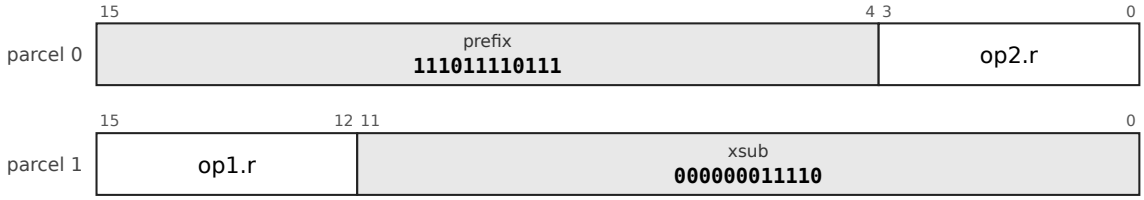



```
FRINTZ Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

FRINTM

FRINTM: R2, prefix 111011110111, sub-opcode 000000011110; operands reg4, reg4; 4 bytes

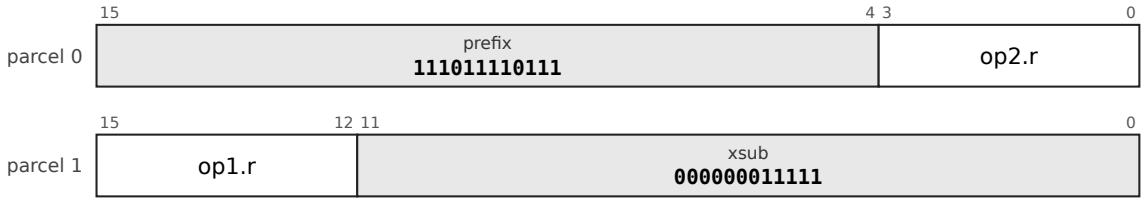



```
FRINTM Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

FRINTP

FRINTP: R2, prefix 111011110111, sub-opcode 000000011111; operands reg4, reg4; 4 bytes

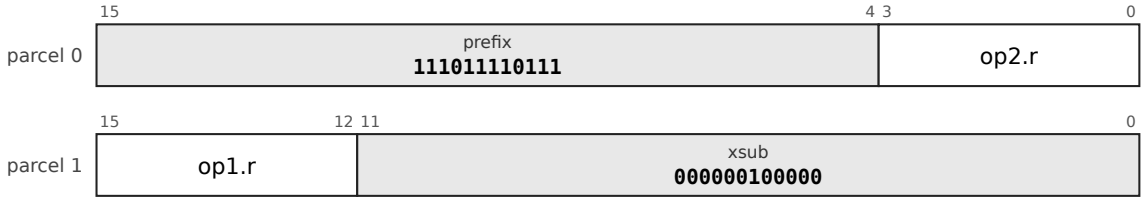



```
FRINTP Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

FRINTA

FRINTA: R2, prefix 111011110111, sub-opcode 000000100000; operands reg4, reg4; 4 bytes




```
FRINTA Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

FRINTD

An alias of FRINT: the assembler writes it as that word.

FRINTDZ

An alias of FRINTZ: the assembler writes it as that word.

FRINTDM

An alias of FRINTM: the assembler writes it as that word.

FRINTDP

An alias of FRINTP: the assembler writes it as that word.

FRINTDA

An alias of FRINTA: the assembler writes it as that word.

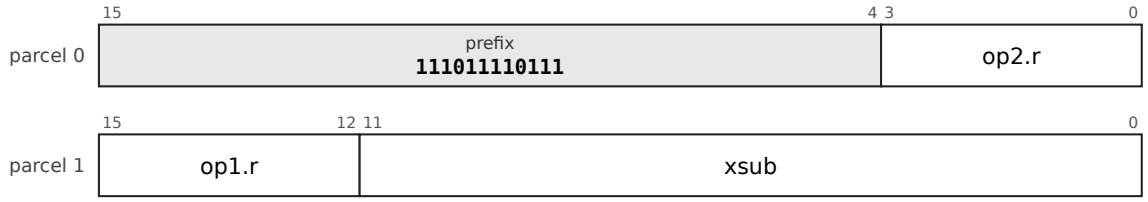
FCVT: Floating conversions

Mnemonics: FCVTDQ, FCVTDQU, FCVTQD, FCVTQUD, FCVTDS, FCVTSD, FCVTSL, FCVTLS

Syntax

```
FCVTDQ Rs, Rd        ; binary64 to signed quad
FCVTDQU Rs, Rd       ; binary64 to unsigned quad
FCVTQD Rs, Rd        ; signed quad to binary64
FCVTQUD Rs, Rd       ; unsigned quad to binary64
FCVTDS Rs, Rd        ; binary64 to binary32
FCVTSD Rs, Rd        ; binary32 to binary64
FCVTSL Rs, Rd        ; binary32 to signed 32-bit integer
FCVTLS Rs, Rd        ; signed 32-bit integer to binary32
```

Encoding



The chart is the leaf 16-R2-1, R2, prefix 111011110111. op1 is reg4: register, 4 bits. op2 is reg4: register, 4 bits. xsub selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): 16-R2-1.

Operation

```
float to integer: truncate toward zero, then saturate; a NaN gives the largest value
integer to float: round to nearest even
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps. A NaN or an out-of-range value raises NV and saturates.

Exceptions None.

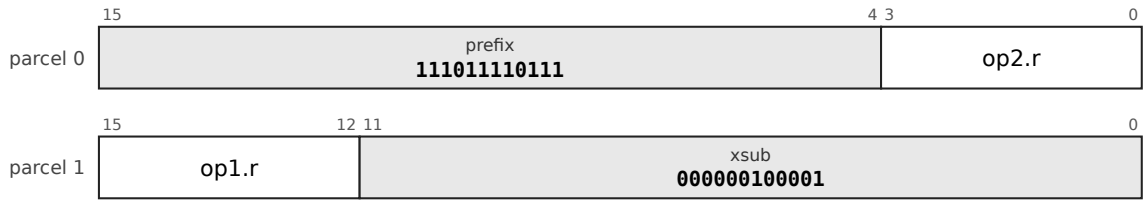
Integer results are 64-bit (FCVTDQ, FCVTDQU) or 32-bit sign-extended (FCVTSL). Unsigned conversion of a negative value is invalid and gives 0.

Why it is here. FCVTDQU and FCVTQUD convert unsigned 64-bit integers to and from binary64. C needs both, and without them each takes a branch and several instructions.

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

FCVTDQ

FCVTDQ: R2, prefix 111011110111, sub-opcode 000000100001; operands reg4, reg4; 4 bytes

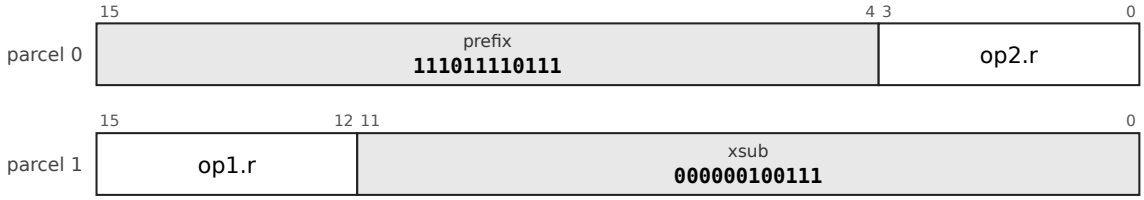



```
FCVTDQ Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTDQU

FCVTDQU: R2, prefix 111011110111, sub-opcode 000000100111; operands reg4, reg4; 4 byte

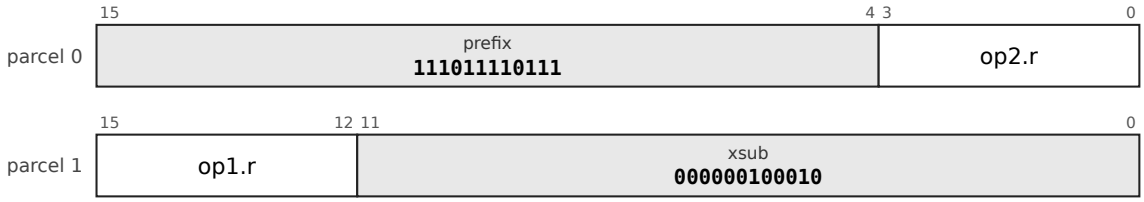



```
FCVTDQU Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTQD

FCVTQD: R2, prefix 111011110111, sub-opcode 000000100010; operands reg4, reg4; 4 bytes

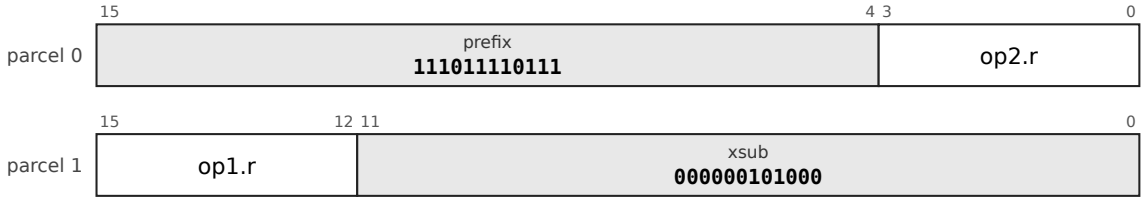



```
FCVTQD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTQUD

FCVTQUD: R2, prefix 111011110111, sub-opcode 000000101000; operands reg4, reg4; 4 byte

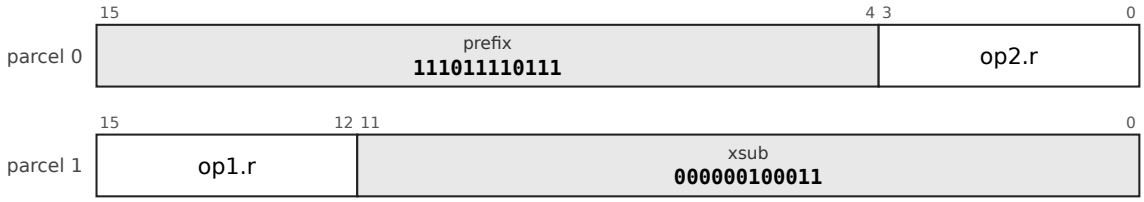



```
FCVTQUD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTDS

FCVTDS: R2, prefix 111011110111, sub-opcode 000000100011; operands reg4, reg4; 4 bytes

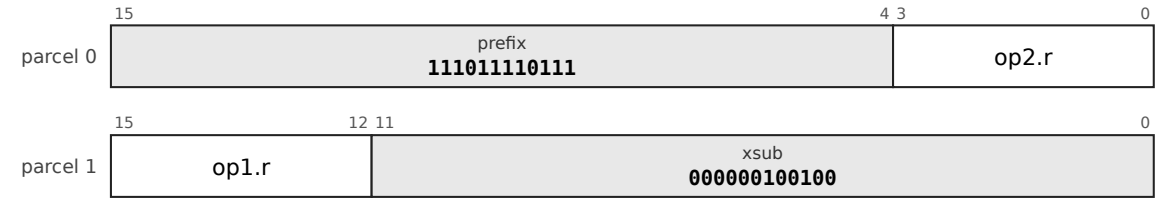



```
FCVTDS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTSD

FCVTSD: R2, prefix 111011110111, sub-opcode 000000100100; operands reg4, reg4; 4 bytes

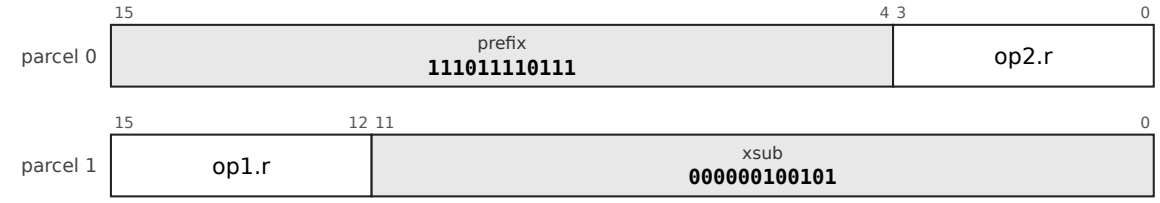



```
FCVTSD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTSL

FCVTSL: R2, prefix 111011110111, sub-opcode 000000100101; operands reg4, reg4; 4 bytes

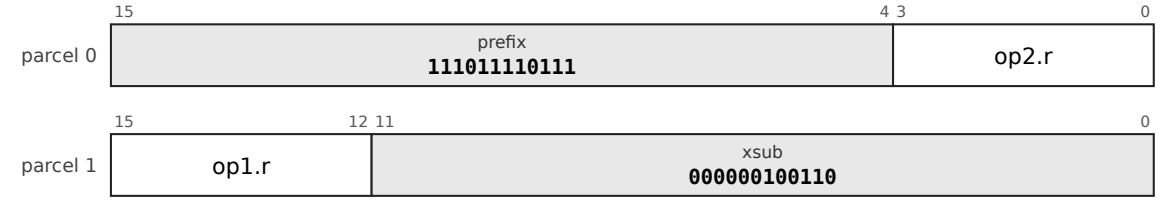



```
FCVTSL Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FCVTLS

FCVTLS: R2, prefix 111011110111, sub-opcode 000000100110; operands reg4, reg4; 4 bytes

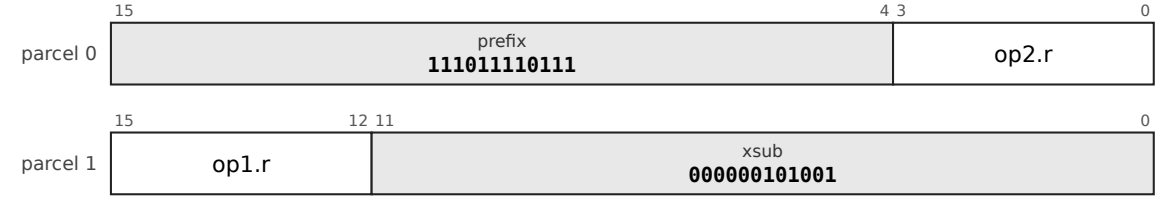



```
FCVTLS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

FSFLAGS: Swap the floating-point exception flags

FSFLAGS: R2, prefix 111011110111, sub-opcode 000000101001; operands reg4, reg4; 4 byte

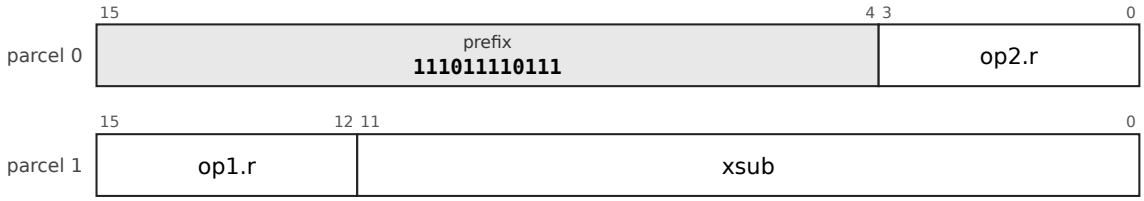


Mnemonics: `FSFLAGS`

Syntax

```
FSFLAGS Rs, Rd
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
old ← FPSR
FPSR ← Rs[4:0]
Rd ← old
```

Flags None.

Exceptions None.

`FPSR` is the sticky five-bit register (`NX` bit 0, `UF` 1, `OF` 2, `DZ` 3, `NV` 4). C99 `<fenv.h>` is built on this one instruction.

Why it is here. IEEE 754 requires five status flags, and C99 `<fenv.h>` needs software access to them. The architecture has a sticky five-bit `FPSR`, laid out as RISC-V's `fflags`, written by every operation that can raise a flag and never trapping, and one swap instruction to read and write it. It is a record of what happened, not a mode, so the rule against mode bits is untouched.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

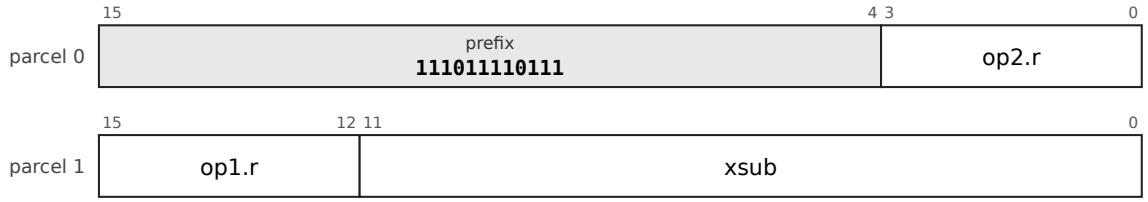
FHALF: Floating arithmetic, binary16

Mnemonics: `FADD16`, `FSUB16`, `FMUL16`, `FDIV16`, `FSQRT16`, `FCMP16`, `FMADD16`, `FMSUB16`, `FNMADD16`, `FNMSUB16`

Syntax

```
FADD16 Rs, Rd
FSUB16 Rs, Rd
FMUL16 Rs, Rd
FDIV16 Rs, Rd
FSQRT16 Rs, Rd
FCMP16 Rd, Rs
FMADD16 Rs, Rt, Ru, Rd ...
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`, `16-R4-1`.

Operation

```
as the binary32 words, in binary16: Rd ← Rd op Rs ; the fused forms take Rs, Rt, Ru, Rd
a half lives in the low 16 bits of a register, sign-extended to 64
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP16` writes `N Z V C` as `FCMP`.

Exceptions None.

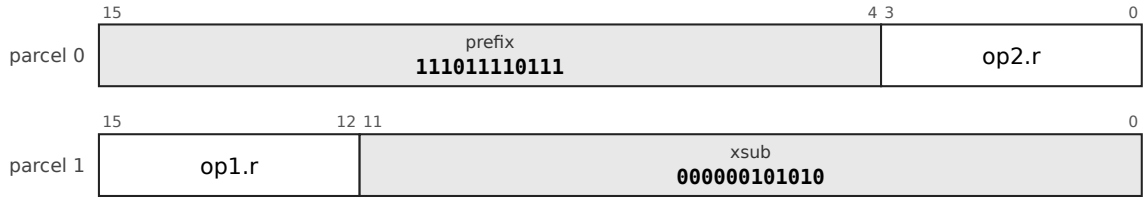
Results are correctly rounded to nearest even, tininess judged after rounding; a NaN result is the canonical `0x7E00`. Half `FMIN`, `FMAX`, `FRINT*`, bfloat16 and conversions to and from 64-bit integers are open items.

Why it is here. Half precision began as a GPU format (NVIDIA's `half`, 2002), spread to machine learning and image formats, and is in CPUs (x86 `F16C` and AVX-512 FP16, ARMv8.2, RISC-V `Zfh` and `Zfhmin`); this account is from memory and not checked against a manual. Two levels are possible: conversions only (about 500 to 1,000 lookup tables, no new operation by the folding rule) and full arithmetic (about 2,000 to 3,500 tables; estimates), and the existing double datapath can compute half correctly rounded. The architecture has both conversions and arithmetic. The leftovers (half `FMIN`, `FRINT*`, bfloat16, 64-bit integer conversions) are open items. 16-bit *integer* arithmetic is not provided: the ALU has one width (9.2.9).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FADD16

FADD16: R2, prefix 111011110111, sub-opcode 000000101010; operands reg4, reg4; 4 bytes

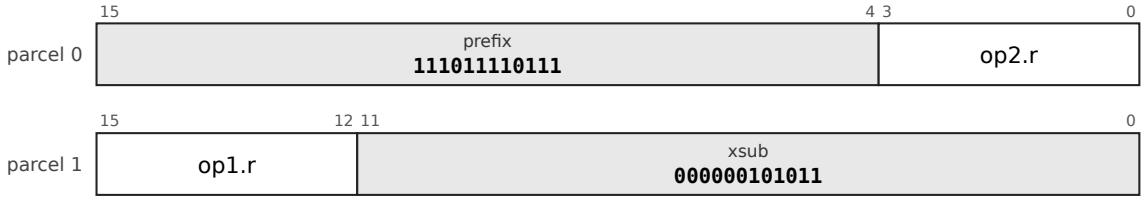



```
FADD16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FSUB16

FSUB16: R2, prefix 111011110111, sub-opcode 000000101011; operands reg4, reg4; 4 bytes

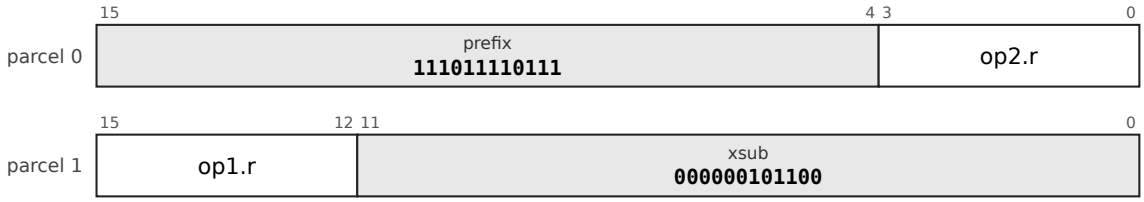



```
FSUB16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FMUL16

FMUL16: R2, prefix 111011110111, sub-opcode 000000101100; operands reg4, reg4; 4 bytes

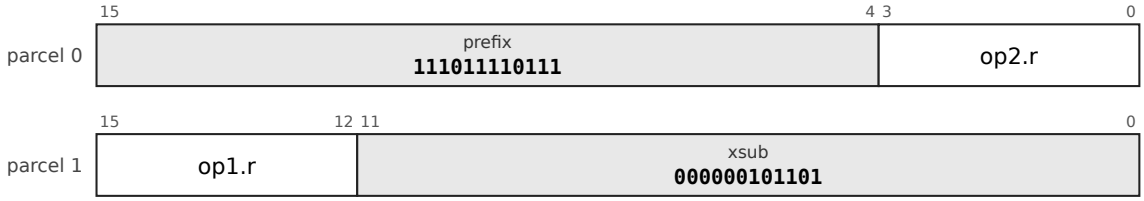



```
FMUL16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FDIV16

FDIV16: R2, prefix 111011110111, sub-opcode 000000101101; operands reg4, reg4; 4 bytes

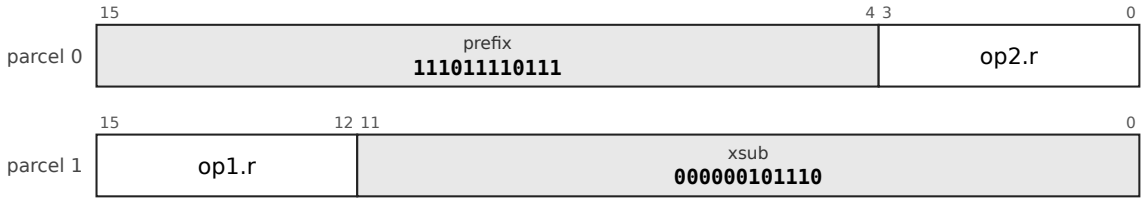



```
FDIV16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FSQRT16

FSQRT16: R2, prefix 111011110111, sub-opcode 000000101110; operands reg4, reg4; 4 byte

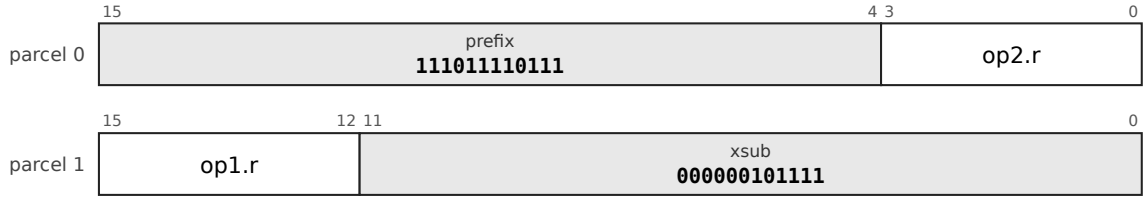



```
FSQRT16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FCMP16

FCMP16: R2, prefix 111011110111, sub-opcode 000000101111; operands reg4, reg4; 4 bytes

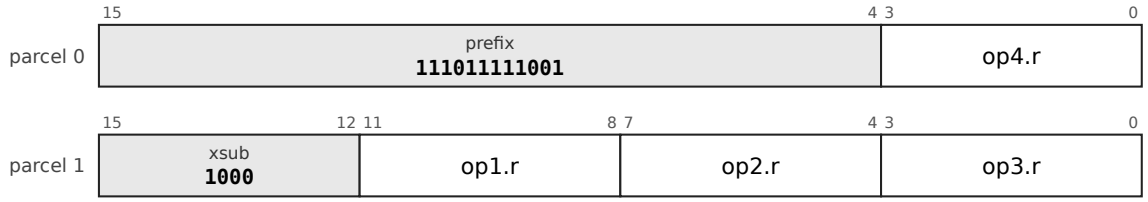



```
FCMP16 Rd, Rs
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FMADD16

FMADD16: R4, prefix 111011111001, sub-opcode 1000; operands reg4, reg4, reg4, reg4; 4 by

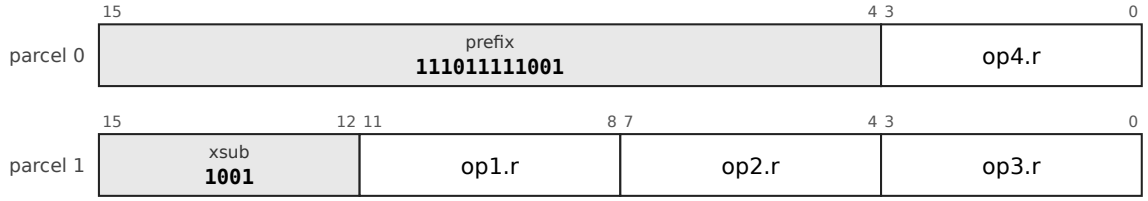



```
FMADD16 Rs, Rt, Ru, Rd ...
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FMSUB16

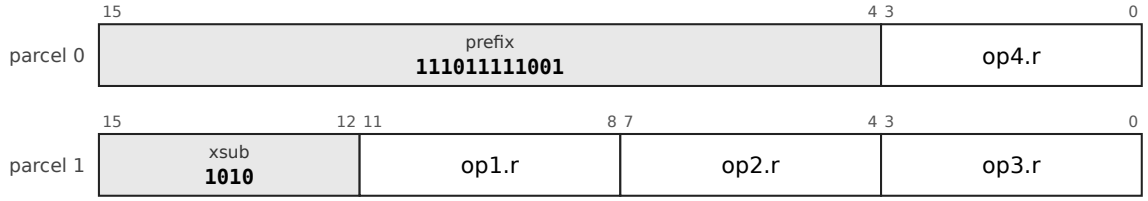
FMSUB16: R4, prefix 111011111001, sub-opcode 1001; operands reg4, reg4, reg4, reg4; 4 by



Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

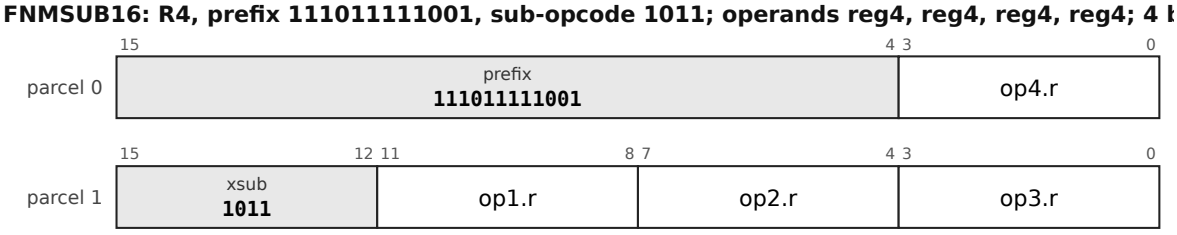
FNMADD16

FNMADD16: R4, prefix 111011111001, sub-opcode 1010; operands reg4, reg4, reg4, reg4; 4 I



Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

FNMSUB16



Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

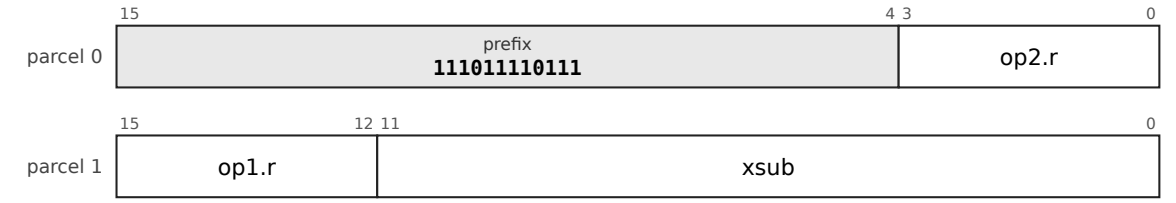
FCVTH: Conversions to and from binary16

Mnemonics: `FCVTSH`, `FCVTHS`, `FCVTDH`, `FCVTHD`, `FCVTHL`, `FCVTLH`

Syntax

```
FCVTSH Rs, Rd        ; binary32 to half
FCVTHS Rs, Rd        ; half to binary32
FCVTDH Rs, Rd        ; binary64 to half
FCVTHD Rs, Rd        ; half to binary64
FCVTHL Rs, Rd        ; half to signed 32-bit integer, toward zero
FCVTLH Rs, Rd        ; signed 32-bit integer to half
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
narrowing: round to nearest even ; widening: exact
FCVTHL of a NaN or an infinity is invalid and saturates; truncating a fraction is inexact
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

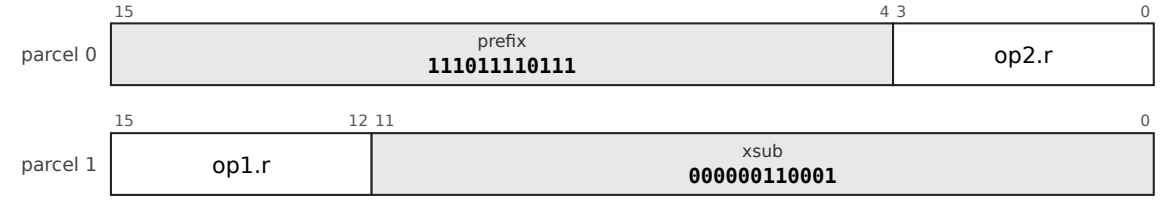
Exceptions None.

Why it is here. The conversion half of the `binary16` support (see `FHALF`). The conversions are the part machine-learning code needs most, because a half is normally loaded and stored with `MOVW`, converted, computed in single or double, and converted back.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FCVTSH

FCVTSH: R2, prefix 111011110111, sub-opcode 000000110001; operands reg4, reg4; 4 bytes

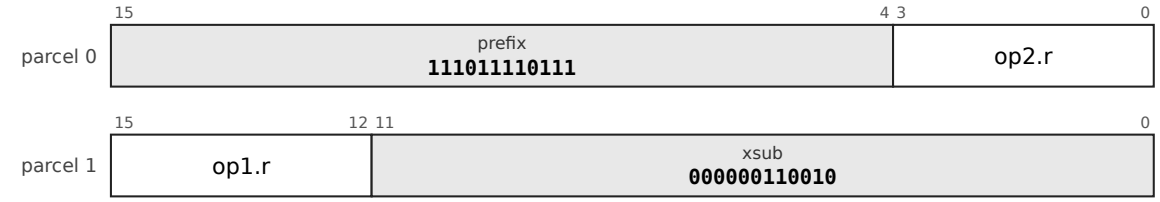



```
FCVTSH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

FCVTHS

FCVTHS: R2, prefix 111011110111, sub-opcode 000000110010; operands reg4, reg4; 4 bytes

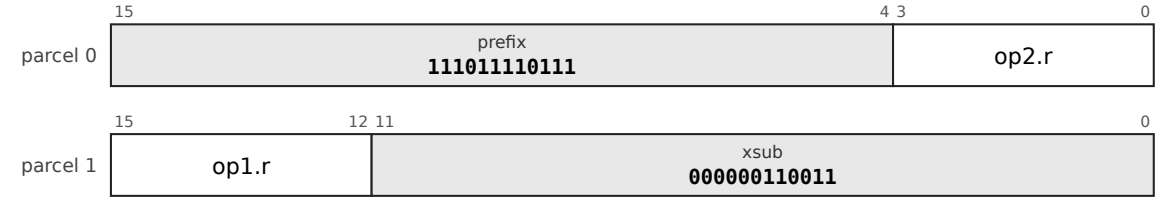



```
FCVTHS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

FCVTDH

FCVTDH: R2, prefix 111011110111, sub-opcode 000000110011; operands reg4, reg4; 4 bytes

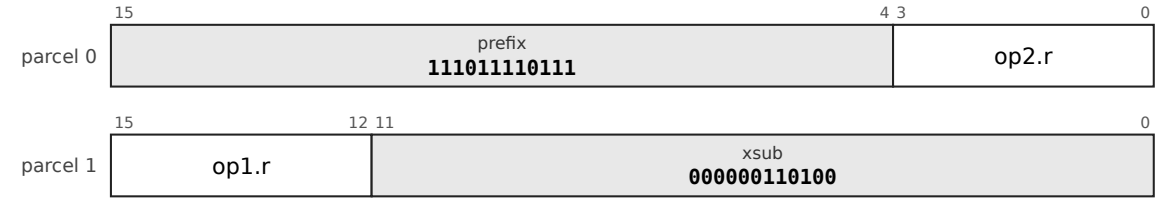



```
FCVTDH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

FCVTHD

FCVTHD: R2, prefix 111011110111, sub-opcode 000000110100; operands reg4, reg4; 4 bytes

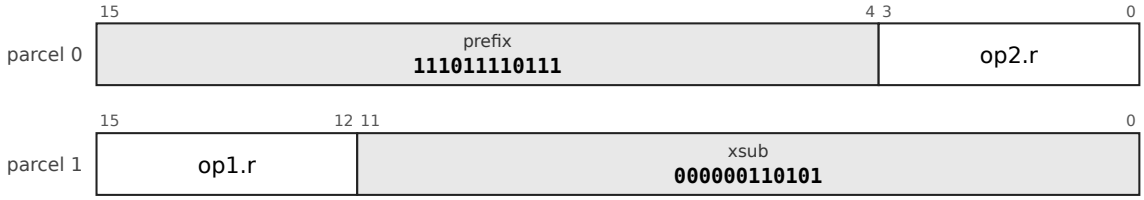



```
FCVTHD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

FCVTHL

FCVTHL: R2, prefix 111011110111, sub-opcode 000000110101; operands reg4, reg4; 4 bytes

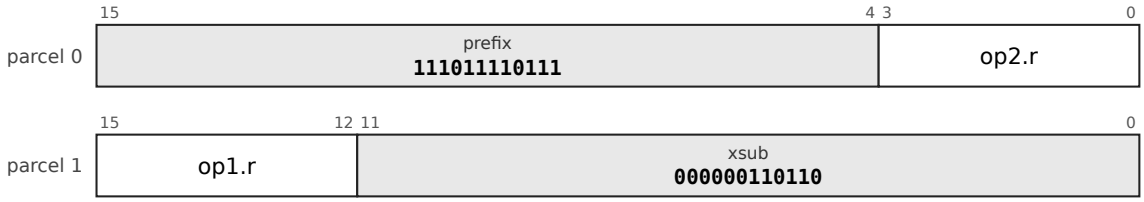



```
FCVTHL Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

FCVTLH

FCVTLH: R2, prefix 111011110111, sub-opcode 000000110110; operands reg4, reg4; 4 bytes




```
FCVTLH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

9.13 Atomics and ordering

Cf. chapter 7, section 7.12 (the atomic and ordering instructions), section 7.3 (the atomics in detail) and section 7.2 (the memory model).

9.13.1 The atomic group ATOM holds CAS, SWAP, XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU and XMINU, each with a _DB variant that adds a barrier, and LL and SC; FENCE, WFE, SEV, ISYNC and HINT are in SYS. Their semantics, ordering rules and the reservation monitor are in chapter 7.

9.13.2 CAS operates on 4 or 8 bytes; an 8-byte compare-and-swap may span two adjacent 4-byte fields, because the usual lock-and-count fast path of an operating system is exactly that (an 8-byte whole over two 4-byte members). *(D)* The encoding of the CAS size is unspecified. *(S)*

9.13.3 The grammar places the operands of the atomics as three registers (XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU, XMINU, SWAP, LL, SC) or four (CAS, CASDB), the memory address in a register, in the order written; which register holds the address, which the operand and which receives the old value is the architecture's to fix, and the text has not fixed it. The size of CAS is likewise open. The reference executor does not implement the group. *(P)*

Commentary. The official entry of each instruction is below, in this section; chapter 7 gives the reasons, the memory model and the use. **Impact.** *Kernel:* the entries below are all a kernel needs to write its locks; chapter 7 explains how they compose. *Compiler:* a lock acquire is one _DB instruction. *Hardware:* a reservation monitor beside the data cache.

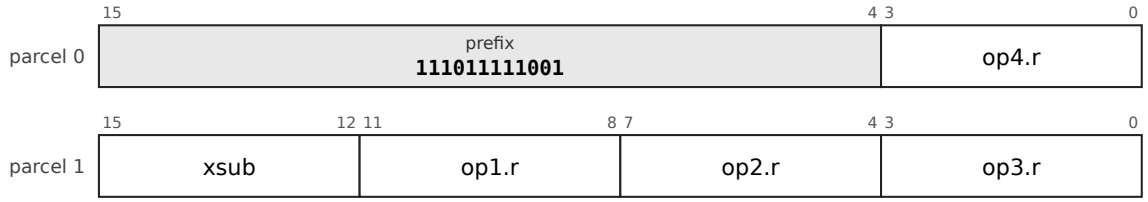
CAS: Compare and swap

Mnemonics: CAS, CASDB

Syntax

```
CAS expected, new, (address)       ; operand placement and size encoding unspecified; size is 4 or 8 bytes
CASDB ...                           ; with a full barrier
```

Encoding



The chart is the leaf `16-R4-1`, R4, prefix `111011111001`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `op4` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R4-1`.

Operation

```
old ← M[address]                        ; 4 or 8 bytes, naturally aligned
if old = expected then M[address] ← new
result ← old          ; the program compares result with expected to learn whether the swap happened
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for `CAS` and `CASDB`), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group.

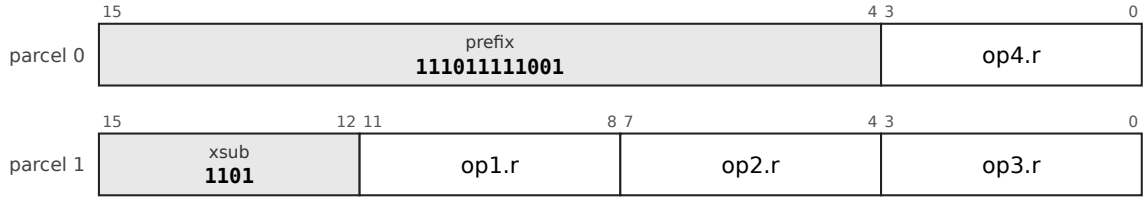
Why it is here. The barrier-fused variants are taken from LoongArch's pragmatism: a lock acquire is one instruction rather than an atomic followed by a `FENCE`, and that pair is in every critical section of the system. With the other multiprocessor operations the atomics make the seventeen-operation SMP umbrella (7.1). They are defined on a reservation monitor and not on a bus lock (2.5).

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

CAS

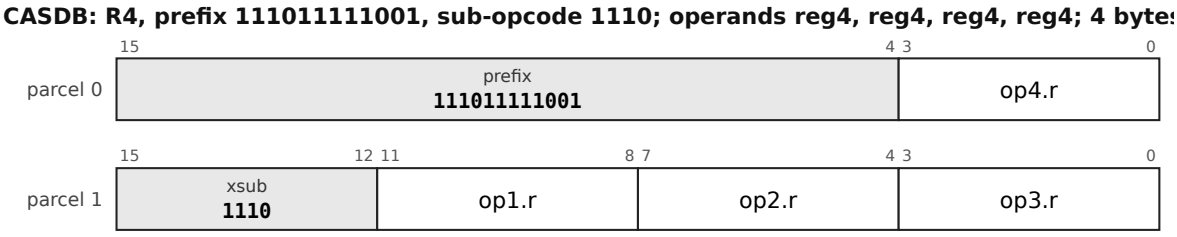
CAS: R4, prefix 111011111001, sub-opcode 1101; operands reg4, reg4, reg4, reg4; 4 bytes




```
CAS expected, new, (address)
```

Operation, flags and exceptions as for the entry CAS above; 1 layout form, each a chart of its own (above).

CASDB




```
CASDB ...
```

Operation, flags and exceptions as for the entry CAS above; 1 layout form, each a chart of its own (above).

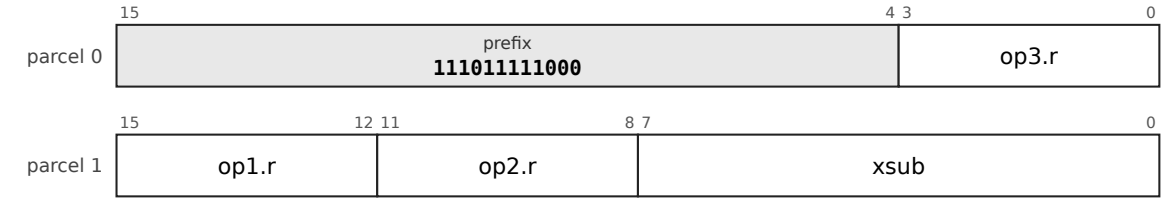
SWAP: Atomic exchange

Mnemonics: `SWAP`, `SWAPDB`

Syntax

```
SWAP value, (address)
SWAPDB ...
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
old ← M[address] ; M[address] ← value ; result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for `CAS` and `CASDB`), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group.

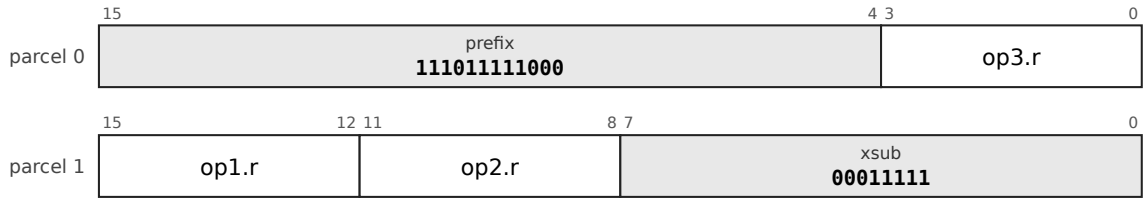
Why it is here. One of the read-modify-write atomics (see `CAS`). No separate reason is recorded for it.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

SWAP

SWAP: R3, prefix 111011111000, sub-opcode 00011111; operands reg4, reg4, reg4; 4 bytes

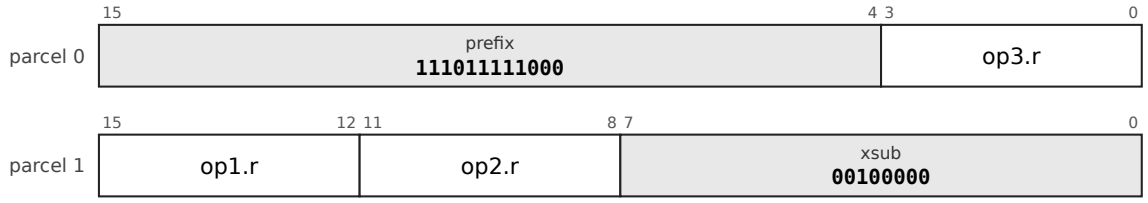



```
SWAP value, (address)
```

Operation, flags and exceptions as for the entry SWAP above; 1 layout form, each a chart of its own (above).

SWAPDB

SWAPDB: R3, prefix 111011111000, sub-opcode 00100000; operands reg4, reg4, reg4; 4 byte




```
SWAPDB ...
```

Operation, flags and exceptions as for the entry SWAP above; 1 layout form, each a chart of its own (above).

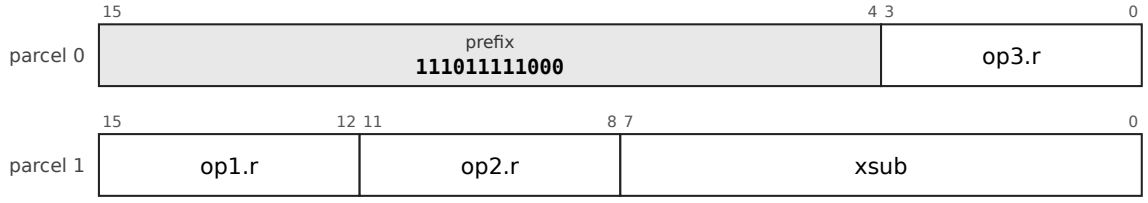
XADD: Atomic fetch and add

Mnemonics: `XADD`, `XADDDB`

Syntax

```
XADD value, (address)
XADDDB ...
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
old ← M[address] ; M[address] ← old + value ; result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for CAS and CASDB), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group.

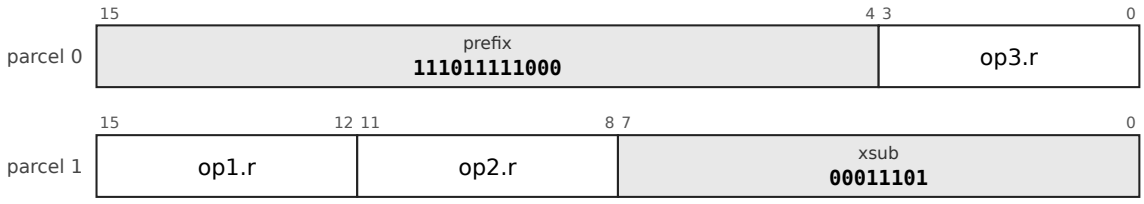
Why it is here. One of the fetch-and-operate atomics (see CAS), in the place of the PDP-11's read-modify-write instructions, which were indivisible on the Unibus through the DATIP/DATO pair (7.1).

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

XADD

XADD: R3, prefix 111011111000, sub-opcode 00011101; operands reg4, reg4, reg4; 4 bytes

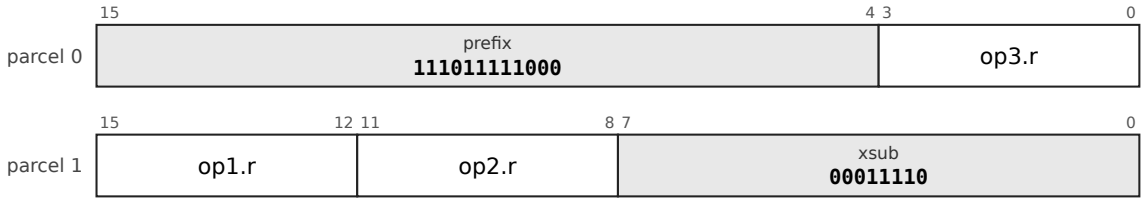



```
XADD value, (address)
```

Operation, flags and exceptions as for the entry XADD above; 1 layout form, each a chart of its own (above).

XADDDB

XADDDB: R3, prefix 111011111000, sub-opcode 00011110; operands reg4, reg4, reg4; 4 byte




```
XADDDB ...
```

Operation, flags and exceptions as for the entry XADD above; 1 layout form, each a chart of its own (above).

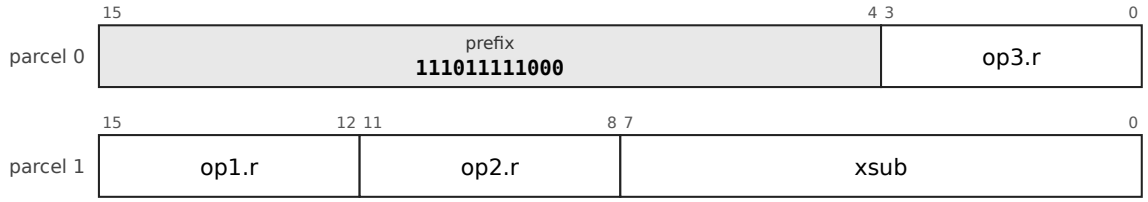
XAND: Atomic fetch and and, or, xor

Mnemonics: XAND, XANDDB, XOR, XOR_DB, XXOR, XXORDB

Syntax

```
XAND value, (address)
XOR_ value, (address)        ; the trailing underscore keeps the mnemonic distinct from XOR
XXOR value, (address)
each with a DB form
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
old ← M[address]
XAND: M[address] ← old AND value
XOR_: M[address] ← old OR value
XXOR: M[address] ← old XOR value
result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for `CAS` and `CASDB`), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group.

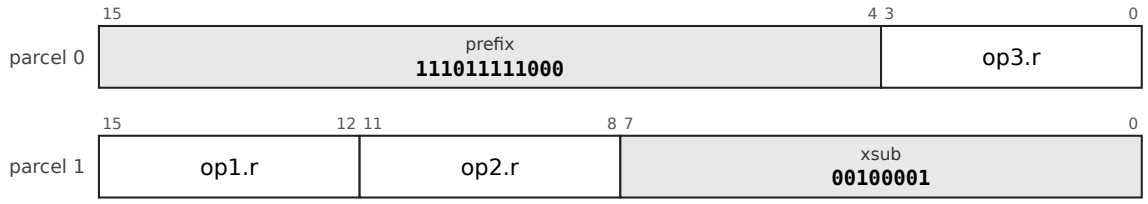
Why it is here. The fetch-and-and, or and xor atomics (see `CAS`). No separate reason is recorded for them.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

XAND

XAND: R3, prefix 111011111000, sub-opcode 00100001; operands reg4, reg4, reg4; 4 bytes

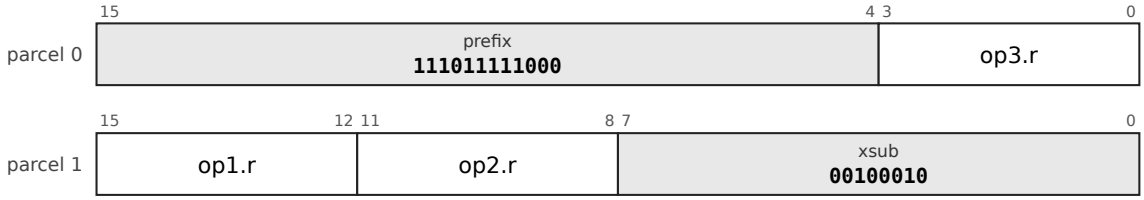



```
XAND value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

XANDDB

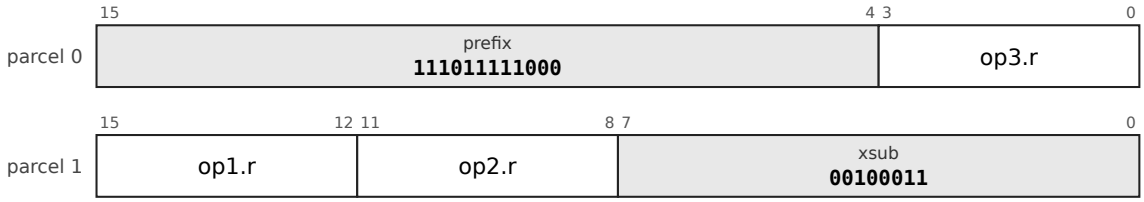
XANDDB: R3, prefix 111011111000, sub-opcode 00100010; operands reg4, reg4, reg4; 4 byte



Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

*XOR_*

XOR_: R3, prefix 111011111000, sub-opcode 00100011; operands reg4, reg4, reg4; 4 bytes

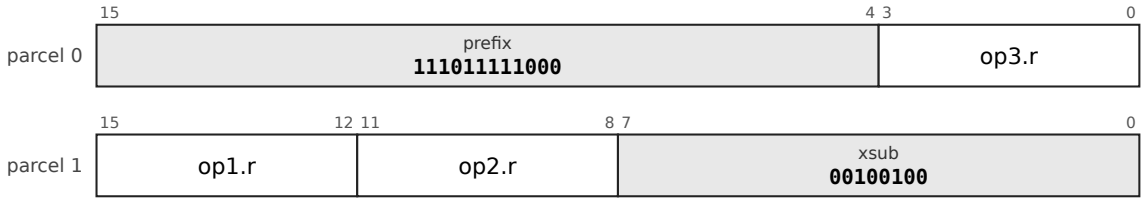



```
XOR_ value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

XOR_DB

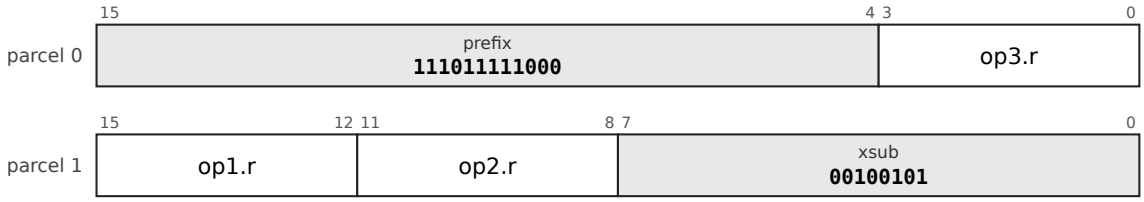
XOR_DB: R3, prefix 111011111000, sub-opcode 00100100; operands reg4, reg4, reg4; 4 byte



Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

XXOR

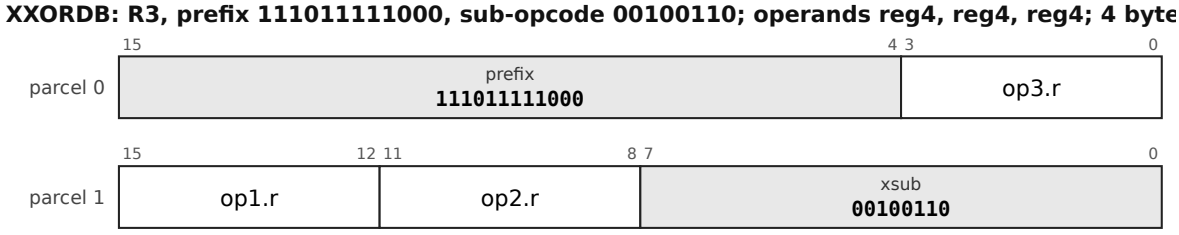
XXOR: R3, prefix 111011111000, sub-opcode 00100101; operands reg4, reg4, reg4; 4 bytes




```
XXOR value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

XXORDB



Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

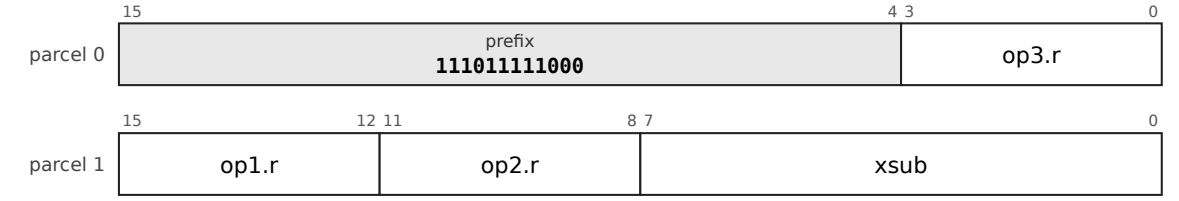
XMAX: Atomic fetch and minimum or maximum

Mnemonics: `XMAX`, `XMAXDB`, `XMIN`, `XMINDB`, `XMAXU`, `XMAXUDB`, `XMINU`, `XMINUDB`

Syntax

```
XMAX value, (address)
XMIN ..., XMAXU ..., XMINU ...
each with a DB form
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
old ← M[address] ; M[address] ← max(old, value) (XMAX, XMAXU) or min (XMIN, XMINU) ; result ← old
signed for XMAX and XMIN, unsigned for the U forms
```

Flags Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for `CAS` and `CASDB`), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group.

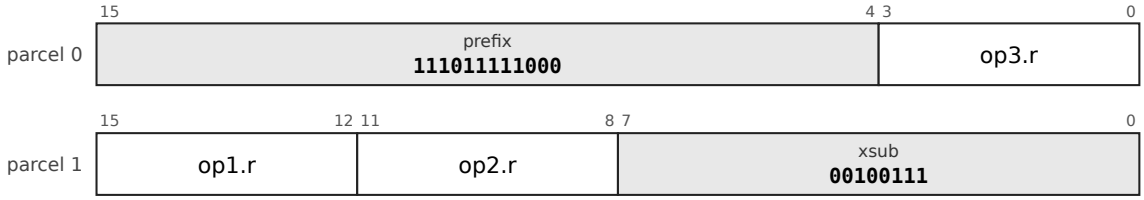
Why it is here. The fetch-and-minimum and maximum atomics, borrowed from LoongArch's richer atomic set: each is a couple of hundred lookup tables and saves a compare-and-swap retry loop.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

XMAX

XMAX: R3, prefix 111011111000, sub-opcode 00100111; operands reg4, reg4, reg4; 4 bytes

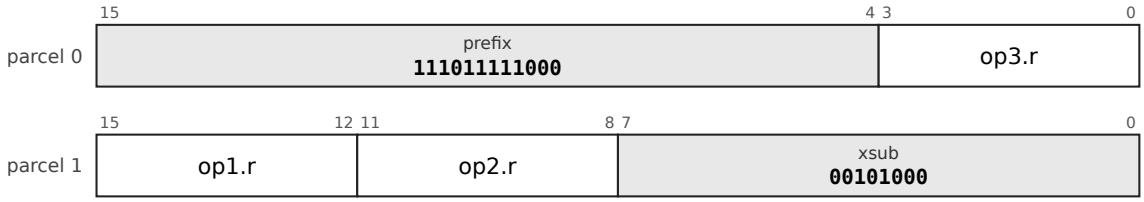



```
XMAX value, (address)
```

Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMAXDB

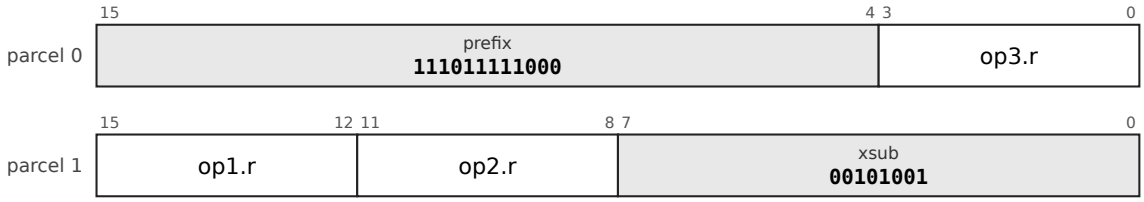
XMAXDB: R3, prefix 111011111000, sub-opcode 00101000; operands reg4, reg4, reg4; 4 byte



Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMIN

XMIN: R3, prefix 111011111000, sub-opcode 00101001; operands reg4, reg4, reg4; 4 bytes

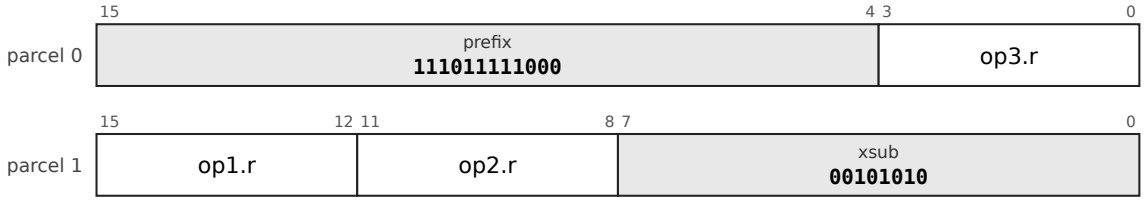



```
XMIN ..., XMAXU ..., XMINU ...
```

Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

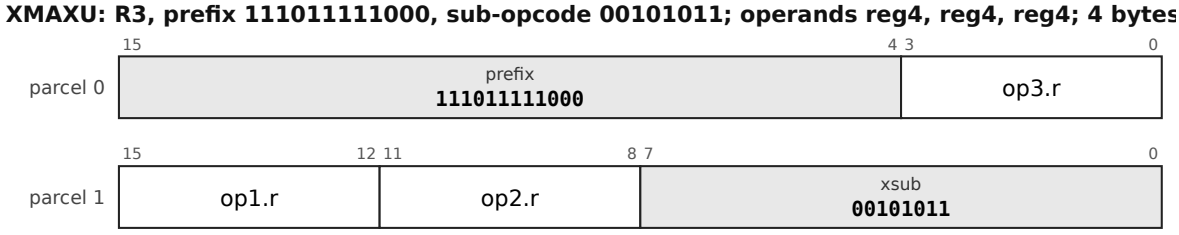
XMINDB

XMINDB: R3, prefix 111011111000, sub-opcode 00101010; operands reg4, reg4, reg4; 4 byte



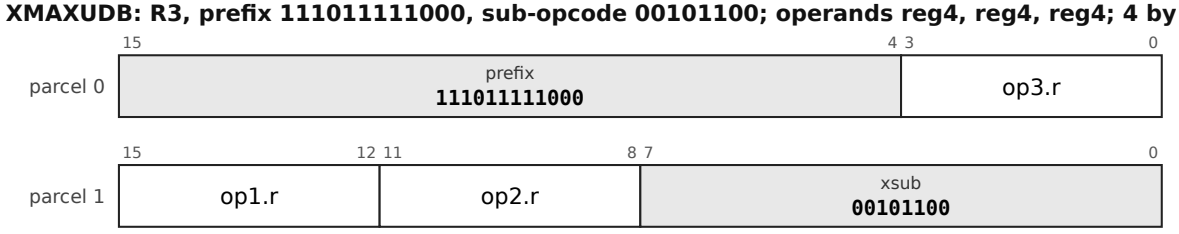
Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMAXU



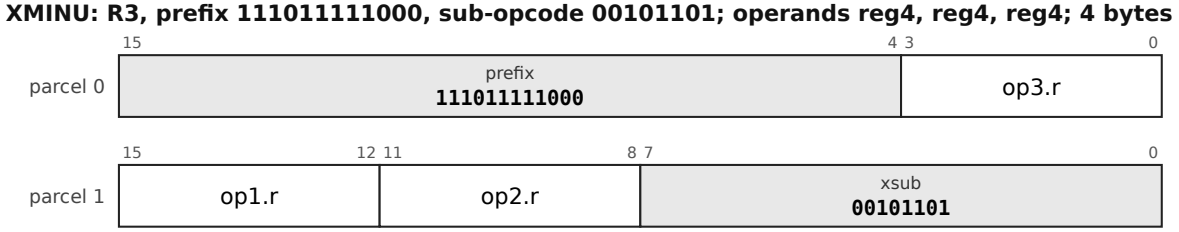
Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMAXUDB



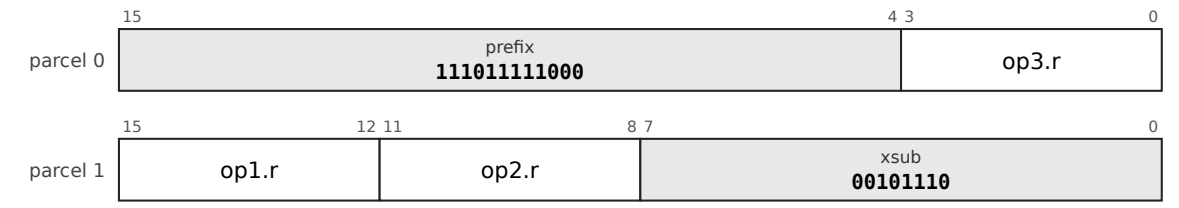
Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMINU



Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

XMINUDB



Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

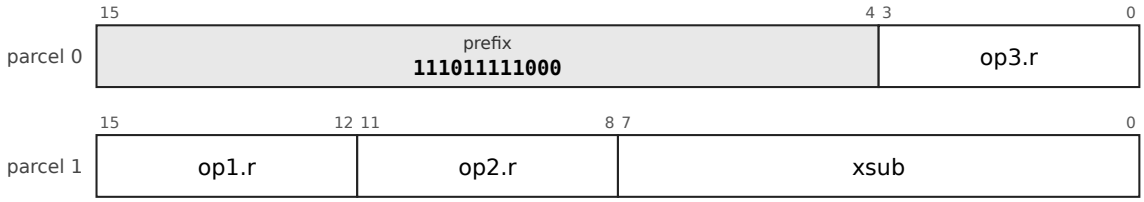
LLSC: Load-linked and store-conditional

Mnemonics: LL, SC

Syntax

```
LL (address), Rd
SC value, (address), result
```

Encoding



The chart is the leaf `16-R3-1`, R3, prefix `111011111000`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `op3` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R3-1`.

Operation

```
LL: Rd ← M[address] ; take a reservation on the granule containing address
SC: if the reservation is intact then M[address] ← value ; result ← 0 (success)
    else nothing is written ; result ← 1 (failure)
a write to the granule by any initiator, core or device, clears the reservation
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

A short LL/SC sequence must eventually succeed if no other initiator keeps writing the granule (the RISC-V guarantee for constrained sequences). The sub-opcodes and the semantics of this group are specified. The ISA text still does not fix where the operands go; the grammars of the prefix-coded encodings **assume** a placement (errata 87, assumption 4: three registers, the address in a register, in the order written; four for CAS and CASDB), so the forms below name the roles and the placement is *(P)*. The reference executor does not implement the group. The result convention shown for SC is the usual one and is unspecified.

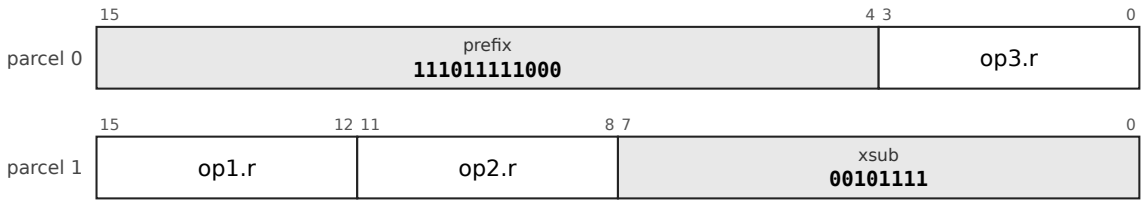
Why it is here. Load-linked and store-conditional, with the reservation a monitor, because a monitor maps onto the exclusive access of AXI and of FPGA and modern memory systems, where the Unibus's locked read-modify-write (DATIP/DATO) does not scale as cores multiply. A short sequence must eventually succeed if no other initiator keeps writing the granule, the RISC-V rule.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

Status specified; **not yet implemented** in the reference executor.

LL

LL: R3, prefix 111011111000, sub-opcode 00101111; operands reg4, reg4, reg4; 4 bytes

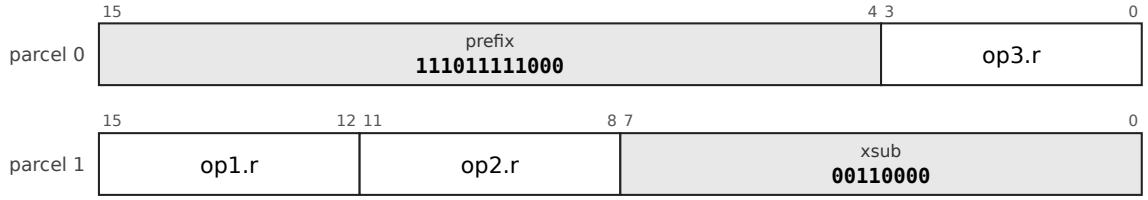



```
LL (address), Rd
```

Operation, flags and exceptions as for the entry LLSC above; 1 layout form, each a chart of its own (above).

SC

SC: R3, prefix 111011111000, sub-opcode 00110000; operands reg4, reg4, reg4; 4 bytes




```
SC value, (address), result
```

Operation, flags and exceptions as for the entry LLSC above; 1 layout form, each a chart of its own (above).

9.14 System instructions, exceptions and interrupts

Cf. chapter 6, section 6.7 (system instructions, exceptions and interrupts), section 6.2 (exceptions and interrupts) and section 6.3 (system registers).

9.14.1 The system group SYS holds the privileged and system instructions: TRAP, TRAPL, RTI, RTT, RET, HALT, BPT, MTPS, MFPS, SPL, MFSR, MTSR, MTDP, MFDP, MFCPUID, PROBER, PROBEW, PUSHX, POPX, and the translation and cache maintenance instructions (TLBI, TLBIA, TLBSYNC, MTASID, MFASID, ICINV, ICINVA). The user-memory guard instructions STUA and CLUA are described with them.

Commentary. The official entry of each instruction is below, in this section; the kernel chapter gives the modes, the entry sequence, the system registers and the reasons. **Impact.** *Kernel:* the entries below are the complete list of what the kernel may execute that user code may not, and of what both may; chapter 6 explains how they are used. *Compiler:* none, apart from TRAP in system-call stubs. *Hardware:* a class decode and one privilege check.

NOP: No operation and landing pad

Mnemonics: NOP, LPAD

Syntax

```
NOP
LPAD
```

Encoding



The chart is the leaf 16-SYS0-1, SYS0, prefix 11101100100. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): 16-SYS0-1.

Operation

```
nothing
```

Flags None written.

Exceptions None.

LPAD marks a legal target of an indirect jump or call; no instruction enforces it in ABI version 1.

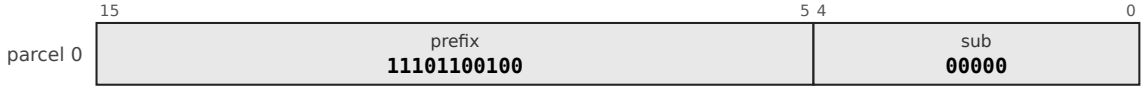
Why it is here. `NOP` is common and has no story; `LPAD` is the landing pad. Landing pads mark the legal targets of indirect jumps and calls so that jump-oriented attacks can be stopped at the transfer, as Intel's indirect-branch tracking and ARM's branch target identification do and OpenBSD enforces. ABI version 1 leaves them off (an open item of the instruction set, 9.21), and the assembler provides `.lpad`; the kernel pain-points and mitigations reports record why deciding later is more expensive (appendix A, section A.1).

Cf. Appendix A, section A.1 (landing pads).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

NOP

NOP: SYS0, prefix 11101100100, sub-opcode 00000; operands no operand; 2 bytes




```
NOP
```

Operation, flags and exceptions as for the entry NOP above; 1 layout form, each a chart of its own (above).

LPAD

LPAD: SYS0, prefix 11101100100, sub-opcode 01001; operands no operand; 2 bytes




```
LPAD
```

Operation, flags and exceptions as for the entry NOP above; 1 layout form, each a chart of its own (above).

RTI: Return from interrupt or exception

Mnemonics: `RTI`, `RTT`

Syntax

```
RTI
RTT
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
PC ← EPC ; PSW ← EPSW          ; a PSW with the reserved mode 2 in either mode field aborts
RTT: and then set the trace bit T after the return
```

Flags Replaced by EPSW, including the flags.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

RTT sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

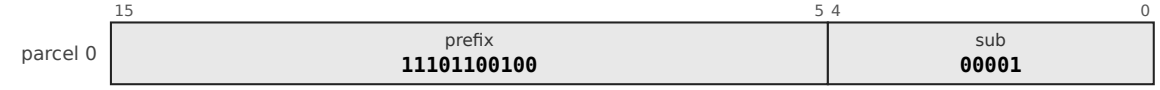
Why it is here. RTT looks redundant against RTI, and stays: it sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

Cf. 6.2 (exceptions and interrupts), 6.7.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

RTI

RTI: SYS0, prefix 11101100100, sub-opcode 00001; operands no operand; 2 bytes




```
RTI
```

Operation, flags and exceptions as for the entry RTI above; 1 layout form, each a chart of its own (above).

RTT

RTT: SYS0, prefix 11101100100, sub-opcode 00010; operands no operand; 2 bytes




```
RTT
```

Operation, flags and exceptions as for the entry RTI above; 1 layout form, each a chart of its own (above).

RET: Return from subroutine

RET: SYS0, prefix 11101100100, sub-opcode 00011; operands no operand; 2 bytes

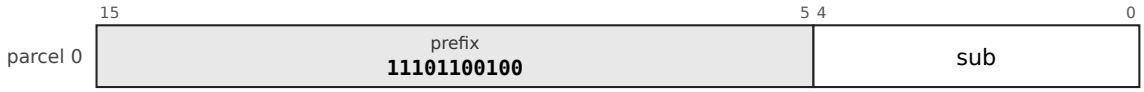


Mnemonics: RET

Syntax

```
RET
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
PC ← M[SP] ; SP ← SP + 8
```

Flags None written.

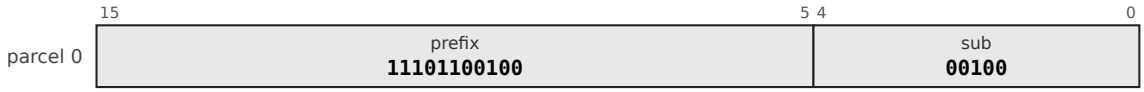
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Not privileged: it is the partner of `CALL`.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

HALT: Halt the processor

HALT: SYS0, prefix 11101100100, sub-opcode 00100; operands no operand; 2 bytes

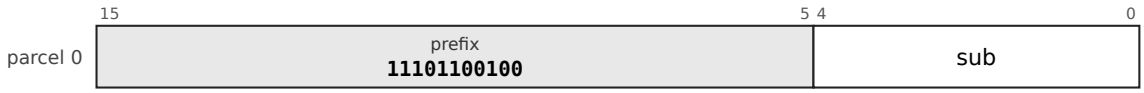


Mnemonics: `HALT`

Syntax

```
HALT
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
stop the processor
```

Flags None written.

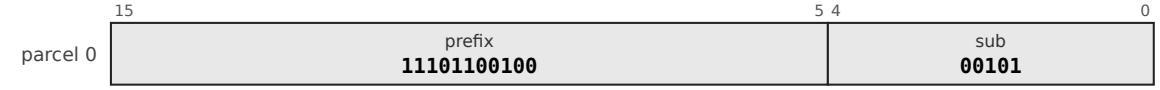
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Cf. 6.6.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

BPT: Breakpoint

BPT: SYS0, prefix 11101100100, sub-opcode 00101; operands no operand; 2 bytes

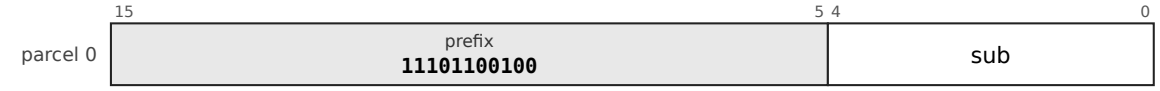


Mnemonics: `BPT`

Syntax

```
BPT
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
take the breakpoint vector with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel by the `BPT` vector.

A trap like any other, but kept as its own two-byte instruction because a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has.

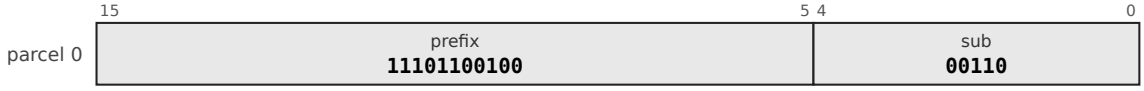
Why it is here. `BPT` is a trap vector, but a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has; that is why it is kept although `IOT` and `EMT` are not.

Cf. 6.2.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

FENCE: Memory fence

FENCE: SYS0, prefix 11101100100, sub-opcode 00110; operands no operand; 2 bytes



Mnemonics: `FENCE`

Syntax

```
FENCE
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
order the memory accesses before it against those after it, for every initiator that observes them
```

Flags None written.

Exceptions None.

Owed to devices (one `FENCE` separates writing a device's registers from setting its go bit) and to other cores.

Why it is here. The memory model is sequentially consistent, so `FENCE` is needed only for what is outside the model: devices and the framebuffer (one `FENCE` between writing a device's registers and setting its go bit, section 3.4) and instruction streams. The `_DB` barrier forms of the atomics exist so that a `FENCE` is not needed in a lock acquire.

Cf. 7.2 (the memory model), 2.2 (what the processor owes the bus).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

WFE: Wait for an event

WFE: SYS0, prefix 11101100100, sub-opcode 00111; operands no operand; 2 bytes



Mnemonics: `WFE` (aliases: `WAIT`)

Syntax

```
WFE
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
sleep until an interrupt is pending or another core executes SEV
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

`WAIT` is an assembler alias. Whether `WFE` wakes on a pending but masked interrupt (the idle-loop question) is open.

Why it is here. `WAIT` was a PDP-11 instruction (sleep until an interrupt); `WFE` subsumes it: it sleeps until an interrupt or another core's `SEV`, and on a uniprocessor it behaves exactly as `WAIT` did. `WFE` and `SEV` belong to the multiprocessor operations (7.6). The wake rule (whether a pending but masked interrupt wakes it) is open item P1.

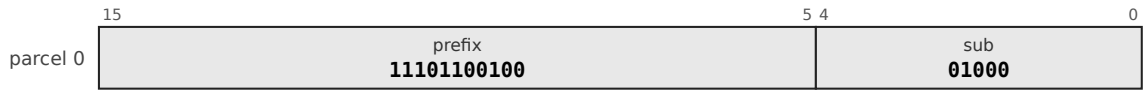
Cf. 7.6 (waiting and signalling).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

`WAIT` is an alias of `WFE`: the assembler writes it as that word.

SEV: Send an event

SEV: SYS0, prefix 11101100100, sub-opcode 01000; operands no operand; 2 bytes

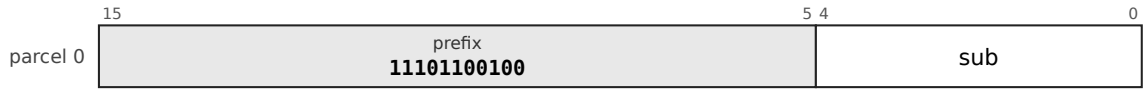


Mnemonics: `SEV`

Syntax

```
SEV
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
wake every core waiting in WFE
```

Flags None written.

Exceptions None.

Why it is here. The signalling half of `WFE`, so that a core waiting for a lock can sleep instead of spinning and be woken by the core that releases it.

Cf. 7.6.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

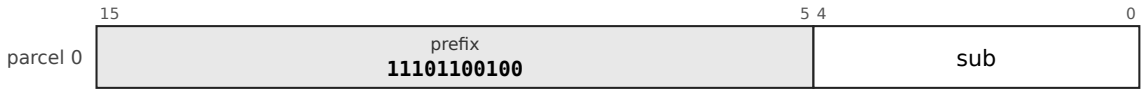
TLBA: Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache

Mnemonics: `TLBIA`, `TLBSYNC`, `ICINVA`

Syntax

```
TLBIA
TLBSYNC
ICINVA
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
TLBIA   : invalidate every translation-buffer entry on this core and, by broadcast, on the others
TLBSYNC : wait until the invalidations this core started have completed on every core
ICINVA  : invalidate the whole instruction cache
```

Flags None written.

Exceptions Privileged (`TLBIA`, `TLBSYNC`); the reference executor treats `ICINVA` as a no-op.

Why it is here. `TLBI` and `TLBIA` broadcast, `TLBSYNC` waits for the other cores, and `ICINV` and `ICINVA` invalidate the instruction cache: a Lisp runtime that compiles a function cannot work without an instruction-cache invalidate.

Cf. 7.5 (translation and instruction-cache coherence).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

TLBIA

TLBIA: SYS0, prefix 11101100100, sub-opcode 01010; operands no operand; 2 bytes

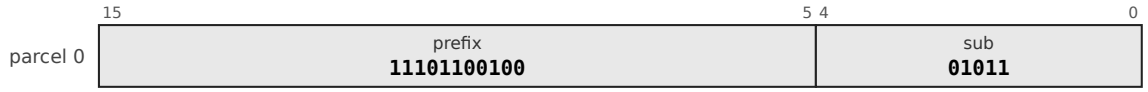



```
TLBIA
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

TLBSYNC

TLBSYNC: SYS0, prefix 11101100100, sub-opcode 01011; operands no operand; 2 bytes




```
TLBSYNC
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

ICINVA

ICINVA: SYS0, prefix 11101100100, sub-opcode 01100; operands no operand; 2 bytes




```
ICINVA
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

PUSHX: Push and pop the exception state

Mnemonics: `PUSHX`, `POPX`

Syntax

```
PUSHX
POPX
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
PUSHX: SP ← SP - 16 ; M[SP] ← EPSW ; M[SP + 8] ← EPC
POPX : EPSW ← M[SP] ; EPC ← M[SP + 8] ; SP ← SP + 16
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. `SP` must be 8-byte aligned (a misaligned-access abort); both words are checked before either is touched.

For a handler that wants to be interrupted: it can save the exception state and lower the priority.

Why it is here. A handler that can be interrupted must save `EPC` and `EPSW` before it lowers the priority; with only `MFSR` that costs two moves and two stores (sixteen bytes) on every such kernel entry, and the same to restore. The instructions may not show up high in the instruction count, but they save bytes and cycles in a tight, very often used kernel loop.

Cf. *6.2*, *6.7*.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

PUSHX

PUSHX: SYS0, prefix 11101100100, sub-opcode 01101; operands no operand; 2 bytes

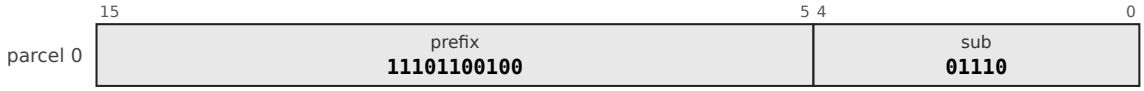



```
PUSHX
```

Operation, flags and exceptions as for the entry PUSHX above; 1 layout form, each a chart of its own (above).

POPX

POPX: SYS0, prefix 11101100100, sub-opcode 01110; operands no operand; 2 bytes




```
POPX
```

Operation, flags and exceptions as for the entry PUSHX above; 1 layout form, each a chart of its own (above).

ISYNC: Instruction synchronisation

ISYNC: SYS0, prefix 11101100100, sub-opcode 01111; operands no operand; 2 bytes



Mnemonics: `ISYNC`

Syntax

```
ISYNC
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
make earlier writes to PTBR, ASID, MMUCTL and VBR, and earlier ICINVs, take effect for the instructions that
follow
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A non-zero pad byte is an illegal instruction.

The architecture promises the effect only after the next `ISYNC`; an in-order implementation may make it immediate, and software must not rely on that.

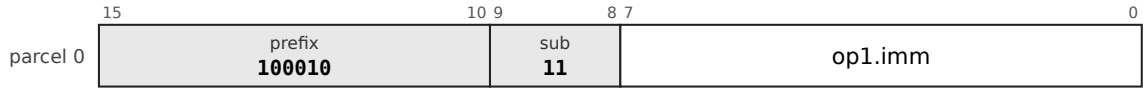
Why it is here. Taken from the out-of-order readiness review: it makes earlier writes to `PTBR`, `ASID`, `MMUCTL` and `VBR` take effect for what follows, so an in-order machine may make them immediate and a speculative one has a place to serialise. It serves the aim that the architecture is efficient on both in-order and out-of-order cores (1.4.4).

Cf. 7.5, 6.3.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

HINT: Hint

HINT: SYSI, prefix 100010, sub-opcode 11; operands imm8u; 2 bytes

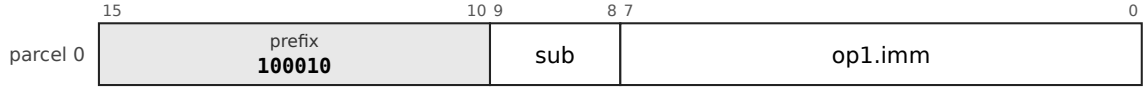


Mnemonics: `HINT`

Syntax

```
HINT #n        ; n = 0 to 255
```

Encoding



The chart is the leaf `16-SYSI-1`, SYSI, prefix `100010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSI-1`.

Operation

```
no architectural effect
```

Flags None written.

Exceptions None.

Defined codes: 0 a plain no-op hint, 1 pause (spin-wait), 2 speculation barrier. Every other code is a reserved hint that executes as a no-op; software shall not use it. Not privileged.

Why it is here. Defined with ISYNC: a no-op with a byte of code, defined for `0` (plain), `1` (pause, for spin-wait loops) and `2` (speculation barrier), every other code a reserved hint that executes as a no-op so that a later machine can give it a meaning without breaking an old binary.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

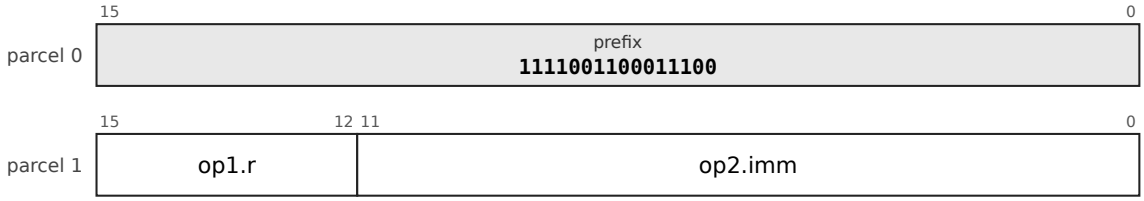
MFSR: Move from and to a system register

Mnemonics: `MFSR`, `MTSR`

Syntax

```
MFSR Rd, name
MTSR name, Rs
```

Encoding



The chart is the leaf `16-MFSR-1`, MFSR, prefix `1111001100011100`. `op1` is `reg4`: register, 4 bits. `op2` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-MFSR-1`, `16-MTSR-1`.

Operation

```
MFSR: Rd ← system register `name`
MTSR: system register `name` ← Rs        ; read-only registers are an illegal instruction
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. An unassigned system register is an illegal instruction.

The register is chosen by name. A write to PTBR, ASID or MMUCTL flushes the translation buffer.

Name	Number	Access
PTBR	0	read and write
EPC	1	read and write
EVA	2	read only
ECAUSE	3	read only
DMW0	4	read and write
DMW1	5	read and write
USP	6	read and write
SSP	7	read and write
EPSW	8	read and write

Name	Number	Access
VBR	9	read and write
MMUCTL	10	read and write
ASID	11	read and write
CPUID	12	read only

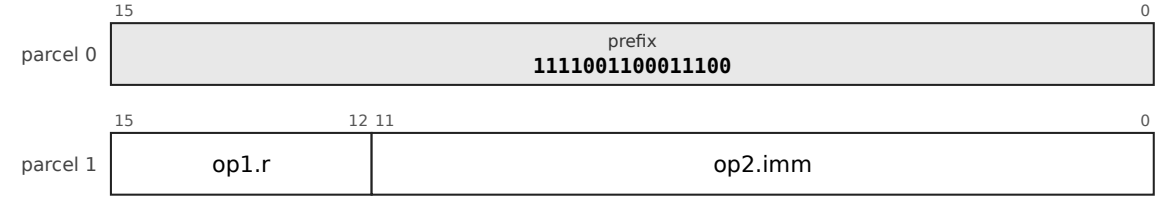
Why it is here. A paging operating system must install a page table and learn why it was entered, which needs instructions to write PTBR and read EPC, EVA and ECAUSE. One general pair reaches all of the system registers by number. The user and supervisor stack pointers are in the set because any kernel must read and write the user stack pointer to build a signal frame, copy a process or switch context.

Cf. 6.3 (system registers).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MFSR

MFSR: MFSR, prefix 1111001100011100; operands reg4, imm12u; 4 bytes

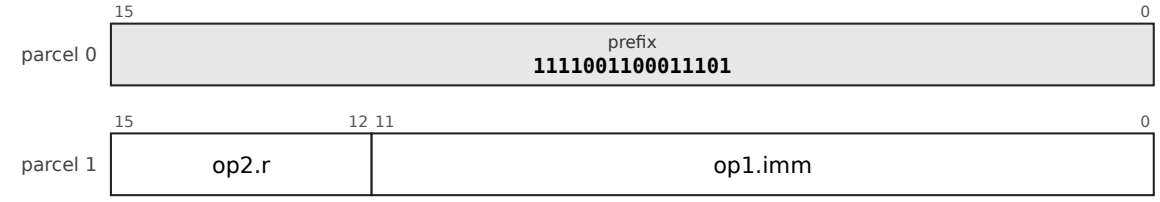



```
MFSR Rd, name
```

Operation, flags and exceptions as for the entry MFSR above; 1 layout form, each a chart of its own (above).

MTSR

MTSR: MTSR, prefix 1111001100011101; operands imm12u, reg4; 4 bytes




```
MTSR name, Rs
```

Operation, flags and exceptions as for the entry MFSR above; 1 layout form, each a chart of its own (above).

MTPS: Move to and from the processor status word

Mnemonics: MTPS, MFPS

Syntax

```
MTPS Rs
MFPS Rd
```

Encoding



The chart is the leaf `16-SYSR-1`, SYSR, prefix `110110100`. `op1` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-1`.

Operation

```
MTPS: PSW ← Rs          ; in user mode only the flags N Z V C are written
MFPS: Rd ← PSW
```

Flags `MTPS` writes the flags from `Rs`; `MFPS` writes none.

Exceptions A `MTPS` value with the reserved mode 2 in a mode field aborts.

Cf. 6.1 (modes and the status word).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MTPS

MTPS: SYSR, prefix 110110100, sub-opcode 000; operands reg4; 2 bytes




```
MTPS Rs
```

Operation, flags and exceptions as for the entry MTPS above; 1 layout form, each a chart of its own (above).

MFPS

MFPS: SYSR, prefix 110110100, sub-opcode 001; operands reg4; 2 bytes

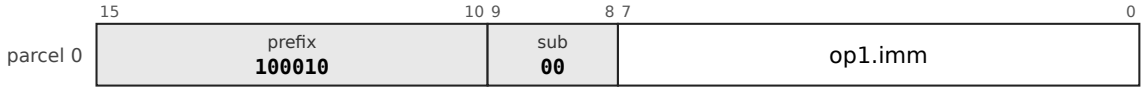



```
MFPS Rd
```

Operation, flags and exceptions as for the entry MTPS above; 1 layout form, each a chart of its own (above).

SPL: Set the interrupt priority level

SPL: SYSI, prefix 100010, sub-opcode 00; operands imm8u; 2 bytes

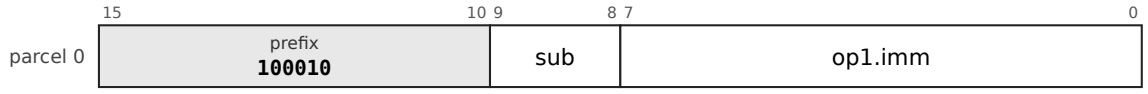


Mnemonics: `SPL`

Syntax

```
SPL #n        ; n = 0 to 7
```

Encoding



The chart is the leaf `16-SYSI-1`, SYSI, prefix `100010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSI-1`.

Operation

```
PSW.priority ← n
```

Flags None written. The condition flags are untouched.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A priority above 7 is an illegal instruction.

Kept despite `MTPS` writing the whole word because it sets the priority alone: a kernel raising priority around a critical section must not clobber the flags.

Why it is here. `SPL` stays although `MTPS` writes the whole status word: it sets the priority alone, leaving the condition codes untouched, so a kernel raising priority around a critical section does not clobber flags. It is the PDP-11 instruction of the same name.

Cf. 6.2.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

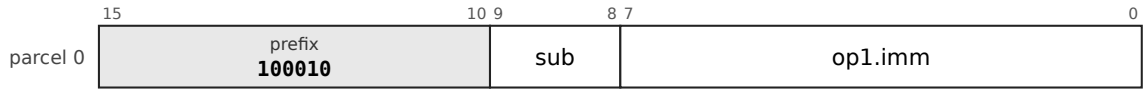
TRAP: System call and trap

Mnemonics: `TRAP`, `TRAPL`

Syntax

```
TRAP #n        ; n = 0 to 15, two bytes
TRAPL #n       ; n = 0 to 255, four bytes
```

Encoding



The chart is the leaf `16-SYSI-1`, SYSI, prefix `100010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSI-1`.

Operation

```
enter the kernel by vector (TRAP_BASE + n), with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel; not privileged.

The replacement for the PDP-11 `IOT` and `EMT`. In these encodings `TRAP` and `TRAPL` are one word with an 8-bit vector number (`docs/encoding-proposal.md`).

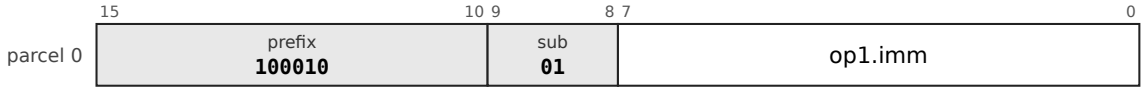
Why it is here. In the original encoding `TRAPL` existed so that the vector range of 256 and the two-byte common case could both be kept (`TRAP #n` for `n` below 16 is two bytes, and the long form reaches 256); here one word carries an 8-bit vector number. `IOT` and `EMT`, which were `TRAP` at two particular vectors on the PDP-11, are not provided.

Cf. 6.2, 6.5 (system calls).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

TRAP

TRAP: SYSI, prefix 100010, sub-opcode 01; operands imm8u; 2 bytes

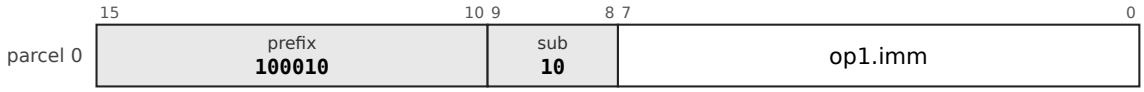



```
TRAP #n
```

Operation, flags and exceptions as for the entry TRAP above; 1 layout form, each a chart of its own (above).

TRAPL

TRAPL: SYSI, prefix 100010, sub-opcode 10; operands imm8u; 2 bytes

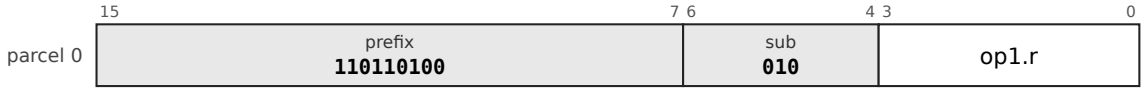



```
TRAPL #n
```

Operation, flags and exceptions as for the entry TRAP above; 1 layout form, each a chart of its own (above).

TLBI: Invalidate a translation

TLBI: SYSR, prefix 110110100, sub-opcode 010; operands reg4; 2 bytes

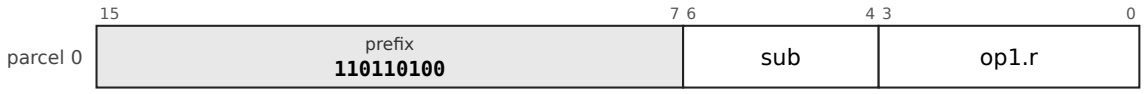


Mnemonics: `TLBI`

Syntax

```
TLBI Rs
```

Encoding



The chart is the leaf `16-SYSR-1`, SYSR, prefix `110110100`. `op1` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-1`.

Operation

```
invalidate the translation-buffer entry for the page containing the address in Rs, on this core and by
broadcast on the others
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Why it is here. It broadcasts: a translation changed by one core must be invalidated on all of them, and `TLBSYNC` waits for it.

Cf. 7.5, 6.4.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

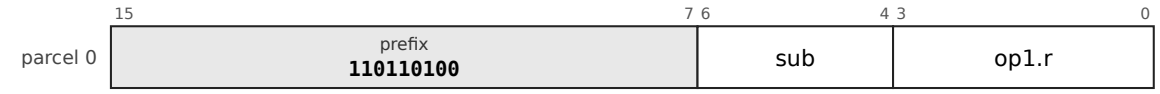
MTASID: Move to and from the address-space identifier

Mnemonics: `MTASID`, `MFASID`

Syntax

```
MTASID Rs
MFASID Rd
```

Encoding



The chart is the leaf `16-SYSR-1`, SYSR, prefix `110110100`. `op1` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-1`.

Operation

```
MTASID: ASID ← Rs[15:0]          ; the translation buffer is flushed in the reference executor
MFASID: Rd ← ASID
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The `ASID` tags translation-buffer entries so a context switch is one store.

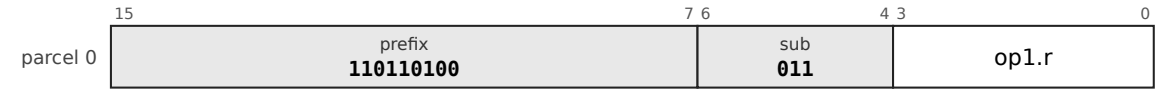
Why it is here. The `ASID` tags translation-buffer entries so a context switch is one store and no flush. No further reason is recorded.

Cf. 6.4.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MTASID

MTASID: SYSR, prefix 110110100, sub-opcode 011; operands reg4; 2 bytes




```
MTASID Rs
```

Operation, flags and exceptions as for the entry MTASID above; 1 layout form, each a chart of its own (above).

MFASID

MFASID: SYSR, prefix 110110100, sub-opcode 100; operands reg4; 2 bytes




```
MFASID Rd
```

Operation, flags and exceptions as for the entry MTASID above; 1 layout form, each a chart of its own (above).

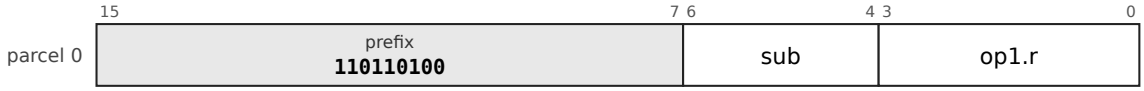
MTDP: Move to and from the direct pointer

Mnemonics: `MTDP`, `MFDP`

Syntax

```
MTDP Rs
MFDP Rd
```

Encoding



The chart is the leaf `16-SYSR-1`, SYSR, prefix `110110100`. `op1` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-1`.

Operation

```
MTDP: DP ← Rs
MFDP: Rd ← DP
```

Flags None written.

Exceptions None.

`DP` is the thread pointer in user mode and the per-CPU pointer in kernel mode; there is one copy per mode.

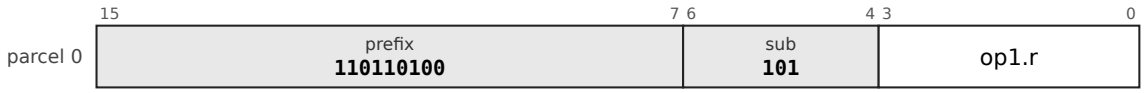
Why it is here. `DP` descends from the 6502's zero page and is kept for code density (a short displacement from `DP`); it is also the thread pointer in user mode and, in kernel mode, the per-CPU pointer that an SMP kernel wants (7.4). `MTDP` and `MFDP` read and write the current mode's copy.

Cf. 7.4 (per-CPU state).

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MTDP

MTDP: SYSR, prefix 110110100, sub-opcode 101; operands reg4; 2 bytes




```
MTDP Rs
```

Operation, flags and exceptions as for the entry MTDP above; 1 layout form, each a chart of its own (above).

MFDP

MFDP: SYSR, prefix 110110100, sub-opcode 110; operands reg4; 2 bytes

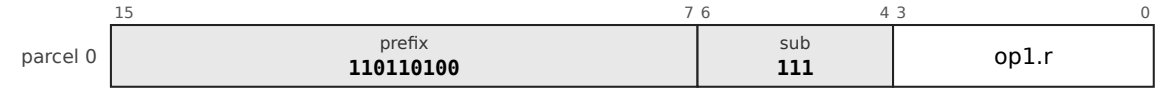



```
MFDP Rd
```

Operation, flags and exceptions as for the entry MTDP above; 1 layout form, each a chart of its own (above).

MTDMW: Move to a direct-mapping window

MTDMW: SYSR, prefix 110110100, sub-opcode 111; operands reg4; 2 bytes



Mnemonics: `MTDMW`

Syntax

```
MTDMW Rs
```

Encoding



The chart is the leaf `16-SYSR-1`, SYSR, prefix `110110100`. `op1` is `reg4`: register, 4 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-1`.

Operation

```
DMW0 ← Rs          ; storage only: the operand names no window and the format of the windows is unspecified
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The direct-mapping windows have no defined format.

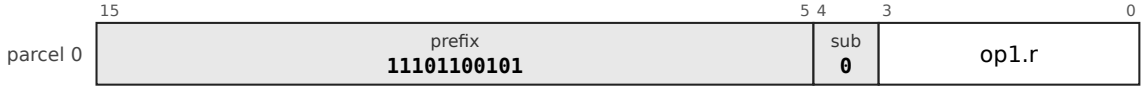
Why it is here. Taken from LoongArch's direct-mapping windows, which stop the kernel taking translation-buffer misses on its own text. The windows are deferred: the instruction is storage only.

Cf. 6.4.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

MFCPUID: Read the processor identifier

MFCPUID: SYSR, prefix 11101100101, sub-opcode 0; operands reg4; 2 bytes



Mnemonics: `MFCPUID`

Syntax

```
MFCPUID Rd
```

Encoding



The chart is the leaf `16-SYSR-2`, SYSR, prefix `11101100101`. `op1` is `reg4`: register, 4 bits. `sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-2`.

Operation

```
Rd ← CPUID
```

Flags None written.

Exceptions None.

The index of this core, for per-CPU data.

Why it is here. Part of the multiprocessor operations, because per-CPU data needs each core to know which one it is.

Cf. 7.4, 7.8.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

ICINV: Invalidate the instruction cache for an address

ICINV: SYSR, prefix 11101100101, sub-opcode 1; operands reg4; 2 bytes

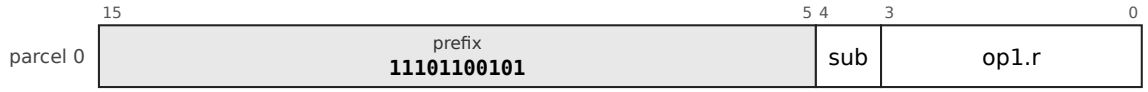


Mnemonics: `ICINV`

Syntax

```
ICINV Rs
```

Encoding



The chart is the leaf `16-SYSR-2`, SYSR, prefix `11101100101`. `op1` is `reg4`: register, 4 bits. `sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYSR-2`.

Operation

```
invalidate the instruction-cache line containing the address in Rs
```

Flags None written.

Exceptions The reference executor treats it as a no-op; privilege as for the other cache-maintenance instructions.

Why it is here. Instruction-cache invalidation, by address (`ICINV`) and whole (`ICINVA`). The reason is the one given for `TLBA`: a runtime that compiles a function cannot work without it.

Cf. 7.5.

Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

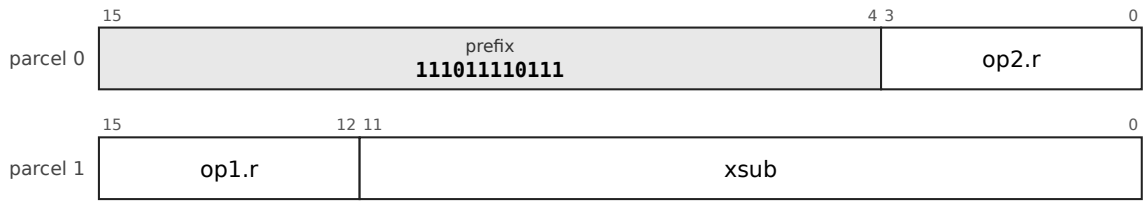
PROBE: Probe an address for read or write access

Mnemonics: `PROBER`, `PROBEW`

Syntax

```
PROBER Rd, Rs
PROBEW Rd, Rs
```

Encoding



The chart is the leaf `16-R2-1`, R2, prefix `111011110111`. `op1` is `reg4`: register, 4 bits. `op2` is `reg4`: register, 4 bits. `xsub` selects the mnemonic.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-R2-1`.

Operation

```
Rd ← 1 if the address in Rs is readable (PROBER) or writable (PROBEW) in the previous mode, else 0
no fault is taken
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

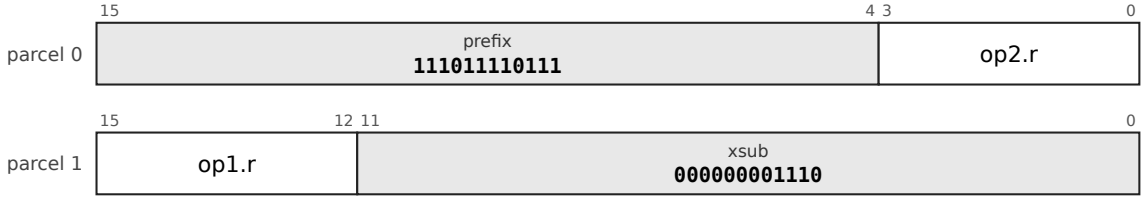
Taken from the VAX `PROBER` and `PROBEW`: the kernel validates a user address before using it.

Why it is here. Taken from the VAX `PROBER` and `PROBEW`: the kernel validates a user address before using it, and it replaces the `nofault` global of the V7 kernel for the address-validation case.

Cf. 6.5 (the user-memory guard), 6.4.
Status implemented in the reference executor (`common/pdp64exec.c` and `common/pdp64exec_enc.inc`, run by `sim64`; QEMU does not yet execute this encoding).

PROBER

PROBER: R2, prefix 111011110111, sub-opcode 000000001110; operands reg4, reg4; 4 bytes

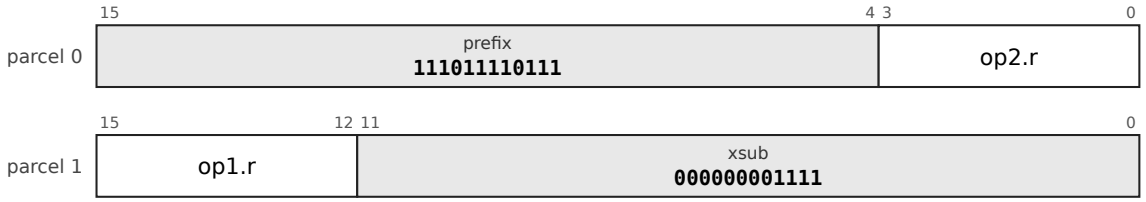



```
PROBER Rd, Rs
```

Operation, flags and exceptions as for the entry PROBE above; 1 layout form, each a chart of its own (above).

PROBEW

PROBEW: R2, prefix 111011110111, sub-opcode 000000001111; operands reg4, reg4; 4 bytes




```
PROBEW Rd, Rs
```

Operation, flags and exceptions as for the entry PROBE above; 1 layout form, each a chart of its own (above).

TRAPCC: Trap on condition

Mnemonics: `TRAPCC.NE`, `TRAPCC.EQ`, `TRAPCC.PL`, `TRAPCC.MI`, `TRAPCC.VC`, `TRAPCC.VS`, `TRAPCC.CC`, `TRAPCC.CS`, `TRAPCC.GE`, `TRAPCC.LT`, `TRAPCC.GT`, `TRAPCC.LE`, `TRAPCC.HI`, `TRAPCC.LOS`

Syntax

```
TRAPCC.cc         ; one word for each of the 14 conditions: TRAPCC.EQ, TRAPCC.VS, TRAPCC.HI ...
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.
All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`.

Operation

```
if condition cc holds on the flags then take the CHECK trap
    EPC ← address of the next instruction ; EVA ← address of this instruction
```

Flags None written.

Exceptions The CHECK trap, when the condition holds. A *trap*, not an abort: the instruction completes, EPC is the next instruction, and RTI resumes after the check.

(D) Errata 85 and 86: TRAPCC is a trap and not an abort, there is no compare-and-trap form, and the fused ADDV, SUBV and MULV carry the register-only cases. There is one word for each of the 14 conditions, in two bytes with no operand: the condition numbers are those of the branches (ne 1 ... los 14; al is not a word). After ADD the check for signed overflow is TRAPCC.VS (docs/study-proposals.md item 9 calls it ADD; TRAPV). EVA is the address of the TRAPCC itself, the one trap that defines it, so a handler need not decode backwards from EPC. The check vector is number 16 (CHECK) in docs/study-proposals.md item 9 and in the reference executor; spec/pdp64-system.hbnf has no vector of that name yet, so the number is *(P)*. The reference executor takes a trap that fires outside the kernel as the end of the program.

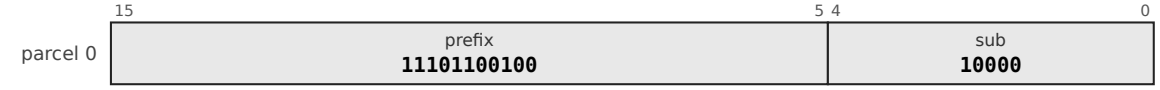
Why it is here. A check that costs two bytes and no branch. The comparison with the alternatives is in the study (docs/study-proposals.md item 9): ADD #1, 8(R2) and a check is four plus two bytes and the add keeps its immediate and memory forms, where ADD and a BVS to a shared stub is six bytes and two instructions; and the same instruction checks what is not overflow (a bounds check after CMP, unsigned overflow after ADD). The OpenBSD kernels of i386, hppa and sparc64 already deliver integer-overflow traps as SIGFPE (FPE_INTOVF), so the signal route exists. The author made it a trap and not an abort (errata 85 and 86), so a handler that promotes a fixnum to a bignum can repair the register and resume; the one case where a trap defines EVA.

Cf. 6.2 (traps and aborts), docs/study-proposals.md item 9.

Status implemented in the reference executor (common/pdp64exec.c and common/pdp64exec_enc.inc, run by sim64; QEMU does not yet execute this encoding).

TRAPCC.NE

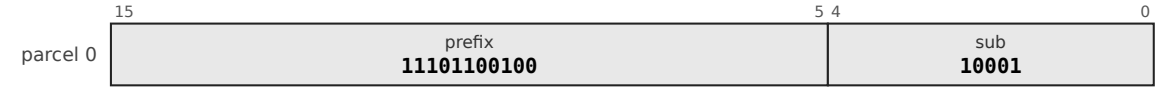
TRAPCC.NE: SYS0, prefix 11101100100, sub-opcode 10000; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.EQ

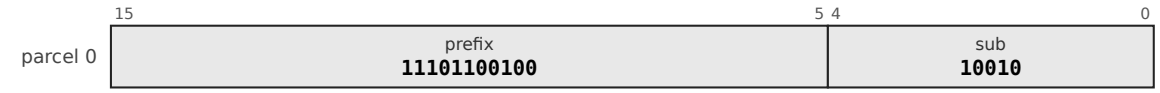
TRAPCC.EQ: SYS0, prefix 11101100100, sub-opcode 10001; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.PL

TRAPCC.PL: SYS0, prefix 11101100100, sub-opcode 10010; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.MI

TRAPCC.MI: SYS0, prefix 11101100100, sub-opcode 10011; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.VC

TRAPCC.VC: SYS0, prefix 11101100100, sub-opcode 10100; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.VS

TRAPCC.VS: SYS0, prefix 11101100100, sub-opcode 10101; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.CC

TRAPCC.CC: SYS0, prefix 11101100100, sub-opcode 10110; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.CS

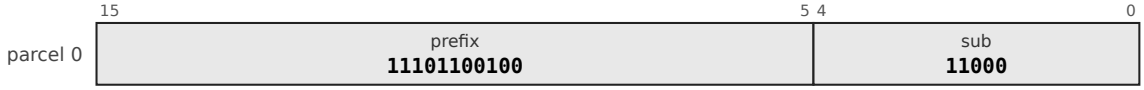
TRAPCC.CS: SYS0, prefix 11101100100, sub-opcode 10111; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.GE

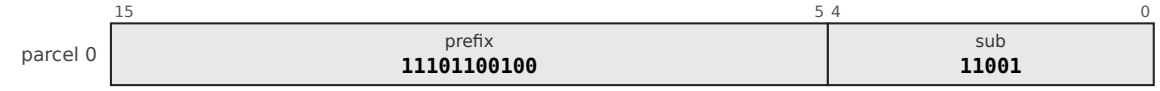
TRAPCC.GE: SYS0, prefix 11101100100, sub-opcode 11000; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.LT

TRAPCC.LT: SYS0, prefix 11101100100, sub-opcode 11001; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.GT

TRAPCC.GT: SYS0, prefix 11101100100, sub-opcode 11010; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.LE

TRAPCC.LE: SYS0, prefix 11101100100, sub-opcode 11011; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.HI

TRAPCC.HI: SYS0, prefix 11101100100, sub-opcode 11100; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

TRAPCC.LOS

TRAPCC.LOS: SYS0, prefix 11101100100, sub-opcode 11101; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

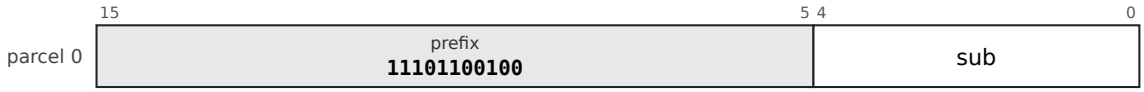
SYSPG: Reserved system words

Mnemonics: `SYSPG0`, `SYSPG1`, `SYSPG2`, `SYSPG3`, `SYSPG4`, `SYSPG5`

Syntax

```
SYSPG0 ... SYSPG5
```

Encoding



The chart is the leaf `16-SYS0-1`, SYS0, prefix `11101100100`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `16-SYS0-1`, `16-SYS0-2`.

Operation

```
none: the six words are reserved
```

Flags None written.

Exceptions Not specified. The reference executor traps them as unimplemented instructions.

(S) Placeholders. The grammars reserve six system words for the page-bit mechanisms that the author approved to be specified (errata 84: landing pads gated by a page bit, the `SYS` page bit), which `docs/retire-stage-candidates.md` describes as needing no opcode; no text of the repository gives these words a meaning, and software must not use them.

Why it is here. The grammars reserve six words next to `LPAD` and `TRAPCC` for the page-bit mechanisms: the author approved landing pads gated by a page bit and the `SYS` page bit "to be specified" (errata 84), and the retire-stage study found that these need no opcode. The placeholders keep the space; the record does not say what any of them will be.

Status specified; **not yet implemented** in the reference executor.

SYSPG0

SYSPG0: SYS0, prefix 11101100100, sub-opcode 11110; operands no operand; 2 bytes




```
SYSPG0 ... SYSPG5
```

Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

SYSPG1

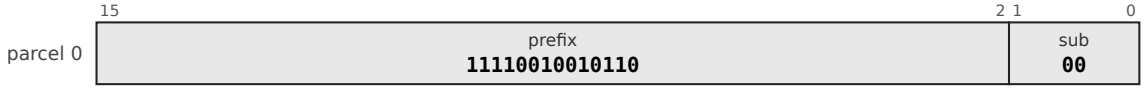
SYSPG1: SYS0, prefix 11101100100, sub-opcode 11111; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

SYSPG2

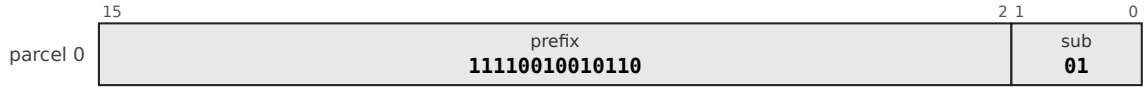
SYSPG2: SYS0, prefix 11110010010110, sub-opcode 00; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

SYSPG3

SYSPG3: SYS0, prefix 11110010010110, sub-opcode 01; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

SYSPG4

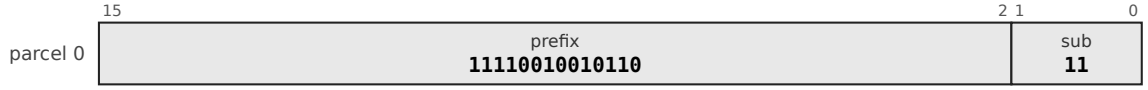
SYSPG4: SYS0, prefix 11110010010110, sub-opcode 10; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

SYSPG5

SYSPG5: SYS0, prefix 11110010010110, sub-opcode 11; operands no operand; 2 bytes



Operation, flags and exceptions as for the entry SYSPG above; 1 layout form, each a chart of its own (above).

9.15 Faults, restart and memory management

Cf. chapter 6, section 6.8 (faults, restart and memory management) and section 6.4 (memory management).

9.15.1 A fault is a trap, an abort, a progress stop in a block operation or a machine check, and every instruction either completes or is annulled and restarted with no state to undo (`MCPY` and its kin keep their progress in registers); the page-table format, the memory types and the translation registers are in the kernel chapter. *(D)*

9.15.2 A fused instruction (`MOVCALL`, `POPRET`, `POPMRET`, `MOV2`) is restarted whole after a fault: no part of it is visible as a separate instruction, and a test that switches the restore off fails. *(D)*

Commentary. The instruction-set consequence is the restart rule: the register update of an auto-increment mode is held until the instruction retires. **Impact.** *Kernel:* see 6.8. *Compiler:* no scheduling restriction arises from faults. *Hardware:* a single-entry undo buffer and a read-only walker.

9.16 Position-independent code

9.16.1 Position-independent code costs nothing extra on this machine: every access to code or data in the same module is PC-relative by an ordinary addressing mode, so there is no GOT register and no prologue to set one up. *(D)*

9.16.2 A call to an imported function is `CALLI got_slot(PC)` and a PLT stub is the single instruction `JMPI got_slot(PC)`; an imported variable's address is one `MOVQ got_slot(PC), Rd`. *(D)*

9.16.3 A position-independent access to a global is no longer than an absolute one: four bytes within 32 KB and six within 2 GB, against six for a 32-bit absolute address. *(D)*

9.16.4 The thread pointer is `DP`, so the local-exec thread-local model is one six-byte `MOVQ tls_offset(DP), Rd`. *(D)*

9.16.5 A six-byte branch reaches ±4 GiB, so no linker veneers are needed for any binary smaller than that; the assembler's and the linker's branch relaxation both reduce to choosing a length. *(D)*

9.16.6 `R12` is the linker's scratch register. The ABI (appendix D) decides that imports are called without a PLT stub; lazy against eager binding is open. *(D for `R12` and no stub; open for binding)*

Commentary. **Precedent.** On i386 a position-independent executable reserved `%ebx` (one of eight registers) for the GOT base and set it in every prologue, which cost a few percent of size and speed; x86-64 fixed that with RIP-relative addressing, and AArch64 needs linker veneers because its branches reach only 128 MB. **Trade-offs.** The two-byte PC-relative form

is the shortest, so position-independent data access is slightly smaller than absolute access; the price is a GOT load per imported call site where a PLT stub would cost a load per function. **Improvements.** Position-independence is a side effect of three decisions made for other reasons: PC-relative as an ordinary mode (adopted because the PC left the register file), variable-length encoding (for density) and `DP` (a 6502 zero-page descendant kept for density), which is exactly the thread pointer. **Benefits.** OpenBSD builds everything position-independent by default, and here that costs nothing. **Modern comparables.** x86-64 RIP-relative addressing; AArch64 `ADRP` with a GOT load and veneers for far calls. **Impact.** *Kernel:* a loader applies only relocations for data pointers, and write-xor-execute and randomisation come with no code penalty. *Compiler:* `-fPIE` is the default and emits the same instructions as `-fno-pic` for local data; no reserved register, no per-function setup. *Hardware:* nothing; the PC-relative mode is in the address adder already.

9.17 What was rejected

9.17.1 The architecture refuses, by design, every feature that adds hidden state, an unbounded instruction, a second operand path or a mode bit that changes what other instructions mean. *(D)*

9.17.2 From the PDP-11 and VAX it refuses general memory-to-memory operands, deferred addressing as a general form (only `JMPI` and `CALLI` are deferred), unbounded instructions with hidden microstate (`MOVC5`, `EDITPC`), packed decimal, hardware call frames (`CALLS`, `CALLG`), the PC as a general register, multiple floating-point formats, `POLY`, the `IOT`, `EMT`, `WAIT` and `RESET` instructions, a distinct low-half unsigned multiply, and the PDP-11's compare operand order. *(D)*

9.17.3 From x86 it refuses partial-register writes that preserve upper bits, operand-size and `REX` prefixes, segmentation, flag writes that vary by instruction, implicit registers for multiply, divide and shift count, the x87 stack, transcendental instructions and byte-granular instruction fetch. *(D)*

9.17.4 From the RISC family it refuses a register hardwired to zero (a sixteenth of a 16-register file), fused compare-and-branch in place of flags, branch delay slots, register windows and software-refilled translation buffers. *(D)*

9.17.5 It also refuses features of its own invention: NUL-terminated string instructions (`MSCN` with an explicit maximum gives a bounded `strlen`), `CRC32` words (carry-less multiply is the primitive under every CRC), reciprocal-estimate instructions, NaN payload propagation, a 32-bit profile, an eight-byte instruction alignment, `MOVT` and `MOVK` partial-immediate inserts, hardware tag checking, store-with-write-barrier, a self-synchronising encoding, and a scalar instruction for sub-1-bit inference (the accelerator of chapter 8 does it). *(D)*

Commentary. **Precedent.** The VAX's `EDITPC` is a bytecode interpreter in microcode, and DEC needed a separate control chip for the PDP-11's commercial instruction set and shipped none; the VAX's hardware call frames ran slower than the equivalent sequence on later models; ARM32's `PC+8` and MIPS's delay slot expose one implementation's pipeline forever. **Trade-offs.** Each refusal gives up a real convenience: the lost `MOV (R1)+,(R2)+` copy loop, the byte-wide `ADDB`, the inline-argument `JSR`. Section 9.18 says what replaces each. **Improvements.** The common pattern is that a refused feature was either subsumed by a more general one (`MCPY` for the copy loop) or paid its cost in every implementation for a few users (`EDITPC`). **Benefits.** An instruction set a small in-order core can implement completely and a compiler can target without exceptions. **Modern comparables.** RISC-V and AArch64 also refuse memory-to-memory operands and delay slots; AArch64 keeps flags where RISC-V refuses them. **Impact.** *Kernel:* no instruction has a restart state except the three that keep their progress in architectural registers, so a fault is always restartable. *Compiler:* no special cases: every operand rule is uniform, one memory operand and the rest registers. *Hardware:* no microcode engine, no second operand path, no hidden-state latch, no alignment rotator in the fetch path.

9.18 Idioms: restored, lost and replaced

9.18.1 Several idioms that the PDP-11 and VAX had and later RISC designs dropped are restored: subtract-one-and-branch (`SOB`, here in both directions), probe an address without faulting (`PROBER`, `PROBEW`), bit-field extract and insert, find-first-set (`CTZ`, `CLZ`), byte swap, block move, fill, compare and scan with their progress held in registers, the register-save mask (`PUSHM`, `POPM`, the useful half of `CALLS`), and carry-less multiply. *(D)*

9.18.2 The VAX's counted-string instructions are largely present: `MOVC3` is `MCPY`, `CMPC3` is `MCMP`, `LOCC` is `MSCN`, `SKPC` is `MSCNN`, and `MOVC5` is `MCPY` plus `MSET`. Table-translated move, table scans and substring search are absent because a per-byte table lookup stops a block operation from streaming. *(D)*

9.18.3 Deliberately lost, with their replacements: memory-to-memory `MOV` (replaced by `MCPY`); autoincrement-deferred and the dispatch-table idiom (replaced by `JTAG` for tag dispatch and a scaled-index `JMP` for the rest); `JSR Rn` and `RTS Rn` with inline

arguments and cheap coroutines (replaced by `CALL` and `RET` with the return address on the stack where every unwinder and debugger expects it); the VAX `CASE` and `INDEX` instructions (replaced by a compare, branch and scaled `JMP`); byte and word ALU operations (`MOVZB`, the operation, `MOVB`: three instructions where the PDP-11 had one); and the flag set and clear instructions. *(D)*

9.18.4 Subsumed, so not losses: `SXT` (the conversion moves), `SWAB` (`BSWAP`), `MARK` (the convention it served is gone), `ASH` and `ASHC` (the shifts take a register count), `MFPI` and `MTPI` (`PROBER` with the previous-mode bits and the user-memory guard), `MULU` (identical to `MUL` in the low half), and `ROL` (an assembler alias for `ROR`). *(D)*

9.18.5 Measurements decide what is included, and the compiler of this CPU uses more of the set than the first compiler did: `SET cc, rd`, `ABS`, `MIN` and `MAX`, the bit-field instructions, the strided auto-increment modes and the scaled index are all emitted, and the fused pairs are 19,661 instructions of the corpus of pdp64/16 (19,786 of pdp64/32). *(D)*

9.18.6 The compiler fuses a move before a call (`MOVCALL`), a pop before a return (`POPRET`, `POPMRET`) and two register moves (`MOV2`); the pairs are matched by dynamic programming after register allocation, two bytes saved a pair, 39,674 bytes (4.16%) in the corpus of pdp64/16 when fusion was first added. The mask of a `POPMRET` names `R8` to `R15`, so a save of `R24` to `R31` in pdp64/32 does not fuse.

9.18.7 A dense `switch` is a jump table with eight-byte entries placed after the return (the analogue of the VAX `CASEL`); any other `switch` is a binary search, which costs 0.27% in static size and takes the opcode dispatch of Lua from 225.1 million to 148.7 million instructions executed. The jump tables are 3.4% to 5.0% fewer cycles on Lua in the timing model and 806 bytes smaller.

9.18.8 Signed division by a power of two, and a pointer difference, are shifts; signed division and remainder of an `int` by a constant are a multiply and a shift (2 to 6 and 6 to 10 bytes longer than the plain code, 30% to 67% fewer cycles in micro-benchmarks). Unsigned and 64-bit division by a constant needs a `MULH`-based tree and is not done.

9.18.9 An array index is scaled by the addressing mode and not by a shift and an add, including an index that is converted from a narrower type first (the inner loop of the `bs_ec` benchmark, 31% fewer instructions); a walk down an array is a strided post-increment; multiplication by 5 or 9 is a `LEA`.

Commentary on 9.18.6 to 9.18.9. **Precedent.** The VAX back end of the portable C compiler used `CASEL`, the autoincrement modes and `INDEX` for these jobs; this list is what survived of that table on this CPU. **Trade-offs.** Each pass was measured against its cost: the size-for-speed passes (division by constants, jump tables) lengthen the code a little and shorten the run a lot, and the static-single-assignment form was left off because it ran 3.2% to 5.1% slower in the timing model. **Impact.** *Kernel:* a switch in a driver or a system-call dispatch is a table. *Compiler:* every item is a switchable pass with its own flag, so a regression can be bisected. *Hardware:* none of these needs an instruction that the entries do not already describe.

Commentary. **Precedent.** The PDP-11's `JSR Rn` put the return address in a register of the caller's choosing, which made coroutines a pair of `JSR`s and inline arguments free; the VAX replaced it with `CALLS` and a frame, which was slower than the code it replaced. **Trade-offs.** The most frequently felt loss in hand-written code is the byte-wide read-modify-write; the compiler never notices it. A bounds-checked `INDEX` for Pascal and Ada is a possible addition. **Improvements.** `SOB` branches either way, with four reaches; the block operations keep their progress in registers, which is what the VAX got wrong, so a fault restarts for free. **Benefits.** The code a compiler emits is the code the table above measures, and the extras are for the people who write assembler. **Modern comparables.** x86 keeps `LOOP` and the string instructions with `REP`; AArch64 has bit-field moves and no memory-to-memory forms; RISC-V has none of them in the base. **Impact.** *Kernel:* `PROBER` and `PROBEW` replace the `nofault` global in the user-copy routines. *Compiler:* idioms the compiler never emits are not built for it, so the compiler's instruction selection stays small. *Hardware:* each restored idiom is a few hundred gates of existing datapath, and each lost one is a decode that never existed.

9.19 Side by side: the PDP-11, the VAX and the pdp64

9.19.1 The word is 64 bits against the PDP-11's 16 and the VAX's 32, the registers are 16 as on the VAX, the virtual address is 39 bits with 40 physical, and the instruction count is 304 instruction words in 270 leaves, against the VAX's 304. *(D)*

9.19.2 The program counter is not a register, there is no argument pointer, and the frame pointer and the linker scratch are ordinary software conventions. *(D)*

9.19.3 An instruction has one or two operands (three for the block operations) and at most one memory operand, against the PDP-11's two and the VAX's six. *(D)*

9.19.4 The condition codes are the same four, `N`, `Z`, `V` and `C`; floating point is in the base and in the general registers, with three IEEE formats (half, single, double) and no mode bits that change the meaning of other instructions, where the PDP-11 had `FPS.FD`, `FL` and `FT` and the VAX had rounding bits in the status word. *(D)*

9.19.5 An instruction is two, four or six bytes (one to three parcels of 16 bits), or ten with a 64-bit literal, against the PDP-11's two to six and the VAX's one to more than fifty. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX instruction formats, with their encodings drawn side by side, are in the instruction-set text, section 13. **Trade-offs.** The pdp64 keeps what made the PDP-11 pleasant, orthogonal addressing and a small register set, and what made the VAX powerful, flags, a register-save mask and string operations, and drops what made both expensive: memory-to-memory operands and unbounded instructions. **Improvements.** The compiler's code averages 3.11 bytes an instruction on the 841-file corpus and 2.68 executed. **Benefits.** A processor that is smaller than either predecessor while running 64-bit code. **Modern comparables.** The closest family is AArch64 in the number of registers and addressing modes and RISC-V compressed in density; neither has flags and a variable length together. **Impact.** *Kernel:* the exception model is the PDP-11's with the VAX's per-mode stacks. *Compiler:* one memory operand makes instruction selection a register-allocation problem, as on a RISC. *Hardware:* a decoder that finds the length from a prefix of the first parcel, and no microcode.

9.20 What was taken from LoongArch, and what was not

9.20.1 LoongArch64 is the most recent clean-room instruction set designed without an ideological commitment: it borrowed from MIPS, RISC-V and ARM as each had the better answer, which is the posture of the pdp64.

9.20.2 Adopted from it: bit manipulation in the base (`CLO`, `CLZ`, `CTZ`, bit-field moves, byte reverse), atomics with the barrier folded in (the `_DB` variants), a richer atomic set (fetch-and-minimum and maximum), and memory type as a page attribute. *(D)*

9.20.3 Declined, with reasons: `R0` hardwired to zero (a sixteenth of a 16-register file, and `CLR` and `NEG` are one parcel already); no condition codes (a third read port in the branch unit against a four-bit latch); `FCSR` rounding-mode bits (hidden state that changes other instructions); four PC-relative address-formation instructions (one `LEA` carries 32 bits); 32 registers (this encoding has no bit to spare for the field); and separate vector register files (chapter 8). *(D)*

9.20.4 Under consideration: separate floating-point condition flags, software-visible translation-buffer manipulation (to pin an entry for a real-time kernel). *(P)*

9.20.5 A branch-free conditional select for constant-time code, `MASKEQZ` and `MASKNEZ`, is adopted; `BSEL`, a bit select, is proposed. *(D for the first two, P for the third)*

Commentary. **Precedent.** LoongArch has a fixed 32-bit encoding with room for two register fields and a displacement, so it can afford compare-and-branch and needs four instructions to form an address; this machine's variable length reverses both trade-offs. **Trade-offs.** The direct-mapping windows LoongArch uses to avoid translation-buffer misses on kernel text are deferred: `DMW0` and `DMW1` are stored without effect (6.3). **Improvements.** The same idea is right or wrong depending on the encoding around it, and each adoption is judged against this encoding. **Benefits.** Borrowing the tested parts saves design time and avoids their bugs. **Modern comparables.** LoongArch itself; the RISC-V bit-manipulation extensions (Zbb, Zbs) do much of the same. **Impact.** *Kernel:* atomics with barriers make a lock acquire one instruction. *Compiler:* bit operations in the base mean the compiler can use them without a feature test. *Hardware:* each adopted idea is a few hundred lookup tables and each declined one is a block not built.

9.21 What is still open in the instruction set

9.21.1 Open, with the reasoning in appendix B or in the instruction-set text, section 12: the thin 6-bit displacement of the shortest scaled-index form; the landing-pad decision (optional first, mandatory later); atomics against the bus (the lock against the monitor; the monitor is chosen); worst-case instruction latency for the block operations (an interrupt-latency claim must state the block granularity); the floating-point condition flags (the signed branches do not treat unordered cleanly); the block operations and the cache (a streaming memory type or a non-temporal variant); directed rounding for arithmetic (a rounding field in the instruction); the leftovers of half precision (minimum, maximum, rounding, bfloat16); lazy against eager binding in the ABI; the verification of the memory model against published litmus tests; and the coherence protocol beyond four cores (the design supports at least four and prefers sixteen or more, errata 92; the leanings are in `docs/smp-decisions.md`). *(P and open)*

9.21.2 Settled: the instruction length (one to three parcels, with `MOVQL` the one exception), the prefix code in place of a reserved-opcode map, and UTF-8 scope (`MVAL` is committed). *(D)*

9.21.3 Open in the encodings, as the repository leaves them: the placement of the operands of the atomic group and the size of `CAS` (9.13.3); the poison bit that the deferred loads need, and the combination `.ud` of the two qualifiers; the full specification of the user-access moves; the vector number of `TRAPCC`; the semantics of the AES and SHA words, still to be checked against FIPS 197 and FIPS 180-4; `BSEL`; the `SYSPG0` to `SYSPG5` placeholders; the three-register form of `ADDV`, `SUBV` and `MULV` against the two-register form of the grammar; a store through `d(DP)`; a profile-tuned variant of the encoding (not tried); and, in the compiler, the use of `MOVUW`, `MOVUL` and `MOVUQ`, division by 64-bit constants, loop-invariant motion and live-range splitting. *(P and open)*

Commentary. **Trade-offs.** None of the open items blocks the first implementation; the three that must be settled before silicon, the memory model verification, the coherence choice and the worst-case latency figure, are verification tasks, not design changes. **Impact.** *Kernel:* the open items touching the kernel are the user-memory guard, the floating-point state and the block-operation granularity. *Compiler:* conditional select and directed rounding would change instruction selection. *Hardware:* the coherence protocol must be chosen before the cache is built, not after.

Chapter 10 — Conclusion: from FPGA to silicon

The pdp64 is built to be run first on a modest FPGA, to be small enough that its whole core is about sixteen thousand lookup tables or ninety thousand transistors, and to reach silicon only after an OpenBSD built by the project's own toolchain can be logged into over `ssh`. The chapter gives the FPGA plan and its numbers, the transistor budget for a chip, why the first implementations are in order and how the architecture is kept ready for a faster one, and the path from the present toolchain to the finish line. Tags: *(D)* decided; *(E)* an estimate, not a measurement.

10.1 The FPGA plan

10.1.1 The first implementation is a soft core on a system-on-chip FPGA whose hard ARM cores serve as the bring-up harness: they load programs, run the tests and provide the console while the pdp64 core is still being debugged. *(D)*

10.1.2 The first board is the Terasic DE10-Nano (Cyclone V SE: about 110,000 logic elements, dual Cortex-A9, 1 GB of memory on the hard processor, HDMI, and the MiSTer ecosystem); the Digilent Zybo Z7-20 or the AMD Kria KV260 are the move when timing becomes the limit; the open-toolchain ULX3S is kept in view but loses the hard-core harness. The device chat staged devices first on a Zynq UltraScale+ (`decisions-pending.md` E6). *(P)*

10.1.3 A complete core is about 16,000 six-input lookup tables, 10 to 16 multiplier blocks and 11 block RAMs, with an error of about ±40%: the register file is 250 lookup tables of distributed RAM, the 64-bit ALU 600, the barrel shifter 450, the caches 300 and 500 plus 8 block RAMs, the bus bridge 600; the figures are for the original encoding, and the decoder and register file of this CPU have not been estimated separately. *(E)*

10.1.4 Block RAM, not logic, is the scarce resource: the caches, the translation buffer, the framebuffer and the blitter scratch compete for about 5.5 megabits. *(E)*

10.1.5 The realistic clock is 50 to 80 MHz on the Cyclone V, 90 to 130 MHz on a Zynq 7020 and 180 to 250 MHz on an UltraScale+; a correct 40 MHz machine is worth more than a fast broken one, and 60 MHz is already some fifty times a PDP-11/70. *(E)*

10.1.6 The work is staged in phases, each ending in a demonstration and a regression suite: specification and tools; an integer core; exceptions; the memory-management unit; the memory system; floating point; a booting kernel; peripherals; and, last, the other cores: the design supports at least four, and the initial device has four. *(D)*

10.1.7 Phase 2, the fault model, is not optional and does not move; the block operations are built with it, because their whole specification is about what state is visible when an exception arrives. *(D)*

10.1.8 On the paper estimates of the SMP decisions, four cores with the floating-point unit fit only a board of the Kria KV260 class, and sixteen cores fit none of the boards of the plan; no core has been synthesised. *(E)*

Commentary. **Precedent.** The PDP-11's own microprocessor, the T-11, was about 12,000 transistors, a complete integer core without a memory-management unit or floating point; the J-11 that implemented the full PDP-11/70 was about 120,000. The MicroVAX 78032 was 125,000 transistors for a subset of the VAX, with 175 of its 304 instructions and no floating point. **Trade-offs.** The 64-bit carry chain is slow on an FPGA and gates the clock more than it would in silicon, the barrel shifter is a large multiplexer tree, and flags serialise branch resolution behind the ALU; none of that argues for changing the instruction set, only for where the pipeline registers go, for example a virtually indexed, physically tagged first-level cache so that the translation buffer is off the critical path. A multiplier built from two to four multiplier blocks over four to eight cycles closes timing where a full combinational one will not. **Improvements.** The register-transfer description is kept vendor-neutral, with every vendor primitive in one thin directory (under 500 lines), so that moving between boards costs that directory and nothing else. **Benefits.** A single engineer working part-time can reach a booting system with floating point in roughly 10 to 19 months, by the plan's wide-error-bar estimate, with phases 2 and 3 the ones that historically double. **Modern comparables.** The RISC-V soft cores Rocket and VexRiscv are the usual FPGA reference points; the plan treats the compiler back end as the schedule risk, as it is for any new architecture.

10.2 The chip: a transistor budget

10.2.1 For a non-pipelined or short-pipeline implementation in a modern process, the whole core excluding caches is estimated at about 92,000 transistors. *(E)*

Block	Transistors (E)
register file, 16 × 64, two read ports and one write	~6,000
ALU, shifter, address adder	~15,000
decode and sequencer	~6,500
bit unit (popcount tree, count zeros, bitfields)	~3,500
block-move sequencer	~1,500
UTF-8 decode and validity	~1,200
translation buffer (32 entries), walker, window compare	~19,000
floating-point unit with fused multiply-add, classify, min and max	~34,000
control, status word, exceptions, undo buffer	~5,500
total, excluding caches	**~92,000**

10.2.2 The bit unit and the UTF-8 decoder together cost less than 5% of the die; the fused multiply-add datapath is the largest single addition and the least negotiable. *(E)*

Commentary. **Precedent.** The figure sits between the T-11 and the MicroVAX: it is a 64-bit machine with floating point and memory management below the 78032's 125,000 transistors, which had neither. **Trade-offs.** The estimate is not a measurement and excludes caches, which dominate the area of a real chip; an implementation of the translation buffer as a content-addressable memory is larger than the figure assumes, which is why the FPGA build uses a four-way set-associative one. **Benefits.** A chip this size fits in the budget of a small process node with room for caches and the bus controller. **Modern comparables.** The ARM1 of 1985 was about 25,000 transistors without a memory-management unit or floating point; the Intel 80386 about 275,000.

10.3 In order first, ready for out of order

10.3.1 The first implementations, on the FPGA and in silicon, are small in-order cores without speculation; the architecture is kept ready for a large out-of-order design. *(D)*

10.3.2 The condition flags were the structural obstacle: 68.2% of compiled instructions wrote a flag and only 1.5% read one, so the flags form two groups that nothing writes in part, and moves, loads, stores and the compare-and-branch instructions write none. *(D)*

10.3.3 An in-order, non-speculative core has none of the speculative side channels of the last decade, and an out-of-order core must keep that property; the rules that keep a future faster core from repeating them are collected in the hardware-vulnerabilities report, and include no speculation across protection domains, no shared predictor state across privilege boundaries without a flush, and operand-independent timing for a declared set of instructions. *(D as a design aim; the rules are proposals)*

10.3.4 Sequential consistency is nearly free on a small in-order core and is the most expensive memory model for a large speculative one; if a large core is built it may be necessary to move to total store order, which is the least painful retreat. *(P)*

Commentary. **Precedent.** One instruction set has often spanned in-order and out-of-order cores: the ARM small cores beside the large ones, the RISC-V Rocket beside the BOOM, x86 from the Pentium to the P6, Alpha from the 21064 to the 21264. The VAX's long, orthogonal instructions with many side effects are generally blamed for making its later fast implementations so hard. **Trade-offs.** Readiness has a cost, visible here as the compiled-code size paid for the flag rules (0.39%) and as design effort spent on a core that may never be built. **Improvements.** The pain-points report finds that the pdp64 avoids the largest classes of kernel pain by design (barriers, floating-point state, a faulting entry, remote translation flushes, the speculative family) and is exposed in a few places: a single page-table base if a core ever speculates, software accessed and dirty bits, uncached DMA buffers, and no specified constant-time or Rowhammer protection. **Benefits.** Any of these can be addressed in the specification while it is still being written, which is cheaper than after silicon. **Modern comparables.** The common vulnerability-driven changes of recent years, page-table isolation and speculation barriers, are what a speculative core of this architecture would have to add.

10.4 The finish line, and the path to it

10.4.1 Before silicon the project delivers five things: an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D)*

10.4.2 The architecture manual, which includes the instruction set but covers the whole machine, is the primary deliverable; the toolchain is advanced as needed to write and check it. *(D)*

10.4.3 The assembler and linker exist; the assembler lacks the atomics and the linker lacks shared libraries; the pcc compiler back end passes 516 C tests on the simulator and on QEMU; the LLVM back end has its machine-code layer, checked against the project's own assembler on 62 programs and 6,304 instruction forms with no disagreement; the simulator and QEMU run a kernel-mode machine with exceptions, a memory-management unit, a timer and a console; no part of OpenBSD is ported. *(measured)*

10.4.4 The long pole is the LLVM back end and clang. OpenBSD's kernel and base cannot be built by pcc, and the fallback of porting Unix/32V with pcc would not reach the goal, because it has no network stack and no `ssh`. *(D)*

10.4.5 The path, in order, with a test for each step: finish the toolchain gaps (atomics, the user-memory guard, the ABI version); an LLVM back end that passes the C tests; clang, the compiler runtime and `lld`; a machine model for the operating system with an enumerating bus, a disk, a network card and an entropy source (virtio devices first, for speed); the kernel booting to single-user; the userland to a login prompt; the network and `sshd`, and then `ssh user@localhost -p 2222` gives a shell; and finally the same system on the FPGA. *(P)*

Commentary. **Precedent.** OpenBSD's riscv64 port is its newest and smallest, and it measures at 35,600 lines of machine-dependent code, of which 14,873 are the core, 11,771 platform drivers, 4,513 the loader and 4,443 headers; arm64 is 61,613 and amd64 79,167. **Trade-offs.** The pdp64 port should be smaller than the riscv64 one in several places: no firmware tables to parse, uniform eight-register devices instead of a platform's driver zoo, a trivial loader, no cache-maintenance code and no floating-point state code; the core machine-dependent code of about 15,000 lines is the part that has to be written by hand *(estimate)*. The LLVM back end is estimated at three to six months for one experienced engineer, a figure that has not been tested. **Benefits.** An OpenBSD that can be logged into is a demonstration that the architecture, the toolchain and the operating-system interface are complete in the way that matters to users, and it forces every piece (the compiler, the linker, the bus, the devices, the boot) to work together. **Modern comparables.** OpenBSD ports to new architectures have followed this shape; the RISC-V port is the nearest in kind.

10.5 What remains open

The open items are collected, with their options and benefits, in appendix B; ideas that are not decisions are in appendix A. The principal open choices are the transport and interrupt controller (chapters 2 and 5), the details of the device layout, the interrupt controller and virtualization position, the vector-math position, and the several smaller instruction-set proposals.

Part II — The instruction reference tables

The instruction reference tables

This is the reference part of the manual: **one entry for each instruction**, in the layout of a processor reference manual. Each entry gives its mnemonics, its syntax, its encoding (a chart of its layout, and the list of all its layouts), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains it. The charts, the hot-register tables and the operand widths are generated from the grammars of the encoding (`spec/pdp64-enc.hbnf` and the grammar of the CPU), the one place where the encoding is written, so a layout here cannot disagree with the assembler or the simulators; the words (operation, flags, exceptions) are in `tools/manual/isa_entries.py` and are checked against the reference executor, `common/pdp64exec.c` and `common/pdp64exec_enc.inc`, which `sim64` runs. The Status line of each entry says whether the executor implements the instruction.

Notation. `Rd` is the destination register, `Rs` the first source, `Rt` the second, `Ru` the third; the assembler writes the destination last. `rm` is a general operand. `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. `N Z V C` are the condition flags. Bit charts number the bits of a parcel from 15 at the left.

Cf. lines point to the chapter that explains the matter an instruction touches: the kernel (chapter 6), the multiprocessor model (chapter 7), the bus (chapter 2).

Where the entries are

- Section 9.6, Moves, conversions and address formation: `MOVQ`, `MOVZL`, `LEA`, `MOVUB`, `MOV2`, `MOVQL`, `MOVQ.D`, `MOVQ.U`, `MOVQ.UD`
- Section 9.7, Integer arithmetic: `ADD`, `SUB`, `CMP`, `MUL`, `MULH`, `ADDM`, `INC`, `DEC`, `NEG`, `ADC`, `SBC`, `DIV`, `MULHU`, `ABS`, `MIN`, `SET`, `MASKEQZ`, `ADDV`
- Section 9.8, Logic, shifts and rotates: `BIC`, `BIS`, `BIT`, `XOR`, `AND`, `SHL`, `CLR`, `COM`, `TST`, `ASL`, `ASR`
- Section 9.9, Bits, bytes and text: `CLZ`, `POPCNT`, `BSWAP`, `BITREV`, `ROR`, `RORI`, `RORI32`, `DSHR`, `EXTB`, `INSB`, `XNORPOP`, `CLMULL`, `UTF8LEN`, `UTF8DEC`, `UTF8ENC`, `BSEL`, `AESES`, `AESKS1`, `SHA256SIG0`, `SHA512SIG0`
- Section 9.10, Branches and control flow: `BR`, `SOB`, `CBNE`, `BTAG`, `JMP`, `CALL`, `JMPI`, `JTAG`, `MOVCALL`, `POPRET`, `POPMRET`
- Section 9.11, The stack and block operations: `PUSHM`, `POPM`, `PUSH`, `POP`, `MCPY`, `MSET`, `MCMP`, `MSCN`, `MVAL`, `MSHA256`, `MAESCTR`
- Section 9.12, Floating point: `FADD`, `FCMP`, `FDIV`, `FADD32`, `FMADD`, `FMADD32`, `FNEG`, `FMIN`, `FCLASS`, `FRINT`, `FCVTDQ`, `FSFLAGS`, `FADD16`, `FCVTSH`
- Section 9.13, Atomics and ordering: `CAS`, `SWAP`, `XADD`, `XAND`, `XMAX`, `LL`
- Section 9.14, System instructions, exceptions and interrupts: `NOP`, `RTI`, `RET`, `HALT`, `BPT`, `FENCE`, `WFE`, `SEV`, `TLBIA`, `PUSHX`, `ISYNC`, `HINT`, `MFSR`, `MTPS`, `SPL`, `TRAP`, `TLBI`, `MTASID`, `MTDP`, `MTDMW`, `MFCPUID`, `ICINV`, `PROBER`, `TRAPCC.NE`, `SYSPG0`

The tables

- Assembler aliases and alternative spellings
- The architectural constants
- The mnemonics, alphabetically
- The bit layouts of every opcode: a chart for each layout, generated from the grammar

The specification of pdp64/16 has 340 mnemonic words (304 instructions and 36 aliases), 70 of them only in the prefix-coded CPUs; the reference executor implements 241 of the 304 instructions; the Status line of the entry of each of the rest says that it is not implemented. There are 125 instruction entries, 270 bit-layout charts of the layouts, and an entry for each of the 304 instructions with a chart of each of its 1175 forms.

Generated by `tools/mkmanual.py`; do not edit.

Assembler aliases and old spellings

A mnemonic that assembles to another instruction. One mnemonic is one instruction (section 9.18); the disassembler never prints an alias.

Generated by `tools/mkmanual.py` from `spec/pdp64-mnemonics.hbnf`; do not edit.

Written	Assembles to	Group
BHIS	BCC	branch
BLO	BCS	branch
CBHIS	CBCC	cbranch
CBLO	CBCS	cbranch
FADDD	FADD	alu
FADDS	FADD32	fesc
FCLASSD	FCLASS	fesc
FCLASSS	FCLASS32	fesc
FCMPD	FCMP	alu
FCMPS	FCMP32	fesc
FDIVD	FDIV	fesc
FDIVS	FDIV32	fesc
FMADDD	FMADD	fesc
FMADDS	FMADD32	fesc
FMAXD	FMAX	fesc
FMIND	FMIN	fesc
FMSUBD	FMSUB	fesc
FMSUBS	FMSUB32	fesc
FMULD	FMUL	alu
FMULS	FMUL32	fesc
FNMADDD	FNMADD	fesc
FNMADDS	FNMADD32	fesc
FNMSUBD	FNMSUB	fesc
FNMSUBS	FNMSUB32	fesc
FRINTD	FRINT	fesc
FRINTDA	FRINTA	fesc
FRINTDM	FRINTM	fesc
FRINTDP	FRINTP	fesc
FRINTDZ	FRINTZ	fesc
FSQRTD	FSQRT	fesc
FSQRTS	FSQRT32	fesc
FSUBD	FSUB	alu
FSUBS	FSUB32	fesc
MULU	MUL	alu
ROL	RORI	besc
WAIT	WFE	sys-none
ROL rs, #n, rd	RORI rs, #(64-n), rd	assembler, immediate count only

The architectural constants

The numbers of the processor status word, the modes, the exception vectors, the page-table entry, the memory types, the translation geometry, the system registers, the addressing modes and the condition codes, each read from the machine-readable specification (`spec/pdp64-system.hbnf` and `spec/pdp64-mnemonics.hbnf`).
Generated by `tools/mkmanual.py`; do not edit.

The processor status word (16 bits)

Field	Bit	Width	Meaning
`C`	0	1	carry (borrow after a subtract)
`V`	1	1	signed overflow
`Z`	2	1	zero
`N`	3	1	negative
`T`	4	1	trace: take a trace trap after the next instruction
`PRI`	5	3	interrupt priority level, 0 to 7
`MODE`	8	2	current mode
`PREV`	10	2	previous mode (the mode before the last exception)

Bits 12 to 15 are reserved and read as zero. Bit 12 is the user-memory guard `UA`.

The modes

Mode	Value
kernel	0
supervisor	1
user	3

Value 2 is reserved: loading it into either mode field of the status word is an illegal instruction.

The exception vectors

The table has 512 entries of eight bytes at the physical address `VBR + 8 * vector`; `ECAUSE` equals the vector number.

Vector	Number	Cause
`RESET`	0	processor reset: the first instruction fetch
`ILLEGAL`	2	a reserved or undefined opcode, a reserved encoding or an unassigned system register
`PRIVILEGED`	3	a system instruction executed outside kernel mode
`MISALIGNED`	4	a data access whose address is not a multiple of its size (the `MOVU` forms excepted)
`NONCANONICAL`	5	a virtual address whose bits 63 to 39 differ from bit 38
`IFAULT`	6	instruction fetch from a page that is not mapped
`LFAULT`	7	load from a page that is not mapped
`SFAULT`	8	store to a page that is not mapped
`IPROT`	9	instruction fetch denied by the page permissions
`LPROT`	10	load denied by the page permissions
`SPROT`	11	store denied by the page permissions
`DIVZERO`	12	division by zero (`DIV`, `DIVU`, `MOD`, `MODU`)
`BUSFAULT`	13	a read of non-existent memory (a precise abort)
`LPAD`	14	an indirect transfer that did not land on `LPAD` (enforcement off in ABI version 1)
`BPT`	15	the `BPT` instruction
`TIMER`	32	the timer interrupt (level 6)
`COUNT`	512	the number of vectors; the table has this many eight-byte entries

The page-table entry (64 bits)

Field	Bit	Width	Meaning
`V`	0	1	valid
`R`	1	1	readable
`W`	2	1	writable
`X`	3	1	executable
`U`	4	1	accessible from user mode
`G`	5	1	global: ignores the ASID
`A`	6	1	accessed (managed by software)
`D`	7	1	dirty (managed by software)
`MT`	8	2	memory type
`PPN`	12	28	physical page number: physical address bits 12 to 39, in place

A non-leaf entry has `V` = 1 and `R` = `W` = `X` = 0 and points to the next table. Every bit not named is zero.

The memory types (`MT`)

Type	Value
cached	0
uncached	1
device	2

The translation geometry

Quantity	Value
levels	3

The system registers (`MFSR`, `MTSR`)

Name	Number	Access
`PTBR`	0	read and write
`EPC`	1	read and write
`EVA`	2	read only
`ECAUSE`	3	read only
`DMW0`	4	read and write
`DMW1`	5	read and write
`USP`	6	read and write
`SSP`	7	read and write
`EPSW`	8	read and write
`VBR`	9	read and write
`MMUCTL`	10	read and write
`ASID`	11	read and write
`CPUID`	12	read only

The addressing modes

Mode	Value
base	`0000`
index	`0001`
pcrel	`0010`
dp	`0011`
imm	`0100`
abs	`0101`
postinc	`0110`
predec	`0111`

Modes `1000` to `1111` are reserved. The syntax and the parcel layout of each mode are in section 9.5.

The condition codes

Name	Number	Aliases
al	0	
ne	1	
eq	2	
pl	3	
mi	4	
vc	5	
vs	6	
cc	7	hs
cs	8	lo
ge	9	
lt	10	
gt	11	
le	12	
hi	13	
los	14	

The mnemonics, alphabetically

Generated by `tools/mkmanual.py`; do not edit.

Mnemonic	Entry	Layouts	Group
ABS	ABS (section 9.7)	1, entry	Arithmetic (AESC)
ADC	ADC (section 9.7)	7, entry	Single operand (SOP)
ADD	ADD (section 9.7)	22, entry	Moves, address formation and the ALU
ADDM	ADDM (section 9.7)	6, entry	Memory escape (MESC)
ADDV	OVF (section 9.7)	1, entry	New in the prefix-coded CPUs
AESDS	AESR (section 9.9)	1, entry	New in the prefix-coded CPUs
AESDSM	AESR (section 9.9)	1, entry	New in the prefix-coded CPUs
AESES	AESR (section 9.9)	1, entry	New in the prefix-coded CPUs
AESESM	AESR (section 9.9)	1, entry	New in the prefix-coded CPUs
AESIM	AESK (section 9.9)	1, entry	New in the prefix-coded CPUs
AESKS1	AESK (section 9.9)	1, entry	New in the prefix-coded CPUs
AESKS2	AESK (section 9.9)	1, entry	New in the prefix-coded CPUs
AND	AND (section 9.8)	19, entry	Moves, address formation and the ALU
ASL	ASL (section 9.8)	7, entry	Single operand (SOP)
ASR	ASR (section 9.8)	7, entry	Single operand (SOP)
BCC	BCC (section 9.10)	3, entry	Branches
BCS	BCC (section 9.10)	3, entry	Branches
BEQ	BCC (section 9.10)	4, entry	Branches
BGE	BCC (section 9.10)	3, entry	Branches
BGT	BCC (section 9.10)	3, entry	Branches
BHI	BCC (section 9.10)	3, entry	Branches
BHIS (alias of BCC)	BCC (section 9.10)	3, entry	Branches
BIC	BIC (section 9.8)	15, entry	Moves, address formation and the ALU
BICM	ADDM (section 9.7)	6, entry	Memory escape (MESC)
BIS	BIS (section 9.8)	17, entry	Moves, address formation and the ALU
BISM	ADDM (section 9.7)	6, entry	Memory escape (MESC)
BIT	BIT (section 9.8)	12, entry	Moves, address formation and the ALU
BITREV	BITREV (section 9.9)	1, entry	Bits and fields (BESC)
BLE	BCC (section 9.10)	3, entry	Branches
BLO (alias of BCS)	BCC (section 9.10)	3, entry	Branches
BLOS	BCC (section 9.10)	3, entry	Branches
BLT	BCC (section 9.10)	3, entry	Branches
BMI	BCC (section 9.10)	3, entry	Branches
BNE	BCC (section 9.10)	4, entry	Branches
BNTAG	BTAG (section 9.10)	1, entry	Branches
BPL	BCC (section 9.10)	3, entry	Branches
BPT	BPT (section 9.14)	1, entry	System (SYS)
BR	BCC (section 9.10)	5, entry	Branches
BSEL	BSEL (section 9.9)	1, entry	New in the prefix-coded CPUs
BSWAP	BSWAP (section 9.9)	1, entry	Bits and fields (BESC)
BSWAPL	BSWAP (section 9.9)	1, entry	Bits and fields (BESC)
BSWAPW	BSWAP (section 9.9)	1, entry	Bits and fields (BESC)
BTAG	BTAG (section 9.10)	1, entry	Branches
BVC	BCC (section 9.10)	3, entry	Branches

Mnemonic	Entry	Layouts	Group
BVS	BCC (section 9.10)	3, entry	Branches
CALL	CALL (section 9.10)	12, entry	Single operand (SOP)
CALLI	JMPI (section 9.10)	8, entry	Indirect transfers (XFER)
CAS	CAS (section 9.13)	1, entry	Atomics (ATOM)
CASDB	CAS (section 9.13)	1, entry	Atomics (ATOM)
CBCC	CBCC (section 9.10)	11, entry	Branches
CBCS	CBCC (section 9.10)	11, entry	Branches
CBEQ	CBCC (section 9.10)	17, entry	Branches
CBGE	CBCC (section 9.10)	7, entry	Branches
CBGT	CBCC (section 9.10)	7, entry	Branches
CBHI	CBCC (section 9.10)	11, entry	Branches
CBHIS (alias of CBCC)	CBCC (section 9.10)	11, entry	Branches
CBLE	CBCC (section 9.10)	7, entry	Branches
CBLO (alias of CBCS)	CBCC (section 9.10)	11, entry	Branches
CBLOS	CBCC (section 9.10)	11, entry	Branches
CBLT	CBCC (section 9.10)	7, entry	Branches
CBMI	CBCC (section 9.10)	2, entry	Branches
CBNE	CBCC (section 9.10)	17, entry	Branches
CBPL	CBCC (section 9.10)	2, entry	Branches
CBVC	CBCC (section 9.10)	2, entry	Branches
CBVS	CBCC (section 9.10)	2, entry	Branches
CLMULH	CLMUL (section 9.9)	1, entry	Bits and fields (BESC)
CLMULL	CLMUL (section 9.9)	1, entry	Bits and fields (BESC)
CLO	COUNT (section 9.9)	1, entry	Bits and fields (BESC)
CLR	CLR (section 9.8)	10, entry	Single operand (SOP)
CLZ	COUNT (section 9.9)	1, entry	Bits and fields (BESC)
CMP	CMP (section 9.7)	17, entry	Moves, address formation and the ALU
COM	COM (section 9.8)	7, entry	Single operand (SOP)
CTO	COUNT (section 9.9)	1, entry	Bits and fields (BESC)
CTZ	COUNT (section 9.9)	1, entry	Bits and fields (BESC)
DEC	DEC (section 9.7)	10, entry	Single operand (SOP)
DIV	DIV (section 9.7)	1, entry	Arithmetic (AESC)
DIVU	DIV (section 9.7)	1, entry	Arithmetic (AESC)
DSHR	DSHR (section 9.9)	1, entry	Bits and fields (BESC)
EXTB	EXTB (section 9.9)	1, entry	Bits and fields (BESC)
EXTZB	EXTB (section 9.9)	1, entry	Bits and fields (BESC)

Mnemonic	Entry	Layouts	Group
FABS	FSIGN (section 9.12)	1, entry	Floating point (FESC)
FADD	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FADD16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FADD32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FADDD (alias of FADD)	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FADDS (alias of FADD32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FCLASS	FCLASS (section 9.12)	1, entry	Floating point (FESC)
FCLASS16	FCLASS (section 9.12)	1, entry	Floating point (FESC)
FCLASS32	FCLASS (section 9.12)	1, entry	Floating point (FESC)
FCLASSD (alias of FCLASS)	FCLASS (section 9.12)	1, entry	Floating point (FESC)
FCLASSS (alias of FCLASS32)	FCLASS (section 9.12)	1, entry	Floating point (FESC)
FCMP	FCMP (section 9.12)	8, entry	Moves, address formation and the ALU
FCMP16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FCMP32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FCMPD (alias of FCMP)	FCMP (section 9.12)	8, entry	Moves, address formation and the ALU
FCMPS (alias of FCMP32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FCOPYSIGN	FSIGN (section 9.12)	1, entry	Floating point (FESC)
FCVTDH	FCVTH (section 9.12)	1, entry	Floating point (FESC)

Mnemonic	Entry	Layouts	Group
FCVTDQ	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTDQU	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTDS	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTHD	FCVTH (section 9.12)	1, entry	Floating point (FESC)
FCVTHL	FCVTH (section 9.12)	1, entry	Floating point (FESC)
FCVTHS	FCVTH (section 9.12)	1, entry	Floating point (FESC)
FCVTLH	FCVTH (section 9.12)	1, entry	Floating point (FESC)
FCVTLS	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTQD	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTQUD	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTSD	FCVT (section 9.12)	1, entry	Floating point (FESC)
FCVTSH	FCVTH (section 9.12)	1, entry	Floating point (FESC)
FCVTSL	FCVT (section 9.12)	1, entry	Floating point (FESC)
FDIV	FDIV (section 9.12)	1, entry	Floating point (FESC)
FDIV16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FDIV32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FDIVD (alias of FDIV)	FDIV (section 9.12)	1, entry	Floating point (FESC)
FDIVS (alias of FDIV32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FENCE	FENCE (section 9.14)	1, entry	System (SYS)
FMADD	FMADD (section 9.12)	1, entry	Floating point (FESC)
FMADD16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FMADD32	FMADD32 (section 9.12)	1, entry	Floating point (FESC)

Mnemonic	Entry	Layouts	Group
FMADDD (alias of FMADD)	FMADD (section 9.12)	1, entry	Floating point (FESC)
FMADDS (alias of FMADD32)	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FMAX	FMIN (section 9.12)	1, entry	Floating point (FESC)
FMAXD (alias of FMAX)	FMIN (section 9.12)	1, entry	Floating point (FESC)
FMIN	FMIN (section 9.12)	1, entry	Floating point (FESC)
FMIND (alias of FMIN)	FMIN (section 9.12)	1, entry	Floating point (FESC)
FMSUB	FMADD (section 9.12)	1, entry	Floating point (FESC)
FMSUB16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FMSUB32	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FMSUBD (alias of FMSUB)	FMADD (section 9.12)	1, entry	Floating point (FESC)
FMSUBS (alias of FMSUB32)	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FMUL	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FMUL16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FMUL32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FMULD (alias of FMUL)	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FMULS (alias of FMUL32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FNEG	FSIGN (section 9.12)	1, entry	Floating point (FESC)
FNMADD	FMADD (section 9.12)	1, entry	Floating point (FESC)
FNMADD16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FNMADD32	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FNMADDD (alias of FNMADD)	FMADD (section 9.12)	1, entry	Floating point (FESC)
FNMADDS (alias of FNMADD32)	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FNMSUB	FMADD (section 9.12)	1, entry	Floating point (FESC)
FNMSUB16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FNMSUB32	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FNMSUBD (alias of FNMSUB)	FMADD (section 9.12)	1, entry	Floating point (FESC)
FNMSUBS (alias of FNMSUB32)	FMADD32 (section 9.12)	1, entry	Floating point (FESC)
FRINT	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTA	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTD (alias of FRINT)	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTDA (alias of FRINTA)	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTDM (alias of FRINTM)	FRINT (section 9.12)	1, entry	Floating point (FESC)

Mnemonic	Entry	Layouts	Group
FRINTDP (alias of FRINTP)	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTDZ (alias of FRINTZ)	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTM	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTP	FRINT (section 9.12)	1, entry	Floating point (FESC)
FRINTZ	FRINT (section 9.12)	1, entry	Floating point (FESC)
FSFLAGS	FSFLAGS (section 9.12)	1, entry	Floating point (FESC)
FSQRT	FDIV (section 9.12)	1, entry	Floating point (FESC)
FSQRT16	FHALF (section 9.12)	1, entry	Floating point (FESC)

Mnemonic	Entry	Layouts	Group
FSQRT32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FSQRTD (alias of FSQRT)	FDIV (section 9.12)	1, entry	Floating point (FESC)
FSQRTS (alias of FSQRT32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FSUB	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FSUB16	FHALF (section 9.12)	1, entry	Floating point (FESC)
FSUB32	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
FSUBD (alias of FSUB)	FADD (section 9.12)	11, entry	Moves, address formation and the ALU
FSUBS (alias of FSUB32)	FSINGLE (section 9.12)	1, entry	Floating point (FESC)
HALT	HALT (section 9.14)	1, entry	System (SYS)
HINT	HINT (section 9.14)	1, entry	System (SYS)
ICINV	ICINV (section 9.14)	1, entry	System (SYS)
ICINVA	TLBA (section 9.14)	1, entry	System (SYS)
INC	INC (section 9.7)	10, entry	Single operand (SOP)
INSB	INSB (section 9.9)	1, entry	Bits and fields (BESC)
ISYNC	ISYNC (section 9.14)	1, entry	System (SYS)
JMP	JMP (section 9.10)	11, entry	Single operand (SOP)
JMPI	JMPI (section 9.10)	8, entry	Indirect transfers (XFER)
JTAG	JTAG (section 9.10)	2, entry	Indirect transfers (XFER)
LEA	LEA (section 9.6)	30, entry	Moves, address formation and the ALU
LL	LLSC (section 9.13)	1, entry	Atomics (ATOM)
LPAD	NOP (section 9.14)	1, entry	System (SYS)
MAESCTR	MAESCTR (section 9.11)	1, entry	New in the prefix-coded CPUs
MASKEQZ	MASK (section 9.7)	1, entry	New in the prefix-coded CPUs
MASKNEZ	MASK (section 9.7)	1, entry	New in the prefix-coded CPUs
MAX	MINMAX (section 9.7)	1, entry	Arithmetic (AESC)
MAXU	MINMAX (section 9.7)	1, entry	Arithmetic (AESC)
MCMP	MCMP (section 9.11)	1, entry	Block memory (MBLK)
MCPY	MCPY (section 9.11)	1, entry	Block memory (MBLK)
MFASID	MTASID (section 9.14)	1, entry	System (SYS)
MFCPUID	MFCPUID (section 9.14)	1, entry	System (SYS)
MFDP	MTDP (section 9.14)	1, entry	System (SYS)
MFPS	MTPS (section 9.14)	1, entry	System (SYS)
MFSR	MFSR (section 9.14)	1, entry	System (SYS)
MIN	MINMAX (section 9.7)	1, entry	Arithmetic (AESC)
MINU	MINMAX (section 9.7)	1, entry	Arithmetic (AESC)
MMOV	MCPY (section 9.11)	1, entry	Block memory (MBLK)
MOD	DIV (section 9.7)	1, entry	Arithmetic (AESC)
MODU	DIV (section 9.7)	1, entry	Arithmetic (AESC)
MOV2	MOV2 (section 9.6)	1, entry	New in the prefix-coded CPUs
MOVB	MOV (section 9.6)	35, entry	Moves, address formation and the ALU

Mnemonic	Entry	Layouts	Group
MOVB.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVB.U	MOVUSER (section 9.6)	6, entry	New in the prefix-coded CPUs
MOVB.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVCALL	MOVCALL (section 9.10)	2, entry	New in the prefix-coded CPUs

Mnemonic	Entry	Layouts	Group
MOVL	MOV (section 9.6)	44, entry	Moves, address formation and the ALU
MOVL.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVL.U	MOVUSER (section 9.6)	6, entry	New in the prefix-coded CPUs
MOVL.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVQ	MOV (section 9.6)	51, entry	Moves, address formation and the ALU
MOVQ.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVQ.U	MOVUSER (section 9.6)	6, entry	New in the prefix-coded CPUs
MOVQ.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVQL	MOVQL (section 9.6)	1, entry	New in the prefix-coded CPUs
MOVUB	MOVU (section 9.6)	13, entry	Memory escape (MESC)
MOVUL	MOVU (section 9.6)	13, entry	Memory escape (MESC)
MOVUQ	MOVU (section 9.6)	13, entry	Memory escape (MESC)
MOVUW	MOVU (section 9.6)	13, entry	Memory escape (MESC)
MOVW	MOV (section 9.6)	34, entry	Moves, address formation and the ALU
MOVW.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVW.U	MOVUSER (section 9.6)	6, entry	New in the prefix-coded CPUs
MOVW.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZB	MOVZ (section 9.6)	27, entry	Moves, address formation and the ALU
MOVZB.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZB.U	MOVUSER (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZB.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZL	MOVZ (section 9.6)	18, entry	Moves, address formation and the ALU
MOVZL.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZL.U	MOVUSER (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZL.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZW	MOVZ (section 9.6)	18, entry	Moves, address formation and the ALU
MOVZW.D	MOVD (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZW.U	MOVUSER (section 9.6)	3, entry	New in the prefix-coded CPUs
MOVZW.UD	MOVUSERD (section 9.6)	3, entry	New in the prefix-coded CPUs
MSCN	MSCN (section 9.11)	1, entry	Block memory (MBLK)
MSCNN	MSCN (section 9.11)	1, entry	Block memory (MBLK)
MSET	MSET (section 9.11)	1, entry	Block memory (MBLK)
MSHA256	MSHA (section 9.11)	1, entry	New in the prefix-coded CPUs
MSHA512	MSHA (section 9.11)	1, entry	New in the prefix-coded CPUs
MTASID	MTASID (section 9.14)	1, entry	System (SYS)
MTDMW	MTDMW (section 9.14)	1, entry	System (SYS)

Mnemonic	Entry	Layouts	Group
MTDP	MTDP (section 9.14)	1, entry	System (SYS)
MTPS	MTPS (section 9.14)	1, entry	System (SYS)
MTSR	MFSR (section 9.14)	1, entry	System (SYS)
MUL	MUL (section 9.7)	15, entry	Moves, address formation and the ALU
MULH	MULH (section 9.7)	11, entry	Moves, address formation and the ALU
MULHU	MULHU (section 9.7)	1, entry	Arithmetic (AESC)
MULU (alias of MUL)	MUL (section 9.7)	15, entry	Moves, address formation and the ALU
MULV	OVF (section 9.7)	1, entry	New in the prefix-coded CPUs
MVAL	MVAL (section 9.11)	1, entry	Block memory (MBLK)
NEG	NEG (section 9.7)	7, entry	Single operand (SOP)
NOP	NOP (section 9.14)	1, entry	System (SYS)
POP	POP (section 9.11)	10, entry	Single operand (SOP)
POPCNT	POPCNT (section 9.9)	1, entry	Bits and fields (BESC)
POPM	POPM (section 9.11)	3, entry	Single operand (SOP)
POPMRET	POPMRET (section 9.10)	1, entry	New in the prefix-coded CPUs
POPRET	POPRET (section 9.10)	1, entry	New in the prefix-coded CPUs
POPX	PUSHX (section 9.14)	1, entry	System (SYS)
PROBER	PROBE (section 9.14)	1, entry	System (SYS)

Mnemonic	Entry	Layouts	Group
PROBEW	PROBE (section 9.14)	1, entry	System (SYS)
PUSH	PUSH (section 9.11)	12, entry	Single operand (SOP)
PUSHM	PUSHM (section 9.11)	3, entry	Single operand (SOP)
PUSHX	PUSHX (section 9.14)	1, entry	System (SYS)
RET	RET (section 9.14)	1, entry	System (SYS)
ROL (alias of RORI)	RORI (section 9.9)	1, entry	Bits and fields (BESC)
ROR	ROR (section 9.9)	1, entry	Bits and fields (BESC)
RORI	RORI (section 9.9)	1, entry	Bits and fields (BESC)
RORI32	RORI32 (section 9.9)	1, entry	Bits and fields (BESC)
RTI	RTI (section 9.14)	1, entry	System (SYS)
RTT	RTI (section 9.14)	1, entry	System (SYS)
SAR	SHIFT (section 9.8)	14, entry	Moves, address formation and the ALU
SBC	SBC (section 9.7)	7, entry	Single operand (SOP)
SC	LLSC (section 9.13)	1, entry	Atomics (ATOM)
SET	SET (section 9.7)	1, entry	Arithmetic (AESC)
SEV	SEV (section 9.14)	1, entry	System (SYS)
SHA256SIG0	SHA256 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA256SIG1	SHA256 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA256SUM0	SHA256 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA256SUM1	SHA256 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA512SIG0	SHA512 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA512SIG1	SHA512 (section 9.9)	1, entry	New in the prefix-coded CPUs

Mnemonic	Entry	Layouts	Group
SHA512SUM0	SHA512 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHA512SUM1	SHA512 (section 9.9)	1, entry	New in the prefix-coded CPUs
SHL	SHIFT (section 9.8)	14, entry	Moves, address formation and the ALU
SHR	SHIFT (section 9.8)	14, entry	Moves, address formation and the ALU
SOB	SOB (section 9.10)	1, entry	Branches
SPL	SPL (section 9.14)	1, entry	System (SYS)
SUB	SUB (section 9.7)	22, entry	Moves, address formation and the ALU
SUBM	ADDM (section 9.7)	6, entry	Memory escape (MESC)
SUBV	OVF (section 9.7)	1, entry	New in the prefix-coded CPUs
SWAP	SWAP (section 9.13)	1, entry	Atomics (ATOM)
SWAPDB	SWAP (section 9.13)	1, entry	Atomics (ATOM)
SYSPG0	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
SYSPG1	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
SYSPG2	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
SYSPG3	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
SYSPG4	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
SYSPG5	SYSPG (section 9.14)	1, entry	New in the prefix-coded CPUs
TLBI	TLBI (section 9.14)	1, entry	System (SYS)
TLBIA	TLBA (section 9.14)	1, entry	System (SYS)
TLBSYNC	TLBA (section 9.14)	1, entry	System (SYS)
TRAP	TRAP (section 9.14)	1, entry	System (SYS)
TRAPCC.CC	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.CS	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.EQ	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.GE	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.GT	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.HI	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.LE	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.LOS	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.LT	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.MI	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.NE	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs

Mnemonic	Entry	Layouts	Group
TRAPCC.PL	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.VC	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPCC.VS	TRAPCC (section 9.14)	1, entry	New in the prefix-coded CPUs
TRAPL	TRAP (section 9.14)	1, entry	System (SYS)
TST	TST (section 9.8)	10, entry	Single operand (SOP)
UTF8DEC	UTF8DEC (section 9.9)	1, entry	Bits and fields (BESC)
UTF8ENC	UTF8ENC (section 9.9)	1, entry	Bits and fields (BESC)
UTF8LEN	UTF8LEN (section 9.9)	1, entry	Bits and fields (BESC)

Mnemonic	Entry	Layouts	Group
WAIT (alias of WFE)	WFE (section 9.14)	1, entry	System (SYS)
WFE	WFE (section 9.14)	1, entry	System (SYS)
XADD	XADD (section 9.13)	1, entry	Atomics (ATOM)
XADDDB	XADD (section 9.13)	1, entry	Atomics (ATOM)
XAND	XAND (section 9.13)	1, entry	Atomics (ATOM)
XANDDB	XAND (section 9.13)	1, entry	Atomics (ATOM)
XMAX	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMAXDB	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMAXU	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMAXUDB	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMIN	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMINDB	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMINU	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XMINUDB	XMAX (section 9.13)	1, entry	Atomics (ATOM)
XNORPOP	XNORPOP (section 9.9)	1, entry	Bits and fields (BESC)
XOR	XOR (section 9.8)	17, entry	Moves, address formation and the ALU
XOR_	XAND (section 9.13)	1, entry	Atomics (ATOM)
XOR_DB	XAND (section 9.13)	1, entry	Atomics (ATOM)
XXOR	XAND (section 9.13)	1, entry	Atomics (ATOM)
XXORDB	XAND (section 9.13)	1, entry	Atomics (ATOM)

The bit layouts of every opcode

Every leaf of the encoding of pdp64/16 has a chart here. A leaf is a prefix of 3 to 16 bits of the first parcel, a sub-opcode that selects one of its mnemonics, and the pieces of its operands; there is no `len` field, because the prefix decides the length. The charts are generated by `tools/mkmanual.py` from the grammars `spec/pdp64-enc.hbnf` and `spec/pdp64-16/pdp64-16.hbnf`, read by `tools/encgram.py`; changing a field in a grammar changes the chart.

How to read a chart. Each row is one 16-bit parcel, bit 15 on the left, the bit numbers above the boxes. A shaded box has a fixed value, written in it; a white box is a field of the instruction. Fields are named `opN.part` for the part of the Nth operand of the assembly syntax (`op1.d` is the displacement of the first operand); a field the layout splits is `name[1]`, `name[2]` from the top. A register field of 3 or 4 bits holds an index into the hot-register table of the ABI, not a register number.

The hot-register tables

The part of the ABI on which the encoding depends: a 3-bit register field holds, in order, `R0 R1 R8 R2 SP R3 R9 R10`. The fused instructions of the encoding are `movcall`, `popmret`, `popret`, `mov2`.

The operand shapes

The widths and the reach of the immediates and displacements of each shape the forms use (the shapes are rules of the grammar):

Shape	Meaning
`abs32`	absolute address, 32 bits
`b3d1u-xsz`	memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access
`b3d2u-xsz`	memory at base register, 3 bits (an index into the hot-register table) + unsigned 2-bit displacement times the size of the access
`b3d3u-xsz`	memory at base register, 3 bits (an index into the hot-register table) + unsigned 3-bit displacement times the size of the access
`b4d0s-xsz`	memory at (base register, 4 bits)
`b4d10s-xsz`	memory at base register, 4 bits + signed 10-bit displacement times the size of the access
`b4d10u-xsz`	memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access
`b4d12s-x1`	memory at base register, 4 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes
`b4d12s-xsz`	memory at base register, 4 bits + signed 12-bit displacement times the size of the access
`b4d12u-xsz`	memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access
`b4d14s-xsz`	memory at base register, 4 bits + signed 14-bit displacement times the size of the access
`b4d28s-x1`	memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes
`b4d2u-xsz`	memory at base register, 4 bits + unsigned 2-bit displacement times the size of the access
`b4d8s-xsz`	memory at base register, 4 bits + signed 8-bit displacement times the size of the access
`b4d8u-xsz`	memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access
`dpd28s-x1`	memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes
`imm10s`	signed 10-bit immediate, -512 to 511
`imm12s`	signed 12-bit immediate, -2048 to 2047
`imm12u`	unsigned 12-bit immediate, 0 to 4095
`imm14s`	signed 14-bit immediate, -8192 to 8191
`imm16s`	signed 16-bit immediate, -32768 to 32767
`imm16u`	unsigned 16-bit immediate, 0 to 65535
`imm18s`	signed 18-bit immediate, -131072 to 131071
`imm32s`	signed 32-bit immediate, -2147483648 to 2147483647
`imm32u`	unsigned 32-bit immediate, 0 to 4294967295
`imm3s`	signed 3-bit immediate, -4 to 3
`imm3u`	unsigned 3-bit immediate, 0 to 7
`imm4s`	signed 4-bit immediate, -8 to 7
`imm4u`	unsigned 4-bit immediate, 0 to 15

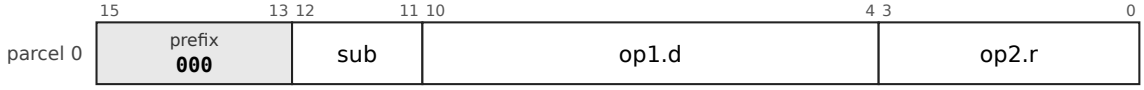
Shape	Meaning
imm5s	signed 5-bit immediate, -16 to 15
imm5u	unsigned 5-bit immediate, 0 to 31
imm64s	signed 64-bit immediate, -9223372036854775808 to 9223372036854775807
imm6s	signed 6-bit immediate, -32 to 31
imm6u	unsigned 6-bit immediate, 0 to 63
imm6u-x8	unsigned 6-bit immediate times 8, 0 to 504
imm7s	signed 7-bit immediate, -64 to 63
imm8s	signed 8-bit immediate, -128 to 127
imm8u	unsigned 8-bit immediate, 0 to 255
imm8u-x256	unsigned 8-bit immediate times 256, 0 to 65280
imm8u-x8	unsigned 8-bit immediate times 8, 0 to 2040

Shape	Meaning
k-imm0	implicit constant 0
k-sp	implicit register r14
pc10-u2	pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes)
pc12-u1	pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes)
pc12-u2	pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes)
pc13-u2	pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes)
pc14-u1	pc-relative, 14 bits counted in 1-byte units (-8192 to 8191 bytes)
pc14-u2	pc-relative, 14 bits counted in 2-byte units (-16384 to 16382 bytes)
pc16-u1	pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes)
pc16-u2	pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes)
pc18-u2	pc-relative, 18 bits counted in 2-byte units (-262144 to 262142 bytes)
pc20-u2	pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes)
pc24-u2	pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes)
pc32-u1	pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes)
pc32-u2	pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes)
pc4-u2	pc-relative, 4 bits counted in 2-byte units (-16 to 14 bytes)
pc5-u2	pc-relative, 5 bits counted in 2-byte units (-32 to 30 bytes)
pc6-u2	pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes)
pc7-u2	pc-relative, 7 bits counted in 2-byte units (-128 to 126 bytes)
pc8-u2	pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes)
pc9-u2	pc-relative, 9 bits counted in 2-byte units (-512 to 510 bytes)
pd4-s12	pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access)
pi4-s12	post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access)
reg3	register, 3 bits (an index into the hot-register table)
reg4	register, 4 bits
spd12s-x1	memory at sp + signed 12-bit displacement times 1, -2048 to 2047 bytes
spd12u-x8	memory at sp + unsigned 12-bit displacement times 8, 0 to 32760 bytes
spd12u-xsz	memory at sp + unsigned 12-bit displacement times the size of the access
spd14u-x8	memory at sp + unsigned 14-bit displacement times 8, 0 to 131064 bytes
spd4u-x8	memory at sp + unsigned 4-bit displacement times 8, 0 to 120 bytes
spd4u-xsz	memory at sp + unsigned 4-bit displacement times the size of the access
spd5u-xsz	memory at sp + unsigned 5-bit displacement times the size of the access
spd6u-x8	memory at sp + unsigned 6-bit displacement times 8, 0 to 504 bytes
spd7u-xsz	memory at sp + unsigned 7-bit displacement times the size of the access
x4-4d22s	memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits
x4-4d2s	memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits
x4-4d4s	memory at base + (index << scale) + signed 4-bit displacement; base register, 4 bits, index register, 4 bits
x4-4d6s	memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits
x4-4d8s	memory at base + (index << scale) + signed 8-bit displacement; base register, 4 bits, index register, 4 bits

The charts

Instructions of 1 parcel (2 bytes)

16-LDSP7-a-1: LDSP7/a, prefix `000`



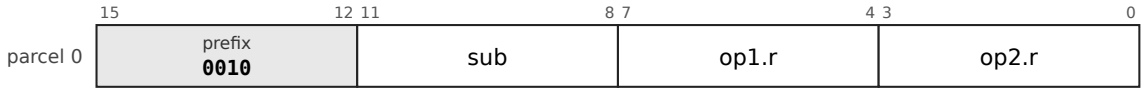
`op1` is `spd7u-xsz`: memory at sp + unsigned 7-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[12:11] op1.d=p0[10:4] op2.r=p0[3:0]`

16-RRF-1: RRF, prefix `0010`



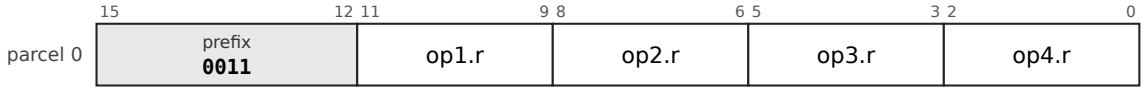
`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[11:8] op1.r=p0[7:4] op2.r=p0[3:0]`

16-FX-MOV2-1: FX.MOV2, prefix `0011`



`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

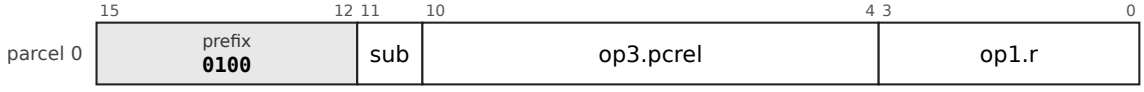
`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`op3` is `reg3`: register, 3 bits (an index into the hot-register table).

`op4` is `reg3`: register, 3 bits (an index into the hot-register table).

Fields: `op1.r=p0[11:9] op2.r=p0[8:6] op3.r=p0[5:3] op4.r=p0[2:0]`

16-CBZ1-7-a-1: CBZ1_7/a, prefix `0100`



`op1` is `reg4`: register, 4 bits.

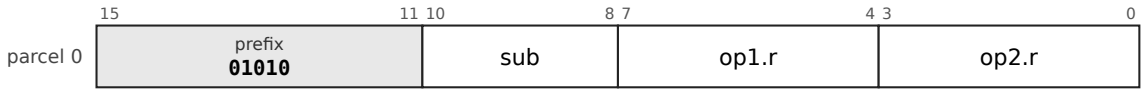
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc7-u2`: pc-relative, 7 bits counted in 2-byte units (-128 to 126 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[11] op3.pcrel=p0[10:4] op1.r=p0[3:0]`

16-RRF-2: RRF, prefix `01010`



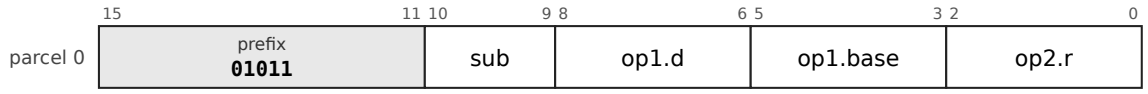
`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `cmp` 110, `fcmp` 111.

Fields: `sub=p0[10:8] op1.r=p0[7:4] op2.r=p0[3:0]`

16-LDHHH3-a-1: LDHHH3/a, prefix `01011`



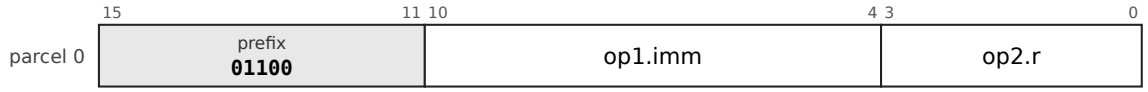
`op1` is `b3d3u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 3-bit displacement times the size of the access.

`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[10:9] op1.d=p0[8:6] op1.base=p0[5:3] op2.r=p0[2:0]`

16-IMM7-a-1: IMM7/a, prefix `01100`

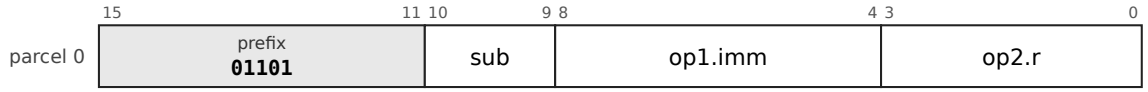


`op1` is `imm7s`: signed 7-bit immediate, -64 to 63.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.imm=p0[10:4] op2.r=p0[3:0]`

16-SHI5-a-1: SHI5/a, prefix `01101`



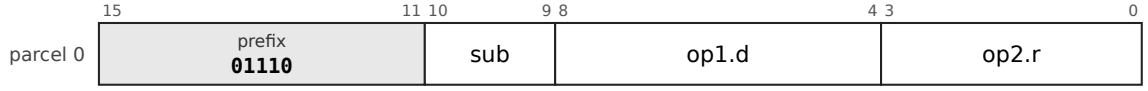
`op1` is `imm5u`: unsigned 5-bit immediate, 0 to 31.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `shl` 00, `shr` 01, `sar` 10, `rol` 11.

Fields: `sub=p0[10:9] op1.imm=p0[8:4] op2.r=p0[3:0]`

16-LDSP5-a-1: LDSP5/a, prefix `01110`



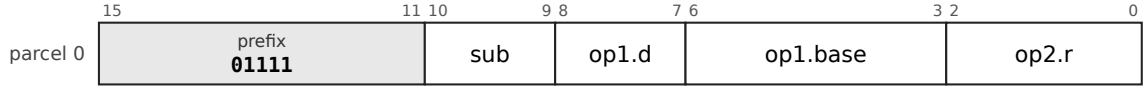
`op1` is `spd5u-xsz`: memory at sp + unsigned 5-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[10:9] op1.d=p0[8:4] op2.r=p0[3:0]`

16-LDHRH2-a-1: LDHRH2/a, prefix `01111`



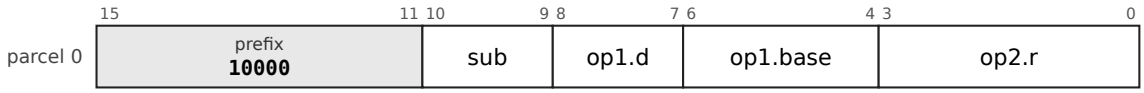
`op1` is `b4d2u-xsz`: memory at base register, 4 bits + unsigned 2-bit displacement times the size of the access.

`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[10:9]` `op1.d=p0[8:7]` `op1.base=p0[6:3]` `op2.r=p0[2:0]`

16-LDHHR2-a-1: LDHHR2/a, prefix `10000`



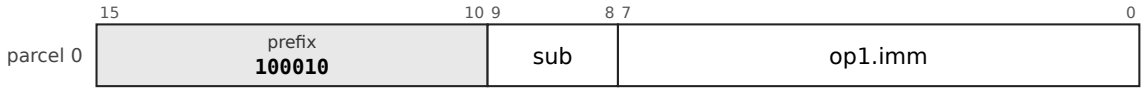
`op1` is `b3d2u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 2-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[10:9]` `op1.d=p0[8:7]` `op1.base=p0[6:4]` `op2.r=p0[3:0]`

16-SYSI-1: SYSI, prefix `100010`

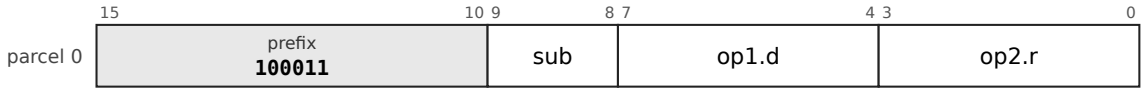


`op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255.

`sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

Fields: `sub=p0[9:8]` `op1.imm=p0[7:0]`

16-LDSP4-a-1: LDSP4/a, prefix `100011`



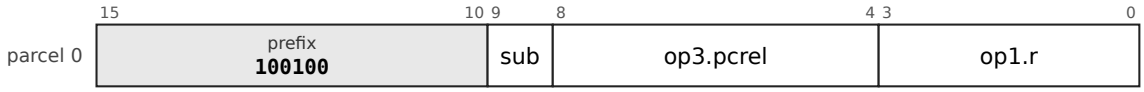
`op1` is `spd4u-xsz`: memory at sp + unsigned 4-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[9:8]` `op1.d=p0[7:4]` `op2.r=p0[3:0]`

16-CBZ1-5-a-1: CBZ1_5/a, prefix `100100`



`op1` is `reg4`: register, 4 bits.

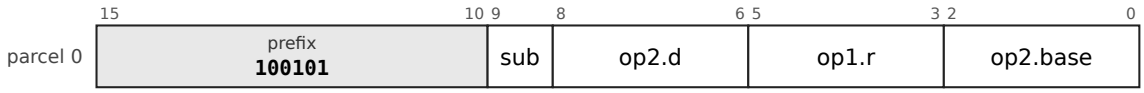
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc5-u2`: pc-relative, 5 bits counted in 2-byte units (-32 to 30 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[9]` `op3.pcrel=p0[8:4]` `op1.r=p0[3:0]`

16-STHHH3-a-1: STHHH3/a, prefix `100101`



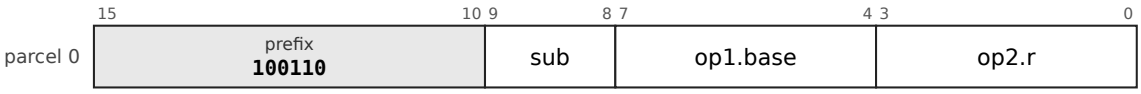
`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

`op2` is `b3d3u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 3-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[9] op2.d=p0[8:6] op1.r=p0[5:3] op2.base=p0[2:0]`

16-LD0-a-1: LD0/a, prefix `100110`



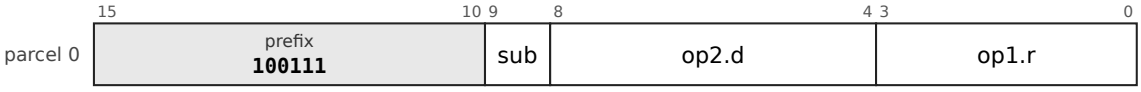
`op1` is `b4d0s-xsz`: memory at (base register, 4 bits).

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[9:8] op1.base=p0[7:4] op2.r=p0[3:0]`

16-STSP5-a-1: STSP5/a, prefix `100111`



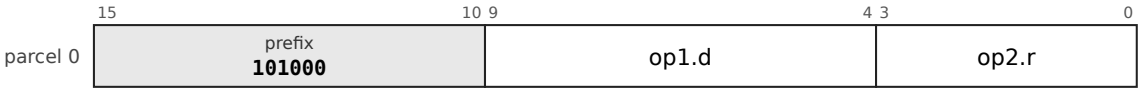
`op1` is `reg4`: register, 4 bits.

`op2` is `spd5u-xsz`: memory at sp + unsigned 5-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[9] op2.d=p0[8:4] op1.r=p0[3:0]`

16-LEASP6-a-1: LEASP6/a, prefix `101000`

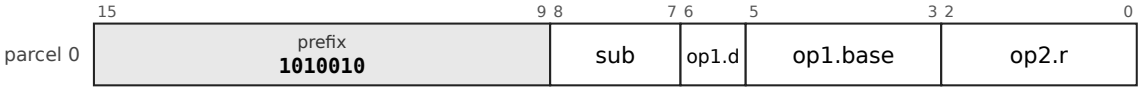


`op1` is `spd6u-x8`: memory at sp + unsigned 6-bit displacement times 8, 0 to 504 bytes.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.d=p0[9:4] op2.r=p0[3:0]`

16-LDHHH1-a-1: LDHHH1/a, prefix `1010010`



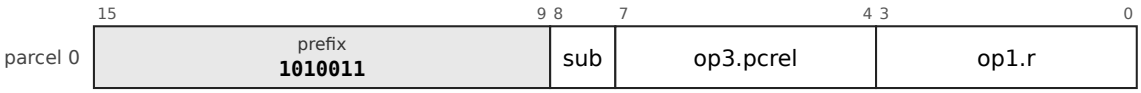
`op1` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[8:7] op1.d=p0[6] op1.base=p0[5:3] op2.r=p0[2:0]`

16-CBZ1-4-a-1: CBZ1_4/a, prefix `1010011`



`op1` is `reg4`: register, 4 bits.

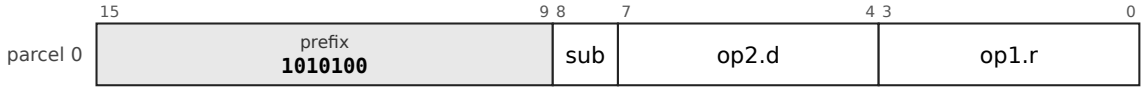
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc4-u2`: pc-relative, 4 bits counted in 2-byte units (-16 to 14 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[8] op3.pcrel=p0[7:4] op1.r=p0[3:0]`

16-STSP4-a-1: STSP4/a, prefix `1010100`



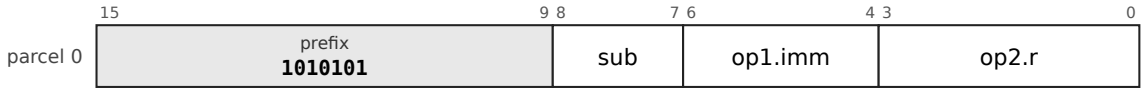
`op1` is `reg4`: register, 4 bits.

`op2` is `spd4u-xsz`: memory at sp + unsigned 4-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[8] op2.d=p0[7:4] op1.r=p0[3:0]`

16-SHI3-a-1: SHI3/a, prefix `1010101`



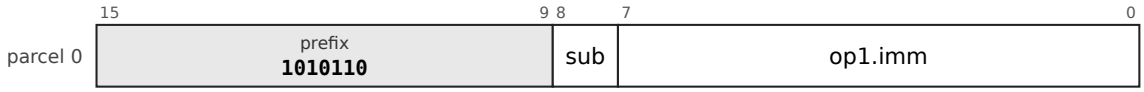
`op1` is `imm3u`: unsigned 3-bit immediate, 0 to 7.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `shl` 00, `shr` 01, `sar` 10, `rol` 11.

Fields: `sub=p0[8:7] op1.imm=p0[6:4] op2.r=p0[3:0]`

16-PUSHM8-a-1: PUSHM8/a, prefix `1010110`

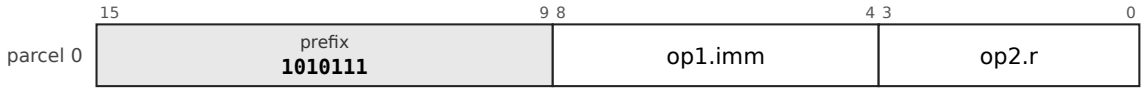


`op1` is `imm8u-x256`: unsigned 8-bit immediate times 256, 0 to 65280.

`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

Fields: `sub=p0[8] op1.imm=p0[7:0]`

16-IMM5-a-1: IMM5/a, prefix `1010111`

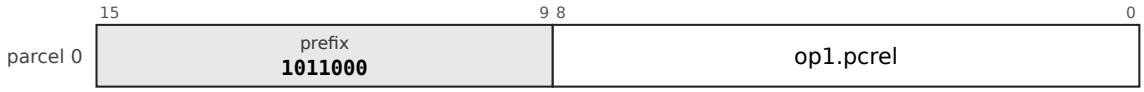


`op1` is `imm5s`: signed 5-bit immediate, -16 to 15.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.imm=p0[8:4] op2.r=p0[3:0]`

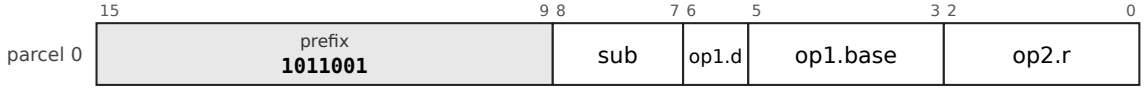
16-BR1-9-a-1: BR1_9/a, prefix `1011000`



`op1` is `pc9-u2`: pc-relative, 9 bits counted in 2-byte units (-512 to 510 bytes).

Fields: `op1.pcrel=p0[8:0]`

16-LDHHH1-b-1: LDHHH1/b, prefix `1011001`



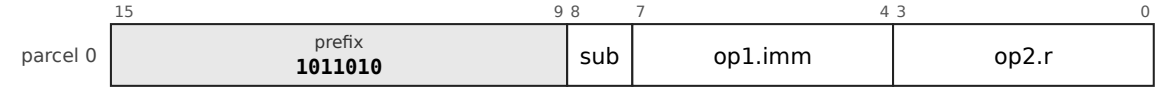
`op1` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[8:7] op1.d=p0[6] op1.base=p0[5:3] op2.r=p0[2:0]`

16-IMM4-b-1: IMM4/b, prefix `1011010`



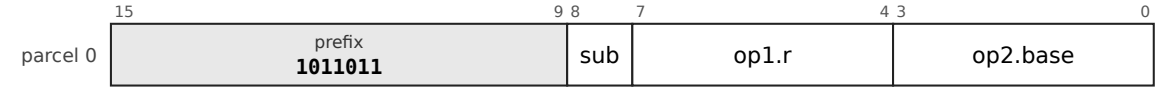
`op1` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[8] op1.imm=p0[7:4] op2.r=p0[3:0]`

16-ST0-a-1: ST0/a, prefix `1011011`



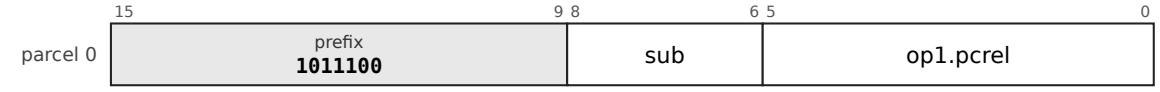
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d0s-xsz`: memory at (base register, 4 bits).

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[8] op1.r=p0[7:4] op2.base=p0[3:0]`

16-BR1-6-c-1: BR1_6/c, prefix `1011100`

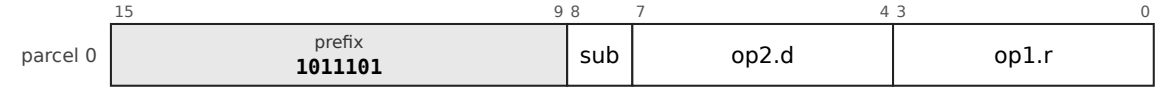


`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).

`sub` (3 bits) selects the mnemonic: `bpl` 000, `bmi` 001, `bvc` 010, `bvs` 011, `bcc` 100, `bcs` 101, `bge` 110, `blt` 111.

Fields: `sub=p0[8:6] op1.pcrel=p0[5:0]`

16-STSP4-b-1: STSP4/b, prefix `1011101`



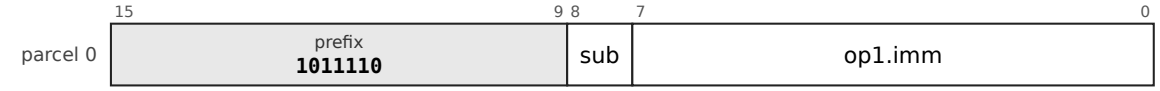
`op1` is `reg4`: register, 4 bits.

`op2` is `spd4u-xsz`: memory at sp + unsigned 4-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[8] op2.d=p0[7:4] op1.r=p0[3:0]`

16-SPADJ8-a-1: SPADJ8/a, prefix `1011110`



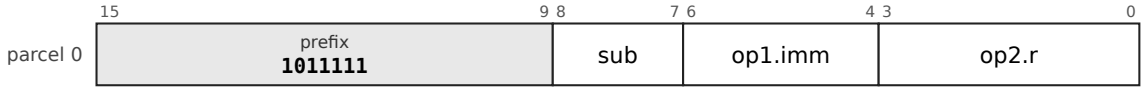
`op1` is `imm8u-x8`: unsigned 8-bit immediate times 8, 0 to 2040.

`op2` is `k-sp`: implicit register r14.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[8] op1.imm=p0[7:0]`

16-IMM3-c-1: IMM3/c, prefix 1011111



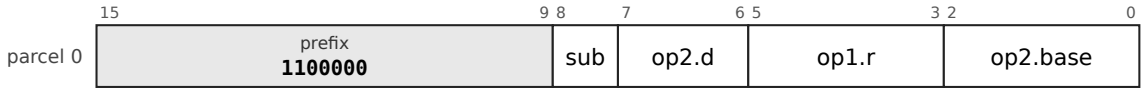
op1 is imm3s: signed 3-bit immediate, -4 to 3.

op2 is reg4: register, 4 bits.

sub (2 bits) selects the mnemonic: bis 00, xor 01, mul 10, bic 11.

Fields: sub=p0[8:7] op1.imm=p0[6:4] op2.r=p0[3:0]

16-STHHH2-b-1: STHHH2/b, prefix 1100000



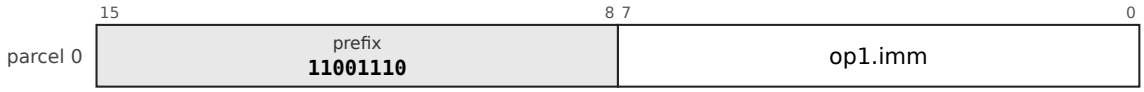
op1 is reg3: register, 3 bits (an index into the hot-register table).

op2 is b3d2u-xsz: memory at base register, 3 bits (an index into the hot-register table) + unsigned 2-bit displacement times the size of the access.

sub (1 bit) selects the mnemonic: movw 0, movb 1.

Fields: sub=p0[8] op2.d=p0[7:6] op1.r=p0[5:3] op2.base=p0[2:0]

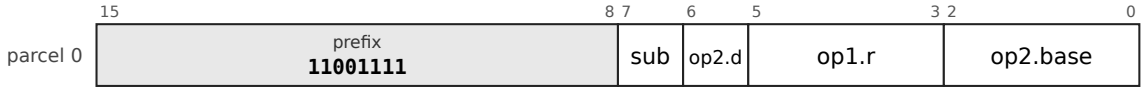
16-FX-POPMRET-1: FX.POPMRET, prefix 11001110



op1 is imm8u-x256: unsigned 8-bit immediate times 256, 0 to 65280.

Fields: op1.imm=p0[7:0]

16-STHHH1-a-1: STHHH1/a, prefix 11001111



op1 is reg3: register, 3 bits (an index into the hot-register table).

op2 is b3d1u-xsz: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

sub (1 bit) selects the mnemonic: movq 0, movl 1.

Fields: sub=p0[7] op2.d=p0[6] op1.r=p0[5:3] op2.base=p0[2:0]

16-LEASP4-a-1: LEASP4/a, prefix 11010000

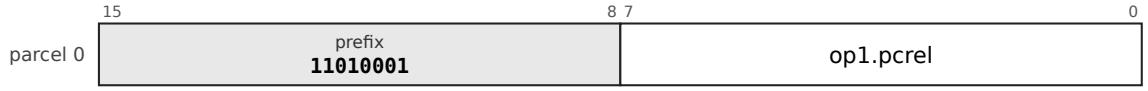


op1 is spd4u-x8: memory at sp + unsigned 4-bit displacement times 8, 0 to 120 bytes.

op2 is reg4: register, 4 bits.

Fields: op1.d=p0[7:4] op2.r=p0[3:0]

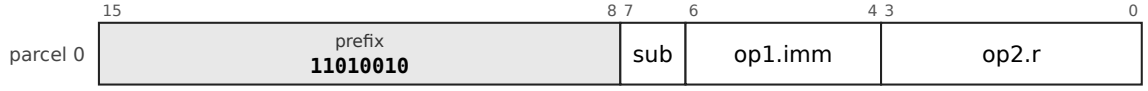
16-BR1-8-a-1: BR1_8/a, prefix `11010001`



`op1` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

Fields: `op1.pcrel=p0[7:0]`

16-IMM3-b-1: IMM3/b, prefix `11010010`



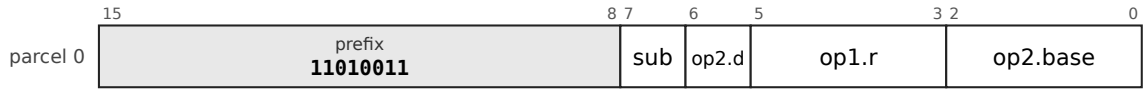
`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[7] op1.imm=p0[6:4] op2.r=p0[3:0]`

16-STHHH1-b-1: STHHH1/b, prefix `11010011`



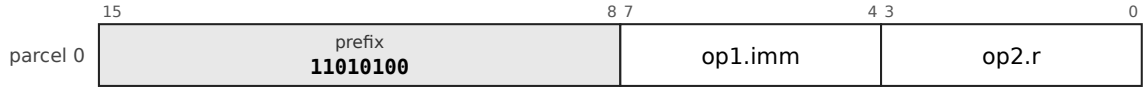
`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

`op2` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[7] op2.d=p0[6] op1.r=p0[5:3] op2.base=p0[2:0]`

16-IMM4-b-2: IMM4/b, prefix `11010100`



`op1` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.imm=p0[7:4] op2.r=p0[3:0]`

16-BR1-6-c-2: BR1_6/c, prefix `11010101`



`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).

`sub` (2 bits) selects the mnemonic: `bgt` 00, `ble` 01, `bhi` 10, `blos` 11.

Fields: `sub=p0[7:6] op1.pcrel=p0[5:0]`

16-BR1-7-b-1: BR1_7/b, prefix `11010111`



`op1` is `pc7-u2`: pc-relative, 7 bits counted in 2-byte units (-128 to 126 bytes).

`sub` (1 bit) selects the mnemonic: `beq` 0, `bne` 1.

Fields: `sub=p0[7] op1.pcrel=p0[6:0]`

16-SYSR-1: SYSR, prefix `110110100`



`op1` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

Fields: `sub=p0[6:4] op1.r=p0[3:0]`

16-SOPR-1: SOPR, prefix `111000001`



`op1` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `clr` 000, `com` 001, `inc` 010, `dec` 011, `neg` 100, `tst` 101, `adc` 110, `sbc` 111.

Fields: `sub=p0[6:4] op1.r=p0[3:0]`

16-IMM3-a-1: IMM3/a, prefix `111000010`



`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.imm=p0[6:4] op2.r=p0[3:0]`

16-SPADJ6-a-1: SPADJ6/a, prefix `111000100`



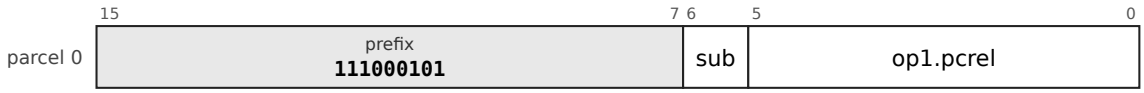
`op1` is `imm6u-x8`: unsigned 6-bit immediate times 8, 0 to 504.

`op2` is `k-sp`: implicit register r14.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[6] op1.imm=p0[5:0]`

16-BR1-6-b-1: BR1_6/b, prefix `111000101`



`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).

`sub` (1 bit) selects the mnemonic: `beq` 0, `bne` 1.

Fields: `sub=p0[6] op1.pcrel=p0[5:0]`

16-IMM3-b-2: IMM3/b, prefix `111000110`

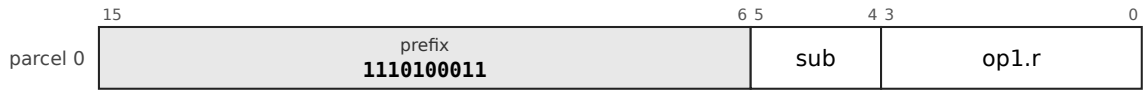


`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.imm=p0[6:4] op2.r=p0[3:0]`

16-SOPR-2: SOPR, prefix `1110100011`



`op1` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `asl` 00, `asr` 01, `push` 10, `pop` 11.

Fields: `sub=p0[5:4] op1.r=p0[3:0]`

16-BR1-6-a-1: BR1_6/a, prefix `1110100110`



`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).

Fields: `op1.pcrel=p0[5:0]`

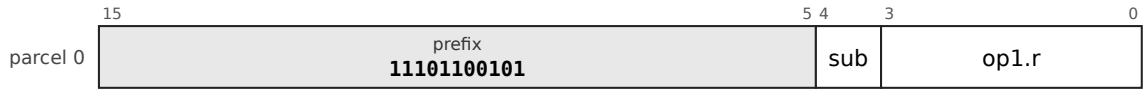
16-SYS0-1: SYS0, prefix `11101100100`



`sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

Fields: `sub=p0[4:0]`

16-SYSR-2: SYSR, prefix `11101100101`

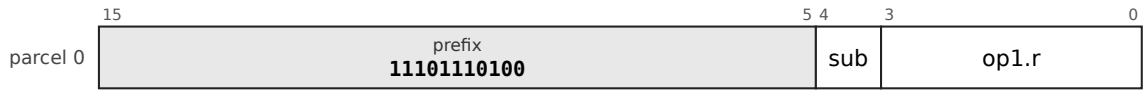


`op1` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.

Fields: `sub=p0[4] op1.r=p0[3:0]`

16-SOPR-3: SOPR, prefix `11101110100`

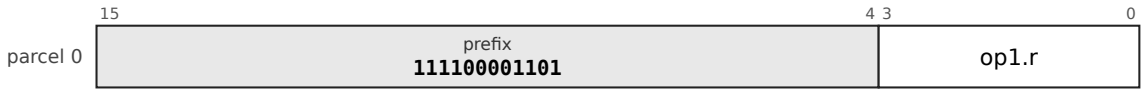


`op1` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `jmp` 0, `call` 1.

Fields: `sub=p0[4]` `op1.r=p0[3:0]`

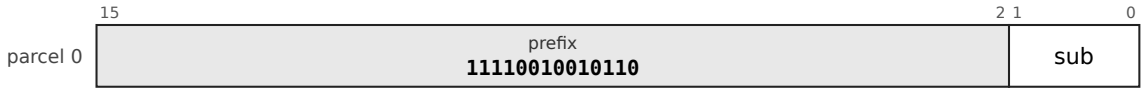
16-FX-POPRET-1: FX.POPRET, prefix `111100001101`



`op1` is `reg4`: register, 4 bits.

Fields: `op1.r=p0[3:0]`

16-SYS0-2: SYS0, prefix `11110010010110`

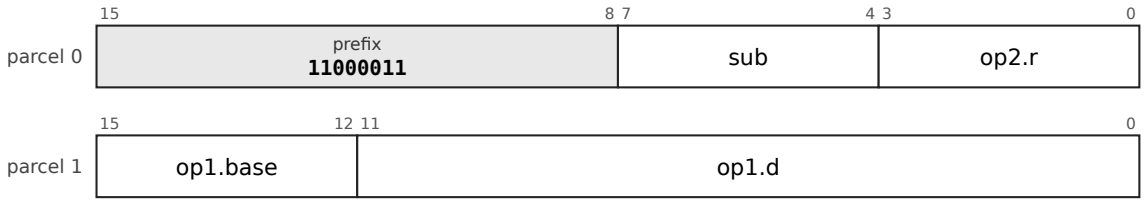


`sub` (2 bits) selects the mnemonic: `syspg2` 00, `syspg3` 01, `syspg4` 10, `syspg5` 11.

Fields: `sub=p0[1:0]`

Instructions of 2 parcels (4 bytes)

16-LDU-B12-1: LDU.B12, prefix `11000011`



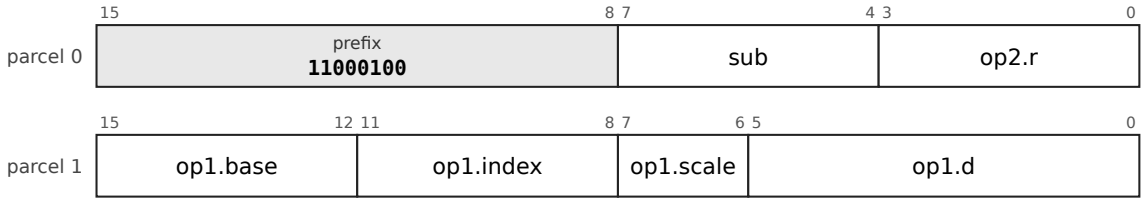
`op1` is `b4d12s-x1`: memory at base register, 4 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

Fields: `sub=p0[7:4]` `op2.r=p0[3:0]` `op1.base=p1[15:12]` `op1.d=p1[11:0]`

16-LDU-X6-1: LDU.X6, prefix `11000100`



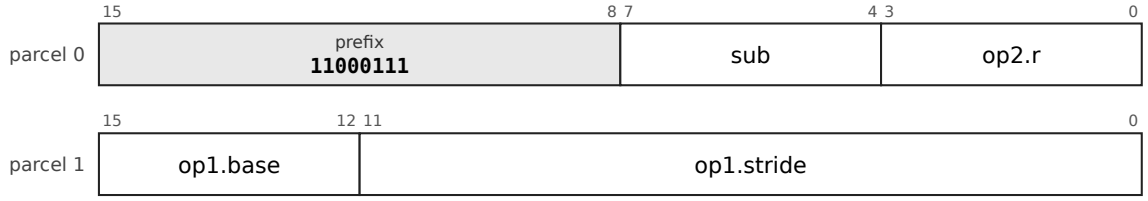
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

Fields: `sub=p0[7:4]` `op2.r=p0[3:0]` `op1.base=p1[15:12]` `op1.index=p1[11:8]` `op1.scale=p1[7:6]` `op1.d=p1[5:0]`

16-LDF-PI12-1: LDF.PI12, prefix `11000111`



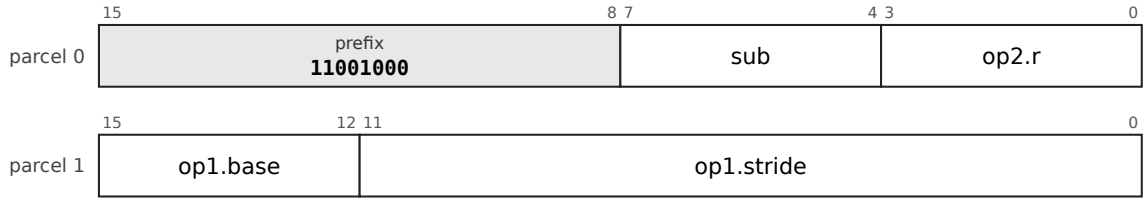
`op1` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-LDF-PD12-1: LDF.PD12, prefix `11001000`



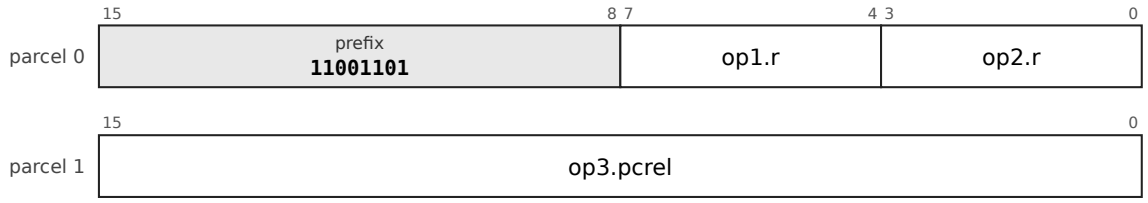
`op1` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-FX-MOVCALL2-1: FX.MOVCALL2, prefix `11001101`



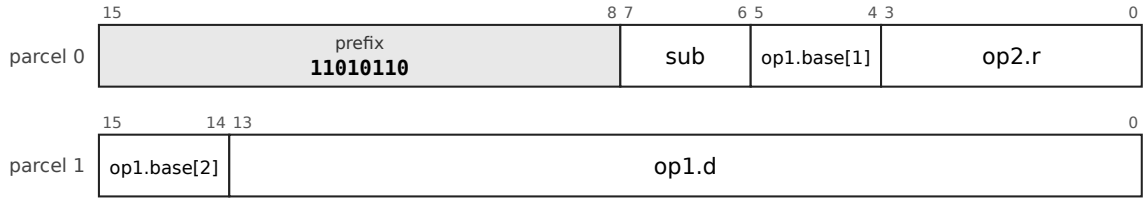
`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`op3` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.r=p0[7:4] op2.r=p0[3:0] op3.pcrel=p1[15:0]`

16-LDBs14-a-1: LDBs14/a, prefix `11010110`



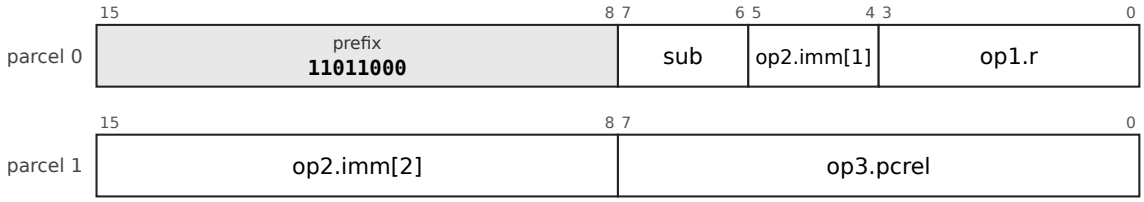
`op1` is `b4d14s-xsz`: memory at base register, 4 bits + signed 14-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[7:6] op1.base-1=p0[5:4] op2.r=p0[3:0] op1.base-2=p1[15:14] op1.d=p1[13:0]`

16-CBI10-8-c-1: CBI10_8/c, prefix `11011000`



`op1` is `reg4`: register, 4 bits.

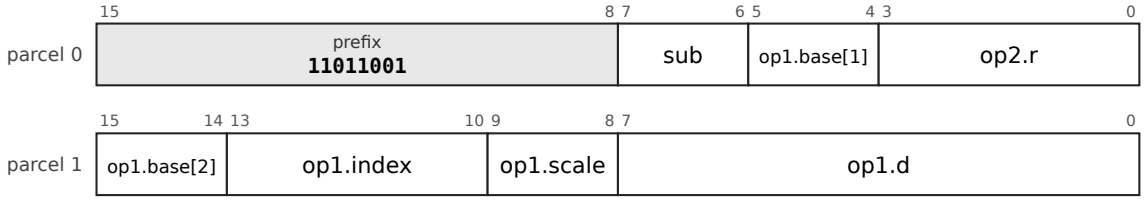
`op2` is `imm10s`: signed 10-bit immediate, -512 to 511.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[7:6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:8] op3.pcrel=p1[7:0]`

16-LDX8-a-1: LDX8/a, prefix `11011001`



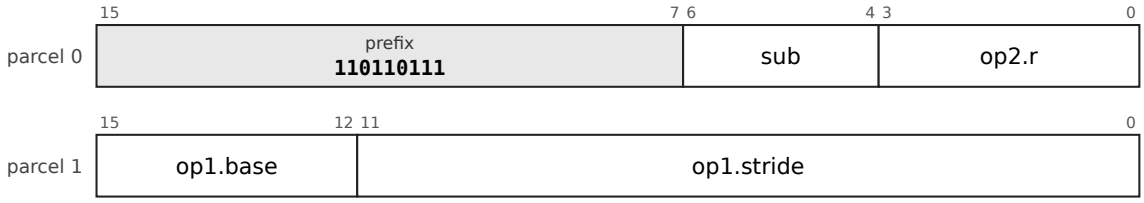
`op1` is `x4-4d8s`: memory at base + (index << scale) + signed 8-bit displacement; base register, 4 bits, index register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[7:6] op1.base-1=p0[5:4] op2.r=p0[3:0] op1.base-2=p1[15:14] op1.index=p1[13:10] op1.scale=p1[9:8] op1.d=p1[7:0]`

16-LDF-PI12-2: LDF.PI12, prefix `110110111`



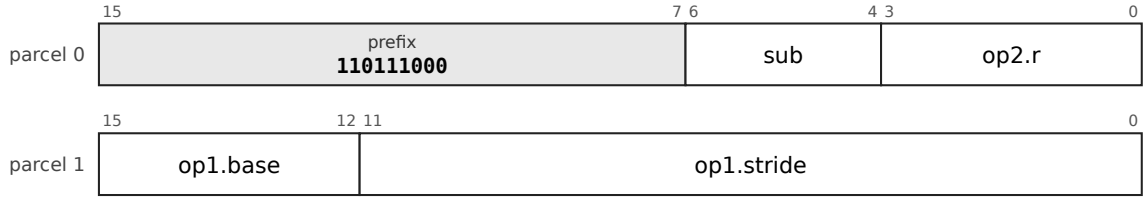
`op1` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).

`op2` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

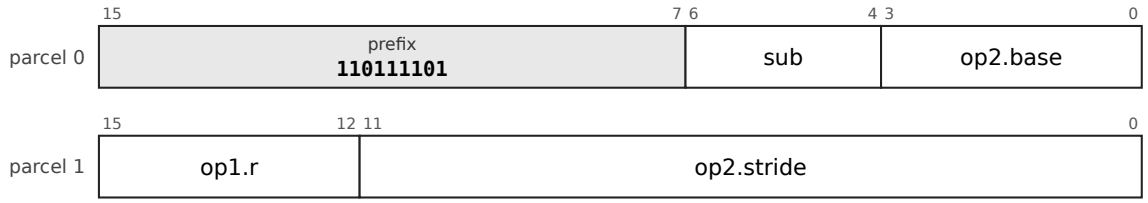
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-LDF-PD12-2: LDF.PD12, prefix `110111000`



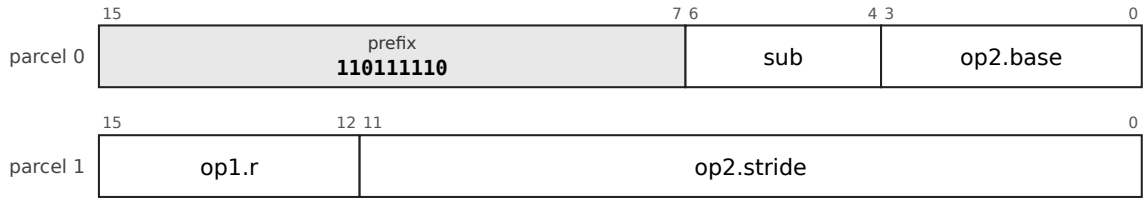
`op1` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`op2` is `reg4`: register, 4 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-STF-PI12-1: STF.PI12, prefix `110111101`



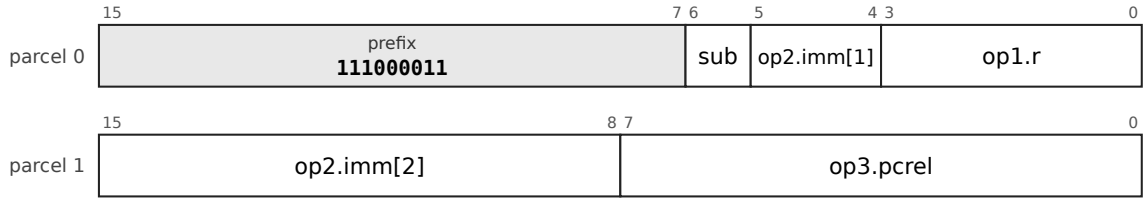
`op1` is `reg4`: register, 4 bits.
`op2` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[6:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]`

16-STF-PD12-1: STF.PD12, prefix `110111110`



`op1` is `reg4`: register, 4 bits.
`op2` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[6:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]`

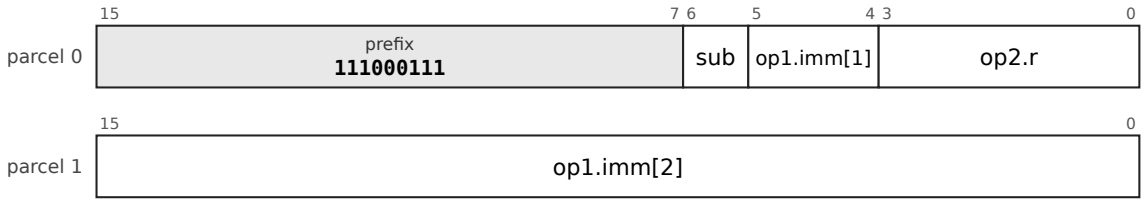
16-CBI10-8-a-1: CBI10_8/a, prefix `111000011`



`op1` is `reg4`: register, 4 bits.
`op2` is `imm10s`: signed 10-bit immediate, -512 to 511.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

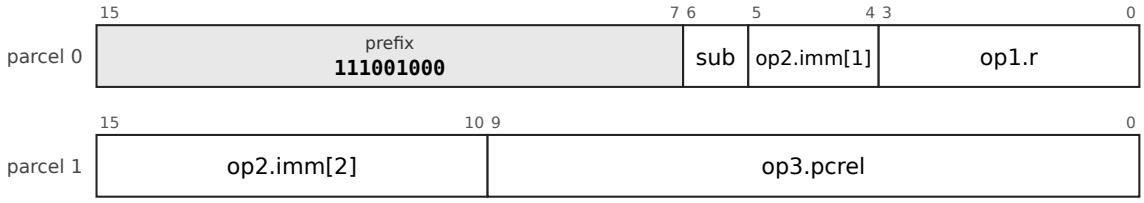
Fields: `sub=p0[6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:8] op3.pcrel=p1[7:0]`

16-LDI18-b-1: LDI18/b, prefix `111000111`



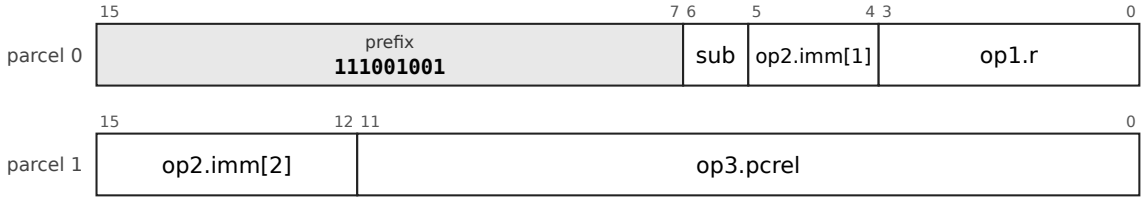
`op1` is `imm18s`: signed 18-bit immediate, -131072 to 131071.
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.
Fields: `sub=p0[6] op1.imm-1=p0[5:4] op2.r=p0[3:0] op1.imm-2=p1[15:0]`

16-CBI8-10-a-1: CBI8_10/a, prefix `111001000`



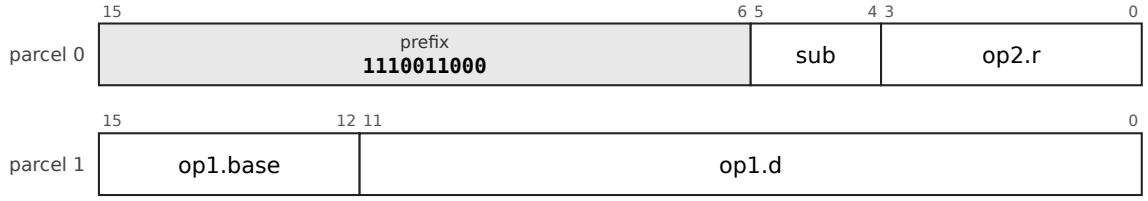
`op1` is `reg4`: register, 4 bits.
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:10] op3.pcrel=p1[9:0]`

16-CBI6-12-a-1: CBI6_12/a, prefix `111001001`



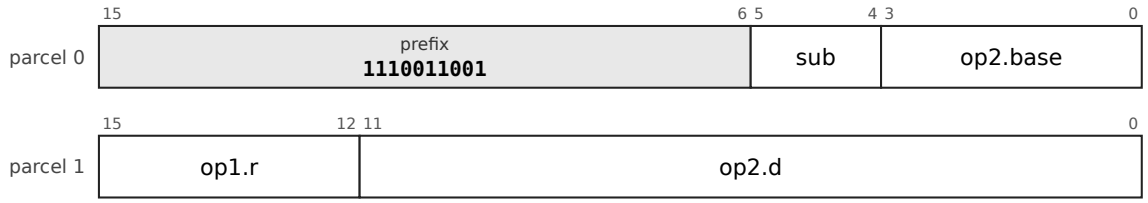
`op1` is `reg4`: register, 4 bits.
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.
`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:12] op3.pcrel=p1[11:0]`

16-LDU-B12-2: LDU.B12, prefix `1110011000`



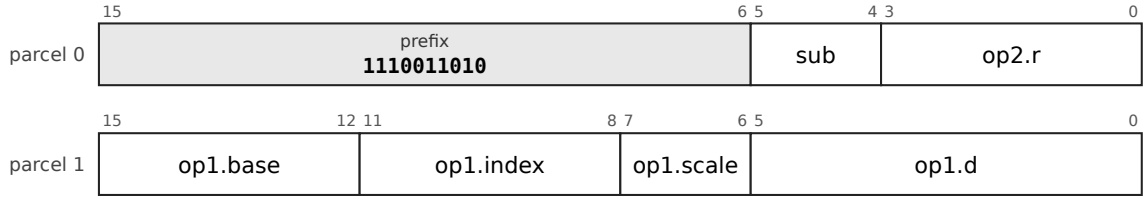
`op1` is `b4d12s-x1`: memory at base register, 4 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw.ud` 00, `movb.ud` 01, `movzl.ud` 10, `movzw.ud` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-STU-B12-1: STU.B12, prefix `1110011001`



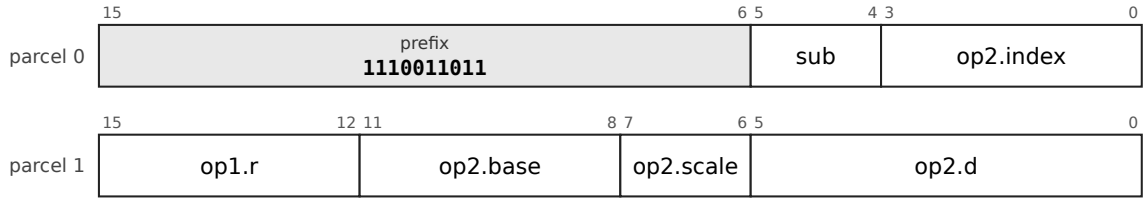
`op1` is `reg4`: register, 4 bits.
`op2` is `b4d12s-x1`: memory at base register, 4 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.
`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.
Fields: `sub=p0[5:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]`

16-LDU-X6-2: LDU.X6, prefix `1110011010`



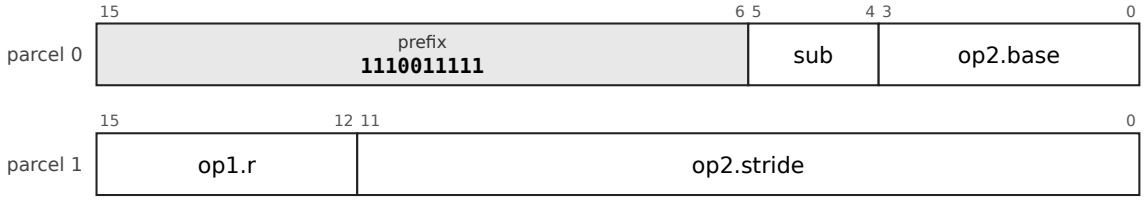
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw.ud` 00, `movb.ud` 01, `movzl.ud` 10, `movzw.ud` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-STU-X6-1: STU.X6, prefix `1110011011`



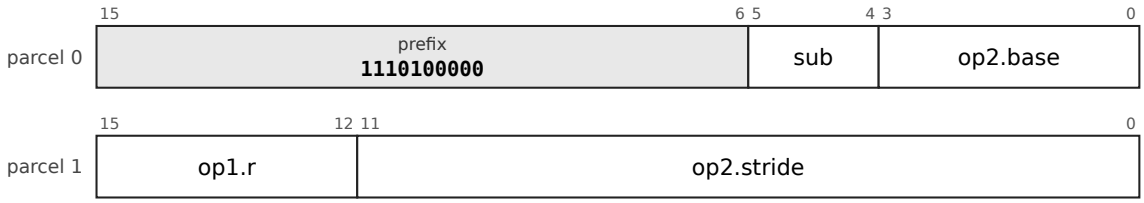
`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.
Fields: `sub=p0[5:4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]`

16-STF-PI12-2: STF.PI12, prefix 1110011111



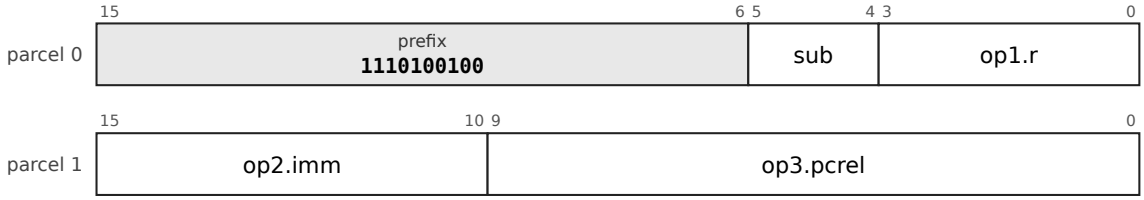
op1 is reg4: register, 4 bits.
op2 is pi4-s12: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
sub (2 bits) selects the mnemonic: addm 00, subm 01, bicm 10, bism 11.
Fields: sub=p0[5:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]

16-STF-PD12-2: STF.PD12, prefix 1110100000



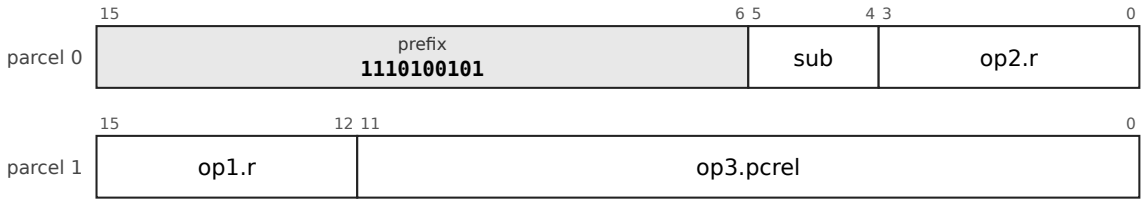
op1 is reg4: register, 4 bits.
op2 is pd4-s12: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
sub (2 bits) selects the mnemonic: addm 00, subm 01, bicm 10, bism 11.
Fields: sub=p0[5:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]

16-CBDI-1: CBDI, prefix 1110100100



op1 is reg4: register, 4 bits.
op2 is imm6s: signed 6-bit immediate, -32 to 31.
op3 is pc10-u2: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).
sub (2 bits) selects the mnemonic: cbpl 00, cbmi 01, cbvc 10, cbvs 11.
Fields: sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:10] op3.pcrel=p1[9:0]

16-CBDR-1: CBDR, prefix 1110100101

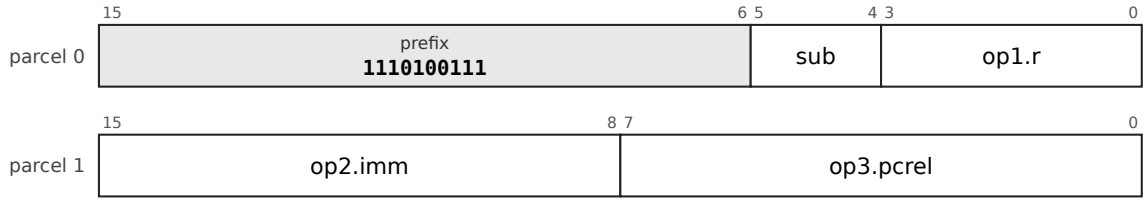


op1 is reg4: register, 4 bits.
op2 is reg4: register, 4 bits.
op3 is pc12-u2: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cbpl` 00, `cbmi` 01, `cbvc` 10, `cbvs` 11.

Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.r=p1[15:12] op3.pcrel=p1[11:0]`

16-CBI8-8-b-1: CBI8_8/b, prefix `1110100111`



`op1` is `reg4`: register, 4 bits.

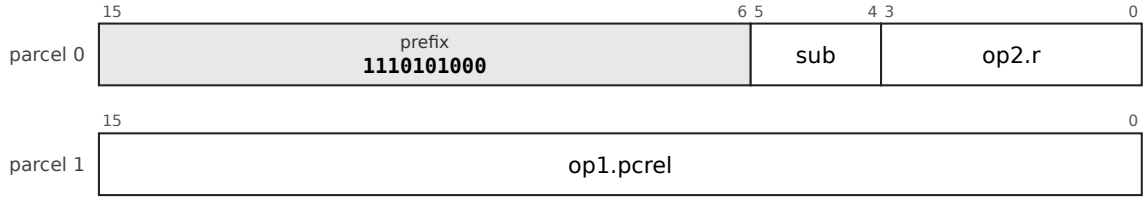
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:8] op3.pcrel=p1[7:0]`

16-LDPC16-a-1: LDPC16/a, prefix `1110101000`



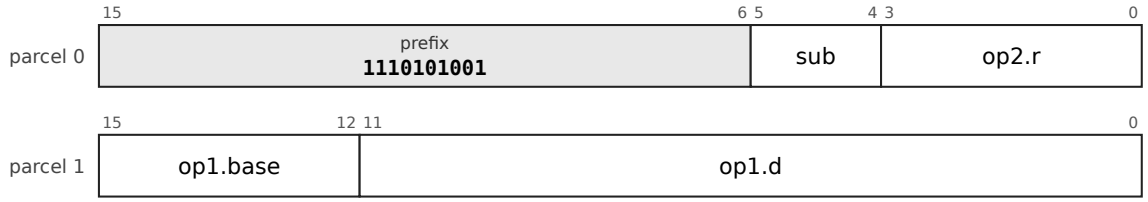
`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]`

16-LDBu12-a-1: LDBu12/a, prefix `1110101001`



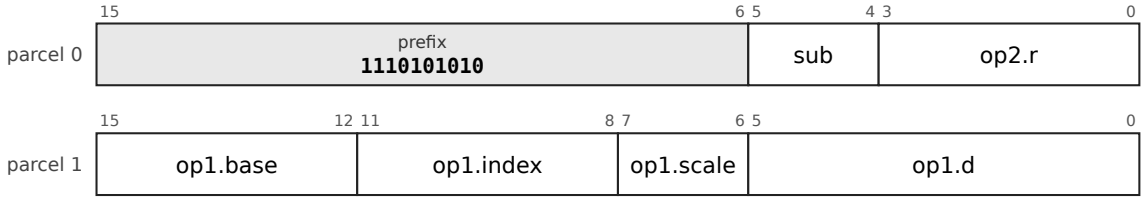
`op1` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

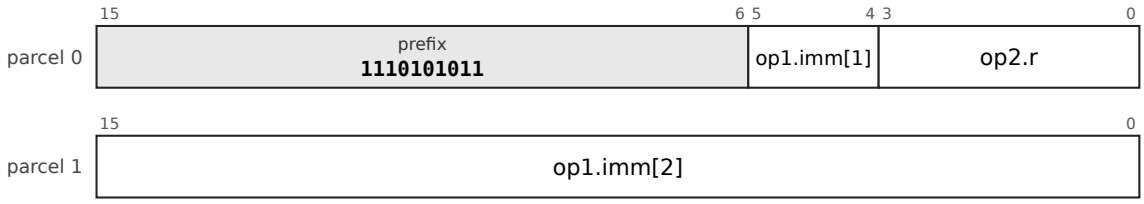
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-LDX6-a-1: LDX6/a, prefix `1110101010`



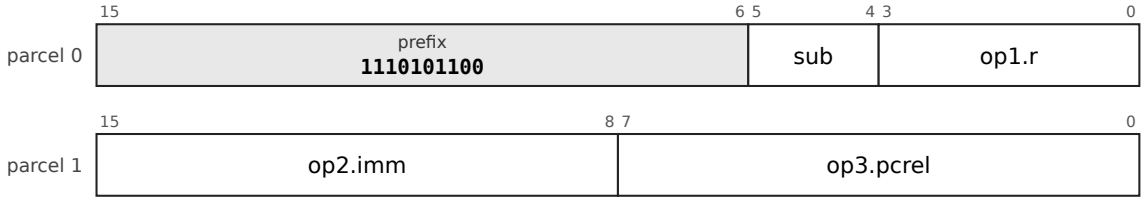
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-LDI18-b-2: LDI18/b, prefix `1110101011`



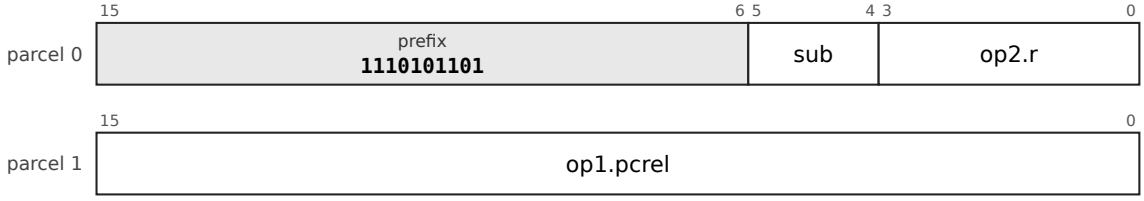
`op1` is `imm18s`: signed 18-bit immediate, -131072 to 131071.
`op2` is `reg4`: register, 4 bits.
Fields: `op1.imm-1=p0[5:4] op2.r=p0[3:0] op1.imm-2=p1[15:0]`

16-CBI8-8-c-1: CBI8_8/c, prefix `1110101100`



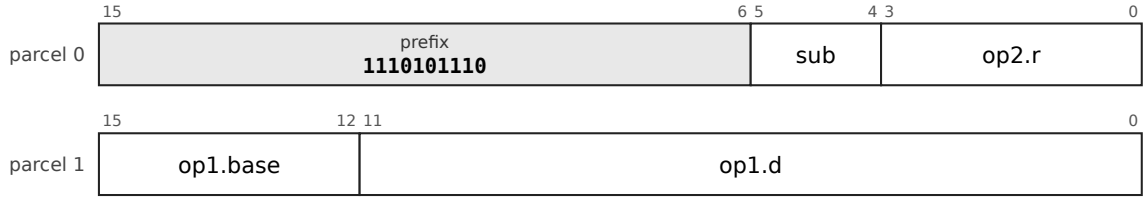
`op1` is `reg4`: register, 4 bits.
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:8] op3.pcrel=p1[7:0]`

16-LDPC16-c-1: LDPC16/c, prefix `1110101101`



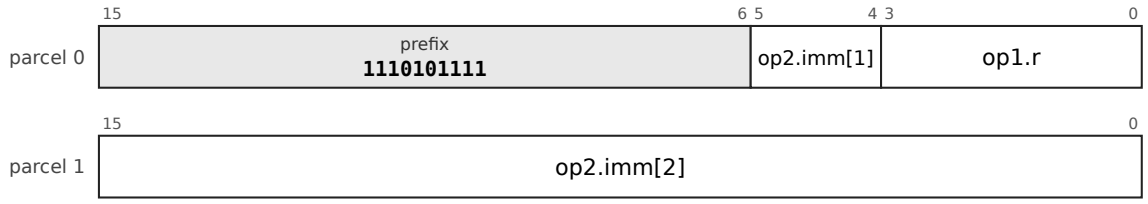
`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]`

16-LDBu12-b-1: LDBu12/b, prefix `1110101110`



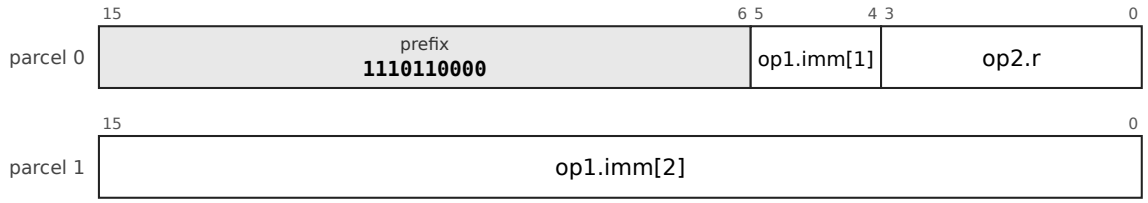
`op1` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-CMI18-a-1: CMI18/a, prefix `1110101111`



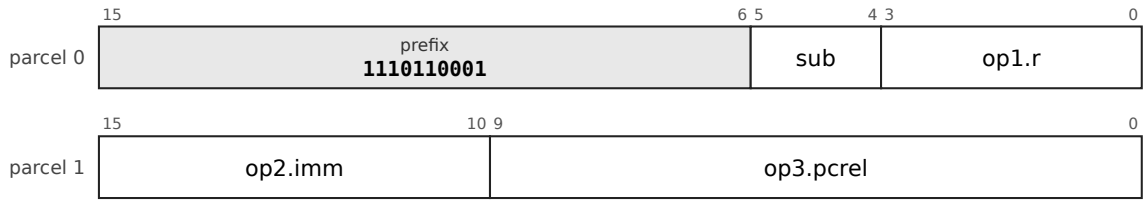
`op1` is `reg4`: register, 4 bits.
`op2` is `imm18s`: signed 18-bit immediate, -131072 to 131071.
Fields: `op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:0]`

16-LDI18-a-1: LDI18/a, prefix `1110110000`



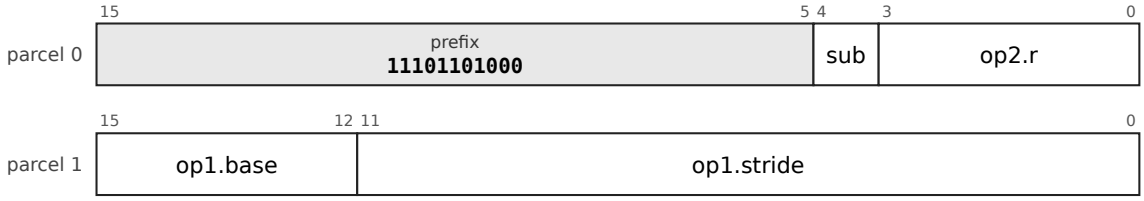
`op1` is `imm18s`: signed 18-bit immediate, -131072 to 131071.
`op2` is `reg4`: register, 4 bits.
Fields: `op1.imm-1=p0[5:4] op2.r=p0[3:0] op1.imm-2=p1[15:0]`

16-CBI6-10-c-1: CBI6_10/c, prefix `1110110001`



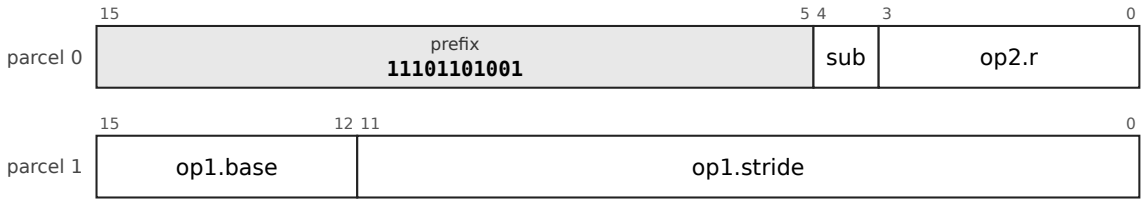
`op1` is `reg4`: register, 4 bits.
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.
`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:10] op3.pcrel=p1[9:0]`

16-LDF-PI12-3: LDF.PI12, prefix `11101101000`



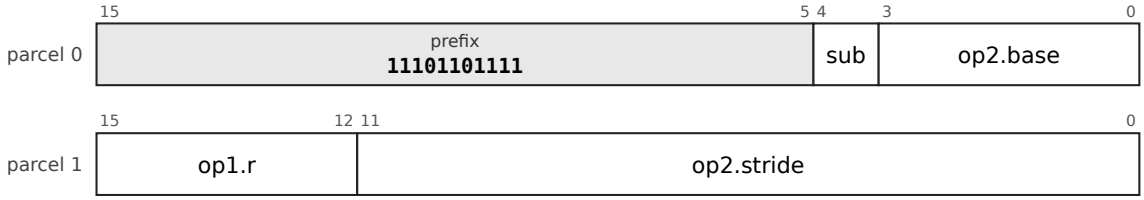
`op1` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-LDF-PD12-3: LDF.PD12, prefix `11101101001`



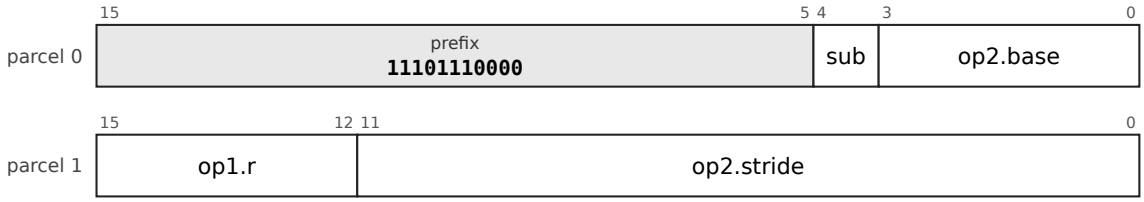
`op1` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[4] op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-CMF-PI12-1: CMF.PI12, prefix `11101101111`



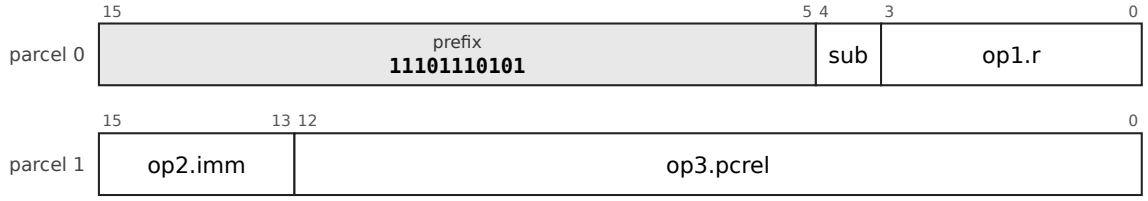
`op1` is `reg4`: register, 4 bits.
`op2` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]`

16-CMF-PD12-1: CMF.PD12, prefix `11101110000`



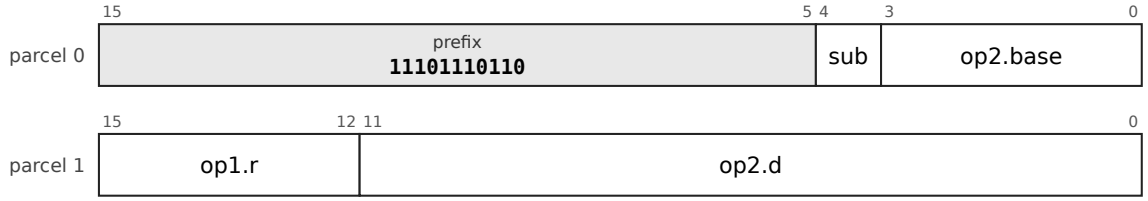
`op1` is `reg4`: register, 4 bits.
`op2` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.stride=p1[11:0]`

16-TAG-1: TAG, prefix `11101110101`



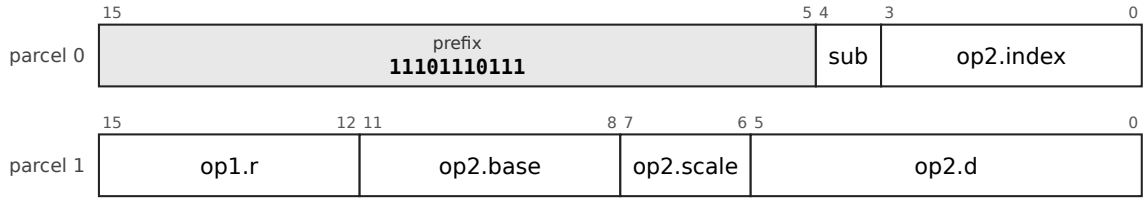
`op1` is `reg4`: register, 4 bits.
`op2` is `imm3u`: unsigned 3-bit immediate, 0 to 7.
`op3` is `pc13-u2`: pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes).
`sub` (1 bit) selects the mnemonic: `btag` 0, `bntag` 1.
Fields: `sub=p0[4] op1.r=p0[3:0] op2.imm=p1[15:13] op3.pcrel=p1[12:0]`

16-STBu12-a-1: STBu12/a, prefix `11101110110`



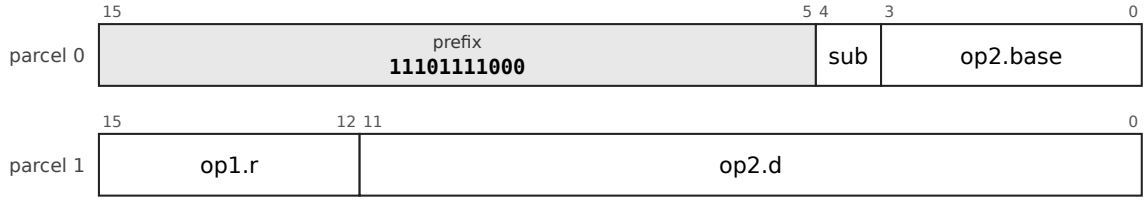
`op1` is `reg4`: register, 4 bits.
`op2` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.
Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]`

16-STX6-a-1: STX6/a, prefix `11101110111`



`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.
Fields: `sub=p0[4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]`

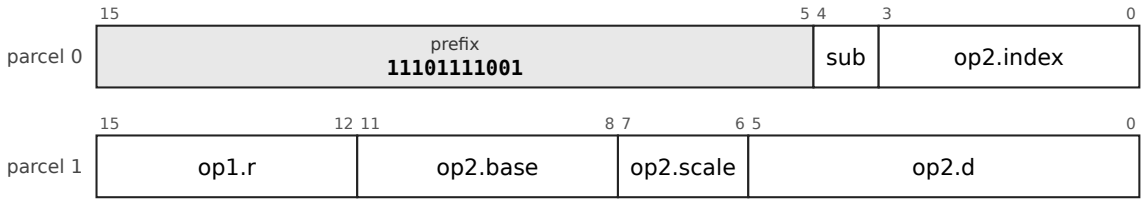
16-STBu12-b-1: STBu12/b, prefix `11101111000`



`op1` is `reg4`: register, 4 bits.
`op2` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

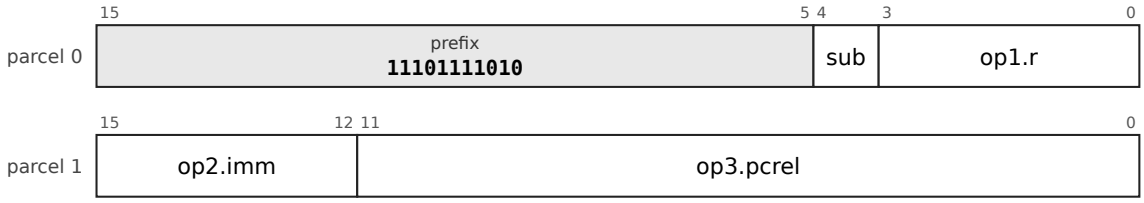
Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]`

16-STX6-b-1: STX6/b, prefix `11101111001`



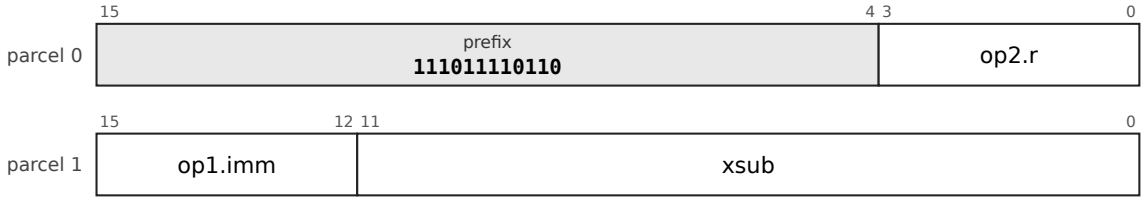
`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.
Fields: `sub=p0[4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]`

16-CBI4-12-a-1: CBI4_12/a, prefix `11101111010`



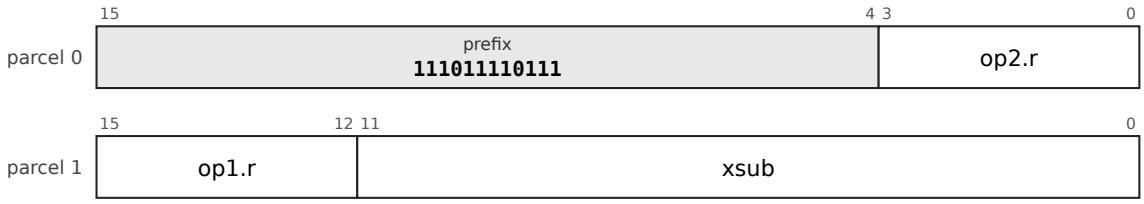
`op1` is `reg4`: register, 4 bits.
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.
`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[4] op1.r=p0[3:0] op2.imm=p1[15:12] op3.pcrel=p1[11:0]`

16-SETF-1: SETF, prefix `111011110110`



`op1` is `imm4u`: unsigned 4-bit immediate, 0 to 15.
`op2` is `reg4`: register, 4 bits.
`xsub` selects the mnemonic.
Fields: `op2.r=p0[3:0] op1.imm=p1[15:12] xsub=p1[11:0]`

16-R2-1: R2, prefix `111011110111`



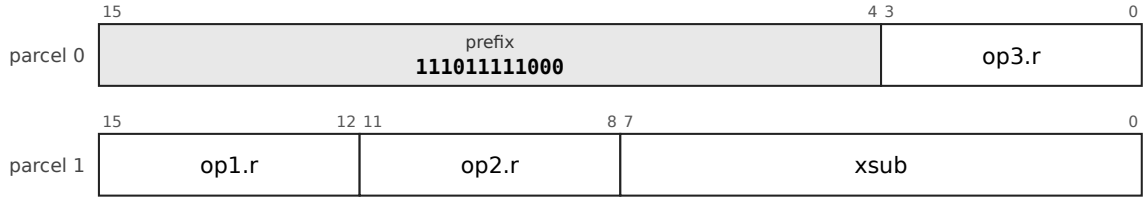
`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`xsub` selects the mnemonic.

Fields: `op2.r=p0[3:0] op1.r=p1[15:12] xsub=p1[11:0]`

16-R3-1: R3, prefix `111011111000`



`op1` is `reg4`: register, 4 bits.

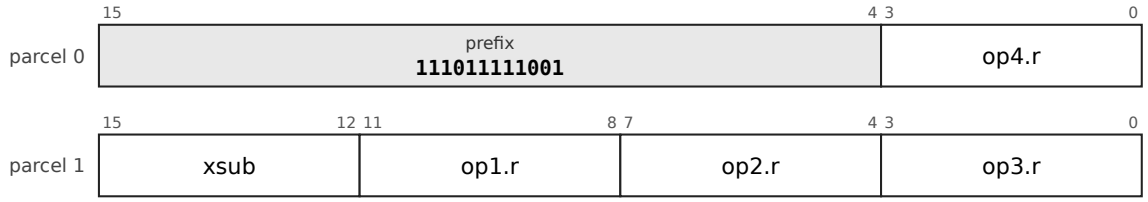
`op2` is `reg4`: register, 4 bits.

`op3` is `reg4`: register, 4 bits.

`xsub` selects the mnemonic.

Fields: `op3.r=p0[3:0] op1.r=p1[15:12] op2.r=p1[11:8] xsub=p1[7:0]`

16-R4-1: R4, prefix `111011111001`



`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

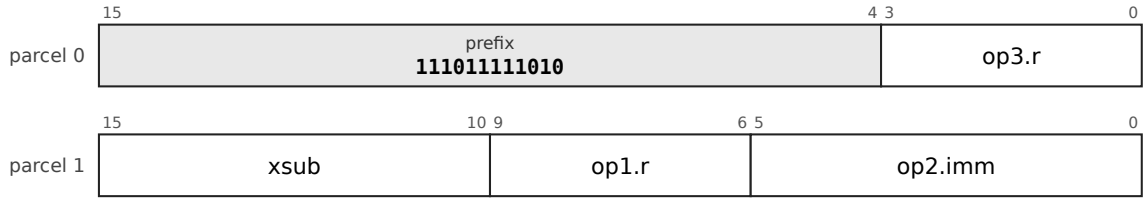
`op3` is `reg4`: register, 4 bits.

`op4` is `reg4`: register, 4 bits.

`xsub` selects the mnemonic.

Fields: `op4.r=p0[3:0] xsub=p1[15:12] op1.r=p1[11:8] op2.r=p1[7:4] op3.r=p1[3:0]`

16-RRI-1: RRI, prefix `111011111010`



`op1` is `reg4`: register, 4 bits.

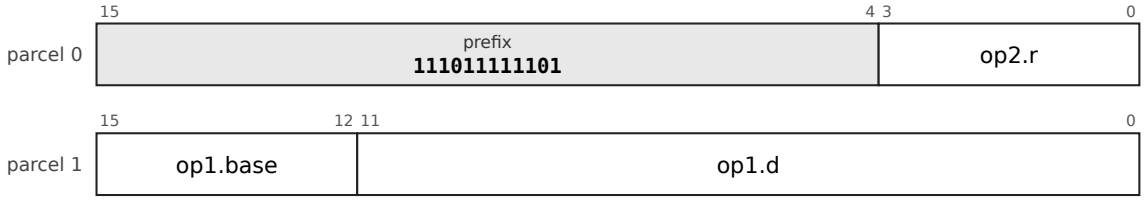
`op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63.

`op3` is `reg4`: register, 4 bits.

`xsub` selects the mnemonic.

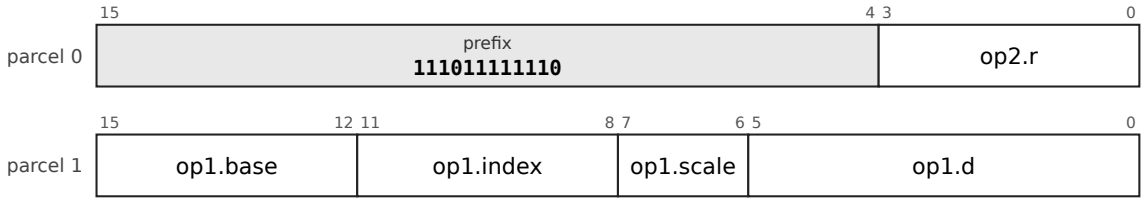
Fields: `op3.r=p0[3:0] xsub=p1[15:10] op1.r=p1[9:6] op2.imm=p1[5:0]`

16-LDU-B12-3: LDU.B12, prefix `111011111101`



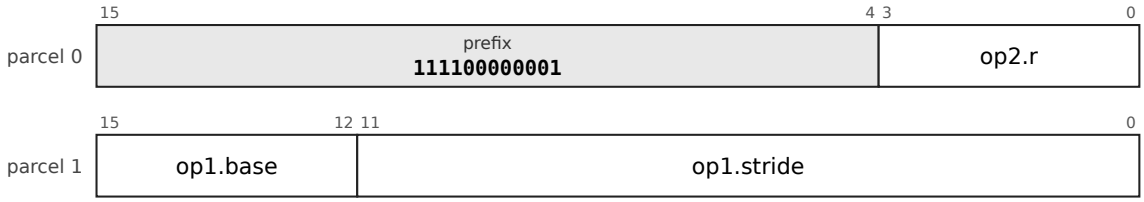
`op1` is `b4d12s-x1`: memory at base register, 4 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-LDU-X6-3: LDU.X6, prefix `111011111110`



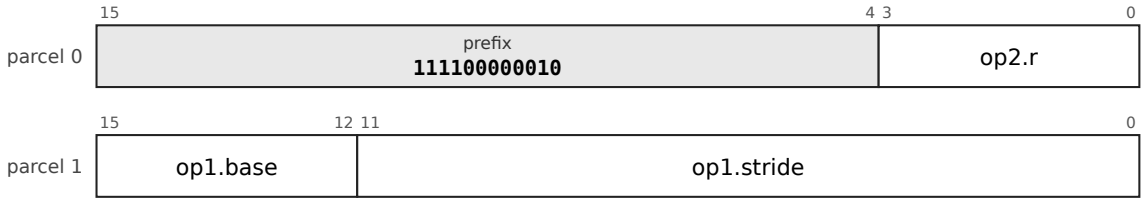
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-LDF-PI12-4: LDF.PI12, prefix `111100000001`



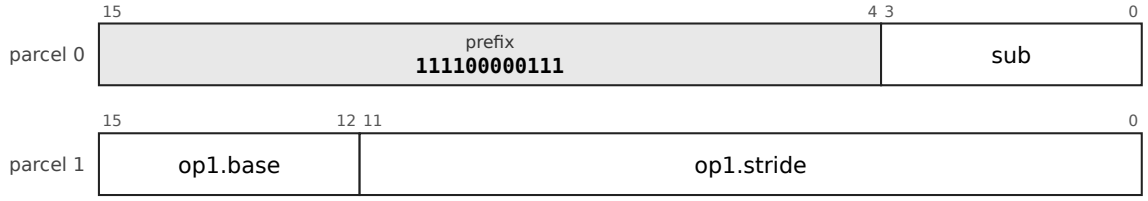
`op1` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-LDF-PD12-4: LDF.PD12, prefix `111100000010`



`op1` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-SOF-PI12-1: SOF.PI12, prefix `111100000111`

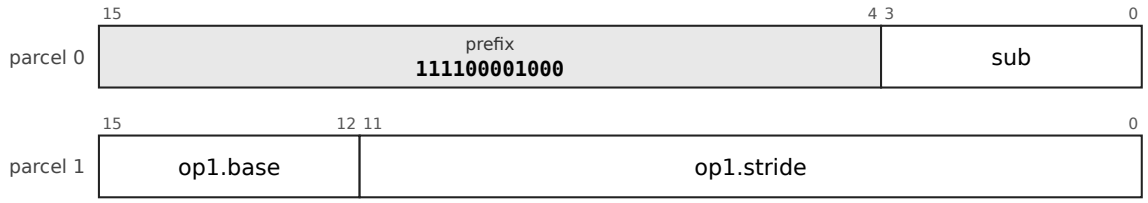


`op1` is `pi4-s12`: post-increment of a base register (register, 4 bits), 12-bit stride (0: the size of the access).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-SOF-PD12-1: SOF.PD12, prefix `111100001000`

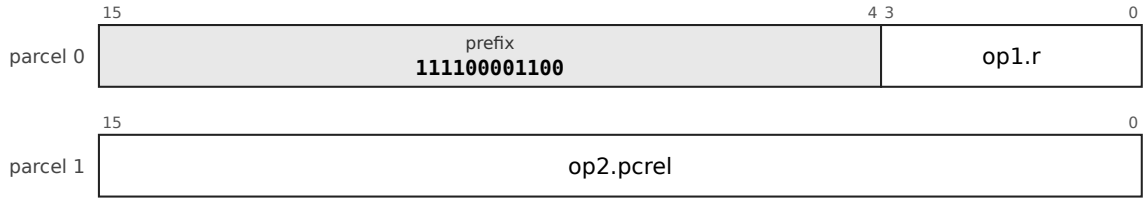


`op1` is `pd4-s12`: pre-decrement of a base register (register, 4 bits), 12-bit stride (0: the size of the access).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.base=p1[15:12] op1.stride=p1[11:0]`

16-SOB-1: SOB, prefix `111100001100`

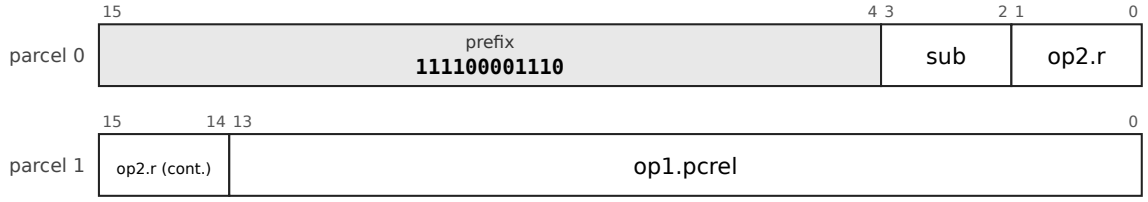


`op1` is `reg4`: register, 4 bits.

`op2` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.r=p0[3:0] op2.pcrel=p1[15:0]`

16-LDPC14-a-1: LDPC14/a, prefix `111100001110`



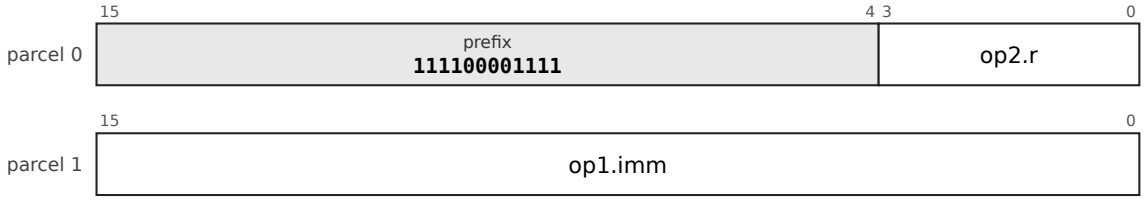
`op1` is `pc14-u1`: pc-relative, 14 bits counted in 1-byte units (-8192 to 8191 bytes).

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

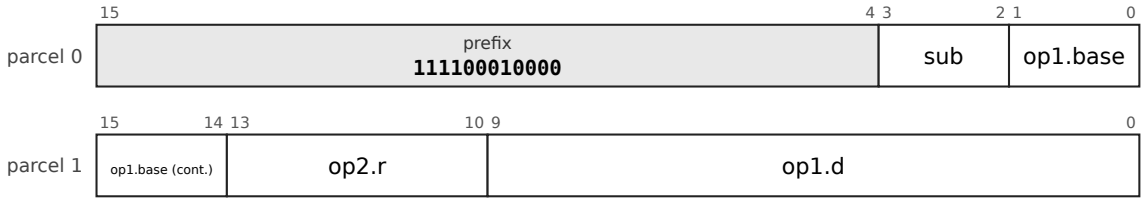
Fields: `sub=p0[3:2] op2.r=p0[1:0]+p1[15:14] op1.pcrel=p1[13:0]`

16-LDI16-a-1: LDI16/a, prefix `111100001111`



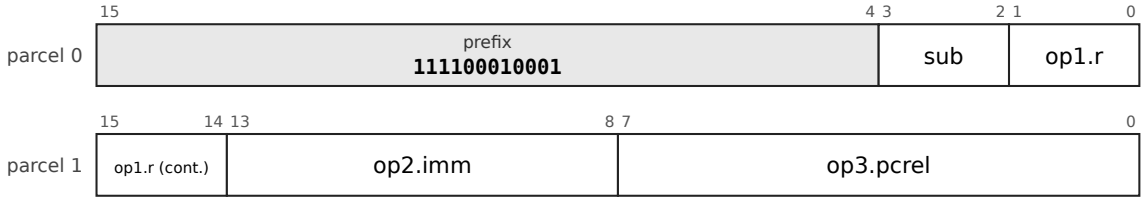
`op1` is `imm16s`: signed 16-bit immediate, -32768 to 32767.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.imm=p1[15:0]`

16-LDBu10-a-1: LDBu10/a, prefix `111100010000`



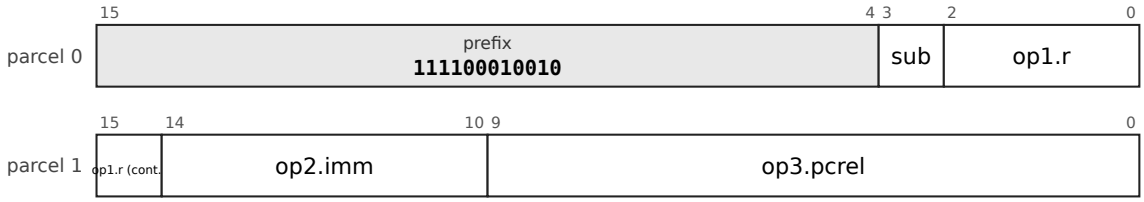
`op1` is `b4d10u-xsz`: memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.
Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:14] op2.r=p1[13:10] op1.d=p1[9:0]`

16-CBI6-8-c-1: CBI6_8/c, prefix `111100010001`



`op1` is `reg4`: register, 4 bits.
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:8] op3.pcrel=p1[7:0]`

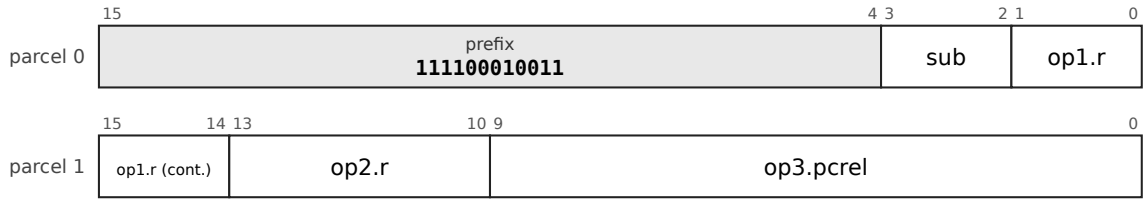
16-CBI5-10-a-1: CBI5_10/a, prefix `111100010010`



`op1` is `reg4`: register, 4 bits.
`op2` is `imm5s`: signed 5-bit immediate, -16 to 15.
`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[3] op1.r=p0[2:0]+p1[15] op2.imm=p1[14:10] op3.pcrel=p1[9:0]`

16-CBR10-c-1: CBR10/c, prefix `111100010011`



`op1` is `reg4`: register, 4 bits.

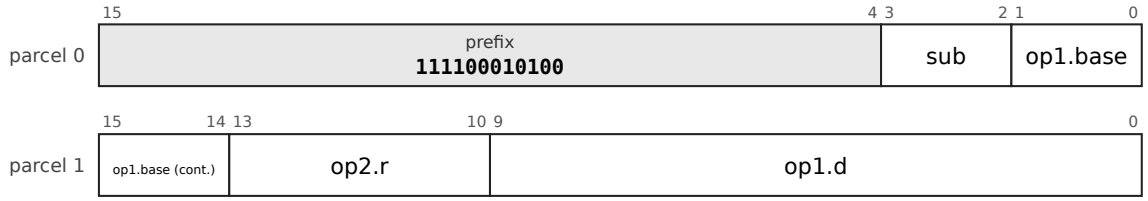
`op2` is `reg4`: register, 4 bits.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.r=p1[13:10] op3.pcrel=p1[9:0]`

16-LDBu10-b-1: LDBu10/b, prefix `111100010100`



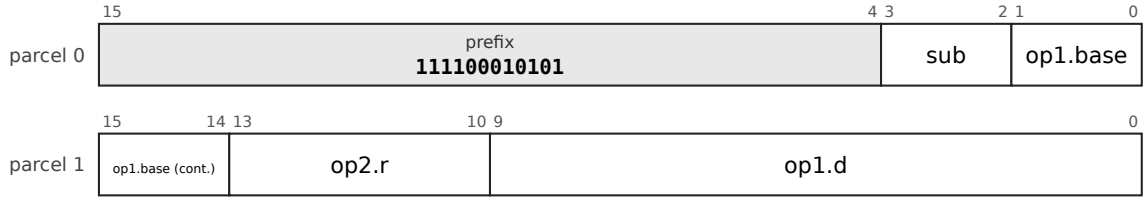
`op1` is `b4d10u-xsz`: memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:14] op2.r=p1[13:10] op1.d=p1[9:0]`

16-LDBs10-a-1: LDBs10/a, prefix `111100010101`



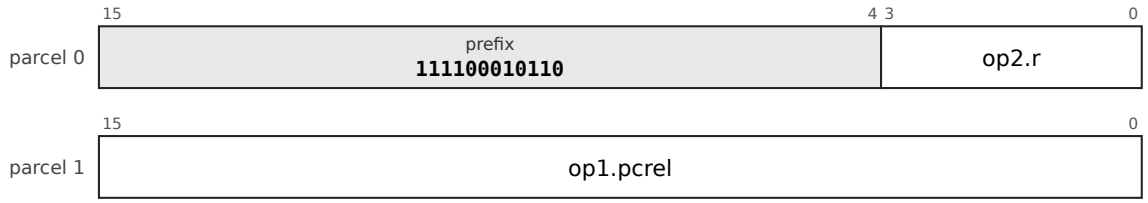
`op1` is `b4d10s-xsz`: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:14] op2.r=p1[13:10] op1.d=p1[9:0]`

16-LDPC16-c-2: LDPC16/c, prefix `111100010110`

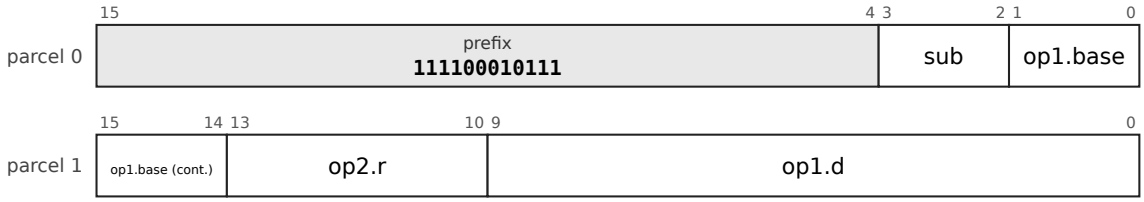


`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).

op2 is reg4: register, 4 bits.

Fields: op2.r=p0[3:0] op1.pcrel=p1[15:0]

16-LDBs10-c-1: LDBs10/c, prefix 111100010111



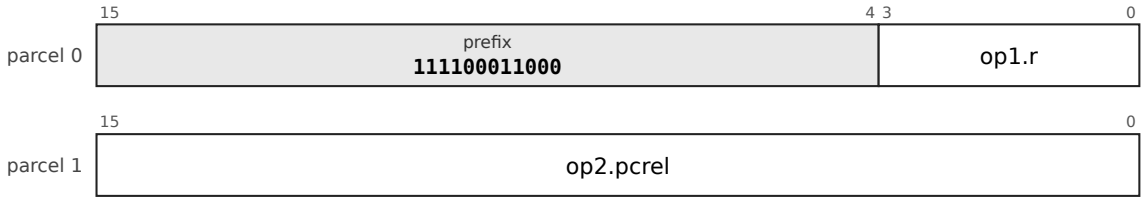
op1 is b4d10s-xsz: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.

op2 is reg4: register, 4 bits.

sub (2 bits) selects the mnemonic: add 00, sub 01, and 10, bis 11.

Fields: sub=p0[3:2] op1.base=p0[1:0]+p1[15:14] op2.r=p1[13:10] op1.d=p1[9:0]

16-CMPC16-a-1: CMPC16/a, prefix 111100011000

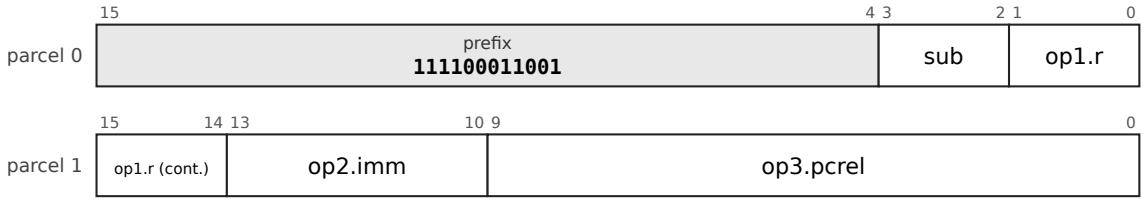


op1 is reg4: register, 4 bits.

op2 is pc16-u1: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).

Fields: op1.r=p0[3:0] op2.pcrel=p1[15:0]

16-CBI4-10-c-1: CBI4_10/c, prefix 111100011001



op1 is reg4: register, 4 bits.

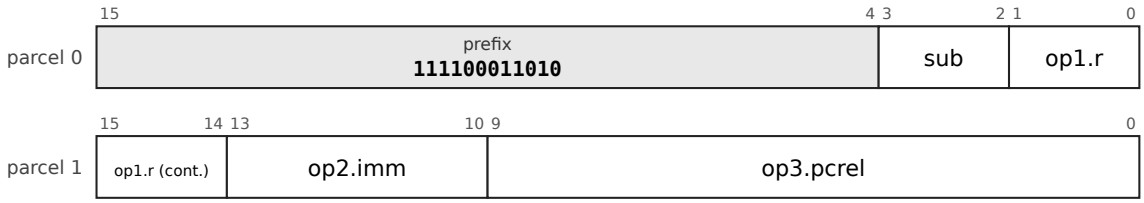
op2 is imm4s: signed 4-bit immediate, -8 to 7.

op3 is pc10-u2: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

sub (2 bits) selects the mnemonic: cbcs 00, cbcc 01, cbhi 10, cblos 11.

Fields: sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:10] op3.pcrel=p1[9:0]

16-CBI4-10-b-1: CBI4_10/b, prefix 111100011010



op1 is reg4: register, 4 bits.

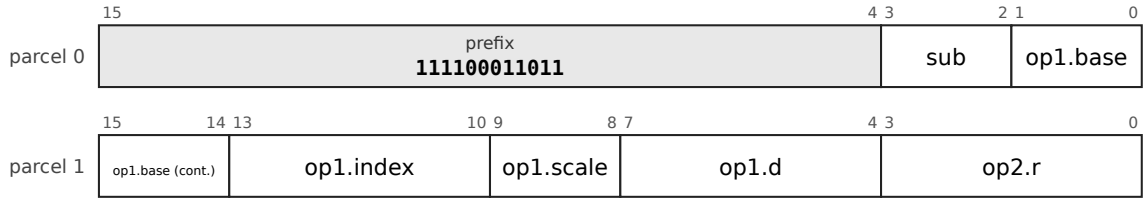
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:10] op3.pcrel=p1[9:0]`

16-LDX4-b-1: LDX4/b, prefix `111100011011`



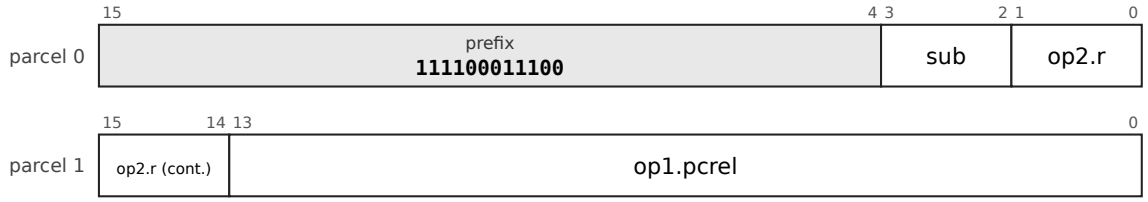
`op1` is `x4-4d4s`: memory at base + (index << scale) + signed 4-bit displacement; base register, 4 bits, index register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:14] op1.index=p1[13:10] op1.scale=p1[9:8] op1.d=p1[7:4] op2.r=p1[3:0]`

16-LDPC14-b-1: LDPC14/b, prefix `111100011100`



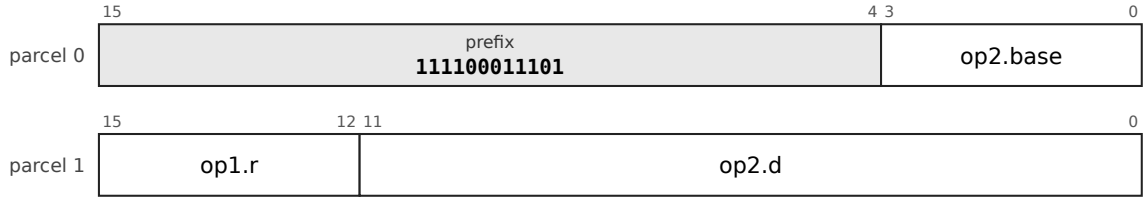
`op1` is `pc14-u1`: pc-relative, 14 bits counted in 1-byte units (-8192 to 8191 bytes).

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[3:2] op2.r=p0[1:0]+p1[15:14] op1.pcrel=p1[13:0]`

16-CMBu12-a-1: CMBu12/a, prefix `111100011101`

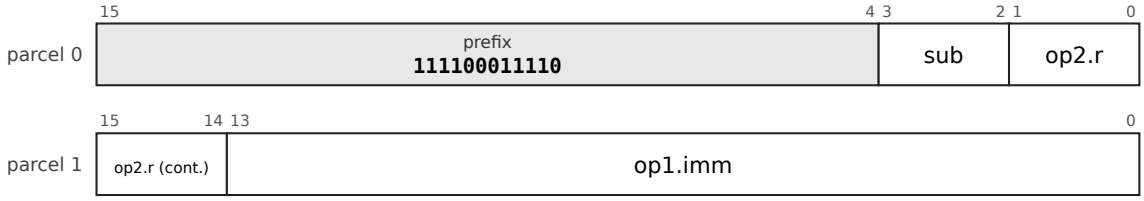


`op1` is `reg4`: register, 4 bits.

`op2` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.

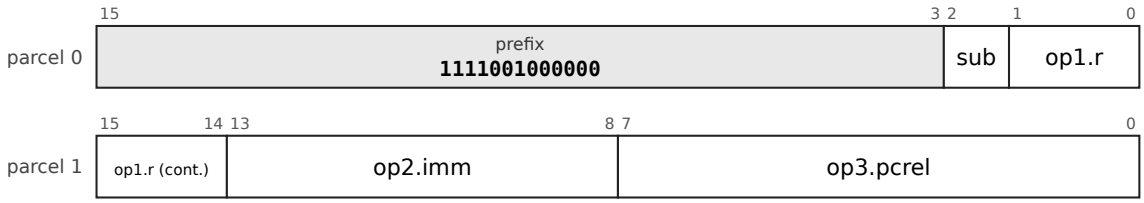
Fields: `op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]`

16-LDI14-c-1: LDI14/c, prefix `111100011110`



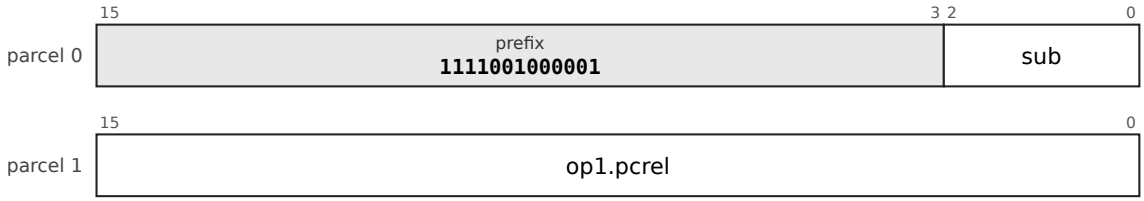
`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `bis` 00, `xor` 01, `mul` 10, `bic` 11.
Fields: `sub=p0[3:2] op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

16-CBI6-8-a-1: CBI6_8/a, prefix `1111001000000`



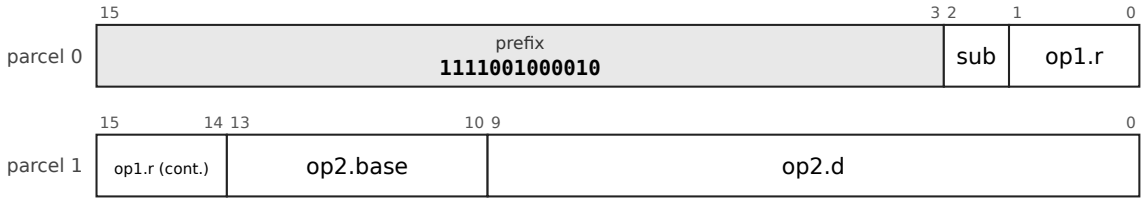
`op1` is `reg4`: register, 4 bits.
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:8] op3.pcrel=p1[7:0]`

16-BR2-16-c-1: BR2_16/c, prefix `1111001000001`



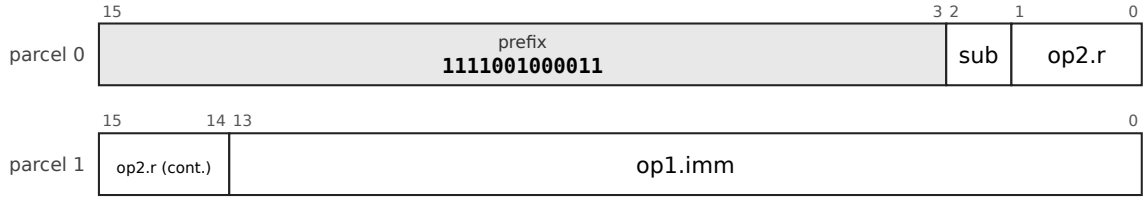
`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).
`sub` (3 bits) selects the mnemonic: `bpl` 000, `bmi` 001, `bvc` 010, `bvs` 011, `bcc` 100, `bcs` 101, `bge` 110, `blt` 111.
Fields: `sub=p0[2:0] op1.pcrel=p1[15:0]`

16-STBu10-a-1: STBu10/a, prefix `1111001000010`



`op1` is `reg4`: register, 4 bits.
`op2` is `b4d10u-xsz`: memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.
Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-LDI14-b-1: LDI14/b, prefix `1111001000011`



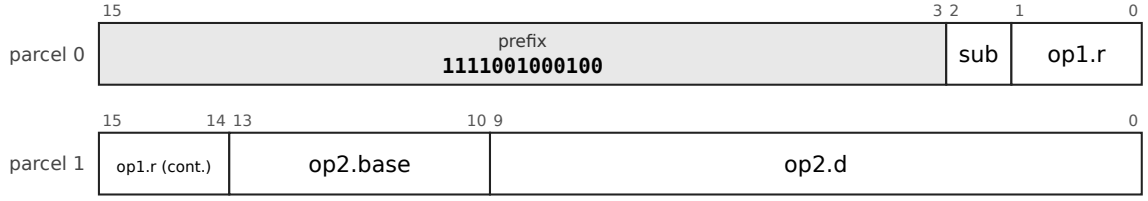
`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[2] op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

16-STBu10-b-1: STBu10/b, prefix `1111001000100`



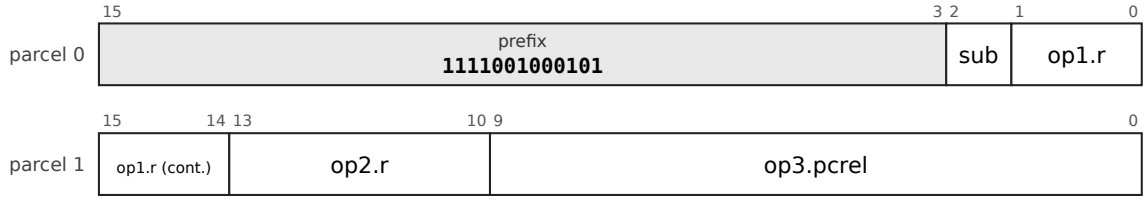
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d10u-xsz`: memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-CBR10-a-1: CBR10/a, prefix `1111001000101`



`op1` is `reg4`: register, 4 bits.

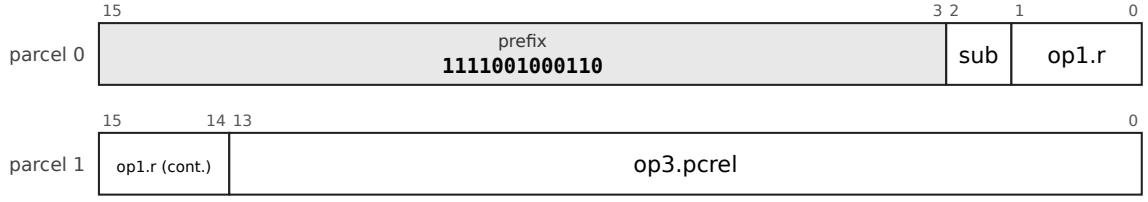
`op2` is `reg4`: register, 4 bits.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.r=p1[13:10] op3.pcrel=p1[9:0]`

16-CBZ2-14-a-1: CBZ2_14/a, prefix `1111001000110`



`op1` is `reg4`: register, 4 bits.

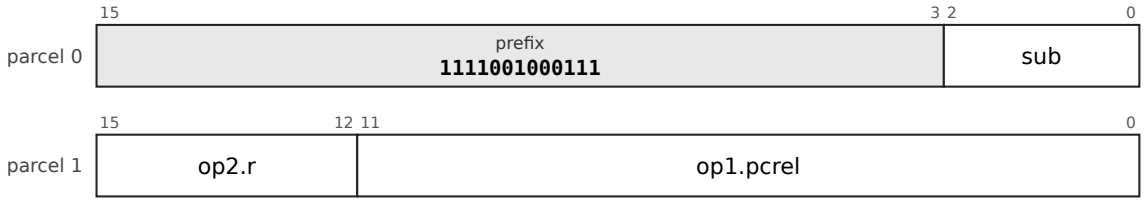
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc14-u2`: pc-relative, 14 bits counted in 2-byte units (-16384 to 16382 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op3.pcrel=p1[13:0]`

16-LDPC12-d-1: LDPC12/d, prefix `1111001000111`



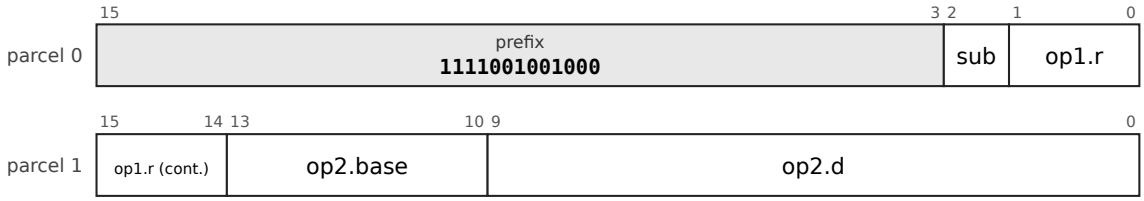
`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).

`op2` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `bic` 000, `mul` 001, `mulh` 010, `shl` 011, `shr` 100, `sar` 101, `fadd` 110, `fsub` 111.

Fields: `sub=p0[2:0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

16-STBs10-a-1: STBs10/a, prefix `1111001001000`



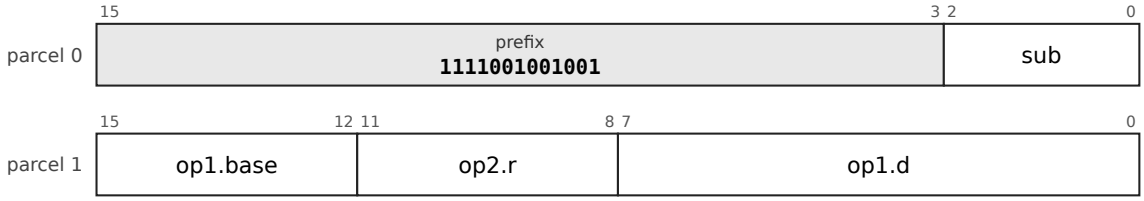
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d10s-xsz`: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-LDBs8-d-1: LDBs8/d, prefix `1111001001001`



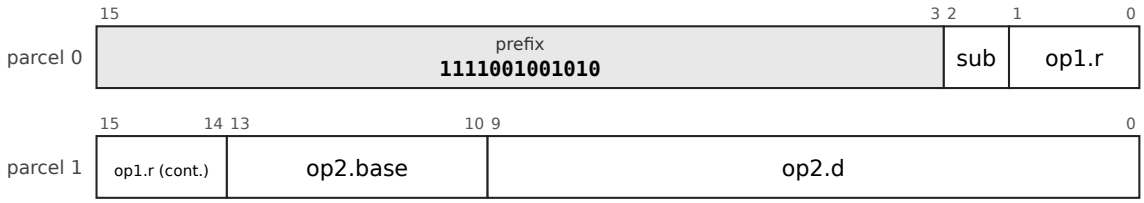
`op1` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `bic` 000, `mul` 001, `mulh` 010, `shl` 011, `shr` 100, `sar` 101, `fadd` 110, `fsub` 111.

Fields: `sub=p0[2:0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

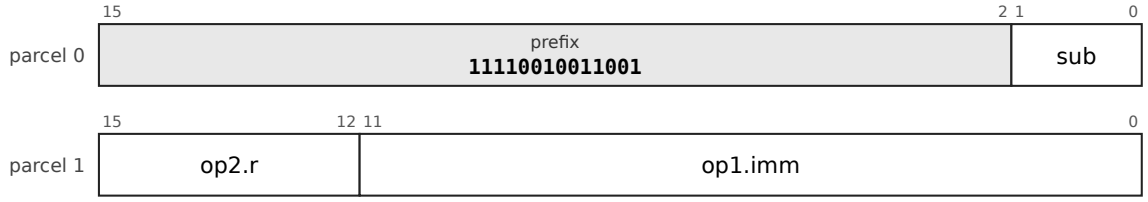
16-STBs10-b-1: STBs10/b, prefix `1111001001010`



`op1` is `reg4`: register, 4 bits.

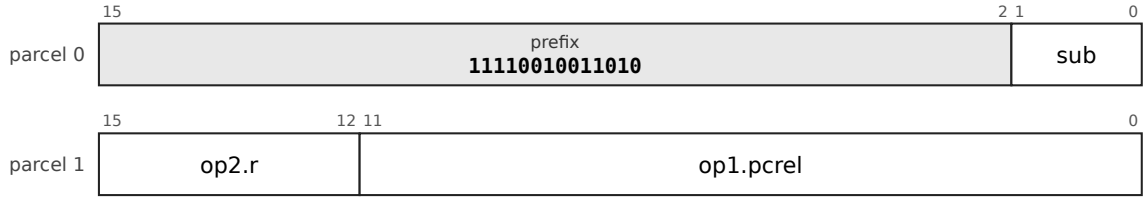
`op2` is `b4d10s-xsz`: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.
Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-LDI12-d-1: LDI12/d, prefix `11110010011001`



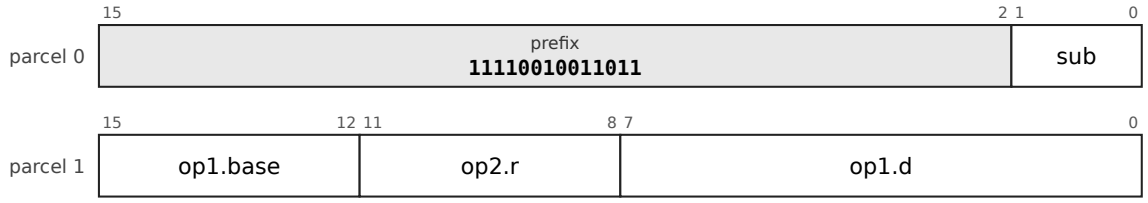
`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `shl` 00, `shr` 01, `sar` 10, `mul` 11.
Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.imm=p1[11:0]`

16-LDPC12-a-1: LDPC12/a, prefix `11110010011010`



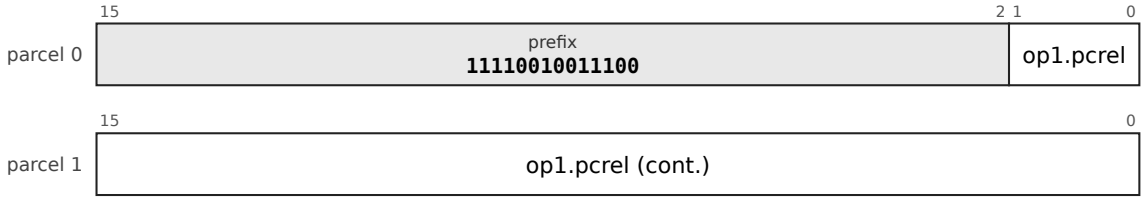
`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.
Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

16-LDBu8-b-1: LDBu8/b, prefix `11110010011011`



`op1` is `b4d8u-xsz`: memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

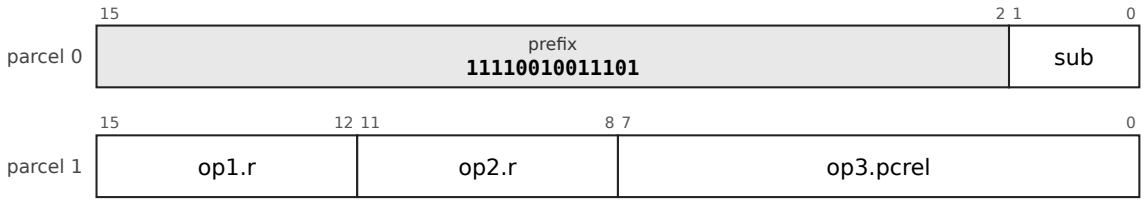
16-CALL2-18-c-1: CALL2_18/c, prefix `11110010011100`



`op1` is `pc18-u2`: pc-relative, 18 bits counted in 2-byte units (-262144 to 262142 bytes).

Fields: `op1.pcrel=p0[1:0]+p1[15:0]`

16-CBR8-c-1: CBR8/c, prefix `11110010011101`



`op1` is `reg4`: register, 4 bits.

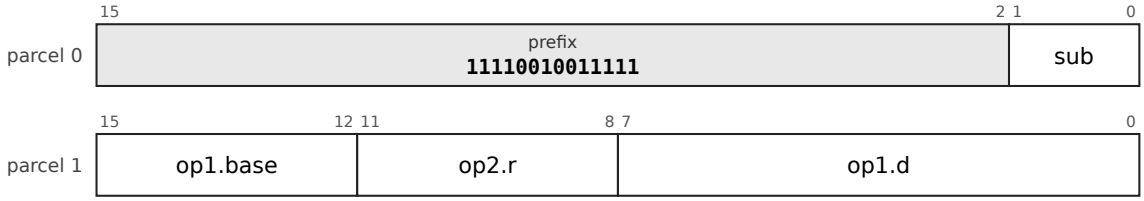
`op2` is `reg4`: register, 4 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

16-LDBs8-a-1: LDBs8/a, prefix `11110010011111`



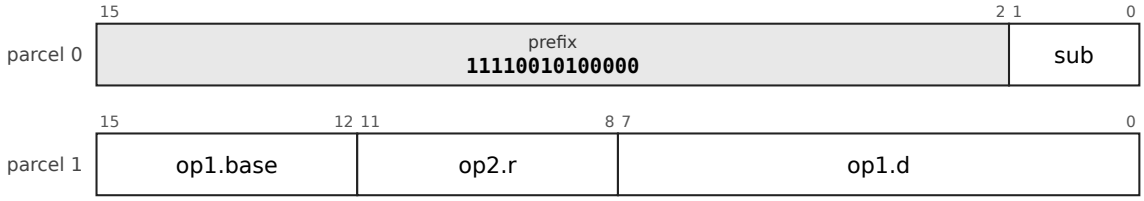
`op1` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

16-LDBu8-c-1: LDBu8/c, prefix `11110010100000`



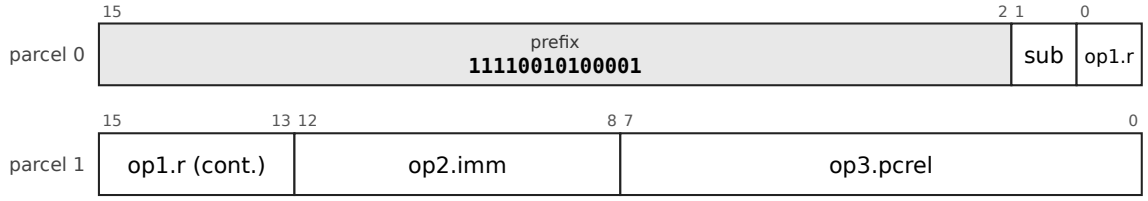
`op1` is `b4d8u-xsz`: memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

16-CBI5-8-a-1: CBI5_8/a, prefix `11110010100001`



`op1` is `reg4`: register, 4 bits.

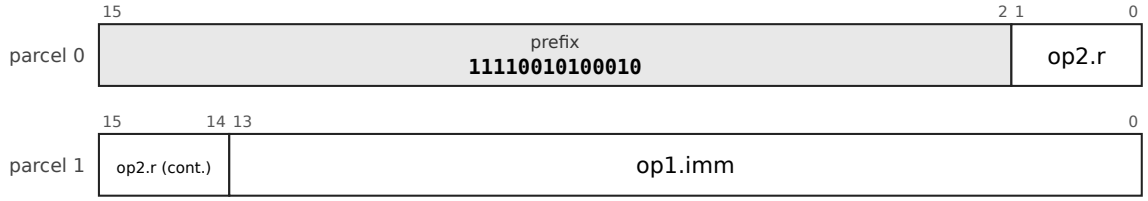
`op2` is `imm5s`: signed 5-bit immediate, -16 to 15.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[1] op1.r=p0[0]+p1[15:13] op2.imm=p1[12:8] op3.pcrel=p1[7:0]`

16-LDI14-a-1: LDI14/a, prefix `11110010100010`

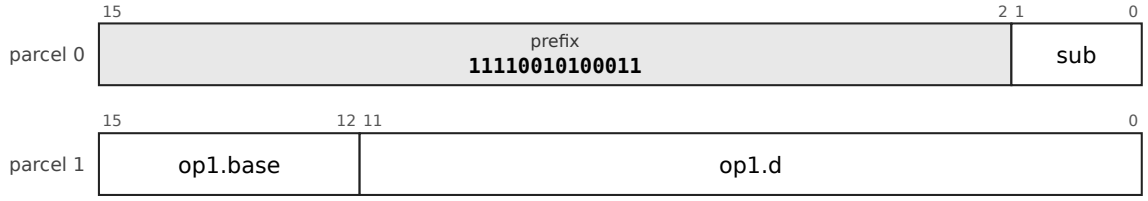


`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

16-SOBu12-a-1: SOBu12/a, prefix `11110010100011`

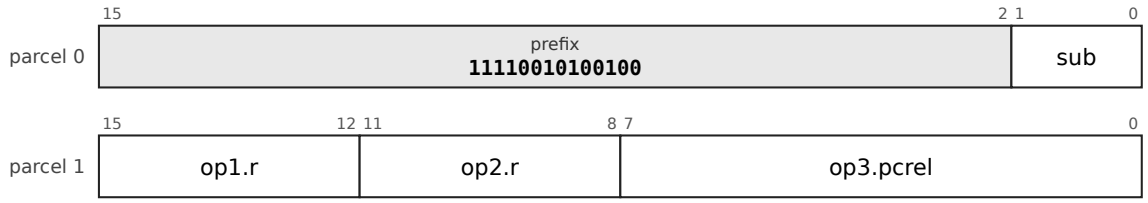


`op1` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.

`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-CBR8-b-1: CBR8/b, prefix `11110010100100`



`op1` is `reg4`: register, 4 bits.

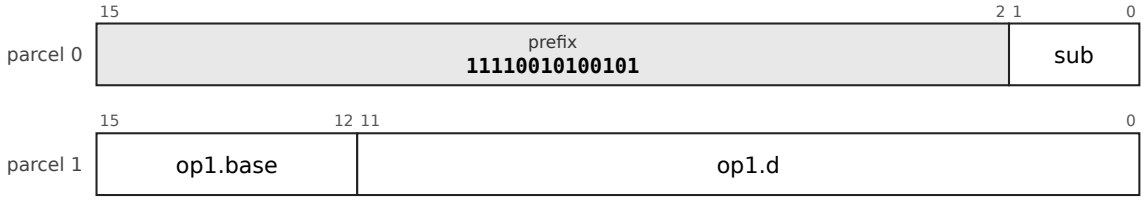
`op2` is `reg4`: register, 4 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

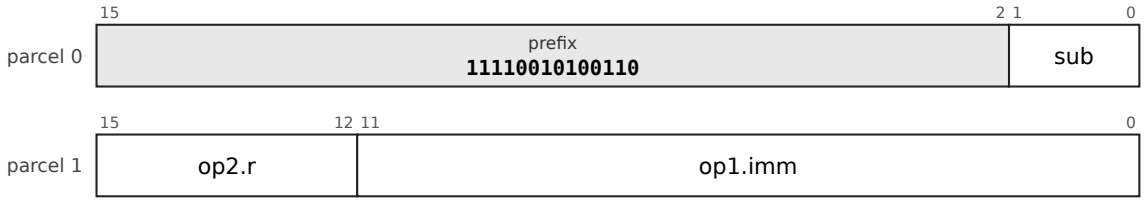
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

16-SOBu12-x-1: SOBu12/x, prefix `11110010100101`



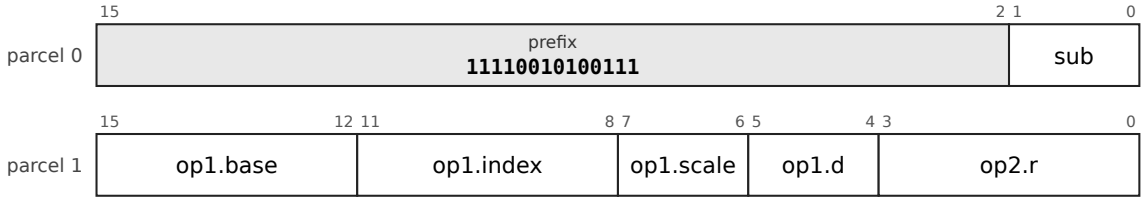
`op1` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.
`sub` (2 bits) selects the mnemonic: `jmp` 00, `call` 01, `jmpi` 10, `calli` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-LDI12-c-1: LDI12/c, prefix `11110010100110`



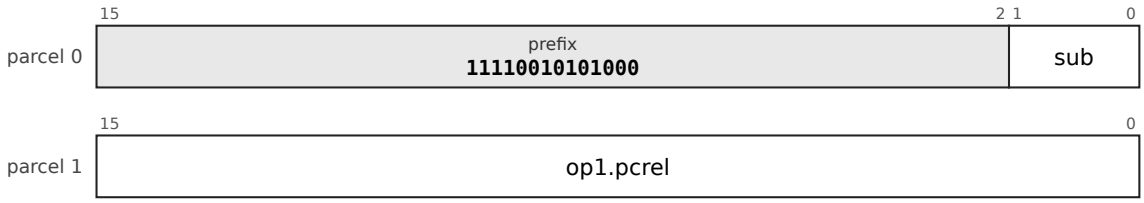
`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `bis` 00, `xor` 01, `mul` 10, `bic` 11.
Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.imm=p1[11:0]`

16-LDX2-b-1: LDX2/b, prefix `11110010100111`



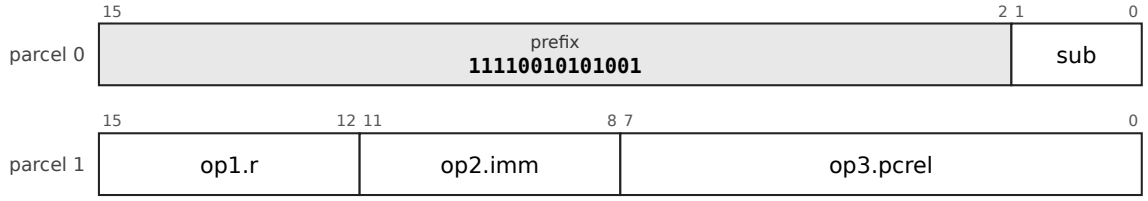
`op1` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:4] op2.r=p1[3:0]`

16-BR2-16-c-2: BR2_16/c, prefix `11110010101000`



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).
`sub` (2 bits) selects the mnemonic: `bgt` 00, `ble` 01, `bhi` 10, `blos` 11.
Fields: `sub=p0[1:0] op1.pcrel=p1[15:0]`

16-CBI4-8-c-1: CBI4_8/c, prefix `11110010101001`



`op1` is `reg4`: register, 4 bits.

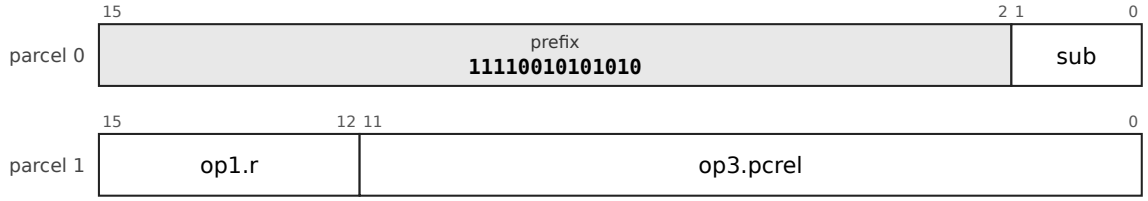
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:8] op3.pcrel=p1[7:0]`

16-CBZ2-12-b-1: CBZ2_12/b, prefix `11110010101010`



`op1` is `reg4`: register, 4 bits.

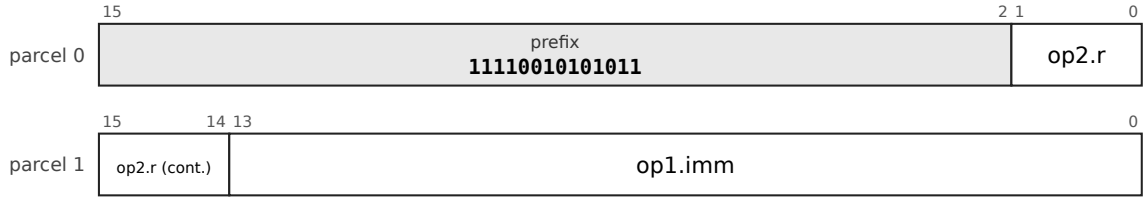
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op3.pcrel=p1[11:0]`

16-LDI14-b-2: LDI14/b, prefix `11110010101011`

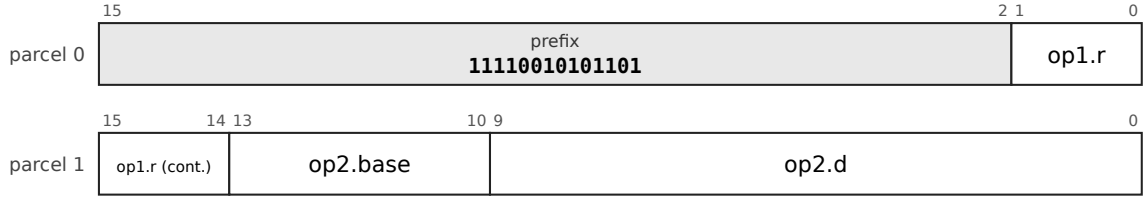


`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

16-CMBu10-a-1: CMBu10/a, prefix `11110010101101`

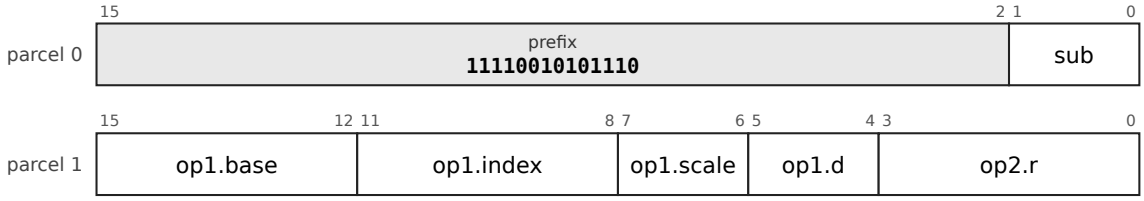


`op1` is `reg4`: register, 4 bits.

`op2` is `b4d10u-xsz`: memory at base register, 4 bits + unsigned 10-bit displacement times the size of the access.

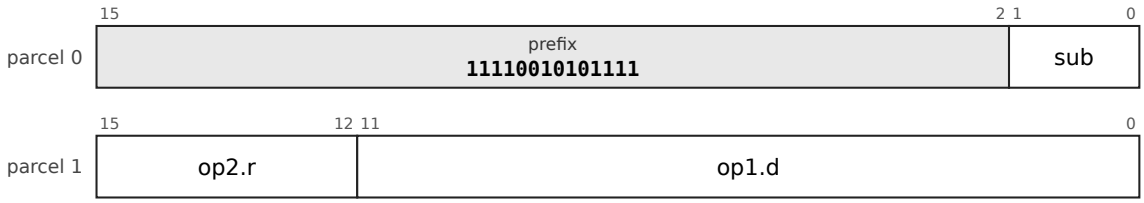
Fields: `op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-LDX2-c-1: LDX2/c, prefix `11110010101110`



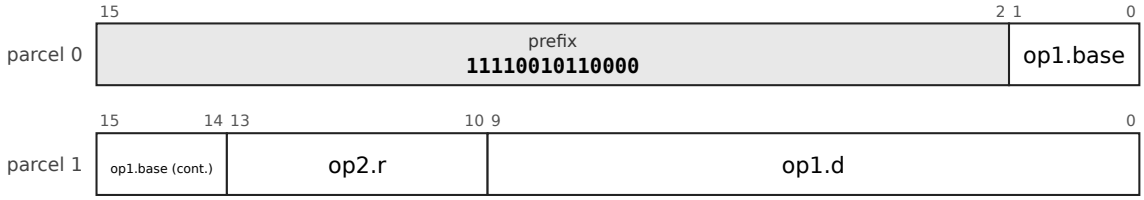
`op1` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.
Fields: `sub=p0[1:0]` `op1.base=p1[15:12]` `op1.index=p1[11:8]` `op1.scale=p1[7:6]` `op1.d=p1[5:4]` `op2.r=p1[3:0]`

16-LDS12-c-1: LDS12/c, prefix `11110010101111`



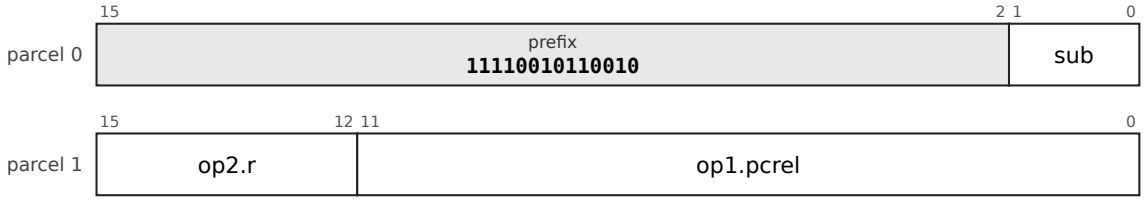
`op1` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.
Fields: `sub=p0[1:0]` `op2.r=p1[15:12]` `op1.d=p1[11:0]`

16-LDBs10-c-2: LDBs10/c, prefix `11110010110000`



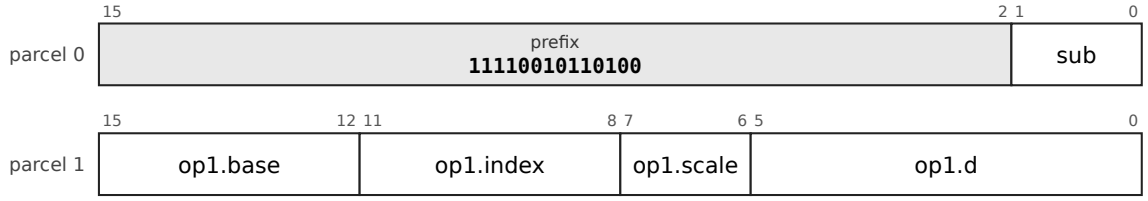
`op1` is `b4d10s-xsz`: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
Fields: `op1.base=p0[1:0]+p1[15:14]` `op2.r=p1[13:10]` `op1.d=p1[9:0]`

16-LDPC12-b-1: LDPC12/b, prefix `11110010110010`



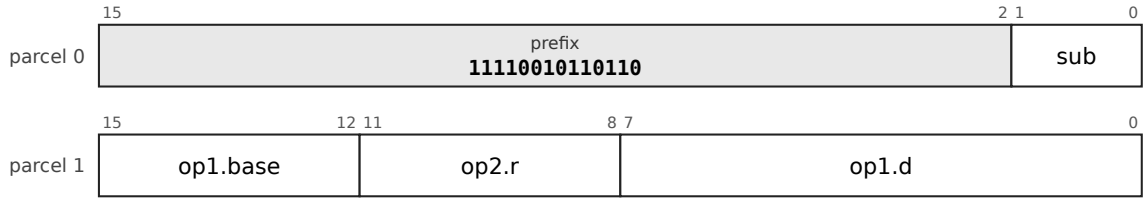
`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0]` `op2.r=p1[15:12]` `op1.pcrel=p1[11:0]`

16-SOX6-a-1: SOX6/a, prefix `11110010110100`



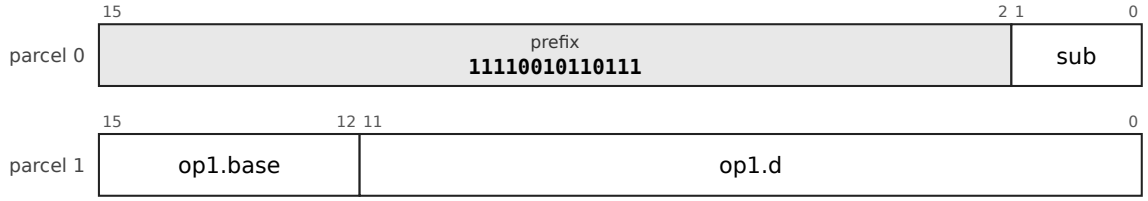
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-LDBs8-b-1: LDBs8/b, prefix `11110010110110`



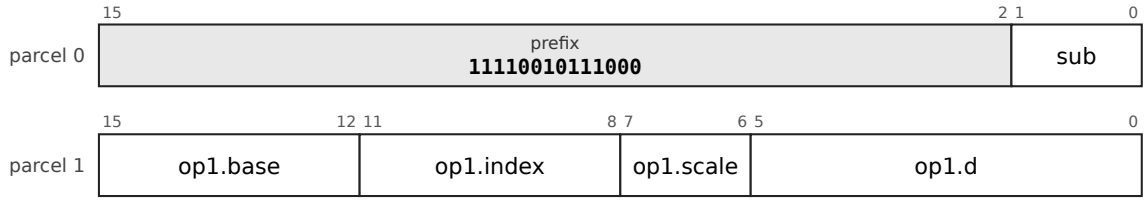
`op1` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

16-SOBs12-a-1: SOBs12/a, prefix `11110010110111`



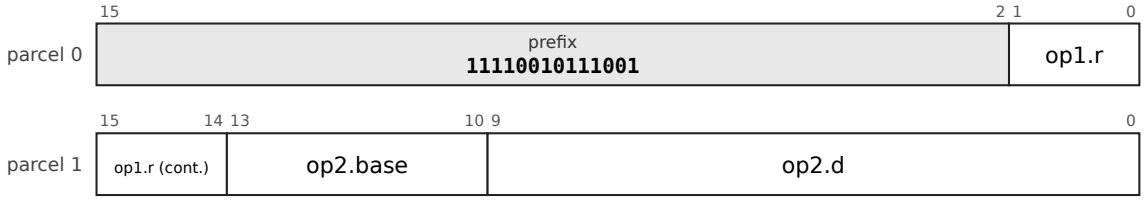
`op1` is `b4d12s-xsz`: memory at base register, 4 bits + signed 12-bit displacement times the size of the access.
`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-SOX6-x-1: SOX6/x, prefix `11110010111000`



`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (2 bits) selects the mnemonic: `jmp` 00, `call` 01, `jmpi` 10, `calli` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-CMBs10-a-1: CMBs10/a, prefix `11110010111001`

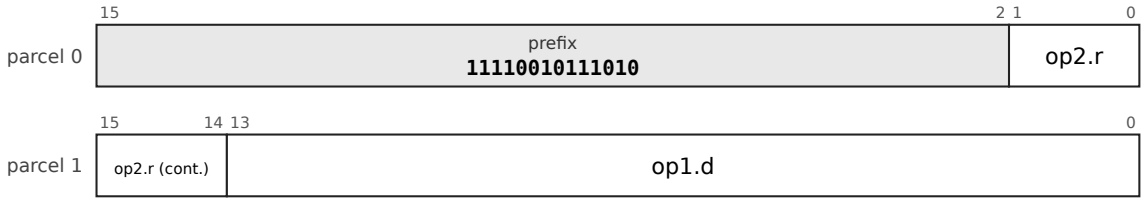


`op1` is `reg4`: register, 4 bits.

`op2` is `b4d10s-xsz`: memory at base register, 4 bits + signed 10-bit displacement times the size of the access.

Fields: `op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

16-LEAS14-a-1: LEAS14/a, prefix `11110010111010`

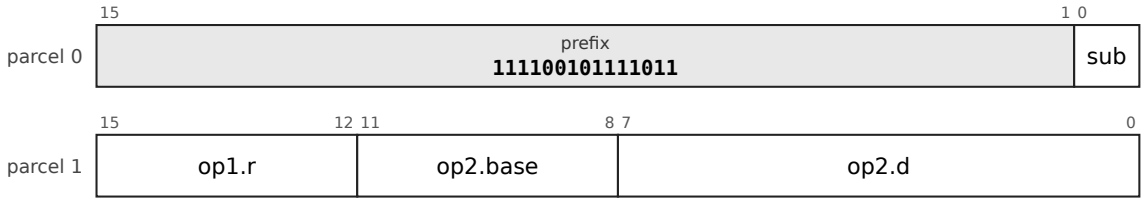


`op1` is `spd14u-x8`: memory at sp + unsigned 14-bit displacement times 8, 0 to 131064 bytes.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p0[1:0]+p1[15:14] op1.d=p1[13:0]`

16-STBs8-a-1: STBs8/a, prefix `111100101111011`



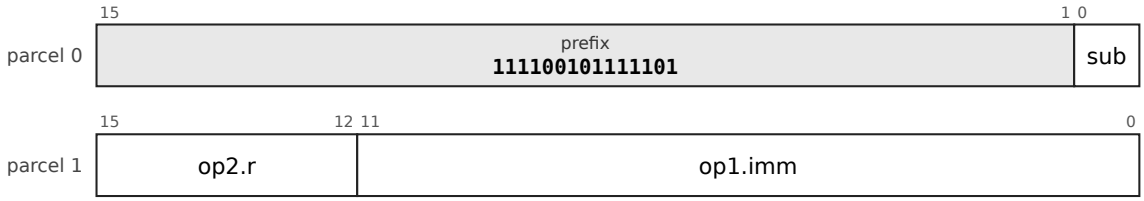
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

16-LDI12-b-1: LDI12/b, prefix `111100101111101`



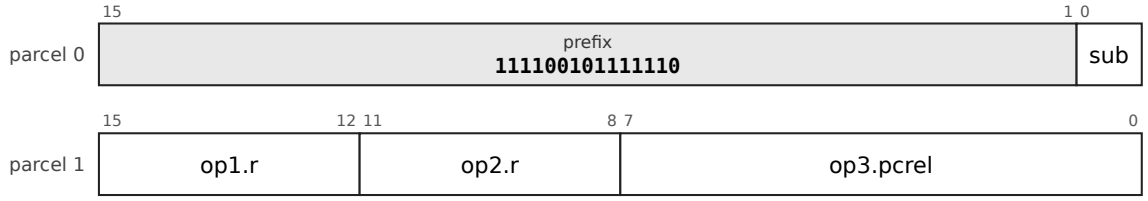
`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[0] op2.r=p1[15:12] op1.imm=p1[11:0]`

16-CBR8-a-1: CBR8/a, prefix `111100101111110`



`op1` is `reg4`: register, 4 bits.

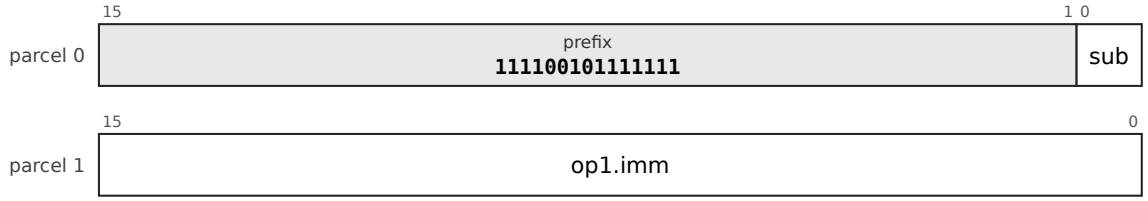
`op2` is `reg4`: register, 4 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

16-PUSHM16a-a-1: PUSHM16a/a, prefix `111100101111111`

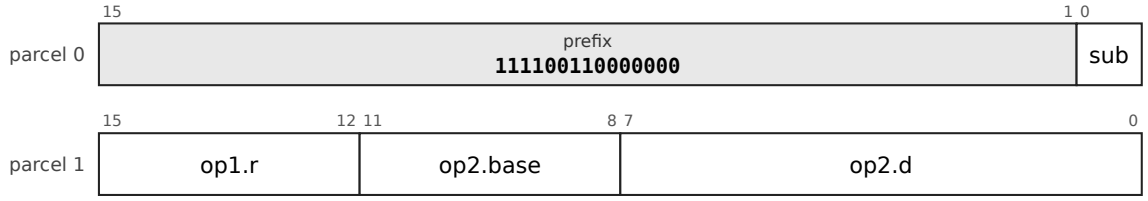


`op1` is `imm16u`: unsigned 16-bit immediate, 0 to 65535.

`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

Fields: `sub=p0[0] op1.imm=p1[15:0]`

16-STBu8-b-1: STBu8/b, prefix `111100110000000`



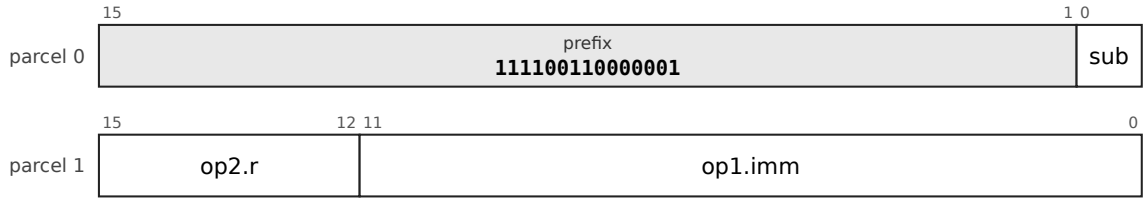
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d8u-xsz`: memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

16-LDI12-d-2: LDI12/d, prefix `111100110000001`



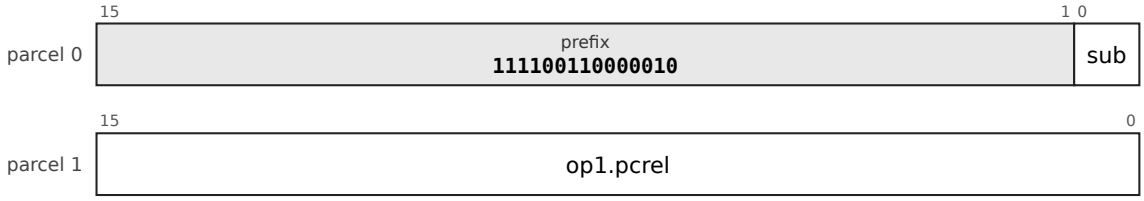
`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `bic` 0, `movb` 1.

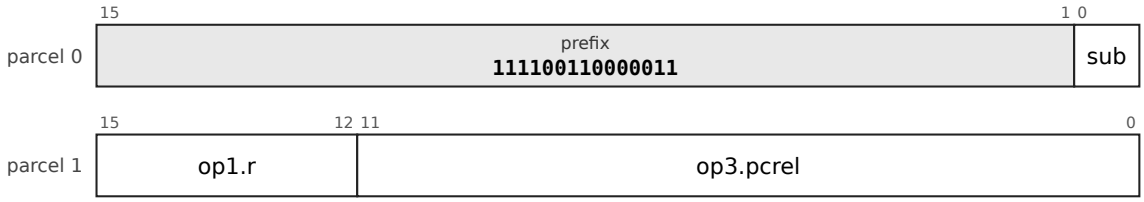
Fields: `sub=p0[0] op2.r=p1[15:12] op1.imm=p1[11:0]`

16-BR2-16-b-1: BR2_16/b, prefix `111100110000010`



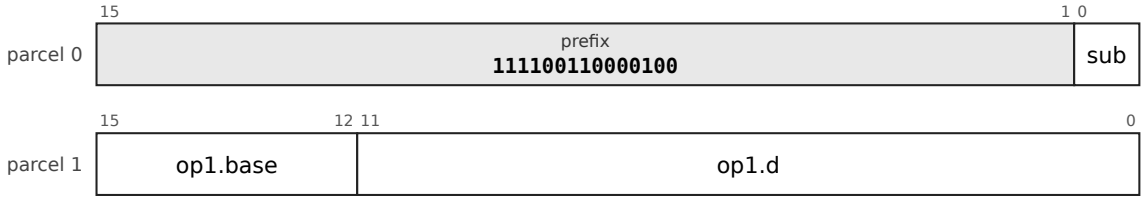
`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).
`sub` (1 bit) selects the mnemonic: `beq` 0, `bne` 1.
Fields: `sub=p0[0] op1.pcrel=p1[15:0]`

16-CBZ2-12-a-1: CBZ2_12/a, prefix `111100110000011`



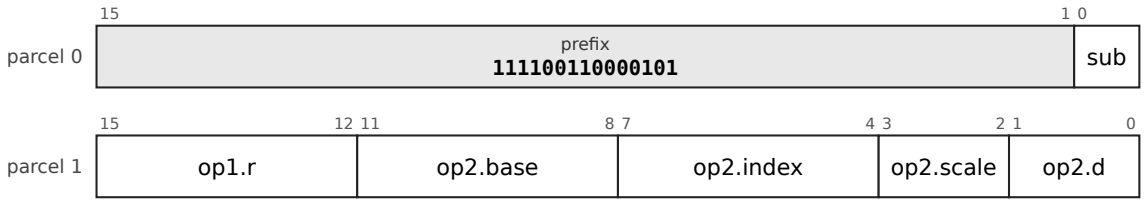
`op1` is `reg4`: register, 4 bits.
`op2` is `k-imm0`: implicit constant 0.
`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op3.pcrel=p1[11:0]`

16-SOBu12-a-2: SOBu12/a, prefix `111100110000100`



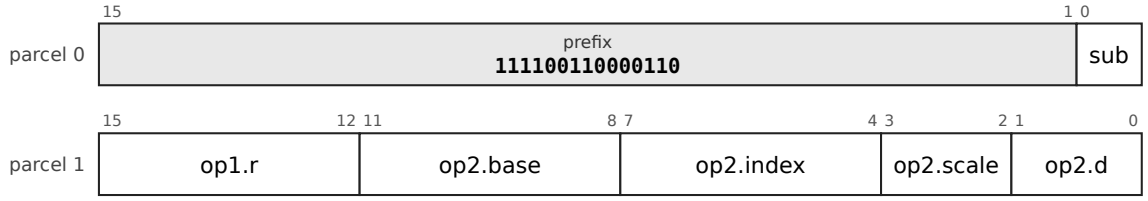
`op1` is `b4d12u-xsz`: memory at base register, 4 bits + unsigned 12-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.
Fields: `sub=p0[0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-STX2-b-1: STX2/b, prefix `111100110000101`



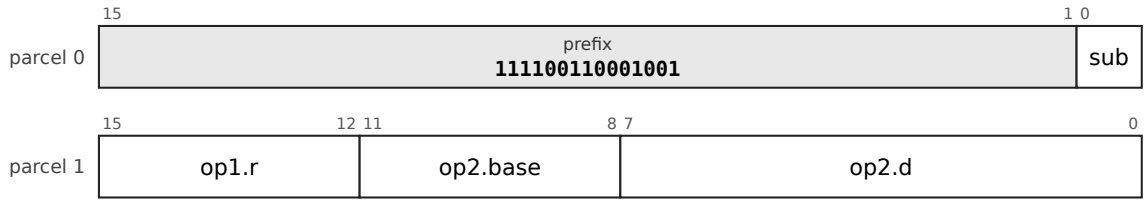
`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.index=p1[7:4] op2.scale=p1[3:2] op2.d=p1[1:0]`

16-STX2-a-1: STX2/a, prefix `111100110000110`



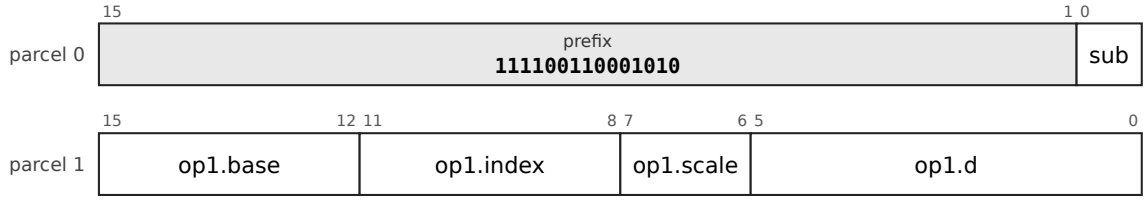
`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.index=p1[7:4] op2.scale=p1[3:2] op2.d=p1[1:0]`

16-STBs8-b-1: STBs8/b, prefix `111100110001001`



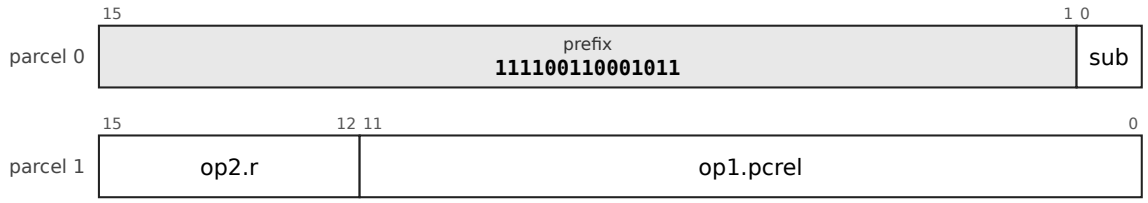
`op1` is `reg4`: register, 4 bits.
`op2` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

16-SOX6-a-2: SOX6/a, prefix `111100110001010`



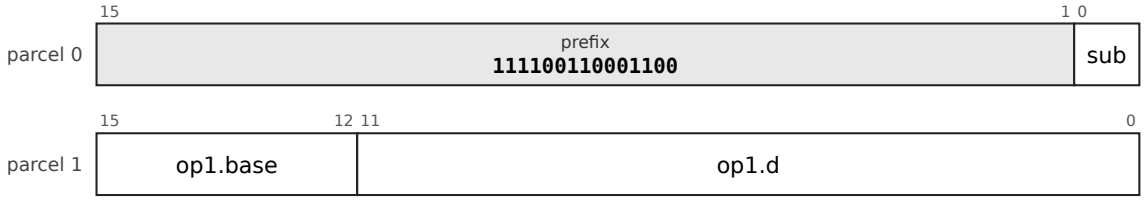
`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.
Fields: `sub=p0[0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

16-LDPC12-d-2: LDPC12/d, prefix `111100110001011`



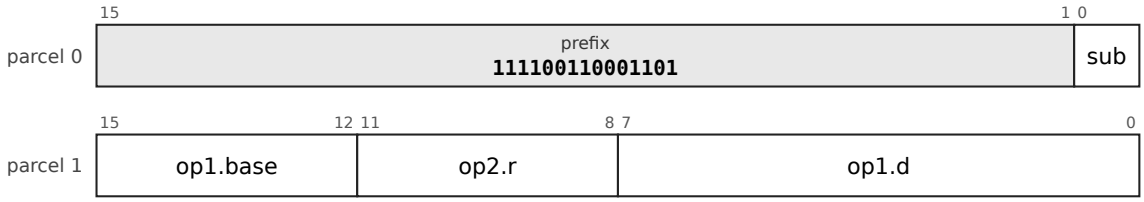
`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `fmul` 0, `bit` 1.
Fields: `sub=p0[0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

16-SOBs12-a-2: SOBs12/a, prefix `111100110001100`



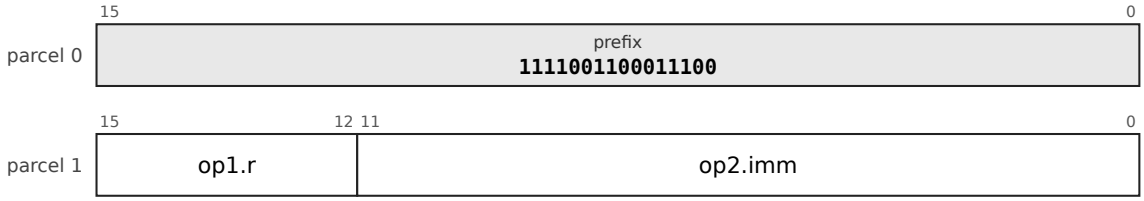
`op1` is `b4d12s-xsz`: memory at base register, 4 bits + signed 12-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.
Fields: `sub=p0[0] op1.base=p1[15:12] op1.d=p1[11:0]`

16-LDBs8-d-2: LDBs8/d, prefix `111100110001101`



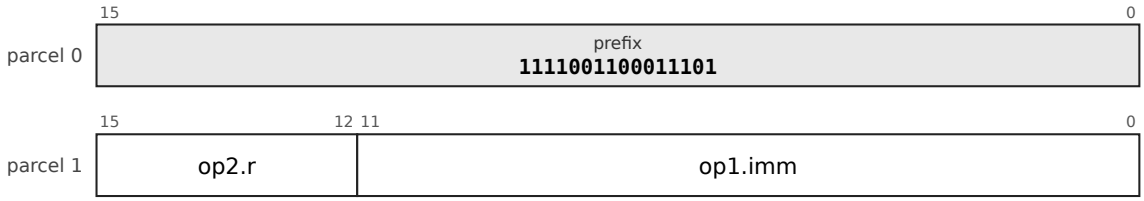
`op1` is `b4d8s-xsz`: memory at base register, 4 bits + signed 8-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `fmul` 0, `bit` 1.
Fields: `sub=p0[0] op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

16-MFSR-1: MFSR, prefix `1111001100011100`



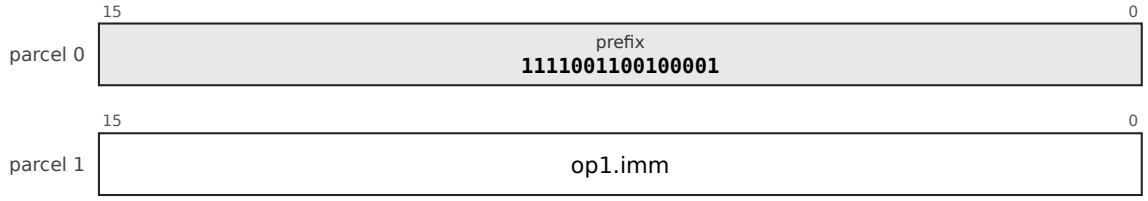
`op1` is `reg4`: register, 4 bits.
`op2` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.
Fields: `op1.r=p1[15:12] op2.imm=p1[11:0]`

16-MTSR-1: MTSR, prefix `1111001100011101`



`op1` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p1[15:12] op1.imm=p1[11:0]`

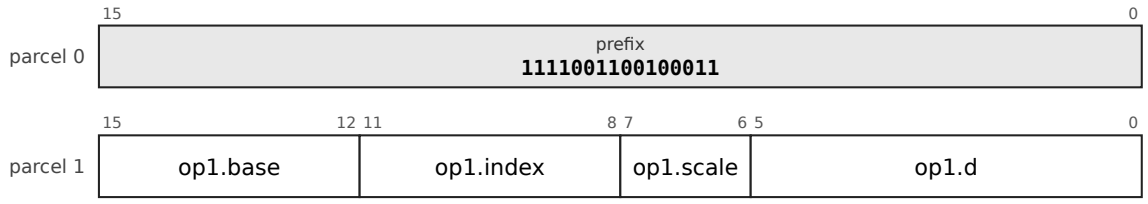
16-PUSHI2-1: PUSHI2, prefix `1111001100100001`



`op1` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

Fields: `op1.imm=p1[15:0]`

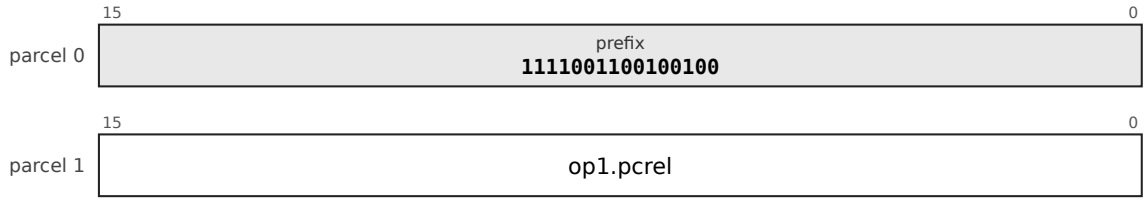
16-JTAG2-1: JTAG2, prefix `1111001100100011`



`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits, index register, 4 bits.

Fields: `op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

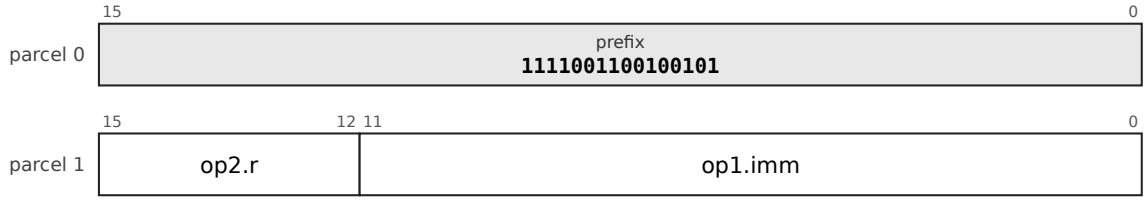
16-BR2-16-a-1: BR2_16/a, prefix `1111001100100100`



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

16-LDI12-a-1: LDI12/a, prefix `1111001100100101`

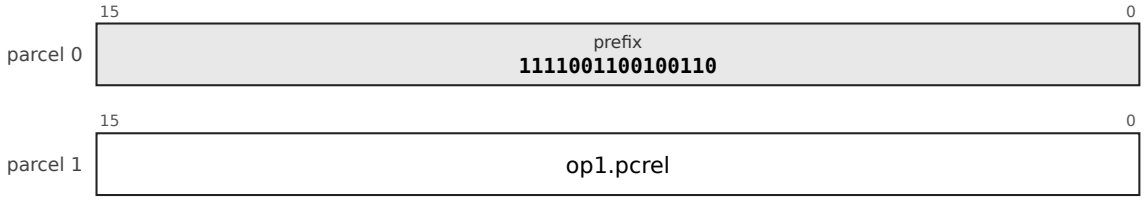


`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p1[15:12] op1.imm=p1[11:0]`

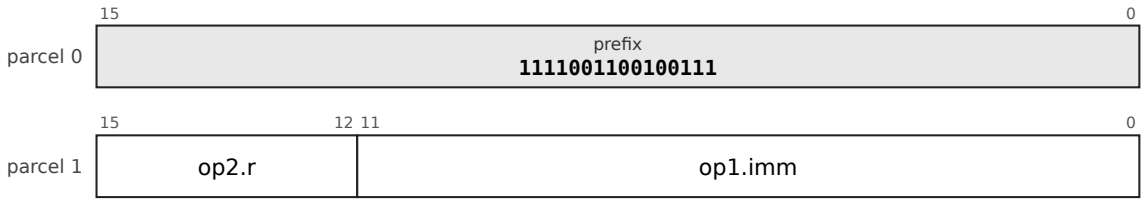
16-CALL2-16-c-1: CALL2_16/c, prefix `1111001100100110`



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

16-LDI12-b-2: LDI12/b, prefix `1111001100100111`

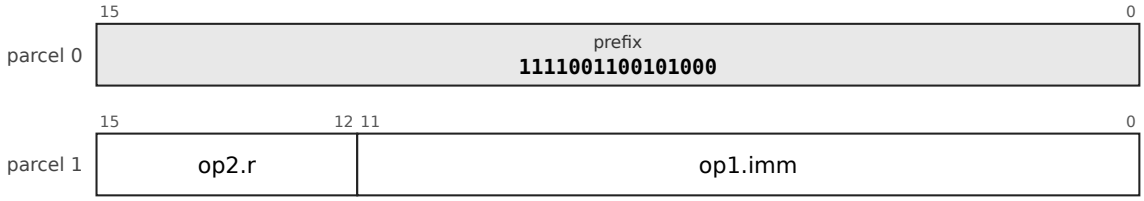


`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p1[15:12] op1.imm=p1[11:0]`

16-LDI12-d-3: LDI12/d, prefix `1111001100101000`

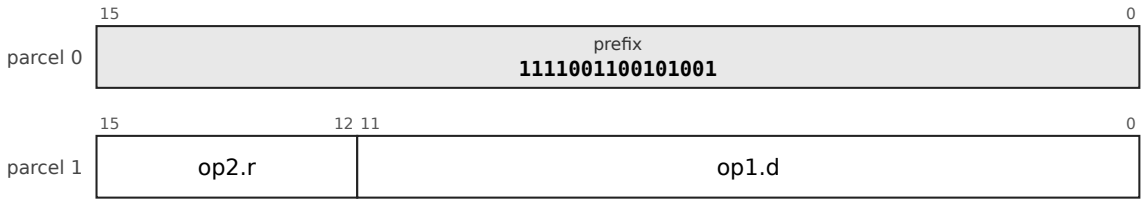


`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p1[15:12] op1.imm=p1[11:0]`

16-LEASx12-a-1: LEASx12/a, prefix `1111001100101001`

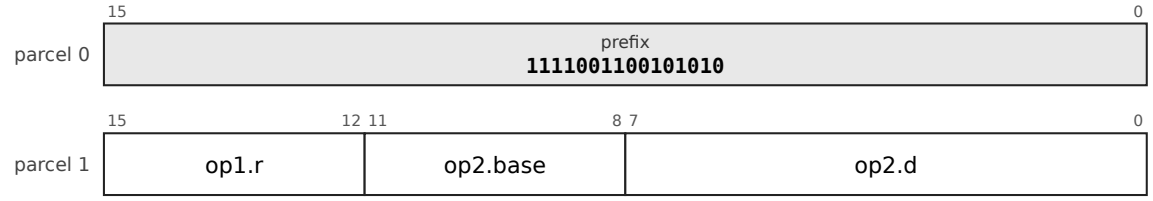


`op1` is `spd12s-x1`: memory at sp + signed 12-bit displacement times 1, -2048 to 2047 bytes.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p1[15:12] op1.d=p1[11:0]`

16-CMBu8-a-1: CMBu8/a, prefix `1111001100101010`

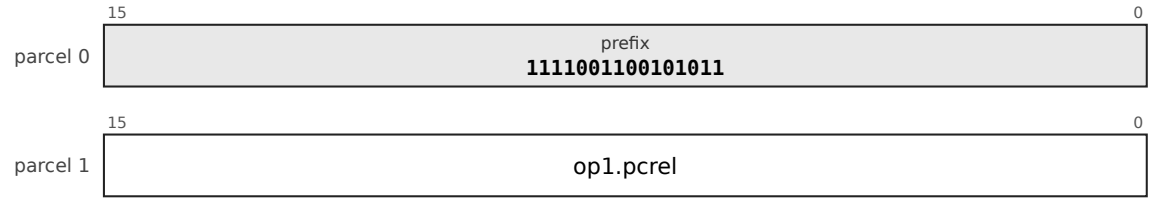


`op1` is `reg4`: register, 4 bits.

`op2` is `b4d8u-xsz`: memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access.

Fields: `op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

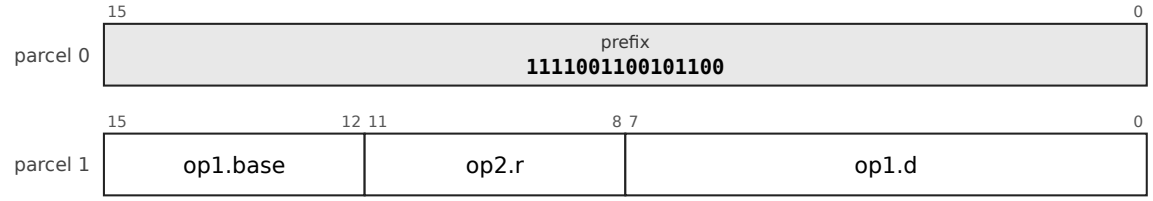
16-CALL2-16-j-1: CALL2_16/j, prefix `1111001100101011`



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

16-LDBu8-c-2: LDBu8/c, prefix `1111001100101100`

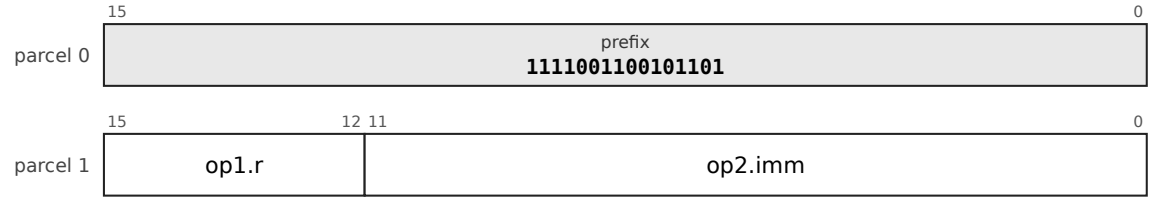


`op1` is `b4d8u-xsz`: memory at base register, 4 bits + unsigned 8-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits.

Fields: `op1.base=p1[15:12] op2.r=p1[11:8] op1.d=p1[7:0]`

16-CMI12-a-1: CMI12/a, prefix `1111001100101101`

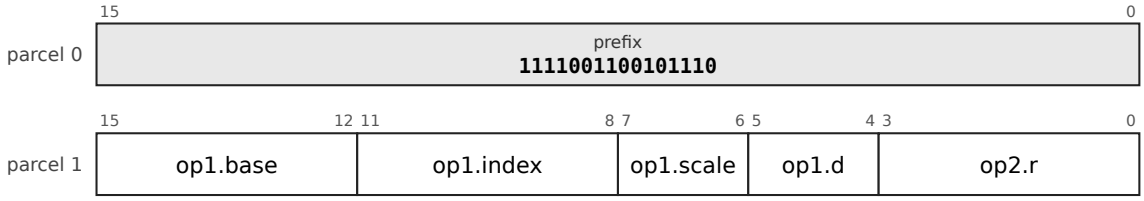


`op1` is `reg4`: register, 4 bits.

`op2` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

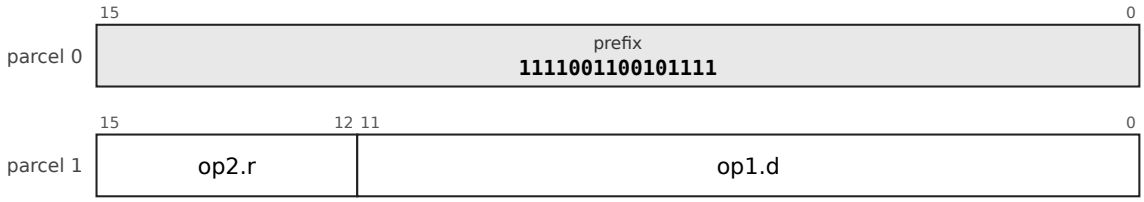
Fields: `op1.r=p1[15:12] op2.imm=p1[11:0]`

16-LDX2-c-2: LDX2/c, prefix `1111001100101110`



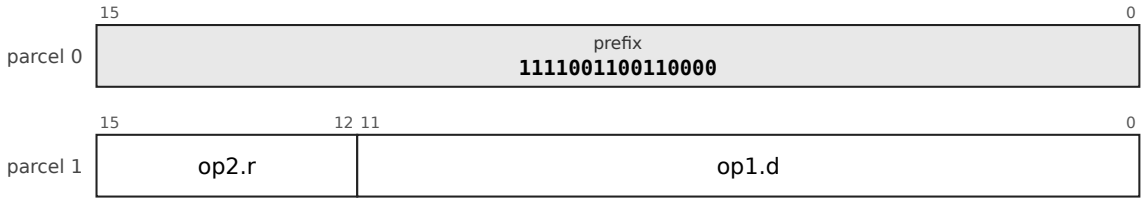
`op1` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
Fields: `op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:4] op2.r=p1[3:0]`

16-LDS12-c-2: LDS12/c, prefix `1111001100101111`



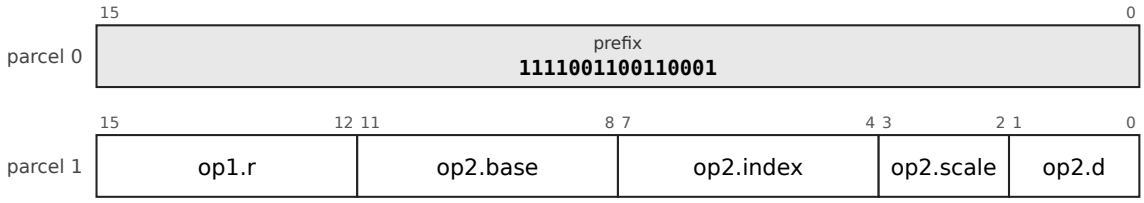
`op1` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p1[15:12] op1.d=p1[11:0]`

16-LEAS12-a-1: LEAS12/a, prefix `1111001100110000`



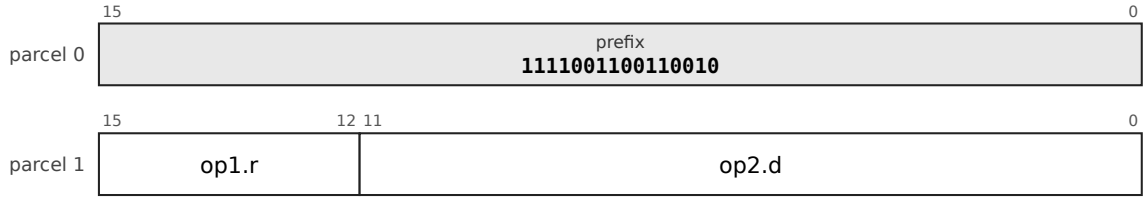
`op1` is `spd12u-x8`: memory at sp + unsigned 12-bit displacement times 8, 0 to 32760 bytes.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p1[15:12] op1.d=p1[11:0]`

16-CMX2-a-1: CMX2/a, prefix `1111001100110001`



`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d2s`: memory at base + (index << scale) + signed 2-bit displacement; base register, 4 bits, index register, 4 bits.
Fields: `op1.r=p1[15:12] op2.base=p1[11:8] op2.index=p1[7:4] op2.scale=p1[3:2] op2.d=p1[1:0]`

16-CMS12-a-1: CMS12/a, prefix `1111001100110010`



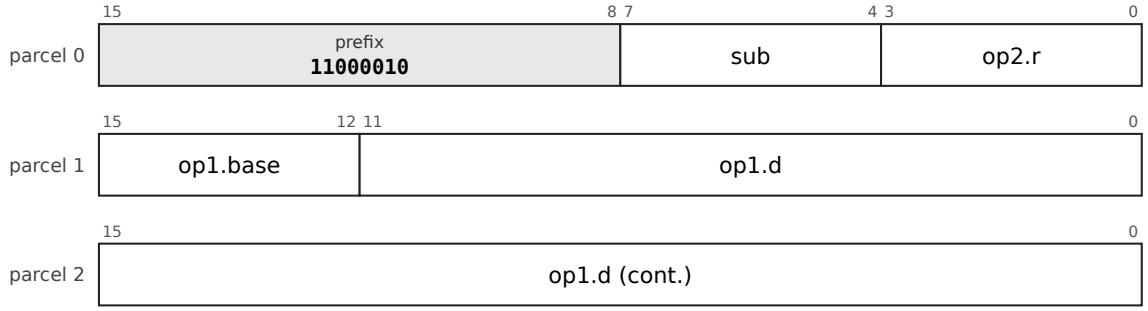
`op1` is `reg4`: register, 4 bits.

`op2` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.

Fields: `op1.r=p1[15:12] op2.d=p1[11:0]`

Instructions of 3 parcels (6 bytes)

16-LDU-B28-1: LDU.B28, prefix `11000010`



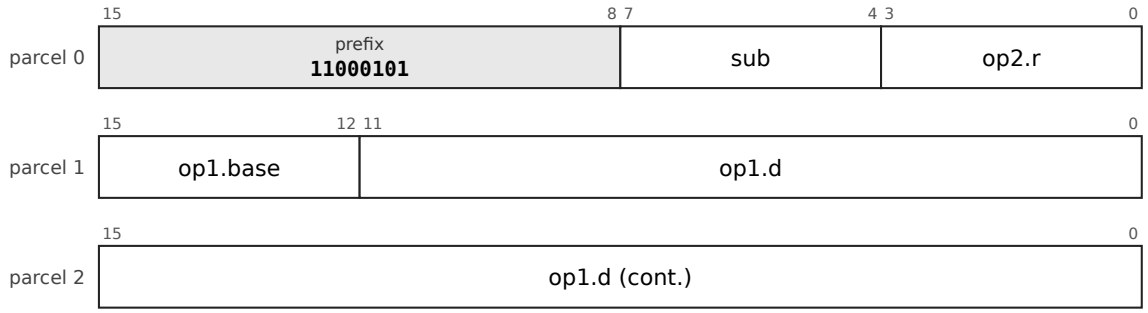
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-B28-1: LDF.B28, prefix `11000101`



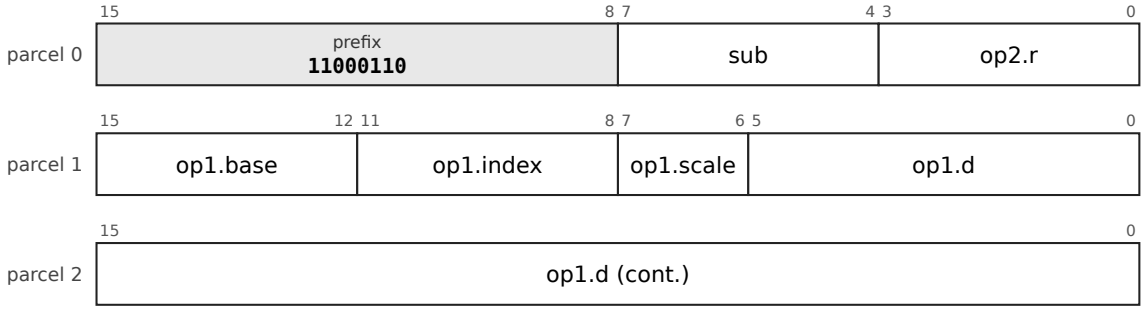
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-X22-1: LDF.X22, prefix `11000110`



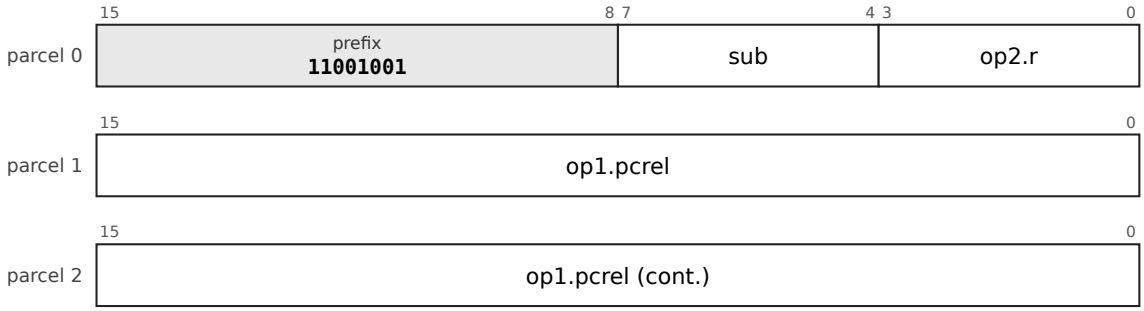
`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

16-LDF-PC32-1: LDF.PC32, prefix `11001001`



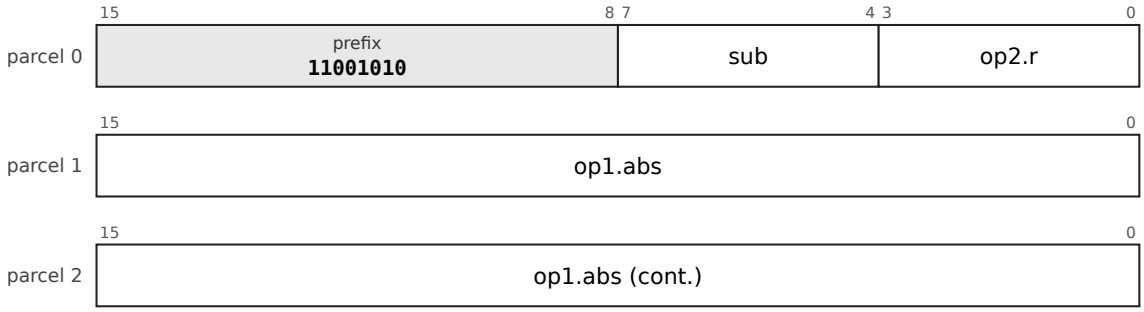
`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-LDF-A32-1: LDF.A32, prefix `11001010`



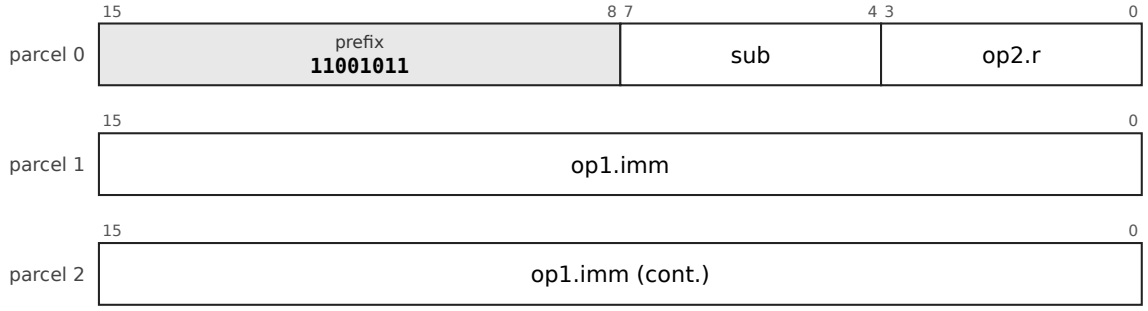
`op1` is `abs32`: absolute address, 32 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

16-LDFI-I32-1: LDFI.I32, prefix `11001011`



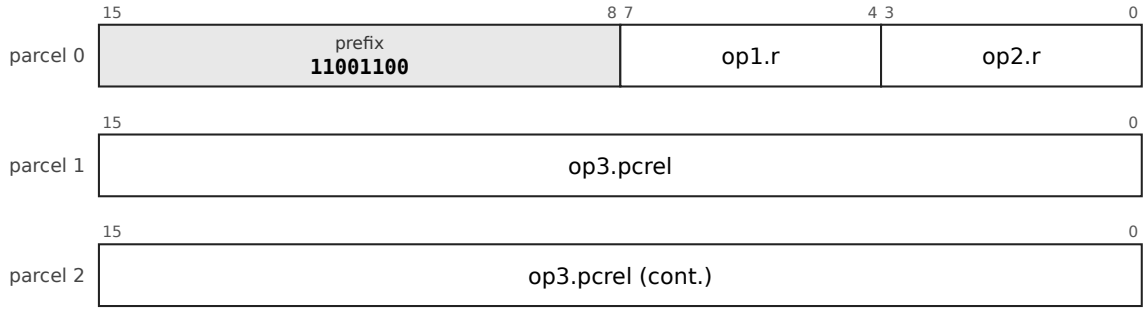
`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `add` 0000, `sub` 0001, `bic` 0010, `bis` 0011, `xor` 0100, `and` 0101, `mul` 0110, `mulh` 0111, `shl` 1000, `shr` 1001, `sar` 1010, `fadd` 1011, `fsub` 1100, `fmul` 1101, `bit` 1110, `movq` 1111.

Fields: `sub=p0[7:4] op2.r=p0[3:0] op1.imm=p1[15:0]+p2[15:0]`

16-FX-MOVCALL3-1: FX.MOVCALL3, prefix `11001100`



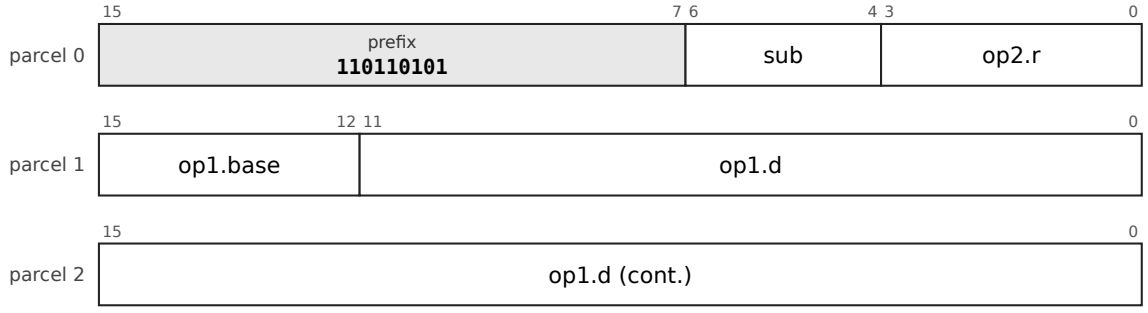
`op1` is `reg4`: register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`op3` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).

Fields: `op1.r=p0[7:4] op2.r=p0[3:0] op3.pcrel=p1[15:0]+p2[15:0]`

16-LDF-B28-2: LDF.B28, prefix `110110101`



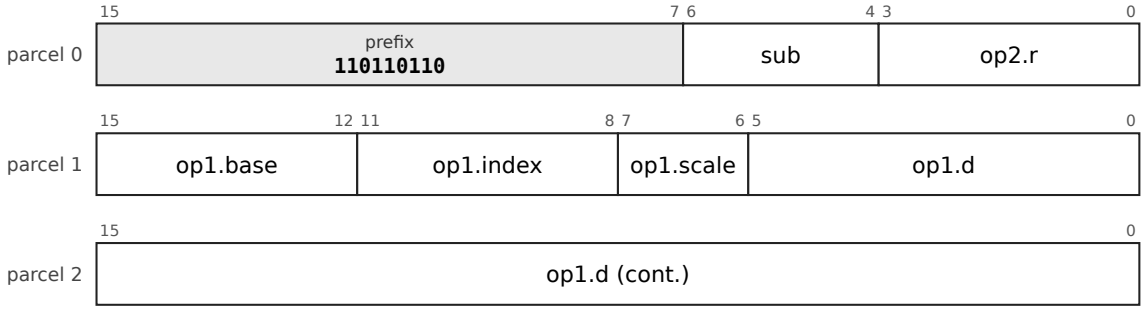
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

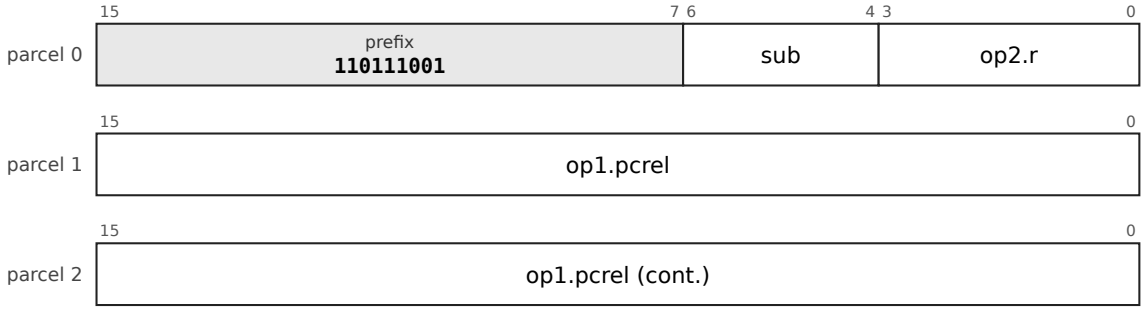
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-X22-2: LDF.X22, prefix `110110110`



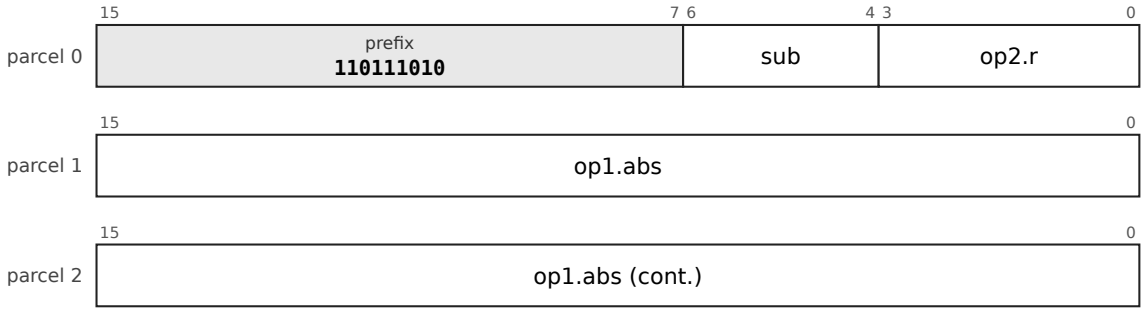
`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

16-LDF-PC32-2: LDF.PC32, prefix `110111001`



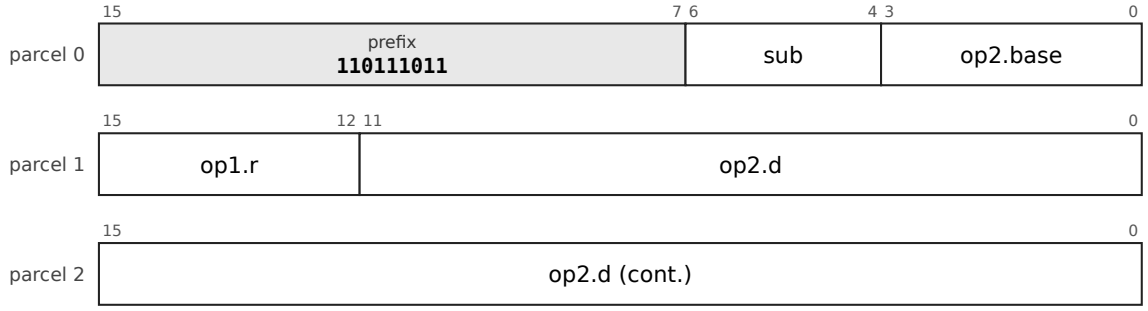
`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-LDF-A32-2: LDF.A32, prefix `110111010`



`op1` is `abs32`: absolute address, 32 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[6:4] op2.r=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

16-STF-B28-1: STF.B28, prefix `110111011`



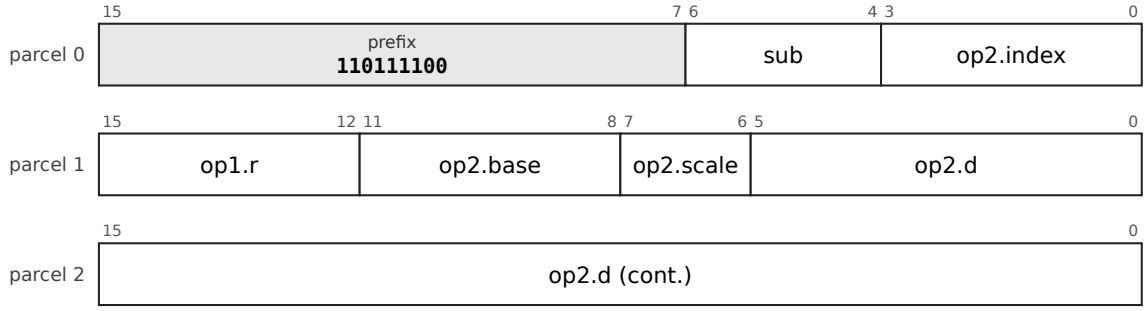
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

Fields: `sub=p0[6:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

16-STF-X22-1: STF.X22, prefix `110111100`



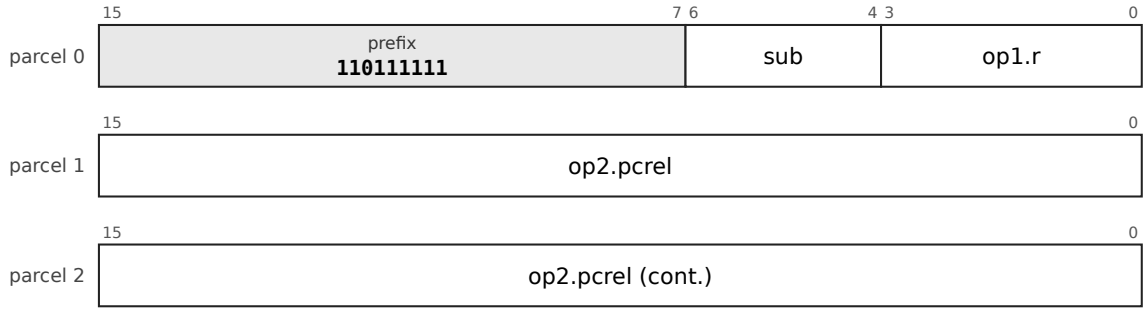
`op1` is `reg4`: register, 4 bits.

`op2` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

Fields: `sub=p0[6:4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

16-STF-PC32-1: STF.PC32, prefix `110111111`



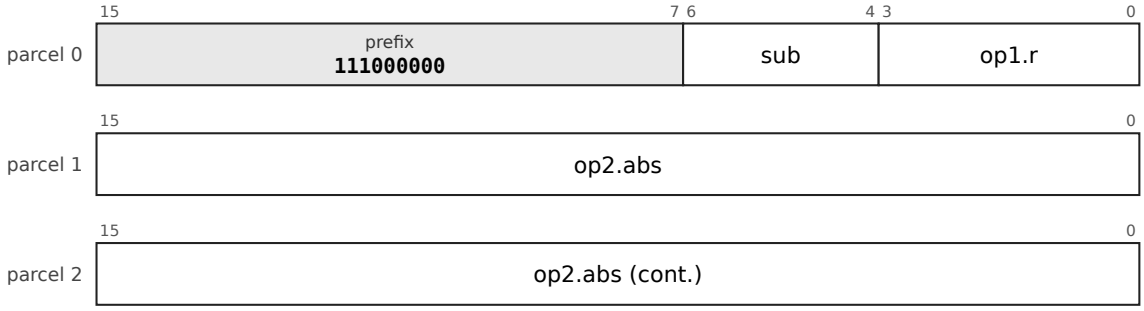
`op1` is `reg4`: register, 4 bits.

`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

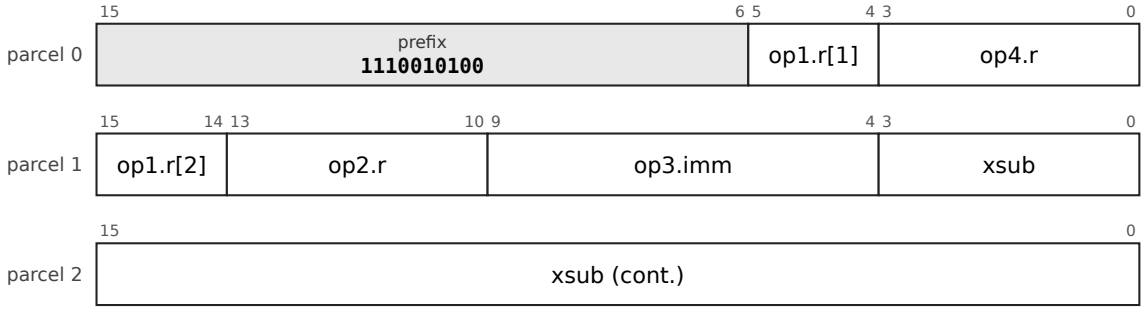
Fields: `sub=p0[6:4] op1.r=p0[3:0] op2.pcrel=p1[15:0]+p2[15:0]`

16-STF-A32-1: STF.A32, prefix `111000000`



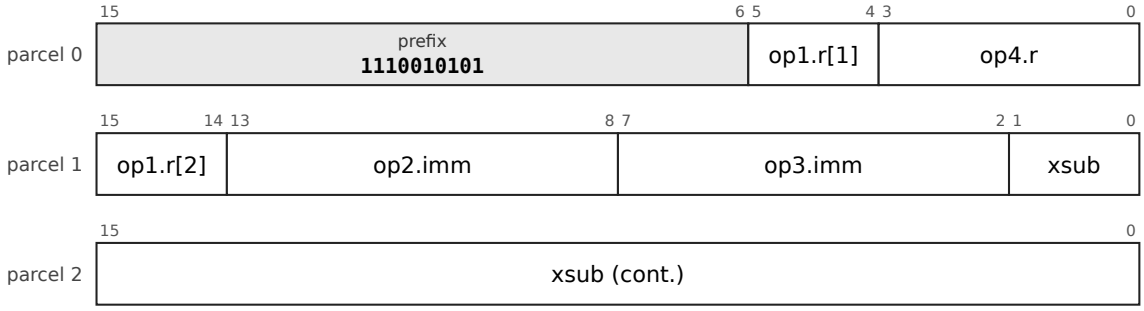
`op1` is `reg4`: register, 4 bits.
`op2` is `abs32`: absolute address, 32 bits.
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[6:4] op1.r=p0[3:0] op2.abs=p1[15:0]+p2[15:0]`

16-BFx-1: BFx, prefix `1110010100`



`op1` is `reg4`: register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63.
`op4` is `reg4`: register, 4 bits.
`xsub` selects the mnemonic.
Fields: `op1.r-1=p0[5:4] op4.r=p0[3:0] op1.r-2=p1[15:14] op2.r=p1[13:10] op3.imm=p1[9:4] xsub=p1[3:0]+p2[15:0]`

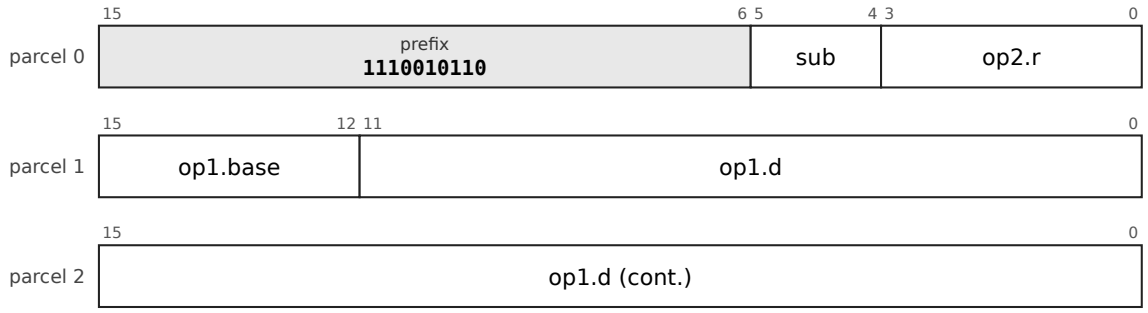
16-BFy-1: BFy, prefix `1110010101`



`op1` is `reg4`: register, 4 bits.
`op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63.
`op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63.
`op4` is `reg4`: register, 4 bits.
`xsub` selects the mnemonic.

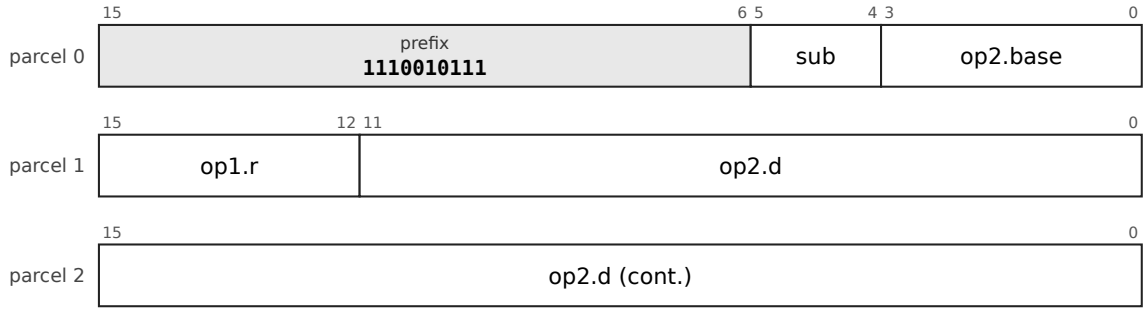
Fields: `op1.r-1=p0[5:4] op4.r=p0[3:0] op1.r-2=p1[15:14] op2.imm=p1[13:8] op3.imm=p1[7:2] xsub=p1[1:0]+p2[15:0]`

16-LDU-B28-2: LDU.B28, prefix `1110010110`



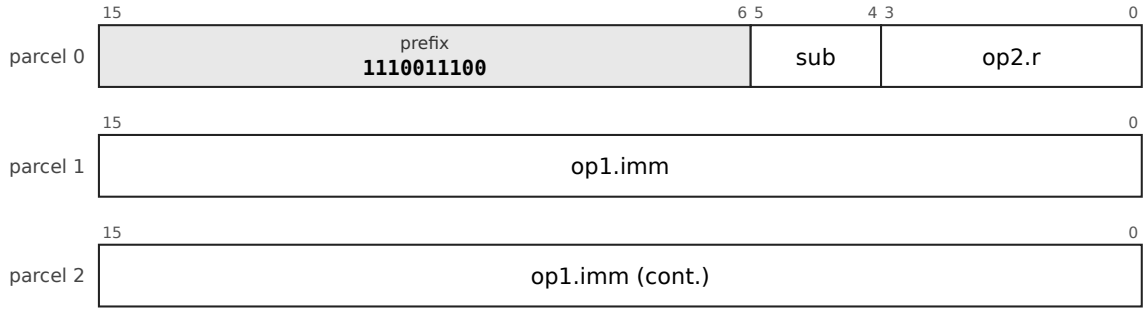
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movw.ud` 00, `movb.ud` 01, `movzl.ud` 10, `movzw.ud` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-STU-B28-1: STU.B28, prefix `1110010111`



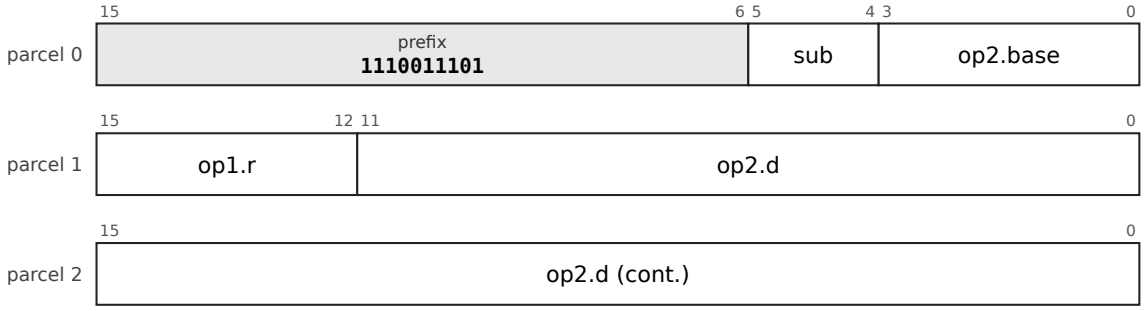
`op1` is `reg4`: register, 4 bits.
`op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.
Fields: `sub=p0[5:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

16-LDFI-I32-2: LDFI.I32, prefix `1110011100`



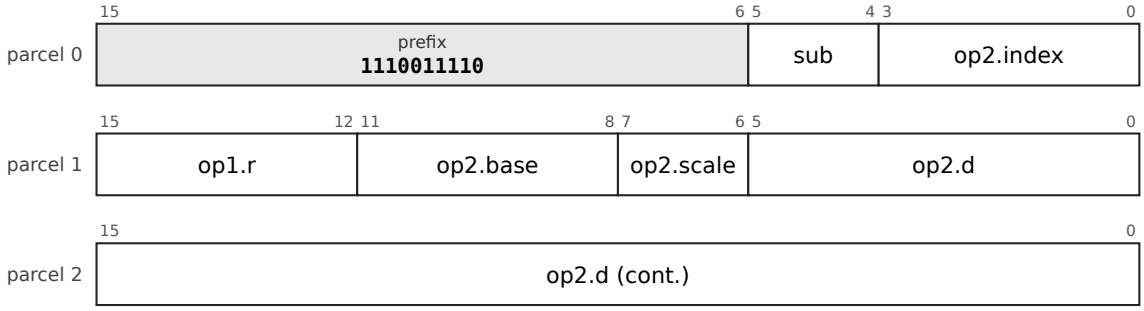
`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
`op2` is `reg4`: register, 4 bits.
`sub` (2 bits) selects the mnemonic: `movl` 00, `movw` 01, `movb` 10, `movzl` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.imm=p1[15:0]+p2[15:0]`

16-STF-B28-2: STF.B28, prefix `1110011101`



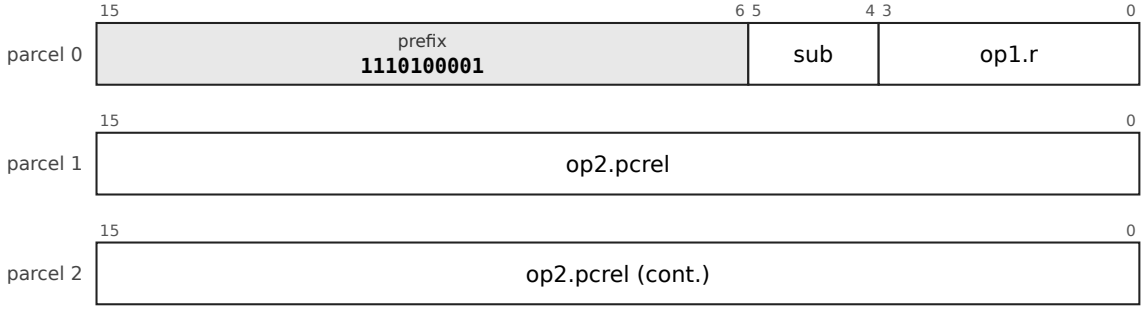
`op1` is `reg4`: register, 4 bits.
`op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.
Fields: `sub=p0[5:4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

16-STF-X22-2: STF.X22, prefix `1110011110`



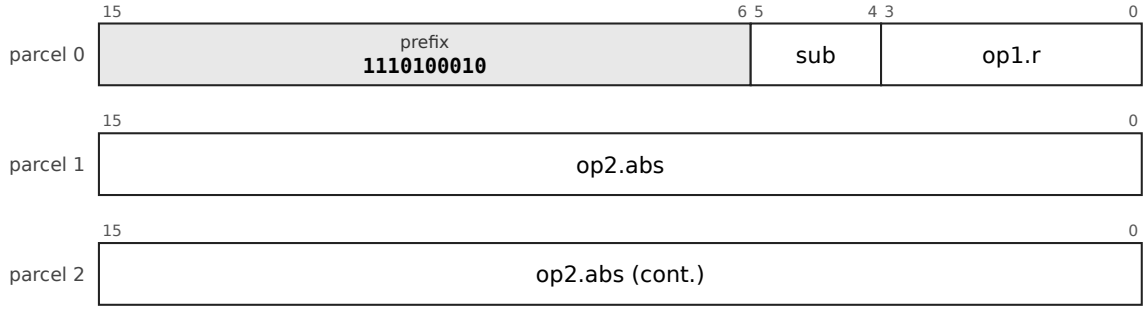
`op1` is `reg4`: register, 4 bits.
`op2` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.
`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.
Fields: `sub=p0[5:4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

16-STF-PC32-2: STF.PC32, prefix `1110100001`



`op1` is `reg4`: register, 4 bits.
`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.
Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.pcrel=p1[15:0]+p2[15:0]`

16-STF-A32-2: STF.A32, prefix `1110100010`



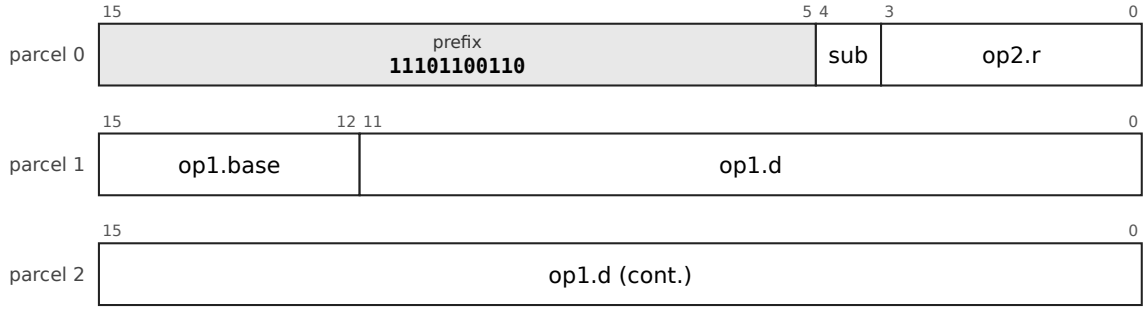
`op1` is `reg4`: register, 4 bits.

`op2` is `abs32`: absolute address, 32 bits.

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.abs=p1[15:0]+p2[15:0]`

16-LDF-B28-3: LDF.B28, prefix `11101100110`



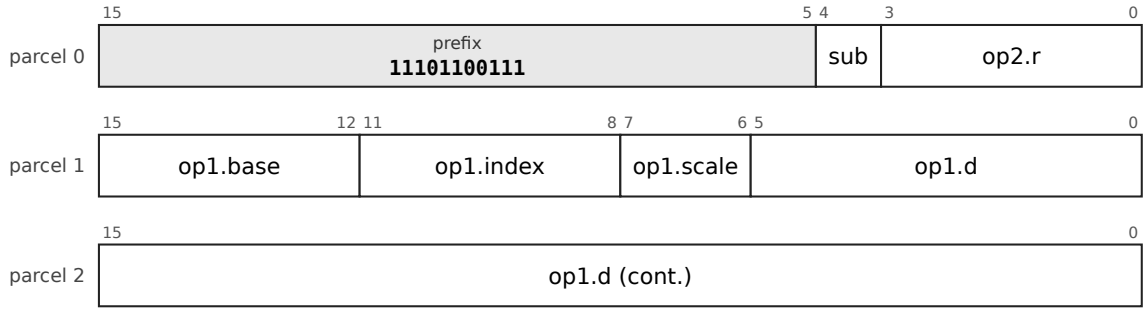
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.

Fields: `sub=p0[4] op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-X22-3: LDF.X22, prefix `11101100111`



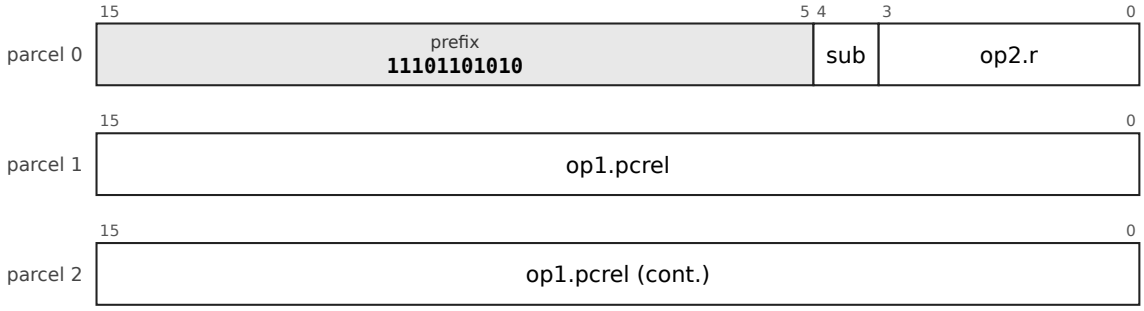
`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

`op2` is `reg4`: register, 4 bits.

`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.

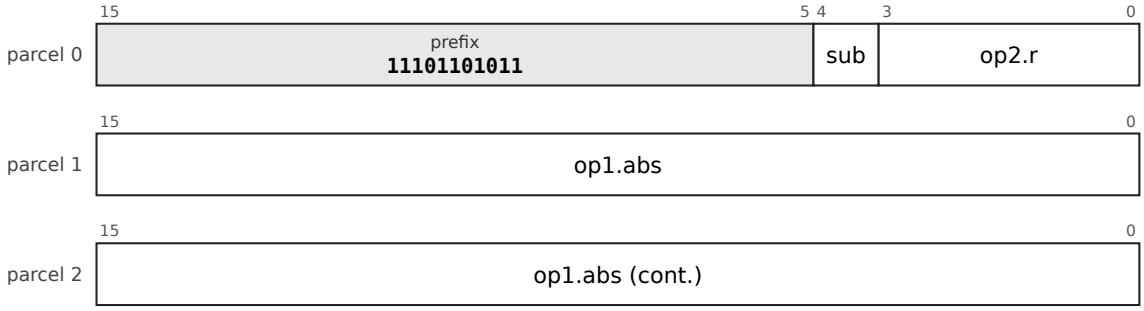
Fields: `sub=p0[4] op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

16-LDF-PC32-3: LDF.PC32, prefix `11101101010`



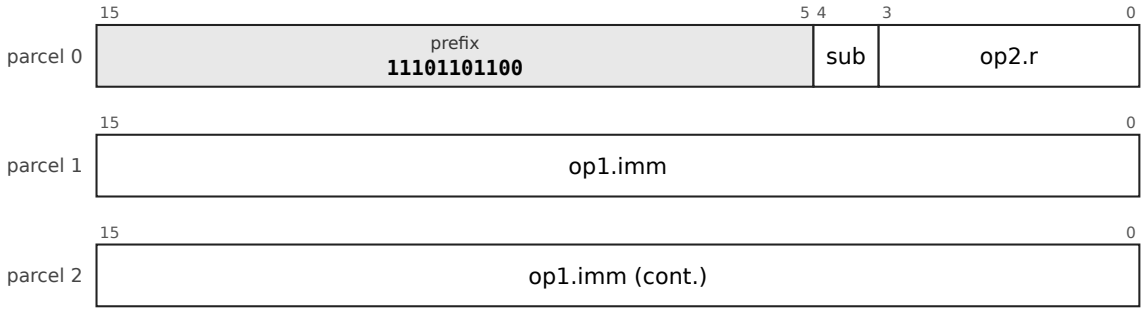
`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[4] op2.r=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-LDF-A32-3: LDF.A32, prefix `11101101011`



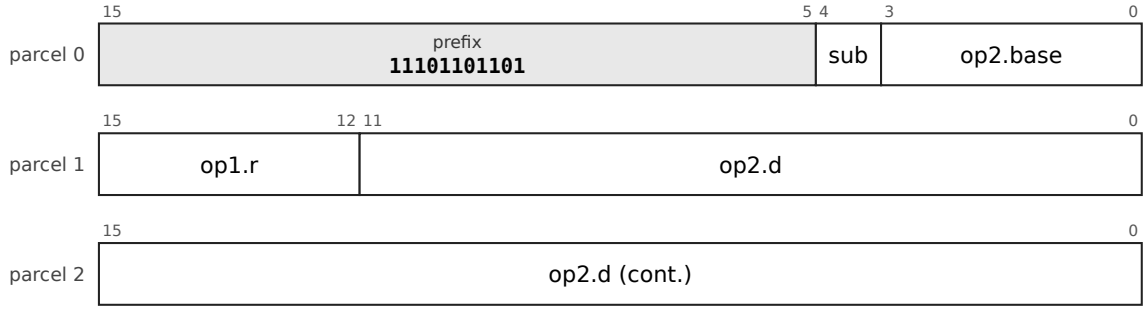
`op1` is `abs32`: absolute address, 32 bits.
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[4] op2.r=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

16-LDFI-I32-3: LDFI.I32, prefix `11101101100`



`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movzw` 0, `movzb` 1.
Fields: `sub=p0[4] op2.r=p0[3:0] op1.imm=p1[15:0]+p2[15:0]`

16-CMF-B28-1: CMF.B28, prefix `11101101101`



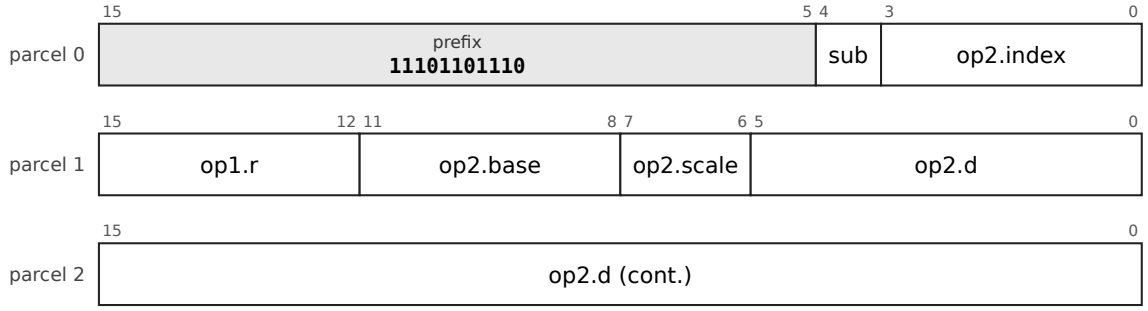
`op1` is `reg4`: register, 4 bits.

`op2` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

16-CMF-X22-1: CMF.X22, prefix `11101101110`



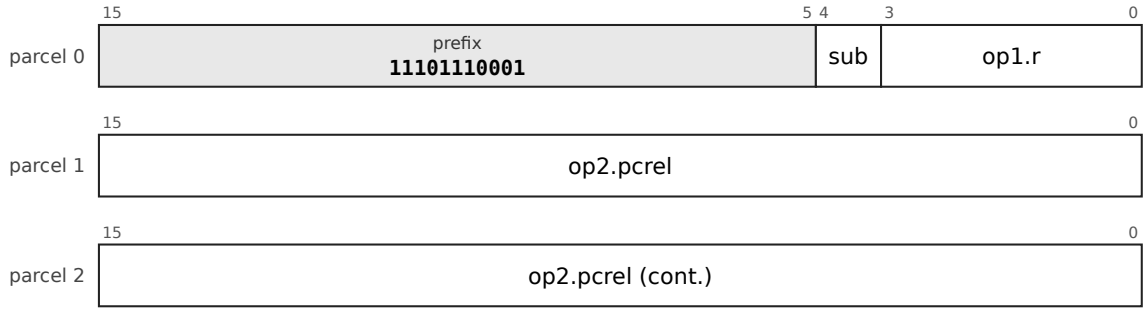
`op1` is `reg4`: register, 4 bits.

`op2` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[4] op2.index=p0[3:0] op1.r=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

16-CMF-PC32-1: CMF.PC32, prefix `11101110001`



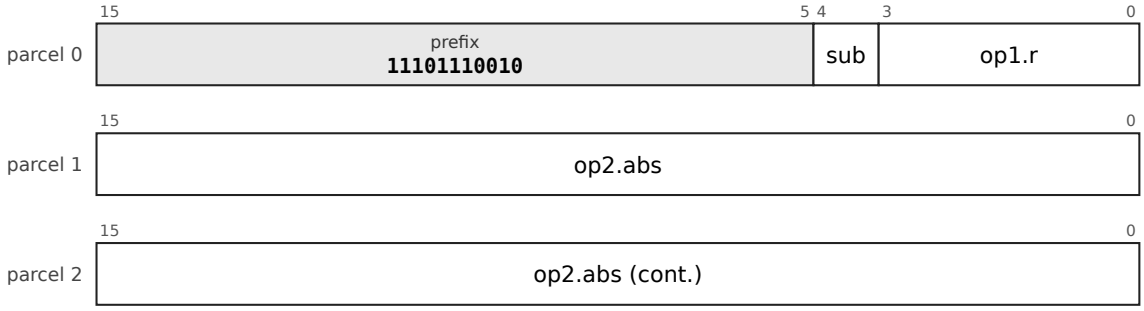
`op1` is `reg4`: register, 4 bits.

`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

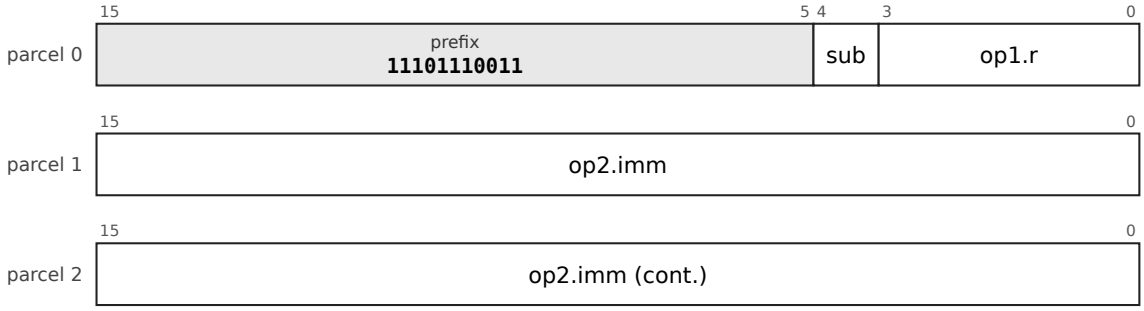
Fields: `sub=p0[4] op1.r=p0[3:0] op2.pcrel=p1[15:0]+p2[15:0]`

16-CMF-A32-1: CMF.A32, prefix `11101110010`



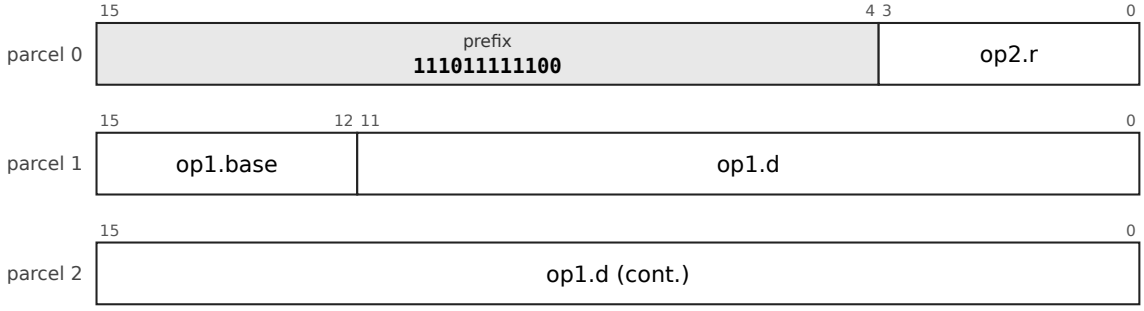
`op1` is `reg4`: register, 4 bits.
`op2` is `abs32`: absolute address, 32 bits.
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[4] op1.r=p0[3:0] op2.abs=p1[15:0]+p2[15:0]`

16-CMF-I32-1: CMF.I32, prefix `11101110011`



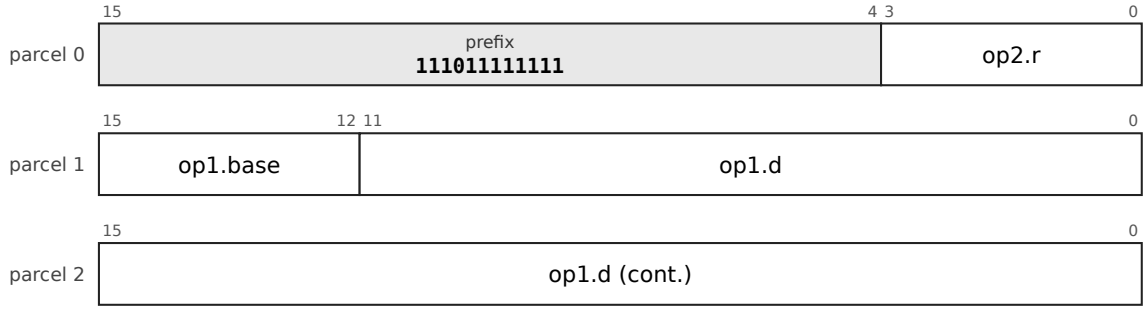
`op1` is `reg4`: register, 4 bits.
`op2` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[4] op1.r=p0[3:0] op2.imm=p1[15:0]+p2[15:0]`

16-LDU-B28-3: LDU.B28, prefix `111011111100`



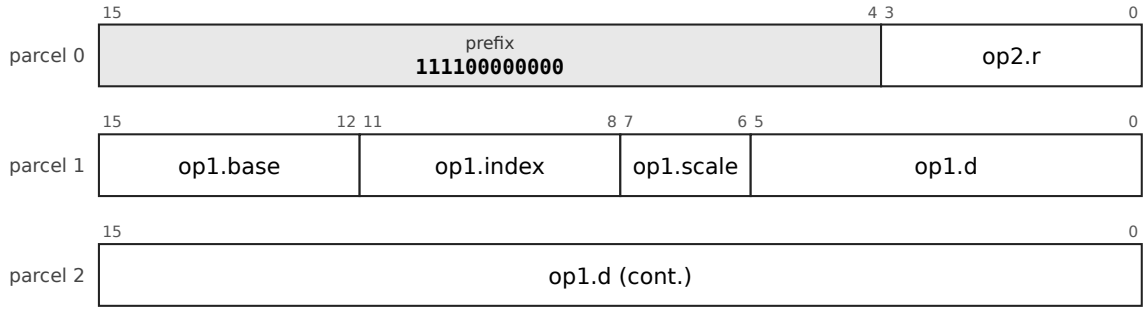
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-B28-4: LDF.B28, prefix `111011111111`



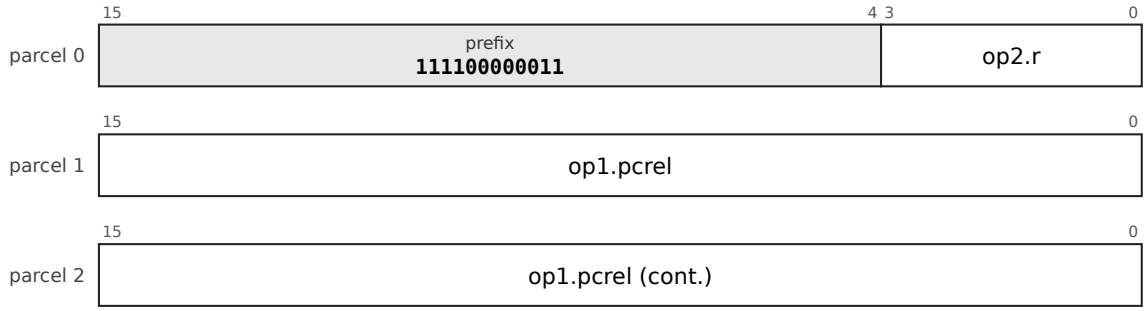
`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-LDF-X22-4: LDF.X22, prefix `111100000000`



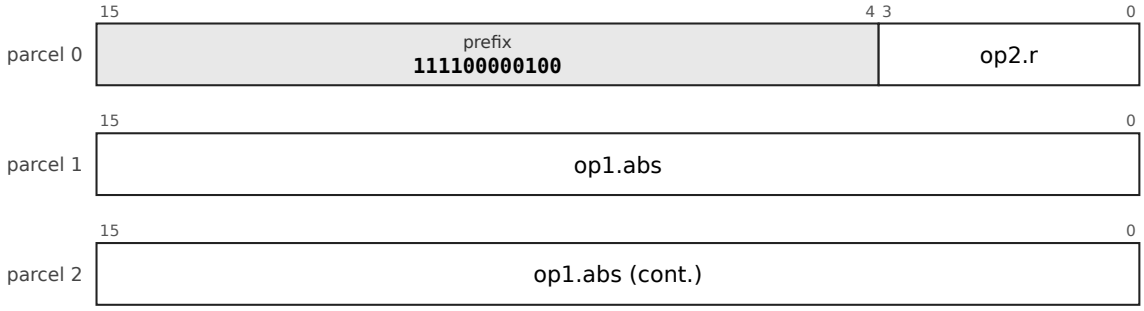
`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

16-LDF-PC32-4: LDF.PC32, prefix `111100000011`



`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`op2` is `reg4`: register, 4 bits.
Fields: `op2.r=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-LDF-A32-4: LDF.A32, prefix `111100000100`

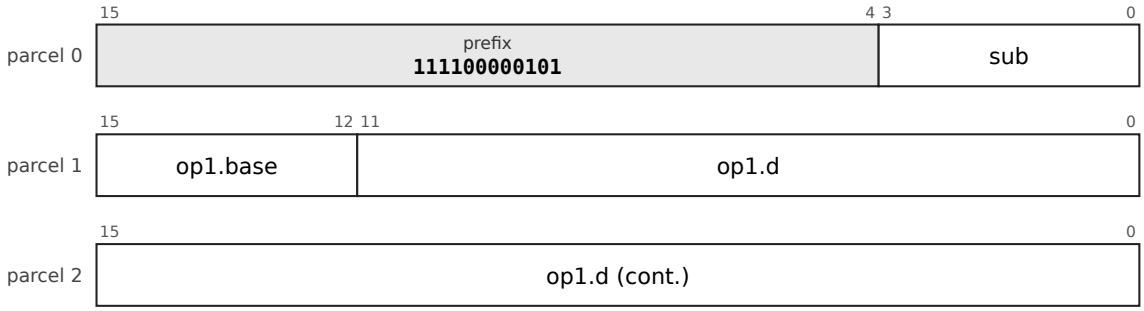


`op1` is `abs32`: absolute address, 32 bits.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

16-SOF-B28-1: SOF.B28, prefix `111100000101`

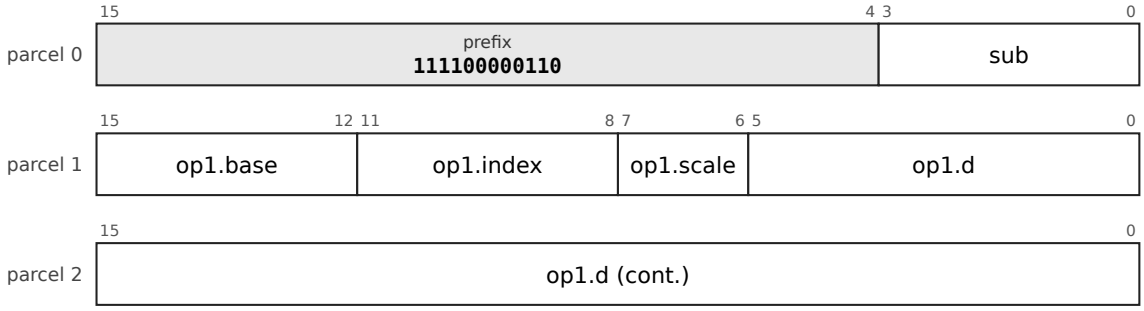


`op1` is `b4d28s-x1`: memory at base register, 4 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.base=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-SOF-X22-1: SOF.X22, prefix `111100000110`

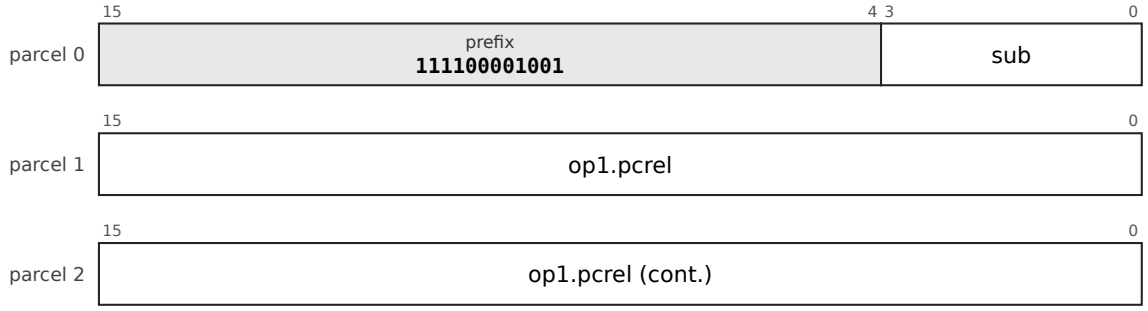


`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

16-SOF-PC32-1: SOF.PC32, prefix `111100001001`

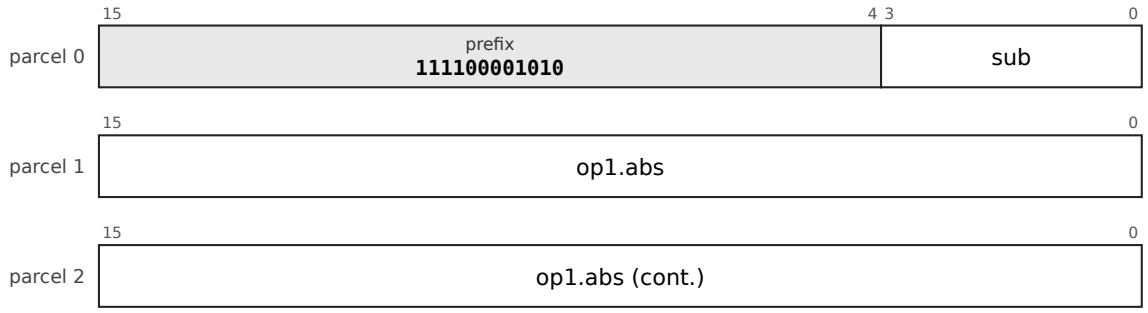


`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-SOF-A32-1: SOF.A32, prefix `111100001010`

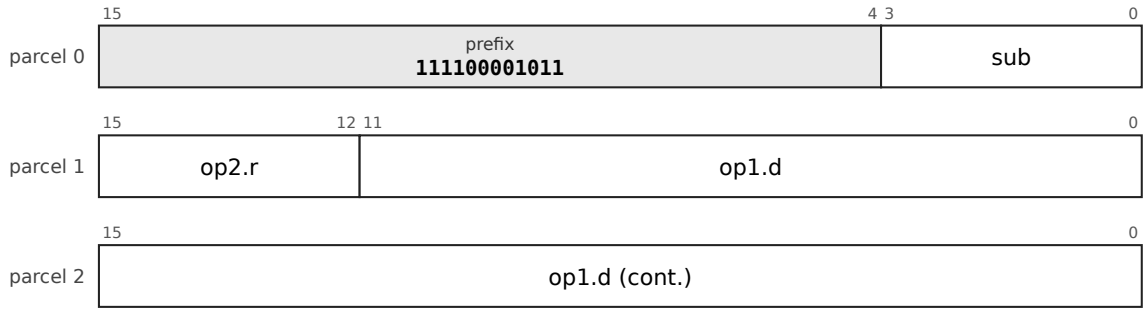


`op1` is `abs32`: absolute address, 32 bits.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

16-DPF-D28-1: DPF.D28, prefix `111100001011`



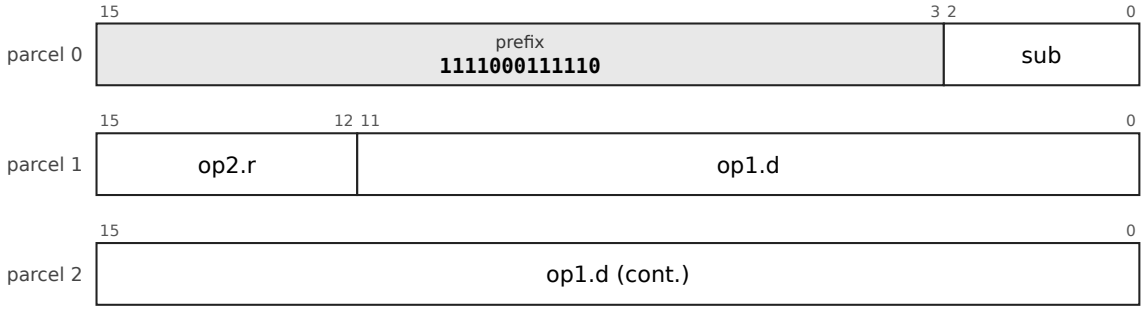
`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

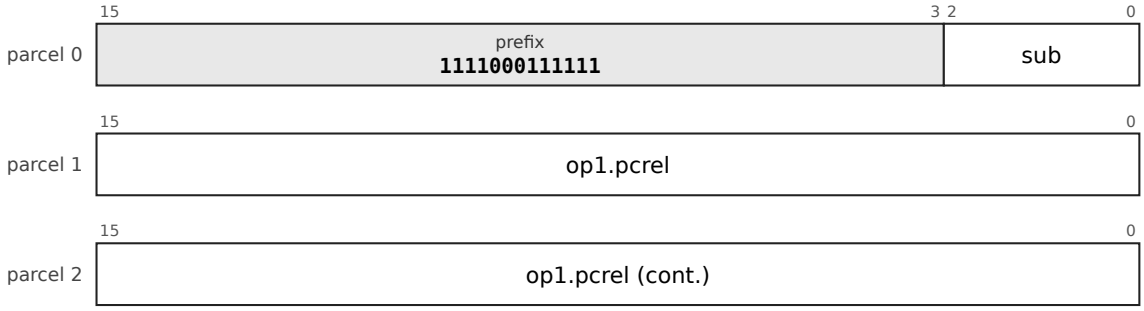
Fields: `sub=p0[3:0] op2.r=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-DPF-D28-2: DPF.D28, prefix `1111000111110`



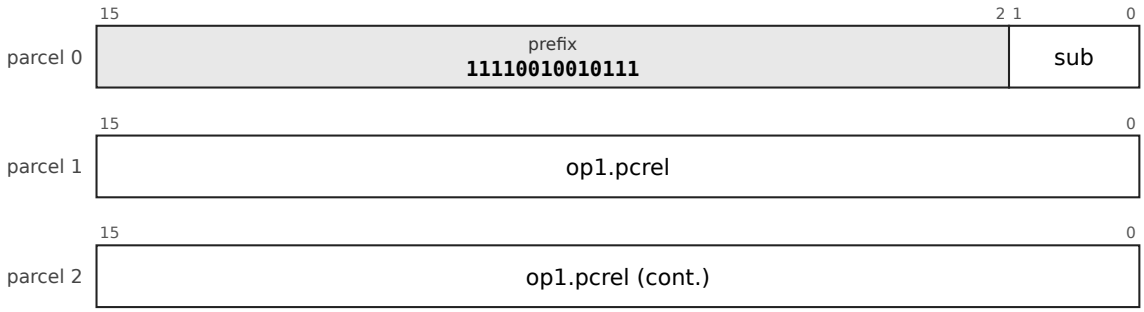
`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg4`: register, 4 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[2:0] op2.r=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-BRF-1: BRF, prefix `1111000111111`



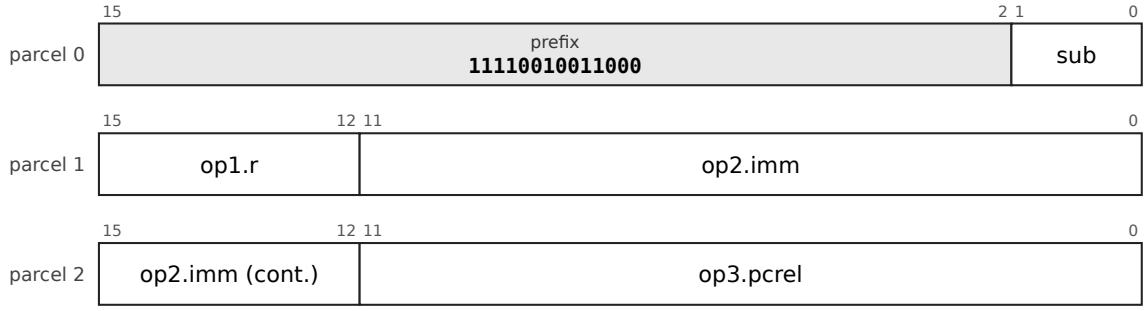
`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (3 bits) selects the mnemonic: `br` 000, `bne` 001, `beq` 010, `bpl` 011, `bmi` 100, `bvc` 101, `bvs` 110, `bcc` 111.
Fields: `sub=p0[2:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-BRF-2: BRF, prefix `11110010010111`



`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (2 bits) selects the mnemonic: `bcs` 00, `bge` 01, `blt` 10, `bgt` 11.
Fields: `sub=p0[1:0] op1.pcrel=p1[15:0]+p2[15:0]`

16-CB3I16-12-b-1: CB3I16_12/b, prefix `11110010011000`



`op1` is `reg4`: register, 4 bits.

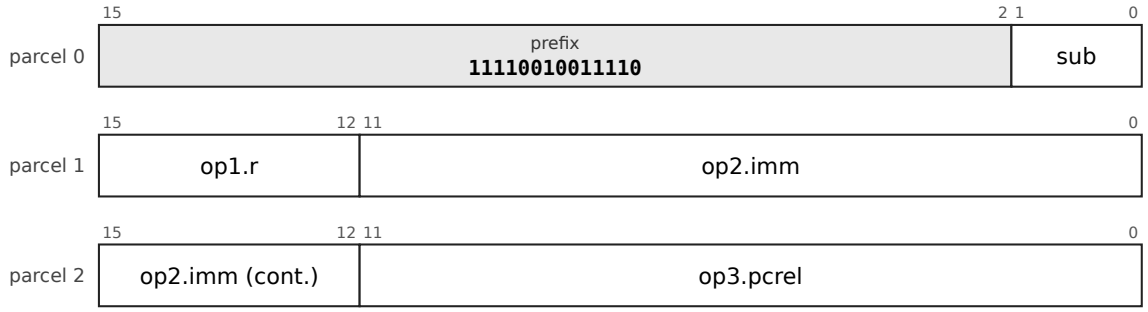
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

16-CB3I16-12-c-1: CB3I16_12/c, prefix `11110010011110`



`op1` is `reg4`: register, 4 bits.

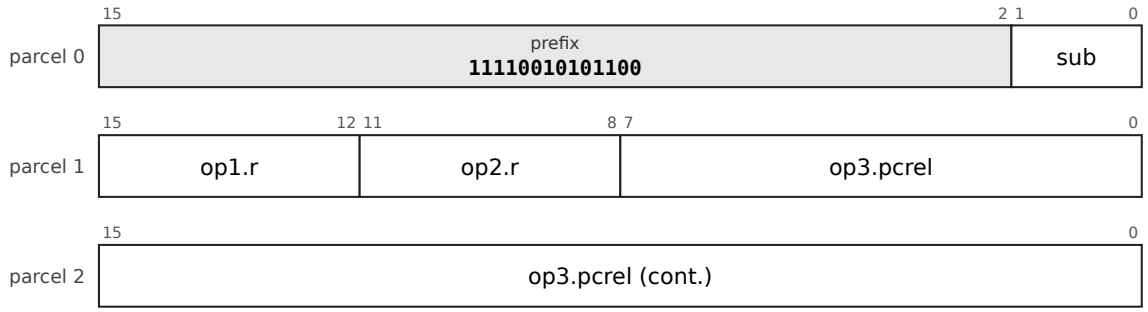
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

16-CB3R24-c-1: CB3R24/c, prefix `11110010101100`



`op1` is `reg4`: register, 4 bits.

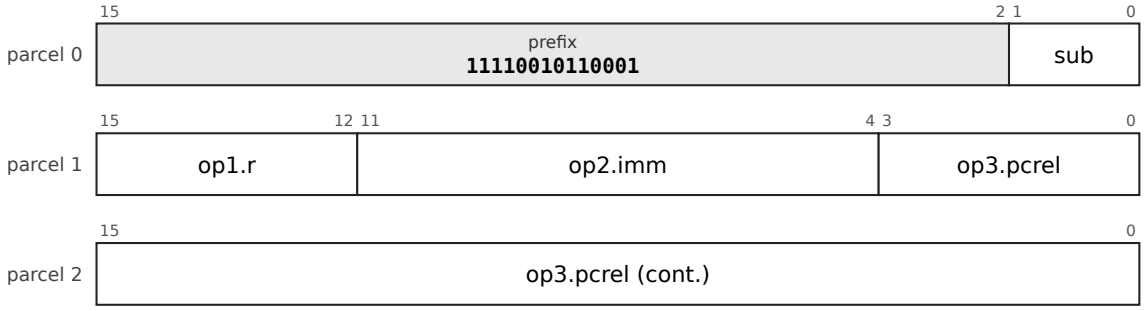
`op2` is `reg4`: register, 4 bits.

`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

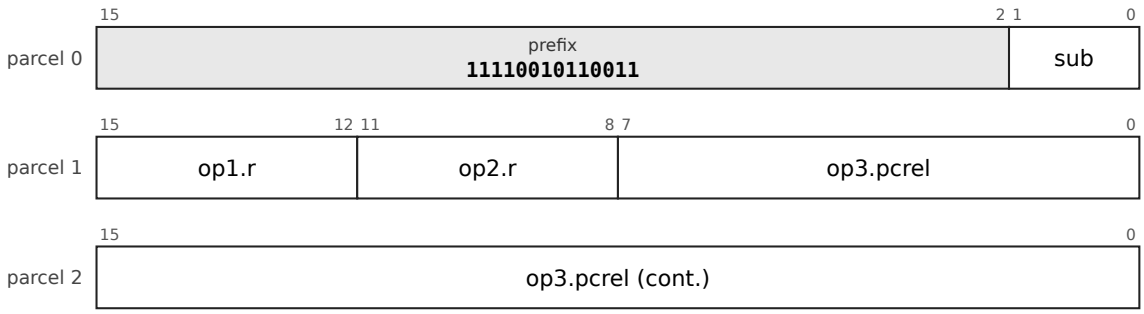
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

16-CB3I8-20-b-1: CB3I8_20/b, prefix `11110010110001`



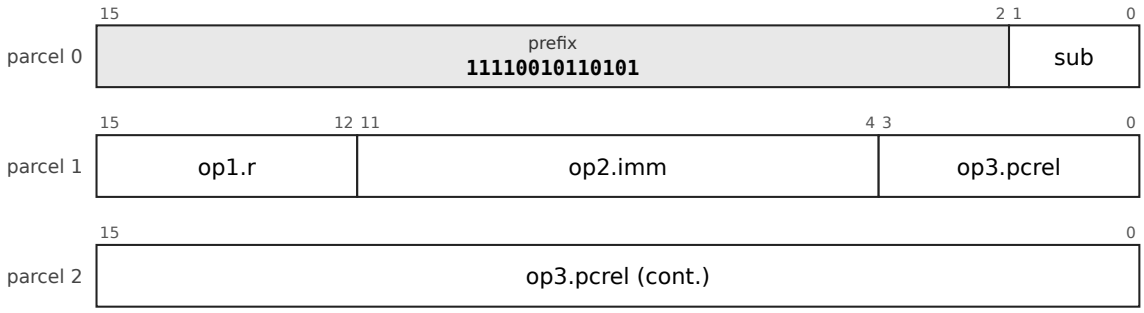
`op1` is `reg4`: register, 4 bits.
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc20-u2`: pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:4] op3.pcrel=p1[3:0]+p2[15:0]`

16-CB3R24-b-1: CB3R24/b, prefix `11110010110011`



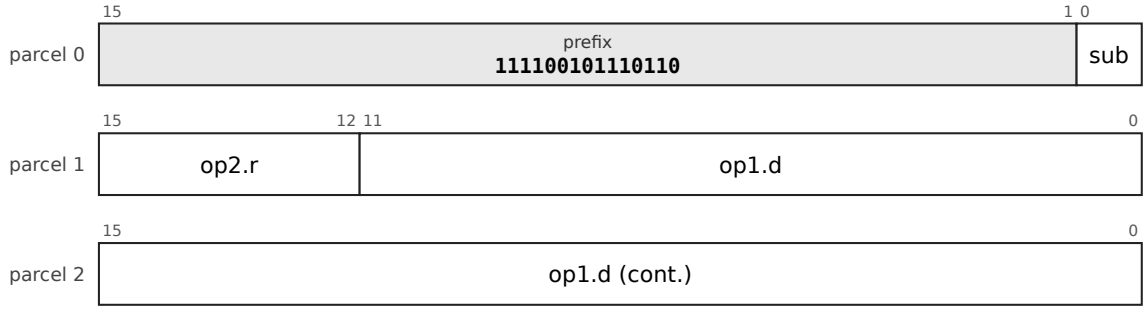
`op1` is `reg4`: register, 4 bits.
`op2` is `reg4`: register, 4 bits.
`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

16-CB3I8-20-c-1: CB3I8_20/c, prefix `11110010110101`



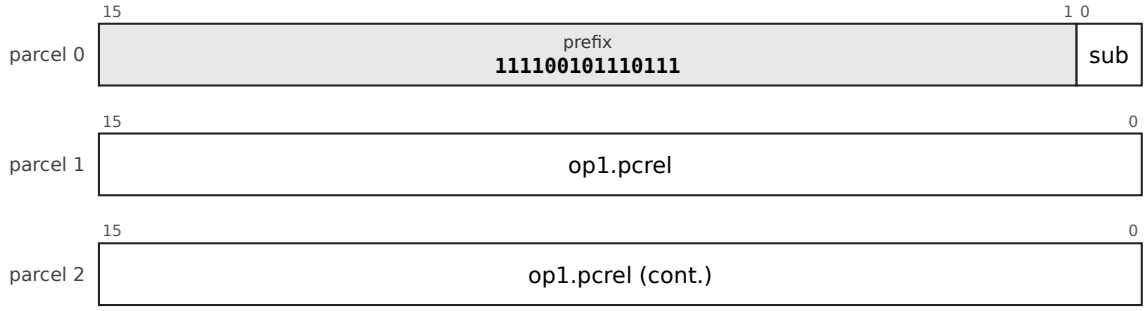
`op1` is `reg4`: register, 4 bits.
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc20-u2`: pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:4] op3.pcrel=p1[3:0]+p2[15:0]`

16-DPF-D28-3: DPF.D28, prefix `111100101110110`



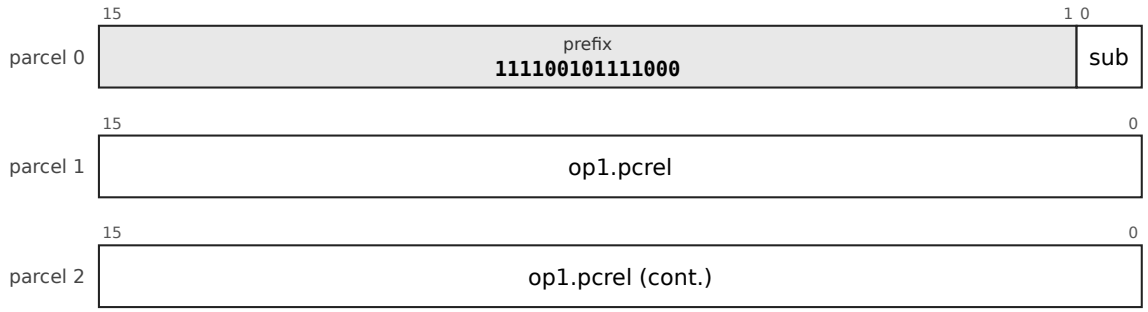
`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg4`: register, 4 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[0] op2.r=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

16-BRF-3: BRF, prefix `111100101110111`



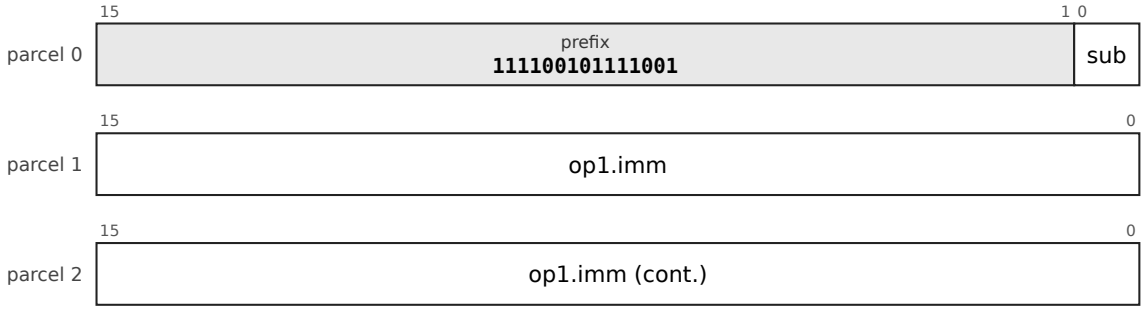
`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (1 bit) selects the mnemonic: `ble` 0, `bhi` 1.
Fields: `sub=p0[0] op1.pcrel=p1[15:0]+p2[15:0]`

16-CALLF-1: CALLF, prefix `111100101111000`



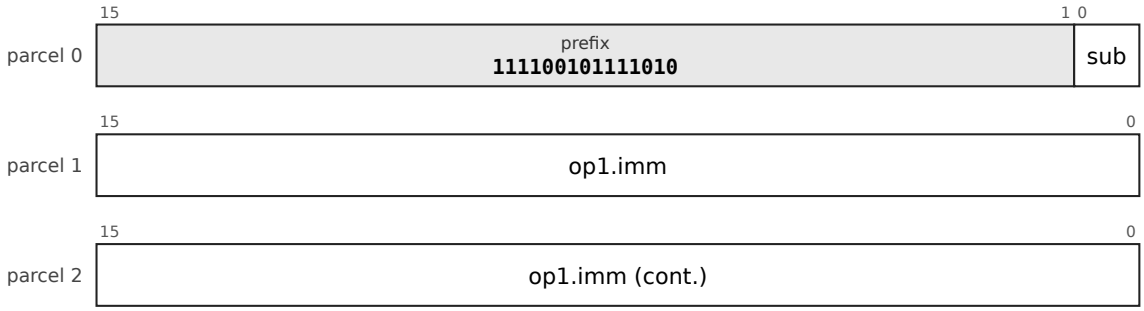
`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (1 bit) selects the mnemonic: `call` 0, `jmp` 1.
Fields: `sub=p0[0] op1.pcrel=p1[15:0]+p2[15:0]`

16-PUSHMF-1: PUSHMF, prefix `111100101111001`



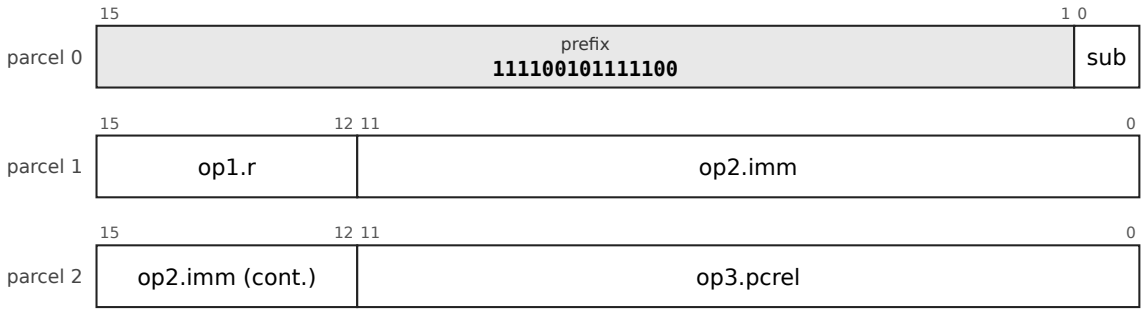
`op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295.
`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.
Fields: `sub=p0[0] op1.imm=p1[15:0]+p2[15:0]`

16-SPADJF-1: SPADJF, prefix `111100101111010`



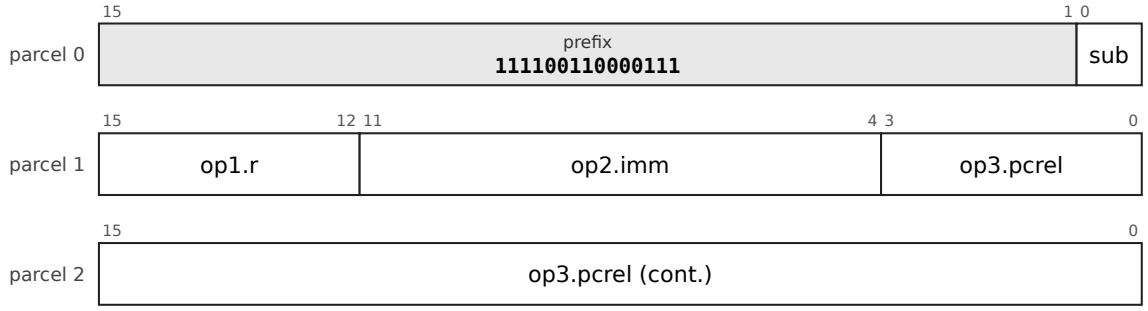
`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
`op2` is `k-sp`: implicit register r14.
`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.
Fields: `sub=p0[0] op1.imm=p1[15:0]+p2[15:0]`

16-CB3I16-12-a-1: CB3I16_12/a, prefix `111100101111100`



`op1` is `reg4`: register, 4 bits.
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.
`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

16-CB3I8-20-a-1: CB3I8_20/a, prefix `111100110000111`



`op1` is `reg4`: register, 4 bits.

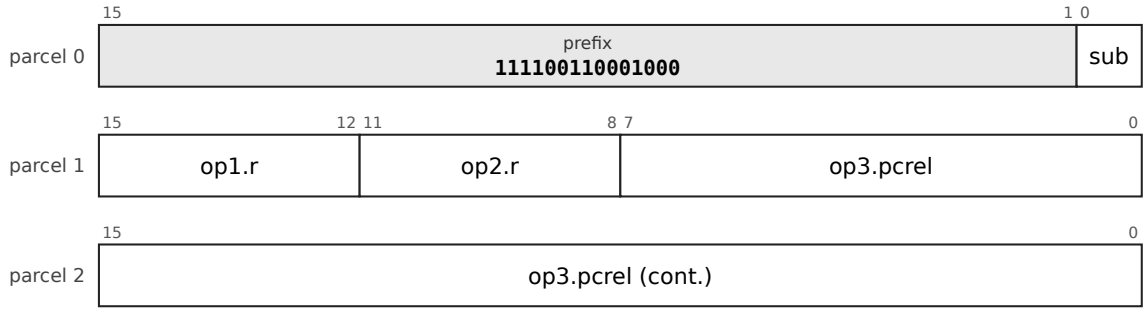
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.

`op3` is `pc20-u2`: pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.imm=p1[11:4] op3.pcrel=p1[3:0]+p2[15:0]`

16-CB3R24-a-1: CB3R24/a, prefix `111100110001000`



`op1` is `reg4`: register, 4 bits.

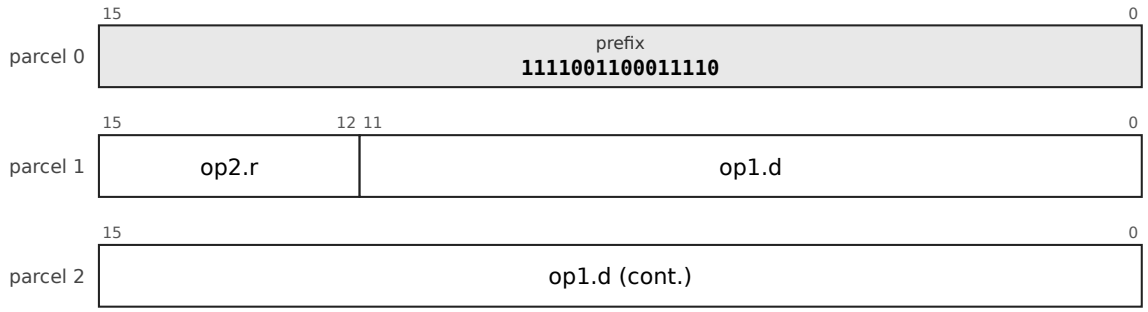
`op2` is `reg4`: register, 4 bits.

`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

16-DPF-D28-4: DPF.D28, prefix `1111001100011110`

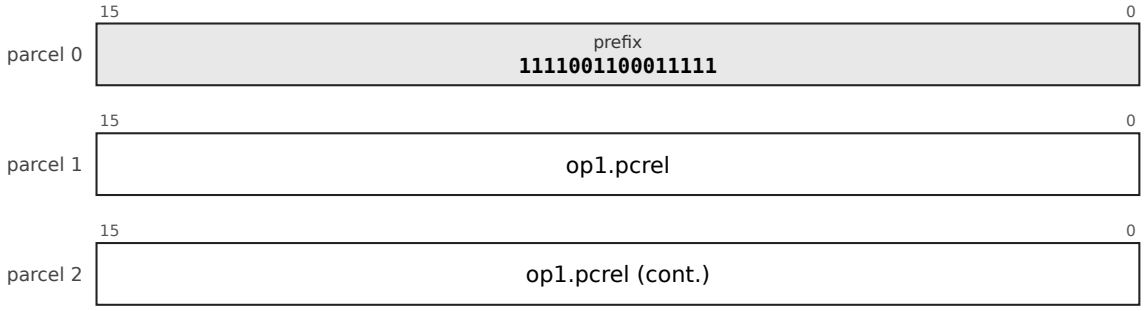


`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

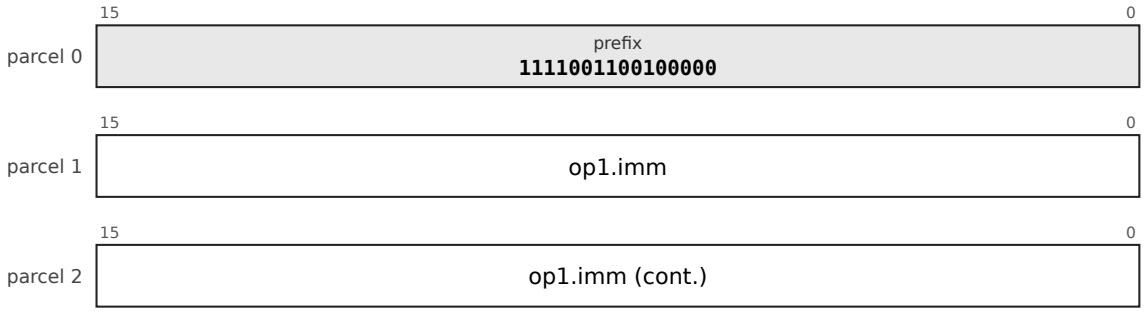
16-BRF-4: BRF, prefix `1111001100011111`



`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).

Fields: `op1.pcrel=p1[15:0]+p2[15:0]`

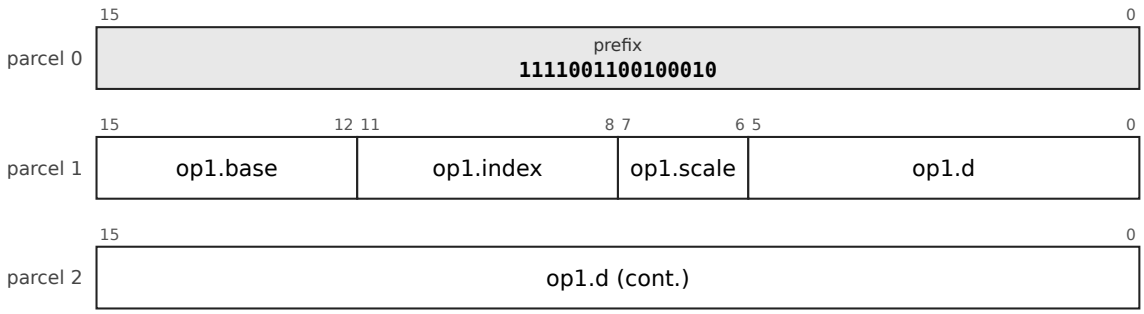
16-PUSHI3-1: PUSHI3, prefix `1111001100100000`



`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

Fields: `op1.imm=p1[15:0]+p2[15:0]`

16-JTAG3-1: JTAG3, prefix `1111001100100010`

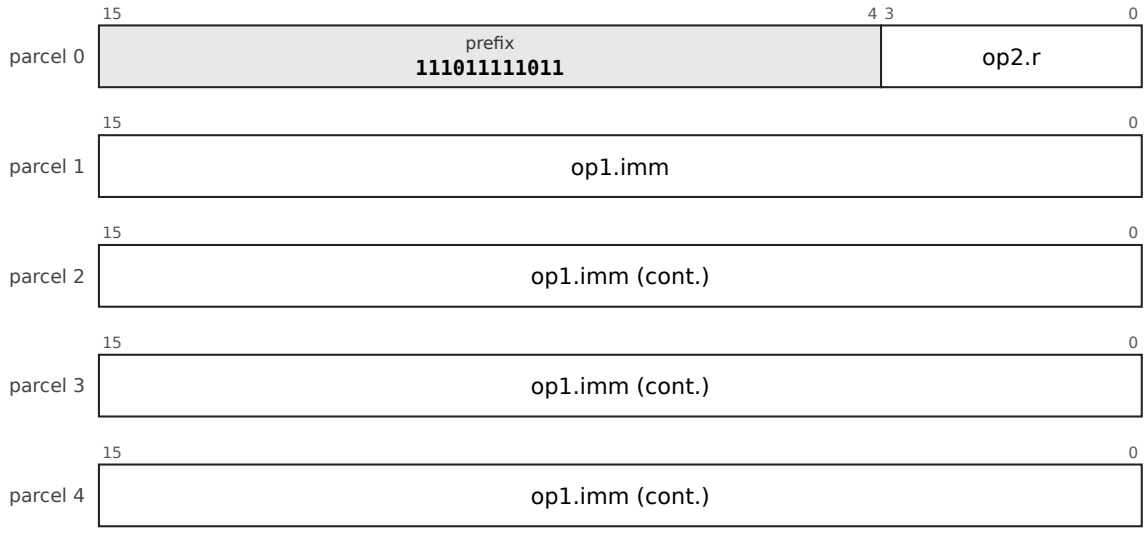


`op1` is `x4-4d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 4 bits, index register, 4 bits.

Fields: `op1.base=p1[15:12]` `op1.index=p1[11:8]` `op1.scale=p1[7:6]` `op1.d=p1[5:0]+p2[15:0]`

Instructions of 5 parcels (10 bytes)

16-MOVQL-1: MOVQL, prefix `111011111011`



`op1` is `imm64s`: signed 64-bit immediate, -9223372036854775808 to 9223372036854775807.

`op2` is `reg4`: register, 4 bits.

Fields: `op2.r=p0[3:0]` `op1.imm=p1[15:0]+p2[15:0]+p3[15:0]+p4[15:0]`

The index of the mnemonics

For every mnemonic, the charts in which it is a sub-opcode or the whole leaf.

- `ABS`: 16-R2-1
- `ADC`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1
- `ADD`: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-SPADJF-1, 16-LDI12-b-1, 16-LDBu8-c-1, 16-SPADJ6-a-1, 16-LDI14-b-1, 16-LDX2-c-1, 16-IMM3-b-1, 16-LDS12-c-1, 16-LDI18-b-1, 16-LDPC16-c-1, 16-LDBs10-c-1, 16-IMM4-b-1, 16-SPADJ8-a-1
- `ADDM`: 16-STF-B28-2, 16-STF-X22-2, 16-STF-PI12-2, 16-STF-PD12-2, 16-STF-PC32-2, 16-STF-A32-2
- `ADDV`: 16-R2-1
- `AESDS`: 16-R3-1
- `AESDSM`: 16-R3-1
- `AESES`: 16-R3-1
- `AESESM`: 16-R3-1
- `AESIM`: 16-R2-1
- `AESKS1`: 16-RRI-1
- `AESKS2`: 16-R3-1
- `AND`: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDI12-b-2, 16-LDBu8-c-1, 16-LDI14-b-2, 16-LDX2-c-1, 16-IMM3-b-2, 16-LDS12-c-1, 16-LDI18-b-2, 16-LDPC16-c-1, 16-LDBs10-c-1, 16-IMM4-b-2
- `ASL`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-2
- `ASR`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-2
- `BCC`: 16-BRF-1, 16-BR2-16-c-1, 16-BR1-6-c-1
- `BCS`: 16-BRF-2, 16-BR2-16-c-1, 16-BR1-6-c-1
- `BEQ`: 16-BRF-1, 16-BR2-16-b-1, 16-BR1-6-b-1, 16-BR1-7-b-1
- `BGE`: 16-BRF-2, 16-BR2-16-c-1, 16-BR1-6-c-1
- `BGT`: 16-BRF-2, 16-BR2-16-c-2, 16-BR1-6-c-2

- BHI: 16-BRF-3, 16-BR2-16-c-2, 16-BR1-6-c-2
- BIC: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDI12-d-2, 16-LDI12-c-1, 16-LDPC12-d-1, 16-IMM3-c-1, 16-LDI14-c-1, 16-LDBs8-d-1
- BICM: 16-STF-B28-2, 16-STF-X22-2, 16-STF-PI12-2, 16-STF-PD12-2, 16-STF-PC32-2, 16-STF-A32-2
- BIS: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDBu8-c-1, 16-LDI12-c-1, 16-LDX2-c-1, 16-LDS12-c-1, 16-LDPC16-c-1, 16-LDBs10-c-1, 16-IMM3-c-1, 16-LDI14-c-1
- BISM: 16-STF-B28-2, 16-STF-X22-2, 16-STF-PI12-2, 16-STF-PD12-2, 16-STF-PC32-2, 16-STF-A32-2
- BIT: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDI12-d-3, 16-LDPC12-d-2, 16-LDBs8-d-2
- BITREV: 16-R2-1
- BLE: 16-BRF-3, 16-BR2-16-c-2, 16-BR1-6-c-2
- BLOS: 16-BRF-4, 16-BR2-16-c-2, 16-BR1-6-c-2
- BLT: 16-BRF-2, 16-BR2-16-c-1, 16-BR1-6-c-1
- BMI: 16-BRF-1, 16-BR2-16-c-1, 16-BR1-6-c-1
- BNE: 16-BRF-1, 16-BR2-16-b-1, 16-BR1-6-b-1, 16-BR1-7-b-1
- BNTAG: 16-TAG-1
- BPL: 16-BRF-1, 16-BR2-16-c-1, 16-BR1-6-c-1
- BPT: 16-SYS0-1
- BR: 16-BRF-1, 16-BR2-16-a-1, 16-BR1-6-a-1, 16-BR1-8-a-1, 16-BR1-9-a-1
- BSEL: 16-R4-1
- BSWAP: 16-R2-1
- BSWAPL: 16-R2-1
- BSWAPW: 16-R2-1
- BTAG: 16-TAG-1
- BVC: 16-BRF-1, 16-BR2-16-c-1, 16-BR1-6-c-1
- BVS: 16-BRF-1, 16-BR2-16-c-1, 16-BR1-6-c-1
- CALL: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-CALLF-1, 16-SOPR-3, 16-CALL2-16-c-1, 16-CALL2-18-c-1, 16-SOBu12-x-1, 16-SOX6-x-1
- CALLI: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOBu12-x-1, 16-SOX6-x-1
- CAS: 16-R4-1
- CASDB: 16-R4-1
- CBCC: 16-CBR8-c-1, 16-CB3I16-12-c-1, 16-CBI4-8-c-1, 16-CBI6-8-c-1, 16-CB3R24-c-1, 16-CBR10-c-1, 16-CBI8-8-c-1, 16-CBI4-10-c-1, 16-CB3I8-20-c-1, 16-CBI10-8-c-1, 16-CBI6-10-c-1
- CBCS: 16-CBR8-c-1, 16-CB3I16-12-c-1, 16-CBI4-8-c-1, 16-CBI6-8-c-1, 16-CB3R24-c-1, 16-CBR10-c-1, 16-CBI8-8-c-1, 16-CBI4-10-c-1, 16-CB3I8-20-c-1, 16-CBI10-8-c-1, 16-CBI6-10-c-1
- CBEQ: 16-CB3I16-12-a-1, 16-CBR8-a-1, 16-CBZ2-12-a-1, 16-CBI5-8-a-1, 16-CBI6-8-a-1, 16-CBI10-8-a-1, 16-CB3I8-20-a-1, 16-CBZ1-4-a-1, 16-CBI5-10-a-1, 16-CB3R24-a-1, 16-CBZ1-5-a-1, 16-CBI8-10-a-1, 16-CBR10-a-1, 16-CBI4-12-a-1, 16-CBZ1-7-a-1, 16-CBZ2-14-a-1, 16-CBI6-12-a-1
- CBGE: 16-CB3I16-12-b-1, 16-CBR8-b-1, 16-CBI8-8-b-1, 16-CBZ2-12-b-1, 16-CB3I8-20-b-1, 16-CB3R24-b-1, 16-CBI4-10-b-1
- CBGT: 16-CB3I16-12-b-1, 16-CBR8-b-1, 16-CBI8-8-b-1, 16-CBZ2-12-b-1, 16-CB3I8-20-b-1, 16-CB3R24-b-1, 16-CBI4-10-b-1
- CBHI: 16-CBR8-c-1, 16-CB3I16-12-c-1, 16-CBI4-8-c-1, 16-CBI6-8-c-1, 16-CB3R24-c-1, 16-CBR10-c-1, 16-CBI8-8-c-1, 16-CBI4-10-c-1, 16-CB3I8-20-c-1, 16-CBI10-8-c-1, 16-CBI6-10-c-1
- CBLE: 16-CB3I16-12-b-1, 16-CBR8-b-1, 16-CBI8-8-b-1, 16-CBZ2-12-b-1, 16-CB3I8-20-b-1, 16-CB3R24-b-1, 16-CBI4-10-b-1

- CBLOS: 16-CBR8-c-1, 16-CB3I16-12-c-1, 16-CBI4-8-c-1, 16-CBI6-8-c-1, 16-CB3R24-c-1, 16-CBR10-c-1, 16-CBI8-8-c-1, 16-CBI4-10-c-1, 16-CB3I8-20-c-1, 16-CBI10-8-c-1, 16-CBI6-10-c-1
- CBLT: 16-CB3I16-12-b-1, 16-CBR8-b-1, 16-CBI8-8-b-1, 16-CBZ2-12-b-1, 16-CB3I8-20-b-1, 16-CB3R24-b-1, 16-CBI4-10-b-1
- CBMI: 16-CBDI-1, 16-CBDR-1
- CBNE: 16-CB3I16-12-a-1, 16-CBR8-a-1, 16-CBZ2-12-a-1, 16-CBI5-8-a-1, 16-CBI6-8-a-1, 16-CBI10-8-a-1, 16-CB3I8-20-a-1, 16-CBZ1-4-a-1, 16-CBI5-10-a-1, 16-CB3R24-a-1, 16-CBZ1-5-a-1, 16-CBI8-10-a-1, 16-CBR10-a-1, 16-CBI4-12-a-1, 16-CBZ1-7-a-1, 16-CBZ2-14-a-1, 16-CBI6-12-a-1
- CBPL: 16-CBDI-1, 16-CBDR-1
- CBVC: 16-CBDI-1, 16-CBDR-1
- CBVS: 16-CBDI-1, 16-CBDR-1
- CLMULH: 16-R3-1
- CLMULL: 16-R3-1
- CLO: 16-R2-1
- CLR: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1, 16-SOBu12-a-1, 16-SOX6-a-1, 16-SOBs12-a-1
- CLZ: 16-R2-1
- CMP: 16-CMF-B28-1, 16-CMF-X22-1, 16-CMF-PI12-1, 16-CMF-PD12-1, 16-CMF-PC32-1, 16-CMF-A32-1, 16-CMF-I32-1, 16-RRF-2, 16-CMBu8-a-1, 16-CMI12-a-1, 16-CMBu10-a-1, 16-CMI18-a-1, 16-CMPC16-a-1, 16-CMX2-a-1, 16-CMS12-a-1, 16-CMBu12-a-1, 16-CMBs10-a-1
- COM: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1
- CTO: 16-R2-1
- CTZ: 16-R2-1
- DEC: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1, 16-SOBu12-a-1, 16-SOX6-a-1, 16-SOBs12-a-1
- DIV: 16-R3-1
- DIVU: 16-R3-1
- DSHR: 16-BFx-1
- EXTB: 16-BFy-1
- EXTZB: 16-BFy-1
- FABS: 16-R2-1
- FADD: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDPC12-d-1, 16-LDBs8-d-1
- FADD16: 16-R2-1
- FADD32: 16-R2-1
- FCLASS: 16-R2-1
- FCLASS16: 16-R2-1
- FCLASS32: 16-R2-1
- FCMP: 16-CMF-B28-1, 16-CMF-X22-1, 16-CMF-PI12-1, 16-CMF-PD12-1, 16-CMF-PC32-1, 16-CMF-A32-1, 16-CMF-I32-1, 16-RRF-2
- FCMP16: 16-R2-1
- FCMP32: 16-R2-1
- FCOPYSIGN: 16-R3-1
- FCVTDH: 16-R2-1
- FCVTDQ: 16-R2-1
- FCVTDQU: 16-R2-1
- FCVTDS: 16-R2-1
- FCVTHD: 16-R2-1
- FCVTHL: 16-R2-1
- FCVTHS: 16-R2-1

- FCVTLH: 16-R2-1
- FCVTLS: 16-R2-1
- FCVTQD: 16-R2-1
- FCVTQUD: 16-R2-1
- FCVTSD: 16-R2-1
- FCVTSH: 16-R2-1
- FCVTSL: 16-R2-1
- FDIV: 16-R2-1
- FDIV16: 16-R2-1
- FDIV32: 16-R2-1
- FENCE: 16-SYS0-1
- FMADD: 16-R4-1
- FMADD16: 16-R4-1
- FMADD32: 16-R4-1
- FMAX: 16-R3-1
- FMIN: 16-R3-1
- FMSUB: 16-R4-1
- FMSUB16: 16-R4-1
- FMSUB32: 16-R4-1
- FMUL: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDPC12-d-2, 16-LDBs8-d-2
- FMUL16: 16-R2-1
- FMUL32: 16-R2-1
- FNEG: 16-R2-1
- FNMADD: 16-R4-1
- FNMADD16: 16-R4-1
- FNMADD32: 16-R4-1
- FNMSUB: 16-R4-1
- FNMSUB16: 16-R4-1
- FNMSUB32: 16-R4-1
- FRINT: 16-R2-1
- FRINTA: 16-R2-1
- FRINTM: 16-R2-1
- FRINTP: 16-R2-1
- FRINTZ: 16-R2-1
- FSFLAGS: 16-R2-1
- FSQRT: 16-R2-1
- FSQRT16: 16-R2-1
- FSQRT32: 16-R2-1
- FSUB: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDPC12-d-1, 16-LDBs8-d-1
- FSUB16: 16-R2-1
- FSUB32: 16-R2-1
- HALT: 16-SYS0-1
- HINT: 16-SYSI-1
- ICINV: 16-SYSR-2
- ICINVA: 16-SYS0-1
- INC: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1, 16-SOBu12-a-1, 16-SOX6-a-1, 16-SOBs12-a-1
- INSB: 16-BFy-1

- ISYNC: 16-SYS0-1
- JMP: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-CALLF-1, 16-SOPR-3, 16-CALL2-16-j-1, 16-SOBu12-x-1, 16-SOX6-x-1
- JMPI: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOBu12-x-1, 16-SOX6-x-1
- JTAG: 16-JTAG3-1, 16-JTAG2-1
- LEA: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-DPF-D28-2, 16-LEASx12-a-1, 16-LDPC12-a-1, 16-LDBs8-a-1, 16-LDPC14-a-1, 16-LDPC16-a-1, 16-LDBu10-a-1, 16-LDHHH1-a-1, 16-LEASP4-a-1, 16-LDSP4-a-1, 16-LDBu12-a-1, 16-LDHHH3-a-1, 16-LDX6-a-1, 16-LDBs10-a-1, 16-LDSP5-a-1, 16-LEAS12-a-1, 16-LD0-a-1, 16-LEASP6-a-1, 16-LDBs14-a-1, 16-LDHRH2-a-1, 16-LDSP7-a-1, 16-LDX8-a-1, 16-LDHHR2-a-1, 16-LEAS14-a-1
- LL: 16-R3-1
- LPAD: 16-SYS0-1
- MAESCTR: 16-R4-1
- MASKEQZ: 16-R3-1
- MASKNEZ: 16-R3-1
- MAX: 16-R3-1
- MAXU: 16-R3-1
- MCMP: 16-R3-1
- MCPY: 16-R3-1
- MFASID: 16-SYSR-1
- MFCPUID: 16-SYSR-2
- MFDP: 16-SYSR-1
- MFPS: 16-SYSR-1
- MFSR: 16-MFSR-1
- MIN: 16-R3-1
- MINU: 16-R3-1
- MMOV: 16-R3-1
- MOD: 16-R3-1
- MODU: 16-R3-1
- MOV2: 16-FX-MOV2-1
- MOVB: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-2, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-1, 16-RRF-1, 16-STBu8-b-1, 16-LDI12-d-2, 16-LDBu8-b-1, 16-STX2-b-1, 16-LDX2-b-1, 16-STBs8-b-1, 16-STBu10-b-1, 16-STHHH1-b-1, 16-STBu12-b-1, 16-LDBu10-b-1, 16-LDHHH1-b-1, 16-LDBu12-b-1, 16-STX6-b-1, 16-LDPC12-b-1, 16-LDBs8-b-1, 16-LDX4-b-1, 16-LDPC14-b-1, 16-STSP4-b-1, 16-STHHH2-b-1, 16-STBs10-b-1
- MOVB.D: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- MOVB.U: 16-LDU-B28-1, 16-STU-B28-1, 16-LDU-B12-1, 16-STU-B12-1, 16-LDU-X6-1, 16-STU-X6-1
- MOVB.UD: 16-LDU-B28-2, 16-LDU-B12-2, 16-LDU-X6-2
- MOVCALL: 16-FX-MOVCALL3-1, 16-FX-MOVCALL2-1
- MOVL: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-2, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-1, 16-RRF-1, 16-STBs8-a-1, 16-LDPC12-a-1, 16-LDBs8-a-1, 16-LDPC14-a-1, 16-STX2-a-1, 16-LDPC16-a-1, 16-LDBu10-a-1, 16-LDHHH1-a-1, 16-STHHH1-a-1, 16-STBu10-a-1, 16-LDSP4-a-1, 16-STSP4-a-1, 16-LDBu12-a-1, 16-LDHHH3-a-1, 16-LDX6-a-1, 16-STBu12-a-1, 16-STX6-a-1, 16-LDBs10-a-1, 16-LDSP5-a-1, 16-STHHH3-a-1, 16-LD0-a-1, 16-STSP5-a-1, 16-ST0-a-1, 16-LDBs14-a-1, 16-STBs10-a-1, 16-LDHRH2-a-1, 16-LDSP7-a-1, 16-LDX8-a-1, 16-LDHHR2-a-1
- MOVL.D: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- MOVL.U: 16-LDU-B28-1, 16-STU-B28-1, 16-LDU-B12-1, 16-STU-B12-1, 16-LDU-X6-1, 16-STU-X6-1

- movl.ud: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movq: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDI12-a-1, 16-STBs8-a-1, 16-LDPC12-a-1, 16-LDBs8-a-1, 16-LDPC14-a-1, 16-LDI14-a-1, 16-STX2-a-1, 16-LDI16-a-1, 16-IMM3-a-1, 16-LDPC16-a-1, 16-LDBu10-a-1, 16-LDHHH1-a-1, 16-STHHH1-a-1, 16-STBu10-a-1, 16-LDSP4-a-1, 16-STSP4-a-1, 16-LDBu12-a-1, 16-IMM5-a-1, 16-LDHHH3-a-1, 16-LDX6-a-1, 16-IMM7-a-1, 16-STBu12-a-1, 16-STX6-a-1, 16-LDBs10-a-1, 16-LDSP5-a-1, 16-STHHH3-a-1, 16-LD0-a-1, 16-STSP5-a-1, 16-LDI18-a-1, 16-ST0-a-1, 16-LDBs14-a-1, 16-STBs10-a-1, 16-LDHRH2-a-1, 16-LDSP7-a-1, 16-LDX8-a-1, 16-LDHHR2-a-1
- movq.d: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movq.u: 16-LDU-B28-1, 16-STU-B28-1, 16-LDU-B12-1, 16-STU-B12-1, 16-LDU-X6-1, 16-STU-X6-1
- movq.ud: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movql: 16-MOVQL-1
- movub: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-2
- movul: 16-LDF-B28-3, 16-LDF-X22-3, 16-LDF-PI12-3, 16-LDF-PD12-3, 16-LDF-PC32-3, 16-LDF-A32-3, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-3
- movuq: 16-LDF-B28-4, 16-LDF-X22-4, 16-LDF-PI12-4, 16-LDF-PD12-4, 16-LDF-PC32-4, 16-LDF-A32-4, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-4
- movuw: 16-LDF-B28-3, 16-LDF-X22-3, 16-LDF-PI12-3, 16-LDF-PD12-3, 16-LDF-PC32-3, 16-LDF-A32-3, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-3
- movw: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-2, 16-STF-B28-1, 16-STF-X22-1, 16-STF-PI12-1, 16-STF-PD12-1, 16-STF-PC32-1, 16-STF-A32-1, 16-DPF-D28-1, 16-RRF-1, 16-STBu8-b-1, 16-LDBu8-b-1, 16-STX2-b-1, 16-LDX2-b-1, 16-STBs8-b-1, 16-STBu10-b-1, 16-STHHH1-b-1, 16-STBu12-b-1, 16-LDBu10-b-1, 16-LDHHH1-b-1, 16-LDBu12-b-1, 16-STX6-b-1, 16-LDPC12-b-1, 16-LDBs8-b-1, 16-LDX4-b-1, 16-LDPC14-b-1, 16-STSP4-b-1, 16-STHHH2-b-1, 16-STBs10-b-1
- movw.d: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movw.u: 16-LDU-B28-1, 16-STU-B28-1, 16-LDU-B12-1, 16-STU-B12-1, 16-LDU-X6-1, 16-STU-X6-1
- movw.ud: 16-LDU-B28-2, 16-LDU-B12-2, 16-LDU-X6-2
- movzb: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-3, 16-DPF-D28-1, 16-RRF-1, 16-LDPC12-a-1, 16-LDBs8-a-1, 16-LDPC14-a-1, 16-LDPC16-a-1, 16-LDBu10-a-1, 16-LDHHH1-a-1, 16-LDSP4-a-1, 16-LDBu12-a-1, 16-LDHHH3-a-1, 16-LDX6-a-1, 16-LDBs10-a-1, 16-LDSP5-a-1, 16-LD0-a-1, 16-LDBs14-a-1, 16-LDHRH2-a-1, 16-LDSP7-a-1, 16-LDX8-a-1, 16-LDHHR2-a-1
- movzb.d: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzb.u: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzb.ud: 16-LDU-B28-3, 16-LDU-B12-3, 16-LDU-X6-3
- movzl: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-2, 16-DPF-D28-1, 16-RRF-1, 16-LDBu8-b-1, 16-LDX2-b-1, 16-LDBu10-b-1, 16-LDHHH1-b-1, 16-LDBu12-b-1, 16-LDPC12-b-1, 16-LDBs8-b-1, 16-LDX4-b-1, 16-LDPC14-b-1
- movzl.d: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzl.u: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzl.ud: 16-LDU-B28-2, 16-LDU-B12-2, 16-LDU-X6-2
- movzw: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-3, 16-DPF-D28-1, 16-RRF-1, 16-LDBu8-b-1, 16-LDX2-b-1, 16-LDBu10-b-1, 16-LDHHH1-b-1, 16-LDBu12-b-1, 16-LDPC12-b-1, 16-LDBs8-b-1, 16-LDX4-b-1, 16-LDPC14-b-1
- movzw.d: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzw.u: 16-LDU-B28-1, 16-LDU-B12-1, 16-LDU-X6-1
- movzw.ud: 16-LDU-B28-2, 16-LDU-B12-2, 16-LDU-X6-2
- mscn: 16-R3-1
- mscnn: 16-R3-1

- `MSET`: 16-R3-1
- `MSHA256`: 16-R3-1
- `MSHA512`: 16-R3-1
- `MTASID`: 16-SYSR-1
- `MTDMW`: 16-SYSR-1
- `MTDP`: 16-SYSR-1
- `MTPS`: 16-SYSR-1
- `MTSR`: 16-MTSR-1
- `MUL`: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDI12-d-1, 16-LDI12-c-1, 16-LDPC12-d-1, 16-IMM3-c-1, 16-LDI14-c-1, 16-LDBs8-d-1
- `MULH`: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDPC12-d-1, 16-LDBs8-d-1
- `MULHU`: 16-R3-1
- `MULV`: 16-R2-1
- `MVAL`: 16-R2-1
- `NEG`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1
- `NOP`: 16-SYS0-1
- `POP`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-2, 16-SOBu12-a-2, 16-SOX6-a-2, 16-SOBs12-a-2
- `POPCNT`: 16-R2-1
- `POPM`: 16-PUSHMF-1, 16-PUSHM16a-a-1, 16-PUSHM8-a-1
- `POPMRET`: 16-FX-POPMRET-1
- `POPRET`: 16-FX-POPRET-1
- `POPX`: 16-SYS0-1
- `PROBER`: 16-R2-1
- `PROBEW`: 16-R2-1
- `PUSH`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-2, 16-PUSHI3-1, 16-PUSHI2-1, 16-SOBu12-a-2, 16-SOX6-a-2, 16-SOBs12-a-2
- `PUSHM`: 16-PUSHMF-1, 16-PUSHM16a-a-1, 16-PUSHM8-a-1
- `PUSHX`: 16-SYS0-1
- `RET`: 16-SYS0-1
- `ROR`: 16-R3-1
- `RORI`: 16-RRI-1
- `RORI32`: 16-RRI-1
- `RTI`: 16-SYS0-1
- `RTT`: 16-SYS0-1
- `SAR`: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDI12-d-1, 16-SHI3-a-1, 16-SHI5-a-1, 16-LDPC12-d-1, 16-LDBs8-d-1
- `SBC`: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1
- `SC`: 16-R3-1
- `SET`: 16-SETF-1
- `SEV`: 16-SYS0-1
- `SHA256SIG0`: 16-R2-1
- `SHA256SIG1`: 16-R2-1
- `SHA256SUM0`: 16-R2-1
- `SHA256SUM1`: 16-R2-1
- `SHA512SIG0`: 16-R2-1
- `SHA512SIG1`: 16-R2-1
- `SHA512SUM0`: 16-R2-1

- SHA512SUM1: 16-R2-1
- SHL: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDI12-d-1, 16-SHI3-a-1, 16-SHI5-a-1, 16-LDPC12-d-1, 16-LDBs8-d-1
- SHR: 16-LDF-B28-2, 16-LDF-X22-2, 16-LDF-PI12-2, 16-LDF-PD12-2, 16-LDF-PC32-2, 16-LDF-A32-2, 16-LDFI-I32-1, 16-DPF-D28-2, 16-RRF-2, 16-LDI12-d-1, 16-SHI3-a-1, 16-SHI5-a-1, 16-LDPC12-d-1, 16-LDBs8-d-1
- SOB: 16-SOB-1
- SPL: 16-SYSI-1
- SUB: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-SPADJF-1, 16-LDI12-b-1, 16-LDBu8-c-1, 16-SPADJ6-a-1, 16-LDI14-b-1, 16-LDX2-c-1, 16-IMM3-b-1, 16-LDS12-c-1, 16-LDI18-b-1, 16-LDPC16-c-1, 16-LDBs10-c-1, 16-IMM4-b-1, 16-SPADJ8-a-1
- SUBM: 16-STF-B28-2, 16-STF-X22-2, 16-STF-PI12-2, 16-STF-PD12-2, 16-STF-PC32-2, 16-STF-A32-2
- SUBV: 16-R2-1
- SWAP: 16-R3-1
- SWAPDB: 16-R3-1
- SYSPG0: 16-SYS0-1
- SYSPG1: 16-SYS0-1
- SYSPG2: 16-SYS0-2
- SYSPG3: 16-SYS0-2
- SYSPG4: 16-SYS0-2
- SYSPG5: 16-SYS0-2
- TLBI: 16-SYSR-1
- TLBIA: 16-SYS0-1
- TLBSYNC: 16-SYS0-1
- TRAP: 16-SYSI-1
- TRAPCC.CC: 16-SYS0-1
- TRAPCC.CS: 16-SYS0-1
- TRAPCC.EQ: 16-SYS0-1
- TRAPCC.GE: 16-SYS0-1
- TRAPCC.GT: 16-SYS0-1
- TRAPCC.HI: 16-SYS0-1
- TRAPCC.LE: 16-SYS0-1
- TRAPCC.LOS: 16-SYS0-1
- TRAPCC.LT: 16-SYS0-1
- TRAPCC.MI: 16-SYS0-1
- TRAPCC.NE: 16-SYS0-1
- TRAPCC.PL: 16-SYS0-1
- TRAPCC.VC: 16-SYS0-1
- TRAPCC.VS: 16-SYS0-1
- TRAPL: 16-SYSI-1
- TST: 16-SOF-B28-1, 16-SOF-X22-1, 16-SOF-PI12-1, 16-SOF-PD12-1, 16-SOF-PC32-1, 16-SOF-A32-1, 16-SOPR-1, 16-SOBu12-a-1, 16-SOX6-a-1, 16-SOBs12-a-1
- UTF8DEC: 16-R2-1
- UTF8ENC: 16-R2-1
- UTF8LEN: 16-R2-1
- WFE: 16-SYS0-1
- XADD: 16-R3-1
- XADDDB: 16-R3-1
- XAND: 16-R3-1
- XANDDB: 16-R3-1

- `XMAX`: 16-R3-1
- `XMAXDB`: 16-R3-1
- `XMAXU`: 16-R3-1
- `XMAXUDB`: 16-R3-1
- `XMIN`: 16-R3-1
- `XMINDB`: 16-R3-1
- `XMINU`: 16-R3-1
- `XMINUDB`: 16-R3-1
- `XNORPOP`: 16-R3-1
- `XOR`: 16-LDF-B28-1, 16-LDF-X22-1, 16-LDF-PI12-1, 16-LDF-PD12-1, 16-LDF-PC32-1, 16-LDF-A32-1, 16-LDFI-I32-1, 16-DPF-D28-1, 16-RRF-1, 16-LDBu8-c-2, 16-LDI12-c-1, 16-LDX2-c-2, 16-LDS12-c-2, 16-LDPC16-c-2, 16-LDBs10-c-2, 16-IMM3-c-1, 16-LDI14-c-1
- `XOR_`: 16-R3-1
- `XOR_DB`: 16-R3-1
- `XXOR`: 16-R3-1
- `XXORDB`: 16-R3-1

Generated by `tools/mkmanual.py`; do not edit.

Part III — Appendices

Appendix A — Possible improvements not decided on

These are ideas that would make the machine better in one respect or another and that are not decisions. They come from the reports on kernel pain points, hardware vulnerabilities and OpenBSD's mitigations, and from the open items of the instruction-set text. Each is stated as a possibility, with its benefit, its cost and what other machines do, so that it can be taken up, rejected or promoted to a decision in appendix B. Nothing here is built or specified unless it says so. The same ordering principle as the rest of the manual applies: code size, then easy recoverability, then speed and efficiency through fewer gates and tighter hot loops.

A.1 Reserve a page bit that enforces landing pads

A.1.1 The `LPAD` instruction marks a legal target of an indirect jump or call; a page bit could make the hardware fault when an indirect transfer lands anywhere else, with enforcement off in ABI version 1 and on from version 2.

A.1.2 A page bit that gates landing pads is approved, with the fault attributed to the branch, to be specified (errata 84, item 5). *(D)*

Commentary. **Precedent.** `LPAD` already exists, and the ABI leaves landing pads off in version 1, with no switch to enforce them. **Trade-offs.** The bit costs one comparator and two bytes per address-taken function, and deciding later gets more expensive because every binary built in the meantime lacks the pads. **Benefits.** It blocks jump-oriented attacks at the transfer, with no recovery code beyond the fault handler that already exists. **Modern comparables.** Intel's indirect-branch tracking and ARM's branch target identification, both of which OpenBSD enforces by default where the hardware has them. **Impact.** *Kernel:* A landing-pad violation is a fault the handler already handles; the kernel sets the page bit at exec time. *Compiler:* The compiler emits `LPAD` at every address-taken function, two bytes each. *Hardware:* One page bit and one comparator on an indirect-transfer target.

A.2 A page bit that says a system call may be made from here

A.2.1 A page-table bit could let the hardware refuse a system call from any page not so marked, so that the kernel's check of where a call came from moves into the instruction.

A.2.2 A page bit for system-call entry (the `SYS` page bit) is approved, to be specified (errata 84, item 8). *(D)*

Commentary. **Precedent.** OpenBSD's `msyscall` and `pinsyscalls` restrict where system calls may be issued from. **Trade-offs.** One bit and one gate; the kernel would no longer need to check the origin on every call. **Benefits.** About four instructions removed from every system call, and region pinning with no hot-path check. **Modern comparables.** No architecture surveyed has this bit; it is done by the kernel in software. **Impact.** *Kernel:* The kernel no longer checks where a system call came from on every call, which removes about four instructions from each. *Compiler:* None. *Hardware:* One page bit and one gate at the `TRAP` decode.

A.3 A check of the stack pointer at system-call entry

A.3.1 The hardware could check that the stack pointer lies in a region registered as a stack when a system call or a fault is taken.

A.3.2 A stack-limit register is approved, to be specified (errata 84, item 4). *(D)*

Commentary. **Precedent.** OpenBSD checks the stack pointer against a `MAP_STACK` region on system calls and faults. **Trade-offs.** It adds a comparison to entry, which is the path the design most wants short, and gives no benefit over the kernel making the same check. **Verdict.** Not recommended; recorded because the question is natural. **Impact.** *Kernel:* None over the check the kernel already makes. *Compiler:* None. *Hardware:* A comparator and a region register on the entry path, which is the path the design most wants short.

A.4 Protecting the return address

A.4.1 The return address lives on the data stack, because `CALL` pushes it and there is no link register; there are four ways to protect it: software canaries and OpenBSD's return-address protection in the compiler, a hardware return stack, a shadow stack, and nothing.

A.4.2 The mirror shadow stack is approved with its recommendations (errata 85, study item 6); return-address authentication is deferred (errata 84, item 7). *(D)*

Commentary. **Precedent.** x86 has the same exposure and uses stack canaries, `retguard` in OpenBSD's clang, and the shadow stacks of Intel's control-flow enforcement; ARM has pointer authentication. **Trade-offs.** A shadow stack is the strongest and brings the most recovery code (a second stack to overflow, switch and unwind), so the analysis does not recommend it by default; software detection costs a few instructions per function and no hardware. **Benefits.** Detection with little or no recovery code. **Verdict.** Start with the compiler; consider a hardware mechanism only if code size allows. **Impact.** *Kernel:* A shadow stack is a second stack for the kernel to manage and unwind; software detection needs only a fault handler. *Compiler:* The compiler adds a canary or a return-address check per function. *Hardware:* Software costs nothing; a hardware return stack or a shadow stack costs a second stack structure and its control.

A.5 Tag or key bits in pointers

A.5.1 The 25 spare bits of a 64-bit pointer (a 39-bit virtual address uses the rest) could carry a tag or a key, as a memory-tagging or pointer-authentication scheme does.

A.5.2 Full 128-bit CHERI is rejected, and a valid-pointer tag per register and per 8-byte word (stage 1) is kept, with bounds deferred (errata 86, study item 3); granule colouring and reserved colours for uninitialised and freed memory are approved, to be specified, and spend pointer bits that the canonical-address rule refuses to software, so that rule must be restated (errata 84, open). *(D)*

Commentary. **Precedent.** ARM's top-byte-ignore, pointer authentication and memory tagging. **Trade-offs.** The cost is canonical-address rules: pointers must be masked before use, and the hardware must define what a tagged pointer does. **Verdict.** No threat that the operating system asked for requires it; it costs gates and state, so it is not recommended. **Impact.** *Kernel:* The kernel would have to define what a tagged pointer means across the user and kernel boundary. *Compiler:* Every pointer use needs masking, and pointer arithmetic must preserve tags. *Hardware:* Gates and state in the address path.

A.6 An optional 48-bit address mode

A.6.1 A fourth page-table level, switched on by a feature bit, could raise the virtual address from 39 to 48 bits.

Commentary. **Trade-offs.** The user half of the 39-bit space is 38 bits, which gives address-space randomisation far less entropy than the 47 bits of amd64; the price of fixing it is a longer walk on every miss. **Benefits.** Entropy and a bigger address space for a database or a memory-mapped file. **Modern comparables.** x86-64 and AArch64 have four levels (and x86-64 five); RISC-V offers three, four and five by feature. **Impact.** *Kernel:* A mode for the kernel to switch on, with a fourth-level table to allocate and walk. *Compiler:* None beyond pointer size, which does not change. *Hardware:* A fourth walker level and a longer walk on every miss.

A.7 A second page-table base register

A.7.1 A second page-table base for the kernel half of the address space would let a kernel and a user program run with different tables without flushing, if a speculative core ever needs the isolation.

Commentary. **Precedent.** The cost of separating kernel and user page tables on x86 after Meltdown was about 5% typical and about 30% in the worst case in the developers' own estimates, and the tagged translation buffers (PCID) made it tolerable. **Trade-offs.** The single base is correct and free on a core that does not speculate; the second base register costs a register and a rule, and the question is whether to pay it before the need exists. **Benefits.** The isolation is in the hardware from the start. **Modern comparables.** AArch64 has two translation-table base registers, one for each half of the address space. **Impact.** *Kernel:* The kernel switches one base register at the privilege change instead of managing a shared table. *Compiler:* None. *Hardware:* A second base register and a rule choosing which half uses which.

A.8 Two-byte forms of the user-memory guard instructions

A.8.1 `STUA` and `CLUA` are not in the grammar of this CPU; whether they are added as one-parcel system words is decided when the kernel exists, and the user-access moves (section 6.5.4) may make them unnecessary; they are kept until the OpenBSD port confirms them (errata 86).

Commentary. **Trade-offs.** It is not worth deciding until a kernel exists to count the copy routines. **Benefits.** Smaller and faster user copies. **Verdict.** Decide when the kernel exists. **Impact.** *Kernel:* Each user copy routine is four bytes smaller. *Compiler:* None. *Hardware:* One more system word.

A.9 Thirty-two-bit forms of add, subtract and shift

A.9.1 If the extension instructions the compiler emits after narrow arithmetic are more than a few percent of the code, 32-bit forms of the common operations in one-parcel leaves would remove them.

Commentary. **Precedent.** AArch64's `W` forms and RISC-V's `ADDW` family. **Trade-offs.** A second set of leaves in the table against extension instructions in the compiled code; the overhead has never been measured over the whole corpus. **Verdict.** Measure first; it is one script over the compiled corpus. **Impact.** *Kernel:* None. *Compiler:* The compiler stops emitting an extension after common 32-bit operations. *Hardware:* A second set of opcodes in the page-2 map and a little ALU decode.

A.10 Packed-lane instructions in the integer registers

A.10.1 A small set of lane operations (add, subtract, saturating add, compare, minimum, maximum, average, sum of absolute differences, shuffle on 8, 16 and 32-bit lanes) in the existing registers could speed up string, codec and image loops.

Commentary. **Precedent.** Alpha's motion-video instructions and PA-RISC's multimedia instructions did exactly this, in integer registers. **Trade-offs.** It adds no state and little hardware, but it is 64 bits wide so the gain per instruction is modest, and compilers use it poorly. **Verdict.** Collect the evidence first: the compiler corpus was built at a size setting that hides how much vectorisable code there is (appendix B, S1). **Impact.** *Kernel:* String and checksum routines get faster. *Compiler:* Vectorisation into integer lanes is weak, so the library routines would be hand-written. *Hardware:* A lane-aware ALU: little extra hardware and no new state.

A.11 Streaming and non-temporal block operations

A.11.1 A large `MCPY` evicts the whole data cache for data never read again; a non-temporal variant or a streaming memory type would avoid it.

Commentary. **Precedent.** x86 has non-temporal stores; ARM has non-temporal load and store pairs. **Trade-offs.** It shows immediately on an 8 KB cache; the fix is an opcode or a memory type, the non-fix is to leave the cache alone. **Verdict.** Open; decide when the memory system is built. **Impact.** *Kernel:* Large copies would not evict the kernel's working set. *Compiler:* `memcpy` would choose the streaming form above a size. *Hardware:* A non-temporal write path in the cache.

A.12 Smaller possibilities

- **`MULHSU`** (the high half of a signed-by-unsigned product, for a signed top limb meeting an unsigned one) is reserved in the arithmetic escape; it is one correction step in software and has one caller, a bignum library's innermost loop.
- **Hardware page zeroing**, and **guard-page states in the page-table entry that do not consume translation-buffer entries**, would make the allocator hardening of OpenBSD's `malloc` cheaper.
- **A mixed leaf size for the kernel's text** is needed if the kernel is to be write-xor-execute with one-gigabyte leaf pages.
- **Hardware breakpoints and debug registers** are not designed; a debugger has only the breakpoint instruction and the trace bit.
- **A sealed (immutable) page bit and a hardware mask for `pledge`** have no benefit over the kernel doing the checks, and are rejected.

Commentary. **Trade-offs.** None of these is ranked above the decisions in appendix B; the first four sections of this appendix (landing pads, the system-call page bit, the stack check and return-address protection) are the ones OpenBSD's developers have shown the most interest in.

Appendix B — Decisions pending

These are the open decisions, each with its question, the proposed answer, and the precedent, trade-offs and benefits that surround it. Each has five options with their costs in the working document `../decisions-pending.md` (the identifiers P, R, S, N and B below are that document's), and the boot choices are worked through in `../boot.md`. Decided items are not repeated here. Each entry is judged by the three principles in order: code size, easy recoverability (less recovery code), and speed and efficiency through fewer gates and tighter hot loops. Every entry is a proposal *(P)* until it is decided.

B.1 The instruction set and the system

B.1.1 The wake rule of `WFE` (P1)

B.1.1.1 The question is whether `WFE` ends its sleep when an interrupt is pending even if the priority masks it; the proposal is yes. *(P)*

Commentary. **Precedent.** An idle loop must check for work and sleep without a gap, or an interrupt that arrives between the two is lost. V7's idle routine lowered the priority and executed `wait` inside a loop that rechecked the run queue after every interrupt, so a lost wake-up cost at most one clock tick. **Trade-offs.** With priority 7 on entry, "lower the priority, then sleep" misses an interrupt that lands between the two, and "sleep at priority 7" never wakes. **Benefits.** The idle loop becomes `spl #7`, check for work, `wfe`, `spl #0`, with no gap and no timer tick to bound it; no opcode, a few gates. **Modern comparables.** AArch64 and RISC-V `wfi` resume on a pending enabled interrupt (RISC-V's manual says the global enable bit is ignored and that an implementation may wake for any reason, so "the same as RISC-V" holds only if the pdp64's priority mask is treated like that bit); x86 closes the gap with the one-instruction interrupt shadow of `STI`, which is hidden state. **Impact.** *Kernel:* The idle loop needs no tick to bound a lost wake-up. *Compiler:* Spin-wait and idle code use `WFE` and `HINT #1`. *Hardware:* A pending-interrupt line into the wait logic, a few gates.

B.1.2 An unprivileged time counter (P3)

B.1.2.1 The proposal is `RDTIME rd`, an unprivileged read of a free-running, constant-rate, monotonic counter. *(P)*

Commentary. **Precedent.** The PDP-11 and VAX kept the time in memory updated by a line clock and read it with a system call. **Trade-offs.** A user-readable counter is a timing side channel, so the kernel needs an enable bit; the counter's width and rate become architecture. **Benefits.** A clock read without a trap, which is what a fast `clock_gettime` in a shared page needs. **Modern comparables.** x86 `RDTSC`, RISC-V `rdtime`, PowerPC `mftb`, s390 `STCKF`, MIPS `RDHWR`, LoongArch `rdtime`; the x86 counter's history of unreliability is the argument for fixing monotonicity and constant rate at the start. **Use.** Timestamps, timeouts, and the fast path of every library's clock call. **Impact.** *Kernel:* A fast `clock_gettime` with no trap. *Compiler:* An intrinsic for the counter read. *Hardware:* One counter, a user-read enable bit and a `SYSX` decode.

B.1.3 Feature and identification registers (P4)

B.1.3.1 The proposal is a block of read-only system registers (numbers 13 and up) giving the version, the optional features, the cache geometry, the page sizes and the address width. *(P)*

Commentary. **Precedent.** The VAX had a system-identification register the kernel switched on; x86 has `CPUID`, which has grown hundreds of leaves and traps under virtualisation. **Trade-offs.** Each register's layout must be written once in a grammar so it cannot drift. **Benefits.** One binary adapts to the machine, and a kernel need not patch itself per model (OpenBSD carries 75 references to its boot-time patching). **Modern comparables.** AArch64's identification registers, RISC-V's `misa`. **Impact.** *Kernel:* The kernel need not patch itself per model. *Compiler:* Runtime selection of routines by feature bits. *Hardware:* A small block of read-only registers.

B.1.4 TLB invalidation by address-space tag (P5)

B.1.4.1 The proposal is one new instruction, `TLBIASID`, that invalidates every translation of one address space, broadcast like `TLBI`. *(P)*

Commentary. **Precedent.** The VAX flushed process translations on every context switch and kept system ones; x86 added tags (PCID) and `INVPCID` much later. **Trade-offs.** A flush-all works until the first rollover of the tag numbers, after which every process loses its translations. **Benefits.** Process exit and tag reuse become cheap. **Modern comparables.** AArch64 `TLBI ASIDE1IS`, RISC-V `sfence.vma` with an address-space operand, x86 `INVPCID`. **Impact.** *Kernel:* Process exit and tag reuse become one broadcast. *Compiler:* None. *Hardware:* A tag compare in the translation buffer's invalidate path.

B.1.5 What a multi-register instruction leaves behind when it faults (P12)

B.1.5.1 The proposal is a rule and a test: every multi-access instruction (`PUSHM`, `POPM`, `PUSHX`, `POPX`, the block operations) is either all-or-nothing or safe to repeat from the start, and an emulator test interrupts each at every access and checks that a restart reaches the same state. *(P)*

Commentary. **Precedent.** The PDP-11 needed a status register (`MMR1`) and, on the 11/40, a kernel routine that decoded the faulting instruction to undo it; the VAX needed a first-part-done bit; ARM's load-multiple restarts with the base register restored. **Trade-offs.** The area below the stack pointer is dead memory, so a partial push is invisible; the risk is another observer (a device or core) seeing a half-written area. **Benefits.** The fault handler restarts blindly, with no instruction-specific cases, which is what recoverability means. **Modern comparables.** RISC-V has no multi-register instruction; ARM and the VAX restart as above. **Impact.** *Kernel:* The fault handler restarts every multi-access instruction blindly. *Compiler:* None. *Hardware:* Each multi-access instruction checks every address before acting, or is repeatable.

B.1.6 Priority 7 on interrupt entry as well as exception entry (P14)

B.1.6.1 Exceptions enter at priority 7, which is decided; the open question is whether interrupts do too, which the implementation does. *(exceptions D; interrupts P)*

Commentary. **Precedent.** The PDP-11 loaded each vector's own priority and pushed the old state on a stack, so an interrupt at any time could not lose it; trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers needs the priority raised, or a higher-level interrupt could overwrite the saved state before the handler stores it; the cost is one `SPL` in a handler that wants to be interruptible. The alternatives are per-vector priority (the PDP-11's choice), a hardware push, and an interrupt shadow. **Modern comparables.** x86, AArch64 and RISC-V all mask interrupts on entry. **Impact.** *Kernel:* One `SPL` in a handler that wants to be interruptible. *Compiler:* None. *Hardware:* Priority 7 loaded into the new status word at entry.

B.1.7 A prefetch instruction (P15)

B.1.7.1 The proposal is none, with the shape of a non-faulting `PREFETCH` with a memory operand reserved on a free sub-opcode. *(P)*

Commentary. **Precedent.** Neither the PDP-11 nor the VAX had one; caches there were transparent. **Trade-offs.** The kernel survey counts over a hundred prefetch instructions across the four kernels, mostly in tuned list and copy code; an in-order core with no data cache gains nothing. **Modern comparables.** AArch64 `PRFM`, ARM `PLD`, PowerPC `DCBT`, x86 `PREFETCHT0`. **Impact.** *Kernel:* List walks and copy loops can warm the cache. *Compiler:* `PREFETCH` would need compiler heuristics. *Hardware:* A non-faulting load path with no destination; nothing on a core with no data cache.

B.1.8 Performance counters (P16)

B.1.8.1 The proposal is two free-running counters, cycles and instructions retired, readable in user mode, added after the clock (P3) with the same enable bit. *(P)*

Commentary. **Precedent.** Unix profiled by sampling the program counter at clock ticks. **Trade-offs.** A user-readable cycle counter is a timing channel; a full counter block with event selection costs a large block and interrupt machinery. **Modern comparables.** x86 `RDPMC`, the AArch64 PMU, RISC-V's `hpm` counters. **Impact.** *Kernel:* A profiler reads counters instead of sampling at ticks. *Compiler:* Intrinsics. *Hardware:* Two counters and an enable bit.

B.1.9 Constant-time execution (P17)

B.1.9.1 The question is how to guarantee that instruction time does not depend on operand values; the proposal is a core set of instructions that is always constant-time plus a mode bit that extends the promise to divide, `MSCN`, `MCMP` and floating point. *(P)*

B.1.9.2 Closed in part: the branch-free select is adopted, at least one secure core has every timing channel removed (all instructions and cache fetches take fixed time, no periodic tick), and on that core each instruction takes a time independent

of its data, with no ticks or timing interrupts introduced to achieve it (errata 85, 86; `docs/crypto-secure-core.md`); the mode bit of the proposal above is superseded for the secure core. *(D)*

Commentary. **Trade-offs.** Making all integer instructions constant-time would cost about 1% on average by a rough estimate, because divide is only 0.04% of compiled instructions (149 of 370,115; multiply costs nothing, since a fixed-latency multiplier needs less logic than one with an early-out); the mode bit keeps fast paths for ordinary code. A constant-time instruction does not make code constant-time: a secret-dependent branch or table index still leaks through the predictor and the cache. **Precedent.** Neither ancestor addressed it; the PDP-11's multiply and divide were microcoded with operand-dependent times. **Modern comparables.** Arm's `DIT` bit, Intel's data-operand-independent timing mode, RISC-V's scalar-crypto timing subset. **Use.** Cryptography on secret keys. The performance cost of those modes is not established. **Impact.** *Kernel:* Crypto code sets the mode around secret-dependent work. *Compiler:* The back end must not turn a constant-time sequence into a variable-time one. *Hardware:* The core set costs nothing; the mode bit needs the fast paths of divide and the scanners to be gateable.

B.1.10 The literal pool and execute-only code (P18)

B.1.10.1 The proposal is an ABI rule: literal pools move out of `.text` into a read-only section beside it, so code pages hold only instructions and can be execute-only. *(P)*

Commentary. **Precedent.** The PDP-11's immediates sit in the instruction stream, and with separate instruction and data spaces a program could not read its own instructions as data. **Trade-offs.** The PC-relative reach limits how far the pool may be, and the linker must keep the two sections near each other. **Benefits.** Zero hardware; a memory-disclosure bug no longer reveals code. **Modern comparables.** OpenBSD makes code execute-only on amd64 and arm64; ARM compilers offer a flag to keep pools out of code. **Impact.** *Kernel:* None. *Compiler:* Pools are emitted into a read-only section beside the text. *Hardware:* None; the linker keeps the sections within PC-relative reach.

B.1.11 Vector and SIMD math (S1 to S6)

B.1.11.1 The device route is decided: the processor has no vector register file and no SIMD instruction set, and wide data-parallel work (graphics, AI and large-language-model inference, the array languages APL and K) is done by memory-mapped devices (chapter 8). *(D)*

B.1.11.2 Open are the number formats and operations of the devices and the reconciliation of the device descriptors with the standard descriptor; the six questions are: where data-parallel work lives (coprocessor, packed lanes in the integer registers, a fixed-width vector file, a scalable one, a matrix unit); which registers hold vectors; fixed or scalable length; how the machine state is saved on a context switch and whether the kernel may use it; which number formats (half precision, bfloat16, 8-bit integer dot products, 8-bit floating point); and the ABI and compiler support. *(P)*

Commentary. **Precedent.** The PDP-11 sent heavy numerical work to attached array processors; the VAX had a vector extension on its 6000 series, with 16 vector registers of 64 elements, supported by VMS; neither Unix used it much. **Trade-offs.** A fixed-width vector file brings a second register file and its context-switch cost, opcodes, and a new width for every generation (the history of SSE, AVX and AVX-512); a scalable design runs one binary on a small and a large implementation at the price of the most complex specification. Lazy saving of vector state was a security hole on x86, and Linux went back to eager restore. **Benefits.** The coprocessor needs no opcodes and no processor state. **Modern comparables.** ARM SVE and RISC-V V (scalable), ARM NEON, x86 AVX, PowerPC VSX, Alpha MVI. **Use.** Neural-network inference (the LB64 accelerator), graphics, signal processing and the library's string and memory routines. The evidence needed first: compile the corpus at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and decide the target workloads. **Impact.** *Kernel:* No vector state to save. *Compiler:* No vectoriser for the processor; the runtime lowers array primitives to descriptors. *Hardware:* No register file; the devices carry the width.

B.2 The bus, boot and devices

B.2.1 Data-cache maintenance by address (P6)

B.2.1.1 The proposal is to reserve nothing: DMA buffers are uncached, so no cache instruction is needed until a non-coherent device appears. *(P)*

Commentary. **Precedent.** The 11/70's cache watched DMA writes; the VAX's adapter had data paths to purge. **Trade-offs.** Uncached buffers are slow to read; cache maintenance would speed them and needs three reserved sub-opcodes.

Modern comparables. ARM, MIPS and embedded PowerPC with non-coherent DMA wrap cache maintenance in a DMA-mapping layer; x86 DMA is coherent. **Impact.** *Kernel:* Drivers need no cache code. *Compiler:* None. *Hardware:* No cache-maintenance operations.

B.2.2 Firmware calls (P7)

B.2.2.1 The proposal is none: everything goes through devices, with a required reset register and a watchdog in the bus controller. The kernel survey disagrees and would specify a firmware-call convention on reserved trap vectors. *(P)*

Commentary. **Precedent.** The PDP-11 kernel never called firmware; the VAX had console registers the kernel read and wrote. **Trade-offs.** No firmware means no firmware to trust and no extra privilege level, and every platform must say where its console and reset registers are, which the others get from firmware. **Modern comparables.** RISC-V's supervisor binary interface, ARM's PSCI and secure monitor calls, Alpha's PALcode, SPARC's Open Firmware. **Impact.** *Kernel:* Reboot and power-off through a device register and a watchdog. *Compiler:* None. *Hardware:* A reset register and a watchdog timer in the controller.

B.2.3 Double-width compare-and-swap and a random-number instruction (P8, P9)

B.2.3.1 The proposals are to add neither: the evidence for a 128-bit compare-and-swap is one Linux use (software can tag a pointer with the 25 spare bits instead), and random numbers come from an entropy device on the bus rather than an instruction. *(P)*

B.2.3.2 The random-number pool is one device shared by all cores, with no per-process pool, no version register and no updatable firmware; it assists the operating system's own generator (errata 85). *(D)*

Commentary. **Precedent.** The VAX solved the lock-free queue with whole-queue interlocked instructions; no machine of the period had a random-number instruction. **Modern comparables.** x86 `CMPXCHG16B`, AArch64 `CASP`, RISC-V `AMOCAS.Q`; x86 `RDRAND` and `RDSEED`, PowerPC `DARN`, s390 `PRNO`, AArch64 `RNDR`. **Impact.** *Kernel:* Entropy from a device at boot. *Compiler:* Tagged pointers in software if a double-width compare-and-swap is not wanted. *Hardware:* An entropy source device; no new opcode.

B.2.4 The interrupt controller and routing to cores (R1)

B.2.4.1 The proposal is wired interrupt lines to core 0 with a per-line target register, keeping the eight priority levels, with message-signalled interrupts as an extension. *(P)*

Commentary. **Precedent.** The PDP-11 had no controller; a device asserted one of four request lines and supplied its vector. **Trade-offs.** The smallest design adds no new failure cases and cannot give a fast multi-queue device its own interrupt. **Modern comparables.** x86's APIC, ARM's GIC, RISC-V's PLIC and AIA. **Impact.** *Kernel:* A handler sees one interrupt model. *Compiler:* None. *Hardware:* Wired lines to one core with a target register.

B.2.5 Inter-processor interrupts and starting other cores (R2, B2)

B.2.5.1 The proposal is a per-core doorbell register and a per-core start register in the bus controller (chapter 4), with `SEV` and `WFE` for waiting. *(P)*

Commentary. **Precedent.** The VAX's console processor started the other processors. **Modern comparables.** x86's INIT and START-UP sequence, ARM's PSCI and spin table, RISC-V's hart-start call. **Impact.** *Kernel:* One register write per core and one per doorbell. *Compiler:* None. *Hardware:* A start register and a doorbell per core.

B.2.6 How coherence scales (R3)

B.2.6.1 The proposal is to keep snooping for the four cores of the first device; the ceiling is now sixteen or more (errata 92), and the protocol for that many is open (7.7.4). *(P)*

Commentary. **Trade-offs.** Directory protocols scale and are large; software coherence is cheap and fragile. **Modern comparables.** x86's snoop filters and directories across sockets, ARM's coherent interconnects. **Impact.** *Kernel:* Cache coherence is invisible to the kernel. *Compiler:* None. *Hardware:* Snooping on two to four cores.

B.2.7 The bus transport (R4)

B.2.7.1 Closed: the transport is PCI Express, with the pdp64's convenience and security circuitry on top (errata 75, 80). *(D)*

Commentary. **Trade-offs.** It saves a bridge and makes the atomics a monitor naturally, and the Unibus's lock is no longer a native transaction. **Modern comparables.** ARM's AMBA, RISC-V's TileLink or AXI, FPGA systems on AXI throughout. **Impact.** *Kernel:* None. *Compiler:* None. *Hardware:* The bridge to AXI is not needed if AXI is the transport.

B.2.8 The device layout and the descriptor (R5)

B.2.8.1 The proposal is the uniform layout and the 32-byte descriptor of chapter 5, with an identification register first, and virtio-compatible devices only if reusing OpenBSD's drivers saves more than it costs. *(P)*

Commentary. **Precedent.** The RK11 and the DEUNA, as in chapter 5. **Trade-offs.** The single ring per device makes duplex devices two slots. **Modern comparables.** `virtio`, NVMe, SATA's command lists, Intel Ethernet's done bit. **Impact.** *Kernel:* One driver skeleton. *Compiler:* The descriptor is a generated C structure. *Hardware:* One ring engine behind all devices.

B.2.9 DMA addressing and protection (R6)

B.2.9.1 Superseded: an IOMMU is part of the design and its five recommendations are accepted (errata 75, 85); bits 40 to 63 of its leaf are free. *(D)*

Commentary. **Precedent.** The Unibus carried only 18 address bits, so the VAX and the larger PDP-11s had map registers that translated device addresses, in effect a simple translation unit for necessity. **Trade-offs.** No protection from a faulty or malicious device; a translation unit matters when virtualisation or untrusted devices do. **Modern comparables.** x86 VT-d, ARM SMMU, the RISC-V IOMMU; Linux's bounce buffers for limited devices. **Impact.** *Kernel:* A driver passes physical addresses. *Compiler:* None. *Hardware:* No translation unit for devices.

B.2.10 Errors raised by devices and the bus (R7)

B.2.10.1 The proposal is the precise and imprecise paths the ISA already has plus a small register block in the bus controller for the cause. *(P)*

Commentary. **Precedent.** A PDP-11 bus timeout trapped through vector 4 and usually ended in a panic; the VAX had a machine-check frame. **Modern comparables.** x86's machine-check banks and PCI Express error reporting, ARM's RAS extension. **Impact.** *Kernel:* One recovery path for device errors. *Compiler:* None. *Hardware:* A small error register block in the controller.

B.2.11 Virtualisation (R8)

B.2.11.1 The proposal is to declare it out of scope for System v1 and fix five constraints: privileged state stays privileged or controllable, the page-table walker could be wrapped by a second level, interrupts are injectable, system-register access can trap, and the translation-enable bit and software accessed and dirty bits are reviewed for what a hypervisor would need. *(P)*

Commentary. **Precedent.** The PDP-11 was not designed for virtualisation and has sensitive instructions that do not trap in user mode; the VAX was not cleanly virtualisable either, and a 1991 paper described the changes needed. **Trade-offs.** Adding virtualisation late cost the VAX instruction changes; settling the constraints early costs a paragraph. **Modern comparables.** x86 VMX with nested paging, ARM exception level 2, the RISC-V hypervisor extension, IBM's trap-and-emulate machines. **Impact.** *Kernel:* A hypervisor could be added without changing a guest. *Compiler:* None. *Hardware:* Constraints on privileged state, the walker and interrupt injection.

B.2.12 The bus controller's mechanisms (N1 to N3)

B.2.12.1 The proposals are a fixed 4 KB page per slot with the mapping register for exceptions (N1), an interrupt with a small event queue and a rescan on overflow (N2), and an absent mark with a precise fault and the queued work left in memory (N3). *(P)*

Commentary. See section 2.6 and its commentary: the VAX's nexus scan, PCI's configuration space and all-ones reads, USB's status-change endpoint. **Impact.** *Kernel:* Enumeration and hot-plug are one loop and one queue. *Compiler:* None. *Hardware:* A fixed page per slot, an event queue and a presence bit.

B.2.13 The boot decisions (B1, B3, B4, B5, B8)

B.2.13.1 The proposals are a boot ROM that reads block 0 (B1), a boot-information block of memory, cores, the bus controller's address and two slots (B3), an `INIT` bit with an optional firmware descriptor (B4), no UEFI, with a simple hand-off a shim could use (B5), and a block list in the protective MBR's boot code (B8). *(P)*

Commentary. See chapter 4 and `../boot.md`: the PDP-11's block 0 and ODT, the VAX's boot block and restart parameter block, the BIOS and UEFI. **Impact.** *Kernel:* A boot-information struct and a core start register. *Compiler:* The loader and first block are built by the project's compiler. *Hardware:* A boot ROM, a block-0 reader and per-core start registers.

B.2.14 How USB attaches, and virtio first

B.2.14.1 Closed in part: USB is attached over PCI Express, and virtio-net, virtio-gpu and virtio-snd are the standard network, graphics and audio interfaces (errata 75, 79). *(D)* What those decisions do not say is whether USB is a controller that shows each attached device as a bus slot, or a conventional ring-based host controller driven by an existing operating-system USB stack; and whether the emulator uses `virtio` devices first to reach an OpenBSD login sooner.

Commentary. **Trade-offs.** Per-slot USB moves the protocol into the controller chip and takes the stack out of the operating system; the conventional controller uses code that already exists. `virtio` reuses OpenBSD's existing drivers in the emulator and defers the project's own eight-register devices to silicon; it gets to `ssh` faster and puts native devices later. **Modern comparables.** xHCI for the conventional controller; `virtio-mmio` on every emulated RISC-V and ARM board. **Impact.** *Kernel:* A disk driver serves a USB disk; virtio reuses the existing drivers. *Compiler:* None. *Hardware:* USB protocol in the controller costs gates; virtio costs nothing for the emulator.

Appendix C — Glossary

Terms the manual uses, collected in one place. Each entry gives the plain meaning and, in brackets, the chapter or section where the manual deals with it. Terms for the instruction groups (the "escape" opcodes) are listed by name and opcode; the full encodings are in the instruction-set text `../pdp64-isa.md`.

C.1 Architecture and encoding

Term	Meaning
pdp64	the 64-bit extension of the PDP-11 this manual describes, as the PDP-11 extended the PDP-10's ideas down and the VAX extended the PDP-11 up (chapter 1)
family	the three CPUs of the architecture, pdp64/r20, pdp64/16 and pdp64/32, each with its own encoding and its own manual (1.8)
parcel	a 16-bit unit of instruction encoding; an instruction is one to three parcels, two to six bytes, or five parcels for `MOVQL` (9.4)
leaf	a group of mnemonics with one form, selected by a prefix of the first parcel; a sub-opcode picks the mnemonic (9.1)
prefix	the top 3 to 16 bits of the first parcel; the prefixes form a prefix code (no prefix begins another), so the first parcel matches at most one leaf and fixes the length of the instruction (9.1)
form	the list of operand shapes of a leaf: how each operand is written in assembly and how many bits each part has (9.1)
sub-opcode	the field after the prefix that selects one mnemonic of the leaf (9.1)
hot register	one of the eight (in pdp64/32, sixteen) most-used registers, which a 3-bit (4-bit) register field of a one-parcel form names by an index into a table; the tables are part of the ABI (9.4, appendix D)
fused instruction	`MOVCALL`, `POPRET`, `POPMRET` or `MOV2`: one instruction for a pair that the compiler finds adjacent (9.10)
relaxation	the linker's choice, after the sections are laid out, of the shortest form that reaches for a call or an address; on by default (9.5)
`CBCC`	compare-immediate-and-branch on condition, the one fused compare and branch (9.10)
`RORI32`	rotate right by an immediate, 32-bit, sign-extended result
`SOB`	subtract one and branch, either direction (9.10)
`MCPY`, `MMOV`, `MSET`, `MCMP`, `MSCN`, `MSCNN`	the block operations: copy, move (overlap-safe), fill, compare, scan for a byte, scan for the first byte not equal; progress is held in registers so a fault restarts for free (9.11)
`MVAL`	block scan of valid UTF-8, one of the string instructions of the same family (9.9)
`PUSHM`, `POPM`	push and pop a register set named by a mask; the useful half of the VAX's `CALLS` without the frame (9.11)
`JMPI`, `CALLI`	jump and call through a memory word; the only deferred forms, used for PLT stubs and imports (9.10, 9.16)
`JTAG`, `TAGOP`	multiway dispatch on a pointer's low tag bits, and the tag-branch group (9.10)
`LPAD`	marks a legal target of an indirect jump or call; optional in ABI version 1 (appendix A.1)
`CLMUL`	carry-less multiply, the primitive under every CRC and Galois-field operation (9.9)
`XNORPOP`, `POPCNT`	bit-population instructions used by binary operations and bitsets (8.3)
`SET cc, rd`	set a register to 0 or 1 from a condition (9.7)
`DP`	the direct pointer: in user mode it is the thread pointer, in kernel mode the per-CPU pointer; a descendant of the 6502's zero page (9.3, 9.16)
`PSW`	the processor status word: flags, mode, priority and the guard bit (6.1)
`N Z V C`	the four condition flags: negative, zero, overflow, carry (9.3)
canonical address	a 64-bit address whose bits above 38 equal bit 38; any other value faults, so software cannot hide tags in the high bits (6.4)
`lowtag`	a type tag stored in the low bits of a pointer, free because objects are quad-aligned (instruction-set text, section 6.2; not otherwise used in this manual)
SWAR	SIMD within a register: using one wide scalar register as several narrow lanes through masking and carry-aware arithmetic (instruction-set text, glossary; the manual's bit and byte instructions of 9.9 make it effective)
`FPSR`	the sticky floating-point exception flag register, read and written by `FSFLAGS` (9.12)
ulp	unit in the last place: the spacing between adjacent floating-point numbers, the unit of rounding error
Payne–Hanek	the argument-reduction algorithm that computes x mod $\pi/2$ to full precision for large x; what hardware transcendentals cannot do (instruction-set text, section 9.7.1; the manual's account of floating point is 9.12)

Term	Meaning
Table Maker's Dilemma	that correct rounding of a transcendental may need unboundedly many bits to resolve a near-tie, so no fixed-time circuit can guarantee it (instruction-set text, section 9.7.1)
block floating point	integer mantissas sharing one exponent, used by the LB64 accelerator so fixed-point accumulation is exact (8.3)

C.2 Memory, protection and the kernel

Term	Meaning
`MT`	memory type, a two-bit page attribute: normal cached, normal uncached, device strongly ordered (2.1, 6.4)
`ASID`	address space identifier: tags translation-buffer entries so a context switch flushes nothing (6.4)
`G` **bit**	global: a translation-buffer entry that ignores the `ASID` (6.4)
TLB	translation lookaside buffer: the cache of page-table entries; `TLBI` invalidates one, `TLBIA` all, `TLBSYNC` waits for other cores (7.5)
`DMW`	direct mapping window: a region translated by a fixed offset with no page table; deferred
`MMUCTL`	the register that enables translation and sets the table base (6.3, 6.4)
walker	the hardware that reads page-table entries on a translation-buffer miss (6.8)
`STUA` / `CLUA`	set and clear the user-access guard bit, which lets kernel code touch user memory only inside the copy routines (6.5)
`PROBER` / `PROBEW`	test whether an address is readable or writable in a given mode without faulting (6.7)
W^X	write xor execute: no page is both writable and executable; OpenBSD enforces it (appendix A)
PIE	position-independent executable: loadable at any address; costs nothing here (9.16)
GOT, PLT	global offset table and procedure linkage table: the indirection that lets position-independent code reach imported symbols (9.16)
ABI	application binary interface: calling convention, register use, data layout; version 1 is given in appendix D
trap, exception, interrupt	a deliberate entry (`TRAP #n`), a fault raised by an instruction, and a signal from a device; all enter through the vector table (6.2)
precise fault	a fault after which every earlier instruction has completed and no later one has changed state (6.8)
`SPL`	set the interrupt priority level (6.2)
`WFE`, `SEV`	wait for an event and send an event: the idle and wake-up pair (7.6)
`FENCE`	orders memory accesses; owed to devices and to other cores (2.2, 7.2)
`ATOM`, **monitor**	the atomic operations, built on a reservation monitor as in AXI and ARM, not a locked bus cycle (7.3)
SMP	symmetric multiprocessing: several identical cores sharing memory (chapter 7)
core class	the read-only register of each core that gives its class and capacity, so that software can tell an in-order core from an out-of-order one (7.8.4)
secure core	a core with every timing channel removed (every instruction and cache fetch takes a time independent of its data), private cache lines and no periodic tick (7.8.6)
dedicated core	a core given to one job, graphics, network or audio; proposed (7.8.7)
coherence, snooping MSI	the protocol that keeps each core's cache consistent with the others; snooping means each cache watches the bus; MSI is its three states (modified, shared, invalid) (7.7)
sequential consistency	the memory model in which all cores see all accesses in one order; the pdp64 declares it (7.2)

C.3 The bus, devices and boot

Term	Meaning
Q64	the system bus: the PDP-11 Q-bus lineage widened to 64-bit data and 40-bit addresses (2.1)
Unibus, Q-bus	the PDP-11's buses; Q64 keeps their programming model, with devices in the address space (2.1)
`DATIP` / `DATO`	Unibus and Q-bus transaction types forming a locked read-modify-write (2.5)
space bit	bit 39 of the physical address: set selects device space, clear selects memory (2.1.6)
slot	a device's own 4 KB (or larger) window of device space, found by enumeration (2.6, 5.1)
bus controller	the device that is the bus: it enumerates slots, maps them, and reports arrivals and departures (2.6)
bridge	a device on Q64 that is itself a bus: it presents the devices behind it as slots with the bus controller's registers, so arbitrary buses (PCI Express, USB, SATA) can be connected to Q64 and enumerated to any depth (2.6.7)
MMIO	memory-mapped input/output: device registers reached by ordinary loads and stores (5.1)

Term	Meaning
DMA	direct memory access: a device moving data to or from memory without the processor (5.4)
ring, descriptor, `OWN`, doorbell	the common device protocol: a ring of 32-byte descriptors in memory, each with an ownership bit the host sets and the device clears, and a register written to tell the device to look (5.2)
`ID` / `CONTROL` / `STATUS` / `RING_BASE` / `RING_SIZE` / `DOORBELL` / `IRQ` / `DEVICE`	the eight common registers every device starts with (5.2)
`INIT`, `READY`	the control bit that makes a device initialise itself and the status bit that says it has (5.2, 5.3)
virtio	a standard family of virtual devices of the same ring-and-descriptor shape; the emulator's first drivers (5.5, appendix B)
NVMe, SCSI, ATA, SATA	storage interfaces, each a command queue in memory and a doorbell, which is the same device pattern (5.2, 5.5)
HDMI transmitter	the chip that takes a pixel stream and does the link protocol; the pdp64 side is a pixel generator and registers (3.3)
`VIC-64X`, `SID-64X`, `BLT64`	the video, sound and blitter devices of the graphics terminal, which is based on the Commodore 64 (video and sound chips) and the Blit terminal (framebuffer and bit-block transfer) (chapter 3)
Blit	Rob Pike's bit-mapped windowing terminal: an 8 MHz Motorola 68000 machine that received downloaded code, characters and editing messages from its host over a serial line and drew every pixel itself (its description as a precursor of the X11 protocol and of NeWS is stated by no source read *(S)*; the lineage the sources confirm runs from its `mux` to 8½ and `rio`); a framebuffer in the address space and a bit-block-transfer operation, done in software, that moves rectangles of pixels; with the C64, one of the two machines the pdp64's terminal is based on (3.1)
blitter, raster operation	a device that copies and combines rectangles of pixels; the raster operation says how source and destination combine (3.1)
BIOS, boot ROM	the first software that runs: it brings the hardware up, finds the disk label and loads the next stage; open source (4.1)
GPT, disk label	the GUID partition table wrapped around the disk's own label so other systems see the disk is in use (4.3)
LB64, MX64, VEC64	the inference accelerator and its companions: sign-flip-accumulate layers, floating-point multiply-accumulate for attention, and element-wise vector work (8.3)
APL, K	array languages whose primitives work on whole arrays; one primitive maps to one descriptor (8.4)

C.4 Process and sources

Term	Meaning
errata	the numbered decision log of the project (`../errata.md`, appendix F)
fair corpus	the 827 or 841 C files that every back end compiles, used for all size comparisons (1.5)
`pcc`	the portable C compiler the project extends to emit pdp64 code (1.5)
`sim64`, **QEMU**	the instruction-set simulator and the full-system emulator the tests run on (1.5, 10.4; not otherwise described in this manual)
(D), (P), (S)	the tags on a numbered statement: decided, proposed, stub (index)
Impact	the line ending each decision's commentary: effect on the kernel, on the compiler and on the hardware layout (index)

Appendix D — The ABI, version 1

The application binary interface fixes how compiled code, the linker and the operating system agree: the sizes of types, which registers carry arguments and results, who saves what, how the stack looks, and how position-independent code reaches its data. These are decided choices (section 9.16 states what the instruction set makes cheap). The source is `../abi.md`; the compiler, the assembler and the linker follow it, and the version number allows a later change without breaking old binaries; in version 1 `long double` is `double`.

D.1 The conventions

Data model	LP64: `char` 8, `short` 16, `int` 32, `long` 64, `long long` 64, pointers 64. Little-endian. `char` is signed. `long double` is `double`.
Alignment	natural; the stack pointer is a multiple of **8** at all times (data alignment is architectural, ISA 7.4: a quad access to an address not a multiple of 8 aborts; the 2-byte parcel alignment is for instructions only). No 16-byte alignment: no type or register is wider than a quad.
Registers	`R0-R5` arguments, `R0` (and `R1`) results, `R0-R7` and `R12` caller-saved; `R8-R11`, `R15` callee-saved; `R12` linker scratch; `R13` frame pointer (callee-saved, optional); `R14` stack pointer.
Floating point	`binary64` in a general register, `binary32` in the low 32 bits sign-extended (ISA 2.3); passed and returned exactly like integers.
Arguments	in order in `R0-R5` (a struct of up to 16 bytes takes one or two consecutive, as many as needed); the rest pushed right to left, each in an 8-byte slot (a larger struct in as many slots as it needs, by value).
Results	`R0`; `R0:R1` for a 128-bit pair or a struct up to 16 bytes; a larger struct through a hidden pointer passed in `R0` (the arguments then start at `R1`), and returned in `R0`.
Calls	`CALL`/`CALLI` push the return address; `RET` pops it. The caller removes stack arguments.
Varargs	the callee stores `R0-R5` in a save area directly below the stack arguments; `va_list` is a pointer into the area, then on into the stack arguments. One area suffices because floats use the same registers.
Red zone	none (version 1).
Symbols	no leading underscore.
Landing pads	off in version 1; the assembler provides `.lpad`.
PIC	globals through `d(PC)`; imported symbols through `MOVQ got(PC),Rd`; calls to imports through `CALLI got(PC)`; TLS local-exec `MOVQ off(DP),Rd`.

D.2 Variadic calls

In a call to a function declared with …, every argument goes in an 8-byte slot, in order: slots 0-5 are `r0-r5`, later slots are on the stack. A structure of any size is passed by value in consecutive slots and may split between `r5` and the stack. A variadic callee leaves its named parameters in their slots (`fp+16+8k`) and `va_start` is the address after the last named slot; `va_list` is a `char *` and `va_copy` is assignment. The prologue pops the return address, pushes `r5`..`r0` and pushes the return address again, so the register save area is contiguous with the stack arguments.

D.3 The hot-register tables

D.3.1 In a one-parcel form a 3-bit register field is an index into a table of the eight hottest registers, and in pdp64/32 a 4-bit field is an index into a table of the sixteen hottest. The tables are part of the ABI, because the encoding depends on them and the compiler's register allocator prefers the hot registers. *(D)*

CPU	3-bit table (index 0 to 7)	4-bit table (index 0 to 15)
pdp64/16	`R0 R1 R8 R2 SP R3 R9 R10`	none (the fields are register numbers)
pdp64/32	`R0 R1 R8 R2 SP R3 R9 R10`	`R0 R1 R8 R2 SP R3 R9 R10 R11 R4 R15 R5 R24 R25 R26 R6`

D.3.2 The 3-bit table covers 90.8% of the register uses of the compiled corpus. `MOV2` takes only registers of the 3-bit table. A different allocation changes the sizes and not the correctness; the measured effect of preferring the hot registers in the allocator is at most 0.03%. *(E)*

D.3.3 A register index beyond a table is an illegal instruction. *(D)*

D.4 The register-save masks and the registers above R15

D.4.1 `pushm`, `popm` and `popmret` name registers by bit: bit n is `rn`. The mask is 16 bits, carried in 8, 16 or 32 bits by the length of the form. The 8-bit mask of `popmret` is scaled by 256 and names `r8` to `r15`, so a save of `r24` to `r31` is not fused with its return. *(D)*

D.5 What the ABI leaves open

Lazy against eager binding, mandatory landing pads and whether imports are always called without a PLT stub beyond version 1 are open (section 9.16, decisions in appendix B).

Appendix E — The assembly language and the object format

The assembler's source layout is the V7 assembler's, `as`; its lexical rules are C's. This appendix is the syntax of a source file: lines, labels, numbers, strings, operators, directives, registers, the operand modes, the mnemonic conventions and the object format. The grammar itself is `spec/pdp64-asm.hbnf`; the source of this appendix is `../asm-syntax.md`. The mnemonics are in the instruction dictionary.

The grammar is `spec/pdp64-asm.hbnf` (hbnf text rules); `tools/check-asmsyntax.sh` generates the parser and runs `tests/asmsyntax.c` over it, and `tools/check-as64.sh` assembles `tests/asm/*.s` and compares the bytes with hand-worked expectations.

- **Lines.** One statement to a line. `;` starts a comment that runs to the end of the line.
- **Labels.** `name:` defines a label; `1:` defines a numeric label, referenced as `1b` (the nearest one before) or `1f` (the nearest after). `sym = expr` assigns. `.` is the location counter. Names are letters, digits, `_`, `.` and `$` and any non-ASCII code point (`café`, `ルート`); a name may start with a digit as long as the whole bareword is not a number (`3dfx:` is a label; `3`, `0x3d`, `0b101`, `0o17` are numbers; `1f` and `1b` are local references), and a bareword of digits only is a numeric local label. Names are case-sensitive and never truncated. ELF symbol names are any bytes, so a name with other characters is written in double quotes (`"odd name!"`). A mnemonic or register name cannot be a label, and neither can `r` followed only by digits: `r16` is an error, not a name (`r2d2` is a name).
- **UTF-8.** A source file is UTF-8, and the parser refuses anything that is not (a stray byte, an overlong form, a surrogate, a truncated sequence), in names, strings, characters and comments alike; a NUL byte is refused too. In a string, UTF-8 text is copied as bytes and `\u00e9`, `\U0001F600` are written as UTF-8; `'é'` is the code point 0xE9, as is `'\u00e9'`.
- **Numbers and characters** are C's: decimal by default (`010` is ten), `0x`, `0b` or `0o` when asked for; `'a'`, `'\n'`, `'\x41'`, `'\101'`.
- **Strings** are `"text\n"` with C's escapes (`\n \t \r \a \b \f \v \0 \\ \' \" \xHH \ooo`), in `.byte` and `.ascii`.
- **Operators** are C's: `* / % & << >>` and `+ - | ^`, unary `- ~ !`, and parentheses. `/` is division. `*`, `/`, `%`, `&` and the shifts need constants; `+` and `-` accept one relocatable symbol, and the difference of two symbols of one section is a constant.
- **Directives.** `.text .data .bss .globl .comm .byte .word .long .quad .ascii .even .align .space .lpad .pool`. `.word` is 16 bits, `.long` 32, `.quad` 64 (the ISA's PDP-11 terminology). `.align n` aligns to n bytes (a power of two).
- **Registers.** `r0`..`r15`; `sp` is `r14`, `fp` is `r13`. `pc` and `dp` appear only as a base: `d(pc)`, `d(dp)`.
- **Operand modes** (ISA section 5): `rN`; `#expr`; `@expr`; `expr(rN)` and `(rN)`; `expr(rN)(rX*s)` with `s` 1, 2, 4 or 8; `expr(pc)`; `expr(dp)`; `(rN)+` and `(rN)+d`; `-(rN)` and `(rN)-d`; a bare expression is a PC-relative reference.
- **Mnemonics** are lowercase (`movq`, `add`, `bne`). A multi-register instruction writes its destination last: `add src, rd`, `xnorpop ra, rb, rd`. `cmp` is the exception, `cmp rd, src`, because the encoding computes `rd - src`.
- **Compare and branch.** `cbCC rd, #imm, label` compares `rd` with an immediate and branches on the condition: `cbeq r3, #5, label`; `cbCC ra, rb, label` compares two registers, and `cbeq` and `cbne` against zero have one-parcel forms. The mnemonics are `cbne cbeq cbpl cbmi cbvc cbvs cbcc cbcs cbge cblt cbgt cble cbhi cblos`. The assembler chooses among the one-, two- and three-parcel forms by the size of the immediate and the distance (the widths are in 9.10.4); when no form reaches, a label in another object or an operand pair that no form holds, it writes `cmp` followed by `bCC`, which writes the flags. `rori32 rs, #n, rd` rotates the low 32 bits right and sign-extends the result.
- **Literal pool: `=expr`.** An operand written with a leading `=` is syntactic sugar for a pooled constant and has no other meaning: `movq =0x123456789ABC, r1` puts the 64-bit value in the literal pool and loads it with one PC-relative `movq`. It is accepted where a quad memory source is (`movq`, `push`, the ALU group with the register first for `cmp`: `cmp rd, =K`, and the floating-point ALU ops), and refused for sized moves, shifts, stores, `lea` and a relocatable symbol. `=` always pools, even a small value; use `#imm` for a constant that fits the signed 32-bit field. A bare `#K` that does not fit that field is an error ("use =K to load it from the literal pool"): the assembler never pools silently. Entries are 8 bytes, 8-byte aligned, one per distinct value per pool, in order of first use, emitted into `.text` at the end of the section or at the directive `.pool` (flush now; code must jump over a mid-section pool). The `=` marks the sugar on the page, which is why it is allowed.

- **Length.** The assembler chooses the shortest form: the first form in the table's order, shorter first and then the order of the catalogue in the grammar, whose fields can hold the operands. Lengths only grow from one pass to the next, and every distance is measured on the layout of the previous pass. The one-parcel forms have no mnemonics of their own, so `movq (r3), r1` assembles to the one-parcel form when it fits and to a longer form when it does not. An unresolved symbol needs a form with a field of 32 bits or more and a relocation; a `movq #imm, rd` whose constant fits no `movq` form is assembled as `movql`.
- **Invocation.** `as64 -m pdp64/r20|pdp64/16|pdp64/32 [-o obj] file.s`: `-m` selects the encoding (the default is `pdp64/r20`; `r20`, `16`, `32` and `old` are accepted) and the object records it in its `e_flags`; `ld64 [-M] [-o out] objects...` refuses to link objects of different encodings. `--stats` prints the instruction counts and bytes, `--dump-lens file` writes the length chosen for every instruction, and `--no-relax` (for `as64` and for `ld64`) gives the plain object and links an object as the assembler laid it out.
- **Words of this CPU.** The qualified moves `movq.u`, `movq.d`, `movq.ud` and the others; `trapcc.<cc>`, one parcel with no operand; `movql #imm64, rd`; the fused `movcall rs, rd, label`, `popret rd`, `popmret #mask` and `mov2 ra, rb, rc, rd`, which the assembler never chooses for itself; `pushm`, `popm` and `popmret` take a mask. `(r)+` and `-(r)` without a stride write stride 0, which means the size of the access, and `(r)+8` writes 8. `extb`, `extzb` and `insb` have a 6-bit position and width, so a width of 64 is refused. The 22 meaningless shapes the original assembler accepts (`clr #imm`, `jmp #imm`, a store to an immediate, `lea #imm` and the others) are refused.
- **Relaxation objects.** By default the assembler marks an object (`e_flags` 0x10, `EF_PDP64_RELAX`) and gives every instruction that holds the address of a symbol a relocation, local symbols included, and every padding of `.text` an `R_PDP64_ALIGN` (type 6; addend = padding bytes << 32 | alignment); the linker then gives each such instruction the shortest form that reaches. An object whose `.text` holds a distance that is not a relocation (a difference of two labels used as a number, a `d(pc)` written as a number) is marked `EF_PDP64_NOADJ_TEXT` (0x20) and its `.text` is left alone. A `cbCC` that the assembler expanded to `cmp` and `bCC` stays expanded. `--no-relax` opts out.
- **`llvm-mc`.** The LLVM back end's assembler (`llvm/`, ISA-checked against `as64` by `llvm/check.sh`) accepts this syntax with these exceptions: no literal pool (`=K` and `.pool`), no names that start with a digit, no non-ASCII names, and `.comm` needs its alignment written (`as64` implies 8). `(pc)` and `(dp)` without a displacement are accepted by `as64` (displacement 0) and by `llvm-mc`.
- **Object format.** ELF64, little-endian, `e_machine` 0xF064 (`EM_PDP64`); relocations `R_PDP64_D64`, `R_PDP64_D32`, `R_PDP64_INSN_ABS`, `R_PDP64_INSN_PC`, `R_PDP64_INSN_BR` (`as64/relocs.h`; LLVM: `ELFRelocs/PDP64.def`).

What it leaves out of V7 `as`: octal numbers by default, `<string>` and the `'\e` escape, `\/ \< \>` and `[ ]` in expressions, `.if`, `.stabs`, symbols cut to eight characters.

Appendix F — The evidence behind the manual

The manual states decisions; these are the documents that hold the evidence, the surveys and the long arguments they were drawn from. Each is a document of the repository, not a chapter of this manual, and each says in its own opening what it is based on and what it did not check. They are listed with what they hold and their status; the *Why it is here* paragraphs of the instruction entries and the *Impact* lines cite them.

F.1 The architecture, in full

Document	What it holds
`pdp64-isa.md`	the full instruction-set text with every encoding, the rejected features, the idioms lost and restored, the comparison with the PDP-11 and the VAX
`pdp64-system-architecture.md`	privilege, exceptions, interrupts, memory management and boot as an operating system sees them, with what is undefined stated plainly
`pdp64-system-design.md`	System v1: the options, how others do it, and how the choices serve the project's goals
`pdp64-q64-bus.md`	the system bus and its devices
`boot.md`	boot and bring-up: how the main systems start themselves, whether UEFI is worth having, and the proposal
`pdp64-in-order-and-out-of-order.md` and `pdp64-ooo-readiness.md`	what the in-order design point costs and buys, and what the architecture must decide to stay ready for an out-of-order core
`pdp64-fpga-plan.md` and `pdp64-lb64-accelerator.md`	the FPGA build plan and the inference accelerator
`abi.md` and `asm-syntax.md`	the ABI and the assembly language (appendices D and E are drawn from them)
`roadmap.md`	the order of work from the present toolchain to an OpenBSD that can be logged into

F.2 The record of decisions

Document	What it holds
`errata.md`	every decision, numbered, with the reasons
`decisions-pending.md`	every open decision, with the need, five options, the survey of what other systems do and what the PDP-11 and VAX did (appendix B is drawn from it)
`fact-check-2026-10-22.md`	the comparison of statements about the outside world with a source where one could be read; the claims it could not check are marked "not confirmed" in the manual

F.3 Measurements and surveys

Document	What it holds
`codegen-analysis.md`	what compiled C says about the instruction set: instruction use, operand forms, the instructions never emitted
`kernel-asm-survey.md`	what Linux, FreeBSD, NetBSD and OpenBSD write in assembly, and what pdp64 can learn from it
`kernel-pain-points.md`	what hurts kernel developers: tight loops, pain points and workarounds in the current kernels, from the source trees and the mailing lists, compared with pdp64
`hardware-vulnerabilities.md`	the processor, memory and system flaws of the last decade that needed mitigations, and the rules that keep pdp64 from repeating them, including the evidence on constant-time modes
`openbsd-mitigations.md`	OpenBSD's memory-safety and exploit mitigations, what pdp64 gives for free and what would make the rest soft and easy
`sim64.md`	the reference emulator
`smp-decisions.md`	the open symmetric-multiprocessing decisions, proposals awaiting the author's annotations (tag P)

Document	What it holds
crypto-secure-core.md, security-proposals.md, study-proposals.md, retire-stage-candidates.md	the proposals on the secure core, the security mechanisms, the studied instructions and the retire-stage mechanisms, with the author's replies in errata 84 to 86
priority-lists.md	the lists of priorities that errata 83 settled

The data behind the tables is in docs/codegen-data/, the generators are in tools/, and the sources of the design discussions are chat01.txt and the archive of the working session.

F.4 The prefix-coded encoding: the measurements

F.4.1 The encoding, the compiler, the assembler, the linker, the simulator and the timing model were built and measured together; the sources of the numbers below are encoding-proposal.md (the study, sections 6 and 12) and encoding-work/integration.md (section 4 is the final table, made on the integrated tree), with the reports of each phase in encoding-work/. The baseline is the original CPU with the compiler as it was before the work. There is no baseline of the original CPU with the new compiler: several of the compiler's passes use instructions only the prefix-coded encodings have. Relaxation is on by default; the model of timing is an in-order toy with ideal memory (128 exact checks of its own pipeline), not hardware.

F.4.2 The static size on the 841-file corpus (instruction bytes from as64 --stats):

	pdp64/r20, old compiler	this CPU, new compiler
instruction bytes	1,083,870	934,470 (-13.78%)
without memcpy-a2.c (27% of the bytes)	785,836	676,354 (-13.93%)
instructions	338,313	300,821
fused pairs (MOVCALL, POPMRET, POPRET, MOV2)	0	10,757, 2,246, 2,609, 4,049 = 19,661
CBCC replaced by CMP and BCC	0	0
text of the objects (840 files, before linking)	1,305,878	1,167,388
the same linked with relaxation	not applicable	1,122,322 (-3.86% against unrelaxed)

Relaxation shortens the text of the objects by 45,066 bytes: external calls 20,333 sites (40,666 bytes), data LEA 12,787 sites (1,802), pc-relative loads (1,588), JMP to another section (718), branches (266). The corpus stub places the symbols right behind each file, so the saving is an upper bound for the reach of the 4-byte call.

F.4.3 Six large programs (lz4, zlib, Lua and three BearSSL kernels, linked with a stand-in C library, run on the simulator; all print exactly the host's output on all three CPUs), totals:

	pdp64/r20, old compiler	this CPU, new compiler
instructions executed	755,816,022	609,755,229 (-19.32%)
bytes fetched	2,365,226,284	1,633,050,156 (-30.96%)
bytes per instruction	3.129	2.678
linked text (relaxed)	226,606	196,772 (-13.17%)
fused instructions executed	0	10,079,611
frame loads and stores executed	68,936,548	65,203,703 (-5.41%)

Linking the same objects with --no-relax instead of the default makes the linked text 1.73% larger and changes the bytes executed by 0.4%.

F.4.4 The timing model (in order, ideal memory, no branch predictor; issue width 1 or 2, fetch window W bytes a cycle; the prefix-coded CPUs pay one cycle of length-decode latency on a taken branch and a refill), all six programs summed:

model	pdp64/r20, old compiler	this CPU, new compiler
issue 1, W = 4: cycles	1,133,787,653	914,840,325 (-19.31%)
issue 2, W = 4: cycles	1,001,691,461	789,767,576 (-21.16%)
issue 1, W = 2: cycles	1,305,661,982	1,007,176,894 (-22.86%)
issue 1, W = 4: fetch-wait, execute-wait (share of cycles)	1.7%, 31.6%	5.5%, 27.9%

At four bytes a cycle and above the programs are bound by execution and not by fetch (widening the fetch from 4 to 16 bytes a cycle takes 0.5% off the cycles at issue width 1); the saving in cycles is the saving in instructions, and the saving in bytes buys nothing at those widths. By program (issue 1, W = 4, against pdp64/r20): lz4 +1.3%, zlib -5.9%, Lua -45.4%, bs_hash -2.1%, bs_sym -1.2%, bs_ec -23.6%. The gain on Lua is the switch dispatch of the new compiler, not the encoding.

F.4.5 The predictions of the study against the final measurement: the model of the study put the corpus at 1,013,598 bytes (-6.48%) and, with the four fused pairs, 979,192 (-9.66%); the measurement is 934,470, because the model measured the old

compiler's code and the compiler now fuses, reuses values, tail-calls, builds jump tables and scales indexes, and the grammar was re-optimised on the new compiler's output. The study put relaxation at 9% of the corpus text and a first measurement at 3.5%; the final measurement is 3.86%.

F.4.6 What these measurements do not establish: the timing is one sweep of an in-order model with ideal memory; the cache variants were not re-run on the final compiler; the 3-parcel `CBCC` and the other form choices were trained on the corpus and not on the six programs, so a profile-tuned variant was not tried; one corpus file does not link in the stub measurement (a 64-bit constant used as a displacement); the compiler does not use `MOVUW`, `MOVUL`, `MOVUQ` (the unaligned-access idiom is 10% of the instructions of lz4), `MULH`-based division by 64-bit constants, loop-invariant motion or live-range splitting; QEMU does not execute this encoding; the atomics, a number of cryptographic words, `MVAL`, `UTF8DEC` and `UTF8ENC` are not implemented in the reference executor.